Modelling and testing

Gábor Árpád Németh, PhD

Version: 2026.02.24.3

Introduction to this lecture

Who am I?

Gábor Árpád Németh

- MSc in Electrical Engineering
- PhD in Computer Science
- Worked in a telecommunications company for 6½ years
  (developer → product owner → system architect)
- Contact:
  - MS Teams, nga AT inf.elte.hu
  - ELTE Southern building, 2nd floor, room 207

Guest lecturers during the term

- Bálint Szél —

Tosca MBT tool
- István Forgács —

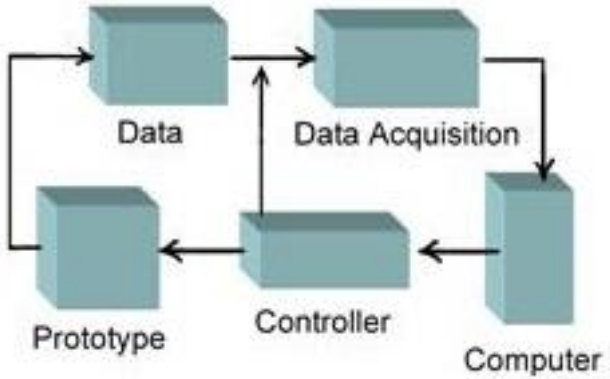
Harmony end-to-end AI MBT test
- Péter Sótér — Jenkins test automation server
- Lugosi Máté —
Model 》 Test 》 Relax MBT framework
- Kristóf Szabados —

Code Quality, code smells

AI: Artificial intelligence
MBT: Model-based testing

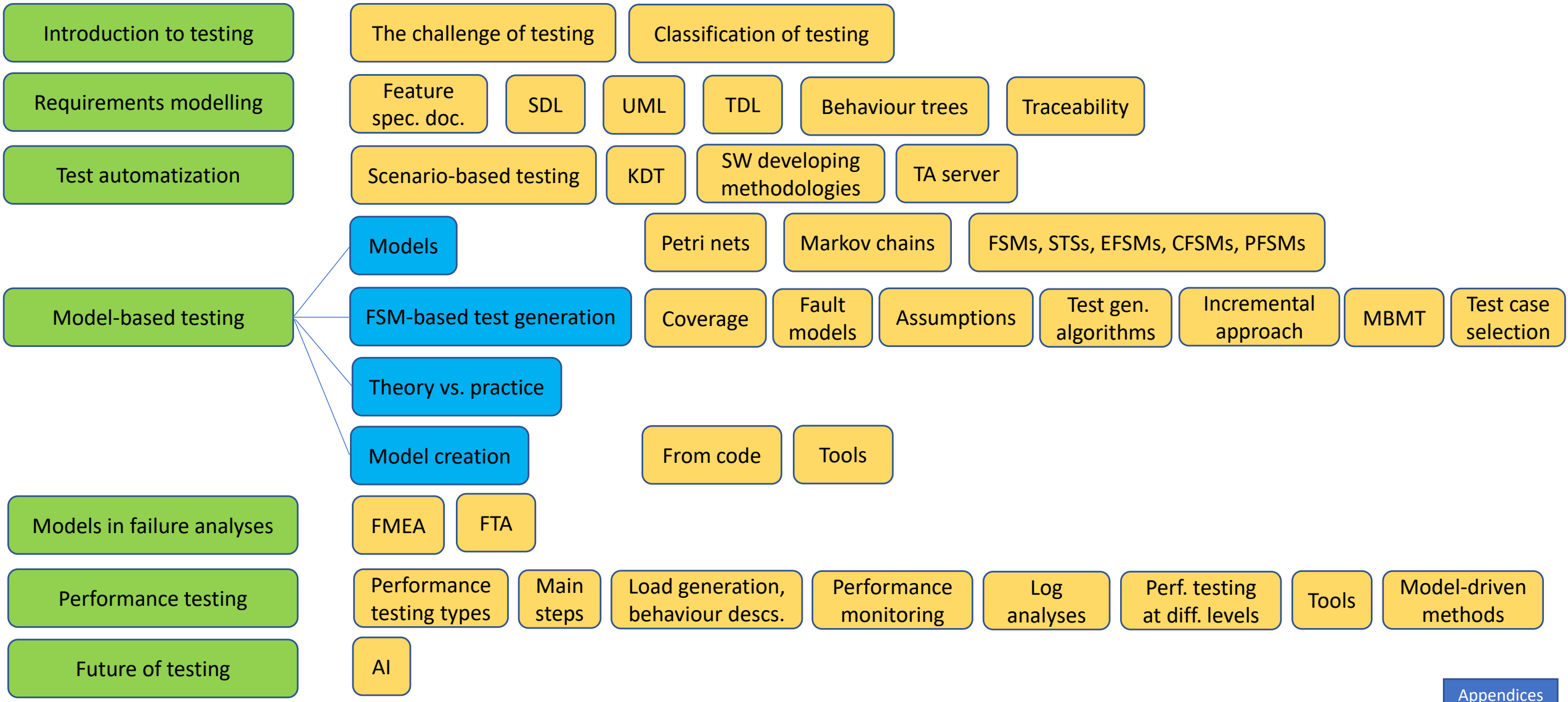
Theory is very good, but
then actual data comes in
and ruins everything...

Theory:



Practice:

An overview of the course



Appendices

An overview of the course

- Introduction to testing
  - The challenge of testing
  - Classification of testing
- Requirements modelling
  - Feature spec. doc.
  - SDL
  - UML
  - TDL
  - Behaviour trees
  - Traceability
- Test automatization
  - Scenario-based testing
  - KDT
  - SW developing methodologies
  - TA server
    - *Live demo of Jenkins test automation server*
- Model-based testing
  - Models
    - Petri nets
    - Markov chains
    - FSMs, STSs, EFSMs, CFSMs, PFSMs
  - FSM-based test generation
    - Coverage
    - Fault models
    - Assumptions
    - Test gen. algorithms
    - Incremental approach
    - MBMT
    - Test case selection
  - Theory vs. practice
  - Model creation
    - From code
    - Tools
      - *Live demo of **3** MBT tools: GraphWalker, 4Test Harmony, Tricentis Tosca*
- Models in failure analyses
  - FMEA
  - FTA
  - *Guest lecture about identifying code smells*
- Performance testing
  - Performance testing types
  - Main steps
  - Load generation, behaviour descs.
  - Performance monitoring
  - Log analyses
  - Perf. testing at diff. levels
  - Tools
    - *Wireshark and Apache JMeter live demo*
  - Model-driven methods
- Future of testing
  - AI
  - *Introduction of a performance testing tool of telecommunication networks using EFSMs for behaviour description*

Appendices

Important things to discuss before we start…

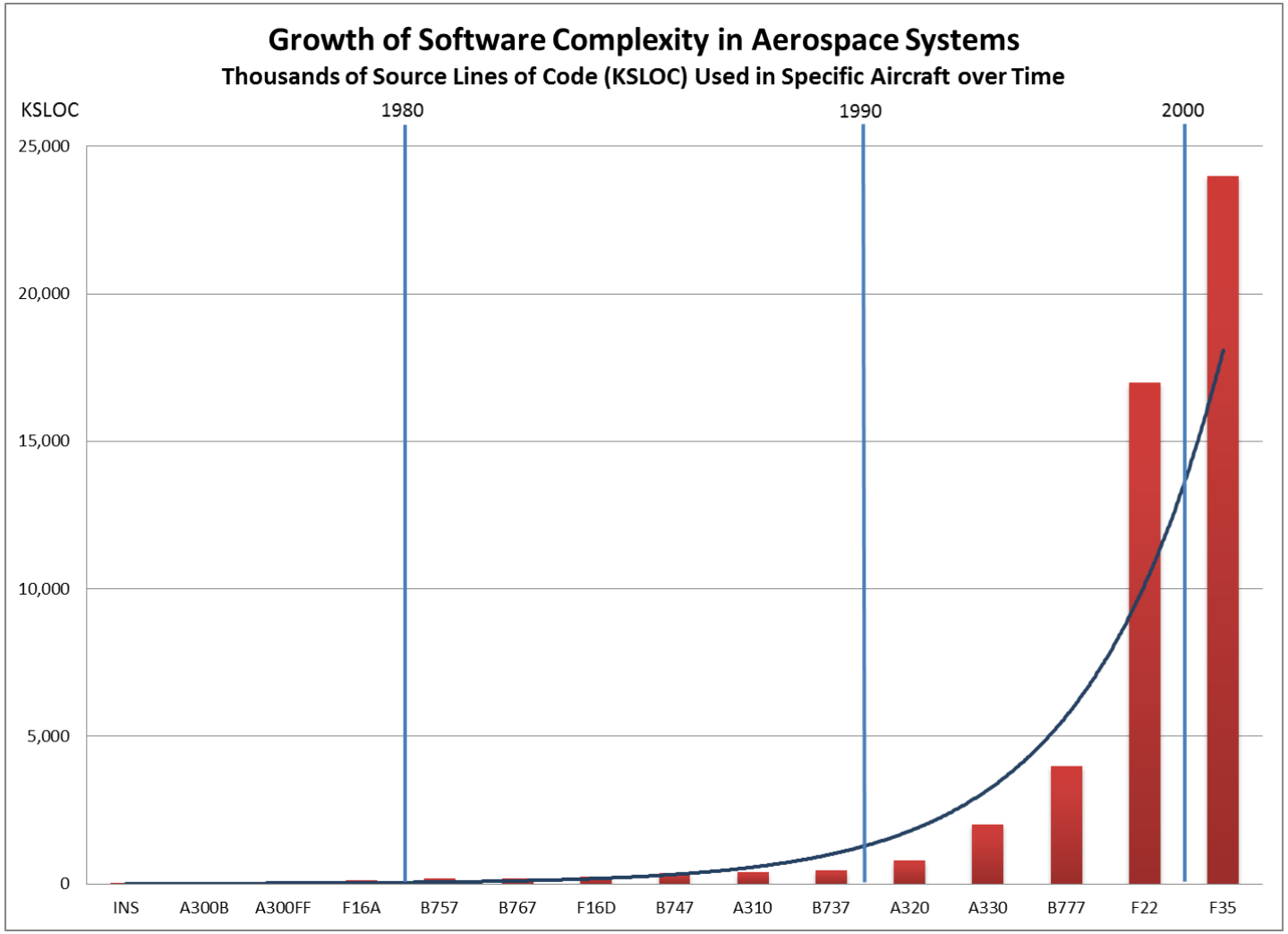
- Start time / end time?
- Evaluating course achievement: Homework / exam?
- How to use slides?
  - Links, appendices, abbreviations, traceability between different materials…etc.
  - Actual version: bottom-right corner of the 1st slide
  - Color codes in the corner of the slides:

- Materials in OneDrive (course videos, example homeworks…etc)
- Model 》 Test 》 Relax git repository
- What is the first PC hardware / OS version you used?

Main ToC

WHEN YOU HEAR THIS:
geek & poke
YESTERDAY IT WORKED
YOU KNOW YOU'RE IN A SOFTWARE PROJECT

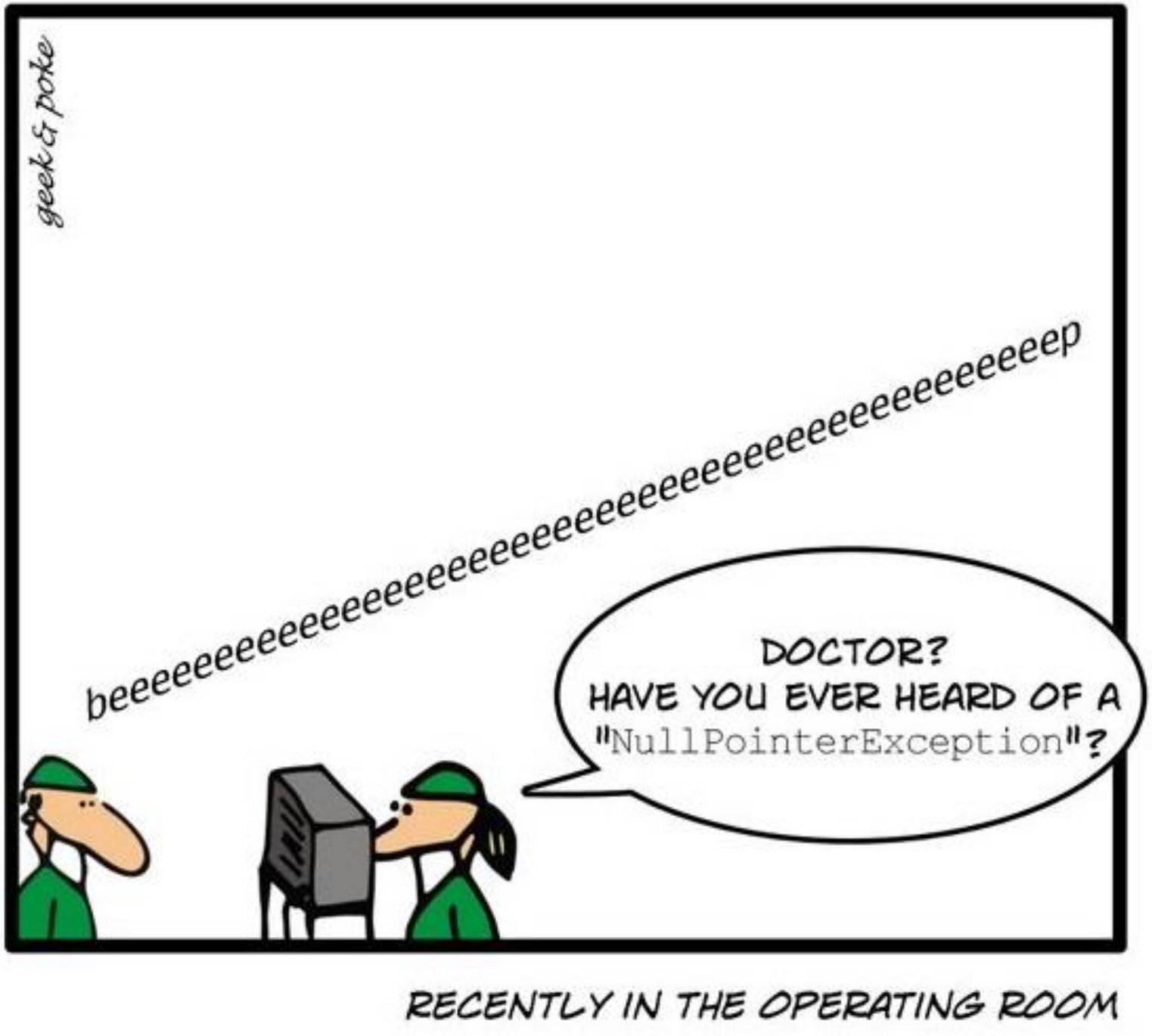
Why is testing important?



Complexity of system grows exponentially…

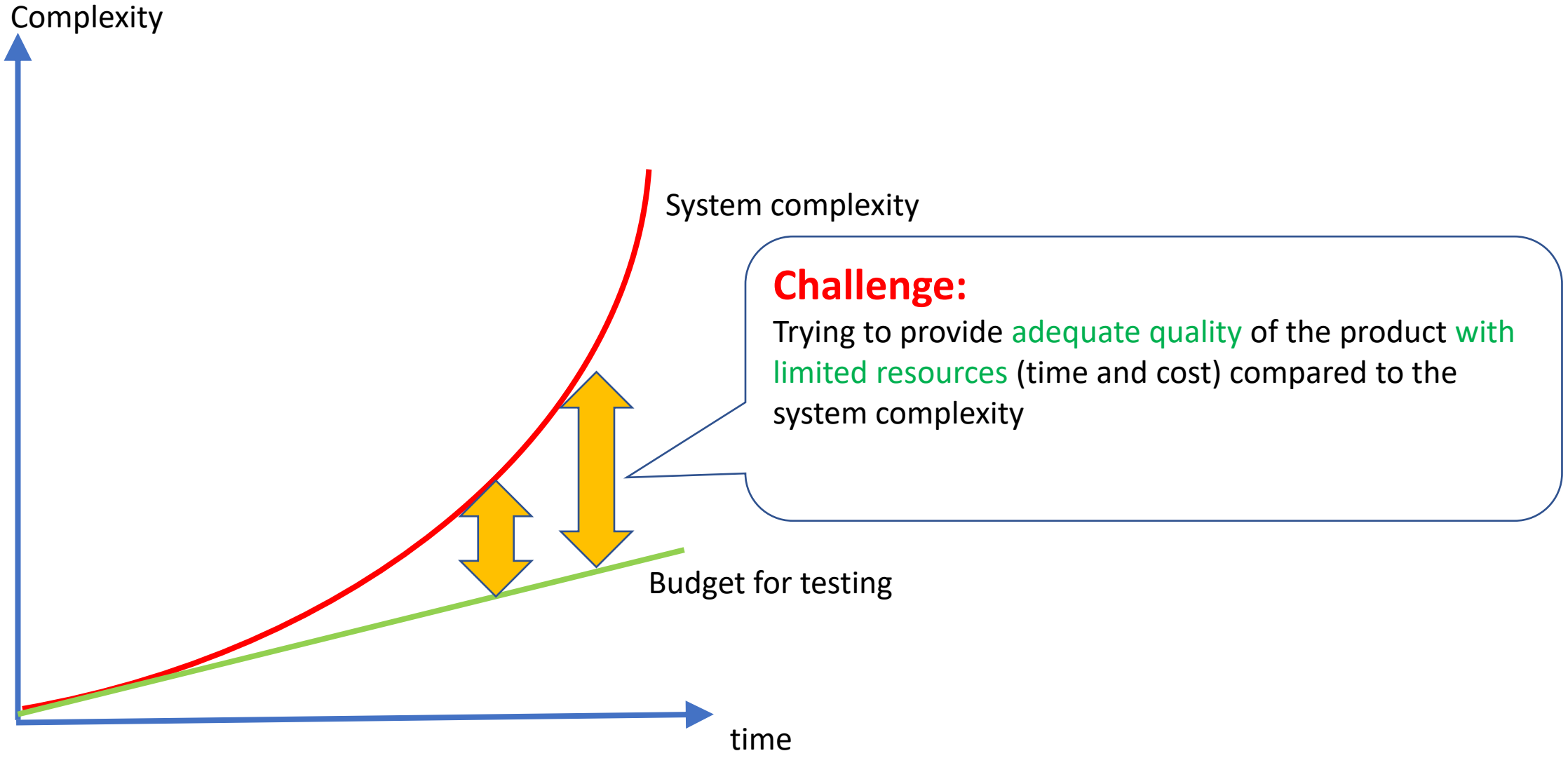
Source of fig

Why is testing important?

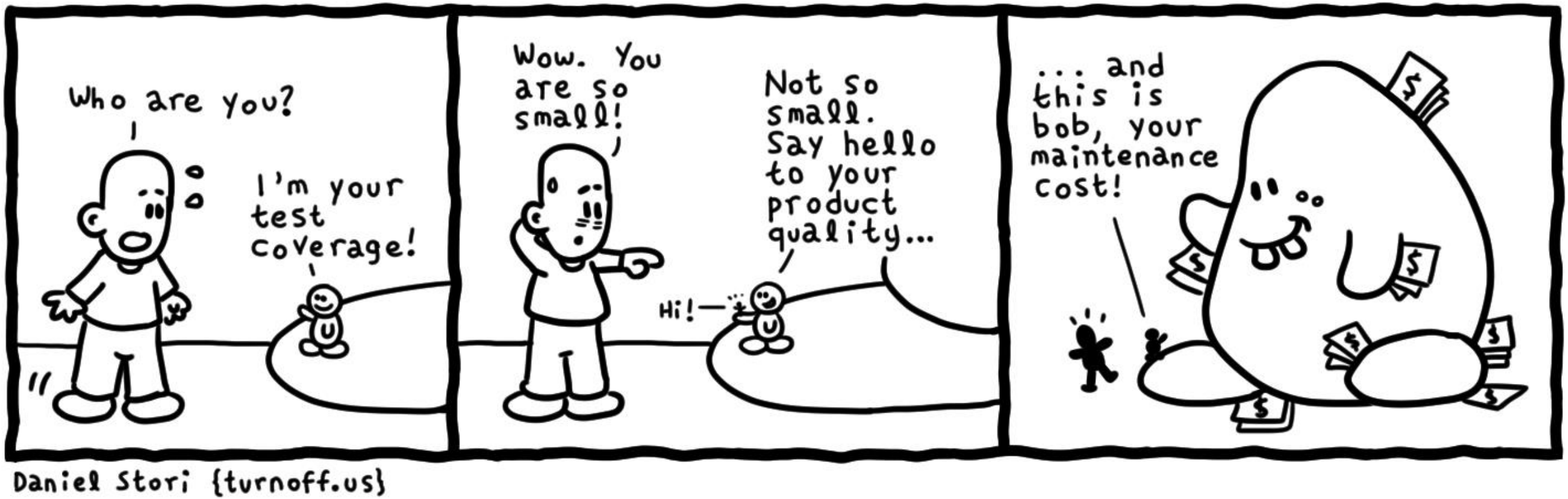

geek & poke
beeee

Why is testing important?

The challenge of testing



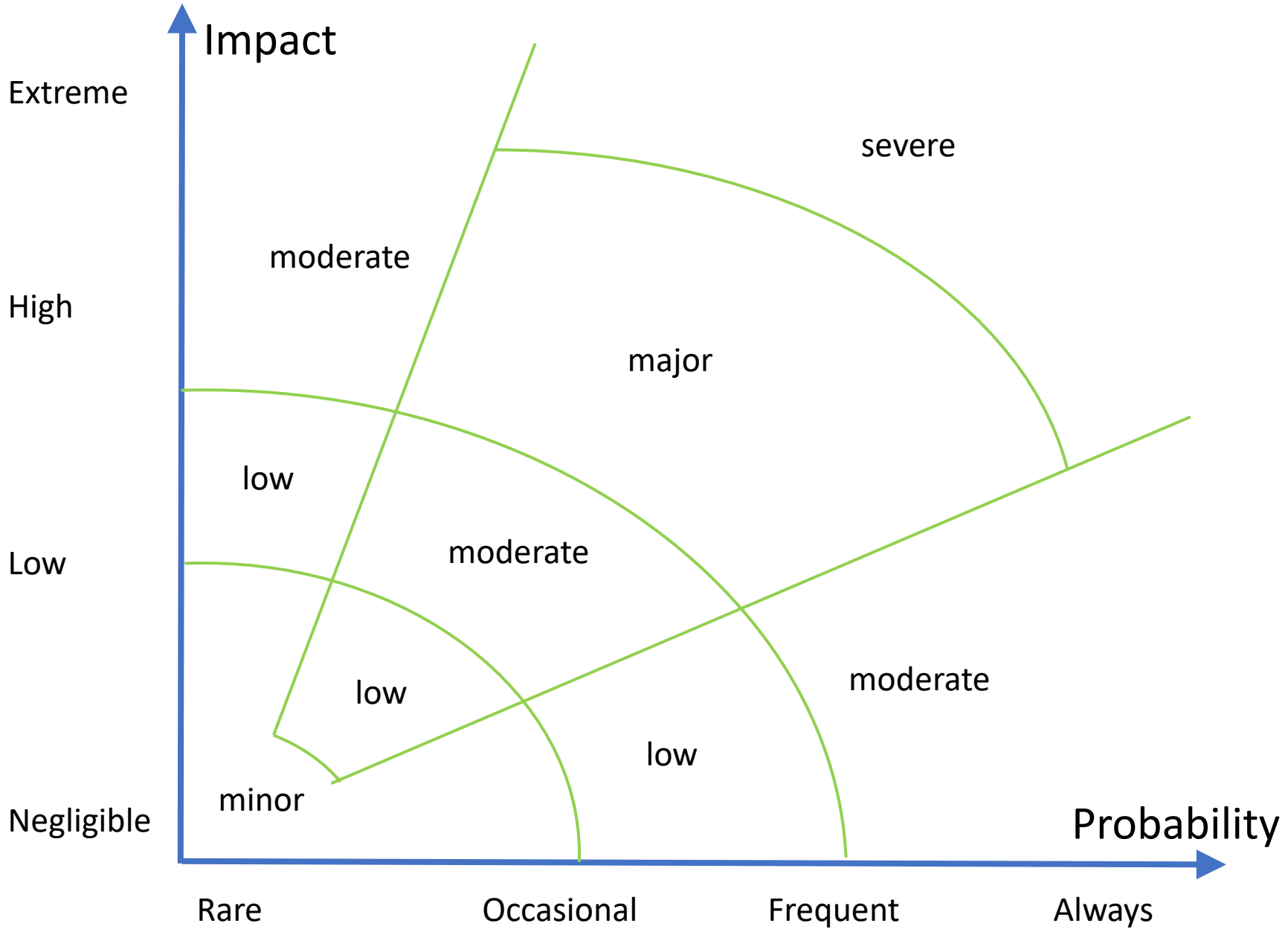
The challenge of testing



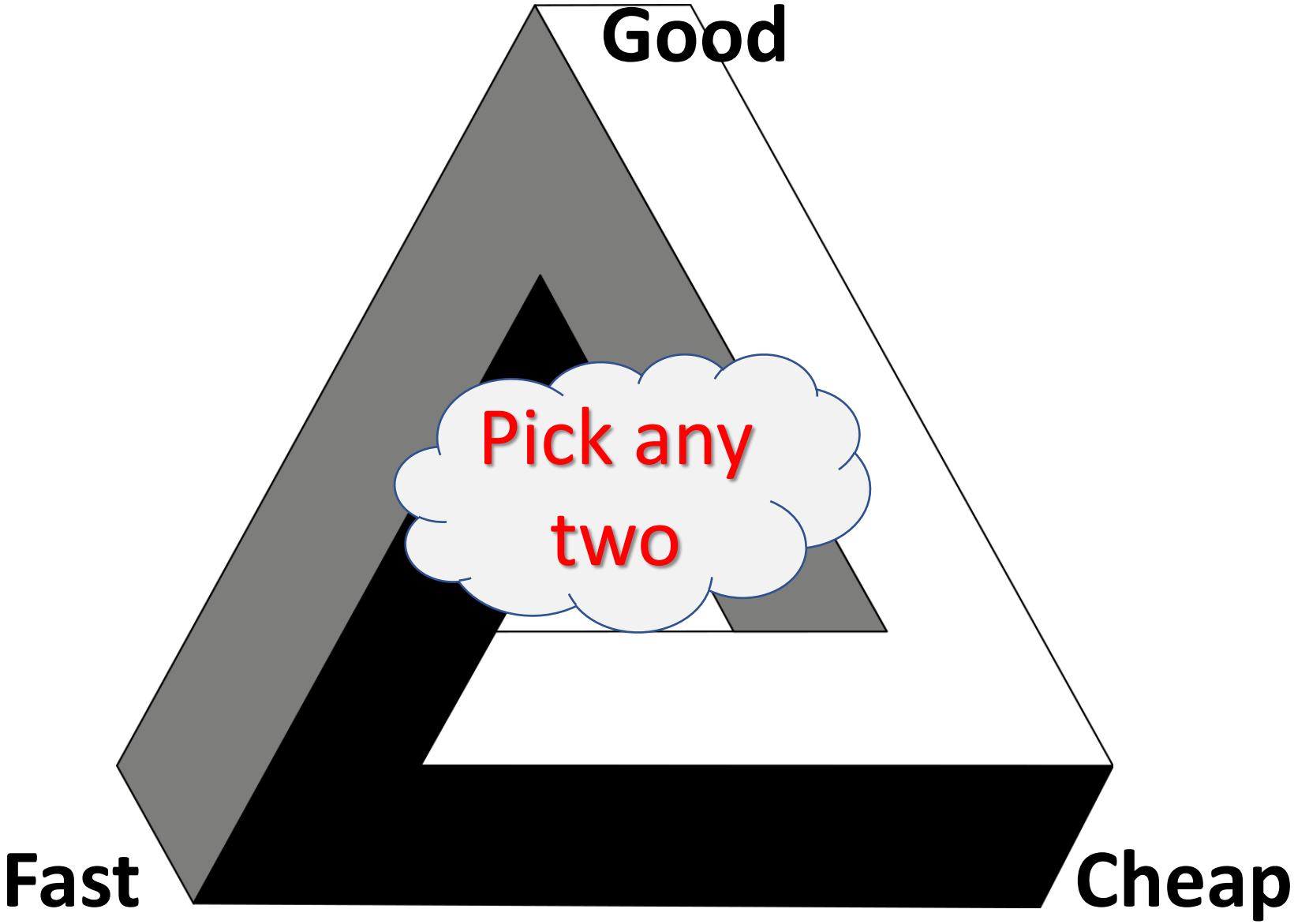
Source

Scoring the risks…

What is adequate quality?

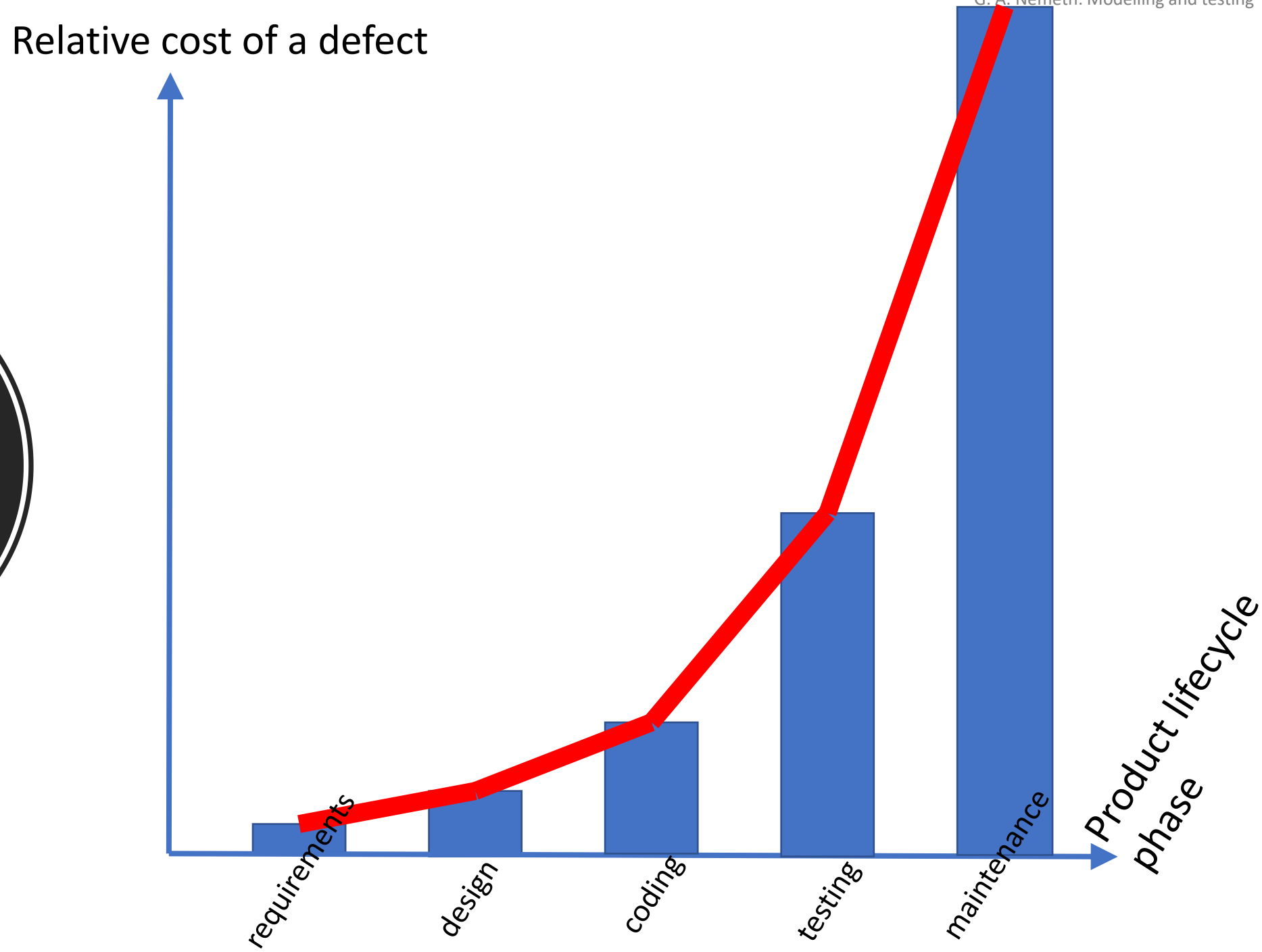


What is adequate quality?

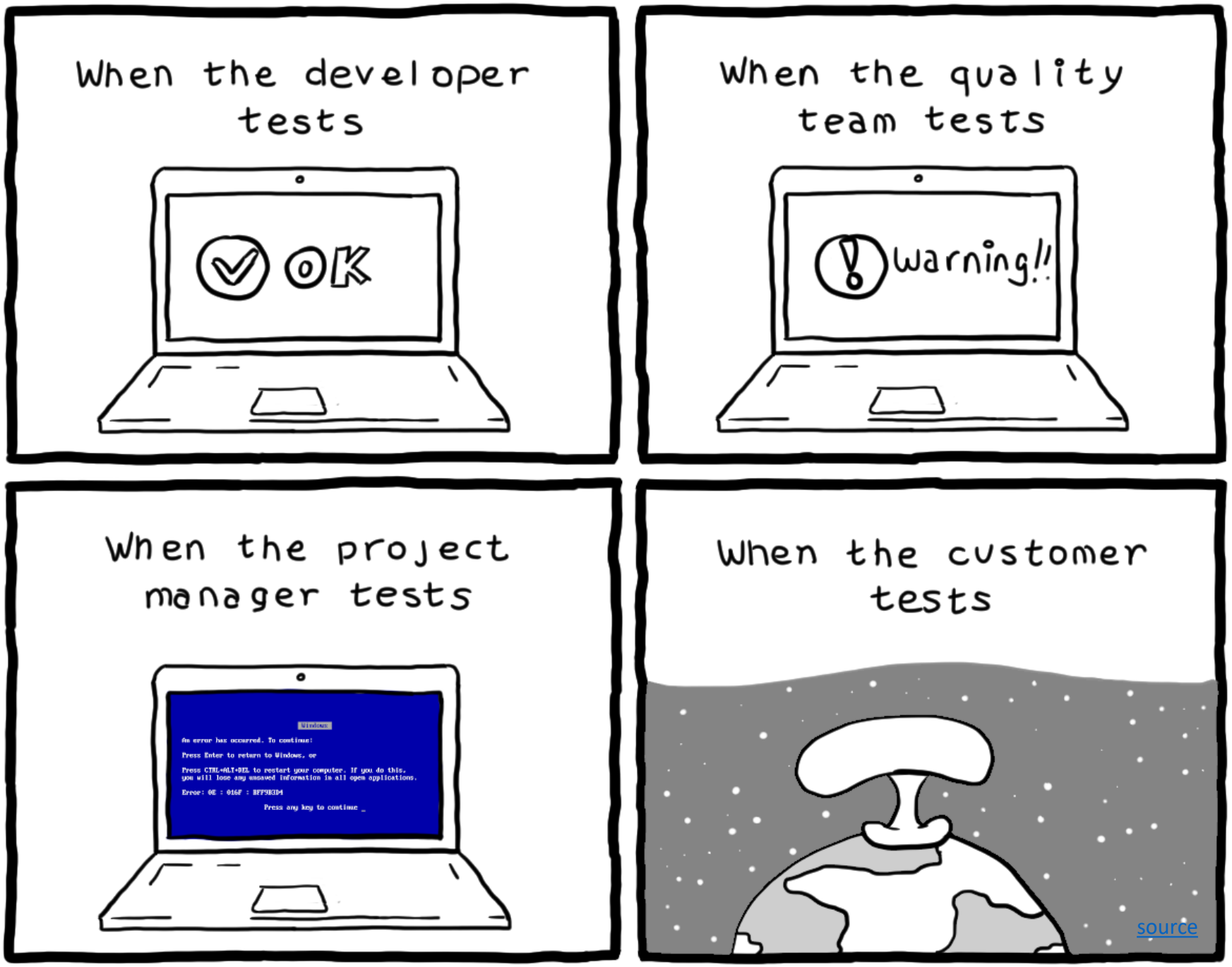


What is adequate quality?

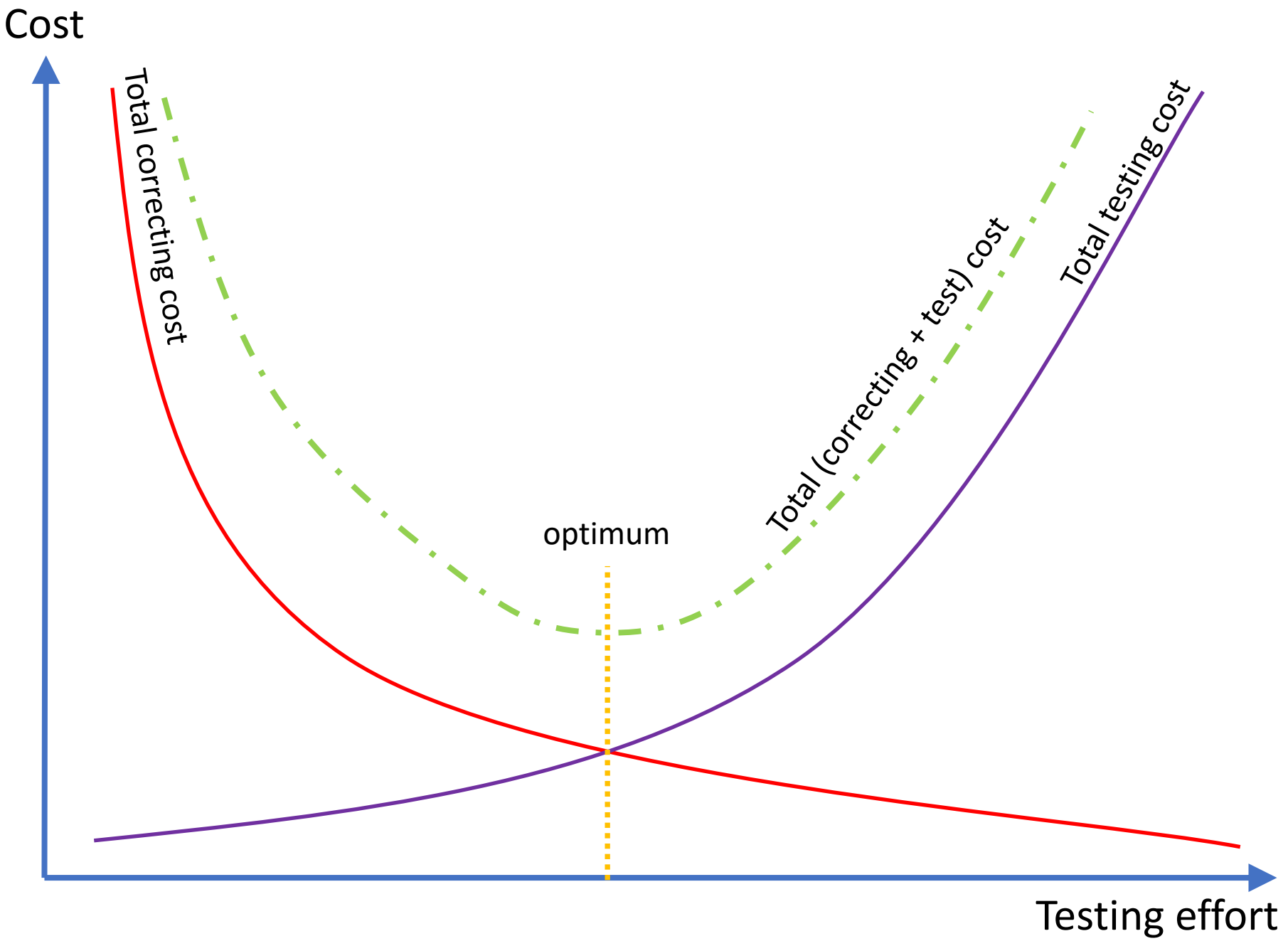
The cost of defects



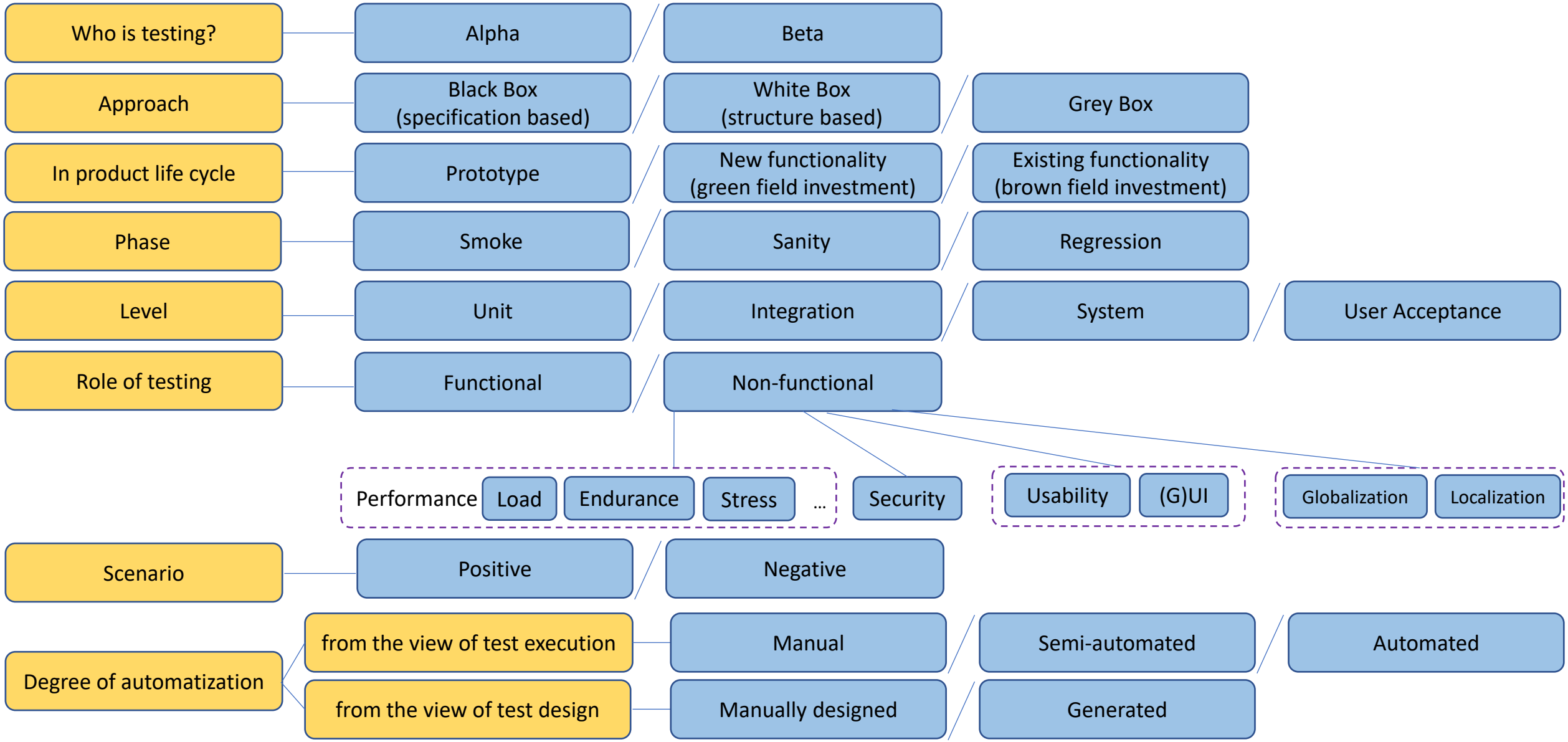
The cost of defects



The optimal cost of testing



Classification of testing



Classification of testing

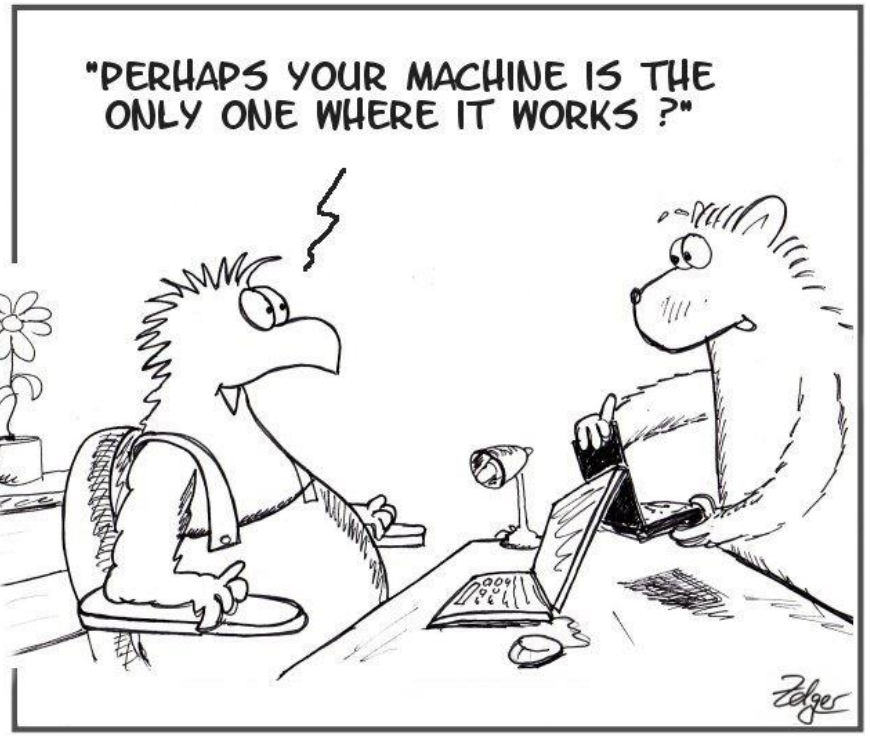
Who is testing?

Alpha

- Before release
- In lab environment
- Testers are internal employers

Beta

- In “real environment”

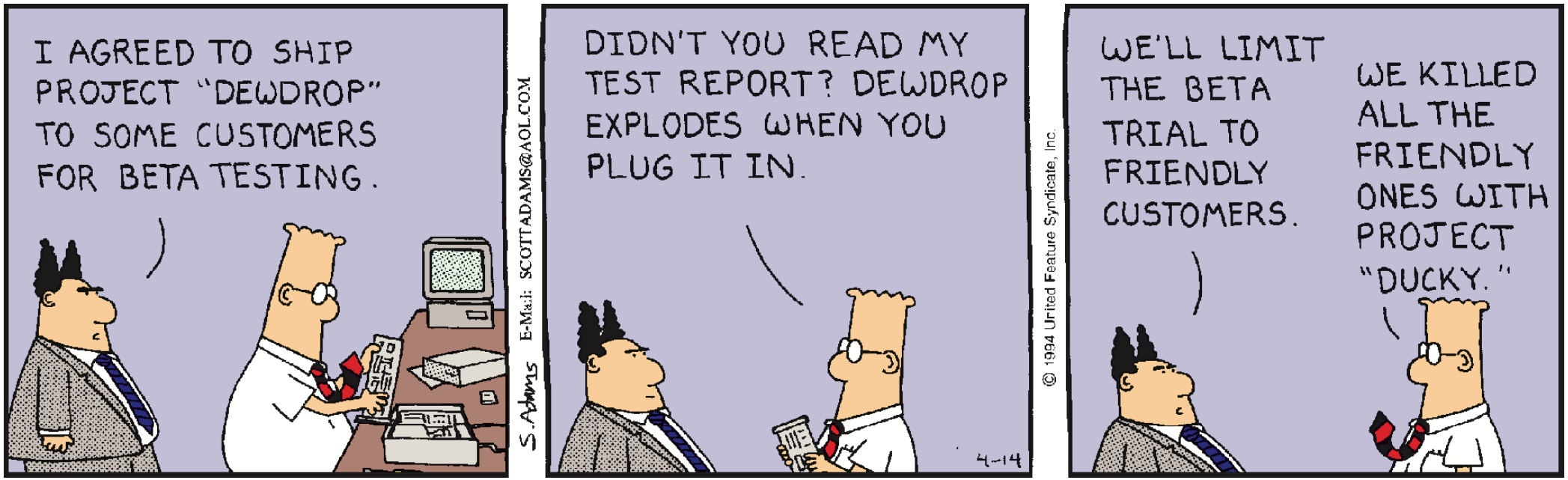


It works on my machine

Classification of testing

Who is testing?

- Alpha
  - Before release
  - In lab environment
  - Testers are internal employers
- Beta
  - In “real environment”
  - Testers are (limited, selected) “real users”
  - Can be considered as External User Acceptance Testing



Classification of testing

approach

Black Box
(specification based)

- **The internal working of the SUT** (System Under Test) **is unknown**
- We can only observe the output of the SUT upon some input
- We know the desired output from some input from the specification

Specification

?

Implementation

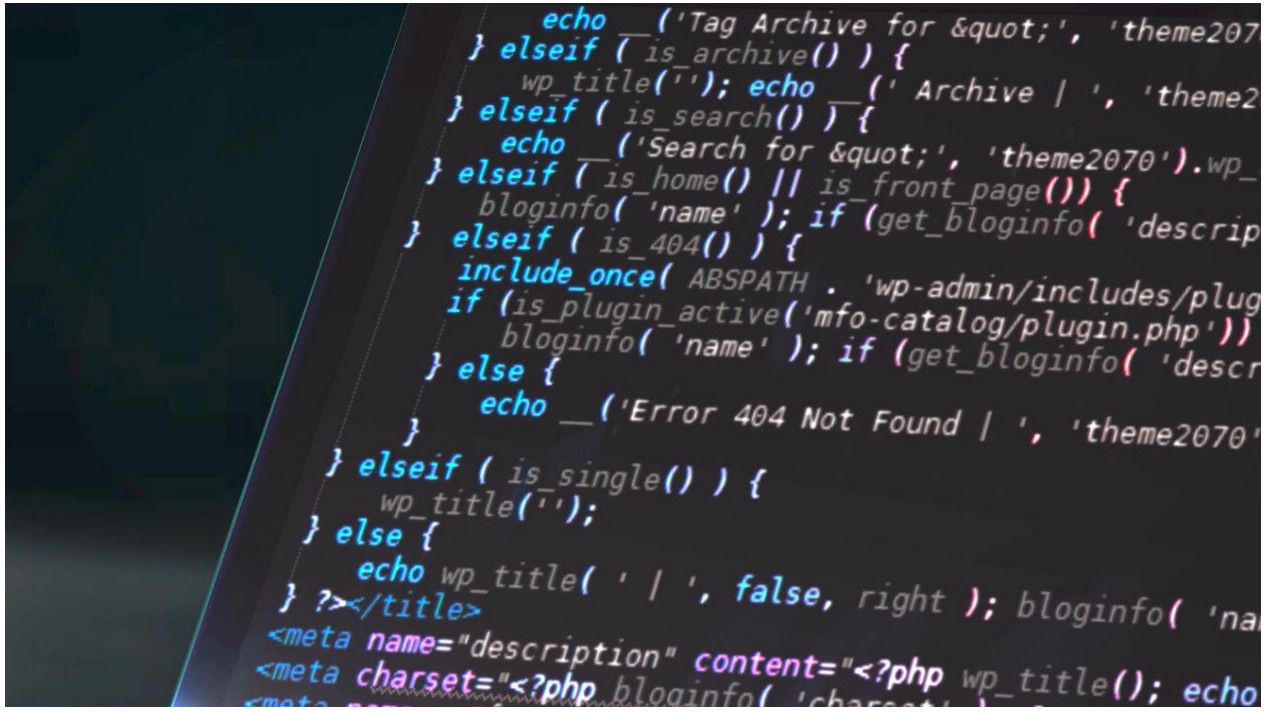
Classification of testing

approach

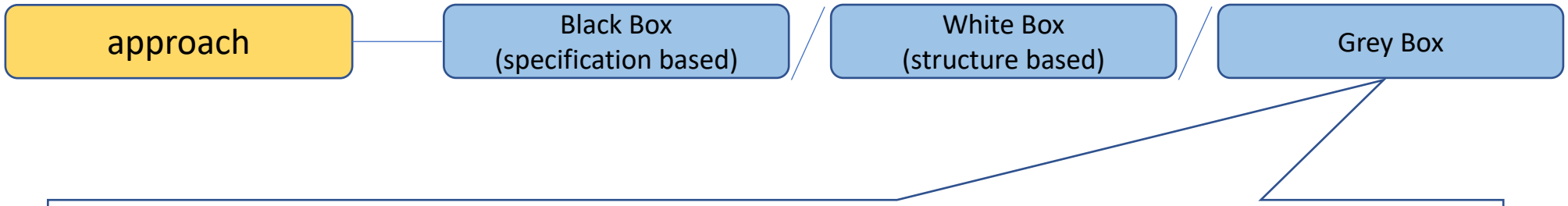
Black Box
(specification based)

White Box
(structure based)

- **The tester knows the internal working** – i.e. the structure – **of the SUT**

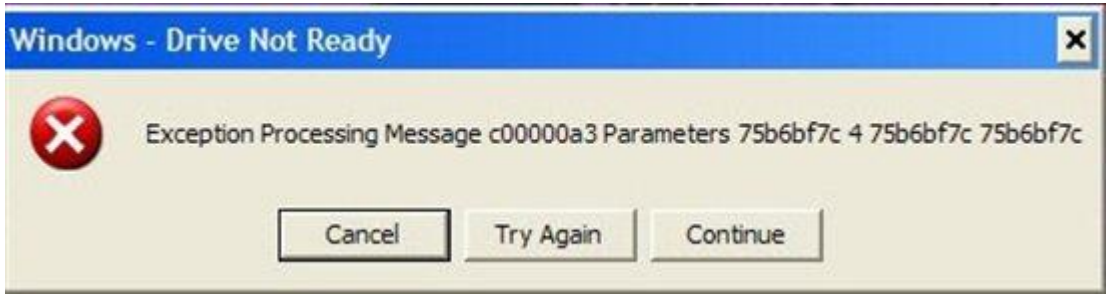


Classification of testing



- Somewhere between black box and white box testing:
  - Some internal variables are known*, but not the entire system

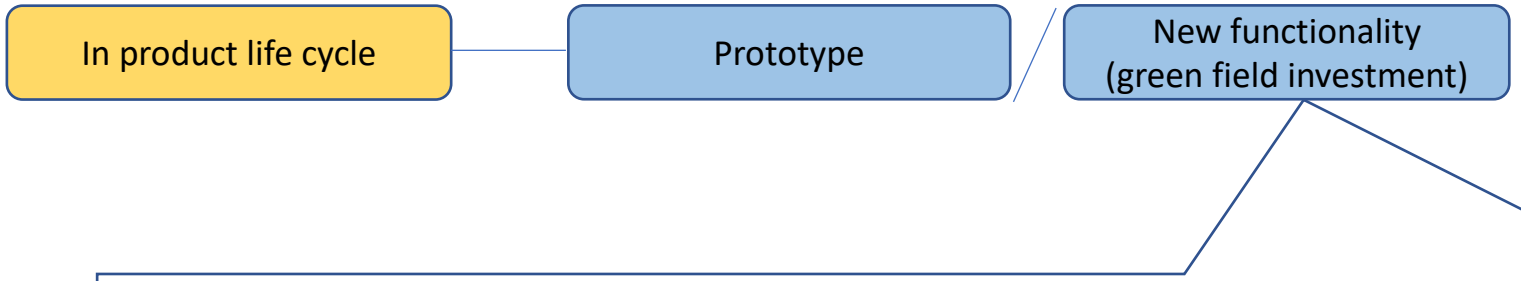
  * For example, the developer team provides access to some internal variables for debugging purposes



Classification of testing

- In prototype phase, the software and its concept are changing rapidly
  → robust test environment would require high maintenance cost
- The impact of bugs are low – as the prototype is not a real product
- Thus, only manual testing, or proof-of-concept of some test scripts are required

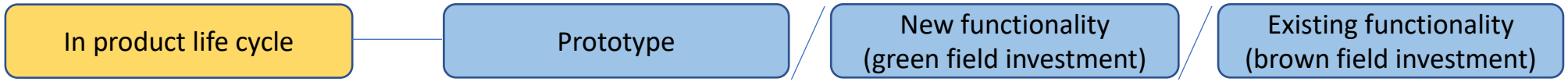
Classification of testing



- In case of new functionalities, the test infrastructure can be designed during feature clarification / specification with the customer

- “White paper chance”:
  - \+ Chance to create a good test design, use new approaches…etc.
  - \+ Motivates thinking
  - \- Big chances always come with big risks if no attention is paid to careful thinking…

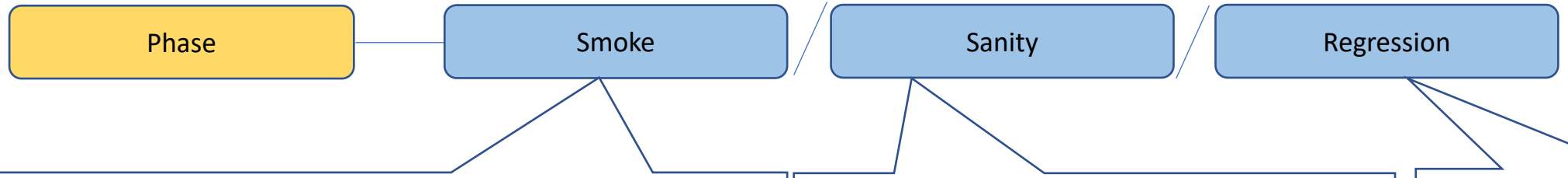
Classification of testing



- In case of an existing product, when testing should be introduced or significantly improved due to quality problems, the first task is to find out how the product should work if the documentation (specification, user documentation…etc.) is not appropriate

- "De-facto problem":
  - People rarely question de-facto things: Is the given solution really appropriate?
  - Maybe significant effect from the existing test infrastructure
  - Maybe hard to maintain
  - Hard to change
  + Existing processes, guidelines can be followed

Classification of testing



- Run after software build **to test that the critical functionalities** of the program is working fine
- Executed "before" any detailed functional or regression tests

- Examples:
  - Verifying that the application launches successfully
  - Checking that the GUI is responsive

- **Run after** receiving a software build, with **minor changes in code**, or functionality, **to check that the bugs have been fixed and no further issues are introduced due to these changes**

- The goal is to determine that the proposed functionality works roughly as expected

- **Re-running** functional and non-functional **tests to ensure that previously developed and tested software still performs after a change**

Classification of testing

Level

Unit

Isolate individual components of source code **and check if they are correct**

- Test basis:
  - Detailed design
  - Code
  - Data model
  - Component specification
- Test objects:
  - Components
  - Code and data structures
  - Classes
  - Database modules

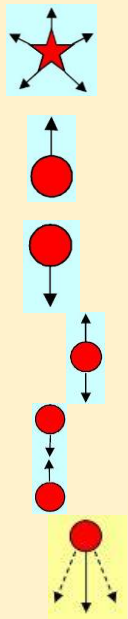
Integration

Individual software modules are combined and tested as a group

- Test basis:
  - Design docs
  - Sequence diagrams, communication protocol specifications, use cases
  - Architecture, workflows, external interface definitions
- Test objects:
  - Subsystems, databases, infrastructure, interfaces, APIs
  - Microservices

Integration strategies:

- Big-bang
- Bottom-up
- Top-down
- Inside-out
- Outside-in
- Branch-wise



Classification of testing

Level

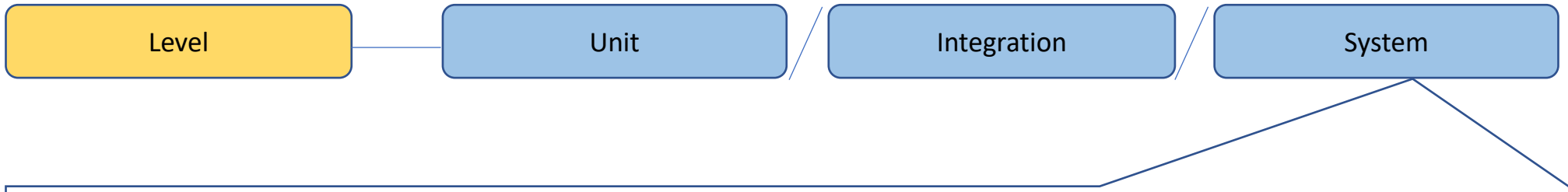
Unit

Integration

Isolate individual components of source code **and check if they are correct**

Individual software modules are combined and tested as a group

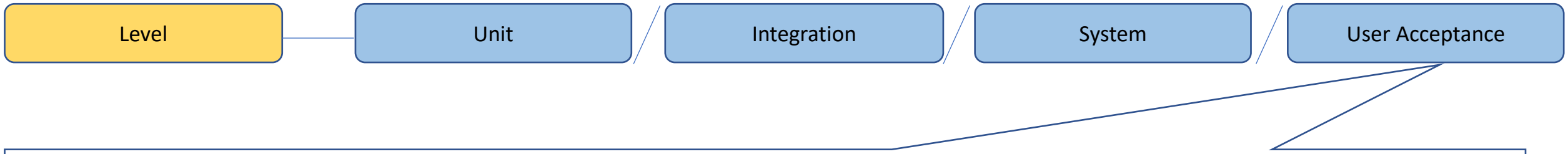
Classification of testing



Focuses on the **behaviour of the whole system** or product **in a live environment**

- Test basis:
  - System requirement spec. (functional and non-functional)
  - Risk inventory, use cases, epics, stories
  - Models of system behaviour, state diagrams, user manuals
- Test objects:
  - Applications, operating systems, SUT
  - System configuration and configuration data

Classification of testing

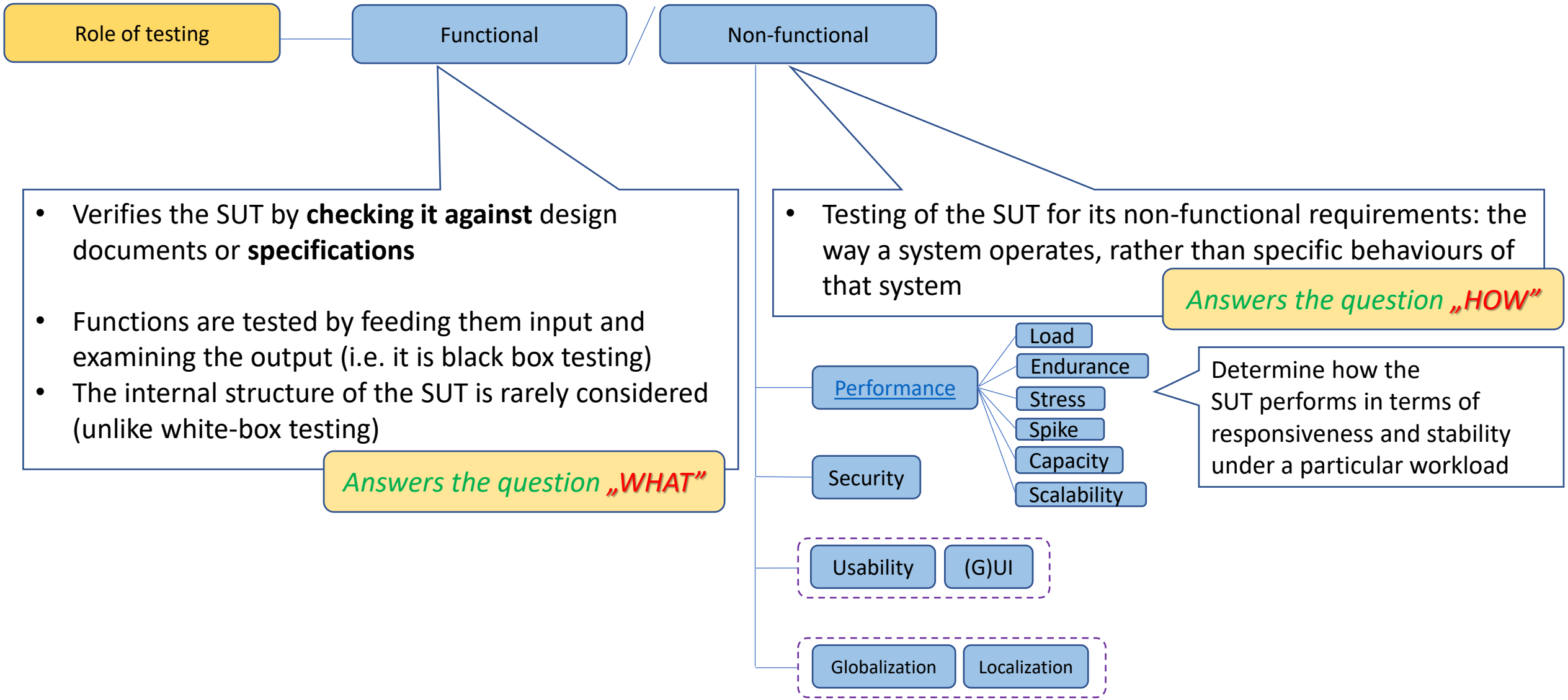


Provide the end users with confidence that the system will function according to their expectations

- Often the responsibility of the customers / users of a system, but project team members may be involved as well

- Test basis:
  - Business processes, user or business requirements
  - Regulations, legal contracts and standards, use cases
  - System requirements, user documentation
  - Installation procedures, risk analysis reports
- Test objects:
  - SUT, system configuration and configuration data
  - Business processes, disaster recovery sites
  - Operational or maintenance processes
  - Forms, reports

Classification of testing



Classification of testing

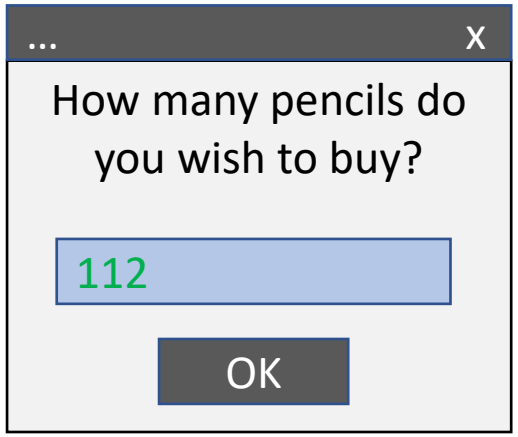
Scenario

Positive

Negative

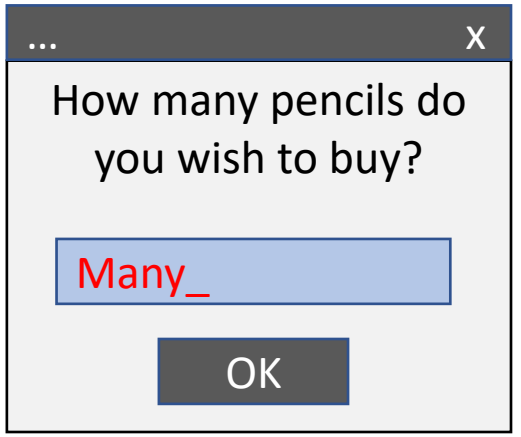
Determines whether the SUT works as expected

- If an error occurs, the test failed



Check how the system handles invalid input or unexpected behaviour

- Gracefully?
- Crashes?



Classification of testing

Degree of automatization — from the view of test execution — Manual / Semi-automated / Automated

- Test engineers simulate the end user and go through the features manually

+:

- Cheap at short term (no program code)
- Can be more dynamic
- Tester sees the application during testing, which may make it more user-friendly

→ suitable in prototype phase or exploratory testing

-:

- Very time-consuming
- Less reliable, ad-hoc, low-accuracy
- Regression testing laborious

- Execute pre-scripted tests against the SUT, then compare the test results to the expected behaviour and automatically report it to the testers

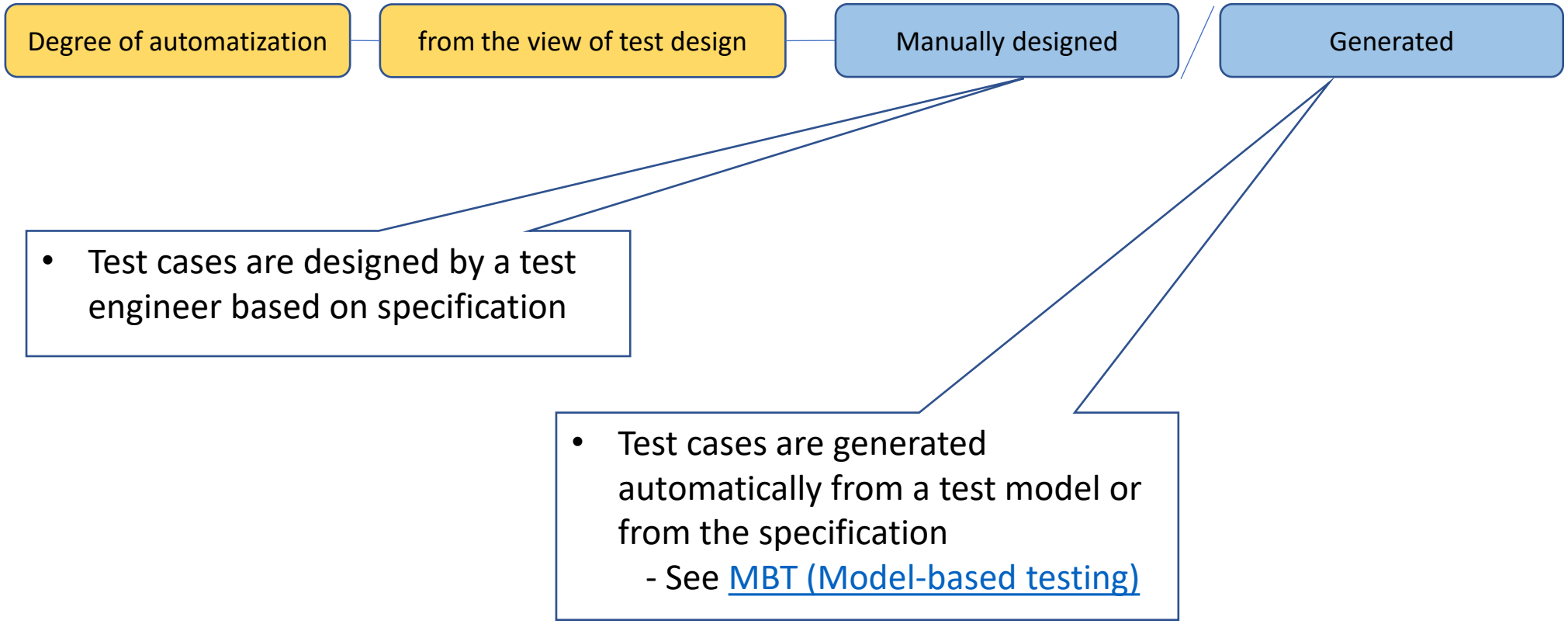
+:

- Much cheaper in long term
- Runs much faster
- Regression testing is easy
- More accurate and reliable results

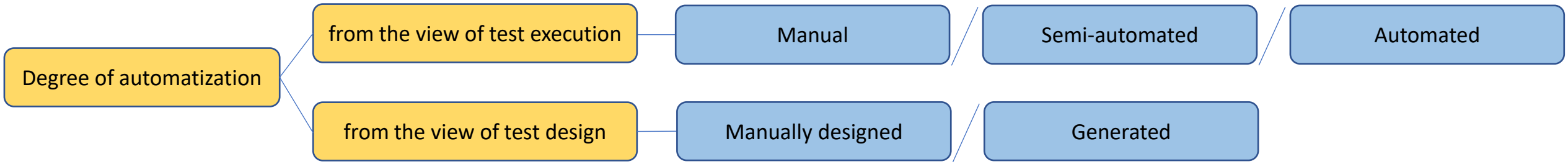
-:

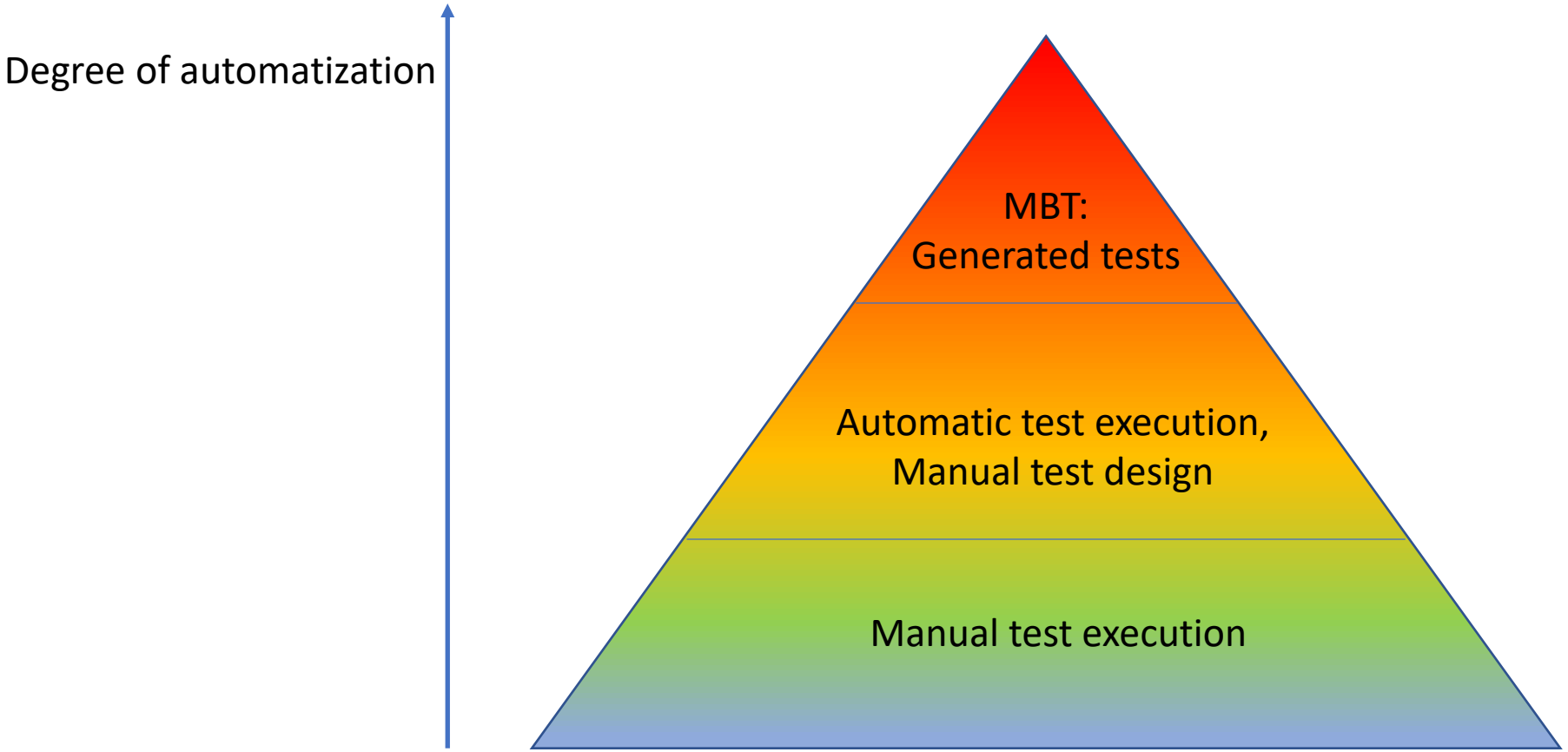
- Higher short term cost (due tools, programming, competence build-up)
- The changes applied to the SUT may result on high maintenance cost of tests

Classification of testing



Classification of testing





Ok, previously we talked about theory…

…but let’s face reality

Bad examples from the industry that should avoided…

- Don’t bother with failed commit tests. They have never worked…
  - Why haven’t they been checked and/or deleted?
- Test fails if a given message does not have the desired structure, but passes otherwise
  - What if the given message is has not been received at all? Do not use “pass” verdict by default. Always check for alternatives!
- The focus is only on performance, but functionality behind has never been tested
  - Test performance only if you are sure that the functionality works properly

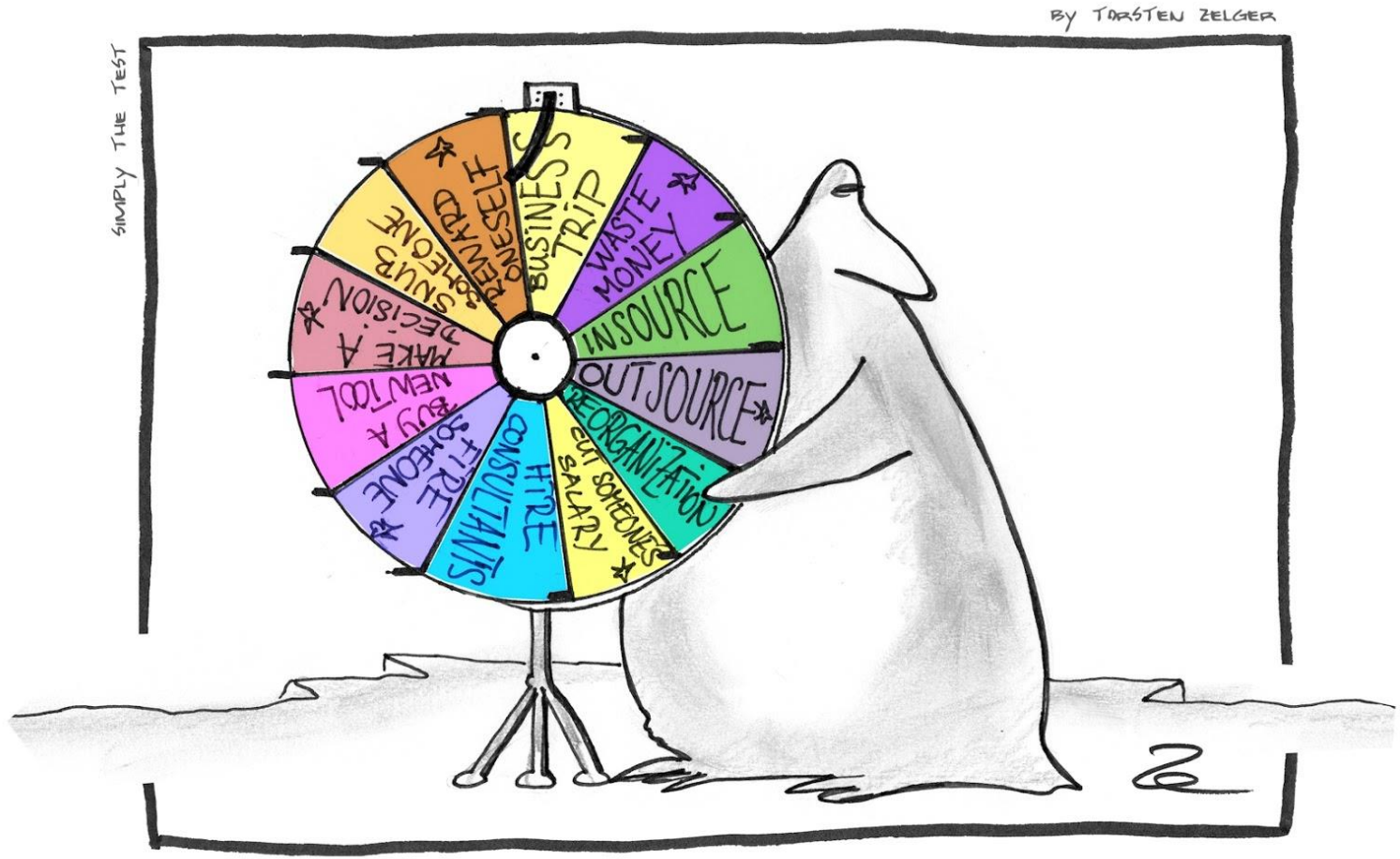
Bad examples from the industry that should avoided…

- It is not necessary to retest all elementary functions - such as install - for each version…
  - Not true. Even the build process can be ruined by a mistake…
- Forgetting to focus on boundary conditions / precision
  - Civilization: Gandhi character: aggression rating: 1 – 2 = -1 → 255 (due int)
  - Siemens train guard system timing problems on parallel tracks
- Using different formats without focusing on conversion
  - Ariane V explosion due to 64 bit floating point → 16 bit unsigned int conversion
  - Mars Climate Orbiter's lost due to the lack of conversion between different units (pounds vs. Newton)
- Synchronization problems
  - Boeing Starliner didn't reach international space station due 11 hours offset in the mission clock of spacecraft

Bad examples from the industry that should avoided…

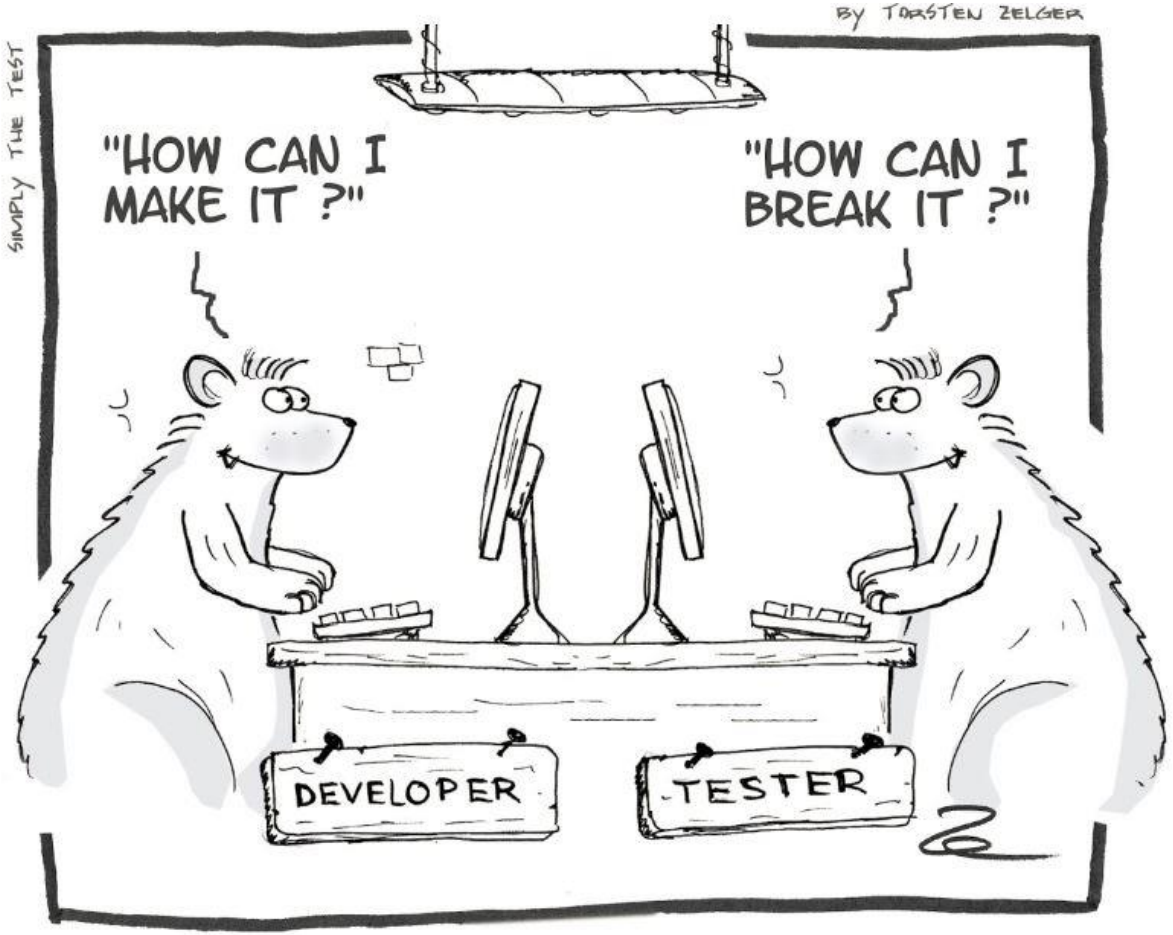
- We have X lines of test code, Y number of tests for a given functionality
  - Management likes to measure numbers like these, but it does not say anything about the quality of the tests
  - Even when the quality of a test is good, it does not say anything about coverage
  - Even the coverage seems to be ok (has a given high percentage), it does not say anything about whether the most important features are covered by test or not

Bad examples from the industry that should avoided…



Where management decisions have their roots

To conclude testing…



They were not so much different, but they had different goals

Testers meeting at the bar

To conclude why we need to test systems:

SYSTEM:

- never ready,
  - if ready doesn't work,
    - if works doesn't do what I want,
      - if does what I want I am wrong.

Definition from Gábor Németh: Parallel Architectures

Requirement modelling & traceability

Main ToC

Overview of Requirement modelling & traceability

- Feature Specification
  - Use cases and user stories
- Standards
- Requirement modelling
  - SDL
  - UML
  - TDL
  - Behaviour trees
- Traceability of requirements

Main ToC

Feature Specification Document

- Description of the required functionalities in a semi-formal way that can be given to the development team
  - classified into use-cases or user stories

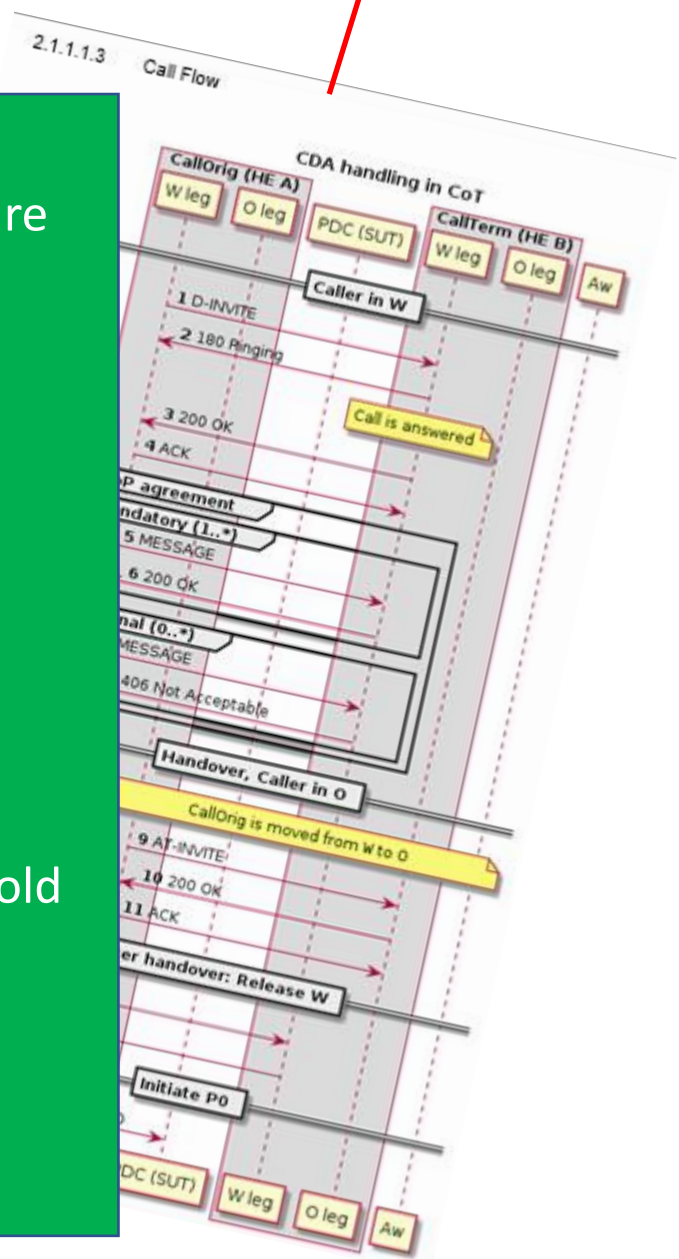
Use cases:

- *Business artefacts* defining some software requirements.
- Describes the actions or steps of events:
  - *Precondition*
  - *Action 1*
  - *Action 2*
  - *Postcondition*

User stories:

- Short, simple descriptions of a feature told *from the perspective of the customer*.
- They typically follow a simple template:
  As a <type of user>, I want <some goal> so that <some reason>.

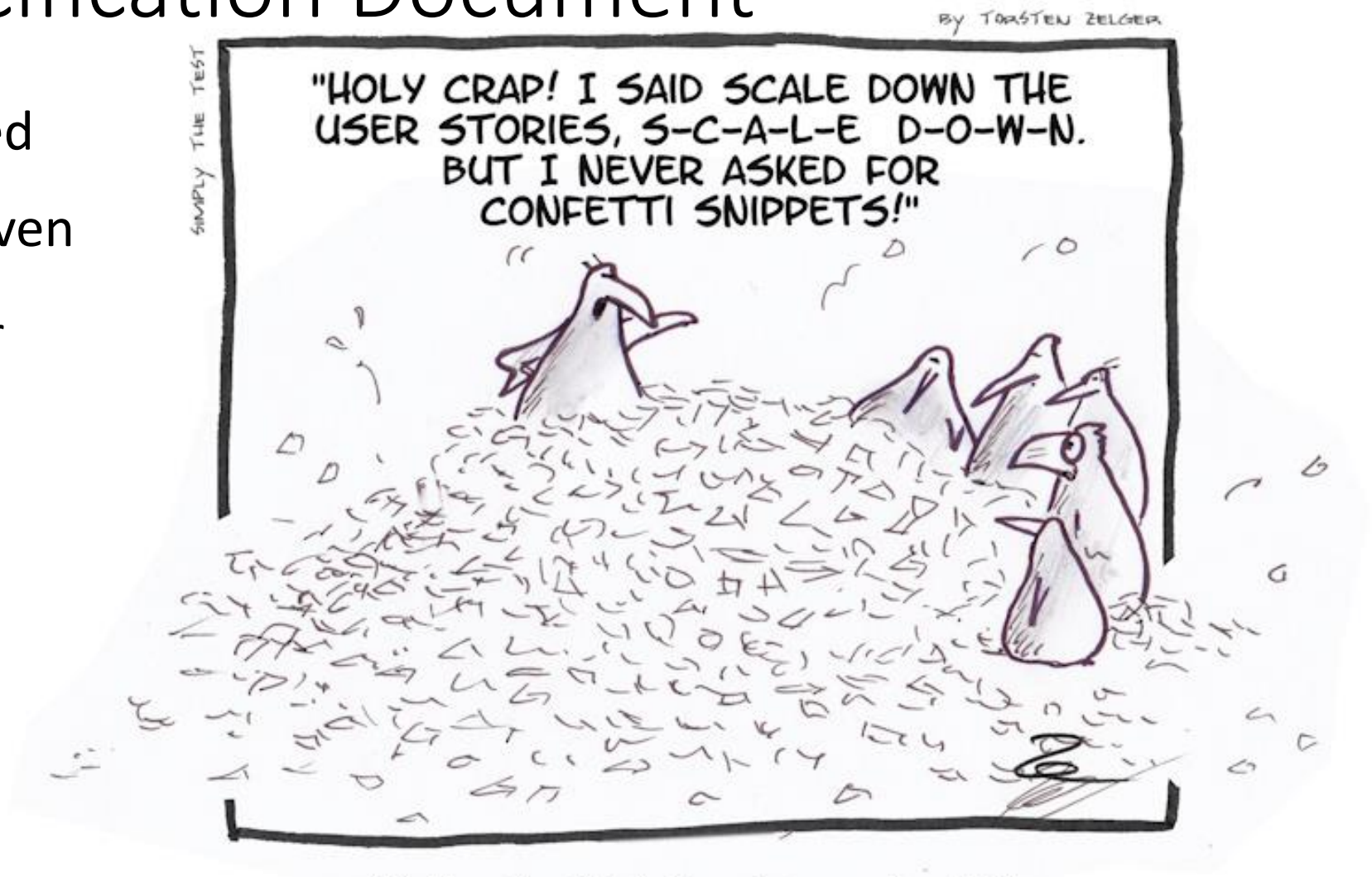
A behaviour description of a use case



A behaviour description of a use case

Feature Specification Document

- Description of the required functionalities in a semi-formal way that can be given to the development team
  - classified into use-cases or user stories



Finding the right balance between "too big" and "too small" was an everlasting discussion

Feature Specification Document

- Description of the required functionalities in a semi-formal way that can be given to the development team
  - classified into use-cases or user stories
  - A part of it may contain formal descriptions *(like the message sequence in the figure)*
- Should be self consistent (provide used abbreviations, references…etc.)
- Should contain information about risks
- May contain information about test design

Artf010416: Efficient CDA handling in CoT

Keyword(s)

D-INVITE, CoT, DSoP, WO handover

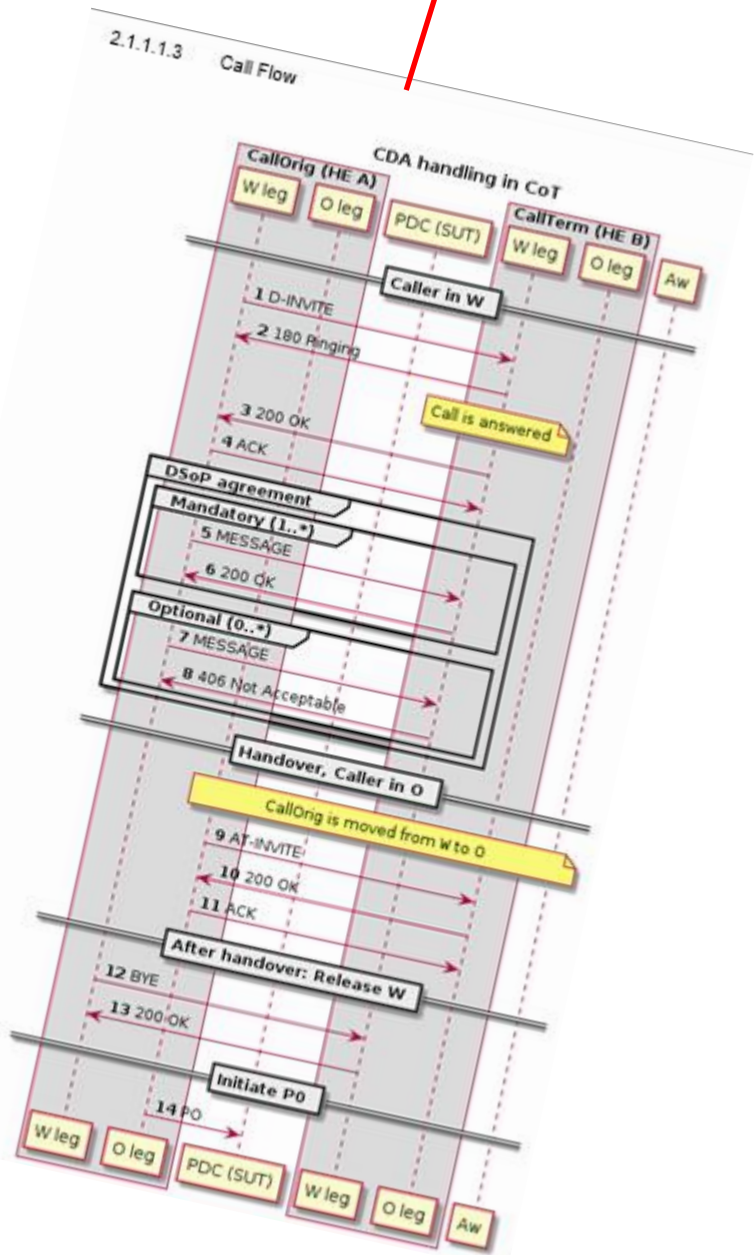
Abstract

This feature specification is intended to be an agreement between t
different CoT of XYZ on the detailed requirements related to CDA.

Contents

1 Introduction
1.1 Revision Information
1.2 Statement of problem
1.3 Input Documents
1.4 Scope
1.5 Abbreviations
1.6 Glossary
2 Expected System Functionality and Characteristics
2.1 Use Case Impacts
2.2 System Impacts
2.3 Test Analysis
2.4 Non-Function Requirements
2.5 Customer Impacts
2.6 Backwards Compatibility
2.7 Technical Risks

A behaviour description of a use case



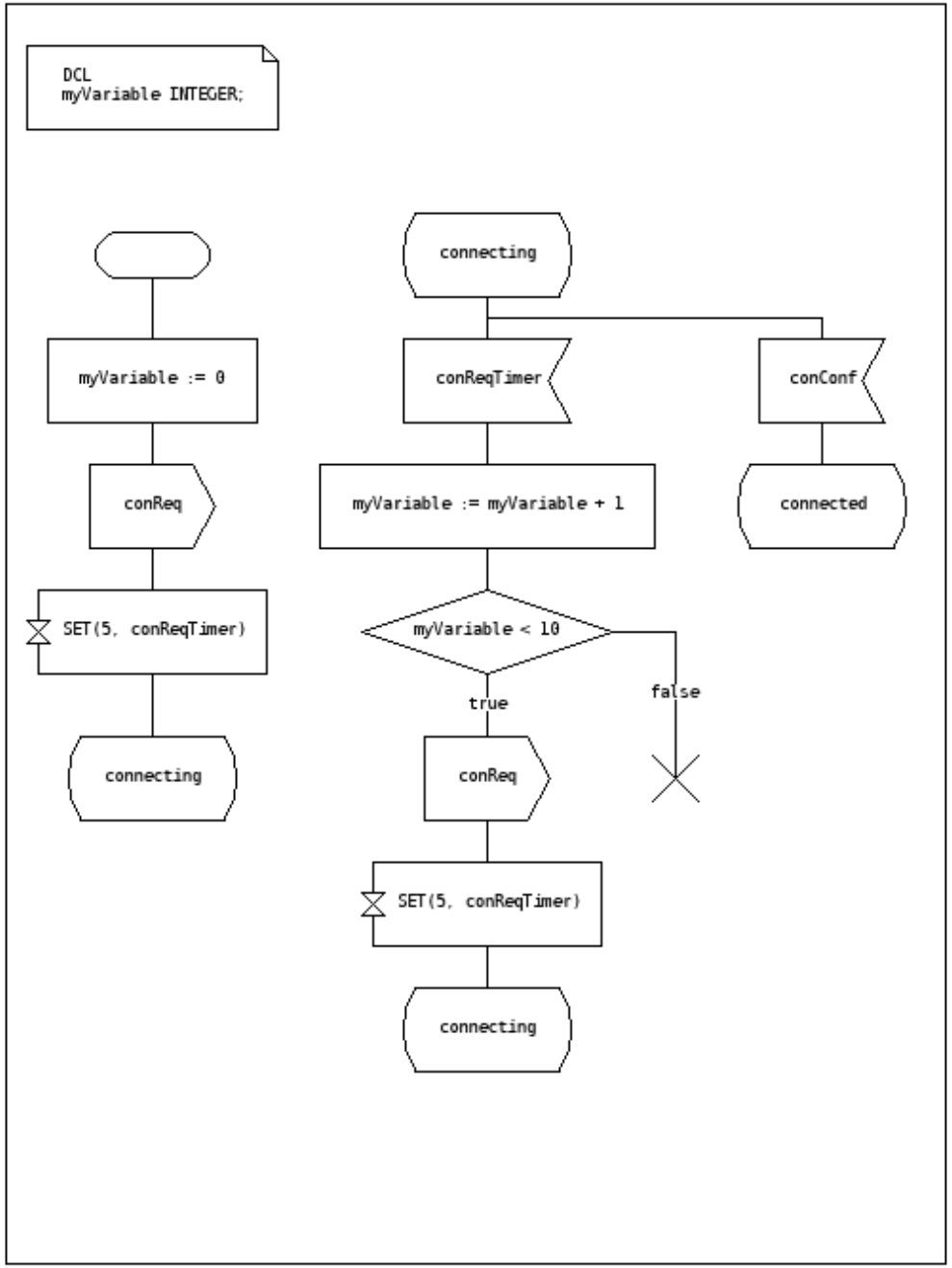
Standards

- Established norm or requirement in regard to technical systems
- Formal document that establishes uniform engineering or technical criteria, methods, processes, and practices

- Examples:
  - An RFC standard: RFC 3261 SIP: Session Initiation Protocol
  - A 3GPP standard: 32.299 Diameter protocol, charging management

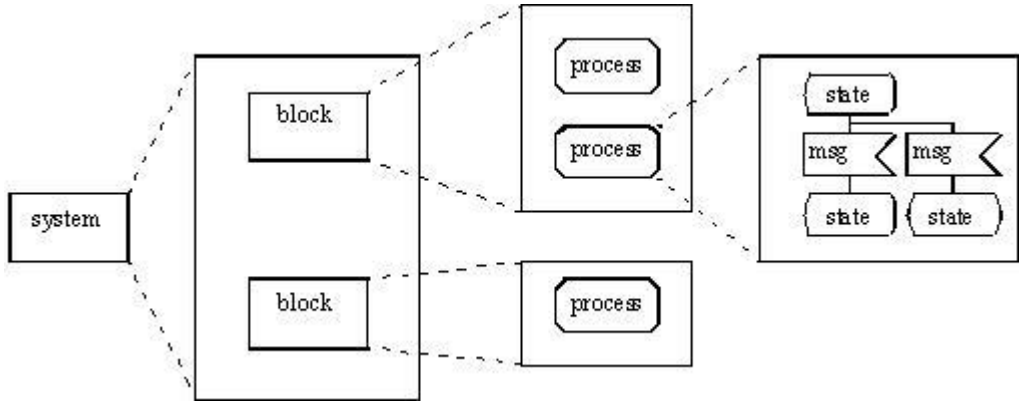
SDL

- Specification and Description Language (SDL)
  - Specification and description of the behaviours of the features in a formal way
  - Extension of the Finite State Machine (FSM) formalism
    - Each process agent is a state machine that contributes to the action carried out by the system
    - A message stimulus coming from the environment or from another agent to an agent is called a signal

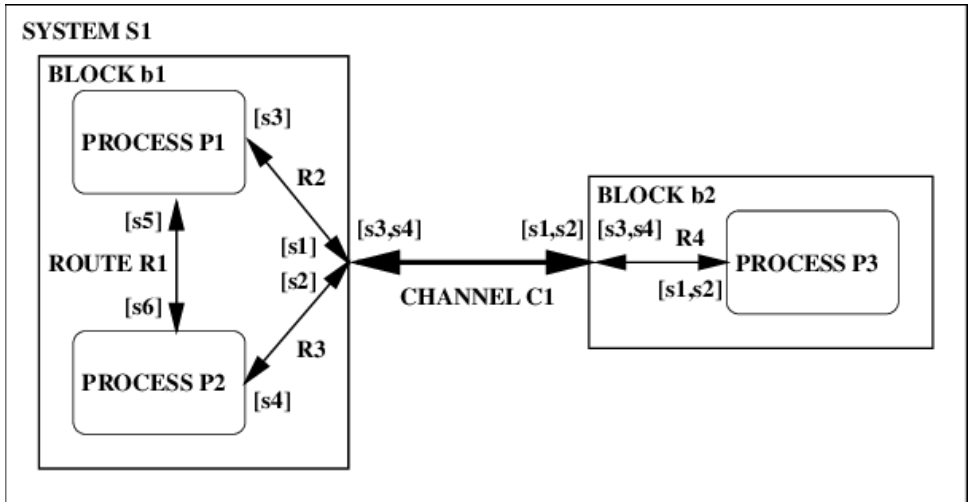


SDL (Specification and Description Language)

- Supports hierarchy: functional blocks can be decomposed in sub-blocks

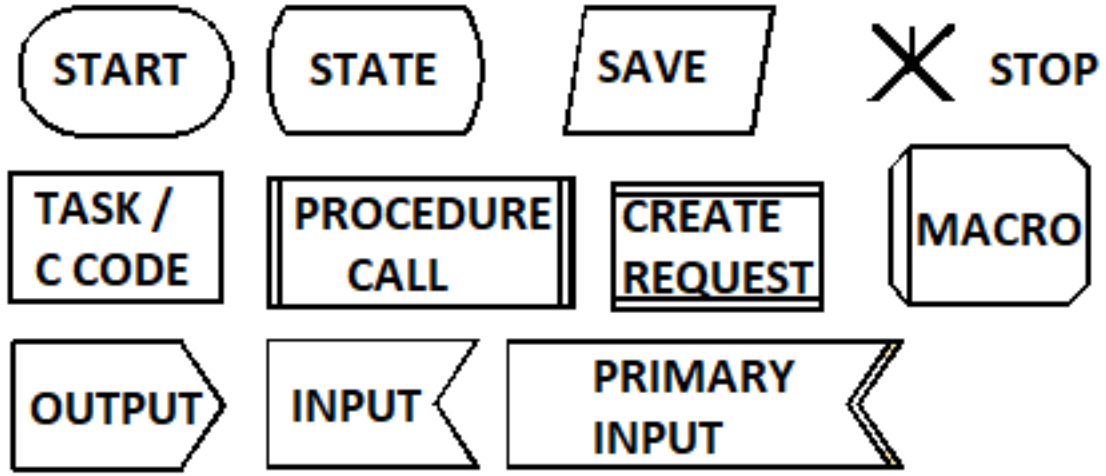


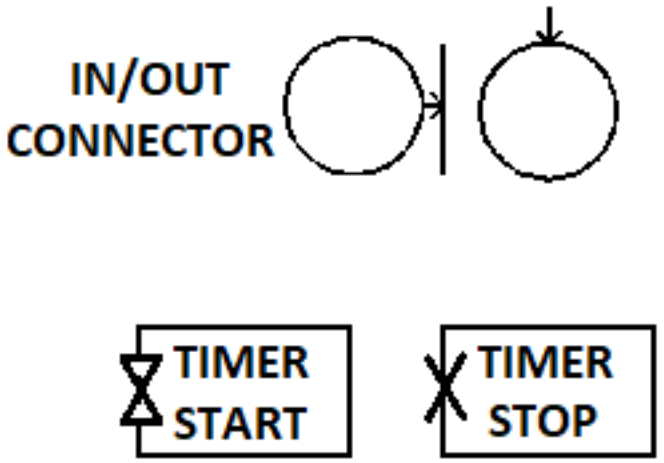
- Blocks are connected through channels that carry the messages (or signals) exchanged between the blocks

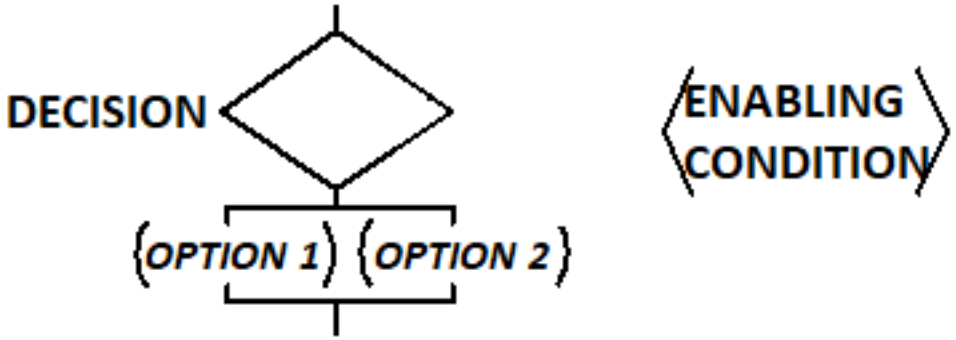


SDL (Specification and Description Language)

- SDL elements:

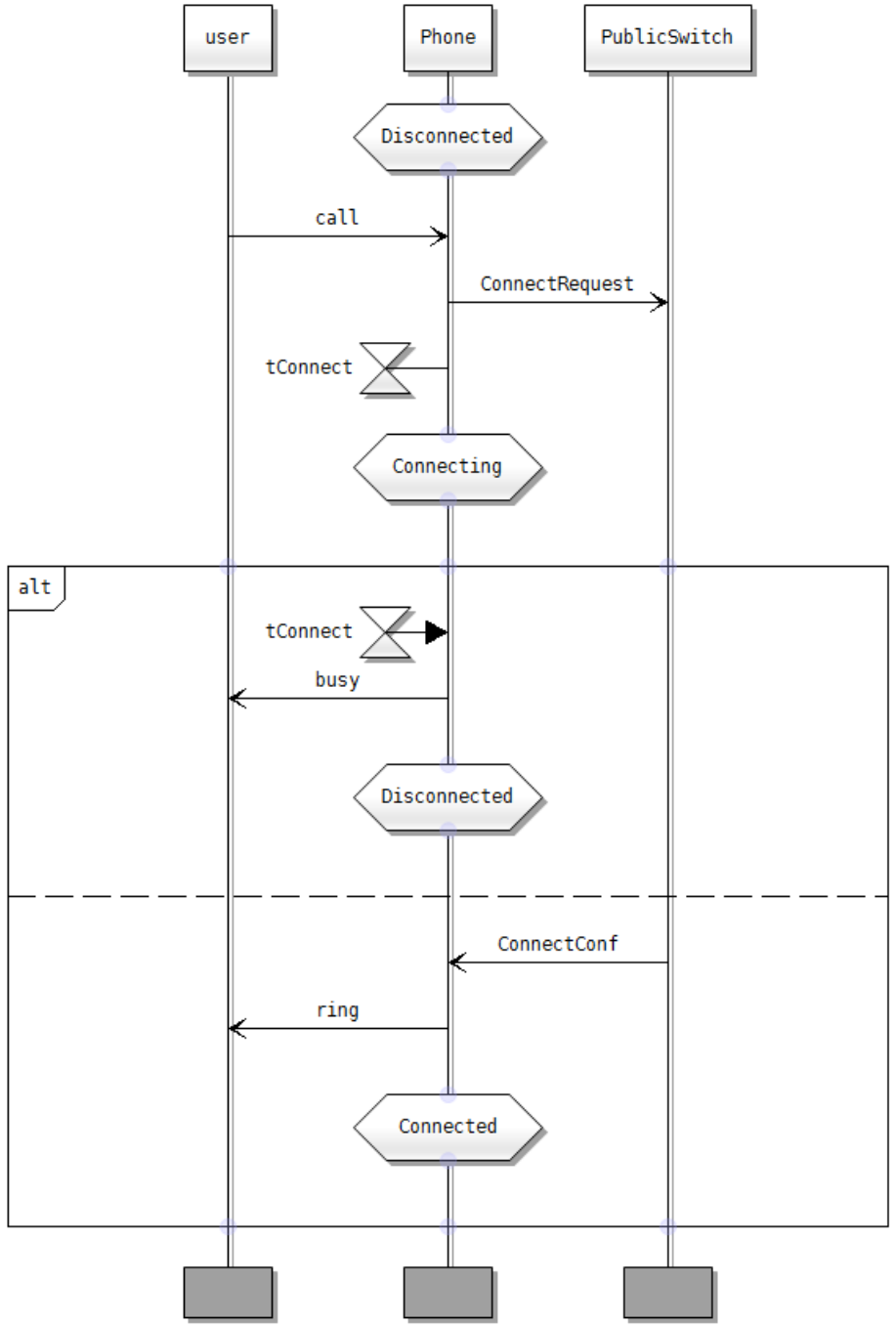






MSC (Message Sequence s)

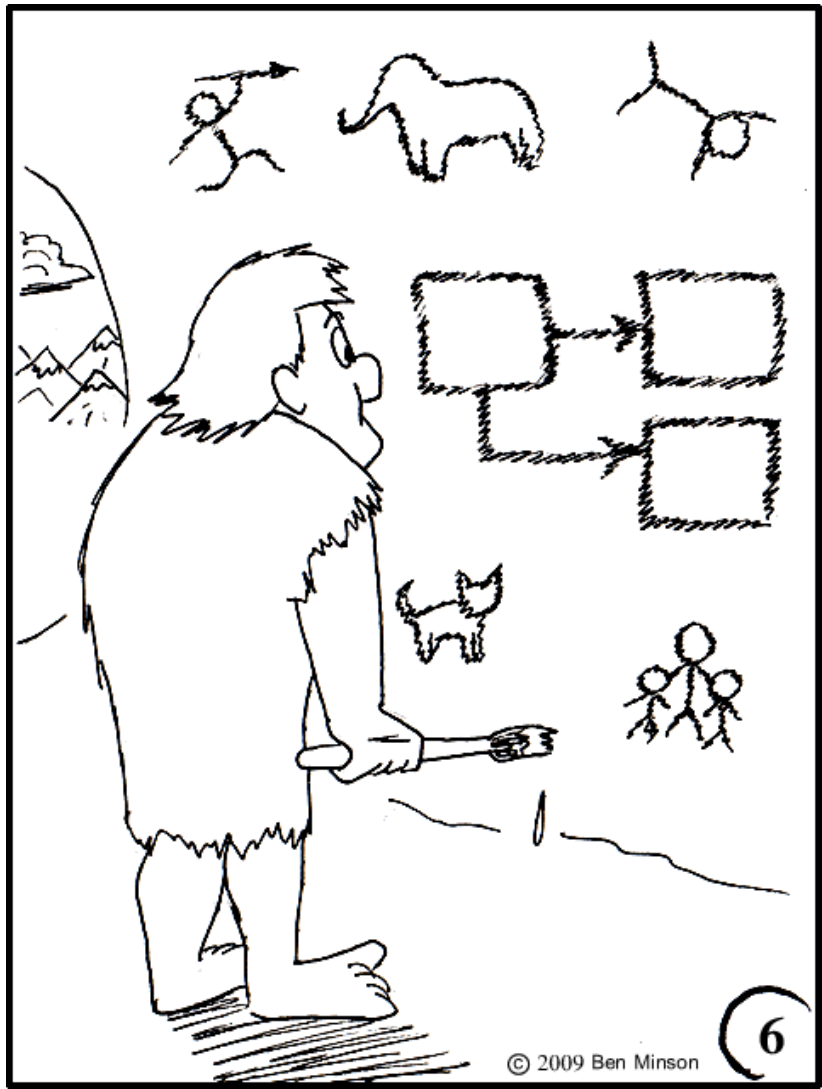
- Interaction diagram from the SDL family
- Describes the communication behaviour of system components and their environment by means of message interchange
- Similar to sequence diagram of UML
- Elements:
  - Participants: Lifelines (vertical lines)
  - Messages: Horizontal arrows between participants with the name of the message written above them
  - Timers



UML (Unified Modeling Language)

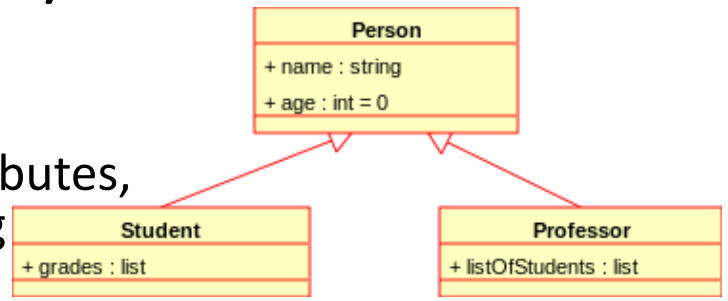
- General-purpose modeling language in the field of software engineering

- Has many types of diagrams, can be classified into two categories:
  - Structural information
  - Behaviour description

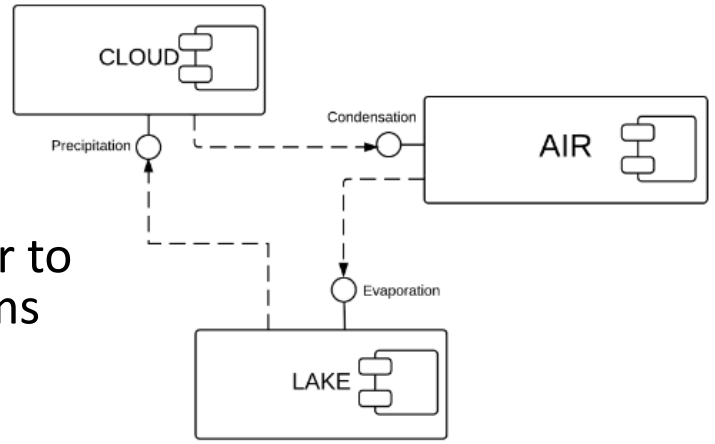


UML (Unified Modeling Language)

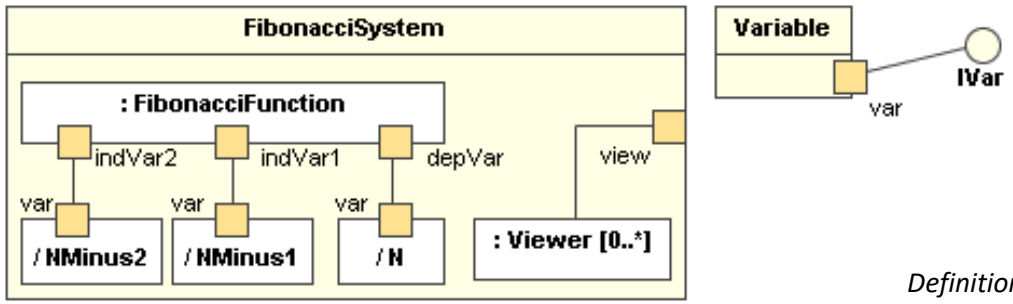
- Structural information diagrams 2/1:
  - Class diagram
    - Describes the structure of a system by showing the system's classes, their attributes, operations and the relationships among objects



  - Component diagram
    - Shows how components are wired together to form larger components or software systems



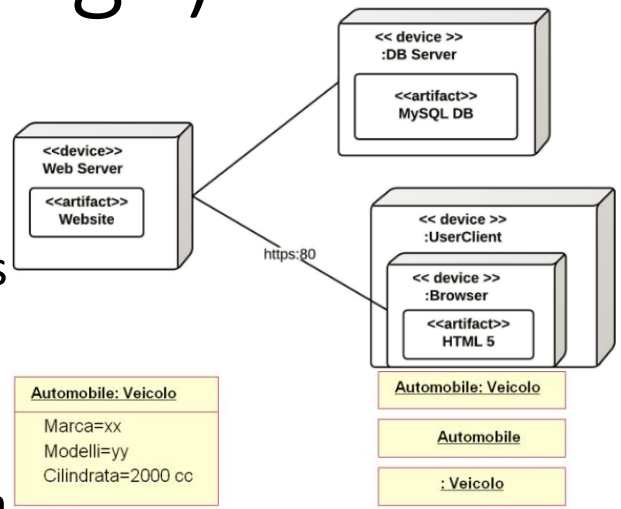
  - Composite structure diagram
    - Shows the internal structure of a class and the collaborations

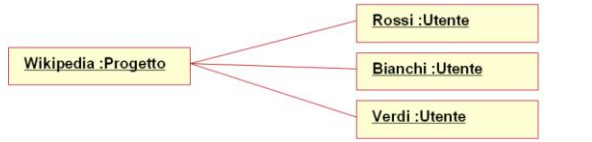


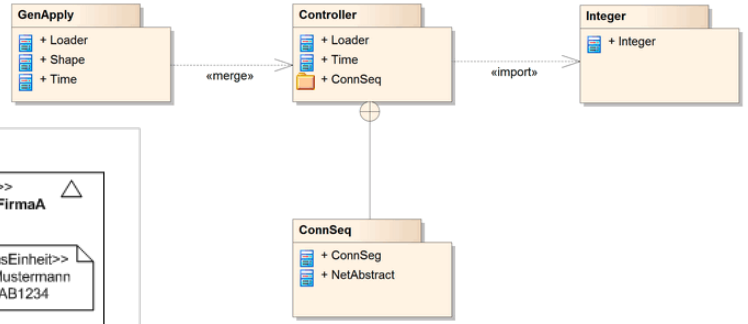
Definitions: wikipedia

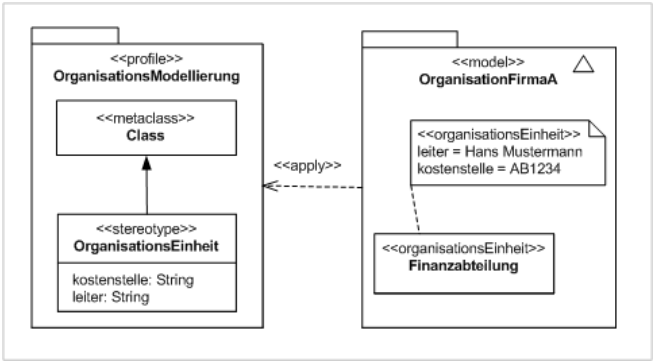
UML (Unified Modeling Language)

- Structural information diagrams 2/2:
  - Deployment diagram
    - Models the physical deployment of artifacts on nodes
  - Object diagram
    - Shows a complete or partial view of the structure of a modeled system at a specific time
  - Package diagram
    - Depicts the dependencies between the packages that make up a model
  - Profile diagram
    - Displays the usage of profiles

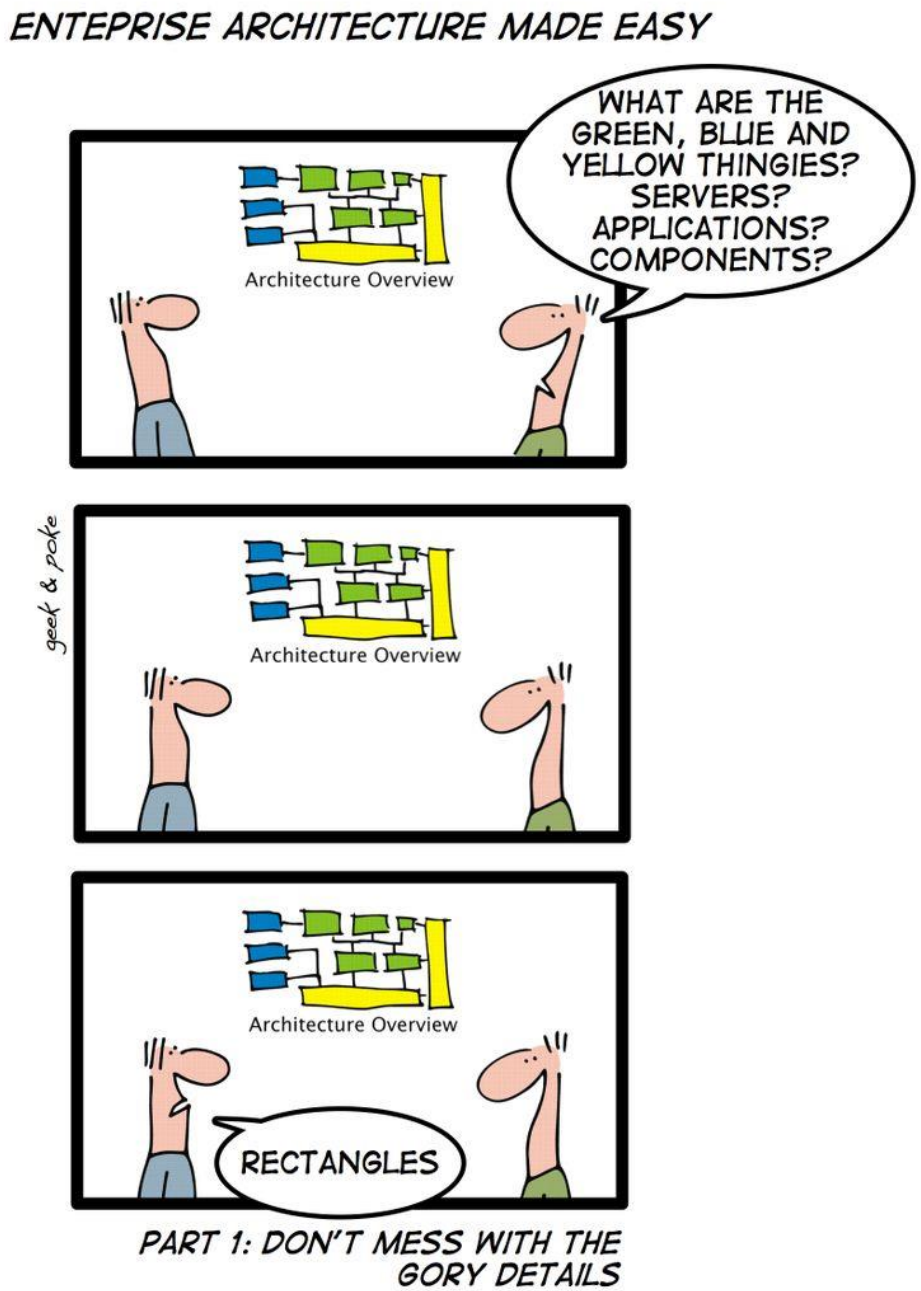






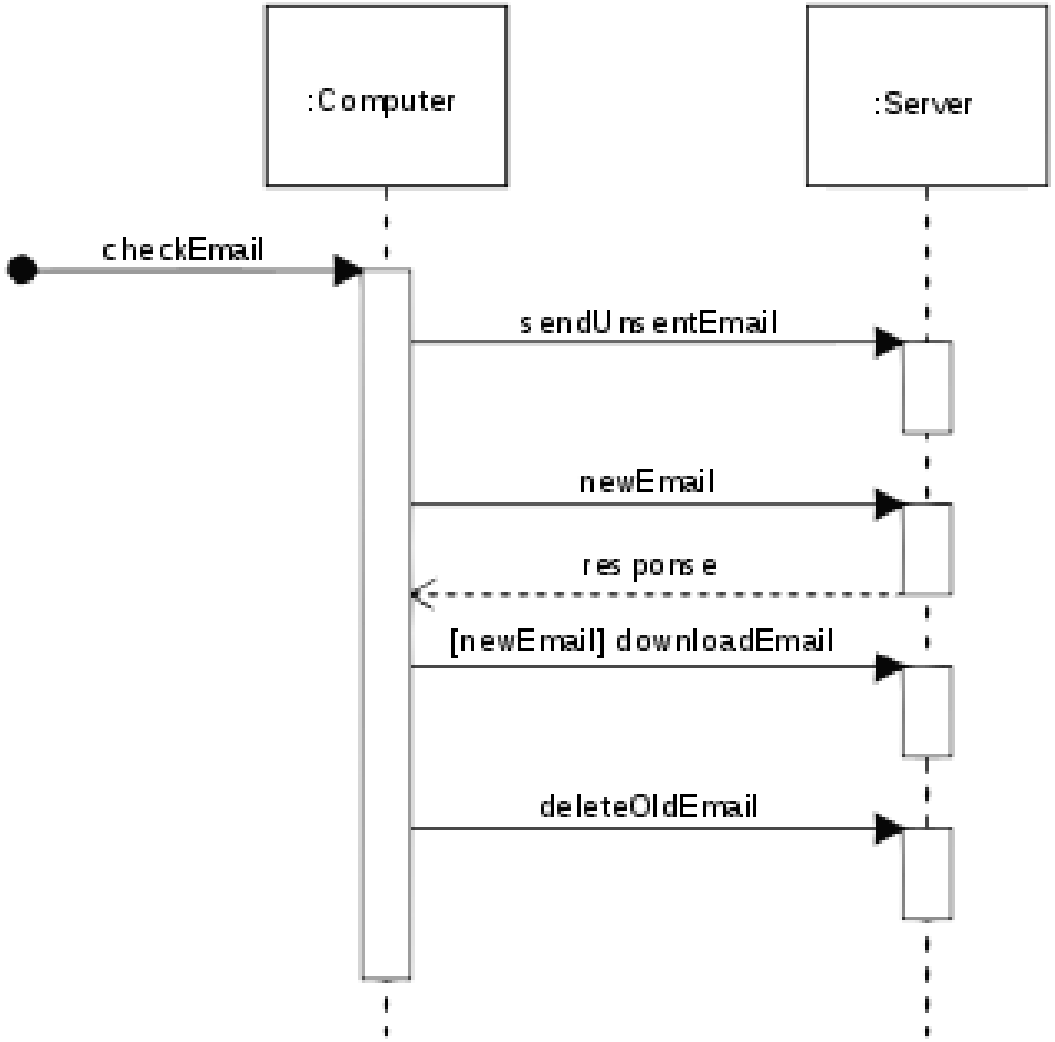


Definitions: wikipedia


ENTEPRISE ARCHITECTURE MADE EASY
WHAT ARE THE GREEN, BLUE AND YELLOW THINGIES? SERVERS? APPLICATIONS? COMPONENTS?
Architecture Overview
Architecture Overview
Architecture Overview
RECTANGLES
geek & poke
PART 1: DON'T MESS WITH THE GORY DETAILS

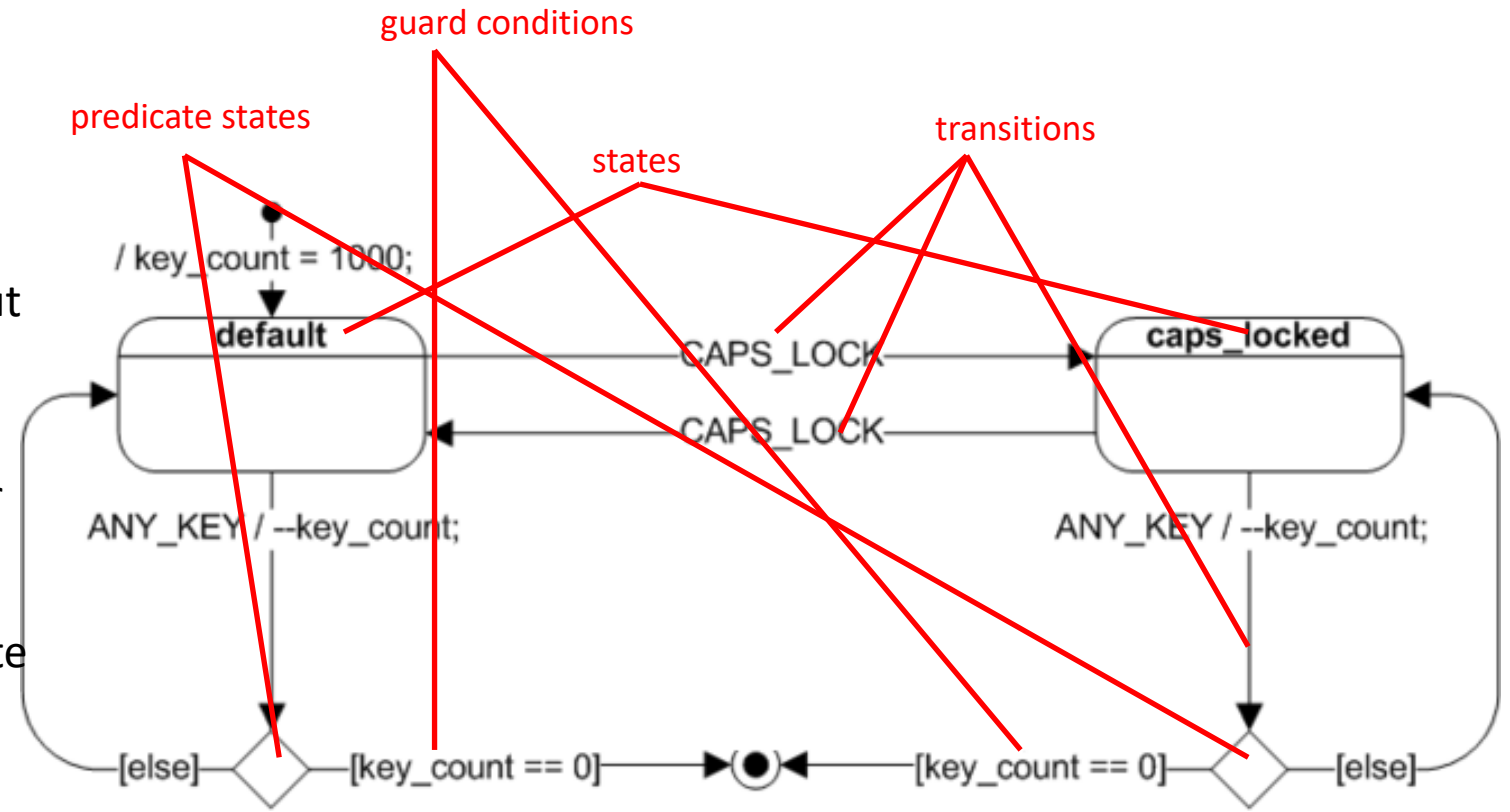
UML (Unified Modeling Language)

- Behaviour descriptions 4/1
  - Sequence (event) diagram
    - Shows the interactions of participants arranged in time sequence
    - Similar to MSC (Message Sequence s)
    - Elements:
      - Participants of communication: Objects, represented with lifelines (vertical lines)
      - Messages: horizontal arrows with the name of the message written above them
        - Solid arrow heads: synchronous calls
        - Open arrow heads: asynchronous messages
        - Dashed lines: reply messages
      - Objects can be created and destroyed during communication



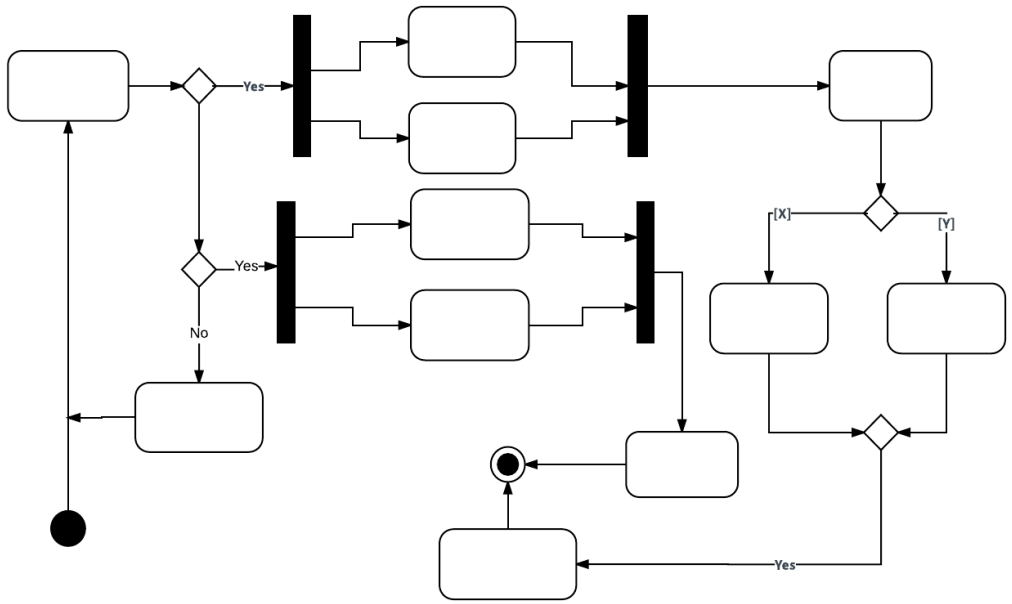
UML (Unified Modeling Language)

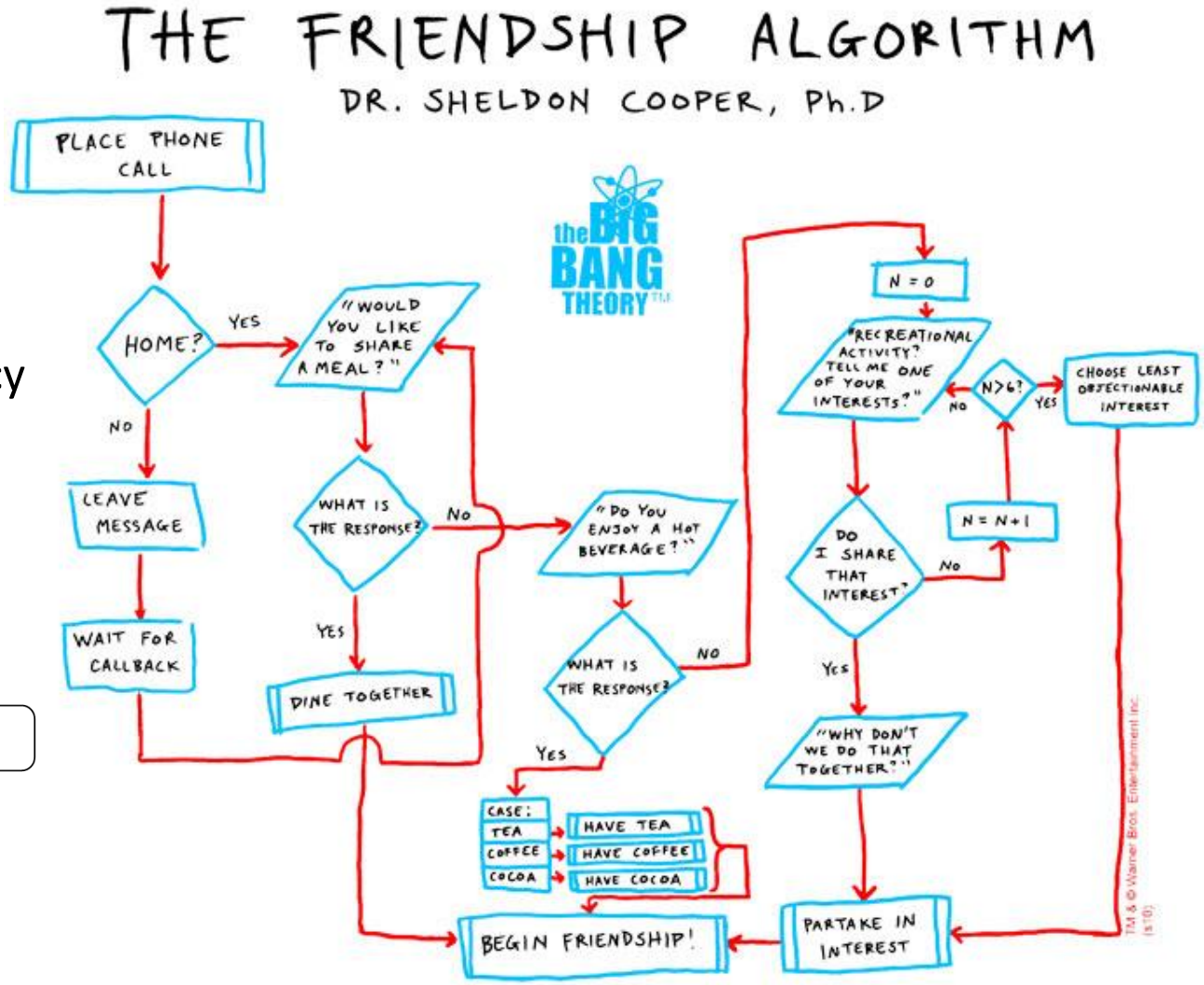
- Behaviour descriptions 4/2
  - State diagram
    - Similar to SDL
    - Extension of the Finite State Machine (FSM) formalism:
      - States
      - Transitions between states
        - Selected with a given input
      - Variables
      - Predicates over variable values
        - The guard conditions over variables decides what transition to choose originated from a predicate state



UML (Unified Modeling Language)

- Behaviour descriptions 4/3
  - Activity diagram
    - Representation of workflows
      (activities & actions)
    - Supports choice, iteration and concurrency

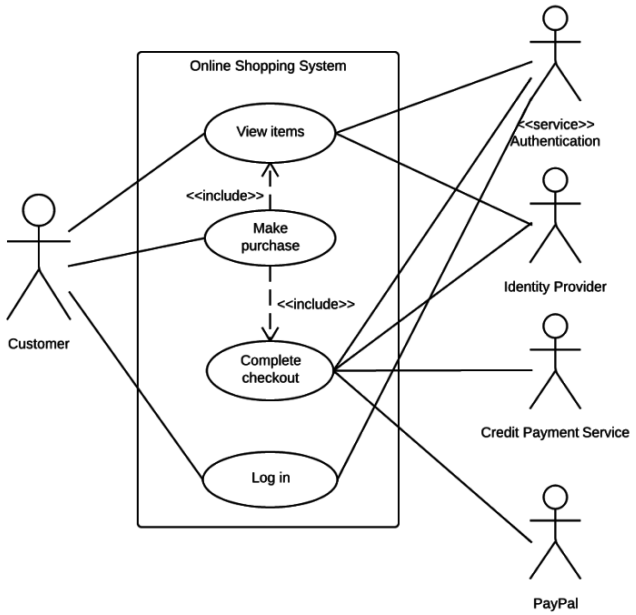


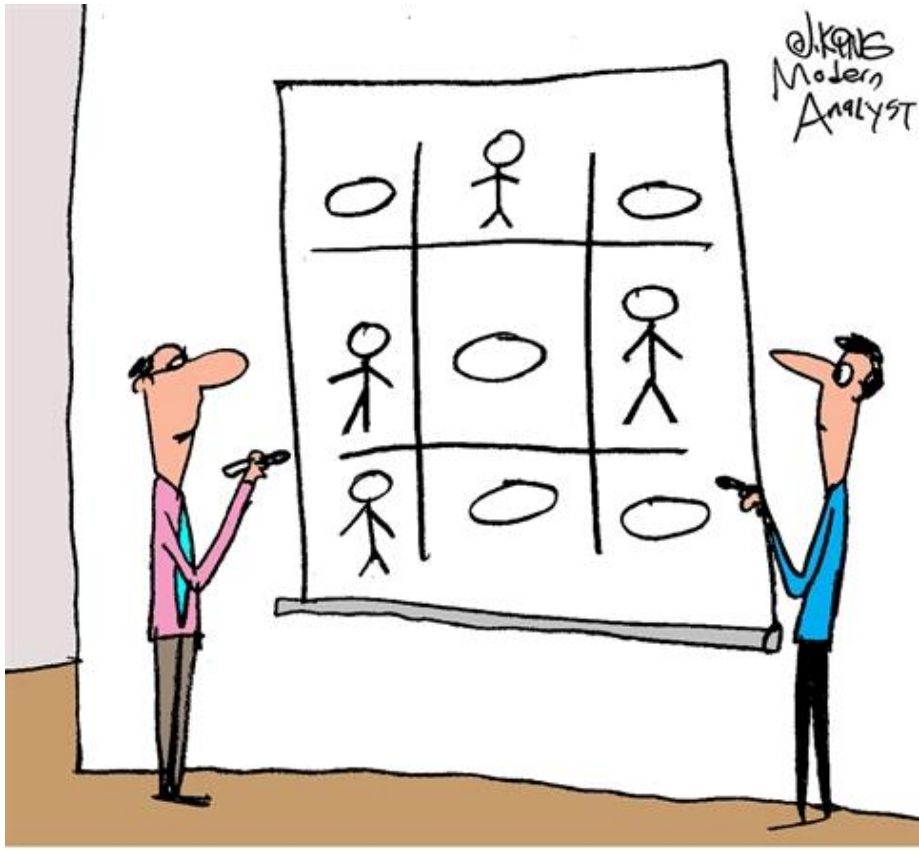


Source of the figure

UML (Unified Modeling Language)

- Behaviour descriptions 4/3
  - Use case diagram
    - Representation of a user's interaction with the system
      - Shows the relationship between the user and the different use cases in which the user is involved
    - Can identify different types of users and different use cases

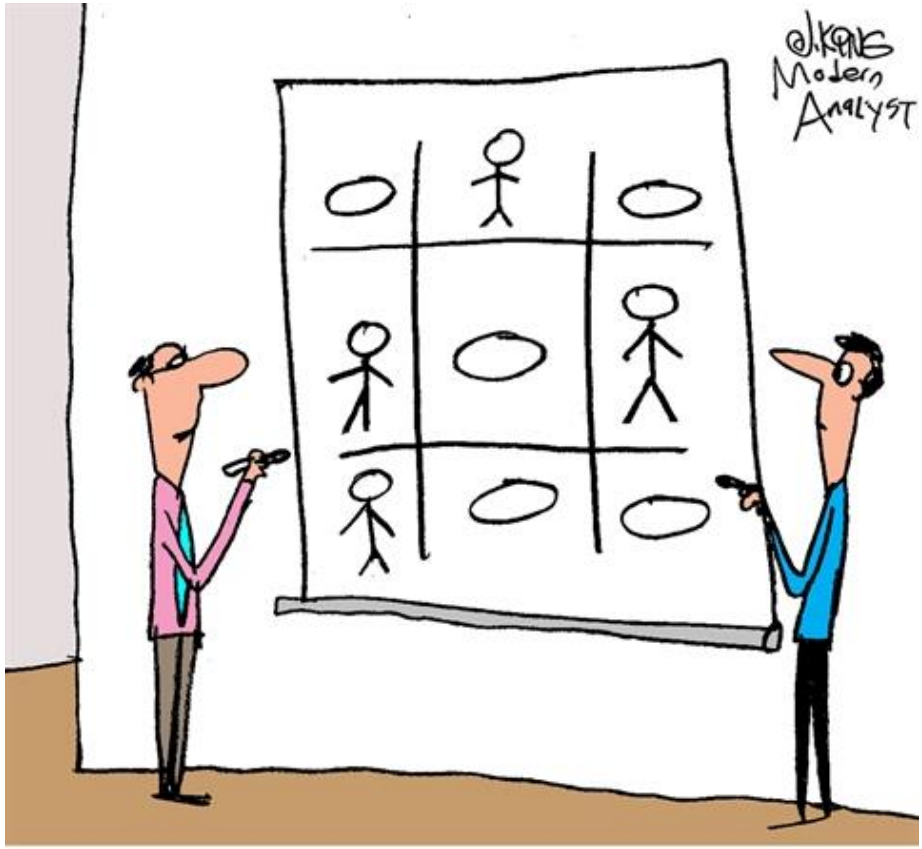




Jerry and Bob, the new business analysts, decided to settle, once for all, whether use cases or actors are more important in the use case diagram.

UML (Unified Modeling Language)

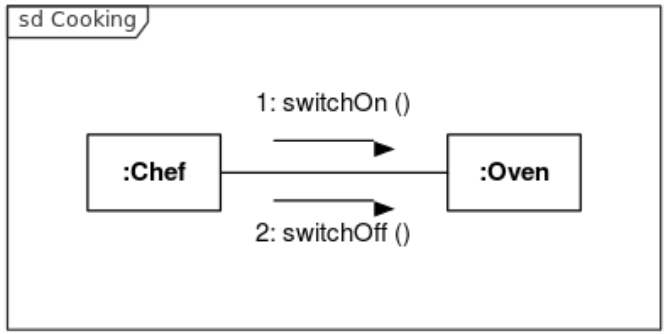
- Behaviour descriptions 4/3
  - Activity diagram
    - Representation of workflows (activities & actions)
    - Supports choice, iteration and concurrency
  - Use case diagram
    - Representation of a user's interaction with the system
      - Shows the relationship between the user and the different use cases in which the user is involved
    - Can identify different types of users and different use cases

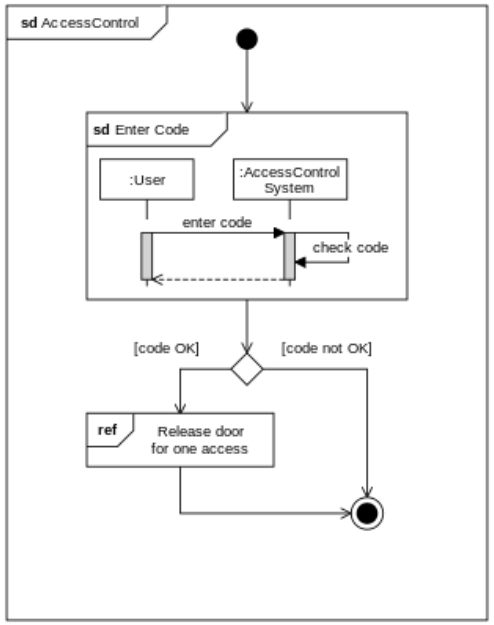


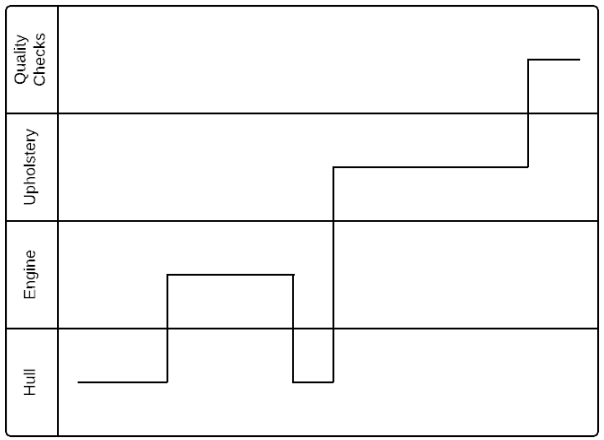
Jerry and Bob, the new business analysts, decided to settle, once for all, whether use cases or actors are more important in the use case diagram.

UML (Unified Modeling Language)

- Behaviour descriptions 4/4
  - Communication diagram
    - Models the interactions between objects or parts in terms of sequenced messages
  - Interaction overview diagram
    - Visualizes a sequence of activities (as the activity diagram does)
    - Each individual activity is pictured as a frame which can contain a nested interaction diagram
  - Timing diagram
    - Focus is on timing constraints



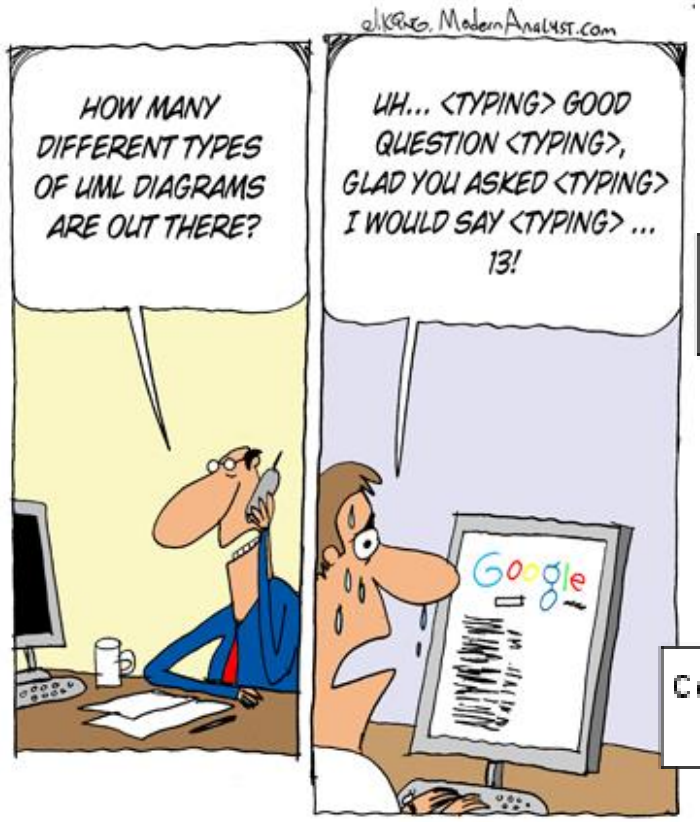


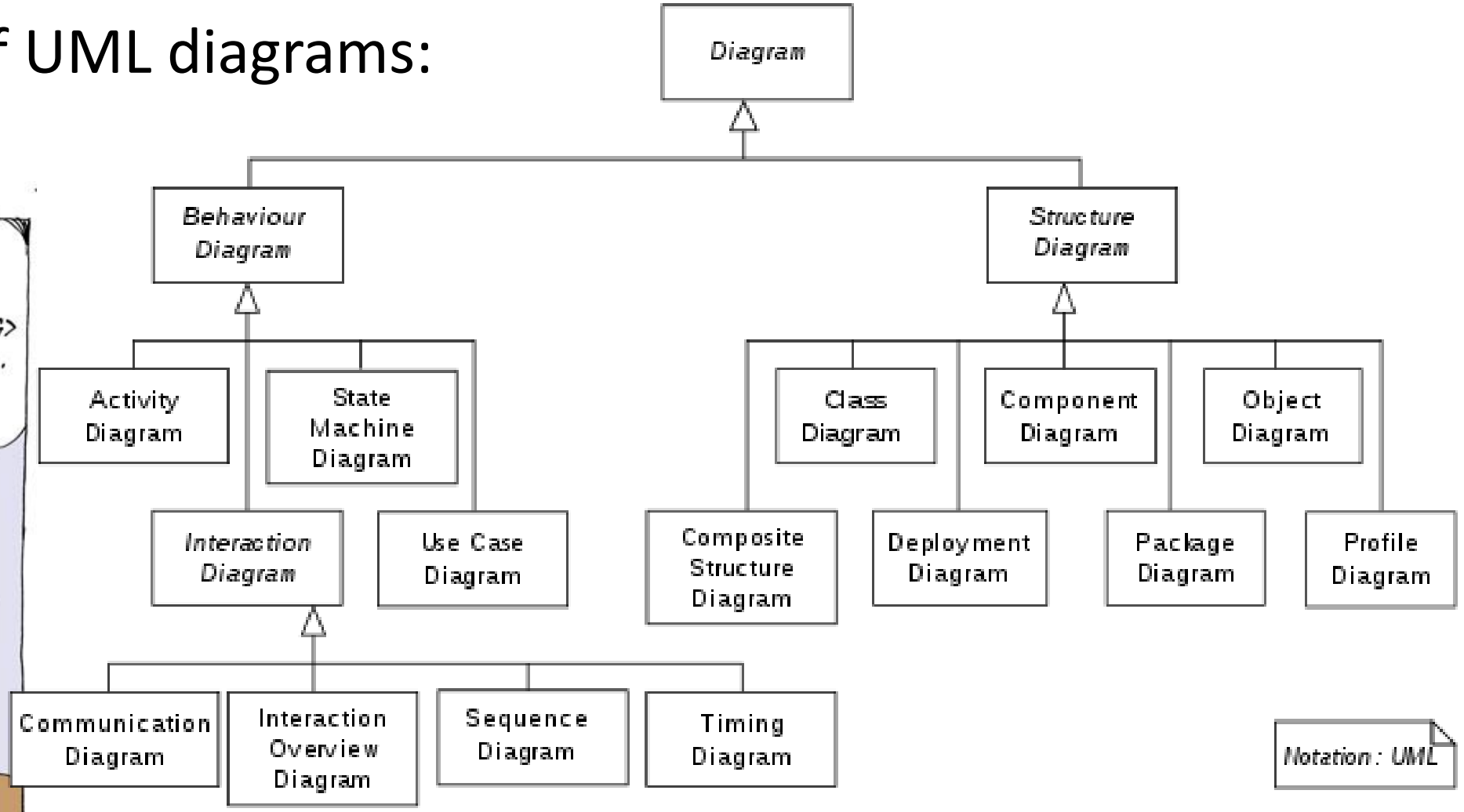


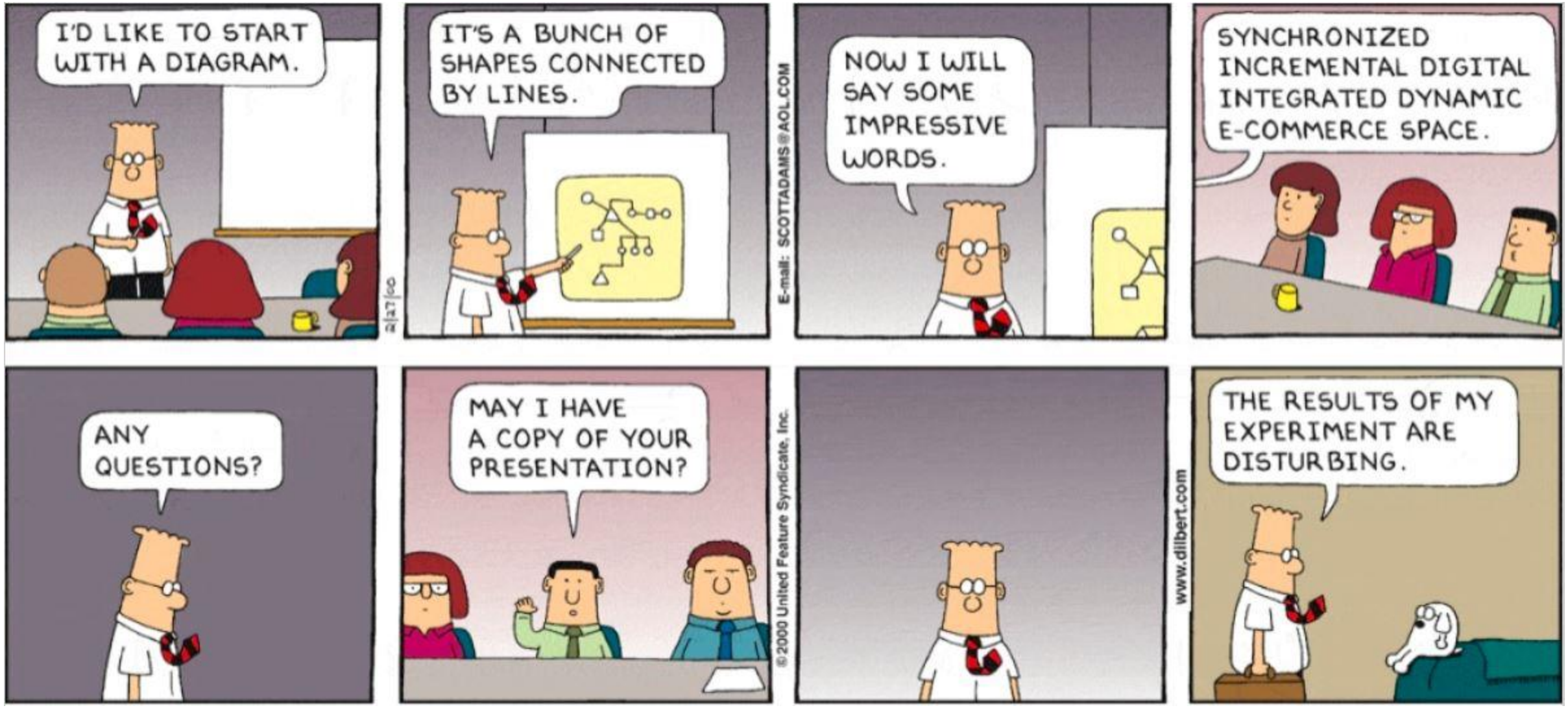
Definitions: wikipedia

UML (Unified Modeling Language)

- Overview of UML diagrams:






I'D LIKE TO START WITH A DIAGRAM.
IT'S A BUNCH OF SHAPES CONNECTED BY LINES.
E-mail: SCOTTADAMS@AOL.COM
NOW I WILL SAY SOME IMPRESSIVE WORDS.
SYNCHRONIZED INCREMENTAL DIGITAL INTEGRATED DYNAMIC E-COMMERCE SPACE.
ANY QUESTIONS?
MAY I HAVE A COPY OF YOUR PRESENTATION?
© 2000 United Feature Syndicate, Inc.
www.dilbert.com
THE RESULTS OF MY EXPERIMENT ARE DISTURBING.

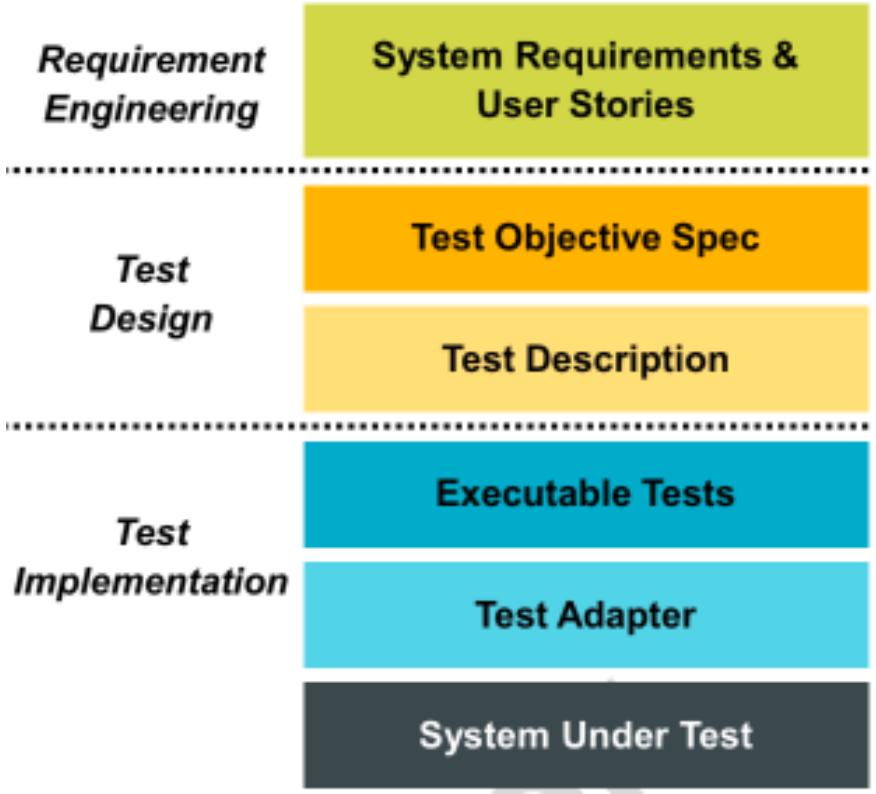
UML (Unified Modeling Language)

- Examples for UML editing tools:
  - plantUML
    - Free, open source
    - Simple text input format → image file output
    - Stand-alone tool
  - Papyrus
    - Free, open source
    - Based on Eclipse, GUI
  - NetBeans
    - Free
    - GUI
    - The UML editor is part of an integrated development environment
  - Lucid
    - Commercial
    - GUI
    - Stand-alone-tool with many interfaces
  - Microsoft Visual Studio
    - Commercial
    - GUI
    - The UML editor is part of an integrated development environment

  - For the comparison of more tools, see this link

TDL (Test Description Language)

- For design, documentation and representation of formal test descriptions
- Standard: ETSI ES 203 119-1
- Try to fill the gap between test objectives and test cases
  - test objectives: derived from system requirements
  - test descriptions: refining the test objectives
  - test cases: what we actually execute on the SUT through a test adapter



TDL – TO (Test Objective)

- Test objective can be described in a structural, formal text
  - Preconditions / Postconditions / Expected behaviour fields
  - Similar description to Gherkin

TP Id	TP_1
Test Objective	Ensure that a request is accepted
Reference	TDL TO Examples
PICS Selection	
Initial Conditions	

with {
the SS entity connected to the UE entity
}	

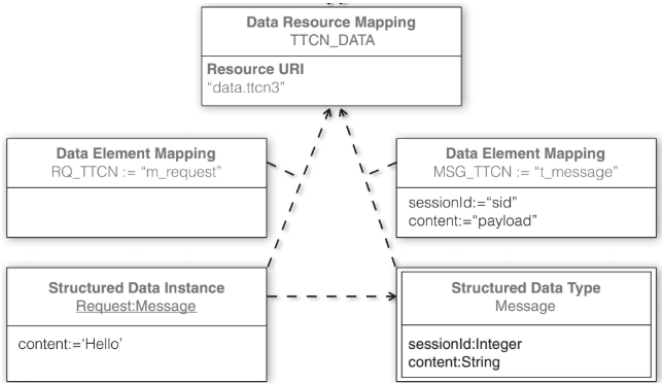
Expected Behaviour	

ensure that {
when {
the UE entity receives a Request message
}
then {
the UE entity sends an Accept message
}
}	

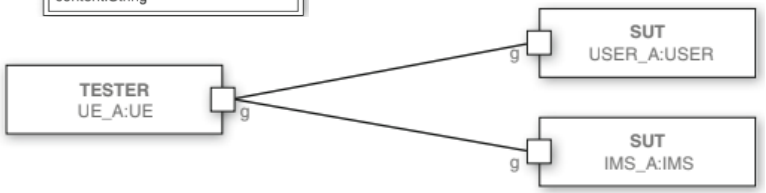
Final Conditions	

TDL – GR (Graphical Representation)

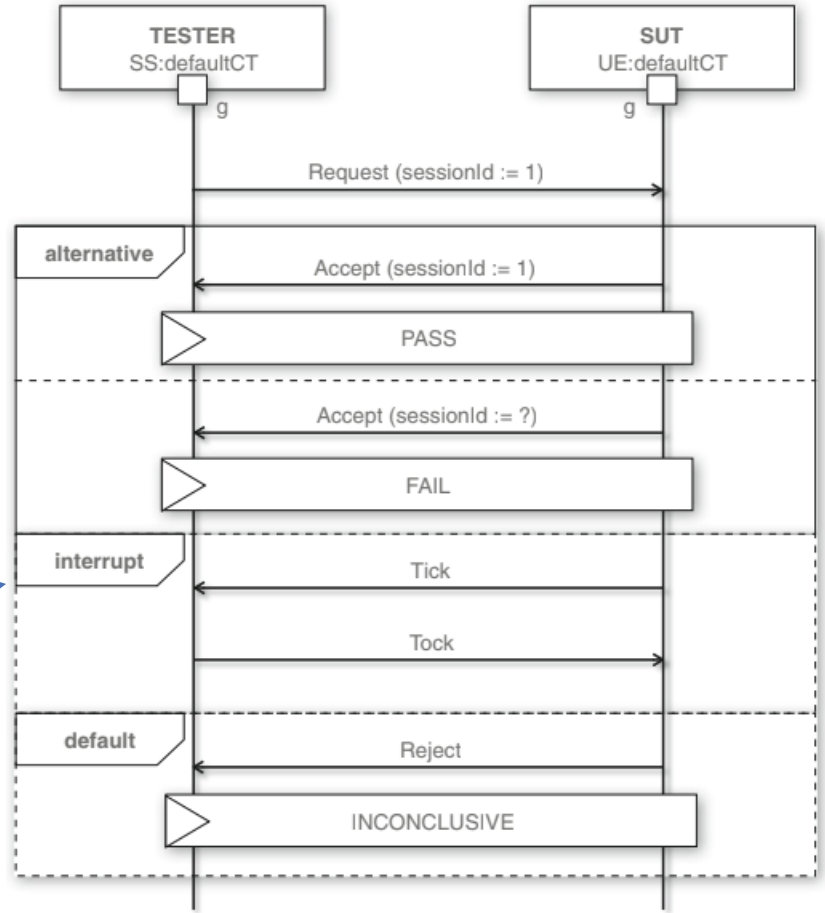
- Its diagrams are very similar to the following UML diagrams:
  - Profile diagram



  - Composite structure diagram



  - Sequence (event) diagram / MSC*



** MSC is not an UML diagram*

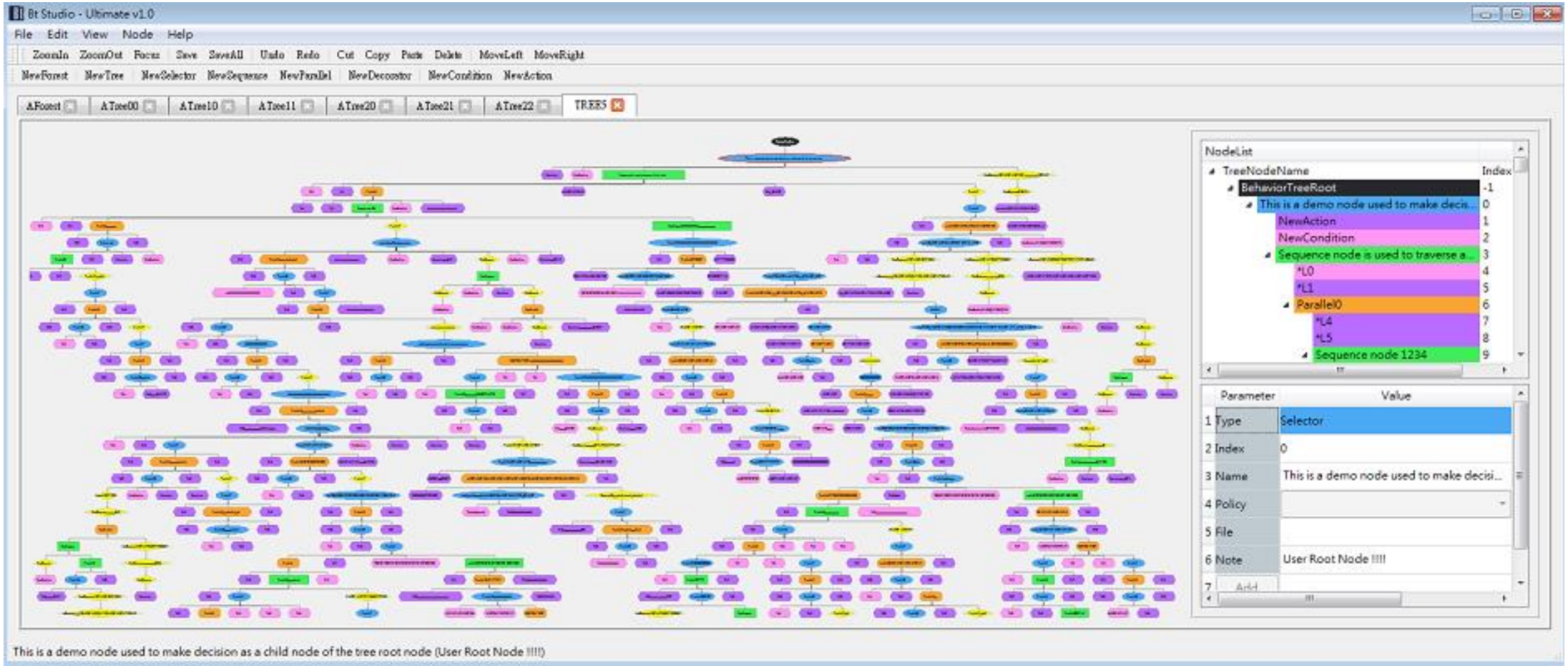
TDL – XF (Exchange format)

- Describes the interchange of TDL models between tools in xml form

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<xsd:schema xmlns:tdl="http://www.etsi.org/spec/TDL/1.4.1"
   xmlns:xmi="http://www.omg.org/XMI"
   xmlns:xsd="http://www.w3.org/2001/XMLSchema"
   targetNamespace="http://www.etsi.org/spec/TDL/1.4.1">
  <xsd:import namespace="http://www.omg.org/XMI" schemaLocation="XMI.xsd"/>
  <xsd:simpleType name="UnassignedMemberTreatment">
    <xsd:annotation>
      <xsd:documentation>
        <p>From package TestDescriptionLanguage::TDL_MetaModel::Data::DataDefinition.</p>
      </xsd:documentation>
    </xsd:annotation>
    <xsd:restriction base="xsd:NCName">
      <xsd:enumeration value="AnyValue"/>
      <xsd:enumeration value="AnyValueOrOmit"/>
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:simpleType name="ParameterKind">
    <xsd:annotation>
      <xsd:documentation>
        <p>From package TestDescriptionLanguage::TDL_MetaModel::Data::DataDefinition.</p>
      </xsd:documentation>
    </xsd:annotation>
    <xsd:restriction base="xsd:NCName">
      <xsd:enumeration value="In"/>
      <xsd:enumeration value="Out"/>
      <xsd:enumeration value="Exception"/>
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:simpleType name="GateTypeKind">
    <xsd:annotation>
      <xsd:documentation>
        <p>From package TestDescriptionLanguage::TDL_MetaModel::TestConfiguration.</p>
      </xsd:documentation>
    </xsd:annotation>
    <xsd:restriction base="xsd:NCName">
      <xsd:enumeration value="Message"/>
      <xsd:enumeration value="Procedure"/>
    </xsd:restriction>
  </xsd:simpleType>
```

Behaviour trees

- Tree structure of component states for behaviour description
- Unambiguous formal semantics given via a process algebra and its operational semantics



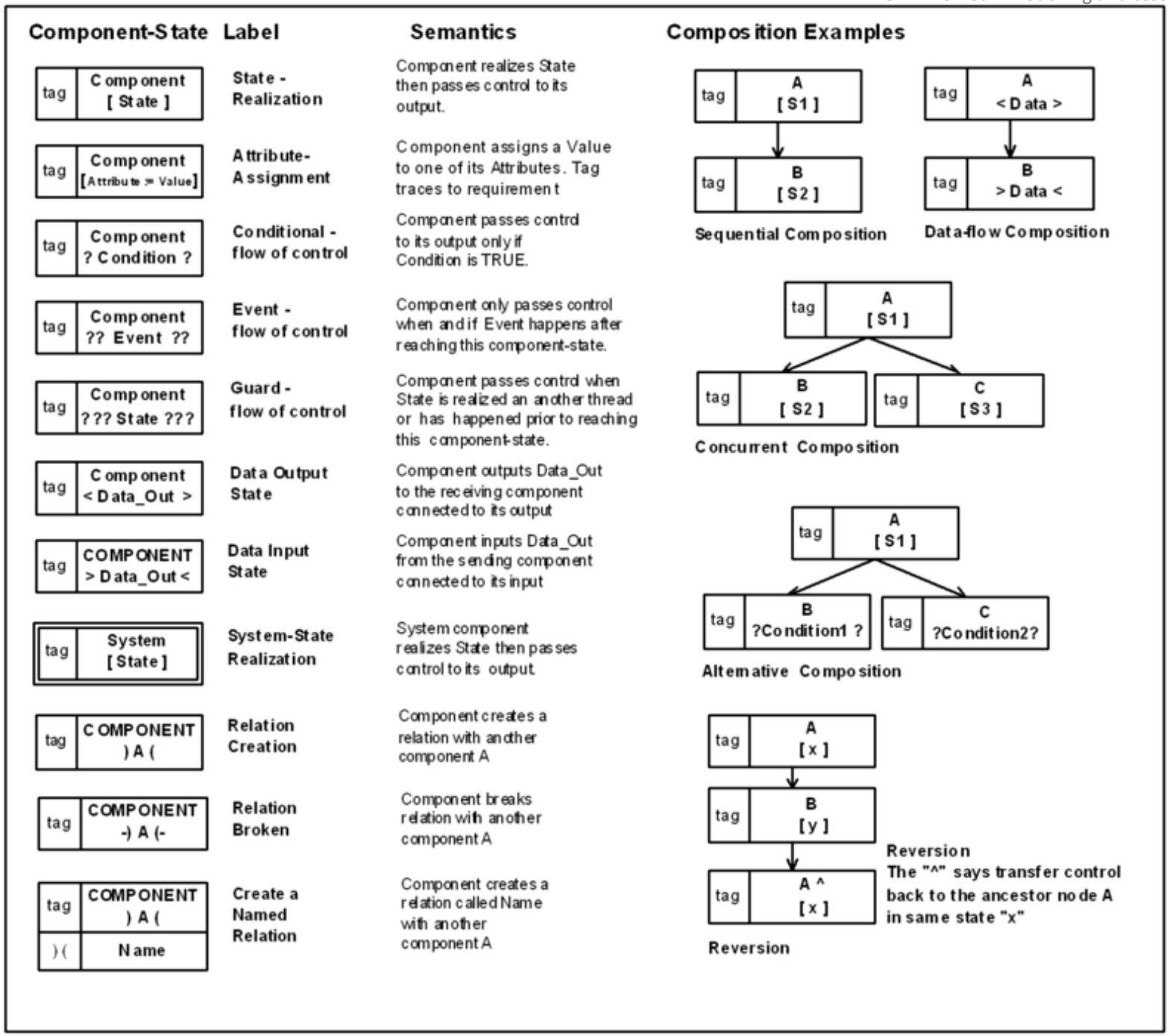
Behaviour trees

Behaviour is always associated with some component

Component-states are composed sequentially or concurrently

Components can support
- actions
- composition
- events
- control-flow
- data-flow
- threads

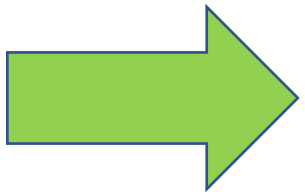
Traceability tags can be used for requirements

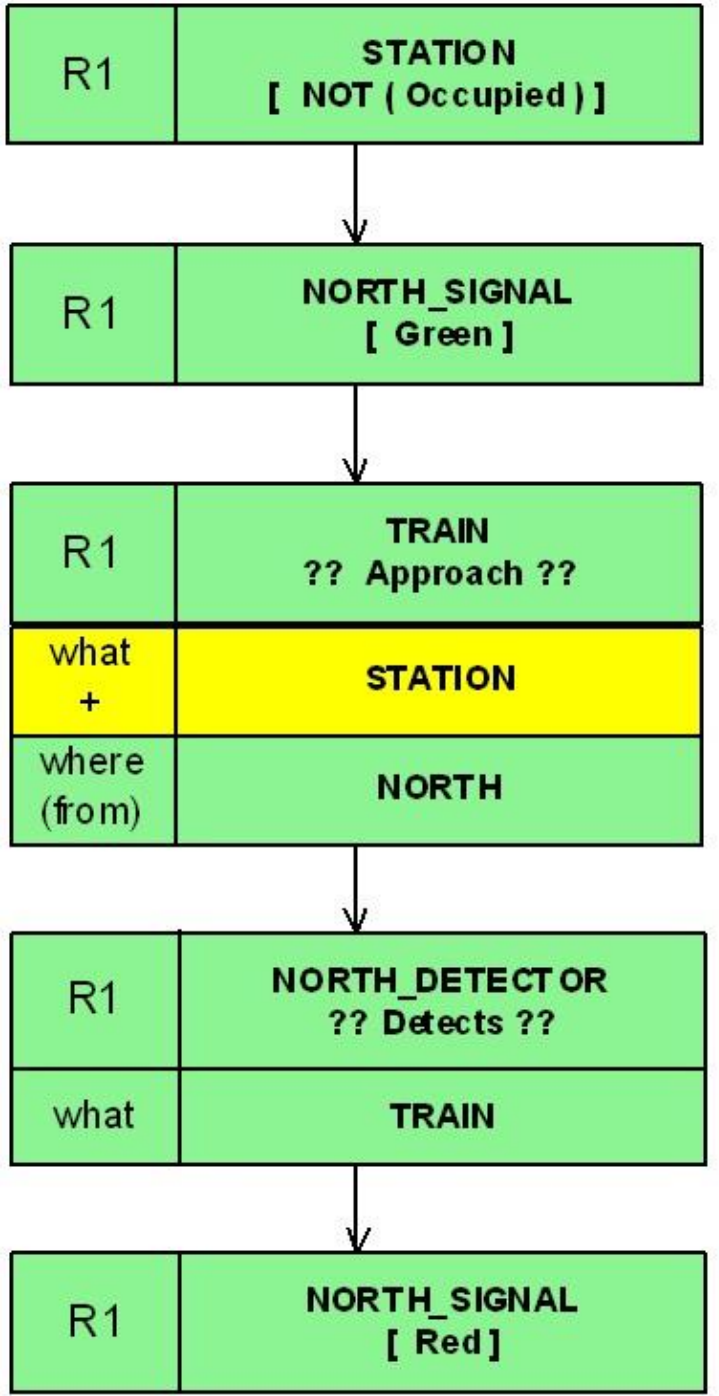


Behaviour trees – process 2/1

1. Requirements translation (informal → formal)
   - Identify components (**bold**)
   - Identify behaviours (underlined)
   - Identify the indication of order (*italics*) for behaviours
   - Construct the corresponding individual behaviour tree

Requirement-R1
Initially the **station** is not occupied. The **north signal** turns green *whenever (seq)* the **station** is not occupied. *Whenever* the **north signal** is green *(seq)* a **train** may approach from the north. *When* approaching from the north, *(seq)* a **train** is detected by the **north detector**, which *causes (seq)* the **north_signal** to turn red.

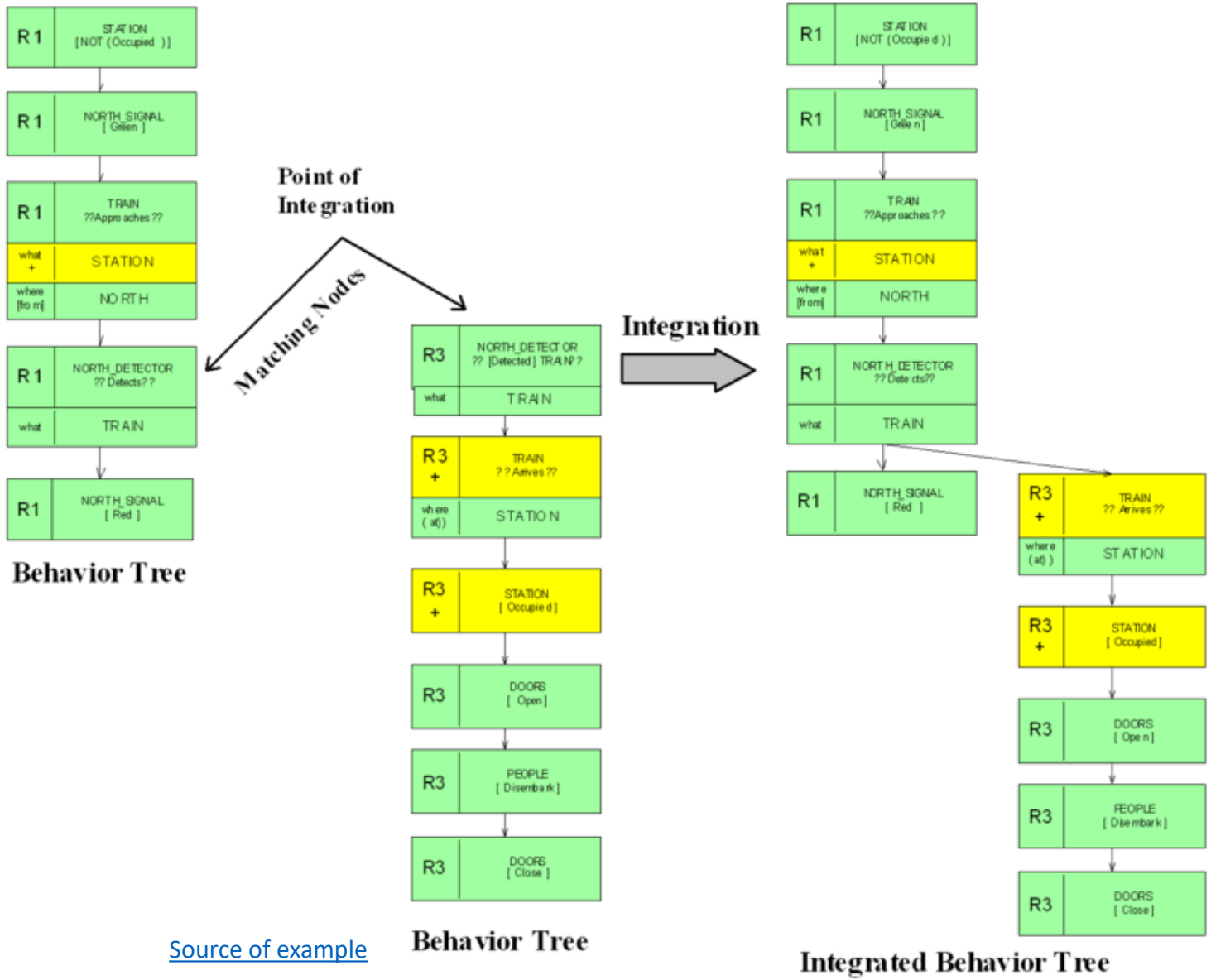




Source of example

Behaviour trees – process 2/2

2. Requirements integration
   - Create an integrated behaviour tree from individual behaviour trees
   - keep attention to preconditions!



Source of example

Behaviour trees

Advantages:

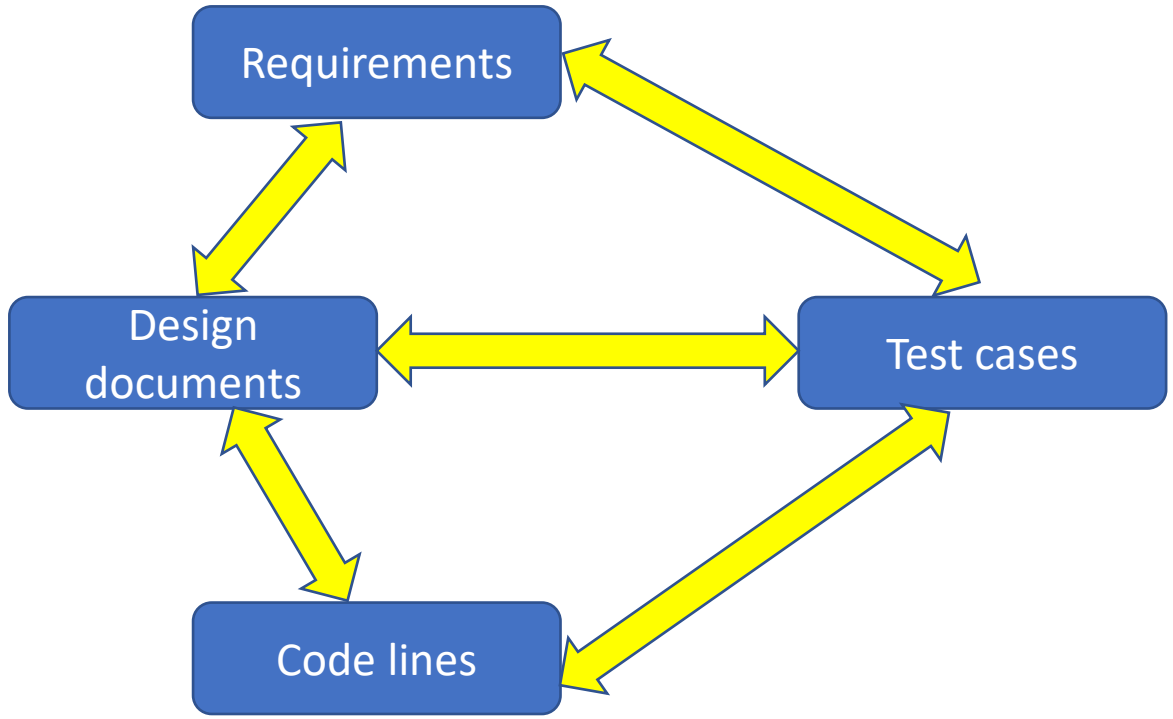
- Well-defined and effective strategy for dealing with requirements complexity
- Single, simple notation for specification, analyses and behaviour design

Disadvantages:

- Not as compact description as state s or state machine descriptions

Traceability matrix

- A document, usually in table form
- Used to determine the connection of any two distinct products of the software development processes
  - The connection can be one-to-one, one-to-many, many-to-one or many-to-many



RTM (Requirement Traceability Matrix)

- A document, usually in table form
- Used to determine the connection of requirements and their corresponding tests
  - Requirement coverage: Which requirements are covered by tests?

Scenario-based testing

Main ToC

Scenario

- A scene that describes the interaction with the system:

1. Define ***who*** the user is;
2. Explain ***why*** the user uses the service *(system, equipment)*;
3. Explain ***what the main goals*** are the user wants to achieve;
4. Define the ***assumptions*** about the service *(software, equipment)*;
5. Outline what the ***steps*** are.

Scenario-based testing

- Focuses on the **interactions between an "actor" and the system** with different roles and environments
  - Actors: humans / external systems
- Performed to verify that the actor can execute the desired actions completely from the beginning to the end
- A test scenario can be an independent test case or a series of test cases that follow each other
- 2 different strategies:
  - Use case testing
  - User story testing

Scenario-based testing: use case testing

- **Use cases:**
  - Describes the actions or steps of events
  - *heavyweight* document

- usual fields in the use case / test design templates:
  1. Use case name
  2. ID;
  3. Description
  4. Primary actor – who initiates an interaction with the system to realize some goal;
  5. Supporting actors – an external actor providing a service to the system under design;
  6. Stakeholders
  7. Preconditions
  8. Postconditions
  9. Failure end condition – what happens when the goal of the primary actor has not been achieved
  10. Triggering condition of the use case
  11. Main (success) scenario /basic flow
  12. Alternate flow or exception
  13. Additional requirements – such as performance, security …etc.

Scenario-based testing: user story testing

- **User story:**
  - *lightweight* document
  - Doesn't capture all the requirement's details, it is more or less informal, short, simple description from the perspective of the user or customer
  - Follow a simple template:
    - *As a <type of user, role>,*
    - *I want <some goal>*
    - *so that <some reason, benefit>.*

- Example syntax: Gherkin language

Gherkin

- Language to define test cases
- Human readable format
  - Acceptance tests are often derived as a result of a conversation with various stakeholders
- Connects the business causes and effects with application input/process/output with **keywords**:
  - **Scenario**: *<short description of the test of how the system shall behave in a given situation>*
  - **Given** *<preconditions / inputs>*,
  - **When** *<actions>*,
  - **Then** *<results / outputs>*.

Gherkin – keywords

- Hierarchy:
  - **Feature:** a Use Case that describes a specific function, high level-description
  - **Scenario:** Each Feature is made of a collection of scenarios. A single scenario is a flow of events and maps 1:1 with an executable test case for the system
  - **Scenario outline:** to specify multiple examples to test against a template scenario by using placeholders
  - **Example(s):** Concrete example(s) that represents a business role (i.e. same as **Scenario:**)
- Steps:
  - **Given** - Describes the preconditions and initial state before the start of a test
  - **When** - Describes actions taken during a test
  - **Then** - Describes the outcome resulting from actions taken in the **When** clause
  - **And** - logical AND
  - **But** - similar as **And**, but invert the condition

Gherkin – example 3/1

```
Feature: Withdrawn money from ATM

A user with an account at a bank would like to withdraw money from an ATM.

Provided he has a valid account and debit or credit card, he should be allowed to make
the transaction. The ATM will tend the requested amount of money, return his card, and
subtract amount of the withdrawal from the user's account.

Scenario 1: Scenario 1
    Given preconditions
    When actions
    Then results

Scenario 2: Scenario 2
    Given preconditions
    When actions
    Then results
```

Example from wikipedia

Gherkin – example 3/2

```
Scenario: A user attempts to withdraw more money
than they have in their account
  Given John has a valid Credit or Debit card
  And his account balance is $20
  When he inserts his card
  And withdraws $40
  Then the ATM displays an error
  And returns his card
  But his balance remains $20
```

Example from wikipedia

Gherkin – example 3/3

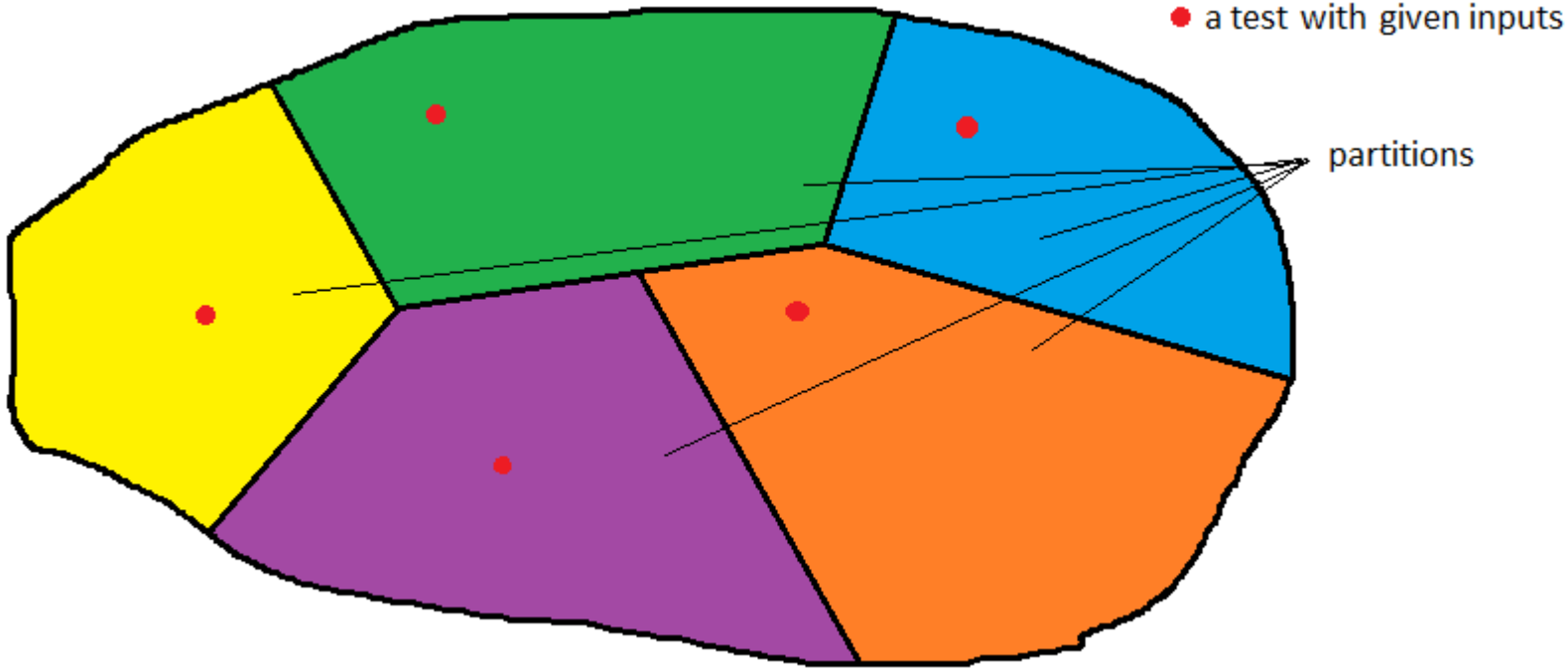
```
Scenario Outline: A user withdraws money from an ATM
      Given <Name> has a valid Credit or Debit card
      And their account balance is <OriginalBalance>
      When they insert their card
      And withdraw <WithdrawalAmount>
      Then the ATM should return <WithdrawalAmount>
      And their account balance is <NewBalance>

      Examples:
      | Name     | OriginalBalance | WithdrawalAmount | NewBalance |
      | Eric     | 100             | 45               | 55         |
      | Gaurav   | 100             | 40               | 60         |
      | Ed       | 1000            | 200              | 800        |
```

The different parameters of Scenario Outline: can be used for EP (Equivalence Partitioning) + BVA (Boundary Value Analyses) testing technique

Example from wikipedia

Background: EP (Equivalence Partitioning)

- The input domain are split into equivalence partitions, ie. into disjoint, non-empty, finite subsets
- Create a test for each subset



Background: BVA (Boundary Value Analyses)

- Focuses on boundary values between equivalence partitions
  - Because most of the faults appear near to boundaries

```
If int_variable_1 > 16 then…
```

```
If int_variable_1 >= 16 then…
```

```
If int_variable_1 == 16 then…
```

```
If int_variable_1 != 16 then…
```

Proposed values for test cases:

15, 16, 17

Note that the EP + BVA topics are discussed with more details and examples here:

- Attila Kovács: Basics of Software testing. Course in ELTE.
- Chapter 5 DOMAIN-BASED TESTING of the following book: I. Forgacs, A. Kovacs: Practical Test Design: Selection of traditional and automated functional tests. BCS. 2019.

Keyword driven testing

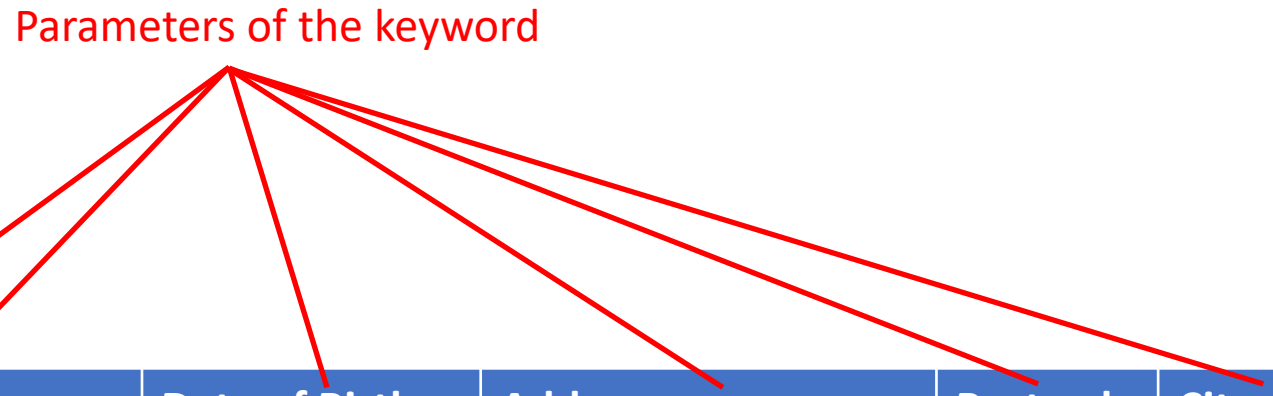
Main ToC

KDT

- Keyword-driven (or table-driven or action word based) testing (KDT):
  - Separates the test design from the implementation
  - Uses keywords to symbolize a functionality to be tested:
    - The given keyword may have input and output parameters
  - Process:
    1. Test case can be described in an implementation-independent way using keywords
       - Functional tests can be developed before any implementation work
    2. Then the keyword is implemented according to the actual code

KDT

- Example:



Keyword
Start_Application
Open_Edit_Mode

Keyword	First name	Surname	Date of Birth	Address	Postcode	City
Enter_Customer	Jakab	Gipsz	22.05.1994	18 Kakadu Avenue	1114	Budapest
Enter_Customer	Johnny	English	16.11.1967	2 Pázsit Street	1022	Budapest
Filter_Customers	*Empty*	*Empty*	*Empty*	*Empty*	1022	*Empty*

Keyword
Exit_Application

Advantages & disadvantages of KDT

Advantages:

- Low maintenance cost through changes in SUT
- Detailed, unambiguous description of test cases
  - Easy-to-read for stakeholders
  - Can be used as an input both for manual and automated tests
- Keywords can be reused in multiple test cases
- Implementation independent

Disadvantages:

- Initial cost, learning curve

Main ToC

Overview of test automatization

Software development methodologies

Waterfall

V-model

Agile

CD

DevOps

TDD

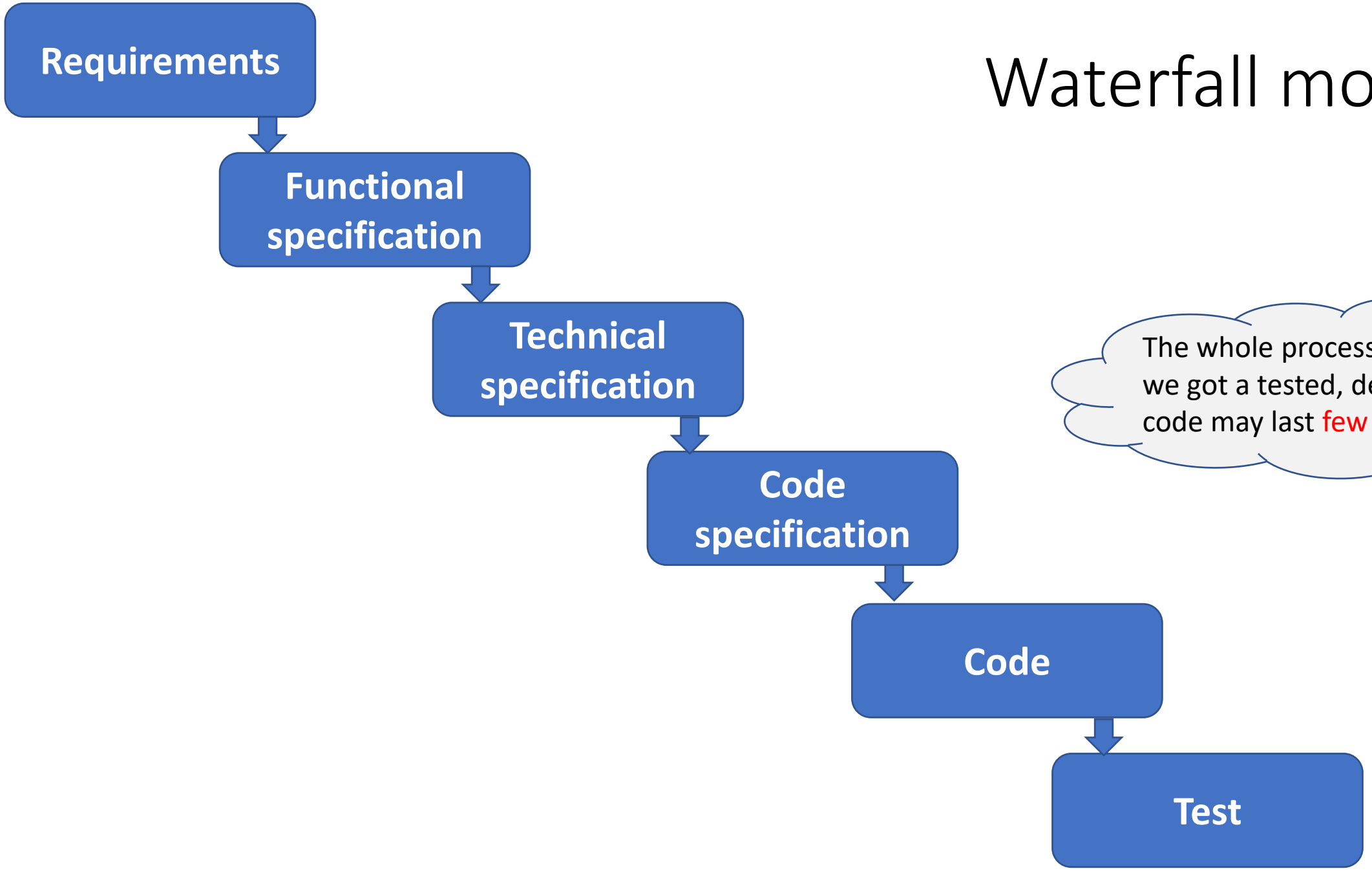
Test Automation

Test Pyramid

Test scheduling

Jenkins demo

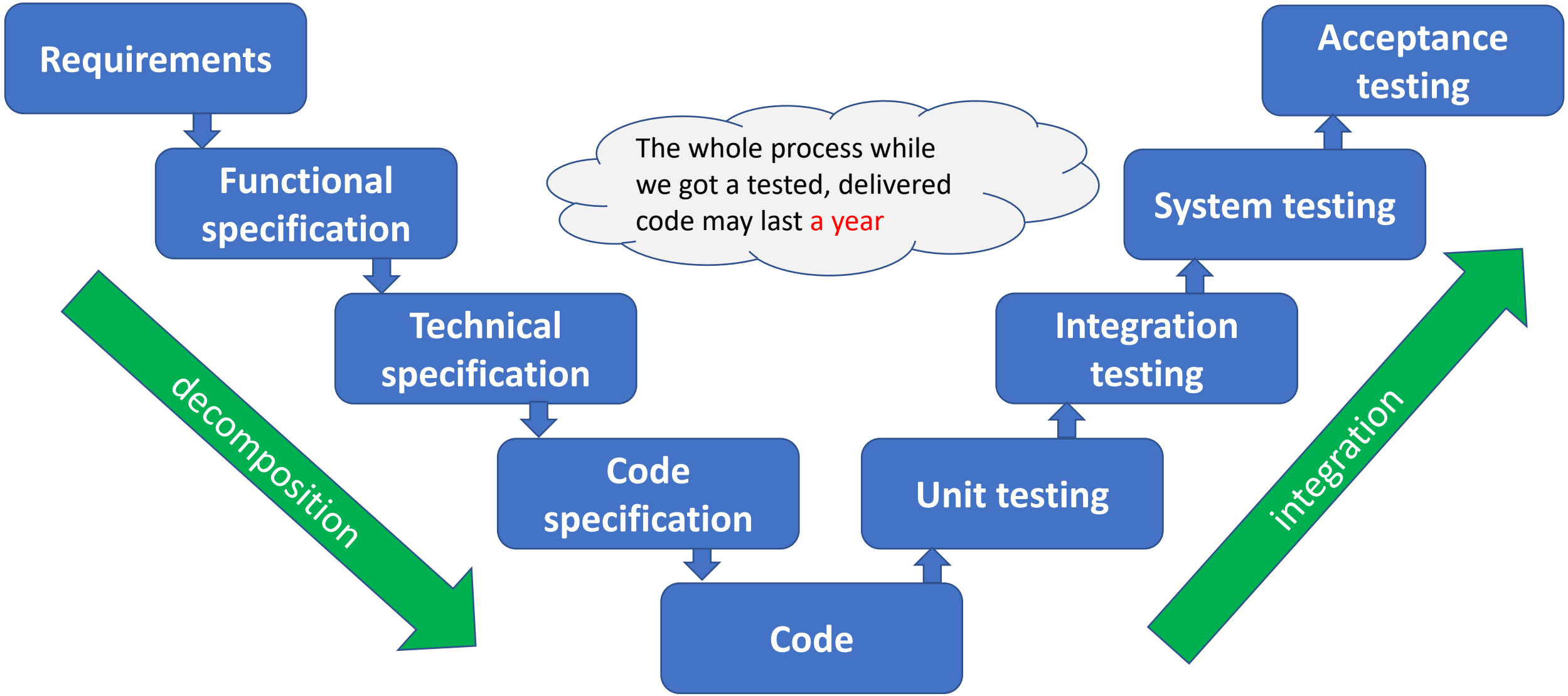
Waterfall model



The whole process while we got a tested, delivered code may last few years

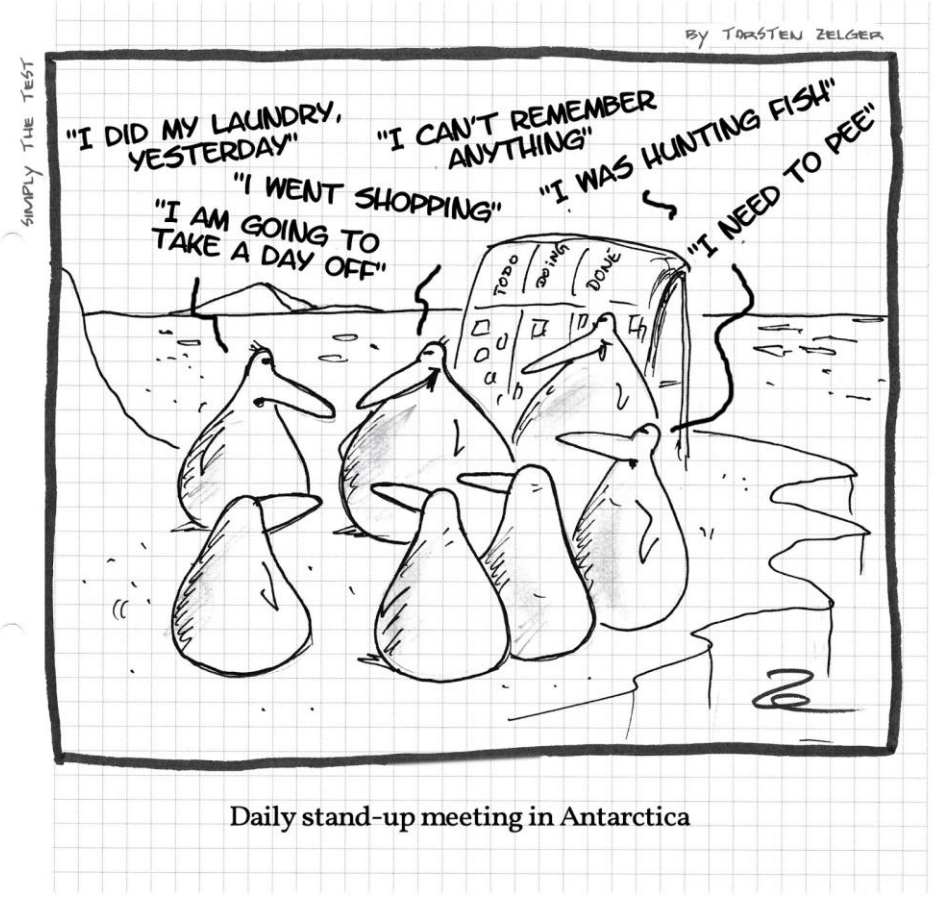
Testing at the end!

V-model


Requirements
Functional specification
Technical specification
Code specification
Code
Unit testing
Integration testing
System testing
Acceptance testing
The whole process while we got a tested, delivered code may last a year
decomposition
integration

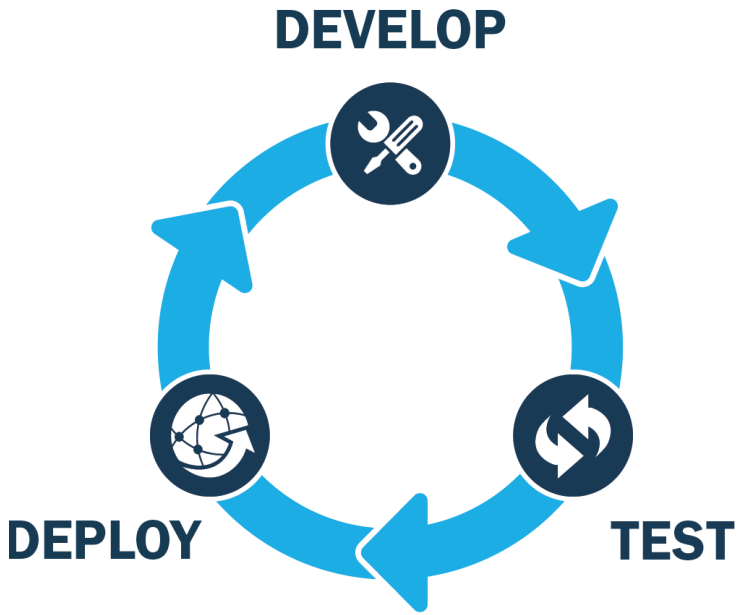
Faster methods 9/1

- Agile:
  - Small tasks (requirement clarifications, design, coding, tests…etc.) that can be done within a 2-4 week sprint



Faster methods 9/2

- For code handling: Continuous Integration (CI), one branch development
  - Commit often / Merge often → make integration easier
    - Don’t crash the code
    - Check test results
    - Code reviews
  - Tested, delivered code after 2-4 weeks



Faster methods 9/3

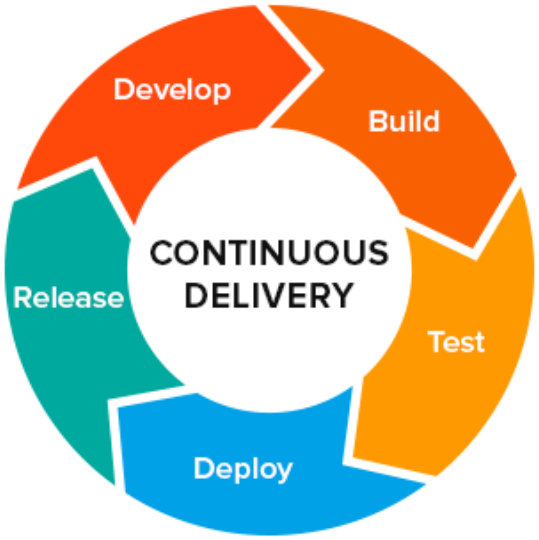
CI gating:

- Execute corresponding test(s) after a code change, but before code commit
- Use the verdict of the test(s) to decide if the given code change can be allowed for commit or not

- Gating tests should be able to be executed within a reasonable time in order to give feedback to the developer about his/her code commit

Faster methods 9/4

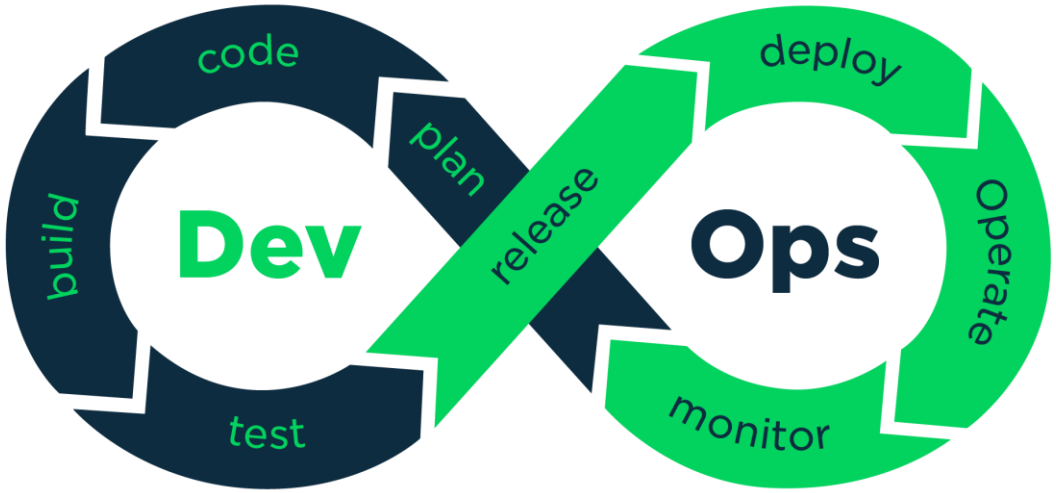
- Continuous Delivery (CD)
  - Goal: deliver the code anytime
    - New feature
    - Bug correction
  - Built top of CI
  - Automatization of delivery process is required
  - Testing before delivery with reduced scope (no endurance tests for example)
  - If code works after commit, then delivery is possible within an hour



Faster methods 9/5

- DevOps:
  - Cross functional teams:
    - Developers
    - Testers
    - Technical Writers
    - IT Operation professionals

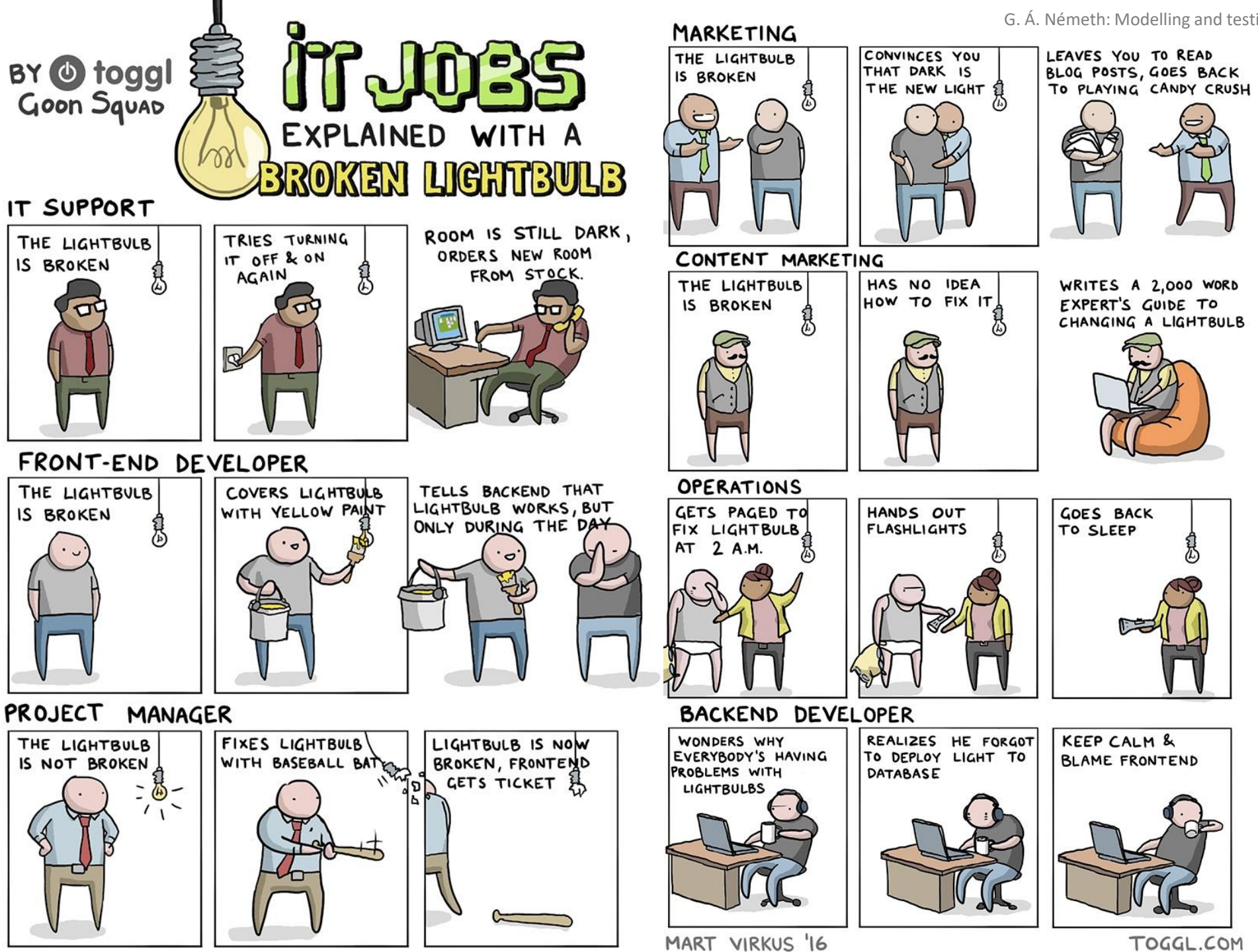
The goal is to near the world of development and business areas



  - Automate the process of software delivery & infrastructure changes
    → reduce the time required from code commit till it appears in normal production (while provide high quality)
  - Built top of CI & CD

Faster methods 9/6

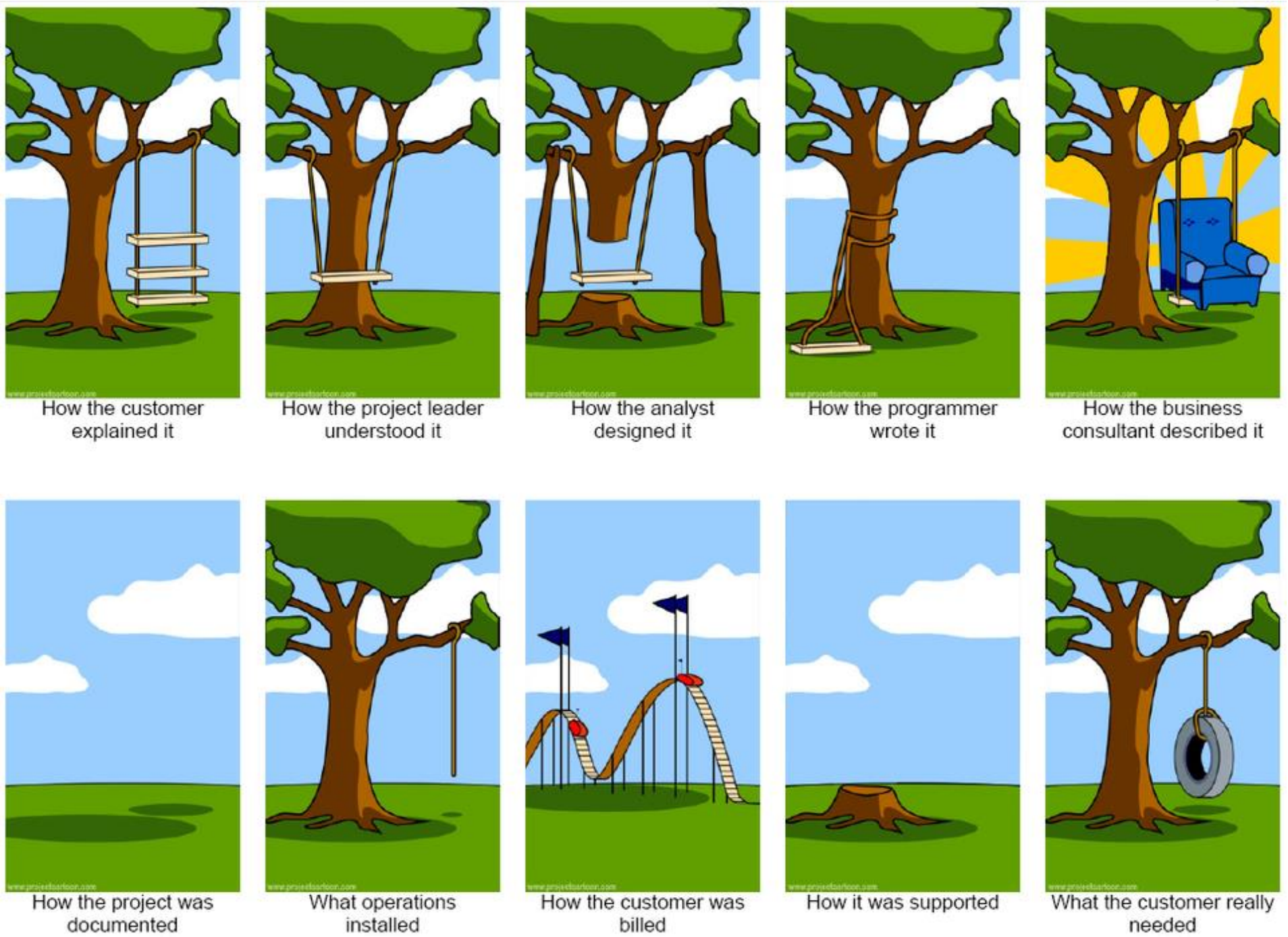
Note that DevOps requires the strong collaboration of people with different background…



Source of the figure

Faster methods 9/7

…which strong collaboration is really needed as they have different things in their mind for the same task.

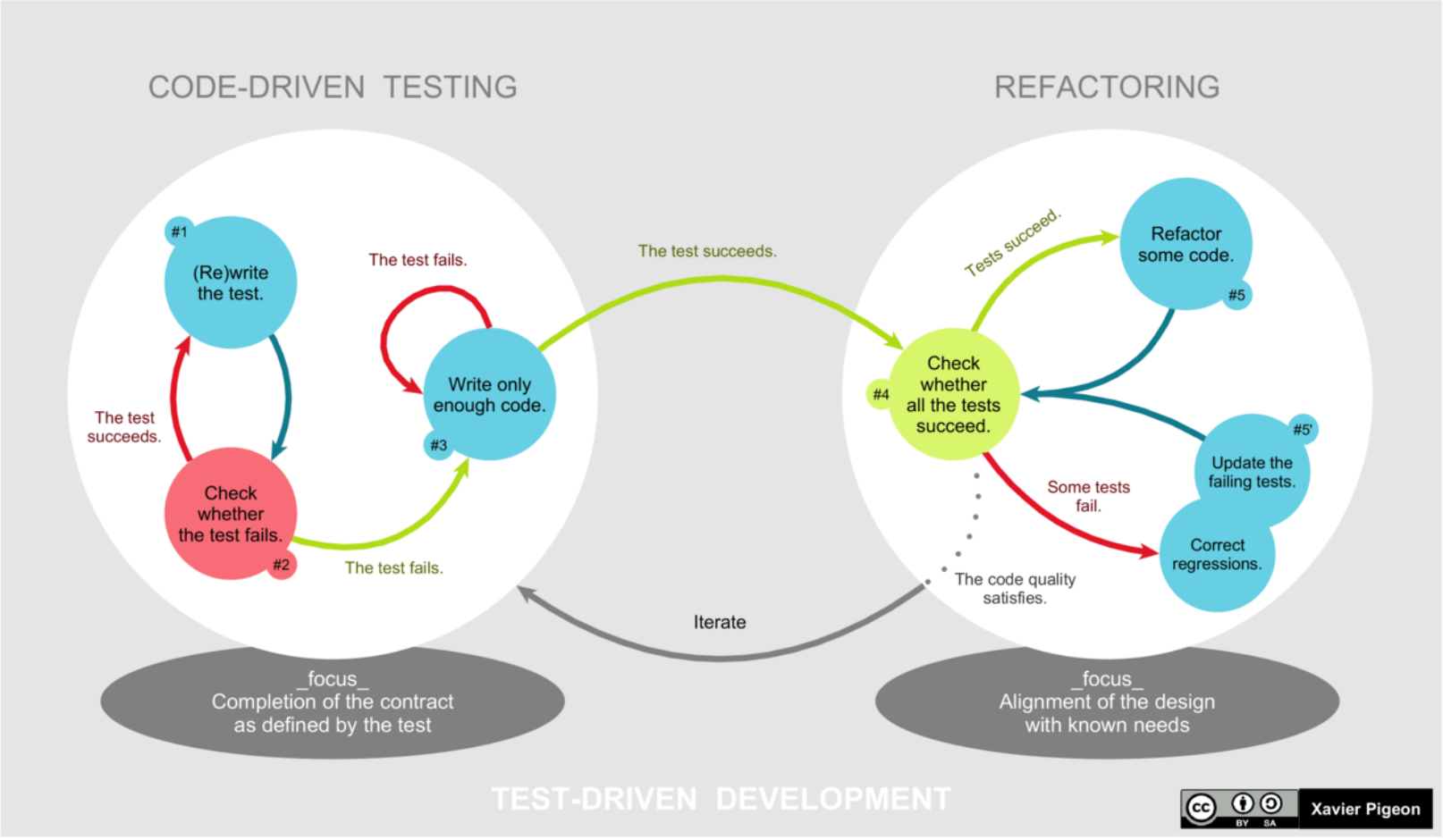


Source of the figure

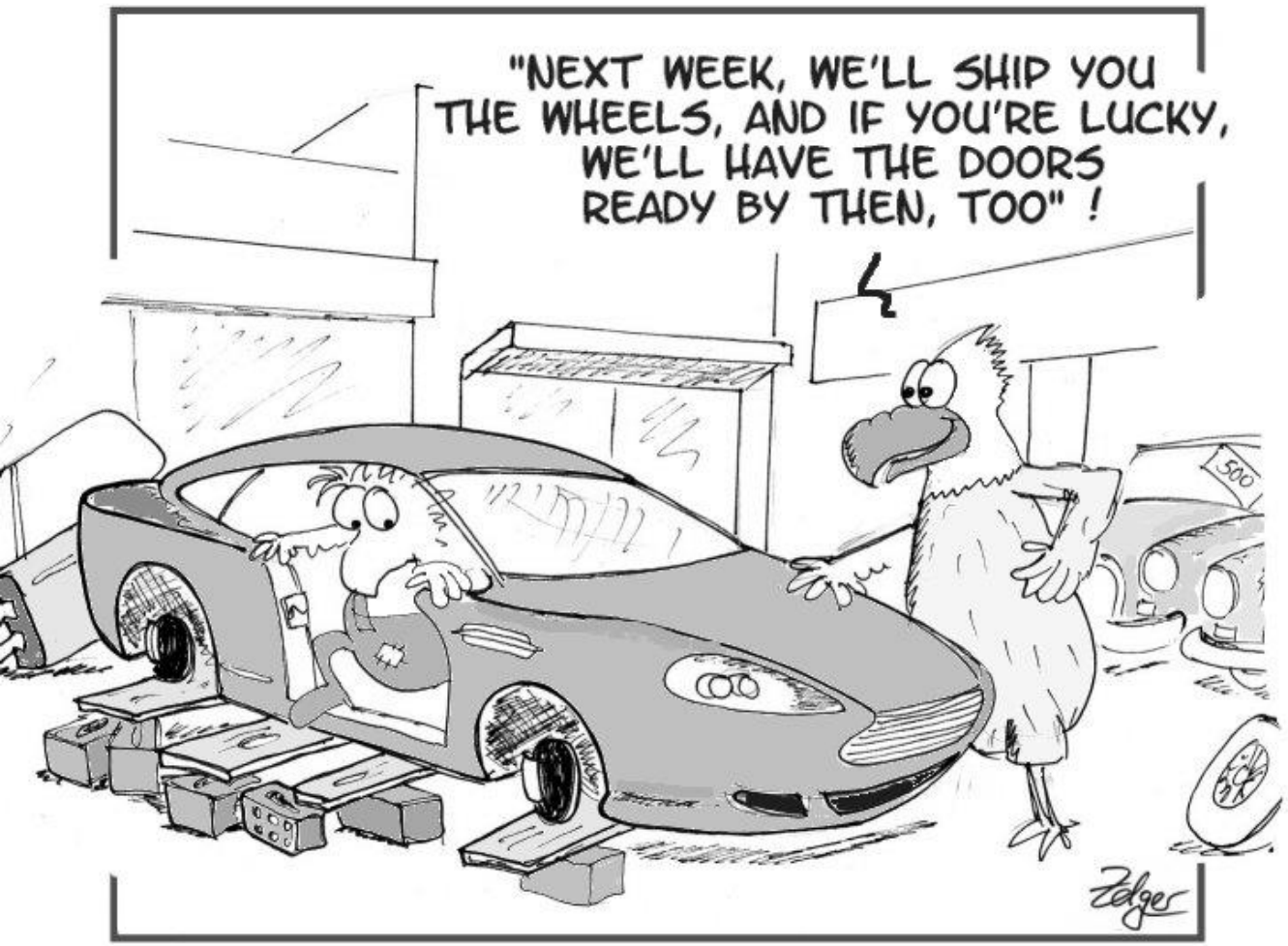
Faster methods 9/8

- Test Driven Development (TDD): “short development cycle: requirements are turned into very specific test cases, then the software is improved to pass the new tests, only.” *wikipedia*

test-driven development lifecycle →
[Source of fig]



Faster methods 9/9



If cars were build like software...

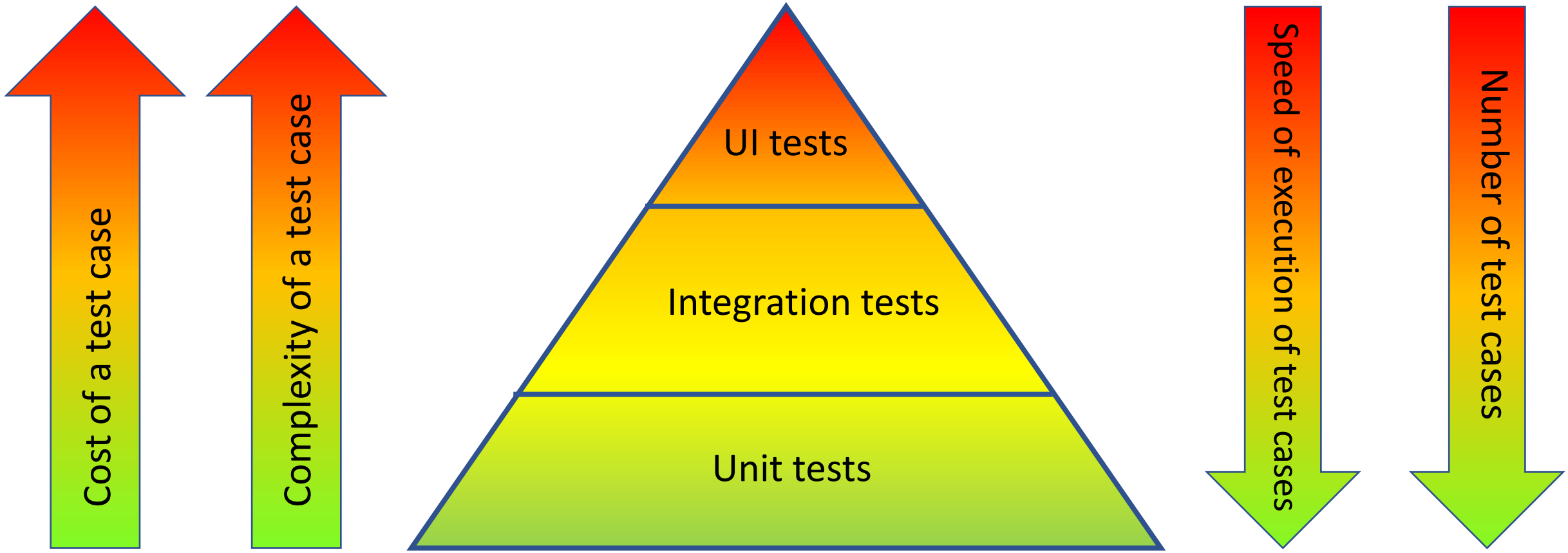
Peep behind the scenes of Chaos Software Ltd.
to discover how it keeps their software services under control

Test Automation

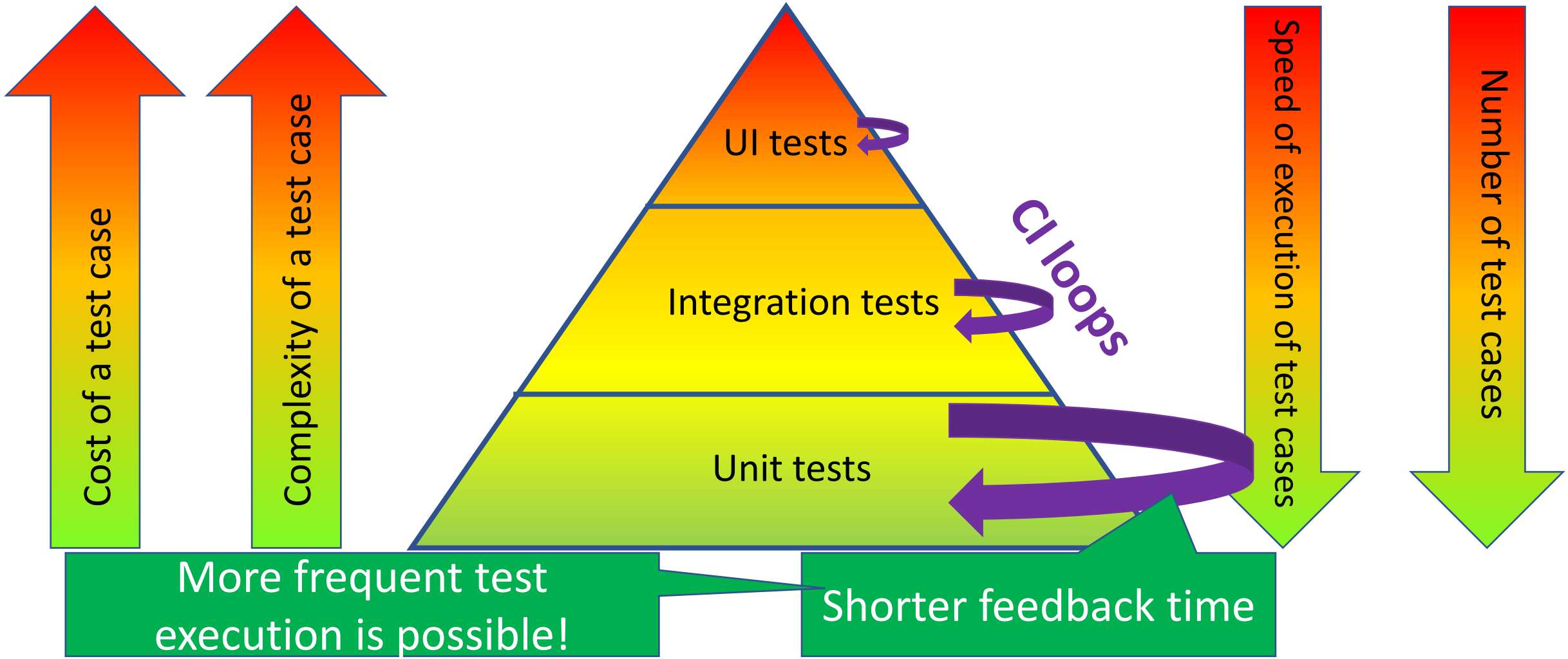
To support faster delivery, test automation is required

- Automates the process of tracking and managing the different tests

Test pyramid in automated testing



Test pyramid in automated testing



Different frequencies of test execution

- Commit tests:
  - Part of the development pipeline, tests for most affected and important parts of code-base
- Nightly tests:
  - Scheduled outside working hours
  - Higher priority tests for the entire product
  - Smoke and Sanity tests
- Weekly tests:
  - Scheduled at weekends
  - Besides higher priority tests, lower priority tests are also performed
  - Performance (mainly endurance (like 24/48/72hrs stability)) tests
- Release tests:
  - Function test for new features, Regression tests
  - User Acceptance test
  - Performance (stability, stress) tests

Right scheduling is very important…

Live demo of Jenkins test automation server

Live demo with Péter Sótér

Jenkins

search log in

Jenkins

People
Build History
Project Relationship
Check File Fingerprint
Job Config History
We Need Beer

Build Queue

No builds in the queue.

Build Executor Status

master
1 Idle
2 Idle

cabbage
1 Idle
2 Idle

celery
1 Idle
2 Idle

All | All Disabled | All Failed | All Unstable | Infrastructure | **Jenkins core** | Libraries

Other Projects

S	W	Name ↓	Last Success	Last Failure	Last Duration	LC
		jenkins_2.0	13 hr - #4	8 days 17 hr - #3	1 hr 2 min	
		jenkins_lts_branch	1 mo 5 days - #197	2 mo 9 days - #189	41 min	
		jenkins_main_maven-3.1.0	2 yr 5 mo - #7	N/A	1 hr 11 min	
		jenkins_main_trunk	2 days 5 hr - #4414	23 days - #4399	1 hr 1 min	
		jenkins_pom	1 yr 3 mo - #264	3 days 6 hr - #334	47 sec	
		jenkins_rc_branch	10 mo - #517	1 yr 7 mo - #424	25 min	
		remoting	2 yr 7 mo - #4	2 yr 7 mo - #3	6 min 19 sec	

Icon: S M L

Legend | RSS for all | RSS for failures | RSS for just latest builds

Live demo of Jenkins test automation server

Live demo with Péter Sótér

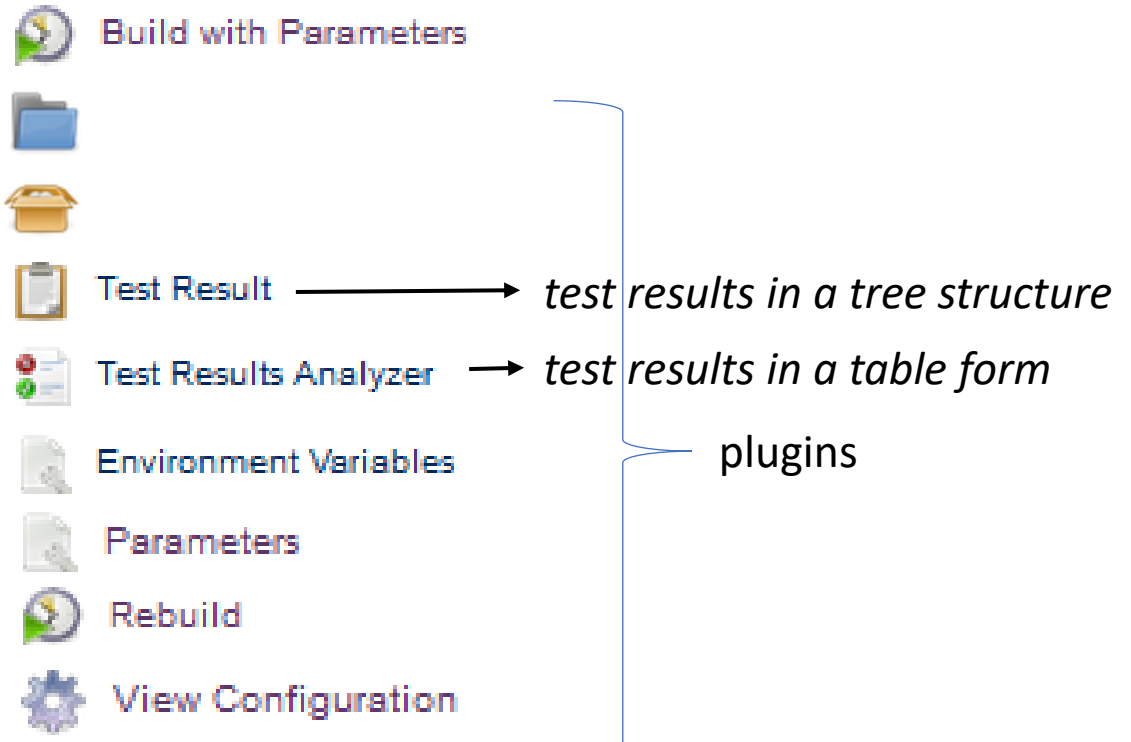
- Open source (java) and free
- Functionality: active connections with nodes (servers) → script execution on nodes by scheduling → test results (active & archives)
- Jenkins jobs:
  - Simple job
  - Pipeline job – stages
    → jobs can be stopped after a stage fails / jobs can be continued after failure
- Feedbacks:
  - Radiators
  - Jenkins home pages
  - E-mails
- Many plugins
  - Instable / not all combinations have been tested → plugin should be installed one-by-one and tested in a playground area first
- Has good built-in documentation

Live demo of Jenkins test automation server

Live demo with Péter Sótér

After log in…

- Start of any Jenkins job
- Access for workspace
- Access for archived files
- Test result of one actual build (Junit)
- Test result of more build
- Environment variable during build
- Parameters of build
- Rebuild
- Configuration of Jenkins job
- Schedule



Live demo of Jenkins test automation server

Live demo with Péter Sótér

Install Jenkins (https://jenkins.io/download/)
Check the parts of Jenkins

Tasks:

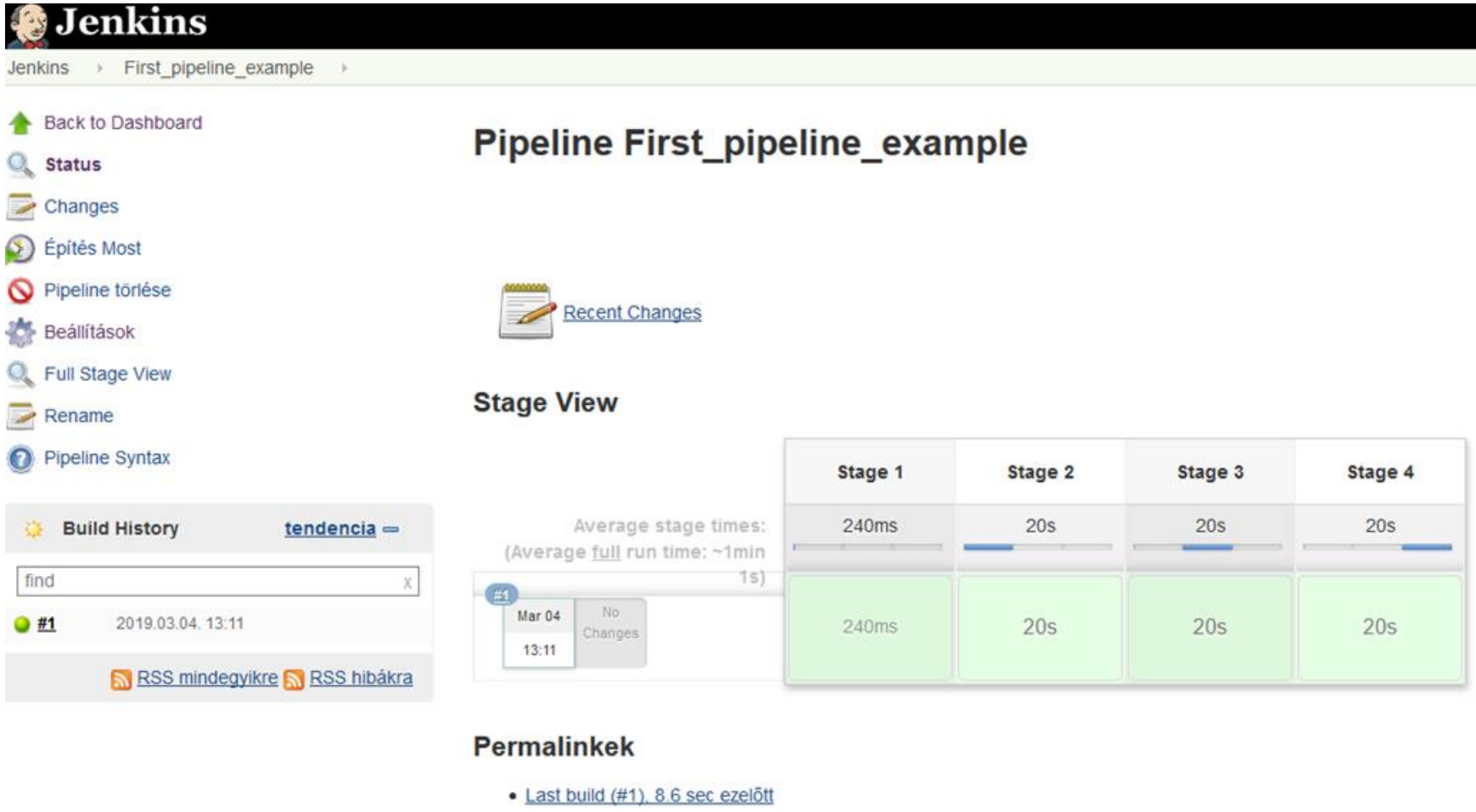
- Create users
- Create a Jenkins job
- Create a pipeline job
- Create a radiator page
- Add new node/ executors
- Install new plugin

Live demo of Jenkins test automation server

- Simple pipeline job example: 4 stages with delay

Live demo with Péter Sótér

```
// Example script with 4 stages
node {
stage ' Stage 1'
echo " BUILD_URL=${env.BUILD_URL}"

def workspace = pwd()
echo " workspace=${workspace}"

stage 'Stage 2'
echo " Stage 2"
sleep(20)

stage 'Stage 3'
echo " Stage 3"
sleep(20)

stage 'Stage 4'
echo " Stage 4"
sleep(20)

}
```

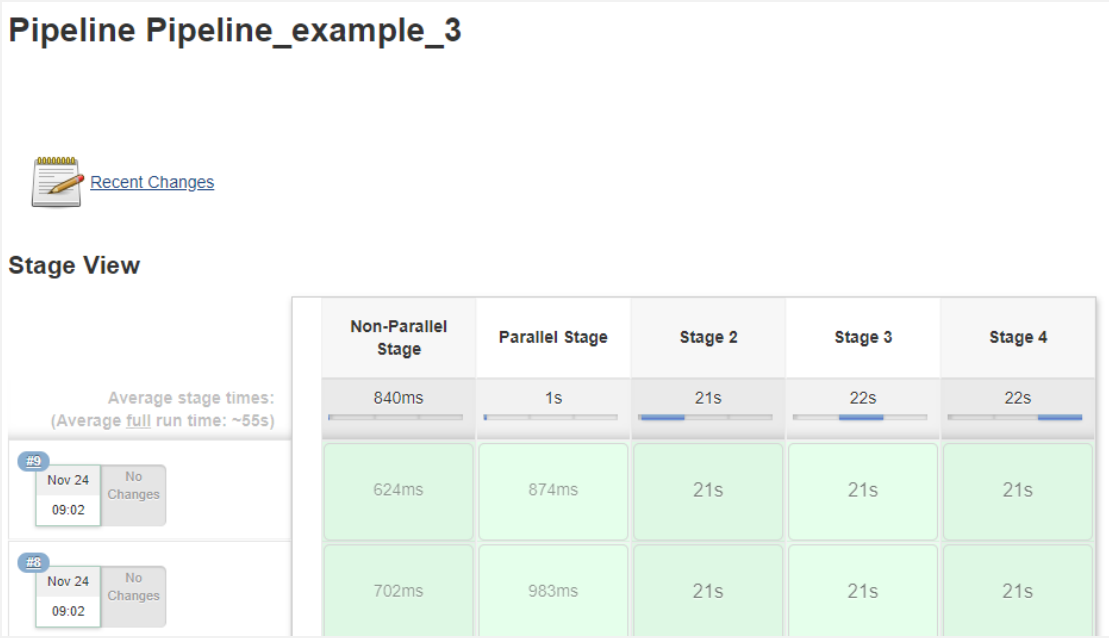


Live demo of Jenkins test automation server

- Parallel pipeline job example: 1+3 parallel stages with delay

Live demo with Péter Sótér

```
pipeline {
    agent any
    stages {
        stage('Non-Parallel Stage') {
            steps {
                echo 'This stage will be executed first.'
            }
        }
        stage('Parallel Stage') {
            parallel {
                stage ('Stage 2'){
                    steps {
                        echo " Stage 2"
                        sleep(20)
                    }
                }
                stage ('Stage 3'){
                    steps {
                      echo " Stage 3"
                      sleep(20)
                      }
                    }
                stage ('Stage 4'){
                  steps {
                    echo " Stage 4"
                    sleep(20)
                  }
                }
            }
        }
    }
```

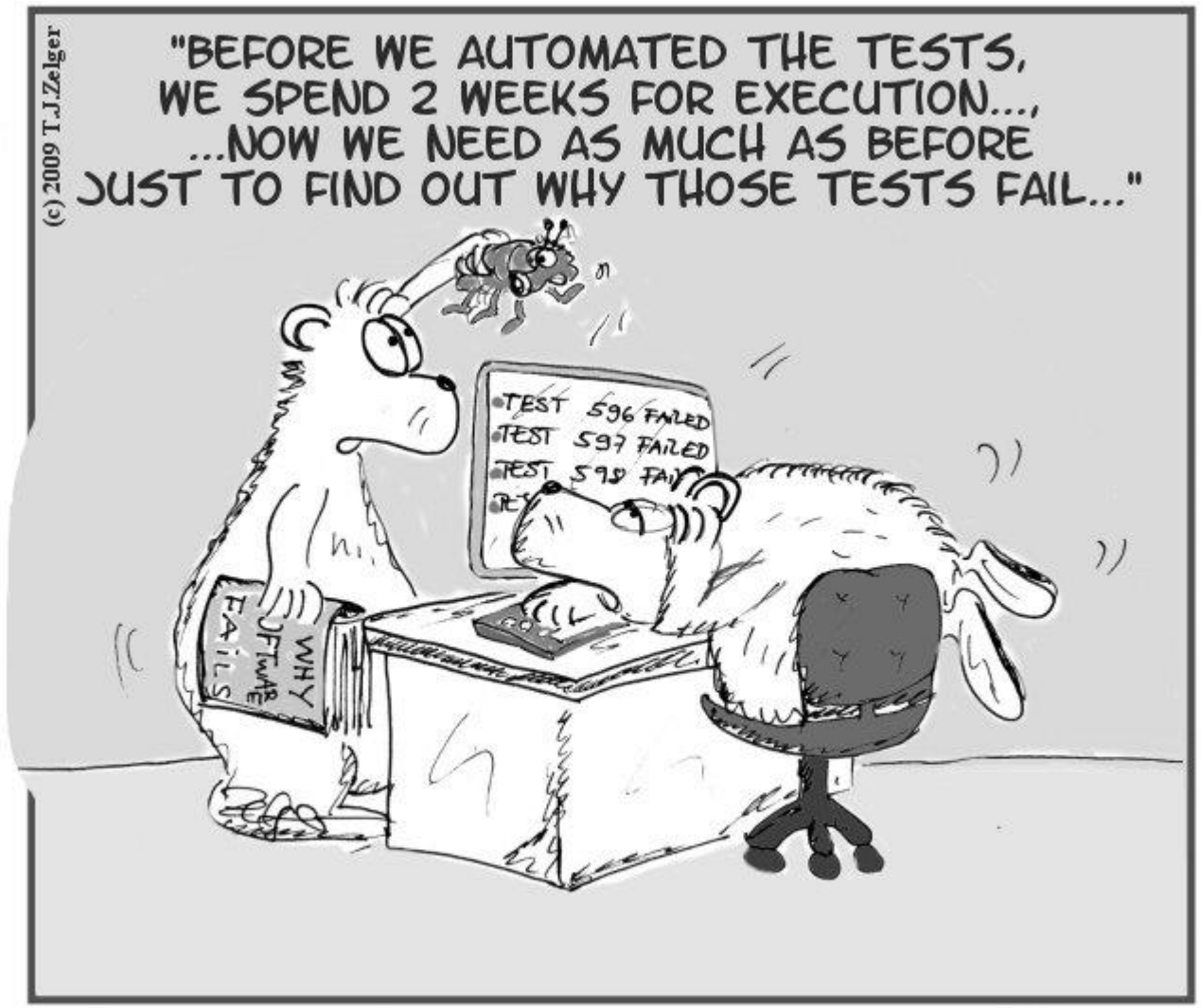


Conclusion of Jenkins demo

Live demo with Péter Sótér

- Jenkins pipeline is a suite of plugins which supports implementing and integrating continuous delivery pipelines into Jenkins
  - Execution environment can be set up
- Jenkins concludes the most important information about test cases in html pages:
  - Results of last and previous executions
  - Test duration
  - Last success run / Last failure run
  - Log
- Test cases can be classified into different groups
  - Scheduling can be set up for these groups
- Even if many try to mystify it, it is not a big deal to set up a working environment ☺

But the environment must be set up properly…



Test Automation side effect

Model-based testing

Main ToC

Overview of Model-based testing

- Models
  - Introduction
  - MBT working principle
  - Petri nets
  - Markov chains
  - FSMs
  - STSs
  - EFSMs
  - CFSMs
  - PFSMs
- FSM-based test generation
  - FSM-based conformance testing
    - Coverage
    - Fault models
    - Assumptions about FSMs
  - Test generation algorithms
    - Overview
    - Transition Tour
    - Algorithms with 2 stages
    - HSI-method
    - H-method
  - Incremental approach
    - General concept
    - Incremental Transition Tour
    - Incremental HSI
  - Model-based mutation testing
    - Mutation testing
    - 2 approaches for model-based mutation testing
  - Test case selection
    - General concept
    - String-distance based method
- Model creation
  - Code → EFSM transformation
  - Manual model creation
- Theory vs. practice
  - Adaptation code
  - Tools
    - *GraphWalker*
    - MTR
  - Online vs. offline MBT
  - Typical problems
  - Advantages of MBT
  - Disadvantages of MBT
- Summary

Main ToC

Models

3 fundamental characteristics of models (in general, in any area):

- Mappings from the original, actual world to a more abstract world
- Serve a specific purpose
- Simplifications, do not reflect all aspects of the actual world

  → 2 strategies: omission or encapsulation of details

Behaviours and Models

The desired behaviour of the system is described with a **model**

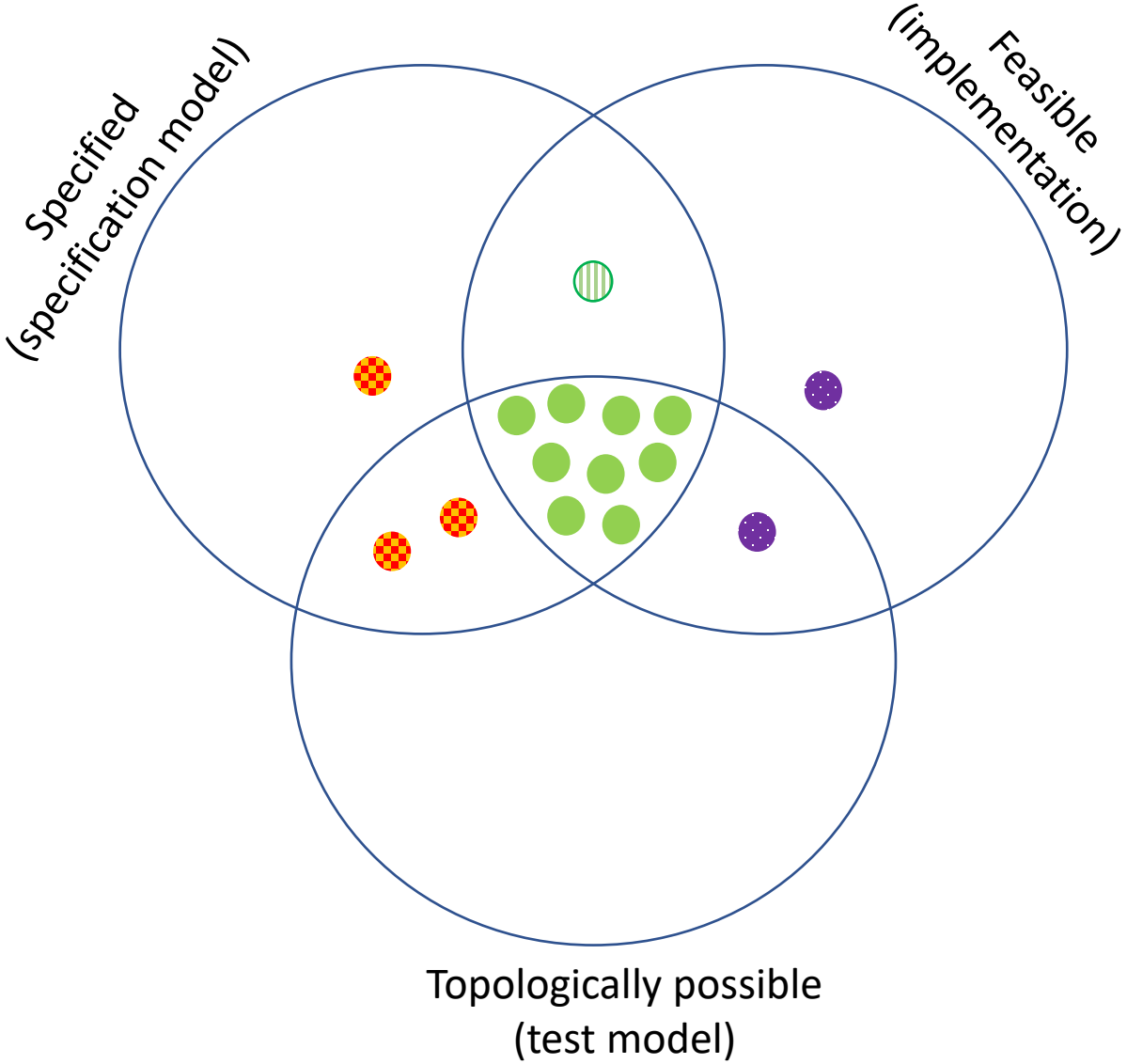
3 types of behaviours:

- Specified

  → Model of the specification

- Feasible
  - Describes the programmed behaviour
- Topologically possible
  - Possible paths that can be reached within the test model

Behaviours and Models



Legend:

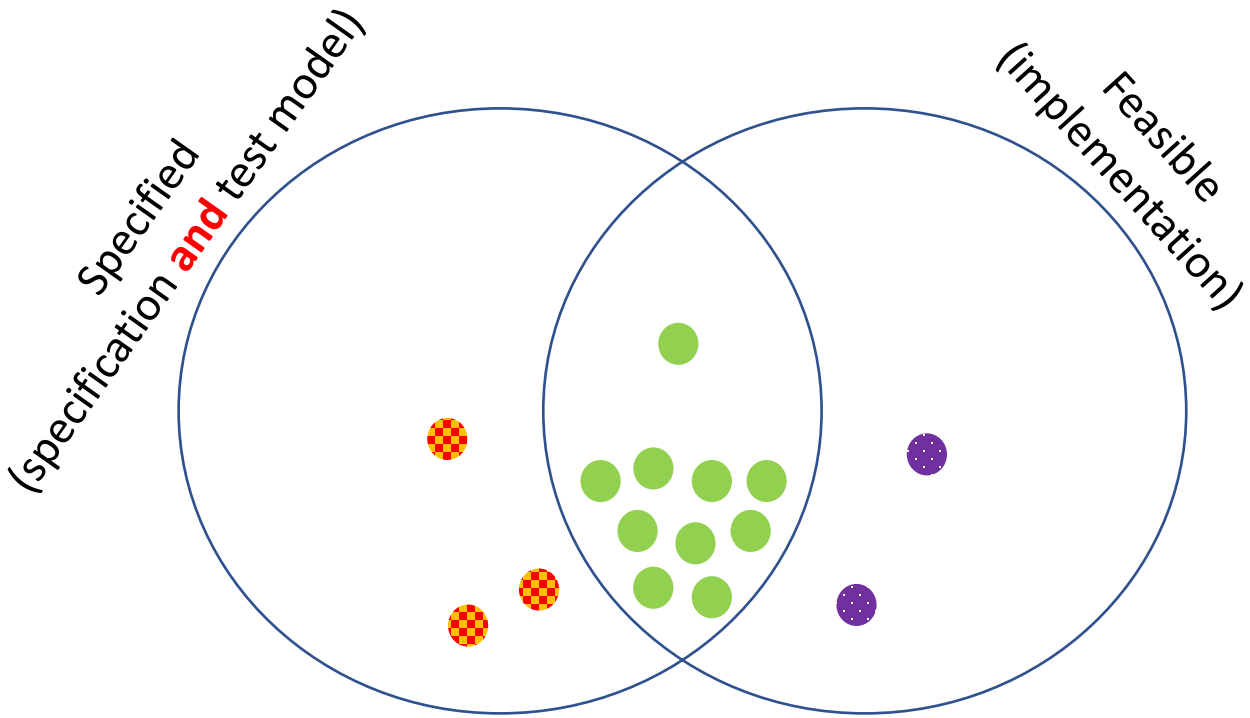
- Desired functionalities (specified & correctly implemented) & the test model is appropriate
- Desired functionalities, but the test model is unable to find them
- Extra, incorrect functionality
- Missing functionality (no implementation exist)

The task in testing is to find missing or incorrect behaviours

Designer analyses the topologically possible behaviour of the test model, which is sometimes feasible and sometimes not, sometimes specified and sometimes not

If something is not topologically possible, but specified, then the model is wrong

Behaviours and Models



Legend:

- Desired functionalities
- Extra, incorrect functionality
- Missing functionality (no implementation exist)

The task in testing is to find missing or incorrect behaviours

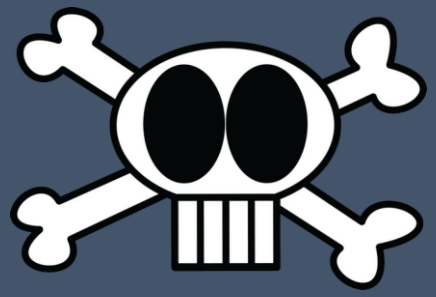
Designer analyses the formal specification, which is sometimes feasible and sometimes not

A formal specification can be used as a test model as well!

Behaviours and Models

Legend:

- Desired functionalities



But please **NEVER** use the implementation as a source to create a test model!

In this case you will only be able to compare the implementation with the implementation…

Designer analyses the formal specification, which is sometimes feasible and sometimes not

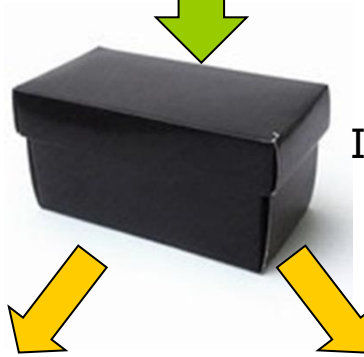
A formal specification can be used as a test model as well!

Model-based testing

- Conformance testing
  - Given
    - Specification model
      - *white box*
    - Implementation model
      - *black box*
  - Test if the implementation conforms to the specification?
    - Ie. determine if the two corresponding model descriptions are equivalent

Specification model

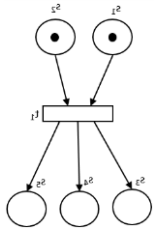
?



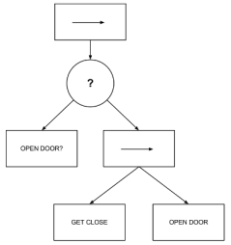
Implementation model

Different models

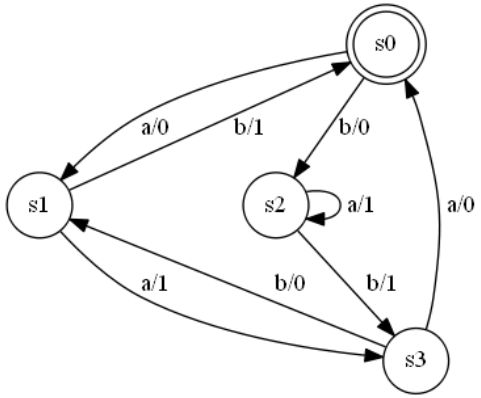
- Petri-net



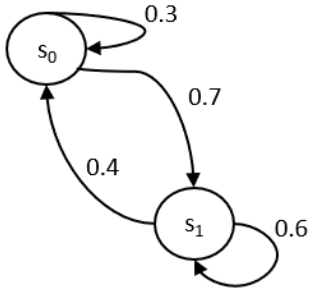
- Behaviour Tree



- FSM (Finite State Machine)



- STS (State Transition System)
- Markov chains

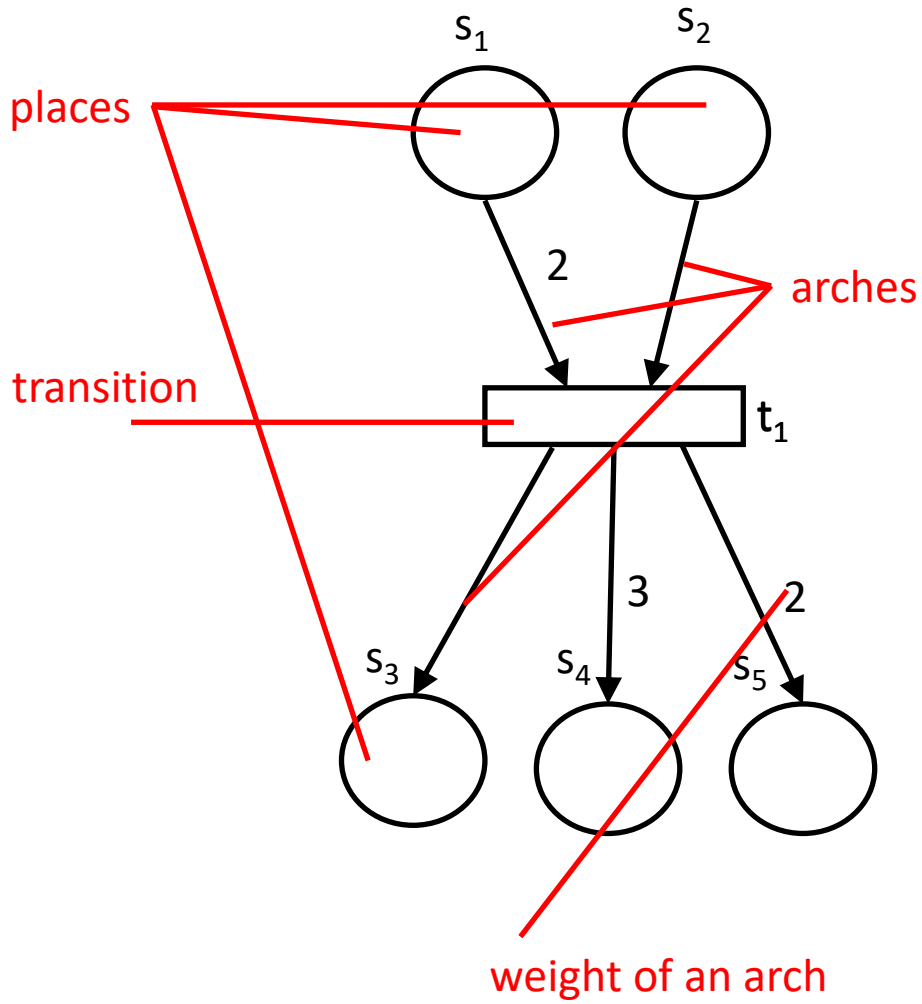


Petri nets

- N=(S, T, W):
  - S is a finite set of places
  - T is a finite set of transitions
  - W is multi set of arches (or flow relations):
    - W: (S X T) U (T X S) → $\mathbb{N}$
      - i.e. it assigns to each arc a non-negative integer

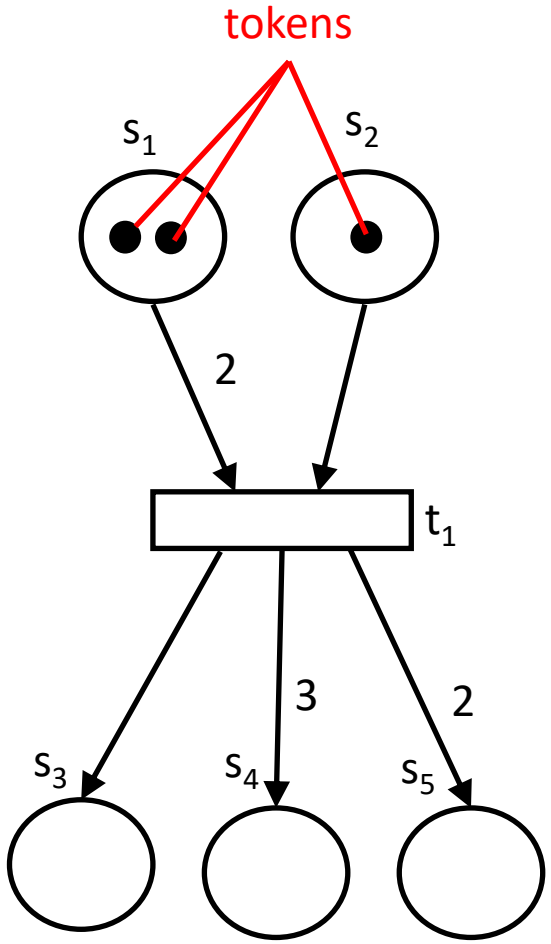
        called *arc weight* (or *multiplicity*)
        - Arcs have capacity 1 by default; if other than 1, the capacity is marked on the arc
      - no arch can connect 2 places or 2 transitions

Disjoint sets!



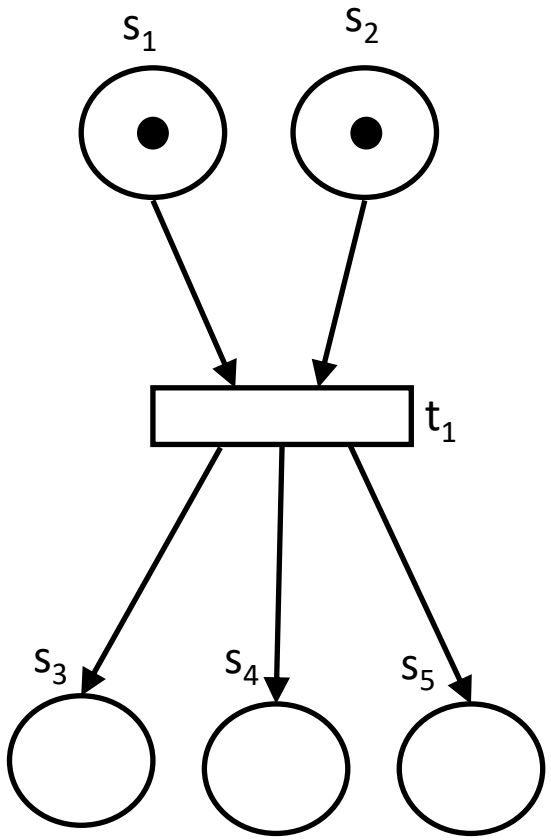
Petri nets

- Places may hold discrete number of tokens
  - Distribution of tokens over the places will represent a configuration of the net called a *marking*



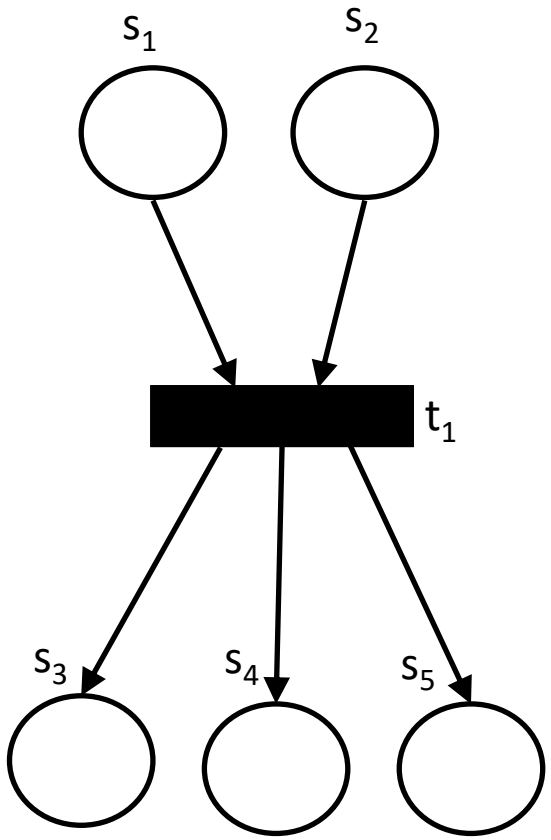
Petri nets

- Firing of transitions:
  - A transition is *enabled* when the number of tokens in each of its input places is at least equal to the arc weight going from the place to the transition



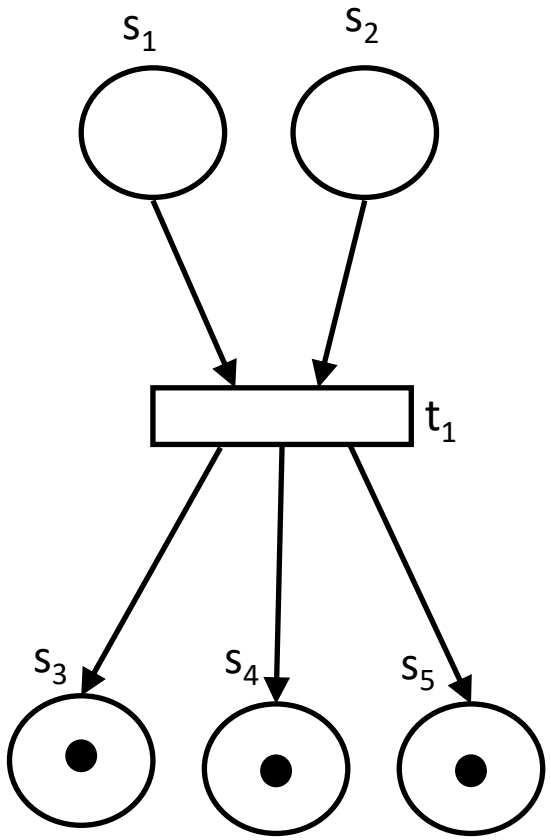
Petri nets

- Firing of transitions:
  - A transition is *enabled* when the number of tokens in each of its input places is at least equal to the arc weight going from the place to the transition
  - An enabled transition **may** *fire* at any time



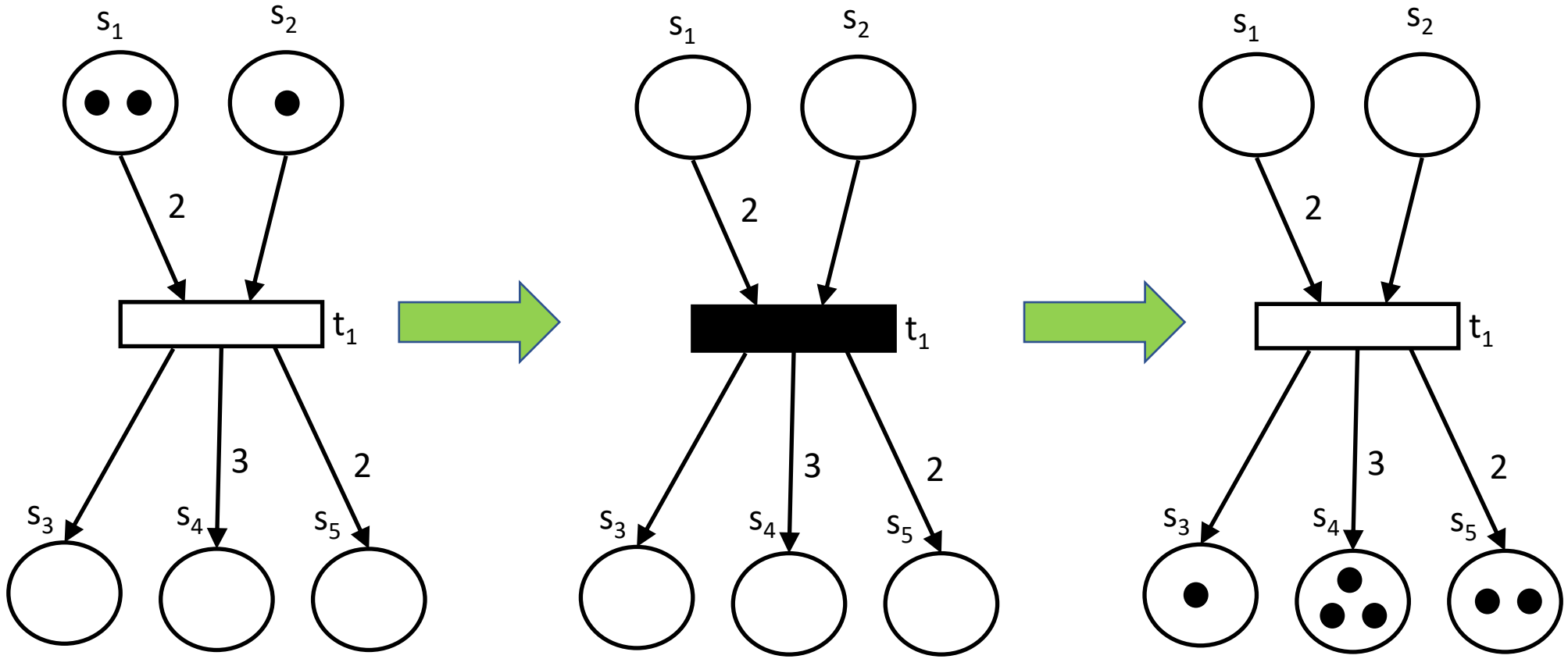
Petri nets

- Firing of transitions:
  - A transition is *enabled* when the number of tokens in each of its input places is at least equal to the arc weight going from the place to the transition
  - An enabled transition **may** *fire* at any time
  - When fired, the tokens in the input places are moved to output places, according to arc weights and place capacities



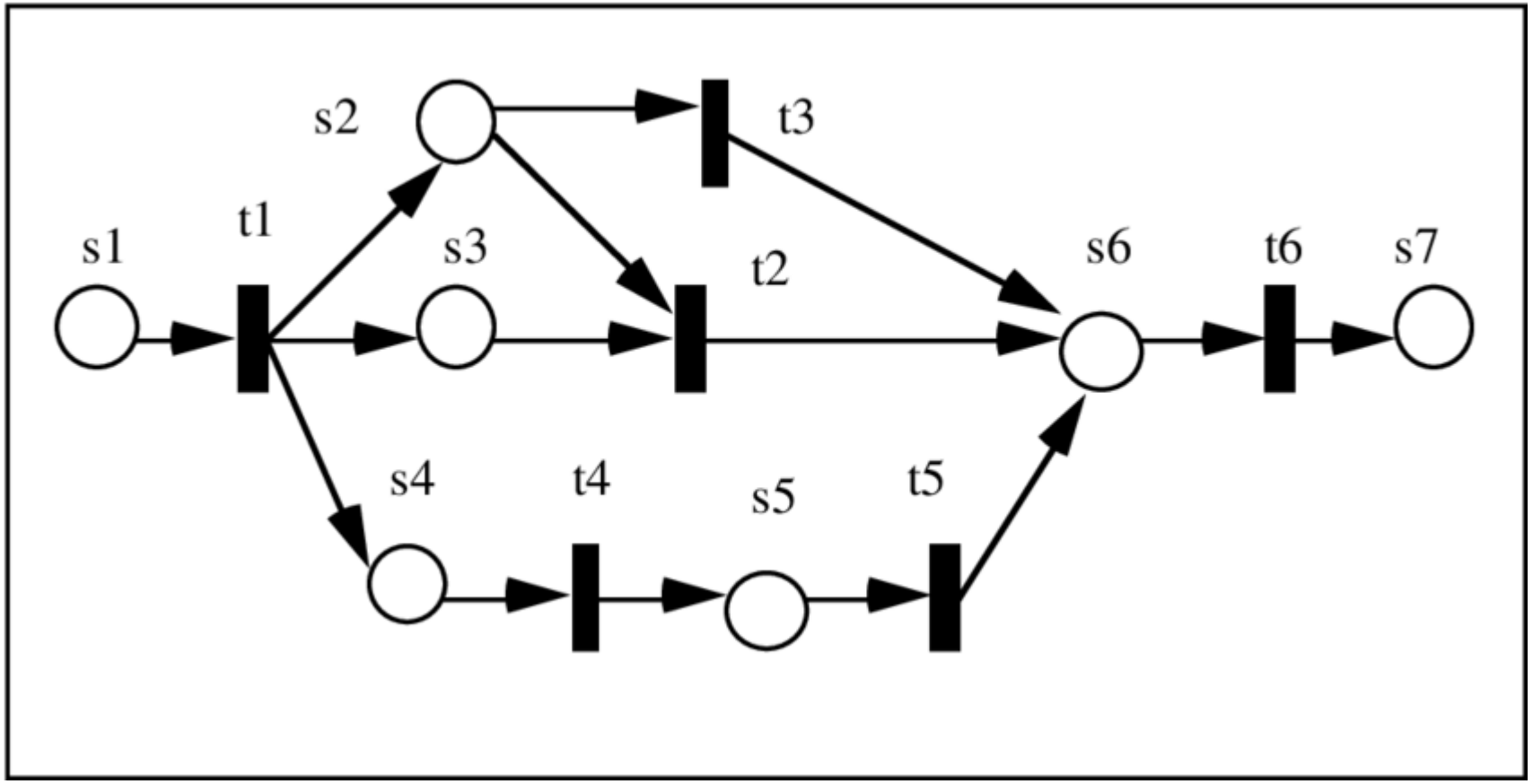
Petri nets

- Firing with different arch weights:



Petri nets

- Formal models build up from places, transitions and arches:



Source of the figure: Chen, Kuo-Tay and Lee, Ronald: Schematic Evaluation of Internal Accounting Control Systems. EURIDIS Research Monograph. (August 11, 1992

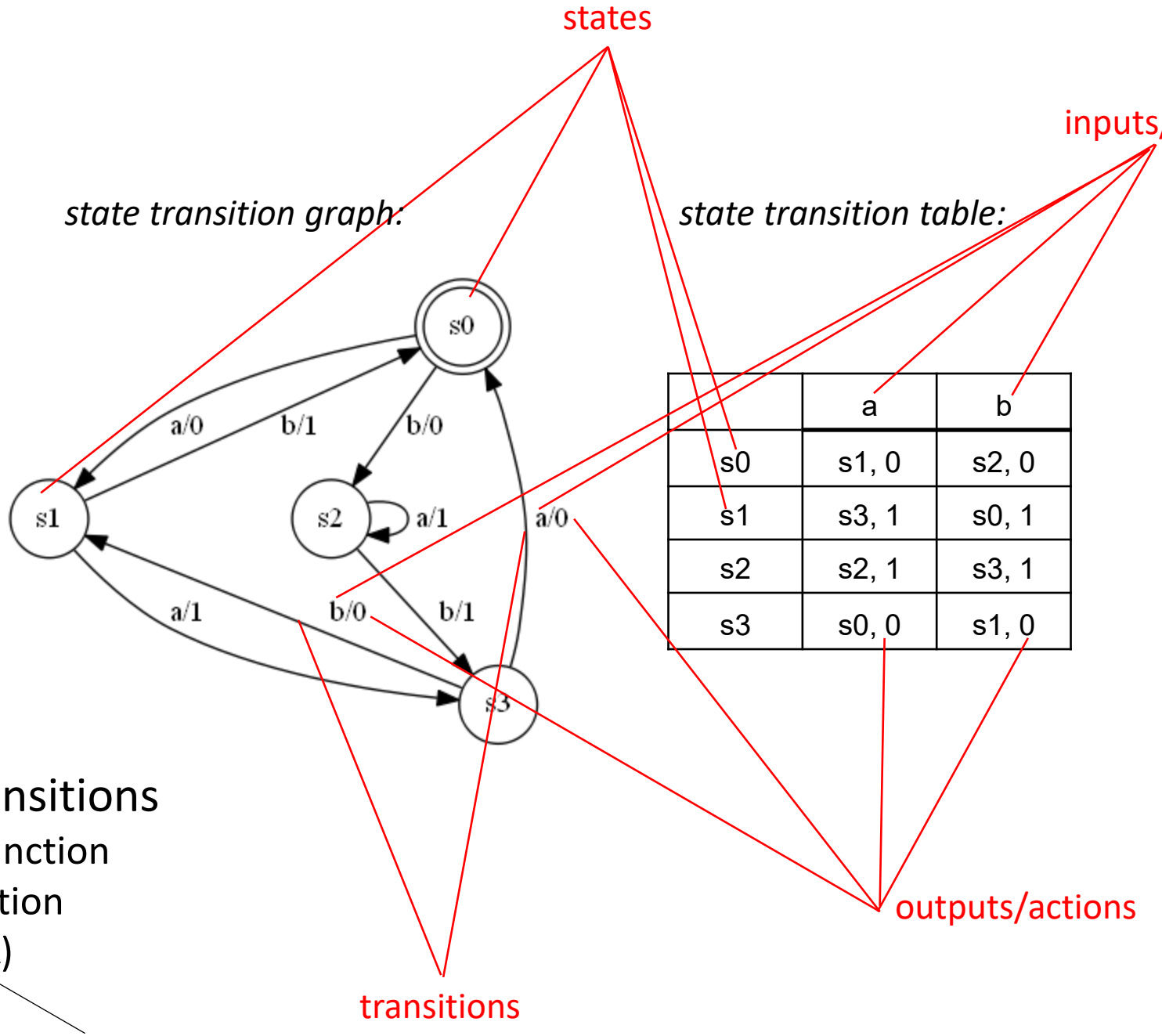
FSM

Finite State Machine

(Mealy-model)

- M=(I, O, S, T)
  - I: set of input symbols
  - O: set of output symbols
  - S: set of states
    - s_o: initial state
  - T: S x I → S x O: set of transitions
    - δ : S x I → S: next state function
    - λ : S x I → O: output function
    - Each t∈T: t=(s_x , s_y , i_x , o_x)

start state next state input output



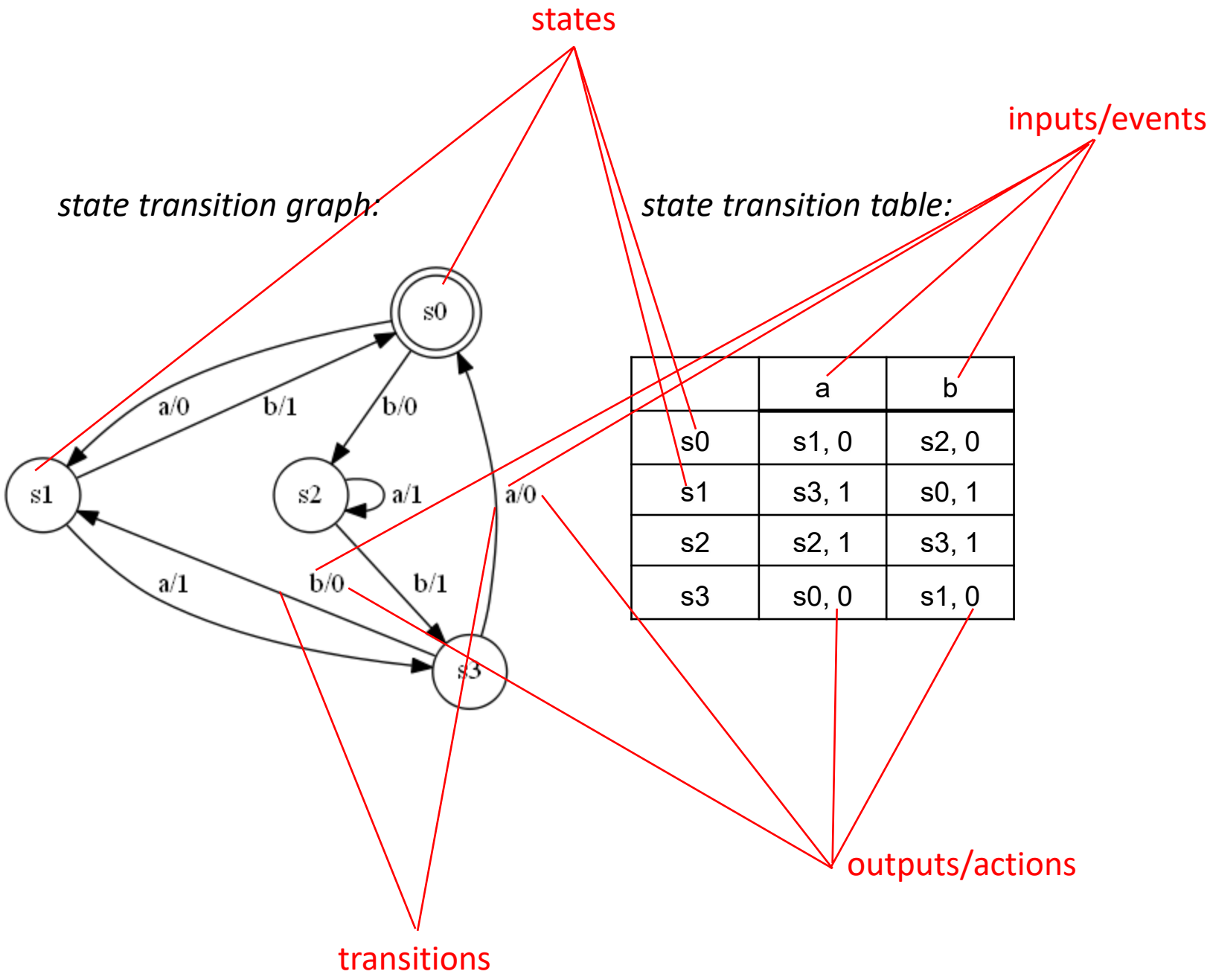
	a	b
s0	s1, 0	s2, 0
s1	s3, 1	s0, 1
s2	s2, 1	s3, 1
s3	s0, 0	s1, 0

STSs vs. FSMs

State Transition System

Same as FSMs, but they do not necessary have finite number of states, transitions, input and output symbols

→ less strict format



	a	b
s0	s1, 0	s2, 0
s1	s3, 1	s0, 1
s2	s2, 1	s3, 1
s3	s0, 0	s1, 0

1st FSM exercise

Märklin Battery-Operated Signal

1. Explore the functionality of the given device & write a specification for it!

- The device has 2 LEDs (red and green), a 2-way switch and a press button.
- The 2-way switch turns the device off or on. When the device is turned on, the green LED lights up.
- The button can be used to change which LED lights; red or green.
- If no button is pressed for 7 secs the device changes the lights.

1st FSM exercise

Märklin Battery-Operated Signal

1. Explore the functionality of the given device & write a specification for it!

2. Create an FSM based on the specification!
   - Create a state transition table
   - Create a state transition graph using Graphviz
     - *user guide*

   Waiting for 7 seconds can be handled as special `timeout` *message*

3. Design a test sequence (pair of input/output sequence) to test all the functionalities of the specification!

1st FSM exercise: solution

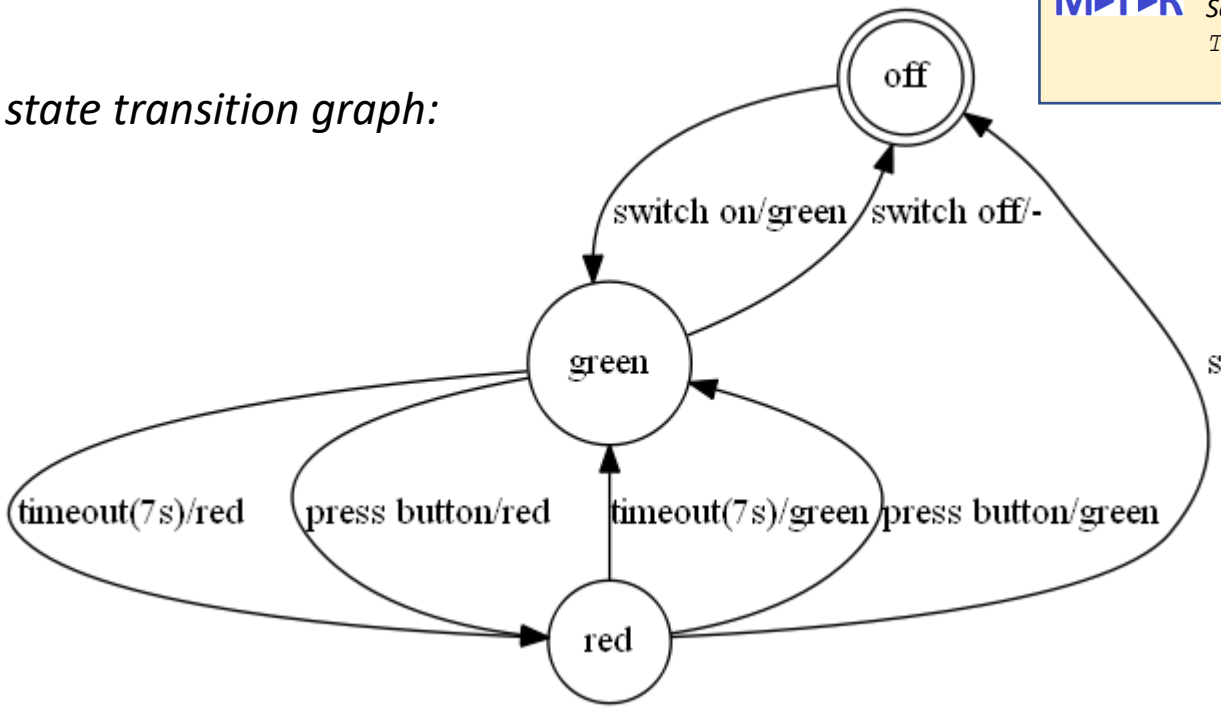
state transition table:

	switch on	switch off	press button	wait for 7 secs
off	s_{green} /green	-	-	-
green	-	s_{off} /-	s_{red} /red	s_{red} /red
red	-	s_{off} /-	s_{green} /green	s_{green} /green

example test sequence:

Action (input)	Desired output
switch on	green
wait for 7 secs	red
wait for 7 secs	green
press button	red
switch off	-
switch on	green
press button	red
press button	green
switch off	-

state transition graph:



***Graphviz* code:**

```
digraph Signal {
            off [shape=doublecircle]
            green [shape=circle]
            red [shape=circle]

            off -> green [label="switch on/green"]
            green -> red [label="timeout(7s)/red"]
            green -> red [label="press button/red"]
            green -> off [label="switch off/-"]
            red -> green [label="timeout(7s)/green"]
            red -> green [label="press button/green"]
            red -> off [label="switch off/-"]
}
```

Note that this model is included in the Model ⟫ Test ⟫ Relax repository: See `sample_models/introduction/Toy_train_signal.json`

2nd FSM exercise: overview

User agent client (UAC) during the registration process of SIP

An overview

Read Section 10 of RFC 3261 SIP: Session Initiation Protocol and Section 2 of RFC 3665 Session Initiation Protocol (SIP) Basic Call Flow Examples on the registration process of SIP

1. Identify the normal flow for the registration process!
   - For the sake of simplicity, please avoid authentication process at this stage
   - You can use plantUML to create the related Message Sequence

2. Create an FSM for the UAC from this normal flow!

3. Extend the previously created FSM with alternate flows described in the standard
   - Note that for some of these added functionalities, variables may be required

2nd FSM exercise: Hints for the 1st and 2nd subtasks

User agent client (UAC) during the registration process of SIP

1. Identify the normal flow for the registration process!

- Section 2.1. Successful New Registration of RFC 3665
  - For the sake of simplicity, please avoid authentication process at this stage
- Section 10.2.2 Removing Bindings of RFC 3261 and Section 2.4 Cancellation of Registration of RFC 3665

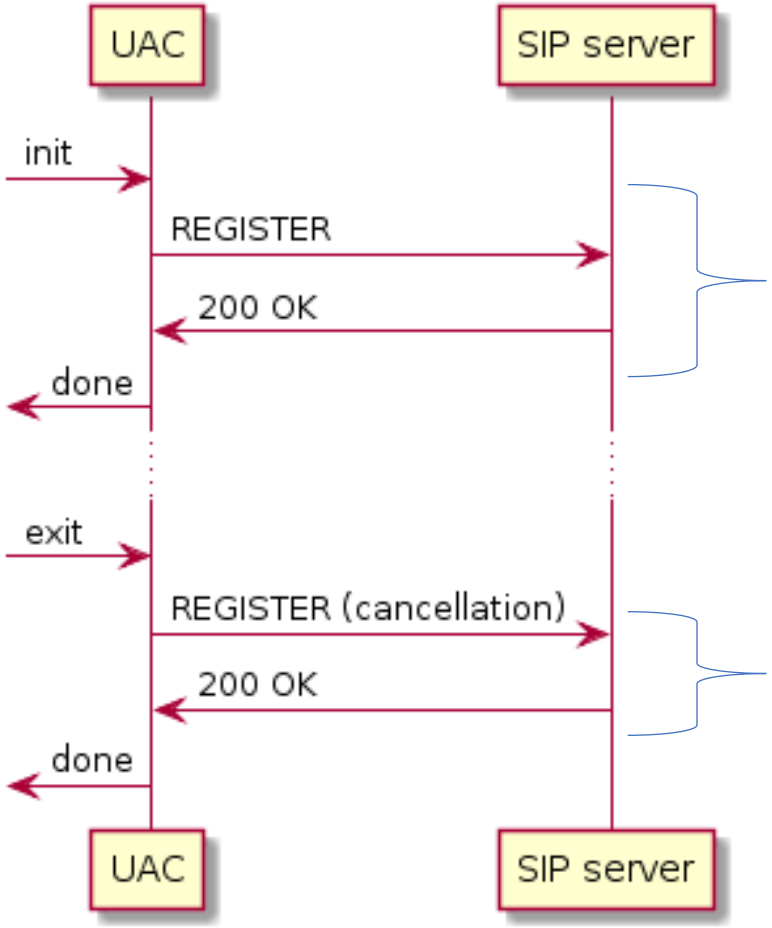
- Create the related Message Sequence with plantUML

2. Create an FSM for the UAC from this normal flow!

- Identify the signals the that UAC sends to/receives from server
- Identify the possible states of the UAC
- Try to find out signals that the UAC sends to/receives from the user

2nd FSM exercise: 1. Identify normal flow

- Normal flow:



Section 2.1. Successful New Registration of RFC 3665 without authentication (i.e. the „REGISTER ->” – „401 Unauthorized <-” message pair has been deleted at the beginning)

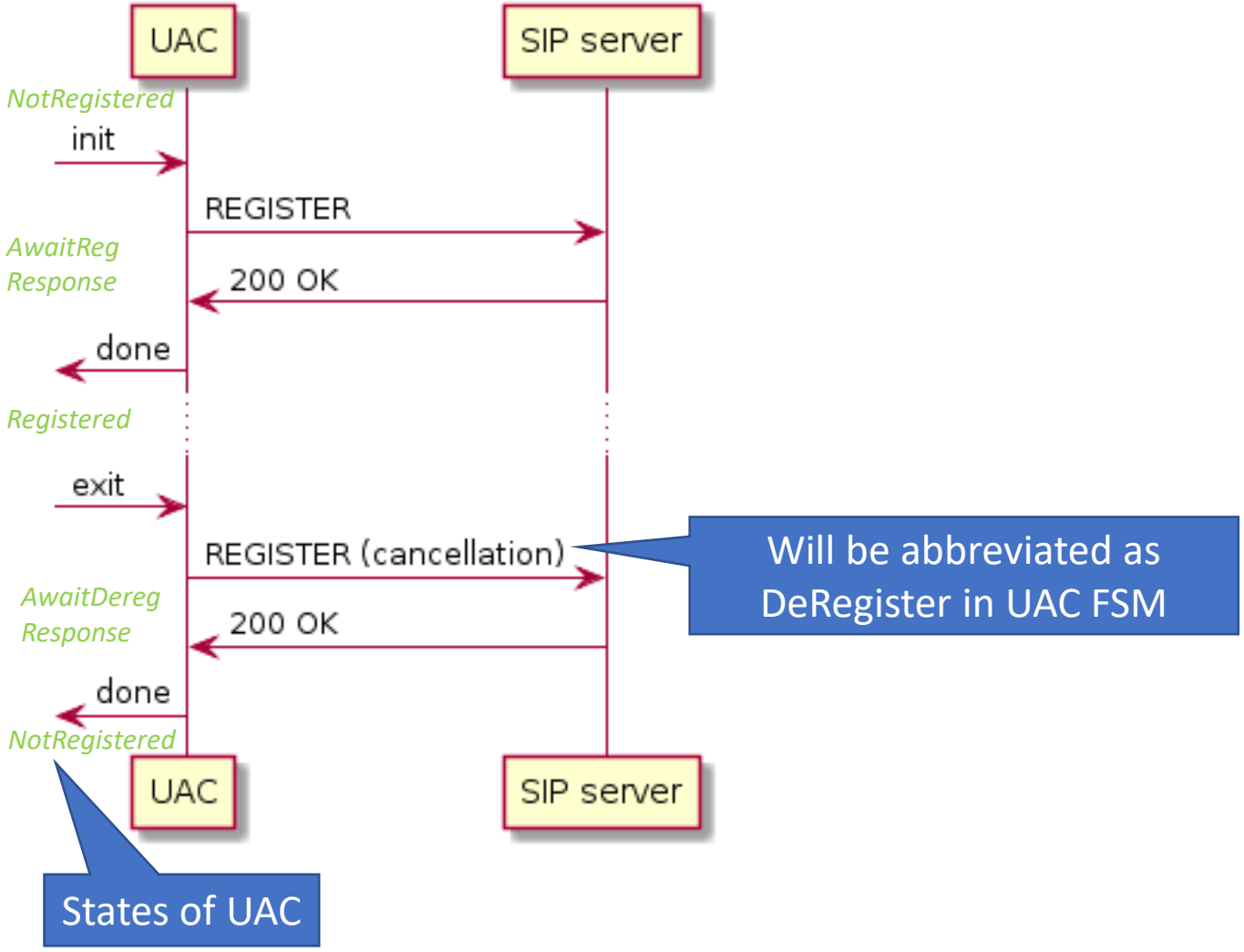
Section 10.2.2 Removing Bindings of RFC 3261 and Section 2.4 Cancellation of Registration of RFC 3665

plantUML code:

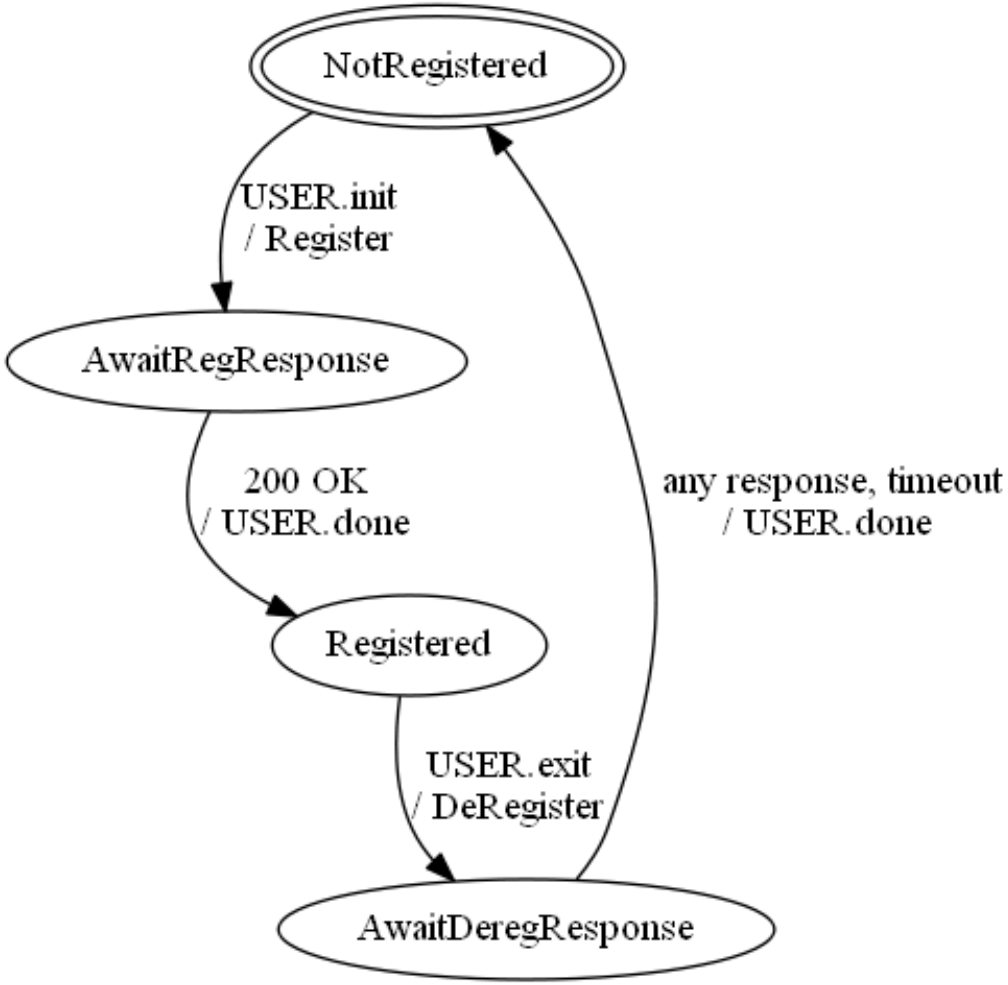
```
@startuml
-> UAC: init
UAC -> "SIP server" : REGISTER
UAC <- "SIP server" : 200 OK
<- UAC: done
...
-> UAC: exit
UAC -> "SIP server" : REGISTER (cancellation)
UAC <- "SIP server" : 200 OK
<- UAC: done
@enduml
```

2nd FSM exercise: 2. Create an FSM for normal flow

- Normal flow:



FSM for normal flow:



2nd FSM exercise: 3. Extend FSM with alternate flows

1. *see Section 2.1 and Section 2.5 of RFC 3665 and Section 10.3 / point 3rd of RFC 3261*

2. *see Section 10.3 / points 4th -6th of RFC 3261*

3. *see Section 10.3 / point 7th of RFC 3261*

4. *see Section 10.2.7 of RFC 3261*

5. *see Section 10.2.1.1 and Section 10.2.4 of RFC 3261*

Which parts of Section 10 of RFC 3261 and Section 2 of RFC 3665 are relevant for this task?

2nd FSM exercise: 3. Extend FSM with alternate flows

1. Handle of authentication – *see Section 2.1 and Section 2.5 of RFC 3665 and Section 10.3 / point 3rd → Section 22 of RFC 3261*

2. Handle negative responses for registration requests – *see Section 10.3 / points 4th -6th of RFC 3261*

3. Interval too brief – *see Section 10.3 / point 7th of RFC 3261*

4. Silent discard: - *see Section 10.2.7 of RFC 3261*

5. Reregister functionality - *see Section 10.2.1.1 and Section 10.2.4 of RFC 3261*

2nd FSM exercise: 3. Extend FSM with alternate flows

1. Handle of authentication: If the server replies with message *401 Unauthorized*, then it requests authentication with a challenge. In this case the UAC should send the authentication data (answer to the challenge) received by the user. – *see Section 2.1 and Section 2.5 of RFC 3665 and Section 10.3 / point 3rd → Section 22 of RFC 3261*

2. Handle negative responses for registration requests: If other than a *200 OK* response (i.e. a negative response) has been received for the *Register* request, then the FSM of UAC should go back to the initial state *NotRegistered* and output to the user that registration was unsuccessful. – *see Section 10.3 / points 4th -6th of RFC 3261*

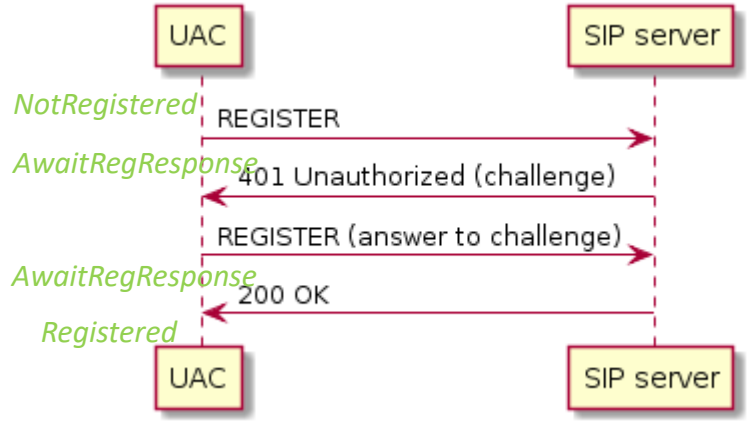
3. Interval too brief: Upon the receive of *423 Interval Too Brief (with* Min-Expires header *value)* the UAC sends a *Register* message to the server again (with an enlarged expiration time value) and stays on *AwaitRegResponse* till it receives message *200 OK. – see Section 10.3 / point 7th of RFC 3261*

4. Silent discard: If no response has been received for registration requests within a predefined time, a *Response timeout* has been received and the UAC should wait some reasonable time sends the *Register* request to the registration server again and goes to state *AwaitRegResponse*. - *see Section 10.2.7 of RFC 3261*

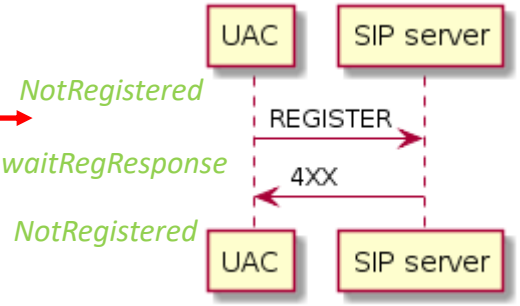
5. Reregister functionality: When a *Registration timeout* message is received in state *Registered*, the UAC should send a *ReRegister* message to the server and go to state *AwaitRegResponse* (note that in this case as in Section 5.6.4 some simplification is made as this action is triggered by an internal timer before the client actually receives *Registration timeout*). - *see Section 10.2.1.1 and Section 10.2.4 of RFC 3261*

2nd FSM exercise: 3. Extend FSM with alternate flows

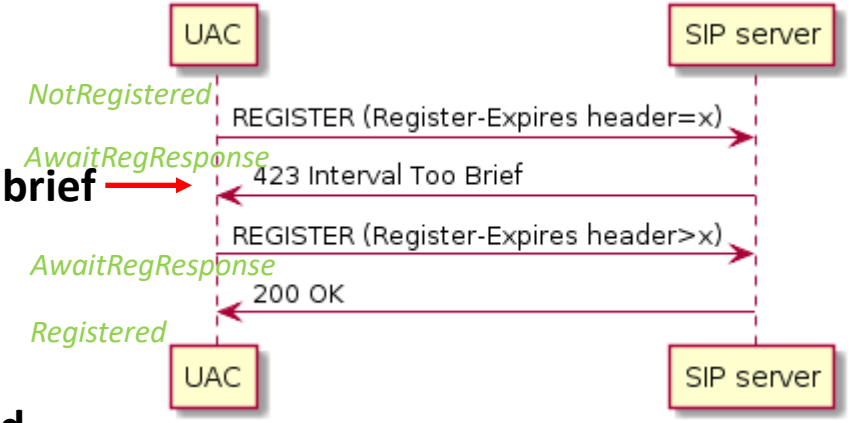
1. Handle of authentication



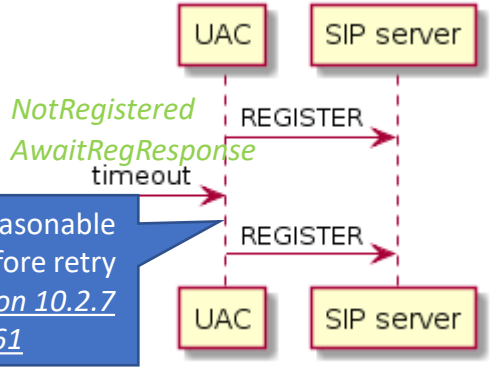
2. Handle negative responses for registration requests



3. Interval too brief

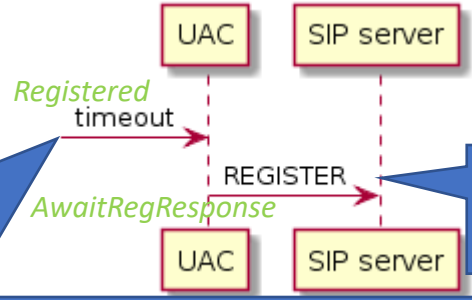


4. Silent discard



„Waiting some reasonable time interval" before retry registration *Section 10.2.7 of RFC 3261*

5. Reregister functionality



Will be denoted as ReRegister in FSM

This timeout is the half of expiration time of SIP server

2nd FSM exercise: 3. Extend FSM with alternate flows

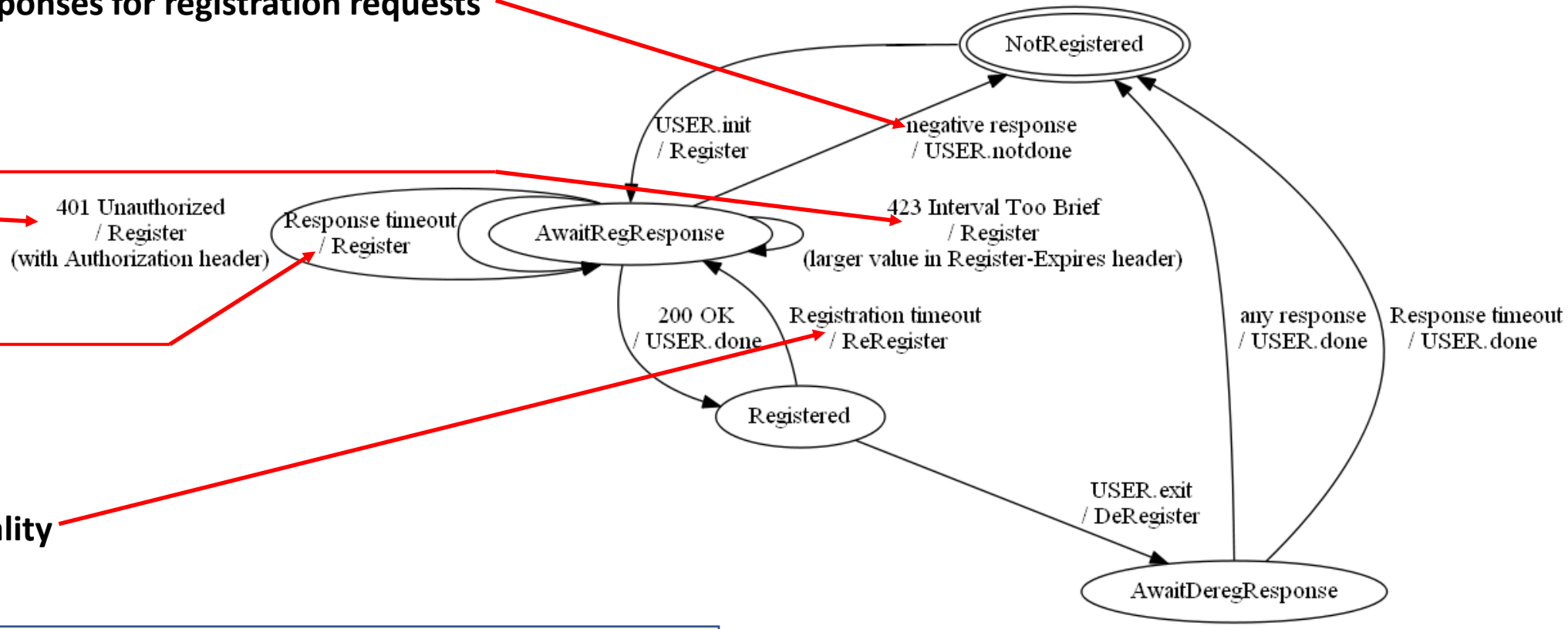
1. Handle of authentication

2. Handle negative responses for registration requests

3. Interval too brief

4. Silent discard

5. Reregister functionality

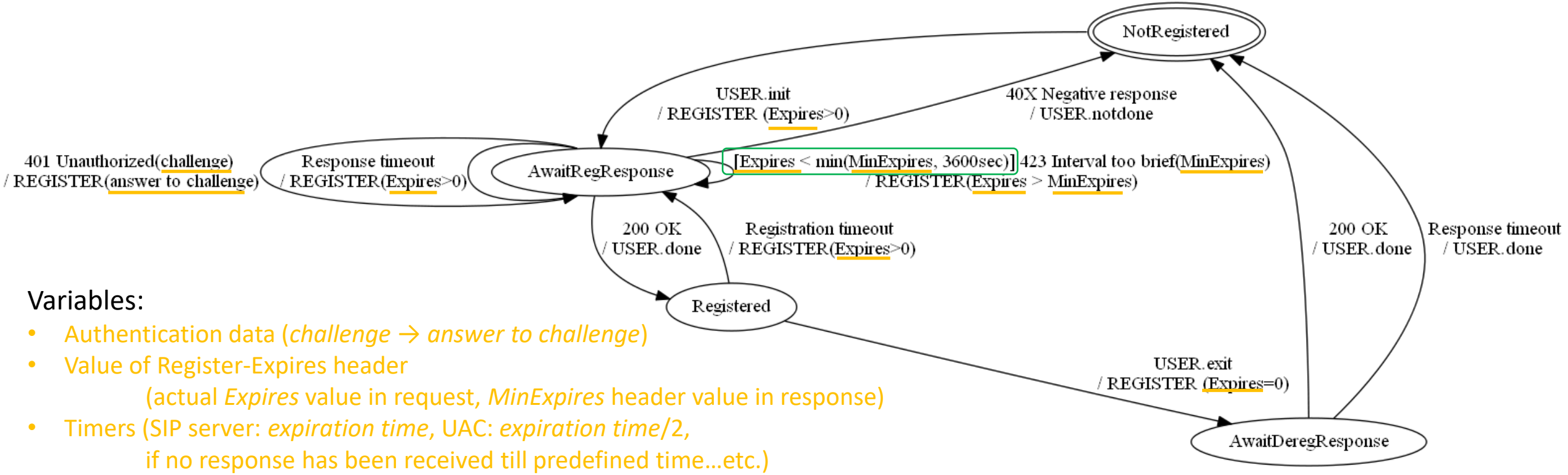


Note that this model is included in the Model ⟫ Test ⟫ Relax repository:
See `sample_models/introduction/SIP_Registration_UAC.json`

Extensions of the FSM formalism: EFSMs

- EFSMs (Extended Finite State Machines)

Example:
SIP (RFC 3261): Registration of UAC



Variables:

- Authentication data (*challenge → answer to challenge*)
- Value of Register-Expires header
  (actual *Expires* value in request, *MinExpires* header value in response)
- Timers (SIP server: *expiration time*, UAC: *expiration time*/2,
  if no response has been received till predefined time…etc.)

Predicate / guarding condition on variable values:

- check received *Expires* value, check *answer to challenge*

Extensions of the FSM formalism: EFSMs

- EFSMs (Extended Finite State Machines)
  - The extension of FSMs with variables and operations based on variable values

- M=(I, O, S, V, T)
  - I: set of input symbols
  - O: set of output symbols
  - S: set of states
  - V: set of variables (each of it with given range)
  - T: set of transitions
    - Each t∈T: t=(s_x , s_y , i_x , o_x , $P_x(V)$, $A_x(V)$)

start state — next state — input — output — predicate on current variable values — action on current variable values

Extensions of the FSM formalism: EFSMs

The predicate on variable values of the EFSM can be denoted…
1. as an input of the transition originating from a distinct predicate state

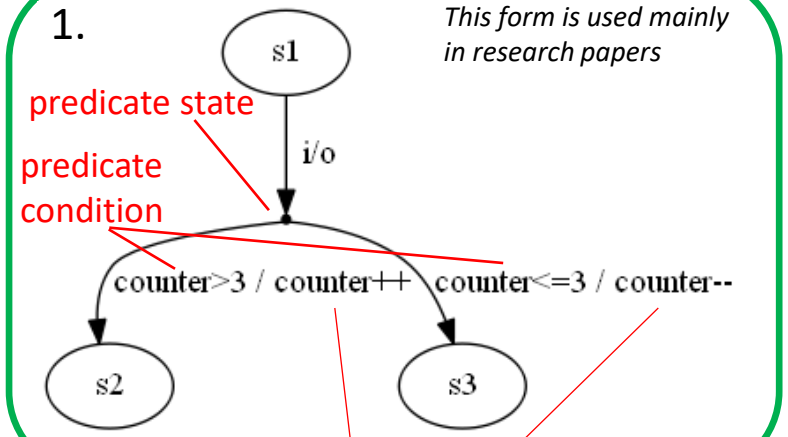
OR
2. as a part of an input (called GUARD condition, written between "[ ]" brackets) of the transition originating from the given state

Similarly the action on current variable values can be…
1. denoted as an output of the transition originating from a distinct predicate state

OR
2. as a part of an output of the transition originating from the given state

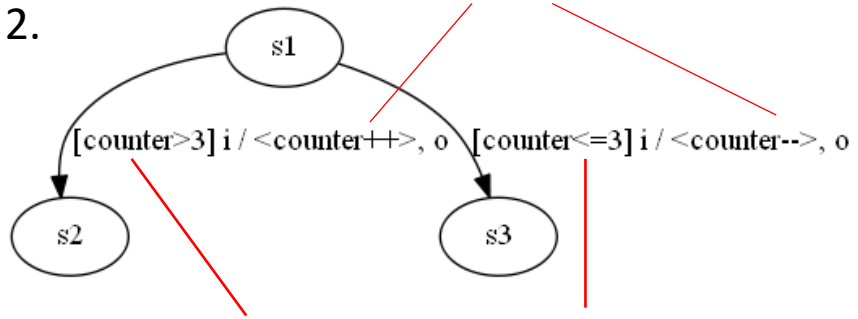
The 2 description modes can be transformed into each other

1.



This form is used mainly in research papers

2.



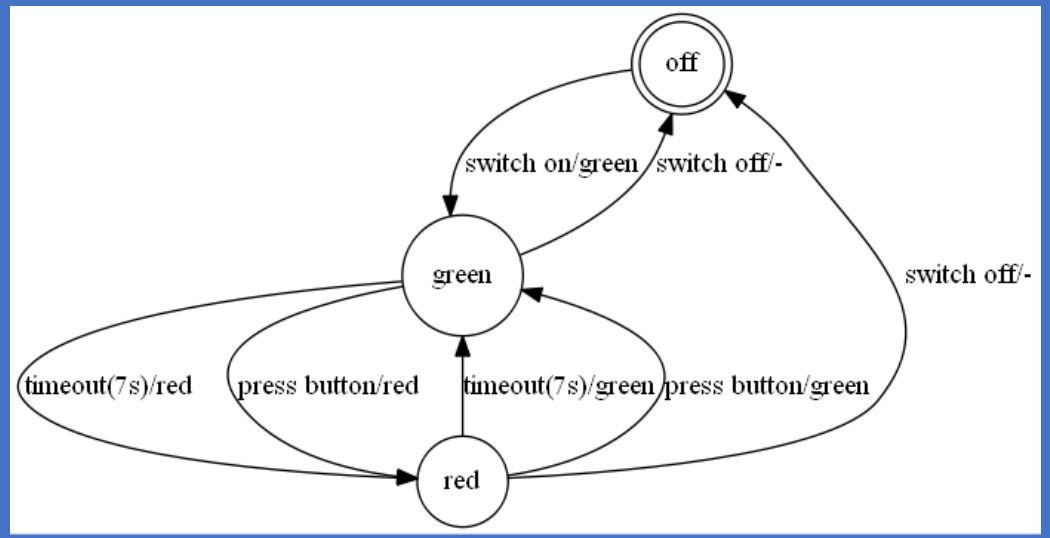
This form is used mainly in MBT tools

- T: set of transitions
  - Each t∈T: t=(s_x , s_y , i_x , o_x , $P_x(V)$, $A_x(V)$)

start state — next state — input — output — predicate on current variable values — action on current variable values

EFSMs - exercise

Battery-Operated Signal – extended version



	switch on	switch off	press button	wait for 7 secs
off	s_{green} /green	-	-	-
green	-	s_{off} /-	s_{red} /red	s_{red} /red
red	-	s_{off} /-	s_{green} /green	s_{green} /green

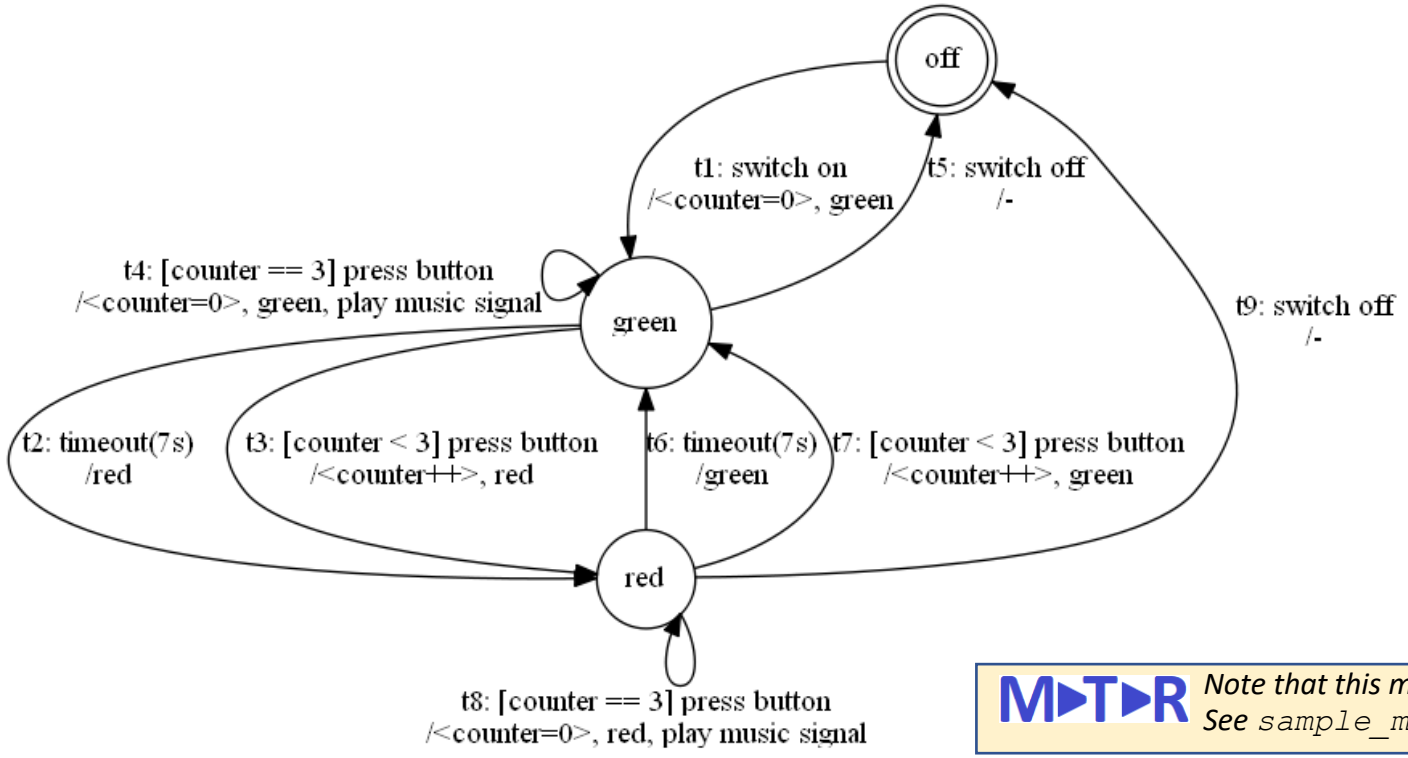
Extend the functionality of the previously discussed battery-operated signal (see spec above) with the following:

- If the button has been pressed *4* times, then the device will play the MÁV music signal and it won't change its' state
- The counter of button presses is set to *0* when the music signal ends or if the device is switched on .

EFSMs - solution

Note that the counter is incremented after condition evaluation (as input proceeds output)

	switch on	switch off	[counter < 3] press button	[counter == 3] press button	wait for 7 secs
off	green /<counter=0>, green	-	-	-	-
green	-	s_{off} /-	s_{red} /<counter++>, red	s_{green} /<counter=0>, green, play signal	s_{red} /red
red	-	s_{off} /-	s_{green} /<counter++>, green	s_{red} /<counter=0>, red, play signal	s_{green} /green



MTR *Note that this model is included in the Model ⟫ Test ⟫ Relax repository:*
See `sample_models/introduction/Toy_train_signal_EFSM.json`

EFSMs – conversion into FSM

- EFSMs (Extended Finite State Machines)
  - Can be converted into FSMs
    - All state-variable value combinations of the EFSM → distinct state in the FSM
      - It results in a state explosion problem
    - Many approaches to solve the state explosion problem
      - Generate “collapsed FSM” on-the-fly from EFSM
        - only those state-variable combinations that are used in the EFSM and are reachable from the initial state are generated
      - Limit the possible range of variables
      - Decomposition*
        - Functionalities
        - Use cases

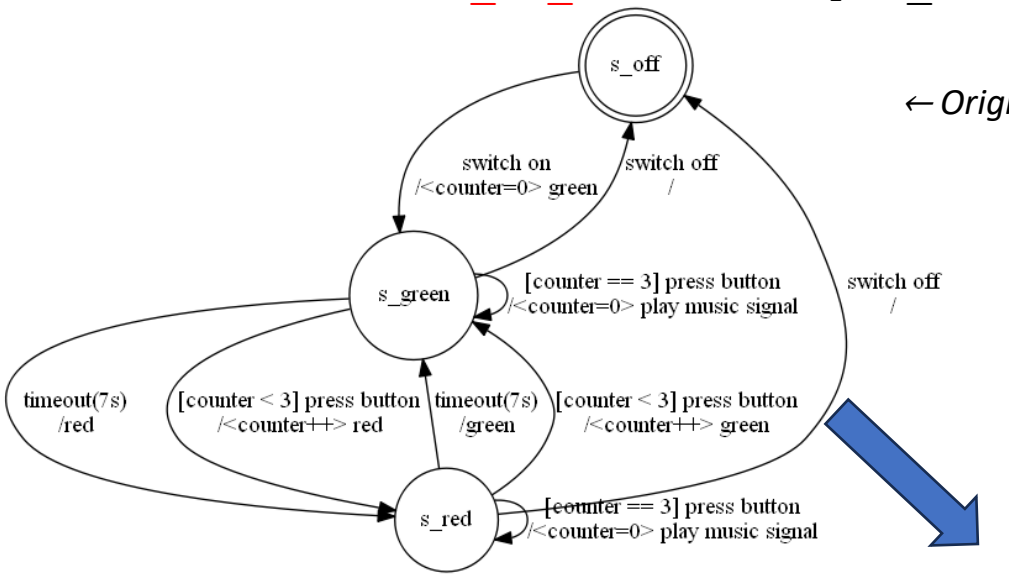
* *Note that decomposition may simplify the original problem as restricts interaction between different components (i.e. trade-off between transparency of the model and the coverage of all possible errors / functionality)*

M▶T▶R *An EFSM→FSM conversion method is implemented in Model ⟫ Test ⟫ Relax framework.*

EFSMs – conversion into FSM with MTR

Apply EFSM→FSM conversion on the extended version of battery-operated Toy Train signal using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

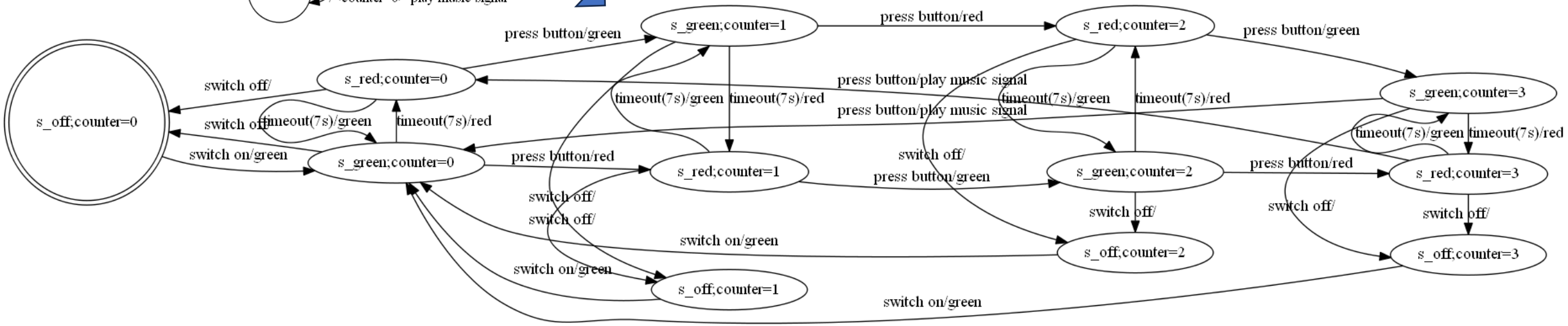
```
./MTR -o c -m efsm_to_fsm -f sample_models/introduction/Toy_train_signal_EFSM.json --graphviz
```



← Original EFSM model

MTR *Note that this model is included in the Model ⟫ Test ⟫ Relax repository:*
See `sample_models/introduction/Toy_train_signal_EFSM.json`

Converted FSM model:



EFSMs – conversion into FSM with MTR

Apply EFSM→FSM conversion on the extended version of locomotive shop using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o c -m efsm_to_fsm -f sample_models/introduction/Model_railway_shop2.json --graphviz
```

Requirements

- **R1:** In the model railway webshop the user can buy locomotives and wagons.
- **R2:** Before any user action is initiated the number of locomotives and wagons are 0.
- **R3:** The user can add locomotives and wagons to the chart, one-by-one.
- **R4:** The user can remove locomotives and wagons from the chart, one-by-one.
- **R5:** The user can empty the chart.
- **R6:** If the user add the 2nd locomotive into the chart, then a 8% discount is offered.
- **R7:** If the user remove a locomotive from the chart and less then 2 remains, then the 8% discount is withdrawn.
- **R8:** If the user add the 4th wagon into the chart, then an exclusive wagon is offered for free.
- **R9:** If the user remove a wagon from the chart and less then 4 remains, then the free, exclusive wagon is withdrawn.
- **R10:** Discounts mentioned in requirements R6 and R8 can be combined, if both conditions are fulfilled.
- **R11:** Limit the number of possible items to chart into 16 (excluding the possible, exclusive free wagon).

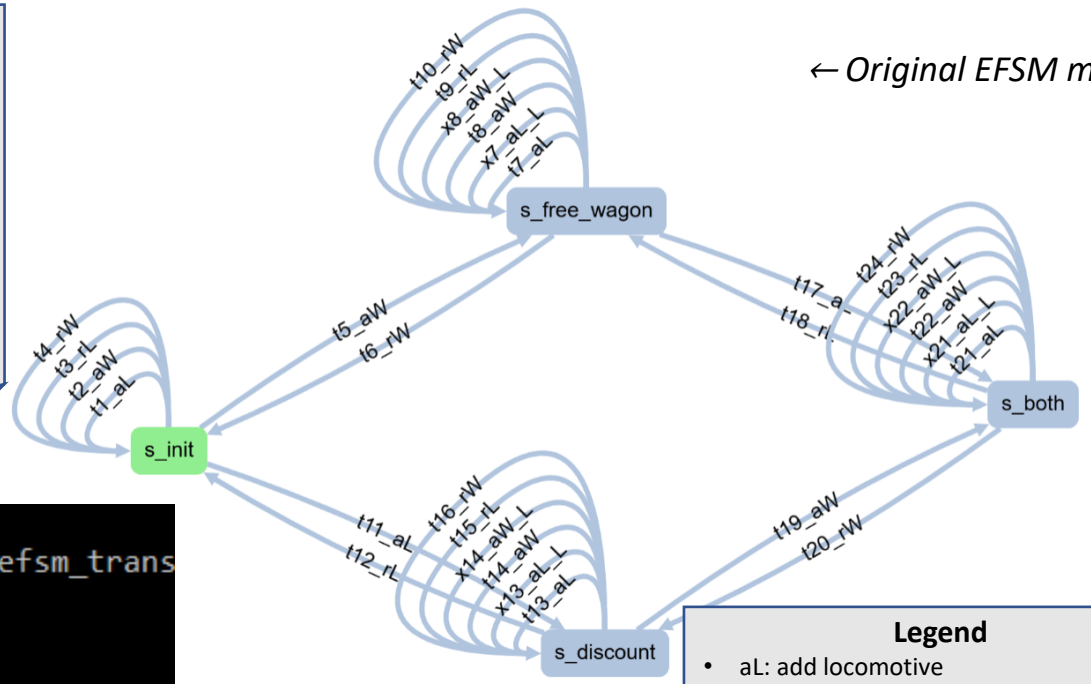
Parameters of the EFSM model:

```
[U-LMI  ] [info] Model summary:
        Name: Model_railway_shop_2
        Type: EFSM
        Number of states: 4
        Number of transitions: 30
        Density: 7.500000
        Number of input symbols: 4
        Number of output symbols: 9
        Completely specified: Yes
        Strongly connected: Yes
        Reset: Yes (UNRELIABLE/FLAG)
        Number of variables: 4
        Number of actions: 56
        Number of guard conditions: 30
```

Parameters of the converted FSM model:

```
[U-LMI  ] [info] Model summary:
        Name: Model_railway_shop_2-efsm_trans
        Type: FSM
        Number of states: 153
        Number of transitions: 578
        Density: 3.777778
        Number of input symbols: 4
        Number of output symbols: 9
        Completely specified: No
        Strongly connected: Yes
        Deterministic: Yes
        Reset: Yes (UNRELIABLE/FLAG)
        Reduced: No
```

← Original EFSM model



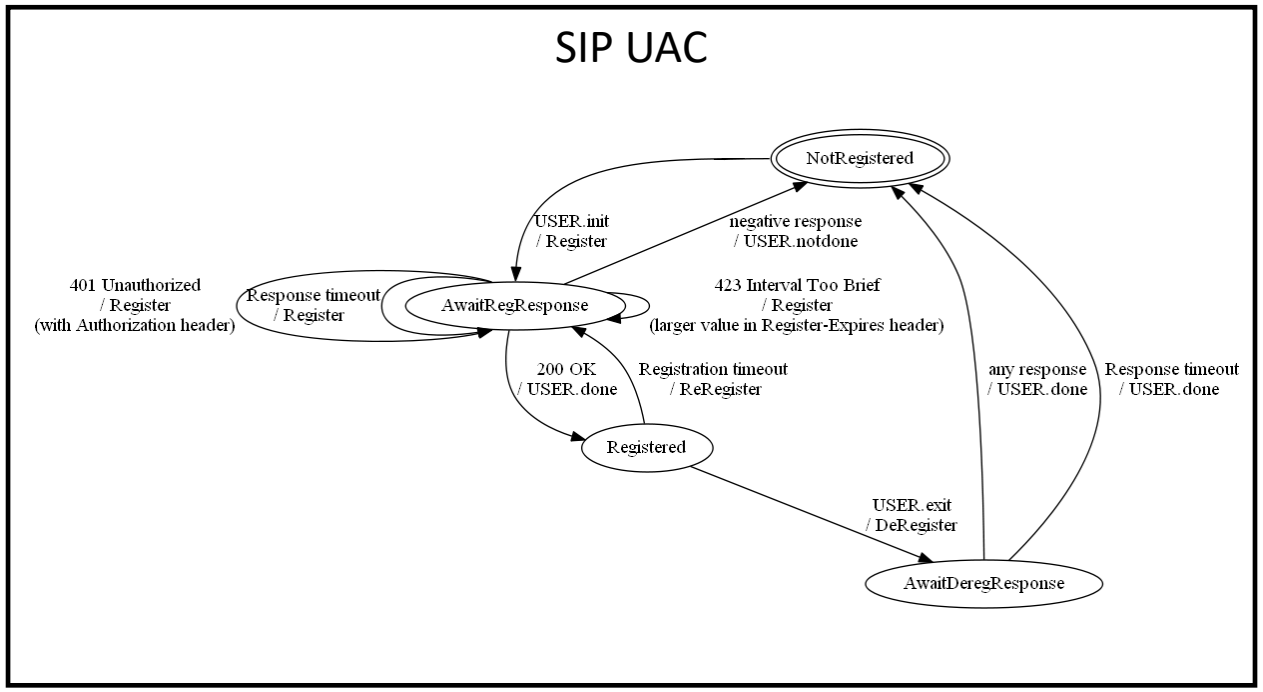
Legend

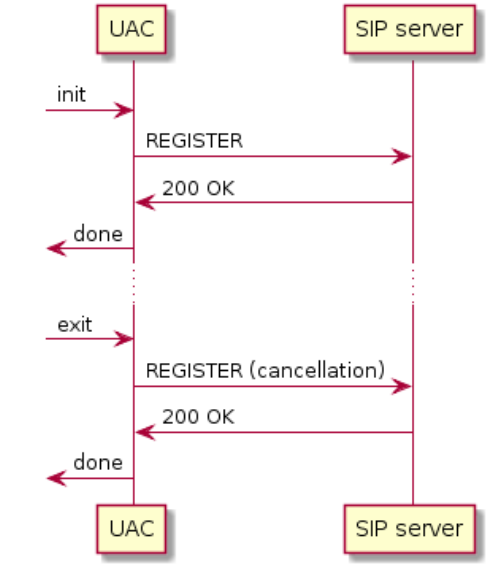
- aL: add locomotive
- aL_L: add locomotive with limit reached
- aW: add wagon
- aW_L: add wagon with limit reached
- rL: remove locomotive
- rW: remove wagon

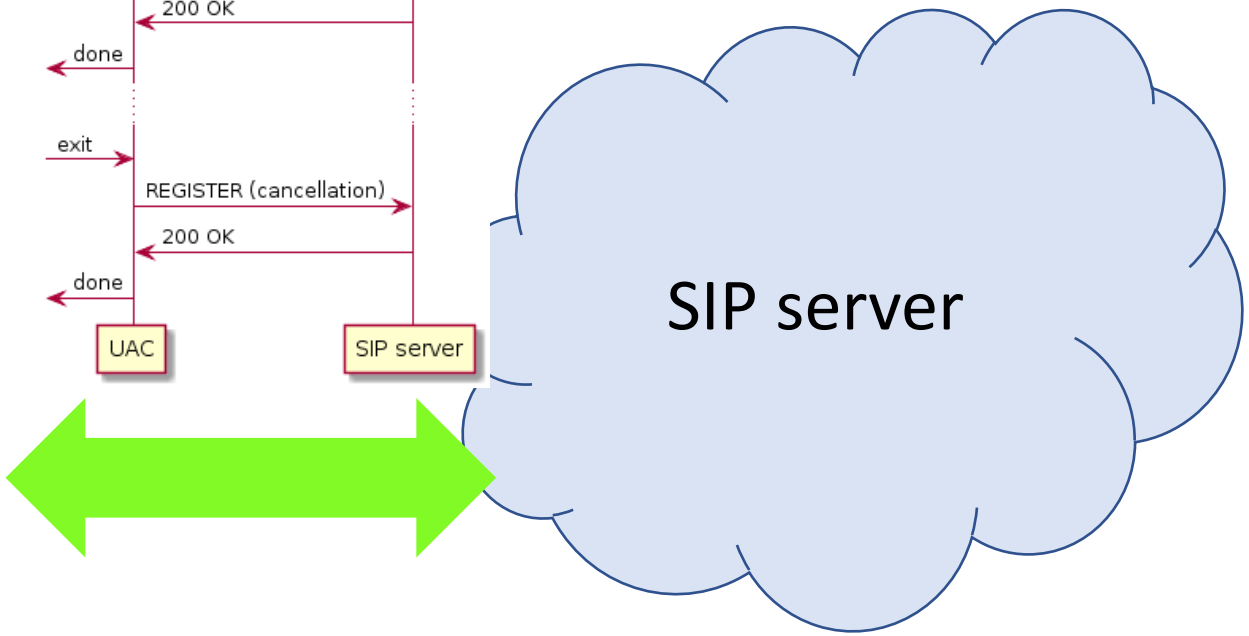
MTR *Note that this model is included in the Model ⟫ Test ⟫ Relax repository:*
See `sample_models/introduction/Model_railway_shop2.json`

Extensions of the FSM formalism: CFSMs

- Ok, but this was only one side of the communication…

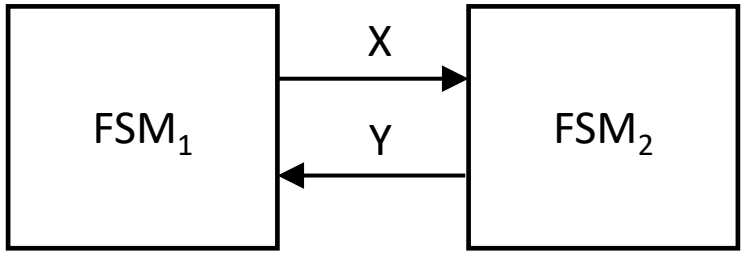


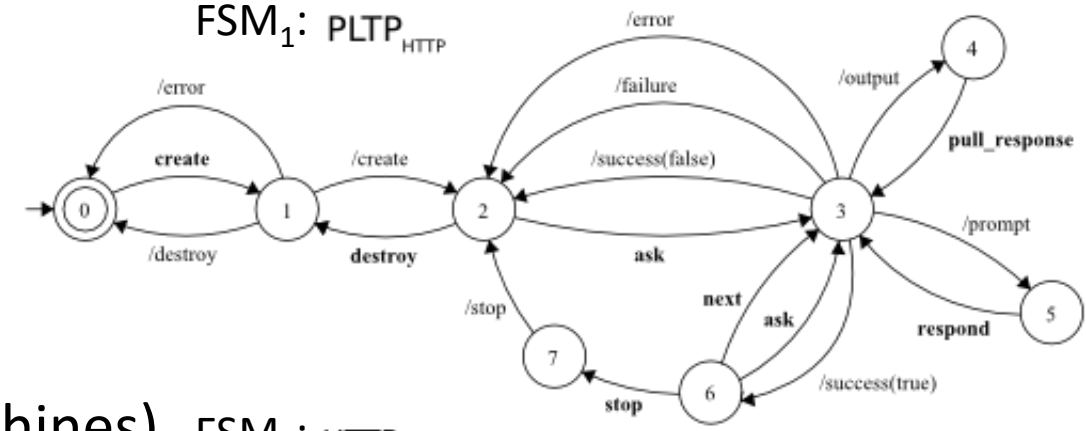




Extensions of the FSM formalism: CFSMs

- CFSMs (Communicating Finite State Machines)
  - Each FSM is a component machine
  - I/O interactions between components
  - CFSM → FSM conversion possible
  - CFSMs can be also extended with variables:
    CEFSM (Communicating Extended Finite State Machines)
    - CEFSM → EFSM → FSM conversion possible





CEFSM model handling and CEFSM→EFSM conversion are implemented in Model 》Test 》Relax framework.

Example: PLTP(HTTP)

CEFSMs – conversion into EFSM with MTR

Apply CEFSM→EFSM conversion on the extended version of locomotive shop using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o c -m cefsm_to_efsm -f sample_models/test/CEFSM/cefsm_transformer/cefsm_transform_simple.json
```

Parameters of the CEFSM model:

```
Model summary:
        Name: cefsm_tr_simple
        Type: CEFSM
        Number of participants: 2
Summary of components:
        Name: c1
                Type: FSM
                Number of states: 2
                Number of transitions: 2
                Density: 1.000000
                Number of input symbols: 1
                Number of output symbols: 1
                Completely specified: Yes
                Strongly connected: Yes
                Deterministic: Yes
                Reset: No

        Communications:
                        Target: c2
                        Type: Send
                        Relationship: 1-to-1
                        Blocking: Single
                        Transitions: e2


        Name: c2
                Type: EFSM
                Number of states: 2
                Number of transitions: 2
                Density: 1.000000
                Number of input symbols: 1
                Number of output symbols: 1
                Completely specified: Yes
                Strongly connected: Yes
                Reset: No
                Number of variables: 5
                Number of actions: 0
                Number of guard conditions: 1

        Communications:
                        Target: c1
                        Type: Receive
                        Relationship: 1-to-1
                        Blocking: Single
                        Transitions: e2
```

Parameters of the converted EFSM model:

```
[U-LMI  ] [info] Model summary:
        Name: cefsm_tr_simple-cefsm_trans
        Type: EFSM
        Number of states: 4
        Number of transitions: 7
        Density: 1.750000
        Number of input symbols: 1
        Number of output symbols: 2
        Completely specified: Yes
        Strongly connected: Yes
        Reset: No
        Number of variables: 11
        Number of actions: 4
        Number of guard conditions: 4
        The following variables are uninitialized: c1_b, c1_i, c1_s
```

Extensions of the FSM formalism: PFSMs

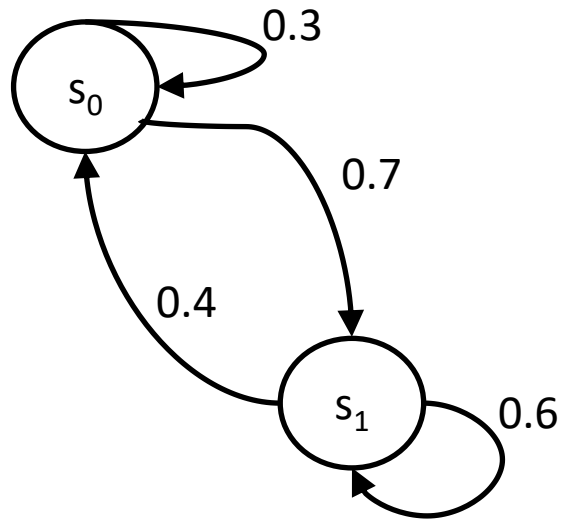
- PFSMs (Probabilistic Finite State Machines)
  - The extension of FSMs with probabilities

- M=(I, O, S, T, P)
  - I: set of input symbols
  - O: set of output symbols
  - S: set of states
  - T: set of transitions
  - P: Probability: function $P(t) \rightarrow [\ 0\ ,\ 1\ ]$ to each transition t

Markov chains

- “A stochastic model describing a sequence of possible events in which the probability of each event depends only on the state attained in the previous event“ *(wikipedia)*

- Can be considered as a special case of PFSM, where there is no input symbol: a transition from a given state is selected only by its probability

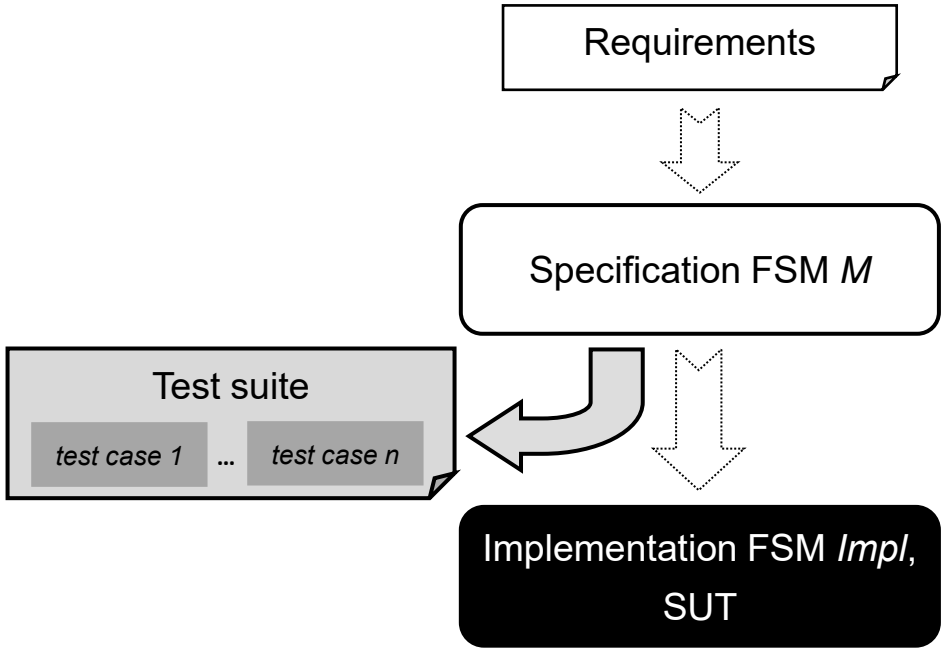


FSM-based conformance testing

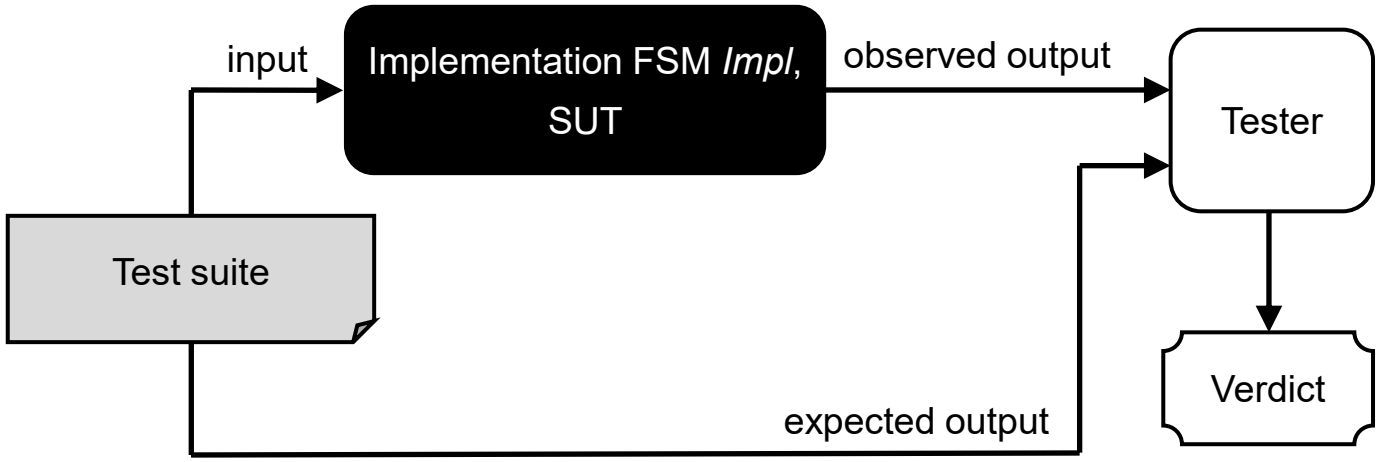
Specification:
FSM *M*
(white box)

?

Implementation:
FSM *Impl*
(black box)



FSM-based test generation



FSM-based testing

Test coverage criteria

n: number of states
p: number of events (input symbols)
a: number of actions (output symbols)
m: number of transitions

	Complexity
• All states	$O(n)$
• All events (all inputs)	$O(p) \leq O(m)$
• All actions (all outputs)	$O(a) \leq O(m)$
• All transitions	$O(m)$
• All transition-pairs	$O(m^2)$
• All n-transition *(n-switch coverage)*	$O(m^{n+1})$

lower complexity, lower fault detection capability

higher complexity, higher fault detection capability

- Every pair of adjacent transitions is traversed at least once
- Every transition sequence generated by n+1 consecutive events is exercised at least once

Test coverage criteria

- All-Transition-State (ATS):

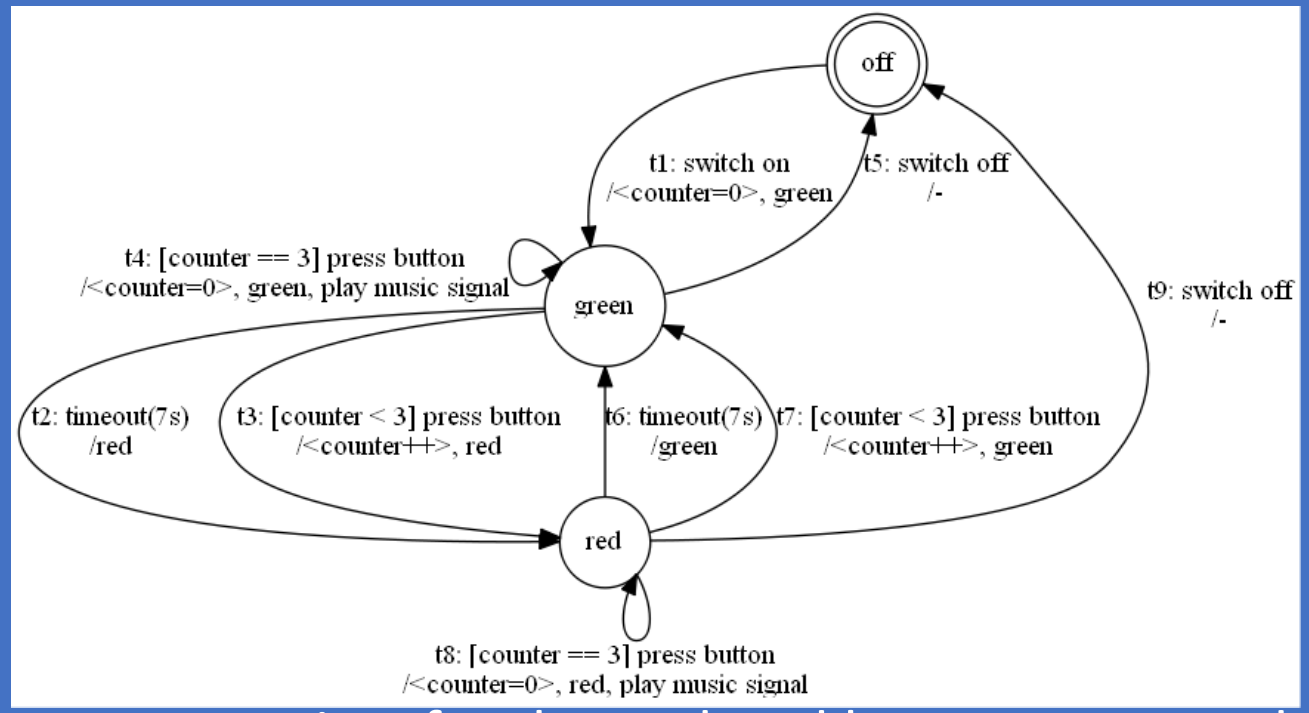
Complexity $O(m*n)$

n: number of states
m: number of transitions

  - S_t: states reachable from transition t
  1. For all t transitions: test set must cover at least one walk via t to S_t
  2. There is at least one walk to all S_t states which does not include transition t (if possible)
  3. When there is a boundary value guard condition in a transition in walk P, then P should be extended in a way that reaches the boundary value of the guard condition (if feasible)
     - i.e. the given transition should be traversed multiple times, till the boundary value of the guard condition is reached

- All-Transition-Transition (ATT): $O(m^2)$
  - Similar to All-Transition-States criterion, but instead of using S_t states reachable from transition t, it uses T_t transitions reachable from transition t

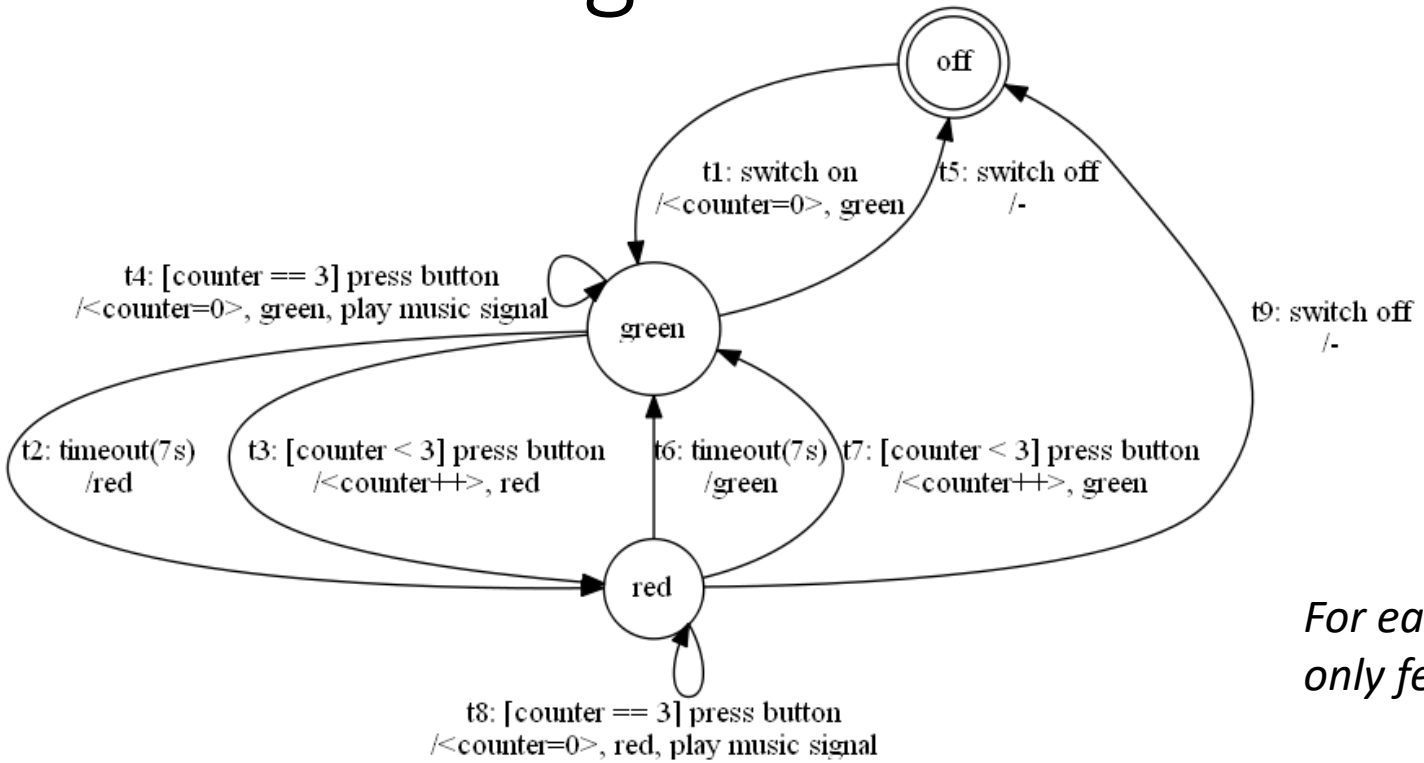
Test coverage criteria – exercise part 1



Create test suites for the updated battery-operated signal FSM with the following coverages:

1. All state coverage
2. All events (inputs) coverage
3. All actions (outputs) coverage
4. All transitions coverage
5. All transition-pairs coverage (only the identification of the pairs are required)

Test coverage criteria – solution part 1.1



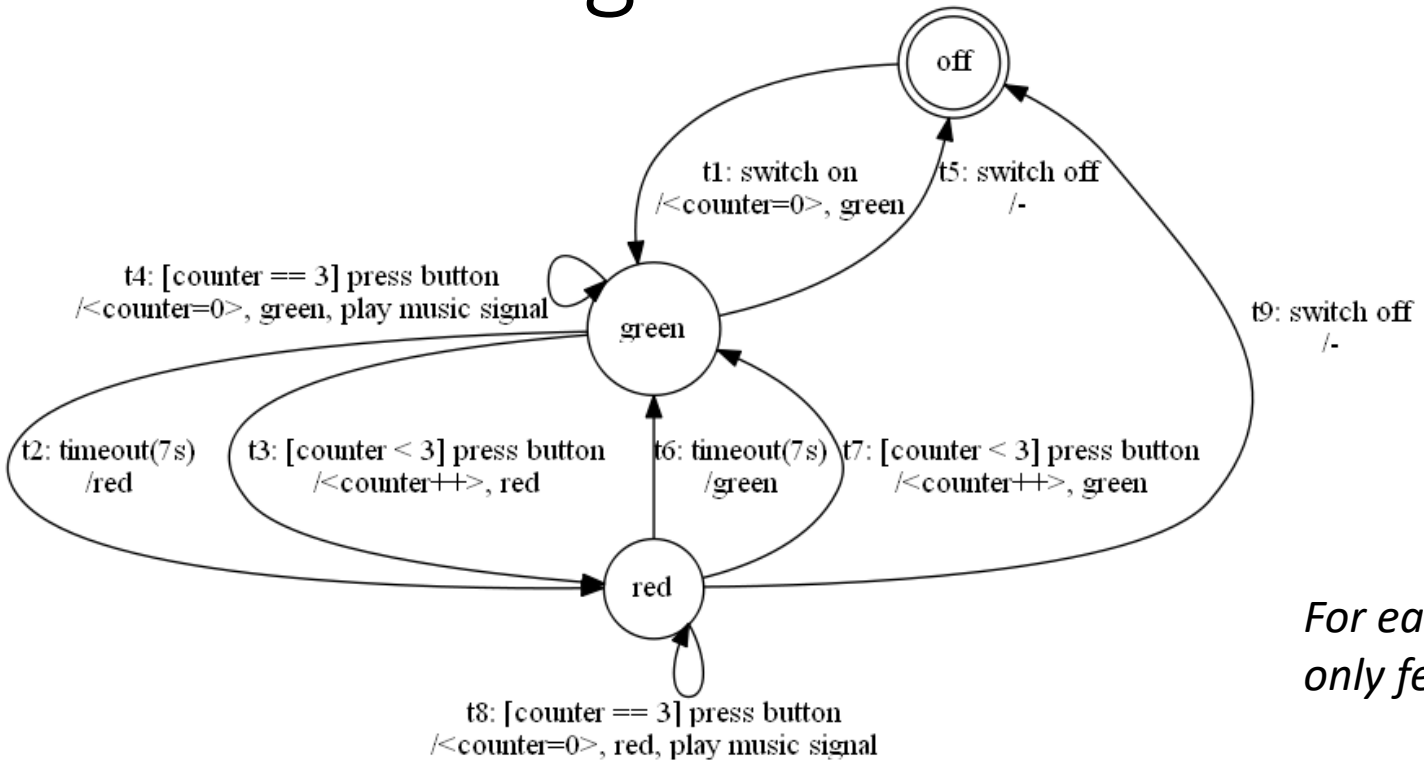
For each coverage multiple solutions exist, only few of them are listed

1. All state coverage: s_{off} , s_{green} , s_{red} states → t1-t3 or t1-t2 → test set is *switch on – button* or *switch on – timeout*

2. All (inputs) events coverage: *switch on*, *switch off*, *press button*, *timeout* events → t1-t3-t6-t5 or t1-t2-t7-t5 → test set is *switch on – button – timeout – switch off* or *switch on –timeout – button – switch off*

3. All (outputs) actions coverage: -, *green*, *red*, *play music signal* outputs → t1-t3-t7-t3-t8-t9 → test set is *switch on – button – button – button – button – switch off*

Test coverage criteria – solution part 1.2

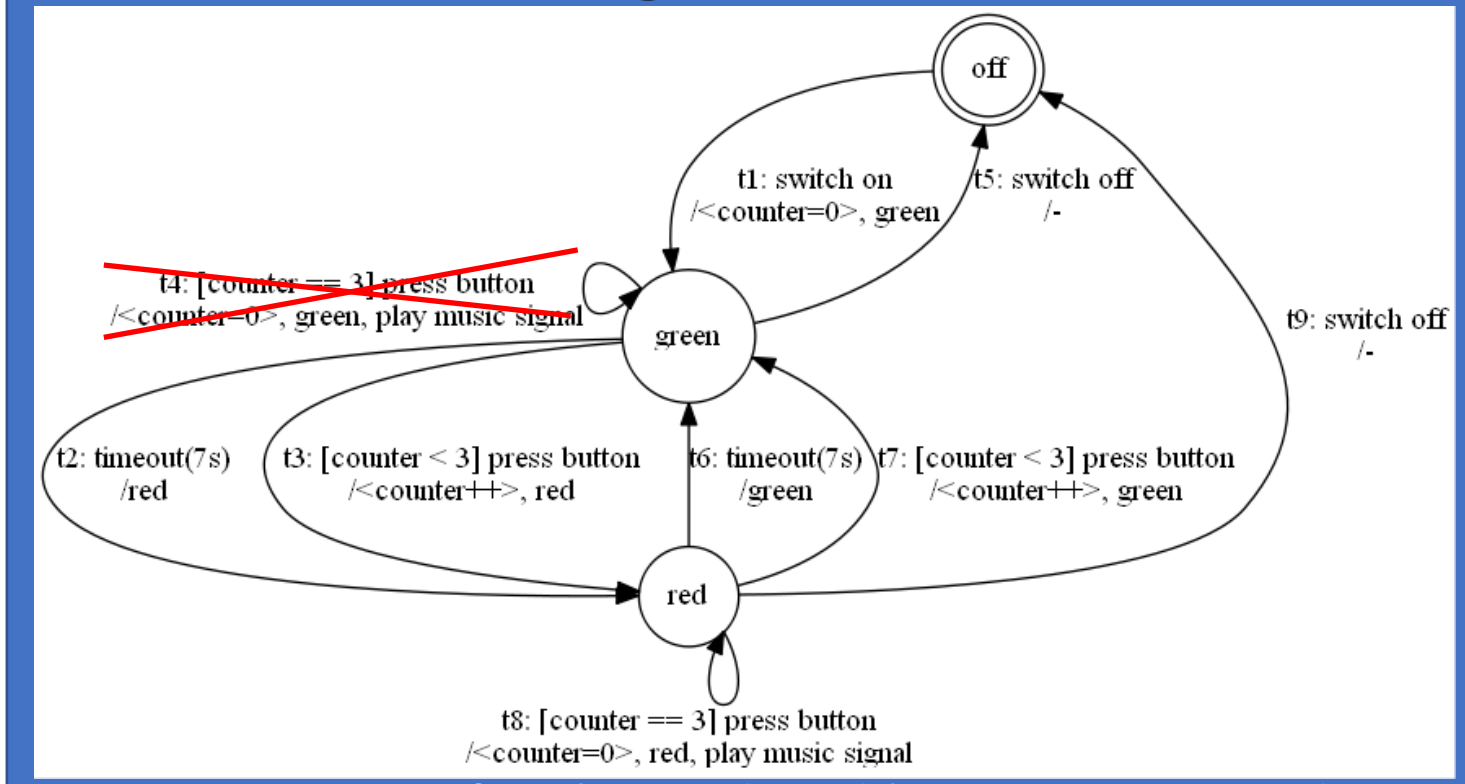


For each coverage multiple solutions exist, only few of them are listed

4. All transitions coverage: *t1-t9* transitions → t1-t3-t7-t3-t8-t9-t1-t2-t6-t3-t7-t3-t6-t4-t5 → test set is *switch on – button – button – button – button – switch off – switch on – timeout – timeout – button – button – button – timeout – button – switch off*

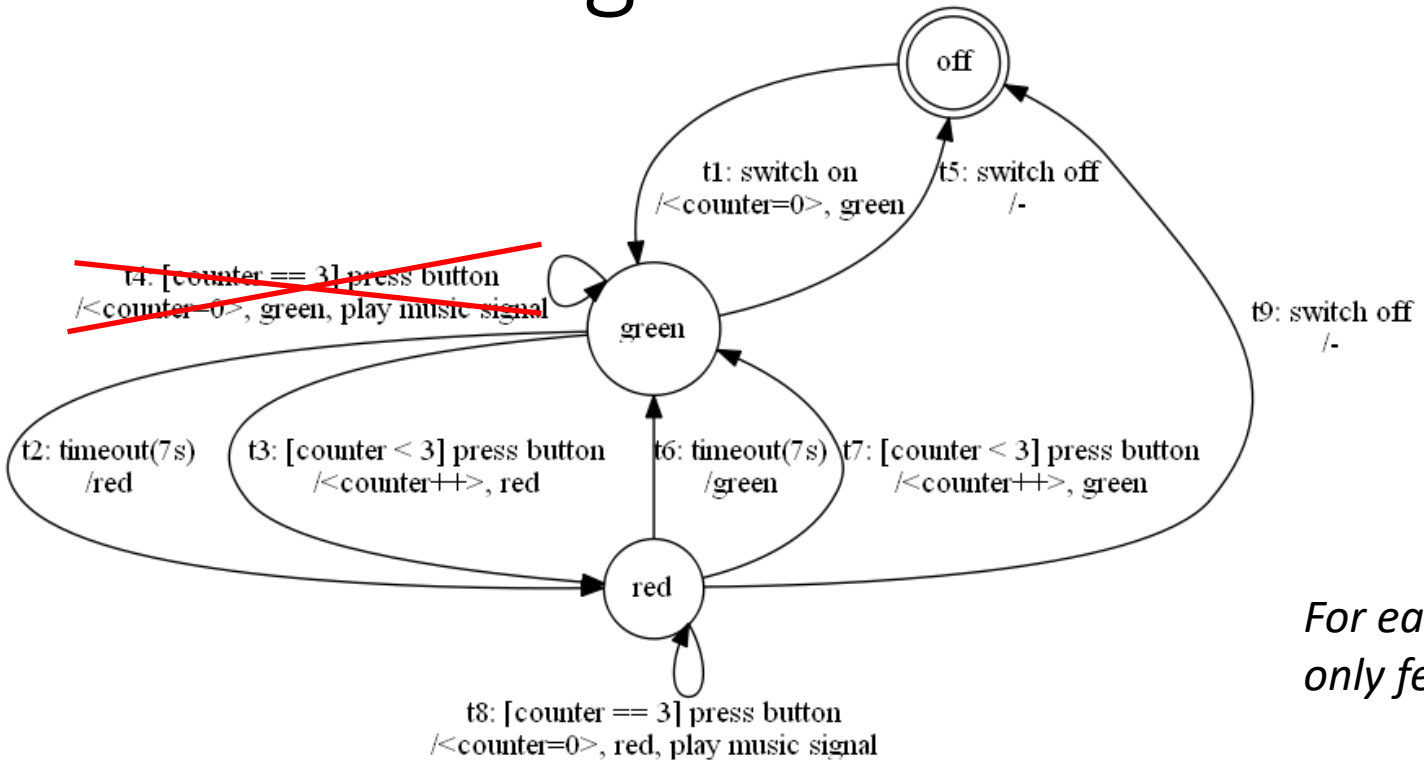
5. All transition-pairs coverage: t1-t2, t1-t3, t1-t4, t1-t5, t2-t6, t2-t7, t2-t8, t2-t9, t3-t6, t3-t7, t3-t8, t3-t9, t4-t2, t4-t3, t4-t4, t4-t5, t5-t1, t6-t2, t6-t3, t6-t4, t6-t5, t7-t2, t7-t3, t7-t4, t7-t5, t8-t6, t8-t7, t8-t8, t8-t9, t9-t1 transition-pairs are adjacent, *but the pairs highlighted with grey background can not be tested due the guard conditions and actions on* `counter`

Test coverage criteria – exercise part 2

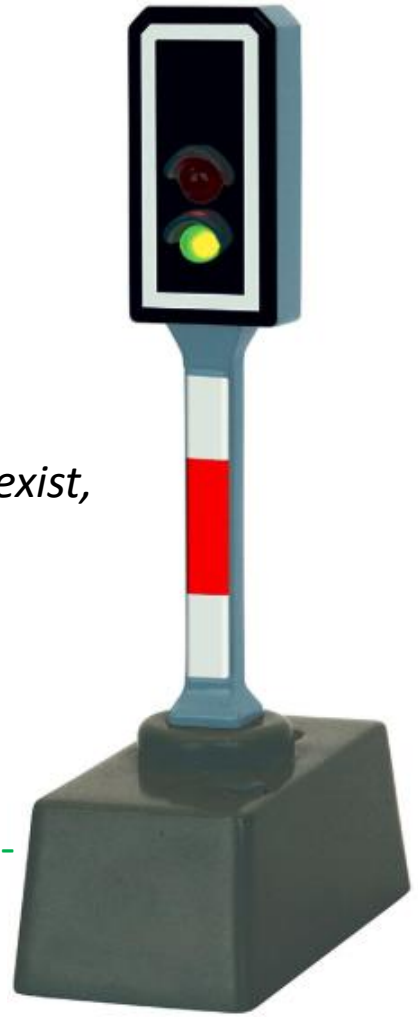


Create test suite for the updated battery-operated signal FSM *(for the sake of simplicity we do not consider t4 loop transition, otherwise the test set would be to big and hard to follow)* with all transition-state coverage

Test coverage criteria – solution part 2



For each coverage multiple solutions exist, only few of them are listed

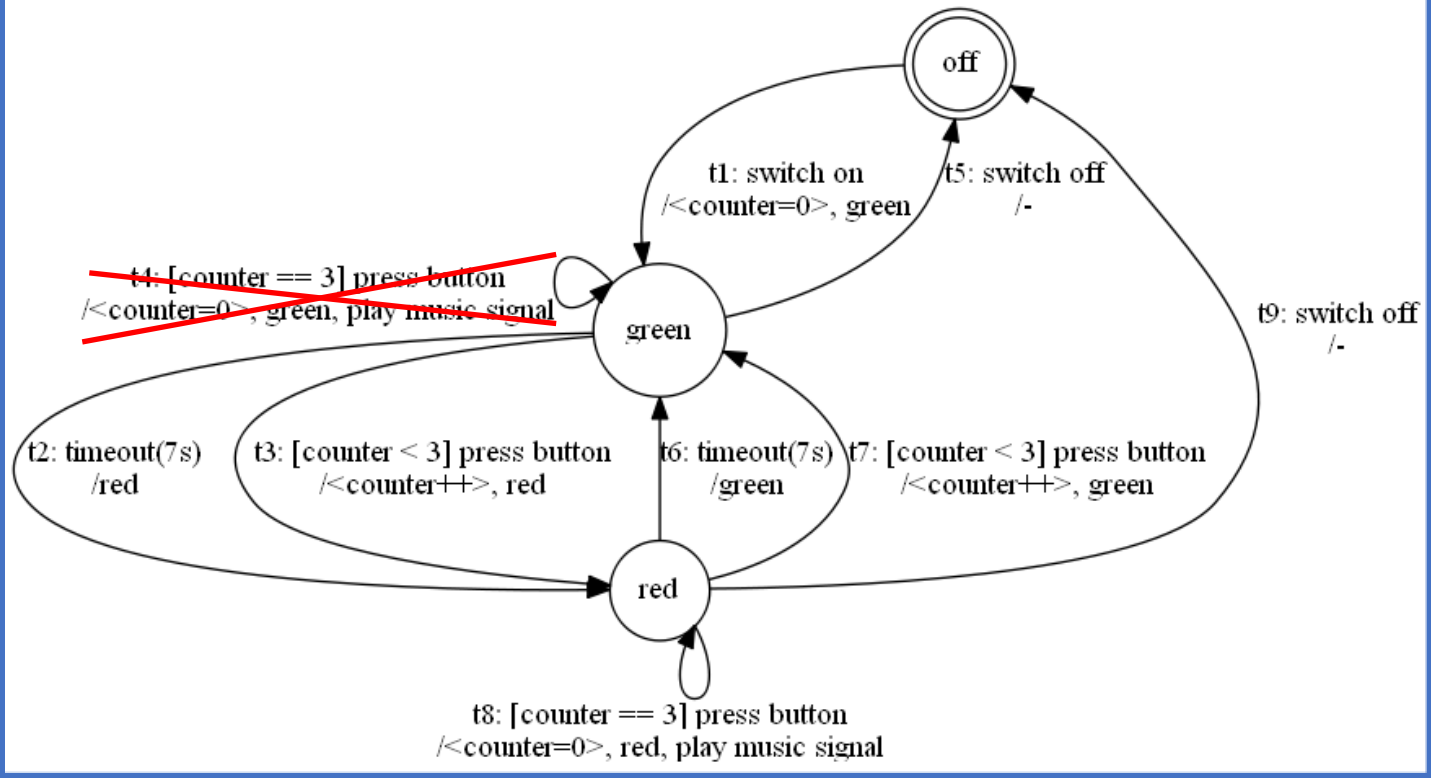


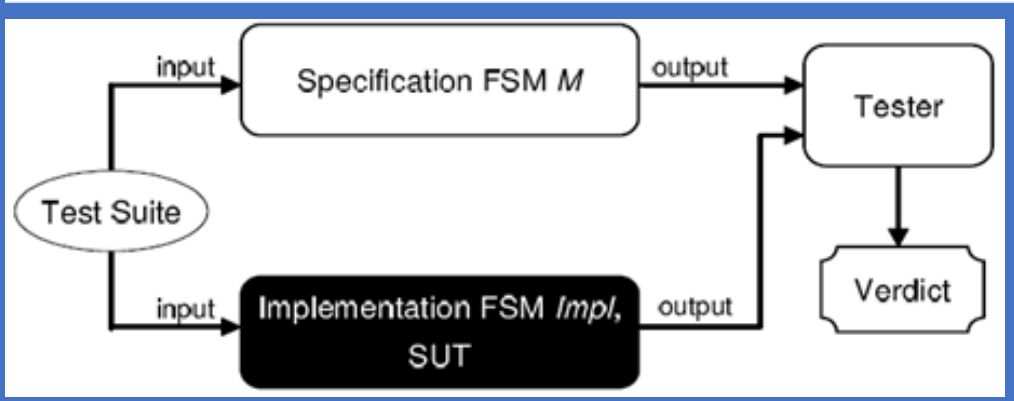
6. All-Transition-State (ATS) coverage:

The state transition graph is strongly connected, moreover, by deleting any transition except *t1*, it remains strongly connected.

- The sequence s_{off}-t1-s_{green}-t5-s_{off}-t1-s_{green}-t2-s_{red}-t6-s_{green}-t3-s_{red}-t7-s_{green}-t3-s_{red}-t8-s_{red}-t7-s_{green}-t3-s_{red}-t9-s_{off}-t1-s_{green}-t2-s_{red} covers the *(t, s)* paths for all (t1, t2, t3, t5, t6, t7, t8, t9) transition *t* and all state *s* (s_{off}, s_{green}, s_{red})
- The sequences s_{off}-t1-s_{green}-t3-s_{red}-t7-s_{green}-t3-s_{red}-t9-s_{off} and s_{off}-t1-s_{green}-t2-s_{red}-t6-s_{green}-t5-s_{off} cover the (*!t*, *s*) paths

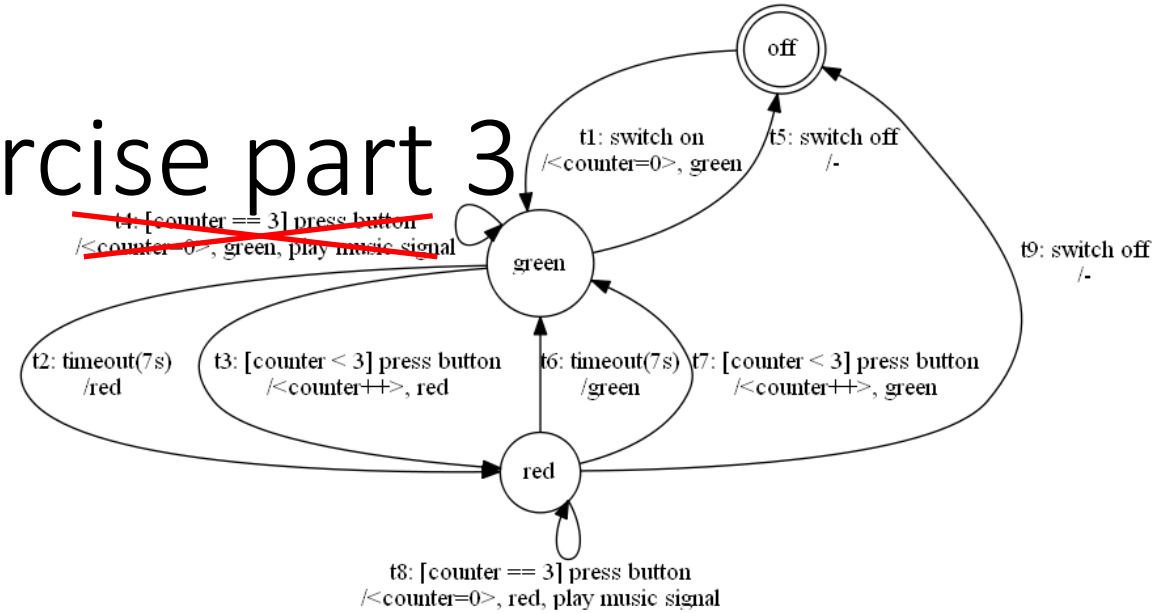
Test coverage criteria – exercise part 3





- Write down the steps of the previous first test sequence with the expected results.
- What will be the difference, if the implementation contains a wrong guarding condition [5 instead of 3]?

Test coverage criteria – exercise part 3



Problem in implementation: 5 in guard condition instead of 3

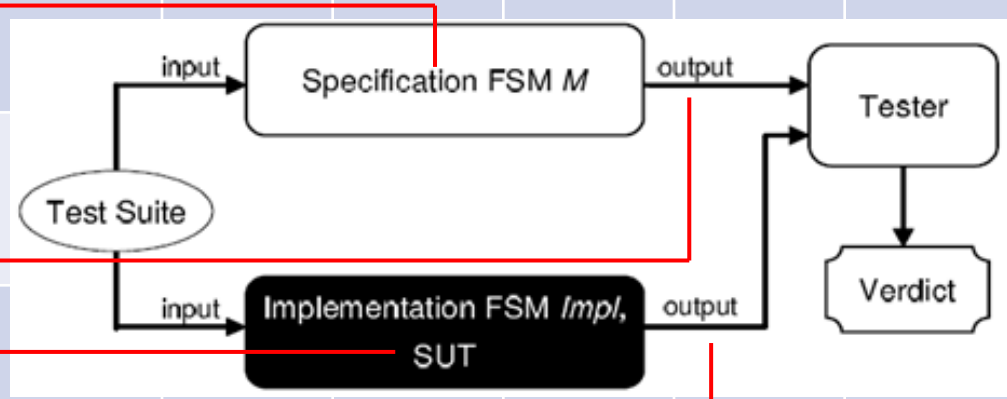
Test sequence (1st sequence of all transition-state coverage test set):

s_{off}-t1-s_{green}-t5-s_{off}-t1-s_{green}-t2-s_{red}-t6-s_{green}-t3-s_{red}-t7-s_{green}-t3-s_{red}-t8-s_{red}-t7-s_{green}-t3-s_{red}-t9-s_{off}-t1-s_{green}-t2-s_{red}

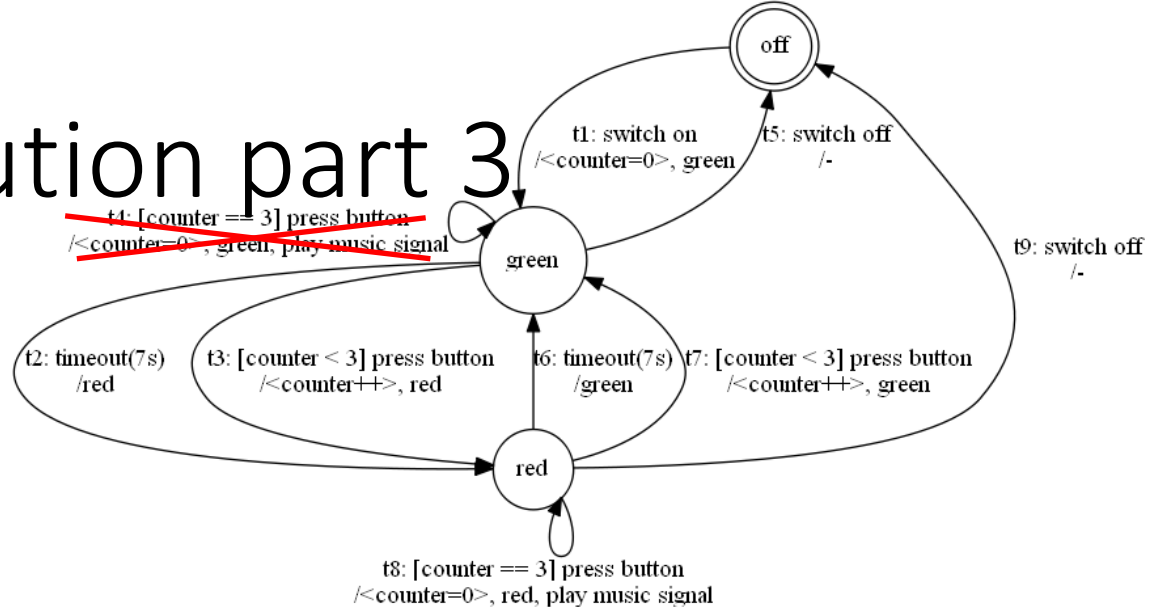
	Step1	Step2	Step3	Step4	Step5	Step6	Step7	Step8	Step9	Step 10	Step 11	Step 12	Step 13	Step 14
State														
Event (Input)														
Next State														
Expected result														
Observed result														

Specification

Implementation



Test coverage criteria – solution part 3



Problem in implementation: 5 in guard condition instead of 3

Test sequence (1st sequence of all transition-state coverage test set):

s_{off}-t1-s_{green}-t5-s_{off}-t1-s_{green}-t2-s_{red}-t6-s_{green}-t3-s_{red}-t7-s_{green}-t3-s_{red}-t8-s_{red}-t7-s_{green}-t3-s_{red}-t9-s_{off}-t1-s_{green}-t2-s_{red}

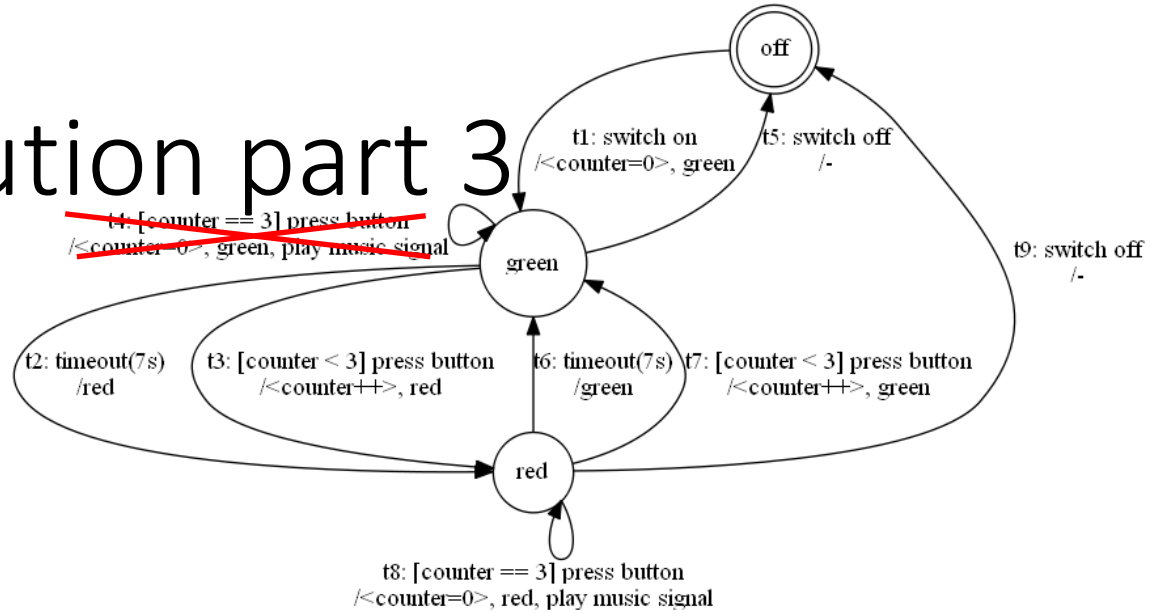
	Step1	Step2	Step3	Step4	Step5	Step6	Step7	Step8	Step9	Step 10	Step 11	Step 12	Step 13	Step 14
State	S_{off}	S_{green}	S_{off}	S_{green}	S_{red}	S_{green}	S_{red}	S_{green}	S_{red}	S_{red}	S_{green}	S_{red}	S_{off}	S_{green}
Event (Input)	Switch on	Switch off	Switch on	Timeout	Timeout	[0<3] Press button	[1<3] Press button	[2<3] Press button	[3=3] Press button	[0<3] Press button	[1<3] Press button	Switch off	Switch on	Timeout
Next State	S_{green}	S_{off}	S_{green}	S_{red}	S_{green}	S_{red}	S_{green}	S_{red}	S_{red}	S_{green}	S_{red}	S_{off}	S_{green}	S_{red}
Expected result	**Green**	**-**	**Green**	**Red**	**Green**	**Red**	**Green**	**Red**	**Red, Music s.**	**Green**	**Red**	**-**	**Green**	**Red**
Observed result														

Test coverage criteria – solution part 3

Problem in implementation: 5 in guard condition instead of 3

Test sequence (1st sequence of all transition-state coverage test set):

s_{off}-t1-s_{green}-t5-s_{off}-t1-s_{green}-t2-s_{red}-t6-s_{green}-t3-s_{red}-t7-s_{green}-t3-s_{red}-t8-s_{red}-t7-s_{green}-t3-s_{red}-t9-s_{off}-t1-s_{green}-t2-s_{red}



	Step1	Step2	Step3	Step4	Step5	Step6	Step7	Step8	Step9	Step 10	Step 11	Step 12	Step 13	Step 14
State	S_{off}	S_{green}	S_{off}	S_{green}	S_{red}	S_{green}	S_{red}	S_{green}	S_{red}	S_{red}	S_{green}	S_{red}	S_{off}	S_{green}
Event (Input)	Switch on	Switch off	Switch on	Timeout	Timeout	[0<3] Press button	[1<3] Press button	[2<3] Press button	[3=3] Press button	[0<3] Press button	[1<3] Press button	Switch off	Switch on	Timeout
Next State	Green	Off	Green	Red	Green	Red	Green	Red	Red	Green	Red	Off	Green	Red
Expected result	**Green**	**-**	**Green**	**Red**	**Green**	**Red**	**Green**	**Red**	**Red, Music s.**	**Green**	**Red**	**-**	**Green**	**Red**
Observed result	**Green**	**-**	**Green**	**Red**	**Green**	**Red**	**Green**	**Red**	**Green**	**Red**	**Red, Music s.**	**-**	**Green**	**Red**

In wrong state (green ↔ red) due implementation error, but we can only observe its' output

Test coverage criteria

- The previous criteria were structural criteria, but other groups of criteria exist:
  - Functional criteria:
    - Go through scenarios, use cases or user profiles
  - Stochastic criteria:
    - Given transitions from given states selected with a given probability
    - The test coverage should reflect on these probabilities →weighted or unweighted random walks
    - This criteria can be used for risk-based testing (more weight to more important features…)

A good overview of different test coverages with step-by-step examples can be found in the following book:
I. Forgacs, A. Kovacs: Practical Test Design: Selection of traditional and automated functional tests. BCS. 2019.
/ Chapter 6 State transition testing

Test coverage – fault coverage in theory

- Generally speaking, testing can only „be used to show the presence of bugs, but never to show their absence” (Dijkstra)

Good news everyone!

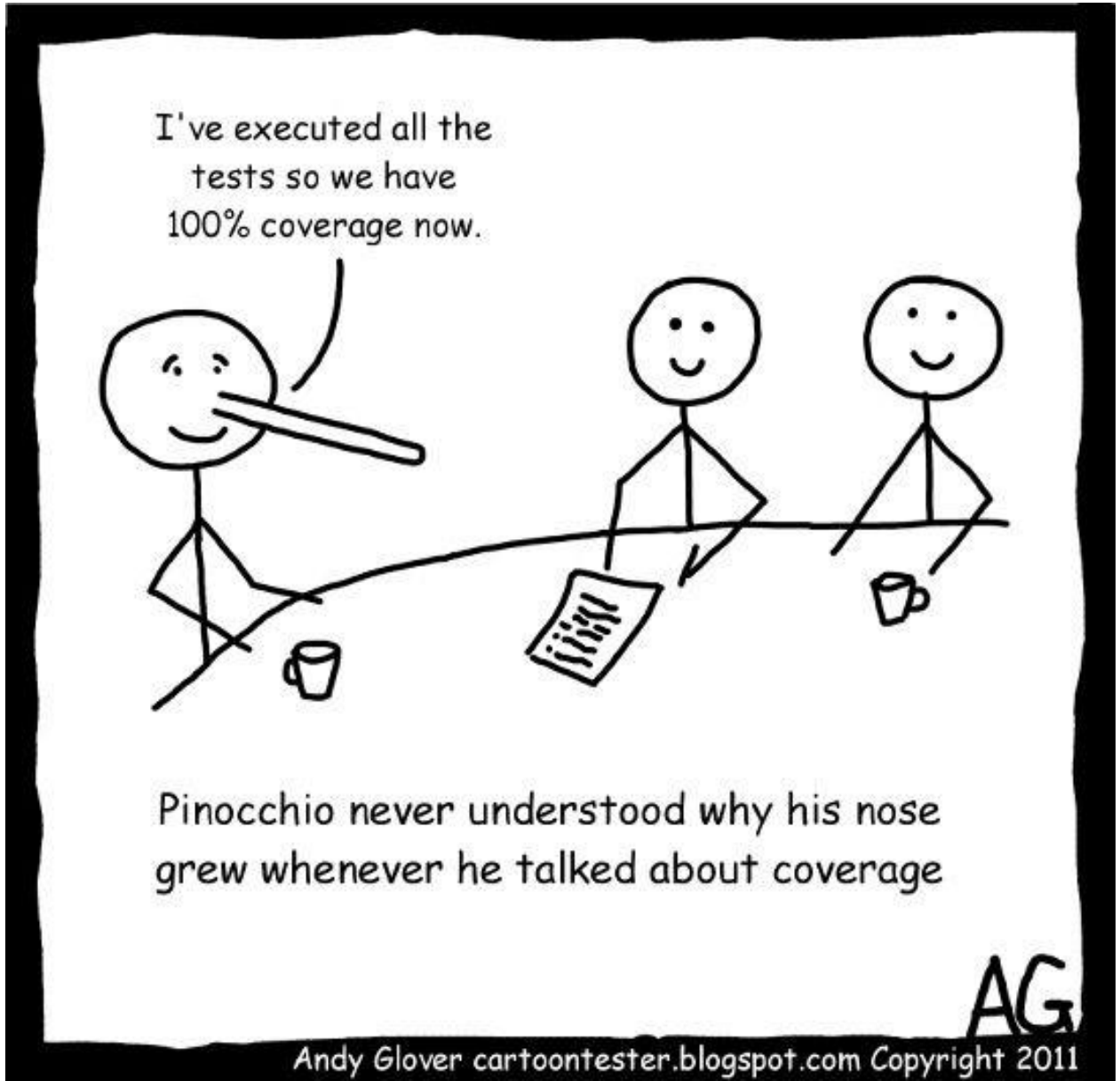
- It is possible to create algorithms that **show the absence** of faults*

  * if some assumptions** hold

  ** that rarely exist in practice

Yehaw!

Test coverage – fault coverage in practice



FSM fault models

- Define the types of faults the test designer is interested in

- Types
  1. Output Fault:
     - FSM *Impl* produces an output that is different from the one that is specified in machine *M*
  2. Transfer Fault
     - FSM *Impl* goes into a state that differs from the state specified in machine *M*
  3. Missing State / Extra State
     - Some type of faults can only be modeled by additional states alongside with transfer faults (example: sink state) or some part(s) of the specification is completely missing in *Impl*
  4. Additional Transition / Missing Transition

M▶T▶R See change injection functionality of Model ⟫ Test ⟫ Relax

Assumptions about FSMs

For undefined state-input pairs typically loop transitons are used (or sometimes a sink error state) without any output

1. FSM *M* (specification machine) is completely specified
   - There is at least one transition for all state-input pairs
2. FSM *M* is deterministic
   - For each state-input pair there exists at most one transition
3. FSM *M* is strongly connected
   - For each pair of states $<s_j, s_l>$, there exists an input sequence which takes M from s_j to s_l
4. FSM *M* is reduced
   - No two states are equivalent: for each pair of states there exists a sequence that produces different output for these states
5. FSM *Impl* does not change during the test experiment and has the same input *I* and output *O* alphabet as *M*.

M▸T▸R See partially specified to completely specified conversion functionality of Model ⟫ Test ⟫ Relax

M▸T▸R See state minimization functionality of Model ⟫ Test ⟫ Relax

Assumptions about FSMs

6. Upper bound for the number of states in FSM *Impl*
   a) *Impl* has no more states than *M*
      → Fault model: (1) output faults, (2) transfer faults
   b) Number of states in *Impl* is limited

Optional assumptions:

7. *reset* message
   - takes FSM back to initial state
8. *status* message
   - reports current state (and stays in that state!), rarely exists
9. *set* message
   - takes FSM to a given state from initial state, rarely exists

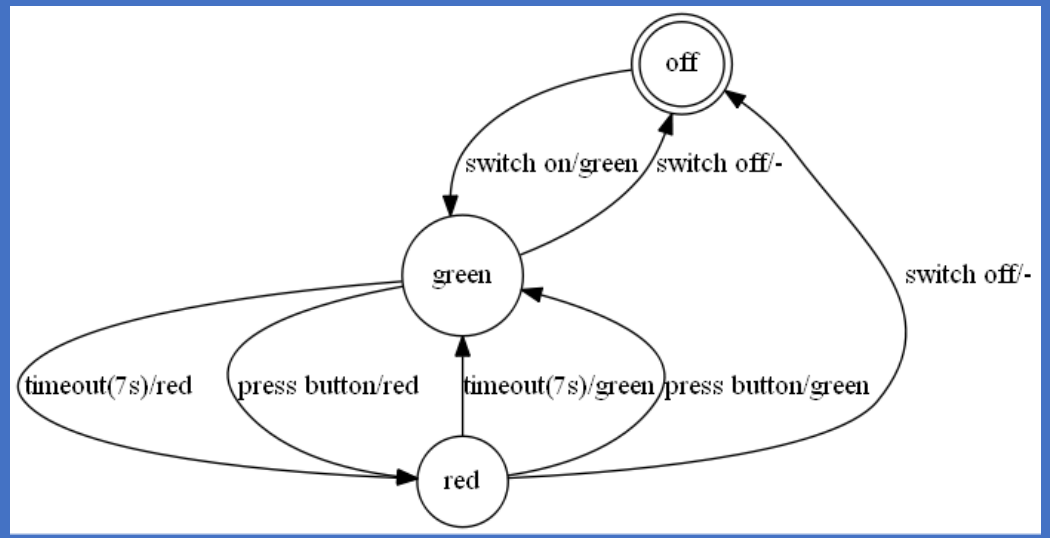
Note that 7.-9. messages can be both reliable or unreliable

- Reliable if it is guaranteed to work properly in any implementation machine *Impl* of *M*

M▸T▸R See testing unreliable resets functionality of Model ⟫ Test ⟫ Relax

Assumptions about FSMs - exercise

Battery-Operated Signal

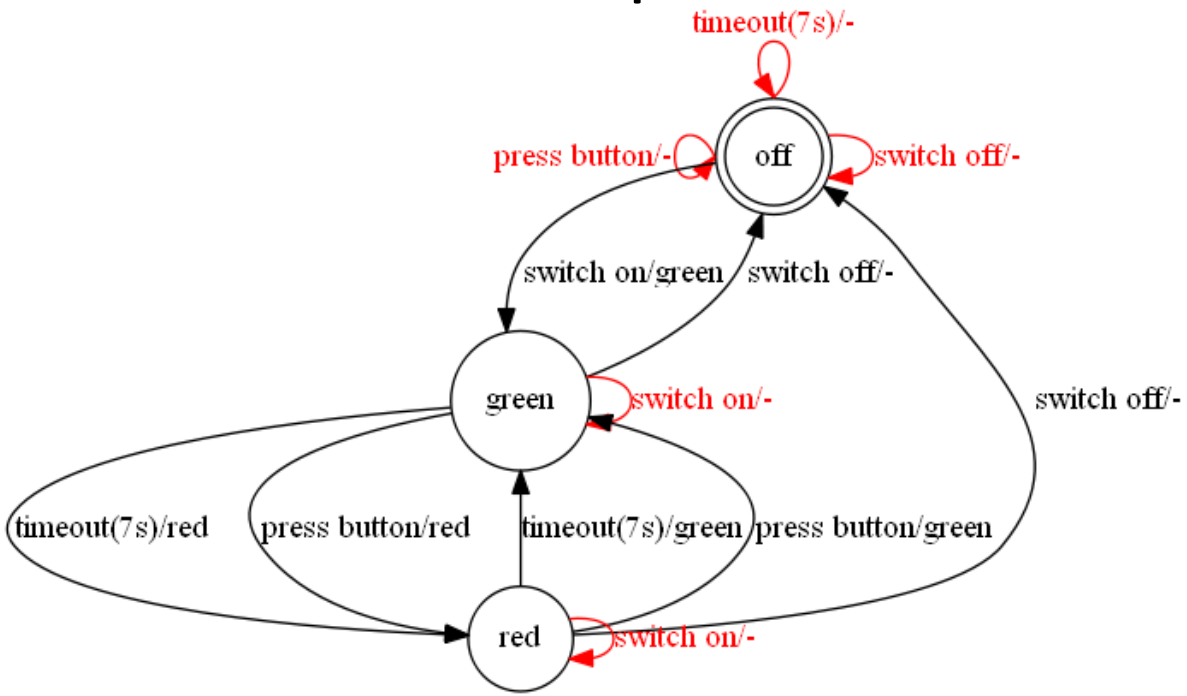


	switch on	switch off	press button	wait for 7 secs
off	s_{green} /green	-	-	-
green	-	s_{off} /-	s_{red} /red	s_{red} /red
red	-	s_{off} /-	s_{green} /green	s_{green} /green

Consider the specification FSM *M* above and answer the following questions:

- Is *M* completely specified? If not, how it can be transformed to be completely specified?
- Is *M* deterministic?
- Is *M* strongly connected?
- Is *M* reduced?
- Does *M* have reset messages? If not, how it can be created?
- Does *M* have status messages? If not, how it can be created?
- Does *M* have set messages? If not, how it can be created?

Assumptions about FSMs - solution



	switch on	switch off	press button	wait for 7 secs
off	s_{green} /green	s_{off} /-	s_{off} /-	s_{off} /-
green	s_{green} /-	s_{off} /-	s_{red} /red	s_{red} /red
red	s_{red} /-	s_{off} /-	s_{green} /green	s_{green} /green

- Is *M* completely specified? No, see modifications above highlighted with red how it can be converted to be completely specified.
- Is *M* deterministic? Yes.
- Is *M* strongly connected? Yes.
- Is *M* reduced? Yes, as no two states are equivalent. *(Completely specified version can be separated with "press button")*
- Does *M* have reset messages? Yes, see input *switch off*.
- Does *M* have status messages? No. Input *press button* shows the state, but it does not stay here.
- Does *M* have set messages? No. Instead of the current light signal, consider one which does not have an on/off switch but has 3 buttons: off, green and red which set the semaphore to the desired state.

M▶T▶R *See log model information feature of Model ⟫ Test ⟫ Relax*

FSM-based test generation algorithms: classifications 3/1

Structure:

- Algorithms with one stage
  - Check all the transitions of FSM
- Algorithms with two stages
  1. State identification
     - Check for each state of the specification if it exists in the implementation as well
  2. Checking remaining transitions
     - Check if output and next state conforms to the specification

FSM-based test generation algorithms: classifications 3/2

Adaptiveness:

- Adaptive
  - The observed outputs can be looped back to the test cases → decision trees
- Non-adaptive
  - Preset test cases are applied regardless of the observed output

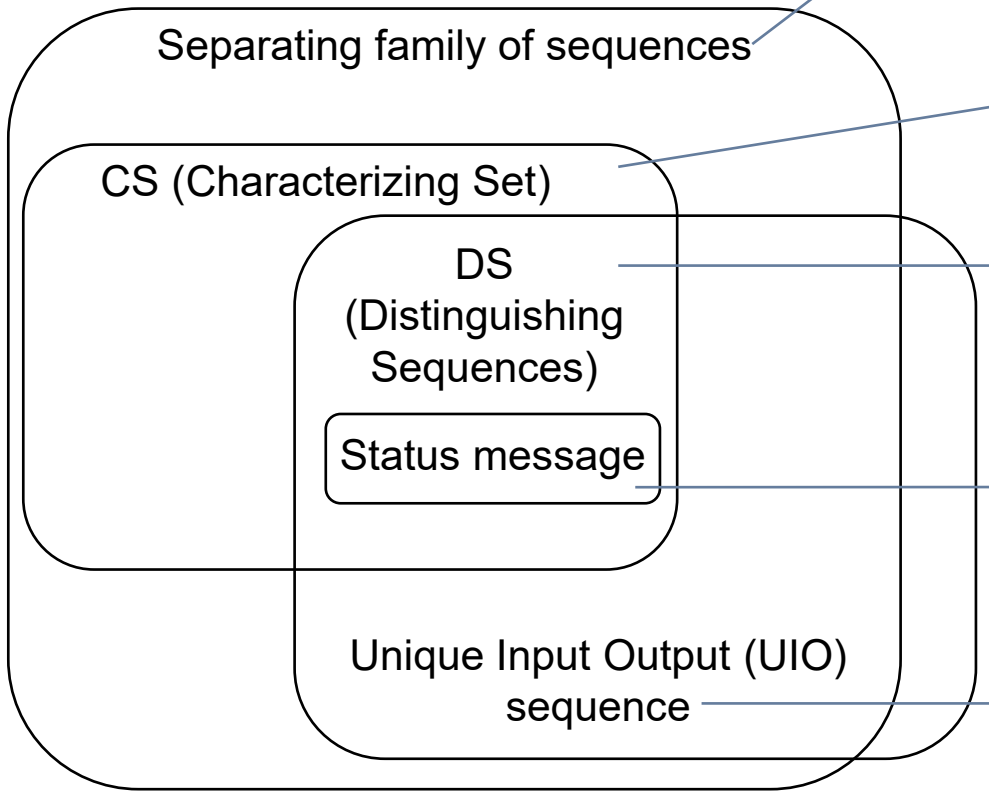
Note that the test sequences may be shorter in the adaptive case than in the non-adaptive case

Note that the feedback may takes time due to message delays…etc. and non-adaptive test cases may be more suitable for regression testing

FSM-based test generation algorithms: classifications 3/3

Algorithms with two stages:

› Different approaches for state identification:



Most general solution: Structured set of sets: $Z=\{Z_0...Z_{n-1}\}$, *where* $Z_i=\{z_{i0}...z_{i\,n-1}\}$, where z_{ij} separates states s_i and s_j

Set of input sequences:
For every state pair at least one input sequence gives a different output

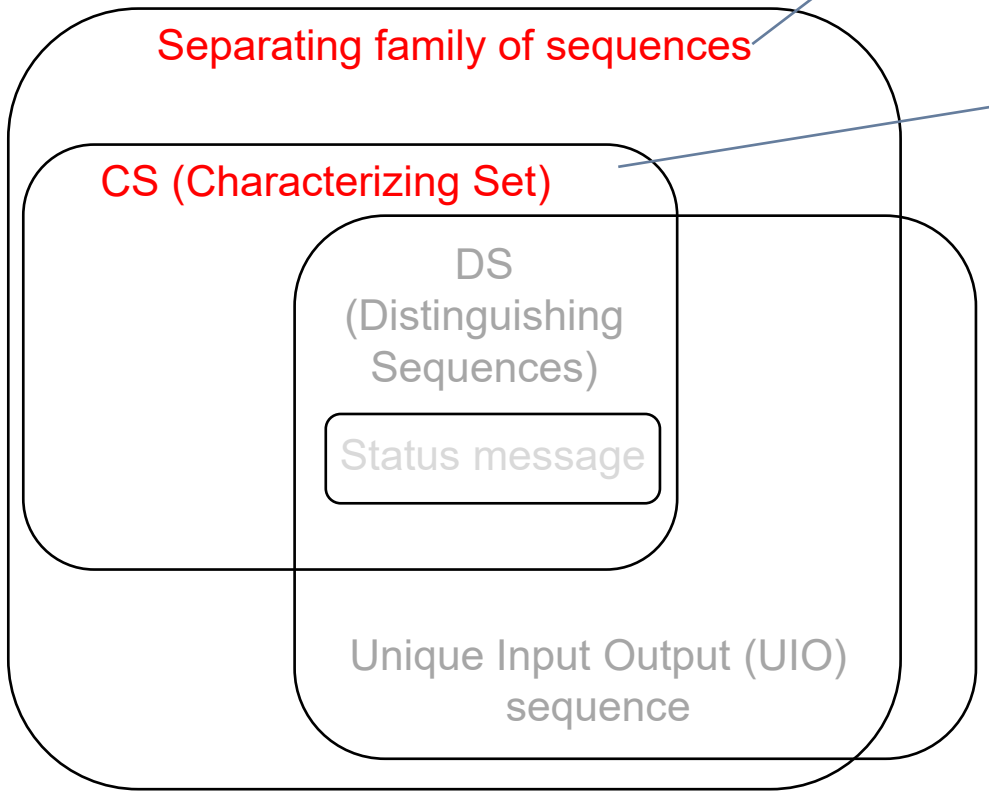
An input sequence that produces different output for each state
(few spec FSMs has DS)

Reports current state using a special message (rarely exists)

For each state s_l an UIO sequence u_l verifies the state (not all FSMs have UIO sequences for all states)

FSM-based test generation algorithms: classifications 3/3

State identification:



Most general solution: Structured set of sets:
$Z=\{Z_0 \ldots Z_{n-1}\}$, *where* $Z_i=\{z_{i0} \ldots z_{i\,n-1}\}$, where z_{ij} separates states s_i and s_j

Set of input sequences:
For every state pair at least one input sequence gives a different output

The CS is a special case of the separating family of sequences: the sets Z_i are identical.

FSM-based test generation algorithms – An overview

Method	Structure	Required assumptions	State Identification	Complexity of test generation	Length of test	Comments
S (naive approach)	1 stage	1-9	status	$O(p \cdot n)$	$O(p \cdot n)$	Applicable only if both set and status messages exist.
D	2 stages	1-6, DS	DS	PSPACE	Exp.	Applicable only if DS exists (it rarely exists).
adapt. D	2 stages	1-6, DS	adaptive DS	no preset test	$O(p \cdot n^3)$	Adaptive version of D.
TT v1	1 stage	2, 3, 5, 6	-	$O(n^3 + m)$	$O(m)$	Only guarantees to find output faults.
TT v2	1 stage	2-6, 8	status	$O(n^3 + m)$	$O(m)$	Applicable only if status message exists.
UIO	1 stage	1-7, UIO seq. for $\forall s_k \in S$	UIO seq.	PSPACE	Exp.	Does not guarantee to find every fault. Applicable only if UIO sequence exists for each state. Similar to W.
UIOv	2 (3) stages	1-7, UIO seq. for $\forall s_k \in S$	UIO seq.	PSPACE	Exp.	Applicable only if UIO sequence exists for each state. Similar to Wp.
W	1 stage	1-7	CS	$O(p \cdot n^3)$	$O(p \cdot n^3)$	-
Wp	2 stages	1-7	CS	$O(p \cdot n^3)$	$O(p \cdot n^3)$	An improved version of W.
HIS /HSI	2 stages	2-7	sep. family of seq.	$O(p \cdot n^3)$	$O(p \cdot n^3)$	Similar to Wp.
H	2 stages	2-7	adaptive selection from sep. family of seq.	no preset test	$O(p \cdot n^3)$	An improved version of HSI

Assumptions:

1. Completely specified
2. Deterministic
3. Strongly connected
4. Minimized
5. *Impl* does not change during test
6. Upper bound for states in *Impl*
7. *reset* msg
8. *status* msg
9. *set* msg

legends

- n: number of states
- p: number of input symbols
- m: number of transitions

Abbreviations

CS: Characterizing Set
DS: Distinguishing Sequence
UIO: Unique Input Output
HSI: Harmonized State Identifiers
TT: Transition Tour
UIO: Unique Input Output

FSM-based test generation algorithms: The most important ones

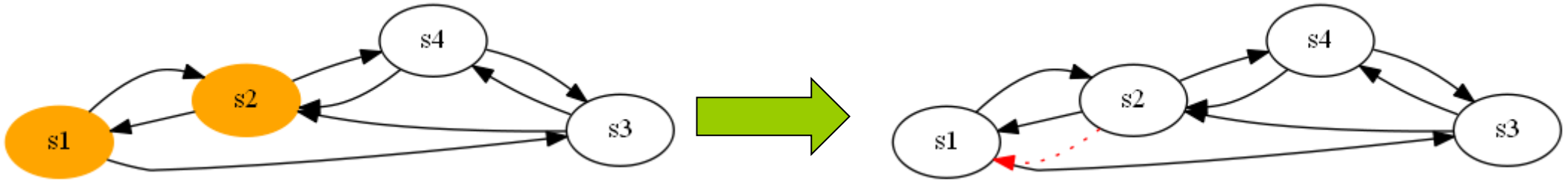
- TT (Transition Tour)-method
  - For deterministic FSMs
  - Unstructured: a test sequence
  - 100% state, event, action and transition coverage
  - Guarantees to find all output faults

- W / Wp / HSI/HIS*-method triple:
  - For completely (W/Wp)/partially specified (HSI), deterministic FSMs with reliable reset capability
  - Structured test suite
    - Preamble + tested transition + Postamble
  - Provides a checking sequence:
    - Guarantees to find **all** output and transfer faults
    - Besides that it has 100% state, event, action and transition coverage

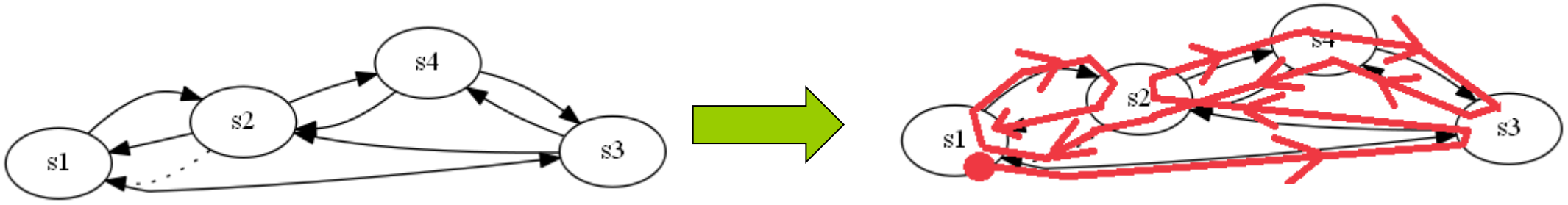
* *Sometimes referred to as HIS, sometimes as HSI (*Harmonized State Identifiers*), but the same method*

FSM-based test generation algorithms:
TT-method: an approach with one stage

- Based on the Directed Chinese Postman Problem
  - Solutions typically consist of two parts:
    1. Augment the original graph to make it Eulerian



    2. Find an Euler circuit over this augmented graph



M▸T▸R The TT-method is implemented in Model 》 Test 》 Relax framework

FSM-based test generation algorithms:
TT-method: an approach with one stage

- Based on the Directed Chinese Postman Problem
  - Solutions typically consist of two parts:
    1. Augment the original graph to make it Eulerian

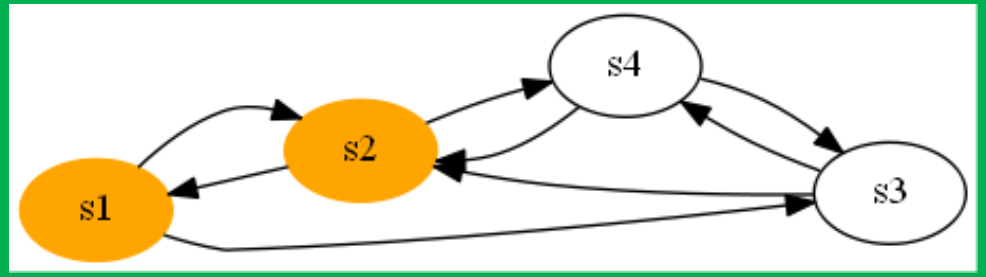
This problem can be reduced to a minimum cost network flow problem
Complexity: $O(n^3 + m)$

n: number of states
m: number of transitions

How?

Using the improved version of Edmond's maximum weighted perfect matching algorithm:

a. Identify the unbalanced nodes of the graph,
   i.e. all s_x nodes of the graph where $deg^+(s_x) \neq deg^-(s_x)$



$deg^+(s_x)$: the number of directed edges originating from the node
$deg^-(s_x)$: number of directed edges that lead to node

FSM-based test generation algorithms:
TT-method: an approach with one stage

- Based on the Directed Chinese Postman Problem
  - Solutions typically consist of two parts:
    1. Augment the original graph to make it Eulerian

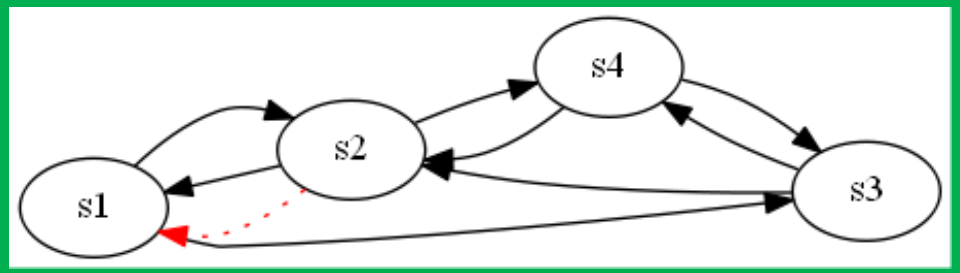
This problem can be reduced to a minimum cost network flow problem
Complexity: $O(n^3 + m)$

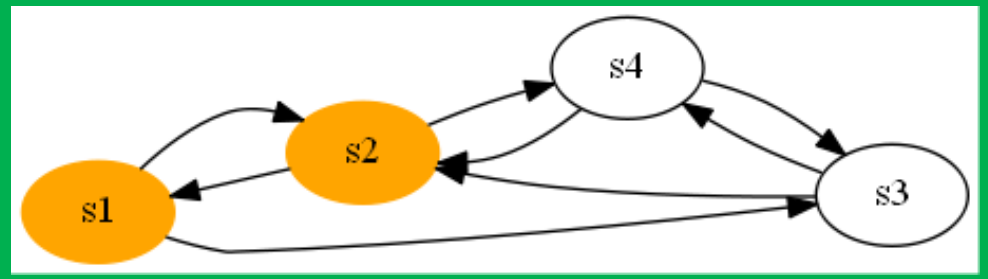
n: number of states
m: number of transitions

How?

Using the improved version of Edmond's maximum weighted perfect matching algorithm:

a. Identify the unbalanced nodes of the graph,
   i.e. all s_x nodes of the graph where $deg^+(s_x) \neq deg^-(s_x)$
b. Duplicate some edges of s_x so that for all s_x: $deg^+(s_x) = deg^-(s_x)$





$deg^+(s_x)$: the number of directed edges originating from the node
$deg^-(s_x)$: number of directed edges that lead to node

FSM-based test generation algorithms:
TT-method: an approach with one stage

- Based on the Directed Chinese Postman Problem
  - Solutions typically consist of two parts:
    1. Augment the original graph to make it Eulerian

This problem can be reduced to a minimum cost network flow problem
Complexity: $O(n^3 + m)$

n: number of states
m: number of transitions

    2. Find an Euler circuit over this augmented graph

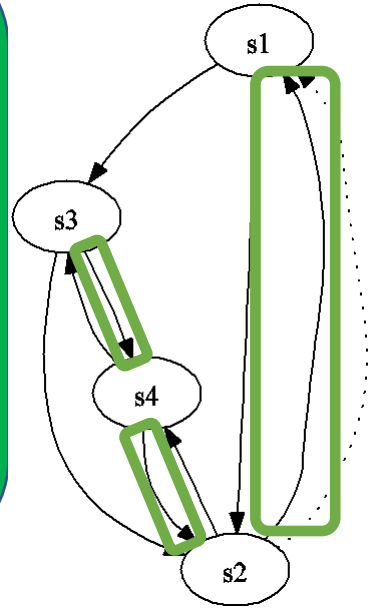
This problem can be reduced to create a spanning tree, which edges are directed towards the initial state of the FSM
Complexity: O(m)

FSM-based test generation algorithms:
TT-method: an approach with one stage

How?

(using next-node representation form)

• **STEP 1:** Find a spanning tree *ST* of Eulerian graph G_E, whose edges are directed toward the root s_r of *ST*.



Find an Euler circuit over this augmented graph

This problem can be reduced to create a spanning tree, which edges are directed towards the initial state of the FSM
Complexity: O(m)

In the *next-node representation*, order of edges is defined for each node and the node is left after the l^{th} visit across the edge labeled with *l*

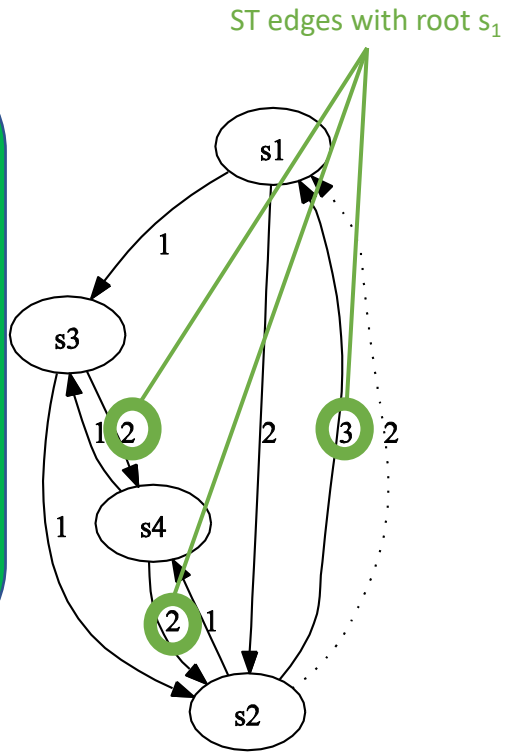
FSM-based test generation algorithms:
TT-method: an approach with one stage

How?

(using next-node representation form)

• **STEP 1:** Find a spanning tree *ST* of Eulerian graph G_E, whose edges are directed toward the root s_r of *ST*.

• **STEP 2:** For any node s_k (except $k = r$) specify an arbitrary order for the edges directed away from s_k with the restriction that the edge of the spanning tree *ST* is the last in the ordering. For the root s_r, specify an arbitrary order for the edges.



Find an Euler circuit over this augmented graph

This problem can be reduced to create a spanning tree, which edges are directed towards the initial state of the FSM
Complexity: O(m)

In the *next-node representation*, order of edges is defined for each node and the node is left after the l^{th} visit across the edge labeled with *l*

FSM-based test generation algorithms:
TT-method: an approach with one stage

How?

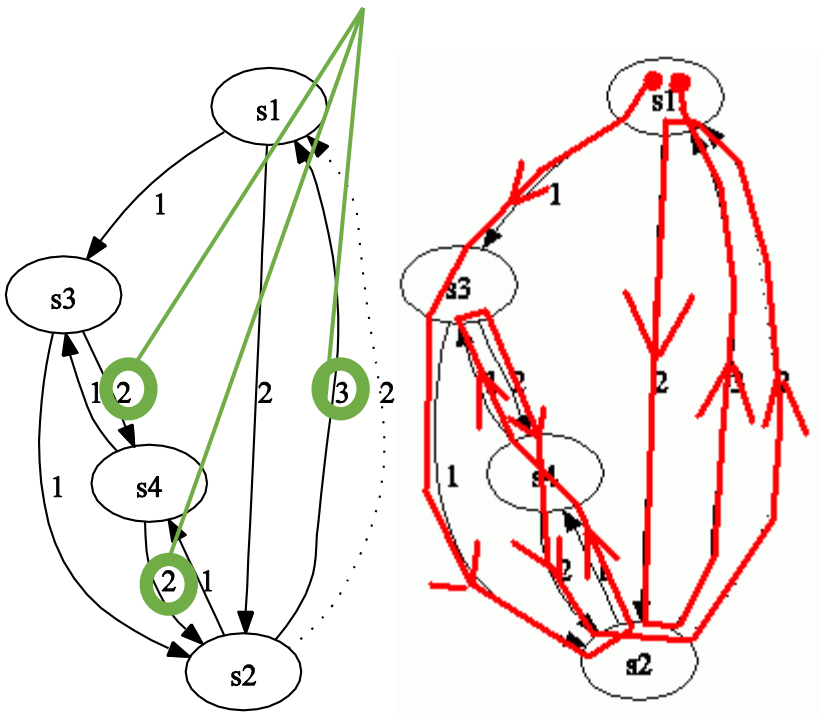
(using next-node representation form)

• **STEP 1:** Find a spanning tree *ST* of Eulerian graph G_E, whose edges are directed toward the root s_r of *ST*.

• **STEP 2:** For any node s_k (except $k = r$) specify an arbitrary order for the edges directed away from s_k with the restriction that the edge of the spanning tree *ST* is the last in the ordering. For the root s_r, specify an arbitrary order for the edges.

• **STEP 3:** The Euler tour of G_E can be found by starting from the node *sr* and always using a previously untraversed edge with the lowest sequential number. If the start node s_r is reached and all edges of this node have been traversed, then terminate.

ST edges with root s_1



Find an Euler circuit over this augmented graph

This problem can be reduced to create a spanning tree, which edges are directed towards the initial state of the FSM

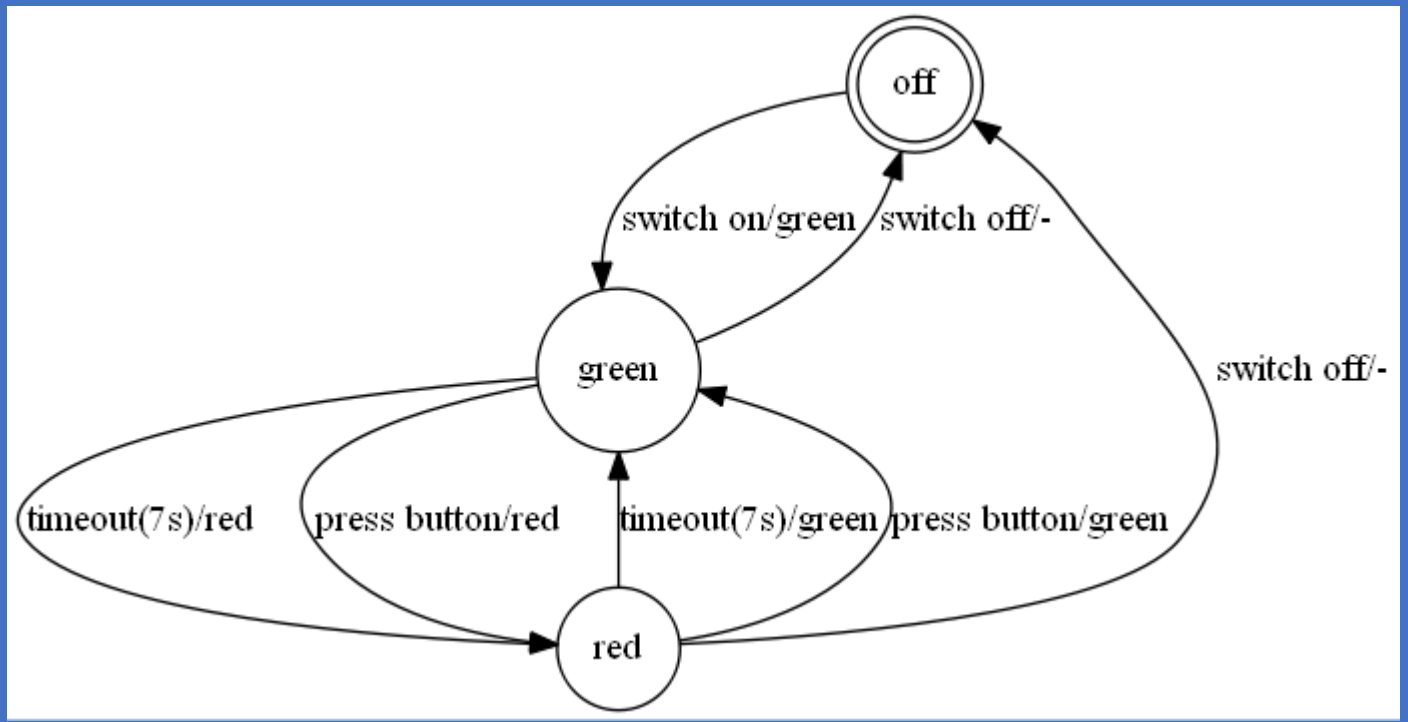
Complexity: O(m)

In the *next-node representation*, order of edges is defined for each node and the node is left after the l^{th} visit across the edge labeled with *l*

FSM-based test generation algorithms:
TT-method: an approach with one stage – 1st exercise

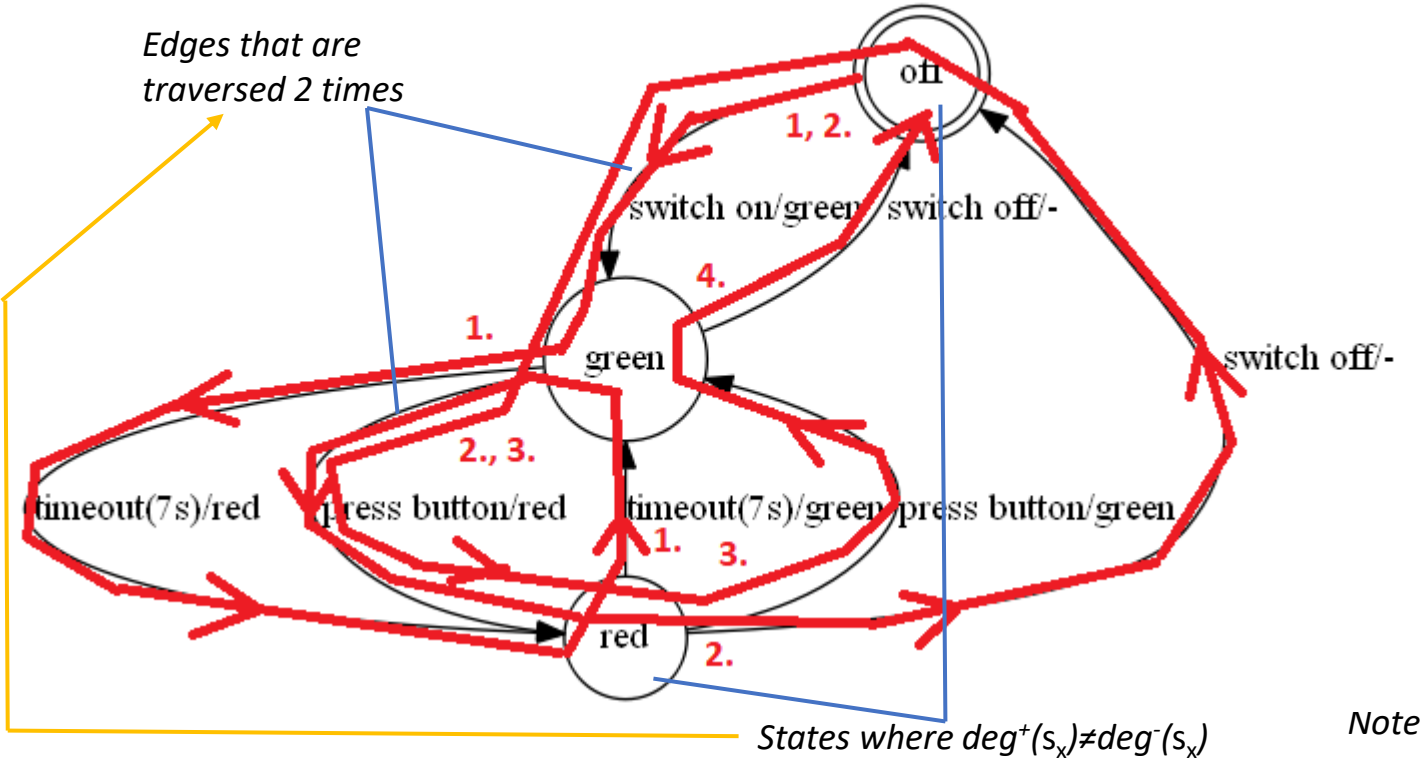
Create TT test sequence for the battery-operated signal!

- Identify unbalanced states (i.e. states where $deg^+(s_x) \neq deg^-(s_x)$)!
- Which edges should be used more than once in the state transition graph?
- Specify the order for the edges directed away from states
- Write down the resulting test sequence (input/desired output)



FSM-based test generation algorithms: TT-method: an approach with one stage – solution for the 1st exercise

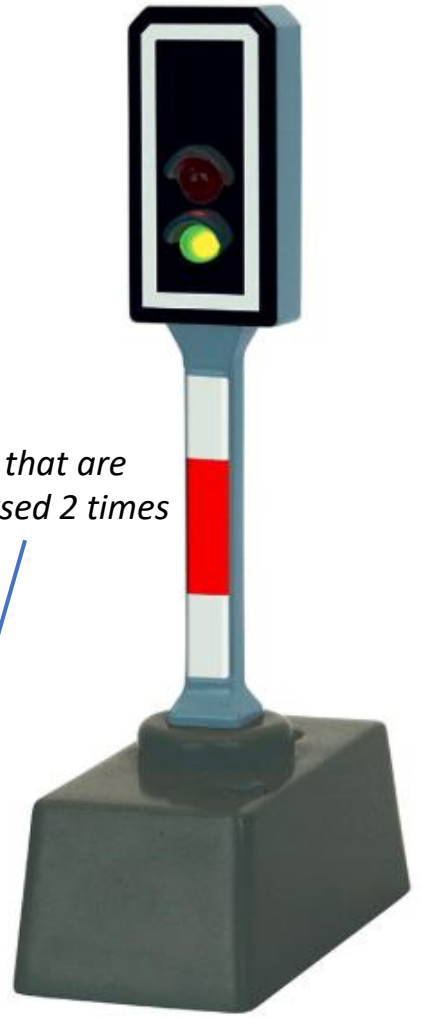
- TT test sequence for the battery-operated signal



TT test sequence:

Action (input)	Desired output
switch on	green
wait for 7 secs	red
wait for 7 secs	green
press button	red
switch off	-
switch on	green
press button	red
press button	green
switch off	-

Edges that are traversed 2 times



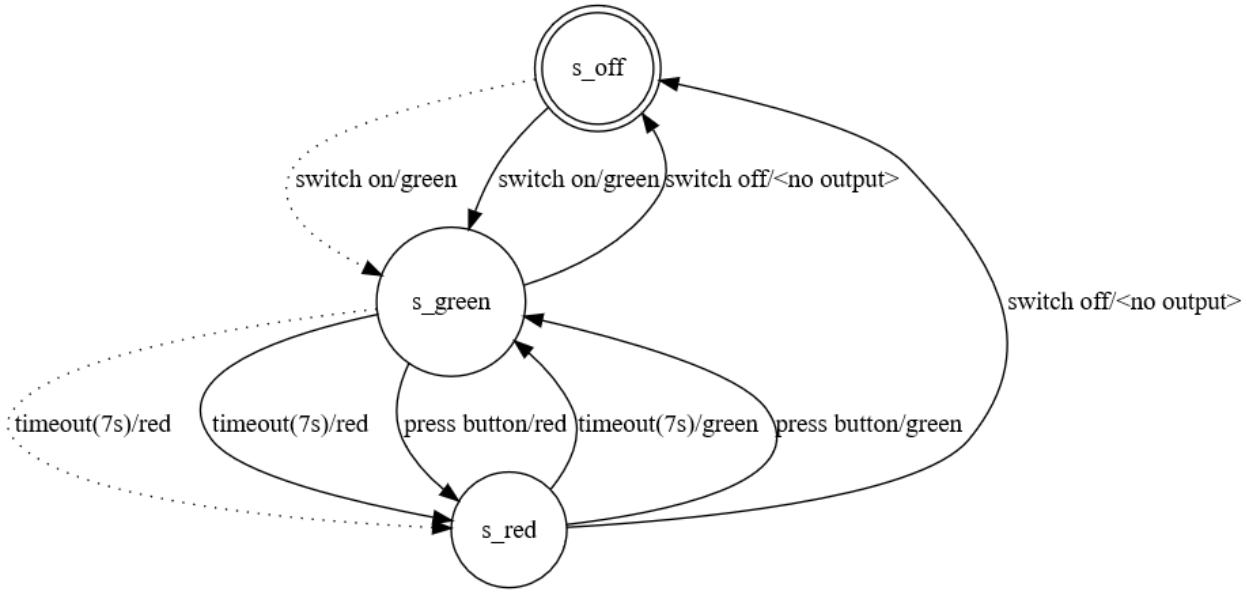
Note that other shortest TT test sequences are also exist

FSM-based test generation algorithms:
TT-method: an approach with one stage – test generation with MTR

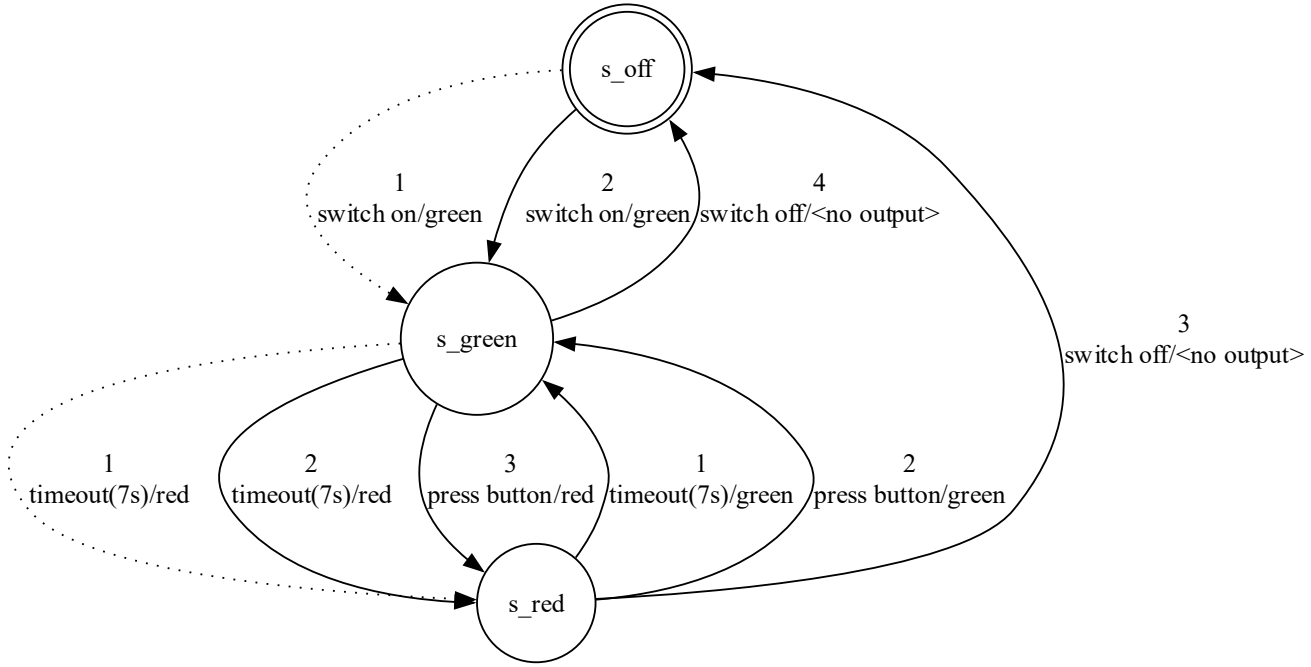
Apply TT test generation on the battery-operated Toy Train signal using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o t -m tt -f sample_models/introduction/Toy_train_signal.json --graphviz
```

Eulerian augmented graph:



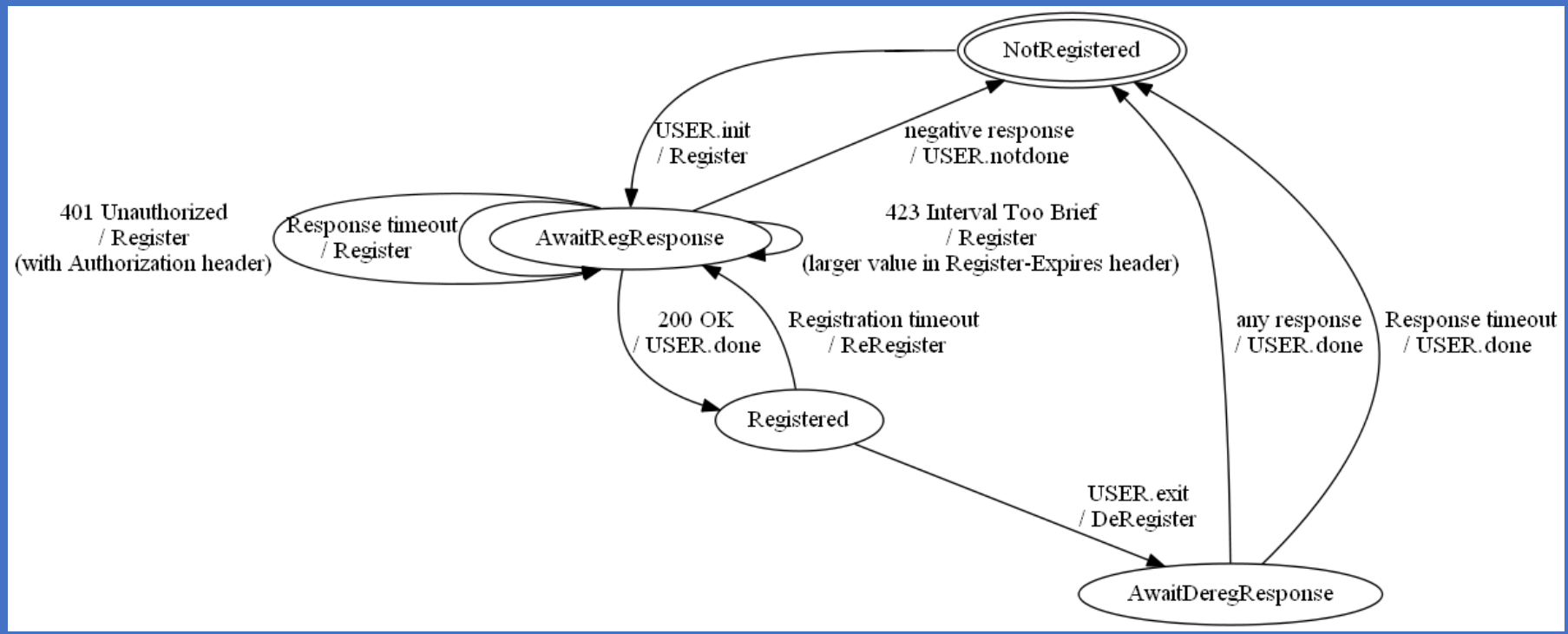
Euler tour in next node representation:



FSM-based test generation algorithms:
TT-method: an approach with one stage – 2nd exercise

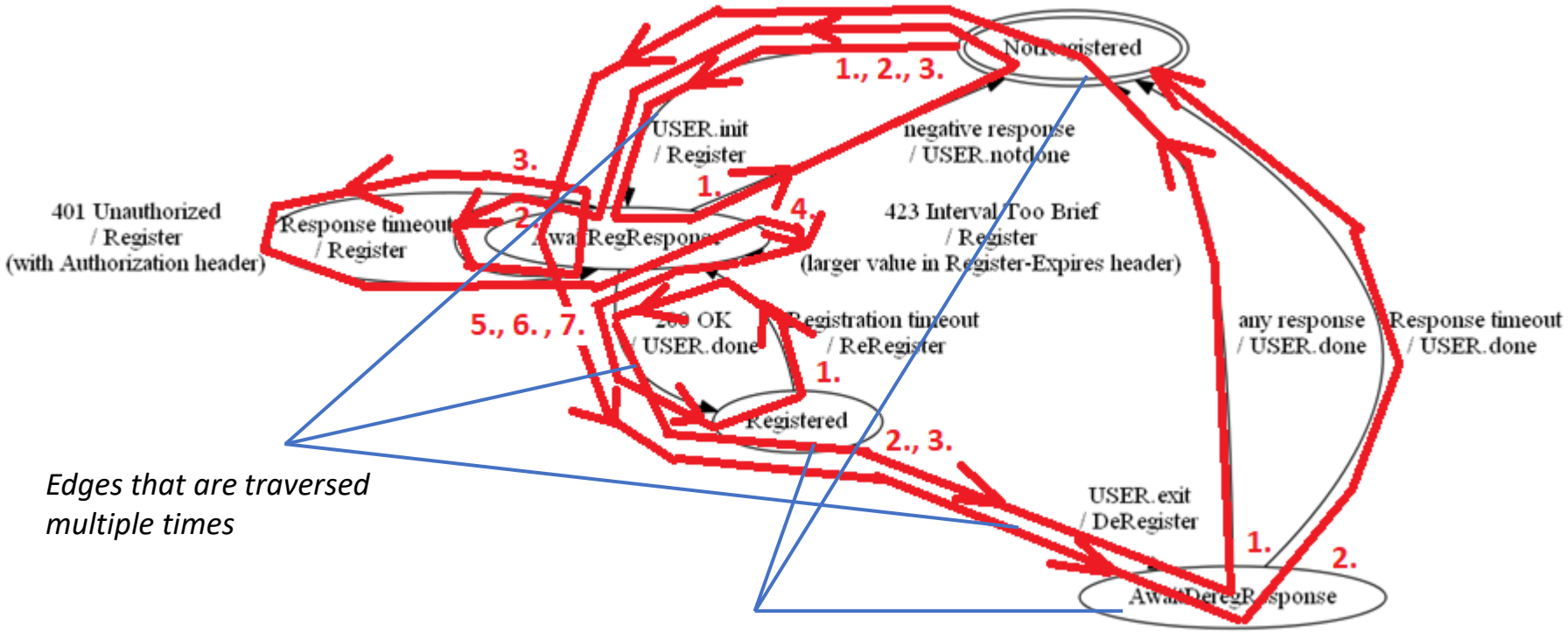
Create TT test sequence for the SIP registration process of UAC (only for signalling level)!

- Identify unbalanced states (i.e. states where $deg^+(s_x) \neq deg^-(s_x)$)!
- Which edges should be used more than once in the state transition graph?
- Specify the order for the edges directed away from states
- Write down the resulting test sequence (input/desired output)



FSM-based test generation algorithms: TT-method: an approach with one stage – solution for the 2nd exercise

- TT test sequence for the SIP registration process of UAC (only for signalling level)



Edges that are traversed multiple times

Unbalanced states, i.e. states where $deg^+(s_x) \neq deg^-(s_x)$

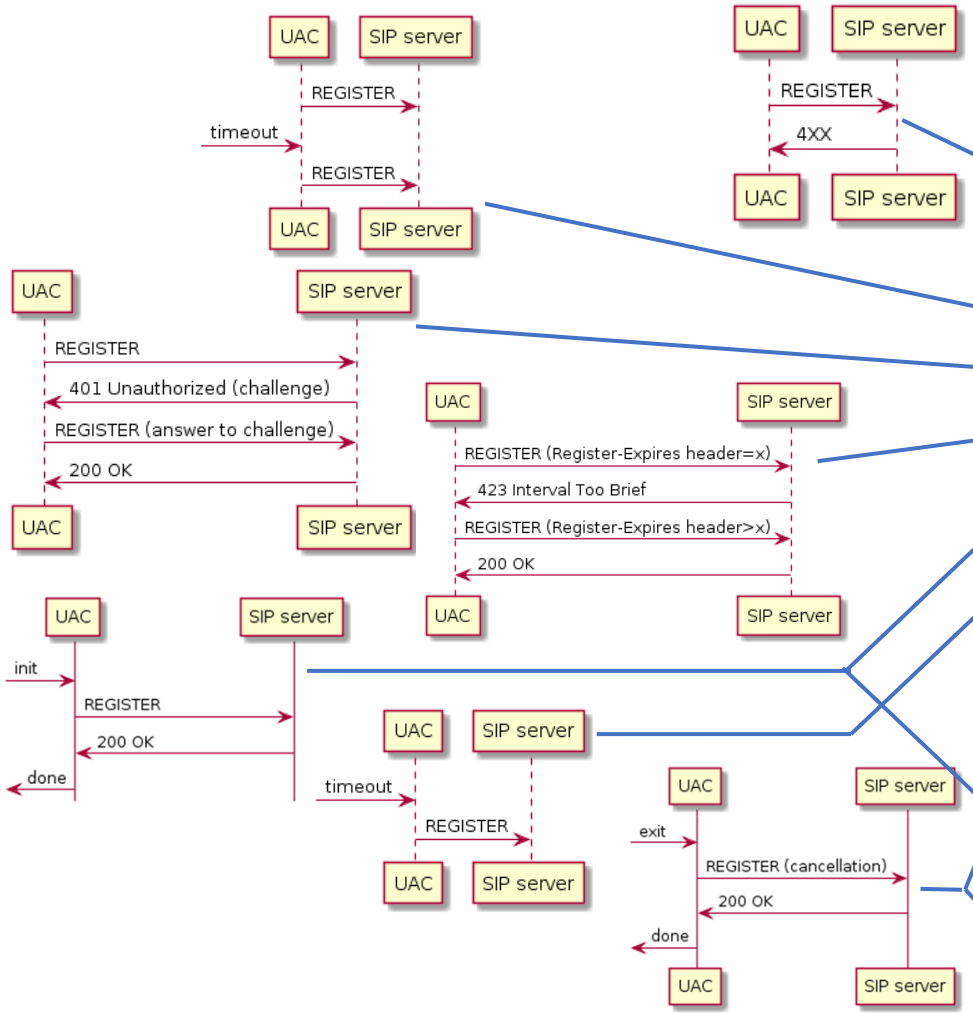
Action (input)	Desired output
USER.init	Register
negative response	USER.notdone
USER.init	Register
Response timeout	Register
401 Unauthorized	Register
423 Interval Too Brief	Register
200 OK	USER.done
Registration timeout	(Re-)Register
200 OK	USER.done
USER.exit	(De-)Register
Any response	USER.done
USER.init	Register
200 OK	USER.done
USER.exit	(De-)Register
Response timeout	USER.done

Apply TT test generation on the SIP UAC example using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o t -m tt -f sample_models/introduction/SIP_Registration_UAC.json --graphviz
```

FSM-based test generation algorithms: TT-method: an approach with one stage – solution for the 2nd exercise

- TT test sequence for the SIP registration process of UAC (only for signalling level)



2. Handle negative responses

4. Silent discard

1. Handle of authentication

3. Interval too brief

0.a Successful registration

5. Reregister functionality

0.b Deregistration (with resp.)

0.a Successful registration (no auth.)

0.b Deregistration (no resp.)

Action (input)	Desired output
USER.init	Register
negative response	USER.notdone
USER.init	Register
Response timeout	Register
401 Unauthorized	Register
423 Interval Too Brief	Register
200 OK	USER.done
Registration timeout	(Re-)Register
200 OK	USER.done
USER.exit	(De-)Register
Any response	USER.done
USER.init	Register
200 OK	USER.done
USER.exit	(De-)Register
Response timeout	USER.done

FSM-based test generation algorithms: TT-method: an approach with one stage – lack of state identification

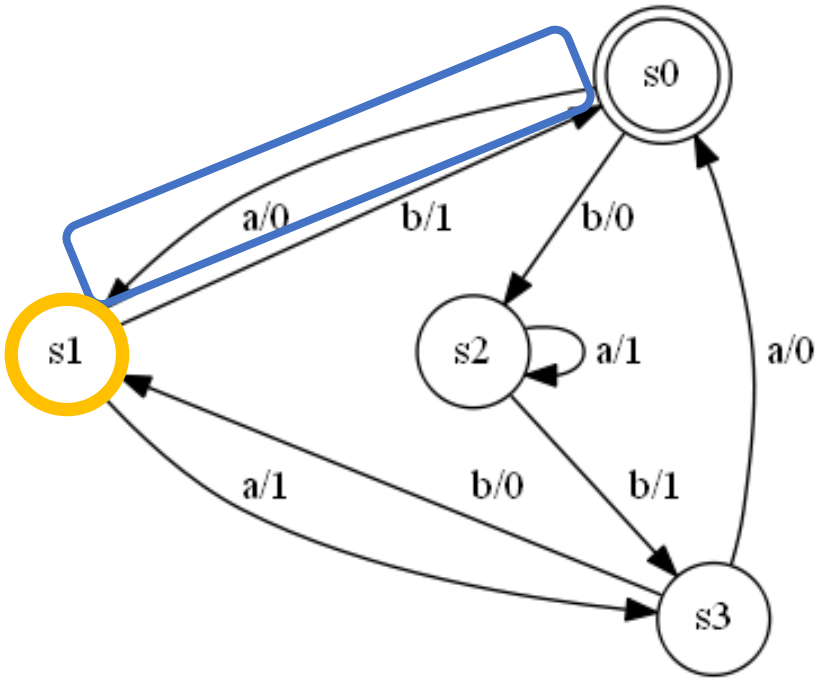
- As the original TT-method (referred as TT v1 in the overview table of test gen. algos.) does not check the end transitions with state identification it does not guarantee to find transfer faults

- If the original TT-method is extended with a *status* message after every step (referred as TT v2 in the overview table of test gen. algos.), it guarantees to find all transfer fault
  - However, status message rarely exists in practice

FSM-based test generation algorithms:
Approaches with 2 stages – the general idea

Stage 1st: Verify all states of the FSM

1. Preamble
   - To reach a given state

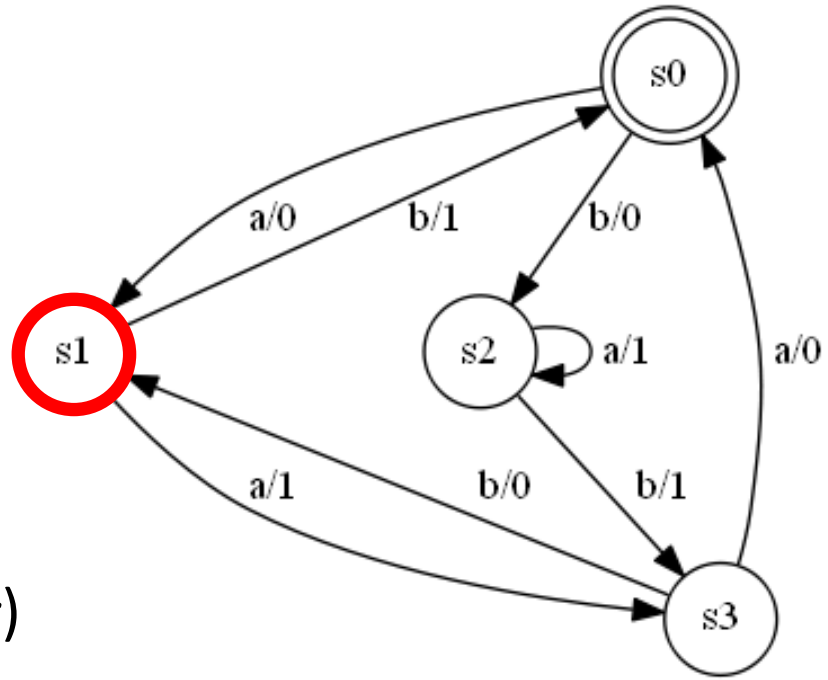


FSM-based test generation algorithms:
Approaches with 2 stages – the general idea

Stage 1st: Verify all states of the FSM

1. Preamble
   - To reach a given state

2. Postamble
   - To verify the given state
     (to detect FSM *Impl* has same states as FSM *Spec*)



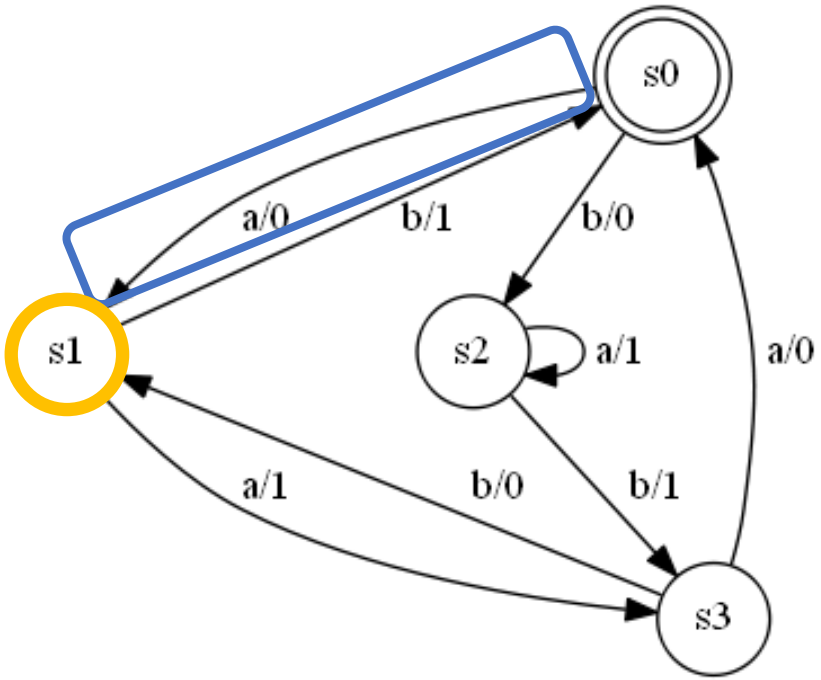
Test sequences for all states s_x:
reset.reach(s_x).verify(state s_x)

- stage 1st: state identification

FSM-based test generation algorithms:
Approaches with 2 stages – the general idea

Stage 2nd: test all remaining transition of the FSM

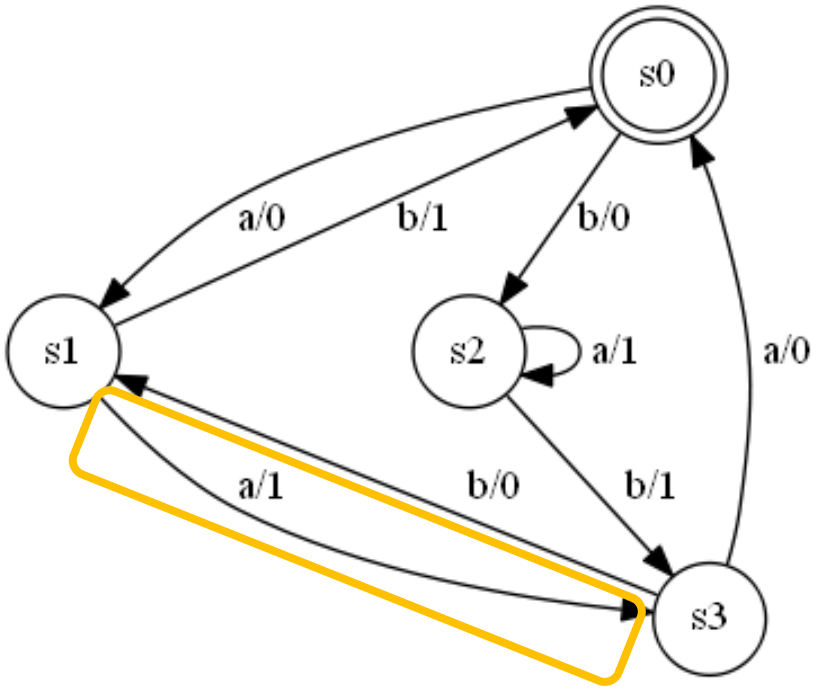
1. Preamble
   - To reach a given state



FSM-based test generation algorithms:
Approaches with 2 stages – the general idea

Stage 2nd: test all remaining transition of the FSM

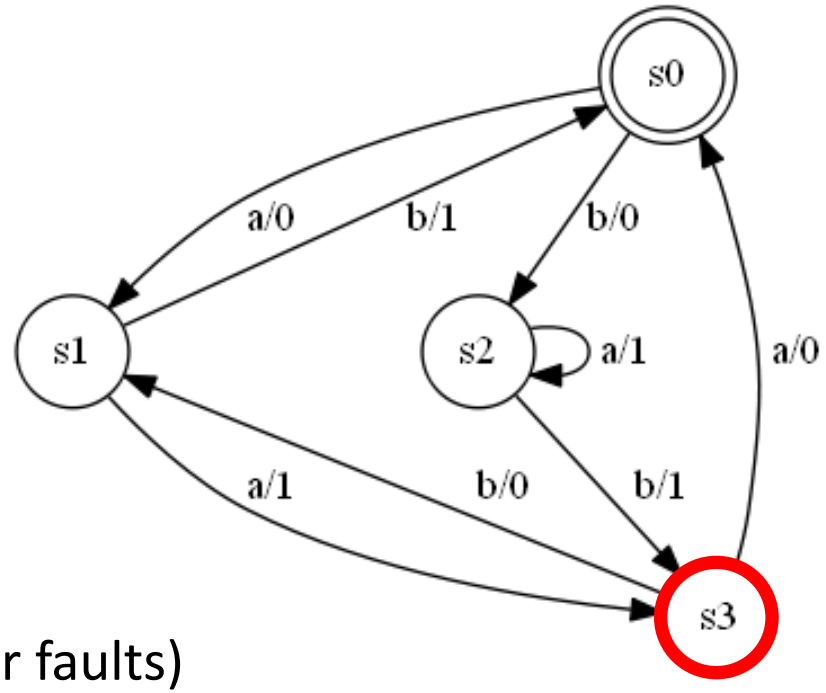
1. Preamble
   - To reach a given state
2. Test a given transition
   - To find possible output faults



FSM-based test generation algorithms:
Approaches with 2 stages – the general idea

Stage 2nd: test all remaining transition of the FSM

1. Preamble
   - To reach a given state
2. Test a given transition
   - To find possible output faults
3. Postamble
   - To verify end state of transition (to find possible transfer faults)



Overall test set consists of test sequences for all states s_x and for all transitions $t_k=(s_x, s_y, i_m o_n)$:

reset.reach(s_x).verify(state s_x) - stage 1st: state identification

reset.reach(s_x).t_k.verify(s_y) - stage 2nd: transition testing

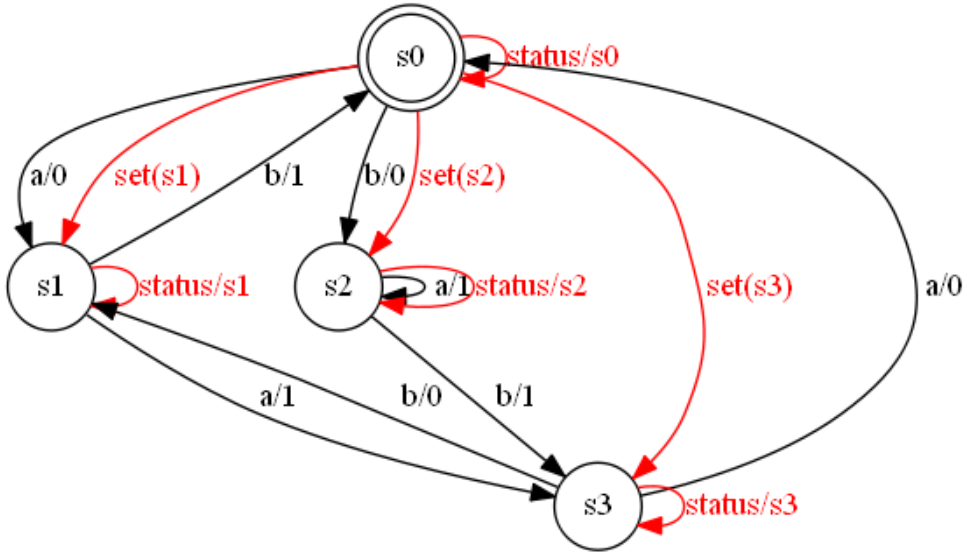
FSM-based test generation algorithms:
The simplest, naive approach with 2 stages

Structure:

1. Preamble
   - To reach given state with set(s_x) set message

Rarely exists in practice

2. Test a given transition *(only in transition testing stage)*



Rarely exists in practice

3. Postamble
   - To verify end state of transition with a status message

Overall test set consists of test sequences for all states s_x and for all transitions $t_k=(s_x, s_y, i_m o_n)$:

reset.set(s_x).status(s_x) - stage 1st: state identification

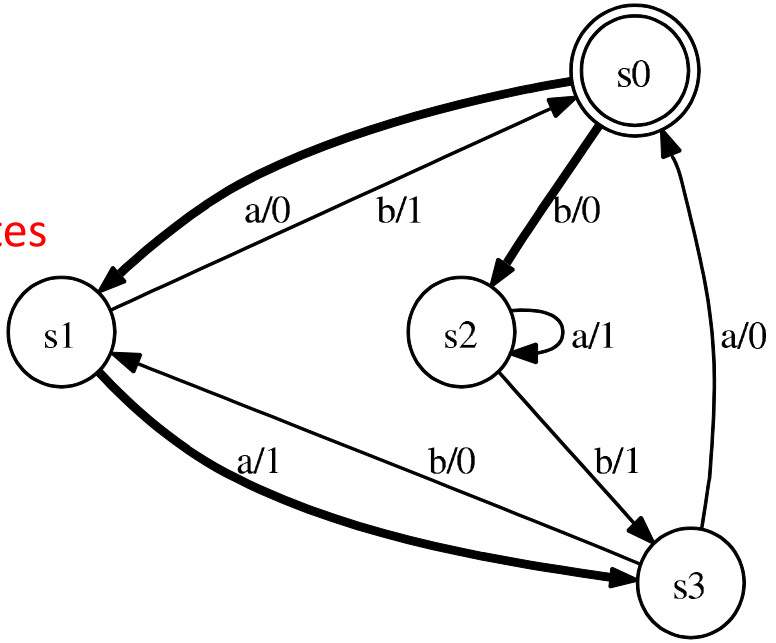
reset.set(s_x).t_k.status(s_y) - stage 2nd: transition testing

FSM-based test generation algorithms:
W / Wp / HSI/HIS*-method triple

* *Sometimes referred to as HIS, sometimes as HSI, but the same method*

Structure:

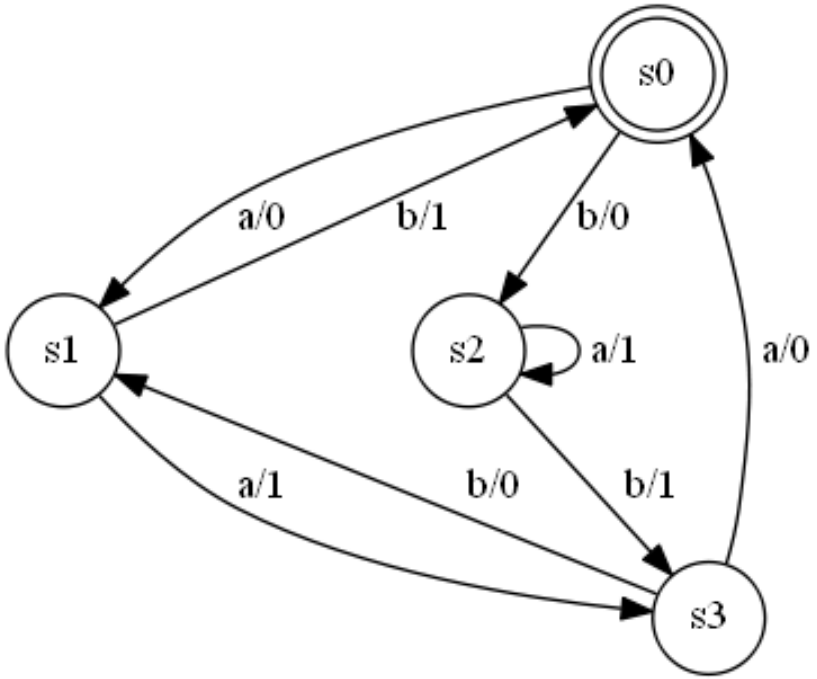
1. Preamble
   - To reach a given state: State Cover Set
     - Create a spanning tree ST from s_0 initial state to all other states *(bold edges)*
2. Test a given transition *(only in transition testing stage)*



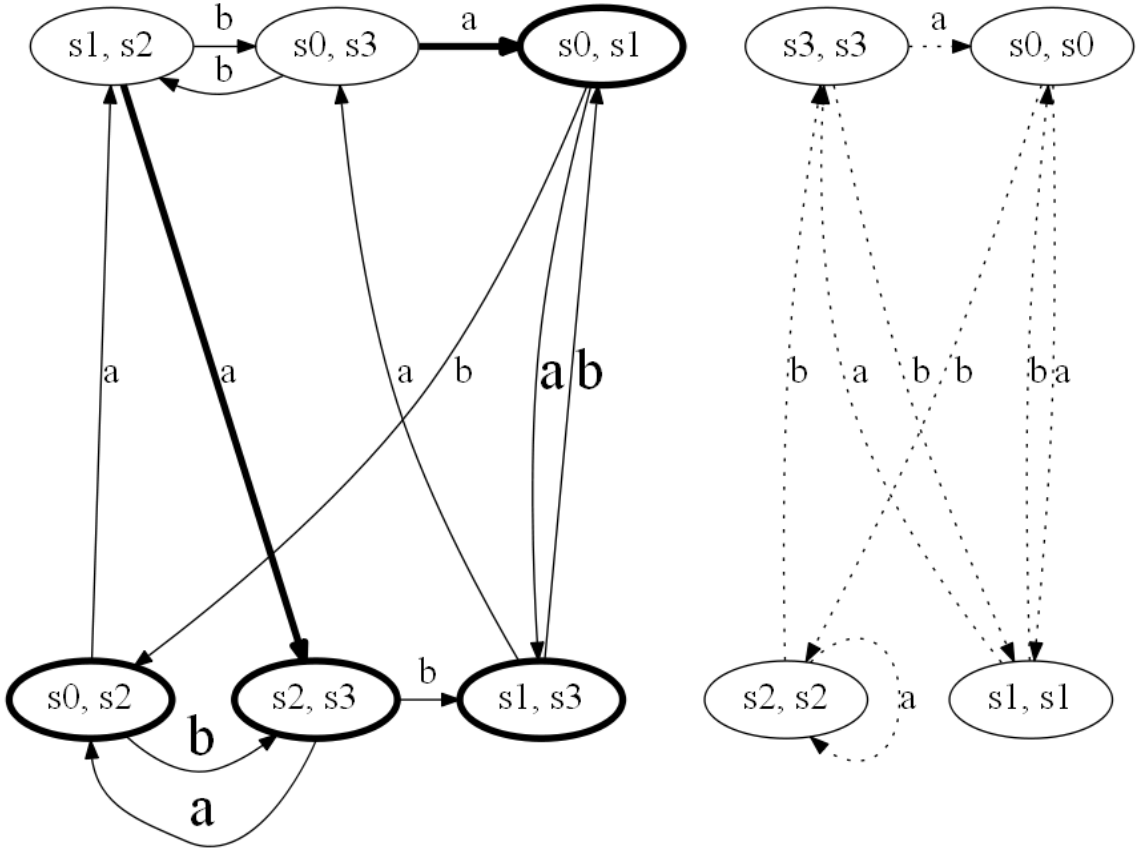
3. Postamble
   - To verify end state
     - Different approaches:
       - W / Wp-method: CS (characterizing set)
       - HSI/HIS-method: separating family of sequences

FSM-based test generation algorithms:
HSI/HIS-method: state verification

Separating family of sequences for state verification:



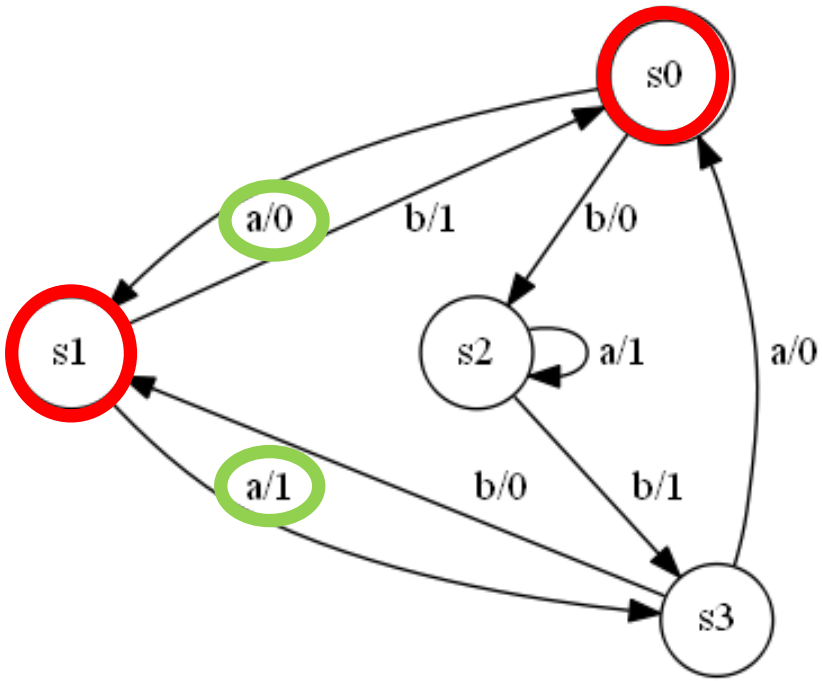
Create an auxiliary state pair graph:

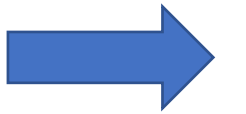


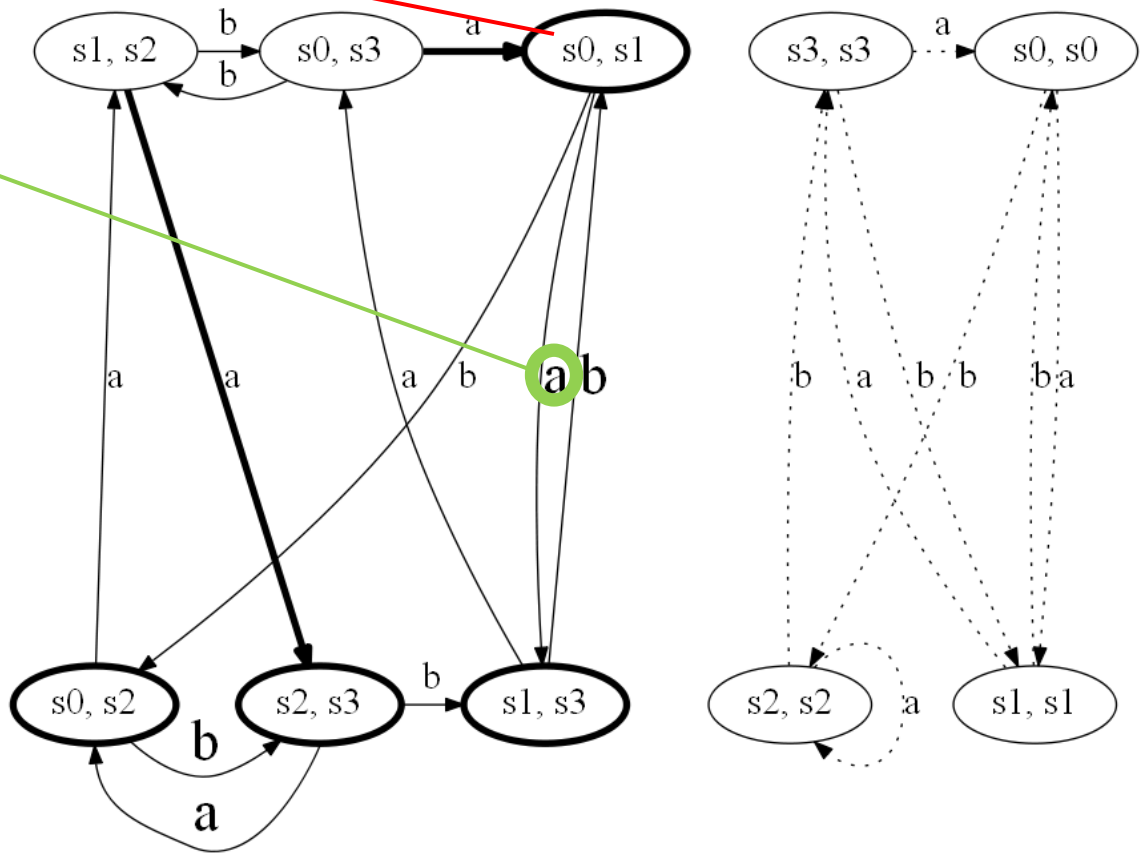
FSM-based test generation algorithms:
HSI/HIS-method: state verification

Separating state pair: has a separating input

Separating input: has different output for different states



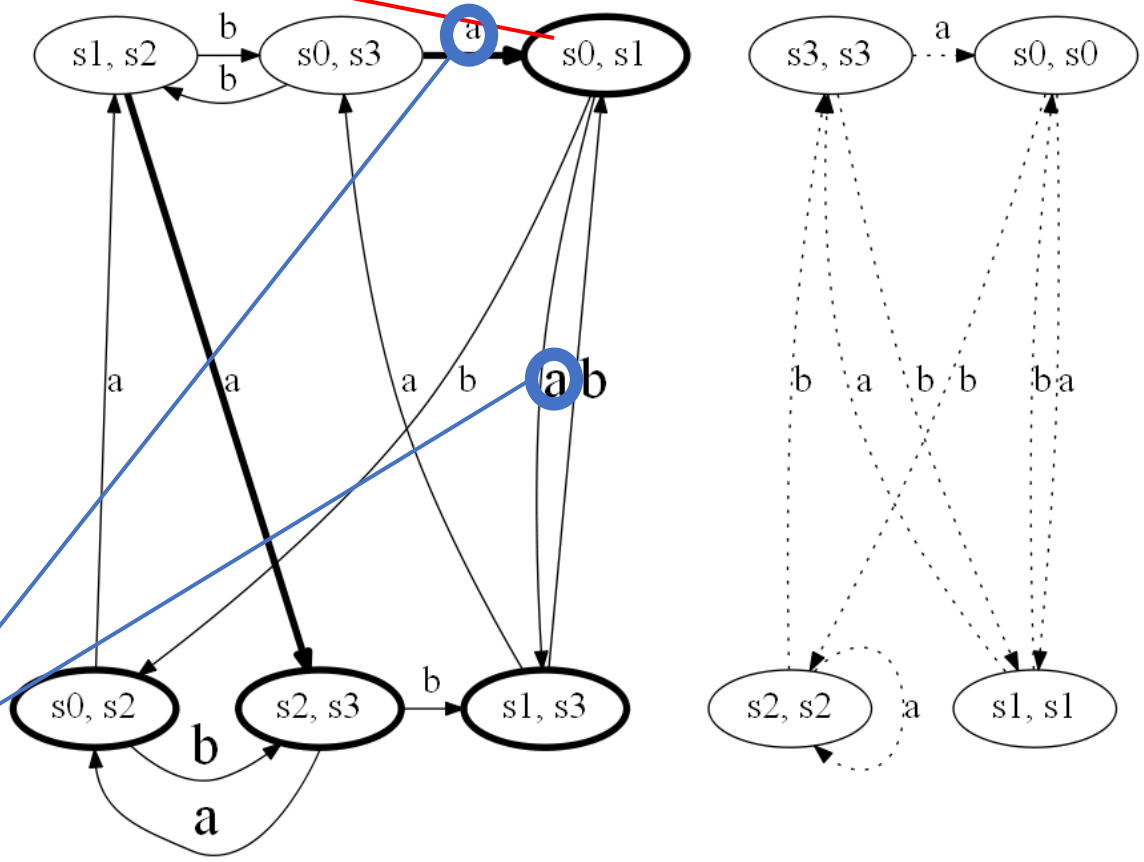




FSM-based test generation algorithms:
HSI/HIS-method: state verification

Separating state pair: has a separating input

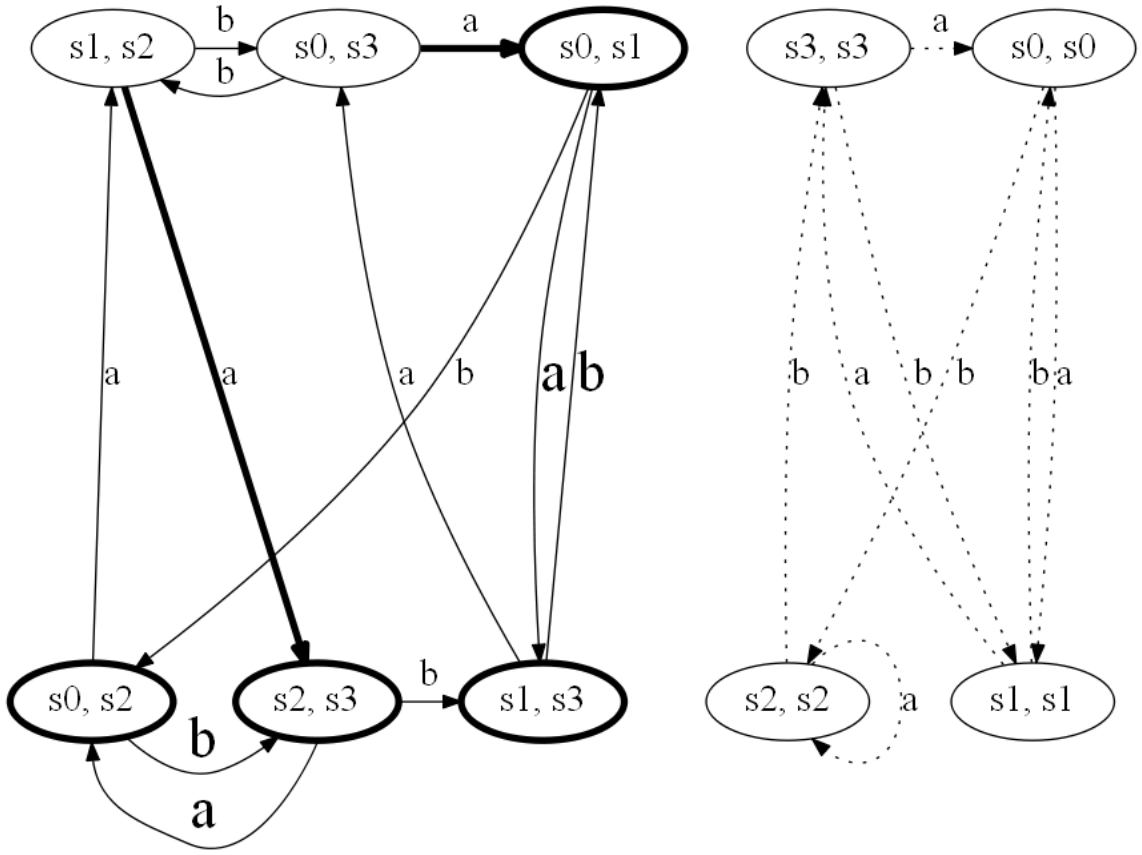
- Find a path from each (non-identical, non separating) state-pair to a separating state pair!
- These paths can be represented with a spanning forest *(bold edges)*
- A z_{ij} separating sequence of state pair s_x, s_y will be a path to a separating state pair concatenated with its separating input
  - *Example for <s_0, s_3>: a.a*



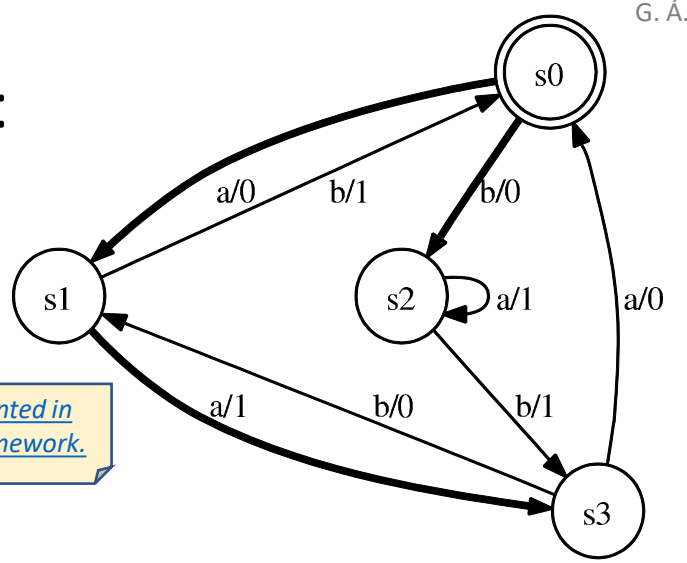
FSM-based test generation algorithms:
HSI/HIS-method: state verification

- Then the separating family of sequences can be built up in the following way:
  - $Z=\{Z_o \ldots Z_i \ldots Z_{n-1}\}$, *where* $Z_i=\{z_{i1} \ldots z_{ij} \ldots z_{in}\}$, z_{ij} separating sequence of state pair s_i, s_j

  - *Example:*
    - $Z_0=\{z_{01}=a, z_{02}=b, z_{03}=a.a\}$
    - $Z_1=\{z_{01}=a, z_{12}=a.a, z_{13}=b\}$
    - $Z_2=\{z_{02}=b, z_{12}=a.a, z_{23}=a\}$
    - $Z_3=\{z_{03}=a.a, z_{13}=b, z_{23}=a\}$

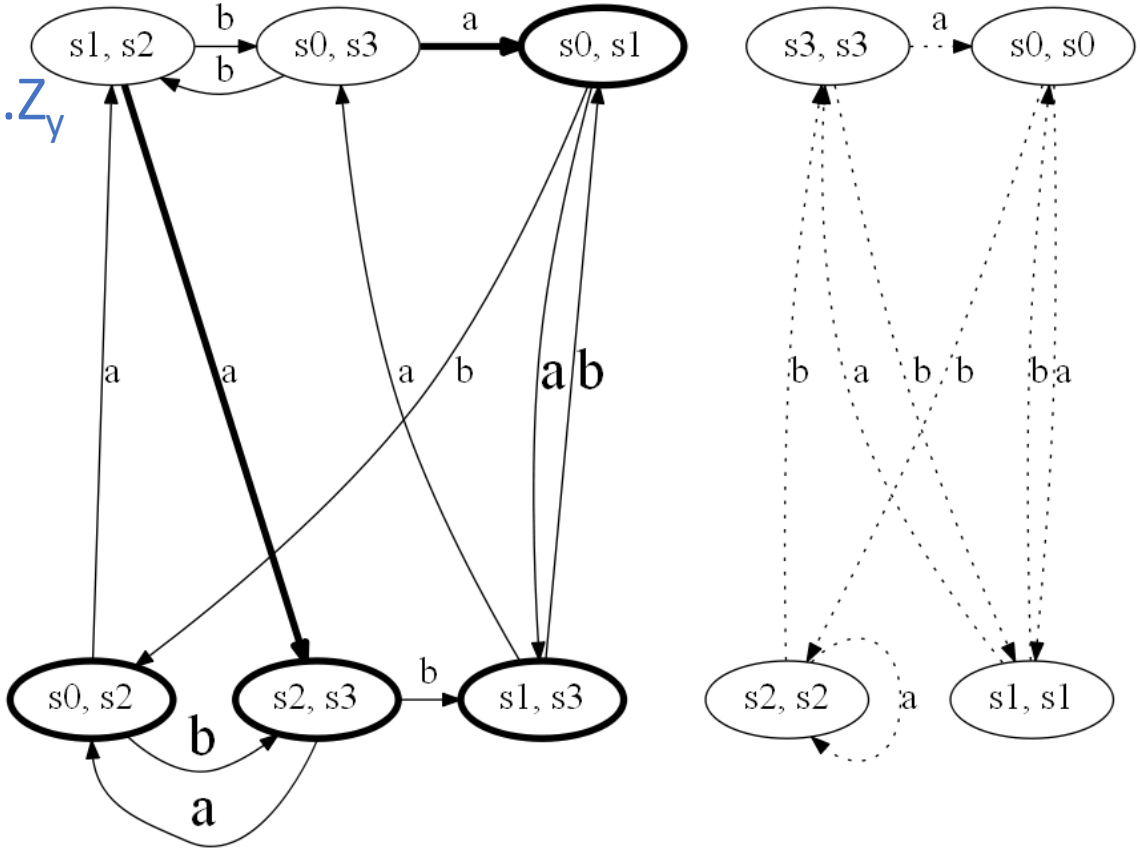


FSM-based test generation algorithms: HSI/HIS-method: final structure



- The test set is the following:
  - Verifying states:
    - For all s_x states: reset.q_x.Z_x
  - Test transitions:
    - For all $t_k=(s_x, s_y, i_m\ o_n)$ transitions: reset.q_x.t_k.Z_y
- *Example:*
  - *Q set for reach all states:*
    - $Q=\{q_0=\varepsilon, q_1=a, q_2=b, q_3=aa\}$
  - $Z=\{Z_0, Z_1, Z_2, Z_3\}$ *set for state verification:*
    - $Z_0=\{z_{01}=a, z_{02}=b, z_{03}=a.a\}$
    - $Z_1=\{z_{01}=a, z_{12}=a.a, z_{13}=b\}$
    - $Z_2=\{z_{02}=b, z_{12}=a.a, z_{23}=a\}$
    - $Z_3=\{z_{03}=a.a, z_{13}=b, z_{23}=a\}$

MTR *The HSI-method is implemented in Model ⟫ Test ⟫ Relax framework.*

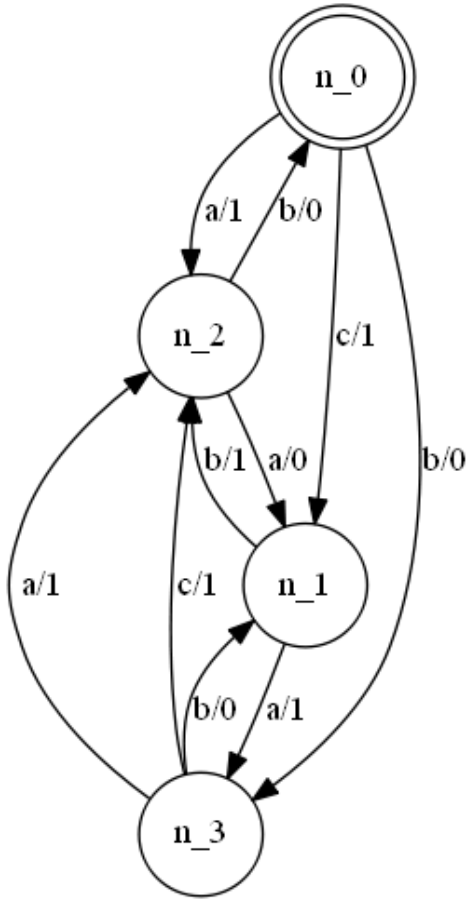


FSM-based test generation algorithms: HSI/HIS-method: test generation with MTR

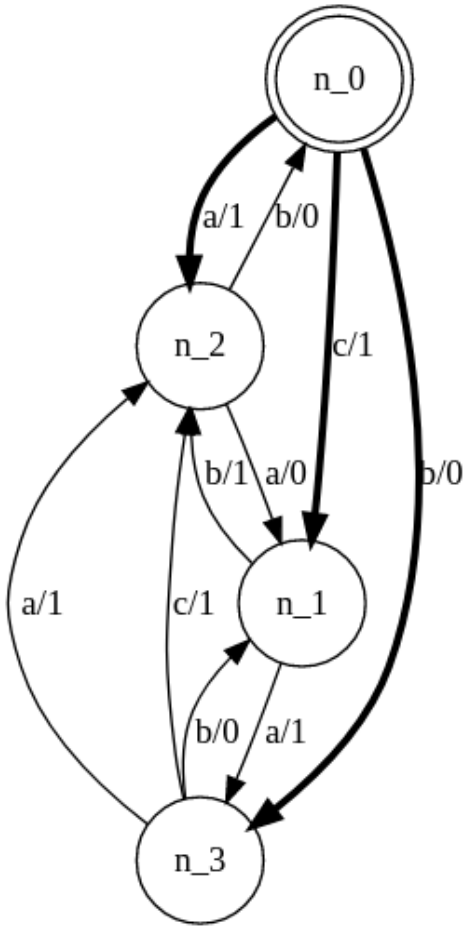
Apply HSI test generation on another example using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o t -m hsi -f sample_models/introduction/h_1.json --graphviz --subsequences
```

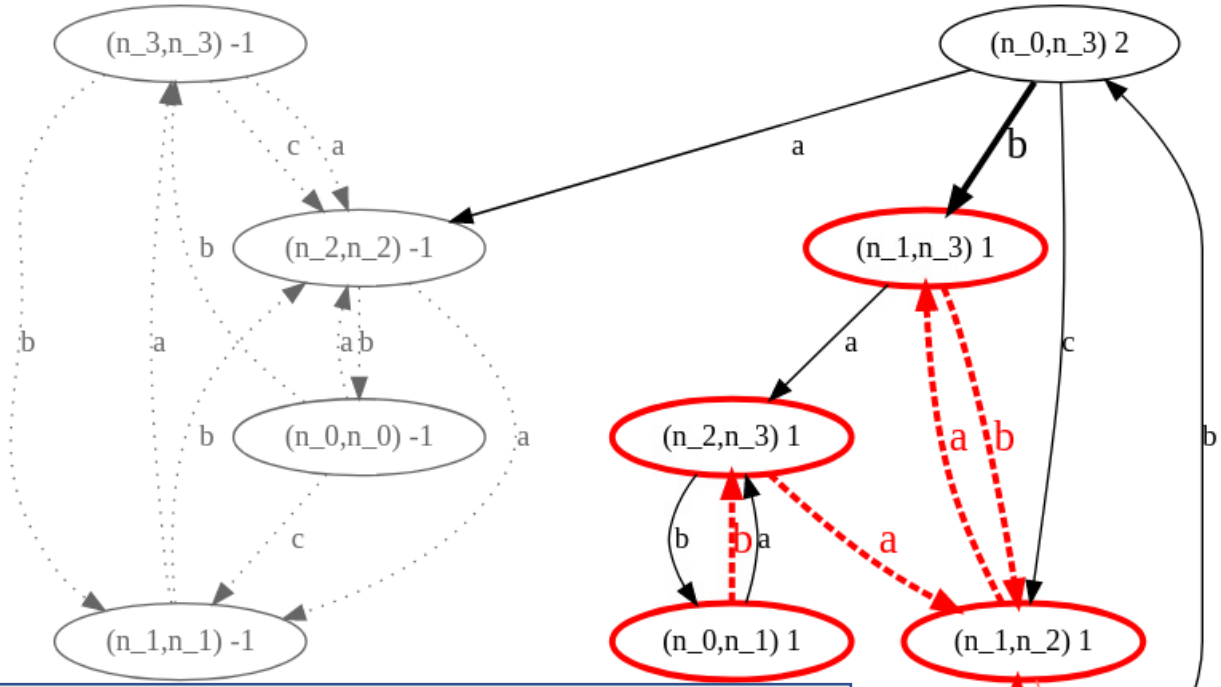
Original graph:



State cover set:



Separating family of sequences over the auxiliary state pair graph:



Legend

- *Red ellipses: separating state pairs*
- *Red dashed edges: separating input*
- *Red bold dashed edges with bigger label: separating input selected by the algorithm*
- *Black bold edges: spanning forest edges (pointing to separating state pairs)*
- *Grey ellipses: identical state pairs*
- *Grey dotted edges: edges between identical state pairs*

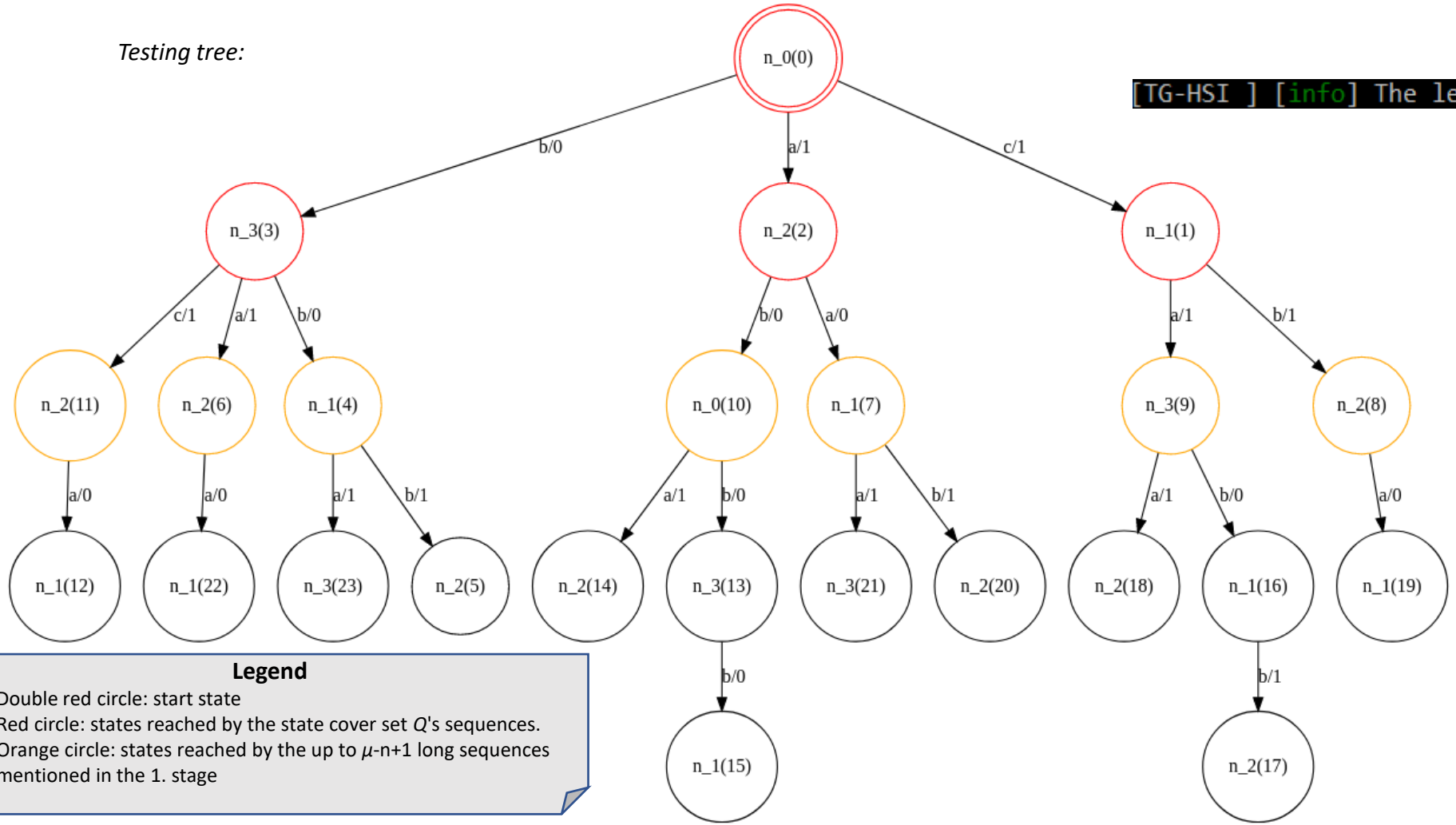
FSM-based test generation algorithms: HSI/HIS-method: test generation with MTR

Apply HSI test generation on another example using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o t -m hsi -f sample_models/introduction/h_1.json --graphviz --subsequences
```

Testing tree:

```
[TG-HSI ] [info] The length of the test suite is 46
```



Legend

- Double red circle: start state
- Red circle: states reached by the state cover set *Q*'s sequences.
- Orange circle: states reached by the up to μ-n+1 long sequences mentioned in the 1. stage

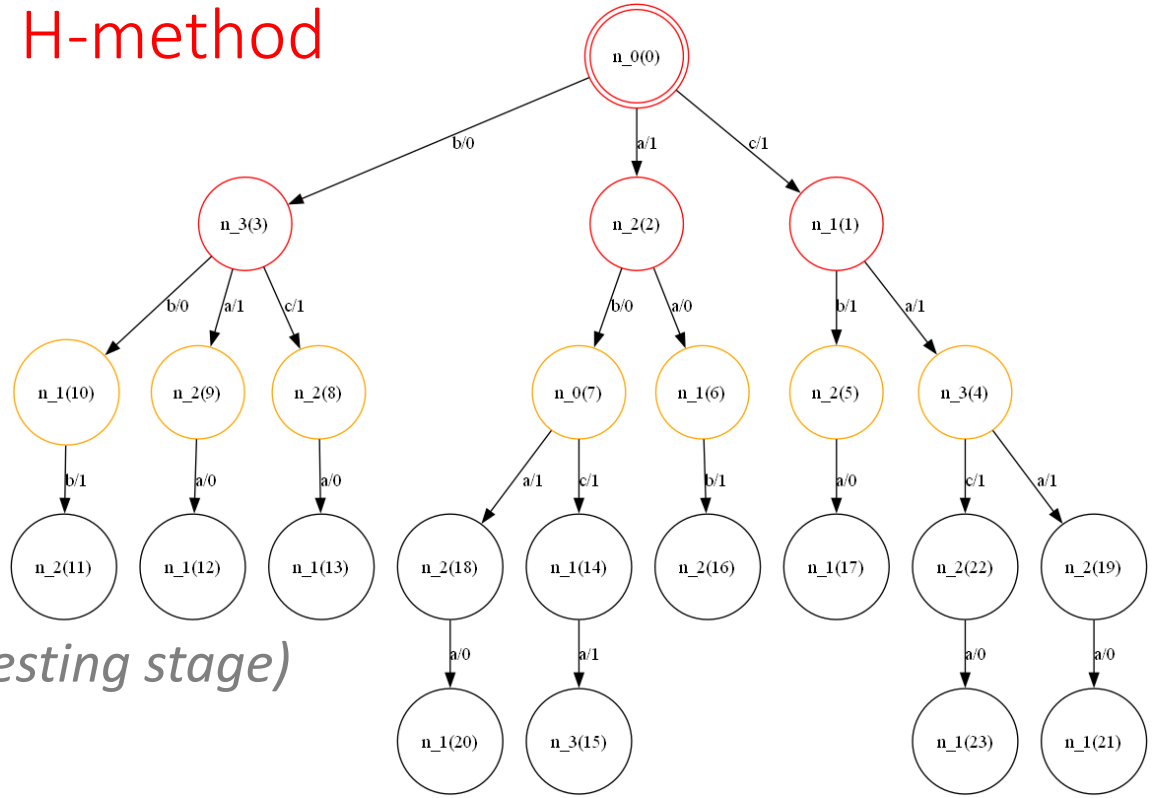
FSM-based test generation algorithms: HSI/HIS-method: test generation with MTR

Apply HSI test generation on another example using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o t -m hsi -f sample_models/introduction/h_1.json --graphviz --subsequences
```


```
"q_state_cover_set_input_list": {
            "q_n_0": [],
            "q_n_1": [
                "c"
            ],
            "q_n_2": [
                "a"
            ],
            "q_n_3": [
                "b"
            ]
        },
"z_separating_family_of_sequences_input_list":
{
            "z_n_0": {
                "z(n_0,n_1)": [
                    "b"
                ],
                "z(n_0,n_2)": [
                    "a"
                ],
                "z(n_0,n_3)": [
                    "b",
                    "b"
                ]
            },
            "z_n_1": {
                "z(n_0,n_1)": [
                    "b"
                ],
                "z(n_1,n_2)": [
                    "a"
                ],
                "z(n_1,n_3)": [
                    "b"
                ]
            },
            "z_n_2": {
                "z(n_0,n_2)": [
                    "a"
                ],
                "z(n_1,n_2)": [
                    "a"
                ],
                "z(n_2,n_3)": [
                    "a"
                ]
            },
            "z_n_3": {
                "z(n_0,n_3)": [
                    "b",
                    "b"
                ],
                "z(n_1,n_3)": [
                    "b"
                ],
                "z(n_2,n_3)": [
                    "a"
                ]
            }
        },
```

FSM-based test generation algorithms:
An improved version of HSI/HIS: the H-method



Structure:

1. Preamble
   - Same as HIS/HIS (uses State Cover Set)
2. Test a given transition *(only in transition testing stage)*
   - Same as HIS/HIS
3. Postamble (to verify end state)
   - It uses similar structure, but instead of using predetermined state identifiers, it selects the appropriate ones on-the-fly, thus shortening the test suite.

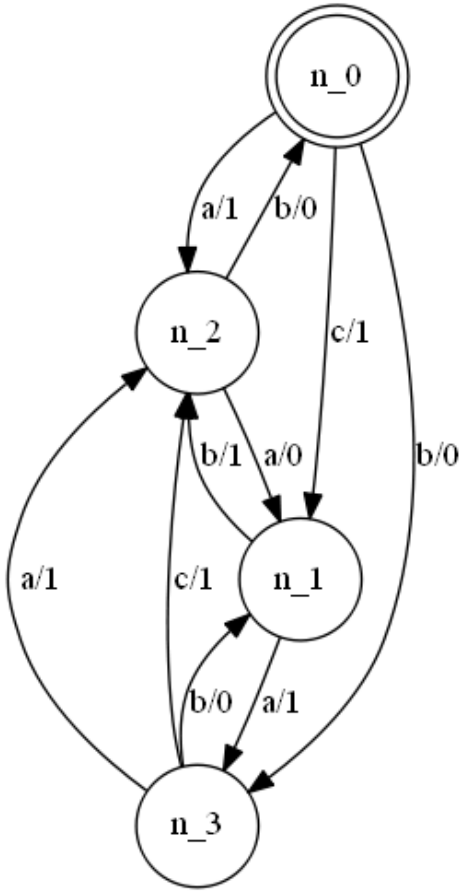
M▶T▶R *The H-method is implemented in the Model ⟫ Test ⟫ Relax framework.*

FSM-based test generation algorithms: H-method: test generation with MTR

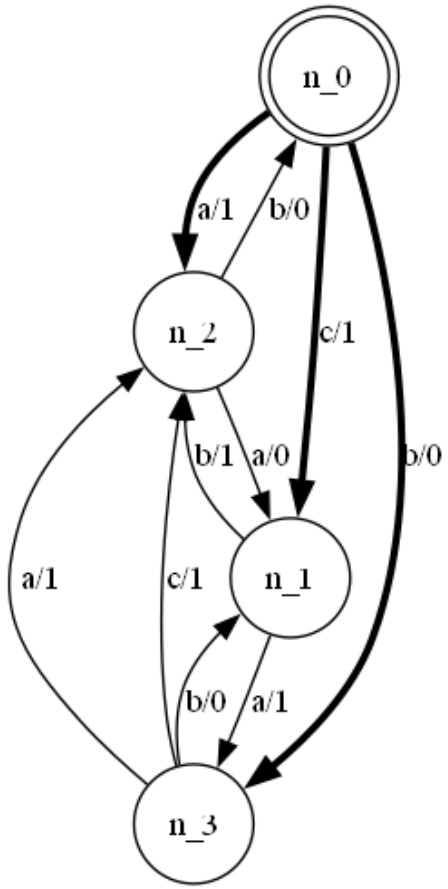
Apply HSI test generation on the previous example using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o t -m h -f sample_models/introduction/h_1.json --graphviz
```

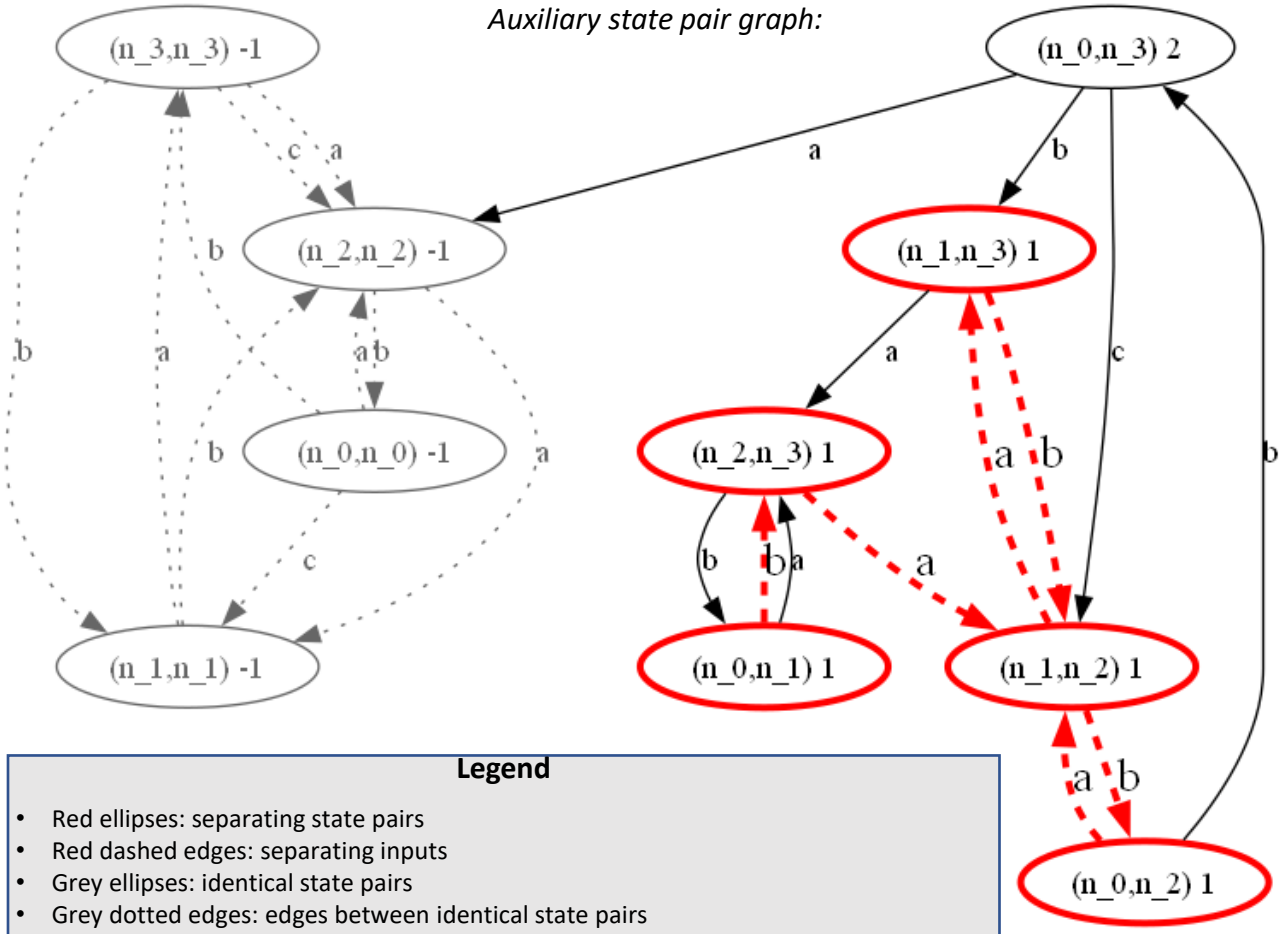
Original graph:



State cover set:



Auxiliary state pair graph:



Legend

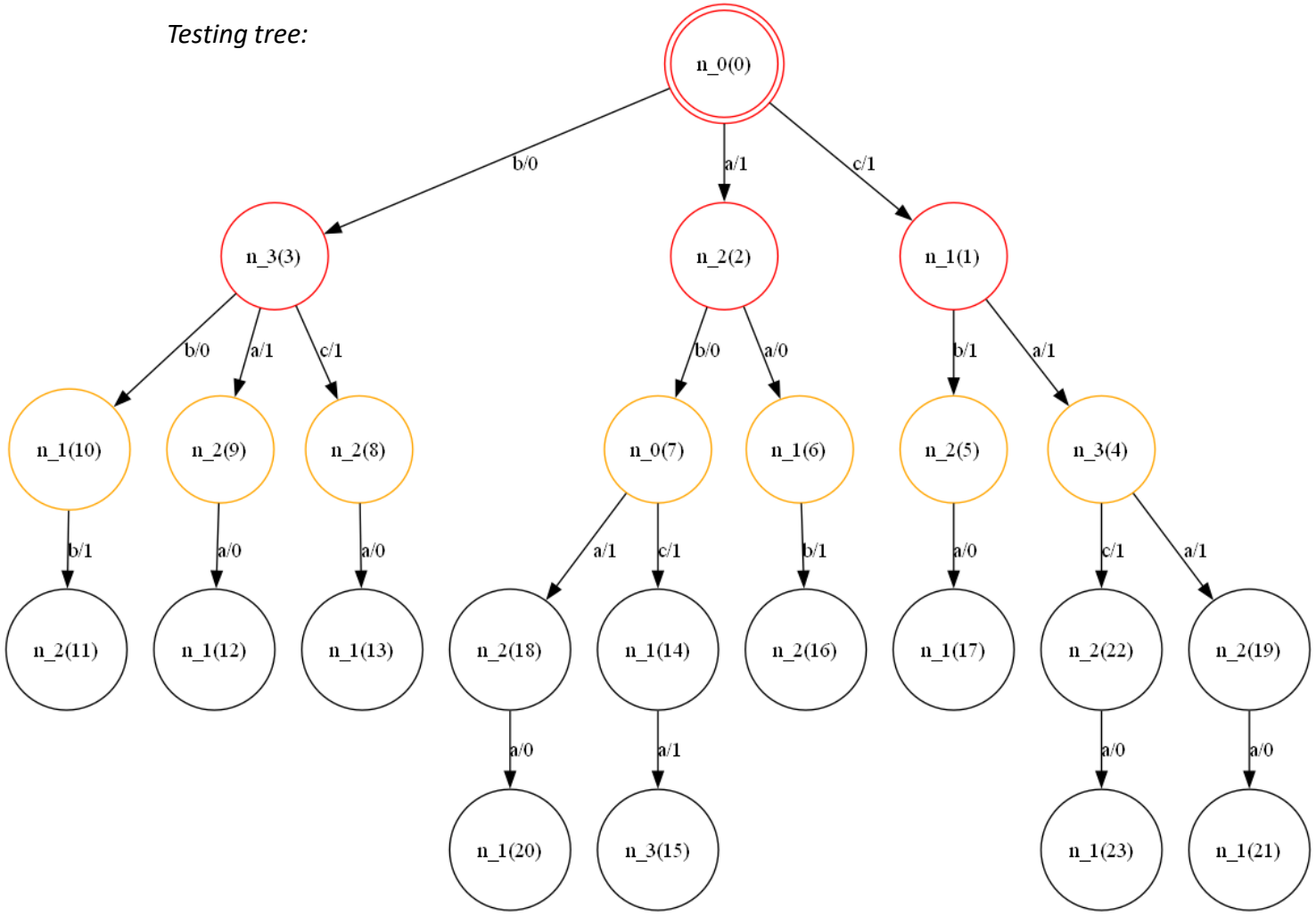
- Red ellipses: separating state pairs
- Red dashed edges: separating inputs
- Grey ellipses: identical state pairs
- Grey dotted edges: edges between identical state pairs
- Integer after the state pair: the length of the shortest possible separating sequence

FSM-based test generation algorithms: H-method: test generation with MTR

Apply H-method test generation on the previous example using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o t -m h -f sample_models/introduction/h_1.json --graphviz
```

Testing tree:



Legend

- Double red circle: start state
- Red circle: states reached by the state cover set *Q*'s sequences.
- Orange circle: states reached by the up to μ-n+1 long sequences mentioned in the 1. stage

Compare the testing tree with the [one generated by HSI]. What is the difference?

- The H-method prioritizes to minimize the branching extension of the testing tree (that would result in multiple test suite from starting state) instead of to select the shortest sequence (i.e. to minimize the depth of testing tree), in another word, the algorithm tries to reach a global optimum (although with a heuristic) instead of local ones.

```
[TG-H   ] [info] The length of the test suite is 40
```

Finding additional states in implementation

- Note that the HSI and H methods are also able to discover additional states in the implementation compared to specification, if the number of states in implementation is fixed.
- Depending the number of possible extra states in implementation, the complexity of the resulting test suite (and test generation) increases

Finding additional states in implementation using MTR

Apply HSI test generation on the previous example with 0 possible extra state using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -o t -m hsi -f sample_models/introduction/h_1.json --extra_states=0 --graphviz
```

```
[TG-HSI ] [info] The length of the test suite is 46
```

Apply HSI test generation on the previous example with 1 possible extra state*:*

```
./MTR -o t -m hsi -f sample_models/introduction/h_1.json --extra_states=1 --graphviz
```

```
[TG-HSI ] [info] The length of the test suite is 141
```

Apply HSI test generation on the previous example with 2 possible extra states*:*

```
./MTR -o t -m hsi -f sample_models/introduction/h_1.json --extra_states=2 --graphviz
```

```
[TG-HSI ] [info] The length of the test suite is 385
```

Apply HSI test generation on the previous example with 3 possible extra states*:*

```
./MTR -o t -m hsi -f sample_models/introduction/h_1.json --extra_states=3 --graphviz
```

```
[TG-HSI ] [info] The length of the test suite is 1052
```

Apply HSI test generation on the previous example with 10 possible extra states*:*

```
./MTR -o t -m hsi -f sample_models/introduction/h_1.json --extra_states=10 --graphviz
```

```
[TG-HSI ] [info] The length of the test suite is 808042
```

…etc

Apply H test generation on the previous example with 1 possible extra state*:*

```
./MTR -o t -m h -f sample_models/introduction/h_1.json --extra_states=1 --graphviz
```

…etc

FSM-based test generation algorithms – Summary

Method	Structure	Required assumptions	State Identification	Complexity of test generation	Length of test	Comments
S (naive approach)	1 stage	1-9	status	$O(p \cdot n)$	$O(p \cdot n)$	Applicable only if both set and status messages exist.
D	2 stages	1-6, DS	DS	PSPACE	Exp.	Applicable only if DS exists (it rarely exists).
adapt. D	2 stages	1-6, DS	adaptive DS	no preset test	$O(p \cdot n^3)$	Adaptive version of D.
TT v1	1 stage	2, 3, 5, 6	-	$O(n^3 + m)$	$O(m)$	Only guarantees to find output faults.
TT v2	1 stage	2-6, 8	status	$O(n^3 + m)$	$O(m)$	Applicable only if status message exists.
UIO	1 stage	1-7, UIO seq. for $\forall s_k \in S$	UIO seq.	PSPACE	Exp.	Does not guarantee to find every fault. Applicable only if UIO sequence exists for each state. Similar to W.
UIOv	2 (3) stages	1-7, UIO seq. for $\forall s_k \in S$	UIO seq.	PSPACE	Exp.	Applicable only if UIO sequence exists for each state. Similar to Wp.
W	1 stage	1-7	CS	$O(p \cdot n^3)$	$O(p \cdot n^3)$	-
Wp	2 stages	1-7	CS	$O(p \cdot n^3)$	$O(p \cdot n^3)$	An improved version of W.
HIS /HSI	2 stages	2-7	sep. family of seq.	$O(p \cdot n^3)$	$O(p \cdot n^3)$	Similar to Wp.
H	2 stages	2-7	adaptive selection from sep. family of seq.	no preset test	$O(p \cdot n^3)$	An improved version of HSI

Assumptions:

1. Completely specified
2. Deterministic
3. Strongly connected
4. Minimized
5. *Impl* does not change during test
6. Upper bound for states in *Impl*
7. *reset* msg
8. *status* msg
9. *set* msg

legends

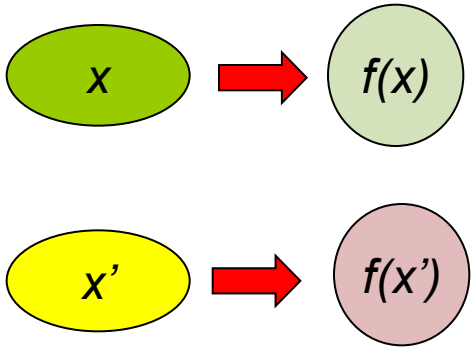
- n: number of states
- p: number of input symbols
- m: number of transitions

Abbreviations

CS: Characterizing Set
DS: Distinguishing Sequence
UIO: Unique Input Output
HSI: Harmonized State Identifiers
TT: Transition Tour
UIO: Unique Input Output

Traditional vs. incremental approach

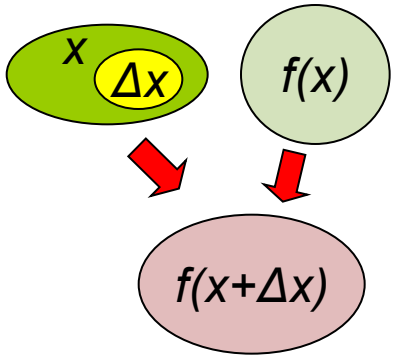
Traditional, batch algorithms:



Input:
• system specification: *x*

Output:
• test suite: *f(x)*

Incremental algorithms:



Input:
• system specification: *x*
• changes: Δx
• **previous test suite: *f(x)***

Output:
• new test suite: $f(x+\Delta x)$

- Assume static specifications
- Test suites are generated "from scratch" after a change

- Assume dynamic specifications
- Utilize additional information:
  - Previous test suite

- Complexity:
  - Depends on the **size of the specification**

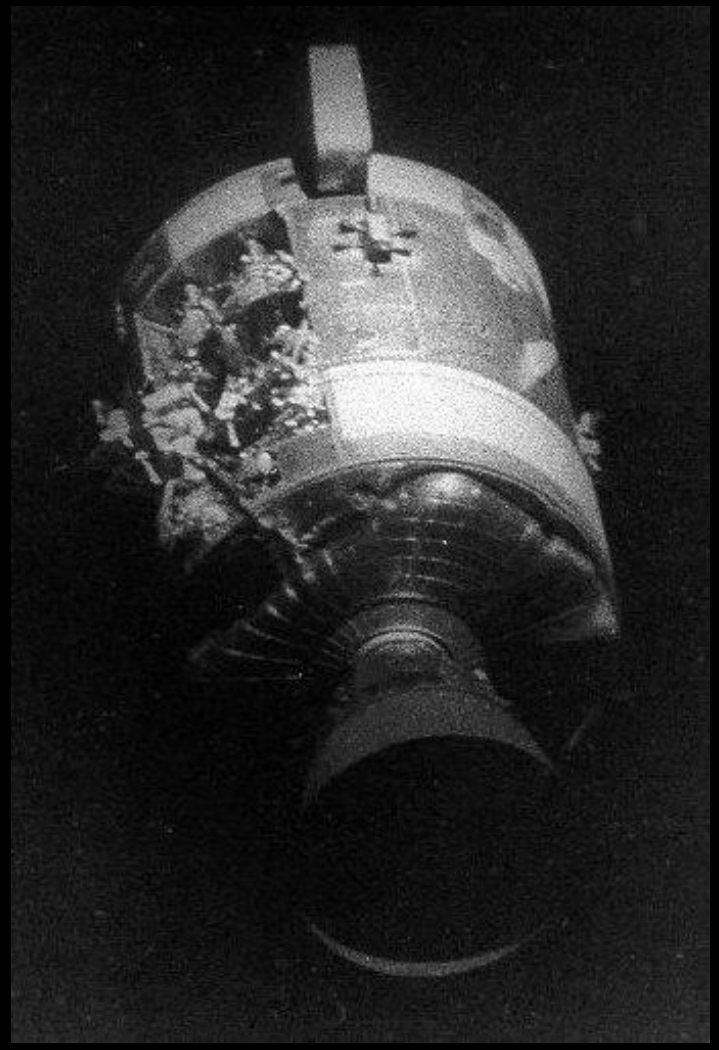
⟺

- Complexity:
  - Depends on the **extent of change**

INEFFICIENT APPROACH ⟺ **EFFICIENT APPROACH**

Δ - „extent of changes”: modified part of the input, **affected part of the output**

G. Ramalingam and T. Reps. On the computational complexity of dynamic graph problems. Theoretical Computer Science, 158(1–2):233–277, 1996.



Advantages of incremental approach over batch approach

- Generates test cases significantly faster for evolving systems

- Supports **change-focused testing**:
  - Retest only the parts that are affected by changes
  - Function test of new features and their cross effects

- Preserves the old test cases of the previous system if reusable
  - Does not force the test designer to learn a new structure of test cases for the unchanged parts
  - Supports **regression testing**
  - Supports performance testing

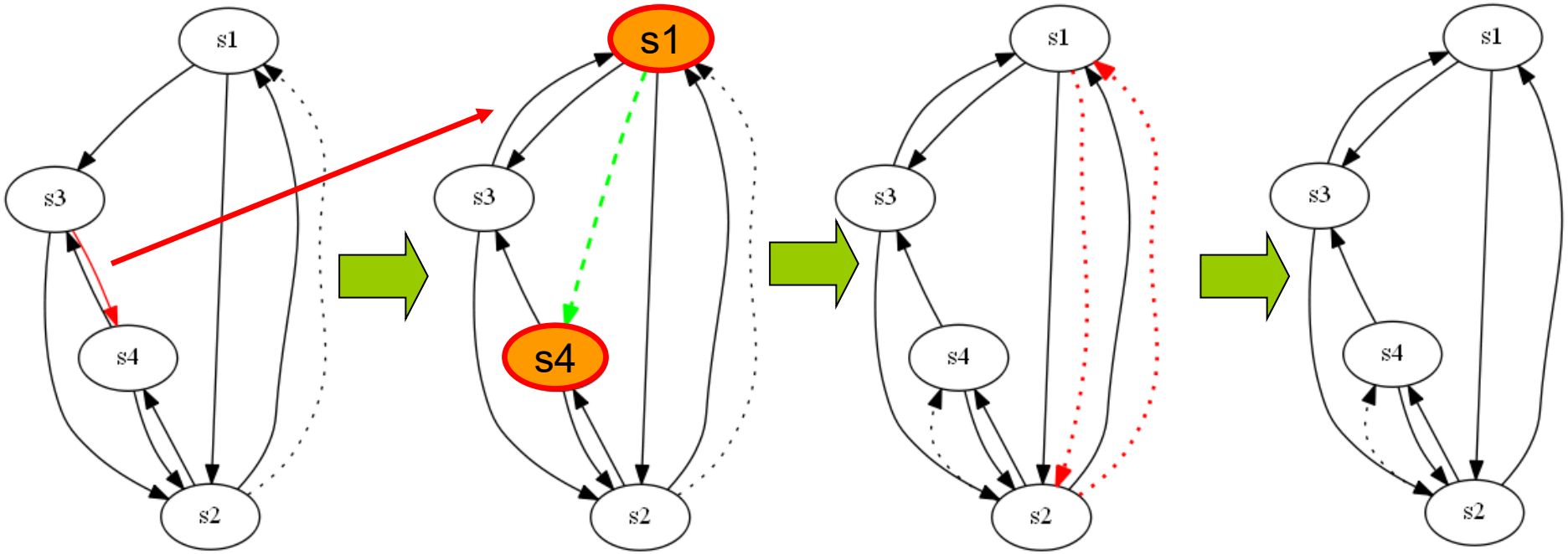
Representing changes in FSMs

- Available atomic changes on graphs:
  - Remove/add isolated nodes
  - Remove/add edge
  - Remove or change the label of an edge
- FSM fault models:
  - Output fault
  - Transfer fault
  - Missing/additional states
  - Missing/additional transitions
- Changes on FSMs based on fault models:
  - Output change
  - Next state change

  Keep completely specified and deterministic FSMs completely specified and deterministic
  - Remove states (and transitions leading to or originating from the removed state) /add states (with one incoming and one outgoing transition)
  - Remove/add transitions

Incremental TT-method

1. Keep the graph Eulerian: balance the nodes that become unbalanced after the change
   - Create a virtual edge
   - Substitute it step by step to extra edges (using breadth-first search)
   - Eliminate cycles over extra edges



Incremental TT-method

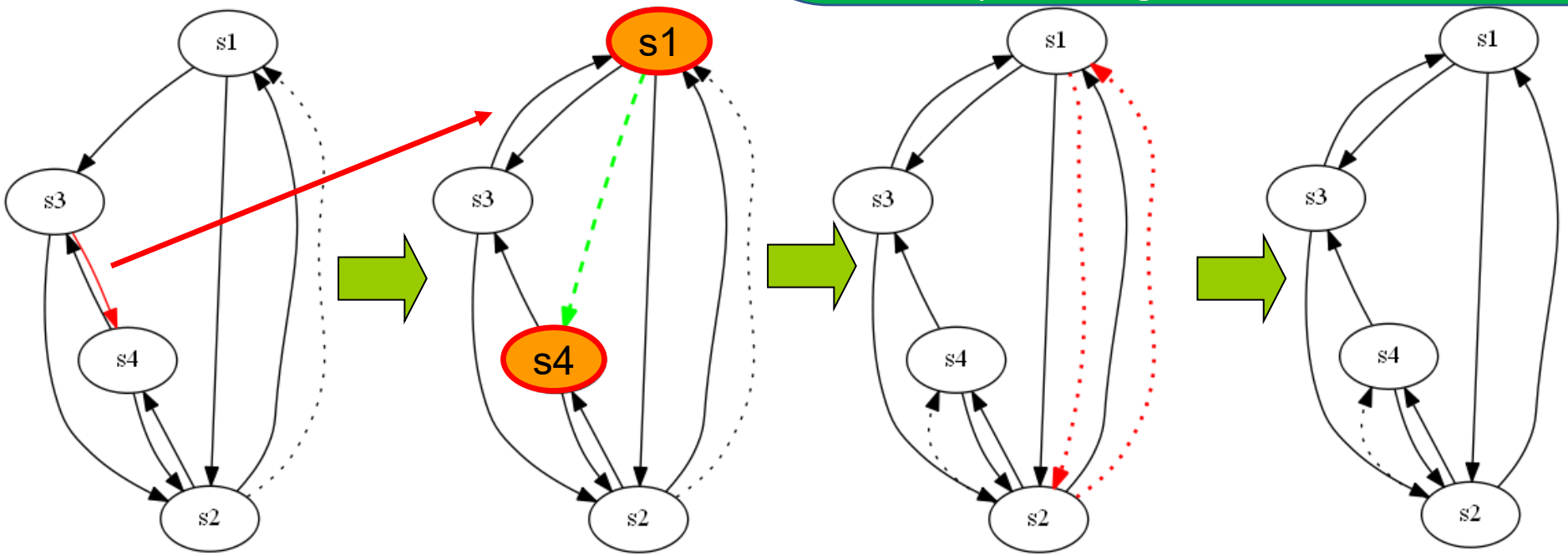
1. Keep the graph Eulerian: balance the nodes t
   - Create a virtual edge
   - Substitute it step by step to extra edges
   - Eliminate cycles over extra edges

Why cycles over extra edges can be eliminated?

- Each extra edge (shown with dotted lines) is a duplication of an edge (transition) in the original graph (FSM)
- Thus each edge (transition) of the original graph (FSM) will be covered the Euler tour and the length of this tour can be reduced with the elimination of cycles formed by extra edges

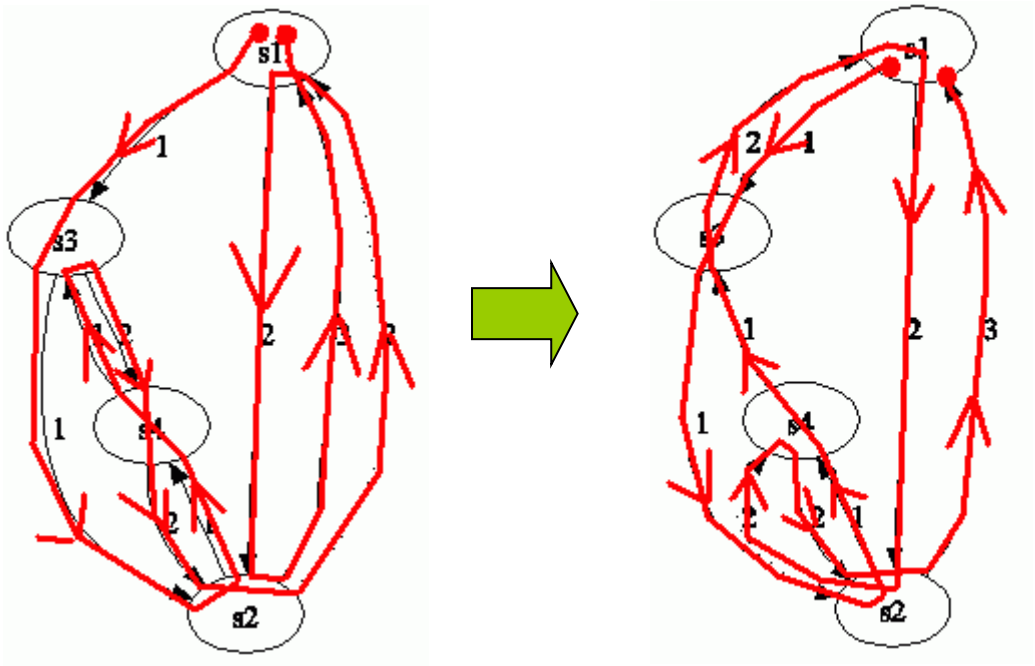
How?

- Identify SCC (strongly connected components) of the graph formed by extra edges using Tarjan's algorithm
- run depth-first search in each SCC and to delete cycles formed by extra edges



Incremental TT-method

2. The maintenance of the Euler tour can be reduced to a maintenance of a spanning tree (when using next-node representation form)



Incremental TT-method

Required assumptions (both for batch and incremental)

- Deterministic, strongly connected specification FSM
- *Impl* does not change during test
- *Impl* has no more states than specification FSM

Handled types of changes

- Remove/add transition
- Remove/add state
- Output change
- Next state change

Complexity Calculation (test generation)

Traditional, batch TT solutions:

$O(n^3+m+\chi)$ or

$O(m(n^2+m)+\chi)$

Incremental TT*:*

$O(\rho(m+\chi))$

Complexity Calculation (length of test sequence)

Traditional, batch TT solutions:

$O(m+\chi)= O(m)$

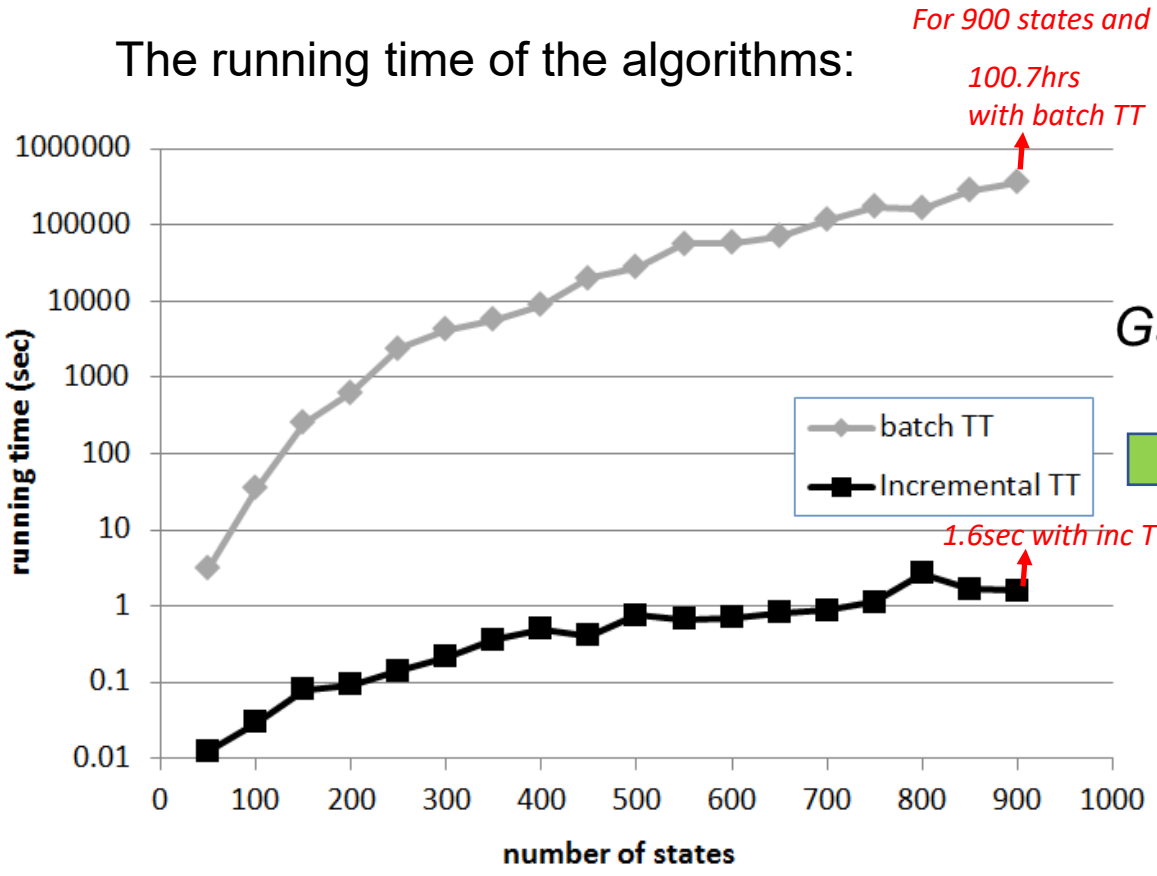
Incremental TT*:*

$O(m+\chi) = O(m)$

n: number of states (nodes)
m: number of transitions (edges)
χ*:* number of extra instances of transitions
ρ*:* number of changed transitions

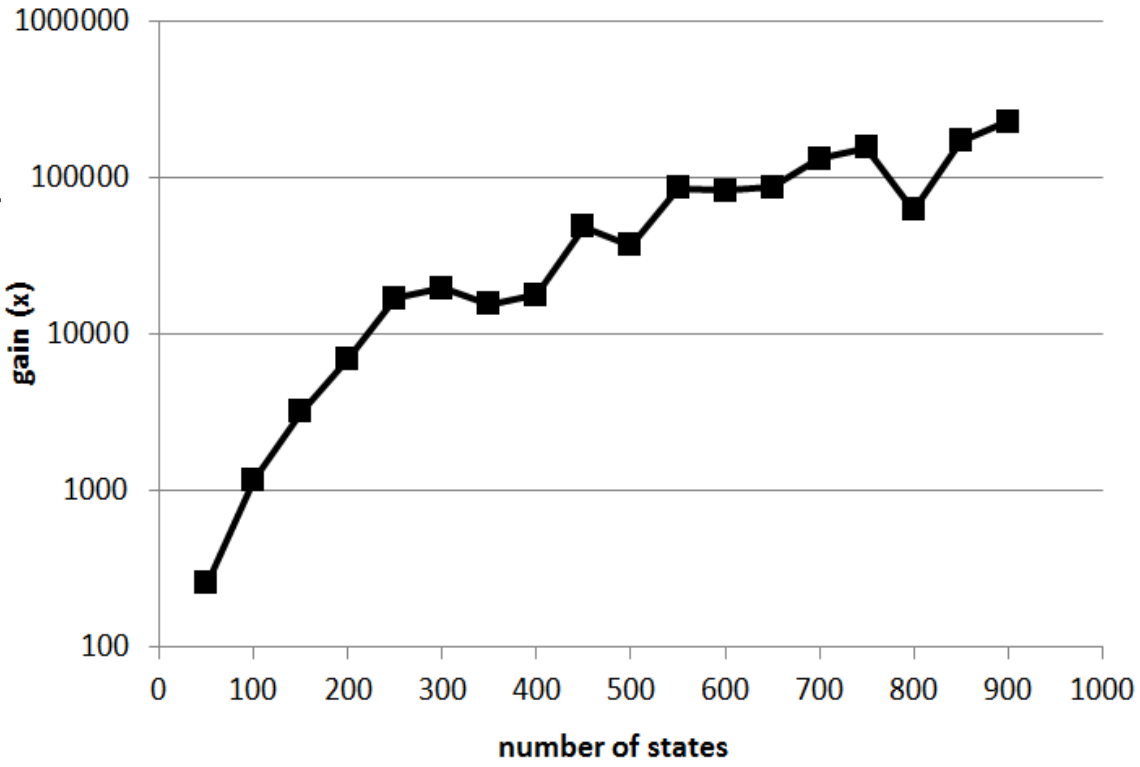
Incremental TT-method

The running time of the algorithms:



$$Gain=\frac{t_{batch}}{t_{inc}}$$

The Gain of the algorithms:



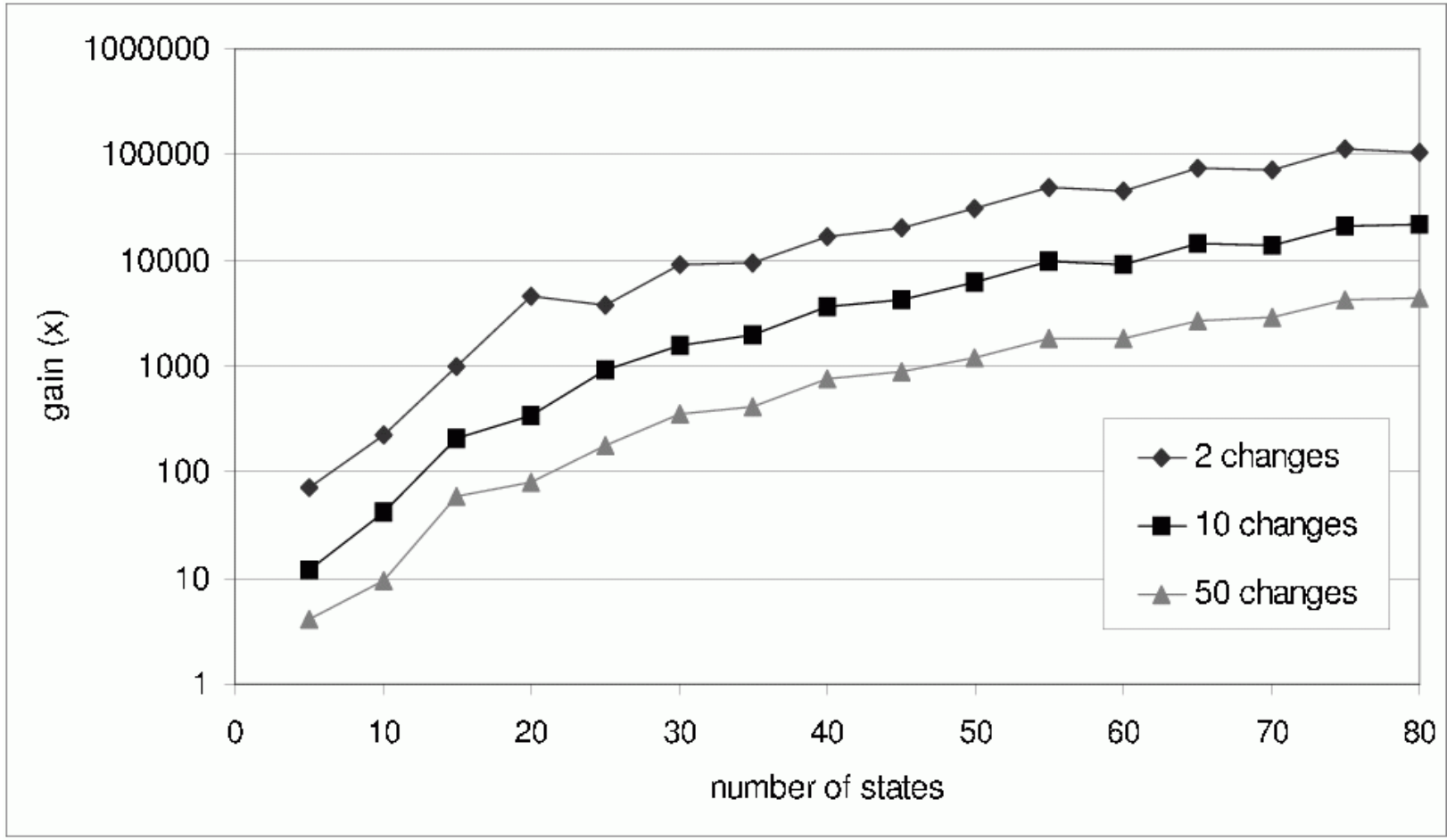
Average number of edges originating from a node: 5, Number of states: between 50 & 900
Number of edge changes: 10

operating system: Linux Gentoo 3.4.6, CPU: AMD Opteron 246 processor with 1993 MHz clock frequency, 1024 KB cache Memory: 4 GB

Incremental TT-method

The Gain of the algorithms in more dense graphs:

$$Gain=\frac{t_{batch}}{t_{inc}}$$

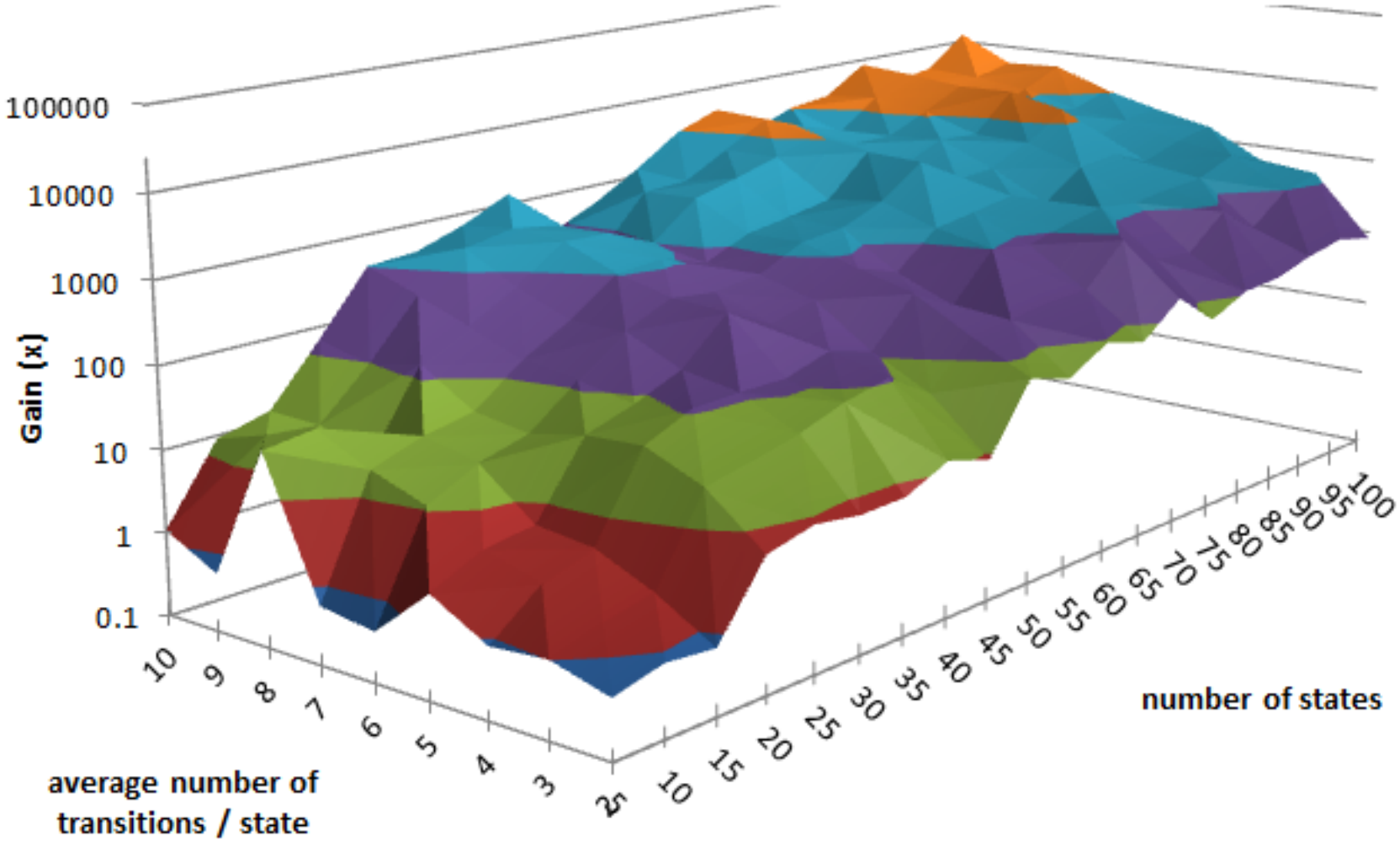


Average number of edges originating from a node: 25, Number of states: between 5 & 150
Number of edge changes: 2, 10 & 50

Incremental TT-method

Gain of incremental TT algorithm compared to batch TT:

$$Gain = \frac{t_{batch}}{t_{inc}}$$

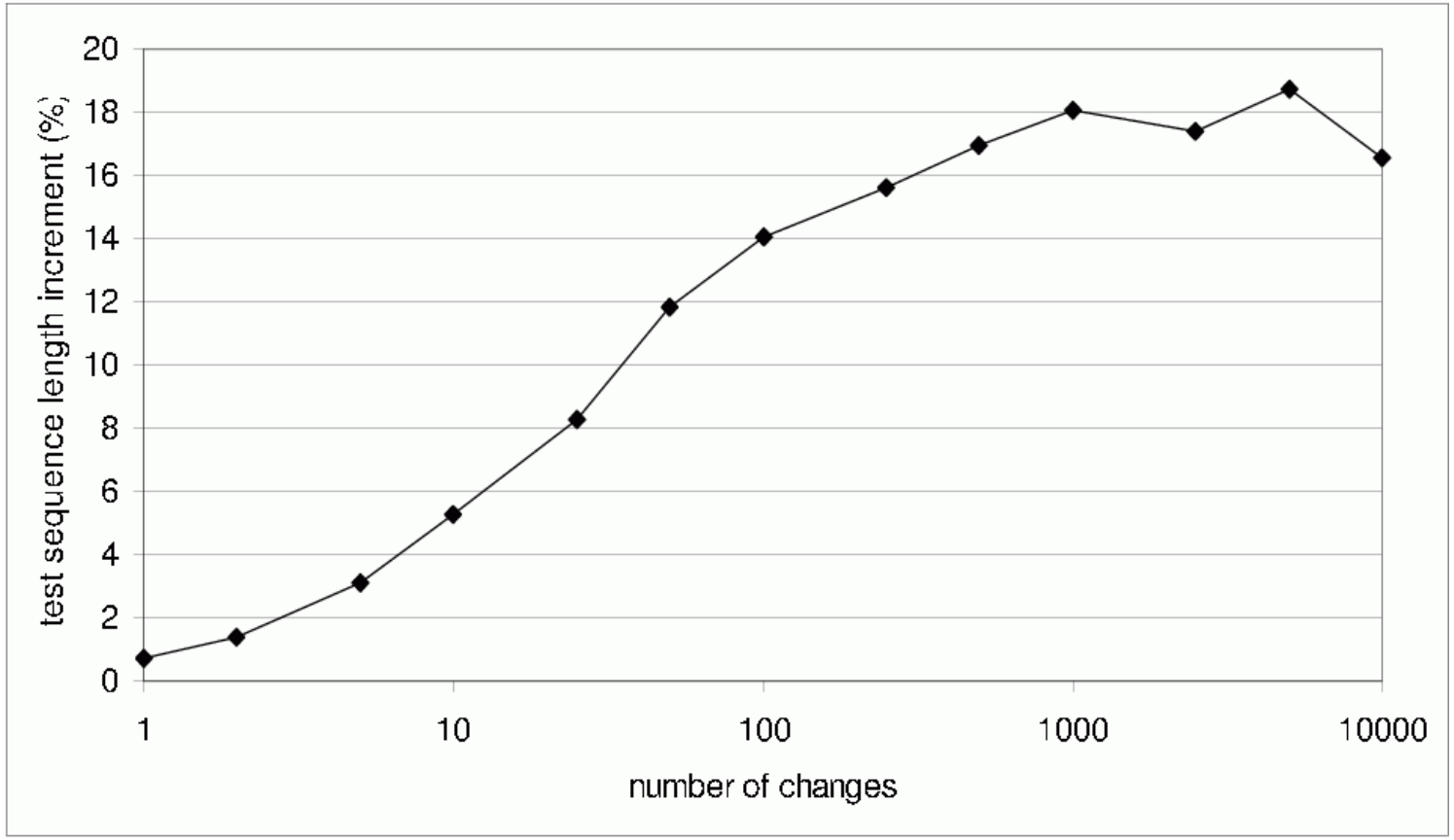


Number of edge changes: 10

As the size of the system increases, the gain of the incremental method compared to the batch increases as the complexity of incremental method depends only on the extent of change

Incremental TT-method

Difference in test sequence length from optimal solution:

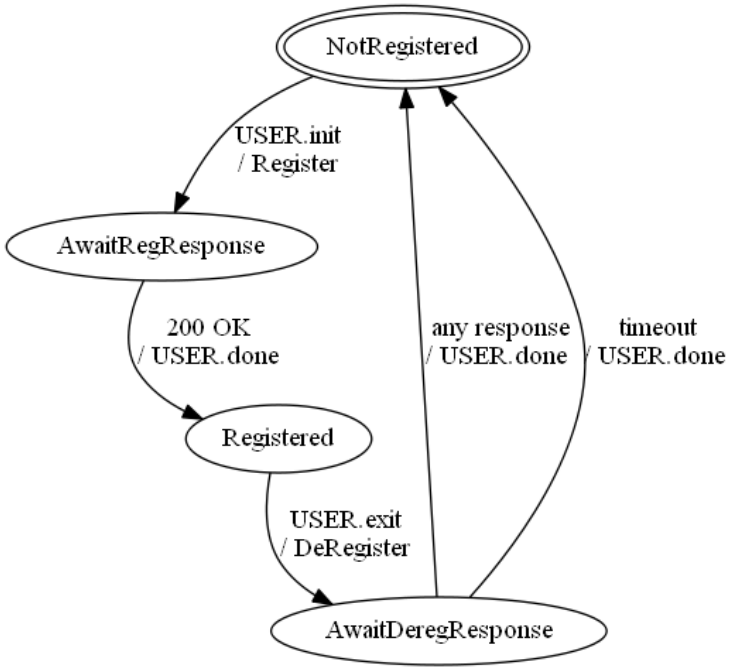


Number of states: 50, Number of edges: 250,
(Average number of edges originating from a node: 5),
Number of edge changes: 1-10.000

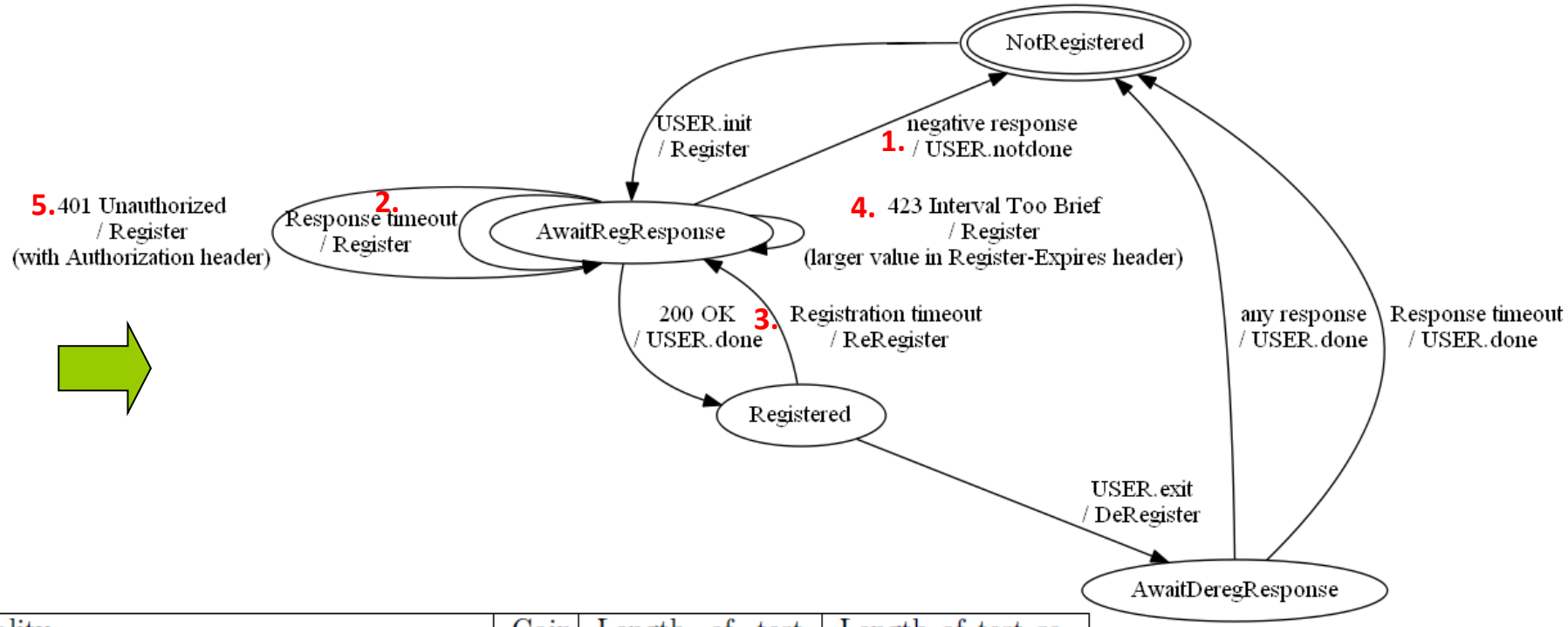
Incremental TT-method

An application example: Continuously evolving specification of SIP (RFC 3261): Registration of UAC

I. EFSM for the easy, all-successful way of working:



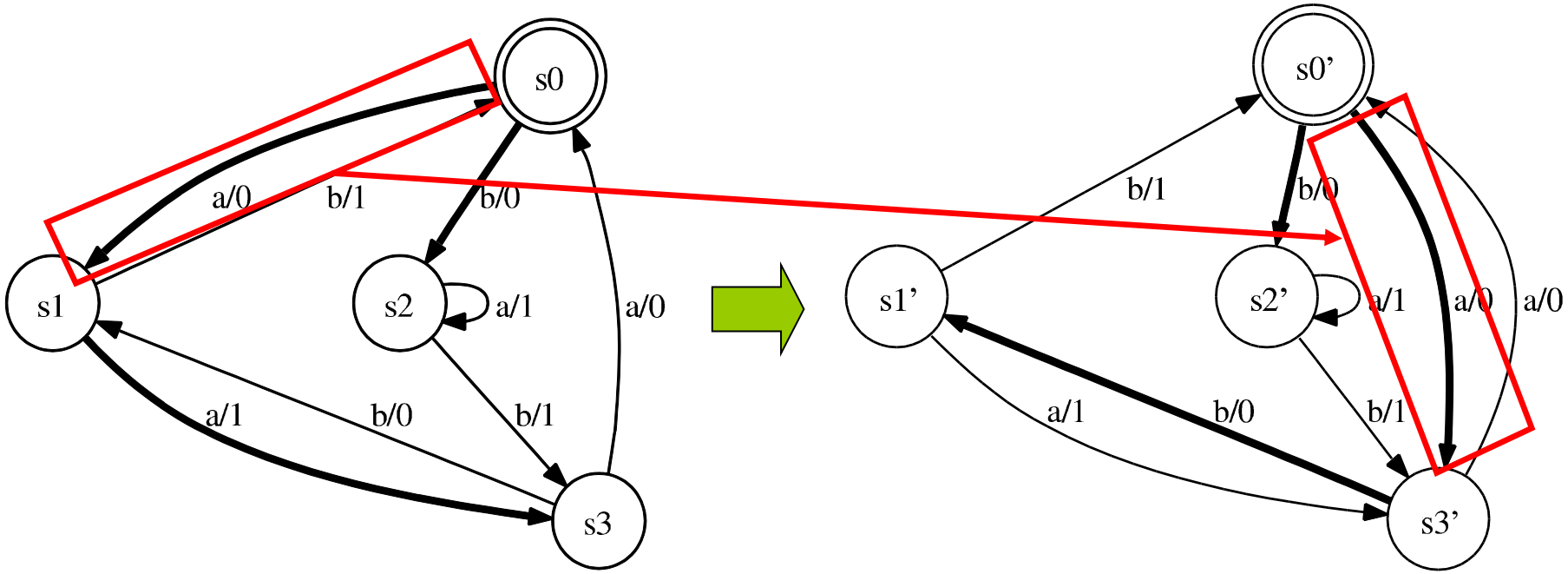
II. Extended EFSM with alternative behaviours:



Added functionality	Gain (x)	Length of test sequence (batch algorithm)	Length of test sequence (incremental algorithm)
1. Handle negative responses for registrations requests	1.6	10	10
2. Silent discard	3.6	11	11
3. Reregister functionality	1.5	13	13
4. Interval too Brief	3.4	14	14
5. Handle of authentication	3.7	15	15

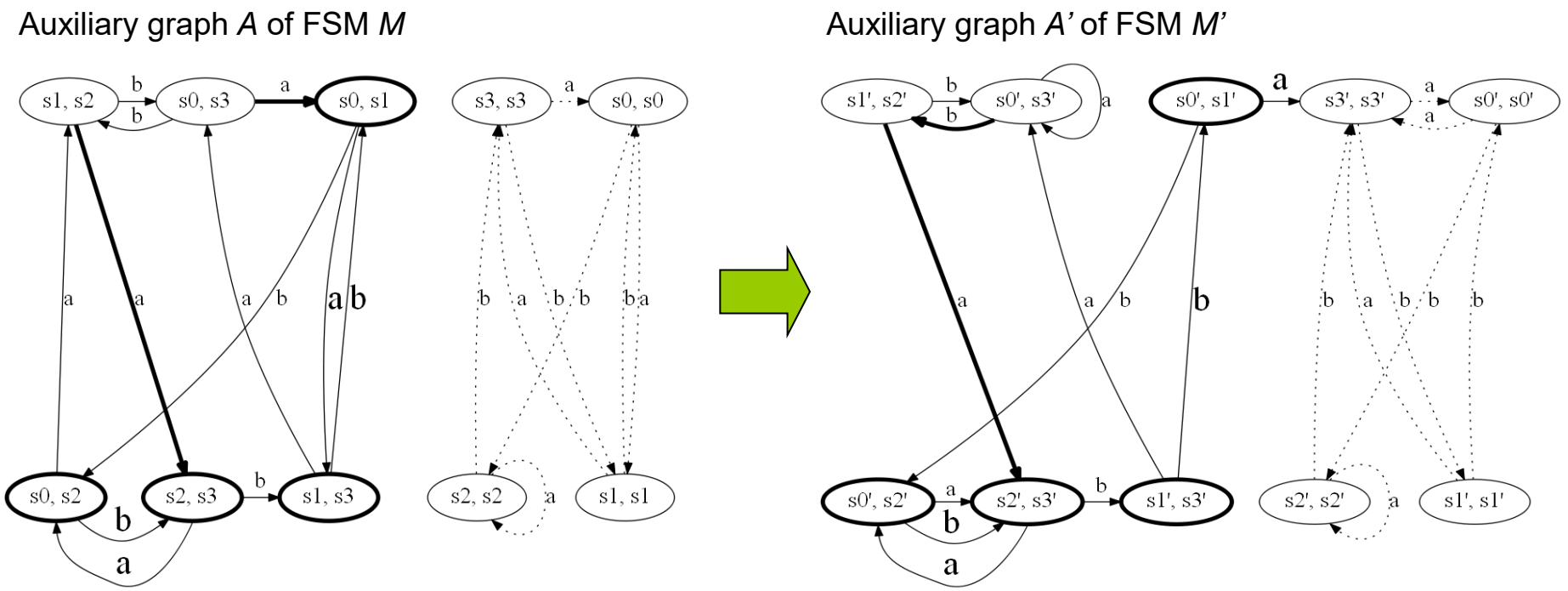
Incremental HSI/HIS-method

- The maintenance of the State Cover Set can be reduced to the maintenance of a spanning tree from s_0 initial state



Incremental HSI/HIS-method

- The maintenance of the separating family of sequences can be reduced to the maintenance of a spanning forest over an auxiliary state pair graph



Incremental HSI/HIS-method

Required assumptions (both for batch and incremental)

- Deterministic, **completely specified**, strongly connected, **minimized** specification FSMs **with reliable reset**
- *Impl* does not change during test, *Impl* has no more states than specification FSM

Handled types of changes

- Output change
- Next state change

Complexity Calculation (test generation)

First algorithm – to reach all states:

- Traditional, batch HSI: $O(p*n)$
- Incremental HSI: $O(p*\Delta_Q)$, where $1 \le \Delta_Q \le n$

Second algorithm – to verify all states:

- Traditional, batch HSI: $O(p*n^2)$
- Incremental HSI: $O(p*\Delta_Z)$, where $n \le \Delta_Z \le n^2$

$p=|I|$ (number of input symbols)
$n=|S|$ *(number of states)*

Δ_Q *:* extent of changes respect to state cover set Q

Δ_Z *:* extent of changes respect to separating family of sequences set Z

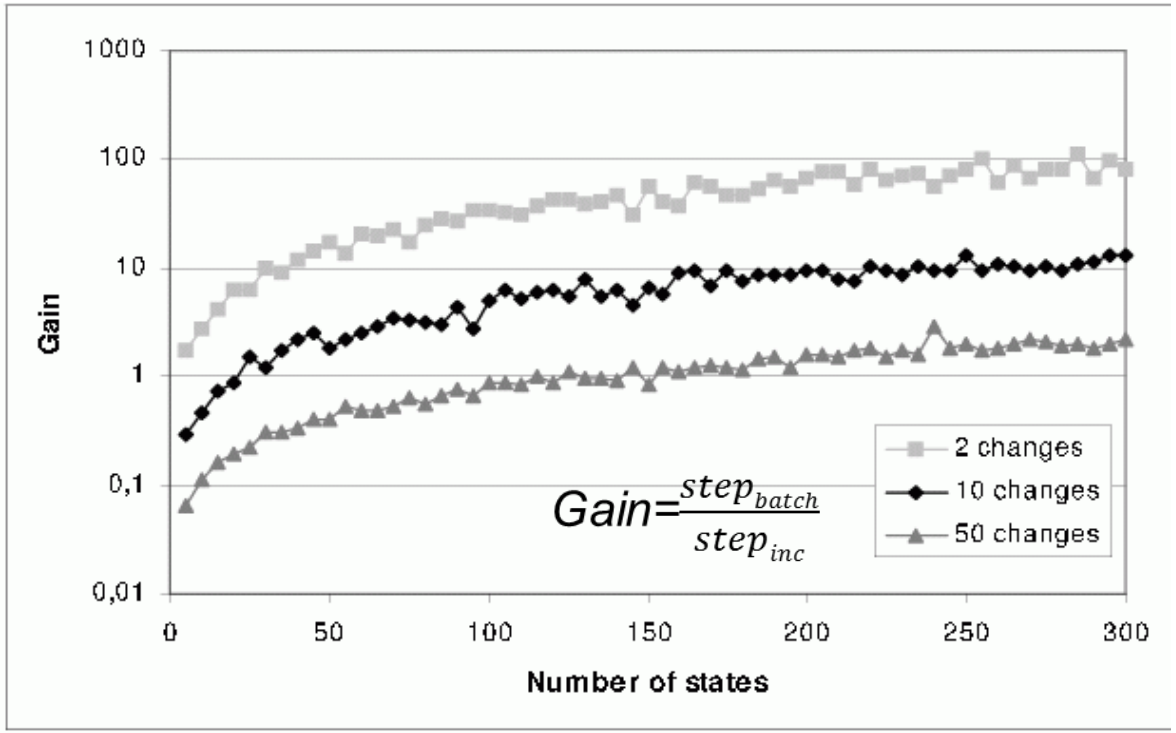
Complexity Calculation (overall length of test sequences)

- Traditional, batch HSI (checking sequence): $O(p*n^3)$
- Incremental HSI (checking sequence): $O(p*n^3)$,
- Incremental HSI (change focused testing): $O(p*\Delta_Q*n^2)+O(2*p*\Delta_Z)= O(p*\Delta*n^2)$, where $1 \le \Delta_Q \le n$, $n \le \Delta_Z \le n^2$ and $1 \le \Delta \le n$

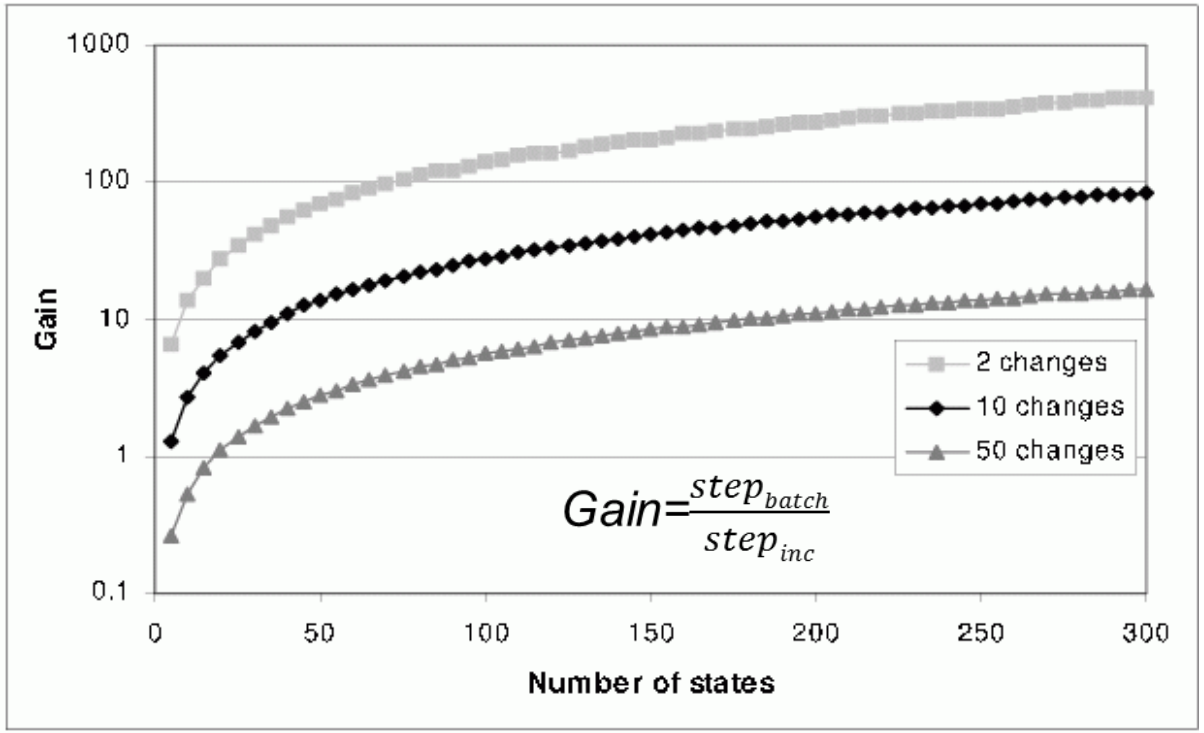
Incremental HSI/HIS-method

Gain of incremental HSI algorithm compared to batch HIS/HSI:

Generating State Cover Set (to reach states):



Generating separating family of sequences (to verify end state):

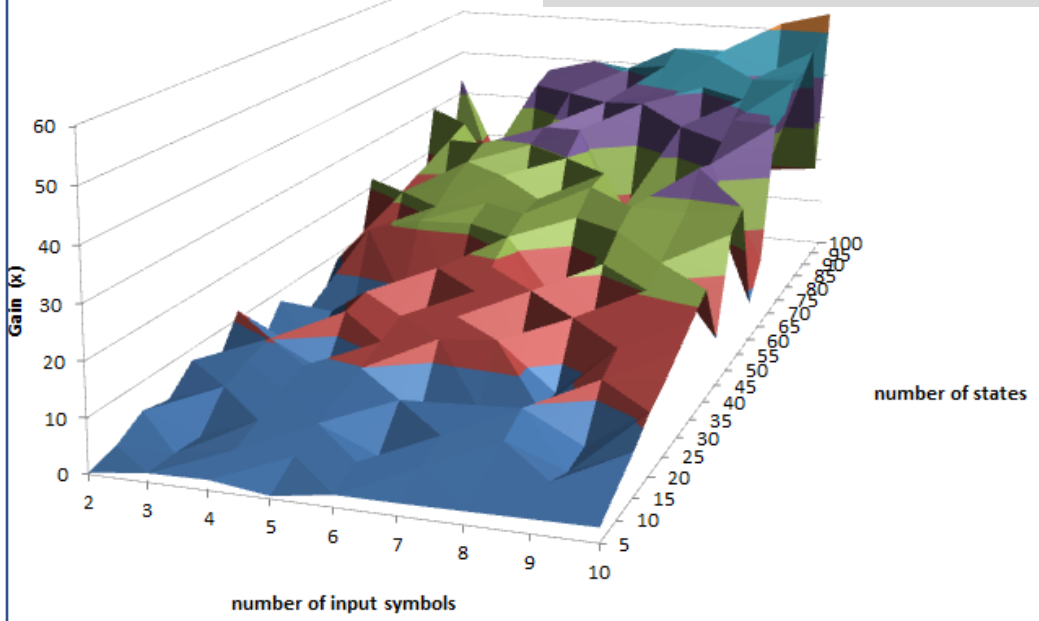


Number of inputs: 5, number of output symbols: 5, number of states: between 5 & 300
*Number of **next state changes**: 2, 10 & 50*

Incremental HSI/HIS-method

$Gain=\frac{step_{batch}}{step_{inc}}$ of incremental HSI algorithm compared to batch HIS/HSI:

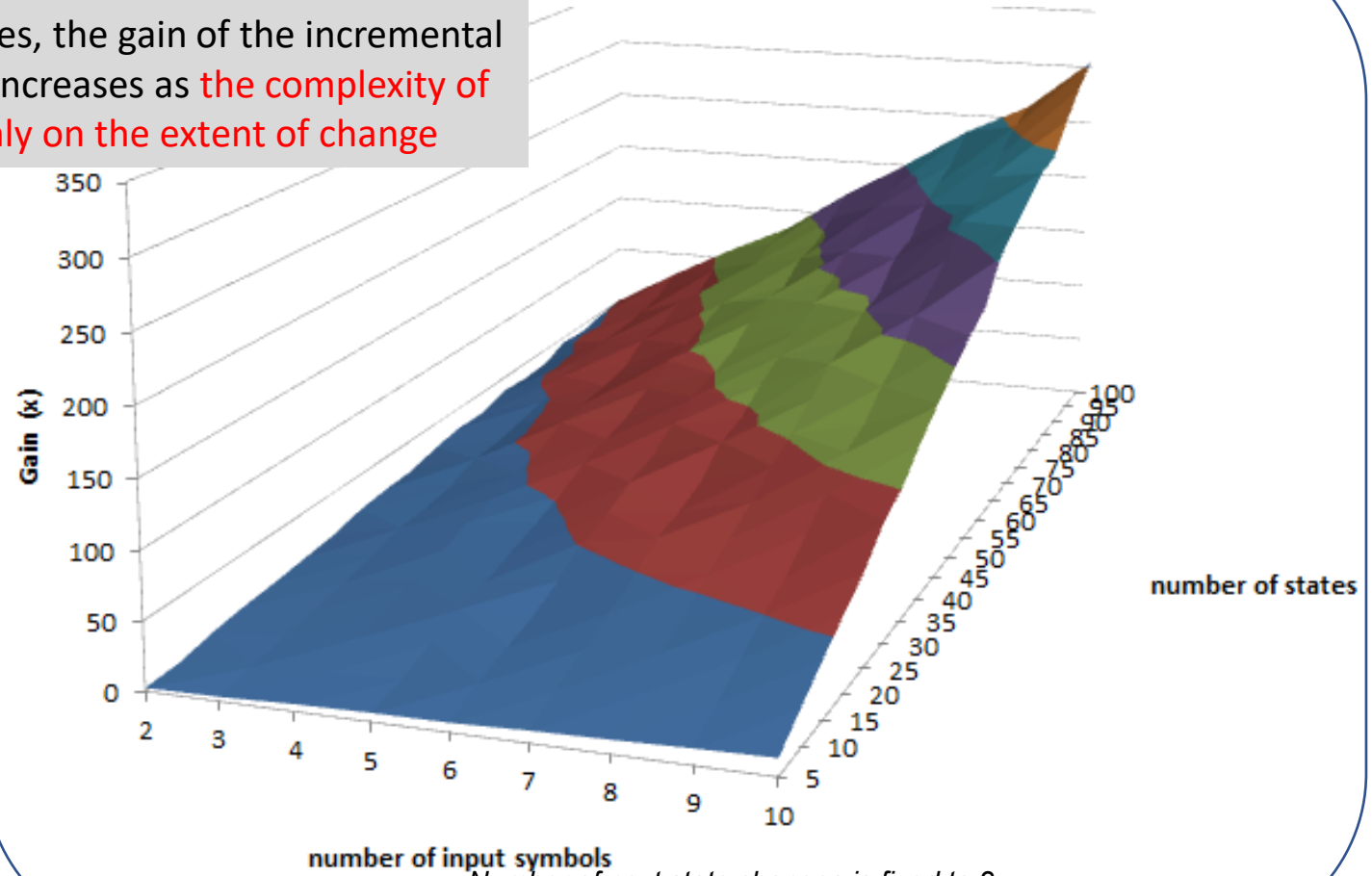
Generating State Cover Set (to reach states):



Number of next state changes is fixed to 2

As the size of the system increases, the gain of the incremental method compared to the batch increases as the complexity of incremental method depends only on the extent of change

Generating separating family of sequences (to verify end state):

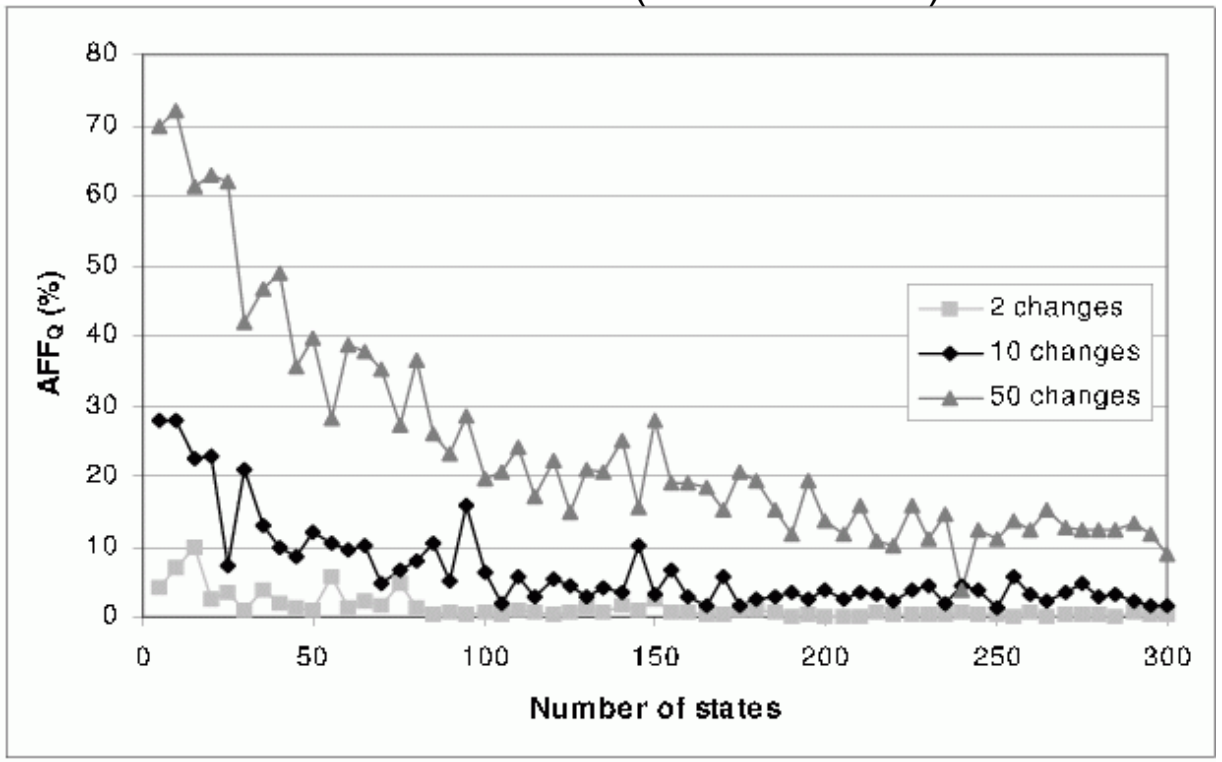


Number of next state changes is fixed to 2
Number of output symbols is fixed to 2

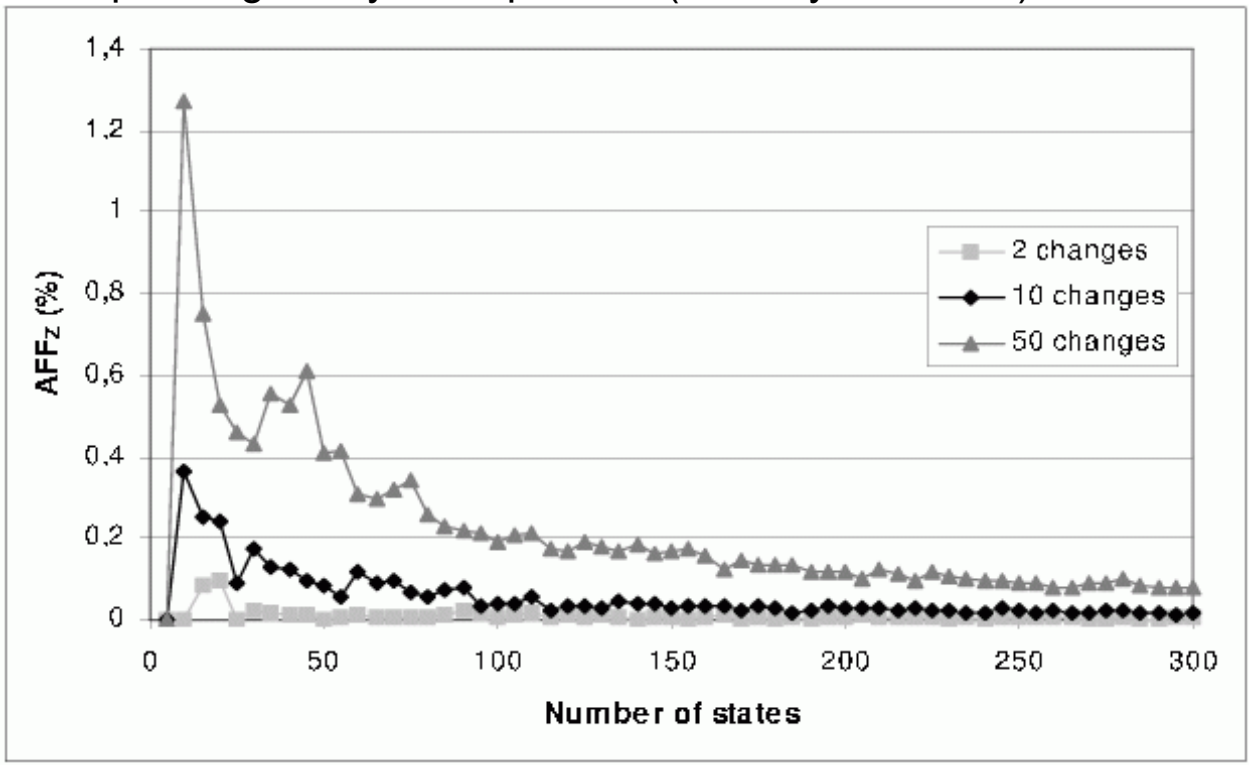
Incremental HSI/HIS-method

Percentage amount of affected elements (i.e. where new test sequence has to be generated):

In State Cover Set (to reach states):



In separating family of sequences (to verify end state):

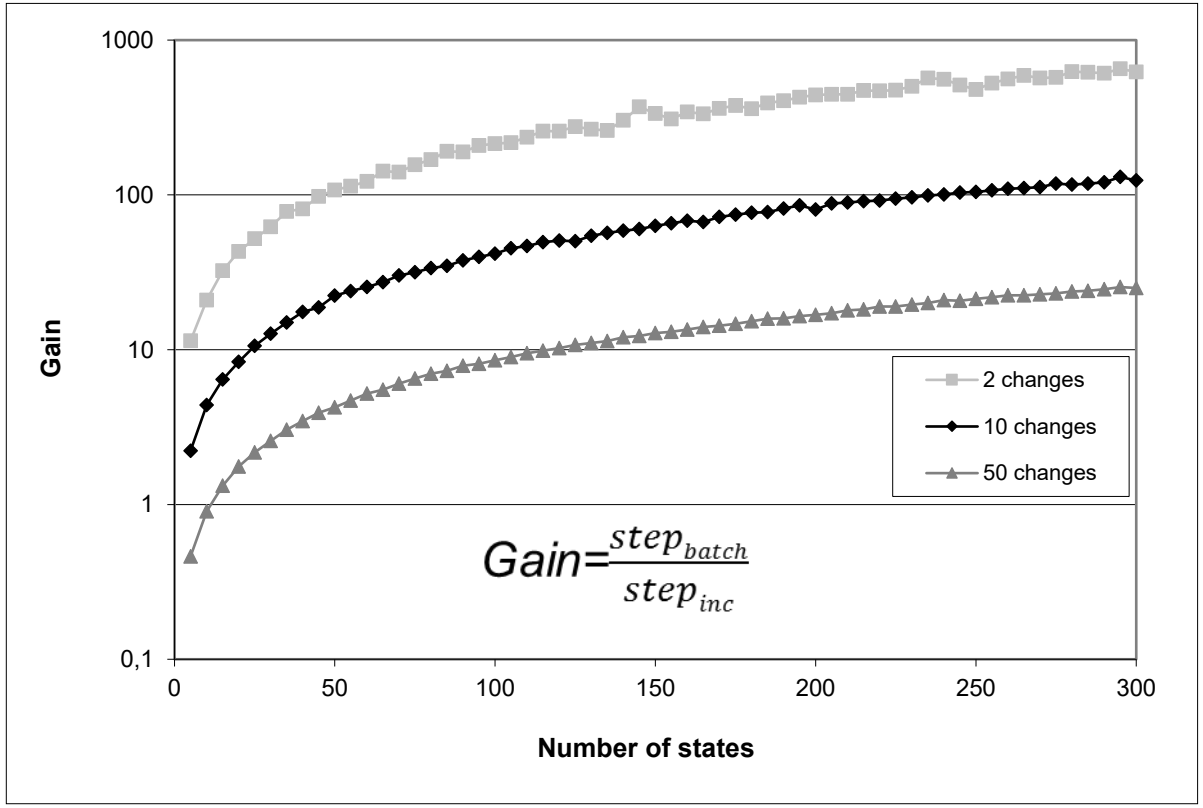


Number of inputs: 5, number of output symbols: 5, number of states: between 5 & 300
*Number of **next state changes**: 2, 10 & 50*

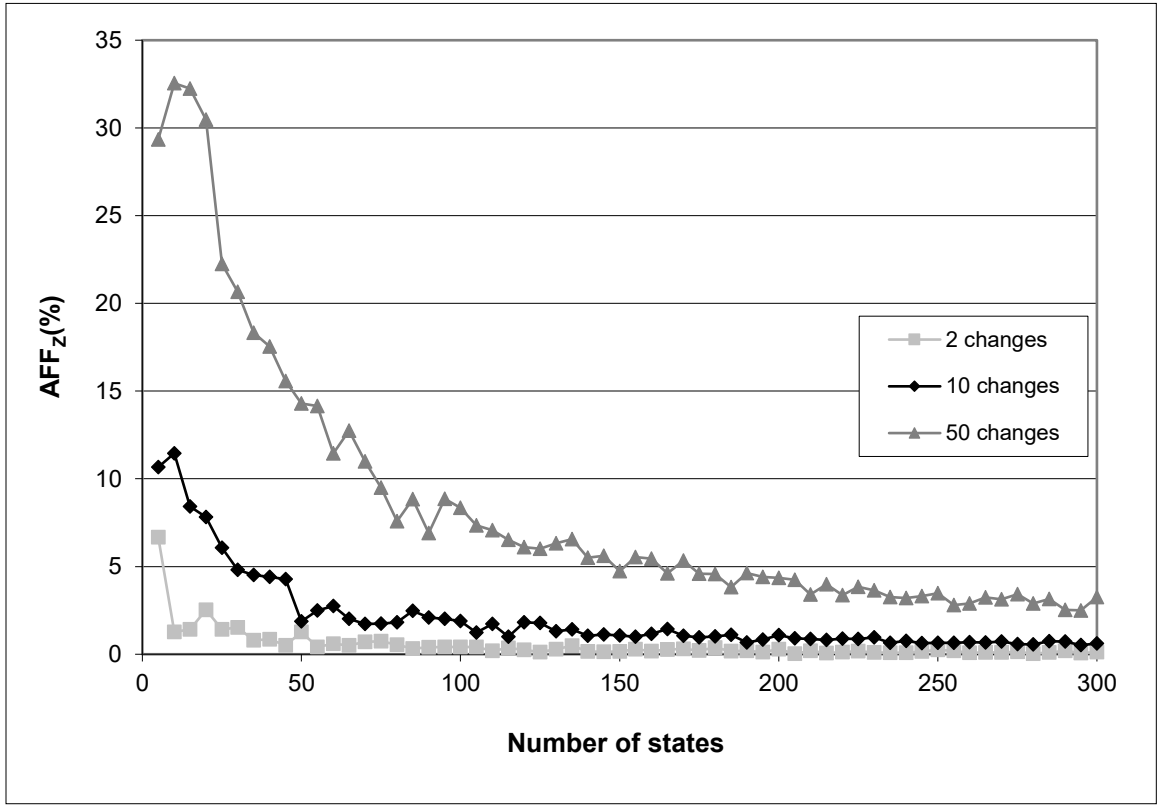
Incremental HSI/HIS-method

In separating family of sequences (to verify end state):

Gain of incremental HIS/HSI algorithm compared to batch HIS/HSI:



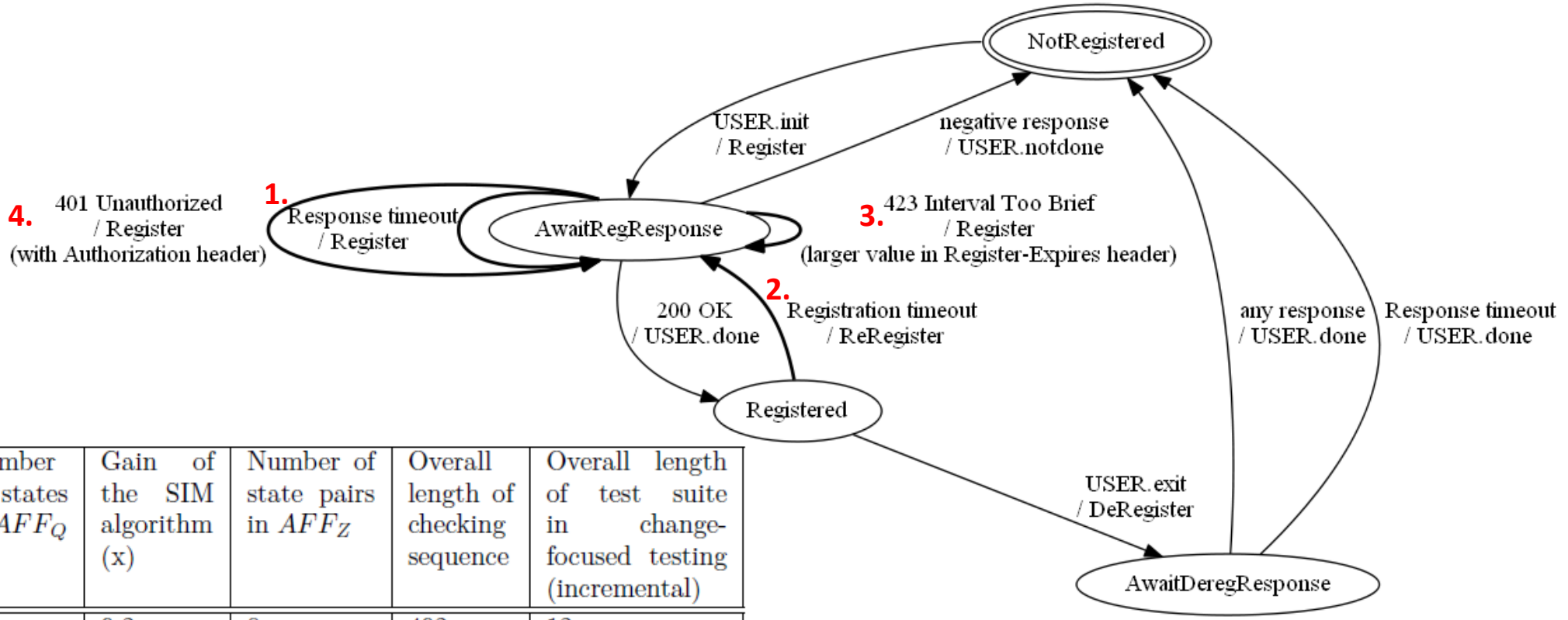
Percentage amount of affected elements (i.e. where new test sequence has to be generated):



Number of inputs: 5, number of output symbols: 5, number of states: between 5 & 300
*Number of **output changes**: 2, 10 & 50*

Incremental HSI/HIS-method

An application example: Continuously evolving specification of SIP (RFC 3261): Registration of UAC



Modified functionality	Gain of the SCSM algorithm (x)	Number of states in AFF_Q	Gain of the SIM algorithm (x)	Number of state pairs in AFF_Z	Overall length of checking sequence	Overall length of test suite in change-focused testing (incremental)
1. Silent discard	3.6	0	9.2	0	492	12
2. Reregister functionality	3.6	0	9.7	0	492	15
3. Interval too Brief	3.6	0	8.9	0	492	12
4. Handle of authentication	3.6	0	8.8	0	492	12

Mutation testing (MT)

- *White-box* technique
- Faulty system versions – *mutants* – are created by applying simple modifications on the SUT that represents typical programming errors
- Check if a test can detect the modification from the SUT:
  - If yes: it *kills* a given mutant,
  - If no: the mutant remains *live*
- *Mutation score* shows the ratio of the number of killed and *non-equivalent* mutants
- MT relies on the *competent programmer hypothesis* and the *coupling effect*
  - *Competent programmer hypothesis:* Experienced programmers tend to "create programs that are close to being correct"
    (i.e. it differs from the specification with relatively simple syntax and semantic faults)
  - *Coupling effect*: Those tests that identify all simple faults probably uncover more complex ones too

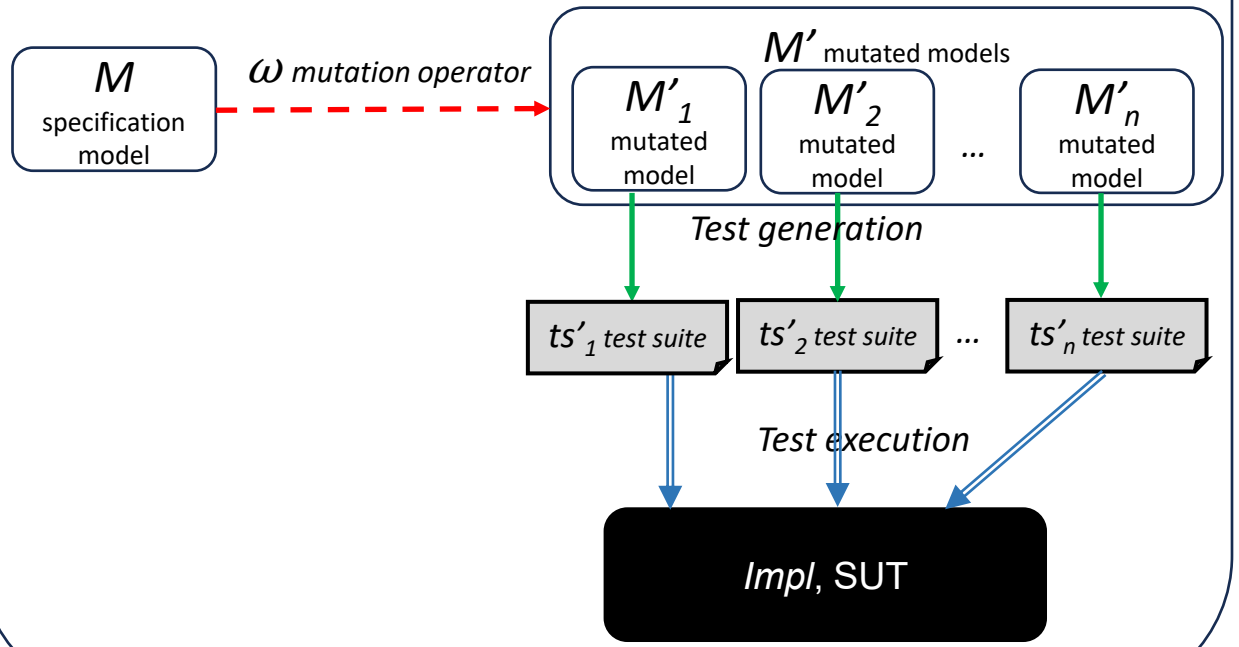
Model-based mutation testing (MBMT)

- The SUT is considered as a *black-box* (i.e. its source code can not be mutated)

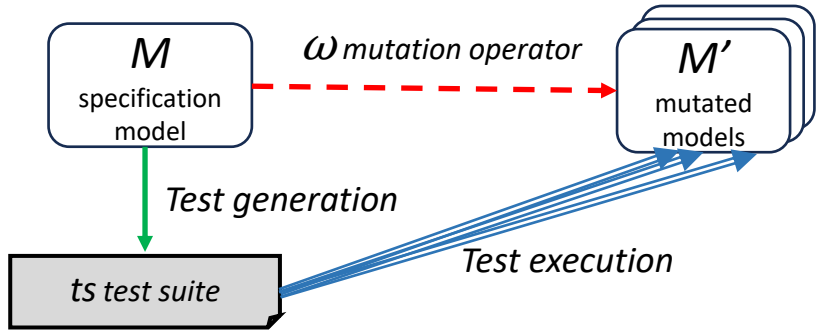
→ Introduce mutation operators in the system specification model

- 2 approaches:

1. Test suites ts'_x are generated from each M'_x mutant model separately and then executed on the SUT



2. Test suites *ts* are generated from the *M* original model separately and then executed on all mutated models *M'*



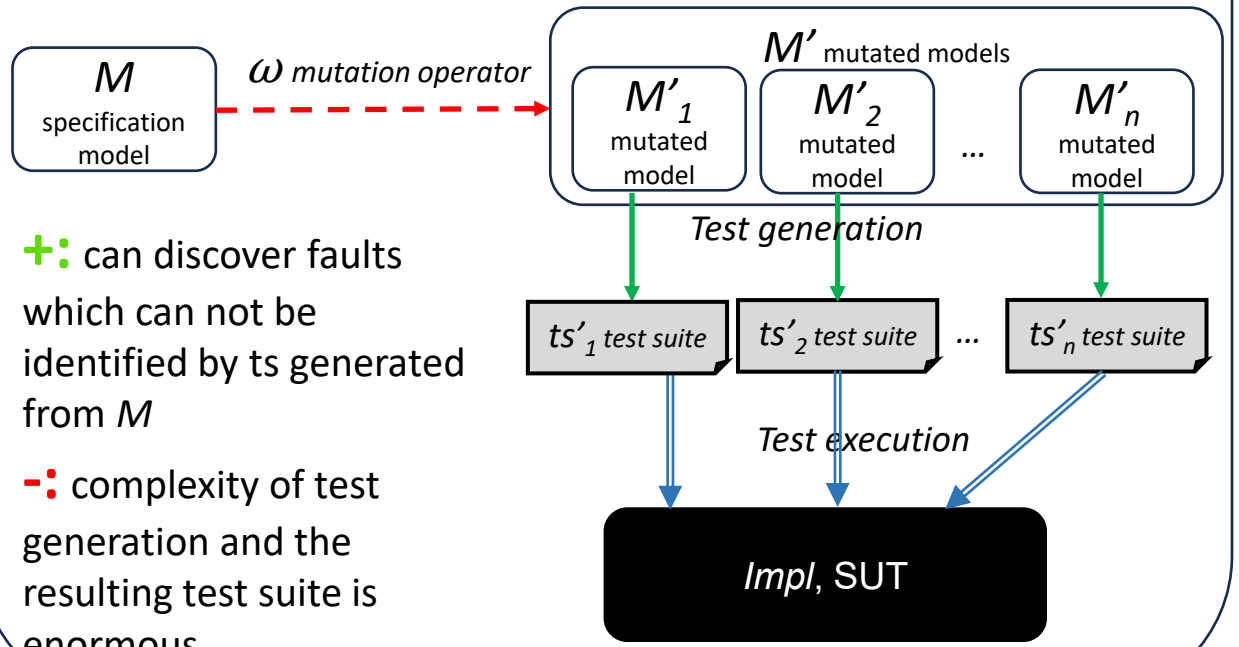
Model-based mutation testing (MBMT)

- The SUT is considered as a *black-box* (i.e. its source code can not be mutated)

→ Introduce mutation operators in the system specification model

- 2 approaches:

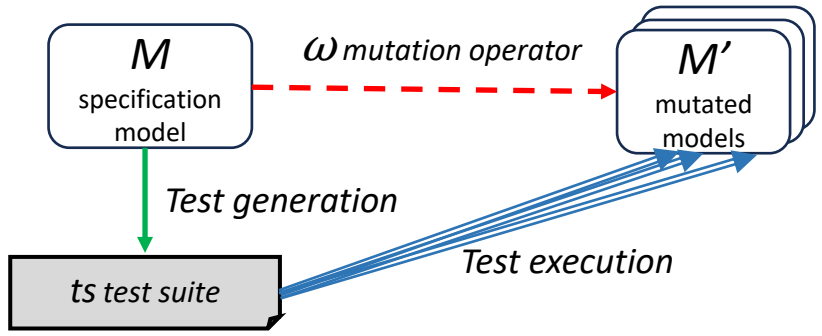
1. Test suites ts'_x are generated from each M'_x mutant model separately and then executed on the SUT



+: can discover faults which can not be identified by ts generated from *M*

-: complexity of test generation and the resulting test suite is enormous

2. Test suites *ts* are generated from the *M* original model separately and then executed on all mutated models *M'*



+: complexity of test generation and the resulting test suite is the fraction of the 1st approach

-: some mutants can not be discovered by *ts* generated from *M*

Model-based mutation testing (MBMT)

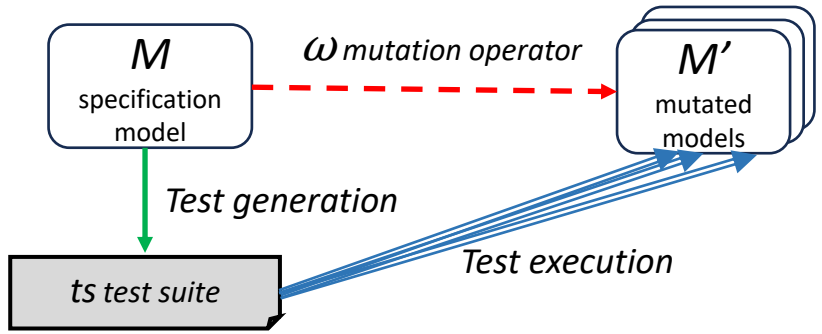
- The SUT is considered as a *black-box* (i.e. its source code can not be mutated)

→ Introduce mutation operators in the system specification model

- 2 approaches:

M▸T▸R The 2nd approach of MBMT is implemented in Model 》 Test 》 Relax framework

2. Test suites *ts* are generated from the *M* original model separately and then executed on all mutated models *M'*



+: complexity of test generation and the resulting test suite is the fraction of the 1st approach

-: some mutants can not be discovered by *ts* generated from *M*

Model-based mutation testing (MBMT)

Model-based mutation testing (2nd approach):

1. Goes through the *Ω* list of mutation operators and generates all possible *M'* mutant models for the given ω_n operator
2. Generates *ts* test suite from *M* according to the *ALG* list of test gen algos and their params
3. Execute *ts* on each mutant model of *M'*
   - If the predefined *target_score* is reached, then at *ts* to the *TS* set of test suites
   - Otherwise continue 2.
4. Return *TS* set of test suites and the actually achieved *mutation_score*

Algorithm 1: Model-based mutation testing (MBMT)

```
   input : M; Ω = {ω_1, .., ω_K}; target_score, ALG
   output: TS; mutation_score
1  T := {}, TS := {}
2  foreach ω_n ∈ Ω do
3      M' := generateMutants(M, ω_n)
4      foreach ts ∈ T do
5          execTestSuite(M', ts)
6          if reachedTarget(M',ts,target_score) and ts
            ∉ TS then
7              TS := TS ∪ {ts}
8      if ∄ ts ∈ T: reached target_score for ω_n then
9          while ¬ reachedTarget(M',ts,target_score)
            ∧ ∃ unused ∈ ALG do
10             alg := First unused element of ALG
11             ts := generateTestSuite(M, alg)
12             T := T ∪ {ts}
13             execTestSuite(M', ts)
14             if reachedTarget(M',ts,target_score)
                then
15                 TS := TS ∪ {ts}
16                 break // goto line 2
17     if ∄ ts ∈ T: reached target_score for ω_n then
18         cts := element of ALG with the highest score
19         TS := TS ∪ {cts}
20 mutation_score := calcMutScore(M, Ω, TS)
21 return TS, mutation_score
```

Gábor Árpád Németh. Model-based mutation testing for Finite State Machine specifications with MTR. Infocommunications Journal, 17(3): 84-91, September 2025. DOI: 10.36244/ICJ.2025.3.10

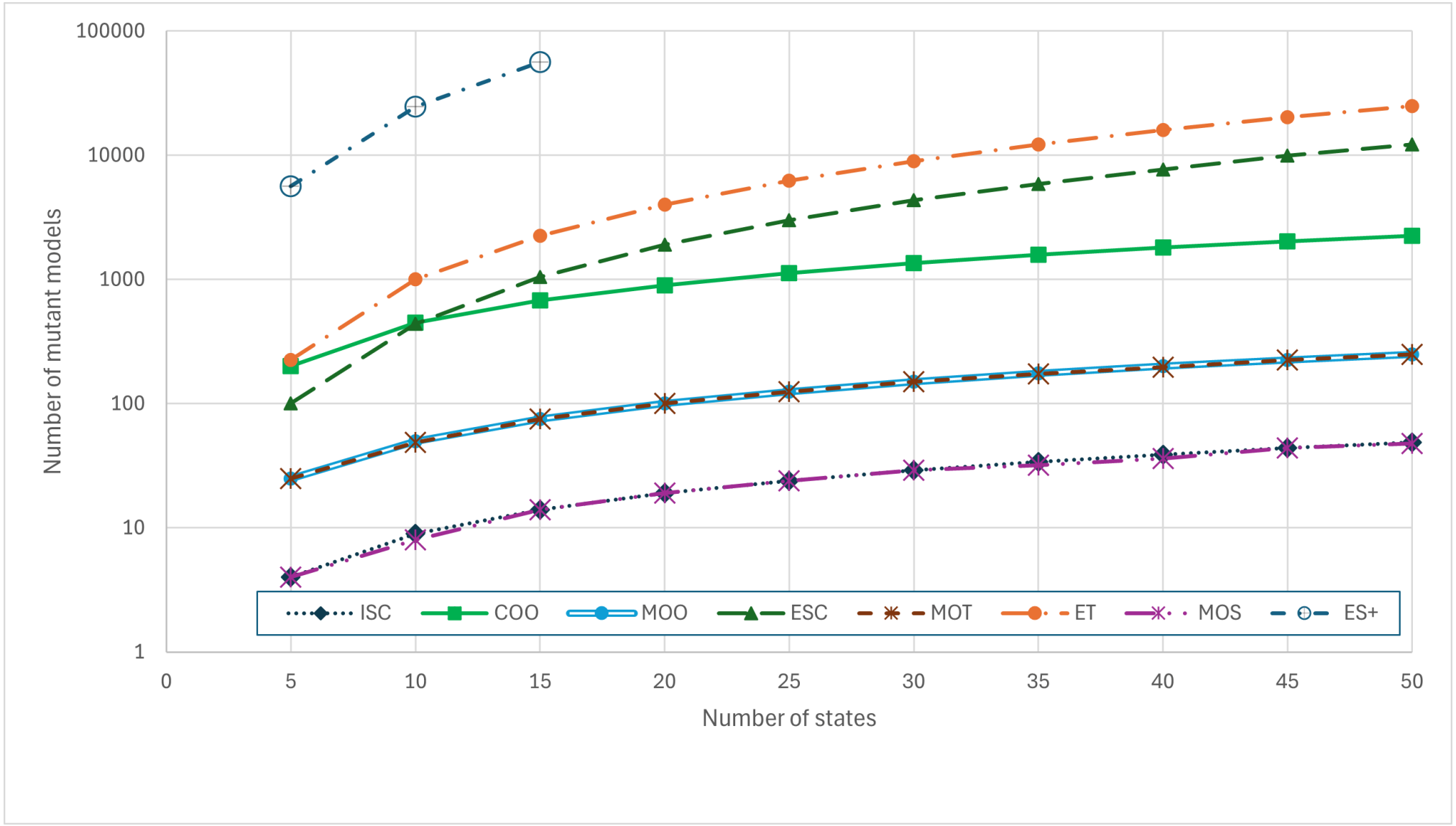
Model-based mutation testing (MBMT)

Used terminology in...			
[4], [6]	[12]	[2], [18]	Notes
-	Wrong-start-state	ISC (Initial State Changed)	-
-	Event-exchanged	COI (Change of Input)	May result in a non-det. FSM
-	Event-missing	MOI (Missing of Input)	Special case of COI. Contradicts to FSM formalism.
Output fault	Output-exchanged	COO (Change of Output)	-
-	Output-missing	MOO (Missing of Output)	Special case of COO
Transfer fault	Destination-exchanged	ESC (End State Changed)	May result in a non-strongly connected FSM
-	-	ROT (Reverse of Transition)	Combination of MOT and Extra Transition
Missing transition	Arc-missing	MOT (Missing of Transition)	May result in a non-strongly connected FSM
Extra transition	-	DOT (Redundant of Transition)	May result in a non-det. FSM
Missing state	State missing	MOS (Missing of State)	Associated transitions are also removed. May result in a non-strongly connected FSM
Extra state	State extra	-	Results in a non-strongly connected FSM
-	-	SSR (Start State Redundant)	Special case of extra state
-	-	ESR (End State Redundant)	Special case of extra state

Mutation operators (ω)**:**

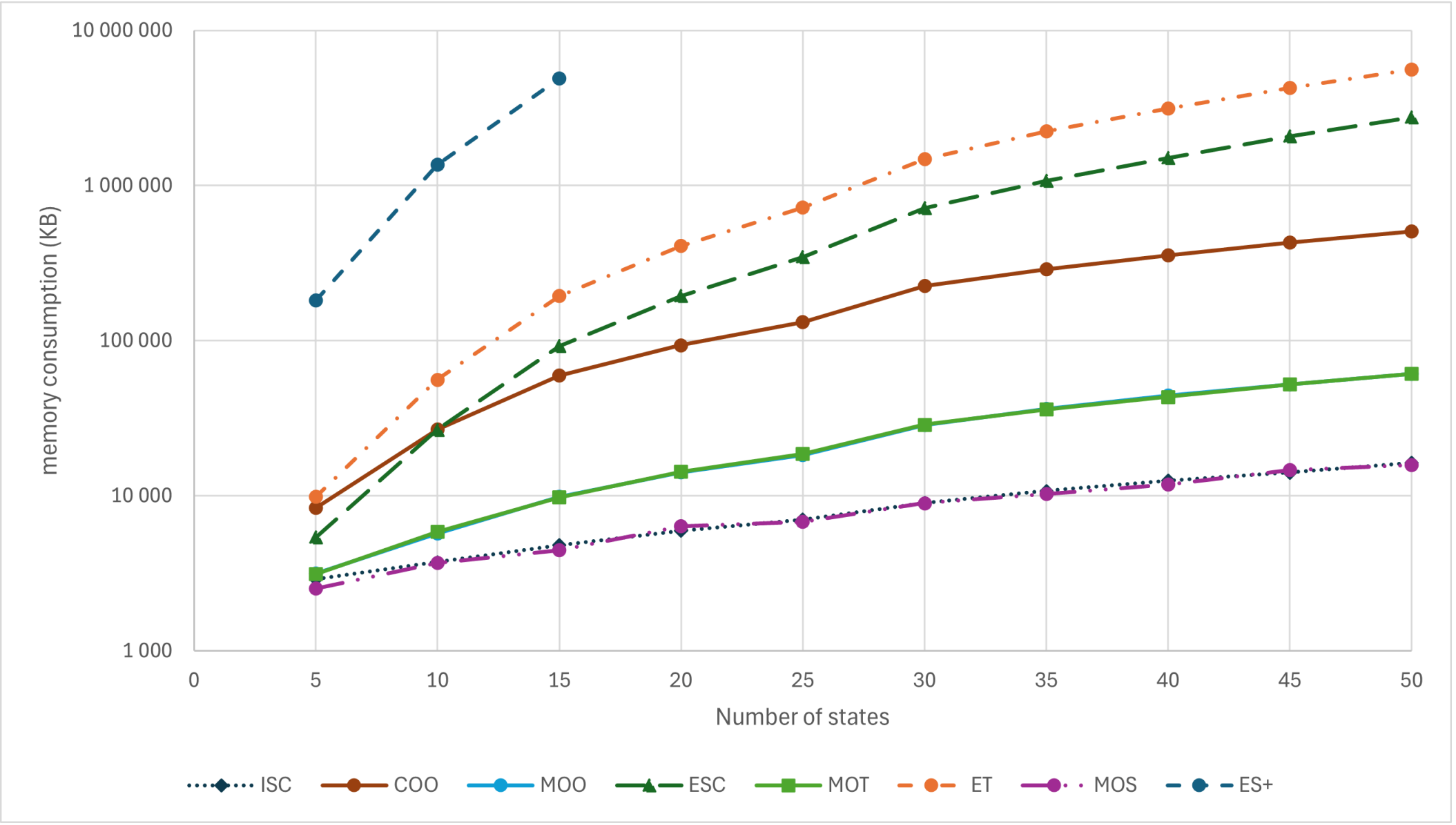
- Initial State Changed (ISC)
- Change of Output (COO)
- Missing of Output (MOO)
- End State Changed (ESC)
- Missing of Transition (MOT)
- Extra Transition (ET)
- Missing of State (MOS)
- Extra State with 1-1 incoming/outgoing transitions (ES+)

Model-based mutation testing (MBMT)



States: 5..50
Input Symbols/Density: 5
Output symbols: 10

Model-based mutation testing (MBMT)



States: 5..50
Input Symbols/Density: 5
Output symbols: 10

Model-based mutation testing (MBMT)

Mutation operator	Mutation scores				
	min	average	median	90th percentile	max
ISC	1	1	1	1	1
COO	1	1	1	1	1
MOO	1	1	1	1	1
ESC	0.96	0.9971	1	0.9921	1
MOT	1	1	1	1	1
ET	0	0	0	0	0
MOS	1	1	1	1	1
ES+	0.9557	0.9937	0.9999	0.9773	1

Mutation score distribution for all mutated models and test generation algorithms

Input Symbols/Density: 5
Output symbols: 10

Model-based mutation testing (MBMT)

Test gen. alg.	Mutation scores for different number of states									
	5	10	15	20	25	30	35	40	45	50
TT	.96	.968	0.969	0.983	0.987	0.992	0.988	0.996	0.996	0.993
ATS0	1	.997	0.999	0.998	0.999	0.999	0.999	0.999	0.999	0.999
HSI	1	1	1	1	1	1	1	1	1	1
HSI (θ=1)	1	1	1	1	1	1	1	1	1	1
1-SC	1	1	1	1	1	1	1	1	1	1
2-SC	1	1	1	1	1	1	1	1	1	1

Mutation scores of different test suites for ESC mutation operator

Input Symbols/Density: 5
Output symbols: 10

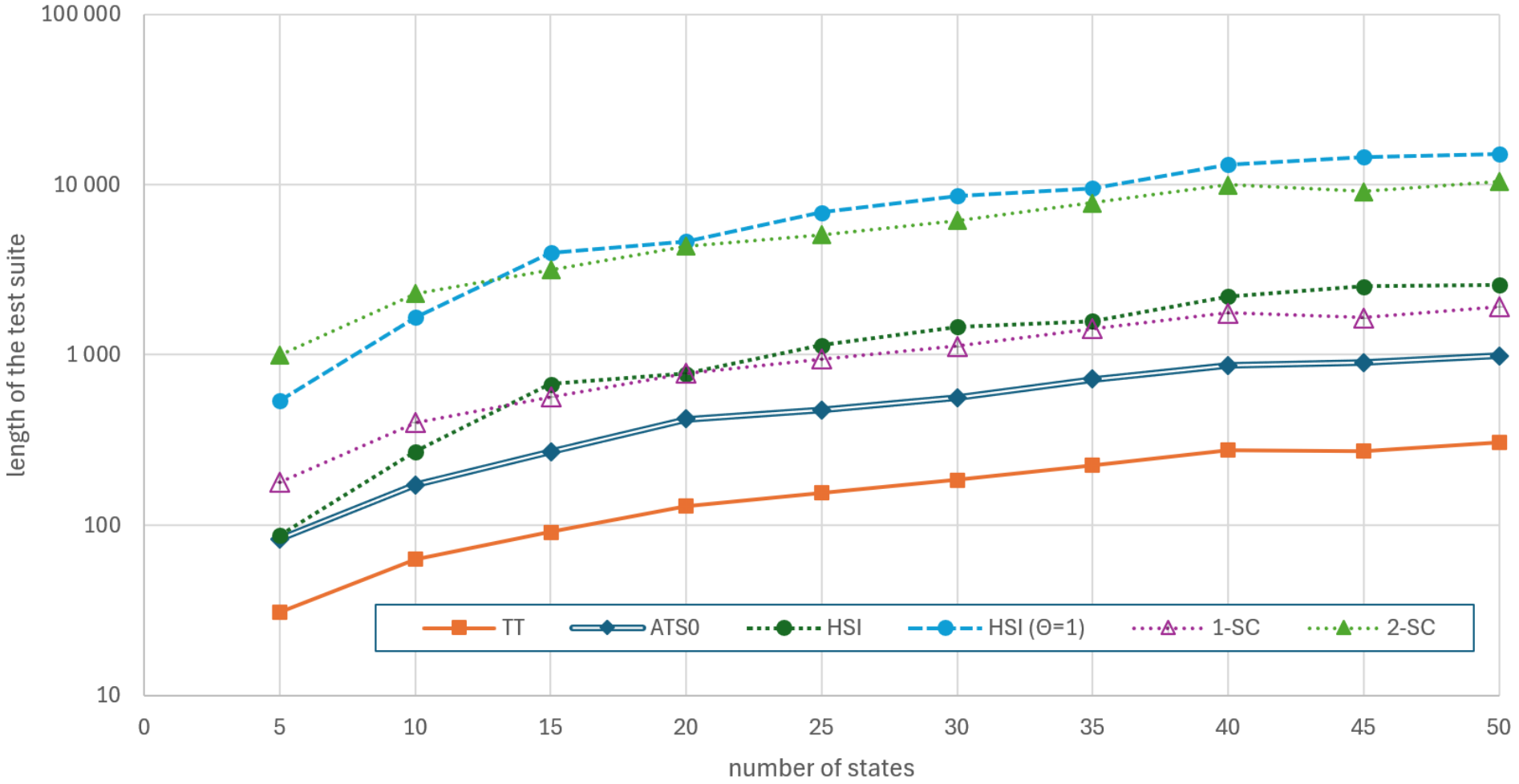
Model-based mutation testing (MBMT)

Test generation algorithm	Mutation scores for different number of states		
	5	10	15
TT	0.9557	0.9773	0.9849
ATS0	0.9989	0.9995	0.9998
HSI	0.9813	0.9902	0.9994
HSI (θ=1)	1	1	1
1-SC	1	1	1
2-SC	1	1	1

Mutation scores of different test suites for ES+ mutation operator

Input Symbols/Density: 5
Output symbols: 10

Model-based mutation testing (MBMT)



Length of different test suites

Input Symbols/Density: 5
Output symbols: 10

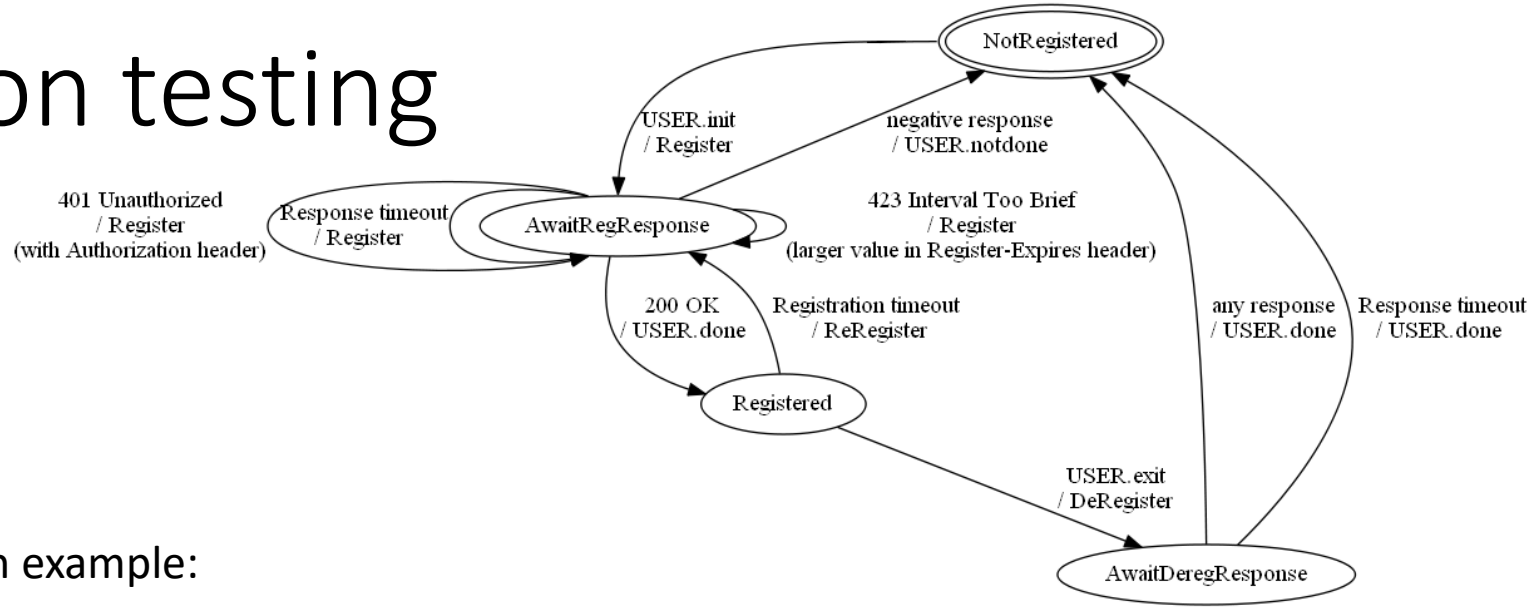
Model-based mutation testing (MBMT)

ALG / target score	Length of the test suite for different number of states									
	5	10	15	20	25	30	35	40	45	50
ATS0, HSI / 1	83	271	672	774	1139	1468	1583	2203	2523	2561
ATS0, 1-SC / 1	83	400	565	780	945	1124	1431	1763	1652	1907
ATS0, HSI / 0.998	83	271	271	420	474	561	719	864	905	989
ATS0, 1-SC / 0.998	83	400	271	420	474	561	719	864	905	989
ATS0, HSI / 0.99	83	173	271	420	474	561	719	864	905	989
ATS0, 1-SC / 0.99	83	173	271	420	474	561	719	864	905	989

Length of test suites using different strategies to kill all, 99.8% and 99% of ESC mutants

Input Symbols/Density: 5
Output symbols: 10

Model-based mutation testing



Mutation scores for the SIP UAC Registration example:

Mutation operator	Mutation scores for PS / CS machines... TT	ATS0	HSI	HSI (θ=1)	1-SC	2-SC
ISC	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1
COO	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1
MOO	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1
ESC	0.8928 / 0.9758	1 / 1	0.0357 / 1	1 / 1	1 / 1	1 / 1
MOT	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1
ET	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0
MOS	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1
ES+	0.8939 / 0.9707	0.9939 / 0.9948	0.1 / 0.9951	0.9848 / 1	0.9939 / 1	0.9962 / 1

Model-based mutation testing (MBMT)

Mutation operator	Proposed test generation algorithm for... complete coverage	good balance	Notes
ISC	HSI	TT	HSI for CS FSMs with reliable reset or if a separating sequence exist for all state pairs[4,5].
COO	TT	TT	-
MOO	TT	TT	-
ESC	HSI / (2-SC)	ATS0, 1-SC	HSI for CS FSMs with reliable reset or if a separating sequence exist for all state pairs[4,5]. 1-SC over ATS0 is advised for sparse FSMs. No proof for 100% coverage of 2-SC.
MOT	TT	TT	-
ET	-	-	This type of fault can not be discovered.
MOS	TT	TT	-
ES+	HSI (θ=1) / (2-SC)	ATS0, HSI / 1-SC	HSI (θ=1) for CS FSMs with reliable reset or if a separating sequence exist for all state pairs[4,5]. No proof for 100% coverage of 2-SC. 1-SC & 2-SC over other options are advised for sparse FSMs. HSI over ATS0 is advised for higher coverage.

Guidelines for selecting test generation algorithms

sparse: density<5 *4: PS → CS conversion can be done easily with MTR*
5: MTR throws a warning if separating sequence does not exist for a state pair

Model-based mutation testing (MBMT) using MTR

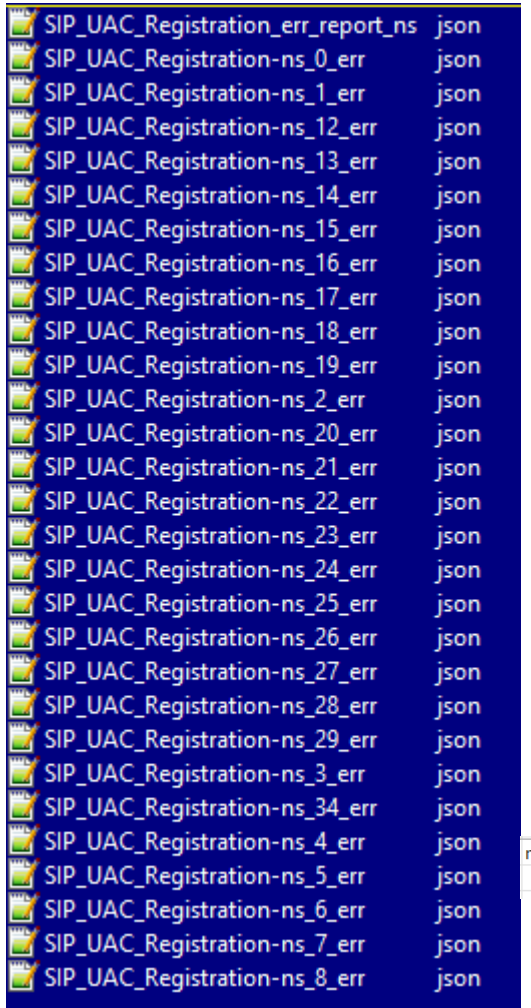
Apply model-based mutation testing using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -m MUT -f sample_models/introduction/SIP_Registration_UAC.json --mutation_config
config_files/mutation/mutation_cfg1.json --save_mutants --mutation_map
```

Mutation configuration (`mutation_cfg1.json`):

```
{
    "profile_name" : "MUTATION_CFG1",
    "mutants" : [
        ["NEXT_STATE"]
    ],
    "sc_check" : true,
    "redirect_transitions" : false,
    "preserve_input_alphabet" :
false,
    "methods" : [
        {
            "method_name" : "ATS",
            "ats_depth" : "0"
        },
        {
            "method_name" : "NS",
            "ns_value" : "1"
        },
        {
            "method_name" : "NS",
            "ns_value" : "2"
        }
    ],
    "target_score" : 1
}
```

List of mutated models →



Console output:

```
[U-CTRL ] [info] Version: 3.10.12 R6: Great spotted woodpecker, profile: DEFAULT, verbosity: 3, debug mode: off
[U-LMI  ] [info] Model Name: SIP_UAC_Registration, Type: FSM, Reset: No
[U-CTRL ] [info] Running MutationTesting test generation
[TG-MUT ] [info] Used mutation configuration: MUTATION_CFG1, path: config_files/mutation/mutation_cfg1.json
[TG-MUT ] [info] 28 mutant models have been created using next_state change injection(s)
[TG-MUT ] [info] Generating test suite using All-Transition-State test generation method and the original model
                 Test generation parameters: (ats_depth : 0)
[U-TW   ] [info] Test suite written: sets_of_test_suites/SIP_UAC_Registration-MUT-test_suites/SIP_UAC_Registration-ATS0-test_suite.json
[U-TSW  ] [info] Test set of test suites written: sets_of_test_suites/SIP_UAC_Registration-MUT-test_set.json
[TG-MUT ] [info] Generated mutants : 28
        Theoretically possible number of mutants: 36
        Target Mutation Score : 1.0
        Achieved Mutation Score : 1.0
        Number of test suites in test set : 1
        Overall number of test steps in test set : 47
[TG-MUT ] [info] Finished computation at 2026-02-24 13:53:13.2591709
        elapsed time: (real time) 0.14367 s
        elapsed time: (user time) 0.143000 s
        mutant generation time: 0.123000 s
        mutation check time 0.010000 s
        test generation time 0.007000 s
[TG-MUT ] [info] Mutation testing cover map file written: sets_of_test_suites/SIP_UAC_Registration-MUTATION_CFG1-MUT-mutation_map.json
[TG-MUT ] [info] Efficiency score of test generation methods:
        next_state              ATS_ats_depth=0         Efficiency score:       1       (28/28)

[TG-MUT ] [info] Efficiency score report written: sets_of_test_suites/SIP_UAC_Registration-MUTATION_CFG1-efficiency_score.csv
[TG-MUT ] [info] Detection score of test generation methods:
        next_state      1

[TG-MUT ] [info] Detection score report written: sets_of_test_suites/SIP_UAC_Registration-MUTATION_CFG1-detection_score.csv
[TG-TGR ] [info] Test generation summary written: test_summary/mut_result.csv
```

Test generation summary (`mut_result.csv`):

number_of_mutants	theoretical_upper_limit	test_set_overall_steps	test_set_used_metl	mutation_generation_time	mutation_check_time	test_generation_time	target_score	mutation_score
28	36	47	(ATS_ats_depth=0)	0.123000	0.010000	0.007000	1.000000	1.000000

Model-based mutation testing (MBMT) using MTR

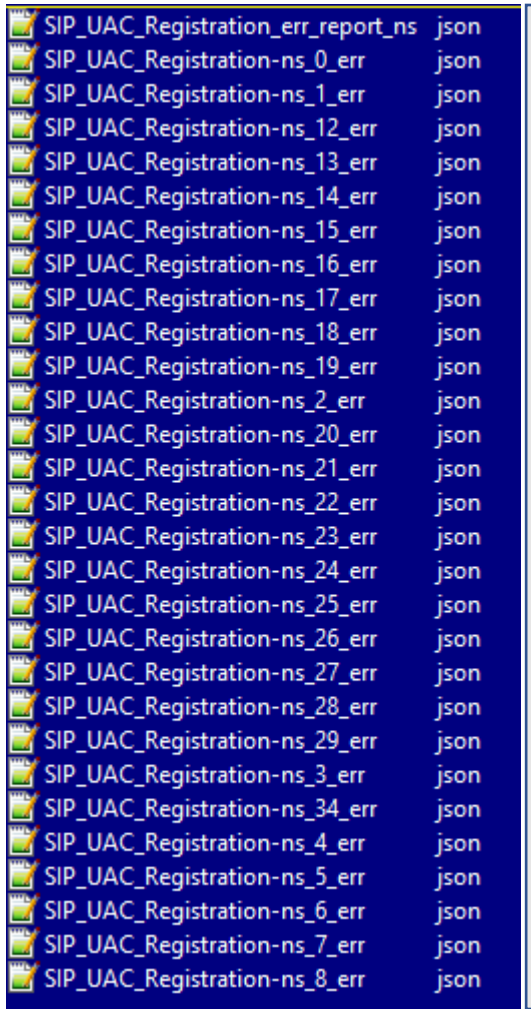
Apply model-based mutation testing using the *Model ⟫ Test ⟫ Relax (MTR) framework:*

```
./MTR -m MUT -f sample_models/introduction/SIP_Registration_UAC.json --mutation_config
config_files/mutation/mutation_cfg1.json --save_mutants --mutation_map
```

Mutation configuration (`mutation_cfg1.json`*):*

```
{
    "profile_name" : "MUTATION_CFG1",
    "mutants" : [
        ["NEXT_STATE"]
    ],
    "sc_check" : true,
    "redirect_transitions" : false,
    "preserve_input_alphabet" :
false,
    "methods" : [
        {
            "method_name" : "ATS",
            "ats_depth" : "0"
        },
        {
            "method_name" : "NS",
            "ns_value" : "1"
        },
        {
            "method_name" : "NS",
            "ns_value" : "2"
        }
    ],
    "target_score" : 1
}
```

List of mutated models →



← Mutation cover map

```
{
    "mutation_type_cover_map": {
        "next_state": "ATS_ats_depth=0"
    },
    "mutant_cover_map": {
        "SIP_UAC_Registration": "",
        "SIP_UAC_Registration-ns_0_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_12_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_13_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_14_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_15_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_16_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_17_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_18_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_19_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_1_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_20_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_21_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_22_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_23_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_24_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_25_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_26_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_27_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_28_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_29_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_2_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_34_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_3_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_4_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_5_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_6_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_7_err": "ATS_ats_depth=0",
        "SIP_UAC_Registration-ns_8_err": "ATS_ats_depth=0"
    }
}
```

Efficiency score of applied algorithms:

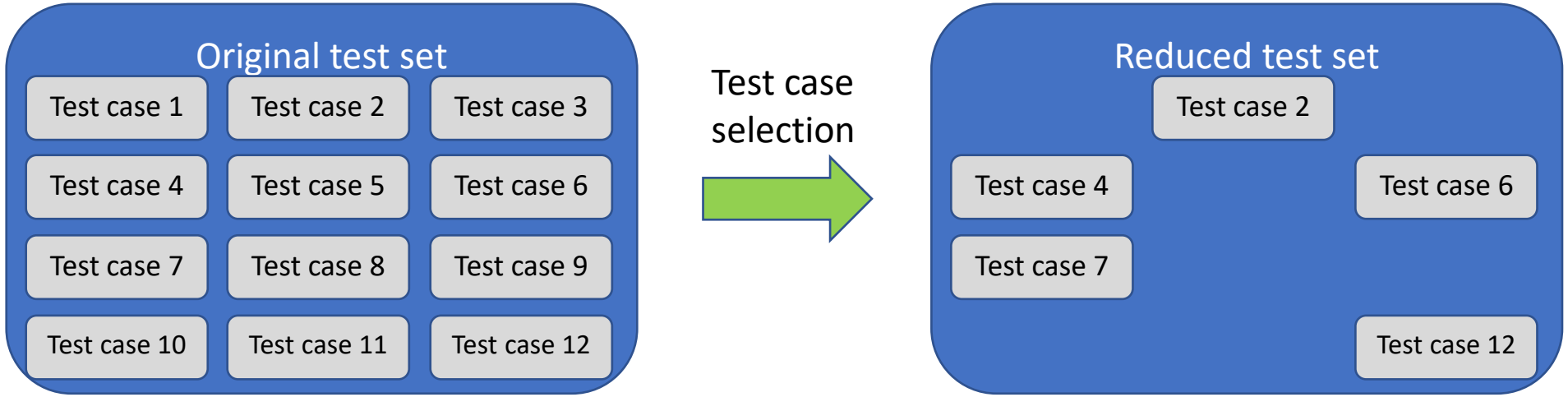
A	B	C	D	E
mutation	test_suite	efficiency_score	killed_mutants_	all_mutants_num
next_state	ATS_ats_depth=0	1.000000	28	28

- Check with other *Ω* list mutation operators (`mutants`) and *ALG* list of algorithms (`methods`)!

Test case selection

- Previously we discussed methods that **generate a test set** that (if the given assumptions are hold) **guarantee a given coverage**, but may result in a huge number of test cases

- Another approach – reduce the size of test set:
  - Given a test set with *n* number of test cases…
  - Select a subset of test set with *k* number of test cases…
  - That covers as different areas/functionalities…etc. as possible (depending on the selected coverage metrics)

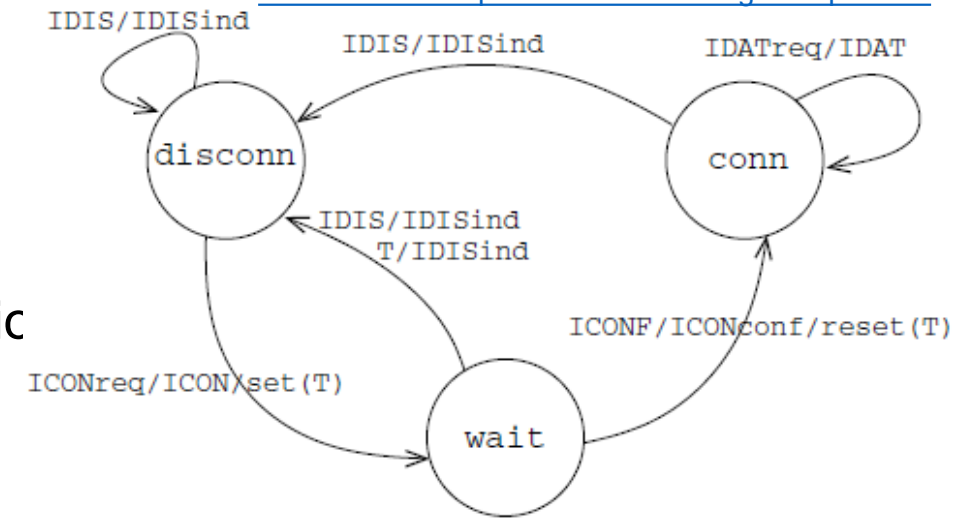


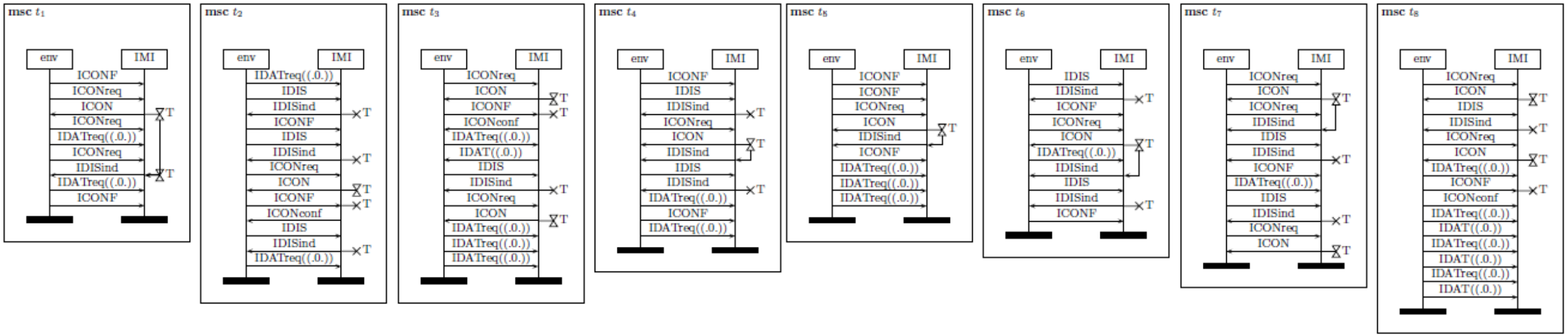
Test case selection – approximation methods

- Reducing the cardinality of the target test set without sacrificing its coverage is an NP-hard problem

- Approximation methods can be used
  - For MSC (Message Sequence) tests sets in telecommunicatic domain string edit distance-based test coverage metric is proposed

EFSM of INRES protocol ISAP Manager Ini process:



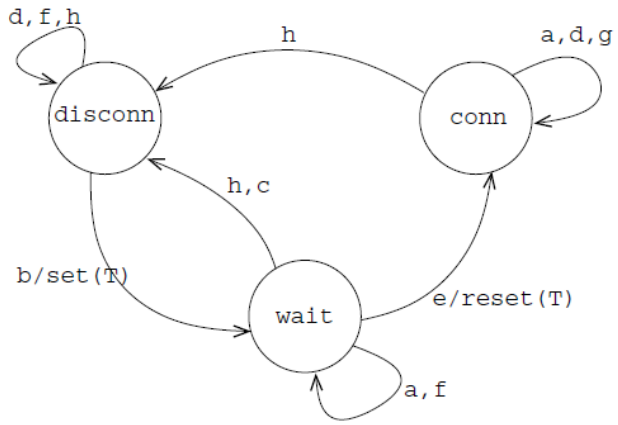


Test case selection using string distance metrics 4/1

- Using string edit distance metrics between test cases
  - Edit operations with unit costs: character insert, character delete, character overwrite
- Select the test cases that differ from each other as much as possible using this metric
  - For this, a distance matrix *D* is used that contains the editing distance between any 2 test cases

Example:

EFSM of INRES protocol ISAP Manager Ini process with character mapping:



Test cases:

t_1=dbafacfd
t_2=fhdhbehf
t_3=beghbfff
t_4=dhbchfdf
t_5=ddbcdfff
t_6=hdbafchd
t_7=bachdfhb
t_8=bhbfeggg

Distance matrix of test cases:

$$\mathbf{D} = \begin{bmatrix} 0&8&7&6&6&3&6&7 \\ 8&0&5&5&7&6&6&7 \\ 7&5&0&6&5&8&5&6 \\ 6&5&6&0&3&6&6&6 \\ 6&7&5&3&0&6&6&7 \\ 3&6&8&6&6&0&6&7 \\ 6&6&5&6&6&6&0&7 \\ 7&7&6&6&7&7&7&0 \end{bmatrix}$$

Test case selection using string distance metrics 4/2

- The threshold of the selection can be fine-tuned with an ε approximation parameter
- Algorithm: Select test cases of a test set with minimum cardinality and maximum internal distance assuming ε
  1. Determines which test cases of *T* ε-cover each other, and if they do, it is marked in matrix *A* with *1* value
  2. Lines of *A* with full *0* values can not be eliminated
  3. If exactly the same coverage is found for two test cases, then one of them is eliminated
  4. The reduced *A* matrix represented in a bipartite graph *G'*
     - Use maximum cardinality matching in *G'* *(bold lines in figure)*

Example (continued):

$$\mathbf{D} = \begin{bmatrix} 0&8&7&6&6&3&6&7 \\ 8&0&5&5&7&6&6&7 \\ 7&5&0&6&5&8&5&6 \\ 6&5&6&0&3&6&6&6 \\ 6&7&5&3&0&6&6&7 \\ 3&6&8&6&6&0&6&7 \\ 6&6&5&6&6&6&0&7 \\ 7&7&6&6&7&7&7&0 \end{bmatrix}$$

ε =5

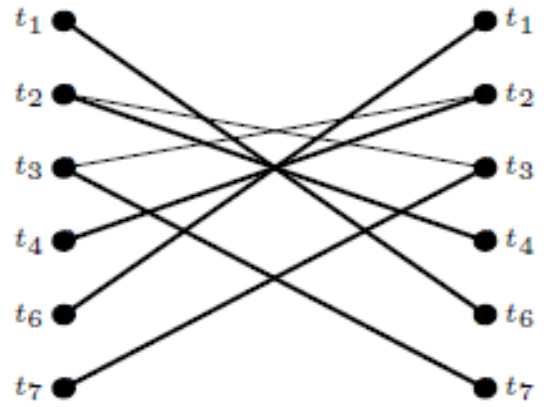
$$\mathbf{A}_{\varepsilon=5} = \begin{bmatrix} 0&0&0&0&0&\boxed{1}&0&0 \\ 0&0&1&\boxed{1}&0&0&0&0 \\ 0&1&0&0&1&0&\boxed{1}&0 \\ 0&\boxed{1}&0&0&1&0&0&0 \\ 0&0&1&1&0&0&0&0 \\ \boxed{1}&0&0&0&0&0&0&0 \\ 0&0&\boxed{1}&0&0&0&0&0 \\ 0&0&0&0&0&0&0&0 \end{bmatrix}$$

t_2 t_5 t_8

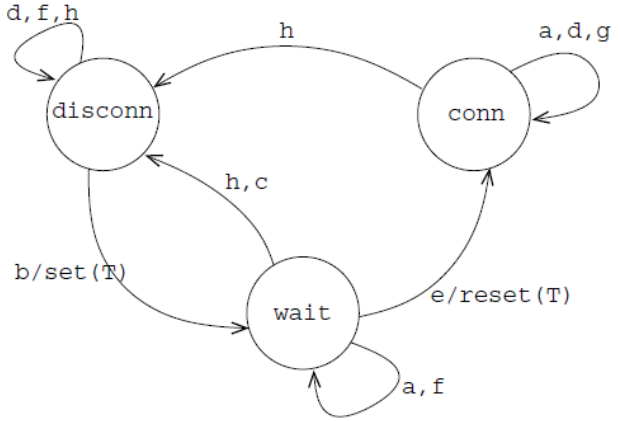
Exactly same ε-cover → one of them can be eliminated

Can not be ε-covered by any other test cases

Bipartite graph *G'* constructed from A:



Test case selection using string distance metrics 4/3



Initial, *T* set of test cases:

t_1=dbafacfd
t_2=fhdhbehf
t_3=beghbfff
t_4=dhbchfdf
t_5=ddbcdfff
t_6=hdbafchd
t_7=bachdfhb
t_8=bhbfeggg

T' set of selected test cases:

t_1=dbafacfd
t_2=fhdhbehf
t_3=beghbfff
t_8=bhbfeggg

These test cases have been selected from maximum-cardinality matching

Example (continued):

$$\mathbf{D} = \begin{bmatrix} 0&8&7&6&6&3&6&7 \\ 8&0&5&5&7&6&6&7 \\ 7&5&0&6&5&8&5&6 \\ 6&5&6&0&3&6&6&6 \\ 6&7&5&3&0&6&6&7 \\ 3&6&8&6&6&0&6&7 \\ 6&6&5&6&6&6&0&7 \\ 7&7&6&6&7&7&7&0 \end{bmatrix}$$

ε =5

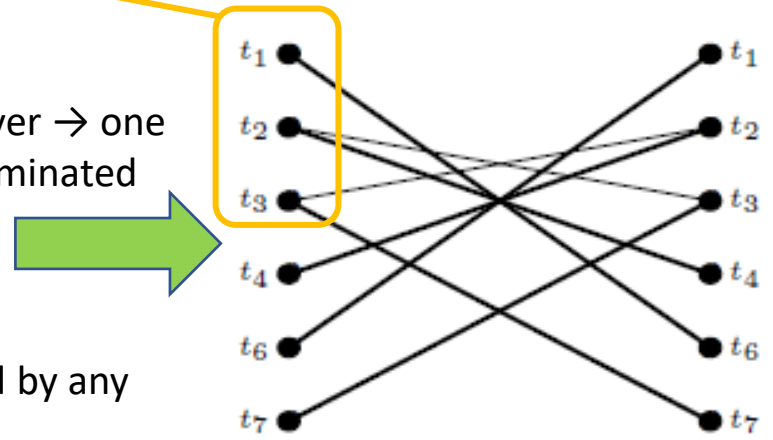
$$\mathbf{A}_{\varepsilon=5} = \begin{bmatrix} 0&0&0&0&0&\boxed{1}&0&0 \\ 0&0&1&\boxed{1}&0&0&0&0 \\ 0&1&0&0&1&0&\boxed{1}&0 \\ 0&\boxed{1}&0&0&1&0&0&0 \\ 0&0&1&1&0&0&0&0 \\ \boxed{1}&0&0&0&0&0&0&0 \\ 0&0&\boxed{1}&0&0&0&0&0 \\ 0&0&0&0&0&0&0&0 \end{bmatrix}$$

t_2, t_5, t_8

Exactly same ε-cover → one of them can be eliminated

Can not be ε-covered by any other test cases

Bipartite graph *G'* constructed from A:



Test case selection using string distance metrics 4/4

+:

- Number of test cases can be significantly reduced
- The complexity of test case selection is $O(|T|^3)$,

where $|T|$ is the number of test cases in the original test set

-:

- Approximation (does not guarantee to find all given type of faults)
- No guidelines how to set up ε parameter properly, experiences required for the given domain to find the optimum setting

Test case generation using string distance metrics

- Based on string distance metrics an algorithm also exist that generate test cases from scratch for given ε threshold parameter

Algorithm Selective automatic test generation

input : ε; K iteration limit; Sp specification
output: T set

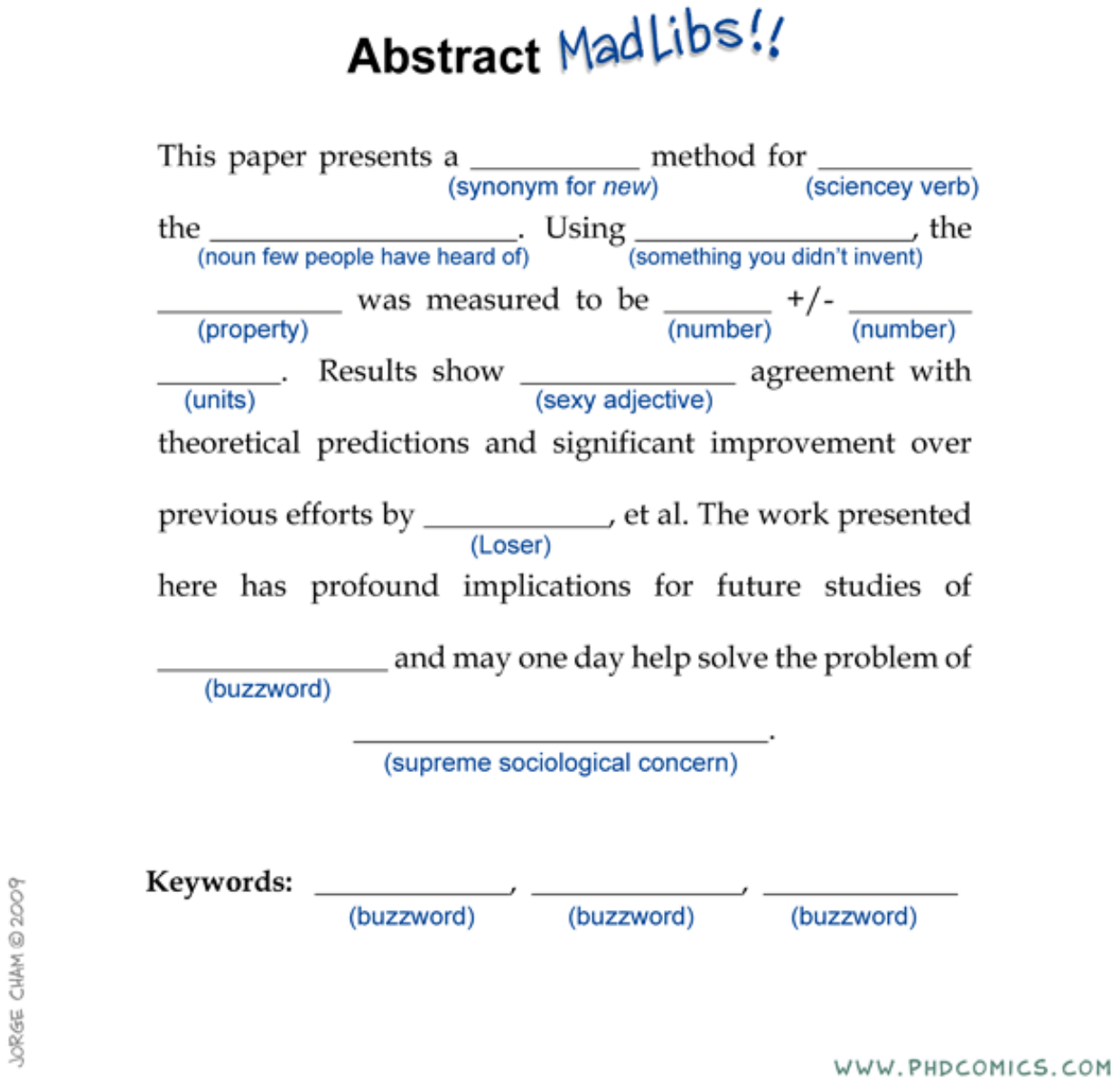
1 $T[0] := \emptyset$;
2 $k := 1$;
3 **repeat**
4 $\quad t' :=$ derive(Sp);
5 $\quad$ **if** $d(t', T) < \varepsilon$ **then** $T[k] :=$ reduce$(T[k-1] \cup t')$;
6 $\quad$ **else** $T[k] := T[k-1] \cup t'$ $k := k + 1$;
7 **until** $k > K$;
8 **return** $T[k]$

MBT in theory

Further information about the theory of MBT:

- Good overview of different FSM models and algorithms:
  - M. Broy, B. Jonsson, J.-P. Katoen, M. Leucker, and A. P. (Eds.): Model-based testing of reactive systems. Springer, 2005.
  - D. Lee and M. Yannakakis. Principles and Methods of Testing Finite State Machines – A Survey. Proceedings of the IEEE, 84(8):1090–1123, 1996.
- Specific related subtopics of MBT:
  - EFSM → FSM conversion: D. Lee and M. Yannakakis. Online minimization of transition systems. In Proceedings of STOC '92, pages 264–274, New York, NY, USA, 1992. ACM.
  - Test coverages with step-by-step examples : I. Forgacs, A. Kovacs: Practical Test Design: Selection of traditional and automated functional tests. BCS. 2019. / Chapter 6 State transition testing
  - Incremental test generation: G. Á. Németh: Incremental test generation algorithms. PhD dissertation. 2015.
    - Chapter 3: An overview of FSM-based test generation
    - Chapters 4-6: The incremental approach, 2 incremental algorithms
  - Model-based mutation testing: Gábor Árpád Németh. Model-based mutation testing for Finite State Machine specifications with MTR. Infocommunications Journal, 17(3): Under publication.
  - String distance metric based test case selection: G. Kovács, G. Á. Németh, M. Subramaniam and Z. Pap: Optimal String Edit Distance Based Test Suite Reduction for SDL Specifications. In Proceedings of SDL 2009. Pages 82-97. 2009.

How to read an article?



Abstract MadLibs!!

This paper presents a _____ method for _____
(synonym for *new*) (sciencey verb)
the _____. Using _____, the
(noun few people have heard of) (something you didn't invent)
_____ was measured to be _____ +/- _____
(property) (number) (number)
_____. Results show _____ agreement with
(units) (sexy adjective)
theoretical predictions and significant improvement over
previous efforts by _____, et al. The work presented
(Loser)
here has profound implications for future studies of
_____ and may one day help solve the problem of
(buzzword)
_____.
(supreme sociological concern)

Keywords: _____, _____, _____
(buzzword) (buzzword) (buzzword)

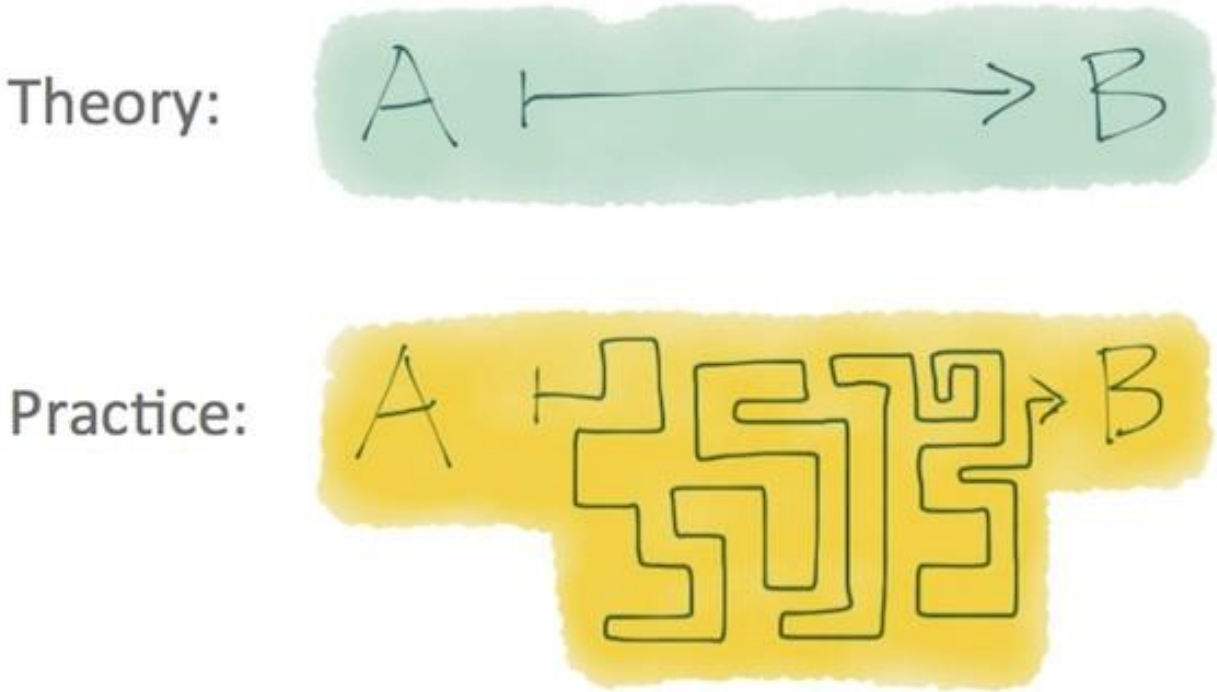
JORGE CHAM © 2009

WWW.PHDCOMICS.COM

MBT
theory vs. practice

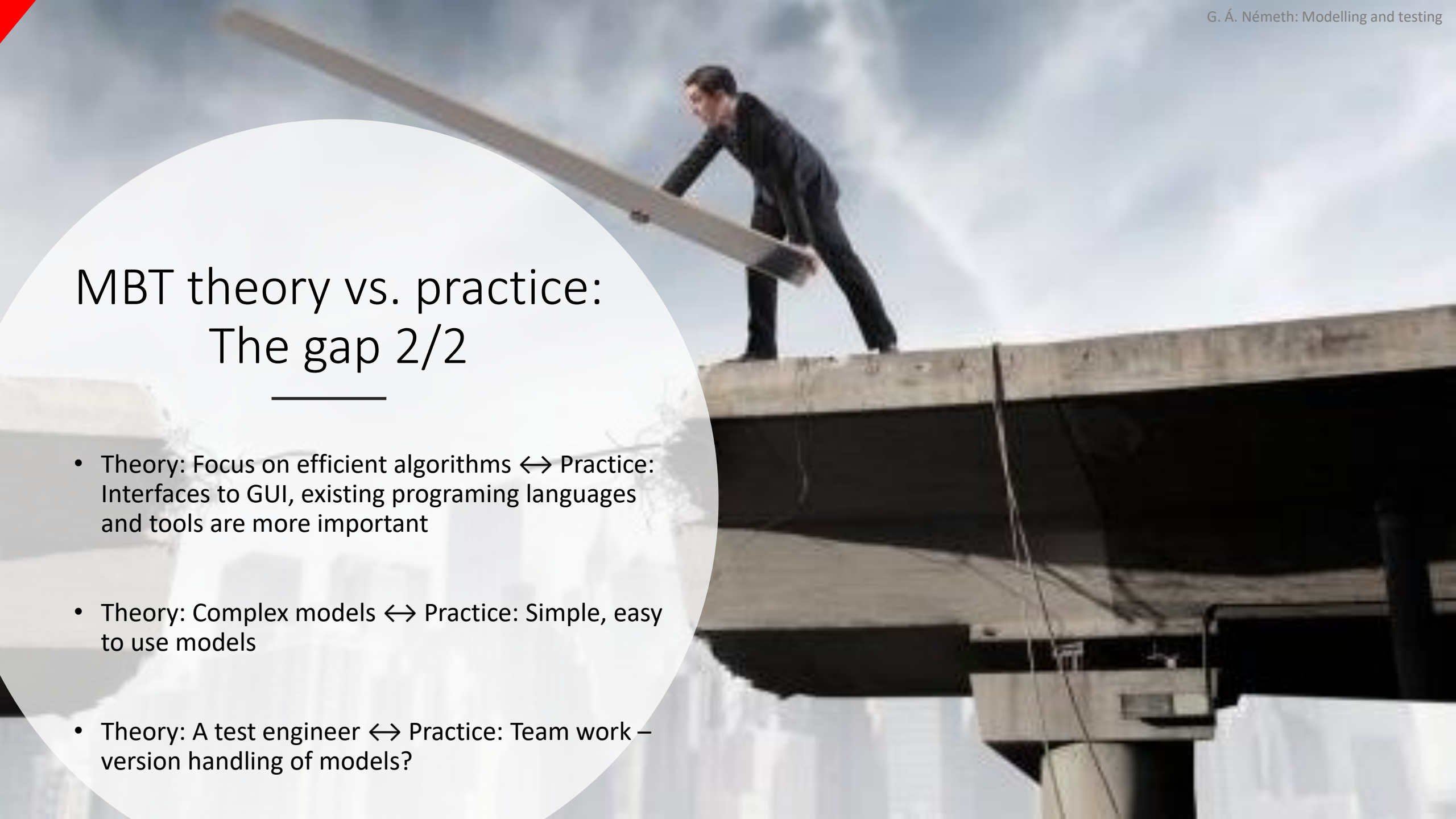
- What is the difference between theory and practice?

- **Theoretically** nothing…



MBT theory vs. practice: The gap 2/1

- Theory: FSM models ↔ Practice: EFSM models + additional adaptation code (glue code) may be required
- Theory: focus on good coverage ↔ Practice: focus shorter length of test sets
- Theory: Assumes rigid specifications ↔ Practice: evolving systems



MBT theory vs. practice: The gap 2/2

- Theory: Focus on efficient algorithms ↔ Practice: Interfaces to GUI, existing programing languages and tools are more important
- Theory: Complex models ↔ Practice: Simple, easy to use models
- Theory: A test engineer ↔ Practice: Team work – version handling of models?

MBT in practice: adaptation codes and interfaces

- Most of the MBT tool generate test code only for **symbolic execution**, i.e. to run on implementation model rather than on actual implementation*
  - **Adaptation code** is required which "adapt" the MBT generated test case to the SUT to be executable
    - i.e. adaptation code (glue code) implement model elements:
      - Transitions in FSM, appropriate keywords in keyword driven testing
    - The used approach depending on the tested application and its interfaces (GUI, API, CLI…etc)

- Typical MBT tools have only interfaces to few other tools that makes difficult to use them as a part of a toolchain

* In some cases – if additional information exist in the specification model about how to connect this model to the SUT, and connections to other tools are provided as well – the adaptation code can be partially or completely generated. But in this case the adaptation of test cases will be restricted to some predefined problem domains based on the applied MBT tool.

API: application programming interface *CLI: Command Line Interface* *GUI: Graphical User Interface* *SUT: System Under Test*

Adaptation code (glue code) examples

GUI testing of a Web application with Selenium:

```
//Adaptation code that simulates the clicking on an element
public static void MakeClick(ChromeDriver driver, int i, string from, string input, string output)
{
    try
    {
        if (input == "press Previous")
        {
           IWebElement pressNext = driver.FindElement(By.XPath("/html/body/div/main/a[2]"));
           pressNext.Click();
        }
        else if(input == "select a category")
        {
           Functions.GoToCategoriesPage(driver);
        }
        IWebElement nextAction = driver.FindElement(By.XPath(Dictionaries.xpaths[input]));
        nextAction.Click();
    }
    catch (Exception e)
    {
        throw new Exception("ERROR: Could not find element by XPath on source page!" +
        "\n" + "test No.: " + i.ToString() + ", test from: " + from + ", test input: " +
        input + ", test output: " + output);
    }
}
```

Testing through HTTP *(Docker)*:

```
//Corresponding model element:
input "start container - successful - not running" {
guard {
return container_state in [CONTAINER_STATE_CREATED, CONTAINER_STATE_EXITED]
}
body { container_state = CONTAINER_STATE_RUNNING }
//Adaptation code for "Start container" transition of the model:
adapter {
res = client.post("/containers/"
+ TEST_CONTAINER_NAME + "/start")
assert res.status_code == 204, "Status code 204 expected"
}
```

Testing through an API *(Telegram API)*:

```
//Create connection between model elements and adaptation code:
var tests = new Dictionary<string, Func<Task>> {
    ...
    {"create Chat", CreateChatTest},
...
//Adaptation code for "create Chat" transition of the model:
async Task CreateChatTest()
{
    var contacts = await client.Contacts_GetContacts();
    user = contacts.users[contacts.contacts.First().user_id];

    var update = await client.Messages_CreateChat(new InputUserBase[]
    {
        new InputUserSelf(),
        user
    }, "Test");
    CustomAssert.Assert(update.Chats.Values.FirstOrDefault(x => x.Title
    == "Test") != null, nameof(CreateChatTest));

    chatId = update.Chats.Values.First(x => x.Title == "Test").ID;
}
```

Testing through CLI *(GitHub CLI)*:

```
//Adaptation code for "Open New PR" transition of the model:
def OpenNewPR(self):
    self.prid = self.__get_next_pr_id()
    branch_name = git_new_branch(self.prid)
    sh.gh.pr.create("--fill", "--head",
                    branch_name,
                    _env=get_env("github-fmbt.token"))
    return self.prid
```

Test code in Java → C++ code adapted with a dll file, JNA *(CNC Controller)*:

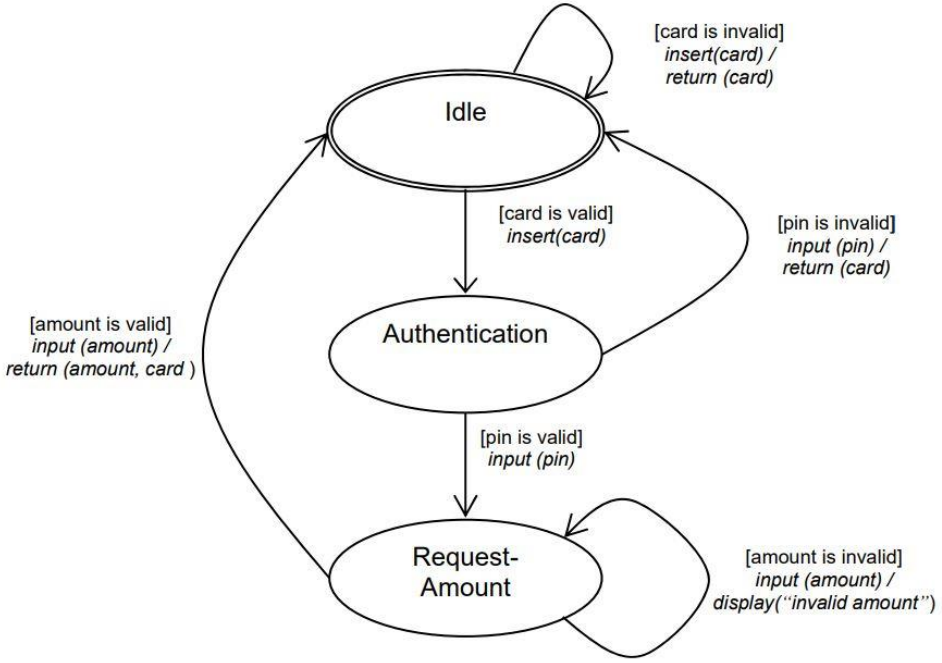
```
@Override
public void e_start()
{
  step();
}

private void step()
{
  dll.setMotorMoverTarget(ptr, x_on ? 1 : 0, y_on ? 1 : 0, z_on ? 1 : 0, 1, false);
}
```

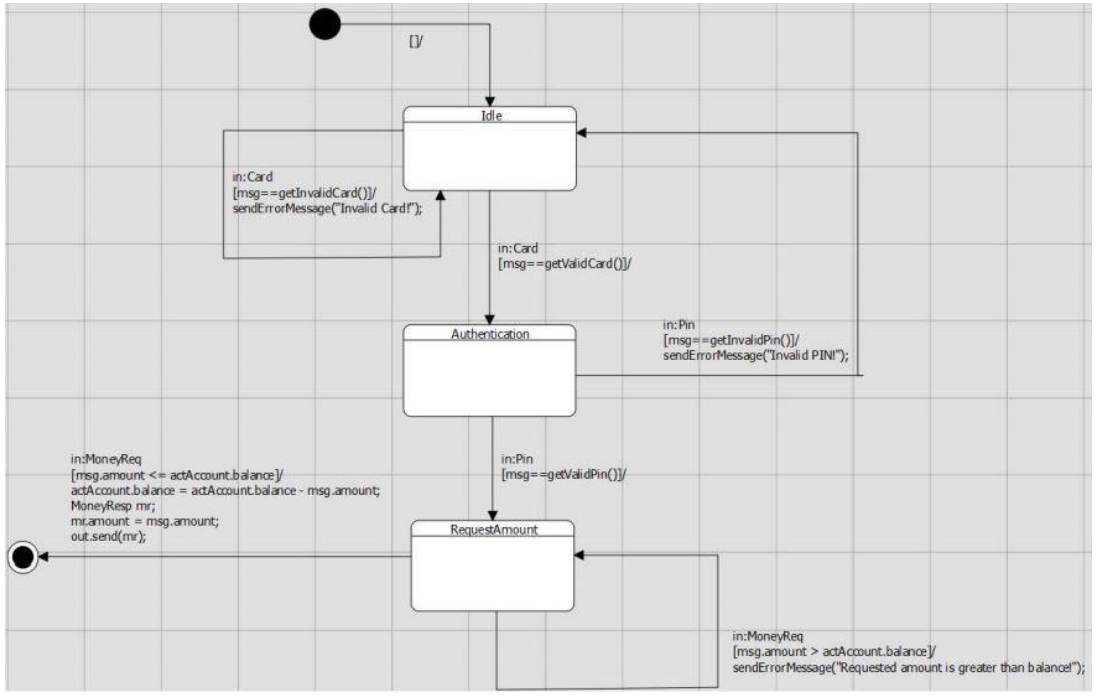
- More adaptation code examples are provided under Appendix D: Sample Homeworks
- Sample test projects with their adaptation code also provided in Model 》 Test 》 Relax

API: application programming interface
CLI: Command Line Interface
DLL: Dynamic Link Library
GUI: Graphical User Interface
HTTP: HyperText Transfer Protocol
JNA: Java Native Access

MBT in practice: How models are created? 2/1

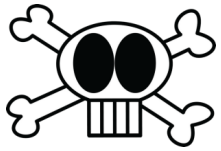
1. Most of the time models are created manually by the test analyst
   - Based on specification, standards
   - Iterative process
     - Validation and improvement cycles
   - Supported by (usually graphical) tools





MBT in practice: How models are created? 2/2

2. In some cases models are generated from the code

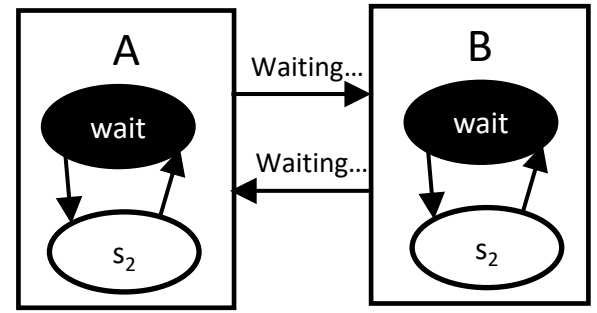


- **Very dangerous! The case when specification is derived from implementation should be avoided…**
  - …as in that case we will compare implementation with implementation

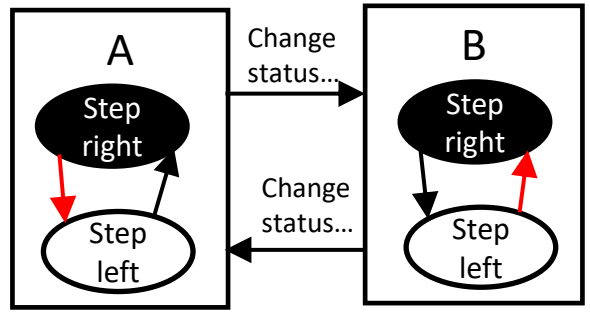
- But there is **one exception**…
- …this technic can be used efficiently **to identify interoperability problems**
  - Deadlocks
  - Livelocks
  - Infinite loops

Interoperability problems

1. **Deadlock**: 2 or more participants are waiting for each other
   - *Example (communication systems): the receiver asks the sender to wait with the transmission and later, it notifies the sender to continue the transmission, but this notification is lost.*
     - *If no reliability functions are implemented to recover this lost notification, then the 2 participants wait for each other forever.*



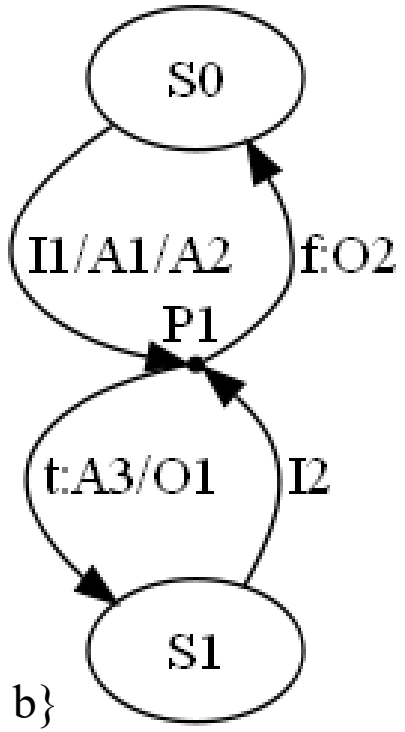
2. **Livelocks**: similar to the deadlock; however, instead of waiting for each other, the participants are continually processing regarding the other one without a sense: the specified task is not progressing.
   - *Example: two people try to pass each other facing each other, but they always try to step aside in the same direction.*
   - The risk of livelock occurs when we would like to resolve a deadlock and force more than one participant to change.



MBT in practice: Code → (C)EFSM conversion 5/1

- *for* cycle:

```
public void example()
{
        int b=3;
        for(int a=0;a<b;a++)
        {

        System.out.println("example");
        }
}
```



set of states: S: $\{S_0, S_1\}$
set of variables: V: $\{V_1$: a, V_2: b$\}$
set of input symbols: I: $\{I_1$: *example()*, I_2: *ret_System.out.println()*$\}$
set of output symbols: O: $\{O_1$: *System.out.println()*, O_2: *ret_example()*$\}$
set of predicates: P: $\{P_1$: $a<b\}$
set of actions: A: $\{A_1$: $b=3$, A_2: $a=0$, A_3: $a{+}{+}\}$

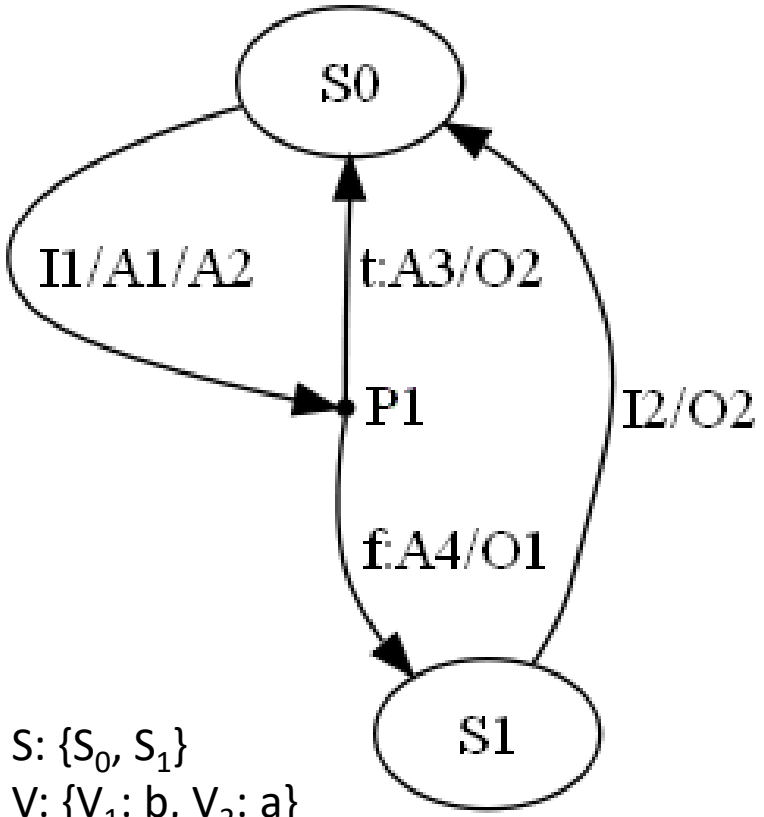
Note that the *while* and *do-while* loops result in similar EFSMs

Source of examples: Balázs Szépe: Java-kód transzformáció Kiterjesztett Véges Automata modellre. MSc diplomawork. BME. 2011.

MBT in practice: Code → (C)EFSM conversion 5/2

- *if-else* statement:

```
public int example()
{
        int b=3;
        int a=10;
        if(a<b){
                a=b;
        }
        else{
                b=100;

        System.out.println("b=100");
        }
        return b;
}
```



set of states: S: {S_0, S_1}
set of variables: V: {V_1: b, V_2: a}
set of input symbols: I: {I_1: *example()*, I_2: *ret_System.out.println()*}
set of output symbols: O: {O_1: *System.out.println()*,O_2: *ret_example(b)*}
set of predicates: P: {P_1: *a<b*}
set of actions: A: {A_1: *b=3*, A_2: *a=10*, A_3: *a=b*, A_4: *b=100*}

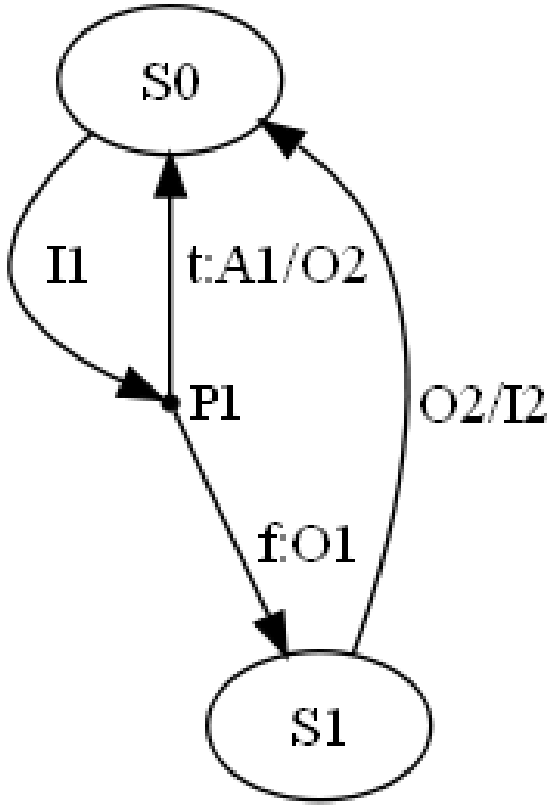
Note that the *switch-case* statement can be converted to if-else statement, then to EFSM

Source of examples: Balázs Szépe: Java-kód transzformáció Kiterjesztett Véges Automata modellre. MSc diplomawork. BME. 2011.

MBT in practice: Code → (C)EFSM conversion 5/3

- Exception handling:

```
public void example(int a)
{ try
        { if(a>5) throw new Exception();
        a++;
        }catch(Exception e){System.out.println("error");}
}
```



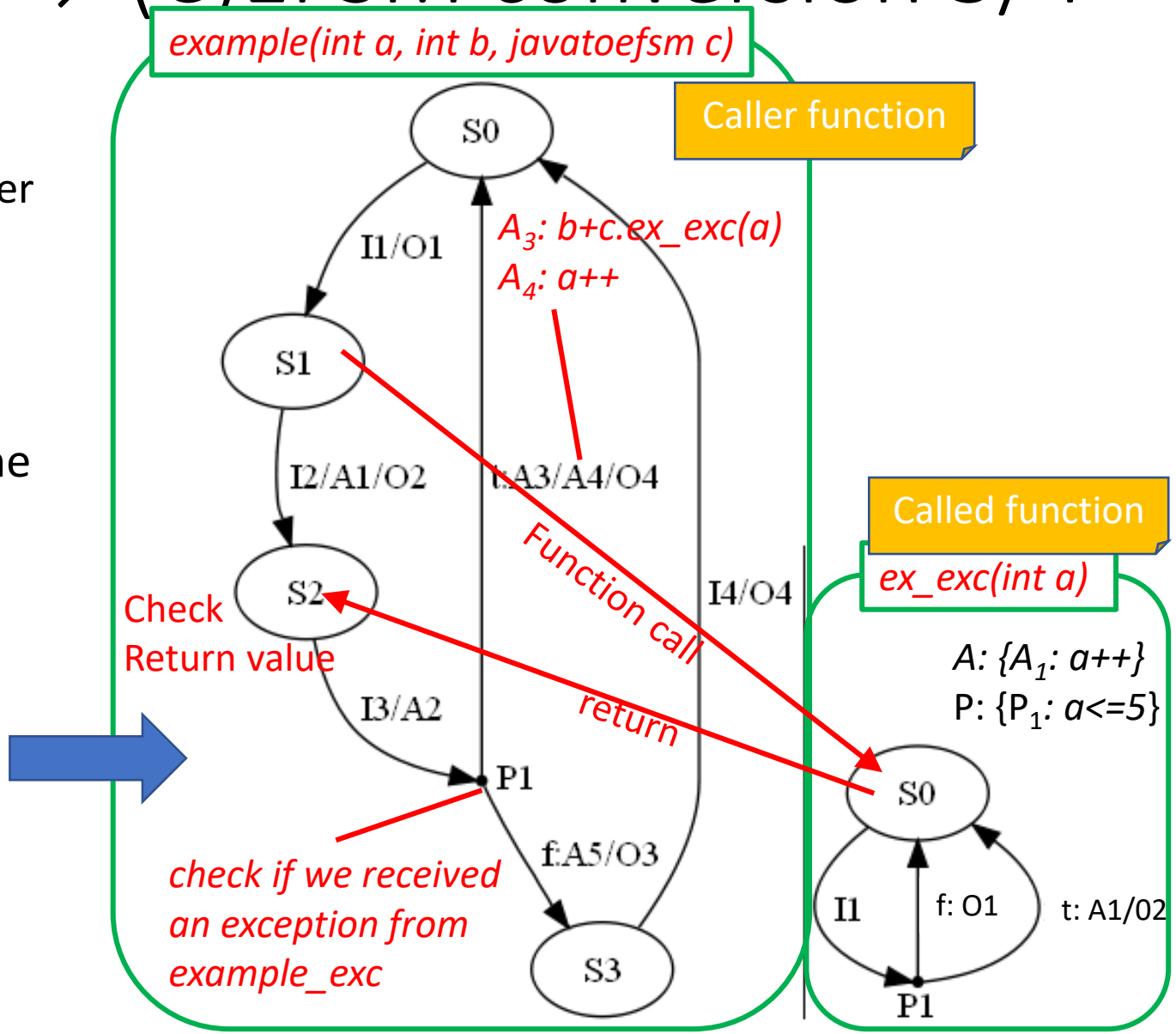
set of states:	S: $\{S_0, S_1\}$
set of variables:	V: $\{V_1: a\}$
set of input symbols:	I: $\{I_1$: *example()*, I_2: I_2: *ret_System.out.println()*$\}$
set of output symbols:	O: $\{O_1$: *System.out.println(„error")*, O_2: *ret_example(a)*$\}$
set of predicates:	P: $\{P_1$: ***a<=5***$\}$ *(negate of condition of "if(a>5)")*
set of actions:	A: $\{A_1: a{+}{+}\}$

Source of examples: Balázs Szépe: Java-kód transzformáció Kiterjesztett Véges Automata modellre. MSc diplomawork. BME. 2011.

MBT in practice: Code → (C)EFSM conversion 5/4

- Function call:
  - Create a new sub EFSM
    - It's input will be the output of the caller function
  - When a function is called, the caller function stays in the given state till the called function returns to it's s_0 initial state
  - The output of called function will be the input of the caller function.

```
public int ex_exc(int a) throws Exception{
if(a>5) throw new Exception();
a++;
return a;}

public int example(int a, int b, javatoefsm c){
try{        a=b+c.ex_exc(a);
            a++;
}catch(Exception e){System.out.println(„error");}
return a;}
```



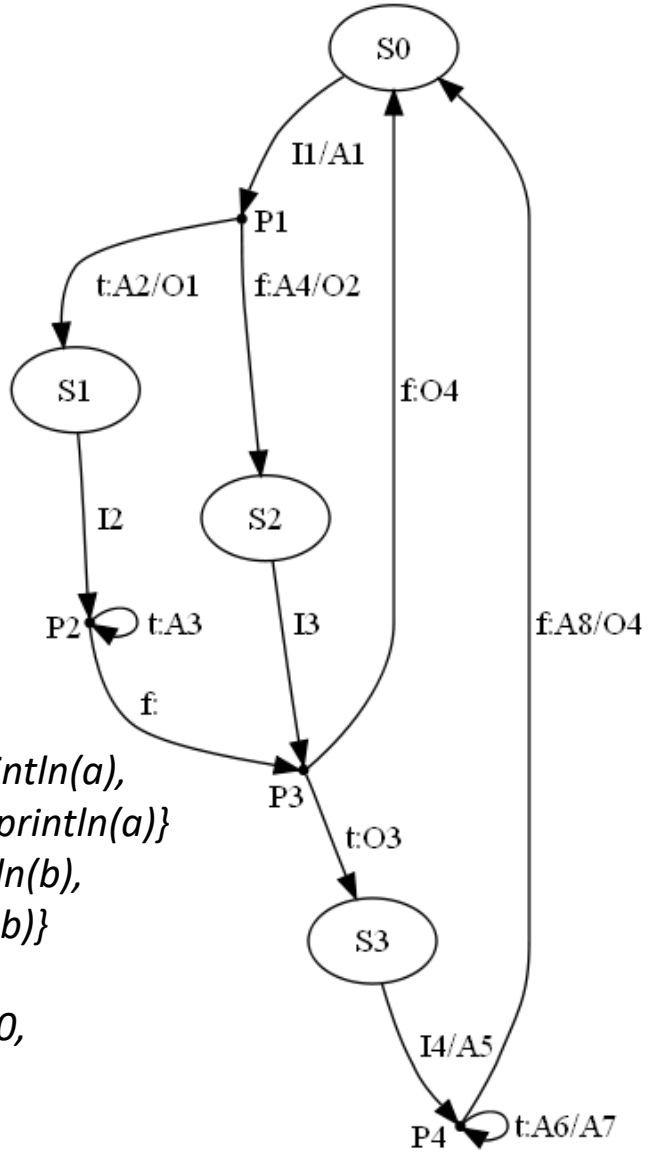
Source of examples: Balázs Szépe: Java-kód transzformáció Kiterjesztett Véges Automata modellre. MSc diplomawork. BME. 2011.

MBT in practice: Code → (C)EFSM conversion 5/5

- A compound example:

```
public class javatoefsm {
public int example_2(int a, int b){
int c=10;
if(a == 5) {
        b=a+2*b;
        System.out.println(a);
        while(c>0) c--;
}
else {
        b=b+2;
        System.out.println(b);
}
if(a == 3) {
        System.out.println(a);
        for(int i=0;i<5;i++) {
                a=i*a;
        }
        a=a/100;
}
```

$S:\{S_0, S_1, S_2, S_3\}$
$V:\{V_1: a, V_2: b, V_3: c\}$
$I:\{I_1: example_2(int\ a, int\ b), I_2: ret_System.out.println(a),$
$I_3: ret_System.out.println(b), I_4: ret_System.out.println(a)\}$
$O:\{O_1: System.out.println(a), O_2: System.out.println(b),$
$O_3: System.out.println(a), O_4: ret_example_2(a+b)\}$
$P:\{P_1: a == 5, P_2: c>0, P_3: a == 3, P_4: i<5\}$
$A:\{A_1: c=10, A_2: b=a+2*b, A_3: c--, A_4: b=b+2, A_5: i=0,$
$A_6: a=i*a, A_7: i++, A_8: a=a/100\}$



Source of examples: Balázs Szépe: Java-kód transzformáció Kiterjesztett Véges Automata modellre. MSc diplomawork. BME. 2011.

MBT in practice: Code → EFSM conversion

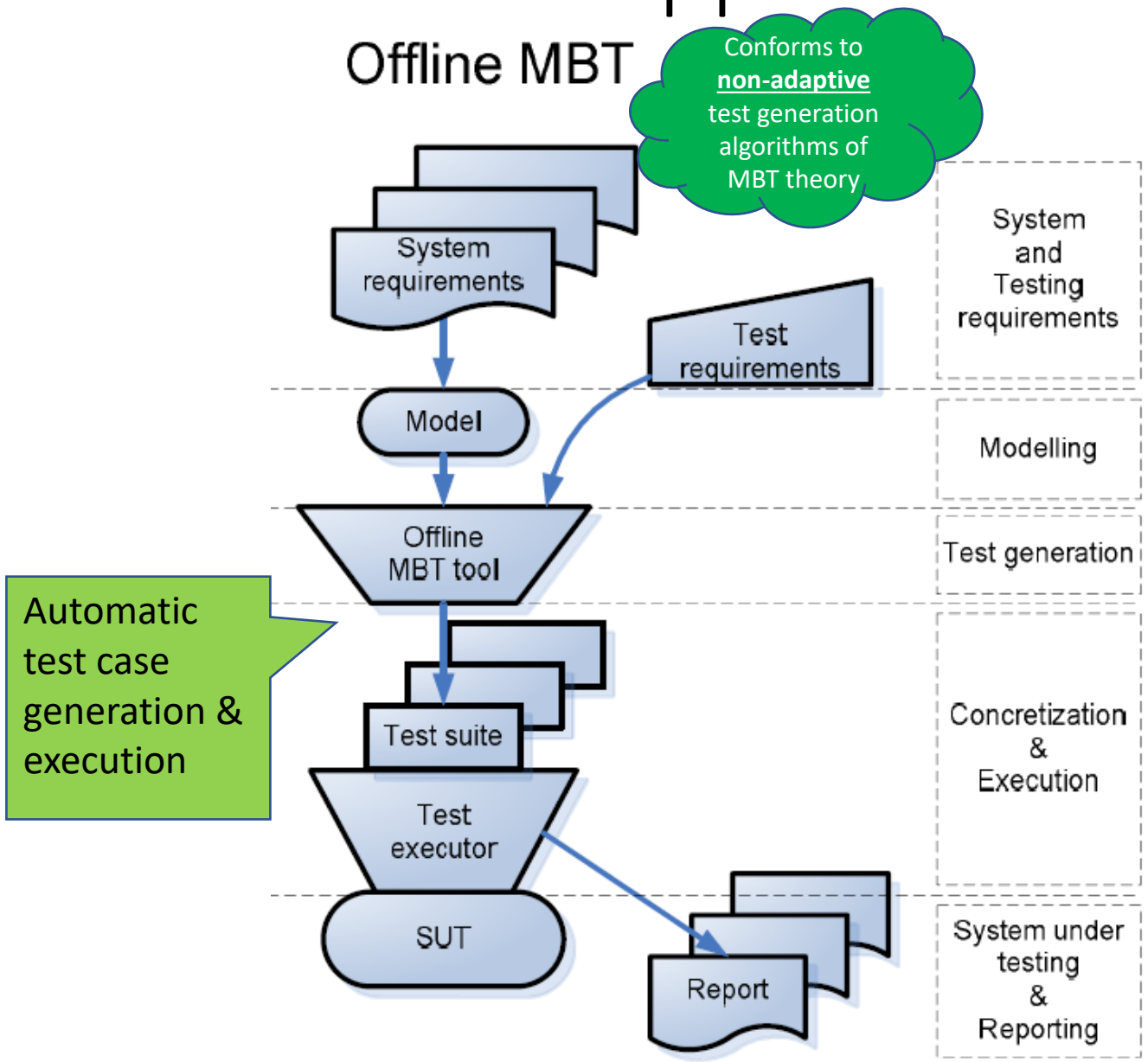
Further information about the topic

- Klaus Havelund, Thomas Pressburger: Model checking JAVA programs using JAVA PathFinder. International Journal on Software Tools for Technology Transfer. March 2000, Volume 2, Issue 4, pp 366–381
- Thomas A. Henzinger, Ranjit Jhala, Rupak Majumdar, Grégoire Sutre: Lazy Abstraction March 2002 ACM SIGPLAN Notices 37(1)

MBT in practice: How models are created?

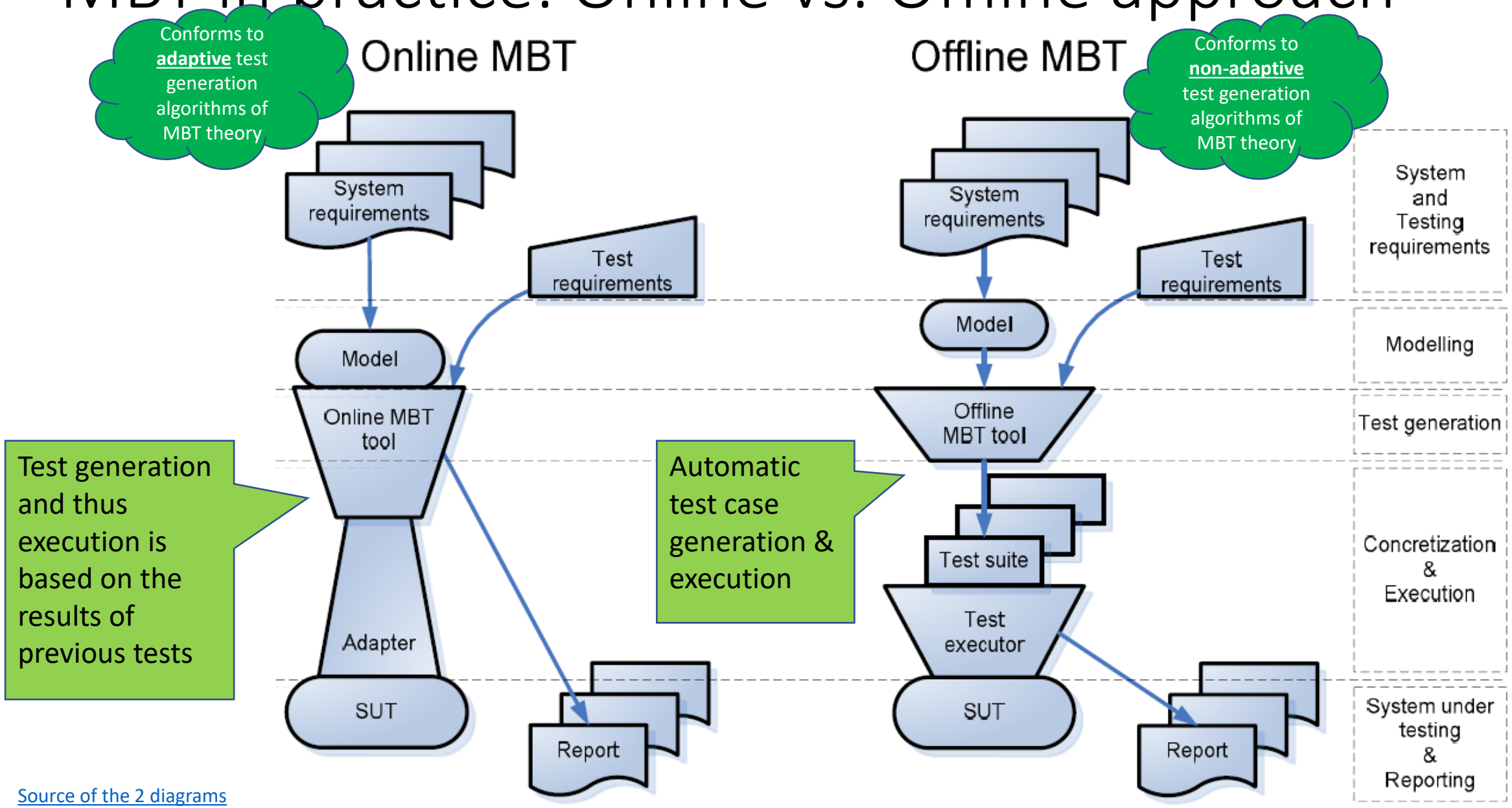
Ok, let’s see the much more frequently used case, when models are created manually by test designer with tool support…

MBT in practice: Online vs. Offline approach



Source of the 2 diagrams

MBT in practice: Online vs. Offline approach



Source of the 2 diagrams

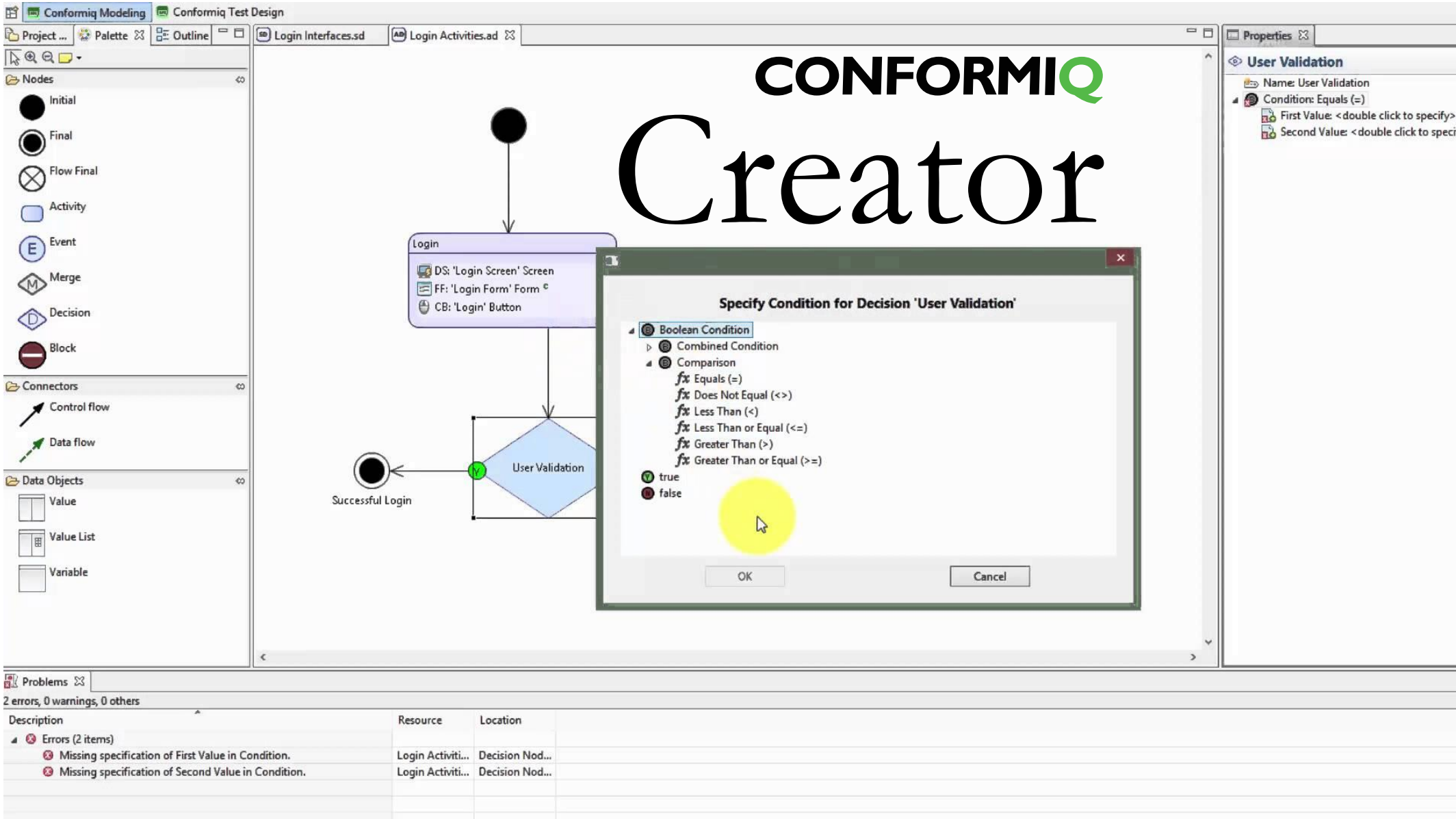
MBT in practice: some industry leader tools

CONFORMIQ

- Different commercial tools:
  1. **Creator**:
     - Model: activity diagram
     - Many import possibilities: BPMN, Flow, Gherkin, manual tests…etc.
     - Requirements can be downloaded from requirement management tools

BPMN: Business Process Model and Notation

MBT in practice: some industry leader tools



MBT in practice: some industry leader tools

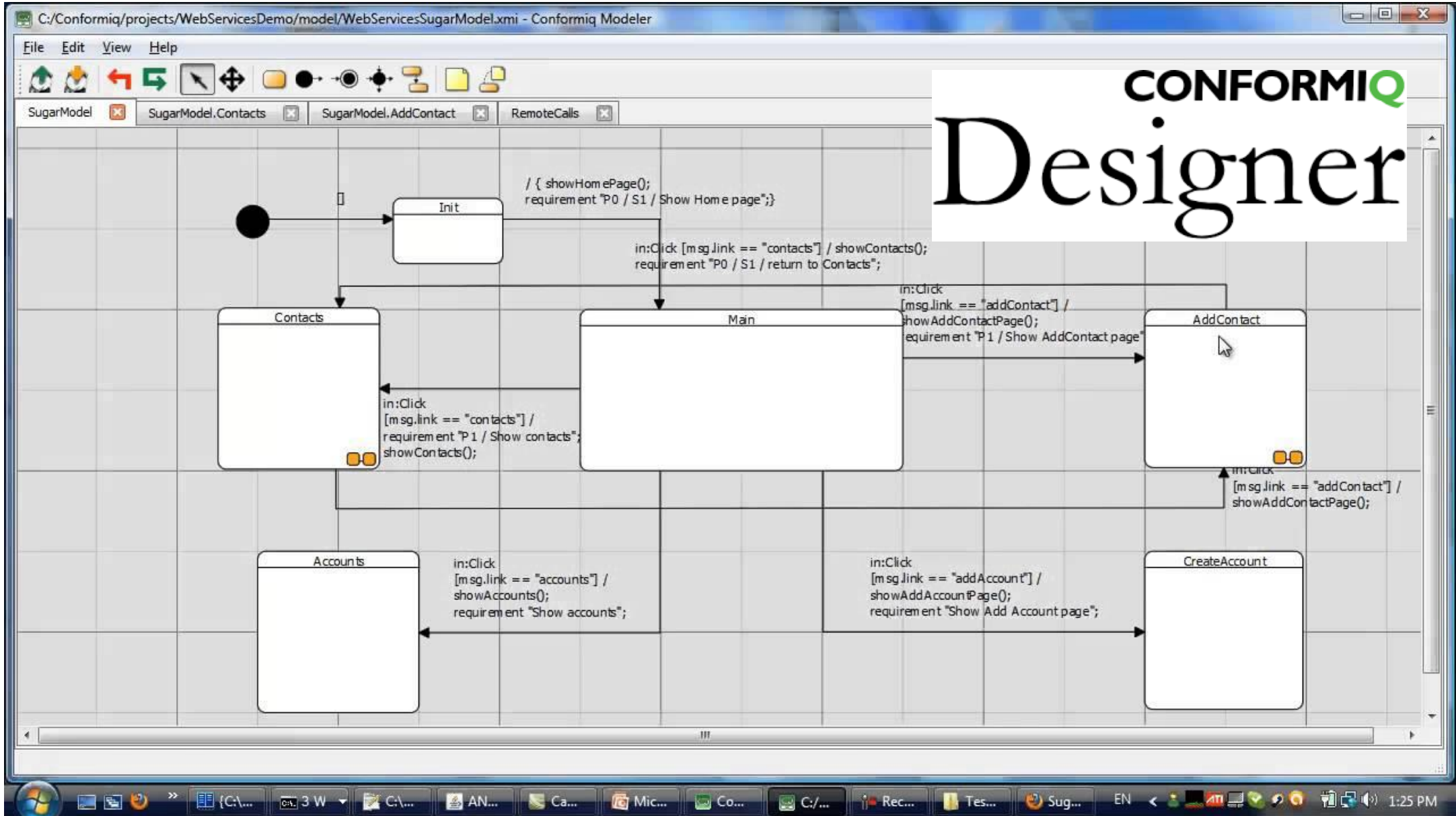
CONFORMIQ

2. **Designer** (previously Qtronic)
   - Model: EFSMs
   - Input format: UML State Machines with some Java-like code *(in Qtronic Modeling Language)*
   - Offline and online MBT
   - Various coverages:
     - Requirements coverage, state/transition coverage, branch coverage, atomic condition coverage or boundary value analysis
   - Incremental test generation: reuses old tests after change if possible

     (but test cases are generated by batch method)
   - Tests can be exported to test management tools or TTCN-3
   - No proper version handling of models → teamwork?

EFSM: Extended Finite State Machine
UML: Unified Modeling Language
TTCN-3: Testing and Test Control Notation version 3

MBT in practice: some industry leader tools

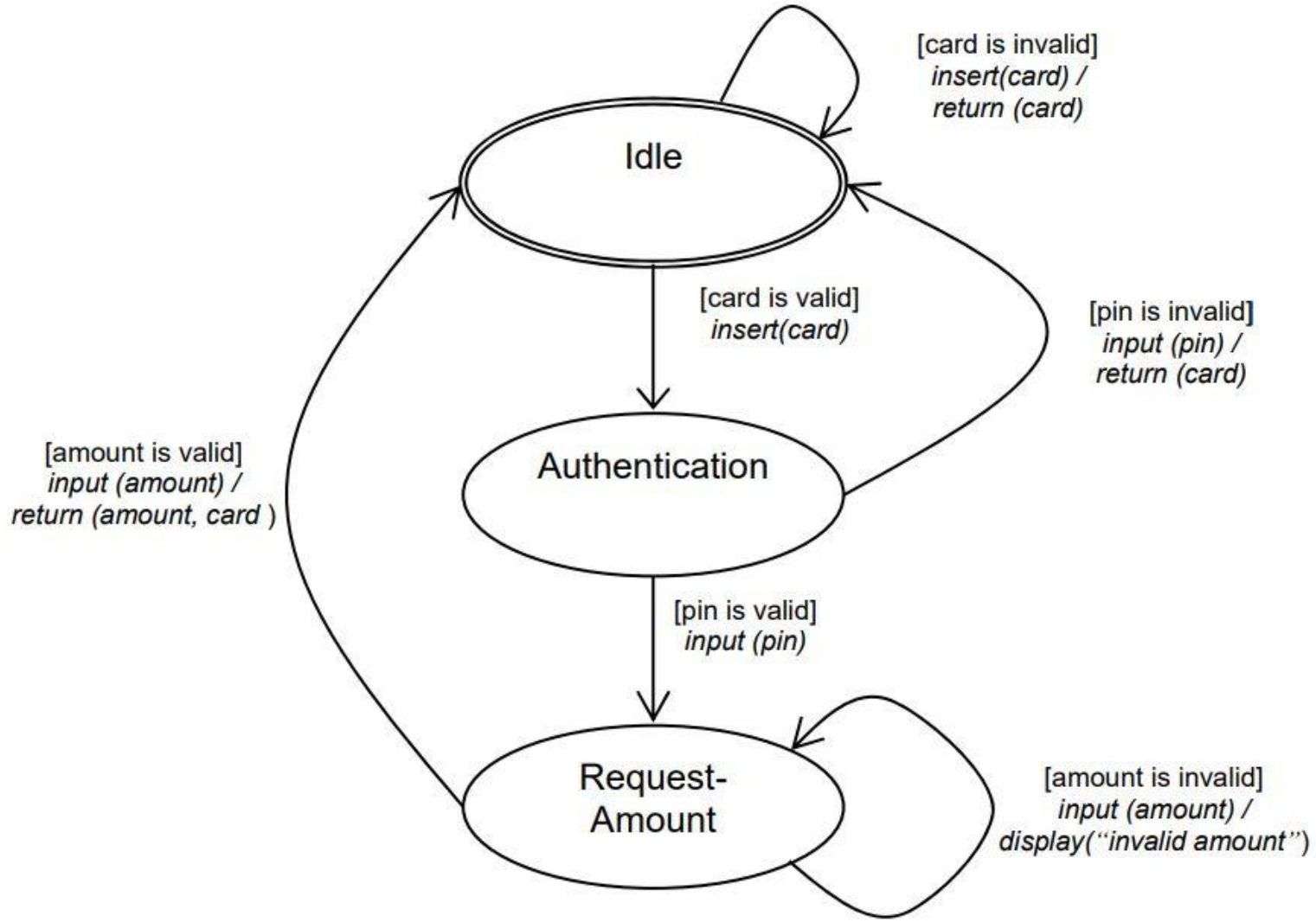


MBT in practice: some industry leader tools

The ATM example with

CONFORMIQ Designer

1. Initial general FSM model of a bank ATM



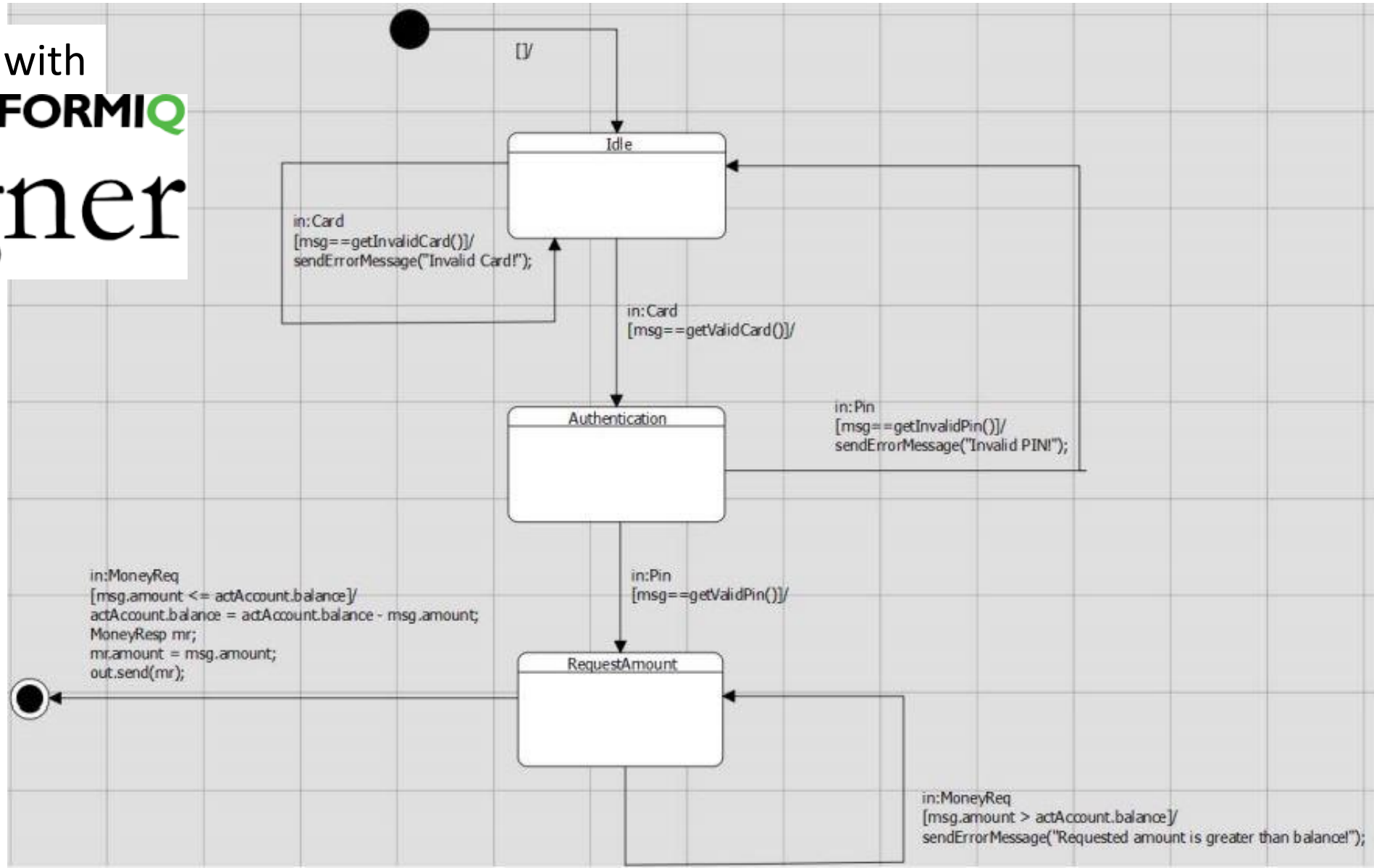
Source: ETSI study

MBT in practice: some industry leader tools

The ATM example with

CONFORMIQ Designer

2. Model in Conformiq Modeler

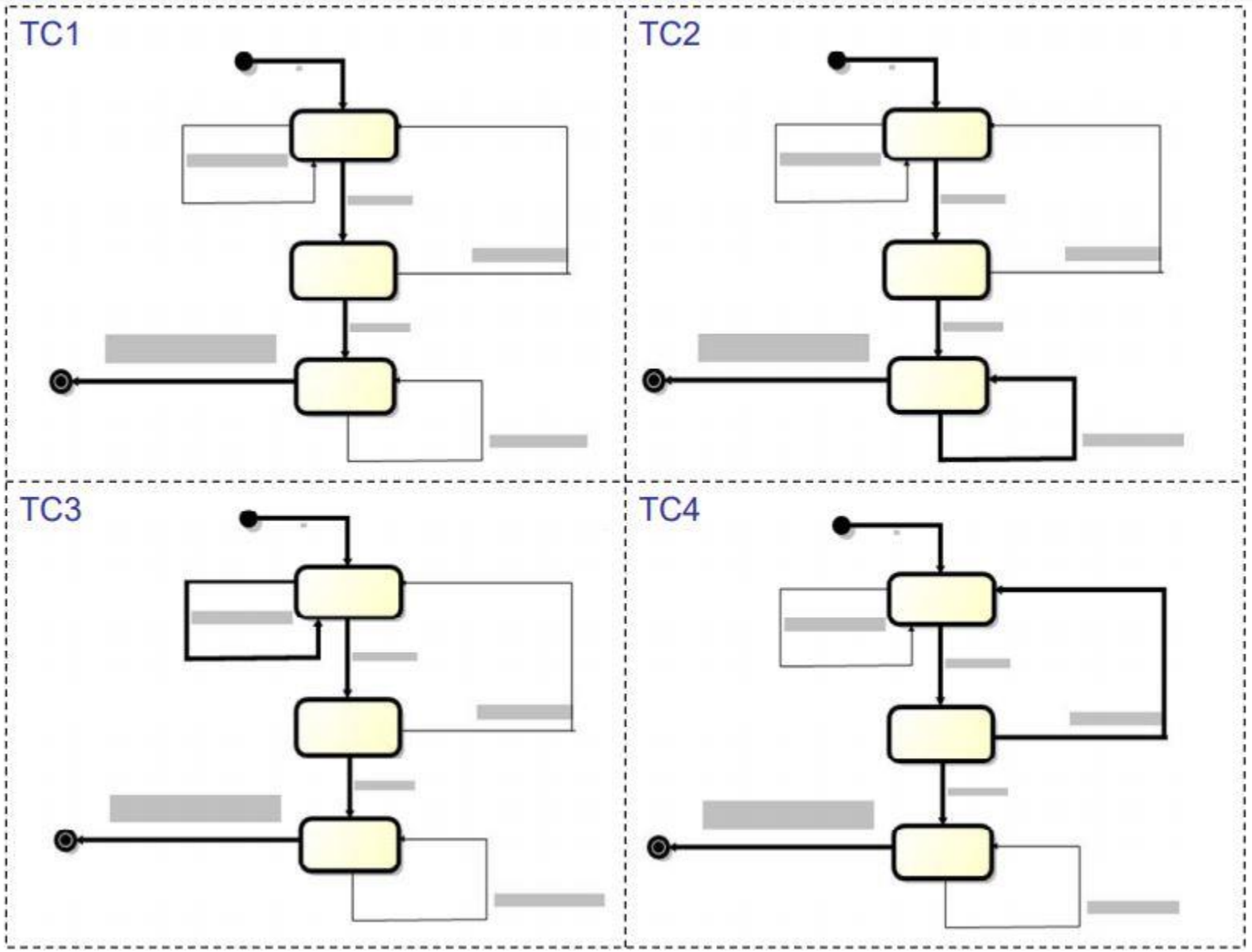


Source: ETSI study

MBT in practice: some industry leader tools

The ATM example with

CONFORMIQ Designer



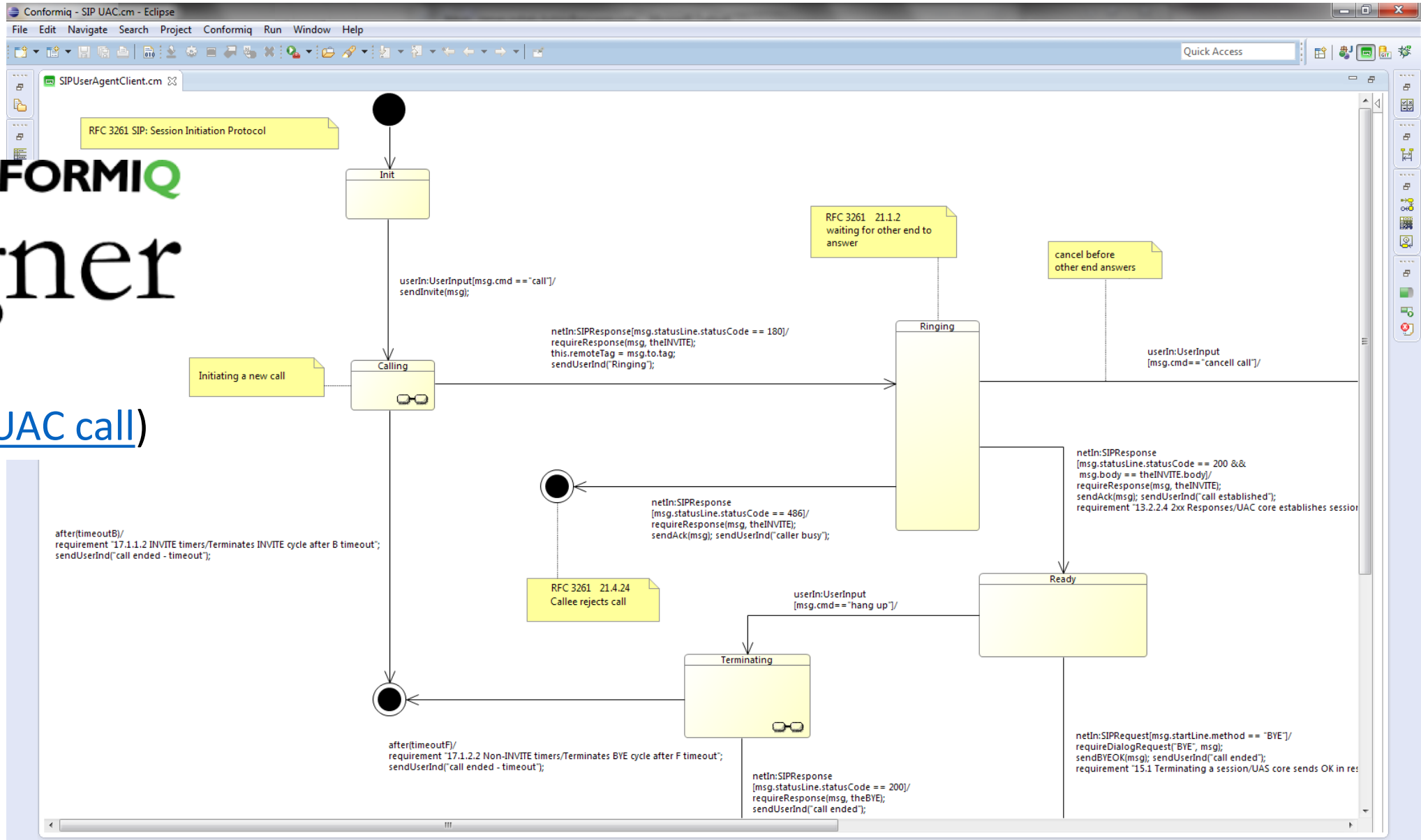
3. Test cases with transition and state coverage generated by Conformiq Designer

Source: ETSI study

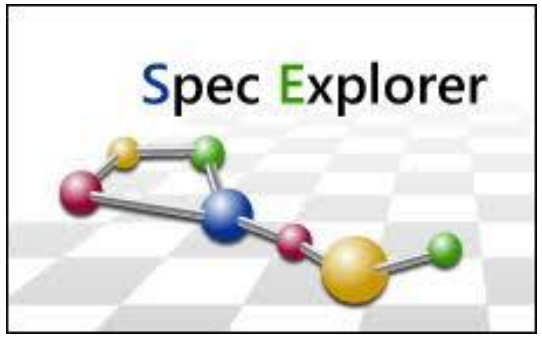
MBT in practice: some industry leader tools

CONFORMIQ Designer

A bigger project
([RFC 3261](#) / [SIP UAC call](#))

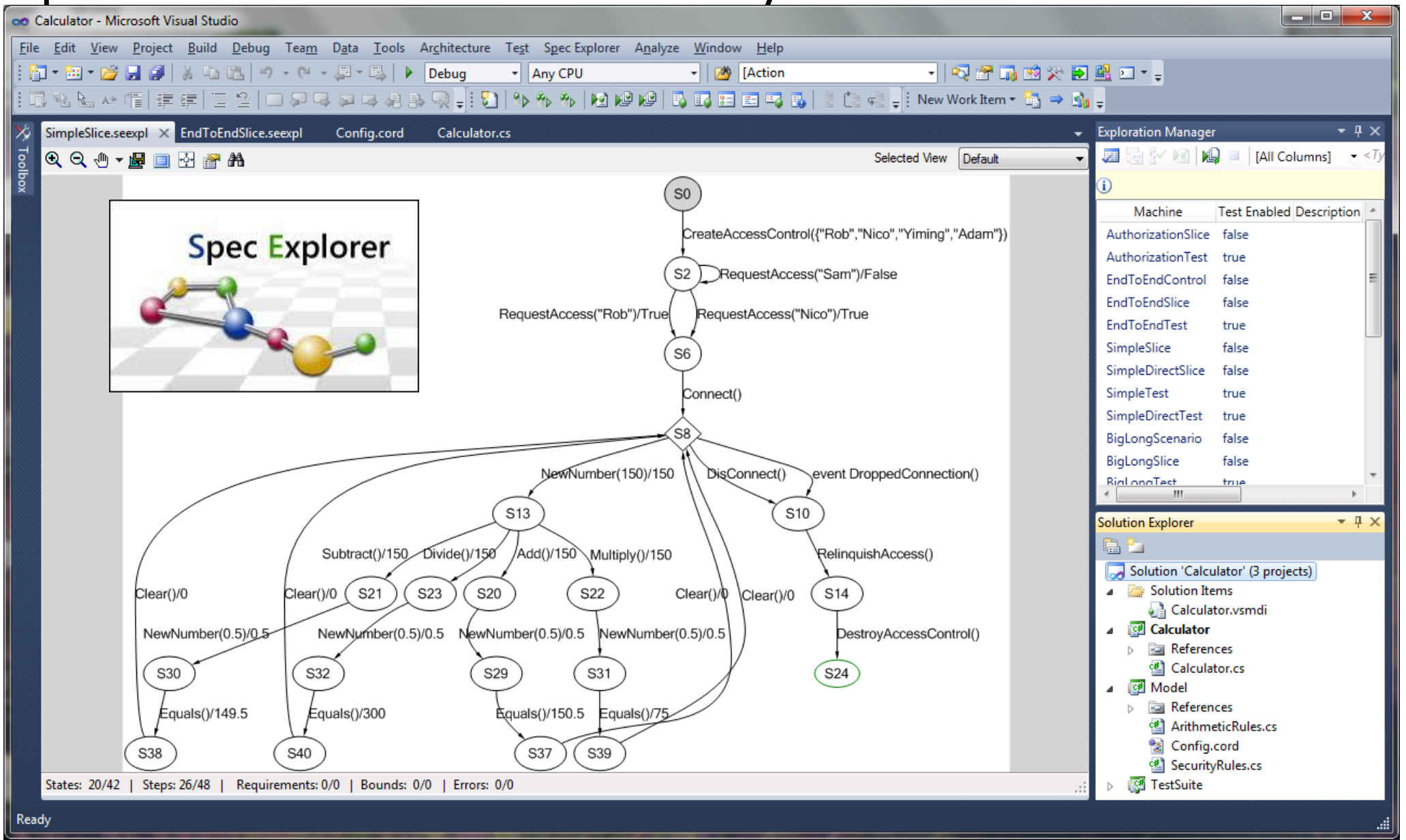


MBT in practice: some industry leader tools

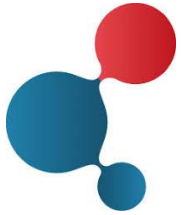


- It was a commercial product, but **not developed anymore**
- Model: FSM (visualization only)
- Input: model programs in C#
  - Good, but low level specification language
- Offline and online MBT
- The number of tests generate and their coverage, can be controlled only indirectly by the parameters of state machine exploration
  - Random walk, shortest path, transition coverage

MBT in practice: some industry leader tools



MBT in practice: some industry leader tools

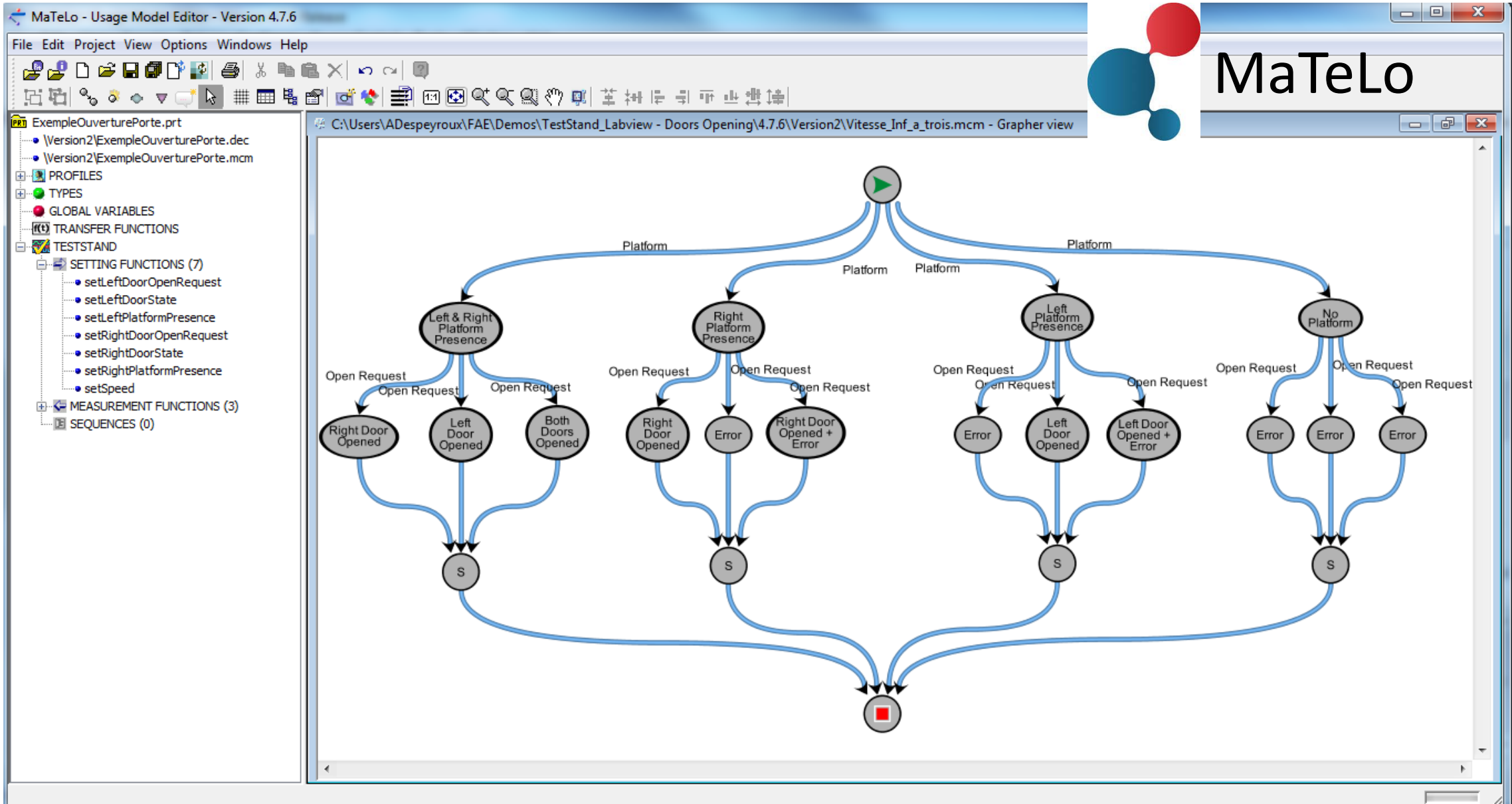


MaTeLo

- Commercial tool
- Model: Markov chains
- Offline MBT
- Strategies:
  - Random /most probable route generation oriented by profiles
  - All states/transitions coverage
  - Boundary value
- Can be connected to numerous test platforms
  - Test script generation for Selenium, Java, TTCN-3, UFT, Matlab…etc.
- Supports teamwork on the same model

UFT: Unified Functional Testing

MBT in practice: some industry leader tools



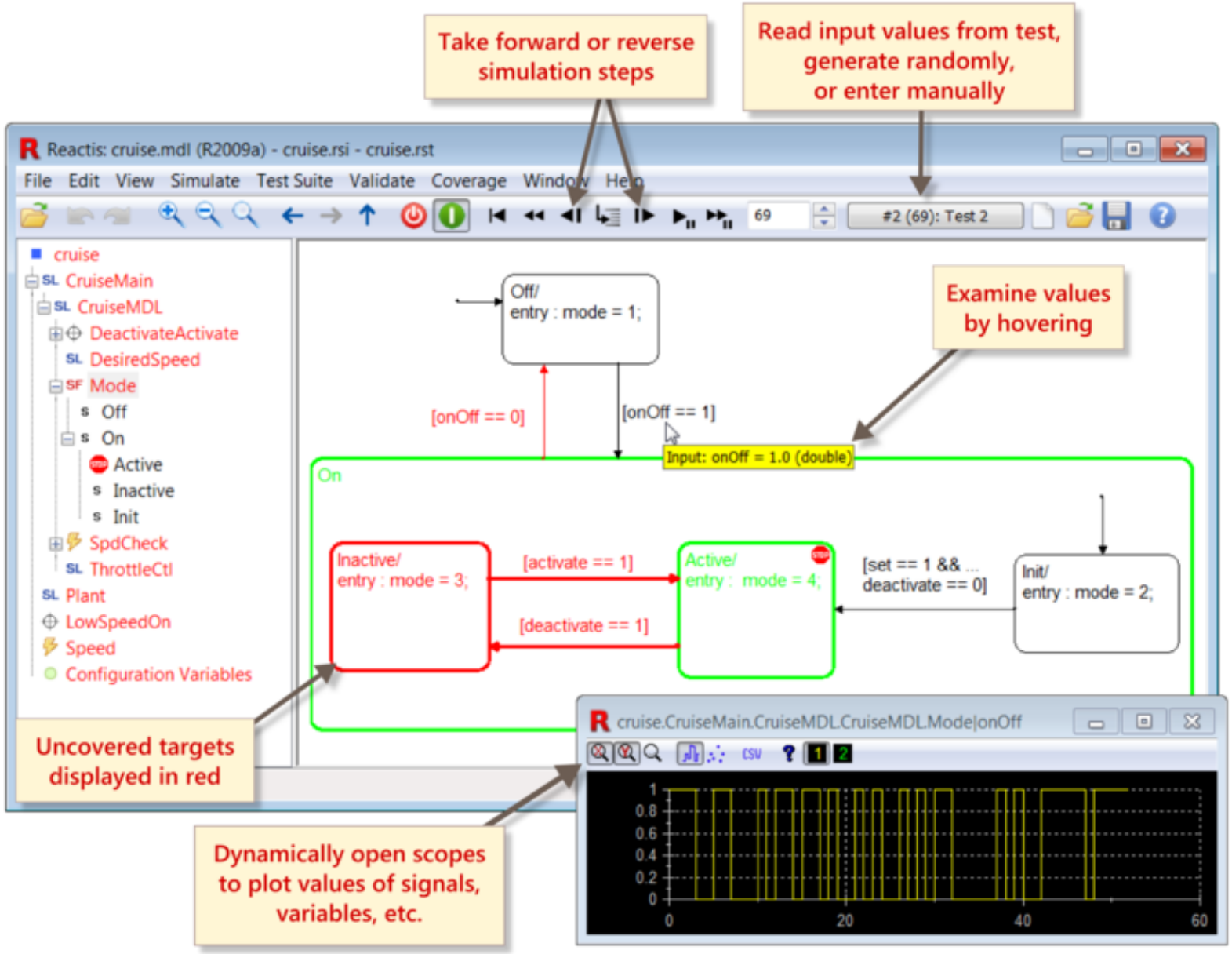
MBT in practice: some industry leader tools

Reactis®

- Commercial tool
- Build upon Matlab/Simulink
- Offline MBT
- Model: EFSM
  - Model editing with an external tool: Matlab/Simulink/Stateflow
- Different coverage techniques:
  - State, Condition-Action, Transition-Action
  - Branch, Lookup Table…etc.
  - Boundary value
  - Individual coverage targets
- Random walk

MBT in practice: some industry leader tools

Reactis®



MBT in practice: some open source and free tools

Tool	Model	Algorithms / coverage / test case selection	Online/ offline MBT	languages	Interfaces	Notes
GraphWalker *Demo through an example project*	FSM/EFSM like (graph)	(weighted/unweighted) random (various stopping conditions), A*	both	C#, python, perl	Maven, Selenium, JUnit	Can import test suites generated by Model 》 Test 》 Relax
Model 》 Test 》 Relax M▶T▶R	FSM/EFSM	Simple/weighted Random, All-State, Transition Tour, All-Transition-State, All-Transition-Transition, HSI-method, H-method, N-Switch Coverage	offline	-	GraphWalker	Developed in ELTE Faculty of Informatics
fMBT	EFSM (trans-formed FSM)	Heuristics (permutations of consecutive elements)	both	Python, C++, JavaScript, shell script	Selenium	
Modbat	FSM like	Heuristics	both	Java		Specialized to API testing
MBT with JUnit	FSM/EFSM	Different coverage metrics	offline	Java	JUnit	
Harmony *Demo from István Forgács*	Domain specific test language	Test cases are generated from each acceptance criterion	offline	Gherkin++	OpenKeyWord (Playwright)	Free, but not open source. Commercial version also exists.

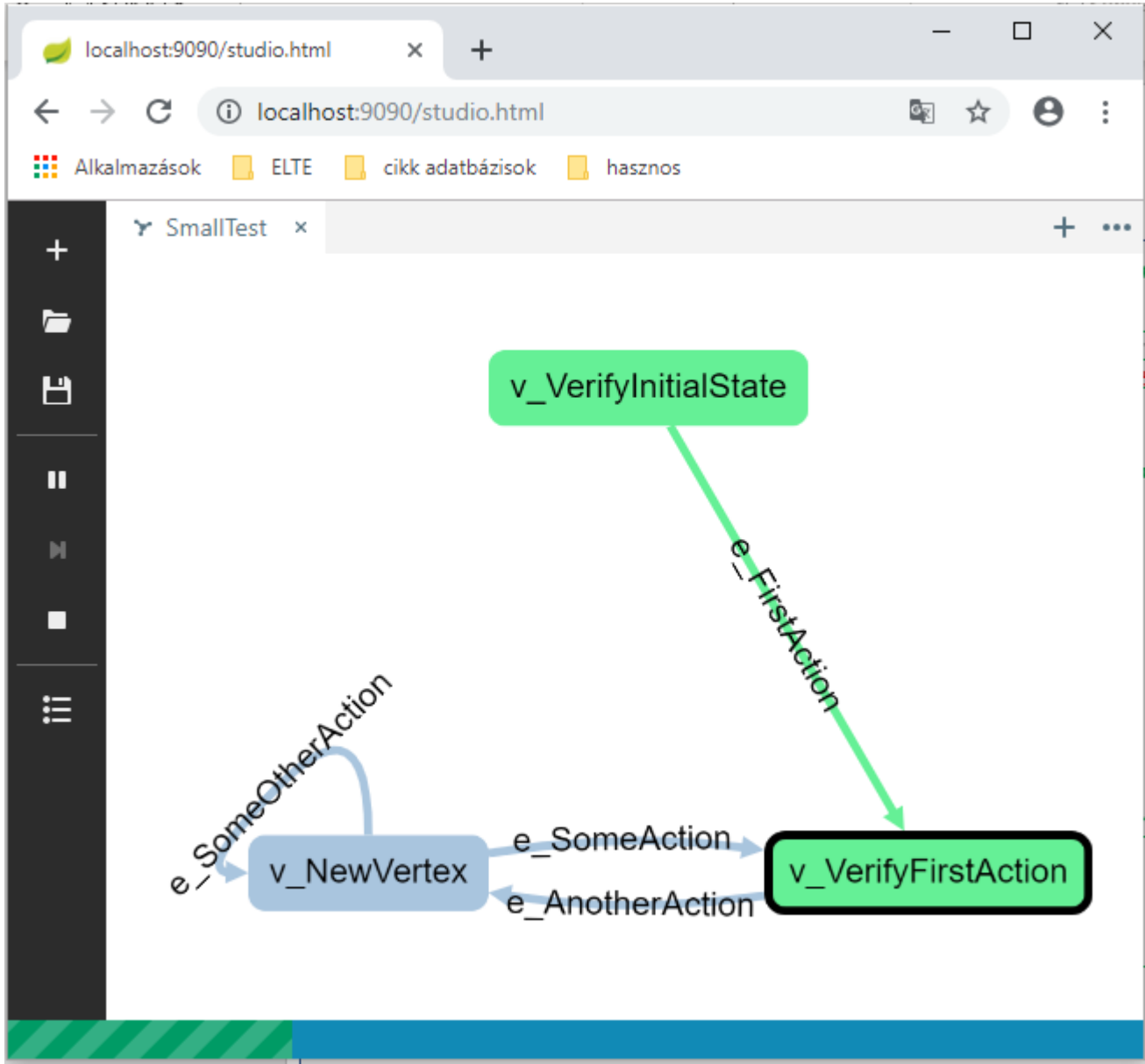
MBT in practice: trends

- The simple, easy-to-use tools are more popular than the robust tools that can handle complex behaviours
- Interfaces to existing tools are more important for users than extensive set of functionalities

GraphWalker demo: Introduction 2/1

- Open source, free
- Easy-to-use
- Has good documentation
- Support in GW forum

- Model: “EFSM like” graphs
  - Not follows EFSM formalism 100%:
    - No input/output symbols in the model
    - Actions can be added into states
- Offline & online MBT
- Other tools should be used for test execution: Maven interface, can be used together with Junit and Selenium



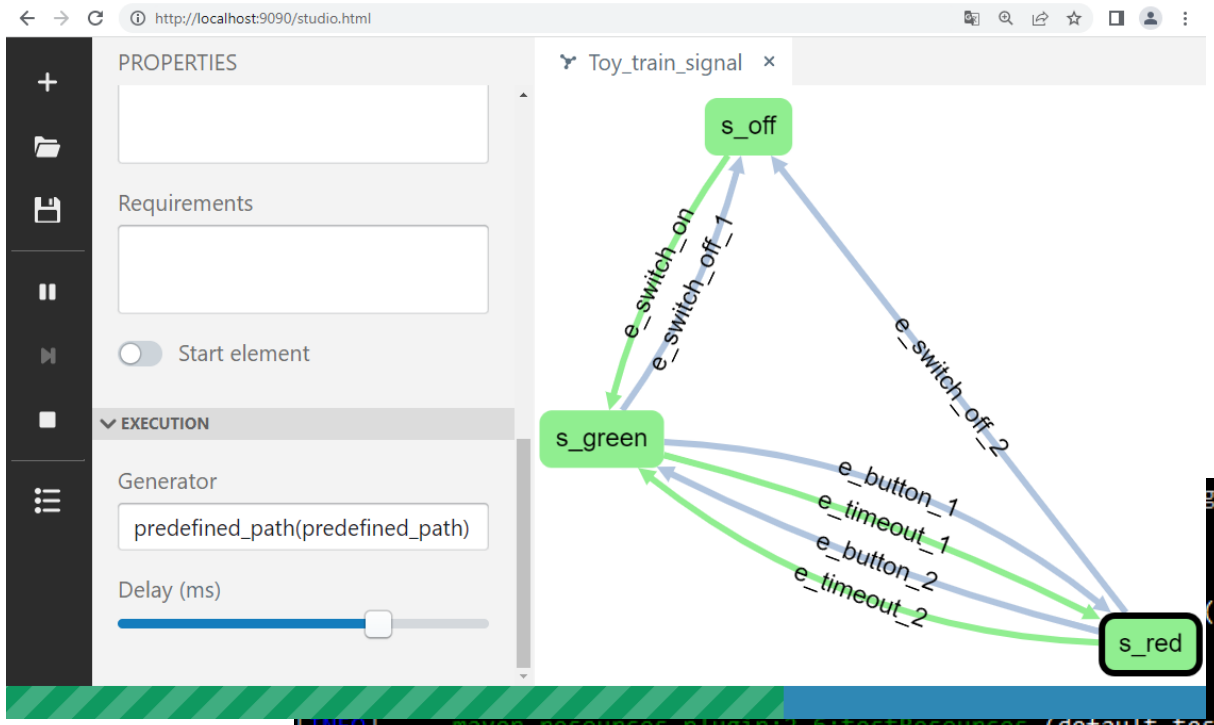
GraphWalker demo: Introduction 2/2

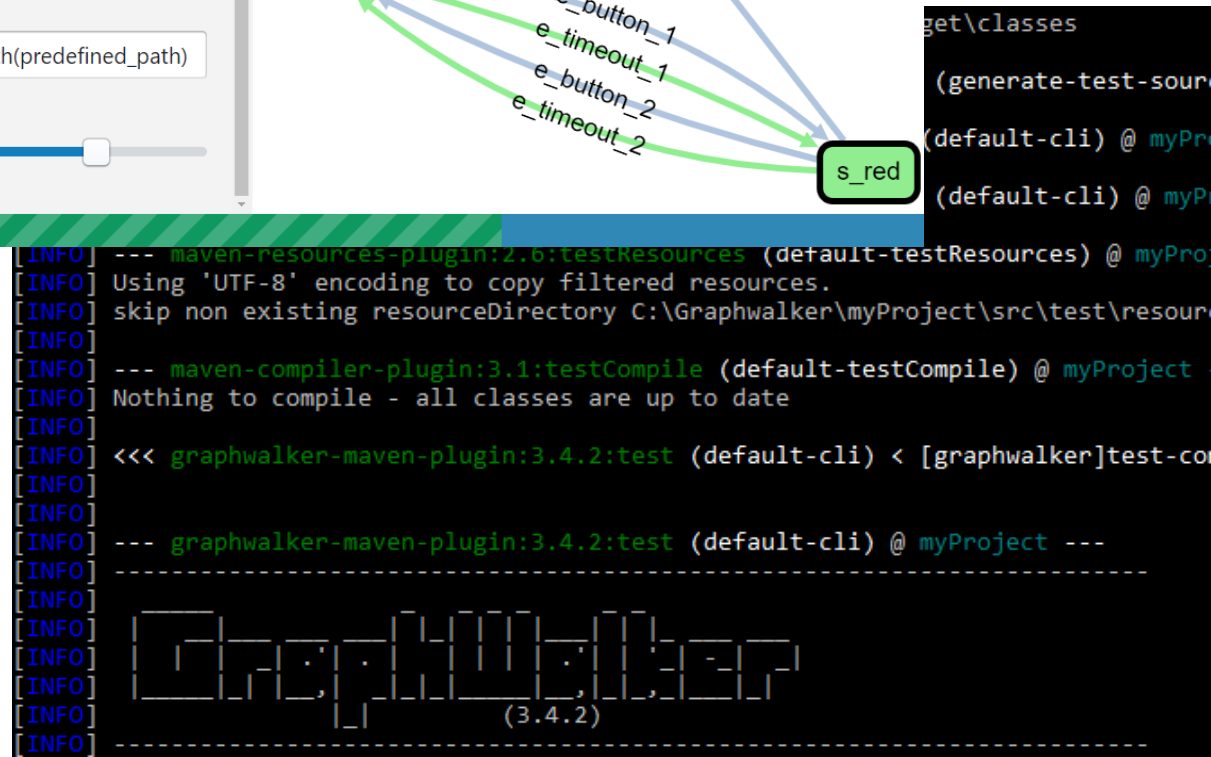
Working process:

1. Creating EFSM (graph) model
2. Creating the adaptation code (in C#, python or perl) related to the graph that interacts with the SUT
3. Executing the test sequences
   - Online or offline path generation
   - Algorithms:

     (weighted, simple, quick) random walk, A*
   - Stopping conditions:

     edge / vertex / requirement / length / duration / reached vertex/edge coverages





GraphWalker demo: An Overview

- GraphWalker installation and prerequisites

1) Create a small GraphWalker test from scratch
   - Investigate main components, understand working principle
2) Create a sample project for GraphWalker using maven
   - Practice in graph editing, investigate generators and stopping conditions
3) GraphWalker Studio:
   - Editing models, investigate generators and stopping conditions
4) Learn by doing: Battery-operated signal example
   - Create new project
   - Draw FSM
   - Write corresponding code skeleton
   - Run tests, investigate different generators and stopping conditions
   - Extend the FSM with variables and guarding conditions into an EFSM

Note that step 1) can be skipped and one can start with step 2), but due to possible problems with maven integration it is advised to check Graphwalker without maven first…

GraphWalker demo: Installation

I. Prerequisites:
   1. Download and install Java JDK and add to path

      (using *JAVA_PATH* system environment variable in Windows)

      JAVA_HOME C:\Program Files\Java\jdk1.8.0_192

   2. Download and install Maven and add to path

      (using system environment variables in Windows)

      C:\Program Files\apache-maven-3.6.0\bin

      Check it with `mvn -ver`

II. (E)FSM editor:
   - Download GW studio

III. Set up GraphWalker environment:
   1. Download GraphWalker CLI and put it into a desired folder

GraphWalker demo: small test from scratch 2/1

1) Create a small GraphWalker test from scratch
   - Create *gw_test/src/test/java/* folder under the folder of GraphWalker
   - Create a sample test file called *ExampleTest.java* in this folder
     - What the *ExampleTest.java* actually does?
       - Import
       - Functions for graph/EFSM elements (3 vertices/states, 1 edge/transition, 1 action, 2 guard conditions)
       - 3 test cases (called success / failure / exception):

```
40      @Test
41      public void success() {
42          Vertex start = new Vertex();
43          Model model = new Model().addEdge(new Edge()
44                  .setName("edge1")
45                  .setGuard(new Guard("isTrue()"))
46                  .setSourceVertex(start
47                          .setName("vertex1"))
48                  .setTargetVertex(new Vertex()
49                          .setName("vertex2"))
50                  .addAction(new Action("myAction();")));
51          this.setModel(model.build());
52          this.setPathGenerator(new RandomPath(new VertexCoverage(100)));
53          setNextElement(start);
54          Machine machine = new SimpleMachine(this);
55          while (machine.hasNextStep()) {
56              machine.getNextStep();
57          }
58      }
```

1. Creating a start node
2. Creating the graph/EFSM model with adding edge with related parameters
3. Build the graph model
4. Set path generator for test with given type and percent of coverage used as stopping condition
5. Set start vertex as entry point of execution
6. Create machine that will control execution
7. Run next step of test case while stopping condition does not hold

GraphWalker demo: small test from scratch 2/2

- Create a Maven *pom.xml* Project Object Model file in *gw_test/* folder
  - *Please check that if GraphWalker version is appropriate:*
- Type `mvn test` (under folder *gw_test/*)
  - You should get a screen similar to this:

```
<dependency>
    <groupId>org.graphwalker</groupId>
    <artifactId>graphwalker-core</artifactId>
    <version>${graphwalker.version}</version>
</dependency>
```

```
<graphwalker.version>[redacted]</graphwalker.version>
```

```
C:\Graphwalker\gw_test>mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------------< org.myorg:example >-------------------------
[INFO] Building GraphWalker Test 1.0.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ example ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory C:\Graphwalker\gw_test\src\main\resources
[INFO] -------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running ExampleTest
10:19:09.481 [main] DEBUG o.g.core.machine.SimpleMachine - Context: ExampleTest@3d1cfad4
10:19:09.483 [main] DEBUG o.g.core.machine.ExecutionContext - Execute vertex1
vertex1
10:19:09.534 [main] DEBUG o.g.core.machine.SimpleMachine - Context: ExampleTest@3d1cfad4
10:19:09.535 [main] DEBUG o.g.core.machine.ExecutionContext - Execute org.graphwalker.core.model.Guard@1649b0e6 isTrue()
10:19:09.549 [main] DEBUG o.g.core.machine.ExecutionContext - Execute myAction();
Action called
Results :

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.588 s
[INFO] Finished at: 2019-04-04T10:19:09+02:00
[INFO] ------------------------------------------------------------------------
```

Also check the *target* subfolder for logs and reports…

GraphWalker demo: create project using Maven 2/1

2) Create a sample project for GraphWalker using Maven (in GraphWalker's folder):

```
mvn archetype:generate -B -DarchetypeGroupId=org.graphwalker -
DarchetypeArtifactId=graphwalker-maven-archetype -DgroupId=com.company -
DartifactId=myProject -DarchetypeVersion=LATEST
```

➜ Write appropriate version here (`json vs. graphml format`):

- `3.4.2`: will generate sample graph in graphml for yED
- `LATEST`: use latest version (**4.0.0 & ↑** uses json format for Studio)

You should get a screen similar to this:

```
C:\Graphwalker>mvn archetype:generate -B -DarchetypeGroupId=org.graphwalker -DarchetypeArtifactId=graphwalker-maven-archetype -DgroupId=com.company -DartifactId=myProject
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.0.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.0.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.0.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] Archetype [org.graphwalker:graphwalker-maven-archetype:3.4.2] found in catalog remote
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Archetype: graphwalker-maven-archetype:3.4.2
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: groupId, Value: com.company
[INFO] Parameter: artifactId, Value: myProject
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] Parameter: package, Value: com.company
[INFO] Parameter: packageInPathFormat, Value: com/company
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] Parameter: package, Value: com.company
[INFO] Parameter: groupId, Value: com.company
[INFO] Parameter: artifactId, Value: myProject
[INFO] Project created from Archetype in dir: C:\Graphwalker\myProject
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:49 min
[INFO] Finished at: 2019-03-22T09:09:44+01:00
[INFO] ------------------------------------------------------------------------

C:\Graphwalker>
```

GraphWalker demo: create project using Maven 2/2

A sample project has been created in folder *myProject* with the following files:

- *pom.xml*: Project Object Model xml file that contains information about the project and configuration details used by Maven to build the project
- *src/main/resources/com/company/SmallTest.json:* a small sample graph that can be edited with GW studio
- *src/main/java/com/company/SomeSmallTest.java*: creates a sample test over the graph

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.company</groupId>
  <version>1.0-SNAPSHOT</version>
  <artifactId>myProject</artifactId>
  <name>GraphWalker Example</name>

  <properties>
    <graphwalker.version>3.4.2</graphwalker.version>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.graphwalker</groupId>
      <artifactId>graphwalker-core</artifactId>
      <version>${graphwalker.version}</version>
    </dependency>
    <dependency>
      <groupId>org.graphwalker</groupId>
      <artifactId>graphwalker-io</artifactId>
      <version>${graphwalker.version}</version>
    </dependency>
    <dependency>
      <groupId>org.graphwalker</groupId>
      <artifactId>graphwalker-java</artifactId>
      <version>${graphwalker.version}</version>
    </dependency>
    <dependency>
      <groupId>org.graphwalker</groupId>
      <artifactId>graphwalker-maven-plugin</artifactId>
      <version>${graphwalker.version}</version>
    </dependency>
  </dependencies>
```

src/main/resources/com/company/SmallTest.json

```
{
  "models": [
    {
      "name": "SmallTest",
      "id": "cbca8c67-6d0c-40b4-8afb-032b069a4bc1",
      "generator": "random(edge_coverage(100))",
      "actions": [],
      "vertices": [
        {
          "id": "d9c15804-24de-47ca-9430-9cb93202d9ff",
          "name": "v_VerifyInitialState",
          "actions": [],
          "requirements": [],
          "properties": {
            "x": 206.9166717529297,
            "y": 54.41667175292969
          }
        },
        {
          "id": "fd4435f7-6a42-4a8b-a522-49c6b91a85e1",
          "name": "v_VerifyFirstAction",
```

src/main/java/com/company/SomeSmallTest.java

```
@GraphWalker(value = "random(edge_coverage(100))", start = "e_FirstAction")
public class SomeSmallTest extends ExecutionContext implements SmallTest {

  @Override
  public void e_FirstAction() {
    System.out.println("Running: e_FirstAction");
  }

  @Override
  public void e_AnotherAction() {
    System.out.println("Running: e_AnotherAction");
  }

  @Override
  public void e_SomeAction() {
    System.out.println("Running: e_SomeAction");
  }

  @Override
  public void e_SomeOtherAction() {
    System.out.println("Running: e_SomeOtherAction");
  }

  @Override
  public void v_VerifySomeAction() {
    System.out.println("Running: v_VerifySomeAction");
  }

  @Override
  public void v_VerifySomeOtherAction() {
    System.out.println("Running: v_VerifySomeOtherAction");
  }
}
```

GraphWalker: Define generators and stopping conditions (CLI mode)

- Open the model file (*src/main/resources/com/company/SomeSmallTest.json*):

```
{
  "models": [
    {
      "name": "SmallTest",
      "id": "cbca8c67-6d0e-40b4-8afb-032b069a4bc1",
      "generator": "random(edge_coverage(100))",
      "actions": [],
      "vertices": [
        {
          "id": "d9c15804-24de-47ca-9430-9cb93202d9ff",
          "name": "v_VerifyInitialState",
          "actions": [],
          "requirements": [],
          "properties": {
            "x": 206.9166717529297,
            "y": 54.41667175292969
          }
        },
        {
          "id": "fd4435f7-6a42-4a8b-a522-49c6b91a85e1",
          "name": "v_VerifyFirstAction",
          "actions": [],
          "requirements": [],
          "properties": {
            "x": 348.9166717529297,
            "y": 368.4166717529297
          }
        },
        ...
      ],
      "edges": [
        ...
      ],
      "startElementId": "d9c15804-24de-47ca-9430-9cb93202d9ff"
    }
  ]
}
```

Note that these information are included in the test file (*src/main/java/com/company/SomeSmallTest.java*) in case of GW versions 3.4.2 & ↓

Define generator:

- **random** (some stop condition(s))
- **weighted_random** (some stop condition(s))
- **quick_random** (some stop condition(s))
- **a_star** (a stop condition that names a vertex or an edge)
- **predefined_path** (predefined_path)

Define stopping conditions for tests:

- **edge_coverage** (an integer representing percentage of desired edge coverage)
- **vertex_coverage** (an integer representing percentage of desired vertex coverage)
- **requirement_coverage** (an integer representing percentage of desired requirement coverage)
- **dependency_edge_coverage** (an integer representing dependency threshold)
- **reached_vertex** (the name of the vertex to reach)
- **reached_edge** (the name of the edge to reach)
- **time_duration** (an integer representing the number of seconds to run)
- **length** (an integer representing the total numbers of edge-vertex pairs)
- **never**

Define starting state

Continue with generated interfaces

Continue with GraphWalker CLI

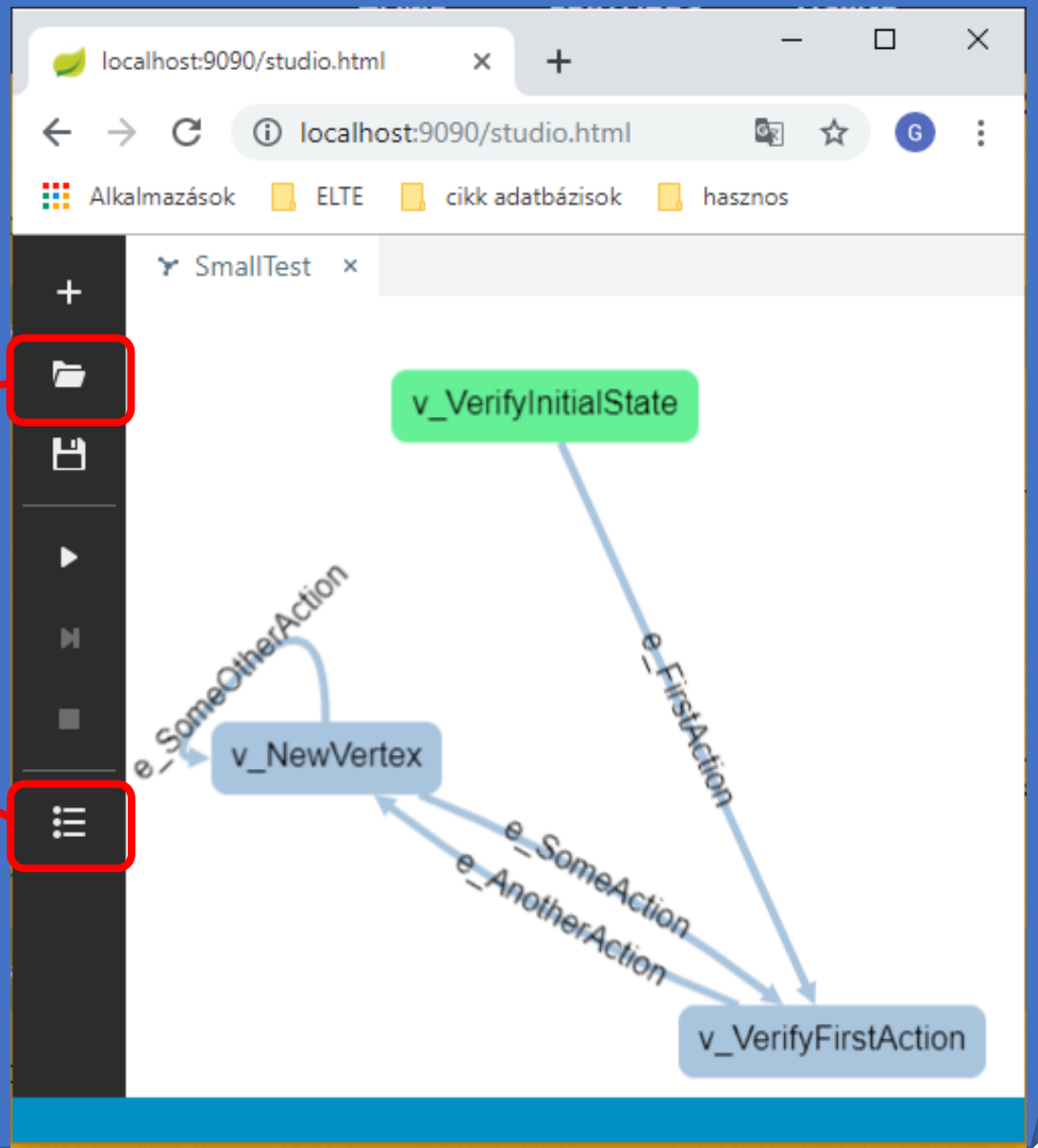
Continue with GraphWalker Studio

More info in user guide

GraphWalker demo: GW studio 5/1

3) GraphWalker studio

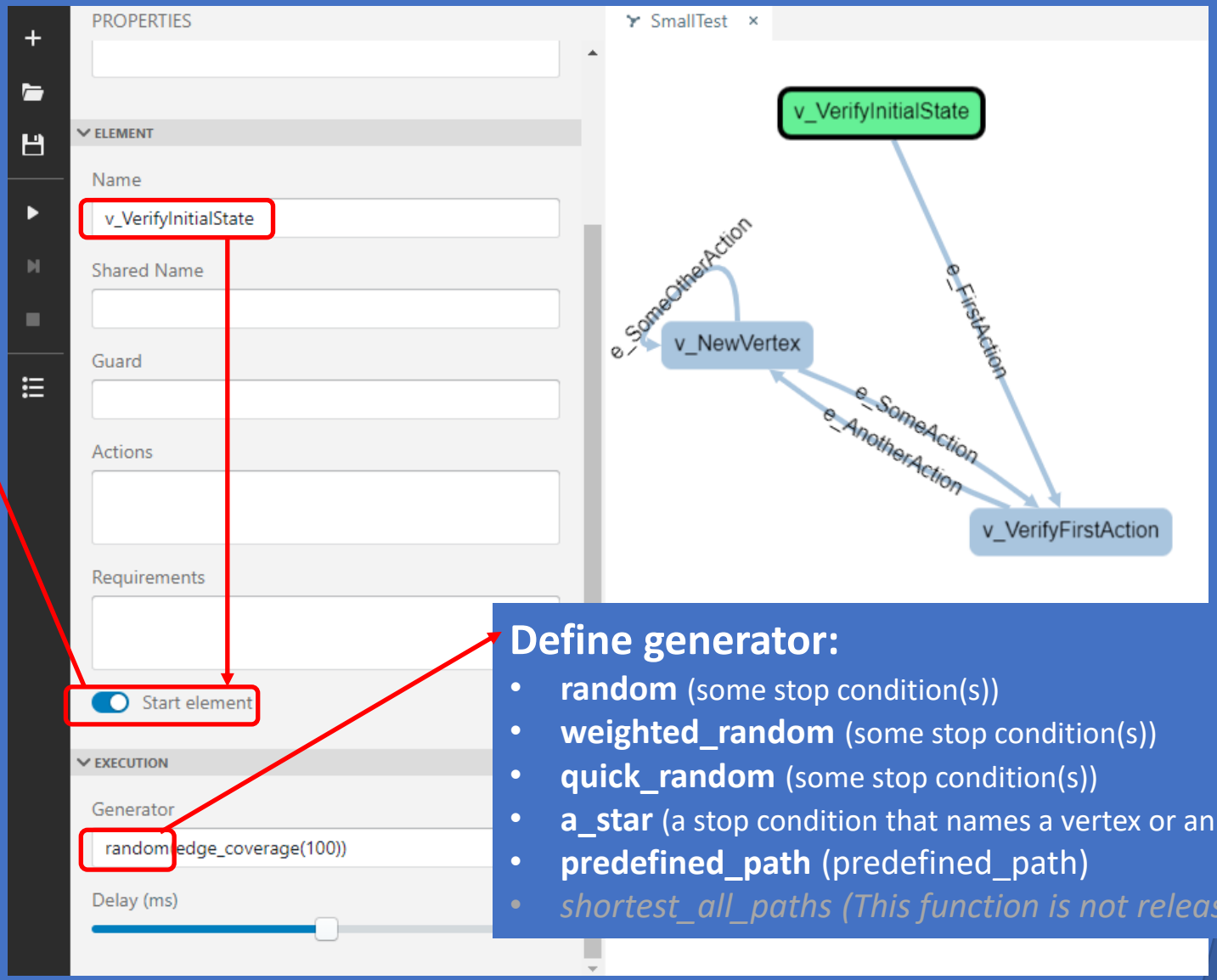
- Start GW studio and type http://localhost:9090/studio.html in your web browser
- Open the sample graph (*src/main/resources/com/company/SmallTest.json*)
- Click on properties icon



GraphWalker demo: GW studio 5/2

- Investigate properties

Define starting state



Define generator:

- **random** (some stop condition(s))
- **weighted_random** (some stop condition(s))
- **quick_random** (some stop condition(s))
- **a_star** (a stop condition that names a vertex or an edge)
- **predefined_path** (predefined_path)
- *shortest_all_paths (This function is not released yet)*

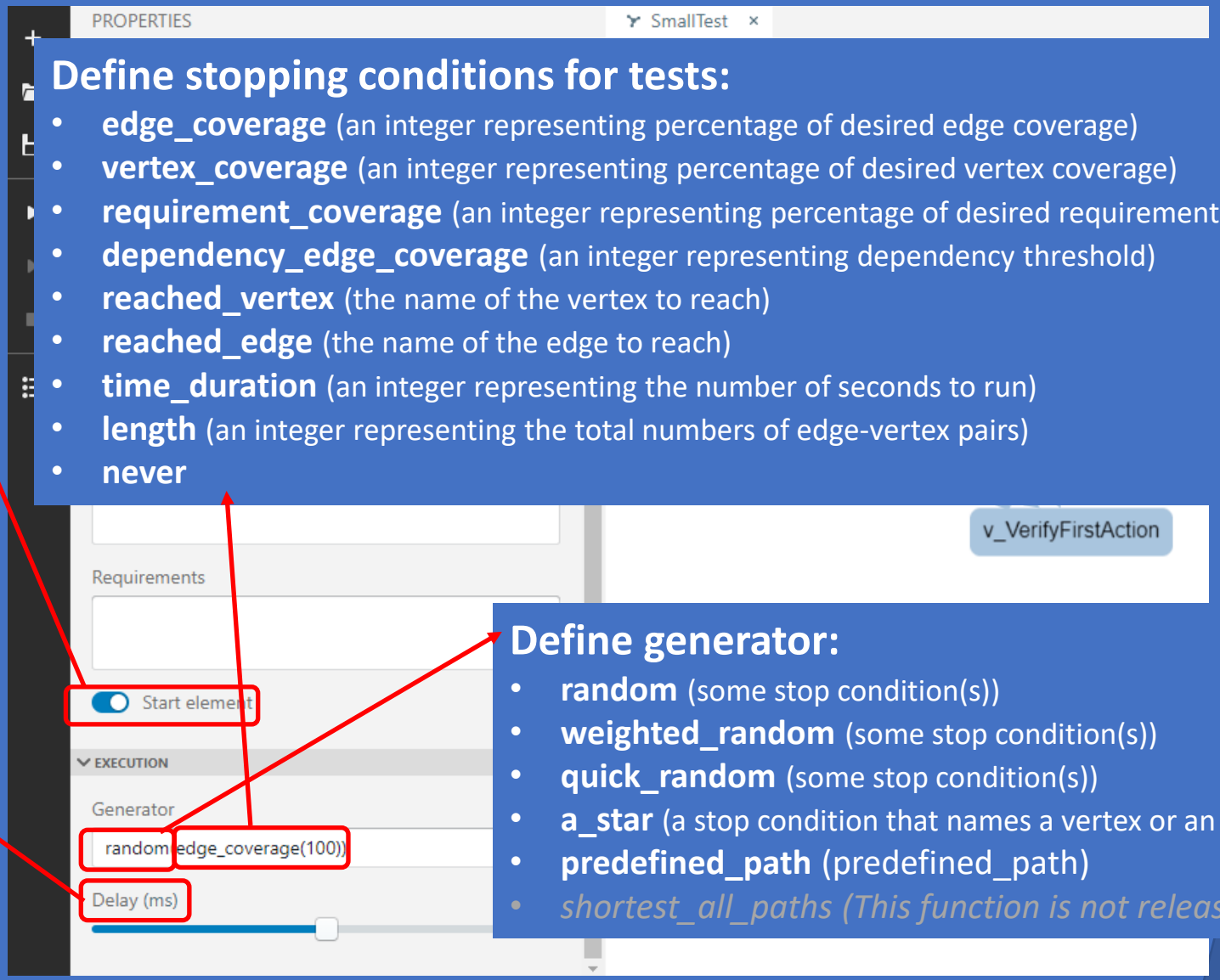
GraphWalker demo: GW studio 5/3

- Investigate properties

Define starting state

Define delays after each step

More info in user guide



Define stopping conditions for tests:

- **edge_coverage** (an integer representing percentage of desired edge coverage)
- **vertex_coverage** (an integer representing percentage of desired vertex coverage)
- **requirement_coverage** (an integer representing percentage of desired requirement coverage)
- **dependency_edge_coverage** (an integer representing dependency threshold)
- **reached_vertex** (the name of the vertex to reach)
- **reached_edge** (the name of the edge to reach)
- **time_duration** (an integer representing the number of seconds to run)
- **length** (an integer representing the total numbers of edge-vertex pairs)
- **never**

Define generator:

- **random** (some stop condition(s))
- **weighted_random** (some stop condition(s))
- **quick_random** (some stop condition(s))
- **a_star** (a stop condition that names a vertex or an edge)
- **predefined_path** (predefined_path)
- *shortest_all_paths (This function is not released yet)*

GraphWalker demo: GW studio 5/4

- Investigate properties

Define starting state

```
{
  "models": [
    {
      "name": "SmallTest",
      "id": "cbca8c67-6d0c-40b4-8afb-032b069a4bc1",
      "generator": "random(edge_coverage(100))",
      "actions": [],
      "vertices": [
        {
          "id": "d9c15804-24de-47ca-9430-9cb93202d9ff",
          "name": "v_VerifyInitialState",

          ...

        }
      ],
      "startElementId": "d9c15804-24de-47ca-9430-9cb93202d9ff"
    }
  ]
}
```

src/main/resources/com/company/SmallTest.json

Define stopping conditions for tests:

- **edge_coverage** (an integer representing percentage of desired edge coverage)
- **vertex_coverage** (an integer representing percentage of desired vertex coverage)
- **requirement_coverage** (an integer representing percentage of desired requirement coverage)
- **dependency_edge_coverage** (an integer representing dependency threshold)
- **reached_vertex** (the name of the vertex to reach)
- **reached_edge** (the name of the edge to reach)
- **time_duration** (an integer representing the number of seconds to run)
- **length** (an integer representing the total numbers of edge-vertex pairs)
- **never**

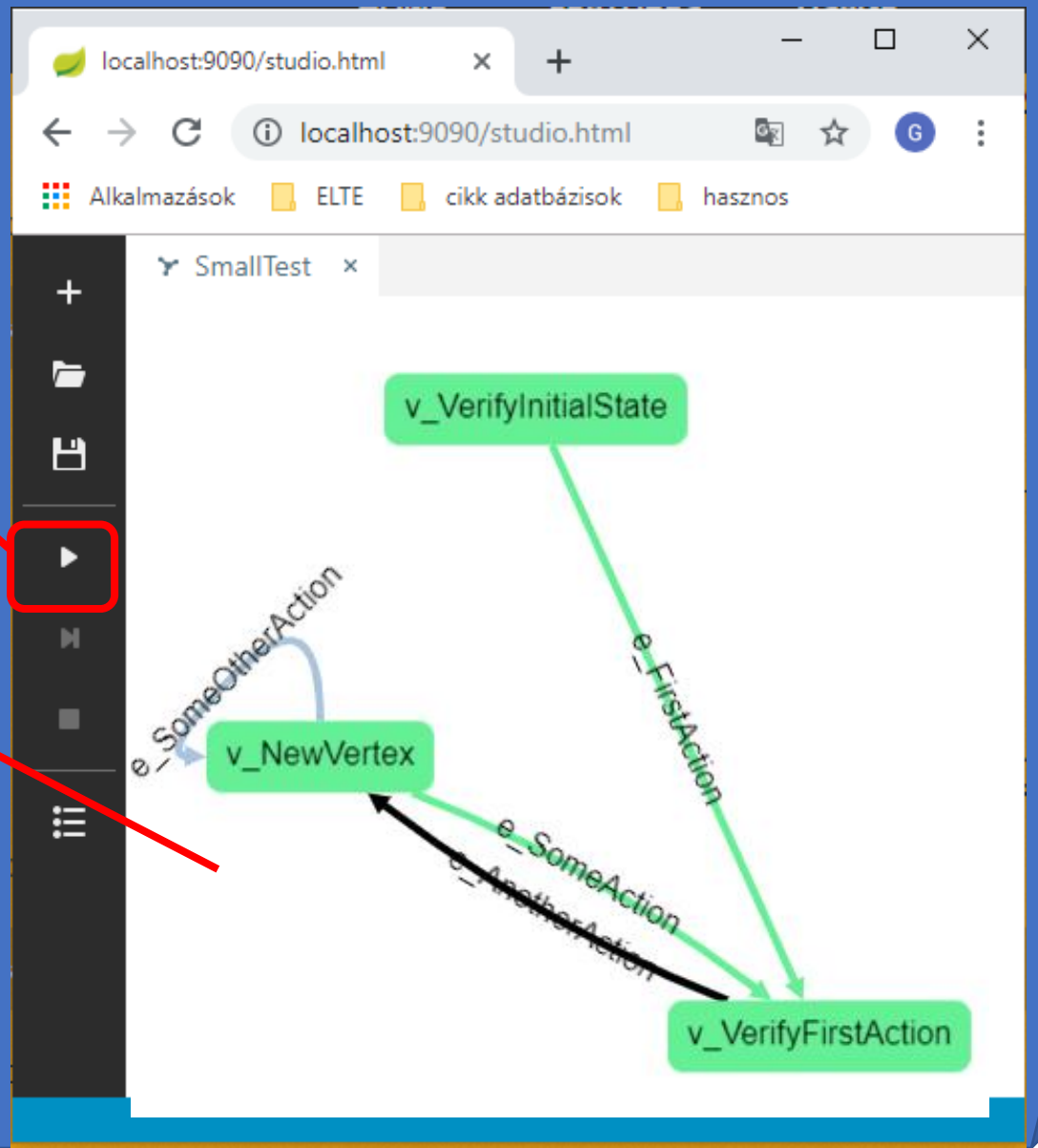
Define generator:

- **random** (some stop condition(s))
- **weighted_random** (some stop condition(s))
- **quick_random** (some stop condition(s))
- **a_star** (a stop condition that names a vertex or an edge)
- **predefined_path** (predefined_path)
- *shortest_all_paths (This function is not released yet)*

GraphWalker demo: GW studio 5/5

- Execute the tests:

- Blue: unvisited transition / state
- Green: traversed transition / visited state
- Black: currently executes



GraphWalker demo: Generated interfaces

- Check the generated files in *target* folder:
  - Note that *target/generated-sources/graphwalker/com/company/* folder contains a java file, that list the interfaces generated from the graph (*src/main/resources/com/company/SmallTest.graphml OR SmallTest.json in this project*):
    - This can be used as a starting point to see what we should implemented in the test

(*src/main/java/com/company/SomeSmallTest.java* in the current sample project)

```java
// Generated by GraphWalker (http://www.graphwalker.org)
package com.company;

import org.graphwalker.java.annotation.Model;
import org.graphwalker.java.annotation.Vertex;
import org.graphwalker.java.annotation.Edge;

@Model(file = "com/company/SmallTest.graphml")
public interface SmallTest {

    @Edge()
    void e_FirstAction();

    @Edge()
    void e_SomeOtherAction();

    @Vertex()
    void v_VerifySomeOtherAction();

    @Edge()
    void e_AnotherAction();

    @Vertex()
    void v_VerifySomeAction();

    @Edge()
    void e_SomeAction();
}
```

Note that if we do not want to run GW tests, this file can be also generated with the following command:

`java -jar graphwalker-cli-<version>.jar methods -m [model file]`

OR

`mvn graphwalker:generate-sources` *(in the maven project folder)*

GraphWalker demo: using CLI 2/1

- Run the tests with CLI (from *myProject* folder):

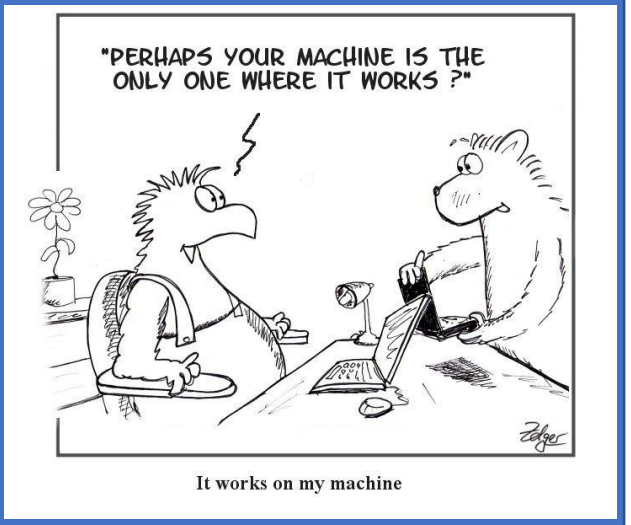
  `mvn graphwalker:test`

  You should get a screen similar to this:

```
[INFO]
[INFO] <<< graphwalker-maven-plugin:3.4.2:test (default-cli) < [graphwalker]test-compile @ myPr
[INFO]
[INFO]
[INFO] --- graphwalker-maven-plugin:3.4.2:test (default-cli) @ myProject ---
[INFO] ------------------------------------------------------------------------
[INFO]   _____                 _    _ _ _     _ _
[INFO]  |  __ \               | |  | | | |   | | |
[INFO]  | |  \/_ __ __ _ _ __ | |__| | | | __| | | _____ _ __
[INFO]  | | __| '__/ _` | '_ \|  __  | | |/ _` | |/ / _ \ '__|
[INFO]  | |_\ \ | | (_| | |_) | |  | | | | (_| |   <  __/ |
[INFO]   \____/_|  \__,_| .__/|_|  |_|_|_|\__,_|_|\_\___|_|
[INFO]                   |_|          (3.4.2)
[INFO] ------------------------------------------------------------------------
[INFO] Configuration:
[INFO]     Include = [*]
[INFO]     Exclude = []
[INFO]      Groups = [*]
[INFO]
[INFO] Tests:
[INFO]     SomeSmallTest(RandomPath, VertexCoverage, 100)
[INFO]
[INFO] ------------------------------------------------------------------------
13:02:06.902 [main] DEBUG o.g.core.machine.SimpleMachine - Context: com.company.SomeSmallTest@2
13:02:06.902 [main] DEBUG o.g.core.machine.ExecutionContext - Execute e_FirstAction
Running: e_FirstAction
13:02:06.996 [main] DEBUG o.g.core.machine.SimpleMachine - Context: com.company.SomeSmallTest@2
13:02:07.012 [main] DEBUG o.g.core.machine.ExecutionContext - Execute v_VerifySomeAction
Running: v_VerifySomeAction
13:02:07.027 [main] DEBUG o.g.core.machine.SimpleMachine - Context: com.company.SomeSmallTest@2
13:02:07.043 [main] DEBUG o.g.core.machine.ExecutionContext - Execute e_AnotherAction
Running: e_AnotherAction
13:02:07.059 [main] DEBUG o.g.core.machine.SimpleMachine - Context: com.company.SomeSmallTest@2
13:02:07.059 [main] DEBUG o.g.core.machine.ExecutionContext - Execute v_VerifySomeOtherAction
Running: v_VerifySomeOtherAction
13:02:07.090 [main] DEBUG o.g.core.machine.SimpleMachine - Context: com.company.SomeSmallTest@2
13:02:07.090 [main] DEBUG o.g.core.machine.ExecutionContext - Execute e_SomeAction
Running: e_SomeAction
Running: v_VerifySomeOtherAction
[INFO] ------------------------------------------------------------------------
[INFO]
[INFO] Result :
[INFO]
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 4,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 4,
  "totalNumberOfVisitedVertices": 2,
  "edgeCoverage": 100,
  "totalNumberOfVertices": 2,
  "totalNumberOfUnvisitedEdges": 0
}
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.577 s
[INFO] Finished at: 2019-04-04T12:53:09+02:00
[INFO] ------------------------------------------------------------------------
```

GraphWalker demo: using CLI 2/2 - Troubleshooting

```
[INFO] --- graphwalker-maven-plugin:3.4.2:test (default-cli) @ myProject ---
[INFO] ------------------------------------------------------------------------
[INFO]
[INFO]  |  __|___ ___ ___| |_| | | |___| | |_ ___ ___
[INFO]  |  |  | _| .'| . |  _| | | | .'| | '_| -_|  _|
[INFO]  |_____|_| |__,|  _|_| |_____|__,|_|_,_|___|_|
[INFO]                |_|          (3.4.2)
[INFO] ------------------------------------------------------------------------
[ERROR] null
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  6.931 s
[INFO] Finished at: 2019-04-04T13:15:52+02:00
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.graphwalker:graphwalker-maven-plugin:3.4.2:test (default-cli) on project myProject
: Execution default-cli of goal org.graphwalker:graphwalker-maven-plugin:3.4.2:test failed: java.lang.reflect.Invocat
ionTargetException: No suitable context factory found -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/PluginExecutionException
```



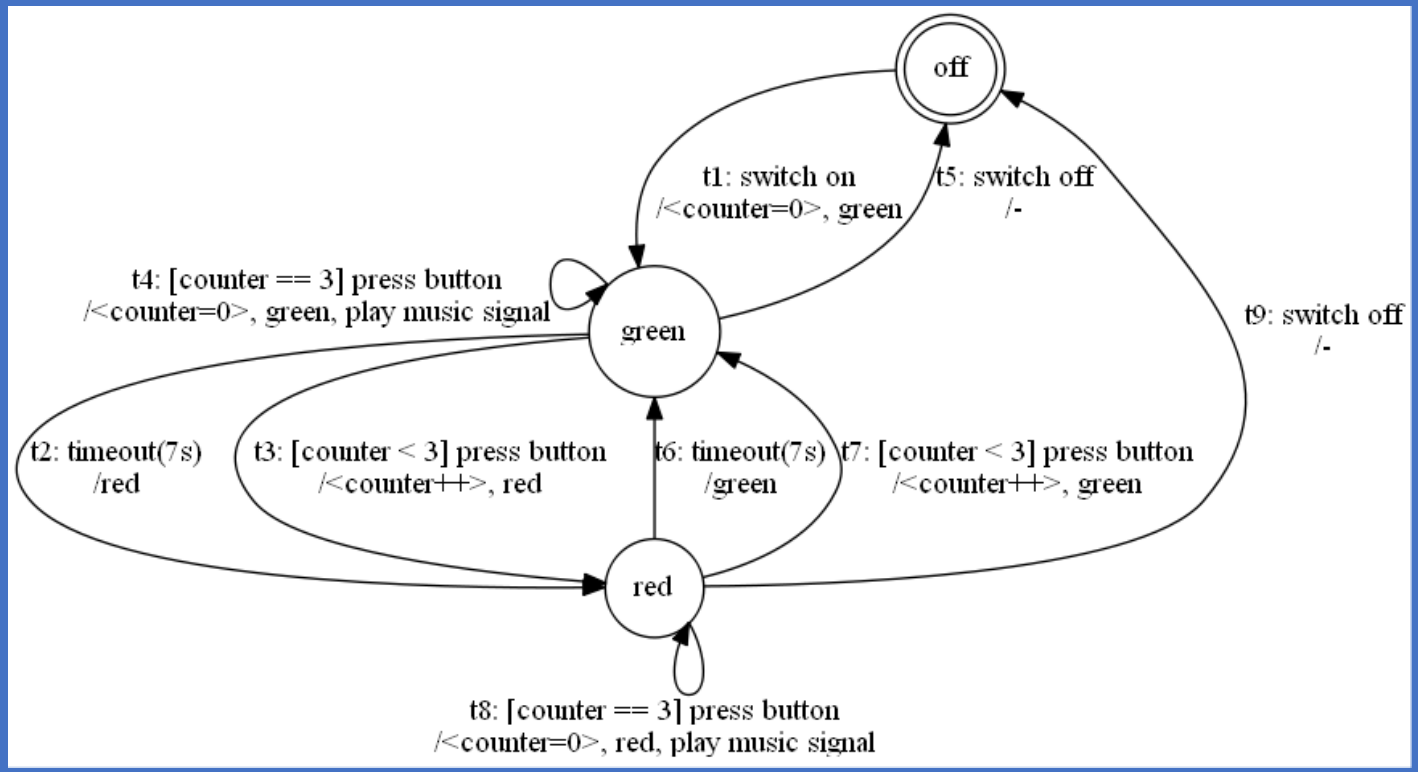
It works on my machine

If build does not work with `mvn graphwalker:test`, **some ideas for troubleshooting**:

- Try `mvn clean` (and optionally `mvn graphwalker:generate-sources`) before run `mvn graphwalker:test`
- In windows use only a user whose name does not contain spaces ☺
- Do not use shared directory
- Check if GraphWalker version is appropriate in *pom.xml* file
- Check the parameter of the generator in the Java code (but in the sample, delivered project it is right)
- Check if your JDK java version is supported

GraphWalker demo: Battery-operated signal

4) Learn by doing using the extended battery-operated signal example
   - Create new project
   - Draw an FSM
   - Write corresponding code skeleton
   - Run tests, investigate different generators and stopping conditions
   - Extend the FSM with variables and guarding conditions



GraphWalker demo: Battery-operated signal

1. Create a new project called TrainSignal using Maven for the battery operated signal:

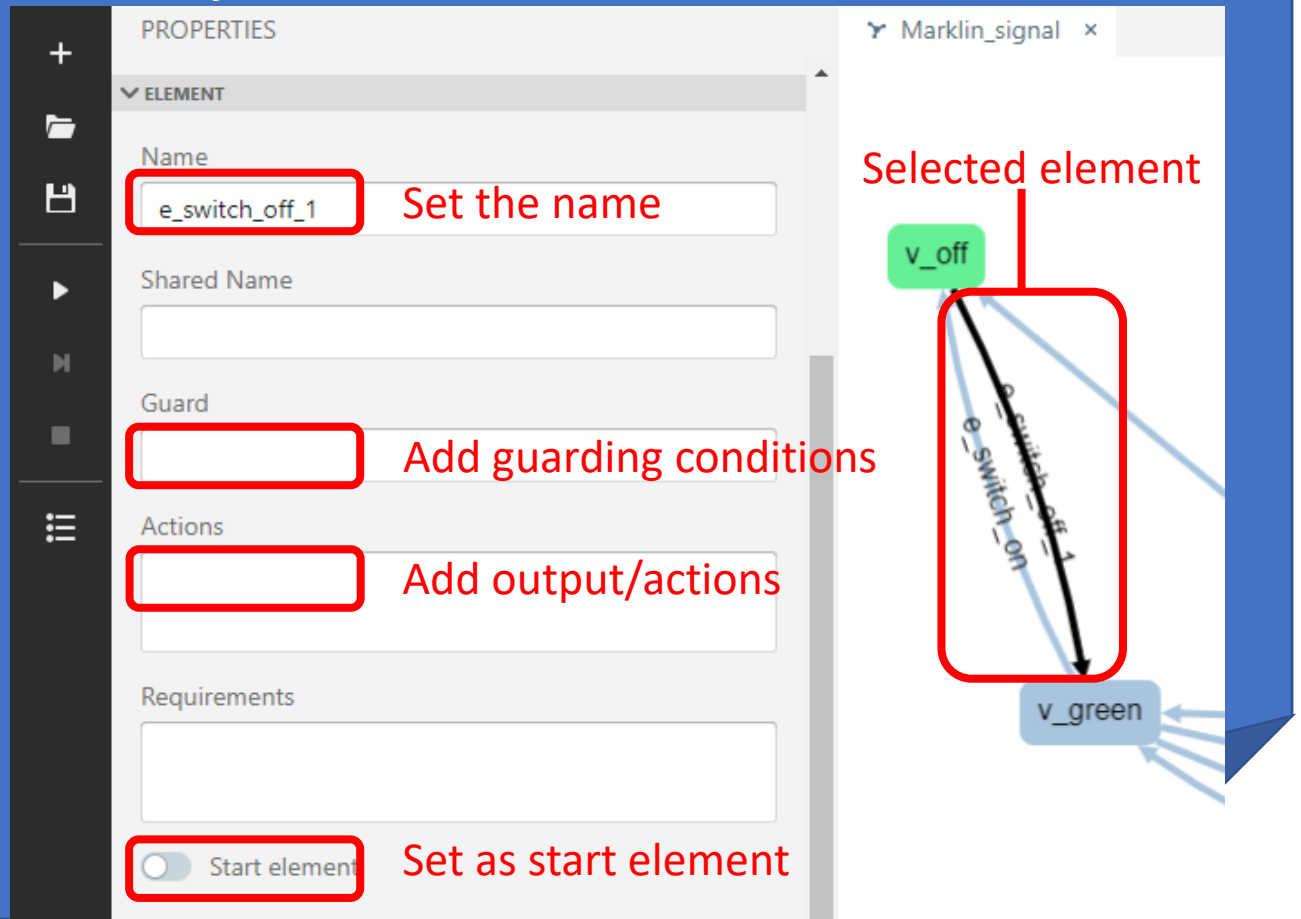
```
mvn archetype:generate -B -DarchetypeGroupId=org.graphwalker -
DarchetypeArtifactId=graphwalker-maven-archetype -DgroupId=com.company -
DartifactId=TrainSignal -DarchetypeVersion=LATEST
```

2. Start GW studio and type http://localhost:9090/studio.html in your web browser

GraphWalker demo: Battery-operated signal

Help for GW studio:

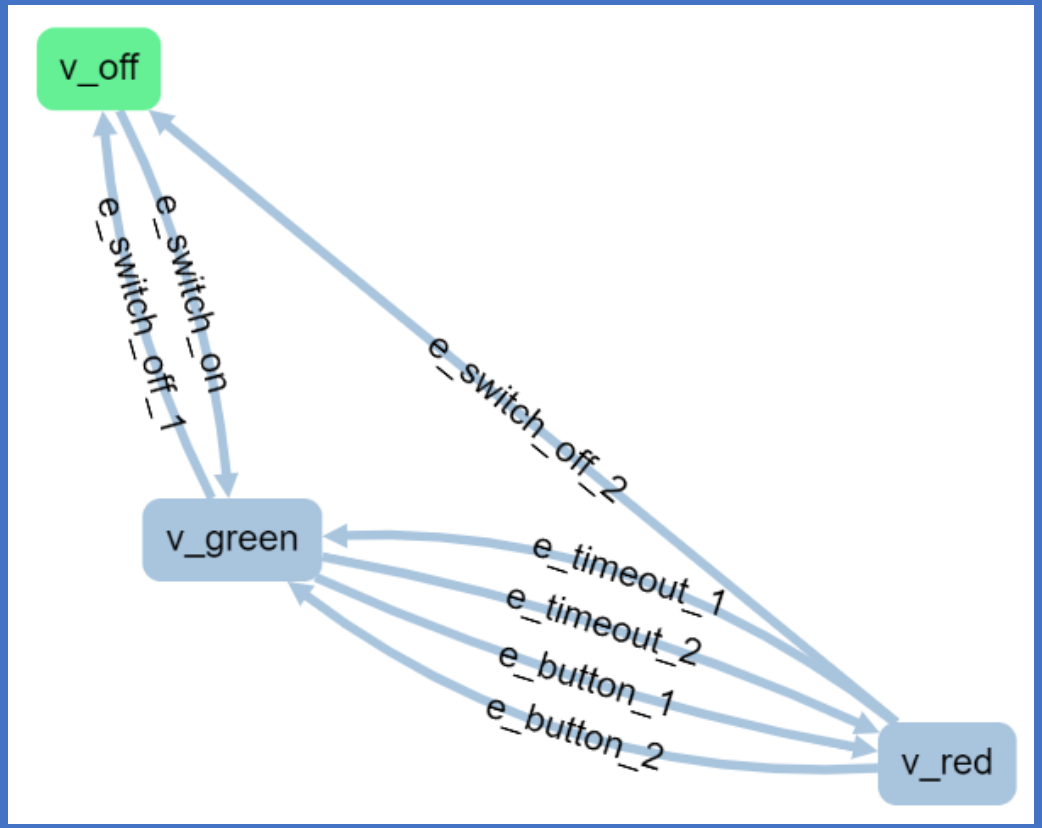
- **Create a vertex**: On the editor area, while pressing the keyboard key *v*, click the *left mouse button*.
- **Create an edge**: On the editor area, while pressing the keyboard key *e*
  - *Click and hold the left mouse button on the first vertex.*
  - *Drag the mouse cursor to the second vertex and release the left mouse button over that vertex.*
- **Change the properties of a vertex/edge**:
  1. Select the given element with mouse click
  2. then click on at the left panel and
  3. Set the given property
     - Name
     - Guarding conditions
     - Output/actions
     - Start element or not?



GraphWalker demo: Battery-operated signal

3. Draw the FSM of battery-operated signal in GW Studio
   - Set up names for vertices and edges
   - Set *v_off* as starting state
   - Add output to edges as actions
     - Example: `red=false;green=true;`
   - Save as *Marklin_signal.json*

	switch on	switch off	press button	wait for 7 secs
v_off	v_green /green	-	-	-
v_green	-	v_off /-	v_red /red	v_red /red
v_red	-	v_off /-	v_green /green	v_green /green



GraphWalker demo: Battery-operated signal

4. Run the tests in GW Studio

…using different generators:

- `random` / `quick_random` / `weighted_random` / `a_star` / `predefined_path`

- …using different stopping conditions:
  - `edge_coverage` / `vertex_coverage` (% of desired coverage)
  - `reached_vertex` / `reached_edge` (desired destination)
  - `time_duration` / `length`

- Run the generator with given settings multiple times!
  - What can you observe? Why?
  - See results here

GraphWalker demo: Battery-operated signal

5. Running test using the CLI interface of GW

- To achieve this, you must first create a corresponding test code:

  i. Modify the test of the sample project (*src/main/java/com/company/SomeSmallTest.java*) to create test in ***SignalTest.java***:
     - Set start node or edge
     - Modify the class name according to filename
     - Create printout functions for all nodes and edges of the graph
       - It can be made based on the interface generated in the java file of folder *target/generated-sources/graphwalker/com/company/*

         (after executing `mvn graphwalker:test` *from the project folder)*

  ii. Modify the Runner (*src/main/java/com/company/Runner.java*) to call ***SignalTest.java***

GraphWalker demo: Battery-operated signal

src/main/java/com/company/ SignalTest.java:

```
package com.company;
import org.graphwalker.core.machine.ExecutionContext;
import org.graphwalker.java.annotation.GraphWalker;

@GraphWalker(value = "random(edge_coverage(100))", start = "e_switch_on")
public class SignalTest extends ExecutionContext implements Marklin_signal
{

 @Override
 public void v_off() {
   System.out.println("In state: v_off");
 }

 @Override
 public void v_green() {
   System.out.println("In state: v_green");
 }

 @Override
 public void v_red() {
   System.out.println("In state: v_red");
 }

 @Override
 public void e_switch_on() {
   System.out.println("Traverses transition: e_switch_on");
 }

 @Override
 public void e_switch_off_1() {
   System.out.println("Traverses transition: e_switch_off_1");
 }

 @Override
 public void e_switch_off_2() {
   System.out.println("Traverses transition: e_switch_off_2");
 }

 @Override
 public void e_timeout_1() {
   System.out.println("Traverses transition: e_timeout_1");
 }

 @Override
 public void e_timeout_2() {
   System.out.println("Traverses transition: e_timeout_2");
 }

 @Override
 public void e_button_1() {
   System.out.println("Traverses transition: e_button_1");
 }

 @Override
 public void e_button_2() {
   System.out.println("Traverses transition: e_button_2");
 }
```

Stopping condition

Starting transition

Generator

Name of the graph

states

transitions

GraphWalker demo: Battery-operated signal

src/main/java/com/company/ Runner.java:

```
package com.company;

import com.company.SignalTest;
import org.graphwalker.java.test.Result;
import org.graphwalker.java.test.TestExecutor;

import java.io.IOException;

public class Runner {
  public static void main(String[] args) throws IOException {
    TestExecutor executor = new TestExecutor(
      SignalTest.class
    );

    Result result = executor.execute(true);
    System.out.println("Done: [" + result.getResults().toString(2) + "]");
  }
}
```

GraphWalker demo: Battery-operated signal

Run the tests in CLI (from *TrainSignal* folder):

```
mvn graphwalker:test
```

```
16:53:11.478 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute: 'e_timeout_1' in model: 'Marklin_signal'
Traverses transition: e_timeout_1
16:53:11.483 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.SignalTest@68805d40
16:53:11.484 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute: 'v_red' in model: 'Marklin_signal'
In state: v_red
16:53:11.487 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.SignalTest@68805d40
16:53:11.488 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute: 'red=false;green=true;' in model: 'Marklin_signal'
16:53:11.490 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute: 'e_timeout_2' in model: 'Marklin_signal'
Traverses transition: e_timeout_2
16:53:11.495 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.SignalTest@68805d40
16:53:11.495 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute: 'v_green' in model: 'Marklin_signal'
In state: v_green
16:53:11.498 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.SignalTest@68805d40
16:53:11.498 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute: 'red=false;green=false;' in model: 'Marklin_signal'
16:53:11.500 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute: 'e_switch_off_1' in model: 'Marklin_signal'
Traverses transition: e_switch_off_1
16:53:11.510 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.SignalTest@68805d40
16:53:11.511 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute: 'v_off' in model: 'Marklin_signal'
In state: v_off
[INFO] ------------------------------------------------------------------------
[INFO]
[INFO] Result :
[INFO]
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 7,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 7,
  "totalNumberOfVisitedVertices": 3,
  "edgeCoverage": 100,
  "totalNumberOfVertices": 3,
  "totalNumberOfUnvisitedEdges": 0
}
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  11.240 s
[INFO] Finished at: 2020-10-14T16:53:11+02:00
[INFO] ------------------------------------------------------------------------

C:\GW4\TrainSignal>
```

Note that as GraphWalker uses random walk to satisfy a given coverage, test cases are typically longer, then they would be in optimal case.

For example the generated tests (using generator `random`) in my 5 conducted experiments traverses **10, 21, 22, 22** and **34** edges, respectively, to fulfill 100% edge cover condition. However, the optimal solution* derived by TT-method requires only **9** edge traversals (the graph has 7 edges)

* Note that TT test sequences always go back to initial state, why the test sequences of GraphWalker typically do not. It means that slightly shorter test sequence may exist than the one generated by TT. In the current graph, however it does not count.

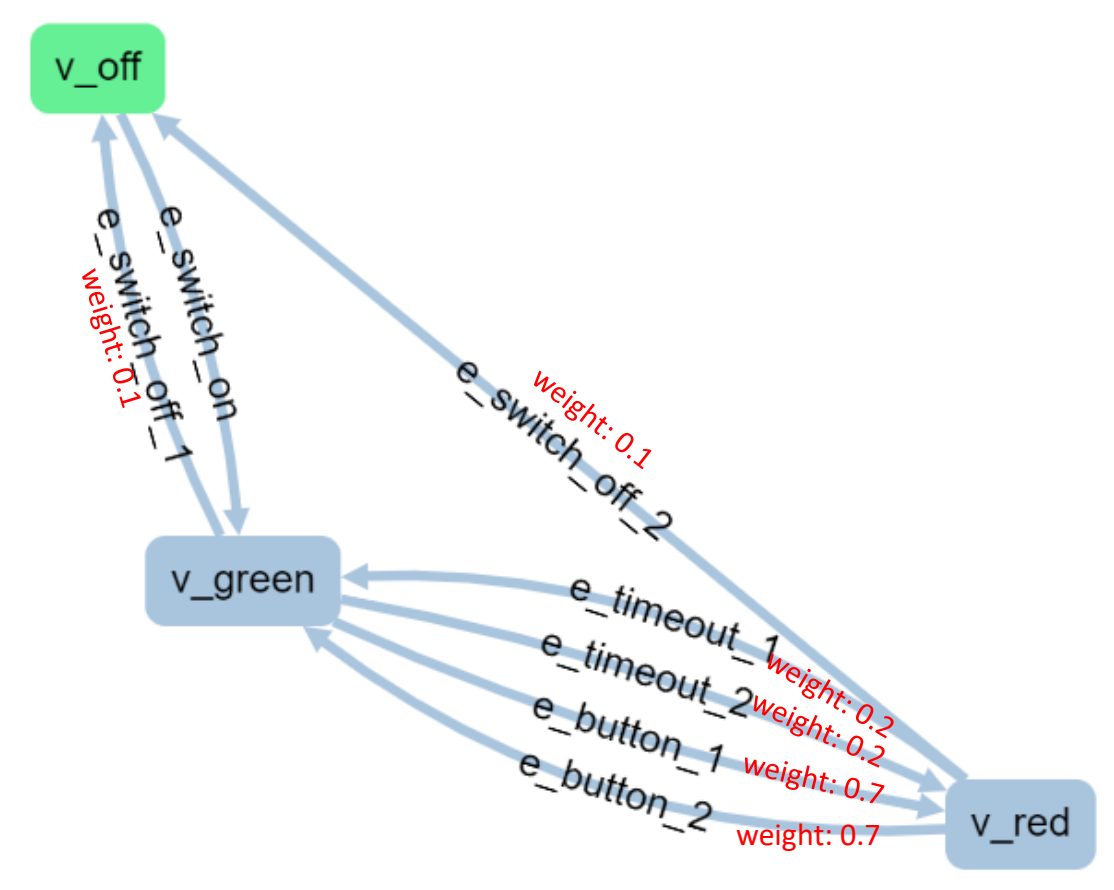
GraphWalker demo: Battery-operated signal

GW versions 4.0.0 & ↑
&
GW versions 3.4.2 & ↓

- Some examples using different generators:

```
@GraphWalker(value = "weighted_random(edge_coverage(100))", start = "e_switch_on")
```

  - Same as `random`, but uses `weight` keyword (a float value between 0.0 and 1.0 can be assigned to edges of the graph) as a probability to select an edge when generate a path
    - If no `weight` keyword is used for edges, than the probabilities will be the same for these edges



Compared to `random` generator the length of test sequences are longer due the weighted distribution of edges (in my 5 conducted experiments they were **14, 24, 25, 40**, and **41**)

GraphWalker demo: Battery-operated signal

- Some examples using different generators:

```
@GraphWalker(value = "quick_random(edge_coverage(100))", start = "e_switch_on")
```

1. Choose an edge not yet visited by random and find the shortest path to that edge using Dijkstra's algorithm
2. Mark all edges in this park as visited then continue with step 1 till stopping condition fulfilled

- Gives reasonable short test sequences, but Step 1 may choose an edge or path that blocked by guard condition of an EFSM

- In my 5 conducted experiments `quick_random` traverses **9, 9, 11, 11** and **12** edges, respectively, to fulfill 100% edge cover condition, thus it is better than generator `random`. The optimal solution derived by TT-method requires only **9** edge traversals (the graph has 7 edges)
- Thus in multiple executions we can observe different test sequence lengths due random walking approach.

```
@GraphWalker(value = "a_star(reached_vertex(v_red))", start = "e_switch_on")
```

- Reaches a given vertex or edge (defined in stopping condition) with the least cost
- Coverage will be smaller, but test sequence length will be the same due the A* algorithm


```
"vertexCoverage": 66,
"totalNumberOfEdges": 7,
"totalNumberOfVisitedVertices": 2,
"edgeCoverage": 28,
"totalNumberOfVertices": 3,
"totalNumberOfUnvisitedEdges": 5
```

GraphWalker demo: Battery-operated signal

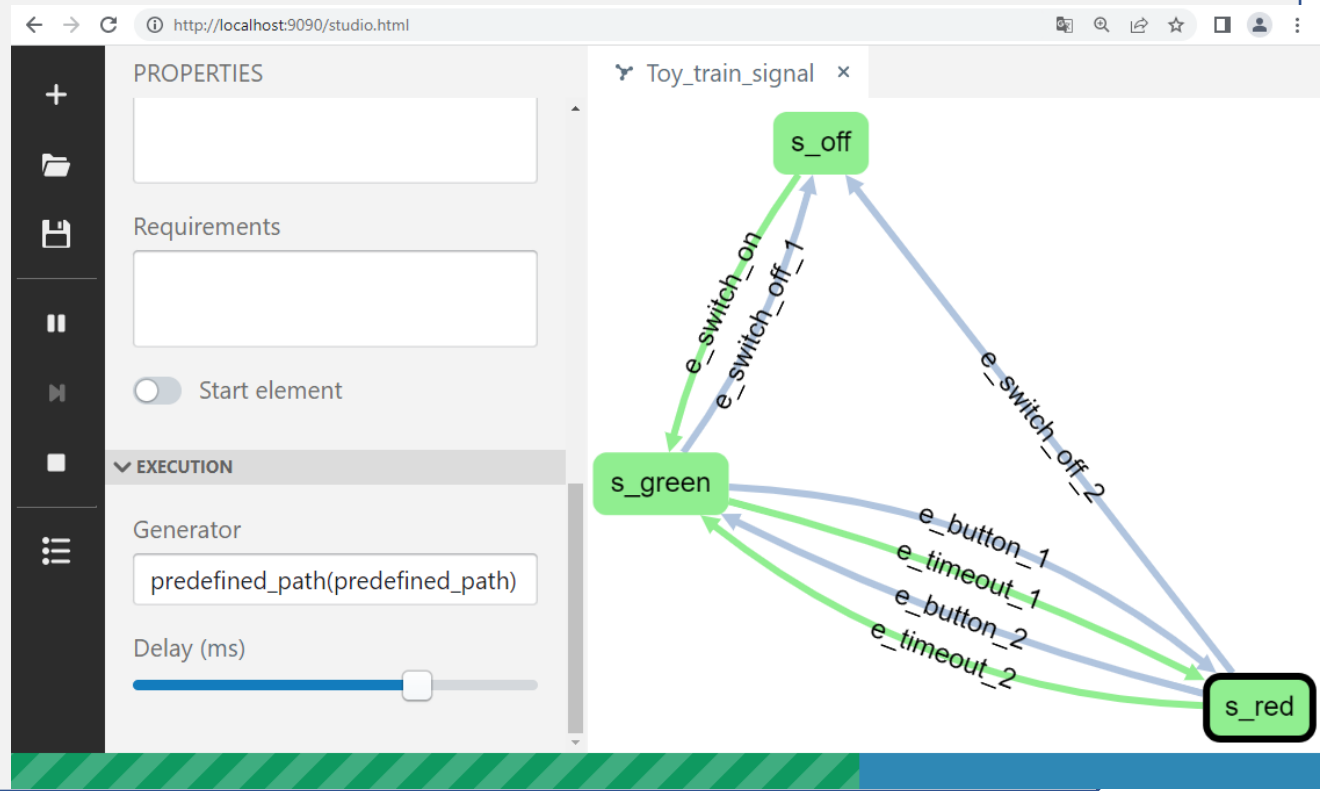
- Some examples using different generators:

`@GraphWalker(value = "predefined_path( predefined_path )`

Enables the user to define an edge sequence that GraphWalker will use when executing. Currently a predefined path can only be specified in the GraphWalker JSON file.

This functionality has been added to GraphWalker from Model 》 Test 》 Relax side. The aim is this feature is to enable GraphWalker to import test suites generated by Model 》 Test 》 Relax.

```
{
   "models":[
      {
          "generator": "predefined_path(predefined_path)",
         "name":"Toy_train_signal",
         "startElementId":"n2",
         "id":"08414683-68cb-4f68-be4c-670cd27f6cbe",
         "vertices":[
                                ...
         ],
         "edges":[
                                ...
         ],
         "predefinedPathEdgeIds":[
           "e1„, "e3", "e5", "e3", "e4",
           "e0", "e2", "e1", "e6"
         ]
      }
   ]
}
```



GraphWalker demo: Battery-operated signal

- Some examples using different coverages:

```
@GraphWalker(value = "quick_random(vertex_coverage(100))", start = "e_switch_on")
```


```
"vertexCoverage": 100,
"totalNumberOfEdges": 7,
"totalNumberOfVisitedVertices": 3,
"edgeCoverage": 57,
"totalNumberOfVertices": 3,
"totalNumberOfUnvisitedEdges": 3
```

```
"vertexCoverage": 100,
"totalNumberOfEdges": 7,
"totalNumberOfVisitedVertices": 3,
"edgeCoverage": 85,
"totalNumberOfVertices": 3,
"totalNumberOfUnvisitedEdges": 1
```

```
"vertexCoverage": 100,
"totalNumberOfEdges": 7,
"totalNumberOfVisitedVertices": 3,
"edgeCoverage": 42,
"totalNumberOfVertices": 3,
"totalNumberOfUnvisitedEdges": 4
```


In multiple executions of vertex coverage (100%) we can observe different edge coverage and test sequence lengths due random walking approach.

```
@GraphWalker(value = "quick_random(edge_coverage(50))", start = "e_switch_on")
```


```
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 4,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [
    {
      "modelName": "Marklin_signal",
      "edgeId": "e2",
      "edgeName": "e_switch_off_2"
    },
    {
      "modelName": "Marklin_signal",
      "edgeId": "e3",
      "edgeName": "e_timeout_1"
    },
    {
      "modelName": "Marklin_signal",
      "edgeId": "e5",
      "edgeName": "e_timeout_2"
    }
  ],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 7,
  "totalNumberOfVisitedVertices": 3,
  "edgeCoverage": 57,
  "totalNumberOfVertices": 3,
  "totalNumberOfUnvisitedEdges": 3
}
```

```
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 4,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [
    {
      "modelName": "Marklin_signal",
      "edgeId": "e4",
      "edgeName": "e_button_2"
    },
    {
      "modelName": "Marklin_signal",
      "edgeId": "e5",
      "edgeName": "e_timeout_2"
    },
    {
      "modelName": "Marklin_signal",
      "edgeId": "e6",
      "edgeName": "e_switch_off_1"
    }
  ],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 7,
  "totalNumberOfVisitedVertices": 3,
  "edgeCoverage": 57,
  "totalNumberOfVertices": 3,
  "totalNumberOfUnvisitedEdges": 3
}
```

```
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 4,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [
    {
      "modelName": "Marklin_signal",
      "edgeId": "e2",
      "edgeName": "e_switch_off_2"
    },
    {
      "modelName": "Marklin_signal",
      "edgeId": "e3",
      "edgeName": "e_timeout_1"
    },
    {
      "modelName": "Marklin_signal",
      "edgeId": "e5",
      "edgeName": "e_timeout_2"
    }
  ],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 7,
  "totalNumberOfVisitedVertices": 3,
  "edgeCoverage": 57,
  "totalNumberOfVertices": 3,
  "totalNumberOfUnvisitedEdges": 3
}
```

```
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 1,
  "verticesNotVisited": [{
    "modelName": "Marklin_signal",
    "vertexName": "v_off",
    "vertexId": "n2"
  }],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 4,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [
    {
      "modelName": "Marklin_signal",
      "edgeId": "e2",
      "edgeName": "e_switch_off_2"
    },
    {
      "modelName": "Marklin_signal",
      "edgeId": "e5",
      "edgeName": "e_timeout_2"
    },
    {
      "modelName": "Marklin_signal",
      "edgeId": "e6",
      "edgeName": "e_switch_off_1"
    }
  ],
  "vertexCoverage": 66,
  "totalNumberOfEdges": 7,
  "totalNumberOfVisitedVertices": 2,
  "edgeCoverage": 57,
  "totalNumberOfVertices": 3,
  "totalNumberOfUnvisitedEdges": 3
}
[INFO]
```


In multiple executions of edge coverage (50%) we can observe the same edge coverage (57%) due stopping condition, but the different vertex coverage and test sequence lengths due random walking approach.

GraphWalker demo: Battery-operated signal

How the actual testing can be made?

(i.e. How the model can be connected to the SUT?)

- GraphWalker generates only a path on the graph/FSM
- Output validation shall be done in the adaptation/glue code (i.e. in transitions).
  This can be done for example, by applying…
  - Junit
  - Selenium (GUI tests)

- See PetClinic example in GraphWalker documentation or examples in this material for further information

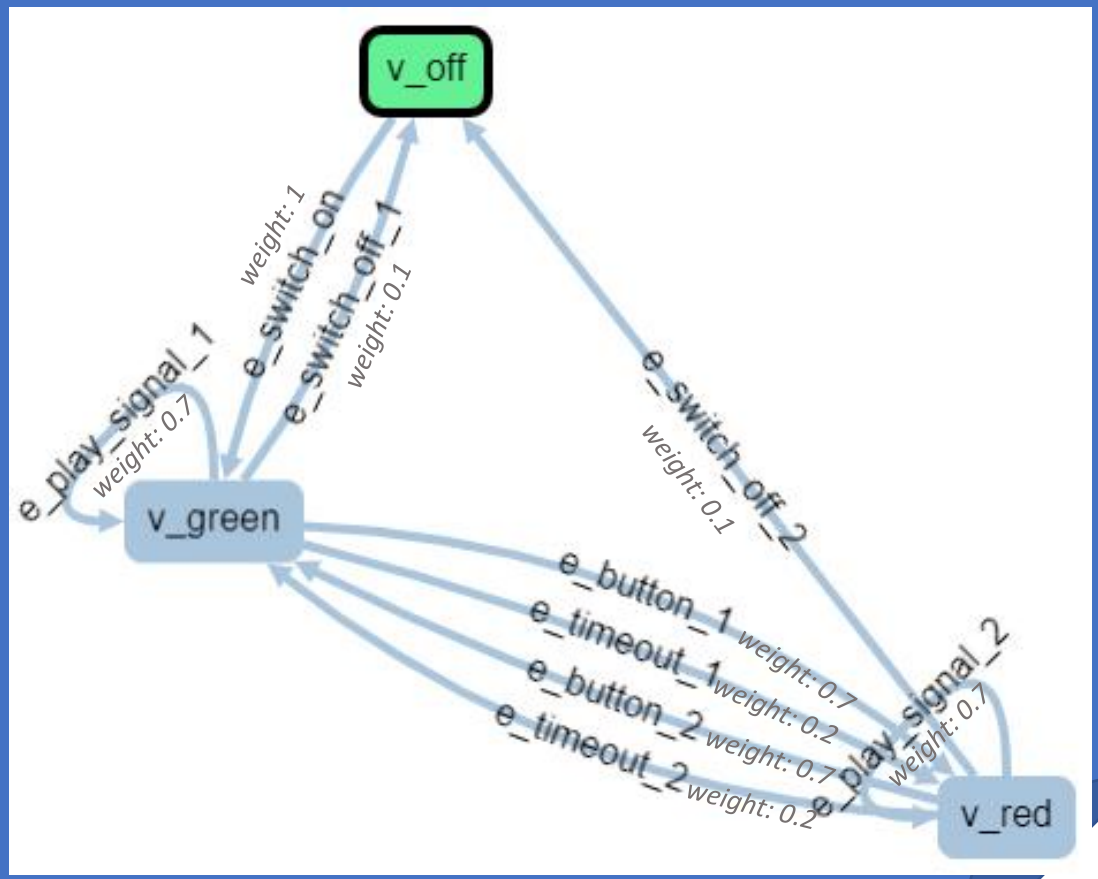
```
@Override
public void e_switch_on() {
        /* INSERT CODE HERE */
}
```

```
@Override
public void e_switch_off_1() {
        /* INSERT CODE HERE */
}

@Override
public void e_switch_off_2() {
        /* INSERT CODE HERE */
}

@Override
public void e_timeout_1() {
        /* INSERT CODE HERE */
}

@Override
public void e_timeout_2() {
        /* INSERT CODE HERE */
}

@Override
public void e_button_1() {
        /* INSERT CODE HERE */
}

@Override
public void e_button_2() {
        /* INSERT CODE HERE */
}
```

GraphWalker demo: Battery-operated signal

6. Extend the battery-operated signal with guard conditions to play the MÁV music signal after 4th button press instead of changing it's state using yED graph editor
   - Use *counter* variable (initialization, increment, and guard conditions) – see table

	switch on	switch off	[counter < 3] press button	[counter == 3] press button	wait for 7 secs
off	green /<counter=0>, green	-	off /-	off/-	off /-
green	-	off /-	red /<counter++>, red	green /<counter=0>, green, play signal	red /red
red	-	off /-	green /<counter++>, green	red /<counter=0>, red, play signal	green /green



- Add bigger `weights` *(like 0.7)* to button presses and smaller `weights` *(like 0.1)* to switch_off *(otherwise the test would be too long)*
- Use `weighted_random` as generator
- Rerun the tests, what will be the difference?

GraphWalker demo: Battery-operated signal

The syntax of guard conditions and variables:

PROPERTIES

ELEMENT

Name

e_button_1

Shared Name

Guard

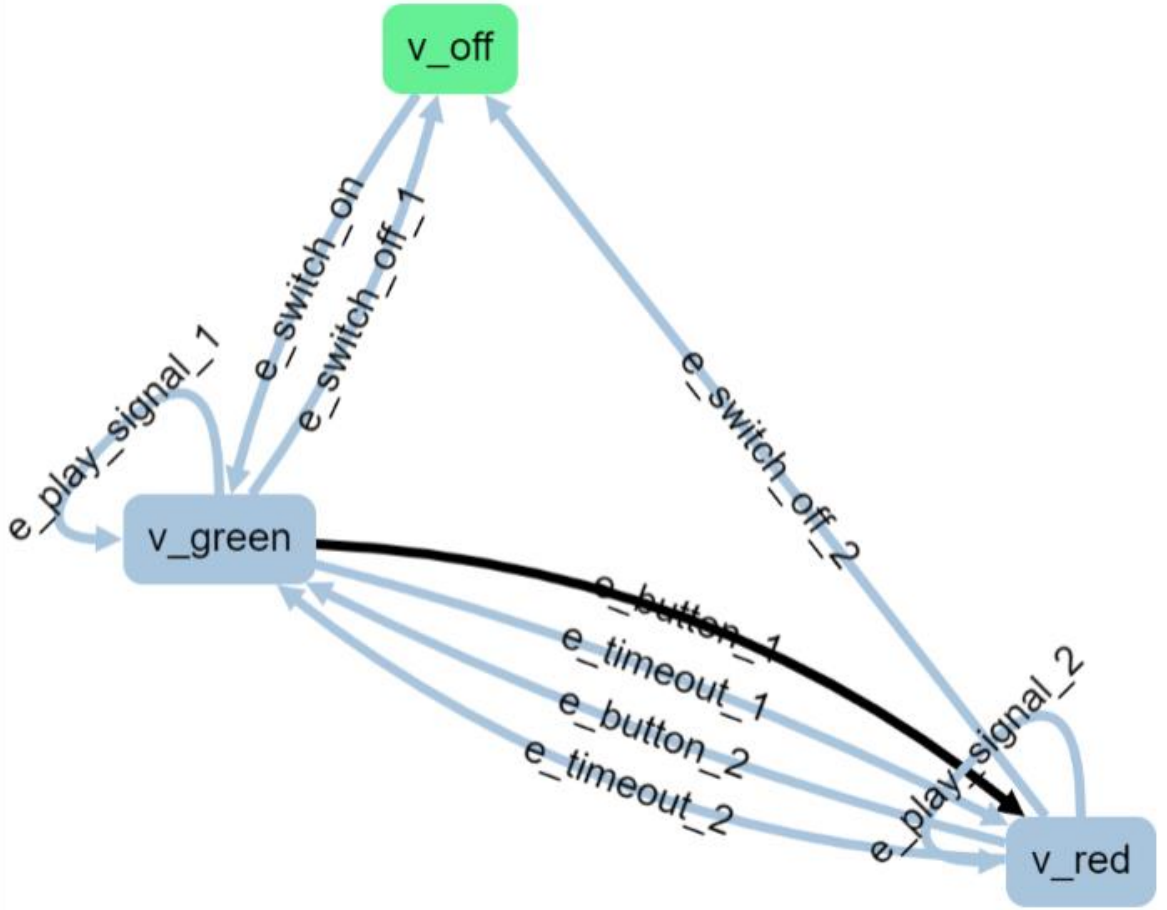
counter<3

Actions

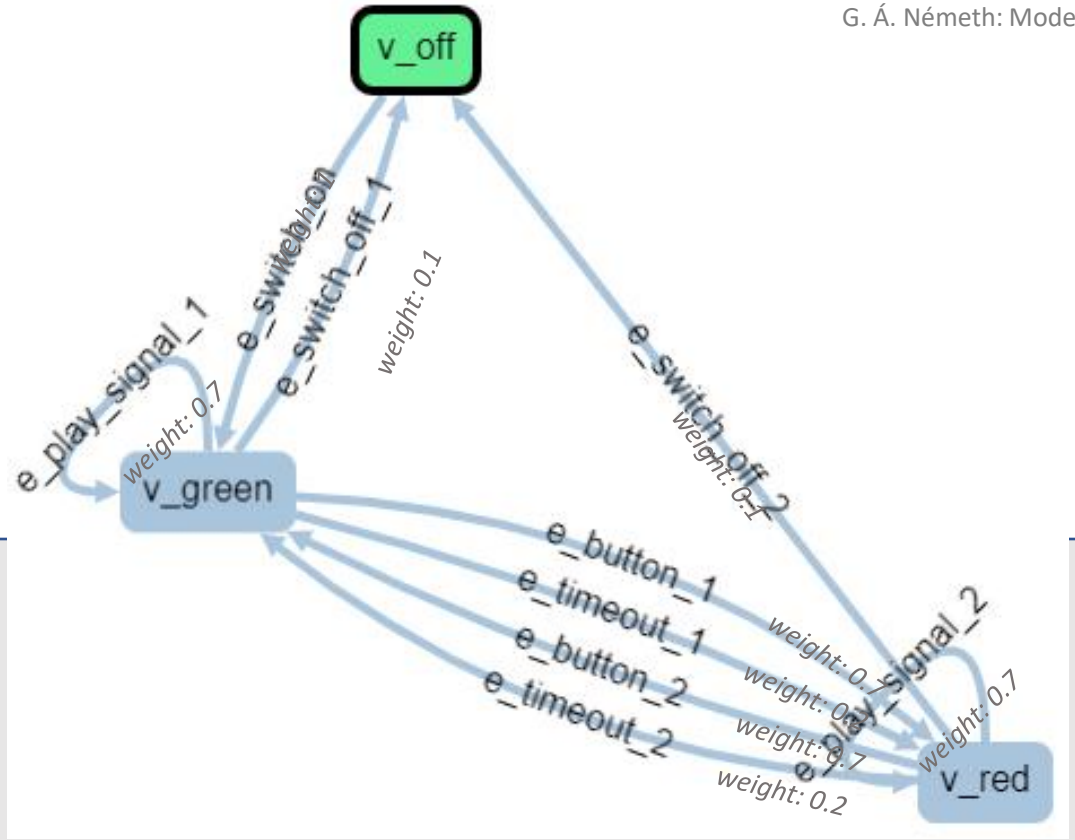
red=true;green=false;counter++;

Requirements

Marklin_signal ×



GraphWalker demo: Battery-operated signal



JSON file of the EFSM (Marklin_signal.json):

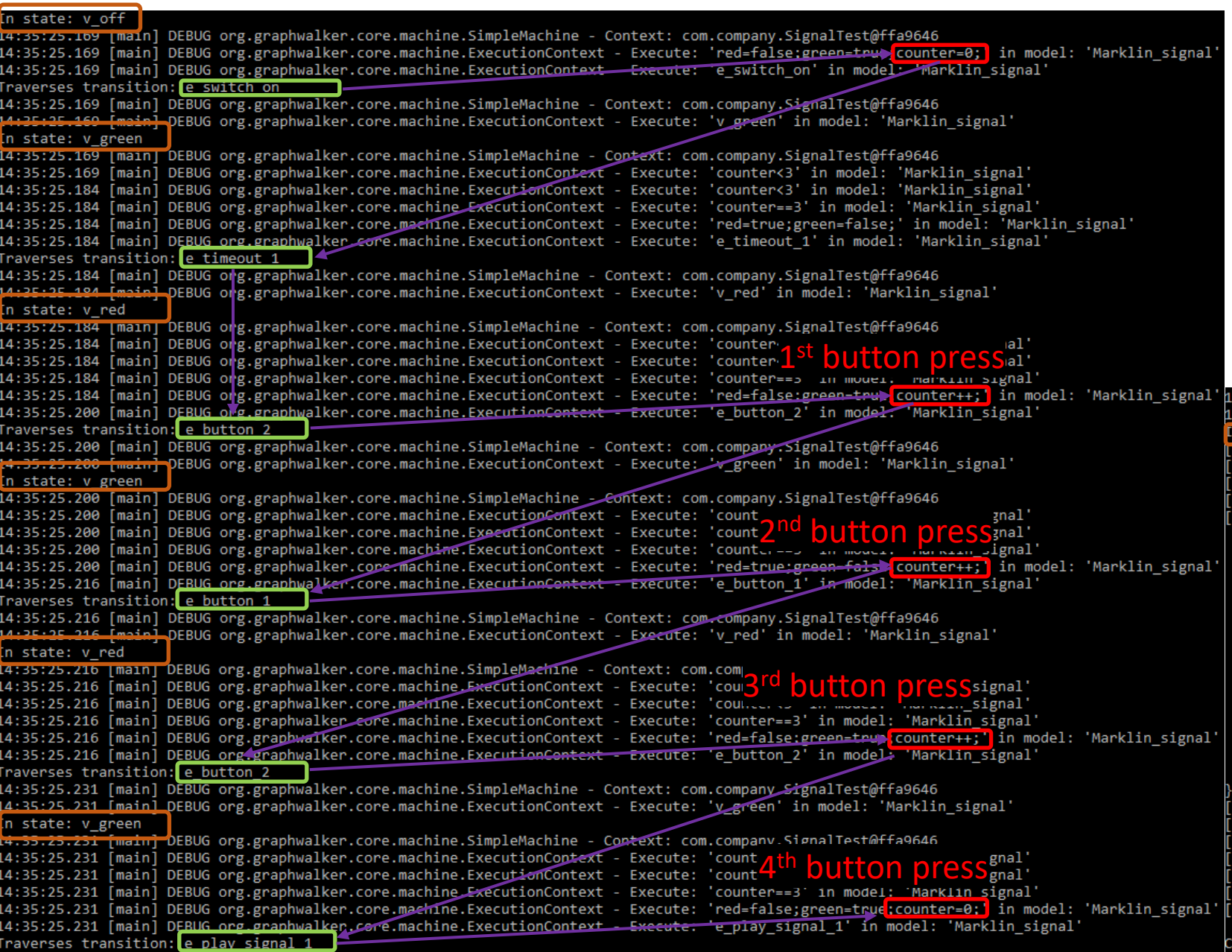

```
{"models":
[
{"name":"Marklin_signal",
                "id":"08414683-68cb-4f68-be4c-670cd27f6cbe",
                "vertices":
                [
                                {"id":"n0","name":"v_green","properties":{"x":20,"y":263.5}},
                                {"id":"n1","name":"v_red","properties":{"x":364,"y":392}},
                                {"id":"n2","name":"v_off","properties":{"x":112,"y":68}}
                ],
                "edges":
                [
                {"id":"e0","name":"e_button_1","guard":"counter<3","actions":["red=true;green=false;counter++;"],"weight":"0.7","dependency":0,"sourceVertexId":"n0","targetVertexId":"n1"},
                {"id":"e1","name":"e_switch_on","actions":["red=false;green=true;counter=0;"],"weight":"1","dependency":0,"sourceVertexId":"n2","targetVertexId":"n0"},
                {"id":"e2","name":"e_switch_off_2","actions":["red=false;green=false;"],"weight":"0.1","dependency":0,"sourceVertexId":"n1","targetVertexId":"n2"},
                {"id":"e3","name":"e_timeout_1","guard":"","actions":["red=true;green=false;"],"weight":"0.2","dependency":0,"sourceVertexId":"n0","targetVertexId":"n1"},

                {"id":"e4","name":"e_button_2","guard":"counter<3","actions":["red=false;green=true;counter++;"],"weight":"0.7","dependency":0,"sourceVertexId":"n1","targetVertexId":"n0"},
                {"id":"e5","name":"e_timeout_2","guard":"","actions":["red=false;green=true;"],"weight":"0.2","dependency":0,"sourceVertexId":"n1","targetVertexId":"n0"},
                {"id":"e6","name":"e_switch_off_1","actions":["red=false;green=false;"],"weight":"0.1","dependency":0,"sourceVertexId":"n0","targetVertexId":"n2"},

                {"id":"e7","name":"e_play_signal_1","guard":"counter==3","actions":["red=false;green=true;counter=0;"],"properties":{"description":""},"weight":"0.7","dependency":0,"sourceVertexId":
"n0","targetVertexId":"n0"},

                {"id":"e8","name":"e_play_signal_2","guard":"counter==3","actions":["red=false;green=true;counter=0;"],"properties":{"description":""},"weight":"0.7","dependency":0,"sourceVertexId":
"n1","targetVertexId":"n1"}
                ],
                "generator":"weighted_random(edge_coverage(75))","startElementId":"n2"
}
],
"selectedModelIndex":0,"selectedElementId":"n2"}
```

GraphWalker demo: Battery-operated signal

Do not forget to add the corresponding new transitions to the code (SignalTest.java) to be able to check the results in GW CLI

```
package com.company;

import org.graphwalker.core.machine.ExecutionContext;
import org.graphwalker.java.annotation.GraphWalker;

@GraphWalker(value = "random(edge_coverage(100))", start = "e_switch_on")
public class SignalTest extends ExecutionContext implements Marklin_signal {

  /*states*/
  @Override
  public void v_off() {
    System.out.println("In state: v_off");
  }

  @Override
  public void v_green() {
    System.out.println("In state: v_green");
  }

  @Override
  public void v_red() {
    System.out.println("In state: v_red");
  }


  /*transitions*/
  @Override
  public void e_switch_on() {
    System.out.println("Traverses transition: e_switch_on");
  }

  @Override
  public void e_switch_off_1() {
    System.out.println("Traverses transition: e_switch_off_1");
  }

  @Override
  public void e_switch_off_2() {
    System.out.println("Traverses transition: e_switch_off_2");
  }

  @Override
  public void e_timeout_1() {
    System.out.println("Traverses transition: e_timeout_1");
  }

  @Override
  public void e_timeout_2() {
    System.out.println("Traverses transition: e_timeout_2");
  }

  @Override
  public void e_button_1() {
    System.out.println("Traverses transition: e_button_1");
  }

  @Override
  public void e_button_2() {
    System.out.println("Traverses transition: e_button_2");
  }

  @Override
  public void e_play_signal_1() {
    System.out.println("Traverses transition: e_play_signal_1");
  }

  @Override
  public void e_play_signal_2() {
    System.out.println("Traverses transition: e_play_signal_2");
  }

}
```

GraphWalker demo: Battery-operated signal




```
14:35:25.247 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context:
14:35:25.247 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execut
In state: v_green
[INFO] ------------------------------------------------------------------------
[INFO]
[INFO] Result :
[INFO]
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 9,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 9,
  "totalNumberOfVisitedVertices": 3,
  "edgeCoverage": 100,
  "totalNumberOfVertices": 3,
  "totalNumberOfUnvisitedEdges": 0
}
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  14.577 s
[INFO] Finished at: 2020-10-14T14:35:25+02:00
[INFO] ------------------------------------------------------------------------

C:\GW4\TrainSignal>
```

GraphWalker demo: Troubleshooting 2/1

See Appendix B for example homeworks using GraphWalker

Typical problems that should be keep in mind:

- Multiple java files in folder `src/main/java/com/company` are not handled correctly. Either the tests will run for infinite time or the wrong tests will be performed (i.e. Graphwalker will not use the selected generator or stopping condition).

GraphWalker demo: Troubleshooting 2/2

Other possible GW problems and their solutions:

- Maven didn't add the generated-sources folder to the CLASS-PATH and because of this `maven:compile` lifecycle kept failing even if `graphwalker:test` is worked.
  - Add *build-helper-maven-plugin* to the project. This plugin allows the developer to specify additional sources to be added to the classpath. The following lines should be added to *pom.xml*:

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>build-helper-maven-plugin</artifactId>
    <executions>
        <execution>
            <phase>generate-sources</phase>
                <goals>
                    <goal>add-source</goal>
                </goals>
                <configuration>
                    <sources>
                        <source>${project.build.directory}/source</source>
                    </sources>
                </configuration>
            </execution>
    </executions>
</plugin>
```

- When adding GW tests to implementation, and an API incompatibility occurs during running `graphwalker:test`:
  - Excluding the incompatible parts.
    - An example to exclude Guava from gtfs-lib – see the corresponding lines of *pom.xml*:

```
<dependency>
    <groupId>com.conveyal</groupId>
    <artifactId>gtfs-lib</artifactId>
    <version>5.0.7</version>
    <exclusions>
        <exclusion>
            <groupId>com.google.guava</groupId>
            <artifactId>guava</artifactId>
        </exclusion>
</exclusions>
</dependency>
```

GraphWalker demo: Summary

- Advantages:
  - Easy-to-learn, easy-to-use, has good documentation*
    * although, possible problems with maven integration and their solutions should be documented
  - Free, open source, easy to extend
  - Different generators and stopping conditions
  - Random walking can be useful when discover new functionalities
- Disadvantages:
  - Random tests
    - Not repeatable, no suitable for regression tests → suitable only for educational purposes/small projects/augmenting of other testing infrastructure
    - Tests are not optimal
      - Although A* also exist it provides the shortest path only to a given vertex/edge
    - No real test generation or selection algorithms
  - Sometimes not follows EFSM formalism (no input/output symbols, actions can be added to states)
  - Test reports could be better
  - Missing error messages in GUI if something mistyped or the model is corrupt…etc.
- Neutral:
  - Should be used together with other tools (function validation…etc.)

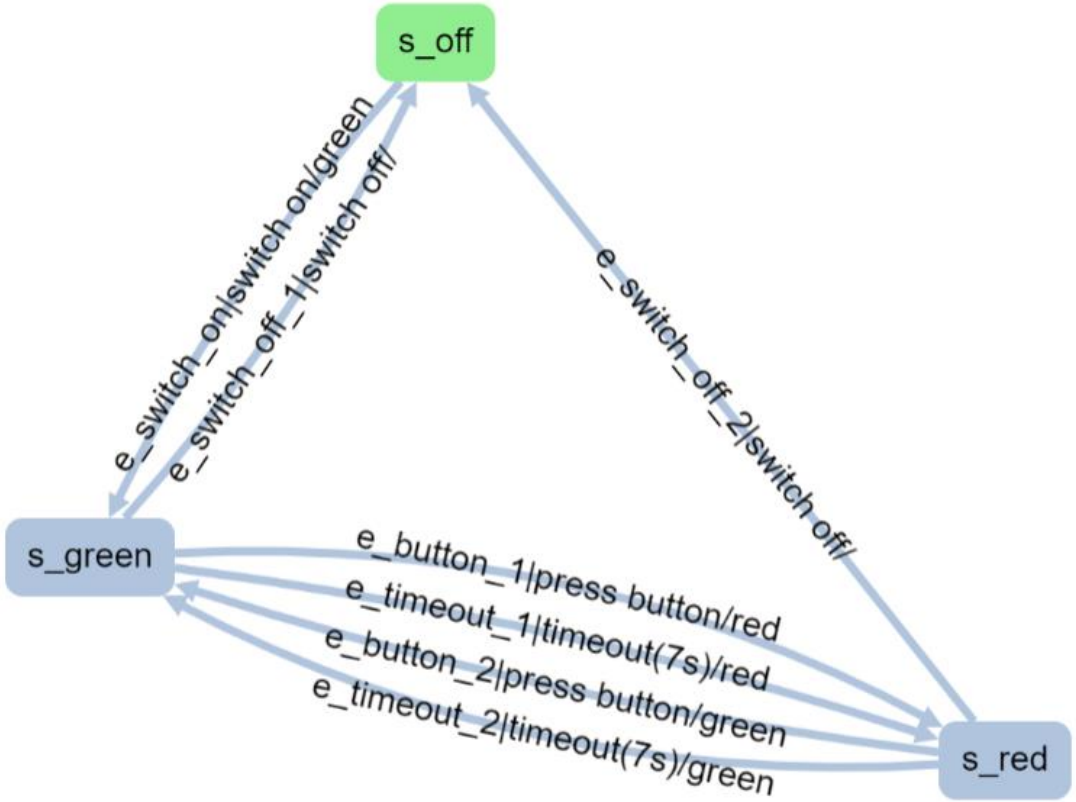
Further processing the model created by GraphWalker: Model 》 Test 》 Relax MBT framework

- Open source, free MBT tool
- Developed in ELTE Faculty of Informatics
- Easy-to-use

- Models can be imported from GraphWalker Studio
- Has systematic test generation algorithms, model conversions
- Has good user guide and sample test projects
- An overview of functionalities can be found here

Model 》 Test 》 Relax: Import model from GraphWalker Studio

Models created with GraphWalker Studio does not handle input and output symbols.
There are 2 options to handle this:
1. Add i/o symbols to the model encoded in the edge names using the following format: `edge_name|input/output`



Then you can apply [Add input/output symbols to the model functionality] of Model 》 Test 》 Relax:

```
MTR -o conversion -m add_iosymbols -f <path>/<name of your model>
```

Note that the converted model is included in the [Model 》 Test 》 Relax] repository:
See `sample_models/introduction/Toy_train_signal.json`

Model 》 Test 》 Relax: Import model from GraphWalker Studio

OR:

2. Add input/output symbols manually into in the *Toy_train_signal.json* JSON model (see "edges" in model file) after the "input" and "output" keywords:

```
{
   "models":[
      {
         "name":"Toy_train_signal",
         "startElementId":"n2",
         "id":"08414683-68cb-4f68-be4c-
670cd27f6cbe",
         "vertices":[
            {
               "id":"n0",
               "name":"s_green",
               "properties":{
                  "x":20,
                  "y":263.5
               }
            },
            {
               "id":"n1",
               "name":"s_red",
               "properties":{
                  "x":364,
                  "y":392
               }
            },
            {
               "id":"n2",
               "name":"s_off",
               "properties":{
                  "x":112,
                  "y":68
               }
            }
         ],
         "edges":[
            {
               "id":"e0",
               "name":"e_button_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n1",
               "input" : "press button",
               "output" : "red",
               "weight": "0.3"
            },
            {
               "id":"e1",
               "name":"e_switch_on",
               "sourceVertexId":"n2",
               "targetVertexId":"n0",
               "input" : "switch on",
               "output" : "green",
               "weight": "1"
            },
            {
               "id":"e2",
               "name":"e_switch_off_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n2",
               "input" : "switch off",
               "output" : "",
               "weight": "0.1"
            },
            {
               "id":"e3",
               "name":"e_timeout_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n1",
               "input" : "timeout(7s)",
               "output" : "red",
               "weight": "0.6"
            },
            {
               "id":"e4",
               "name":"e_button_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n0",
               "input" : "press button",
               "output" : "green",
               "weight": "0.3"
            },
            {
               "id":"e5",
               "name":"e_timeout_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n0",
               "input" : "timeout(7s)",
               "output" : "green",
               "weight": "0.6"
            },
            {
               "id":"e6",
               "name":"e_switch_off_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n2",
               "input" : "switch off",
               "output" : "",
               "weight": "0.1"
            }
         ]
      }
   ]
}
```

Note that this model is included in the [Model 》 Test 》 Relax](#) repository:
See sample_models/introduction/ Toy_train_signal.json

Model 》 Test 》 Relax: Test generation

```
./MTR -o t -m tt -f sample_models/introduction/Toy_train_signal.json --graphviz --transition_list
```

Console output:

Test generation algorithm: TT: Transition Tour

Create output to visualize models with Graphviz

Add transition lists into test suite

```
[U-CTRL ] [info] Version: 3.3.5 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-LMI  ] [info] Model Name: Toy_train_signal, Type: FSM, Reset: No
[U-CTRL ] [info] Created graphviz dot file of original model: ./graphviz-TT
[U-CTRL ] [info] Running Transition Tour test generation
[TG-TT  ] [info] Not Eulerian, augment
[TG-TT  ] [info] Building bipartite graph
[TG-TT  ] [info] Bipartite graph successfully built
[TG-TT  ] [info] Creating matching
[TG-TT  ] [info] Matching done
[TG-TT  ] [info] Duplicating transitions according to matching
[TG-TT  ] [info] Augmenting to Eulerian graph successful
[TG-TT  ] [info] Ordering edges
[TG-TT  ] [info] Edges ordered
[TG-TT  ] [info] Generating test sequence
[TG-TT  ] [info] Starting traversal
[TG-TT  ] [info] Tour length: 9
[TG-TT  ] [info] Augmented model graphviz file created: graphviz/graphviz-TT/Toy_train_signal_augmented.dot
[TG-TT  ] [info] Finished computation at 2024-02-19 15:06:06.5910571
        elapsed time: (real time) 0.0039878 s
        elapsed time: (user time) 0.004000 s
[TG-TGR ] [info] Test generation summary written: test_summary/tt_result.csv
[U-TW   ] [info] Test suite written: test_suites/Toy_train_signal-TT-test_suite.json
```

Short summary about programs settings

Short summary about input model

TG: Test generator component

Main steps of test generation

Short summary about the resulting test sequence

Resulting test suite is saved here

Model 》 Test 》 Relax: Systematic test generation

- Open a generated test sequence in *test_suites* folder (`Toy_train_signal-TT_suite.json` → *test sequence of Transition Tour)*:

```
{
    "test_suite": {
        "name": "Toy_train_signal",
        "id": "08414683-68cb-4f68-be4c-670cd27f6cbe",
        "method": "TT",
        "input_list": [
            "switch on",
            "timeout(7s)",
            "timeout(7s)",
            "timeout(7s)",
            "press button",
            "press button",
            "switch off",
            "switch on",
            "switch off"
        ],
        "output_list": [
            "green",
            "red",
            "green",
            "red",
            "green",
            "red",
            "",
            "green",
            ""
        ],
        "transition_id_list": [
            "e1",
            "e3",
            "e5",
            "e3",
            "e4",
            "e0",
            "e2",
            "e1",
            "e6"
        ],
        "transition_name_list": [
            "e_switch_on",
            "e_timeout_1",
            "e_timeout_2",
            "e_timeout_1",
            "e_button_2",
            "e_button_1",
            "e_switch_off_2",
            "e_switch_on",
            "e_switch_off_1"
        ]
    }
}
```

Name of the model

Used test generation algorithm

List of applied input symbols

List of desired output symbols derived from specification model

Transition lists are optional (see `--transition_list` switch) It can be used for learning/self checking or if one would like to import the results to GraphWalker

Model 》 Test 》 Relax: Import generated tests into GraphWalker

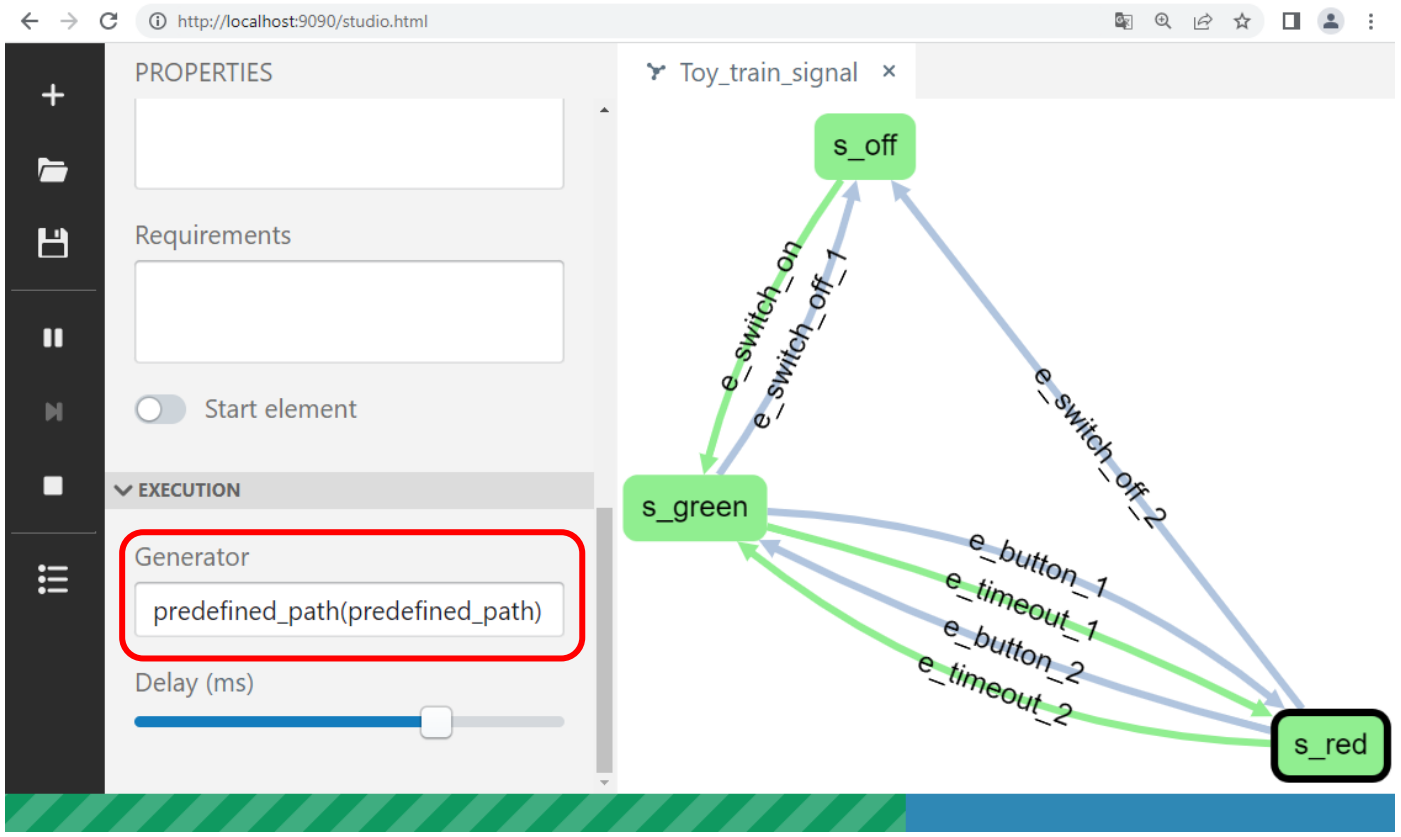
1. Generate test suite with Model 》 Test 》 Relax using the `--predefined_path flag`:

```
./MTR -o t -m tt -f sample_models/introduction/Toy_train_signal.json --predefined_path
```

This creates a new JSON model in the same folder as the original model, with the test generation method appended to the end of the original file name.

2. Execute the test suite in GraphWalker CLI

   or GraphWalker Studio.



MBT in practice: tools with non-FSM models

Green Sweet Lemon

- Uses flow-s for calculation intensive applications
- Uses petri nets for event-driven applications
  - generates test cases from Swim Lane Event-Driven Petri Nets models
  - export test cases to the TITAN TTCN-3 test execution engine
- MBT is offered as a service with GSL

MBT in practice: tools with non-FSM models

Live demo with István Forgács

Harmony

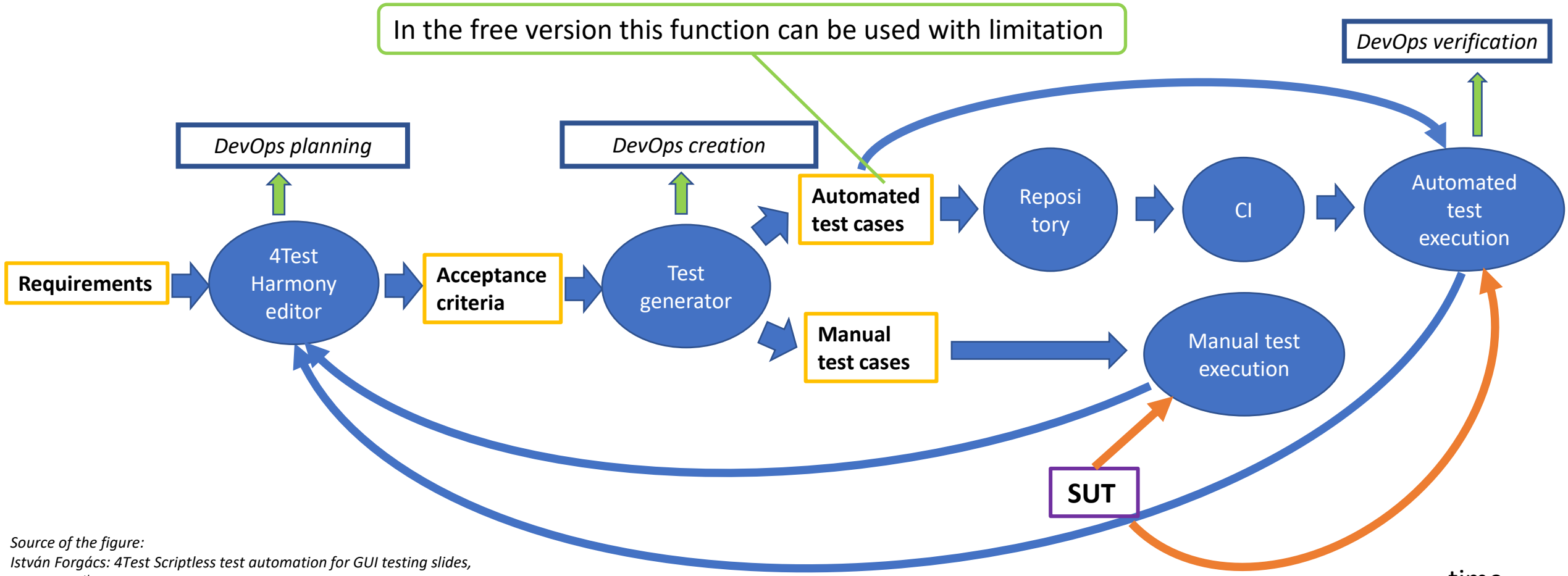
- Gherkin-based (Gherkin++) model, text-based MBT
- No installation required, only a web browser
- Multiple versions:
  - Free
  - Commercial: added functionalities (like test automatization)
  - Academic: full set of functionalities for limited project size
- Keyword-driven and test-first approach
- Risk-based testing for select appropriate test method
- Has good documentation, easy-to-learn, suitable for small projects

- Step by step instruction with code can be found for manual test case generation in Chapter 13 12 GHERKIN-BASED MBT of the following book: I. Forgacs, A. Kovacs: Practical Test Design: Selection of traditional and automated functional tests. BCS. 2019.
- Note that the free version of Harmony also contains sample projects with documentation

MBT in practice: tools with non-FSM models

Harmony – testing process:

Live demo with István Forgács harmony



Source of the figure:
István Forgács: 4Test Scriptless test automation for GUI testing slides, version: 15th April 2019

MBT in practice: tools with non-FSM models

Harmony

Live demo with István Forgács

- Uses the following from test theory:
  - State transition systems
  - EP (Equivalence partitioning) + BVA (Boundary value analysis) and their combinations
- Structure:
  - Projects
  - Features
    - Requirements
    - Categories – choices
    - Acceptance criteria
  - Supports test modularization

Description is similar to Gherkin:

- **GIVEN** describes preconditions (optional)
- **INITIALLY** describes initial state (optional)
- **WHEN** contains the inputs (mandatory)
- **THEN** contains the output (mandatory)
- **AND** connects two **GIVEN**/**WHEN**/**THEN** expressions
- **IS**/**ARE** connects a category and a choice of this category, such as

```
button IS #pressed
```

MBT in practice: tools with non-FSM models

Harmony – an example:

Live demo with István Forgács

Feature (Login example) description in Gherkin-like language:

```
Logged in:
WHEN Login name IS Hal9000 AND Password IS Daisy2001
WHEN Login button IS #pressed
THEN Message IS successful login
```

Generated code:

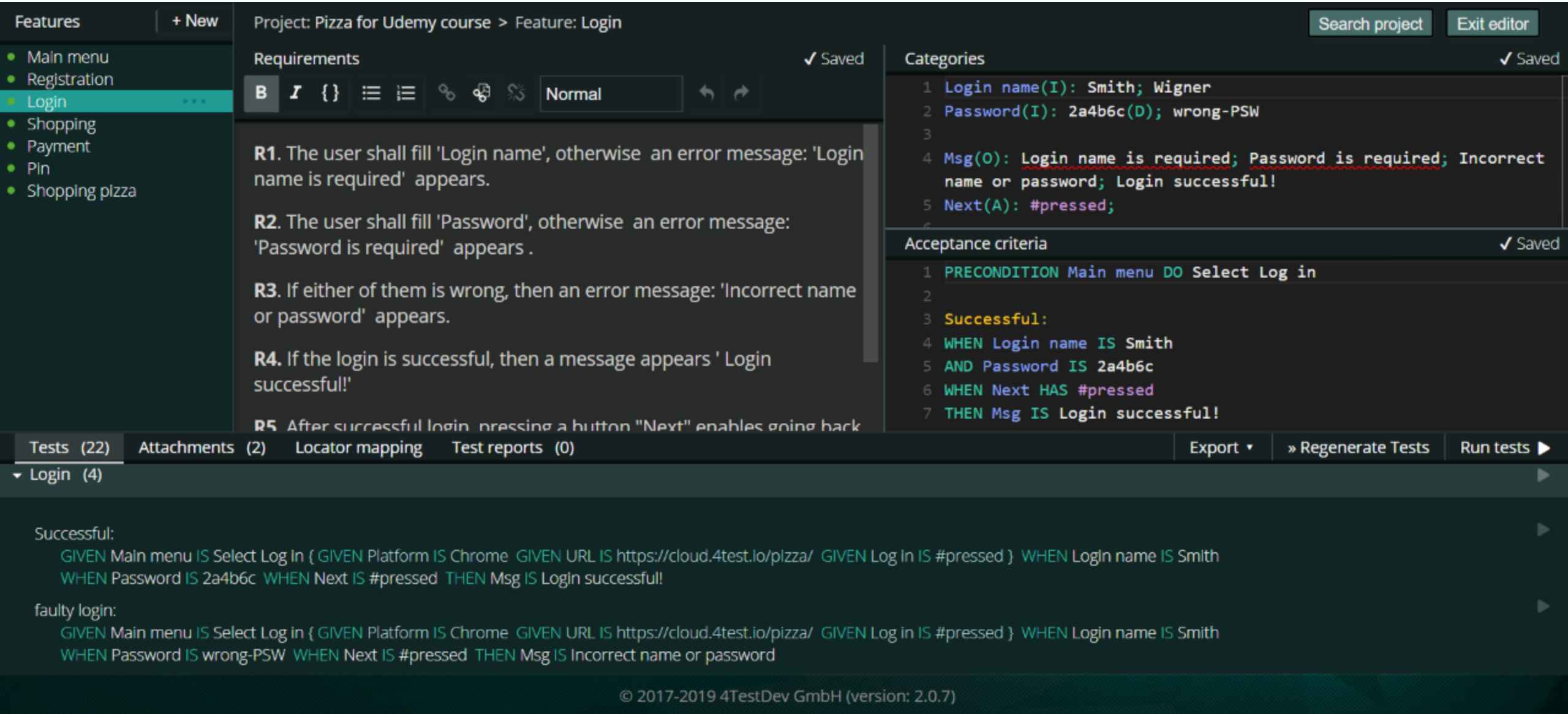
```
SelectWindow( "Login" );
SetValue( "Login name", "Hal9000" );
SetValue( "password", "Daisy2001" );
ClickOn( "Login button");
VerifyValue( "Message", "successful login");
```

This example based on the one presented here:
István Forgács: 4Test Scriptless test automation for GUI testing slides, version: 15th April 2019

MBT in practice: tools with non-FSM models

Harmony

Live demo with István Forgács

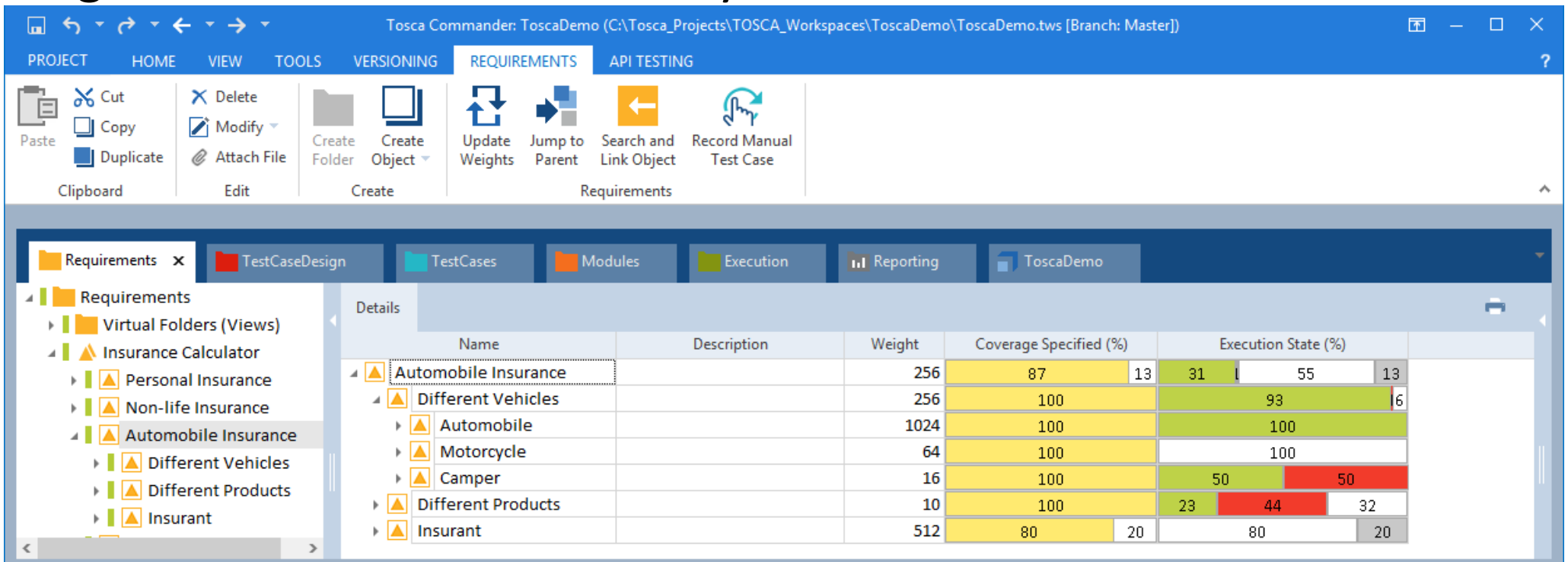


MBT in practice: tools with non-FSM models

Live demo with Bálint Szél

Tricentis Tosca TestSuite

- Commercial tool, 14 day free trial
- Instead of behavioural models of previous tools, it focuses on data model
- Concentrate on higher level (UI, API) tests
- Target audience: Business Analyst



MBT in practice: tools with non-FSM models

Live demo with Bálint Szél

Tricentis Tosca TestSuite – workflow:

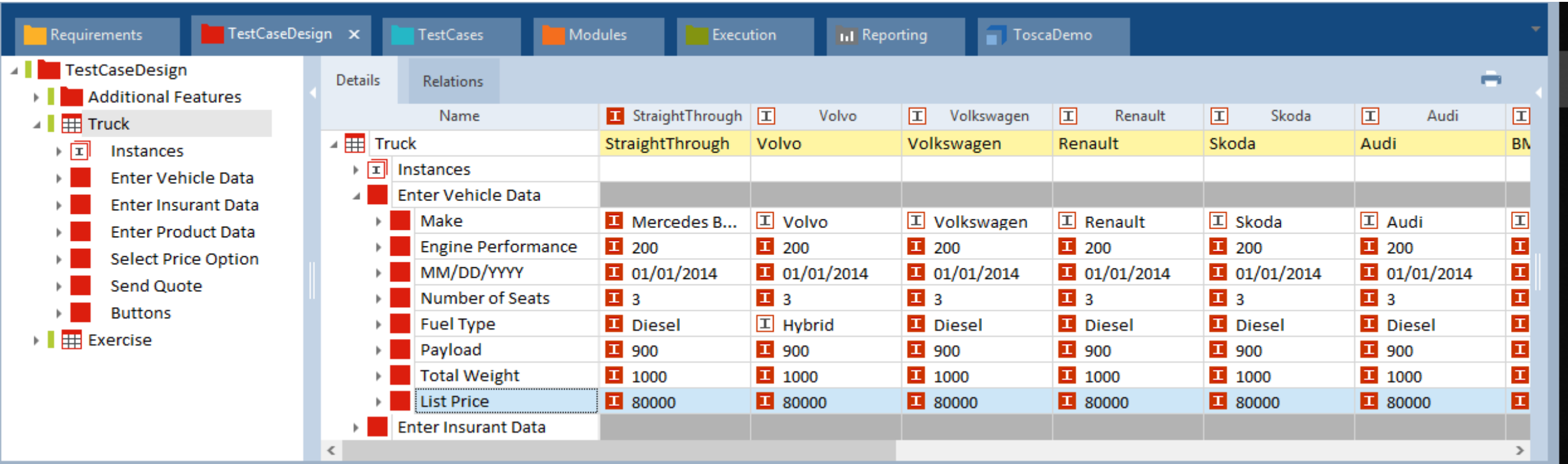
1. Requirements → assign given weights to them
   - Coverage:
     - For test requirements specified by test designer
     - Risk-based: based on the preset weights
2. Test case design
   - Test cases: Fill data model with concrete values
   - Optimize the number of test cases
3. Modules can be created to serve as building blocks
4. Execution of test cases
5. Reporting of test case execution

MBT in practice: tools with non-FSM models

Live demo with Bálint Szél

Tricentis Tosca TestSuite

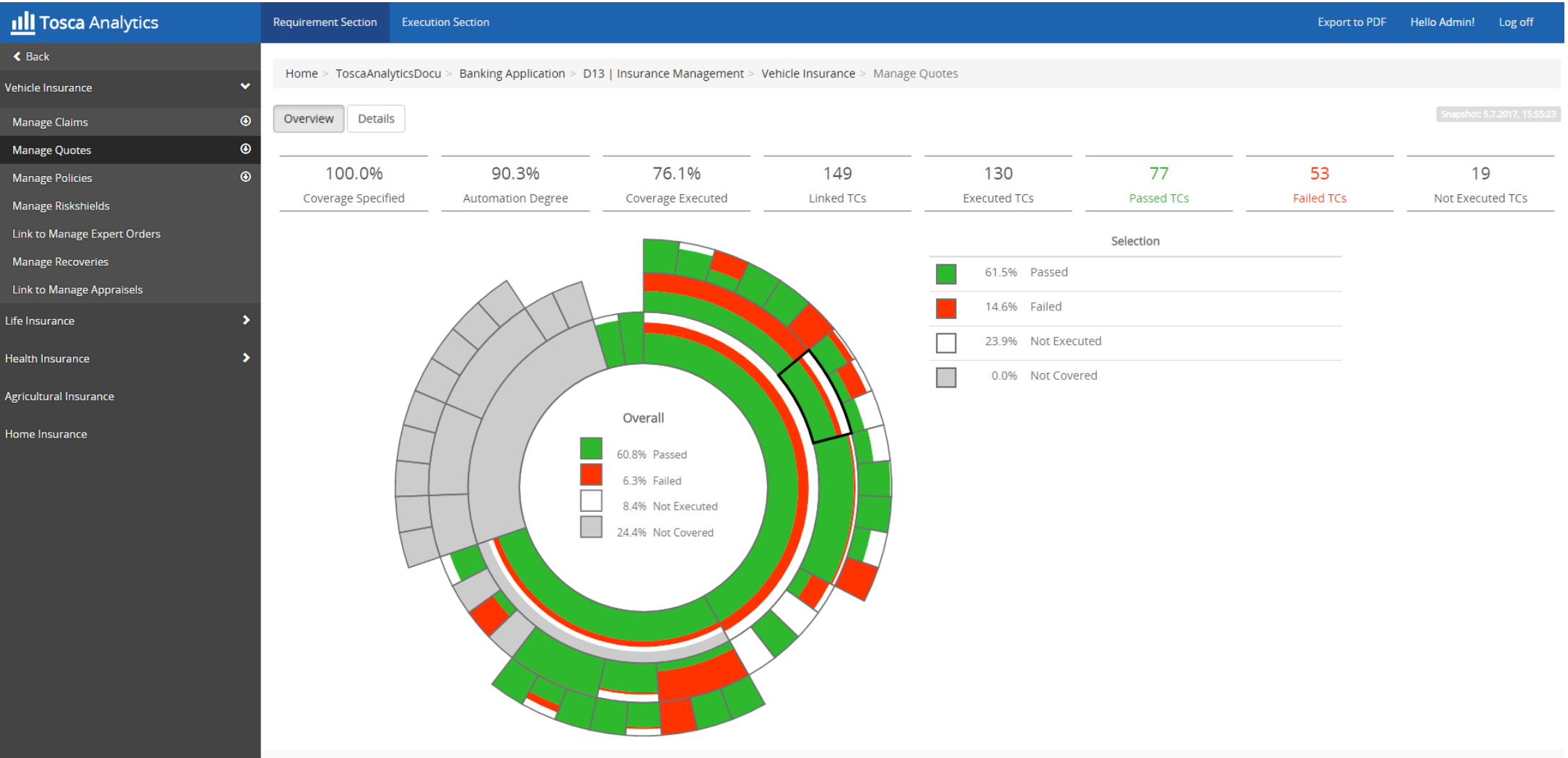
- Data model
  - Structure of the test data objects can be automatically read from implementation
  - Different techniques exist to fill the data model with values
    - Example: StraighTrough (default value for all variables) + alter one variable



MBT in practice: tools with non-FSM models

Tricentis Tosca TestSuite – test report:

Live demo with Bálint Szél



MBT in practice: some industry leader tools

Further information about different MBT tools, case studies

- A list of MBT tools by Zoltán Micskei
- An ETSI study from 2013 that compares tools through common examples
- A short summary of 5 tools
- 2019 Model-based Testing User Survey: Results

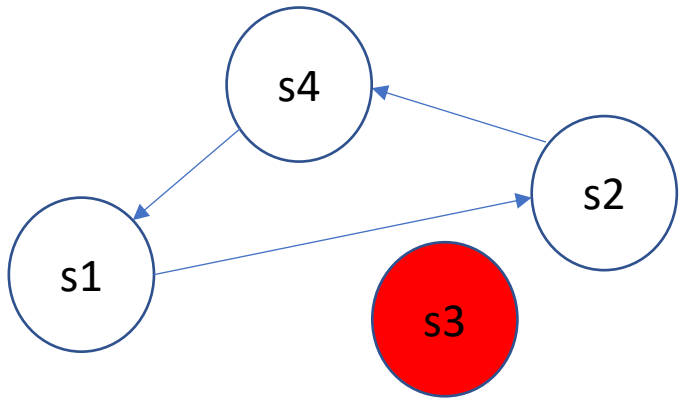
MBT in practice: typical problems 3/1

Not surprisingly the typical problems appears in the modeling…

- Wrong abstraction level is selected for models
  - Too few or too many details are provided

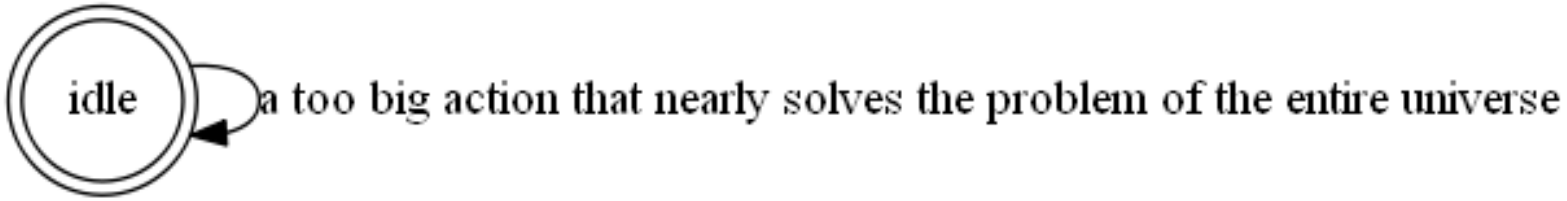
- No classification/nested state is used, i.e. try to model everything with only one model

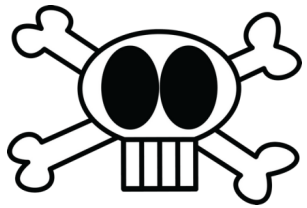
- Unreachable state is used



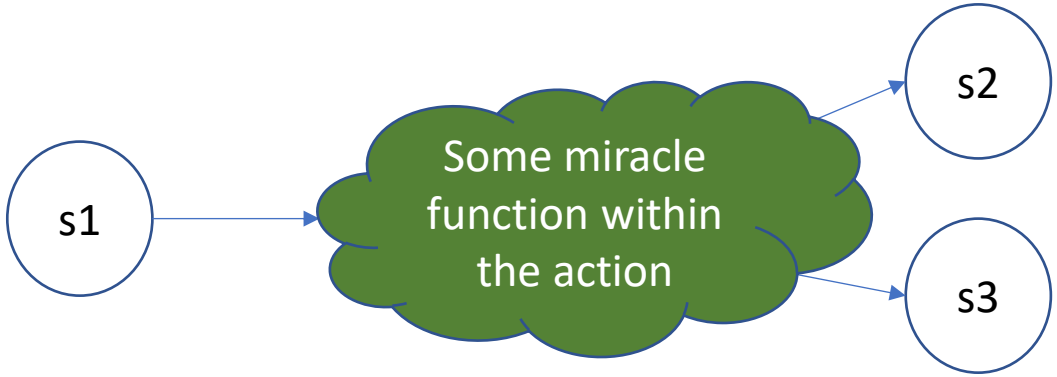
MBT in practice: typical problems 3/2

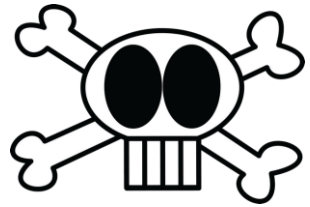
- Bad examples *(where not truly FSM model is used)*:
  - Too big actions with conditions inside
    - In extreme case one action in the FSM:





  - Teleport: next state is identified within an action not in the model:

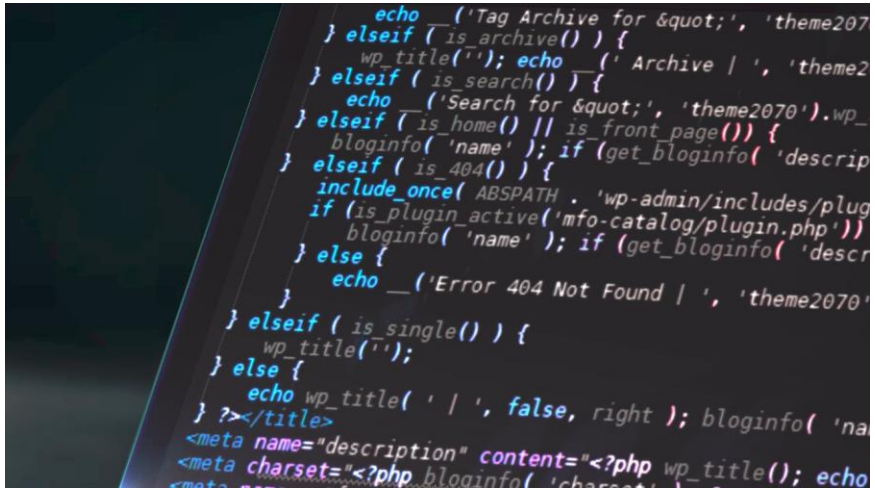


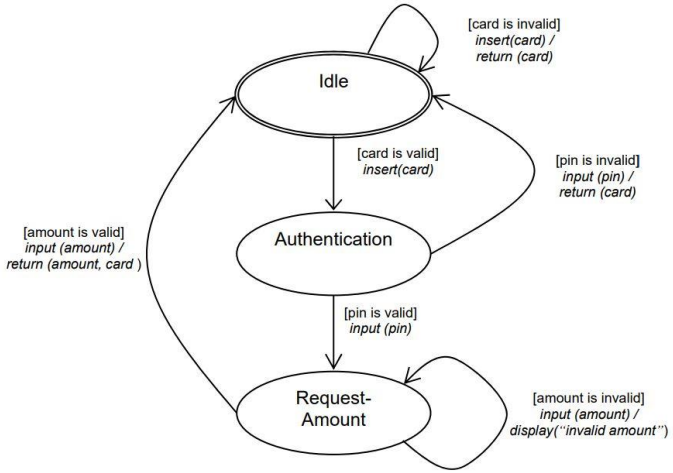


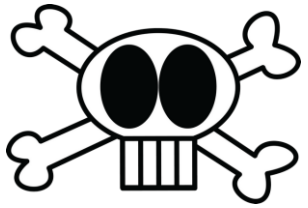
MBT in practice: typical problems 3/3

- Assuming that the models are always correct
  - Many iterations (review...etc.) required for verification

- In brown-field project the implementation is used to create a test model







In this case we will only able compare the implementation with the implementation!

MBT in practice: Advantages 2/1

- Modeling process
  - Close communication with stakeholders
    - Graphical description helps

    → common understanding of the problem → identify misunderstandings
  - Requirement engineers and testers can work together
  - For engineers models are more understandable than test cases
- Early modeling
  - Helps to avoid defects* at early stages

    ** incomplete, inconsistent and unclear requirements*
  - Test generation and analyzation is possible before any real system exists
- Traceability between use cases and tests

MBT in practice: Advantages 2/2

- Models can handle large complexity
  - Nested states, hierarchical description
- Tests can be generated
  - Reduces or eliminates the time of test design
- Models can be reused in similar problems
- Help to reduce maintenance costs when requirements change
  - Models are easier to maintenance, then test cases *(see incremental approach)*
- Flexible
  - Modify test selection criteria / required coverage in the tool
    → overall length of test suite changes

MBT in practice: Disadvantages 2/1

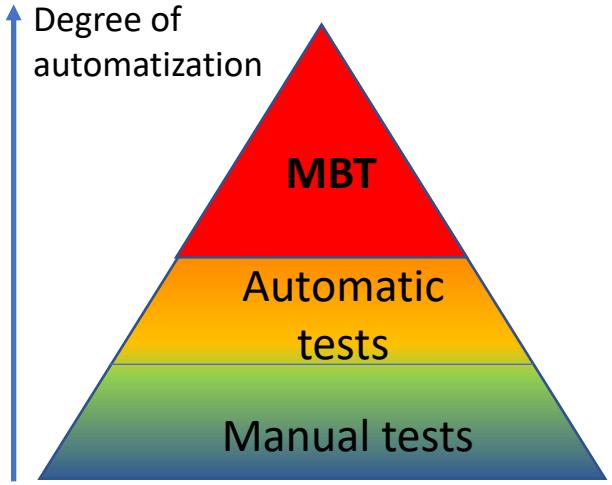
- Requires different thinking
  - Change management is hard
  - More qualified testers (system architect level)
- High initial cost
- Models can also contain bugs
  - Attention must be paid for this
- Still a big gap between theory and practice

MBT in practice: Disadvantages 2/2

- Lack of really good tools that have **all of these**:
  - Handle complex behaviours, have many coverage criteria and routing algorithms
  - Easy-to-learn
  - Interfaces to most important existing tools

    (programming language, test management and automation, requirement management, GUI)
    - Typically many glue code (adaptation code) required
  - Support team work, version handling of models

Summary of MBT



- Next level of test automatization:
  - Automated test design: model → generated test
- Good at defect prevention, easy maintenance through the evolving of SUT
- Can handle complex systems with hierarchical models
- Requires different type of thinking, high initial cost

Models in failure analyses

Main ToC

Overview of models in failure analyses

What is failure?

FMEA | Overview | Columns of an FMEA table | FMEA process

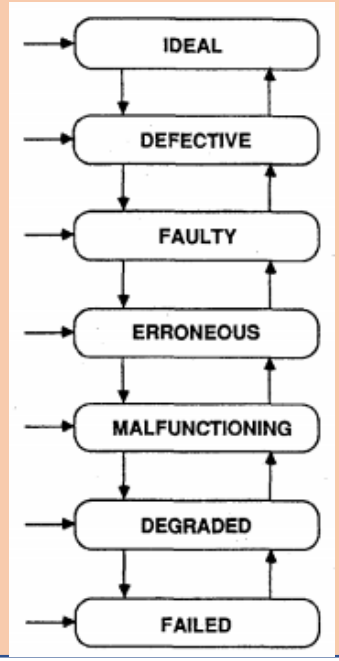
FTA | Overview | FTA elements | FTA tools

Main ToC

What is failure?

- Function loss under stated conditions

Fault (Bug): It is a condition that causes the software to fail to perform its required function. *Incorrect code*
Error: Refers to difference between actual output and expected output. *Cause of fault*
Failure: It is the inability of a system or component to perform required function according to its specification.

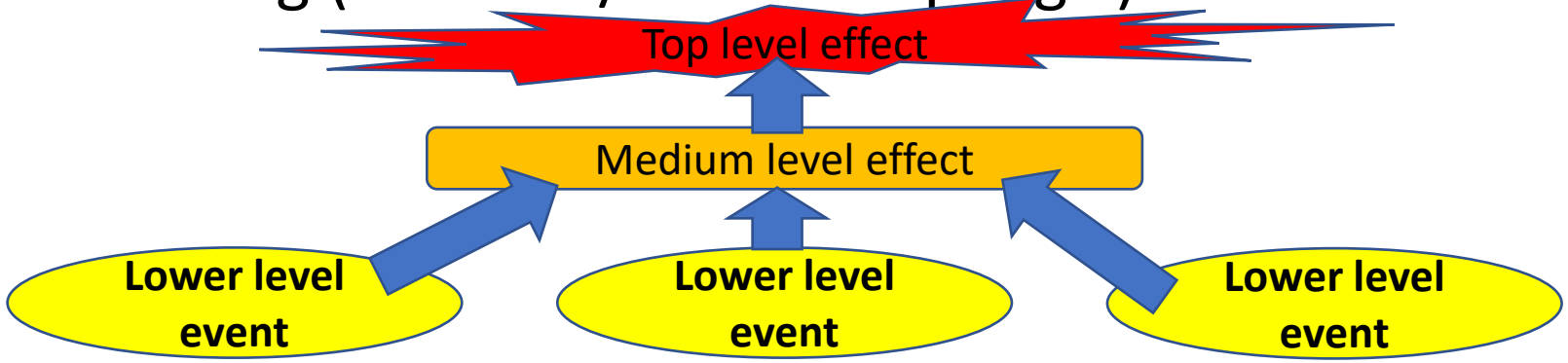


End condition we would like to avoid

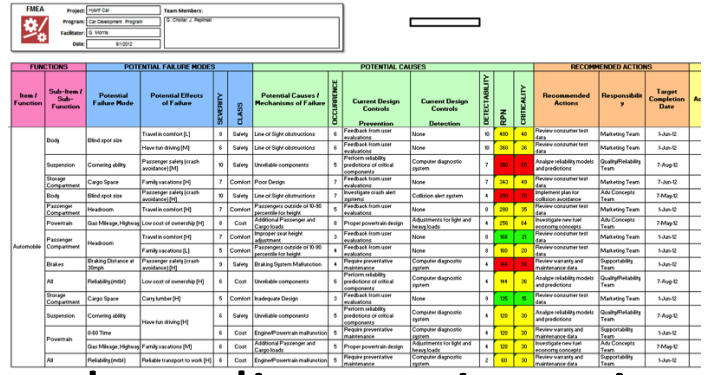
See B. Parhami: Defect, Fault, Error,..., or Failure? In: IEEE Transactions on Reliability (Volume: 46 , Issue: 4 , Dec 1997) article for further details about levels (source of the figure)

FMEA (Failure mode and effects analysis)

- Structured, systematic technique for failure analysis
- Inductive reasoning (forward / bottom-up logic)



- Analyzing component failures systematically and identifying their effects
  - The corresponding information is collected in a table:
    - Columns: Different aspects of analysis
    - Rows: List of failures



- Used in reliability engineering, safety engineering and quality engineering

FMEA (Failure mode and effects analysis)

- Developed by NASA in 1966 (as FMECA)

N70-76197
RA-006-013-1A

OFFICE OF MANNED SPACE FLIGHT | APOLLO PROGRAM

PROCEDURE

FOR

FAILURE MODE, EFFECTS, AND CRITICALITY ANALYSIS

(FMECA)

AUGUST 1966

NASA

REPRODUCED BY
NATIONAL TECHNICAL
INFORMATION SERVICE
U.S. DEPARTMENT OF COMMERCE
SPRINGFIELD, VA. 22161

NATIONAL AERONAUTICS AND SPACE ADMINISTRATION
WASHINGTON, D.C. 20546

1.4 OBJECTIVES OF FMECA

The FMECA provides:

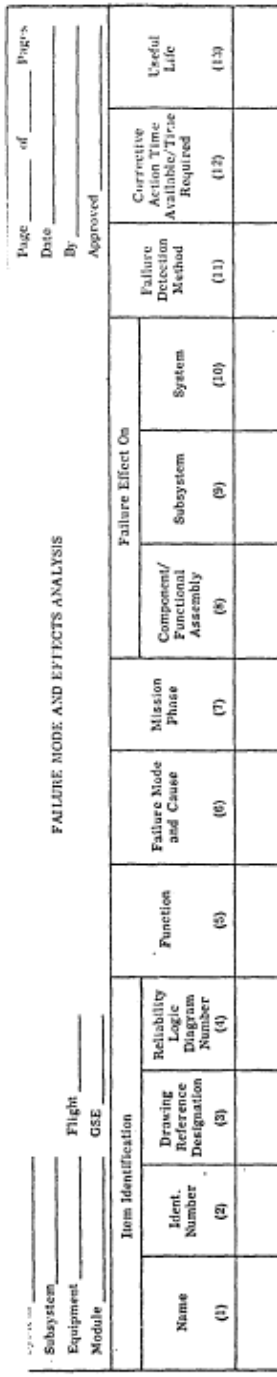
a. The design engineer with a method of selecting a design with a high probability of operational success and crew safety.
b. Design engineering with a documented method of uniform style for assessing failure modes and their effect on operational success of the system.
c. Early visibility of system interface problems.
d. A list of possible failures which are ranked according to their category of effect and probability of occurrence.
e. Identification of single failure points critical to mission success or to crew safety.
f. Early criteria for test planning.
g. Quantitative and uniformly formatted data input to the reliability prediction, assessment, and safety models.

1.5 USE OF FMECA

The FMECA is normally accomplished before a reliability prediction is made to provide basic information. An FMECA should be initiated as an integral part of the early design process of system functional assemblies and should be periodically updated to reflect design changes. This analysis may also be used to provide a model for analyzing already-built systems.

An updated FMECA is a major consideration in the design reviews, inspections, and certifications defined in NASA Apollo Program Directive No. 6, Office of Manned Space Flight, August 12, 1965, subject, "Sequence and Flow of Hardware Development and Key Inspection, Review and Certification Checkpoints."

An FMECA should be performed initially at the highest system level feasible. The purpose of this analysis should be to determine the criticality ranking of the major system elements so FMECA program effort may be scoped and allocated for subsystems and equipments critical to system operational success.



Columns of an FMEA table 4/1

1. Item: Where the failure occurs
2. Potential failure mode: describes the way the failure occurs
3. Potential cause(s) of failure
4. Effects of failure:
   - Local effect of failure: consequences of failure to the item under analysis
   - Next (higher) level effect of failure
   - End effect of failure: consequences of failure to the total system

Columns of an FMEA table 4/2

5. Detection: Method by which a failure is detected, isolated by operator/maintainer and the time it may take
   - *A Certain (fault will be caught on test)*
   - *B Almost certain*
   - *C High*
   - *D Moderate*
   - *E Low*
   - *F Fault is undetected by operators or maintainers*
6. Probability / occurrence of the given failure:
   - *1 Extremely unlikely*
   - *2 Remote*
   - *3 Occasional*
   - *4 Reasonably possible*
   - *5 Frequent*

These are just examples, other number of levels may occur depending on the application domain

Columns of an FMEA table 4/3

7. Severity/Impacts: The consequences of a failure mode
   - *I* *No relevant effect on reliability or safety*
   - *II* *Very minor, no damage, no injuries, only results in a maintenance action*
   - *III* *Minor, low damage, light injuries*
   - *IV* *Critical (causes loss of primary function; Loss of all safety margins, 1 failure away from a catastrophe, severe damage, severe injuries)*
   - *V* *Catastrophic* (product becomes inoperative; the failure may result in completely unsafe operation and possible multiple deaths)
8. RPN (Risk Priority Number): Detection * Probability * Severity
   - The higher the number, the bigger the risk priority
   - Can be classified into different levels
   - Tasks with high priority require action
     - *Depends on the application domain, but for example if Detection, Probability, Severity are numbers in the [0, 10] interval, then in a typical application area RPN>120 requires proper action*

Columns of an FMEA table 4/4

9. Latency (dormancy period):
   - The average time in that a failure mode may stay undetected (if known)
10. Required actions / mitigations / additional remarks

FMEA process

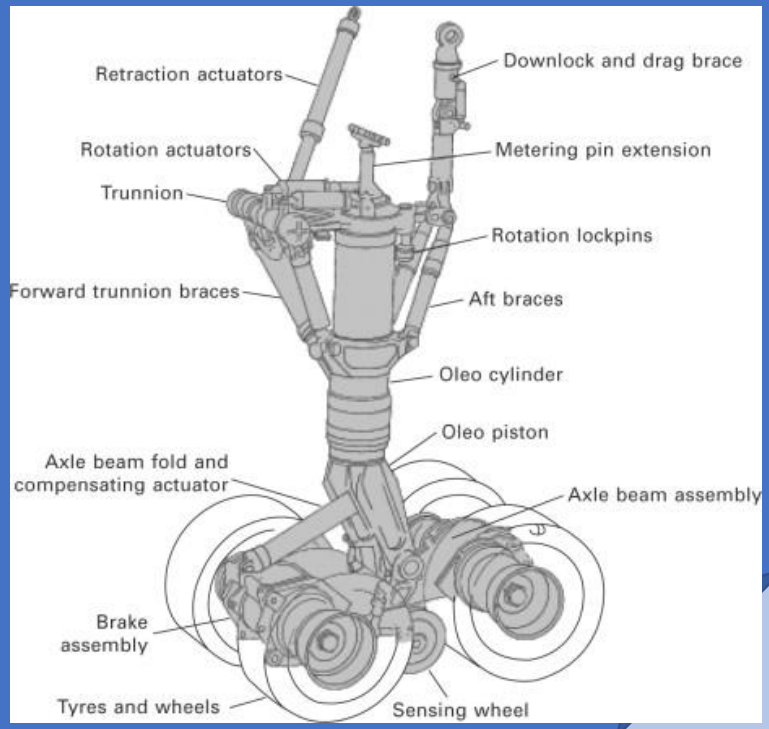
1. Clarify the scope of the FMEA. Is it for concept, system, design, process or service? Keep attention to the boundaries of FMEA!
2. Create a cross-functional team of people with diverse knowledge
3. Use the proper FMEA template (i.e. valid columns for the given scope)
4. Identify all functionality of your scope
5. For each functionality identify failure modes (i.e. all the ways failure could happen)
6. For each failure modes fill the columns (root cause, effects in diff levels, probability, severity, detection…etc.)
7. Identify required actions based on previous columns!

Wake up, exercise!

FMEA exercise

Item	Phase	Potential Failure Mode	Possible Root Causes	Failure Effects			Detection	Proba-bility	Severity	Risk level (P*S)	RPN	Required Actions
				Local	Next Higher	End Item						

- Create an FMEA table to identify the possible problems of the landing gear of an airplane.
  - Beside possible failures, their root causes, effects, please do not forget to create the list of required actions that should be followed when the given failure occurs
    - see the columns of the table above
      - In different phases, different failures may occur
      - Propagate failure effects to higher levels step by step



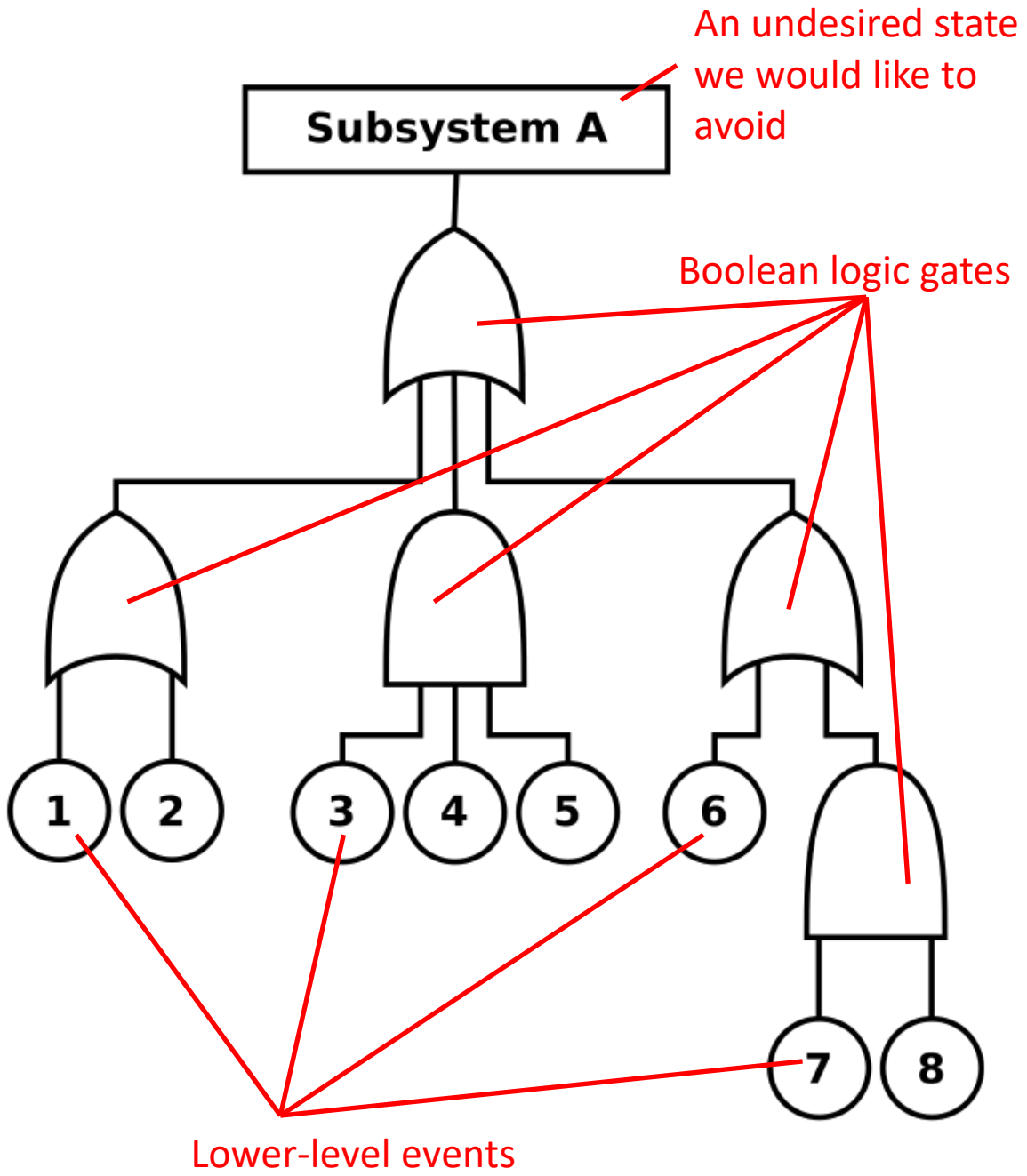
Source of the figure:
Chapter 3.2.4 Landing gear In Introduction to Aerospace Materials, Editor(s): Adrian P. Mouritz,Woodhead Publishing, 2012, https://doi.org/10.1533/9780857095152.39.

FMEA: solution for the task

Item	Phase	Potential Failure Mode	Possible Root Causes	Failure Effects			Detection	Proba-bility	Severity	Risk level (P*S)	RPN	Required Actions
				Local	Next Higher	End Item						
Landing gear	On ground	Steering failure	control failure, mechanical failure	Can not be steered	Unable to go into the runway	Unable to start flight	3 High	C Occasional	I No relevant effect	Extremely low	9	Report the problem to tower and try again. If it still does not work ask a traction engine to tow it into a parking place.
Landing gear	Landing	Fail to brake	control failure, hydraulic failure	Unable to use brakes, try emergency brakes	Unable to use emergency brakes	Unable to stop till the end of runway, possible crash	3 High	B Remote	V Catastrophic	Medium	15	Try again. Report to the tower and depending on fuel level and remaining runaway length (1) try emergency braking and set engines to reverse or (2) overshot and continue this procedure again in next landing try.
Landing gear	Landing	Fail to extend	control failure, mechanical failure	Unable to extend landing gears automatically, try mechanically	Unable to extend landing gears mechanically	Forced landing without gear, possible crash	2 Almost Certain	B Remote	V Catastrophic	Medium	15	Try to extend manually. If this does not help, report the problem and try to retract and extend manually multiple times. If there's no effect, discuss with tower which other methods may help. If the fuel runs out then release remaining fuel and try forced landing without gear.
Landing gear	In flight	Fail to retract	control failure, electrical motor failure, mechanical failure	Unable to rectract the gears automatically	Drag on vehicle performance	Unable to continue flight	2 Almost Certain	B Remote	I No relevant effect	Extremely low	9	Report the problem to tower. Set to extend and try retract again. If there's no effect, then go back to the airport and land.

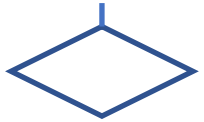
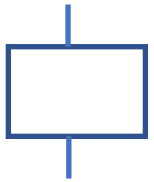
FTA (Fault Tree Analysis)

- Top-down, deductive failure analysis
- An undesired state of a system is analyzed using Boolean logic to combine a series of lower-level events
- Focus on risk and fault analyses
- Mainly used in safety and reliability engineering
  - Application areas:
    - Automotive and aerospace industry
    - Medical industry
    - Security/Vulnerability analyses



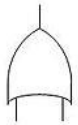
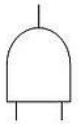
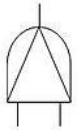
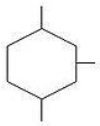
FTA: elements 3/1

- Event symbols:

Name	*Graphical symbol*	*Description*
• Basic event		failure or error in a system component or element
• External event		normally expected to occur
• Undeveloped event		an event about which only insufficient information is available
• Conditioning event		an event that affects the working of logic gates
• Intermediate event		an output of logic gates

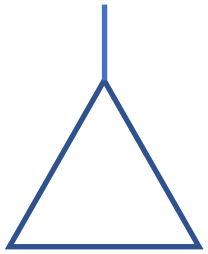
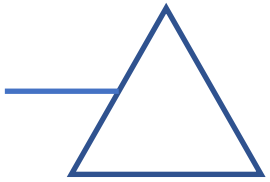
FTA: elements 3/2

- Boolean logical gates:

Name	*Graphical symbol*	*Description*
• OR		output occurs if any input occurs
• AND		output occurs if all input occurs
• XOR (Exclusive OR)		output occurs if exactly one input occurs
• PAND (Priority AND)		output occurs if all input occur in a specified order
• Inhibit		output occurs if specified conditioning event occurs

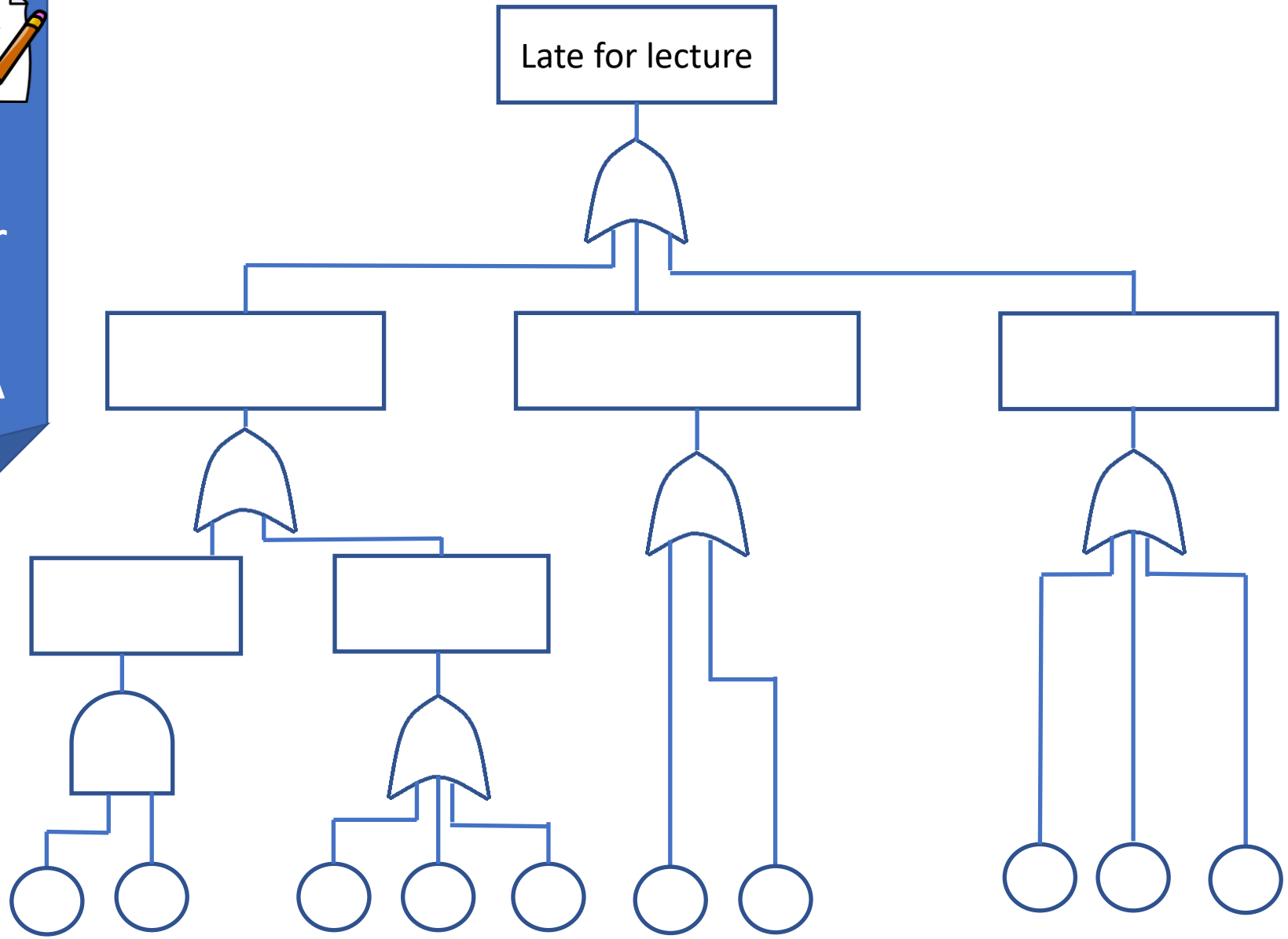
FTA: elements 3/3

- Transfer symbols:
  - Used to connect the FTAs of different subsystems:

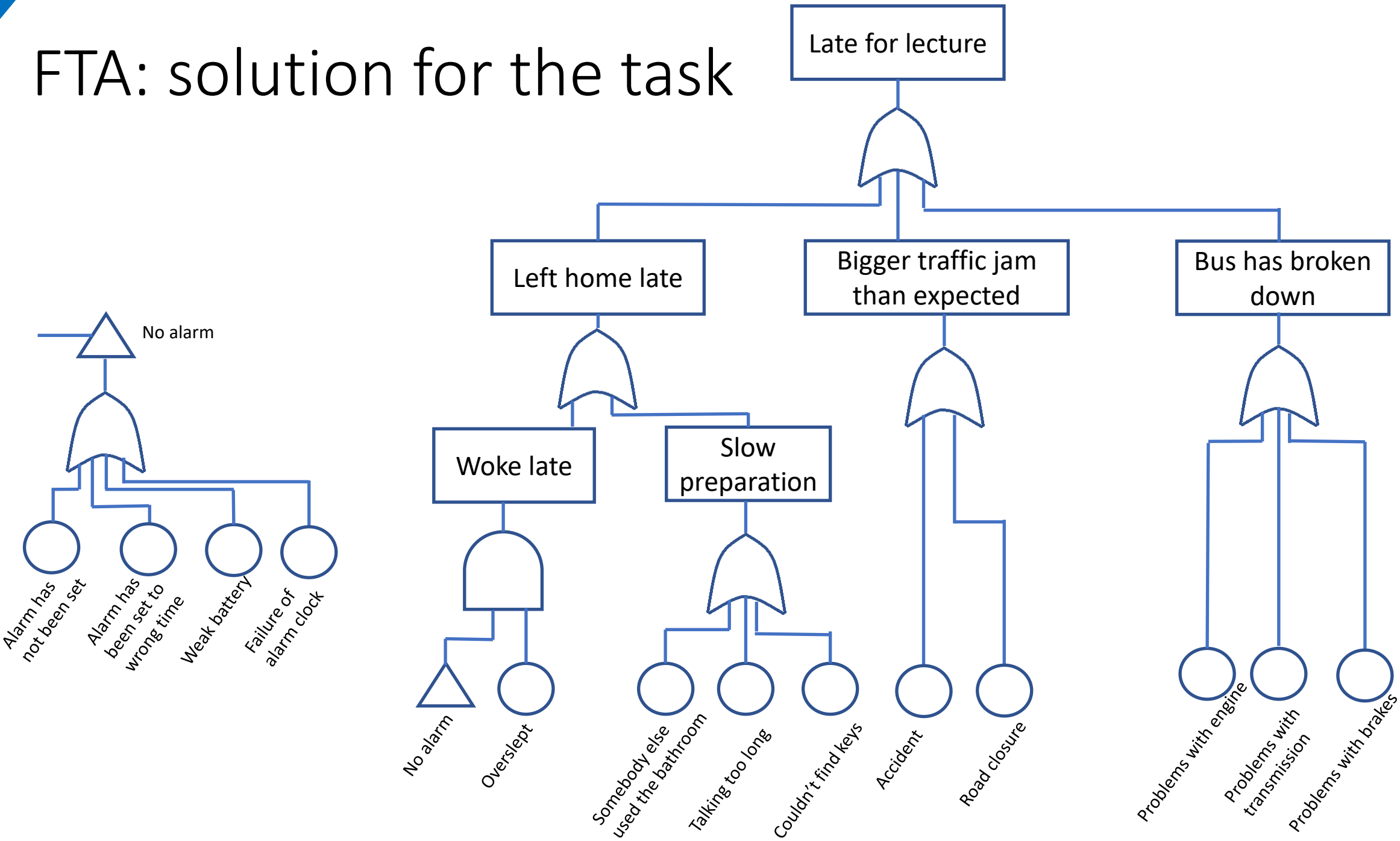
Name	*Graphical symbol*	*Description*
• Transfer in		Import the output of a given subsystem's FTA as an input event in the FTA of the main system
• Transfer out		Export the output of the given FTA

FTA exercise

- We would like to avoid arriving late for the lecture.
- Create a possible FTA for this problem!

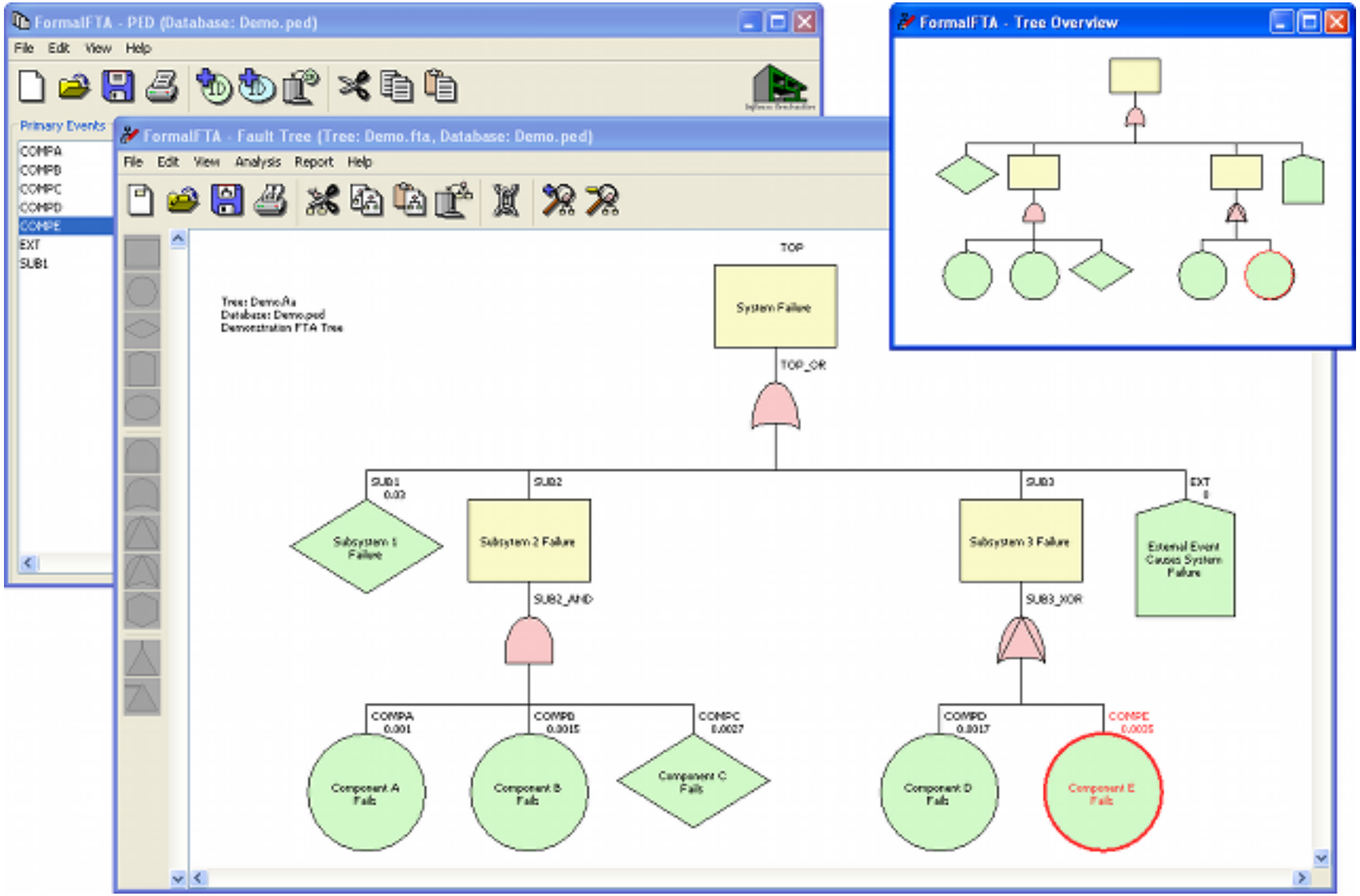


FTA: solution for the task



FTA: Tools

- Free FTA tools:
  - OpenFTA
  - FormalFTA



Performance testing

Main ToC

Overview of performance testing

- Performance testing types
  - Introduction
  - Load testing
  - Endurance testing
  - Stress testing
  - Capacity testing
  - Spike testing
  - Scalability testing
  - Scheduling
  - At different test levels
  - Integration with other testing attributes
- Main steps of performance testing
- Tools in performance testing
  - Load generation
    - Working principle
    - Behaviour description
    - Model-driven methods
      - *JMeter demo*
      - *TitanSim demo*
  - Performance monitoring
    - Measured parameters
      - *Wireshark demo*
  - Log Analyses
- Summary

Main ToC

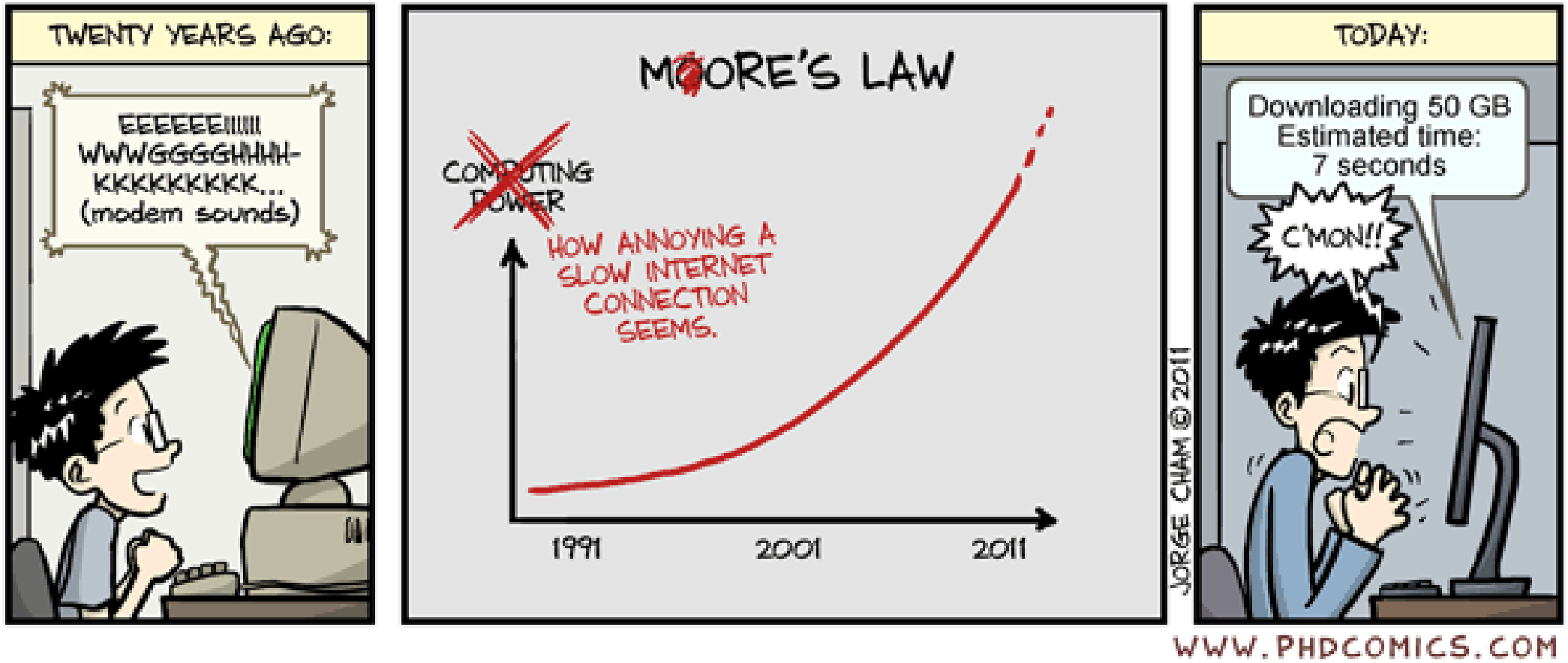
Performance testing

- Umbrella term, encompassing a wide range of test types
- Deals with questions related to application speed, efficiency, and stability
- Also deals with resource consumption, capacity planning and hardware sizing (e.g. memory, processors, disk space, and network bandwidth)

- Types:
  - Load testing
  - Endurance (stability, soak) testing
  - Stress testing
  - Capacity (volume, flood) testing
  - Spike testing
  - Scalability testing

Why is performance testing more and more important?

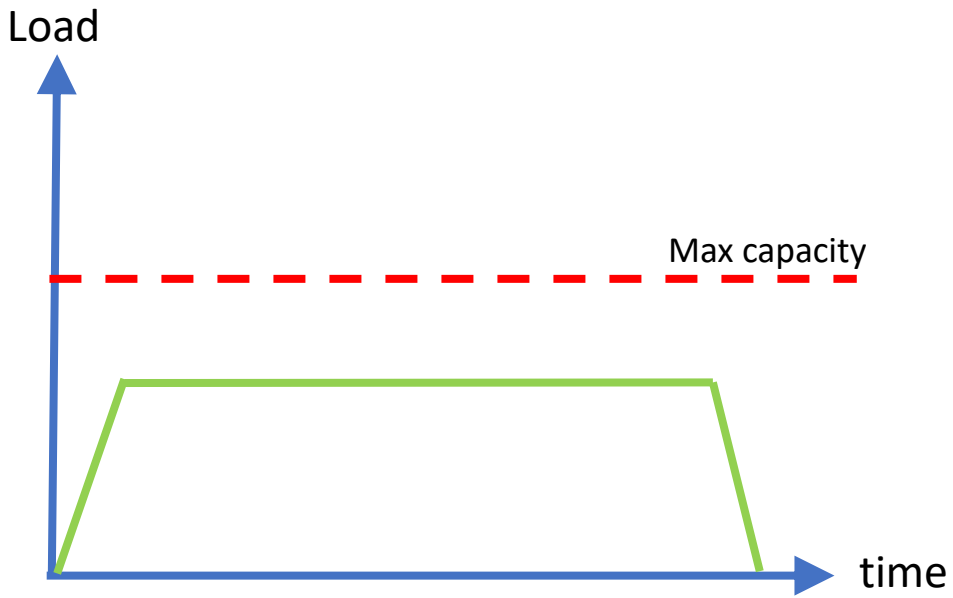
Increasing demands…



Performance testing types

Load testing:

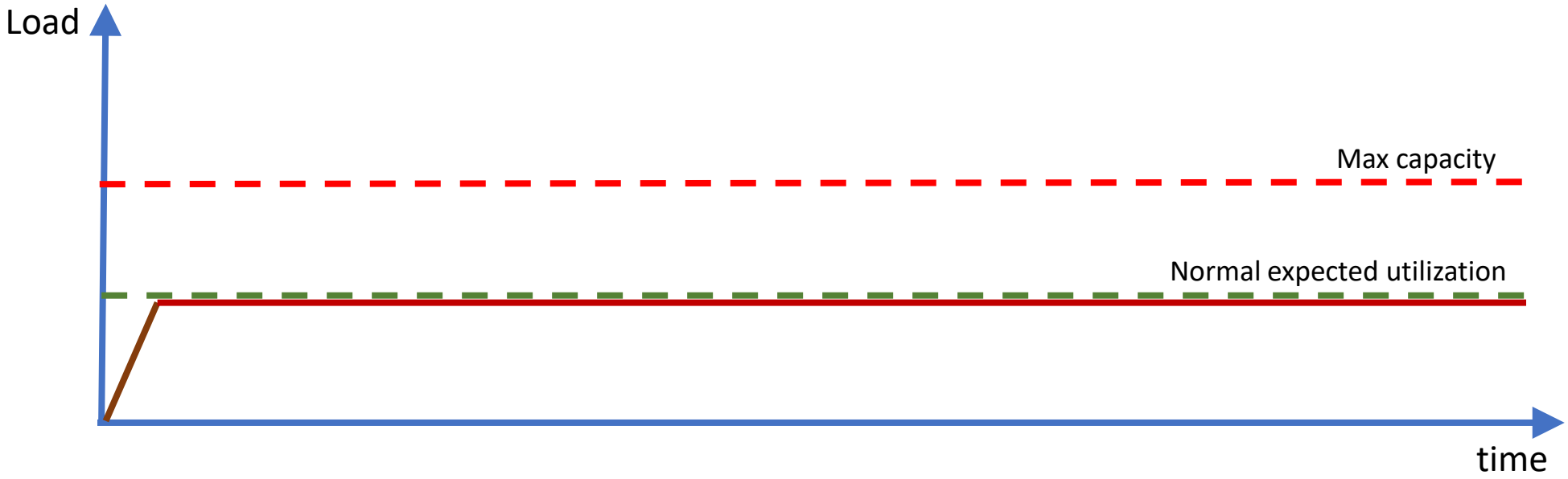
- Used to understand the behaviour of the SUT under a specific expected load
- Usually applied in a controlled (laboratory) environment



Performance testing types

Endurance (stability, soak) testing:

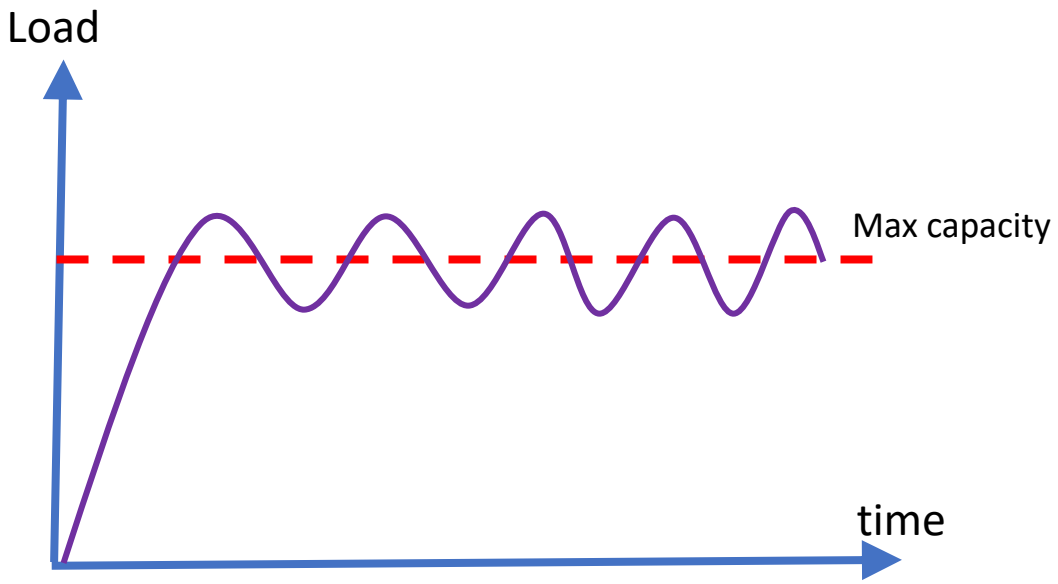
- Similar to load test, but focuses on the stability of the system over a bigger predefined time frame (it depends on the domain, 72hrs for example)
- Used to verify memory leaks, thread problems, database problems, overflows / underflows…etc. that may degrade performance or cause crash



Performance testing types

Stress testing:

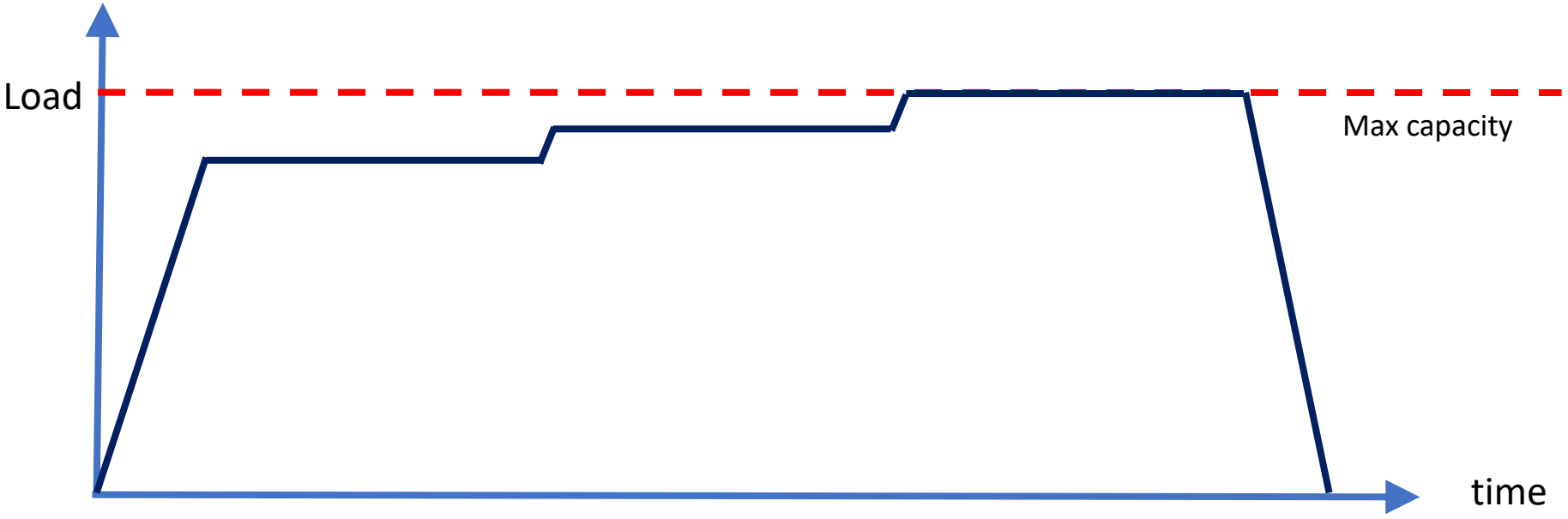
- Used to understand the upper limits of capacity of the SUT
  - Test above normal operational capacity, around max. design capacity
    - (It checks if a relatively small overload on short scale can be handled by scheduling, buffering, etc.)



Performance testing types

Capacity (volume, flood) testing:

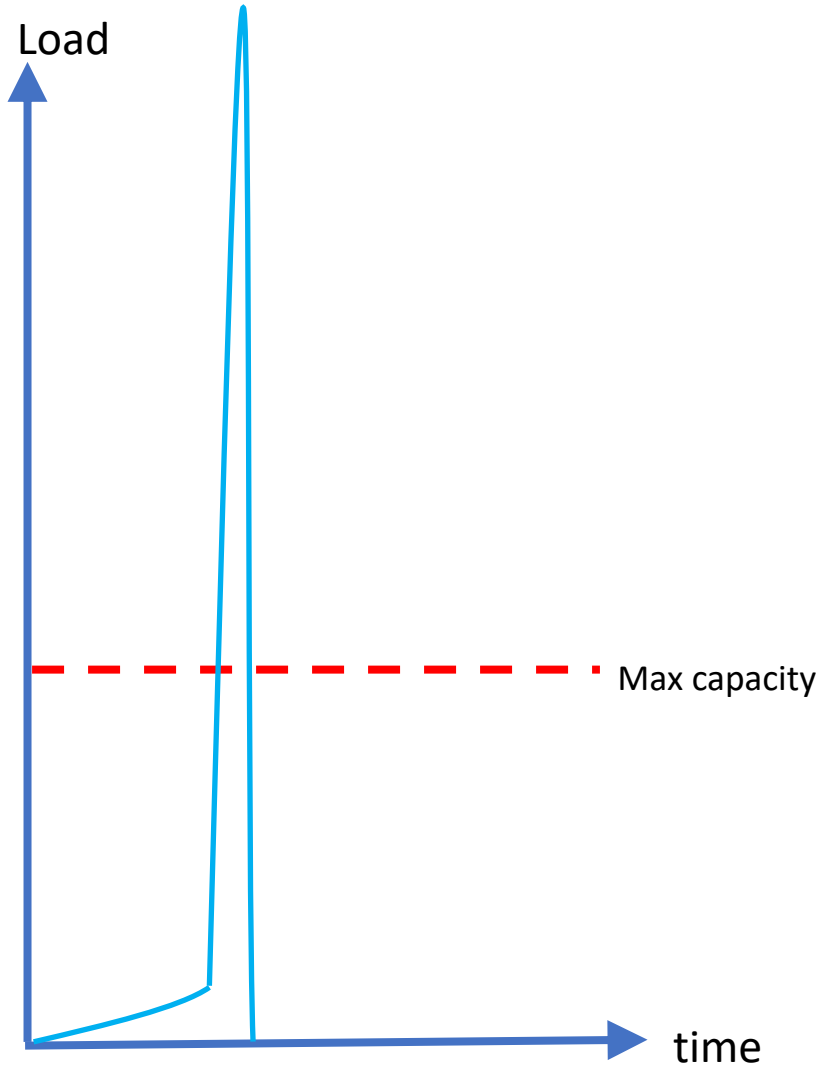
- Determines whether the SUT can manage the amount of workload that was designed for
- When this boundaries are not known in advance, it *benchmarks* the number of users or transactions that the system is able to handle
  - This can be used as a *baseline* for later testing
- The volume of data is increased step-by-step to analyse the actual capacity



Performance testing types

Spike testing:

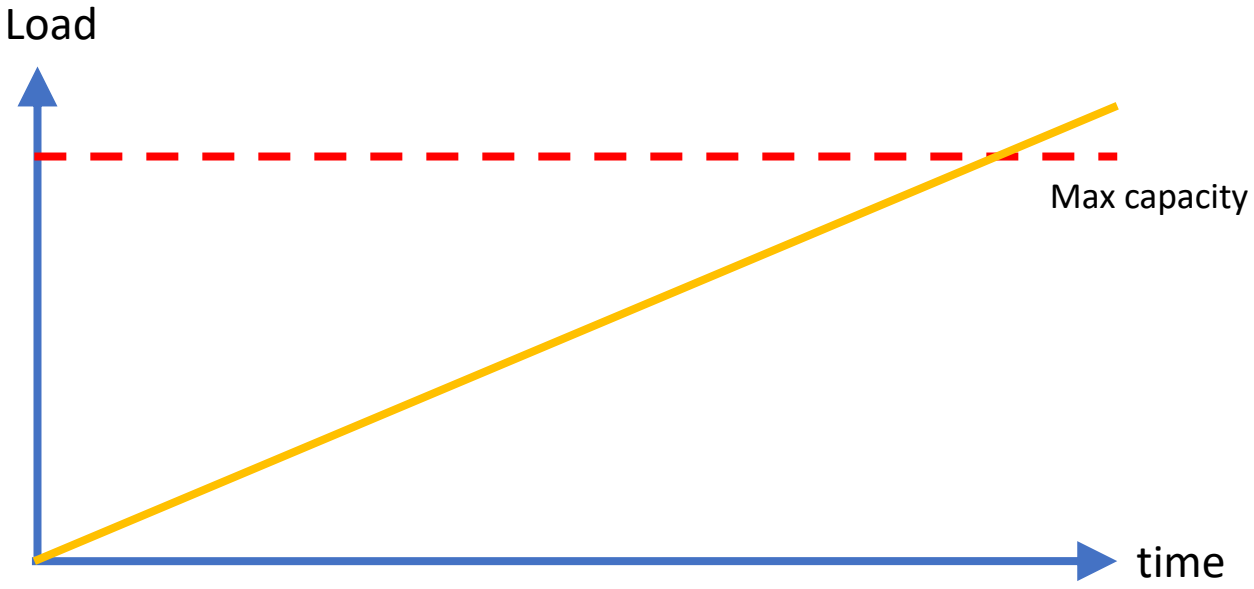
- Used to understand the functioning of a system if the load consumedly exceeds the max design capacity for a short time period
- Investigates whether the SUT crashes, terminates gracefully or just dismisses/delays the processing due to the sudden bursts of the requests



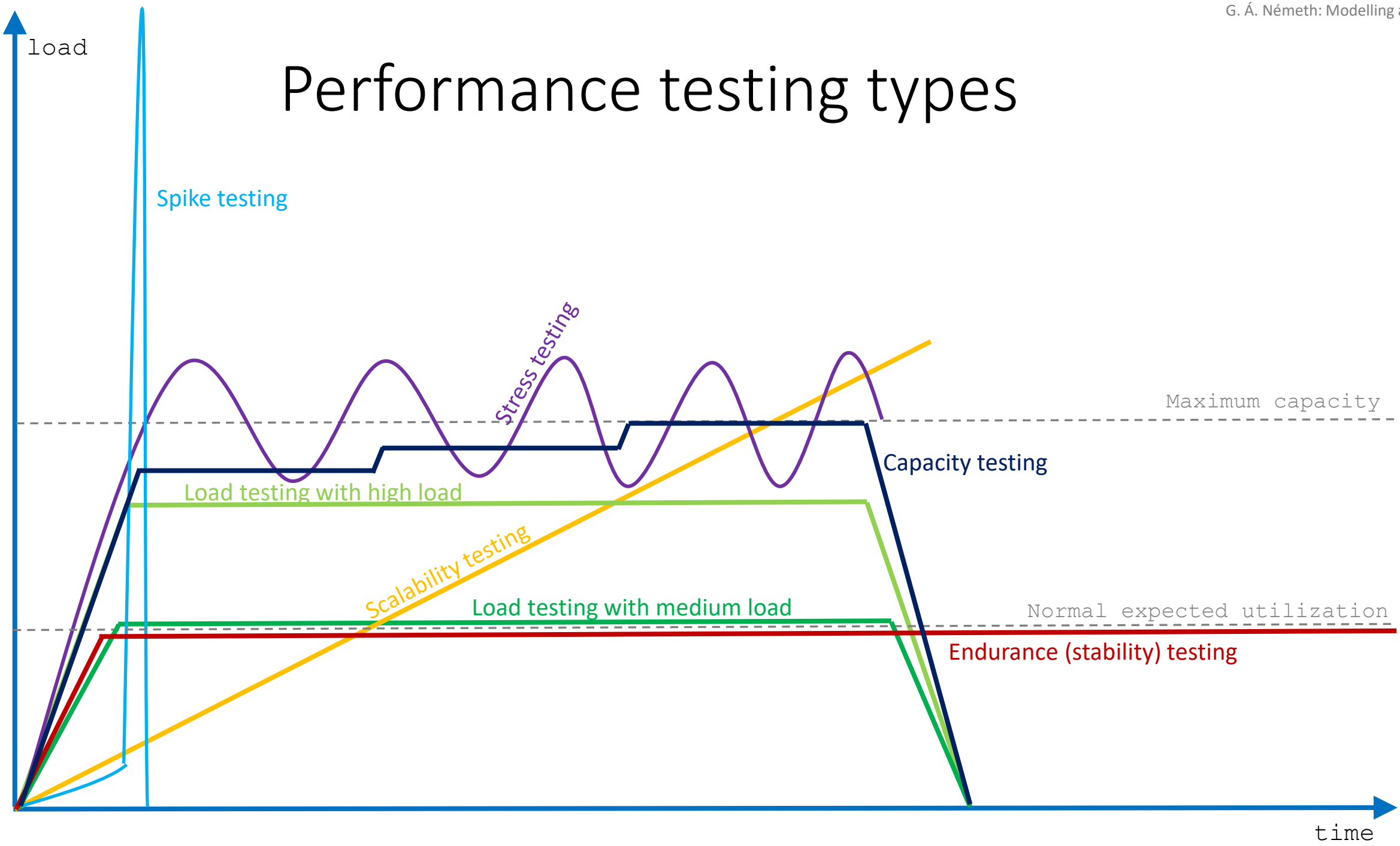
Performance testing types

Scalability testing:

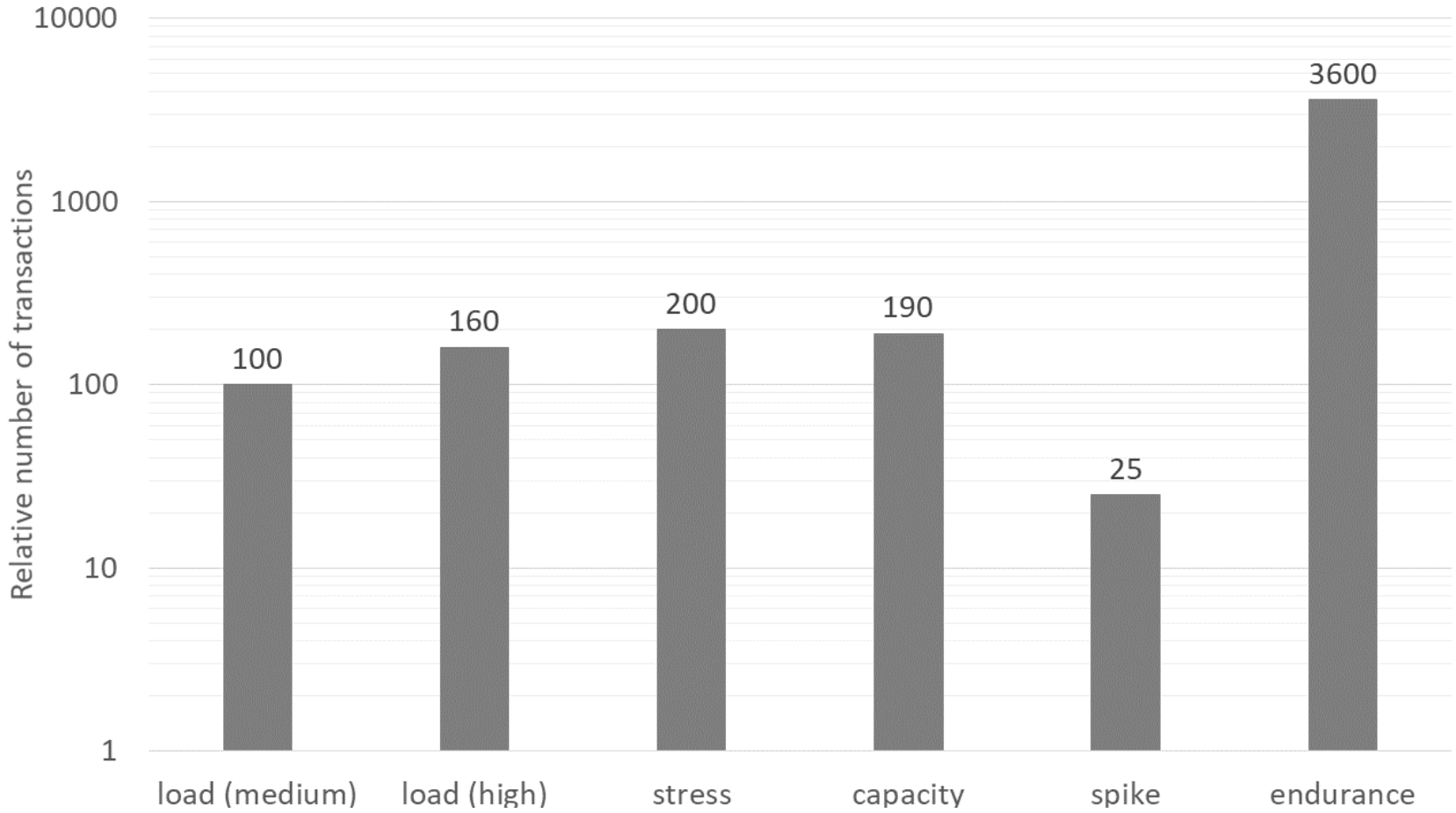
- Shows how the SUT is capable of scaling up/out/down considering some resources (CPU, GPU, memory or network usage)
- 2 approaches (both can be used to find possible bottlenecks):
  - Increase the load to monitor the amount of different types of resources used
  - Scaling up/out the resources of the SUT with the same level of load



Performance testing types



Performance testing types



Relative number of transactions in the performance testing of an industrial application. The requirements:

(1) test duration: 5 minutes for spike test, 72 hrs for endurance tests, 2 hrs for all other test types;

(2) load level:

- *50% for load testing with medium load and for endurance testing, 80% for load testing with high load,*
- *100% ±20% for stress testing*
- *90-95-100% for capacity testing*
- *300% for spike testing for 5 minutes, otherwise 0%,*

Scheduling of different types of performance tests

- Load tests:
  - Regularly
- Endurance tests:
  - Only at major milestones (less critical systems)

  OR
  - Continuously in a dedicated server (more critical systems)
- Capacity tests:
  - Measure actual capacity once & retest sometimes
  - Recalibrate if necessary
- Spike tests:
  - Frequency depends how critical the SUT is
- Scalability tests:
  - Once and apply again if we would like to increase the performance of the SUT or if the environment has been changed

Scheduling of different types of performance tests

- Number of measurements, reference points:
  - **One measurement is not a measurement!**
    - Multiple performance tests are required to ensure consistent findings due to the problems related to hardware and frameworks used below the measured SUT
  - **A baseline is also required** where the measured parameters can be compared with
    - The reason for this is that the SUT is not completely independent from its environment (i.e. it can be connected to a real network...etc.).

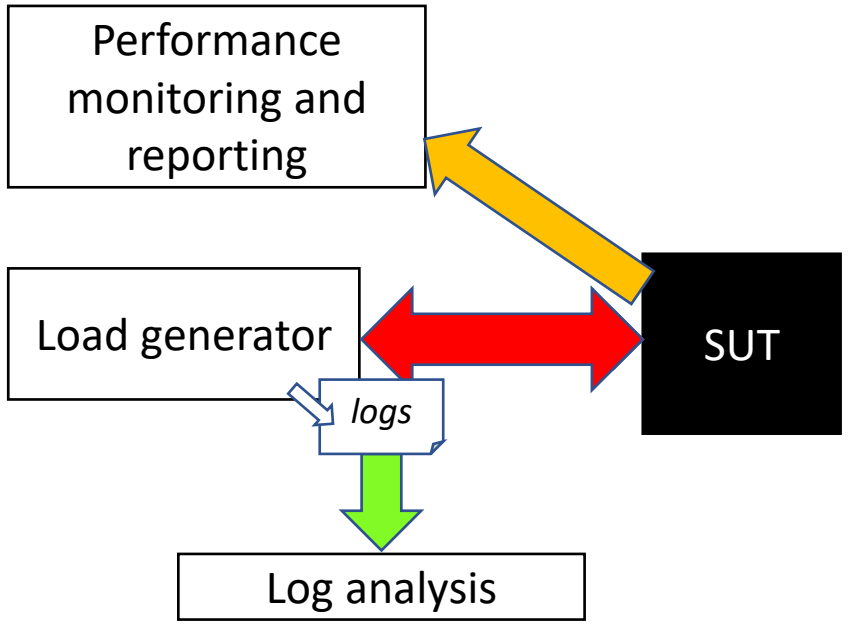
Main steps of performance tests 2/1

1. High level analysis
   - Understand…
     - Requirements, objectives
     - The main characteristics of the SUT, the architecture of the SUT
     - Possible high-level risks
2. Technical analysis and planning
   - Identify key scenarios
   - Finalize objectives with risks, environments, available tools and configurations
   - Define scheduling, monitoring methods and exit criteria
3. Design
   - Load profiling: define variability and distribution of load
   - Various workload models using different approaches for behaviour description

Main steps of performance tests 2/2

4. Implementation
   - Configure load generator, monitoring and analyses tools
   - Develop scripts and procedures, run preliminary tests
5. Execution
   - Run and monitor tests, validate the results
6. Results analysis and reporting
   - Individually or by cross-functional teams
   - 3 strategies:
     1. Verifying against threshold values
     2. Detecting known types of problems (memory leaks, deadlocks, unhealthy system states, throughput problems)
     3. Detecting anomalous behaviour
   - Write report (recommendations, risks, costs, and limitations)

Tools in performance testing



3 main functionalities:

1. Load generation
   - Generate a given workload to the SUT

2. Performance monitoring and reporting
   - Investigate some performance related aspects of the SUT
   - May give an alert on lower performance conditions or detect and notify suspicious behaviours
   - Create a report

3. Log analysis
   - Converts existing logs into the desired format *(example: loadgen statistics → statistics for given traffic mix)*, highlights data with the desired metrics…etc.
   - Helps searching on existing logs
   - May adds additional level of warnings and alerts to log data

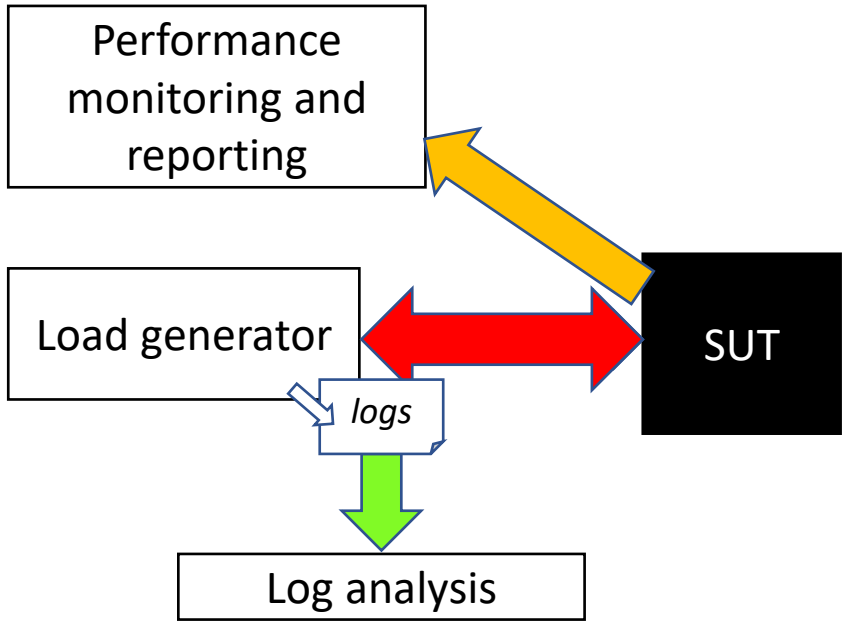
Tools in performance testing

The different tools may consist the 3 main functionalities in different extents:

- A single tool with all the required functionalities

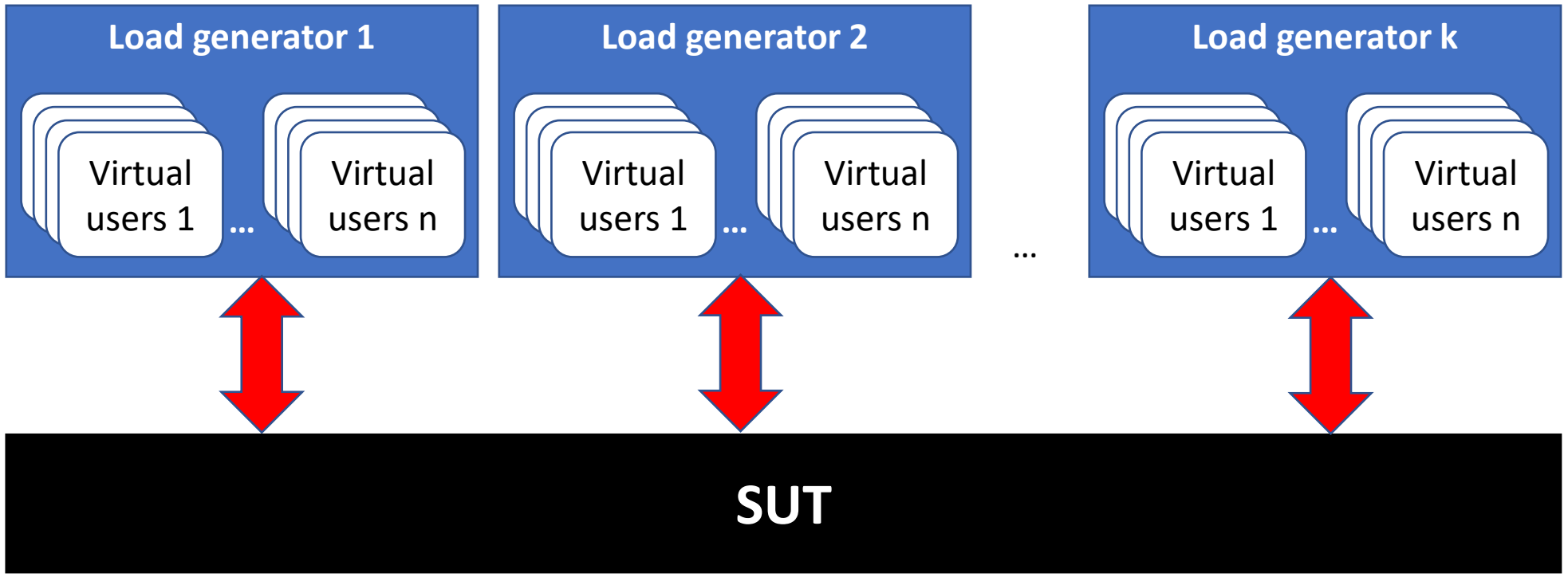
OR

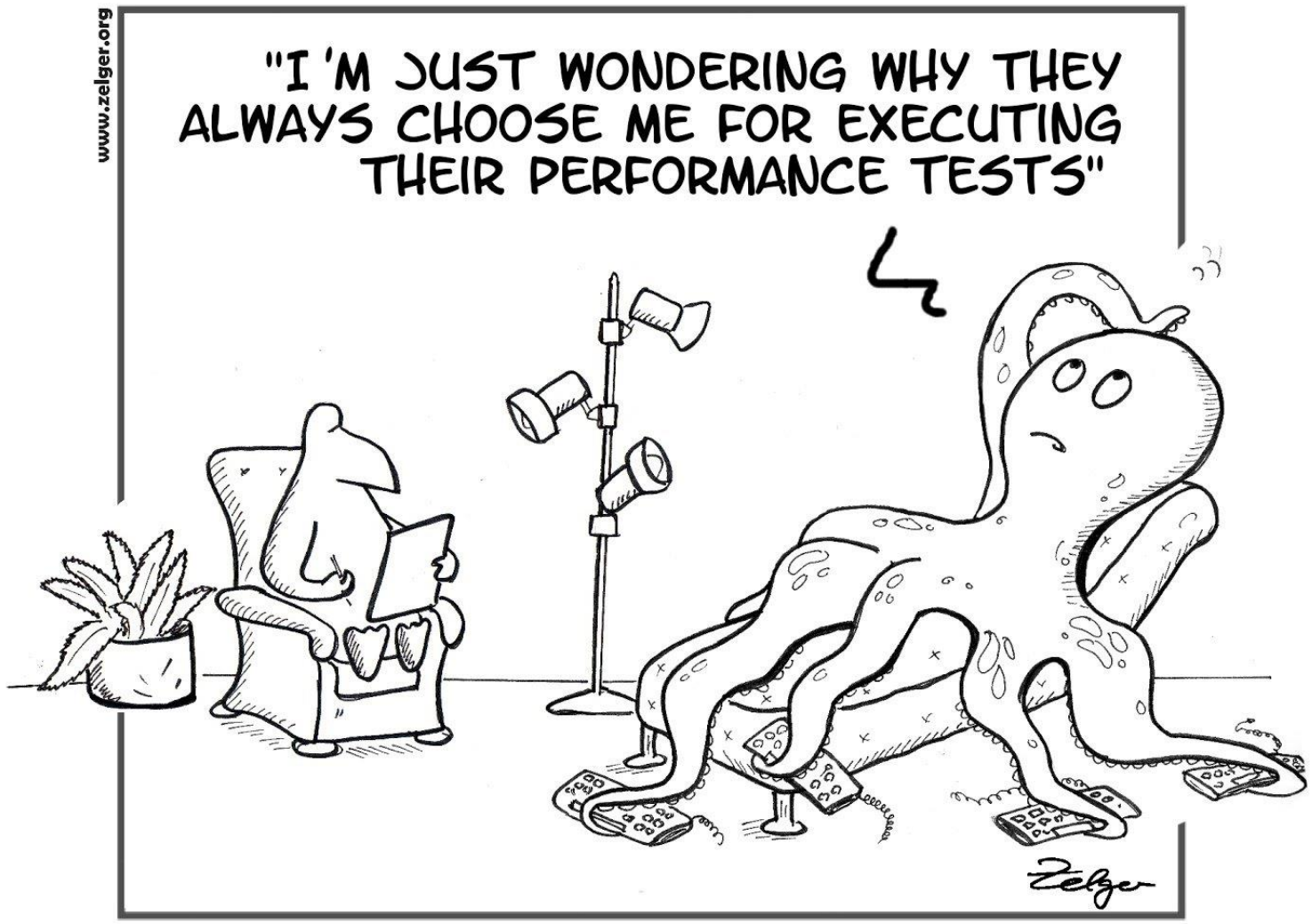
- A load generator with only basic monitoring + monitoring tool + a tool for log analysis



Load generation – working principle

- The simulation or emulation of the workload is achieved by creating virtual users that are distributed into load generators





The multi-threaded octopus was looking for a change

Load generation – behaviour descriptions

- 4 main approaches for the behaviour description of load generation:
  1. Hardwired (packet-generators)
  2. Call-flow (traffic playback tools)
  3. Source code
  4. Model-based descriptions

- These approaches scales between high performance with simple behaviour and medium performance with complex behaviour

Load generation – behaviour descriptions

1. **Hardwired (packet generators)**
   - Create a discrete chunk of communication in a predefined format
     - Some data field of this packet can be changed, but the same for the entire load

   - Pros:
     - Simplest approach →
       1. Easy to learn
       2. Highest possible performance

   - Cons:
     - Not flexible
     - Do not handle alternative behaviours

   - Example tools: Netstress, MikroTik Traffic Generator of router OS

Load generation – behaviour descriptions

1. Hardwired (packet generators)

Screenshot of Netstress

Load generation – behaviour descriptions

2. Call-flow (traffic playback tools)

1. Edit* or record an existing** traffic

   * With a call-flow editor

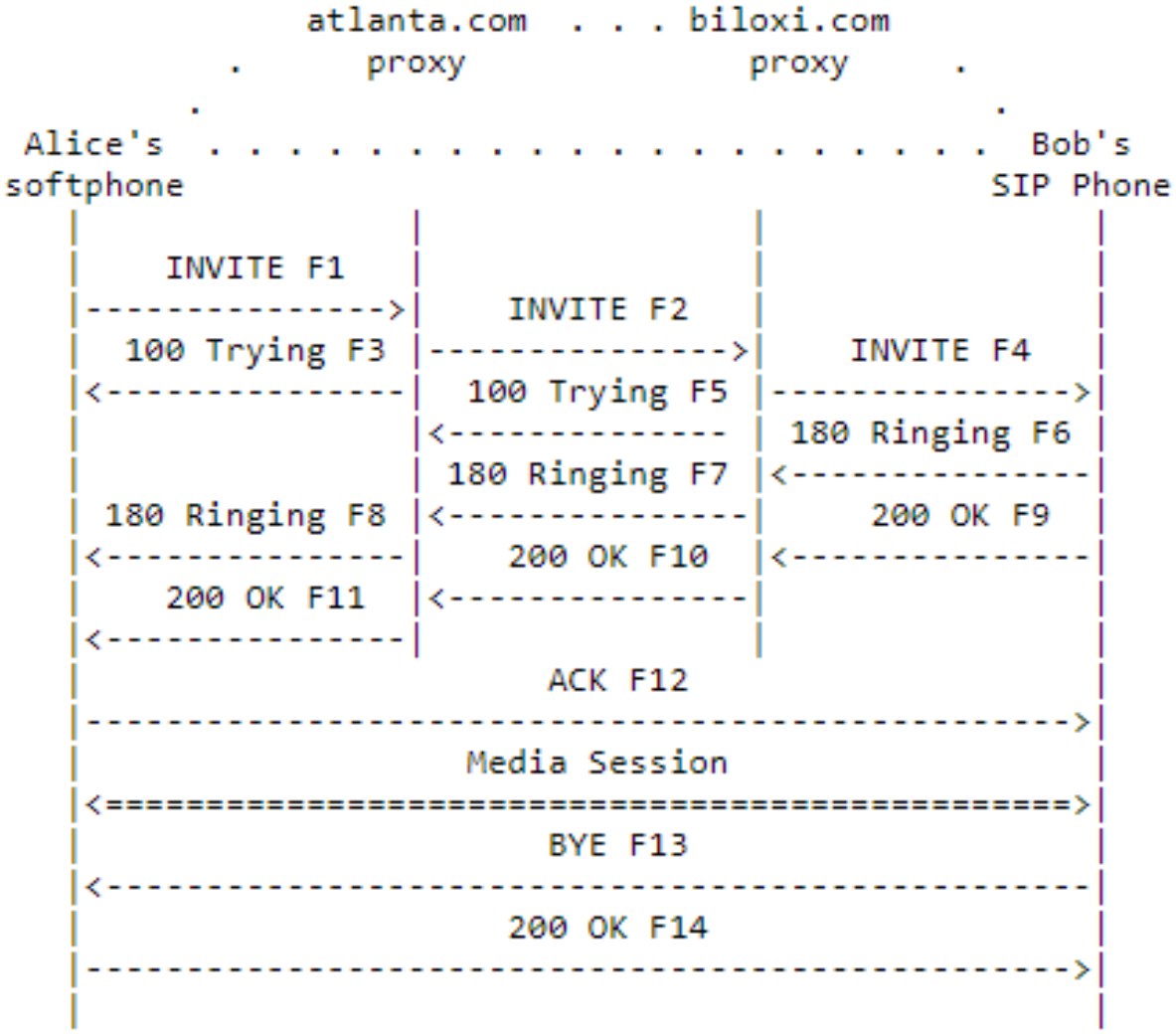
   ** With a network analyzer tool, like Wireshark
2. Play back the given traffic many times to generate load

- Pros:
  - Simple, Easy-to-read format
  - Flexible approach
- Cons:
  - Do not handle alternative behaviours

- Example tools: Apache JMeter, MTS-Ericsson, LoadNinja,
  simple short code that playbacks a WireShark trace

Load generation – behaviour descriptions

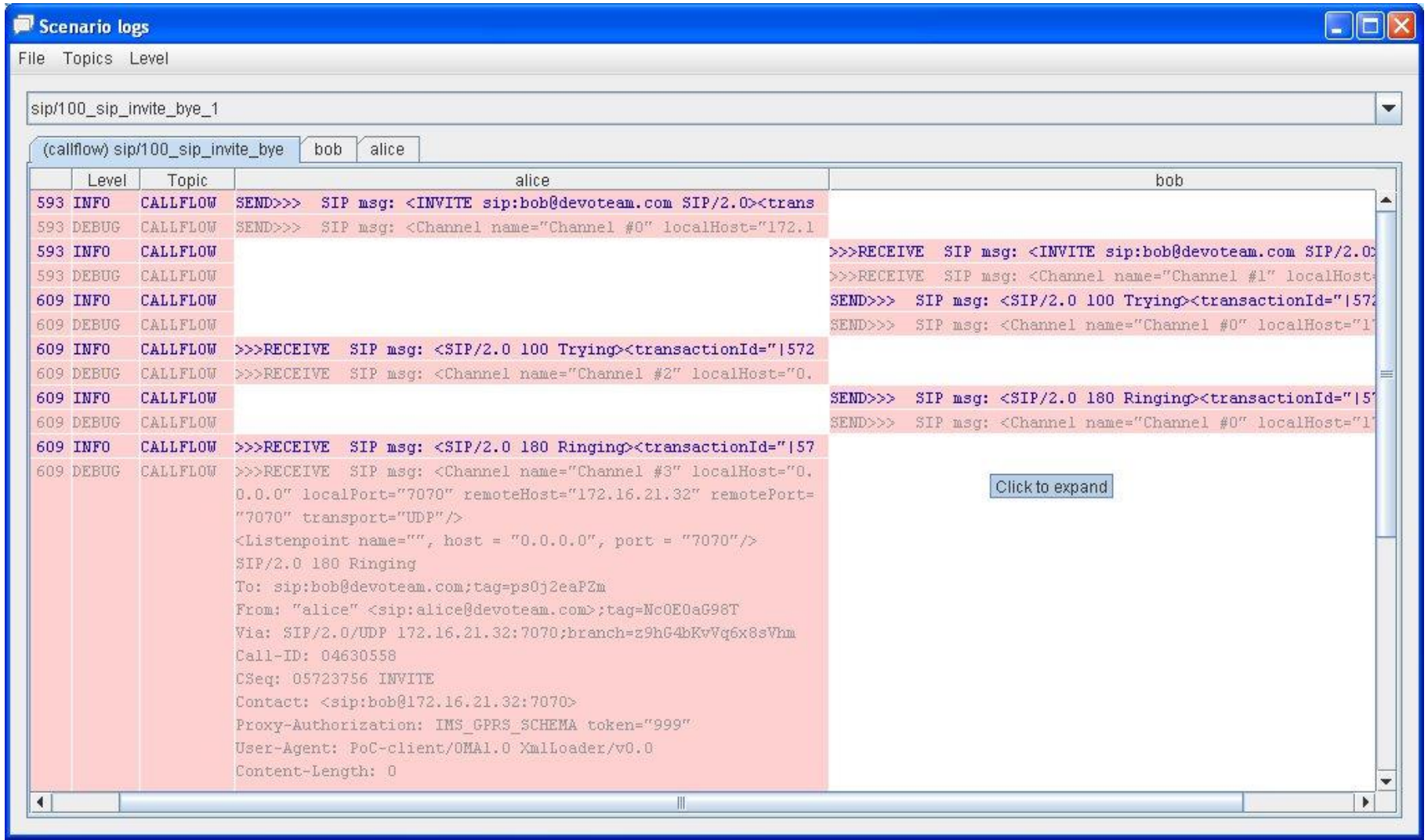
2. Call-flow (traffic playback tools)



Example:
SIP session setup example (RFC 3261/Sec 4.)

Load generation – behaviour descriptions

2. Call-flow (traffic playback tools)

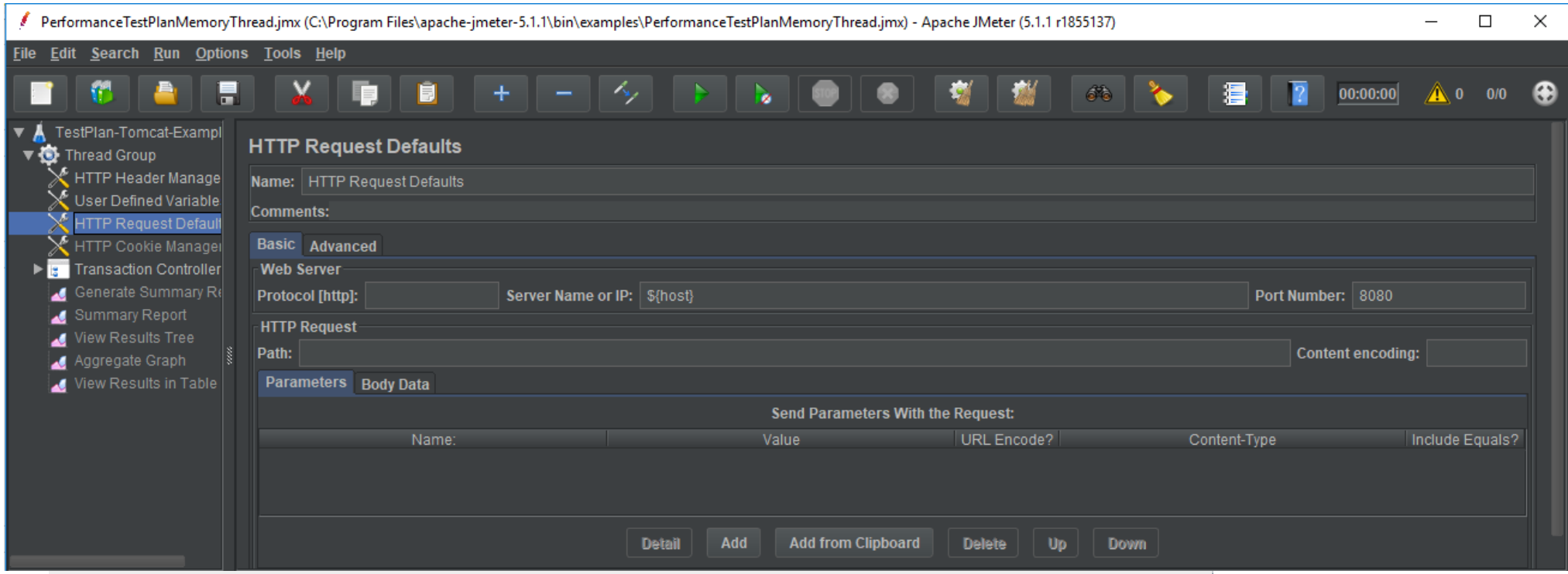


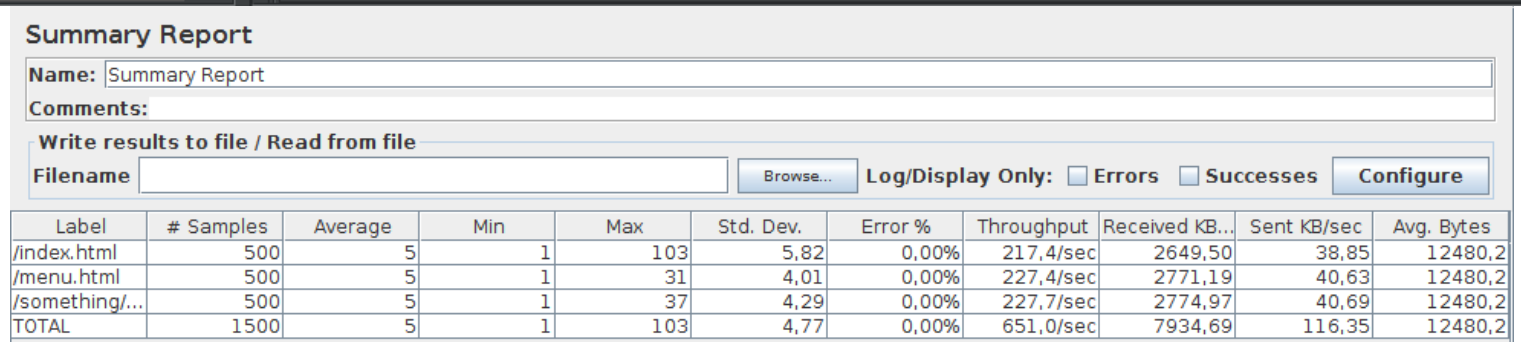
Screenshot of MTS-Ericsson

Load generation – behaviour descriptions

2. Call-flow (traffic playback tools)

JMeter demo





Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB...	Sent KB/sec	Avg. Bytes
/index.html	500	5	1	103	5,82	0,00%	217,4/sec	2649,50	38,85	12480,2
/menu.html	500	5	1	31	4,01	0,00%	227,4/sec	2771,19	40,63	12480,2
/something/...	500	5	1	37	4,29	0,00%	227,7/sec	2774,97	40,69	12480,2
TOTAL	1500	5	1	103	4,77	0,00%	651,0/sec	7934,69	116,35	12480,2

Load generation – behaviour descriptions

3. **Source code**
   - Describe input and internal conditions and appropriate actions for them in program code

- Pros:
  - Flexible approach
  - Can handle alternative behaviours
- Cons:
  - Requires programming skills both to develop and to read tests
  - Maintainability could be a problem
  - No abstract, high level view

Load generation – behaviour descriptions

4. Model-based descriptions

- Use formal models to describe the possible behaviour of the virtual users
- Beside the normal call-flow, alternate flows and exception flows are also considered

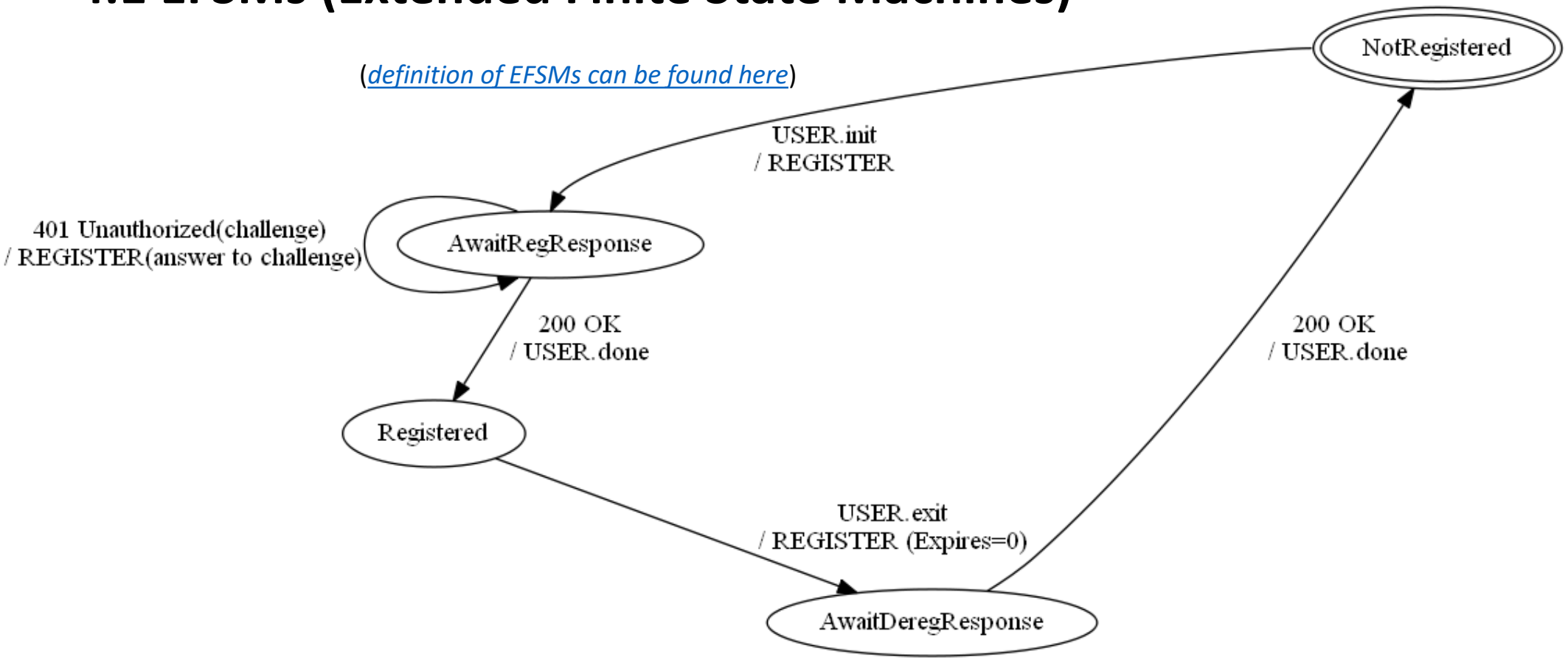
- Possible models:
  - EFSM (Extended Finite State Machine)
  - Petri Nets
  - Markov chains
  - PTA (Probabilistic Timed Automata)
  - ETA (Extended Timed Automata)
  - TCFMM (Timed Communicating Finite Multistate Machine)
  - …etc.

Load generation – behaviour descriptions

4. Model-based descriptions

4.1 EFSMs (Extended Finite State Machines)

(*definition of EFSMs can be found here*)



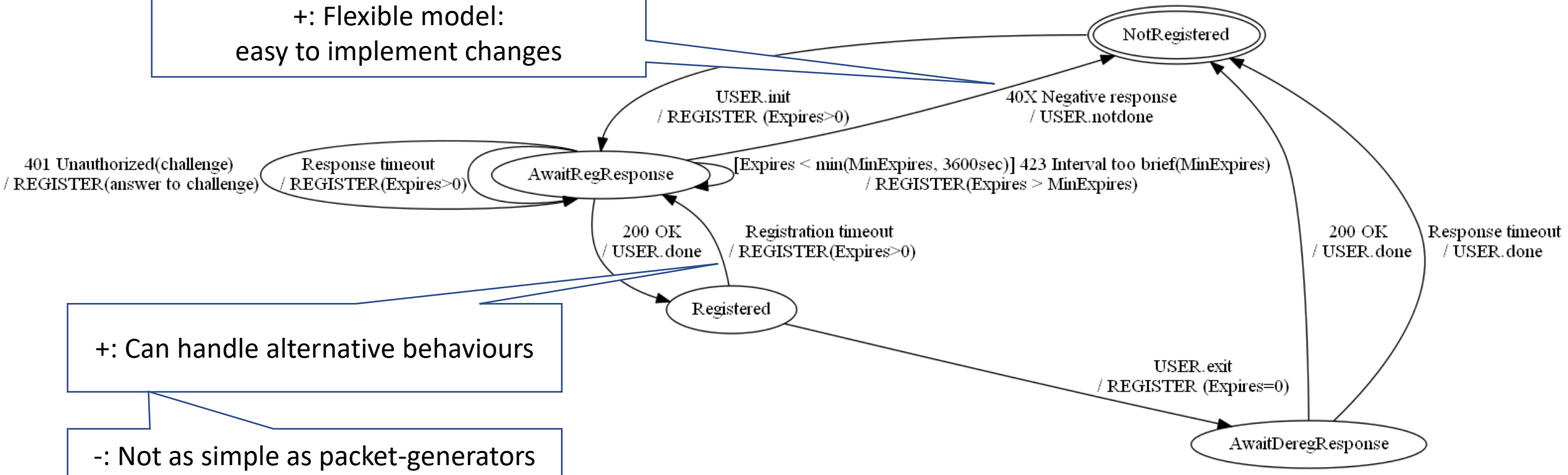
Example:
SIP (RFC 3261): Registration of UAC / default behaviour

Load generation – behaviour descriptions

4. Model-based descriptions

4.1 EFSMs (Extended Finite State Machines)

+: Flexible model:
easy to implement changes



+: Can handle alternative behaviours

-: Not as simple as packet-generators
or traffic playback tools

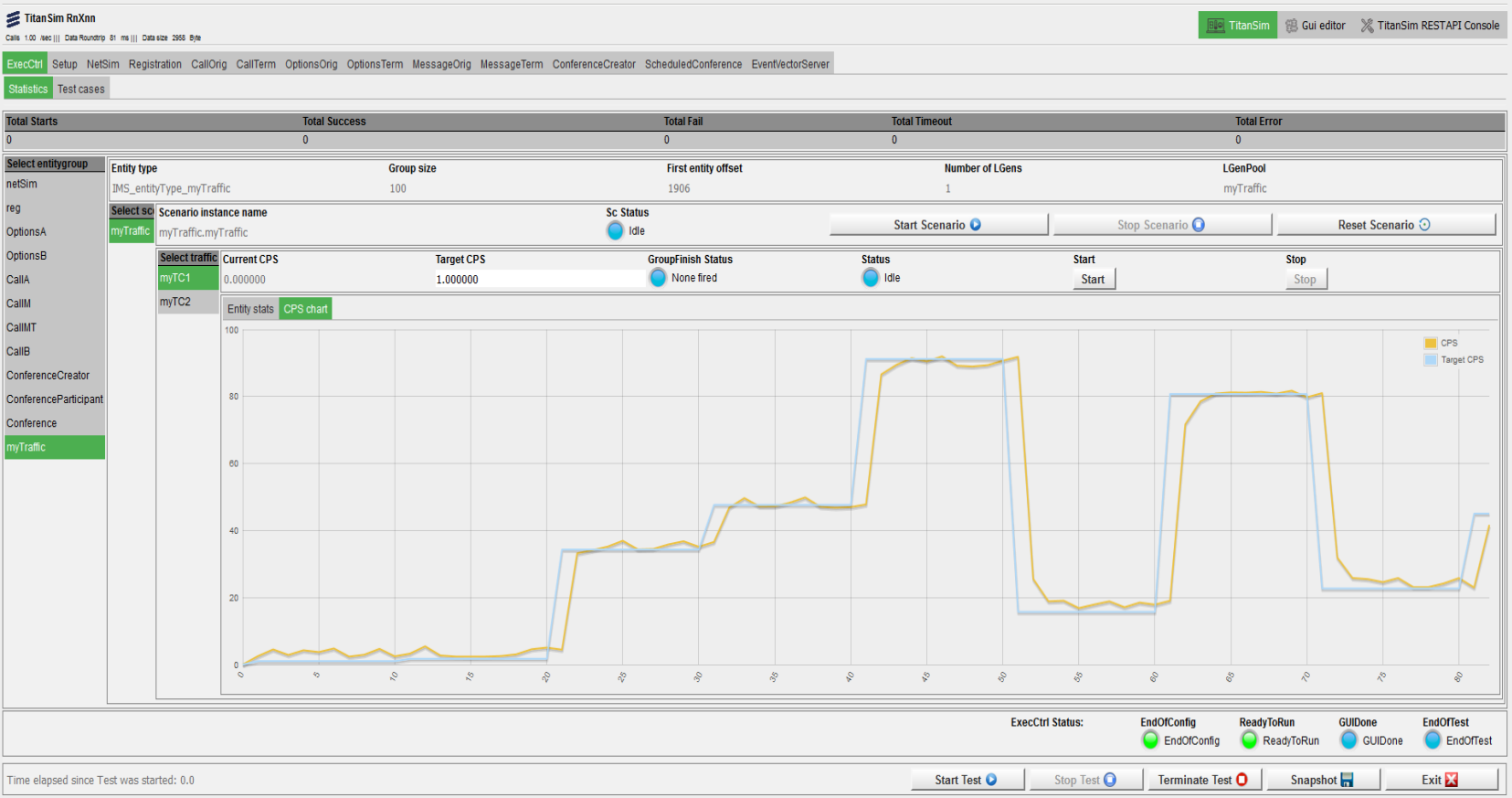
Example:
SIP (RFC 3261): Registration of UAC / with alternative behaviours

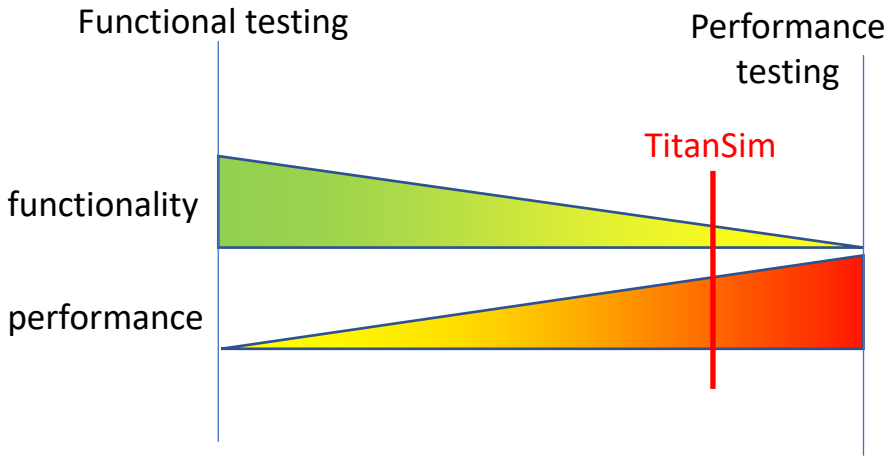
Load generation – behaviour descriptions

4.1 EFSMs (Extended Finite State Machines): An example tool: TitanSim:

- Not commercial tool, but used worldwide inside ERICSSON
- Used for **performance testing of network elements**

TitanSim demo ERICSSON





Source of info: An UCAAT 2016 poster, a HUSTEF 2016 presentation

Load generation – behaviour descriptions

Alternatives for behavior description	Simplicity	Complex behavior	Performance	Description can evolve with SUT
Hardwired (Packet generators)	✓	X	1	X
Call-flow (Traffic playback tools)	✓	X	2	~
Program code	X	✓	3	~
Model-based descriptions (EFSMs…)	~	✓	3	✓

Load generation – behaviour descriptions

Alternatives for behavior description	Simplicity	Complex behavior	Performance	Description can evolve with SUT
Hardwired (Packet generators)	✓	✗	1	
Call-flow (Traffic playback tools)	✓	✗		
Program code	✗	✓	3	
Model-based descriptions (EFSMs…)	~	✓	3	

Less and less performance, but able to test more and more functionality

Performance monitoring

- Performance testing or performance monitoring tools monitor and report the behaviour of the SUT under various input conditions, such as…
  - Number of concurrent users
  - Frequency and distribution of requests
  - The type of requests
  - Different behaviours of users

  …etc.

Performance monitoring – measured parameters

- These parameters are usually monitored during a performance test execution:
  I. Hardware utilization:
    - CPU Utilization
      - *Virtual users provide a load to the SUT, that should be processed by the CPU(s)*
      - *Scalability is important (32…etc. cores) → can the load be distributed evenly into different cores?*
      - *Possible parameters: time, when CPU executes non-idle threads; percentage of available CPU time*
    - Memory Utilization
      - *Traffic (mix) of virtual users and the database queries result in memory consumption*
    - Disk utilization
      - *Databases*
      - *Logs (depending on the test purpose (example: stability tests) & logging level, it can be even Terabytes!)*
      - *Possible parameters: amount/percentage of available storage; disk time; number of read/write requests/sec*
    - Network utilization
    - I/O devices utilization

Performance monitoring – measured parameters

- These parameters are usually monitored during a performance test execution:
  - II. Characteristics of the tested system:
    1. Response time
       - At different levels of protocol stack – *examples:*
         - *Telecommunication:*
           - *Neighbour participants vs. end-to-end participants*
           - *Type/level of message exchange*
         - *Web application: load of a page, the requested transaction is done*
         - *…etc*
       - Different measurable parameters: worst / best / average / 90% percentile
    2. Throughput rate (number of requests processed per unit time), number of handled requests*

       ** with different traffic mixs*

       - Network throughput: rate of successful data delivery over a communication channel
         - Maximum throughput/bandwidth capacity: maximum rate of data transfer via a given channel
       - System throughput: rate of data delivery to all terminal nodes
       - Different types of requests
       - Vary on time

Performance monitoring – measured parameters

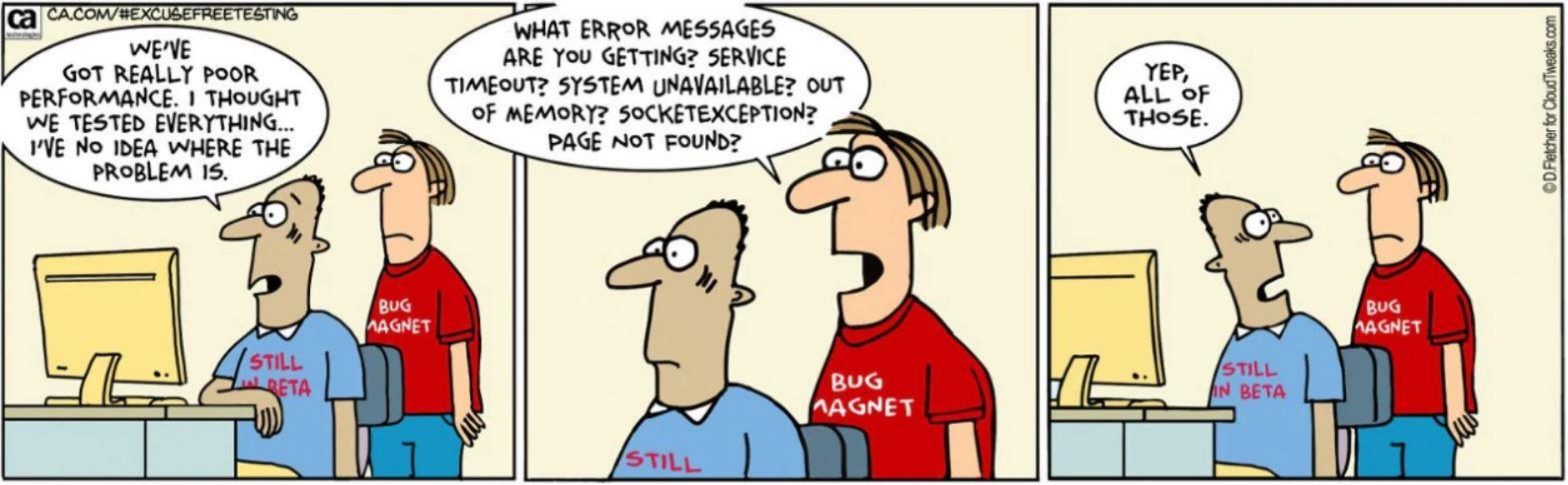
- These parameters are usually monitored during a performance test execution:
  - II. Characteristics of the tested system:
    3. Rate of successfully handled requests*
       - What is successfully handled?
         - The given request handled successfully at 1st/xth trying attempt or within a predefined time?
       - Didn't handled requests were lost, rejected for some reason or just delayed?

    ** with different traffic mixs*

    …etc.

See GoS (Grade of Service) in telecommunication domain!

Performance monitoring



Log analyses

- Collect the desired level of logs for further processing

- Number of tested transactions is high and on-the-fly conversion of statistics requires many resources that decreases load generation performance
  → Conversion is done after test execution or in a dedicated server

- High logging overhead
  → Proper level of logging should be selected
    - Capacity test of a real environment → minimum level
    - Investigate a root cause of a functional problem → more detailed
  → Logging should be switched on/off for each component independently

Log analyses

- Different approaches:
  1. Search for simple regular expressions and create statistics based on them
  2. Create your own solution for log conversions or data highlighting
  3. Use existing log manipulator applications
  4. Mixed solutions that use some of the approaches above

- These approaches scales between high performance with simple behaviour and medium performance with complex behaviour

Performance testing can be applied at difference test levels

- Performance tests at this levels are not common, only for performance critical components

- Examples:
  (1) RestAPI in microservices
  (2) API calls used by GUI
  (3) Fundamental queries of a database

+: One can focus on really performance critical components

-: It can be too sensitive, ie. it can show bottlenecks that are unimportant on higher levels → can not be used for gating in CI

- Resource requirement: low, one working component is enough

- Examples:

(1) Multiple database queries,
(2) Changing the OS or compiler

+:

- One can focus on performance critical components, bottlenecks
- It is a good gating test in CI

-: The tester needs to know before a higher level (system) test which parts he need to concentrate on

- Resource requirement: higher than on unit level, multiple working component are required

OS: Operating system *CI: Continuous Integration*

Performance testing can be applied at difference test levels

- Test the entire application in an “almost real” environment
- Performance tests at this levels are the most common
  - Known as "perf. test" for most of the testers ☺

- Examples:
  (1) Net-bank application with diff. transaction types
  (2) Test in a sample telecommunication network with preset traffic mix or traffic mix from "real" users

+: Tester sees the entire system → tests can be designed based on functionalities

-: The focus can be shifted from the real usage into a theoretical usage of the system

- Resource req.: high, all working components are required

- The performance test takes places on real environment
- Tests are derived from the requirement list of the customer → it describes the required levels of performance for given functionality mixes

- Examples:
  (1) Performance test in a real telecommunication network with real users
  (2) A distributed application used in real environment

+: Tests what really need (functionalities with proper weights)

-: Performance tests may be applied in a too later phase

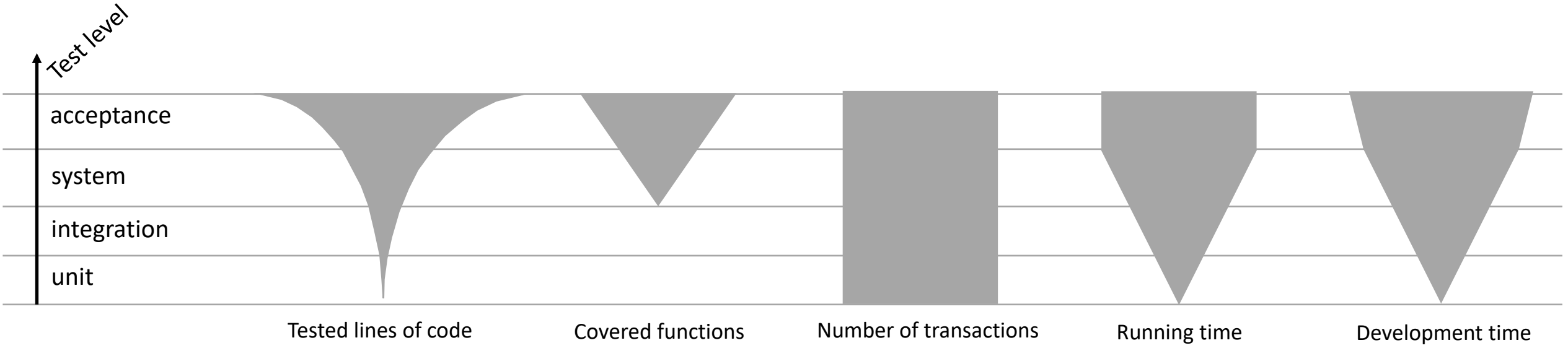
- Resource req.: enormous, special type of resources

Performance testing can be applied at difference test levels

Test level	Applicability, focus	CI gating	Resource Requirement
Unit	Focus on performance-critical components	not applicable	low, 1 working component
Integration	Focus on performance-critical sub-systems	good gating test	medium, multiple working components
System	Investigate capabilities and bottlenecks	long execution time need to be shortened	high, all working components
Acceptance	Check customer's requirements in a real environment	not applicable	enormous, special hardware (real environment or emulation)

Performance testing can be applied at difference test levels

- The proportions of tests at different levels depend highly on the metrics the test engineer is interested in:



A simple performance test at a given level measured by different metrics

Integrate performance testing with other testing attributes

- Performance testing is not a separate testing entity, it can be integrated into other testing attributes:
  - Functionality
    - *Example: besides working properly…*
      - *a webshop should handle given number of users simultaneously*
      - *a given node of telecom. network should handle given number of transactions*
    - A functional test can be run many times as performance test
      → the key point is the capability for parallel execution
  - Availability and usability
    - *Example: Net-bank application*
  - Security
    - *Example: DoS attacks*
      - *flood the system to make it inaccessible/bypass auth. method/do prohibited transactions*
  - …etc.

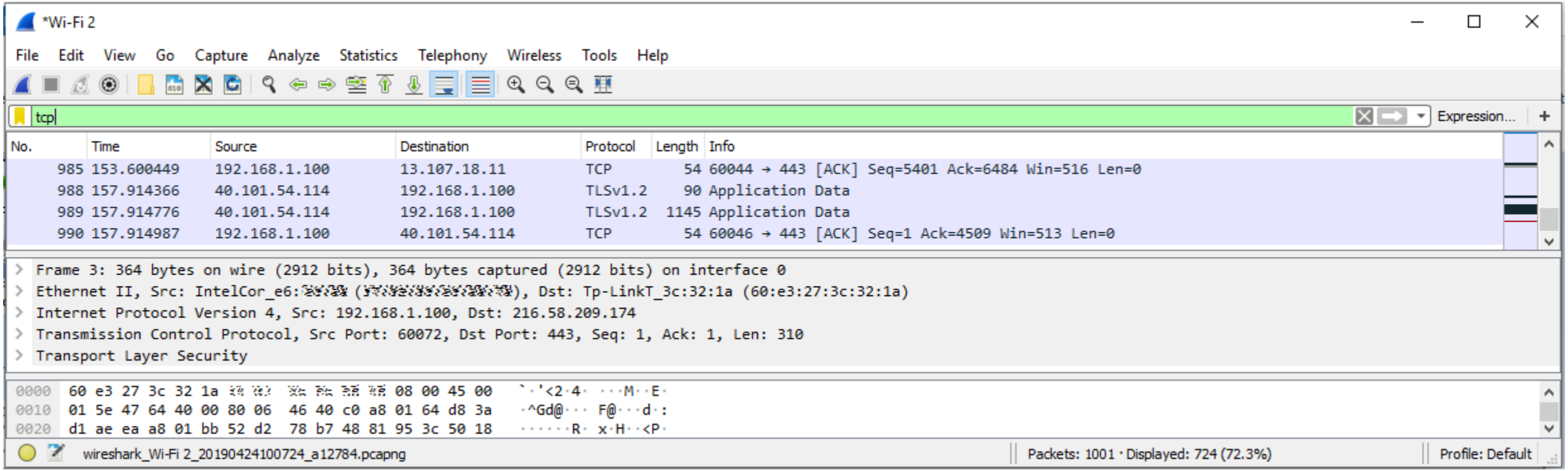
Monitoring tools – Wireshark demo

Wireshark demo

Wireshark packet analyzer tool

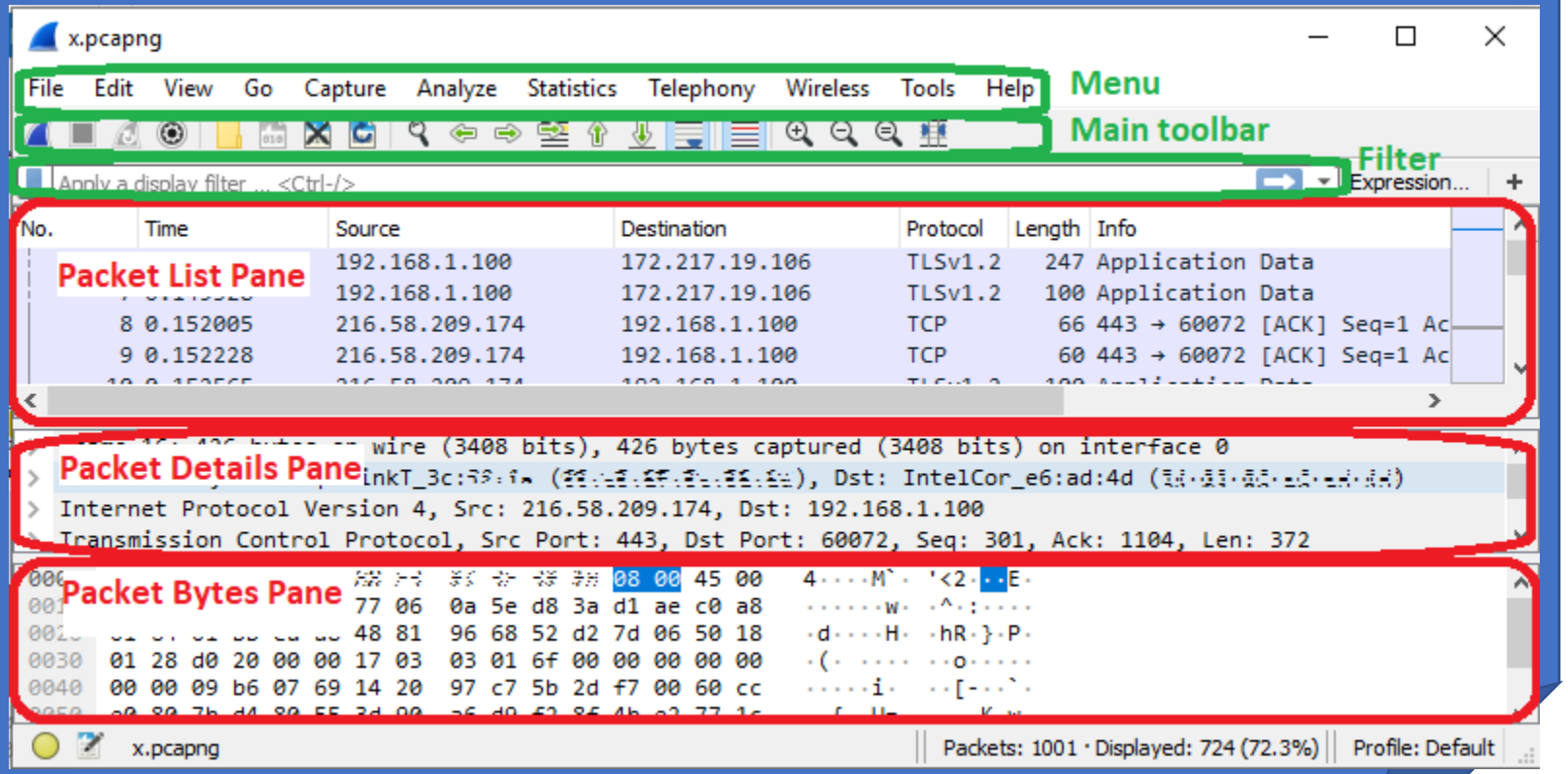
- Analyze logs or live traffic, capture traffic
- Use for network troubleshooting, communication protocol development
- Handle the encapsulation of different protocol layers, display fields, filtering

- Free and open source,
- Easy-to-use, has good documentation



Monitoring tools – Wireshark demo

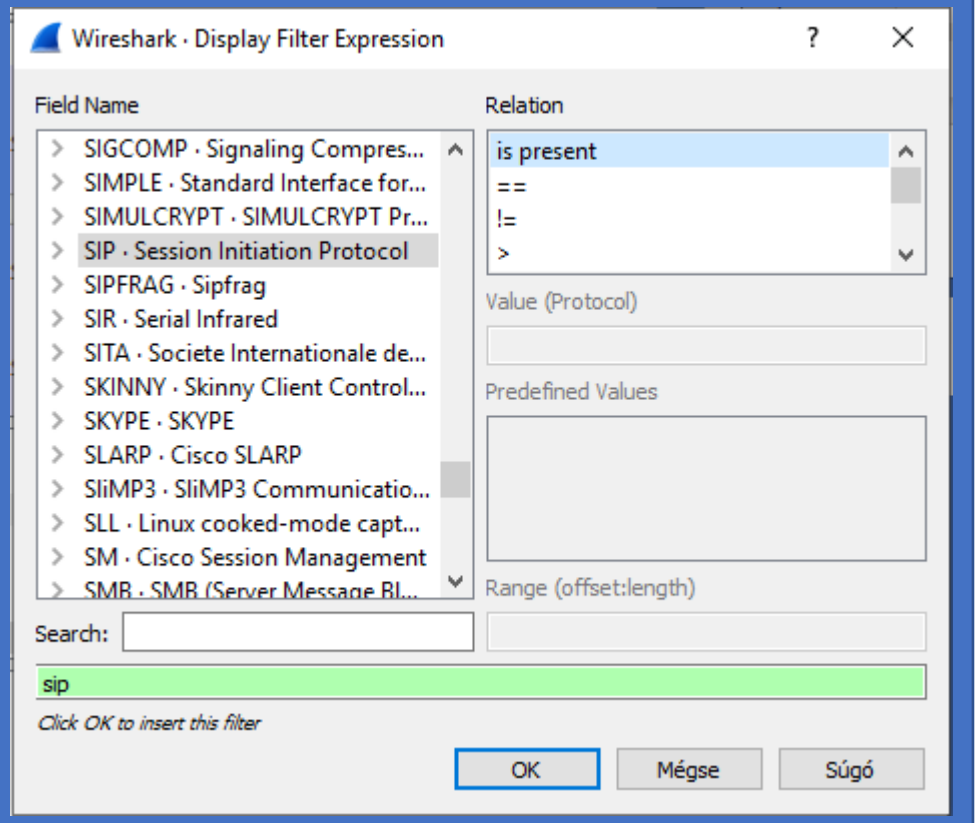
1. Download, install and start Wireshark
2. Select an active network connection (WiFi…etc.) *(You can change it later with Capture/Options)*
3. Start / stop capturing network traffic with the and buttons
4. Explore the 3 main views of network traffic:
   - Packet List pane:
     - Identify columns
     - Click on a given column
     - Double click on given row
   - Packet Details pane:
     - Fold/unfold given layers to explore fields
     - Double click on a given element and check the highlight on the Packet Bytes pane
   - Packet Bytes pane:
     - Double click on a given element and check highlight on the Packet Details pane



Monitoring tools – Wireshark demo

5. Check the Filter toolbar
   - Check preset filters by clicking on

   - Create your own filters



Examples:

```
ip.src==192.168.0.0/16 and ip.dst==192.168.0.0/16
```
Shows only local traffic (source AND destination)

```
ip.addr == 192.168.0.0/16
```
Shows only traffic with local source OR with local destination

```
ip.src != 192.168.1.100
```
Filters out the given source

```
tcp || udp
```
TCP or UDP transport layer protocol

```
tcp.port eq 25
```
Shows only SMTP (=TCP port 25) traffic

```
udp[8:3]==81:60:03
```
Filters UDP packets that contains 0x81, 0x60, 0x03 at the beginning of the UDP payload, skipping the 8-byte UDP header

Note that you can manage (save/load...etc) filters with the button

Check chapters 6.3. Filtering Packets While Viewing and 6.4. Building Display Fil-ter Expressionsof Wireshark's User's Guide and `wireshark-filter` of Manual Pages to get used to how the filter functionality works. The full list of available filters can be found in the Display Filter Reference

Performance testing tools – Apache JMeter demo

JMeter demo

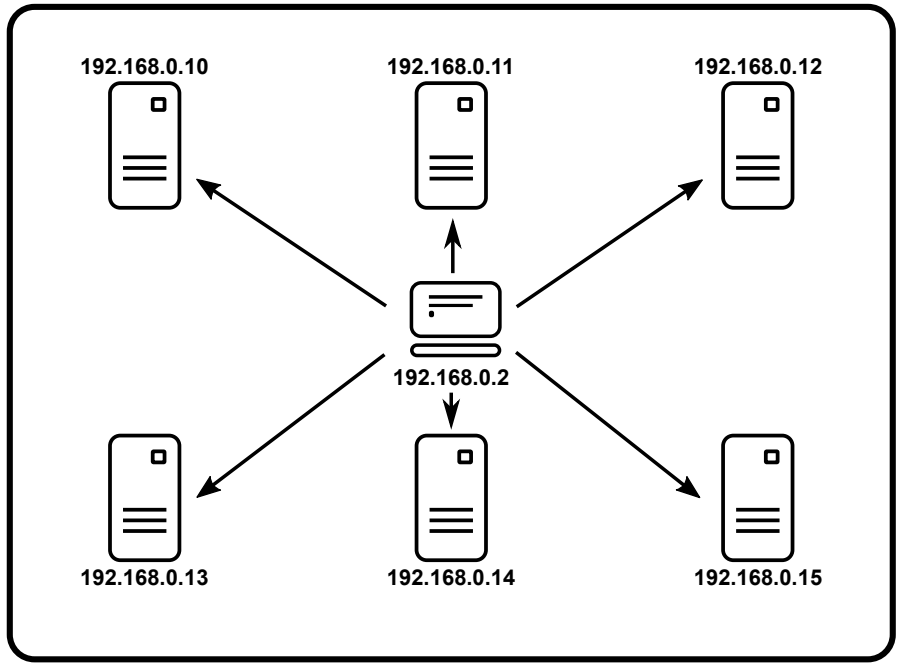
Apache JMeter:

- Simple performance test application
- Free, open source
- Easy to use, has good documentation
- Traffic playback tool: can record an actual traffic (from browser or native application) and play back as load
  - Traffic can be also generated from a manually edited call-flow
- Can handle the following applications/server/protocol types:
  - Web - HTTP, HTTPS (Java, NodeJS, PHP, ASP.NET, …)
  - SOAP / REST Webservices
  - FTP
  - Database via JDBC
  - LDAP
  - Message-oriented middleware (MOM) via JMS
  - Mail - SMTP(S), POP3(S) and IMAP(S)
  - Native commands or shell scripts
  - TCP
  - Java Objects
- Interfaces:
  - GUI: for recording traffic and for setting, debugging and learning purposes
  - CLI: for load test

CLI: Command Line Interface, GUI: Graphical User Interface

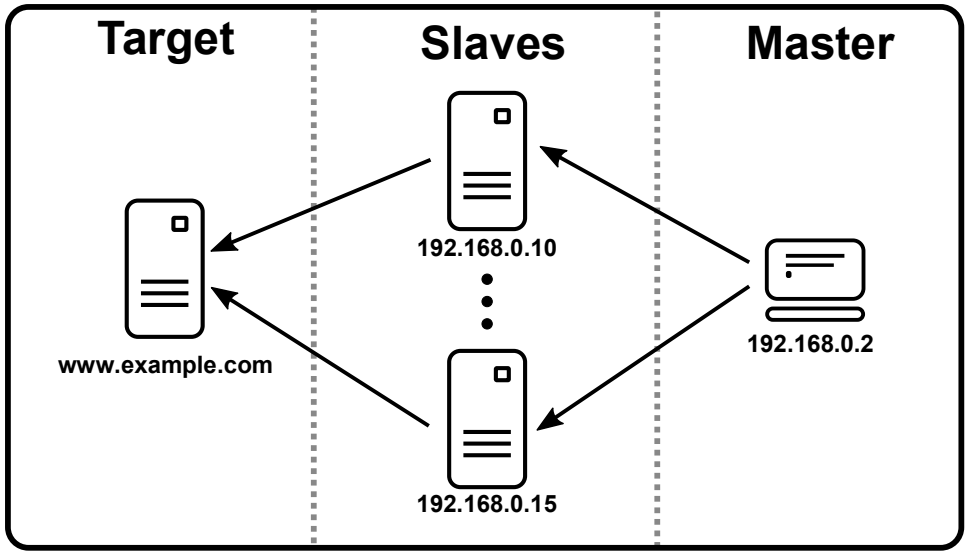
Performance testing tools – Apache JMeter demo

Multi-threading framework allows concurrent sampling by many threads and simultaneous sampling of different functions by separate thread groups.



One master computer control multiple slaves to generate load to the target

- **Master**: the system running JMeter GUI, which controls the test
- **Slave**: the system running JMeter-server, which takes commands from the GUI and send requests to the target system(s)
- **Target**: the webserver we plan to stress test



Performance testing tools – Apache JMeter demo

An overview:

I. Prerequisites: Install and start JMeter
   1. Download JMeter and unpack into a desired folder (where you have write privilages)
   2. Start JMeter (using `jmeter.bat` (Windows) / `jmeter` (Unix) file in `/bin` folder)

II. Record an HTTPs traffic and playback with 5 parallel threads (users)
   - Add reports and analyse measurement parameters

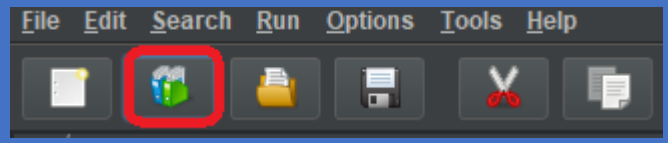
III. Create a Web Test Plan from scratch (edit call-flow manually)

IV. Discuss briefly additional possibilities of Apache JMeter

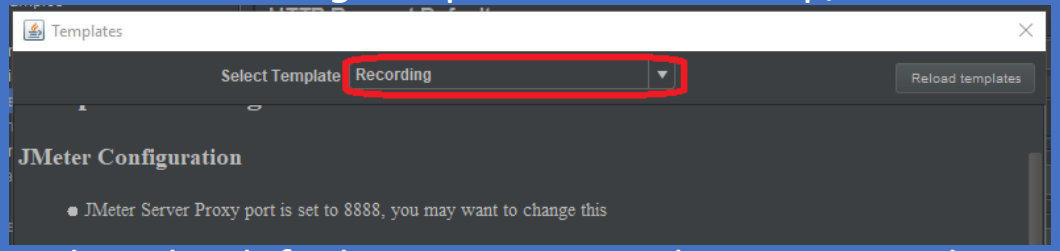
Performance testing tools – Apache JMeter demo

II. Record an HTTPs traffic and playback with 5 parallel threads (users)

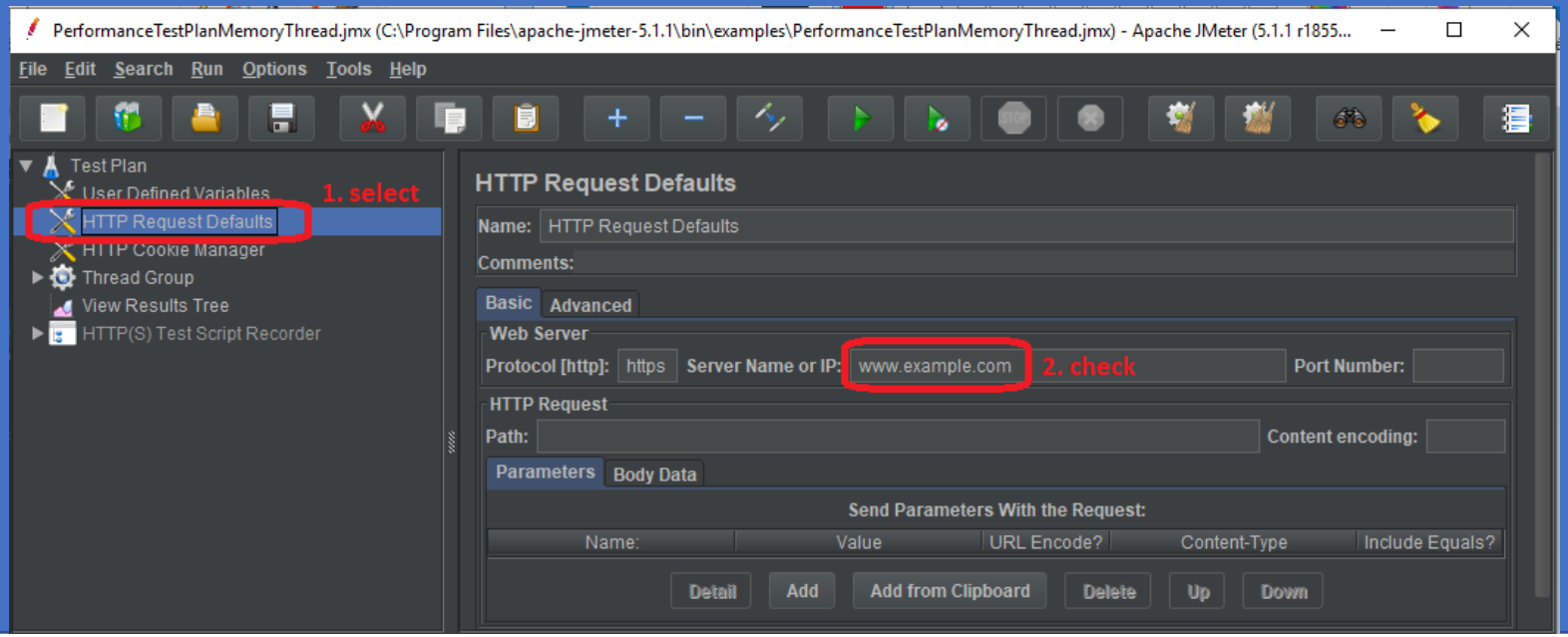
1. Record HTTP/HTTPS traffic using the recording template
   - Select *File / Templates…* menu item or the click on following icon:



   - Select *Recording* template from the drop/down list:

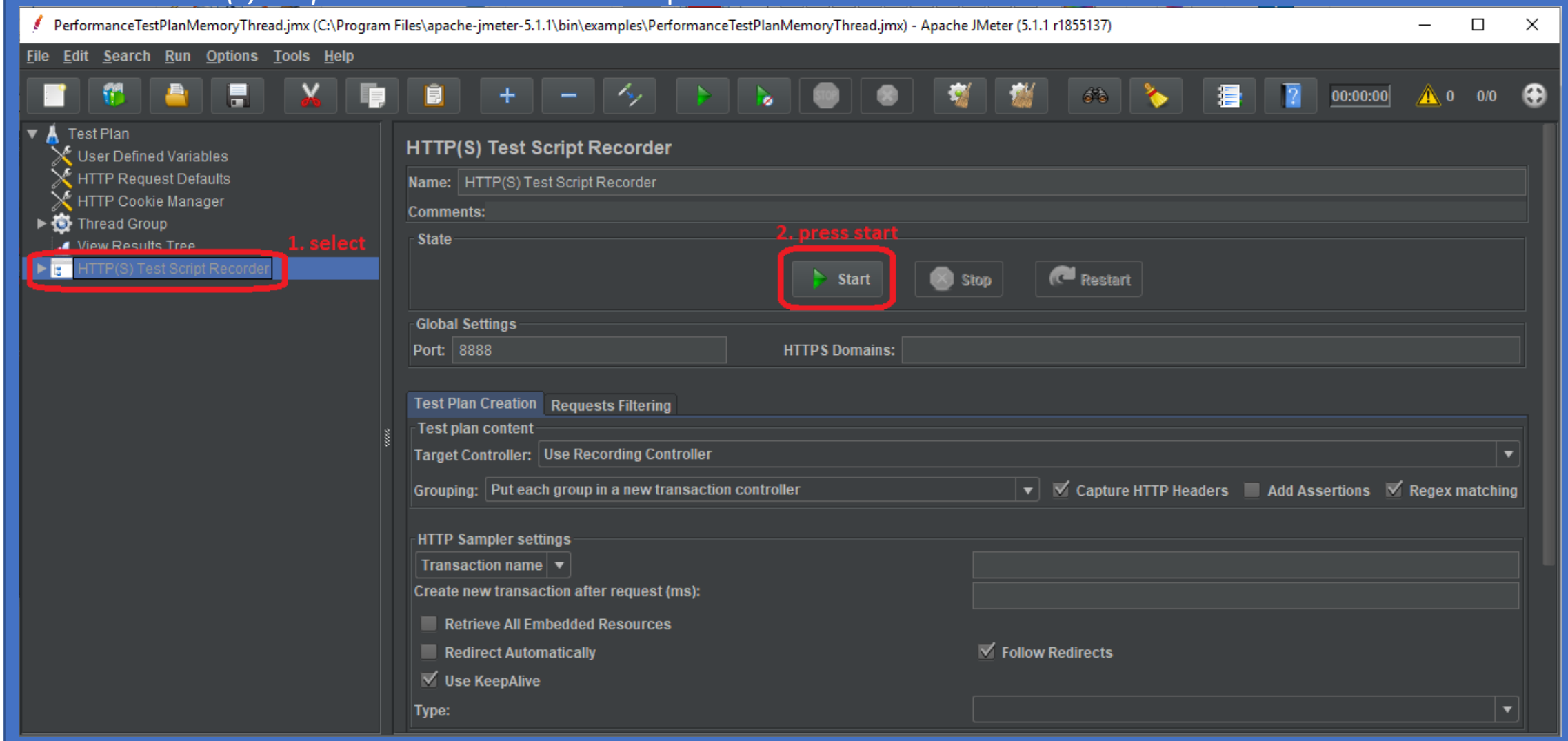


   - Select the default parameters and press *Create* button
2. Select *HTTP Request Defaults* element in the generated Test Plan



Performance testing tools – Apache JMeter demo

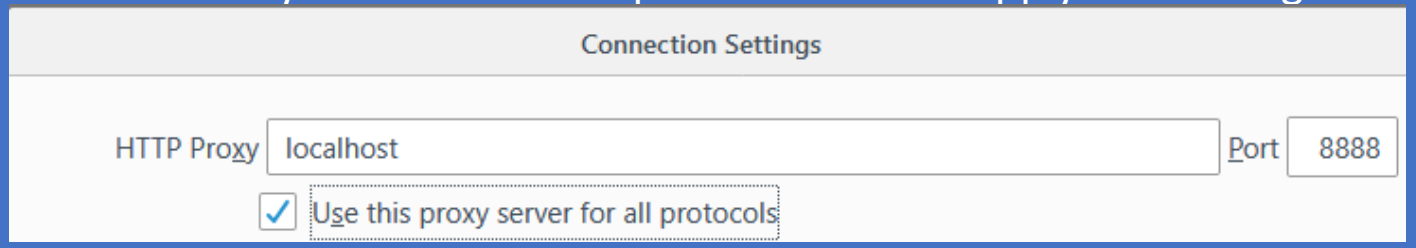
3. Select *HTTP(s) script recorder* at the left and press *Start* button



This will start the JMeter proxy server which is used to intercept the browser requests.

Performance testing tools – Apache JMeter demo

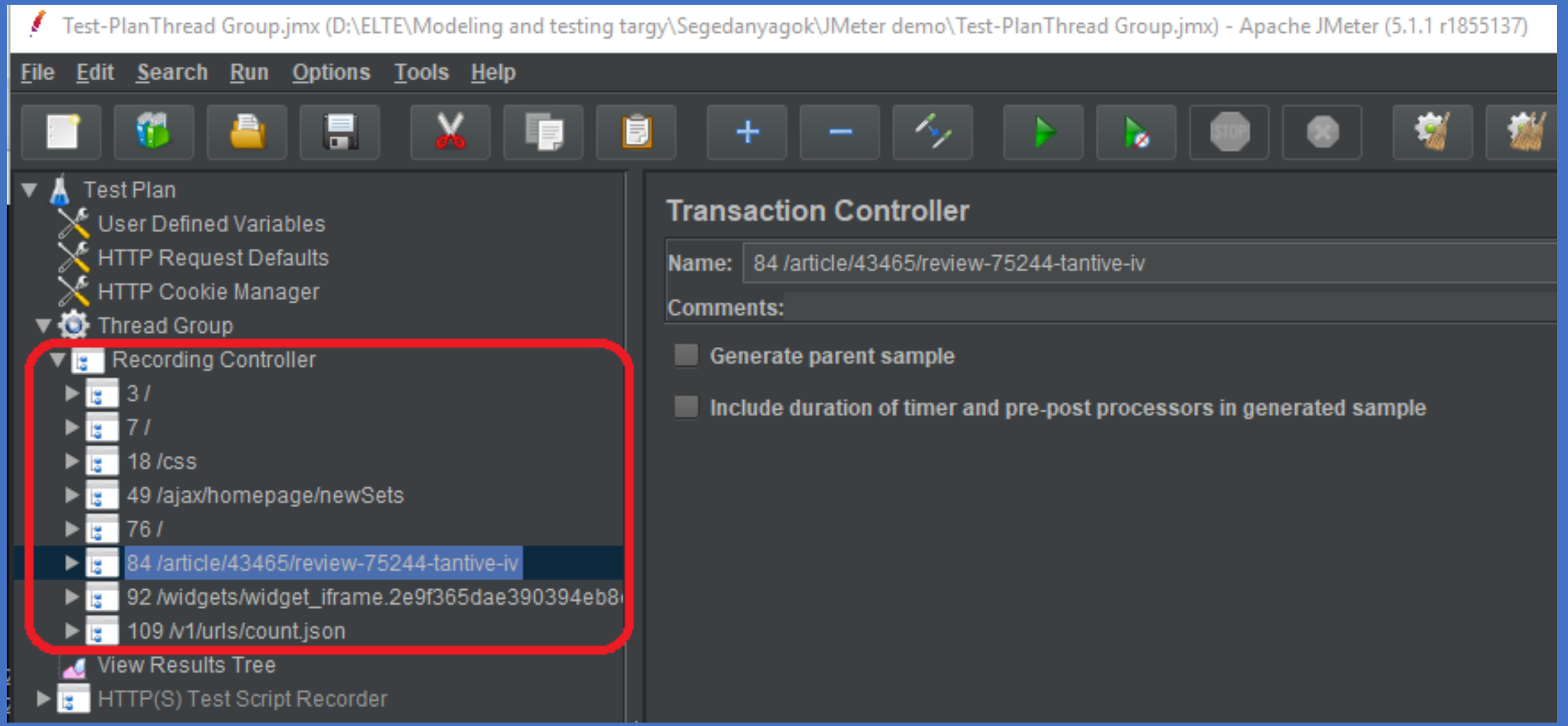
4. Check the *ApacheJMeterTemporaryRootCA.crt* certificate located in *bin* subfolder of Jmeter
   - If this file is not generated (… *java.io.FileNotFoundException: proxyserver.jks (Access is denied)* error msg) due access right problems, then set a folder in *user.properties* file (under *bin* subfolder of Jmeter) where you can write:
     `proxy.cert.directory=<destination folder>`
5. Install the given JMeter CA certificate to your browser
6. In your browser instead of automatically proxy setting, select manually proxy configuration.
   - Set HTTP Proxy to *localhost* and port to *8888* and apply this settings to all protocols



7. Visit a webpage with your browser (it will be very slow…) and click on some links (while JMeter is still running) then close your browser
8. Go back to JMeter, and press Stop *HTTP(s) script recorder*

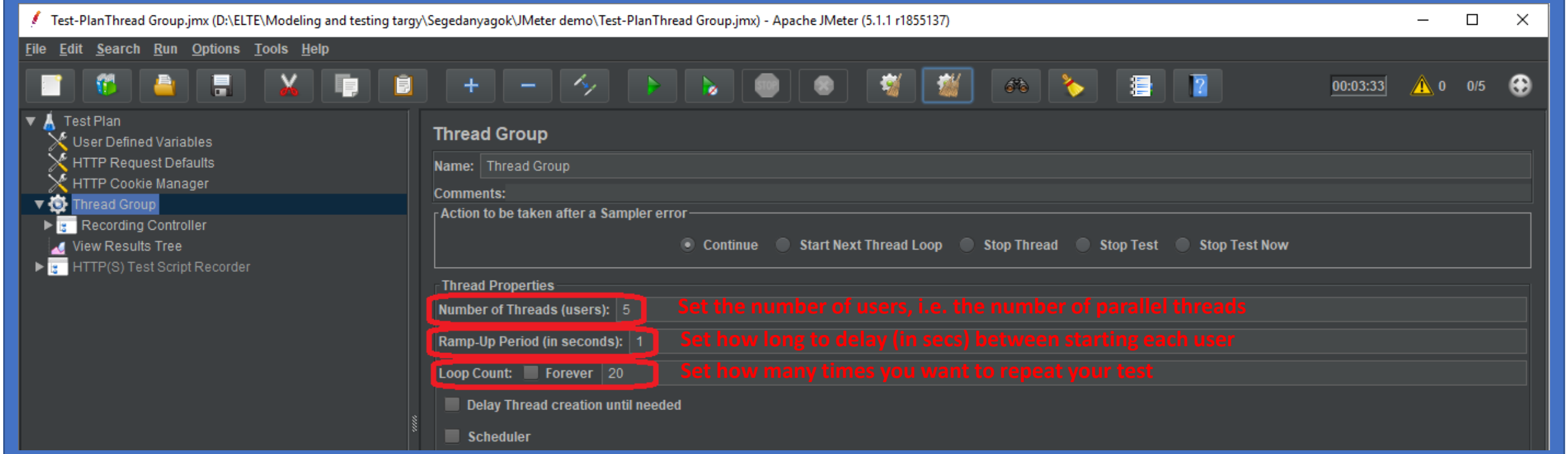
Performance testing tools – Apache JMeter demo

9. Expand the *Thread Group* at the left. There should be several samples, like this:



Performance testing tools – Apache JMeter demo

10. Select Thread Group and adjust some parameters:



11. Save the TestPlan and validate it with clicking on *Thread Group*, then right click and *Validate*

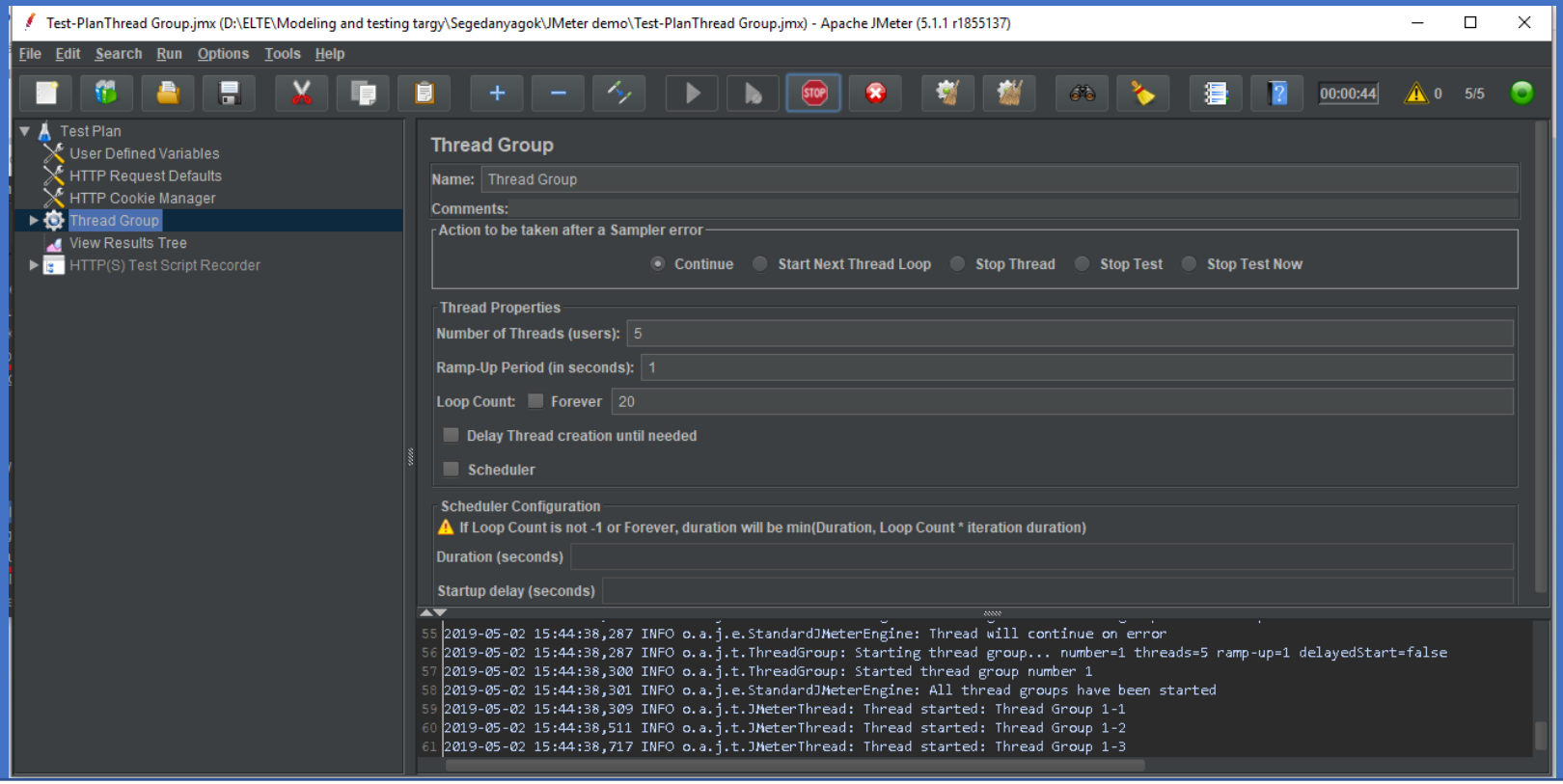
- Check the right-upper corner:
  - Click into ⚠ 0 to see logs:

```
45 2019-05-02 15:43:30,697 INFO o.a.j.t.JMeterThread: Stop Test Now detected by thread: Thread Group 1-1
46 2019-05-02 15:43:30,701 INFO o.a.j.t.JMeterThread: Thread finished: Thread Group 1-1
47 2019-05-02 15:43:30,711 INFO o.a.j.e.StandardJMeterEngine: Notifying test listeners of end of test
48 2019-05-02 15:43:30,708 INFO o.a.j.t.JMeterThread: Stopping: Thread Group 1-1
49 2019-05-02 15:43:30,712 INFO o.a.j.g.u.JMeterMenuBar: setRunning(false, *local*)
```

Performance testing tools – Apache JMeter demo

12. Start the given test with Run / Start menu item or with the button

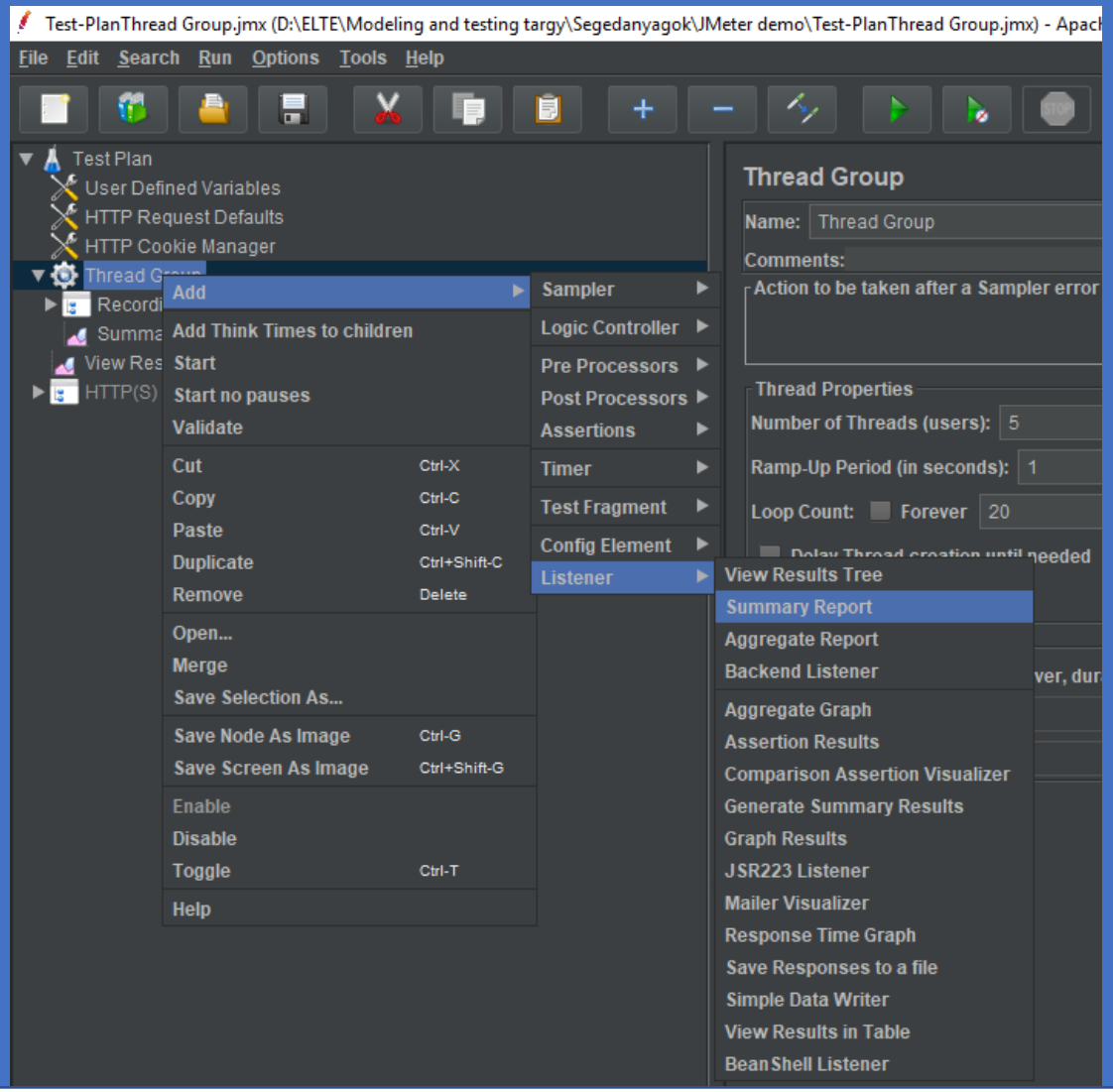
- At the right-upper corner the LED and the numbers show the status of the test:
  - Green LED shows that tests are running
  - The numbers denotes the active threads
    - At the end, the number of active threads begin to shrink
  - When the test has been finished the LED turns into grey
    - 0/5 denotes that there are no more active threads



Performance testing tools – Apache JMeter demo

Ok, good, something running, but we do not know any parameters of the target…

13. Add statistics *Summary Report* as a Listener…



Right click on *Thread Group*, then *Add/Listener/Summary Report*

Performance testing tools – Apache JMeter demo

14. Rerun test and investigate the record during and after execution!

Test-PlanThread Group.jmx (D:\ELTE\Modeling and testing targy\Segedanyagok\JMeter demo\Test-PlanThread Group.jmx) - Apache JMeter (5.1.1 r1855137)

File Edit Search Run Options Tools Help

00:01:23 0 5/5

- Test Plan
  - User Defined Variables
  - HTTP Request Defaults
  - HTTP Cookie Manager
  - Thread Group
    - Recording Controller
    - Summary Report
  - View Results Tree
  - HTTP(S) Test Script Recorder

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename Browse... Log/Display Only: Errors Successes Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/...	Sent KB/sec	Avg. Bytes
3 /	84	506	112	3596	461.32	0.00%	1.0/sec	25.75	1.50	25972.5
4 /v1/buckets/...	42	235	109	3177	463.91	0.00%	30.6/min	0.42	0.17	839.1
5 /v1/buckets/...	42	73	33	331	58.93	0.00%	30.7/min	10.84	0.20	21697.3
6 /v1/buckets/...	42	51	24	242	46.08	0.00%	30.7/min	0.63	0.18	1264.8
7 /	84	129	66	384	75.61	0.00%	1.0/sec	12.73	0.53	12767.0
18 /css	82	199	122	653	119.12	0.00%	1.0/sec	1.27	0.33	1298.0
49 /ajax/hom...	81	416	54	1711	410.14	0.00%	59.4/min	21.97	1.18	22728.9
58 /static/sh...	40	256	74	1194	234.62	0.00%	32.0/min	13.12	0.21	25198.0
56 /assets/js/...	40	86	53	331	54.98	0.00%	32.2/min	0.41	0.28	774.0
75 /s/notosa...	40	269	108	1496	283.11	0.00%	32.1/min	8.63	0.24	16511.0
76 /	79	6274	5749	8229	578.89	100.00%	58.6/min	2.44	0.15	2557.0
81 /live/boost...	39	198	58	814	176.50	0.00%	31.2/min	0.48	0.16	935.8
84 /article/43	76	371	112	1431	235.43	0.00%	1.0/sec	38.50	1.41	37707.2

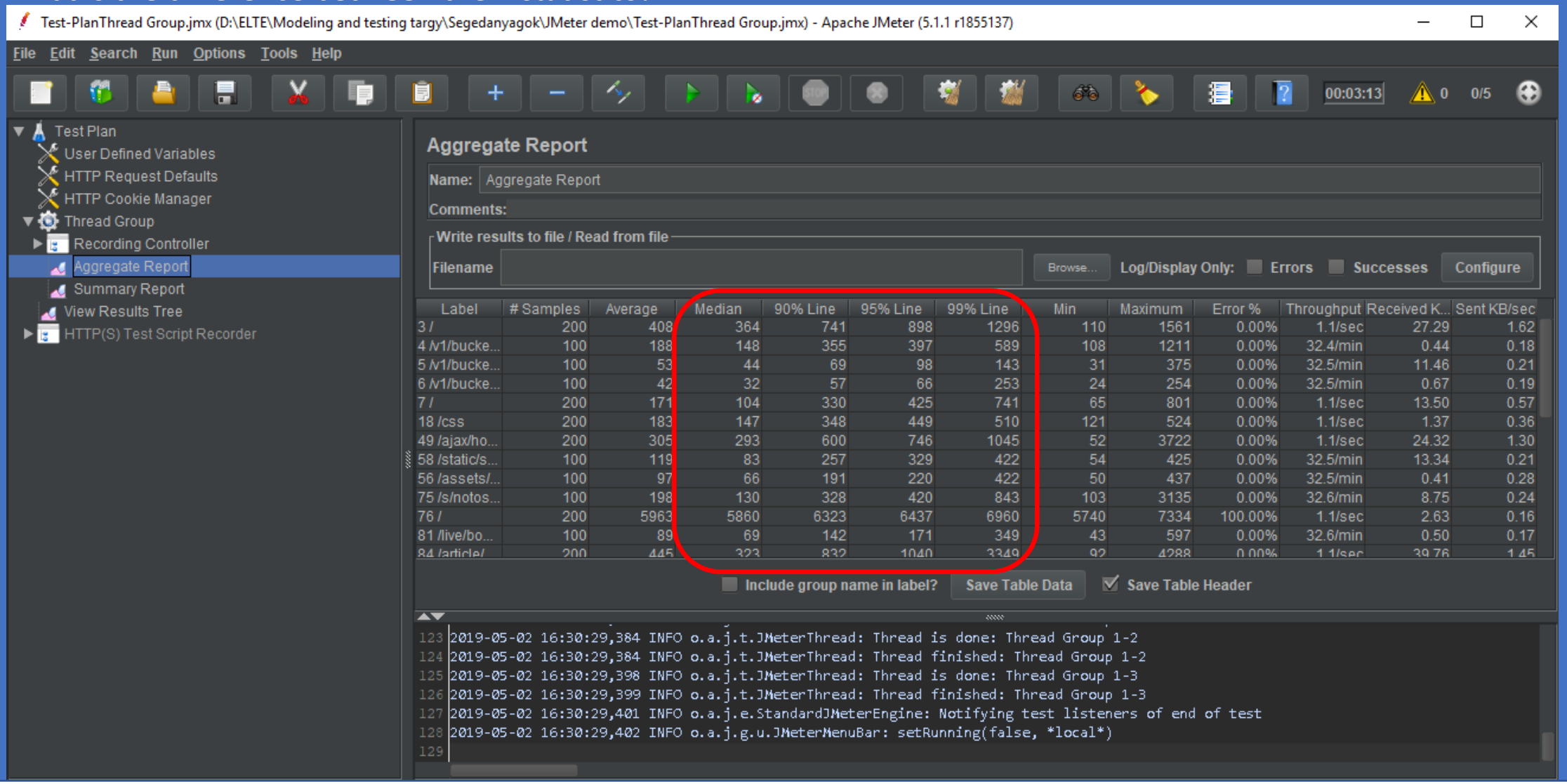
Include group name in label? Save Table Data Save Table Header

```
80 2019-05-02 16:20:25,793 INFO o.a.j.e.StandardJMeterEngine: Starting ThreadGroup: 1 : Thread Group
81 2019-05-02 16:20:25,793 INFO o.a.j.e.StandardJMeterEngine: Starting 5 threads for group Thread Group.
82 2019-05-02 16:20:25,793 INFO o.a.j.e.StandardJMeterEngine: Thread will continue on error
83 2019-05-02 16:20:25,793 INFO o.a.j.t.ThreadGroup: Starting thread group... number=1 threads=5 ramp-up=1 delayedStart=false
84 2019-05-02 16:20:25,804 INFO o.a.j.t.ThreadGroup: Started thread group number 1
85 2019-05-02 16:20:25,804 INFO o.a.j.e.StandardJMeterEngine: All thread groups have been started
86 2019-05-02 16:20:25,805 INFO o.a.j.t.JMeterThread: Thread started: Thread Group 1-1
```

Performance testing tools – Apache JMeter demo

15. Also try *Aggregate Report*!

- What is the difference between the 2 statistics?



Performance testing tools – Apache JMeter demo

Understand the parameters of reports:

- **Label:** name of the sampler
- **#Samples:** Total number of requests sent to the server during the duration of the test
- **Average:** Average is sum of all the sample time divided by total number of requests, also called as Mean (unit: msec)
- **Median:** 50th percentile / 50% line: 50% of the requests were taking less than or equal to this time (unit: msec)
- **90/95/99% line:** 90/95/99% of the requests were taking less than or equal to this time (unit: msec)
- **Min/Max:** Min/Max sample time (unit: msec)
- **Std. Dev.:** Standard Deviation: a measure that is used to quantify the amount of variation or dispersion of a set of data values.
  - A low standard deviation indicates that the data points tend to be close to the mean (also called the expected value) of the set, while a high standard deviation indicates that the data points are spread out over a wider range of values.

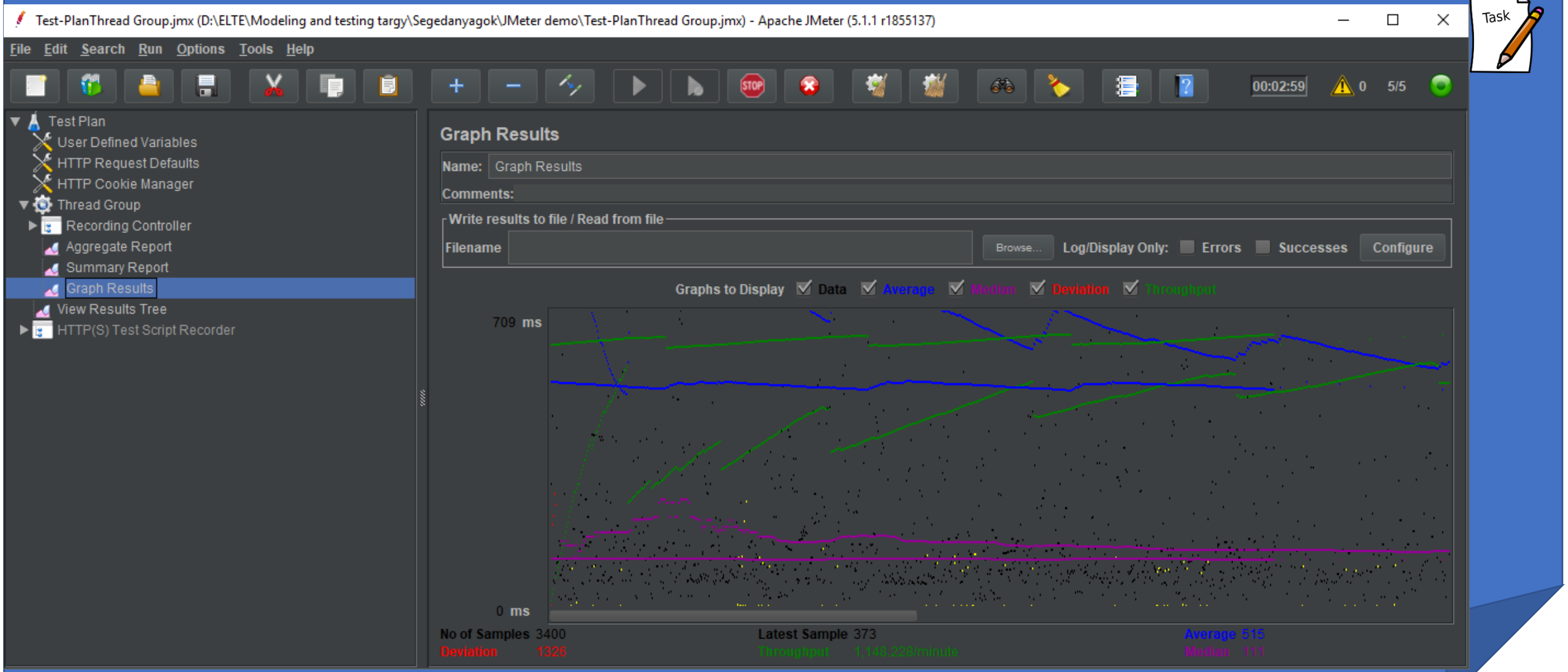
  (description from wikipedia)
- **Error %:** Percentage of failed tests
- **Throughput:** Number of requests processed per unit time. Used to identify the capacity of the server (but the limitation in the capacity can be also come from the limited capacity of the load generator (i.e. the clients used for testing), as in previous example)
- **Received/Sent KB/sec:** self explanatory
- **Avg. Bytes:** Average response size

Note that we used GUI for debugging/educational reasons, but the actual load testing should be done via CLI

CLI: Command Line Interface, GUI: Graphical User Interface

Performance testing tools – Apache JMeter demo

16. Try *Graph Results*!



Performance testing tools – Apache JMeter demo

III./b. The login scenario of a webpage can be tested using the similar approach:

1. Record your login flow with the HTTP Recording script introduced previously

2. Identify the following 2 elements:
   - GET request: gets the login form.
   - POST request: posts the login credentials

3. The response from the GET request will contain tokens which need to be extracted with Regular Expression Extractor
   - See JMeter's Regular Expressions description or BlazeMeter's Regular Expression Extractor description for further details

4. Add the above tokens as parameters in the POST request

Performance testing tools – Apache JMeter demo

CLI: Command Line Interface

17. Run the load test again in CLI mode

- `jmeter -n -t name_of_testplan.jmx -l name_of_log.jtl`
- `jmeter -n -t name_of_testplan.jmx -l name_of_log.jtl` -H my.proxy.server -P 8000

```
D:\ELTE\apache-jmeter-5.1.1\bin>jmeter -n -t MTU2.jmx -l log2.jtl
May 06, 2019 2:14:57 PM java.util.prefs.WindowsPreferences <init>
WARNING: Could not open/create prefs root node Software\JavaSoft\Prefs at root 0x80000002. Windows RegCreateKeyEx(...) returned error code 5.
Creating summariser <summary>
Created the tree successfully using MTU2.jmx
Starting the test @ Mon May 06 14:15:07 CEST 2019 (1557144907447)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary +    341 in 00:00:26 =   13.3/s Avg:   304 Min:     1 Max:  5821 Err:    21 (6.16%) Active: 5 Started: 5 Finished: 0
summary +    485 in 00:00:26 =   18.4/s Avg:   313 Min:     0 Max:  5841 Err:    34 (7.01%) Active: 5 Started: 5 Finished: 0
summary =    826 in 00:00:52 =   15.9/s Avg:   309 Min:     0 Max:  5841 Err:    55 (6.66%)
summary +    530 in 00:00:31 =   16.9/s Avg:   260 Min:     0 Max:  5806 Err:    36 (6.79%) Active: 5 Started: 5 Finished: 0
summary =   1356 in 00:01:23 =   16.3/s Avg:   290 Min:     0 Max:  5841 Err:    91 (6.71%)
summary +    564 in 00:00:29 =   19.7/s Avg:   284 Min:     0 Max:  5856 Err:    39 (6.91%) Active: 5 Started: 5 Finished: 0
summary =   1920 in 00:01:52 =   17.1/s Avg:   288 Min:     0 Max:  5856 Err:   130 (6.77%)
summary +    461 in 00:00:30 =   15.2/s Avg:   298 Min:     0 Max:  6144 Err:    32 (6.94%) Active: 5 Started: 5 Finished: 0
summary =   2381 in 00:02:22 =   16.7/s Avg:   290 Min:     0 Max:  6144 Err:   162 (6.80%)
summary +    519 in 00:00:28 =   18.4/s Avg:   286 Min:     0 Max:  5812 Err:    38 (7.32%) Active: 0 Started: 5 Finished: 5
summary =   2900 in 00:02:50 =   17.0/s Avg:   289 Min:     0 Max:  6144 Err:   200 (6.90%)
Tidying up ...    @ Mon May 06 14:17:58 CEST 2019 (1557145078504)
... end of run
D:\ELTE\apache-jmeter-5.1.1\bin>
```

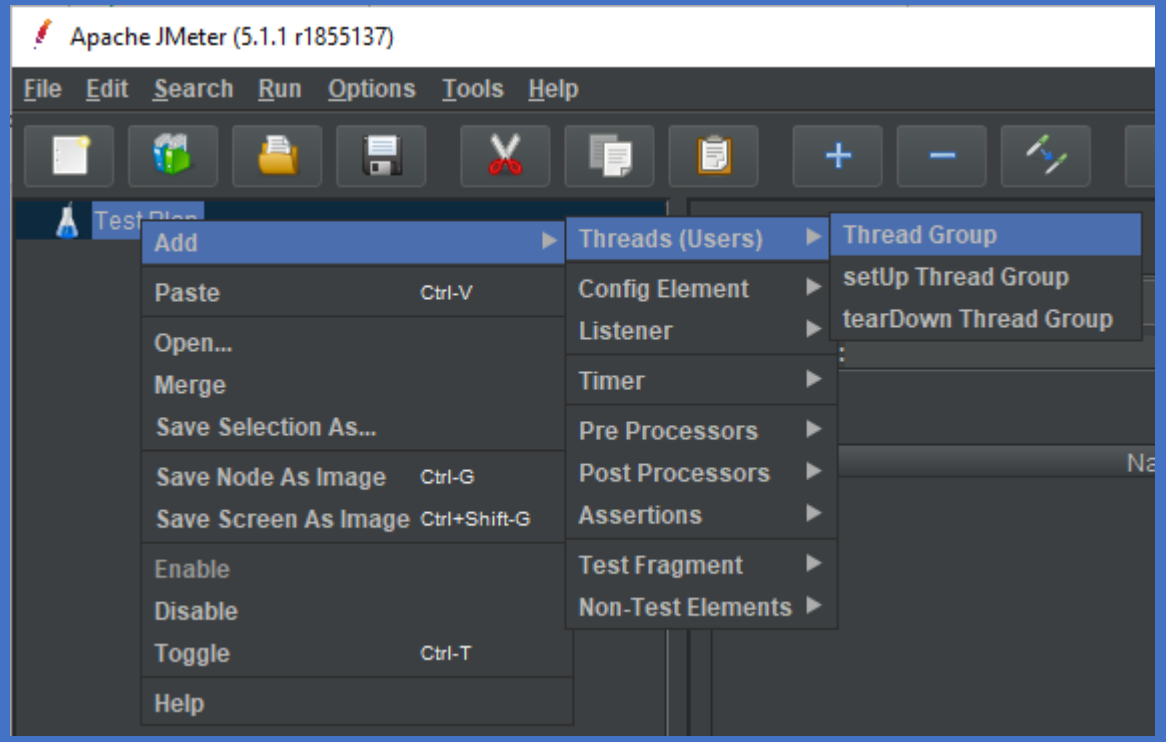
```
timeStamp,elapsed,label,responseCode,responseMessage,threadName,dataType,success,failureMessage,bytes,sentBytes,grpThreads,allThreads,URL,Latency,IdleTime,Connect
1557144908875,1047,3 /,200,OK,Thread Group 1-5,text,true,,14707,644,5,5,https://brickset.com/,10,0,2
1557144908875,10,3 /-0,301,Moved Permanently,Thread Group 1-5,,true,,303,322,5,5,http://brickset.com/,10,0,2
1557144908886,1036,3 /-1,200,OK,Thread Group 1-5,text,true,,14404,322,5,5,https://brickset.com/,1026,0,910
1557144908500,1412,3 /,200,OK,Thread Group 1-3,text,true,,14706,644,5,5,https://brickset.com/,11,0,3
1557144908500,11,3 /-0,301,Moved Permanently,Thread Group 1-3,,true,,303,322,5,5,http://brickset.com/,11,0,3
1557144908512,1400,3 /-1,200,OK,Thread Group 1-3,text,true,,14403,322,5,5,https://brickset.com/,1381,0,1283
1557144908204,1722,3 /,200,OK,Thread Group 1-1,text,true,,14769,644,5,5,https://brickset.com/,60,0,40
1557144908204,60,3 /-0,301,Moved Permanently,Thread Group 1-1,,true,,303,322,5,5,http://brickset.com/,60,0,40
1557144908266,1660,3 /-1,200,OK,Thread Group 1-1,text,true,,14466,322,5,5,https://brickset.com/,1615,0,1528
```

JMeter logfile

Performance testing tools – Apache JMeter demo

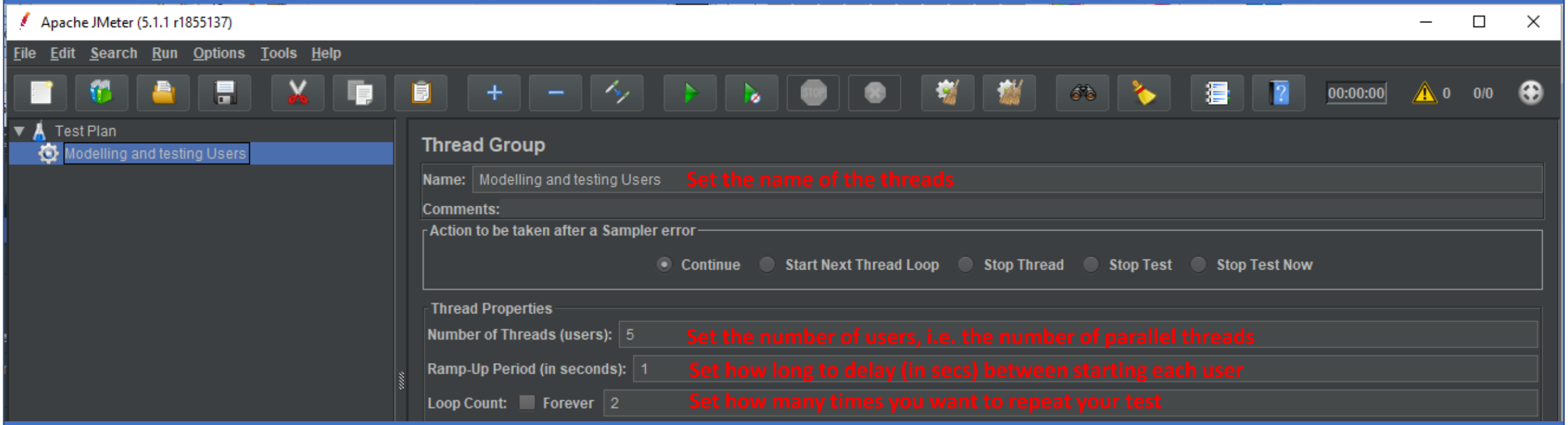
III. Create a Web test plan from scratch! (i.e. edit call-flow manually)

1. Select *File / New* menu option
2. Adding users with the *Thread Group* element:
   - Right click on *TestPlan*, then *Add/Threads (Users)/Thread Group*



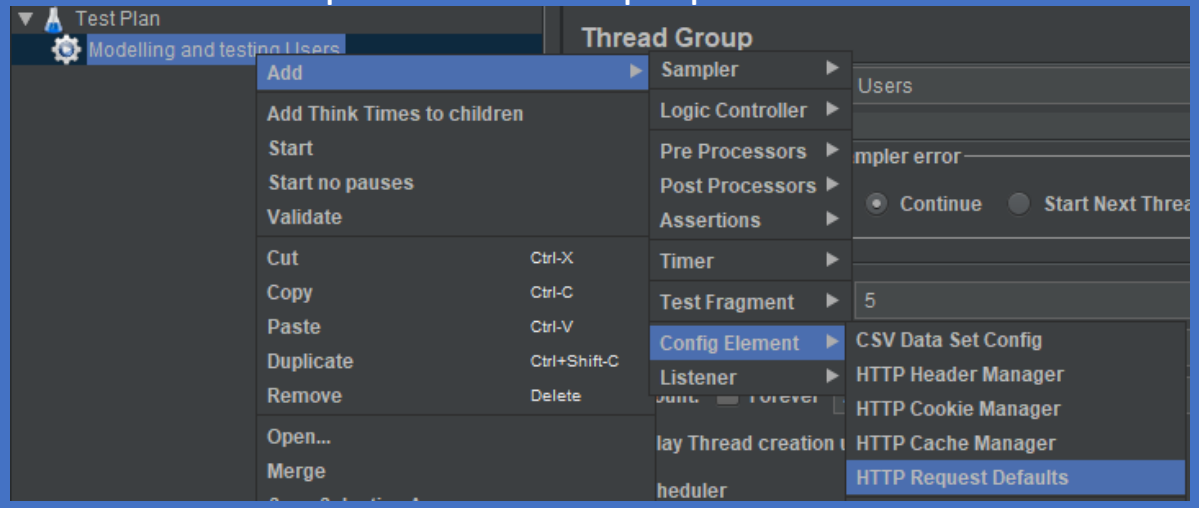
Performance testing tools – Apache JMeter demo

3. Set the properties of Thread Group:



Performance testing tools – Apache JMeter demo

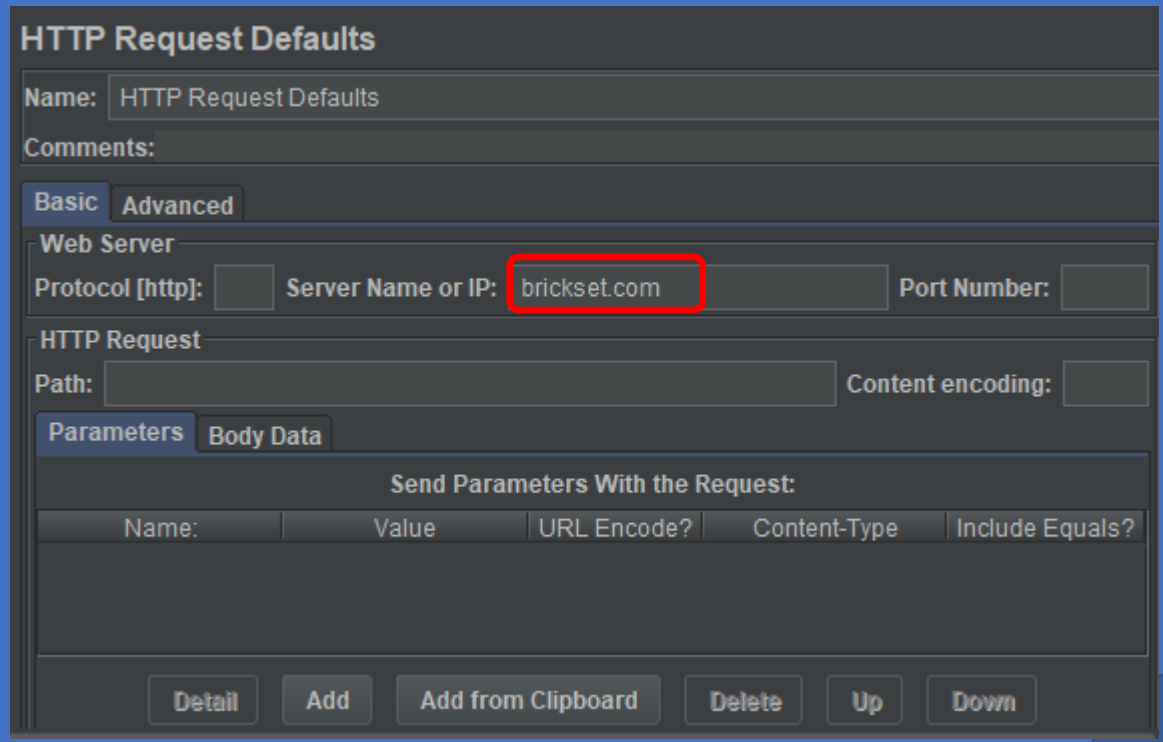
4. Add HTTP Request elements properties:



5. Set the name of the webserver where all HTTP requests will be sent to (*brickset.com* in the current example)

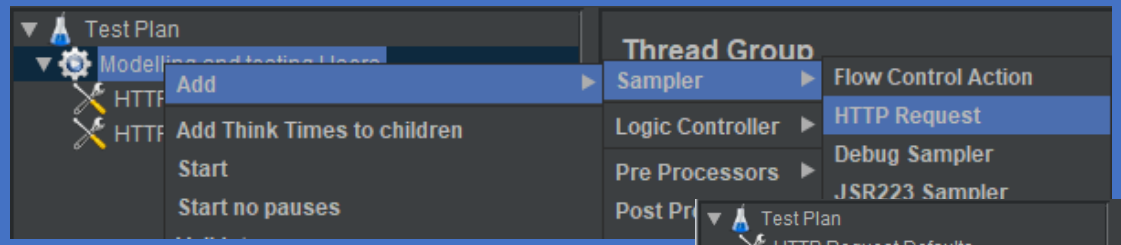
- Note that this is just the setting of defaults that HTTP request elements use, it does not send HTTP elements itself (it will be done in step 7).

Right click on Users group,
then *Add/Config Element/HTTP Request Defaults*

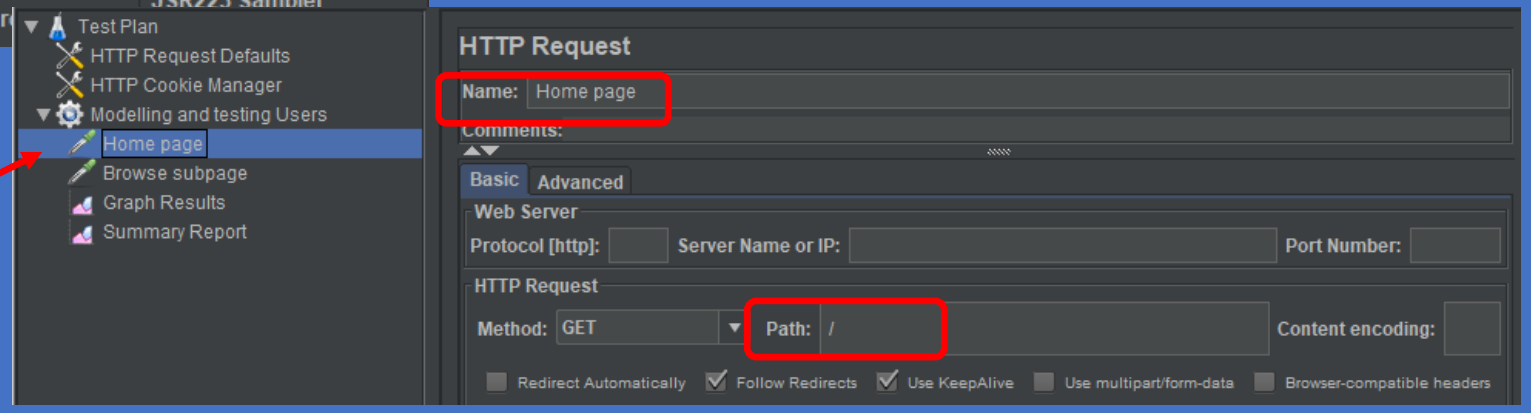


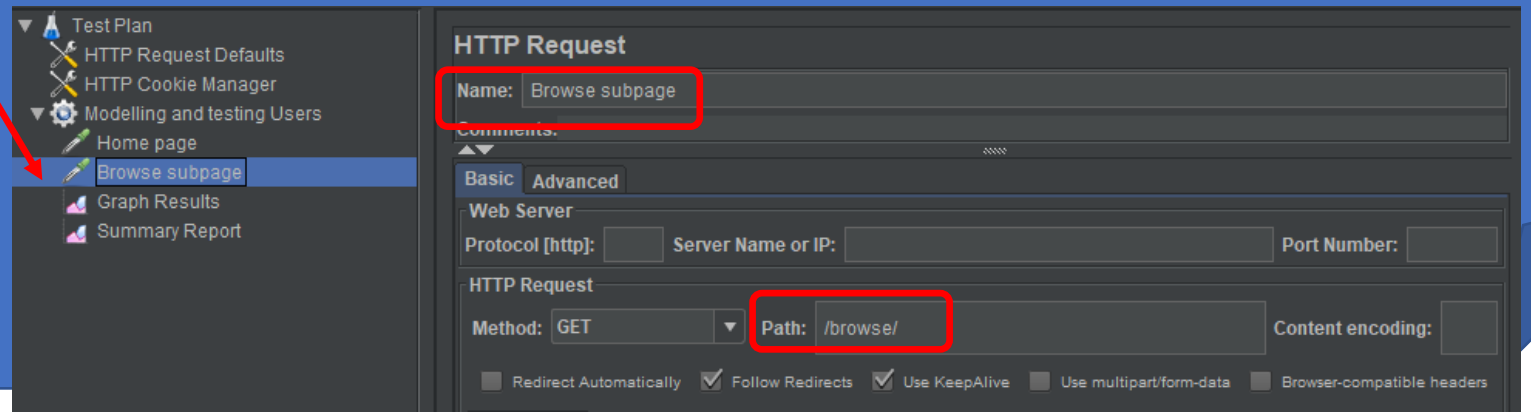
Performance testing tools – Apache JMeter demo

6. Add HTTP cookie manager
   - Right click on Users group, then *Add/Config Element/HTTP Cookie Manager*
   - This will ensure that each thread gets its own cookies
7. Adding HTTP request



   - We will add 2 HTTP requests:
     1. Going into the home page
     2. Going into a subpage





8. Add reports to the TestPlan!
   - Right click on *Thread Group*, then *Add/Listener/*
9. Validate TestPlan
   - Right click on *Thread Group*, then *Validate*

Performance testing tools – Apache JMeter demo

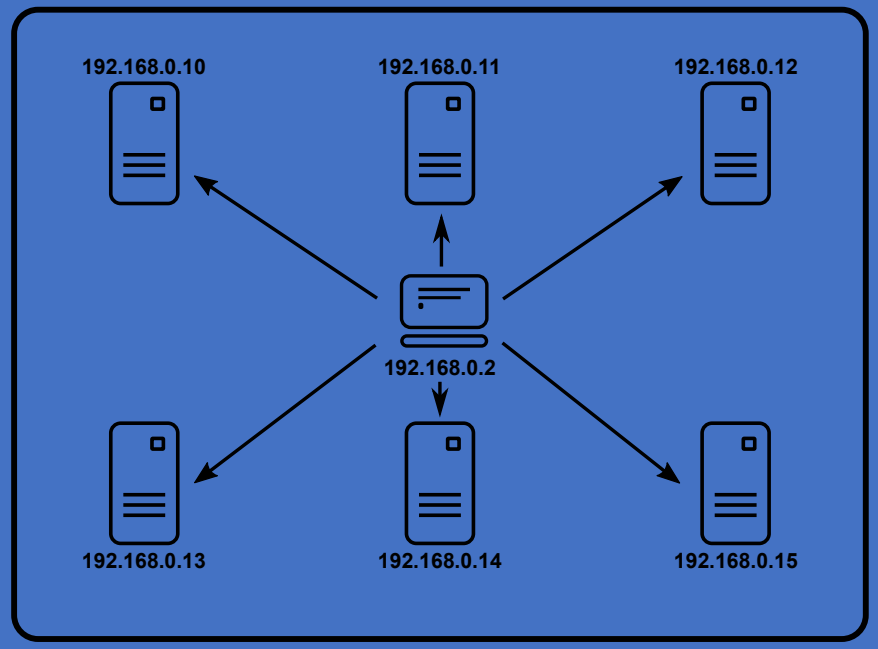
IV. What we did not tried in this demo, but may worth to check 2/1:

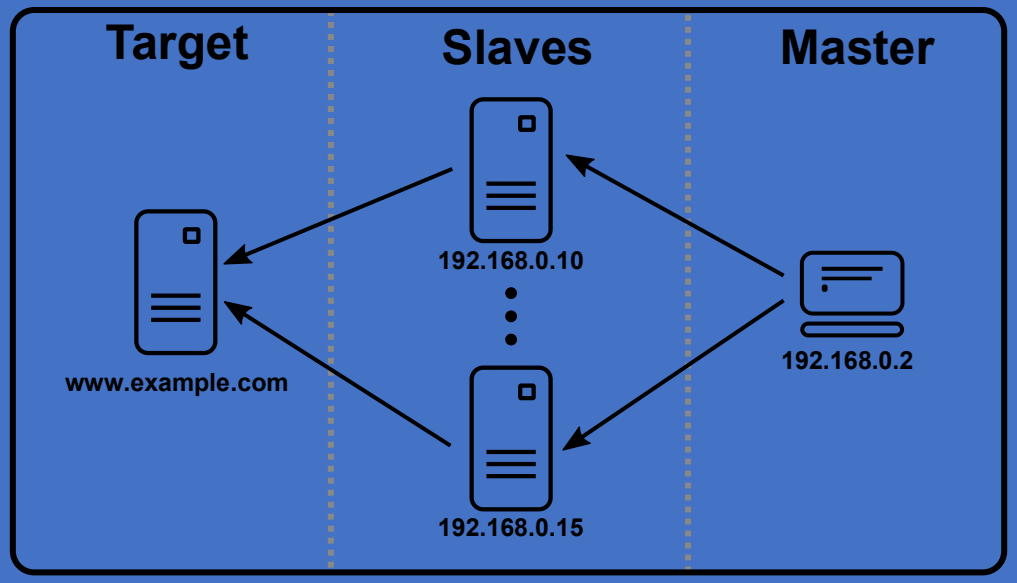
- Discover the possibilities of CLI mode (full list of options)

```
jmeter-server -H my.proxy.server -P 8000
```

```
jmeter -n -t testplan.jmx -r [-Gprop=val] [-Gglobal.properties] [-X]
```

- Testing of other protocols, types of load (like testing a database, an FTP site, an LDAP server, a webservice…etc)
- Distributed testing: handle multiple slaves that generates the same/different load

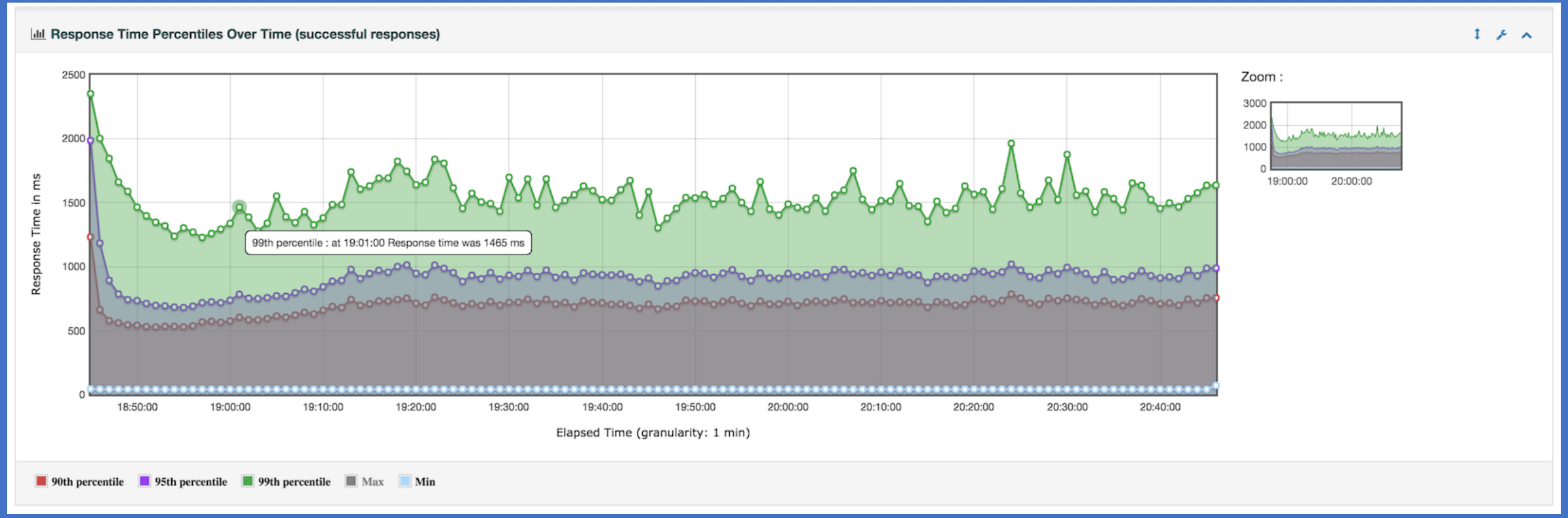




Performance testing tools – Apache JMeter demo

IV. What we did not tried in this demo, but may worth to check 2/2:

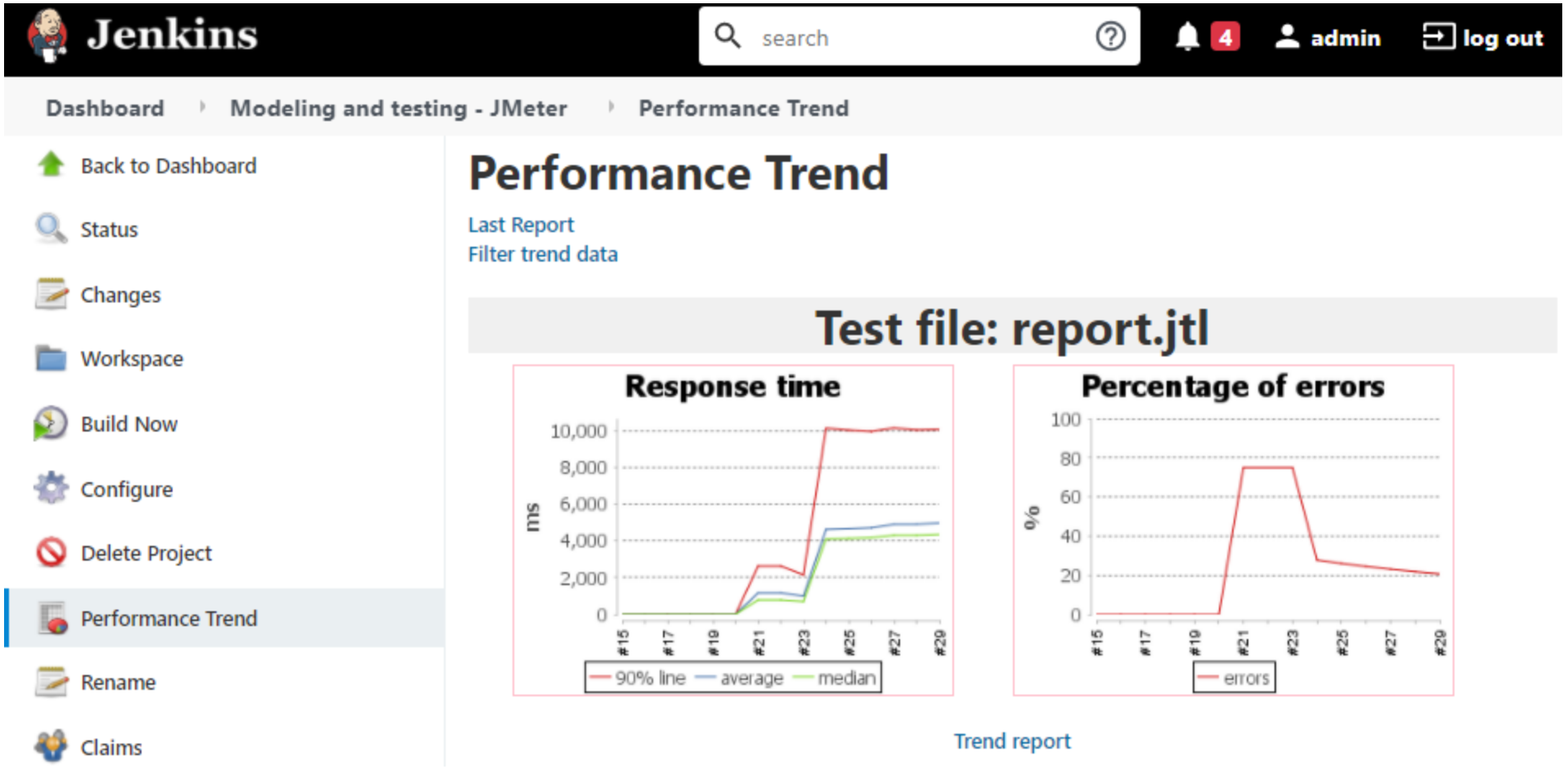
- Handle listeners, create own statistics
- Create HTML test reports



- Pattern matching with regular expressions

...etc.

Performance testing tools – Jenkins performance plugin

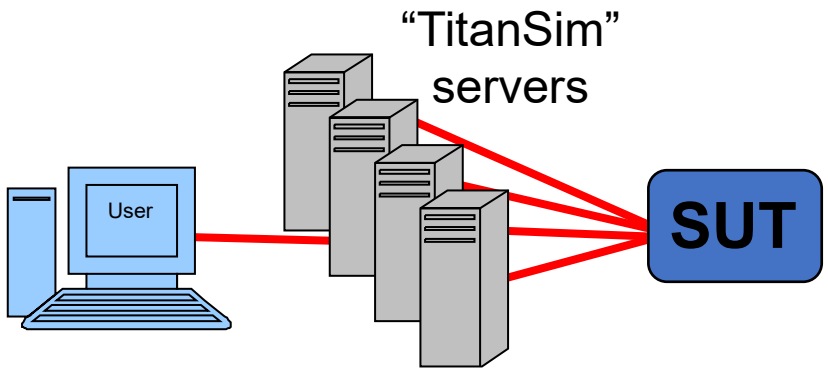


[Performance plugin] of [Jenkins] can be used to integrate [Apache JMeter] tests into the [pipeline process]

Performance testing tools – TitanSim demo

ERICSSON

- Not commercial tool, but used worldwide inside ERICSSON
- Used for **performance testing of network elements:**
  - Simulate users, different nodes of the network and their interactions
    - ~ 100.000 users / 12-core server
      - Scale for few million users
    - ~ 1.000 parallel calls / 12-core server
  - **Simulate complex behaviours:**
    - Alternative flows
    - Realistic traffic mix



- Based on TITAN (open source and free TTCN-3 test execution engine used in functional testing)

Source of info: An UCAAT 2016 poster, a HUSTEF 2016 presentation

Performance testing tools – TitanSim demo

ERICSSON

TitanSim for performance testing*

- Internal tool within Ericsson

Node simulation

User equipment simulation

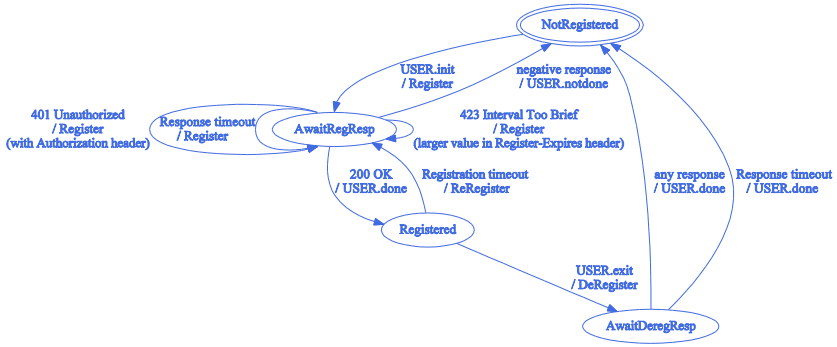
Titan for functional testing (TTCN→C++)

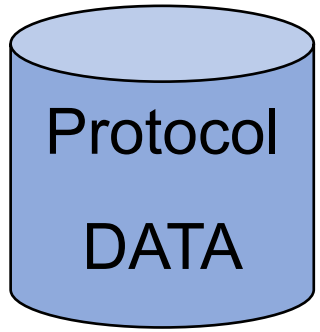
- free and open source

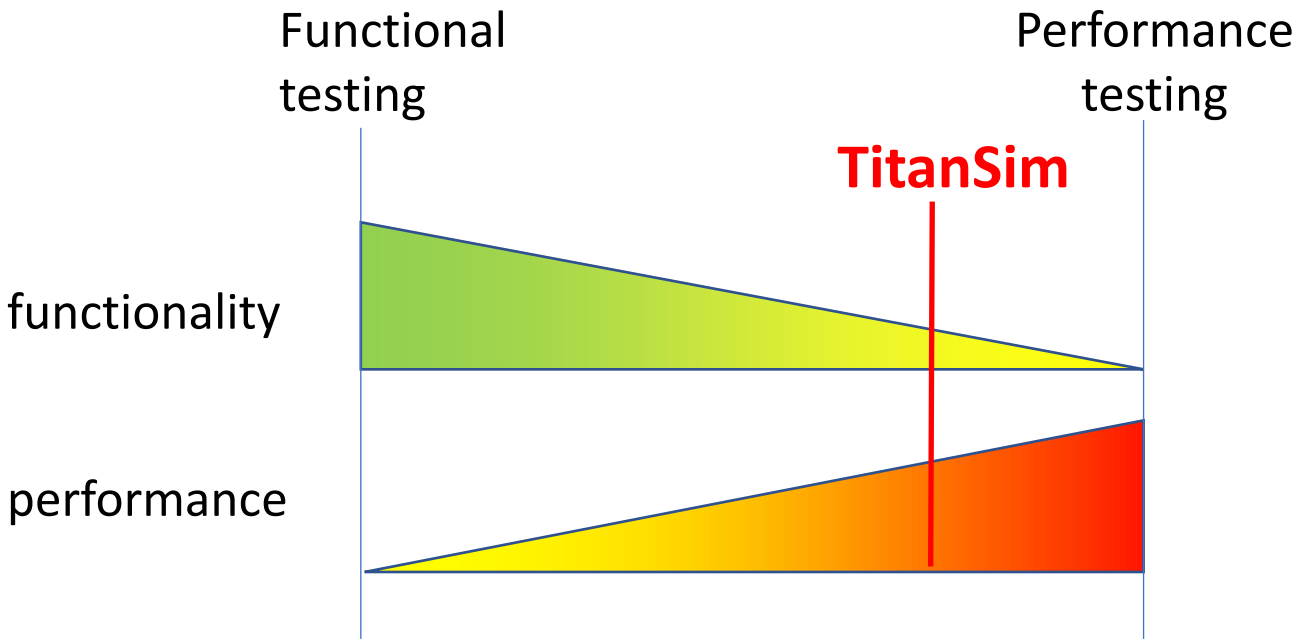
Performance testing tools – TitanSim demo

ERICSSON

- TitanSim uses EFSMs for behaviour description







Source of info: An UCAAT 2016 poster, a HUSTEF 2016 presentation

Performance testing tools – TitanSim demo

ERICSSON

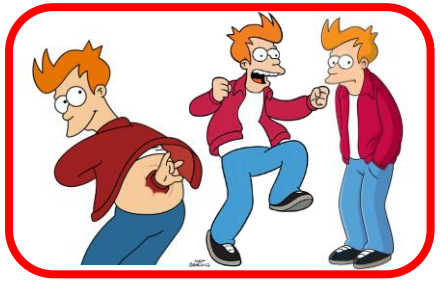
Terminology:

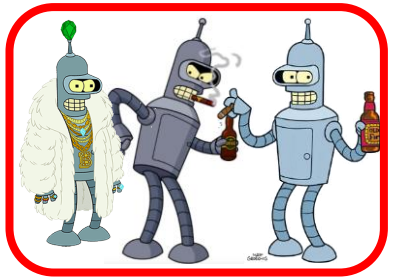
- **Entity**: A single abstract simulated object (e.g. a subscriber)

- **Entity type**: Describes the capabilities* of a given Entity

  * Set of behaviors that can be simulated

- **Entity group**: A set of Entities of the same Entity type



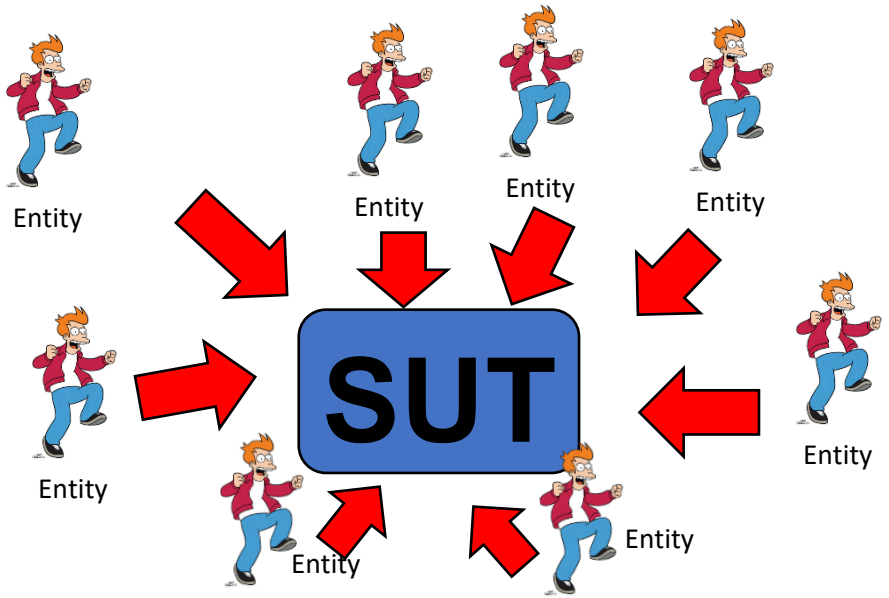


Source of the figs: Matt Groening (Futurama)

Performance testing tools – TitanSim demo

ERICSSON

- **Entity group**: A set of Entities of the same Entity type
  - Used to emulate the traffic flow
  - When entity is finished it will be available for another execution



Source of the figs: Matt Groening (Futurama)

Performance testing tools – TitanSim demo

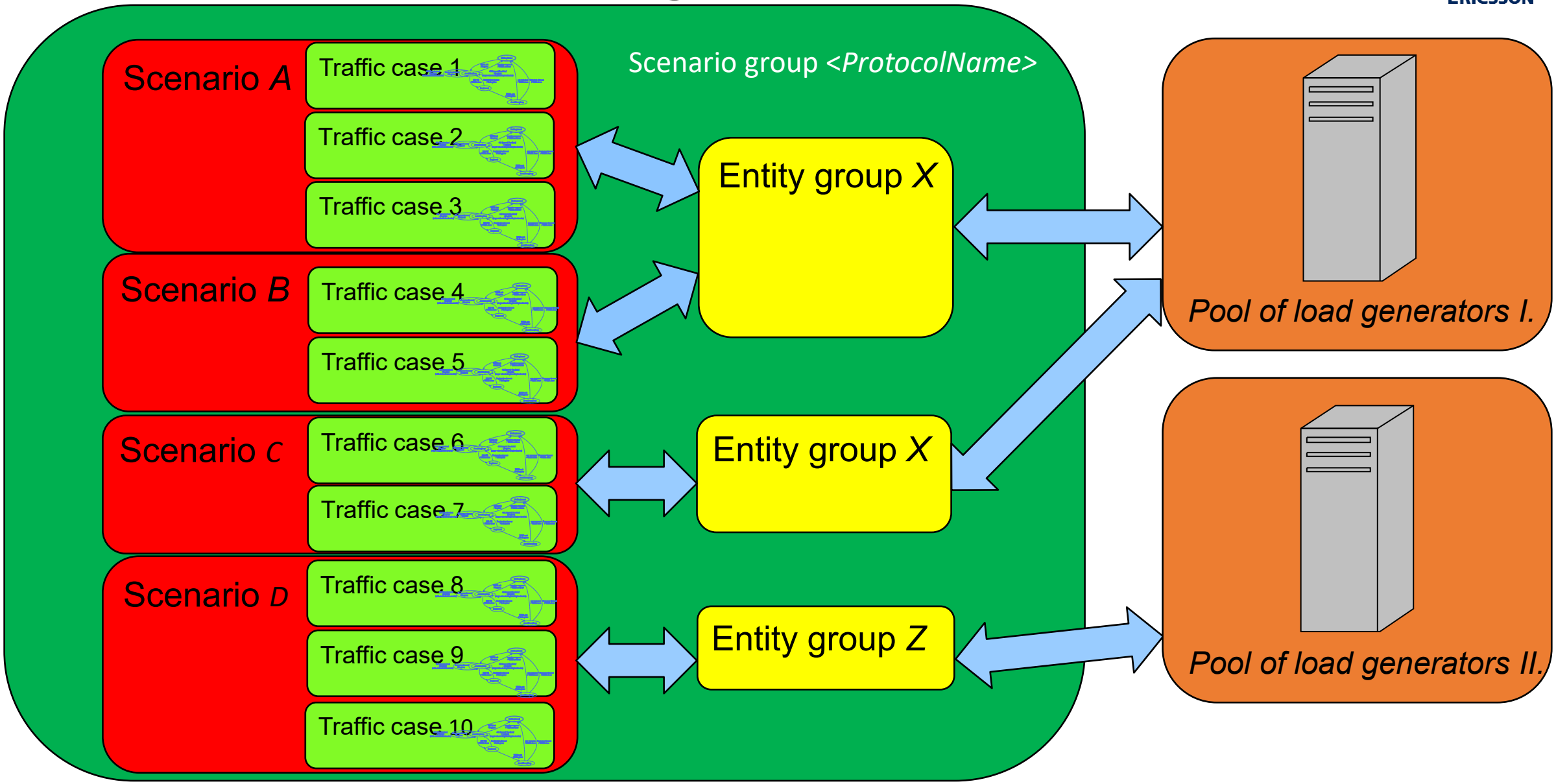
ERICSSON

Assignment of traffic flows to actors:

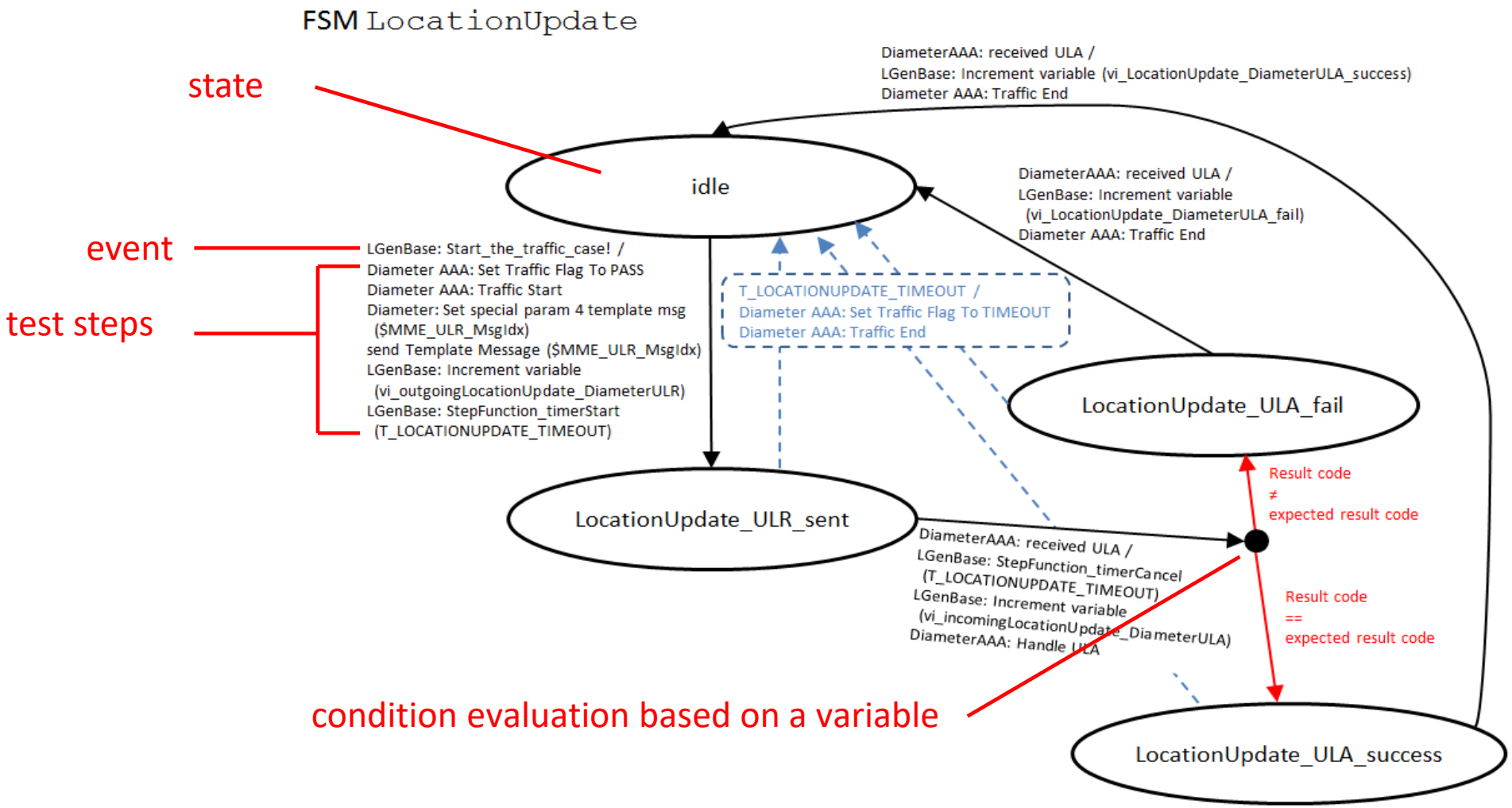
- **TrafficCase**: EntityType ↔ FSM (1-1) assignment
- **Scenario**: set of TrafficCases and their parameters
- **Scenarios2Groups**: assigns Scenarios to Entity groups
- **Scenario group**: set of Scenarios

Performance testing tools – TitanSim demo

ERICSSON



Performance testing tools – TitanSim demo



Performance testing tools – TitanSim demo

ERICSSON

Name of the FSM:

LocationUpdate

List of states:

idle, LocationUpdate_ULR_sent, LocationUpdate_ULA_success, LocationUpdate_ULA_fail

List of timers:

T_LOCATIONUPDATE_TIMEOUT

```
{
  name := "LocationUpdate",
  paramDecls := {
  {stateList := { "idle","LocationUpdate_ULR_sent", "LocationUpdate_ULA_success", "LocationUpdate_ULA_fail" } },
  {timerList := {{ name := "T_LOCATIONUPDATE_TIMEOUT", startValue := 5.0 }}}
  },
  table :=
  {
    extendedTable := {
    {
      events2Listen := { events := { { singleEvent := { bName := "EPTF_CLL_LGenBase",  iName := "LGenBase: Start_the_traffic_case!", eventType := fsm } } } },
      { inState:={ state := "idle"},
        {   actionList := {
            { stepOrFunctionName := "Diameter AAA: Set Traffic Flag To PASS", contextArgs := { stepContextArgs :=  { } } },
            { stepOrFunctionName := "Diameter AAA: Traffic Start",contextArgs := { stepContextArgs := { } } },
            { stepOrFunctionName := "Diameter: Set special param 4 template msg", contextArgs := { stepContextArgs := { ${MME_ULR_MsgIdx,integer}, ${SessionId_AVPId,integer}, 0 } } },
          { stepOrFunctionName := "Diameter: Set special param 4 template msg", contextArgs := { stepContextArgs := { ${MME_ULR_MsgIdx,integer}, ${UserName_AVPId,integer}, 0 } } },
            { stepOrFunctionName := "send Template Message", contextArgs := { stepContextArgs := { ${MME_ULR_MsgIdx,integer}, 0 } } },
            { stepOrFunctionName := "LGenBase: Increment variable", contextArgs := {varParams := {varName := "vi_outgoingLocationUpdate_DiameterULR",paramValue := {intVal := 1}} } },
            { stepOrFunctionName := "LGenBase: StepFunction_timerStart", contextArgs := { timerName := "T_LOCATIONUPDATE_TIMEOUT"} }
            },
            nextStateCalculation := omit
            nextState := "LocationUpdate_ULR_sent"
        }
      }
    }
  },
```

Name of the timer

Value of the timer in seconds

Start of FSM table i.e. the place where we describes transitions with events and test steps

Events *LGenBase: Start_the_traffic_case!*

Start state of the given transition: Where are we yet? *Idle*

(First state in the FSM table is the initial state of the FSM)

Next state:

State where the machine goes after the given event has been occoured and test steps have been executed

LocationUpdate

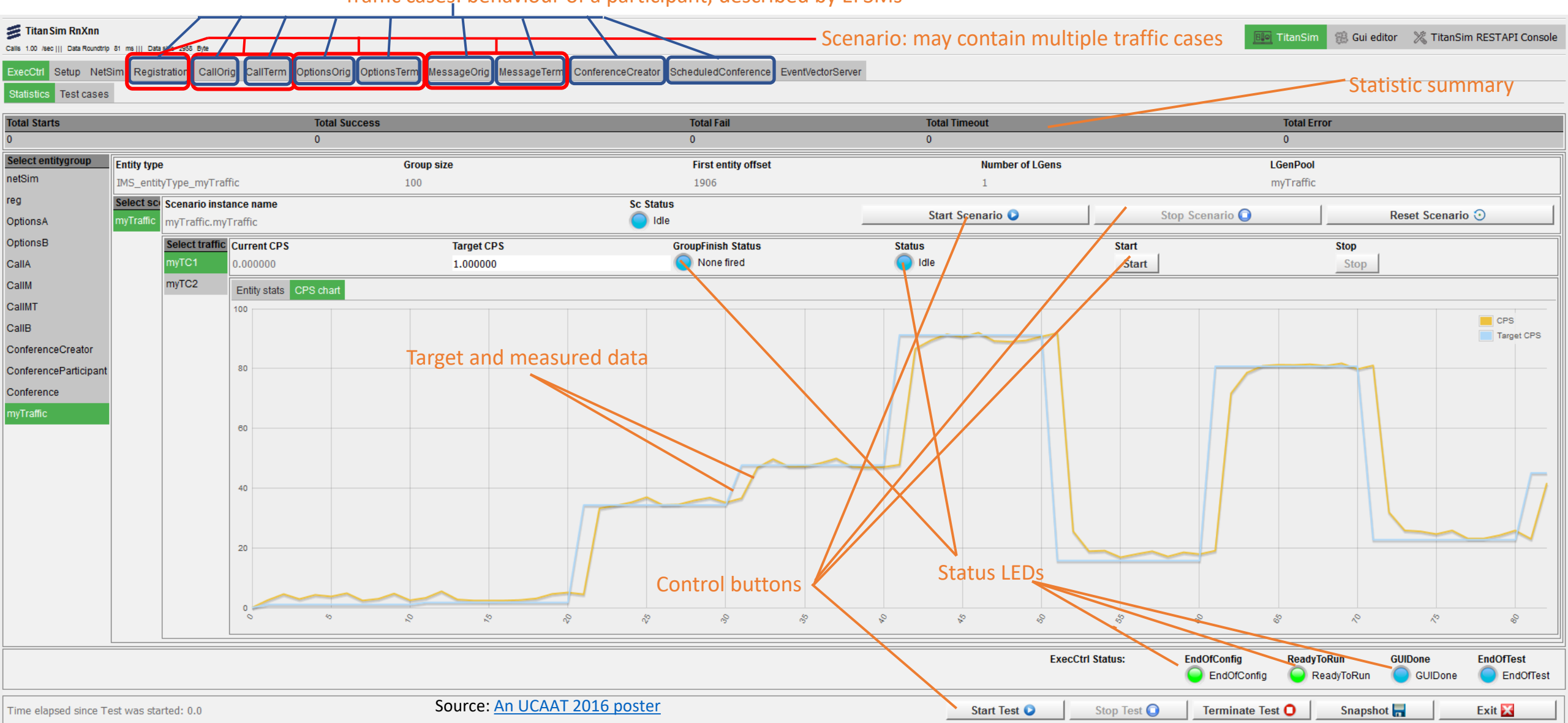
List of test steps for a given transition:

Diameter AAA: Set Traffic Flag To PASS…etc.

Parameters of a test step

Performance testing tools – TitanSim demo

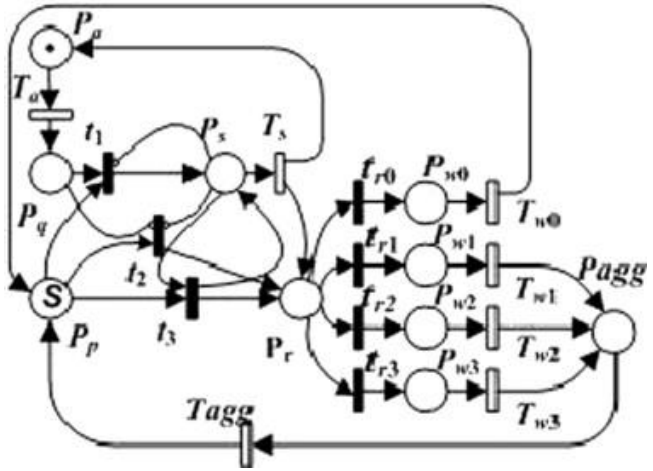
ERICSSON



Source: An UCAAT 2016 poster

Model-driven performance testing methods for communicating systems 6/1

- Different perf. models for communicating systems exist based on Petri nets
  - Analytically prove that their models correspond to the specifications
    - Number of requests served pro second, number of parallel users…etc.



Example Performance testing petri net model from Youness, O. S. and El-Kilani, W. S. and El-Wahed, W. F. A. A: Behavior and Delay Equivalent Petri Net Model for Performance Evaluation of Communication Protocols In Computer Communications, vol. 31 issue 10., pp. 2210–2230, 2008.

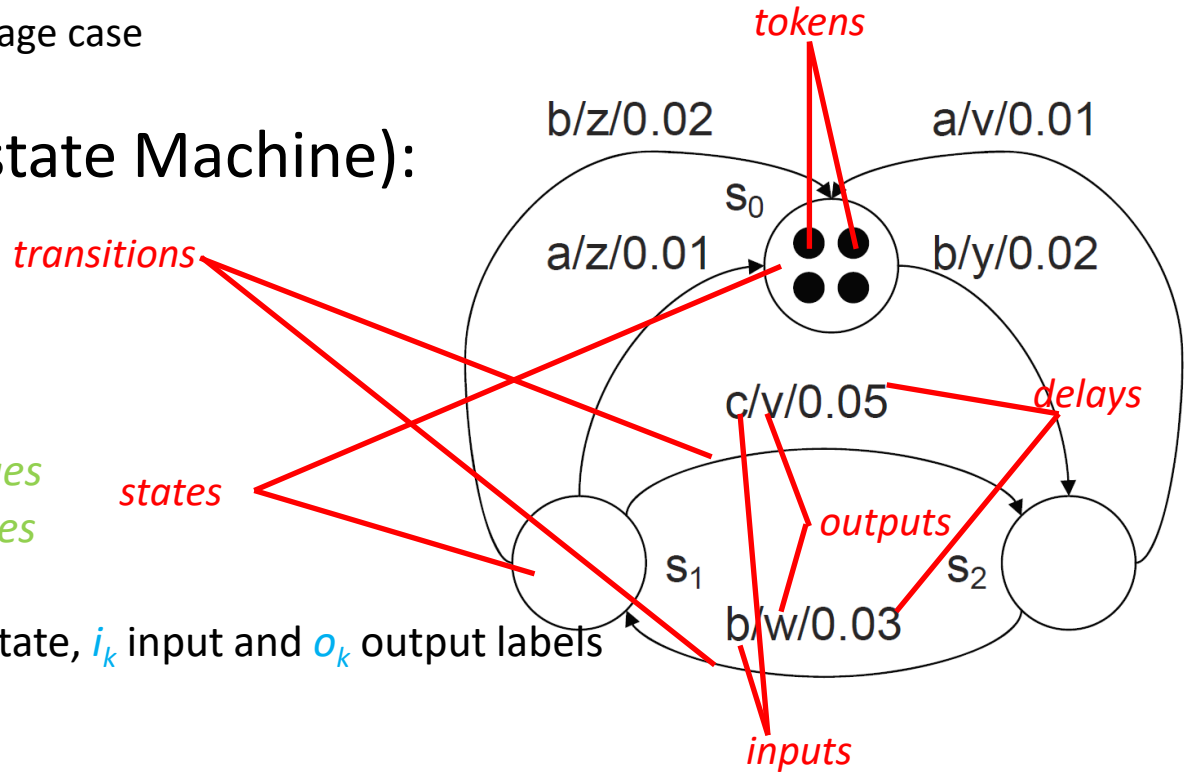
  - Do not deal with the validating of the SUT (i.e. if impl. corresponds to the perf. model)
    - The validation can be made in an ad-hoc way:
      - Emulating the real environment, where software entities emulates users that runs a given traffic mix (*see TitanSim for example*)

        → Only the expected characteristics can be investigated, not the worst case (large deviation in practice!)

Model-driven performance testing methods for communicating systems 6/2

- The goal is to determine if the SUT being capable of processing a given number of requests per second while serving a given number of users
  - Both in worst case (any sequence of different requests) and in average case

- TCFMM (Timed Communicating Finite Multistate Machine):
  - The model is based on FSMs:
    - *TCFMM = (S, I,O, T, U, s_0)*
      - ***S***: set of states
        - where s_0 is the initial state
      - ***I***: set of input sysmbols - *REQUEST messages*
      - ***O***: set of output symbols - *ANSWER messages*
      - ***T***: set of transitions
        - Each transition t_k has an s_x initial and a s_y next state, i_k input and o_k output labels and d_k **delays**
      - ***U* set of users to be served**
        - **Each user is represented with a u_k token** that traverses from one state to another with given delay d_k
        - **Each token from a terminal state goes back automatically to the initial state** (representing that at a time $|U|$ user can be served. If $x \leq |U|$ request is served, then x new request can be initiated parallelly)



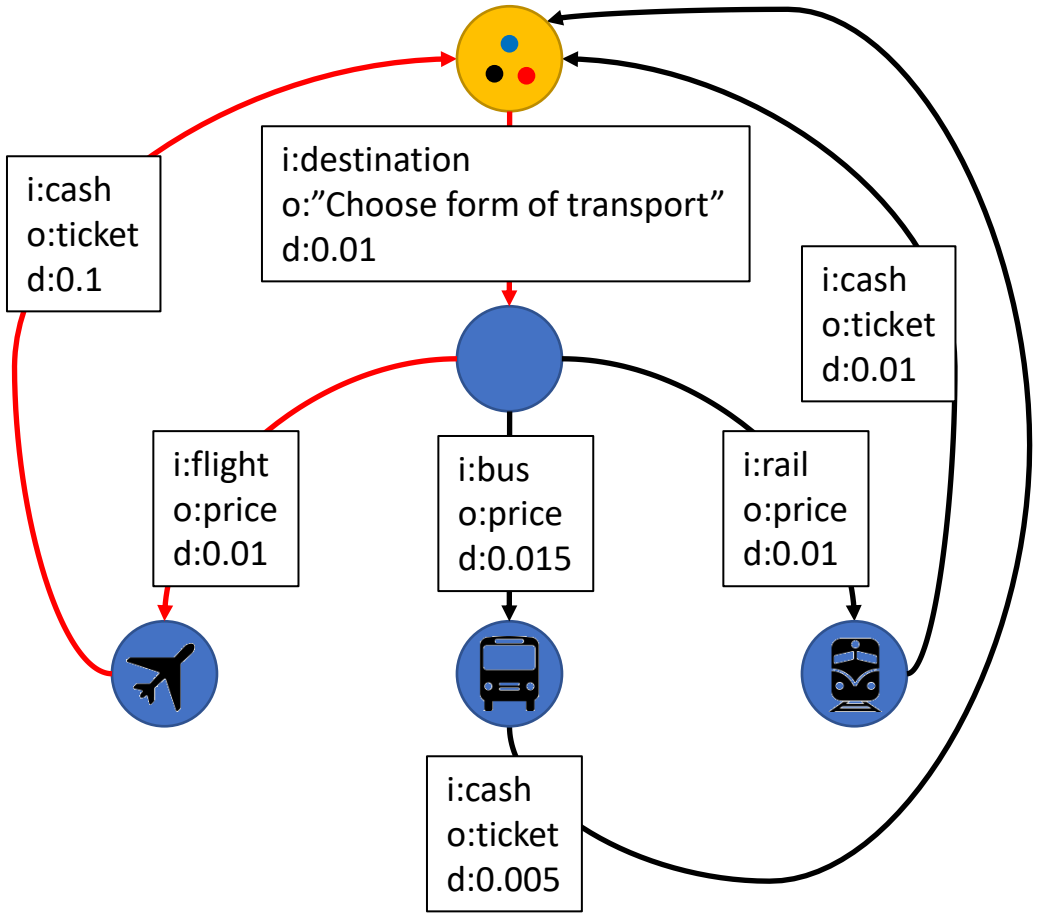
Model-driven performance testing methods for communicating systems 6/3

- The using of TCFMM model:
  - The number of simultaneously served users → the number of tokens: $usr=|U|$
  - The d_k delay of a transition t_k: $d_k=\frac{\max_{u_j \in U}(pd_j)}{usr}$, where
    - pd_j physical delay for a user u_j between sending i_k and receiving o_k
  - TCFMM is able to process C_{req} mps (messages per second) iff for all of its transition cycles c_i:

$$\frac{\sum_{t_j \in c_i} d_j}{|c_i|} \leq \frac{1}{C_{req}}$$

Model-driven performance testing methods for communicating systems 6/4

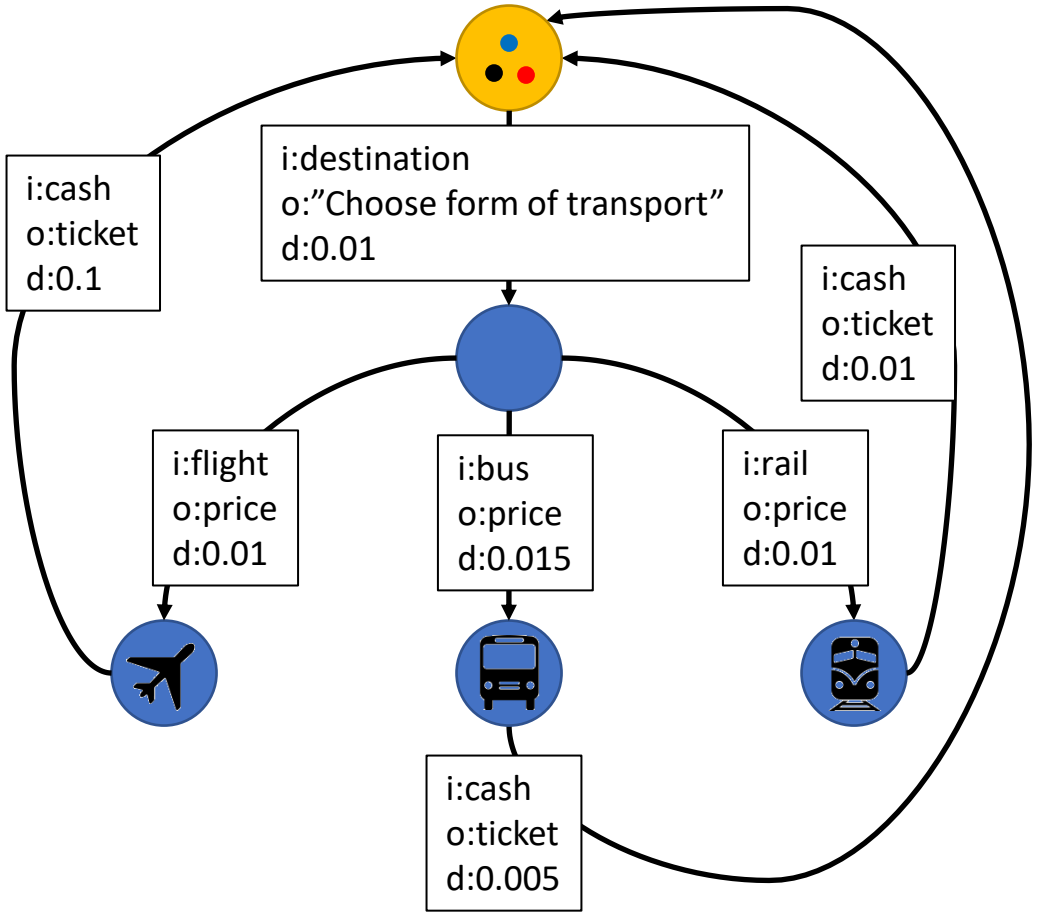
- Comparing TCFMM model to SUT:
  - **Worst case mps**: finding the cycle in the TCFMM traversing for which the SUT processes the lowest possible number of calls within a second.



Source of the animation:
Levente Erős: An Automatic Performance Testing Method Based on a Formal Model for Communicating Systems slides

Model-driven performance testing methods for communicating systems 6/5

- Comparing TCFMM model to SUT:
  - **Calculating average mps**:
    - traffic mix, users select different actions with given probabilities



Source of the animation:
Levente Erős: An Automatic Performance Testing Method Based on a Formal Model for Communicating Systems slides

Model-driven performance testing methods for communicating systems 6/6

- Comparing TCFMM model to SUT:
  - **Worst case mps**: finding the cycle in the TCFMM traversing for which the SUT processes the lowest possible number of calls within a second.
  - **Calculating average mps**: $C_{expected} = \frac{1}{\sum_{t_i \in T} d_i f_i}$, where:
    - d_i : average of transition delays
    - f_i : corresponding stationary transition probabilities

Further information about the topic:
Levente Erős, Tibor Csöndes: An automatic performance testing method based on a formal model for communicating systems. IWQoS 2010: 1-5

Then we can calculate the mps with solving the linear equation system with the observed f_i stationary transition probabilities

These **probabilities come from the investigated SUT**. Thus instead of inputs of the original TCFMM model, we **use** these probabilities **in the calculation to decide where to go** → Markov-model

Performance testing - advices

- Select the appropriate type of performance testing and measured parameters
- One measurement is not measurement, a baseline is required
- Use a tool with appropriate complexity and performance
- Measure performance as early as possible, performance testing is not only applicable to finished products
- Performance testing is not a separate testing entity; it can be integrated into other testing attributes like functionality, usability, reliability and security

Performance testing – hot research topics

- Load generators that use some feedback from the SUT to generate dynamic workload:
  - Identifies workload sensitive input transitions, then increment load for these transitions with monitoring key performance parameters:
    - Osvaldo Huerta-Guevara, Vanessa Ayala-Rivera, Liam Murphy, and A. Omar Portillo-Dominguez. Towards an efficient performance testing through dynamic workload adaptation. In Testing Software and Systems - 31st IFIP WG 6.1 International Conference, ICTSS 2019, Paris, France, October 15-17, 2019, Proceedings, volume 11812 of Lecture Notes in Computer Science, pages 215–233. Springer, 2019.
  - Simply terminate the given tests when they provide no further relevant information:
    - Hammam M. AlGhamdi, Cor-Paul Bezemer, Weiyi Shang, Ahmed E. Hassan, and Parminder Flora. Towards reducing the time needed for load testing. Journal of Software: Evolution and Process, pages –, July 2020.
  - Use machine learning and reinforcement learning to create stress tests to find performance breaking points and bottlenecks:
    - M. Helali Moghadam, M. Saadatmand, M. Borg, M. Bohlin, and B. Lisper. Machine learning to guide performance testing: An autonomous test framework. In 2019 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), pages 164–167, 2019.
    - T. Ahmad, A. Ashraf, D. Truscan, and I. Porres. Exploratory performance testing using reinforcement learning. In 2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pages 156–163, 2019.
    - J. Koo, C. Saumya, M. Kulkarni, and S. Bagchi. Pyse: Automatic worstcase test generation by reinforcement learning. In 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST), pages 136–147, 2019.
- Performance Monitoring - give alert about possible performance deviations in advance:
  - Haroon Malik and Elhadi M. Shakshuki. Performance evaluation of counter selection techniques to detect discontinuity in large-scale systems. J. Ambient Intell. Humaniz. Comput., 9(1):43–59, 2018.
- Extracting logs to detecting performance regressions automatically
  - Lizhi Liao, Jinfu Chen, Heng Li, Yi Zeng, Weiyi Shang, Jianmei Guo, Catalin Sporea, Andrei Toma, and Sarah Sajedi. Using black-box performance models to detect performance regressions under varying workloads: an empirical study. Empir. Softw. Eng., 25(5):4130–4160, 2020.
- Embedding performance testing into cloud and other virtual testing environments

The future of testing

Main ToC

How will the future look like?

- Some past predictions came true, some did not…

Transportation prediction from 60s

Self driving car prediction from 1967

Prediction from 40s: Tokyo 2011

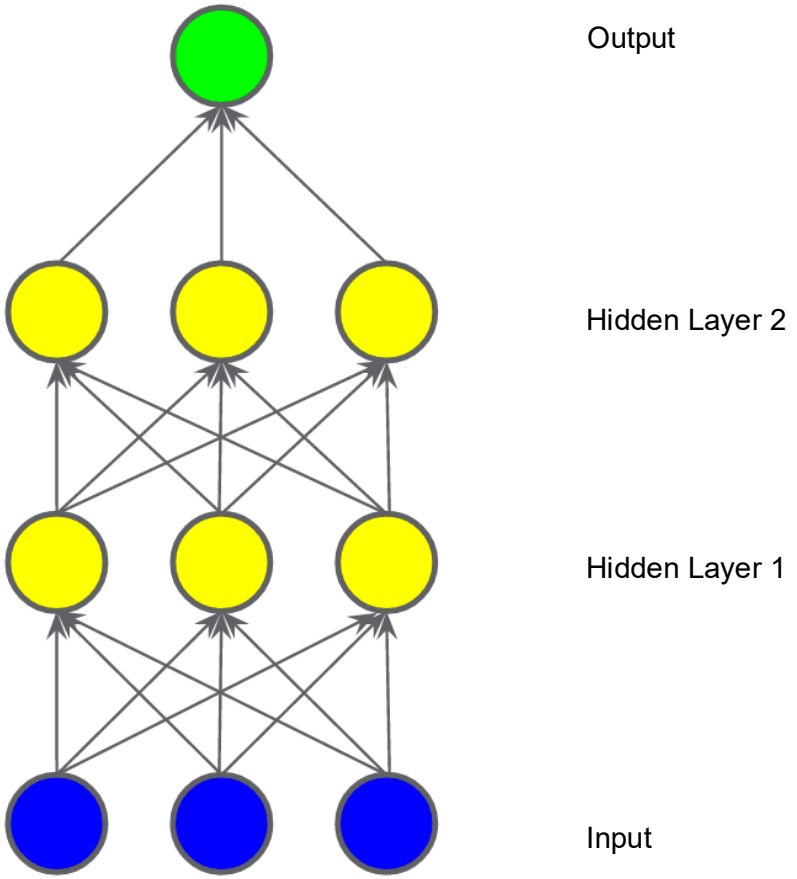
Moon base prediction from 60s

Blade Runner (1982): Los Angeles in 2019

AI – working principles

- Model: Neural net
  - Inputs: features we want to classify
    - Raw data → feature vectors with given encodings
  - Hidden layers and numbers of neurons in a given layer
    → Complexity (speed of classification)
  - Each neuron of the networks gives an output for it's inputs with given weights and activation functions
  - Output: classification

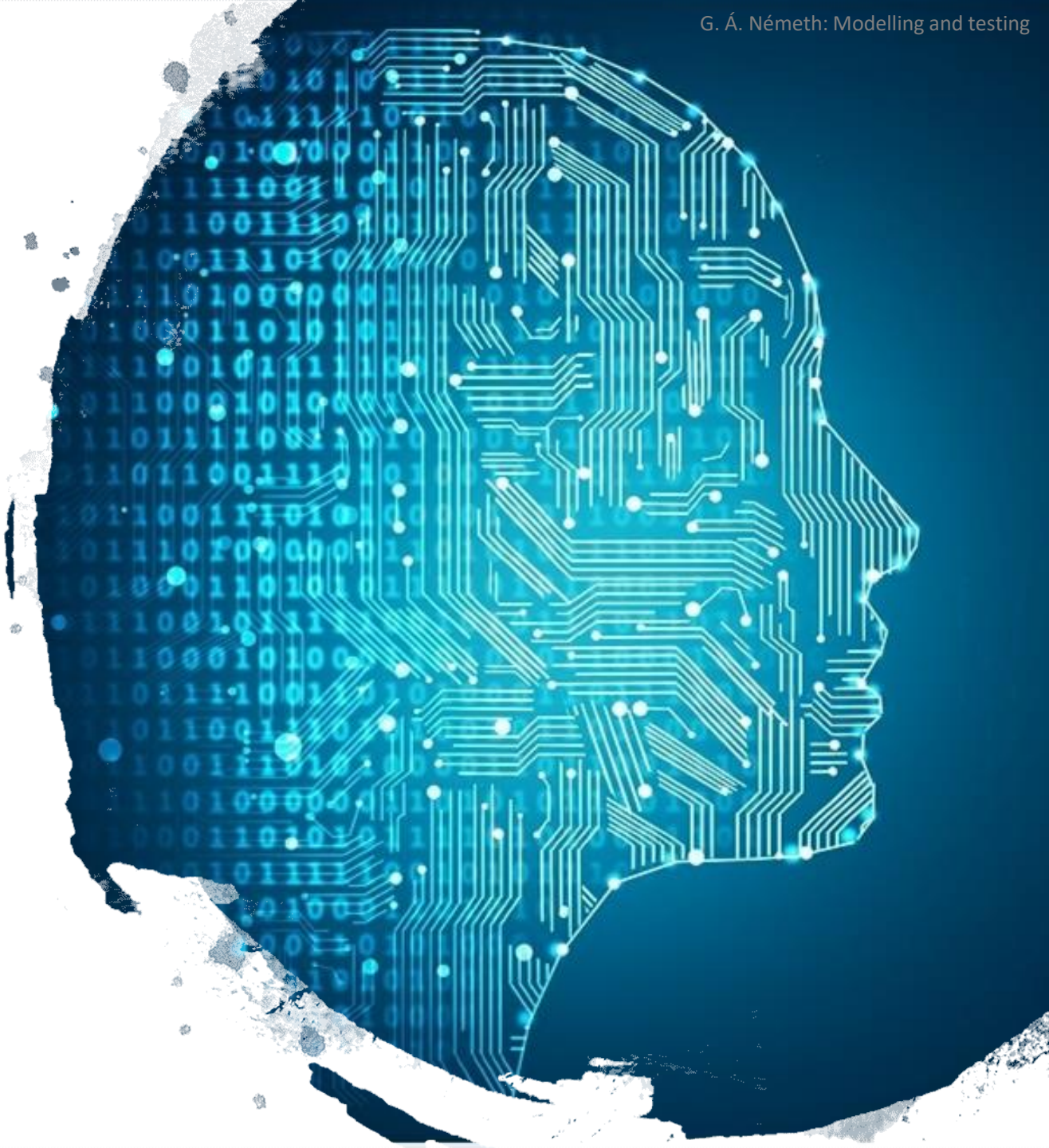
- Deep learning – 3 sets of data:
  - Training set
    - To train model from data
  - Validation set
    - To fine tune model (learning rate, iterations, batch size...etc.)
  - Test Set
    - Confirm results at the end



(Source of image: Google Tensorflow documentation)

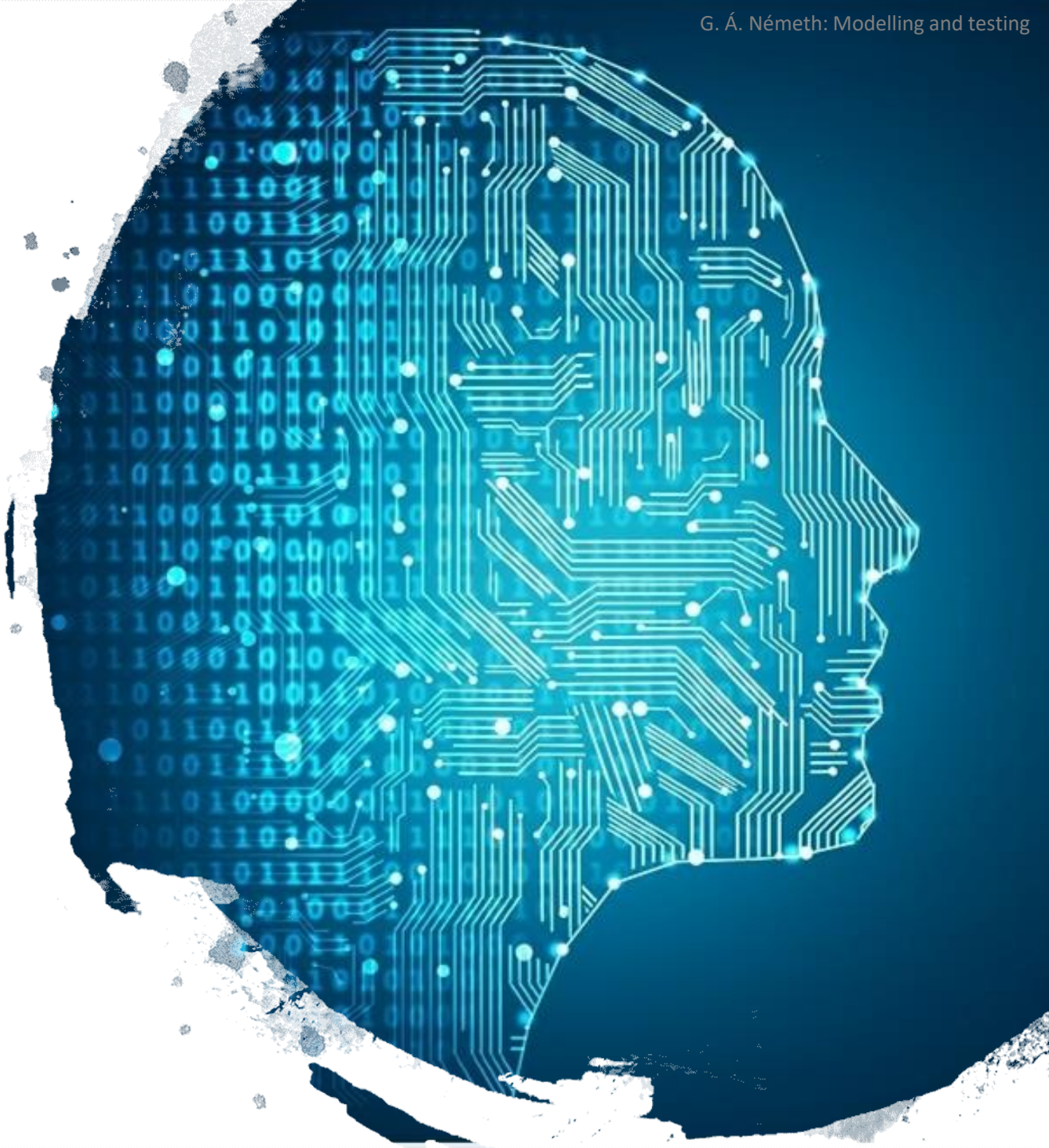
How AI can be used in testing?

- Analyze logging & give more information of specific parts
- Optimize the execution of existing tests
  - Within a given timeframe which test should be executed?
  - If more available time, then the threshold of AI decision can be lowered...
- Focus test generation to given area
- …etc.



How AI can be tested?

- How to test neural networks?
  - If the training phase is already finished, i.e. neural networks with preset weights
    - Statistical methods
  - If the training phase is not finished, i.e neural networks can adjust their weights while observe the world
    - How to prevent attacks that try to teach network with manipulated data?



AI – free general tools 3/1

- Google Tensorflow
  - Free and open source software library for high performance numerical computation, strong support for machine learning and deep learning
  - Many companies using it (Google, AMD, Nvidia, Uber, AirBnd, ebay, SAP, twitter, Airbus, Intel, linkedIn…etc)
  - Has very good general tutorial on AI, tutorial on Tensorflow, documentation

- Keras
  - Running on the top of Tensorflow, CNTK or Theano
    - The below layer will be used as a tensor manipulation library and Keras will be used for build and train models
  - Simple and user friendly
  - Modular
    - Existing building blocks
    - Extension with custom building blocks
  - Open source (python)
  - Has very good documentation

AI – free general tools 3/2

- PyTorch
  - Framework of Meta
  - Similar to TensorFlow, object oriented
  - High abstraction level tools are also provided with GPU support

- Google JAX:
  - Machine learning framework for transforming numerical functions
  - Works with other frameworks (TensorFlow, PyTorch…etc.)
  - Main functionalities: automatic differentiation, just-in-time compilation, auto-vectorization, parallelization
  - Low level tool
  - Several python libraries use JAX as a backend, like Flax, Equinox, Diffrax…etc.

AI – free general tools 3/3 (“also-run” category)

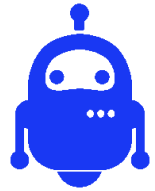
- H_2O, Sparkling water
- Facebook’s Caffee2
  - Last release in 2007
- Acumos AI
- Microsoft Cognitive Toolkit (CNTK)
- Theano

AI – free datasets

- General image datasets:
  - Image net
  - Coco

- Downloadable automotive deep learning data sets in image processing topics:
  - Berkley deep drive
  - Cityscapes
  - Kitti

AI – free testing tools: AGENT 2/1

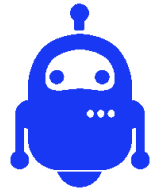


- AGENT: AI Generation and Exploration in Test
  - Simple, open source tool
  - Autonomously learn and explore a website and evaluate its elements
  - Consists of 4 test agents:
    - 1 coordinator:
      - Set testing assignments to working agents
    - 3 work agents:
      - Interacts with SUT, fulfills the given goal set by coordinator
      - 2 mode:
        - Explore and generate test
        - Run generated test

(Source of the info: Tariq KING, Ultimate Software (USA): Demystifying AI-Driven Test Automation. HUSTEF 2019 Tutorial day presentation)

SUT: System Under Test

AI – free testing tools: AGENT 2/2



- AGENT: AI Generation and Exploration in Test
  - AGENT source code in github
    - User's Guide
  - AGENT-X source code in github (Chrome extension to supplement the AGENT)

Additional info:

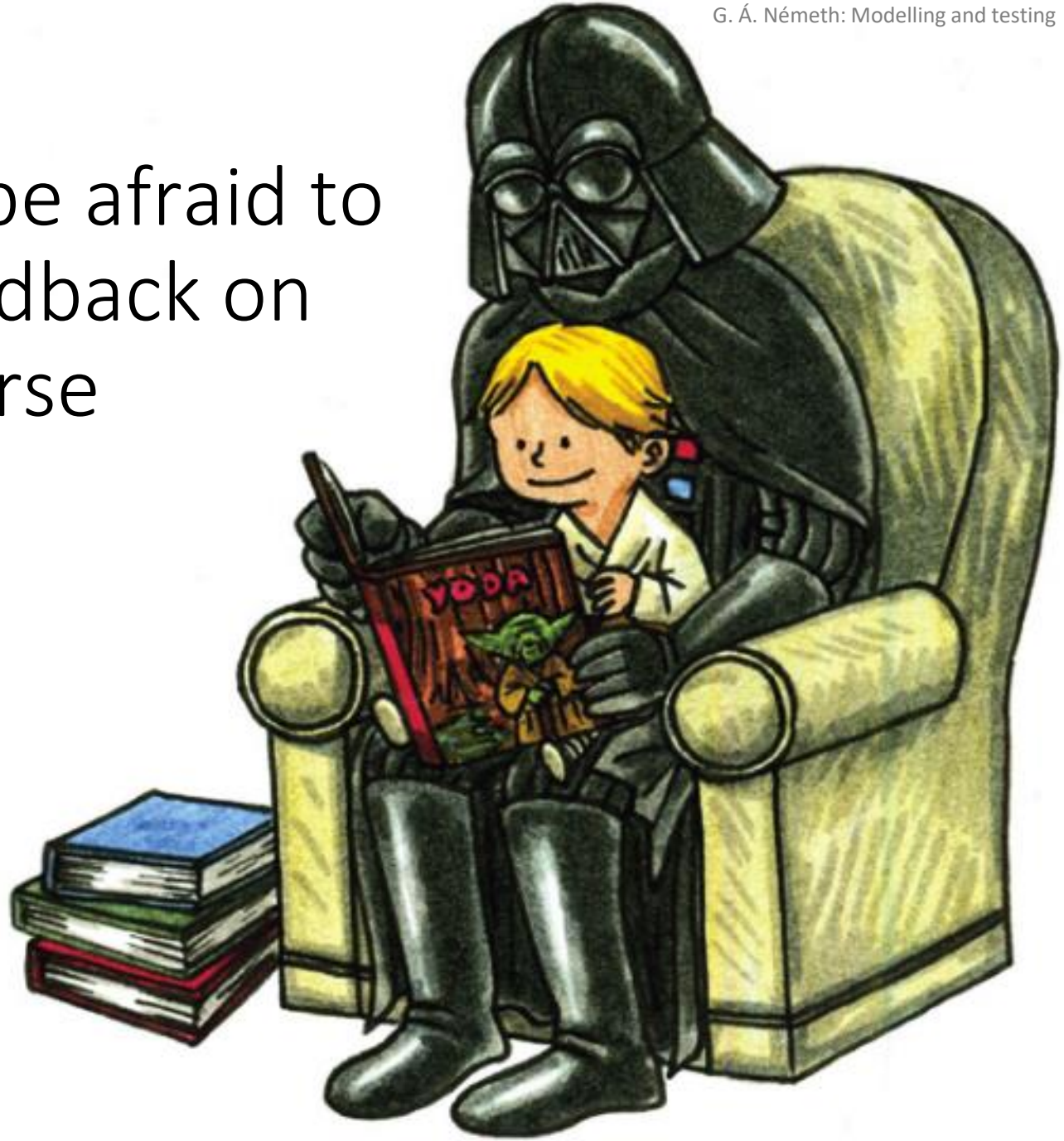
- Santiago, D. (2018). A model-based AI-driven test generation system (Unpublished master's thesis). Miami, Florida, Florida International University. Retrieved February 5, 2019.
- Santiago, D., Clarke, P. J., Alt, P., & King, T. M. (2018). Abstract flow learning for web application test generation. Proceedings of the 9th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation - A-TEST 2018, 49-55. doi:10.1145/3278186.3278194
- Santiago, D., King, T., & Clarke, P. (2018). AI-driven test generation: Machine learning from human testers. 37th Annual Pacific NW Software Quality Conference.

Course evaluation

Give feedback on this course

Source of the figure: Rick and Morty 3x06 Rest and Ricklaxation

Do not be afraid to give feedback on this course



Source of the figs: Jeffrey Brown: Darth Vader and Son (2012)

Appendices

Main ToC

Overview of appendices

- Appendix A: April 1st : All Fools' Day – special edition
- Appendix B: Requirement engineering
  - I. RE in a green field investment
  - II. A case study of changing requirements
  - III. RE in brown field investments
- Appendix C: Evaluating course achievement
  - Appendix D: Homework: guidelines, example projects
  - Appendix E: Example exams
- Appendix F: Model 》 Test 》 Relax MBT framework
- Appendix G: Traceability table for different course materials
- Appendix H: Scheduling of topics during different semesters
- Appendix I: Syllabus
- Appendix J: List of abbreviations

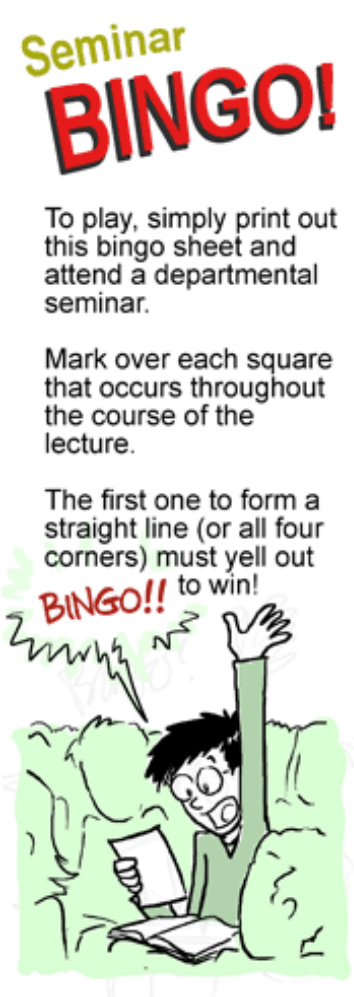
Appendix A: April 1st : All Fools' Day – special edition 4/1

- Standards:
  - IP over avian carriers:
    - RFC 1149 A Standard for the Transmission of IP Datagrams on Avian Carriers
    - RFC 2549 IP over Avian Carriers with Quality of Service
    - More info
    - Implementation:

  - RS 232
    - *Page 168th: "The transmitter sends a start bit to inform the receiver that a character is to be sent in the following bit transmission. This start bit is always a '0'. Next, 5, 6 or 7 data bits are sent as a 7-bit ASCII character, followed by a parity bit and finally either 1, **1.5** or 2 stop bits"*

Appendix A: April 1st : All Fools' Day – special edition 4/2

- Article generator for researchers
- Spuorious correlations
- Acronym generator
- How an anonymus 4chan forum poster gives a lower bound for a superpermutation problem?
  - *[1] Anonymous 4chan poster; Houston, Robin; Pantone, Jay; Vatter, Vince: A lower bound on the length of the shortest superpattern. 2018 October*
  - *[2] Engen, Michael; Vatter, Vincent (2021), "Containing all permutations", American Mathematical Monthly, **128** (1): 4-24*
- The Long Night: Modeling the Climate of Westeros



Appendix A: April 1st : All Fools' Day – special edition 4/3

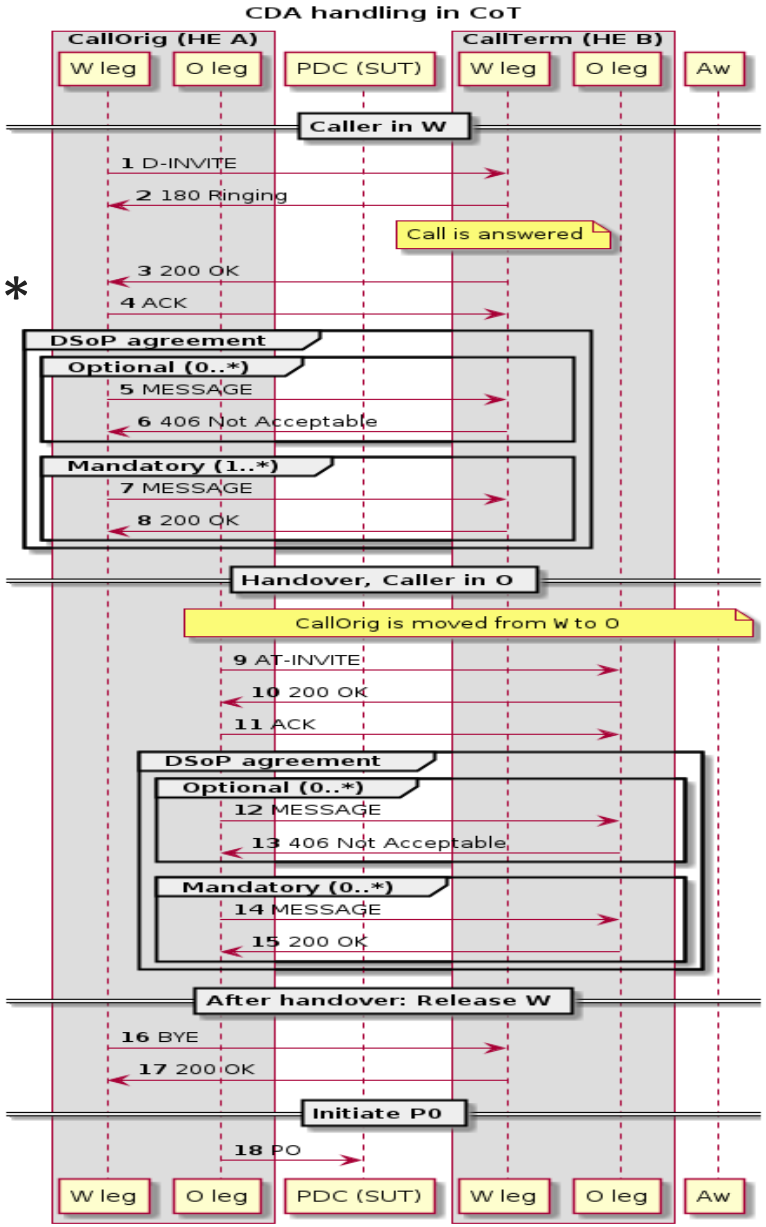
- Feature specification:
  Efficient CDA*
  handling with CooT**

* Common Diner Arrangement

** Colleagues of TST***

*** True System Team

Adobe Acrobat Document



- Bullshit generator for managers (in hungarian)
- Bullshit bingo

Off Site Meeting	Legacy	On the back end	E-business	Resource[s]
Demographic	Total Cost of Ownership (TCO)	Drop the Ball	Gap Analysis	Heavy lifting
Acquisition	Popup	BULLSHIT BINGO (free square)	Cost	Ecosystem
Search Engine Optimization	Segment	Open Source	Secret sauce	Agnostic
Strategic	Robust	Application Service Provider (ASP)	Phase	First to market

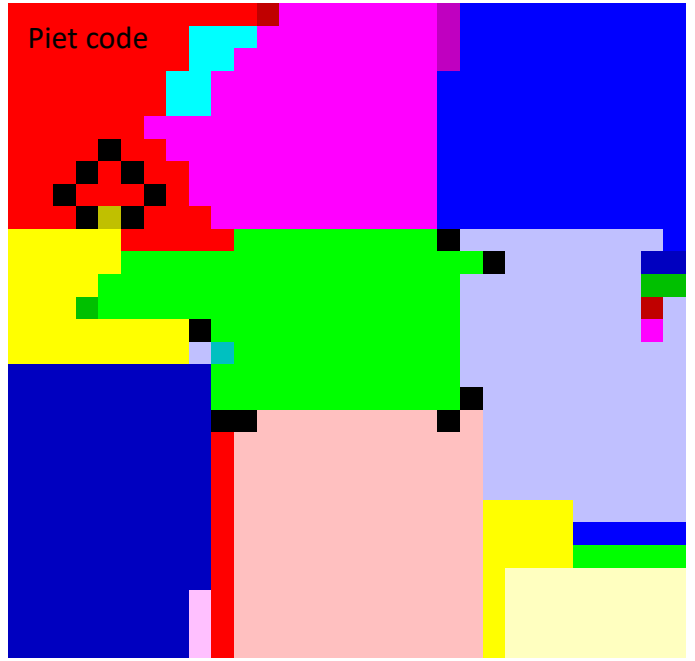
Appendix A: April 1st : All Fools' Day – special edition 4/4

- Programing languages:
  - Whitespace
  - Befunge
  - Piet
  - BrainFuck
  - Shakespeare
  - Rockstar

Whitespace code

Befunge code

```
      060p070                                                                  p'O80v
    pb2*90p4$4>                                                                $4$>v>
  v4$>4$>4$>4$>#                                                               ARGH>!
 <{[BEFUNGE_97]}>                                                              FUNGE!
 ##:-:##   #####*        4$*>4$      >060p>    60g80g -!#v_  60g1+     60p60v
 #vOOGAH                **>4$>^!!eg    nufeB^    $4$4$4 $4<v#<<v-*2a::   v7-1g<
 #>70g>90g-!           #@_^Befunge!!   123456    123456 VvVv!#!>Weird!   >0ggv*
  ^$4$4p07+1g07       ,a<$4<   <$4$4<  <$4$4<    <$4$4< <<#<*-=-=-=-=-*  -=-=v*
   ::48*-#v_>,4$>     4$4$4     $4$4$  4$4$4$    4$4$4$ 4$^*!*   XXXXXX   XXX>
     BOINK>$60g1-7   0g+d2*     %'A+,1 $1$1$1    $1$1$1 $>^<$     HAR!!!  8888
        Befunge_is   such_a     pretty langua    ge,_is n't_i     t?_It_  8888
           looks_so  much_l     ike_li ne_noi    se_and it's_     STILL_  '88'
Turing-     Complet e!_Cam     ouflag e_your    code!! Confu     se_the
hell_out   of_every  one_re     ading_ your_co  de._Oh, AND_y     ou.:-) ,o88o.
 Once_this_thing_i   s_code    d,_rea  ding_it_back_ver ges_on   the_imp 888888
  ossible._Obfusc     ate_the_obfus    cated!_Befunge_ debuggers_are__  888888
   your_friends!       By:_Alexios     Chouchou las... X-X-X-X-X-X-X!   888888
      -=*##*=-           \*****/        9797*  -=97=- !@-*=  *****     '"88P'
                                                      *!@-*
                                                      =*!@-
                                                      -=*!@
                                                      @-=*!
```

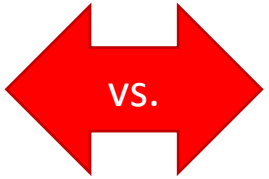


Appendix B/I: Requirements engineering in a green field investment 8/1
From the brainstorming to the prototype

1. Initial thoughts - Brainstorming
   - What we would like to achieve?
   - Define scope
   - Define possible customers
   - Define initial budget (and maybe some forecast for later stages)

2. Refine requirements
   - Risks:

Too general
→ too complex architecture → too complicated to implement a simple functionality
→ most of the possible functionalities will never be used



Too specific
→ bad architecture → dirty hacks over hardwired structure → hard to maintain
→ no future-proof (lack of important features)

Appendix B/I: Requirements engineering in a green field investment 8/2
From the brainstorming to the prototype

2. Refine requirements

- We should get to know possible customers, their goals, problems and current processes…etc.
  - Otherwise: Ivory tower: we make something that nobody really wants…

An example: Ikarus PALT*:

A big engineering achievement that was not applicable to the existing infrastructure

Source of figures: Ikarus archives

*PALT: Passengers And Luggage Together

Appendix B/I: Requirements engineering in a green field investment 8/3
From the brainstorming to the prototype

- An example of collecting requirements – in a structured document:

Requirement Specification of Tool Zebra

Contents

1 The Zebra system: our vision 4
1.1 The philosophy of the tool Zebra 4
1.2 Terminology 4
1.2.1 Behavior 4
1.2.2 Topology 4
1.2.3 User 6
1.2.4 Zebra 6
1.2.5 Zebroid 7
2 Requirements 7
2.1 Topology and behavior views 8
2.2 Setting of protocols and interfaces 13
2.3 Setting of user rights 17
2.4 Setting of behavior dependencies 19
2.5 Report generation 21
2.6 Miscellaneous 25
3 References 25
4 Abbreviations 25

1.2 Terminology

R_G_20

1.2.1 Behavior

The behavior is a dynamic description. It describes the interaction between the different participants of the system.

1.2.2 Topology

The topology is a static description, which defines the participants of the system and the interconnections among them. The parts of the Topology are described in Figure 1.

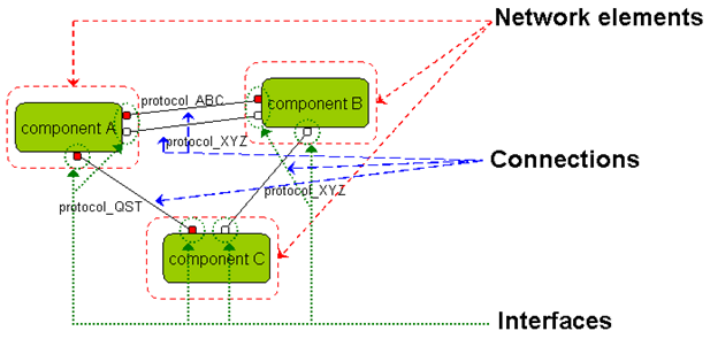


Figure 1: Parts of the topology

1.2.2.1 Network Elements

- The Network Elements are the nodes of the Topology. They are the participants of the interaction in the Topology.

R_G_21

Handle different abstraction levels.

One of the best properties of the engineer is the ability to select the right amount of details for a given task. It means that (s)he can consider a higher abstraction level if required, and not confused by any unimportant details. Thus, the tool should support the visualization and the editing in different abstraction levels. The tool should give the opportunity to the user to fold and unfold the given abstraction level on demand, to hide the details, which he is not interested in at a given time.

There are many requirements, which are related to this high-level requirement, see requirements R_G_21, R_U_05, R_U_06, R_T_03 and R_T_04 for further details.

Handle different abstraction levels of the topology. The tool should support to fold and unfold the different parts of the topology on demand.

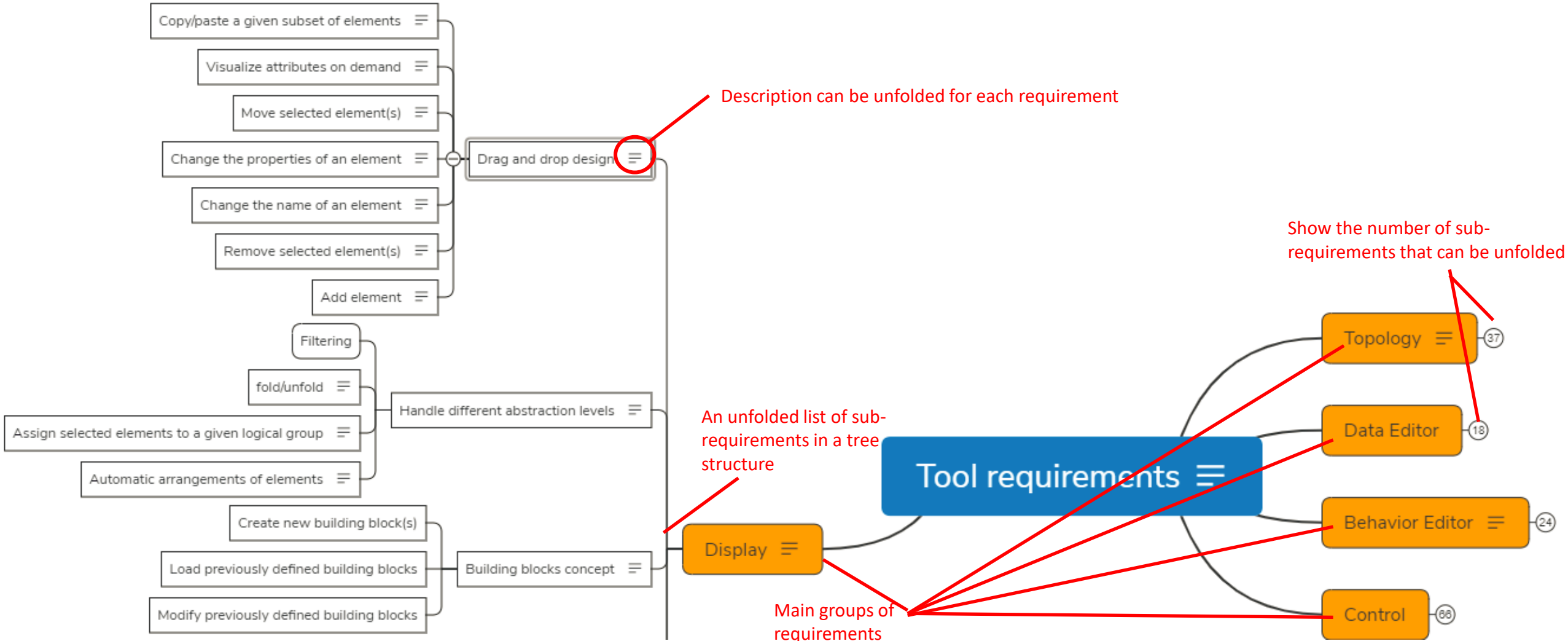
The user wants to display only that part of the network, which he is interested in. He wants to create higher and lower level view on demand.

An implementation idea would be if

(1) the user double clicks to an item, which he

Appendix B/I: Requirements engineering in a green field investment 8/4
From the brainstorming to the prototype

- An example of collecting requirements – tree structure (xmind):



Appendix B/I: Requirements engineering in a green field investment 8/5
From the brainstorming to the prototype

3. Creating a prototype
   - **Only for proof of concept!**

   - Should answer the following questions (2/1):
     - **What** would we like to achieve?
       - List of functionalities
     - **How** would we like to achieve the goal? – non-functional aspects
       - Usability ↔ user interface, assumptions about users, working process
       - Performance related aspects:
         - Responsibility
         - Designed workload
         - Scalability
         - How to handle overload…etc.

         ↔ software architecture & required hardware
       - Security aspects ↔ architecture

Appendix B/I: Requirements engineering in a green field investment 8/6
From the brainstorming to the prototype

3. Creating a prototype
   - Should answer the following questions (2/2):
     - How should we provide the expected **quality**?
       - Manual testing for explanatory testing
       - A few proof-of-concept tests
         - Unit, integration, system levels
         - Functional and non-functional (performance, (G)UI, security…etc.)

Appendix B/I: Requirements engineering in a green field investment 8/7
From the brainstorming to the prototype

- Prototype:
  - **Role: Proof-of-concept**
  - Not an implementation code base for the final product!

- From most of the prototypes no real product has been developed due to the following reasons:
  - Wrong assumptions, when defining requirements and scope
  - Wrong initial thoughts about possible customers and/or their needs
  - Wrong assumptions about budget
  - The protype showed that the development cost and/or time would be too high
  - Organizational changes in the company resulted in cost cut / project closure
  - Similar product has been developed meanwhile in parallel

Appendix B/I: Requirements engineering in a green field investment 8/8
From the brainstorming to the prototype

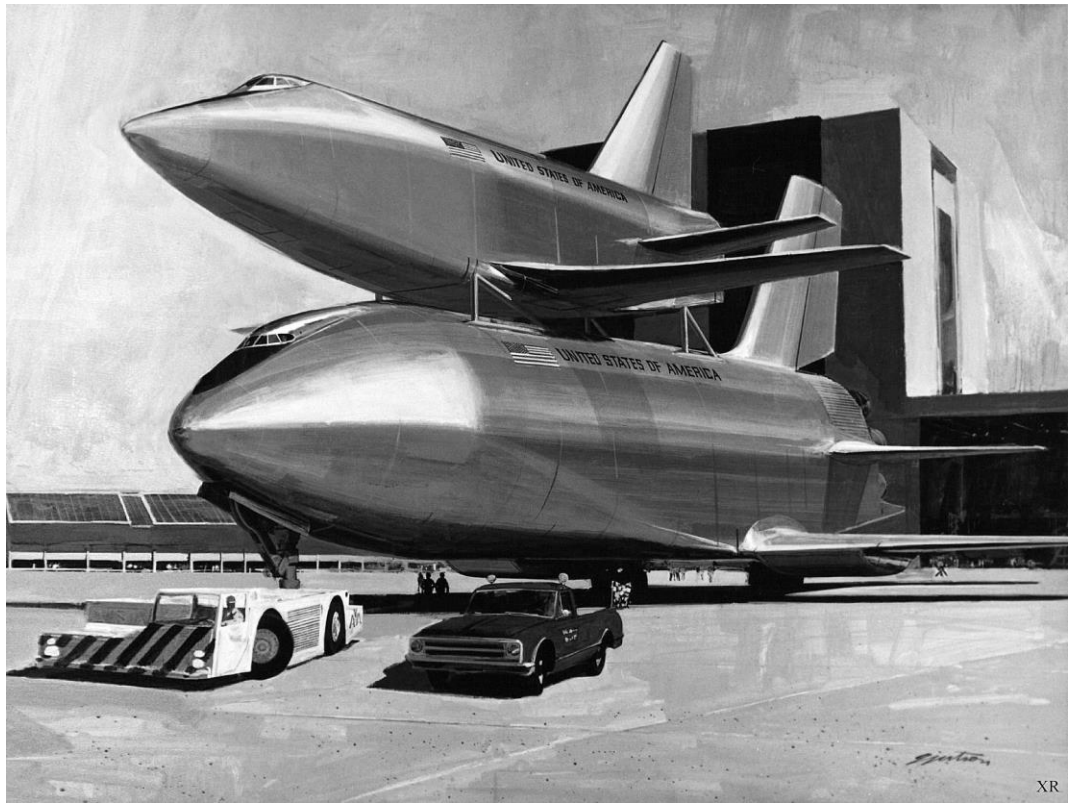
If we succeed than comes…

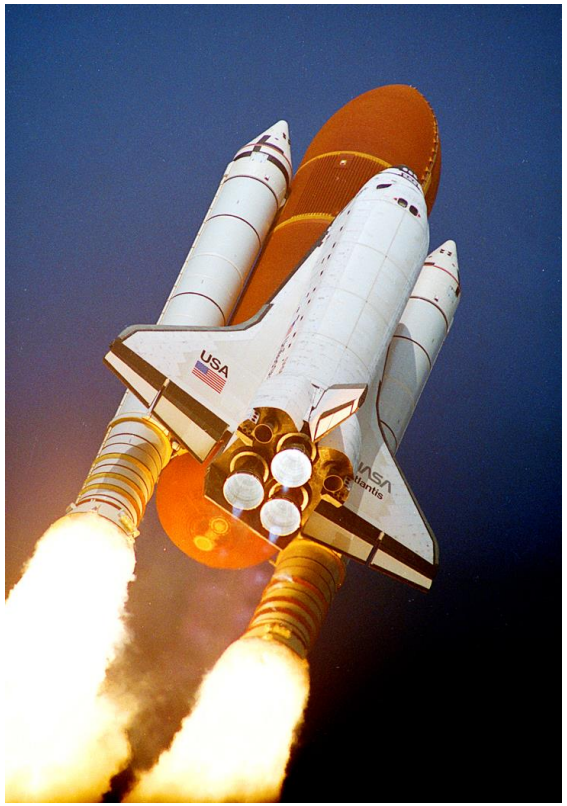
4. Productification
   - But many reviews before this step:
     - Technical reviews (architecture)
     - Management reviews at different levels (financial, customer…etc. aspects)

Project vs. product

	Project	Product
Generic	Unique, customer specific	Generic
Time	Has beginning and end date	Permanent (until phase out)
Planning	One-step/Predictive planning	Iterative/adaptive planning
Input	Project requirements	Evolving customer needs

Appendix B/II: A case study about changing requirements 9/1





The true story of the genesis of the Space Shuttle

References:

- David Baker: NASA Space Shuttle. 1981 onwards (all models). Owner's Workshop Manual. Haynes. 2011.
- Wikipedia: Space Shuttle program, Space Shuttle design process, Criticism of the Space Shuttle program

Appendix B/II: A case study about changing requirements 9/2

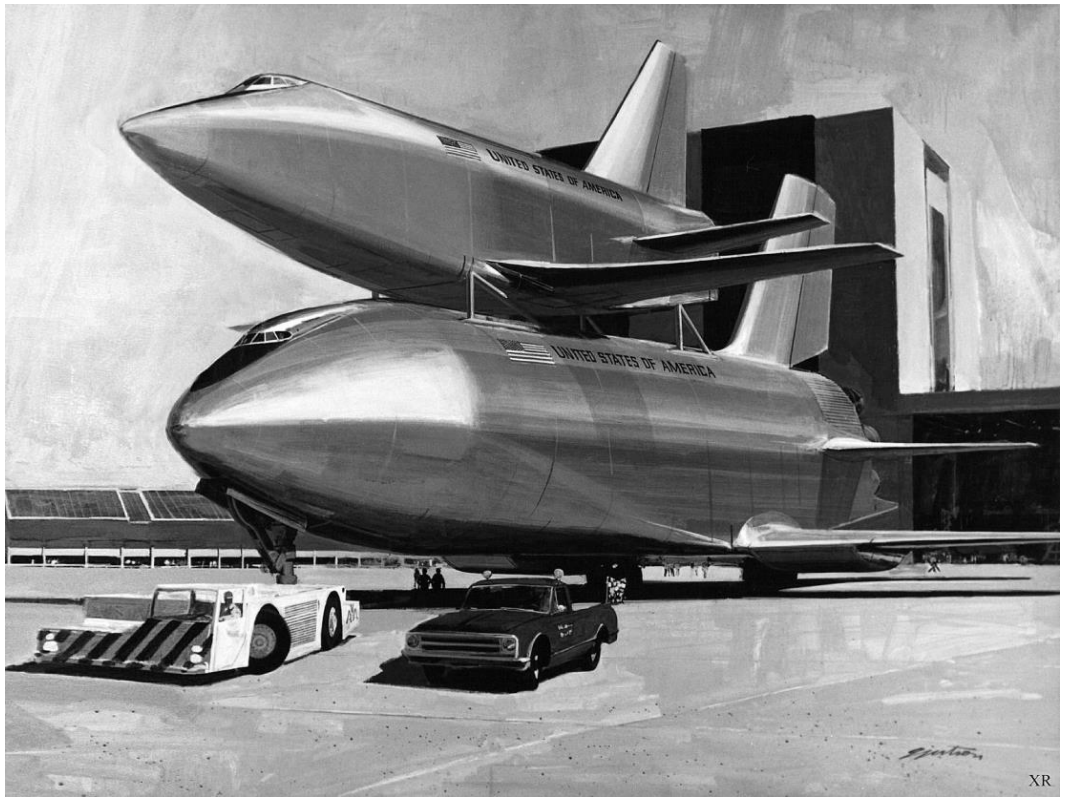
Background:

- After the Apollo (Moon landing program), significant cut on NASA's budget
- NASA plan to develop a fully reusable system – a „Shuttle" – to make space travelling significantly cheaper

Initial design (1970 Phase B studies):

- Fully reusable system:
  - 1.700 ton fly back manned booster with 12 rocket engines (with liquid fuel) and wings
  - 380 ton orbiter with 2 rocket engines
  - Orbiter's Payload capacity: 11 tons to LEO*

    * LEO: Low-Earth Orbit
- This solution was too costly to develop…



Appendix B/II: A case study about changing requirements 9/3

Investigate different concepts:

1. Fully (booster + orbiter) reusable systems
2. Expandable tanks
3. Expandable boosters

…etc

Catch-22:

- Lowest cost-per-flight solutions requires highest development cost
  - NASA have insufficient budget
- If decrease development cost, it results in a higher cost-per-flight
  - Controversial to the initial goal

Appendix B/II: A case study about changing requirements 9/4

Optimizations:

1. Expandable tank has been selected to decrease development cost (smaller orbiter would be enough)
   → compromise: not fully reusable system
2. SRBs (solid rocket boosters) proposed instead of liquid propellant ones to decrease cost-per-flight
   - Advantages:
     - **Simple and cheap**
     - Much easier to handle, no fueling needs before launch
   - Disadvantage:
     - Less efficient than liquid propellant rockets
     - **Once ignited, can not be stopped** – 1st compromise on safety

   (NASA had a rule before to not use them for manned space flights)
3. Insufficient thrust can be gained from solid rockets to lift-off the entire system
   → instead of the usual serial concept, parallel concept is selected for solid rockets
4. Recoverable boosters proposed to decrease cost-per-flight

Appendix B/II: A case study about changing requirements 9/5

Wait, initial design has been altered!

Do we need the cheaper solid rocket boosters if they are reusable?

Appendix B/II: A case study about changing requirements 9/6

Compromise of partners / payload capacity:

- The development cost was still too high → NASA found a partner (USAF*) to share the costs
- What payload is required?**
  - NASA: 6,8 tons to LEO*** (for satellites)
  - USAF*: 18 tons to polar orbit ≈ 30 tons LEO*** (for military satellites)
  - NASA later: 20 tons to LEO*** (to build Freedom space station from modules)

* USAF: US Air Force

** **The most important question when designing the Space Shuttle…**

*** LEO: Low-Earth Orbit

Appendix B/II: A case study about changing requirements 9/7

Canceled Safety functions:

- Liftoff – designed due SRBs:
  - Blow out port for boosters to separate in case of failure during ascending

    → cancelled due weight
  - Abort solid rocket motors

    → cancelled due simplicity

- Landing:
  - Turbofan engines that pop out from a compartment of rear payload bay at landing

    → cancelled due weight and volume

Appendix B/II: A case study about changing requirements 9/8

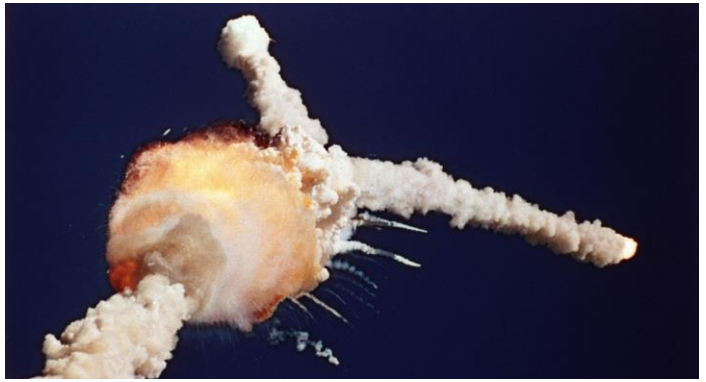
The usage of Space Shuttle - facts:

1. Due SRBs and parallel design, two Space Shuttle disasters:
   - Challenger in 1986

   → decrease flight intensity

   → US Air Force back out from project

   → increase cost-per-flight



   - Columbia in 2003

   → increase cost-per-flight

   → decision about the retirement of the Space Shuttle fleet

Appendix B/II: A case study about changing requirements 9/9

2. Except HST* in 1990, only after 1998 (17 years after first flight!) NASA uses the possible payload capability of the Space Shuttle, when building the ISS**

* HST: Hubble Space Telescope

** ISS: International Space Station

3. The Vandenberg Space Shuttle launch pad build for US Air Force has never been used

4. The initial plan to send Space Shuttle into space bi-weekly has been never achieved

   → the cost-per-flight has been even higher compared to simple, expandable rockets!

5. After Space Shuttle era:
   - More simple designs
   - Concentrate on liquid propellant rocket engines
   - Concentrate on less payload capacity

Appendix B/III: Requirements engineering in brown field investments 8/1

From the customer requirements to the specification through an example process

An overview:

1. CR from customer
2. Early estimation
3. Task clarification → Feature Specification
4. Design documents (architectural, test…etc.)
5. Implementation
6. Tests
7. Documentation
8. Deployment

Focus on this topic

Scheduling of CR at any stage is made by PO according to priorities / available resources / output of previous stages

CR: Change Request
PO: Product Owner

Appendix B/III: Requirements engineering in brown field investments 8/2

From the customer requirements to the specification through an example process

1. Customer requests a change

- Submits a CR (change request) in a CR management system (example tool: Tuleap)
- Describes the requested functionality from the customer perspective
  - May be ambiguous, may not be self consistent, may lack of important details…etc.

Artifact number	Title	Customer
Artf010416	Efficient CDA handling in CoT	XYZ
CoT should handle CDA: A proper mechanism need to be implemented for D-INVITE, message exchange regarding DSoP, and initiating PO. This mechanism should be implemented between HEs of LoSP. Note that a WO handover is also necessary.		

Appendix B/III: Requirements engineering in brown field investments 8/3

From the customer requirements to the specification through an example process

2. Based on the CR an early estimation is made

- By a business analyst/requirement engineer/system architect
- Quickly with limited efforts
- Output:
  - A quick overview of the topic, affected part(s) of the system, possible bigger tasks
  - Polo size: S/M/L/XL

    → determines the rough timeframe in mhrs* required for development, tests, documentation and deployment

    → each domain/company/company units may have different timeframes for each polo size

Polo size	mhrs
S	0-40
M	41-80
L	81-200
XL	200+

Polo size	mhrs
S	0-200
M	200-500
L	500-2000
XL	2000+

* mhrs: men hours

Appendix B/III: Requirements engineering in brown field investments 8/4

From the customer requirements to the specification through an example process

3. Task clarification with customer

- Iterative process
- Transparency - CR management system:
  - The communication should be tracked
    - To avoid later misunderstandings
    - To provide the ability to involve new people from both sides
  - The status of the CR should be updated
- Always check related standards!
  - Conformance to related standards is important
  - If we must deviate from the standard, then write down the reason behind it & the possible risks
- Always checks related existing features!
  - Backward compatibility is important

CR: Change Request

Appendix B/III: Requirements engineering in brown field investments 8/5

From the customer requirements to the specification through an example process

3. Task clarification with customer

- Output: Feature Specification
  - Describes the required functionality in an unambiguous, self-consistent way that can be given to the developers/testers/technical writers

Artf010416: Efficient CDA handling in CoT

Keyword(s)

D-INVITE, CoT, DSoP, WO handover

Abstract

This feature specification is intended to be an agreement between the different CoT of OWU on the detailed requirements related to CDA.

Contents

1 Introduction 1
1.1 Statement of problem 1
1.2 Input Documents 1
1.3 Scope 2
1.4 Abbreviations 2
1.5 Glossary 3
2 Expected System Functionality and Characteristics 3
2.1 Use Case Impacts 3
2.2 System Impacts 7
2.3 Test Analysis 7
2.4 Non-Function Requirements 7
2.5 Customer Impacts 7
2.6 Backwards Compatibility 7
2.7 Technical Risks 7

1 Introduction

1.1 Statement of problem

Nowadays, the CDA of CoT is a more and more pronounced problem that has to be solved with a proper method. The D-INVITE, the message exchange regarding DSoP and PO process should be implemented in the CoT both for W and O legs.

1.2.2 Standards, Specifications and Studies

Title	Number	Revision	Comment
[I] A Standard for the Transmission of IP Datagrams on Avian Carriers	RFC 1149	-	
[II] IP over Avian Carriers with Quality of Service	RFC 2549	-	An enhanced version of RFC1149

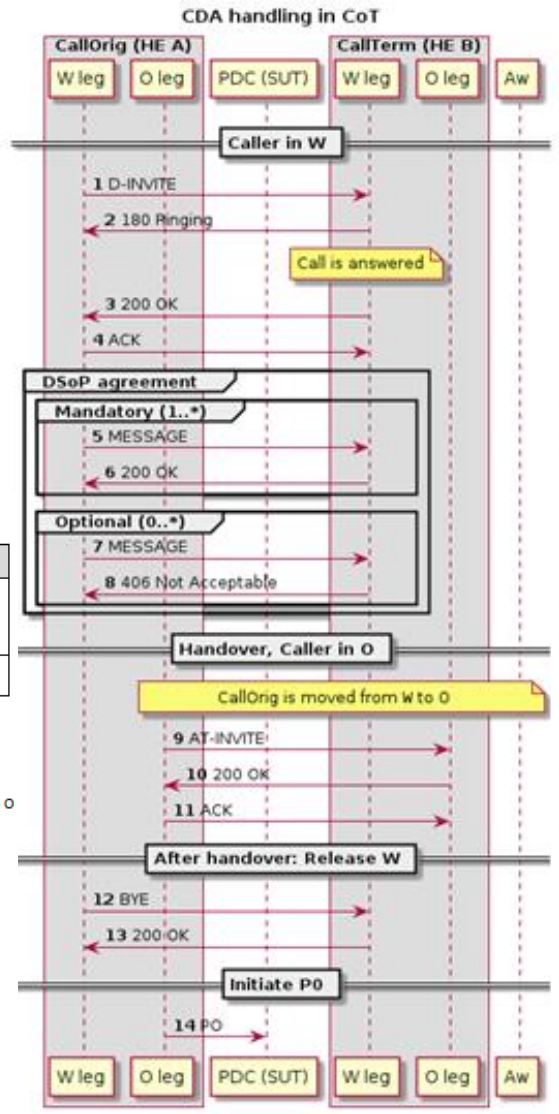
1.3 Scope

The requested feature is the interoperability between the HEs of CoT to perform CDA with DSoP.
The CDA functionality enables handover from a W network to legacy O networks and a fallback from O to W before W leg is released (note that no fallback opportunity is required after the W leg is released).

1.4 Abbreviations

CDA	Common DA
CoT	Colleagues of TST
CFR	Chinese Food Restaurant
DA	Dinner Arrangement
DB	DataBase
DSoP	Detailed Specification of Pizza

2.1.1.1.3 Call Flow



2.2 System Impacts

It is definitely necessary to write an IP for this feature.

For W leg either whitespace [3], gothic [1] or usual character coding [2] can be used.

Please also take attention to create the *CDA handling in CoT* part of TitanSim Online help; significant documentation work is required.

2.3 Test Analysis

The feature cannot be tested in lab1 due to lack of CDA functionality, it can be tested only in CoT environment.

Selftest must to be written for the use cases described in this document, and in the future it is advised to create test also in ONTE.

However, the power of manual testing should not be underestimated, even the postcondition of PO process should be tested.

2.4 Non-Function Requirements

2.4.1 Capacity and Performance

No request has been received

Significant performance drop is expected if W legs of HE A and B are implemented over [I] or [II] instead of legacy IP networks.

2.5 Customer Impacts

No customer impacts on existing functionalities.

2.6 Backwards Compatibility

No backward incompatible issue is foreseen.

2.7 Technical Risks

Problems may occur in the interpretation of the received message is expected if whitespace [3] character coding is used over [I] or [II]. In this case the using of other character coding methods or transport layers is proposed.

Appendix B/III: Requirements engineering in brown field investments 8/6

From the customer requirements to the specification through an example process

3. Task clarification with customer

- Output: Feature Specification
  - Must be accepted by both sides:
    1. Reviewed internally
       - Participants:
         - Business analysts/system architects
         - Developers (who have competence in the related part of the software)
         - Test responsible person
       - Review responsible:
         - Screening
         - Moderate review, give verdict (accepted / accepted with comments / rejected)
         - Check afterlife based on verdict (check modifications to comments / 2nd turn of review…etc.)
         - Update status on CR management system
    2. Approved by customer

Appendix B/III: Requirements engineering in brown field investments 8/7

From the customer requirements to the specification through an example process

Possible risks:

- We want that feature right now!

  → Hardwired, too specific solutions that are hard to be generalized or maintain

- Give too big requirements at one step without priorities and schedule

  → Will be never finished

- Requirements that do not conform with corresponding standards

  → Compatibility problems at a later phase, working mode-switch and other dirty hacks

Appendix B/III: Requirements engineering in brown field investments 8/8

From the customer requirements to the specification through an example process

Possible risks:

- Problems with documentation

1. No proper documentation of task clarification discussions with user

   → misunderstandings at deployment, blaming each other

   → changing requirements

   → delay of delivery, more cost effect

2. No proper documentation of the delivered feature

   (missing or incomplete user / developer / architectural documentations)

   → customer/ developer unable to use the feature properly, requirements and design decisions are mixed up

   → reverse engineering (in code / standards / old e-mail exchanges with customer)

   → try to sort out related documents and get approval by customer

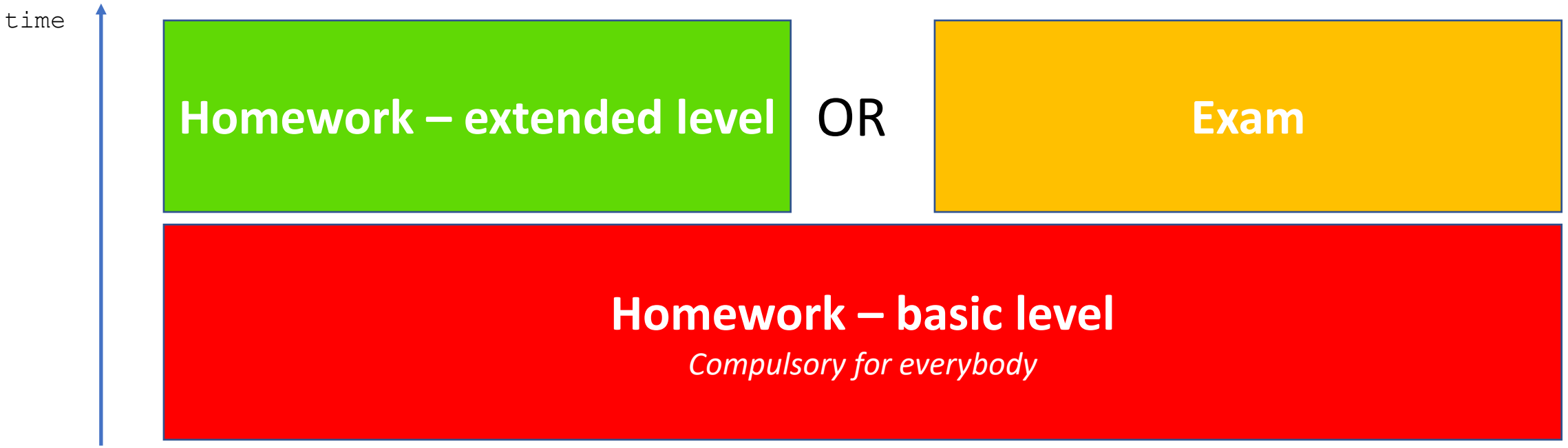
   → huge additional costs, loss of credibility

3. No traceability exists between specification – code – test – documentation 4-tuple

Feature group	Feature name	Uniq ID	Role	Subfeatures (if exist)	Related parameters	Related Standard	Link to user help	Feature Specification	Related tests

Appendix C: Evaluating course achievement 2/1

- Course participants:
  - Teacher (1):
    - Works with an idealistic learn maximization function
  - Students (1..*):
    - Some of them works with a pure energy minimalization function
    - Some of them works with a function that search an optimum between learn maximization and energy minimalization
- How to achieve a grade?



Main ToC

Appendix C: Evaluating course achievement 2/2

- **Homework – basic level**
  - **Compulsory for everybody**
  - Required parts:
    I. **Specification for an own problem**
    II. **Model creation for the specification**
      - Pls. give a name to your model that reflects somehow to your topic *(i.e. not homework.json or test.json)*

    III. **Applying at least 5 different test generation algorithms/settings on the specification model**
      - Just random-based test generation algorithms won't be accepted
      - If random is applied, pls. execute it minimum 5 times and present min/max/average values for main results
  - Required outcome:
    - **Documentation** of the specification and how the formal model and the tests for symbolic execution are created
      - **Pdf** format and in **English**
      - Do not forget to mention the **applied limitations, the used tools and their versions and summary tables for the main params and results of applied test generation algorithms/settings**
    - The **formal model and generated test suites** should be also included!
      - Keep attention to **input and output symbols of the model**!

No homeworks are accepted after deadline!

- Note that **only model-based testing topics are accepted**
- Then the students can choose between to extend their homework to test an actual system or to take an exam to achieve a grade

Main ToC

Appendix D: Homework – Guidelines 6/1

Opportunity to create an **extended level project homework** to achieve an offered grade

No homeworks are accepted after deadline!

Required parts:

I. Specification for an own problem

II. Model creation for the specification

III. Test generation

i.e. the same as for basic homework (but can be refined if required)

IV. Source code of the SUT *(or link to the SUT)*

V. Adaptation code (with it's description*)

*** The used tools/technologies, the structure and the main functions of the adaptation code and it's working through at least one use case should be discussed**

V. Detailed test report and evaluation of the results

Note that the proportion of these parts can vary:
- For example, if you try other tools than the one introduced in the course, then the specification can be simpler

OR
- if you have a very good idea and create a model based on that, you may investigate less different coverage
- …etc.

But pls. always discuss your idea and the proportions of the required parts in advance!

SUT: System Under Test

Main ToC

Appendix D: Homework – Guidelines 6/2

extended level

Required outcome:

- **Documentation** of the project:
  - **Pdf format and in English** with related tables and figures
- **The model, the adaptation code and the generated test cases of the project** should be also included
- **Live presentation of the project in the last week**

From received homework summaries may have been created and published in course slides, so **the homework should not contain any confidential information**

Quality homeworks will be offered a grade which can either be accepted or denied by the student.

Should the offered grade be denied, the exam should be taken in order to achieve the course.

Main ToC

Appendix D: Homework – Guidelines 6/3

basic level & extended level

How should I start selecting my own topic*?

- Think about problems that can be described with *distinct states* and *limited number of variables*
- Read *at least* the following topics in this material**:
  - FSM and EFSM models, FSM-based testing (test coverage criteria, TT method), MBT: differences between theory and practice, free MBT tools (GraphWalker demo), advantages and disadvantages of MBT *(optional: Model ⟫ Test ⟫ Relax MBT FW)*
- Check homeworks from previous years ***:
  - Advised to be checked: homeworks with ★
  - Also check uploaded homework documentation files pdfs in OneDrive
  - It is advised to start checking with the most recent ones

* Note that these advices are only applicable for (E)FSM-based topics, however other topics can be selected! For more information pls. consult with your lecturer!

** Note that course videos also exist. For traceability between videos and slides check this table

*** Note that the documentation of some homeworks can be also checked in the folder of course materials

EFSM: Extended Finite State Machine
FSM: Finite State Machine
FW: Framework
MBT: Model-based testing
TT: Transition Tour

Main ToC

Appendix D: Homework – Guidelines 6/4

extended level

What can be the actual SUT?

- An already existing tool – for example see HWs VI, XII, XIII, XVI, XVIII, XIX, XXIII, XXXIV, XXXIX, XXXXII, XXXXV, XXXXVI

OR

- Your own implementation – for example see HWs VII, VIII, X, XI, XVII, XX, XXXVI, XXXVIII
  - Completely new implementation

    OR
  - Implementation created originally for BSc/MSc thesis / OTDK work / other projects
    - In the latter case please consult with your teacher in advance and write down the boundaries of your existing and new works

OTDK: Országos Tudományos Diákköri Konferencia →National Conference of Scientific Students
SUT: System Under Test

Main ToC

Appendix D: Homework – Guidelines 6/5

extended level

How can I write a **glue (adaptation) code**?

- Depending on the selected problem, you can test the SUT either via GUI, API or CLI
  - There are many opportunities, for inspiration you can check the documentation of some previous homeworks in the folder of course materials in OneDrive
  - Also note that there is a summary table of previous homeworks, which list different approaches
  - **Pls. allocate enough time to select the appropriate tool for adaptation code writing**, check current trends
    - Some hints for GUI testing:
      - Playwright vs. Selenium vs. Cypress comparisons: 1st, 2nd, 3rd …etc.
      - First make a proof of concept if a given tool is selected to find out the followings:
        - How would you like to access an element? *(by id or xpath or search graphically?)*
        - Is a given tool support auto-wait?
        - Compatibility
        - Speed

API: Application Programming Interface
GUI: Graphical User interface
CLI: Command Line interface
SUT: System Under Test

Main ToC

Appendix D: Homework – Guidelines 6/6

extended level

Checklist before homework submission:

1. Did you attach all the required files?
   - *Documentation, model, SUT/link to the SUT, adaptation code, generated test suites (and test summary files if applicable)*
2. Did the documentation format properly?
   - *Pdf in English, contains your name and the name of the topic,*
   - *Structured well, grammar and typos checked before submission*
   - *Figures / tables contains label and references from text*
   - *Contains the required sections (Intro, specification, model description with limitations, test generation, adaptation code description with at least a use case example, detailed test report and evaluation of the results, conclusion)*
   - *The document should be self-consistent, should not contain unresolved abbreviations, should contains links to tools*
3. *Does the documentation answer the following questions clearly?*
   - *What is the SUT? What are the tested functionalities?*
   - *Type of the model? What are the limitations (if there is any) in the tests compared to the functionalities of the SUT?*
   - *Used tools and their versions*
   - *Adaptation code: used language, used tools and techniques*
   - *What was the results of testing?*

- *Note that one can use the optional, early feedback deadline*

Main ToC

Appendix D: Homework – Deadlines & proposed scheduling

Just an advice / Strongly recommended / MANDATORY	Deadline	Task
MANDATORY (for everybody)	**Week 5th (in 1st lecture timeslot)**	**Discussing the selected task (Basic/Extended HW)**
Just an advice	Around week 6th	Showing the model, ideas about testing tools
MANDATORY (for everybody)	**Week 8th Monday 8.00 CET**	**Handing the basic level HW entire document**
Strongly recommended (for Extended level)	**Week 10th Monday 8.00 CET**	**Handing in the entire document if you want early feedback about the proposed mark** (you will have a chance to update it!)
MANDATORY (for Extended level)	**Week 12th Monday 8.00 CET**	**Extended HW: Handing in the entire document**
MANDATORY (for Extended level)	**Week 14th (in 1st lecture timeslot)**	**Extended HW: Give live presentation**

Tasks that are selected and homework that are received after the discussed mandatory deadlines will not be accepted!

Advice: Make a time plan and do not try to do everything in the last week. In my experience from past semesters all homeworks that were started 1 week before the deadline never succeed due to different problems (related to model making or coding, compatibility problem of the selected tool...etc.)
Keep in mind that there are consultations opportunities!

CET: Central Europe Time

Main ToC

Appendix D: Overview of example homeworks 2/1

Note that this overview only contains selected homeworks that have an actual adaptation code

SUT	Model	Model-based testing tool	Adaptation code	Star	Notes
Lot-Art: website for art and luxury items sold	EFSM	Custom solution (PHP Script)	PHP Script	Yes	
GTFS Aggregator travel planning application (Java FX client)	EFSM	GraphWalker, Harmony	Java (GraphWalker tests), Harmony (login), Travis CI (build test)	Yes	
NAV Online Invoicing System API	EFSM	GraphWalker	C#, using FlaUI + Nunit	Yes	
Movie Organiser website	EFSM	GraphWalker (symbolic test exec. only)	Laravel Dusk	Yes	
Autonomous Multi-robot System Simulation	EFSM+DB	GraphWalker	C#, Altwalker + Nunit	Yes	Positions of agents, shelves…etc. read from an input file at each step
Airline path optimization	EFSM	GraphWalker	Java, GUI testing uses Selenium with chromedriver	Yes	
Github Pull Request lifecycle	EFSM (t)	fMBT	Using the CLI + web interface of Github with Selenium	Yes	EFSM (t): Possible variable combinations are coded into states due fMBT
CNC Controller	FSM	GraphWalker	Test code in Java → C++ code adapted with a dll file, JNA	Yes	
Arcade Game	EFSM	GraphWalker	Java, GUI: using Selenide	No	
Warehouse management application	EFSM	GraphWalker (symbolic test exec. only), Harmony	Harmony	Yes	
Docker Engine API	EFSM (t)	fMBT	Python, connects to the Docker Engine through HTTP	Yes	EFSM (t): Possible variable combinations are coded into states due fMBT
Wordle (native Julia application)	EFSM	GraphWalker	Java testing interface	Yes	
Trello (web UI)	EFSM	GraphWalker	Java, GUI: using Selenide	Yes	
Google Keep API	FSM	Model 》 Test 》 Relax	Python	Yes	
Car Rental Registry application	EFSM	GraphWalker	Winium API calls, UISpy tool for element identifications	Yes	
Websigno (web application)	EFSM	GraphWalker	Java, GUI: using Selenium	Yes	
Pazaak Card Game (WPF application)	EFSM	GraphWalker	Winium API calls, UISpy tool for element identifications	Yes	
Neovim (Vim-based text editor program) modes	FSM+EFSM	Model 》 Test 》 Relax	Python, using Neovim API for running RPCs through a TCP socket	Yes	This example is included in the Model 》 Test 》 Relax repository
Archive of Our Own website	EFSM	GraphWalker	C#, GUI: using Selenide, Junit	Yes	
Telegram Client API	EFSM	Model 》 Test 》 Relax	C#, using Telegram API (TelegraApi + TDLib + WTelegramClient)	Yes	
Twitter API v2	FSM	Model 》 Test 》 Relax	Java, using Twitter API v2	Yes	
ELVIRA website	FSM	Model 》 Test 》 Relax	Python, GUI: using Selenium	Yes	
ASP.NET Webshop	FSM	Model 》 Test 》 Relax	C#, GUI: using Selenium with Chromedriver	Yes	This example is included in the Model 》 Test 》 Relax repository
AlcoholLimit Android application	EFSM	Model 》 Test 》 Relax	Python, using Appium	Yes	
Instagram	FSM	Model 》 Test 》 Relax (test gen.), GraphWalker	Java (runner: GraphWalker CLI), GUI: Selenium, OkHttp, Junit	Yes	Tests generated by Model 》 Test 》 Relax are imported back into GW
Train ticket reservation application	FSM	Model 》 Test 》 Relax	C#, Windows Application Driver	Yes	
Dungeon crawler console game	EFSM	Model 》 Test 》 Relax	C++ and python	Yes	

Main ToC

Appendix D: Overview of example homeworks 2/2

Note that this overview only contains selected homeworks that have an actual adaptation code

SUT	Model	Model-based testing tool	Adaptation code	Star	Notes
OpenIddict Oauth2.0/OpenId Connect library	EFSM → FSM	Model 》 Test 》 Relax	C#, using API	Yes	This example is included in the Model 》 Test 》 Relax repository
Git	FSM	Model 》 Test 》 Relax	Python, CLI, full pipeline has been implemented	Yes	This example is included in the Model 》 Test 》 Relax repository
Cookie Shop Backend	FSM	Model 》 Test 》 Relax	Python	Yes	Investigated cases if an error is injected to the (1) SUT, (2) spec model, (3) adaptation code
MadTycoon tycoon simulator game	EFSM → FSM	Model 》 Test 》 Relax	GUI testing was implemented by Java functionalities	Yes	
Web application for betting odds	FSM	Model 》 Test 》 Relax	Java, GUI is tested with Selenium, API with Postman	Yes	
Reddit post-related functionalities	FSM	Model 》 Test 》 Relax	Python, using Reddit API	Yes	
SOCKS5 server	FSM	Model 》 Test 》 Relax	Python, using API	Yes	
Unity Asset Store	EFSM	Model 》 Test 》 Relax	TypeScript, GUI: using Selenium	Yes	
Google Calendar API	FSM	Model 》 Test 》 Relax	Python, using Google Calendar API	No	
Spotify	FSM	Model 》 Test 》 Relax	Python, using Spotify Web API (Node.js)	Yes	
Steam market Place	FSM	Model 》 Test 》 Relax	Python, GUI: using Selenium	Yes	
MisEnapló	FSM	Model 》 Test 》 Relax	Javascript, GUI: using Selenium with Chromedriver	Yes	
Wikipedia	EFSM → FSM	Model 》 Test 》 Relax	Python, GUI: using Selenium with Chromedriver	Yes	
Kubernetes Lifecycle Management	FSM	Model 》 Test 》 Relax	Java, CLI (kubectl commands)	Yes	
OrangeHRM human resource management system	EFSM → FSM	Model 》 Test 》 Relax	Python, GUI: using Selenium with Chromedriver	Yes	
Automatic Transmission controller	EFSM → FSM	Model 》 Test 》 Relax	Java, GUI: AssertJ Swing and custom functions	Yes	
Ory Hydra Oauth2.0 and OpenID Connect server	FSM	Model 》 Test 》 Relax	C#, using API	Yes	
X Synchronization Extension Library	EFSM → FSM	Model 》 Test 》 Relax	C++, using xcb X protocol library	Yes	
Discord	EFSM → FSM	Model 》 Test 》 Relax	Python, using API (GUI testing with Selenium just for one Scenario)	Yes	
Jira	EFSM + FSM	Model 》 Test 》 Relax	Python, using API	Yes	
Farmer2Customer market	EFSM + FSM	Model 》 Test 》 Relax	Python, using API	Yes	
Job Hunting Platform	EFSM → FSM	Model 》 Test 》 Relax	JavaScript, GUI: using Playwright	Yes	
Food ordering	EFSM → FSM	Model 》 Test 》 Relax	C#, GUI: using Playwright	Yes	
PriceWatcher+ Shopping Assistant	FSM	Model 》 Test 》 Relax	C#, GUI: using Selenium	Yes	
Online Animal Shelter	FSM	Model 》 Test 》 Relax	JavaScript, GUI: using Playwright	Yes	

Main ToC

Appendix D: Example homework I. 3/1

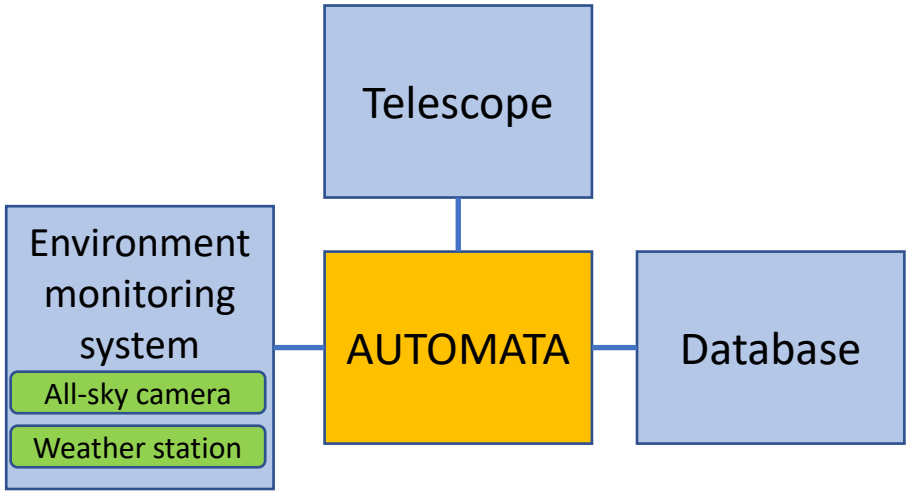
Spring 2018/2019

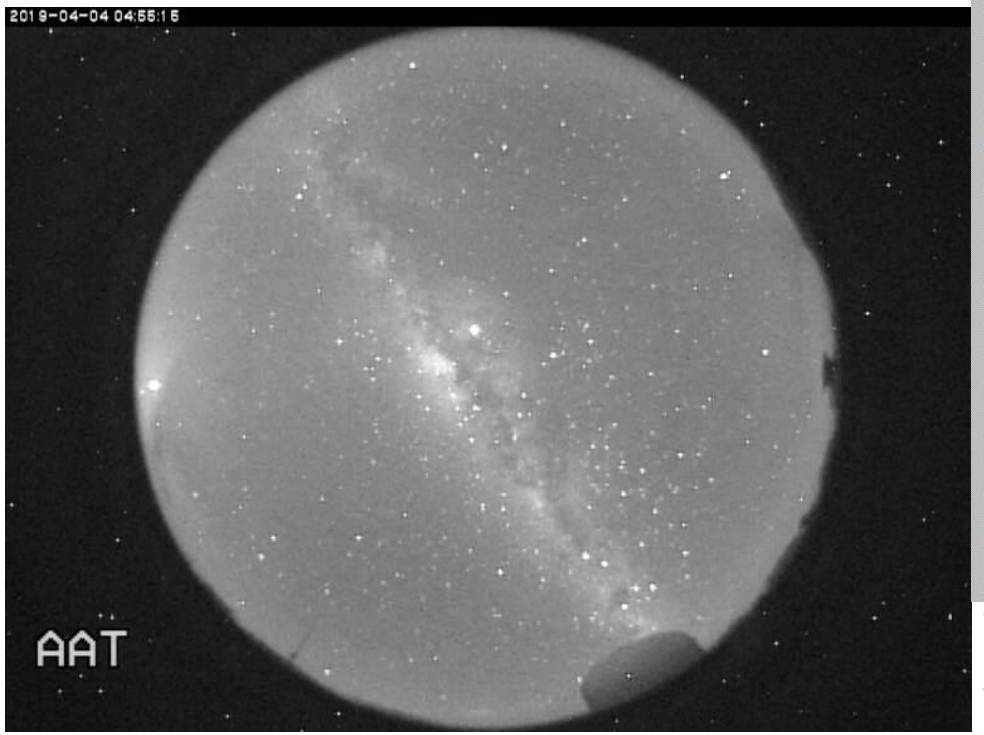
Ottó Hanyecz:

AUTOMATA: AUTOmatic astronoMical observations and dATA processing

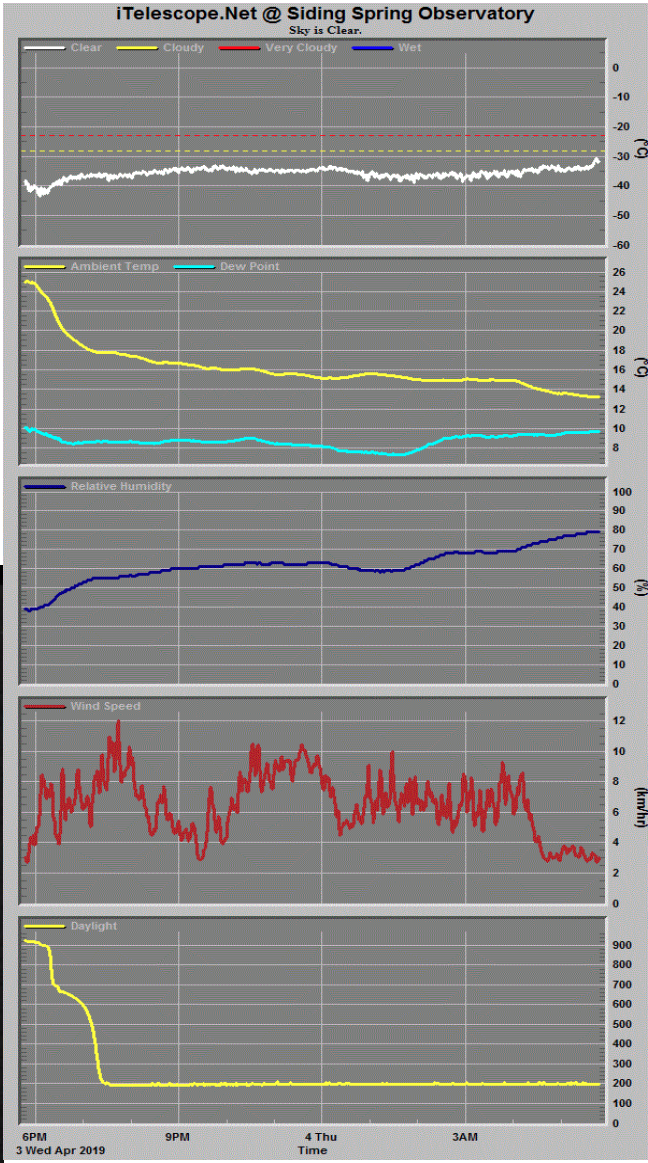
- Simple description of an automatic system which performs automatic astronomical observations, and processes the data automatically
  - Enables measurements if the weather conditions are appropriate and the telescope is tested successfully
  - Measurements can be stopped

  (if conditions are not appropriate for measurements) and continued later





Clear sky over Siding Spring Observatory, Australia at April 4, 2019. (Source: itelescope.net)



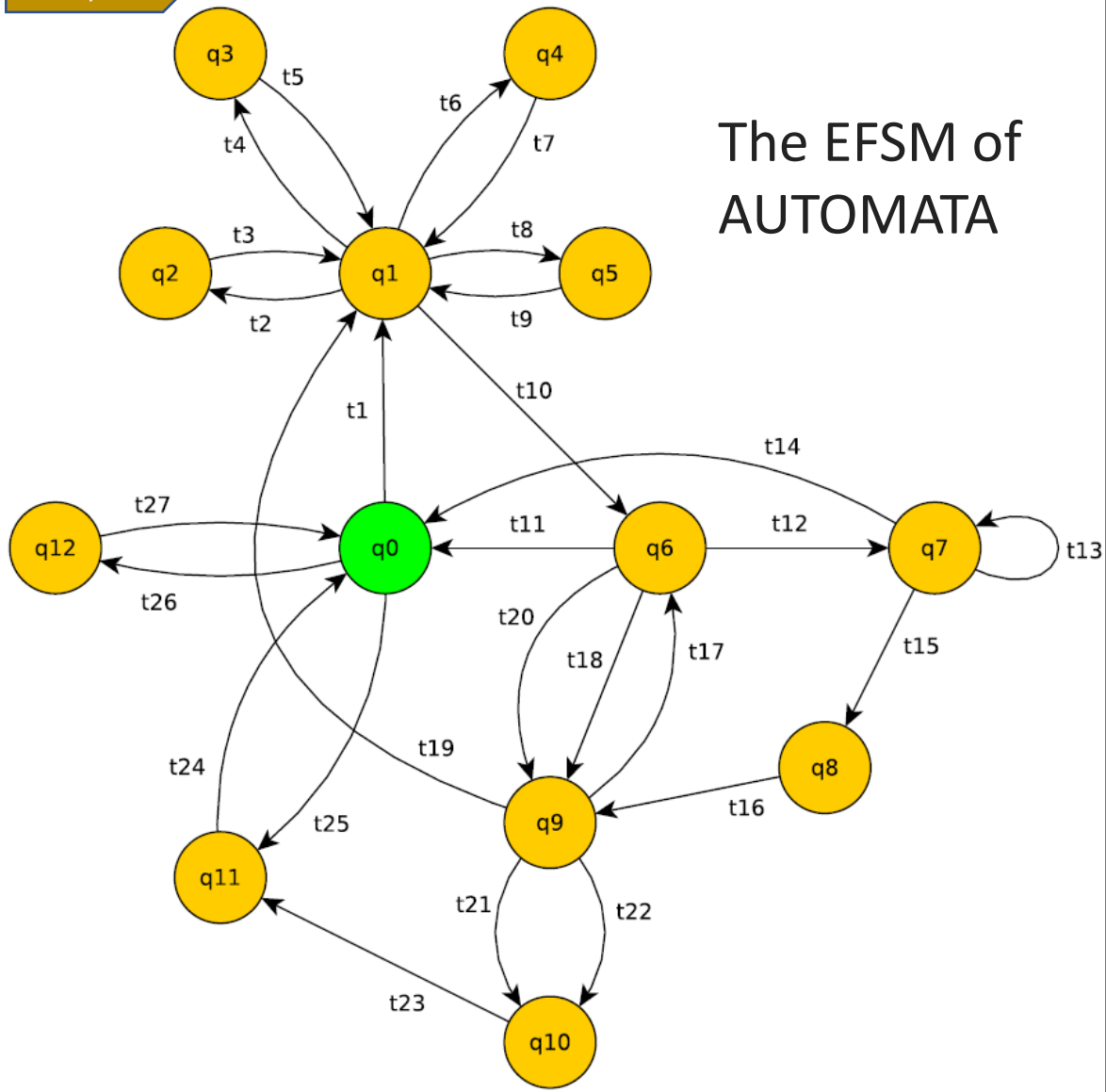
Various data from the weather station at Siding Spring Observatory, Australia at April 4, 2019. From top to bottom: sky quality, temperature and dew point, humidity, wind speed and light intensity. Sky quality is measured by the All-sky Camera. (Source: itelescope.net)

HW ToC

Appendix D: Example homework I. 3/2

Ottó Hanyecz:
AUTOMATA: AUTOmatic astronoMical observations and dATA processing

Spring 2018/2019



The EFSM of AUTOMATA

	Transition			States		Output
	Input	Guard	Action	Current	Next	
t1	check_env		isWindChecked=false isHumChecked=false isTempChecked=false isSkyChecked=false observing=false testing=false	q0	q1	Connected to EMS
t2	check_wind	isWindChecked==false		q1	q2	Wind speed checked
t3	wind_speed		isWindChecked=true	q2	q1	Connected to EMS
t4	check_hum	isHumChecked==false		q1	q3	Humidity checked
t5	hum_info		isHumChecked=true	q3	q1	Connected to EMS
t6	check_temp	isTempChecked==false		q1	q4	Temperature checked
t7	temp		isTempChecked=true	q4	q1	Connected to EMS
t8	check_sky	isSkyChecked==false		q1	q5	Sky Quality checked
t9	sq		isSkyChecked=true	q5	q1	Connected to EMS
t10	write_weather_info	isWindChecked=true isHumChecked=true isTempChecked=true isSkyChecked=true		q1	q6	Wrote data into DB
t11	bad_weather	observing==false		q6	q0	Standy
t12	good_weather	observing==false	tcount=0	q6	q7	Testing
t13	test_again	tcount<3	tcount++	q7	q7	Testing
t14	test_failed	tcount==3		q7	q0	Standby
t15	test_successful			q7	q8	Opening Telescope
t16	opened			q8	q9	Observing
t17	write_obs		observing=true testing=false	q9	q6	Wrote data into DB
t18	wrote_obs	observing==true		q6	q9	Observing
t19	check_conditions		isWindChecked=false isHumChecked=false isTempChecked=false isSkyChecked=false observing=true testing=false	q9	q1	Connected to EMS
t20	weather_info	testing==false observing==true		q6	q9	Observing
t21	bad_conditions			q9	q10	Closing Telescope
t22	end_of_night			q9	q10	Closing Telescope
t23	proc_obs			q10	q11	Data processing
t24	data_proc_comp			q11	q0	Standby
t25	data_proc			q0	q11	Data processing
t26	read_data			q0	q12	Read data
t27	read_finished			q12	q0	Standby

HW ToC

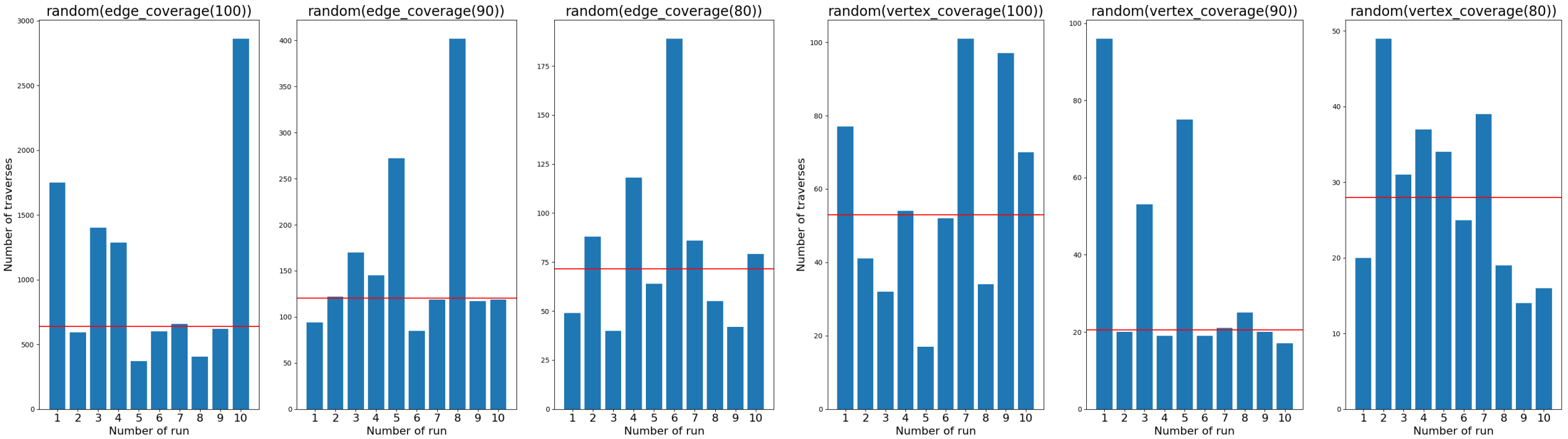
Appendix D: Example homework I. 3/3

Spring 2018/2019

Ottó Hanyecz:

AUTOMATA: AUTOmatic astronoMical observations and dATA processing

- Testing AUTOMATA using GraphWalker:



Number of edge traversals in different test runs while testing the System with 100%, 90%, and 80% edge and vertex coverages. The red horizontal line is the median of the traversals

HW ToC

Appendix D: Example homework II.

Spring 2018/2019

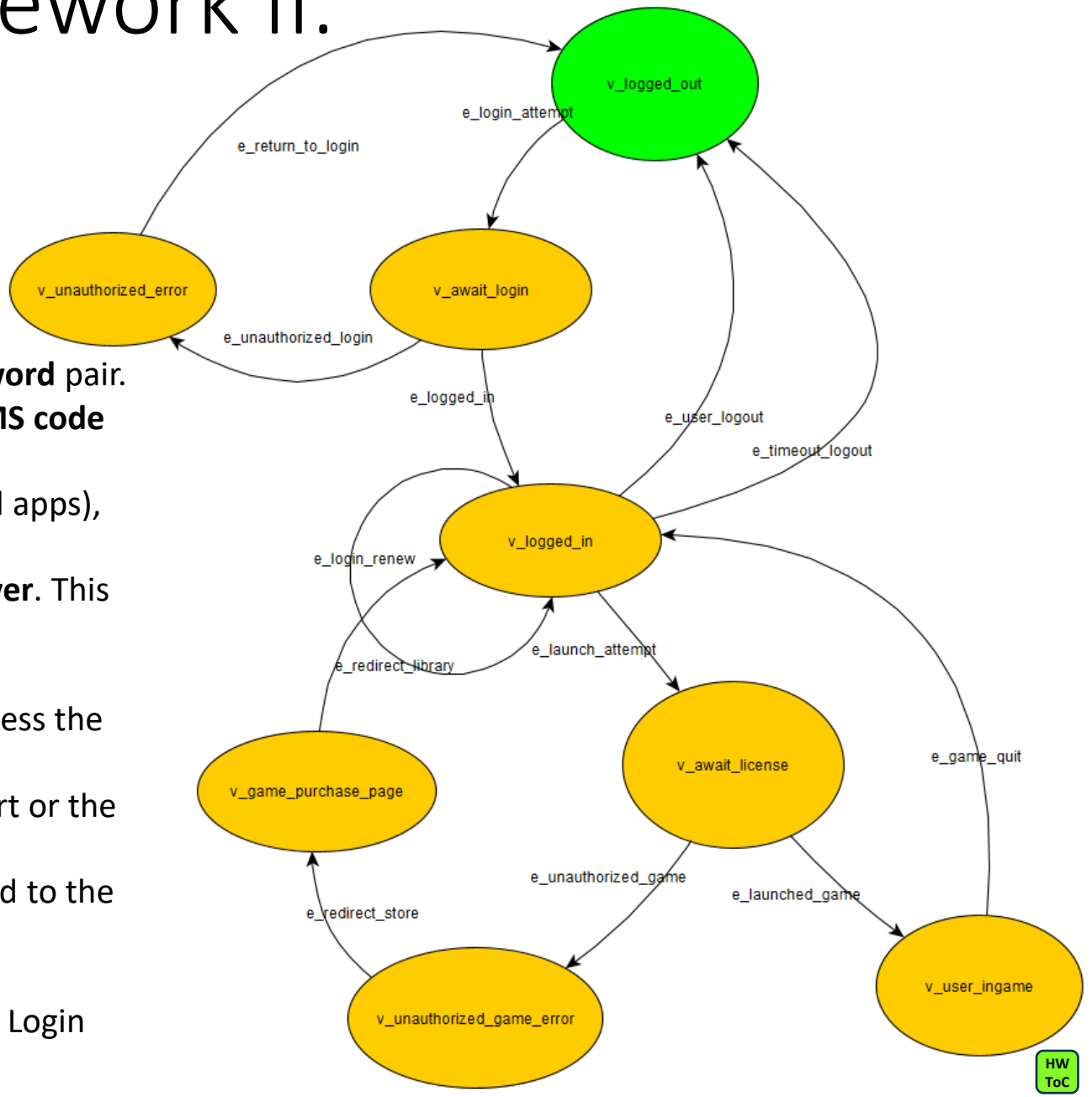
Roland Trainer Kiss:
Generic Game Store

A **user** can get **connected** to the service with a correct **e-mail** and **password** pair.
Optional: **2 step authentication** can be implemented which sends an **SMS code** to the **user's phone** on **each login**.
A **login token** lasts for **3 days** (to ensure that the **user** can **launch** owned apps), but it is renewed **every hour** until the **user logs out**.
Once **logged in**, the **user's** game collection gets requested from the **server**. This ensures that the **user** can **only access** apps that they bought.
If the **user** tries to **launch** or **download** an app, the following happens:
Authentication with the server (checks if the **user** has permission to access the app), the following two options can happen:

1.1. The **user** owns the app, permission is granted, **download** can start or the app can be **launched**.

1.2. The **user** does not own the app, in which case, they get redirected to the purchase page along with an **error message**.

Finally, the **user** can **log out** from the service and **terminate** the session. Login token is destroyed upon **logging out**.



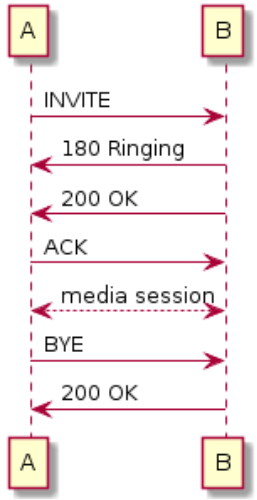
Appendix D: Example homework III.

Spring 2018/2019

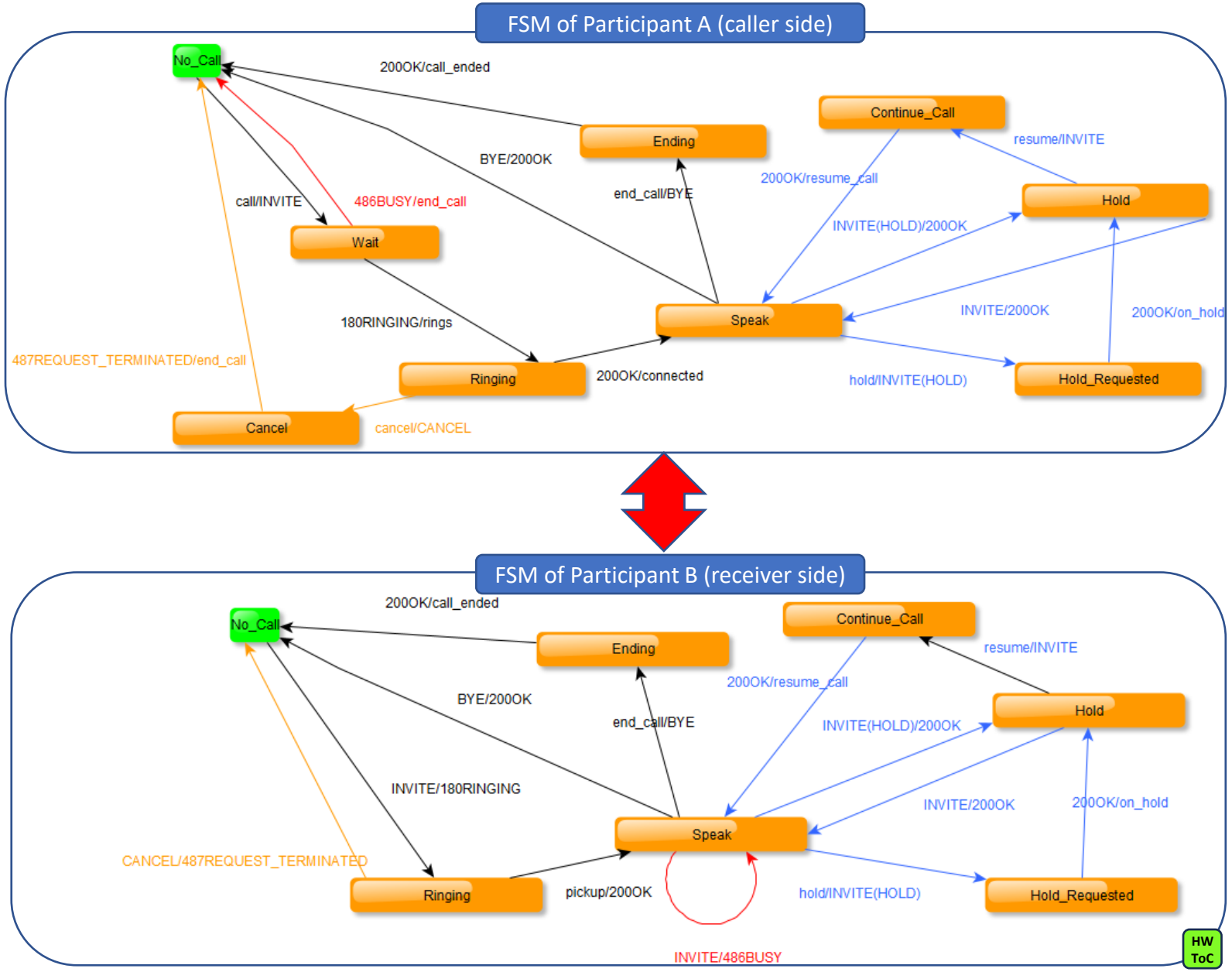
Péter Ferenc Nyitrai: SIP call

Implemented functionalities:

- Normal flow – Basic call:
  RFC 3261 / Section 4



- 486 Busy call:
  RFC 3261 / Section 21.4.24 and RFC 3665 / Section 3.9
- 487 Request Terminated (terminate before pick up):
  RFC 3261 / Section 21.4.25 and RFC 3665 / Section 3.8
- Call hold and resume:
  RFC 5359 / Section 2.1



Appendix D: Example homework IV. 3/1

Spring 2018/2019

Bence Kiss: Game of ELTE

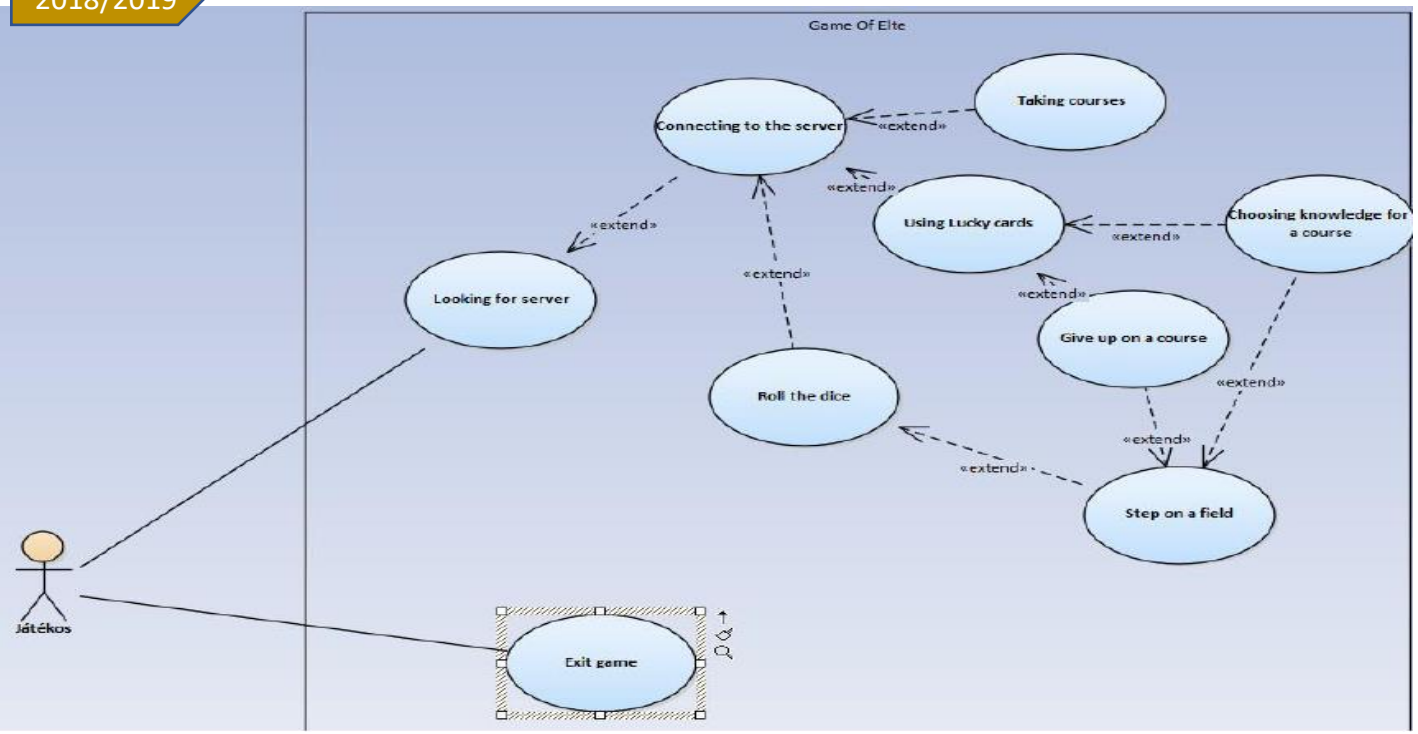
Start	Course	Course	Course	Ousted	Course	Course	Course	Course request
Exam	A board game with the following fields: • **Start:** The beginning of the new semester, players take courses by their choice. • **Course:** If there is already at least one player on this field, the last player can choose a course which he/she can oust the player who has been already standing on this field, from one common course by the player choice. If there are no common courses this field has no meaning. • **Ousted:** If you step on this field it will randomly remove a course from your course list. • **Course request:** You can take one more course if you could not take it at the start field. • **Work or study:** You can choose between study or work, if you choose work you earn (150 000 Ft) money, if you choose study you earn a knowledge for one of your courses by your choice. • **Bad date:** One of your course is overlapping, you have to give up on that course. • **Lucky card:** You draw a lucky card. • **Entertainment:** You had party last night, you pay 10 000 Ft. • **Not attended:** You have not attended a course 4th time, you lose that course. • **Give up course:** One of your course by your choice is too hard for you… you have to give up on that. • **Note:** Luckily you acquired a note for one of your course by your choice, you earn a knowledge for that • **Exam:** You take the exam on these fields. Every knowledge you earned for each course will be a successful exam. • **Impossible exam:** You lose one of your course							Work or Study
Exam								Bad Date
Impossible Exam								Work or Study
Exam								Lucky card
Exam								Work or Study
Exam								Entertainment
Exam								Work or Study
Notes	Work or Study	Entertainment	Work or Study	Lucky card	Work or Study	Give up course	Work or Study	Not attended

HW ToC

Appendix D: Example homework IV. 3/2

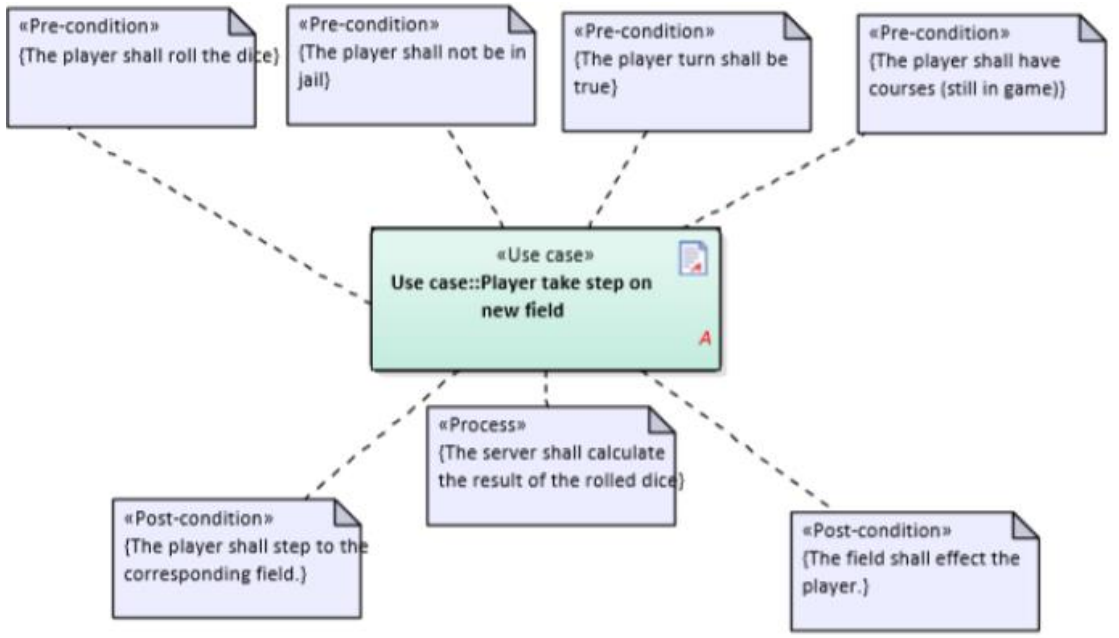
Spring 2018/2019

Use case diagram:

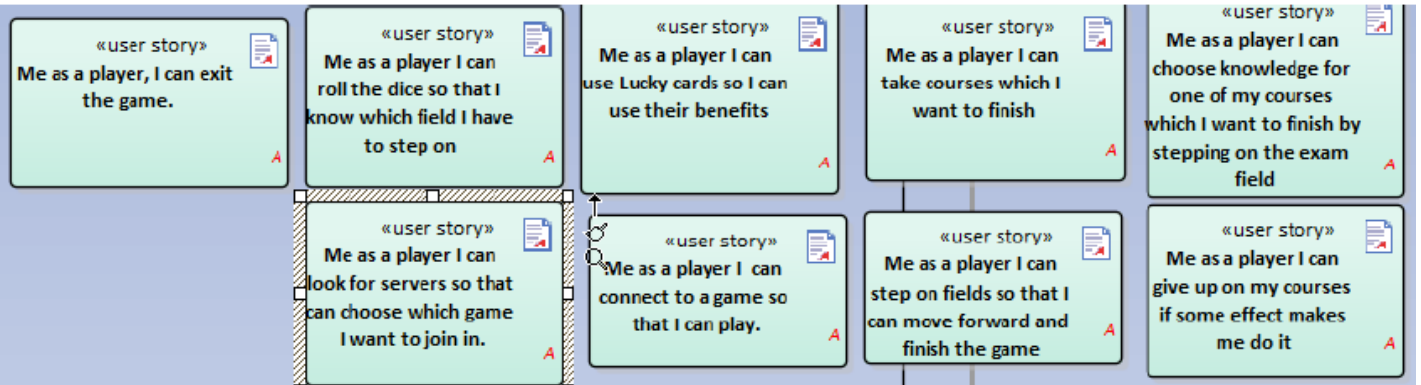


Bence Kiss: Game of ELTE

Preconditions:



User stories:



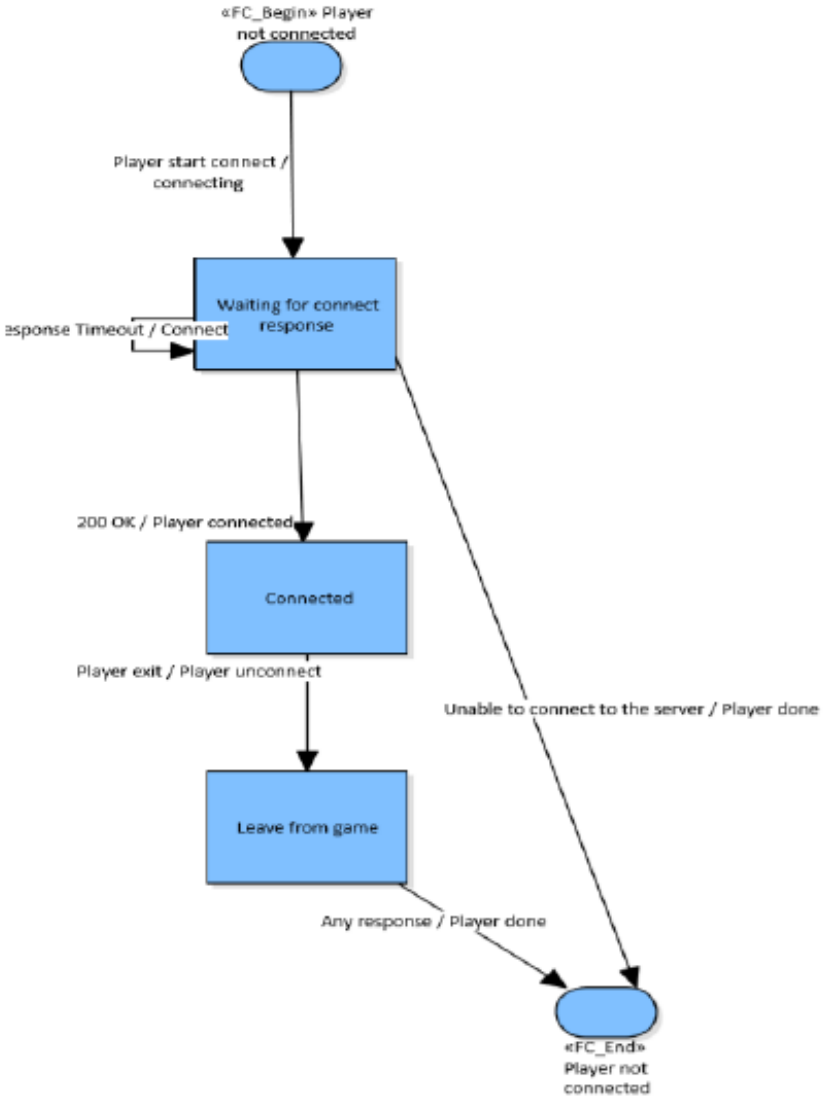
HW ToC

Appendix D: Example homework IV. 3/3

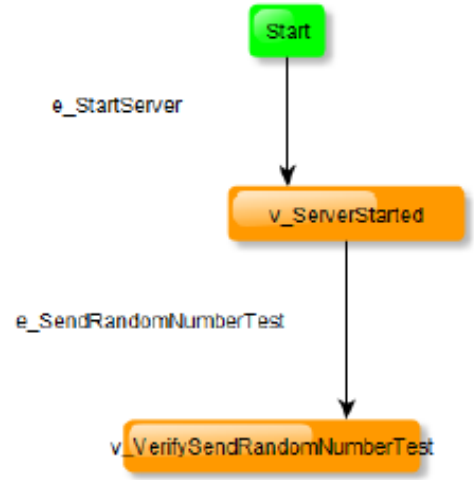
Spring 2018/2019

Bence Kiss: Game of ELTE

Call flows:



A small test in GraphWalker:




```
[INFO] Result :
[INFO]
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 2,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 2,
  "totalNumberOfVisitedVertices": 2,
  "edgeCoverage": 100,
  "totalNumberOfVertices": 2,
  "totalNumberOfUnvisitedEdges": 0
}
[INFO]
[INFO] ------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------
```

HW ToC

Appendix D: Example homework V.

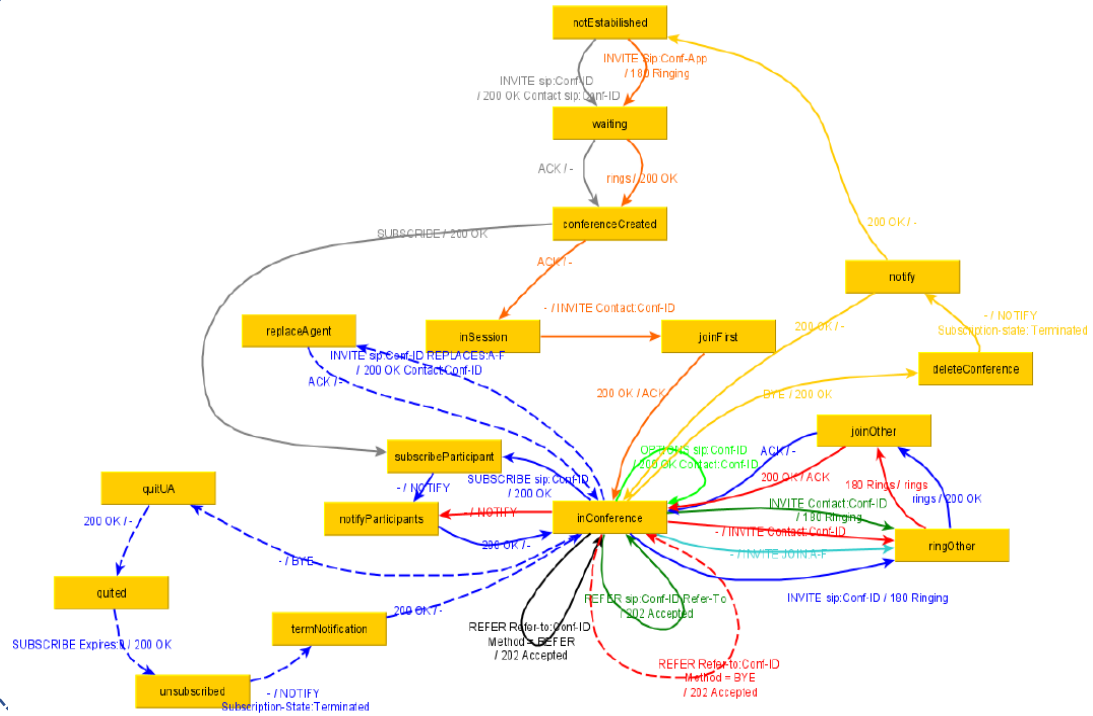
Fall 2019/2020

Adrienn Gyenes:
SIP Conferencing for User Agents

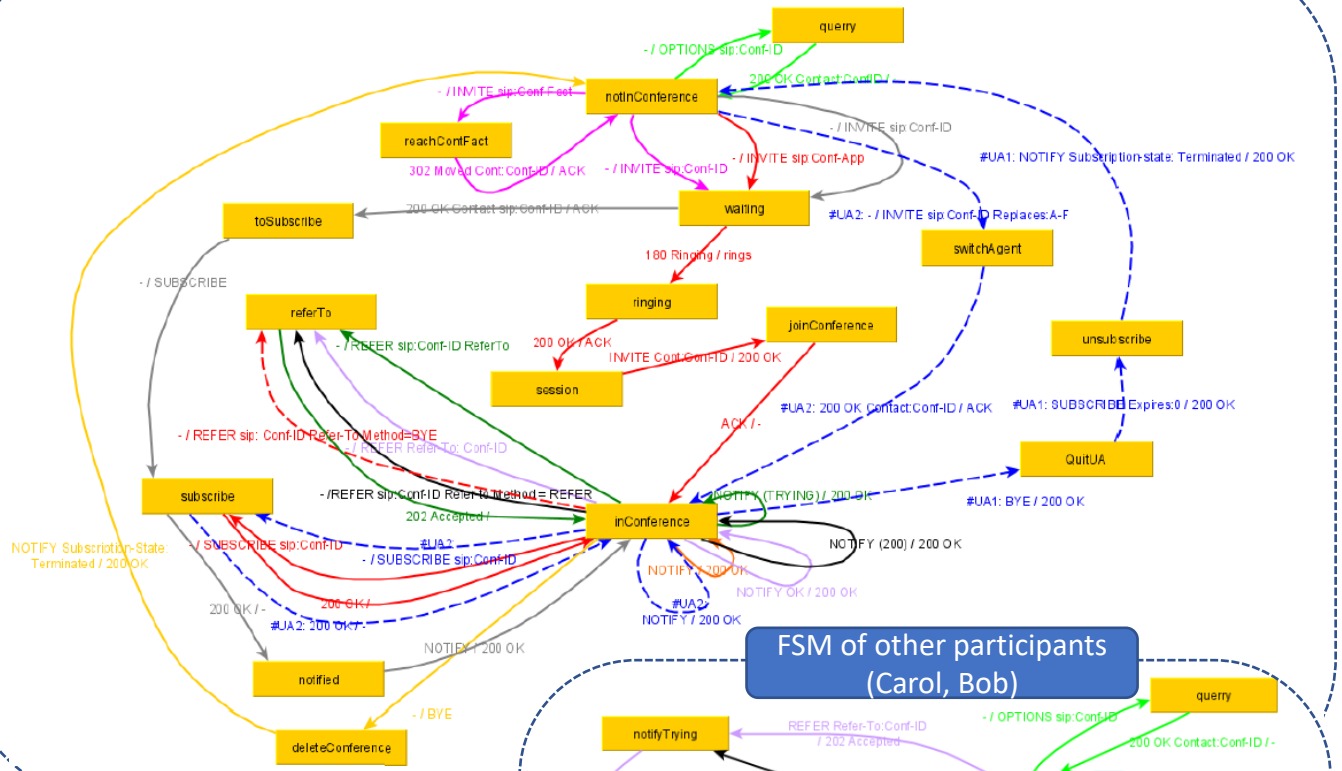
Corresponding sections of standard RFC 4579:

5.1 – 5.2 – 5.3 – 5.4 – 5.5 – 5.6 – 5.7 – 5.8 – 5.9 – 5.10 – 5.11 – 5.12 – 5.13

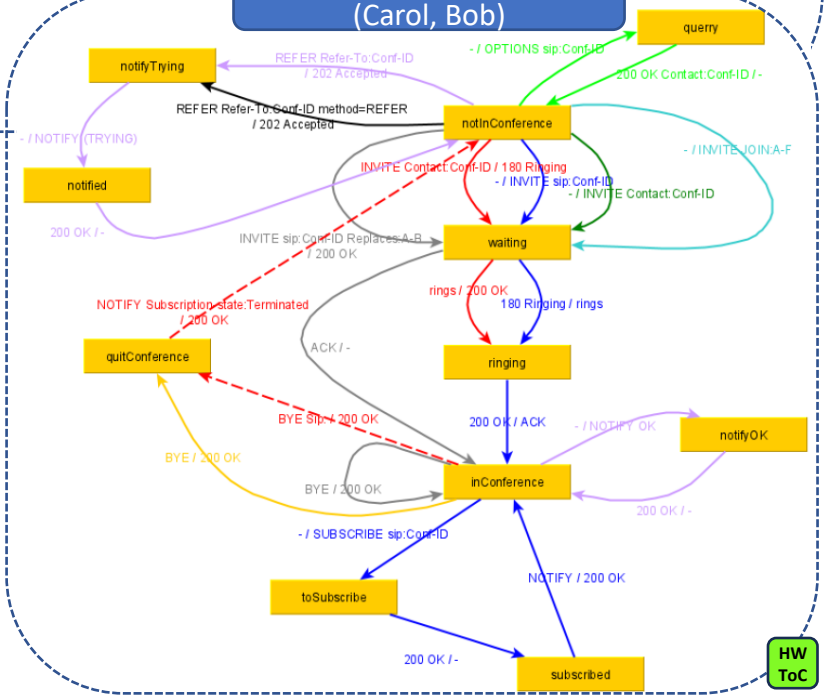
FSM of Focus



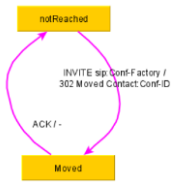
FSM of ConfOwner (Alice)



FSM of other participants (Carol, Bob)



FSM of Conf-Factory app.



HW ToC

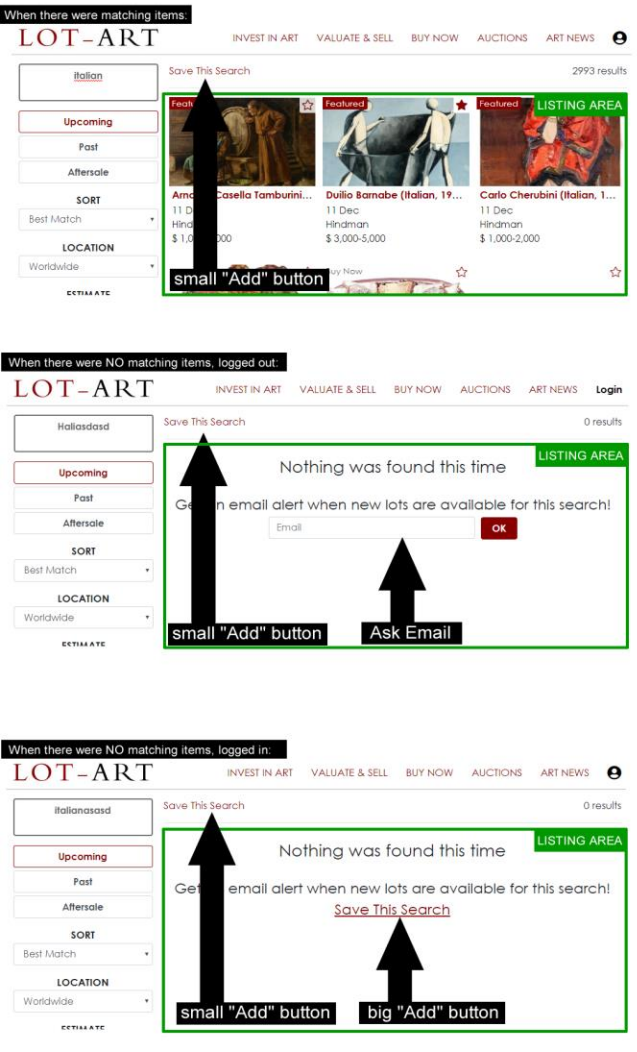
Appendix D: Example homework VI. 3/1

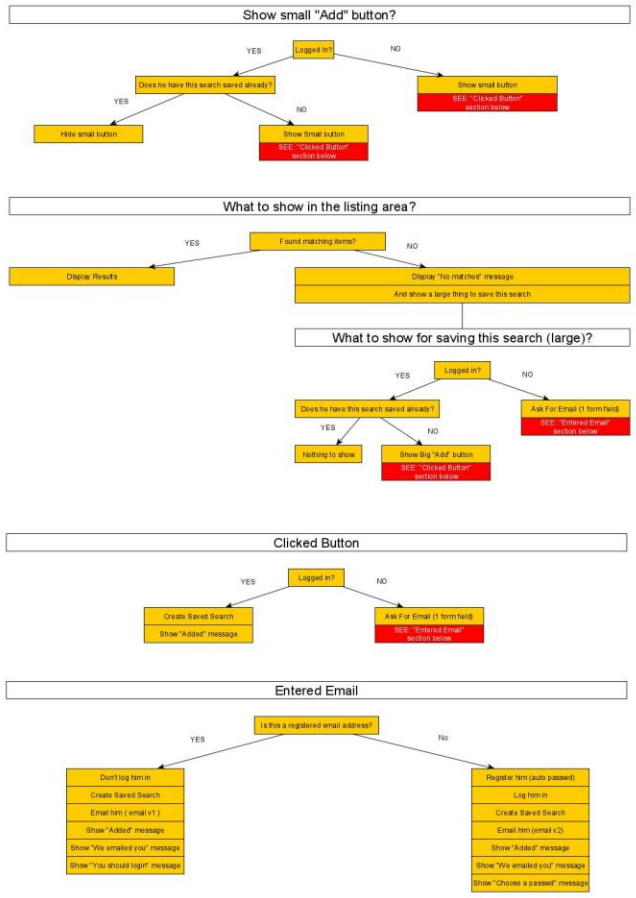
Fall 2019/2020

Milán István Madar:

Lot-Art: Create a Saved Search from the search result page

User made a search and landed on the Search Result Page





- Lot-Art: website for art and luxury items sold at auction
  - The users of this website run the same searches frequently to see if a new item appeared on the market. It is possible to save their searches.
- Problems:
  - complex conditions and dependencies
  - website is evolving step-by-step
  - some user stories may not be foreseen during specification
- Thus it is time consuming to build and maintain manually designed tests.
- Solution:
  - Model: EFSM in yED
  - The model is processed by a PHP script, which also defines some basic routing algorithms

PHP: PHP: Hypertext Preprocessor

HW ToC

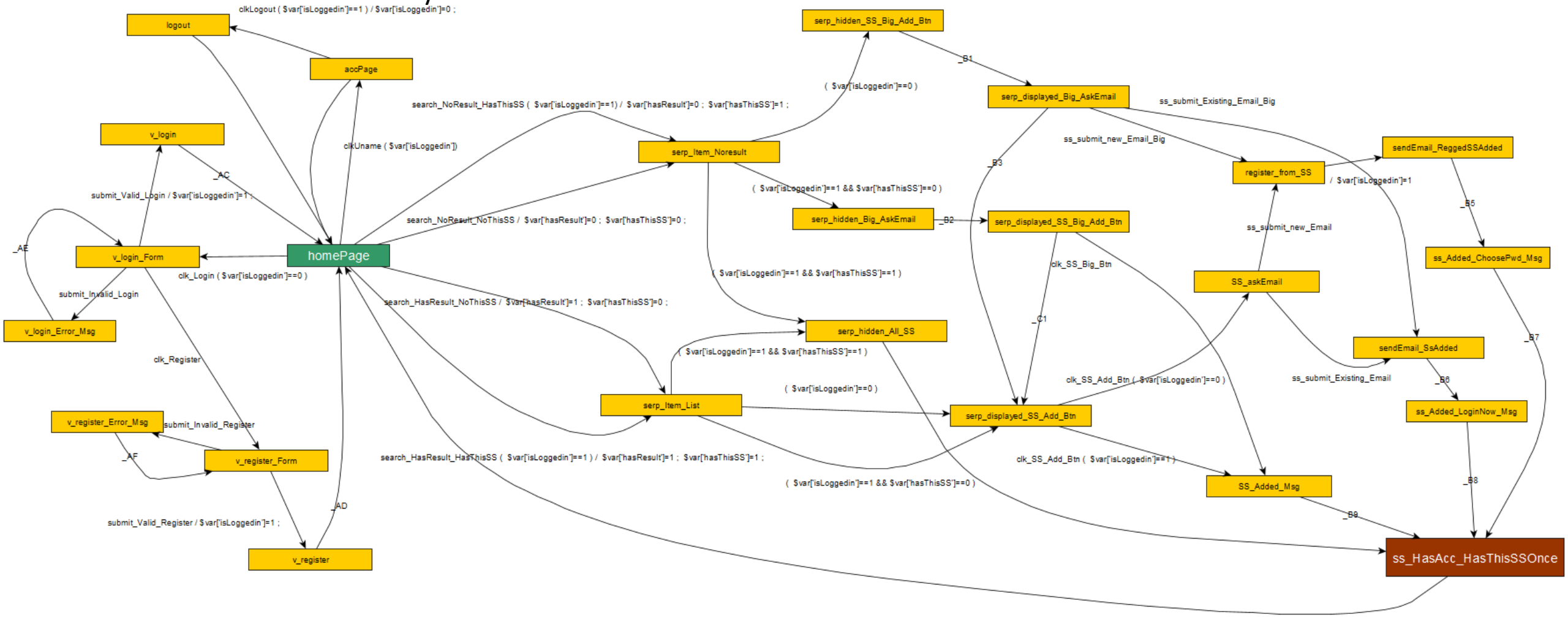
Appendix D: Example homework VI. 3/2

Fall 2019/2020

Milán István Madar:

Lot-Art: Create a Saved Search from the search result page

EFSM for saved search functionality:



Syntax of EFSM transitions: `clickButton ( $var['someVar'] > $var['otherVar'] ) / $var['myVar']=1 ; $weight=0.2`

method name | guard conditions | setters | weight

HW ToC

Appendix D: Example homework VI. 3/3

Fall 2019/2020

Milán István Madar:

Lot-Art: Create a Saved Search from the search result page

Traversing the graph (defining test scenarios)

The starting vertex is always the *HomePage*, *logged out*. The *login* vertex always logs in to the *same account*.

Discover edge cases without a goal:

```
#1 homePage(v) > random for 60 seconds
#2 homePage(v) > random for 60 seconds without login(v)
#3 homePage(v) > login(v) > random for 60 seconds without logout(v)
```

Random logged in with a goal:

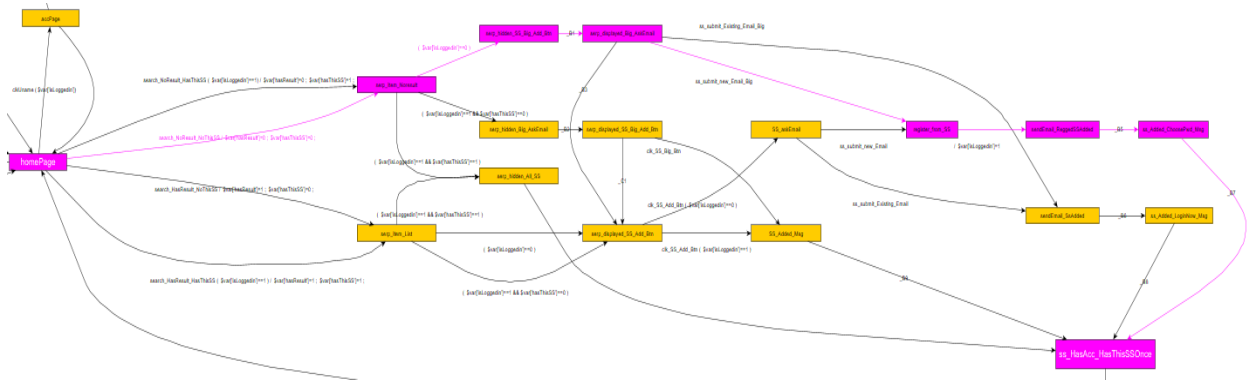
```
homePage(v) > login(v) > random without logout(v) to ss_HasAcc_HasThisSSOnce
```

Random logged out with a goal:

```
homePage(v) > random without login(v) to ss_HasAcc_HasThisSSOnce
```

Confirm a specific important use case:

```
homePage(v) > serp_Item_Noresult(v) > ss_submit_new_Email_Big(e) > ss_HasAcc_HasThisSSOnce(v)
```



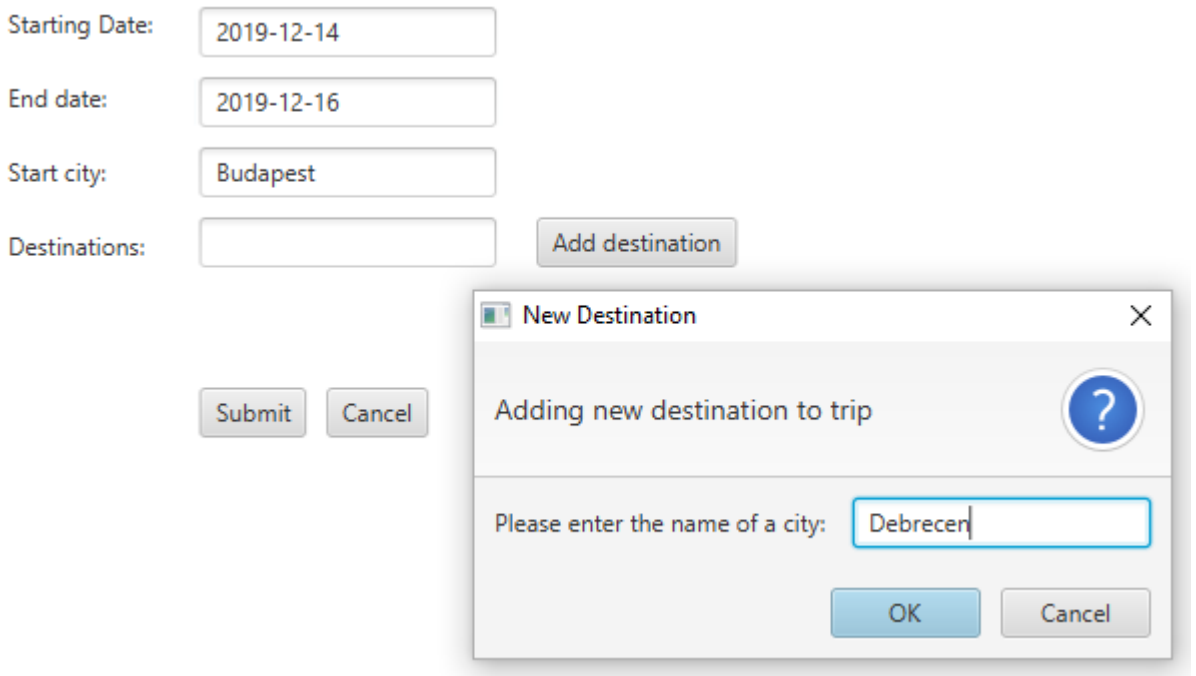
The path is highlighted with purple

HW ToC

Appendix D: Example homework VII. 4/1

Fall 2019/2020

Máté Lugosi:
General Transit Feed Specification (GTFS) Aggregator



- Travel planning application:
  - user specifies…
    - a date interval for travelling
    - the length of the trip
    - starting location
    - variable number of destinations

- Solution:
  - SUT: Basic client implementation in JavaFX
  - EFSM Model and their tests in GraphWalker
  - Harmony tests for login
  - For build tests: Travis CI

EFSM: Extended Finite State Machine
SUT: System Under Test
CI: Continuous Integration

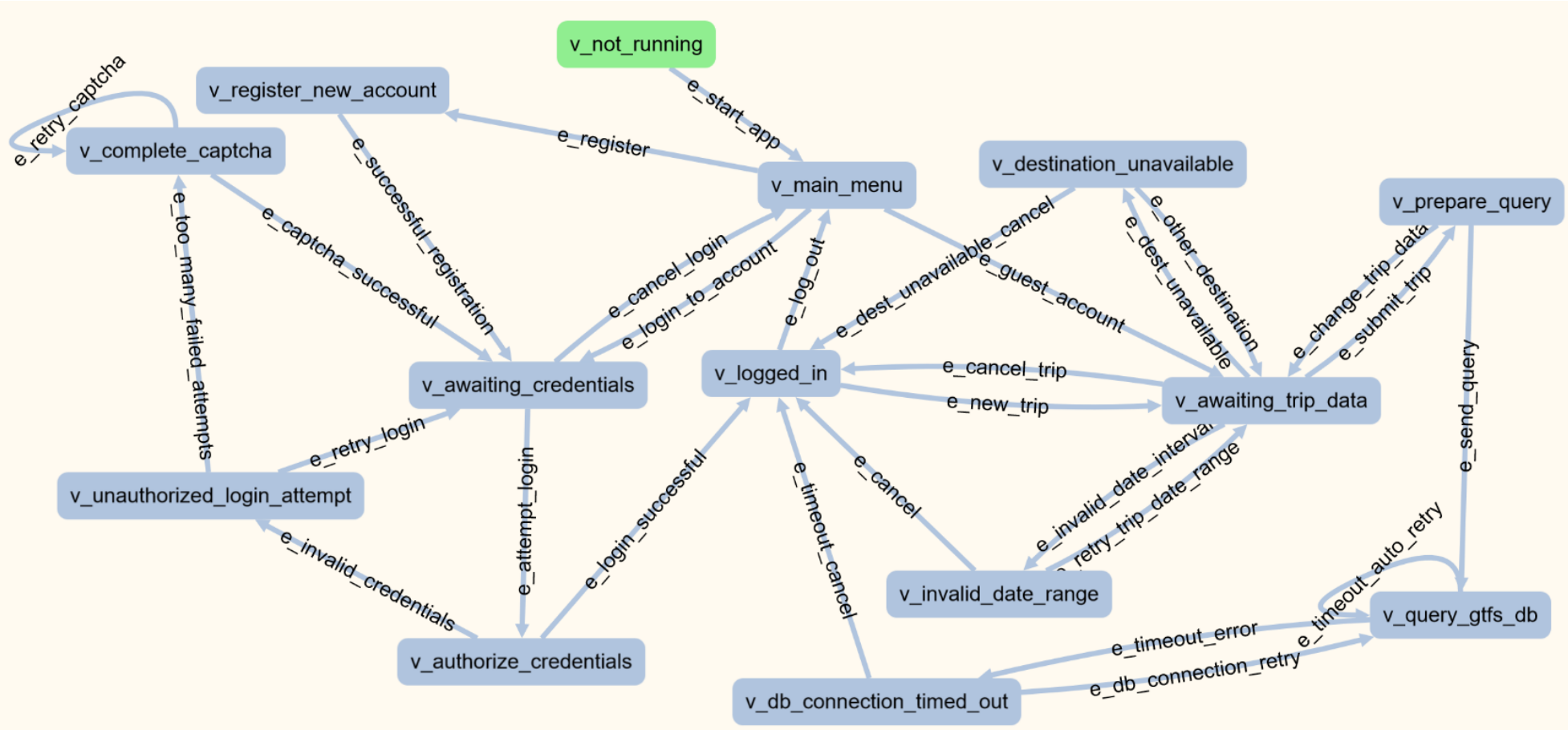
HW ToC

Appendix D: Example homework VII. 4/2

Fall 2019/2020

Máté Lugosi:
General Transit Feed Specification (GTFS) Aggregator

The EFSM of the client application:



Sample from GraphWalker log:

```
Attempting login
** Number of edges visited: 4940
Authorize credentials
E-mail and/or pswd was invalid
** Number of edges visited: 4941
Unauthorized login attempt
Retrying login
** Number of edges visited: 4942
Awaiting credentials
Attempting login
** Number of edges visited: 4943
Authorize credentials
E-mail and/or pswd was invalid
** Number of edges visited: 4944
Unauthorized login attempt
Retrying login
** Number of edges visited: 4945
Awaiting credentials
Attempting login
** Number of edges visited: 4946
Authorize credentials
E-mail and/or pswd was invalid
** Number of edges visited: 4947
Unauthorized login attempt
Too many failed attempts
** Number of edges visited: 4948
Complete captcha
Retrying captcha
** Number of edges visited: 4949
Complete captcha
[INFO] ------------------------
```

HW ToC

Appendix D: Example homework VII. 4/3

Fall 2019/2020

Máté Lugosi:

General Transit Feed Specification (GTFS) Aggregator

Categories ✓ Saved

```
E-mail(I): sample@example.com; invalid@e-mail.com
Password(I): 2a4b6c(D); wrong-PSW(S)

Msg(O): E-mail is required; Password is required; Incorrect e-mail or password; Login successful!

Submit(A): #pressed;
Cancel(A): #pressed;
Homepage(AO): #opened;
```

Acceptance criteria ✓ Saved

```
Login Successful:
WHEN E-mail IS sample@example.com
AND Password IS 2a4b6c
WHEN Submit HAS #pressed
THEN Msg IS Login successful!
AND Homepage HAS #opened

Invalid password:
WHEN E-mail IS sample@example.com
AND Password IS wrong-PSW
WHEN Submit HAS #pressed
THEN Msg IS Incorrect e-mail or password
```

Harmony test for login feature

Login Successful: WHEN E-mail IS sample@example.com WHEN Password IS 2a4b6c WHEN Submit IS #pressed WHEN Cancel IS #pressed THEN Msg IS Login successful! THEN Homepage IS #opened

Invalid password: WHEN E-mail IS sample@example.com WHEN Password IS wrong-PSW WHEN Submit IS #pressed WHEN Cancel IS #pressed THEN Msg IS Incorrect e-mail or password

Invalid e-mail: WHEN E-mail IS invalid@e-mail.com WHEN Password IS 2a4b6c WHEN Submit IS #pressed WHEN Cancel IS #pressed THEN Msg IS Incorrect e-mail or password

HW ToC

Appendix D: Example homework VII. 4/4

Fall 2019/2020

Máté Lugosi:
General Transit Feed Specification (GTFS) Aggregator

Active repositories | My builds

✓	lugosim93 GTFSAggregator	Added client side EFSM model	master	#41 passed	1403149	1 min 8 sec, 8 days ago
✓	lugosim93 GTFSAggregator	Fixed pom build should pass	master	#38 passed	dabf728	1 min 11 sec, 23 days ago
✗	lugosim93 GTFSAggregator	GTFS feed reading and db building func	master	#37 failed	a542935	1 min 3 sec, 23 days ago
✓	lugosim93 GTFSAggregator	Fixes #13, Fixes #12	master	#36 passed	cec826f	1 min 1 sec, 25 days ago
✓	lugosim93 GTFSAggregator	CI should pass	master	#33 passed	47d883f	49 sec, about a month ago
✗	lugosim93 GTFSAggregator	CI test, should fail	master	#32 failed	44d592c	39 sec, about a month ago
✓	lugosim93 GTFSAggregator	calendar parsing func	master	#31 passed	812c63f	44 sec, about a month ago
✓	lugosim93 GTFSAggregator	Calendar data parsing	master	#30 passed	ff27795	46 sec, about a month ago
✓	lugosim93 GTFSAggregator	Very basic gtfs parsing functionality	master	#29 passed	d24b926	44 sec, about a month ago
✓	lugosim93 GTFSAggregator	Build should pass now	master	#27 passed	cd49721	47 sec, about a month ago

Used Travis-CI for build tests in the project's GitHub repository

← *Dashboard showing recent tests*

A simple travis.yml for the project:

```
1  language: java
2
3  jdk:
4    - openjdk8
5
6  install: mvn dependency:resolve
7
8  script: mvn verify
9
```

Summary of a successful test:

master Update README.md
Commit 18dfa40
Compare 1403149..18dfa40
Branch master
Máté Lugosi
JDK: openjdk8 Java
AMD64
#43 passed
Ran for 56 sec
2 minutes ago
Restart build
Debug build

Summary of an unsuccessful test:

master GTFS feed reading and db building functionality added
Commit a542935
Compare cec826f..a542935
Branch master
Máté Lugosi
JDK: openjdk8 Java
AMD64
#37 failed
Ran for 1 min 3 sec
23 days ago
Restart build
Debug build
HW ToC

Appendix D: Example homework VIII. 3/1

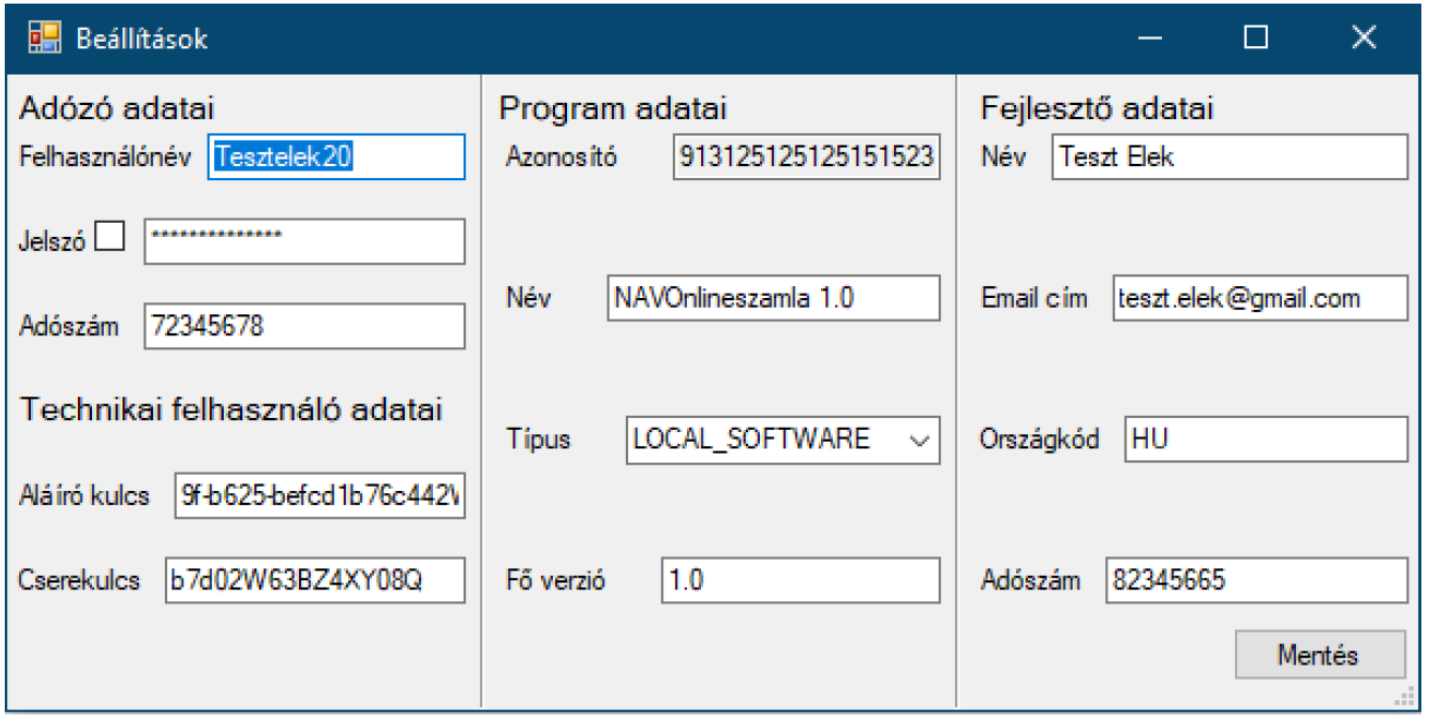
Spring 2019/2020

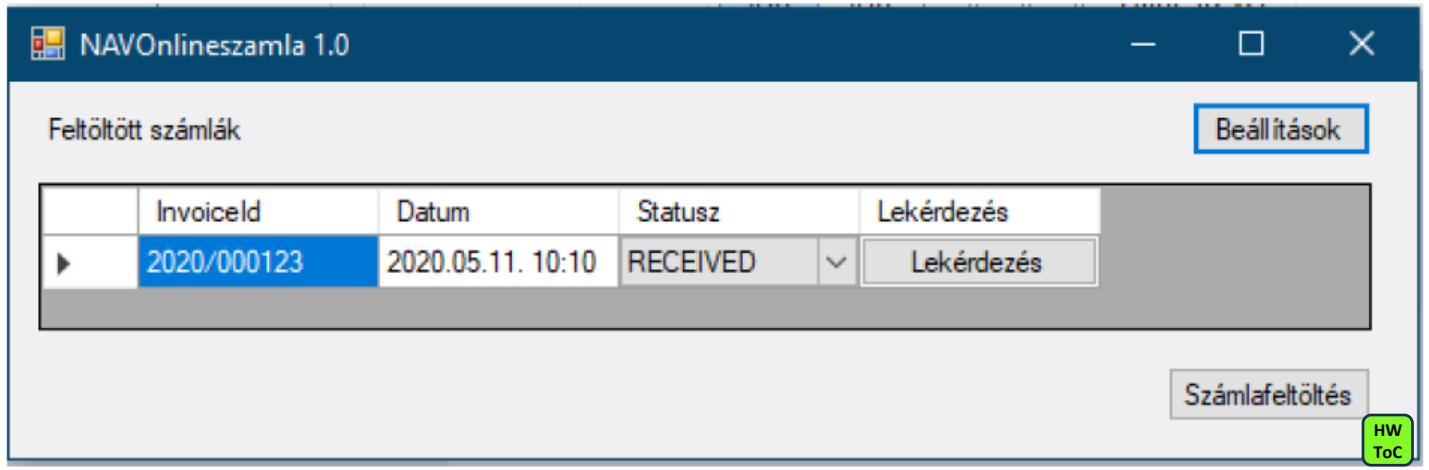
Levente Borsos:
NAV Online Invoicing System

- Import invoices, validate and upload them to the RESTful stateless web service provided by NAV

Short description:

- Before uploading invoices, you have to configure the application in the Settings menu.
- You can upload your invoices one at a time
- The application handles the general technical aspects (handling the http requests/responses, creating schema valid xml files, calculating the request signature, time conversion handling, log files)
- Implemented encoding and encryption algorithms:
  - BASE64 encode/decode (RFC 3548)
  - SHA-512 encode (RFC 6234)
  - SHA3-512 encode (FIPS 202)
  - AES-128 ECB decode (RFC 3826)
- Every submitted invoice needs to be queried that can be done by clicking the upload button.

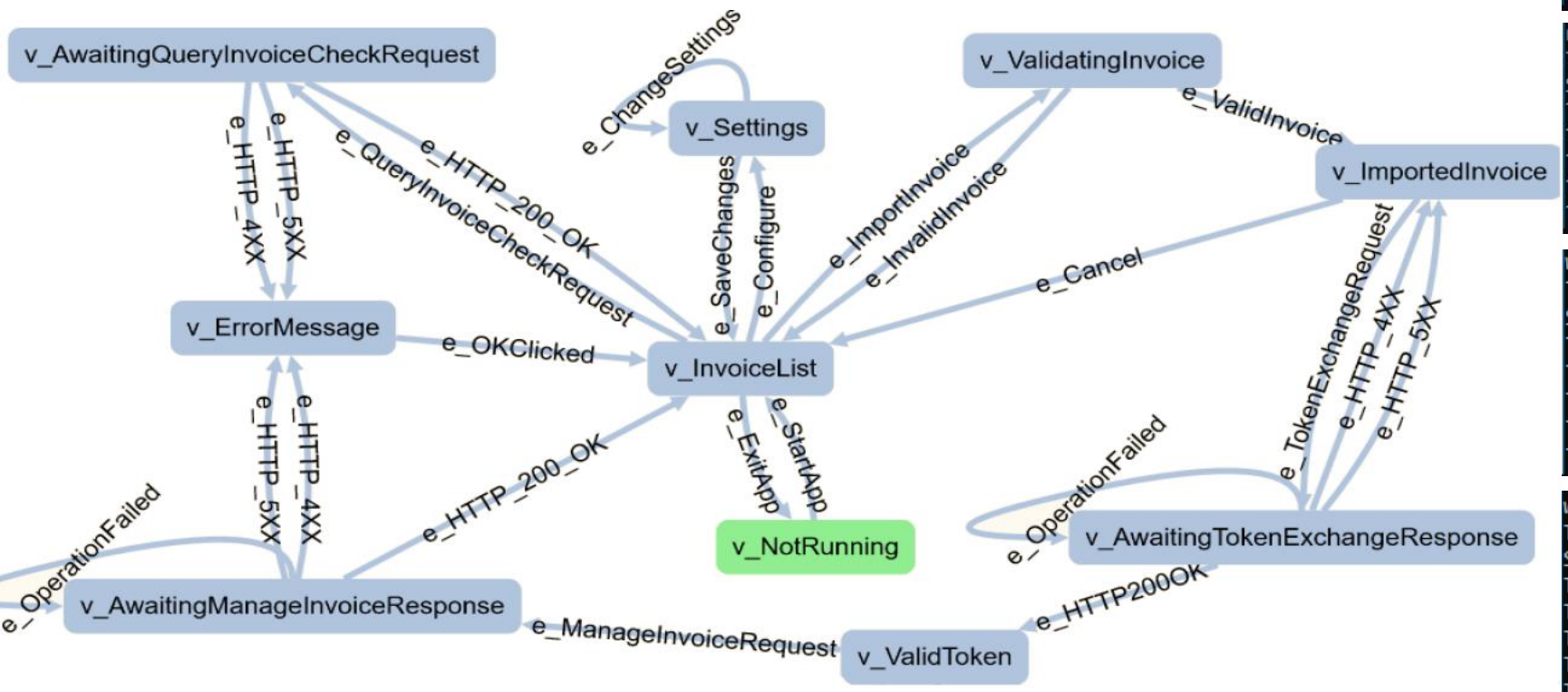




Appendix D: Example homework VIII. 3/2

Spring 2019/2020

Levente Borsos:
NAV Online Invoicing System




```
random(edge_coverage(75)
Total steps: 637.
Settings changed 8 times.
Total number of invoices: 1.
Total number of HTTP 200 responses: 11.
Total number of HTTP 4XX responses: 8.
Total number of HTTP 5XX responses: 20.
Total number of times one of the operations failed: 20.
```

```
random(time_duration(60))
Total steps: 1148995.
Settings changed 9 times.
Total number of invoices: 3899.
Total number of HTTP 200 responses: 46708.
Total number of HTTP 4XX responses: 47008.
Total number of HTTP 5XX responses: 46920.
Total number of times one of the operations failed: 46920.
```

```
weighted_random(edge_coverage(100))
Total steps: 319.
Settings changed 12 times.
Total number of invoices: 1.
Total number of HTTP 200 responses: 5.
Total number of HTTP 4XX responses: 6.
Total number of HTTP 5XX responses: 9.
Total number of times one of the operations failed: 9.
```

```
weighted_random(time_duration(15))
Total steps: 234033.
Settings changed 10 times.
Total number of invoices: 766.
Total number of HTTP 200 responses: 9232.
Total number of HTTP 4XX responses: 9192.
Total number of HTTP 5XX responses: 9459.
Total number of times one of the operations failed: 9459.
```

HW ToC

Appendix D: Example homework VIII. 3/3

Spring 2019/2020

Levente Borsos:
NAV Online Invoicing System

Variables:
• 'invoice': Number of invoices
• 'settings': Number of times the settings changed
• 'count': Number of times an operation failed. The value resets between operations.

Edges:
• e_ImportInvoice: Waits for the user, to select an invoice. If its schema invalid, event e_InvalidInvoice will occur, resulting in an error message. If its schema valid, event e_ValidInvoice will occur, resulting in a success message.
• e_TokenExchangeRequest: Token exchange operation issuing single-use data reporting tokens preceding the invoice data reporting and technical annulment processes. It becomes invalid after 5 minutes.
• e_ManageInvoiceRequest: Invoice data submitting operation. It can only be used with a valid token. (Token exchange request has to be made before submit invoices).
• e_QueryInvoiceCheckRequest: Operation used for checking the success of invoice data reporting. It doesn't require a token.
• e_HTTP_4XX: Any HTTP response, starting with 4. The error seems to have been caused by the client. Check the documentation[1] for more info.
• e_HTTP_5XX: Any HTTP response, starting with 5. The server failed to fulfill a request. Check the website[2] for a potential maintenance.
• e_HTTP_200_OK: The generic response for any correct request. It doesn't necessarily indicate that the business execution of the content of the request was successful. (You need to query the transaction status later.)

Actions:
• e_ChangeSettings increases 'settings' by 1.
• Both e_ManageInvoiceRequest and e_TokenExchangeRequest sets 'count' to 1.
• Both e_OperationFailed edges increases 'count' by 1.
• v_AwaitingManageInvoiceResponse's e_HTTP_200_OK edge increases 'invoice' by 1.

Guard conditions:
• 'settings' must be higher than 7 to use e_ImportInvoice.
• 'settings' must be 7 or lower to use e_Configure.
• 'invoice' must be 1 or higher to use e_QueryInvoiceCheckRequest.
• 'count' must be lower than 4 to use e_OperationFailed (either).

HW ToC

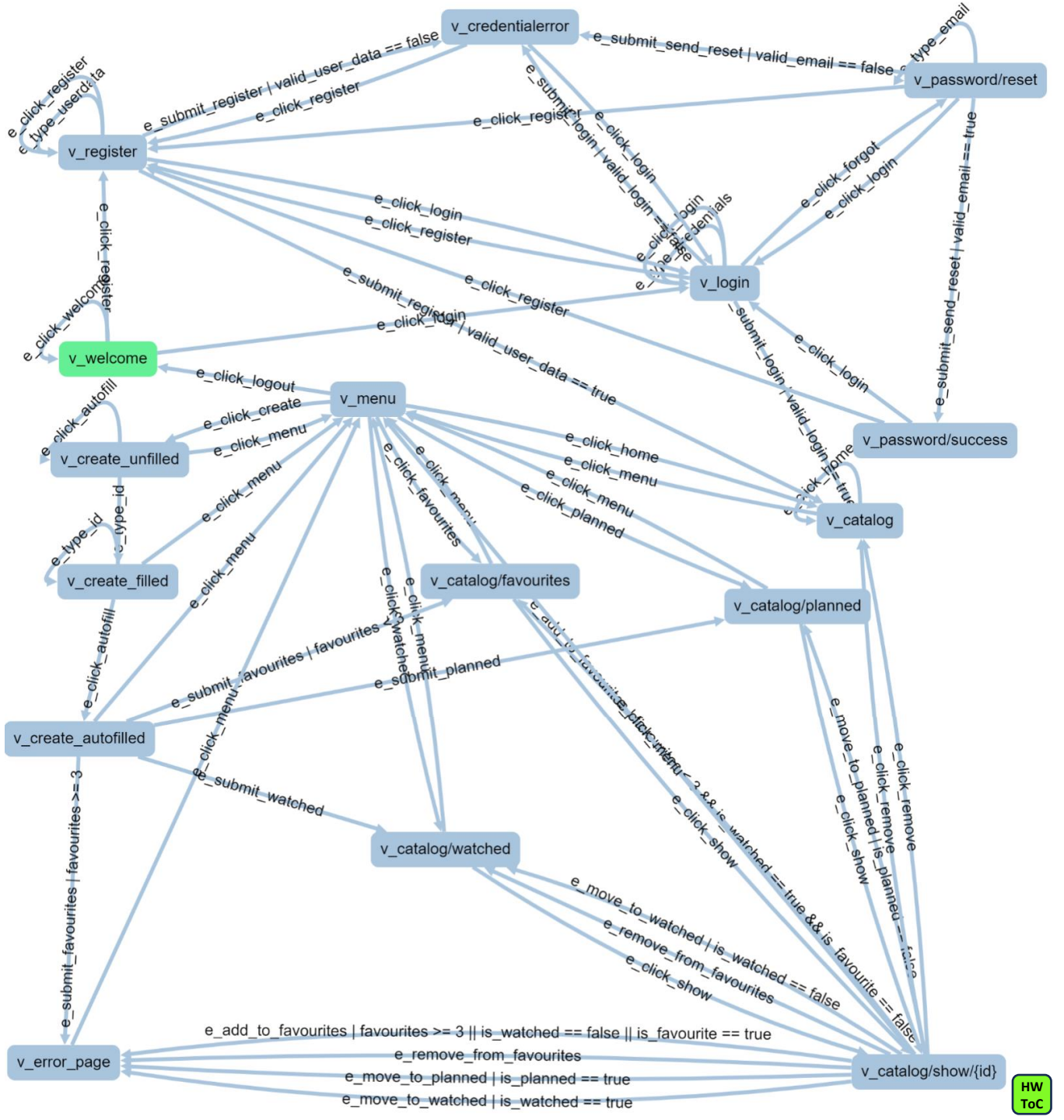
Appendix D: Example homework IX. 2/1

Spring 2019/2020

László Dervalics:
Movie Organiser website

- In the Movie Organiser website registered users can add movies to their collection.
- Users can change the movie status to planned or watched, also they can add maximum of 3 movies to their favourites. The movies in the collection are removable.
- The movies are identified by their IMDB ID's.

- The testing of the model is done with Laravel Dusk



HW ToC

Appendix D: Example homework IX. 2/2

Spring 2019/2020

László Dervalics:
Movie Organiser website

An example for using Laravel Dusk

Login
Clicking the Login button visitors can fill the login form, and login to the website.

Requirements

- The user shall fill 'E-mail'
- The user shall fill 'Password'
- If the Login button is pressed and the 'E-mail' or 'Password' is wrong, the user will be redirected to the error page.
- If the Login button is pressed and the 'E-mail' or 'Password' is correct, the user will be redirected to the homepage.

Dusk tests
Testing to login (valid and invalid) with checking to current page. The pause() function is necessary, because in dusk without pause sometimes it can change the order, or skip some steps.
This tests will go from the Welcome page to the Login page, clicking Login (valid and invalid case):

```
public function testValidLogin()
    {
        $valid_login = true;
        $this->init();
        $this->e_click_login();
        $this->e_type_credentials($valid_login);
        $this->e_submit_login($valid_login);
    }
    public function testInvalidLogin()
    {
        $valid_login = false;
        $this->init();
        $this->e_click_login();
        $this->e_type_credentials($valid_login);
        $this->e_submit_login($valid_login);
    }

//The private functions corresponds to the EFSM's edges:
    private function e_click_login()
    {
        $this->browse(function (Browser $browser) {
            $browser->click('.login')
                ->assertSee('Login page');
        });
    }

    private function e_type_credentials($valid_login)
    {
        if ($valid_login == true)
        {
            $this->browse(function (Browser $browser) {
            $browser->pause(600)
                ->type('email', 'test_elek@test.hu')
                ->pause(600)
                ->type('password', 'valami12')
                ->assertSee('Login page');
        });
        }
        else
        {
            $this->browse(function (Browser $browser) {
            $browser->pause(600)
                ->type('email', 'test@test.com')
                ->pause(600)
                ->type('password', 'valami12')
                ->assertSee('Login page');
        });
        }
    }

    private function e_submit_login($valid_login)
    {
        if ($valid_login == true)
        {
            $this->browse(function (Browser $browser) {
                $browser->press('Login')
                    ->assertSee('Catalog:');
            });
        }
        else
        {
            $this->browse(function (Browser $browser) {
                $browser->press('Login')
                    ->assertSee('Invalid credentials');
            });
        }
    }
```

HW ToC

Appendix D: Example homework X. 4/1

Fall 2020/2021

Miklós Miskolczi:
Autonomous Multi-robot System Simulation

Amazon automated warehouse
Photo by Justin Sullivan/Getty Images

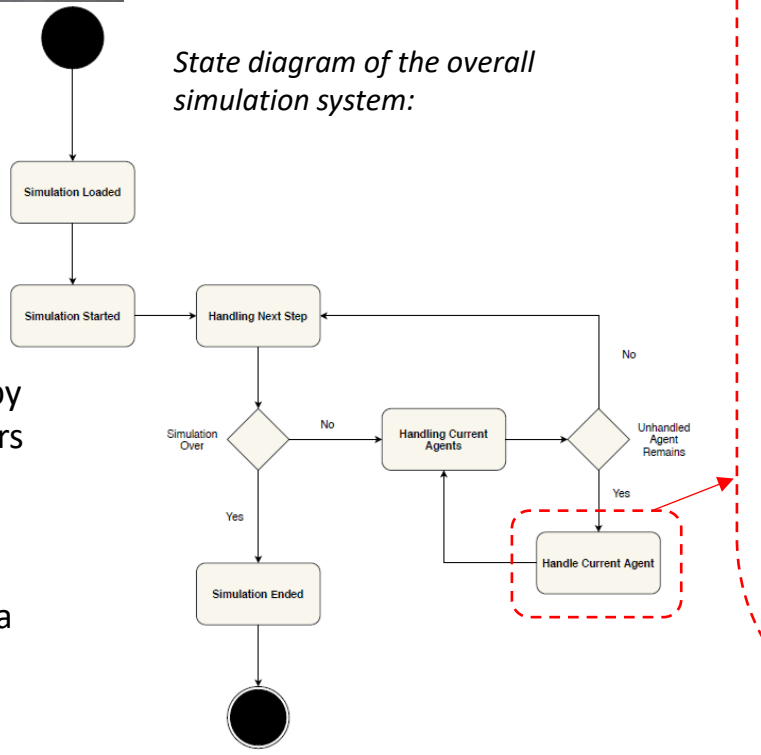
This task simulates an automated shipping warehouse with multiple agents.

Robot/agent:

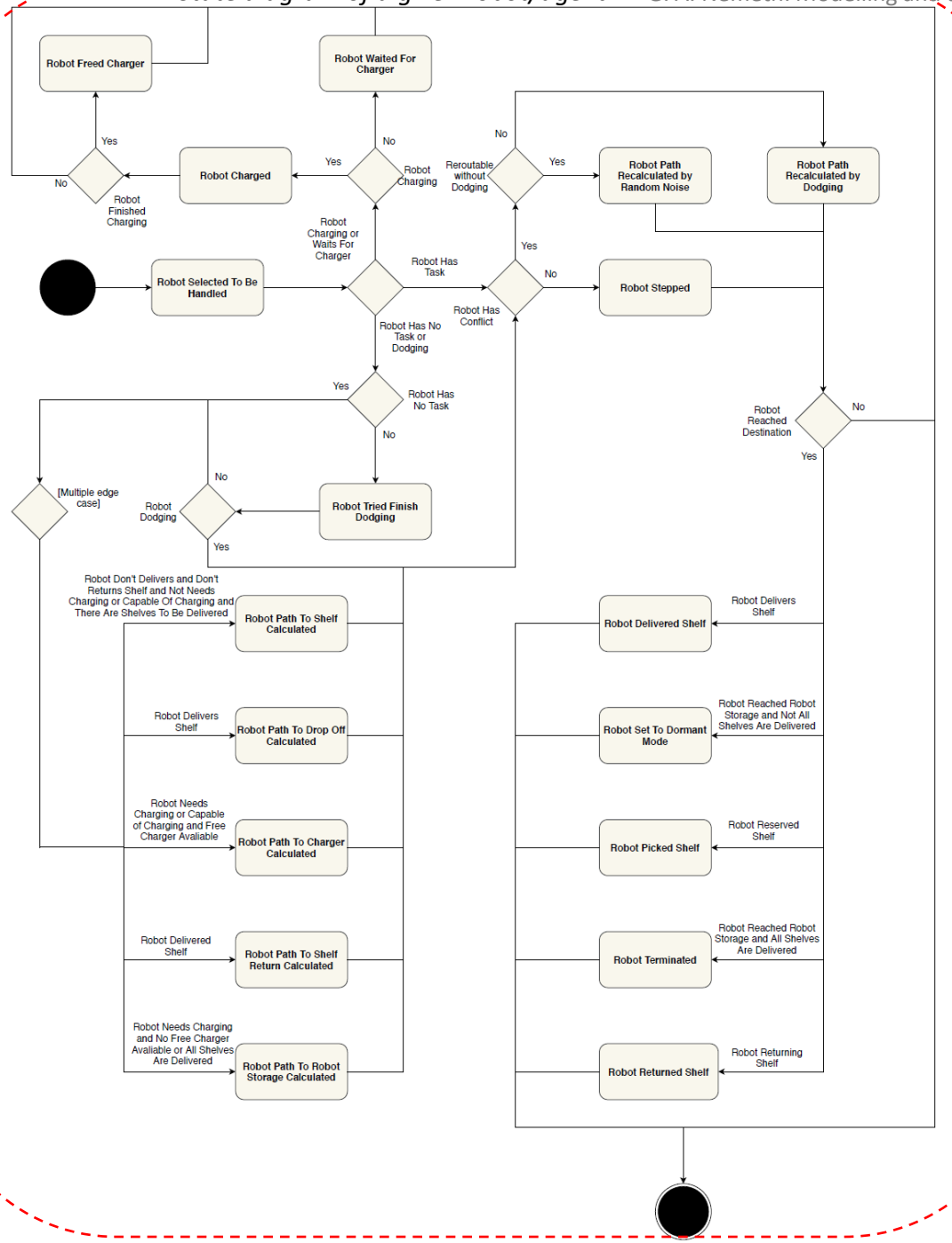
- Autonomous working unit in the warehouse
- Delivers shelves to the given locations and returns them
- Takes care of their energy level by navigating themselves to chargers
- Communicates with the other robots to avoid collision

The **warehouse** is represented with a grid, with *N X M cells*

State diagram of the overall simulation system:



State diagram of a given robot/agent:



HW ToC

Appendix D: Example homework X. 4/2

Fall 2020/2021

Miklós Miskolczi:
Autonomous Multi-robot System Simulation

Description of a given robot/agent:

Dodging: The robot is in the dodging state, if it could not resolve the conflict. In this case the robot searches for a path to a random warehouse cell.

Searching subtask:
- If the robot *must* charge and there are no free chargers available, then it goes to the Robot Storage and switches to *dormant mode*.
- If the robot is in *dormant mode* and there is a free charger, then it reserves it and searches a path to it.
- If the robot *should* or *must* charge and there is a free charger available, then it reserves it and searches a path to it.
- If a robot *should* charge, but there is no charger available, then it does not wait for the charger.
- If there are shelves to be delivered and the robot should not charge, then reserves a shelf and searches a path to it.
- If all shelves are delivered, then the robot searches a path to the Robot Storage, where it can terminate.
- If a robot delivers a shelf (already picked it up), then it searches a path to the shelf's destination.
- If a robot returns shelf (just delivered it), then it searches a path to the shelf's location (where the robot found it). The shelves are delivered to the same place, where they have been found.

Performing task / Navigating: This state includes a low-level conflict solution called *Local Repair A** .If a conflict is noticed, it tries to search for a different shortest path to the current destination but with added random noise, based on its *agitation level*. Each time the robot needs to reroute this way, its *agitation level* is increased, which leads to more random paths. If it can't resolve the conflict this way after an amount of time, it switches to dodging state.

Finished Task: The robot finishes its task if it has reached its current destination and is not dodging or has finished charging. It can be:
- If the robot reserved a shelf, it picks it up.
- If the robot is delivering a shelf, then finishes it.
- If the robot is returning a shelf, it drops the shelf down at the current location.
- If the robot reached a charger, begins the charging.
- If the robot reaches the Robot Storage, it terminates if all shelves are delivered or it goes to dormant mode if waits for charger.
- If the robot finished charging (reached its maximum energy level) then frees it.

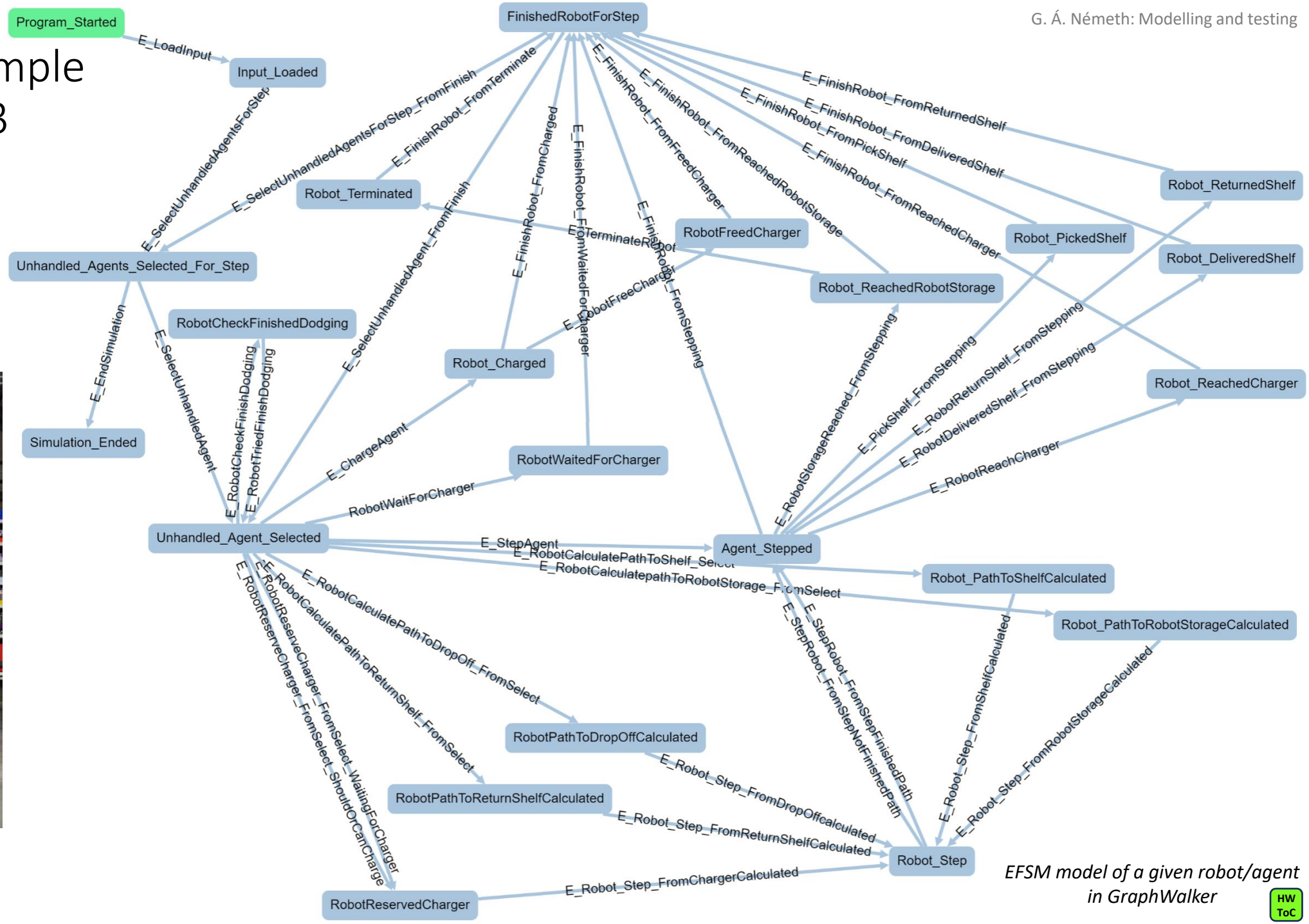
HW ToC

Appendix D: Example homework X. 4/3

Fall 2020/2021

Miklós Miskolczi: Autonomous Multi-robot System Simulation

Amazon automated warehouse
Source of the figure



EFSM model of a given robot/agent in GraphWalker

HW ToC

Appendix D: Example homework X. 4/4

Fall 2020/2021

Miklós Miskolczi:
Autonomous Multi-robot System Simulation

Testing:

- Graphwalker: EFSM model creation and path finding between states
- Altwalker: running tests in states (with unit tests using NUnit)

Test input file:

```
130 140 1 150 8
8 8
0 0 0 0 5 1 0 0
0 0 0 1 0 0 0 0
6 0 0 0 1 0 0 0
0 0 0 0 1 0 1 1
4 0 0 0 0 0 0 0
1 0 1 1 0 1 0 2
0 0 0 2 0 1 0 0
0 0 2 0 0 2 0 2
3
0 0
3 3
0 7
5
5 7 0 4
6 3 0 4
7 2 0 4
7 7 0 4
7 5 0 4
```

Energy levels when agent must/should charge; charge rate / simulation step
max. energy level; Nof LRA tries before dodging*

Size of the warehouse (row, coloumn)

8x8 warehouse grid

0: Empty	*4: Charger*
1: Obstacle	*5: Dropoff*
2: Shelf	*6: RobotStorageEntrance*
3: MissingShelf	

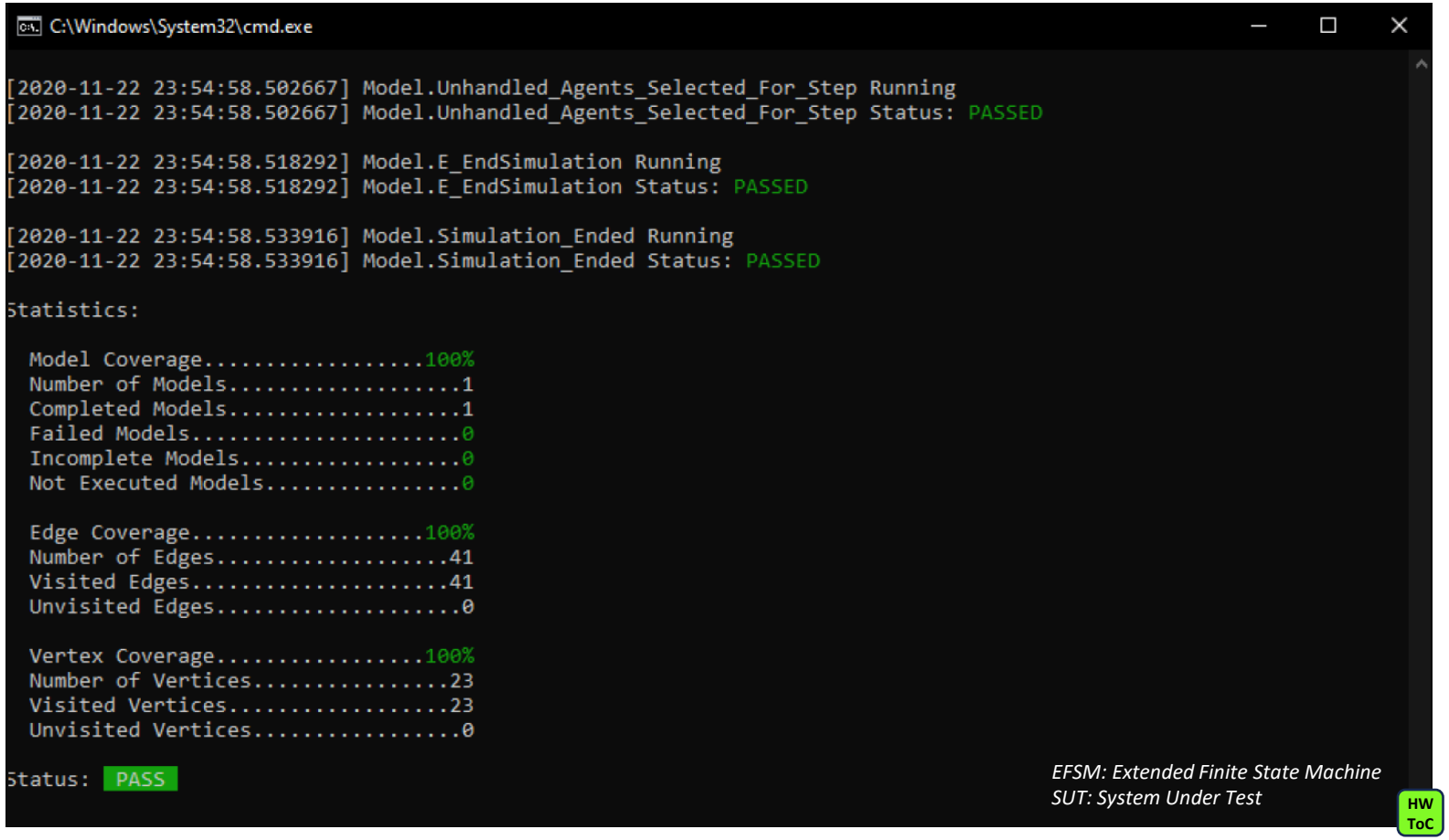
Number of agents

Starting position of the agents

Number of shelves to be delivered

Source and destination address of shelves

In each simulation step, every agent is managed, but only one at a time. A glue code (adaptation code) is created which loads the data from the SUT to the EFSM model if a given agent is selected and based on this information the next step for the given agent can be determined.



EFSM: Extended Finite State Machine
SUT: System Under Test

HW ToC

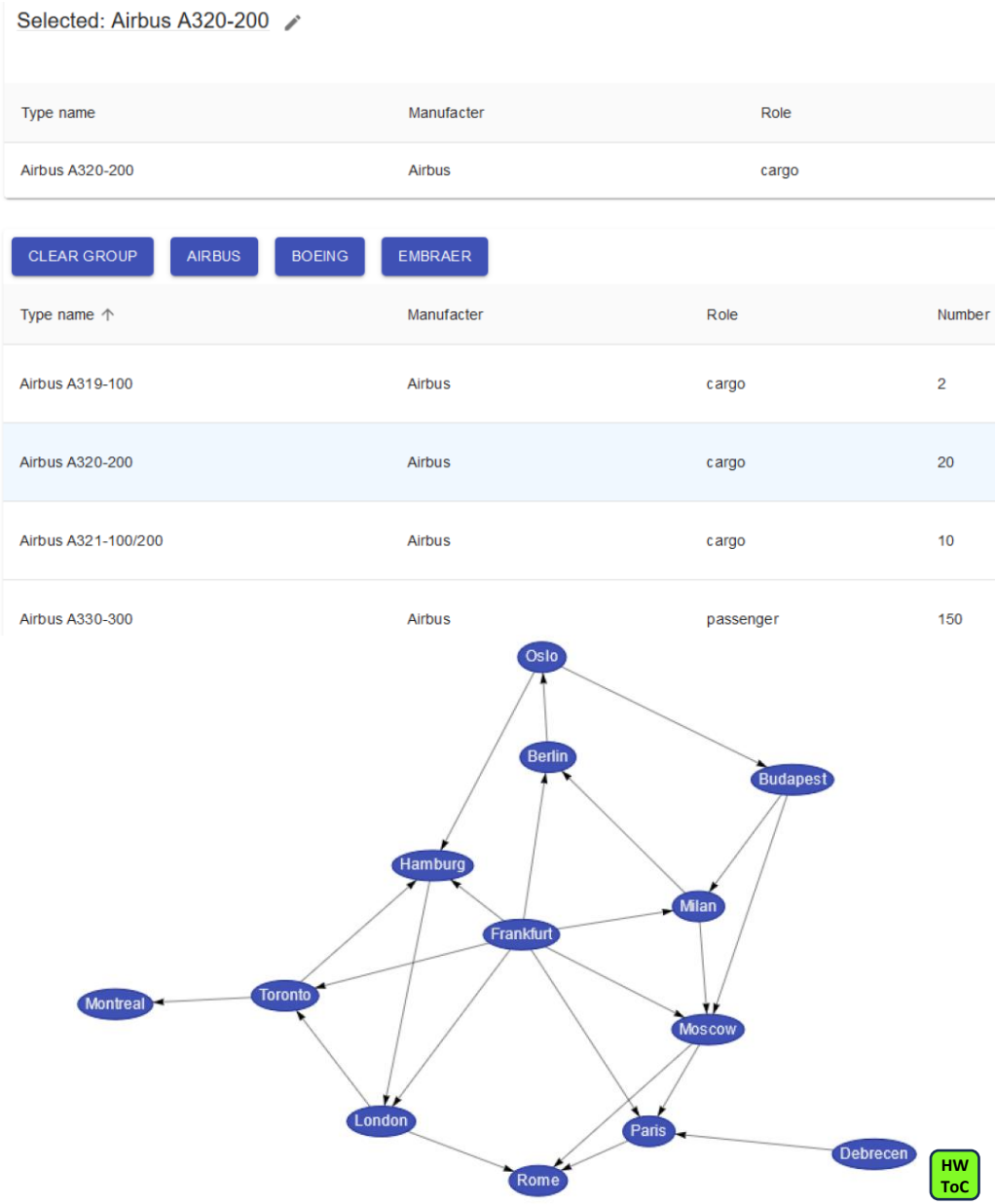
Appendix D: Example homework XI. 3/1

Spring 2020/2021

Norbert Czakó-Bancsi:
Airplane Path Optimalisation

- SUT (System Under Test): Web page application (created for a BSc Thesis):
  - Main functionalities:
    - User management (users are internal employees of an airline)
    - Airplane storage: modify existing airplanes of the airline, sort/group airplanes
    - City storage: add/remove city connections
    - Calculate path between 2 cities
  - Parts:
    - User interface (UI) part was made in ReactJS with Material-UI
    - Back-end was made in NodeJS
    - Storage: MySQL

- Testing:
  - Model-based testing with GraphWalker (project is built on Maven)
    - Edge, vertex and requirement coverages has been investigated
  - Glue code: Java + UI testing uses Selenium with chromedriver
  - Test execution automatization with Jenkins



Appendix D:
Example homework XI. 3/2

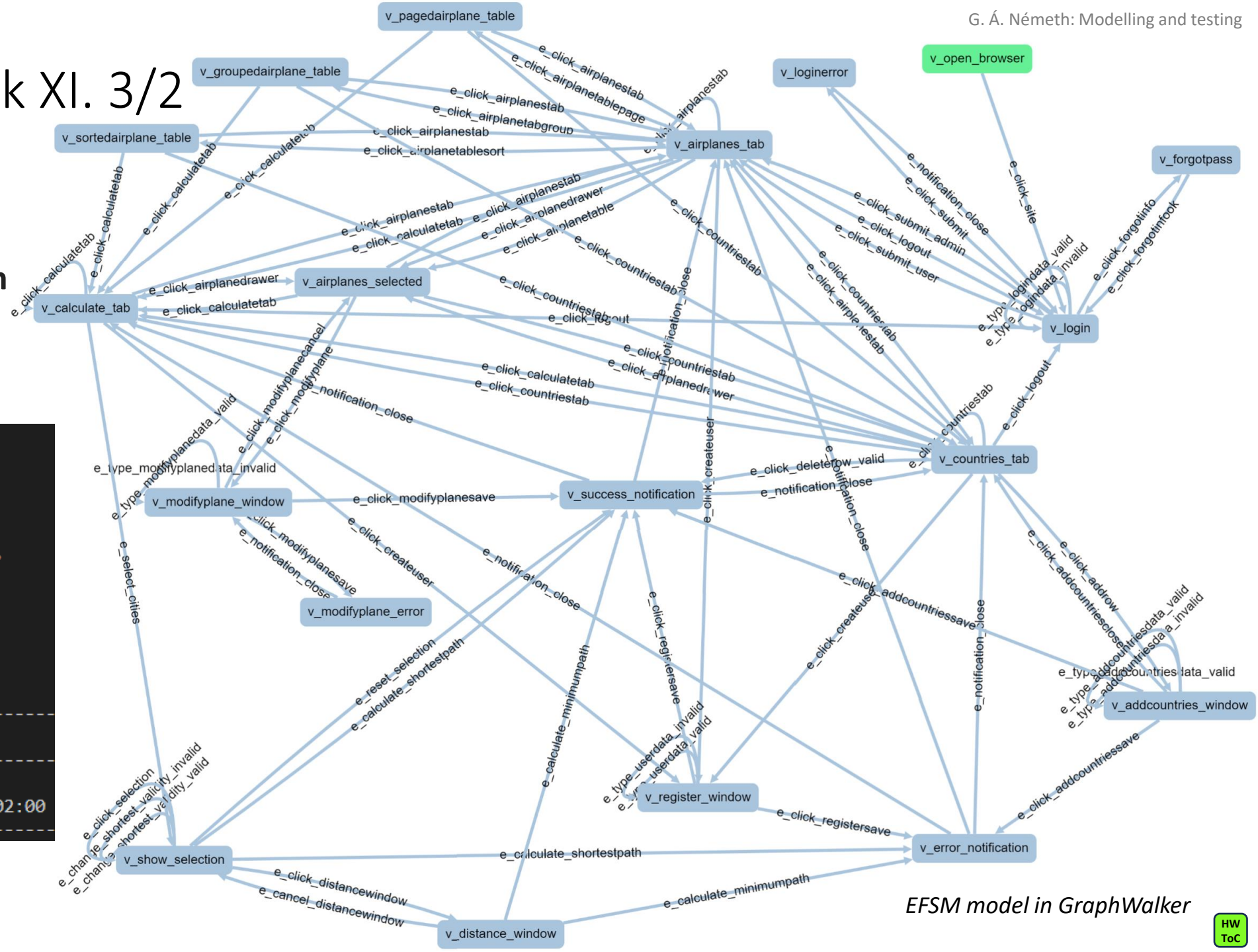
Spring 2020/2021

Norbert Czakó-Bancsi:

Airplane Path Optimalisation


```
  "requirementsFailed": [],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 78,
  "totalNumberOfVisitedVertices": 19,
  "totalNumberOfRequirement": 6,
  "totalNumberOfUncoveredRequirement": 0,
  "edgeCoverage": 100,
  "totalNumberOfVertices": 19,
  "totalNumberOfPassedRequirement": 6,
  "totalNumberOfUnvisitedEdges": 0
}
[INFO]
[INFO] ------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------
[INFO] Total time:  13:47 min
[INFO] Finished at: 2021-04-30T12:41:24+02:00
[INFO] ------------------------------------
```


100% edge cover in GraphWalker


v_pagedairplane_table
v_groupedairplane_table
v_sortedairplane_table
v_airplanes_tab
v_loginerror
v_open_browser
v_forgotpass
v_login
v_airplanes_selected
v_calculate_tab
v_countries_tab
v_modifyplane_window
v_success_notification
v_modifyplane_error
v_addcountries_window
v_register_window
v_show_selection
v_error_notification
v_distance_window
e_click_airplanestab
e_click_airplanetablepage
e_click_airplanetabgroup
e_click_airplanetablesort
e_click_calculatetab
e_click_airplanedrawer
e_click_airplanetable
e_click_countriestab
e_click_countriestabclose
e_click_countriestabcancel
e_click_countriestabsave
e_click_countriestabdelete
e_click_countriestabedit
e_click_countriestabadd
e_click_countriestabaddrow
e_click_countriestabaddcountries
e_click_countriestabaddcountriesclose
e_click_countriestabaddcountriessave
e_click_countriestabdeleterow
e_click_countriestabeditrow
e_click_countriestabsaverow
e_click_countriestabcancelrow
e_click_countriestabaddrowsave
e_click_countriestabaddrowcancel
e_click_countriestabdeleterowsave
e_click_countriestabdeleterowcancel
e_click_countriestabeditrowsave
e_click_countriestabeditrowcancel
e_click_countriestabaddrowvalid
e_click_countriestabaddrowinvalid
e_click_countriestabdeleterowvalid
e_click_countriestabdeleterowinvalid
e_click_countriestabeditrowvalid
e_click_countriestabeditrowinvalid
e_click_countriestabaddcountriesvalid
e_click_countriestabaddcountriesinvalid
e_click_countriestabaddcountriessavevalid
e_click_countriestabaddcountriessaveinvalid
e_click_countriestabaddcountriesclosevalid
e_click_countriestabaddcountriesclosevalid
e_click_countriestabaddcountriesclosev

Appendix D: Example homework XI. 3/3

Spring 2020/2021

Norbert Czakó-Bancsi:
Airplane Path Optimalisation

- Glue code (adaptation code):

```java
// check if the page is open
@Override
public void e_click_site() {
  driver.get("http://localhost:3000/login");
  driver.manage().window().maximize();

  waiter.until(ExpectedConditions.titleContains("Airplane calculator"));
  waiter.until(ExpectedConditions.elementToBeClickable(driver.findElement(By.id("loginButton"))));
}
```

```java
// check if error msg is present
@Override
public void v_loginerror() {
  this.checkIfElementExistsClass("MuiAlert-message");
  this.checkNotificationText("Invalid e-mail/password combination!", "Please enter your email and password!");

}
```

↑ *Example tests with Java and Selenium*

All +

S	W	Name ↓
🔴	⛈	airplane_model_test

Icon: S M L

← *Test automation with Jenkins*


```
[INFO] Result :
[INFO]
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfFailedRequirement": 0,
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "requirementsNotCovered": [],
  "totalNumberOfVisitedEdges": 78,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [],
  "requirementCoverage": 100,
  "requirementsPassed": [
    {
      "modelName": "AirplaneCalculatorModel",
      "requirementKey": "CALCULATE_TAB_OK"
    },
    {
      "modelName": "AirplaneCalculatorModel",
      "requirementKey": "LOGIN"
    },
    {
      "modelName": "AirplaneCalculatorModel",
      "requirementKey": "COUNTRIES_TAB_OK"
    },
    {
      "modelName": "AirplaneCalculatorModel",
      "requirementKey": "SHORTESTPATH_GENERATOR"
    },
    {
      "modelName": "AirplaneCalculatorModel",
      "requirementKey": "MINIMUMPATH_GENERATOR"
    },
    {
      "modelName": "AirplaneCalculatorModel",
      "requirementKey": "AIRPLANES_TAB_OK"
    }
  ],
```

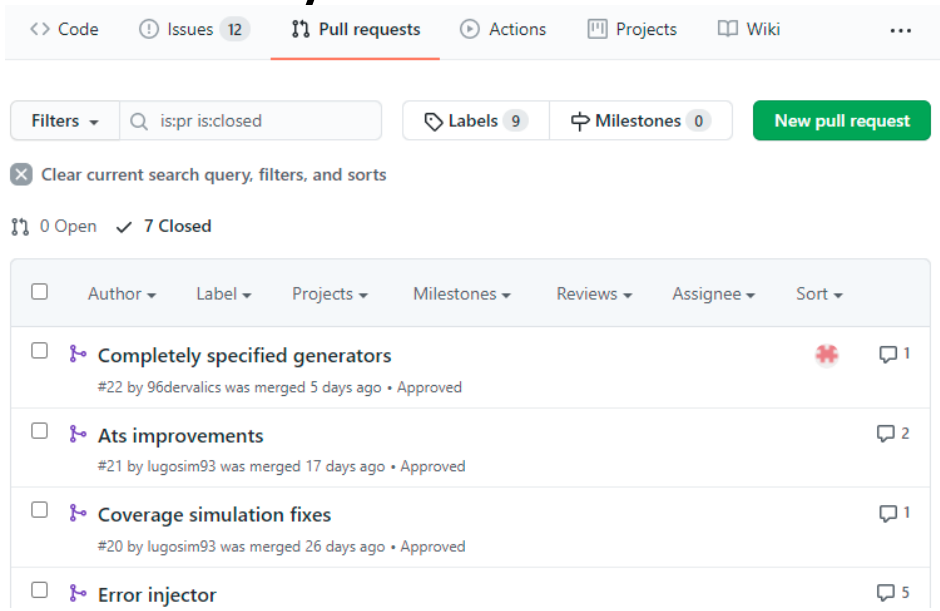

↑ *Requirement coverage in GraphWalker*

HW ToC

Appendix D: Example homework XII. 3/1

Spring 2020/2021

Péter Kókai:
Github Pull Request lifecycle



- SUT (System Under Test): Github Pull Request functionality
  - States: draft, open, merged, closed
- Testing:
  - Model-based testing with fMBT
  - Glue code (adaptation code):
    - Using the CLI of github
    - Using the web interface of Github with Selenium

Examples for CLI glue codes ↓

```
def OpenNewPR(self):
    self.prid = self.__get_next_pr_id()
    branch_name = git_new_branch(self.prid)
    sh.gh.pr.create("--fill", "--head",
                    branch_name,
                    _env=get_env("github-fmbt.token"))
    return self.prid

def ClosePR(self):
    sh.gh.pr.close(self.prid, _env=get_env())
```

Example glue code for testing web interface with Selenium ↓

```
def Open2Draft(self):
    self.__ClickOnText("Convert to draft")
    for_sure=self.driver.find_element_by_xpath("...")
    for_sure.click()
```

CLI: Command Line Interface

HW ToC

Appendix D: Example homework XII. 3/2

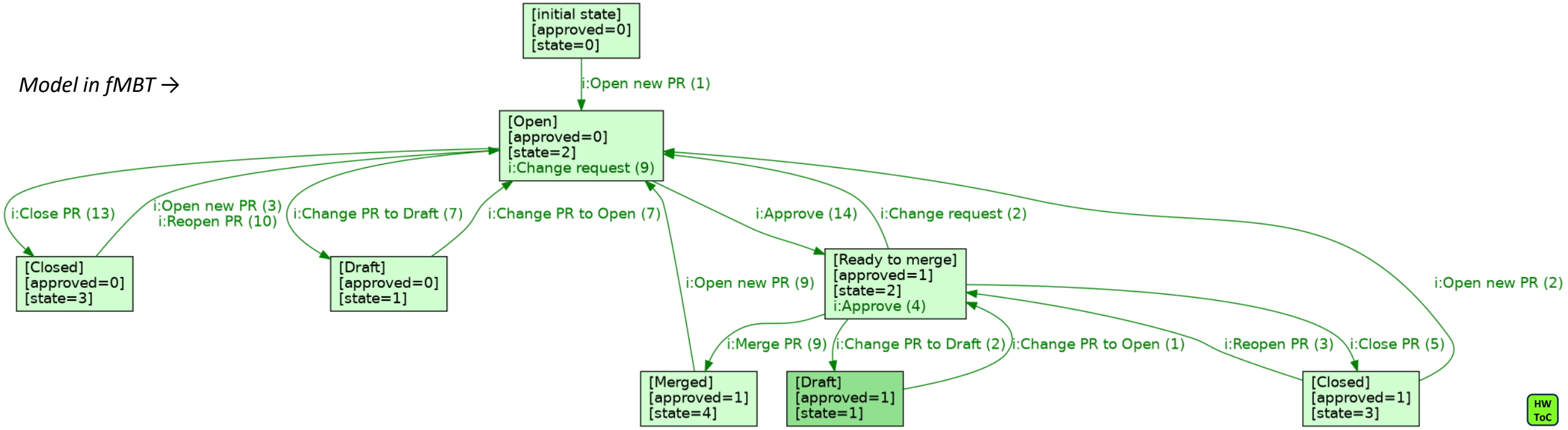
Péter Kókai:
Github Pull Request lifecycle

Spring 2020/2021

Model description in AAL form (truncated) →

```
# preview-show-vars: state, approved
aal "github" {
    language "python" {
        import os
        from git_adapter import *
    }
    variables { state, approved, SUT }
    initial_state {
        state = PRState.NONE
        approved = 0
    }
    adapter_init {
        log("Start")
        SUT=GithubPR("XYZ/github-fmbt")
    }
    adapter_exit {
        if verdict == "pass":
           log("PASS")
        log("cleaning up ")
    }
    input "Open new PR" {
        guard()   { return (state in [PRState.NONE, PRState.CLOSED, PRState.MERGED]) }
        adapter() {
                    prid = SUT.OpenNewPR()
                    log("Create PR {}".format(p))
                  }
        body()    {
                    state = PRState.OPEN
                    approved = 0
                  }
    }
...

    tag "Open" {
        guard()   { return (state == PRState.OPEN) and (0 == approved) }
        adapter() { assert SUT.GetPRStatus() == PRState.OPEN }
    }
...
}
```

Model in fMBT →



HW ToC

Appendix D: Example homework XII. 3/3

Spring 2020/2021

Péter Kókai:
Github Pull Request lifecycle

- Test execution in fMBT:

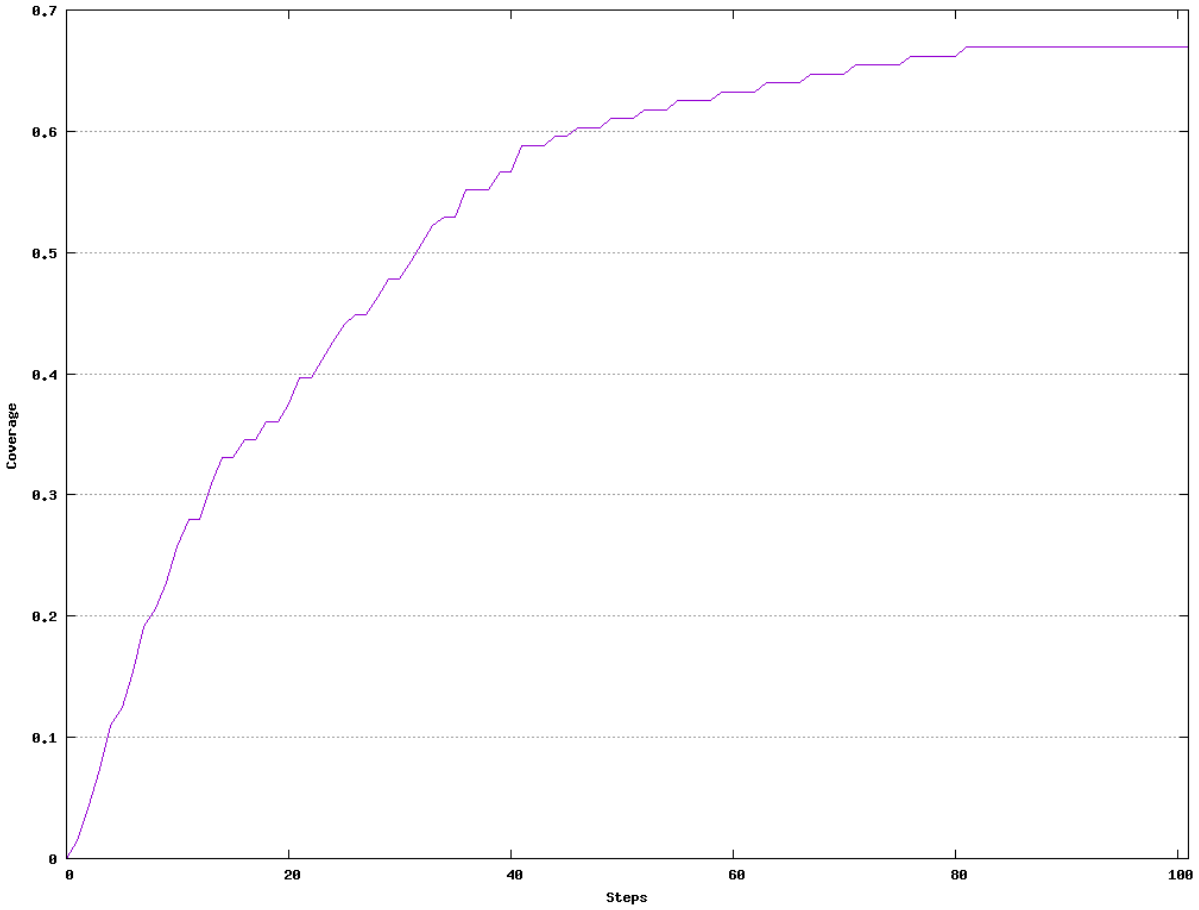
```
heuristic = lookahead(5)
coverage = sum(perm(3, '.*', '.*', 'i:Merge PR'), perm(3, '.*', '.*', 'i:Close PR'))
pass = noprogress(20)
```

..where:

- **lookahead(5)**: generates the possible new walks with 5 length
- **coverage**: defined as a sum of inputs trios where the input ends with either Merge PR or Close PR
- **pass**: if coverage is not increased in 20 step, it stops

Statistics of test execution:

min[ms]	med[ms]	max[ms]	total[ms]	called	"input"
14.794	17.683	27.739	183	10	"i:Change PR to Open"
28.242	33.088	45.721	349	10	"i:Reopen PR"
14.828	24.331	58.756	425	16	"i:Open new PR"
48.486	51.015	83.872	488	9	"i:Merge PR"
43.141	53.403	99.563	555	10	"i:Change PR to Draft"
43.125	50.662	66.917	566	11	"i:Change request"
44.284	52.728	81.250	902	16	"i:Close PR"
43.311	45.685	77.796	960	19	"i:Approve"



↑ *Coverage after each test step*

```
i:Open PR
i:Close PR
i:Reopen PR
i:Change PR to Draft
i:Change PR to Open
i:Approve
i:Approve
...
```

← *Sequence of inputs in log file ()*

HW ToC

Appendix D: Example homework XIII. 3/1

Spring 2020/2021

Dávid Oláh: CNC Controller

- SUT (System Under Test): program that controls a 3-axels CNC (Computer Numerical Control) machining tool
  → Each axis has a stepper motor: can be moved backwards/forward, can be switched on-off
- Testing:
  - Model-based testing with GraphWalker using 2 models:
    (1) *TestDir:* tests all possible transactions between axis directions
    (2) *TestEn:* tests transactions between all on-off states
  - Glue code (adaptation code), interfaces:
    - Test code is implemented in java → C++ code adapted with a dll file
      - JNA (Java Native Access) is used to load the dll file in Java code:

```
private static CNCInterface dll = Native.load("cnc", CNCInterface.class);
```

    - USB controller with the following pins:

Source of the fig.: cncworld.hu

Pin	Function	Controll type	Pin	Function	Controll type
0	x axis step	impulse	6	z axis step	impulse
1	x axis negative direction	high	7	z axis negative direction	high
2	x axis switch off	high	8	z axis switch off	high
3	y axis step	impulse	9	nothing	-
4	y axis negative direction	high	10	nothing	-
5	y axis switch off	high	11	nothing	-

DLL: Dynamic Link Library
USB: Universal Serial Bus

HW ToC

Appendix D: Example homework XIII. 3/2

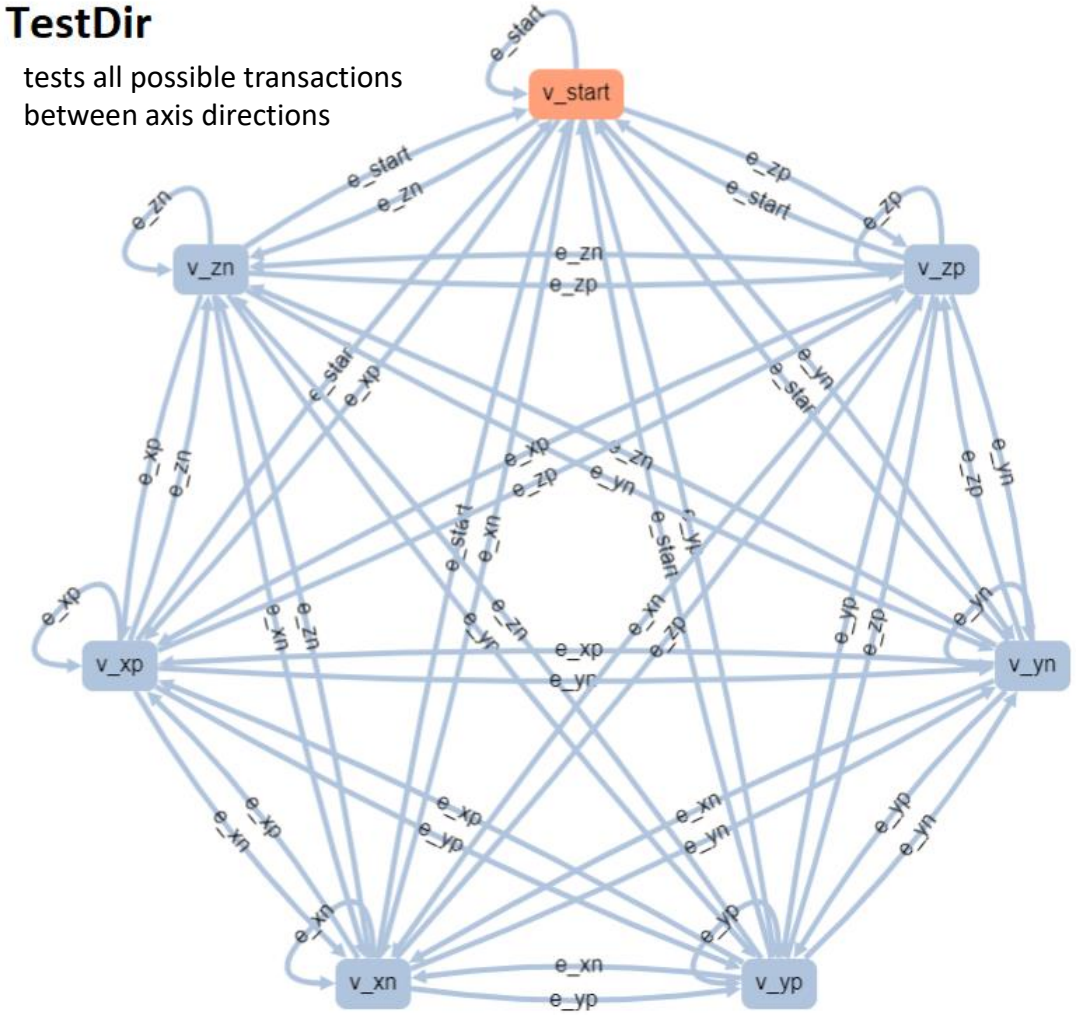
Spring 2020/2021

Dávid Oláh: CNC Controller

Models in GraphWalker:

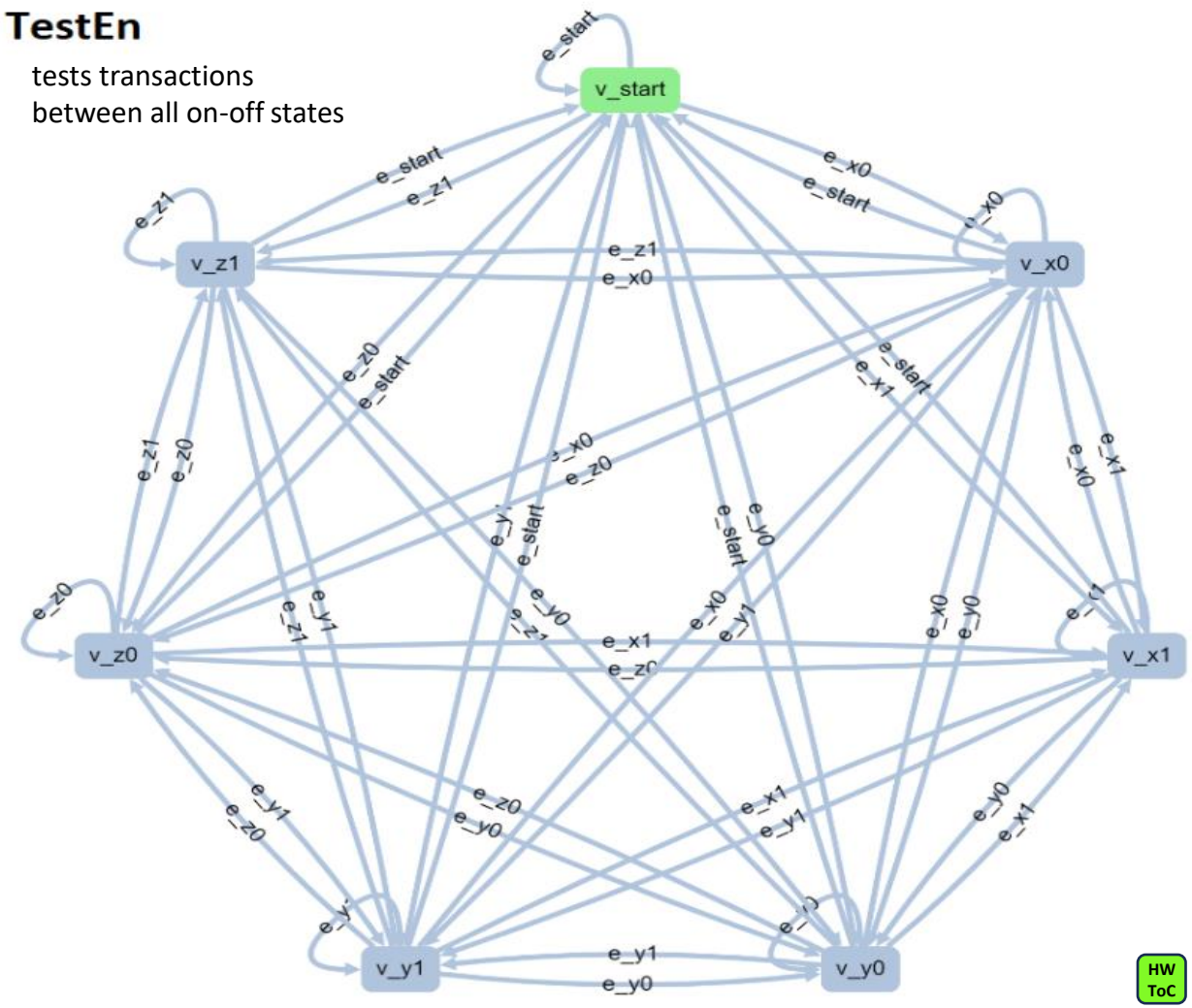
TestDir

tests all possible transactions
between axis directions



TestEn

tests transactions
between all on-off states



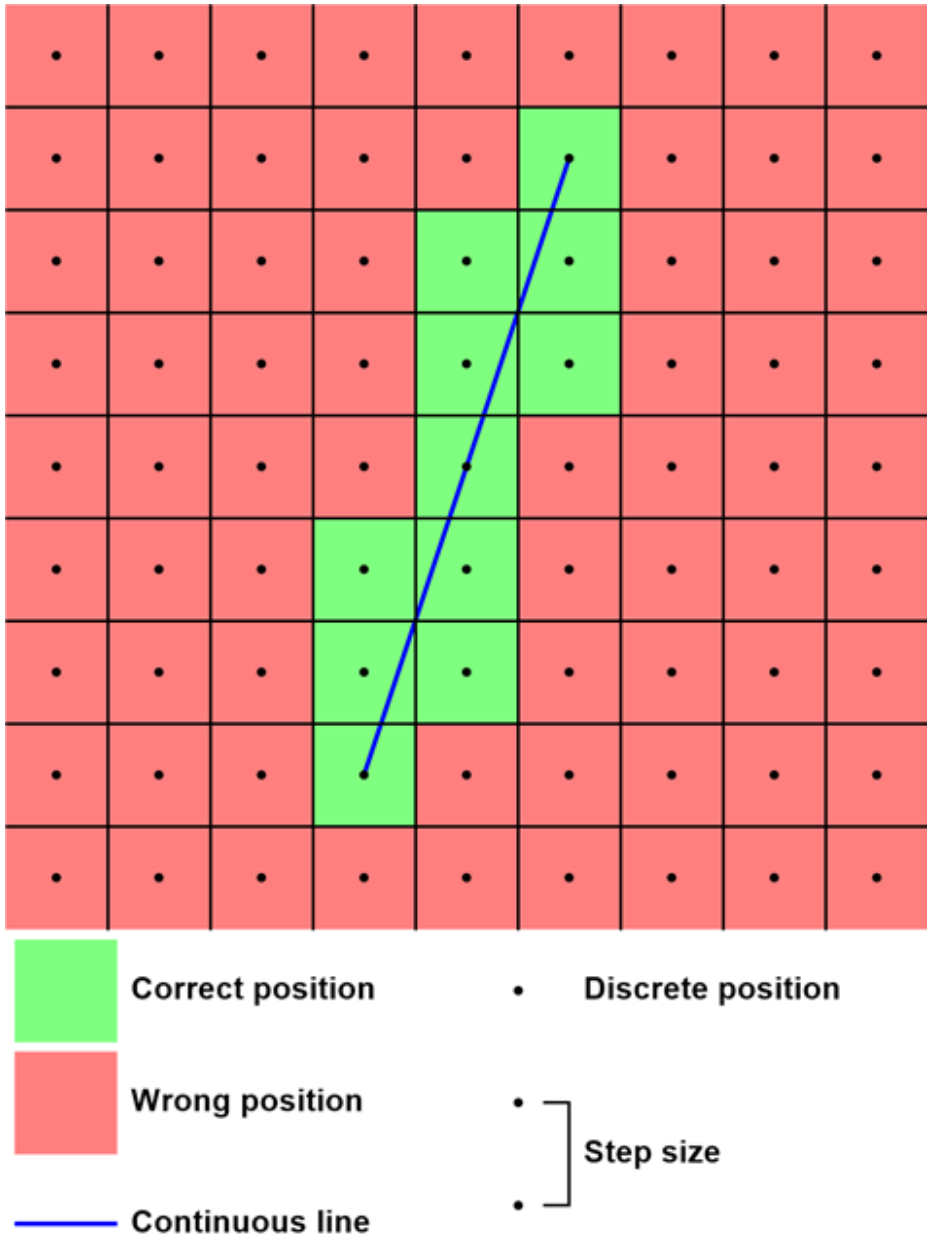
HW ToC

Appendix D: Example homework XIII. 3/3

Spring 2020/2021

Dávid Oláh: CNC Controller

- Unit tests:
  - Test the precision and the ability of the tool
  - These tests are focused on measuring the distance between the discrete positions of the axels and the continuous line fitted to the start and end point of the machine's path. The value of that distance cannot be greater than the amount of distance between two steps measured in millimeters.



HW ToC

Appendix D: Example homework XIV.

Spring 2020/2021

Huang Hongzhi: Arcade Game

- SUT (System Under Test):
  - Simplified arcade game simulation with a javascript web UI

- Testing:
  - Model-based testing with GraphWalker
  - Glue code (adaptation code), interfaces: Selenide

The tested, simplified web UI:

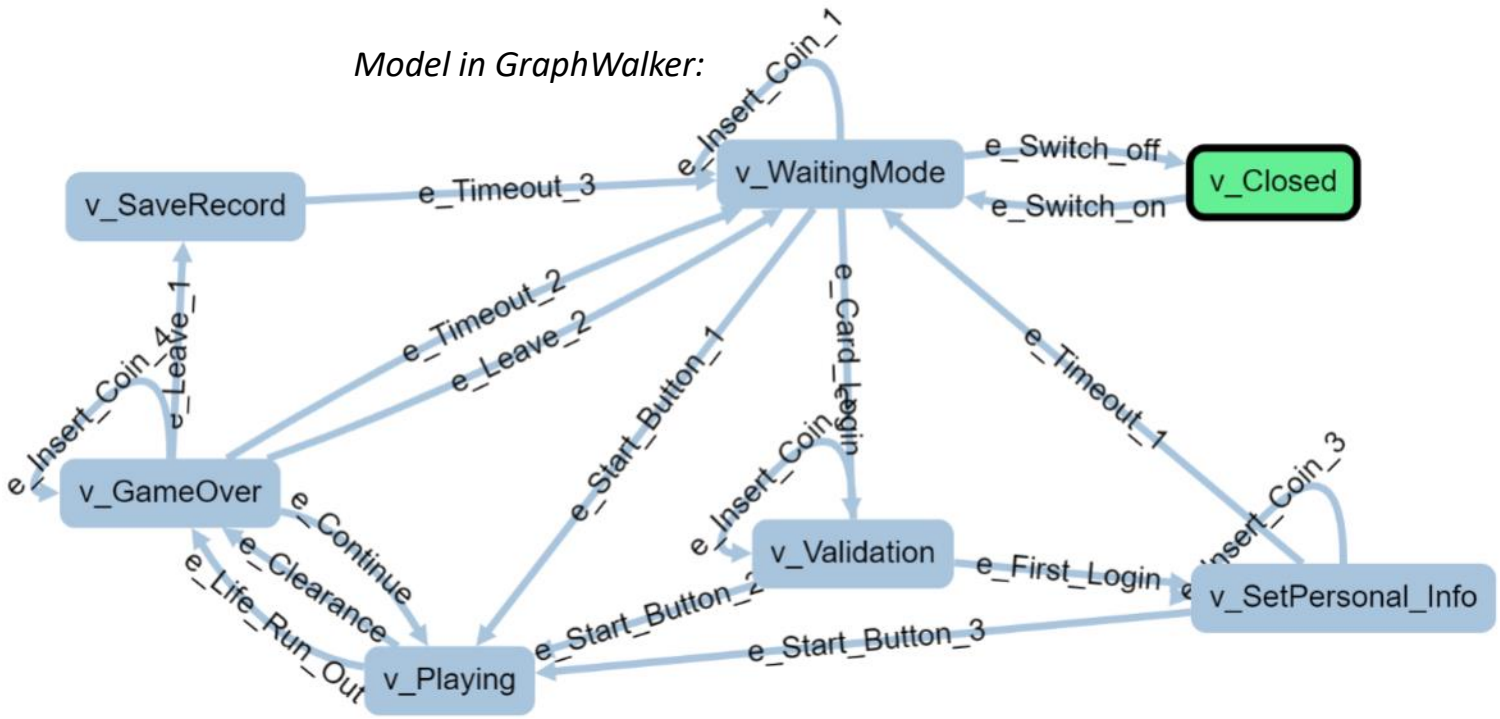
Power | Insert Coin | Cardlogin | Start

Power	Coin	Name	Life	Mode
on	3	guest	0	Waiting_Mode

Test execution in GraphWalker:

```
In state: v_GameOver
Traverses transition: e_Insert_Coin_4
In state: v_GameOver
Traverses transition: e_Continue
In state: v_Playing
Traverses transition: e_Clearance
In state: v_GameOver
Traverses transition: e_Leave_2
In state: v_WaitingMode
[INFO] ------------------------------------------------------------------------
[INFO]
[INFO] Result :
[INFO]
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 19,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 19,
  "totalNumberOfVisitedVertices": 7,
  "edgeCoverage": 100,
  "totalNumberOfVertices": 7,
  "totalNumberOfUnvisitedEdges": 0
}
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  52.983 s
[INFO] Finished at: 2021-04-29T20:47:50+08:00
[INFO] ------------------------------------------------------------------------
```

Model in GraphWalker:



UI: User Interface

HW ToC

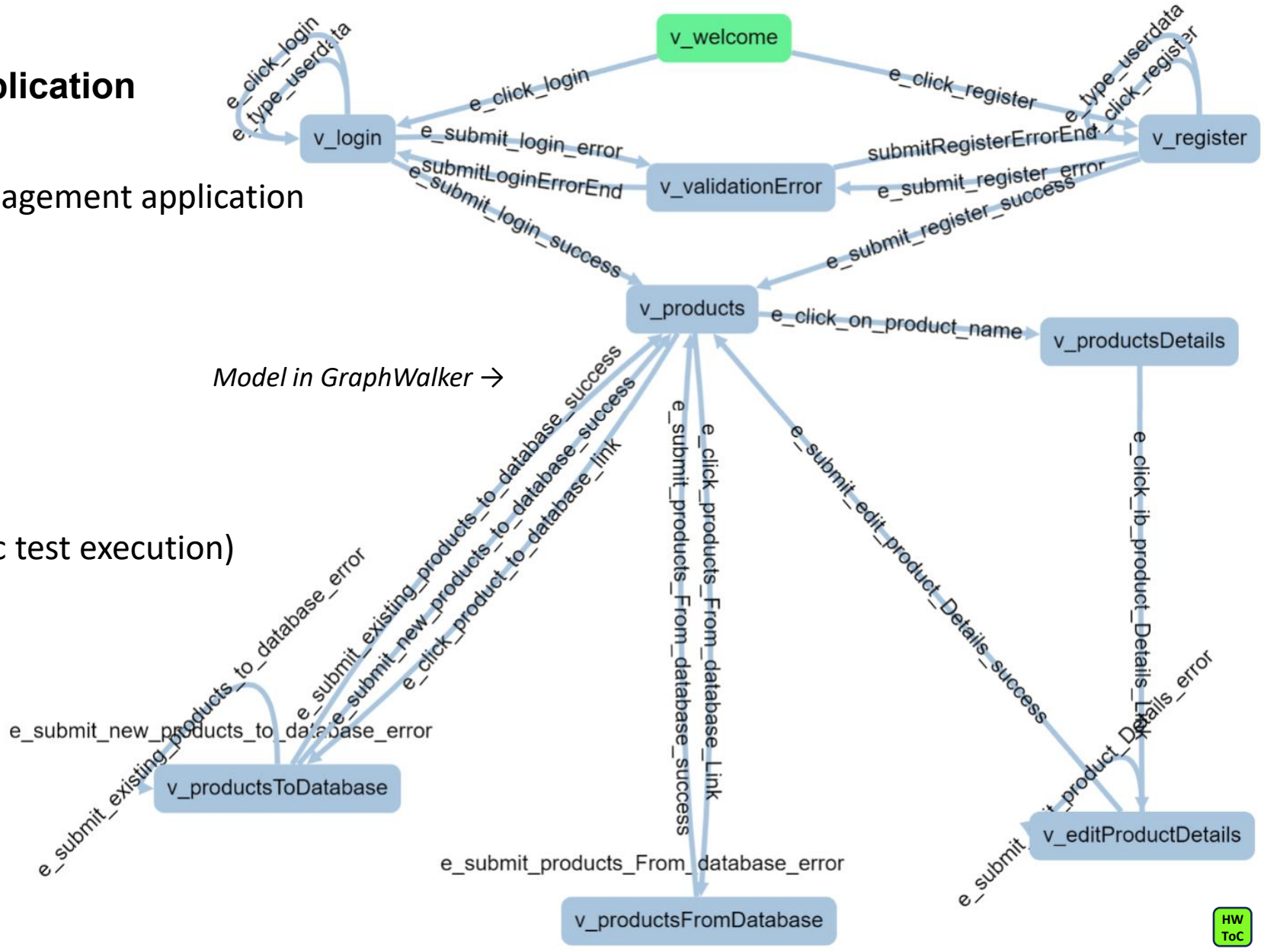
Appendix D: Example homework XV. 2/1

Spring 2020/2021

Csaba Dajka:
Warehouse management application

- SUT (System Under Test): Warehouse management application
  - Functionalities:
    - Registration, login
    - Add/remove/edit products
  - Main parts:
    - Implementation in Laravel
    - Storage: MySQL database
- Testing:
  - Model in GraphWalker (only symbolic test execution)
  - Harmony tests

Model in GraphWalker →



HW ToC

Appendix D: Example homework XV. 2/2

Spring 2020/2021

Csaba Dajka: Warehouse management application

Harmony tests for adding products to database:

Precondition:

```
SUB To ProductsToDatabe when the Customer is not on the products page:
IF ProductsToDatabase IS #non-present
    WHEN #URL IS ProductToDatabase

SUB To ProductsToDatabe when the Customer is on the products page:
IF ProductsToDatabase IS #present
    WHEN ProductsToDatabase IS #pressed

SUB Products To Database page is opened:
WHEN To ProductsToDatabe when the Customer is not on the products page
WHEN To ProductsToDatabe when the Customer is on the products page
```

A negative test:

```
The Customer tries add new product to the database, without
"NewProductName" field filled:
WHEN NewProductCategory IS TestCategory
AND NewProductAmount IS 12
AND NewProductUnit IS m2
AND NewProductTax IS 27
AND NewProductNettPrice IS 12500
AND NewProductDescription IS This is a test NewProduct
AND NewProudctCreate IS #pressed
THEN ProductsJumbotron IS #non-present
```

A positive test:

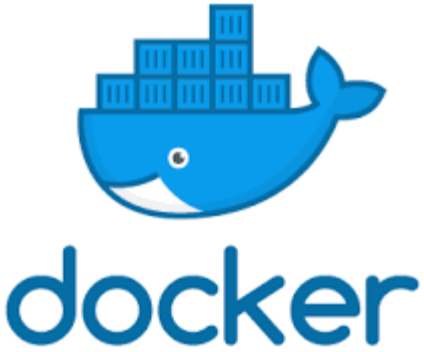
```
Customer adds a new product to the database:
WHEN #URL IS ProductToDatabase
AND  NewProductName IS Test2
AND NewProductCategory IS TestCategory
AND NewProductAmount IS 12
AND NewProductUnit IS m2
AND NewProductTax IS 27
AND NewProductNettPrice IS 12500
AND NewProductDescription IS This is a test NewProduct
AND NewProudctCreate IS #pressed
THEN ProductName-2 IS Test2
AND ProductCategory-2 IS TestCategory
AND ProductAmount-2 IS 12
AND ProductUnit-2 IS m2
```

HW ToC

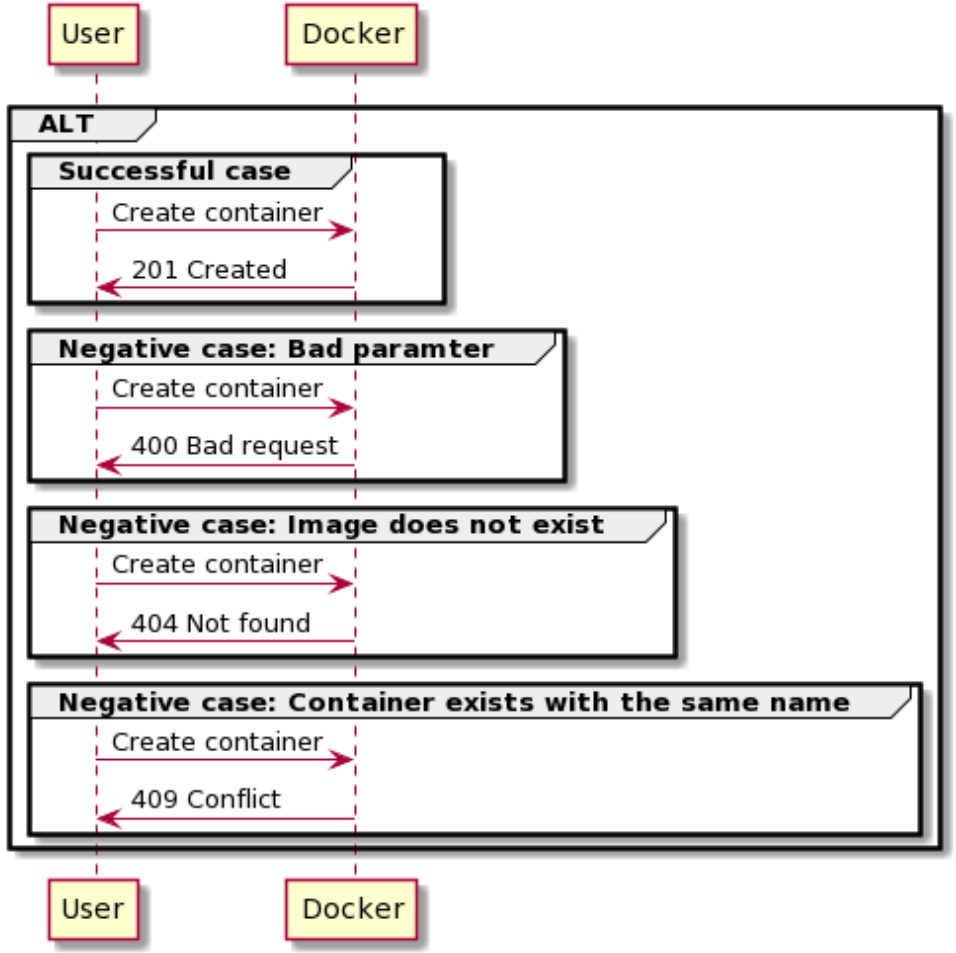
Appendix D: Example homework XVI. 4/1

Fall 2021/2022

Péter Kaszab: Docker Engine API



- SUT (System Under Test): Docker
  - Tested functionalities – (1.41 API specification):
    - Manage images, networks and volumes
    - Create and modify containers, Container life-cycle
- Testing:
  - Model-based testing with fMBT
  - Glue code (adaptation code):
    - Written in Python
    - Connects to the Docker Engine through HTTP



↑ *UML sequence diagram of container creation*

API: Application Programing Interface
UML: Unified Modeling Language

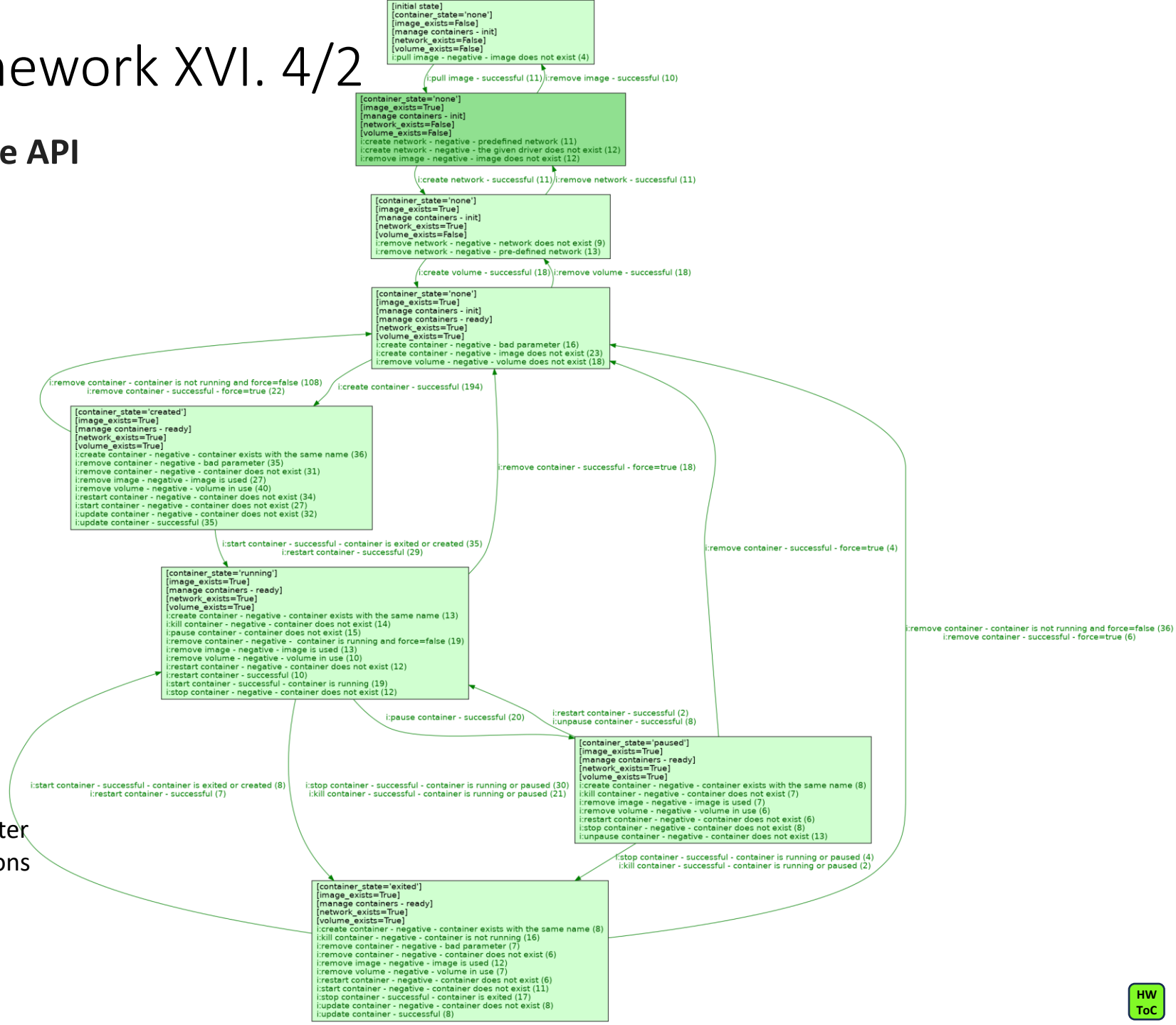
HW ToC

Appendix D: Example homework XVI. 4/2

Fall 2021/2022

Péter Kaszab: Docker Engine API

1. The model in fMBT:



Variables:

- `container_state`: possible values are `none`, `created`, `running`, `paused`, and `exited`
- `image_exists`: boolean variable
- `volume_exists`: boolean variable
- `network_exists`: boolean variable

Tags:

Guard conditions that are either true or false:

- `manage containers - init`: if true, the tester can run image, volume, and network manager actions
- `manage containers - ready`: if true, the tester can run container manager actions

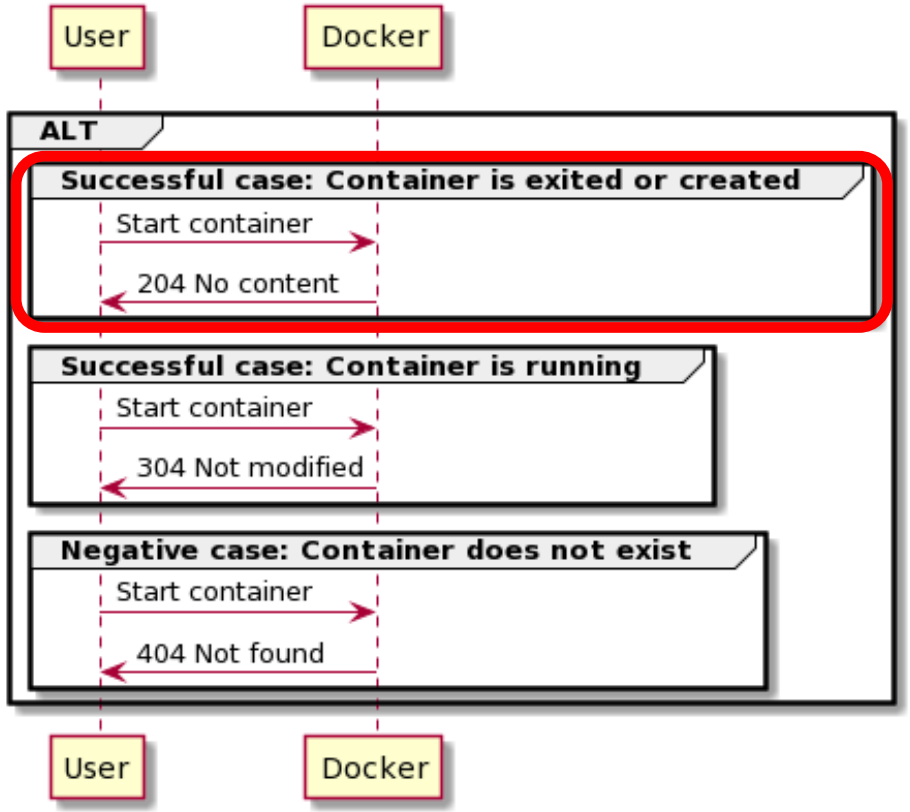
HW ToC

Appendix D: Example homework XVI. 4/3

Fall 2021/2022

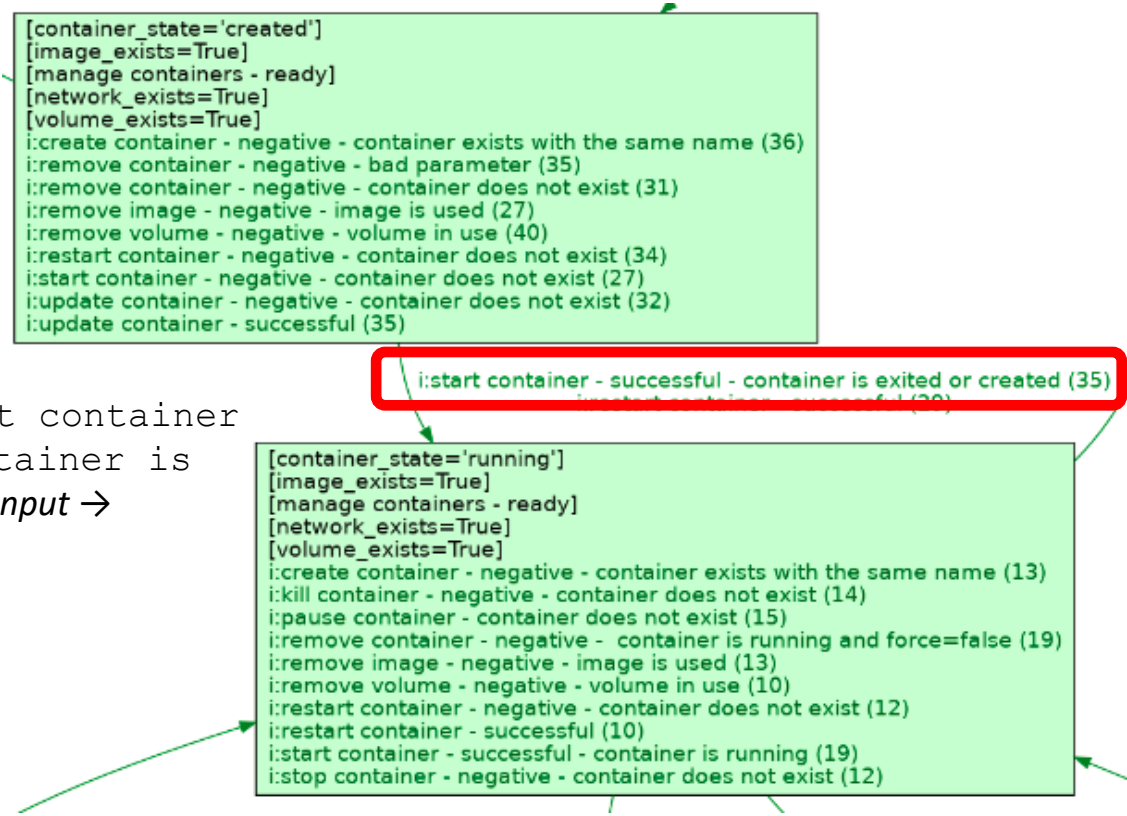
Péter Kaszab: Docker Engine API

2. An example for testing a Docker functionality:



↑ *UML sequence diagram: start container*

fMBT model part for `start container - successful - container is exited or created` *input →*




```
input "start container - successful - not running" {
guard {
return container_state in [
CONTAINER_STATE_CREATED,
CONTAINER_STATE_EXITED
]
}
body { container_state = CONTAINER_STATE_RUNNING }
adapter {
res = client.post("/containers/"
+ TEST_CONTAINER_NAME + "/start")
assert res.status_code == 204, "Status code 204 expected"
```

Adaptation code for the `start container - successful - container is exited or created` *input →*

HW ToC

Appendix D: Example homework XVI. 4/4

Fall 2021/2022

Péter Kaszab: Docker Engine API

3. The tests:

smoke.conf *Simulates execution of all possible paths of length 4. The test passes when every action in the model has been executed at least once.*

```
heuristic = lookahead(4)
coverage = perm(1)
pass = coverage(1)
```

random.conf *Chooses randomly from the available input actions, runs for 5 minutes.*

```
heuristic = random(42)
coverage = perm(2)
pass = duration(5 minutes)
```

init-tag.conf *Simulates execution of all possible paths of length 4. The test passes when the lookahead algorithm cannot improve the coverage.*

```
heuristic = lookahead(4)
coverage = usecase(@["manage containers - init"](perm(2)))
pass = lookahead_noprogress
```

ready-tag.conf *Simulates execution of all possible paths of length 6. The test passes when the coverage stays the same for 20 steps.*

```
heuristic = lookahead(6)
coverage = usecase(@["manage containers - ready"](perm(2)))
pass = no_progress(20)
```

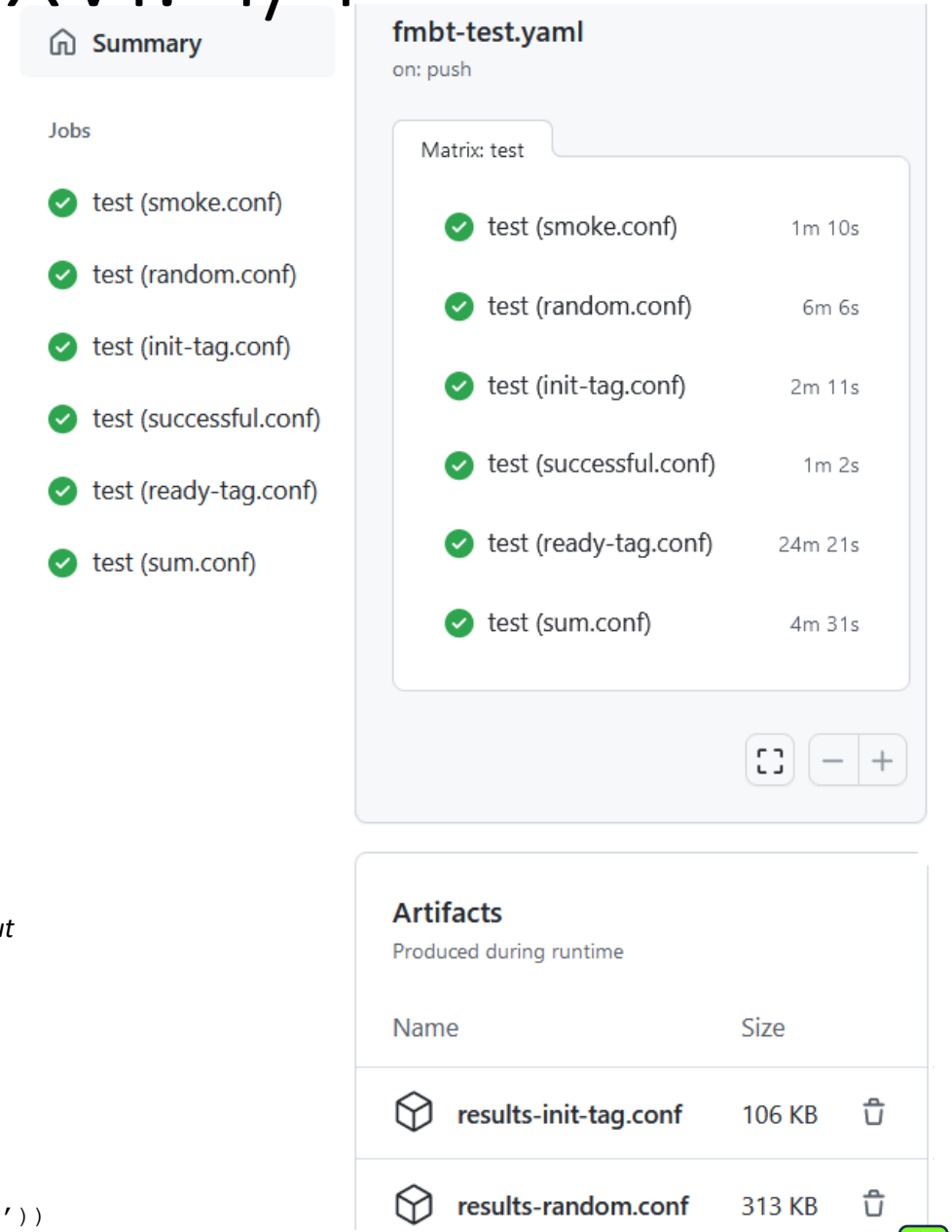
successful.conf *Simulates execution of all possible paths of length 4. It includes only 2 permutations successful input actions in coverage. The test passes when the lookahead algorithm cannot improve the coverage.*

```
heuristic = lookahead(4)
coverage = include('.*successful.*', perm(2))
pass = lookahead_noprogress
```

sum.conf *Use the random heuristic with 20% probability and lookahead(4) with 80% probability*

```
heuristic = mrandom(20,random(42),80,lookahead(4))
coverage = sum(perm(3, 'i:create container - successful', '.*', '.*'),
               perm(3, '.*', '.*', 'i:remove container - container is not running and force=false'))
pass = noprogress(30)
```

↑ *fMBT tests*



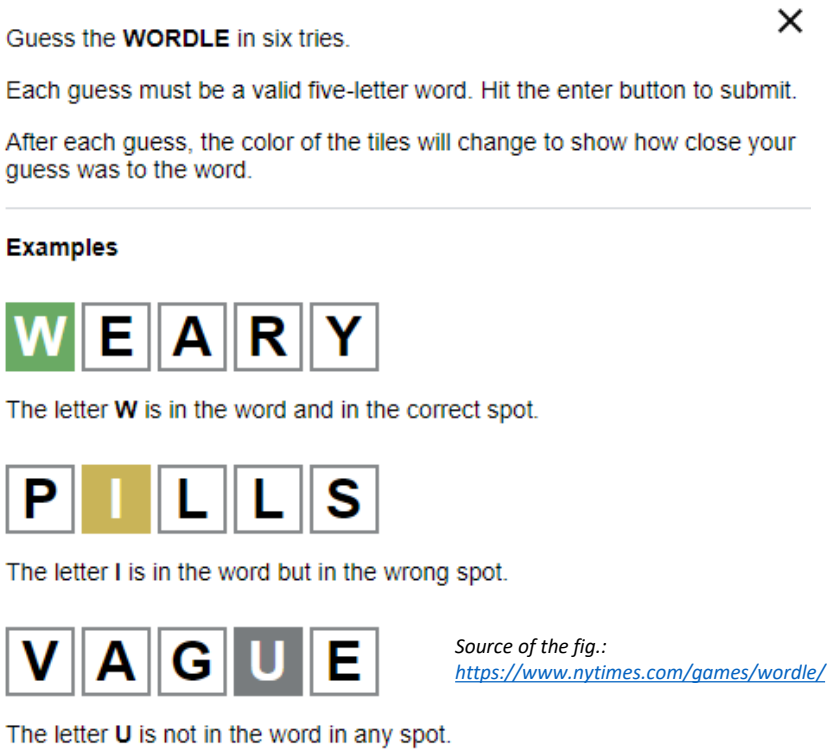
↑ *Test results in GitHub Actions web interface*

HW ToC

Appendix D: Example homework XVII. 2/1

Spring 2021/2022

Angéla Czirkos: Wordle application



Source of the fig.: https://www.nytimes.com/games/wordle/

```
07:10:08.177 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.WordleTest@5364681b
07:10:08.177 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute method: 'e_OptionPrintLetters' in model: 'WordleEFSM'
Running: e_OptionPrintLetters
07:10:08.178 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.WordleTest@5364681b
07:10:08.178 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute method: 'v_PrintLetters' in model: 'WordleEFSM'
Running: v_PrintLetters
07:10:08.179 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.WordleTest@5364681b
07:10:08.179 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute method: 'e_FinishAction' in model: 'WordleEFSM'
Running: e_FinishAction
07:10:08.179 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.WordleTest@5364681b
07:10:08.179 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute method: 'v_ChooseMenuOption' in model: 'WordleEFSM'
Running: v_ChooseMenuOption
 a  b  c  d  e  f  g  h  i  j  k  l  m
 n  o  p  q  r  s  t  u  v  w  x  y  z

Choose from the following options:
     0 : print current state of game (words and evaluations)
     1 : enter new word guess (5 characters, lowercase, English alphabet)
     2 : ask wordle solver for best guess
     3 : print remaining vocabulary
     4 : print letter status
     5 : # of remaining guesses
     6 : end game
Type guess:
07:10:08.515 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.WordleTest@5364681b
07:10:08.515 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute method: 'e_ValidWord' in model: 'WordleEFSM'
Running: e_ValidWord
07:10:08.516 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.WordleTest@5364681b
07:10:08.516 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute method: 'v_EvaluateWord' in model: 'WordleEFSM'
Running: v_EvaluateWord
07:10:08.516 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.WordleTest@5364681b
07:10:08.516 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute action: 'win=true' in model: 'WordleEFSM'
07:10:08.518 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Data: 'WordleTest: com.company.WordleTest@5364681b, gue
sses: 5, playing: true, win: true, '
07:10:08.519 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute method: 'e_CorrectWord' in model: 'WordleEFSM'
Running: e_CorrectWord
07:10:08.519 [main] DEBUG org.graphwalker.core.machine.SimpleMachine - Context: com.company.WordleTest@5364681b
07:10:08.519 [main] DEBUG org.graphwalker.core.machine.ExecutionContext - Execute method: 'v_GameOver' in model: 'WordleEFSM'
Running: v_GameOver
YOU WIN!
leave
lunch
Solution was lunch
Do you want to start a new game? (y/n)
```

↑ *Communication with the SUT*

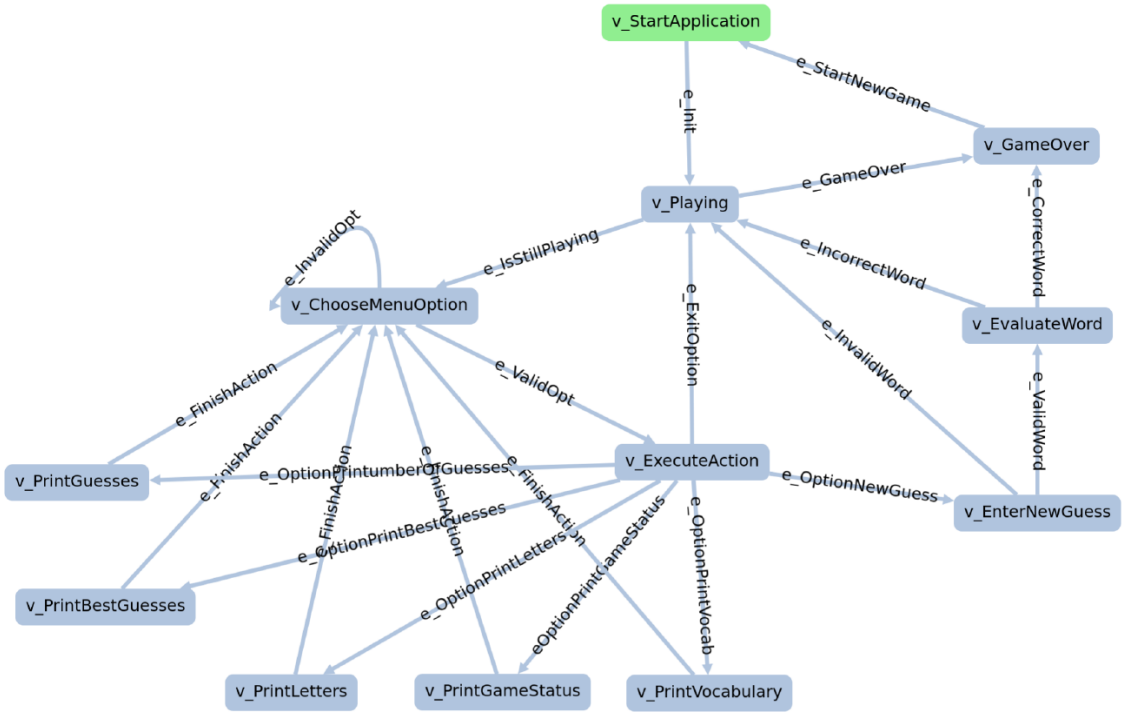
- SUT (System Under Test):
  - Wordle native Julia application (using ZMQ library)
- Testing:
  - Model-based testing with GraphWalker
  - Glue code (adaptation code):
    - Java testing interface

HW ToC

Appendix D: Example homework XVII. 2/2

Spring 2021/2022

Angéla Czirkos: Wordle application



↑ *Model in GraphWalker*

adaptation code →

1. opening a socket and connecting to the server

```
@GraphWalker()
public class WordleTest extends ExecutionContext implements WordleEFSM {

    ZContext context = null;
    ZMQ.Socket socket = null;

    @BeforeExecution
    public void setup(){

      context = new ZContext();
        //  Socket to talk to server
        System.out.println("Connecting to WORDLE SERVER");

        socket = context.createSocket(SocketType.REQ);
        socket.connect(...);
```

2. sending input to the server

```
    @Override
    public void e_OptionPrintBestGuesses(){
      System.out.println("Running: e_OptionPrintBestGuesses");
      String request = "2";
      socket.send(request.getBytes(ZMQ.CHARSET), 0);
    }
```

3. receiving output from the server

```
    @Override
    public void v_ChooseMenuOption(){
      System.out.println("Running: v_ChooseMenuOption");
      byte[] reply = socket.recv(0);
      System.out.println(new String(reply, ZMQ.CHARSET));
    }
```

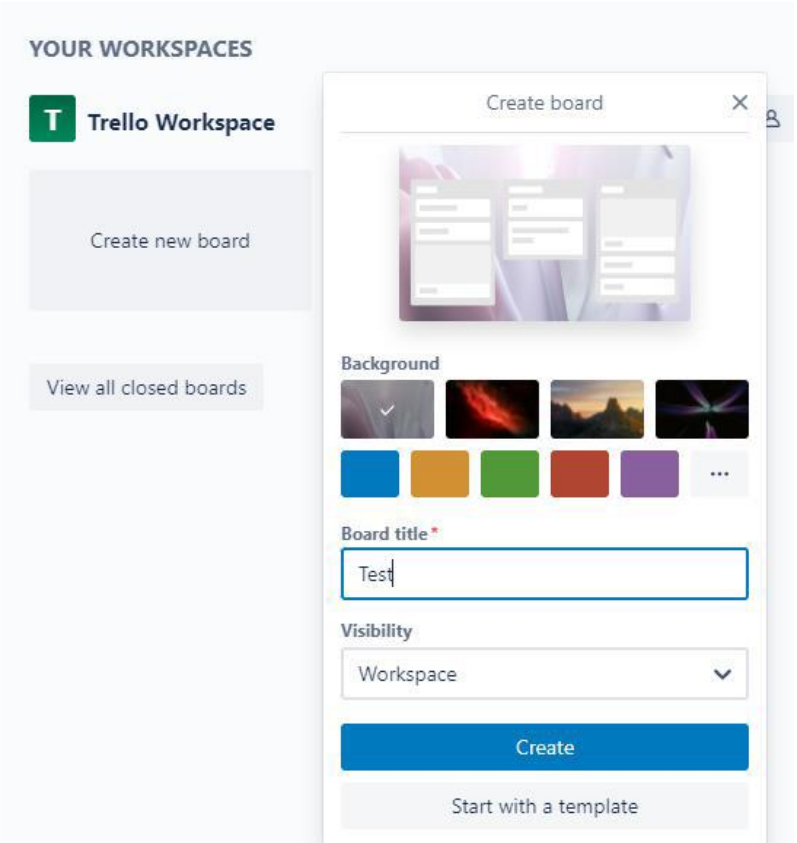
HW ToC

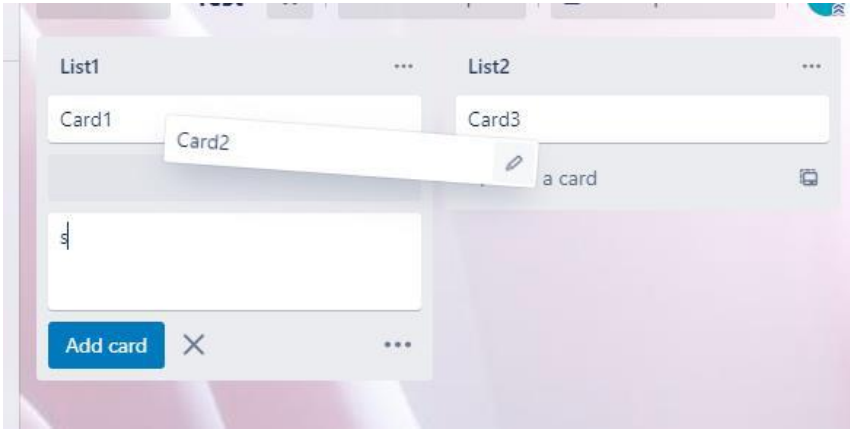
Appendix D: Example homework XVIII. 3/1

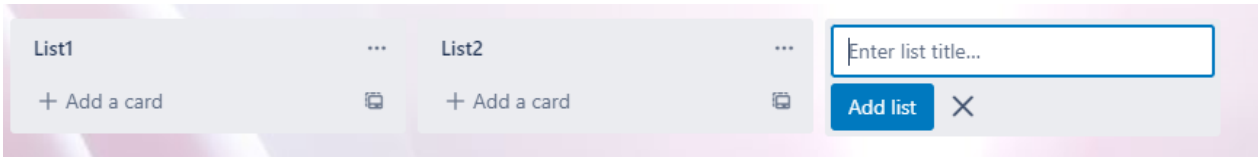
Spring 2021/2022

Andrea Góga: Trello

- SUT (System Under Test):
  - Trello project/task/workflow management tool with web UI
    - Tested functionalities:
      - Creating a new board
      - Creating lists
      - Creating, archiving, and moving cards
      - Deleting a board
- Testing:
  - Model-based testing with GraphWalker
  - Glue code (adaptation code):
    - Selenide





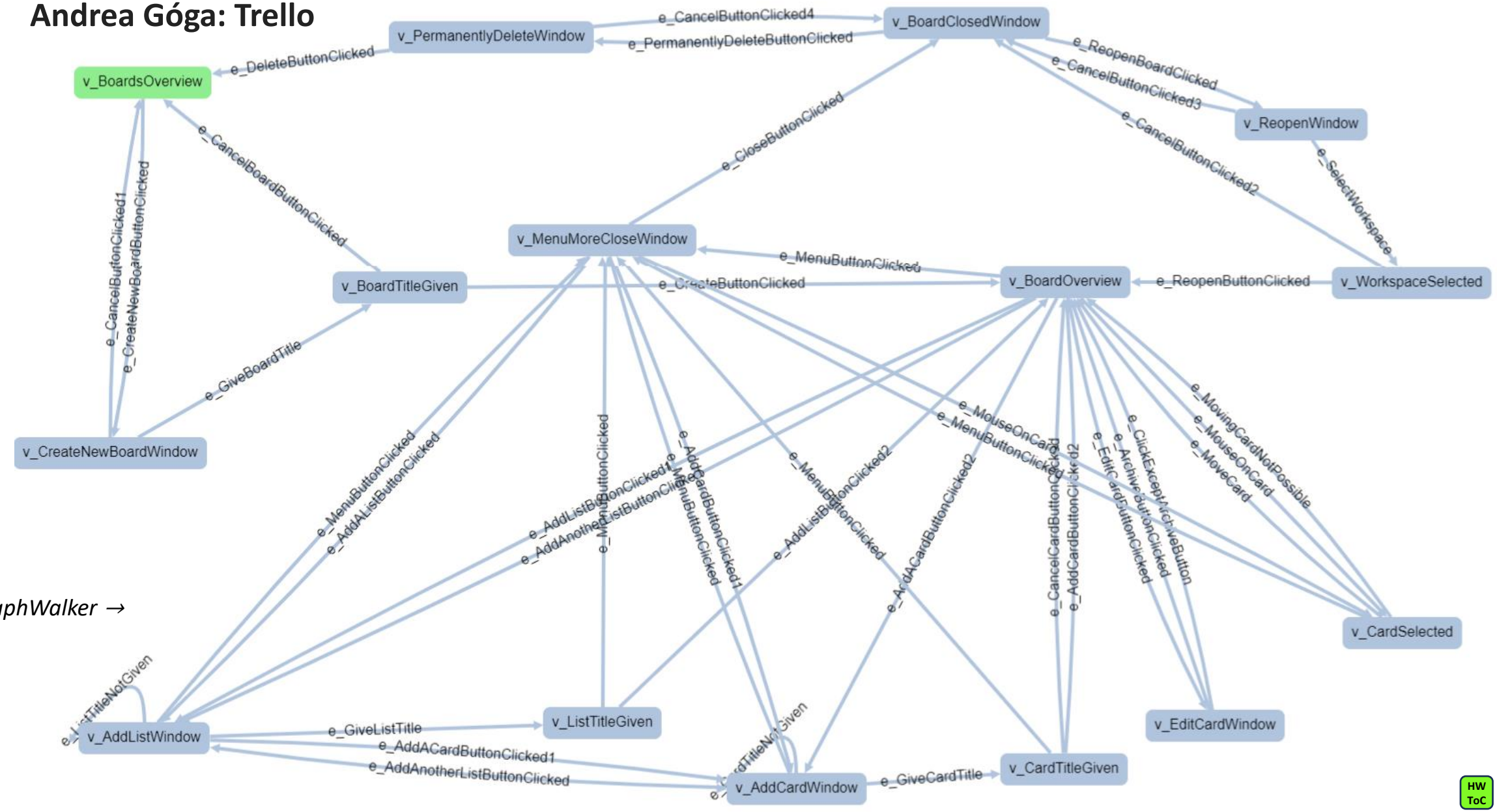


HW ToC

Appendix D: Example homework XVIII. 3/2

Spring 2021/2022

Andrea Góga: Trello



Model in GraphWalker →

HW ToC

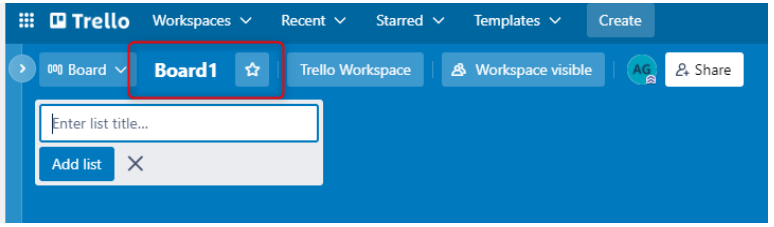
Appendix D: Example homework XVIII. 3/3

Spring 2021/2022

Andrea Góga: Trello

Example 1: Check the name of the board:

1. element is picked by the CSS selector:



2. Checking if the board has the expected name:

```
@Override
public void v_BoardOverview() {

    logger.info("Board overview");
    String board = "Board" + boardCount;
    $( cssSelector: "#content > div > div.board-main-content > " +
            "div.board-header.u-clearfix.js-board-header.collapsed-workspace-nav > " +
            "div.board-header-btn.mod-board-name.inline-rename-board.js-rename-board > h1").shouldHave(text(board));
}
```

Example 2: Check if a window pops up:

```
@Override
public void v_ReopenWindow() {

    logger.info("Reopen window");
    sleep( milliseconds: 1000);
    $( cssSelector: "body > div.atlaskit-portal-container > div > section").isDisplayed();
    $( cssSelector: "body > div.atlaskit-portal-container > div > section > header > div").shouldHave(text("Select a Workspace"));
```

Adaptation code examples

Example 3: Move a card functionality:

```
@Override
public void e_MoveCard() {
    logger.info("Move card");
    sleep( milliseconds: 1000);
    logger.info("cardInList: " + Integer.toString(cardInList));
    //logger.info(Integer.toString(cardLeft));
    //logger.info(Integer.toString(cardRight));
    if (cardInList == 1){
        $( cssSelector: "#board > div:nth-child(1) > div > " +
                "div.list-cards.u-fancy-scrollbar.u-clearfix.js-list-cards.js-sortable.ui-sortable > " +
                "a:nth-child(1) > div.list-card-details.js-card-details").dragAndDropTo( s: "#board > " +
                "div:nth-child(2) > div");
        sleep( milliseconds: 500);
        cardLeft--;
        cardRight++;
        // after moving the number of elements on the target list has been increased
        String movedCardPos = "#board > div:nth-child(2) > div > " +
                "div.list-cards.u-fancy-scrollbar.u-clearfix.js-list-cards.js-sortable.ui-sortable > " +
                "a:nth-child(" + cardRight + ") > div.list-card-details.js-card-details";
        $(movedCardPos).isDisplayed();
        cardInList = 2;
    } else if (cardInList == 2) {
        $( cssSelector: "#board > div:nth-child(2) > div > " +
                "div.list-cards.u-fancy-scrollbar.u-clearfix.js-list-cards.js-sortable.ui-sortable > " +
                "a:nth-child(1) > div.list-card-details.js-card-details").dragAndDropTo( s: "#board > " +
                "div:nth-child(1) > div");
        cardInList = 1;
        cardLeft++;
        cardRight--;
        // after moving the number of elements on the target list has been increased
        String movedCardPos = "#board > div:nth-child(1) > div > " +
                "div.list-cards.u-fancy-scrollbar.u-clearfix.js-list-cards.js-sortable.ui-sortable > " +
                "a:nth-child(" + cardLeft + ") > div.list-card-details.js-card-details";
        $(movedCardPos).isDisplayed();
    }
}
```

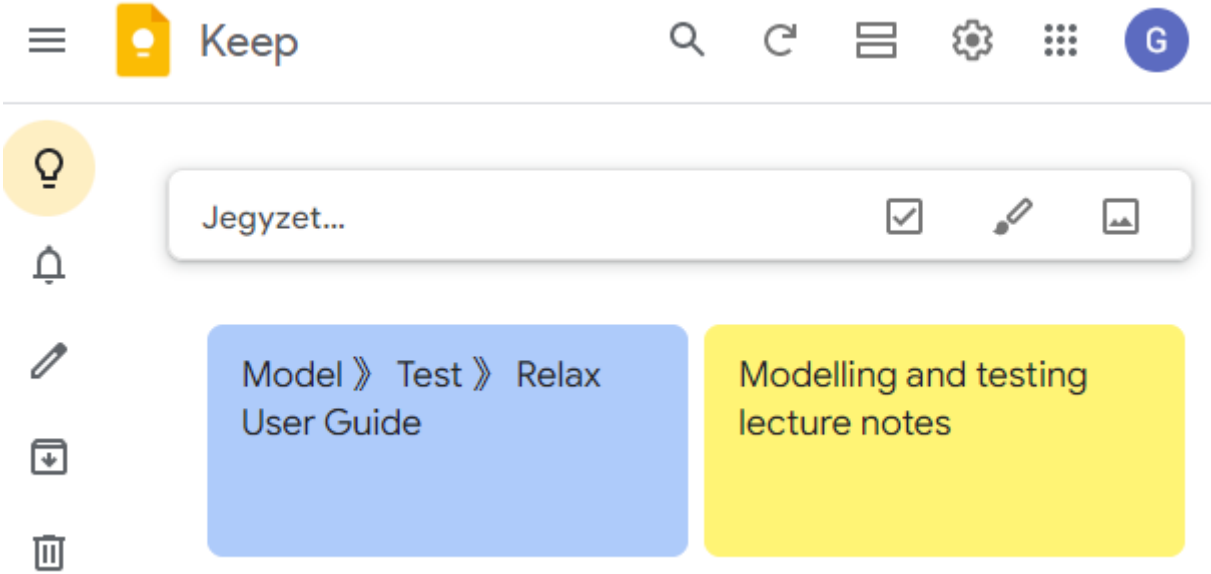
HW ToC

Appendix D: Example homework XIX. 3/1

Spring 2021/2022

Bálint Nádas: Google Keep API

- SUT (System Under Test): Google Keep API
  - Tested functionalities:
    - Login to the API
    - Fetch notes and lists
    - Edit notes (change title, text, colour, pin, archive)
    - Navigate through lists (backward, forward)
    - Edit lists (change text, title, colour)
    - Send new notes and list elements to the API
- Testing:
  - Model-based testing with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python



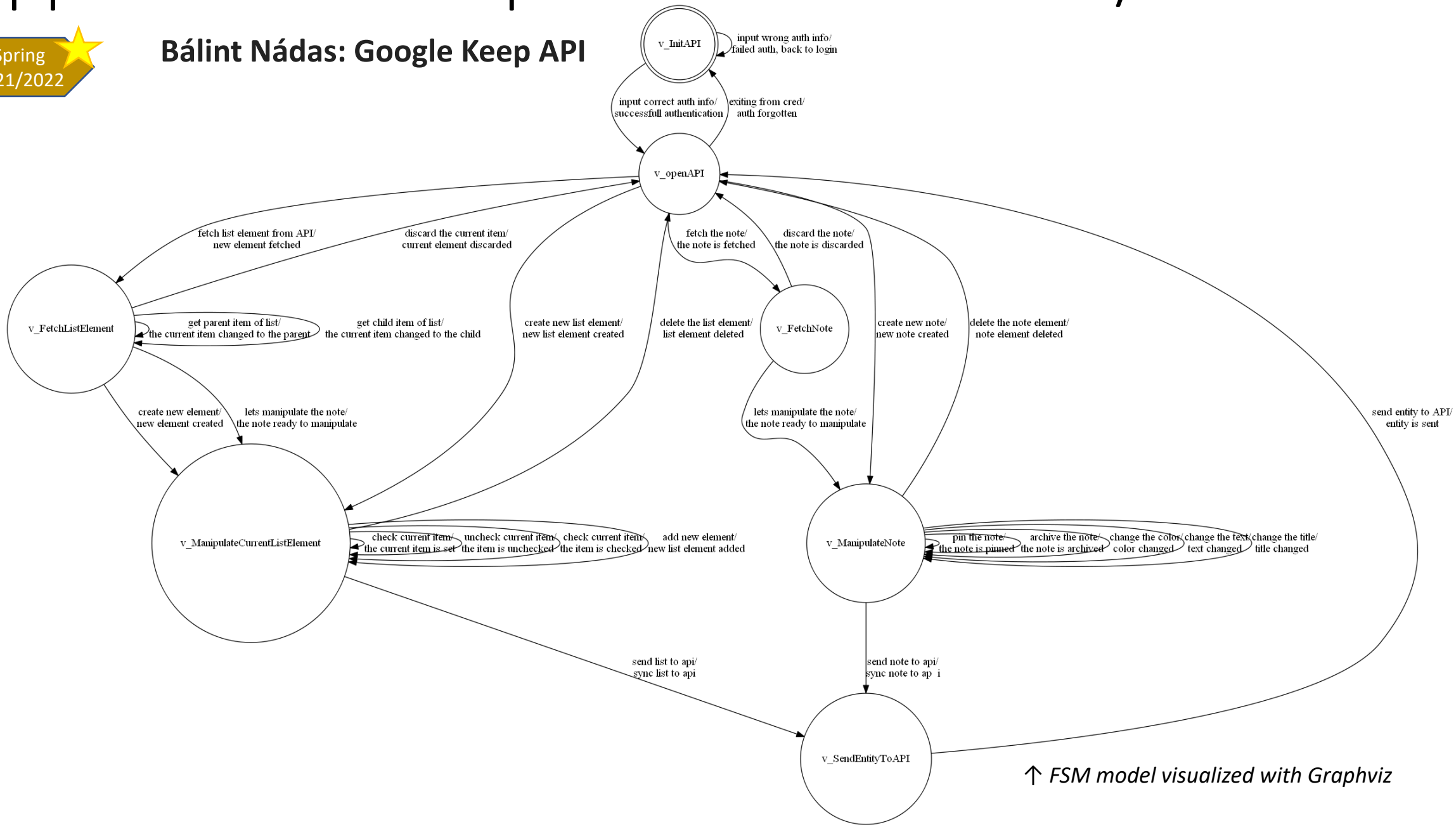
API: Application Programing Interface

HW ToC

Appendix D: Example homework XIX. 3/2

Spring 2021/2022

Bálint Nádas: Google Keep API



↑ *FSM model visualized with Graphviz*

HW ToC

Appendix D: Example homework XIX. 3/3

Spring 2021/2022

Bálint Nádas: Google Keep API

Adaptation code example:

```
## Redefine some constants in order to run the program successfully:
filePath = '' ## path to the generated test suite file
emailaddr = '' ## address to the Google Account which you want to use
goodPsw = '' ## password for the same Account
badPsw = 'badpassword'
testName = 'testName'
testText = 'testText'
testColor = gkeepapi.node.ColorValue.Red
runner = unittest.TextTestRunner()
note = None
item = None
keep = None
…
## Example test cases:
def archiveNote(self):
      self.assertTrue(note.archived)

def wrongAuth(self):
      with self.assertRaises(gkeepapi.exception.LoginException) as cont:
            keep.login(emailaddr, badPsw)
      self.assertTrue('BadAuthentication' in str(cont.exception))

def changeTitle(self):
      self.assertEqual(note.title, testName)

def pinTheNote(self):
      self.assertTrue(note.pinned)
```

```
…

## Example state changes for the input:
def discardCurrentItem():
      global item
      item = None
      runner.run(suiteGenerator(inspect.currentframe().f_code.co_name))

def discardTheNote():
      global note
      note = None
      runner.run(suiteGenerator(inspect.currentframe().f_code.co_name))

def changeTitle():
      note.title = testName
      runner.run(suiteGenerator(inspect.currentframe().f_code.co_name))

def pinTheNote():
      note.pinned = True
      runner.run(suiteGenerator(inspect.currentframe().f_code.co_name))

def archiveNote():
      note.archived = True
      runner.run(suiteGenerator(inspect.currentframe().f_code.co_name))
…
```

```
exiting from cred
.
----------------------
Ran 1 test in 0.000s

OK
input correct auth info
.
----------------------
Ran 1 test in 0.000s
```

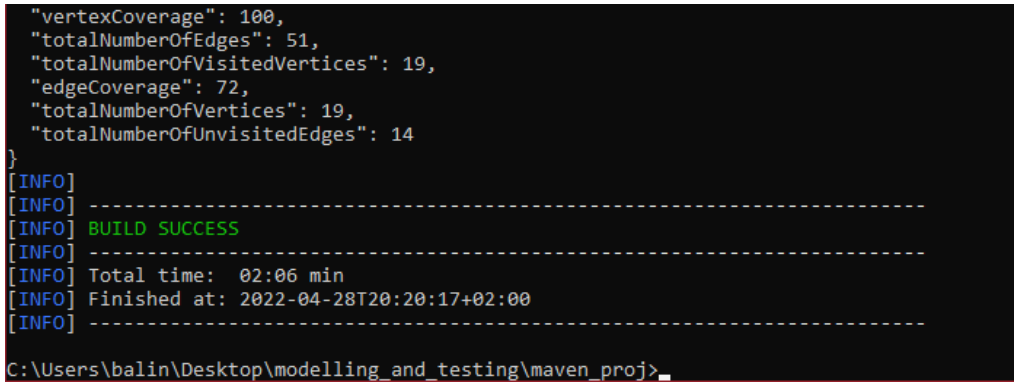
HW ToC

Appendix D: Example homework XX. 3/1

Spring 2021/2022

Bálint Nátrán: Car Rental Registry

- SUT (System Under Test): Car Rental Registry application
  - Tested functionalities:
    - Login
    - Adding/modifying a car entity
- Testing:
  - Model-based testing with GraphWalker
  - Glue code (adaptation code):
    - Using Winium API calls, UISpy tool for element identifications



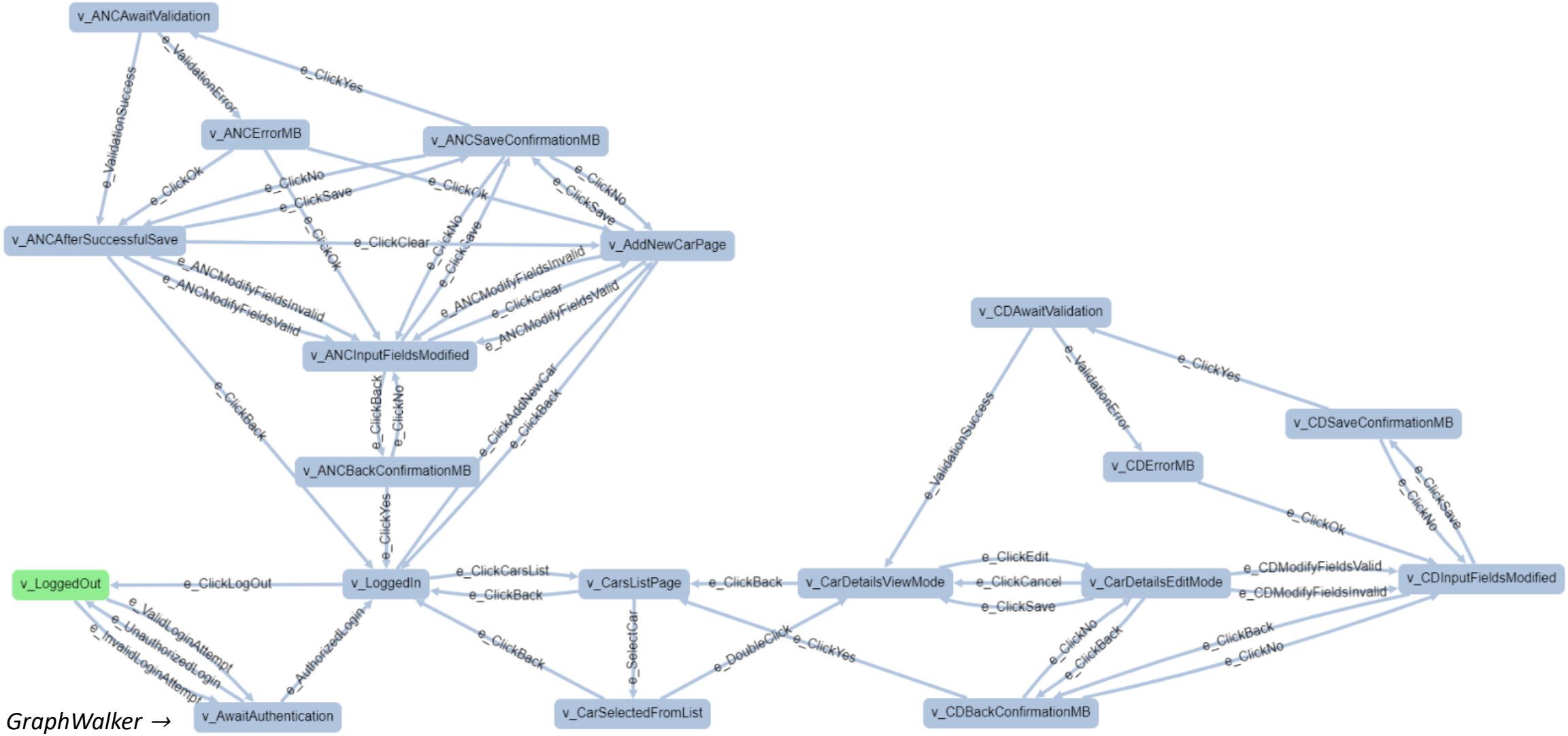
API: Application Programing Interface

HW ToC

Appendix D: Example homework XX. 3/2

Spring 2021/2022

Bálint Nátrán: Car Rental Registry



Model in GraphWalker →

HW ToC

Appendix D: Example homework XX. 3/3

Spring 2021/2022

Bálint Nátrán: Car Rental Registry

Adaptation code example for simple steps →

```
@Override
public void v_ANCInputFieldsModified() {
    System.out.println("In state: v_ANCInputFieldsModified.");
}

@Override
public void e_ClickCancel() {
    System.out.println("Traverses transition: e_ClickCancel.");
    driver.findElement(By.id("CancelBtn")).click();
}

@Override
public void e_ClickCarsList() {
    System.out.println("Traverses transition: e_ClickCarsList.");
    new Actions(driver).moveToElement(driver.findElement(By.id("CarsListBtn"))).click().build().perform();
}
```

Adaptation code example 2for adding a new car element:

```
@Override
public void e_ANCModifyFieldsValid() {
    System.out.println("Traverses transition: e_ANCModifyFieldsValid.");
    driver.findElement(By.id("BrandTB")).sendKeys("aaaaaa"); // Brand
    driver.findElement(By.id("TypeTB")).sendKeys("aaaaaa"); // Type
    driver.findElement(By.id("LicenceNumberTB")).sendKeys(String.valueOf(System.currentTimeMillis())); // unique licence number
    List<WebElement> numericUpDowns = driver.findElements(By.id("PART_TextBox")); // Special numeric up-down elements
    numericUpDowns.get(0).sendKeys("2000"); // Production Year
    numericUpDowns.get(3).sendKeys("1200"); // Weight
    driver.findElement(By.id("FuelTypeCB")).click();
    List<WebElement> fuelTypes = driver.findElementById("FuelTypeCB").findElements(By.className("ListBoxItem")); // Fuel types
    fuelTypes.get(0).click();
    numericUpDowns.get(6).sendKeys("401555"); // Mileage
    numericUpDowns.get(7).sendKeys("10000"); // Daily fee
    numericUpDowns.get(8).sendKeys("06.06.2024."); // Service permit expiry
    driver.findElement(By.id("StatusCB")).click();
    List<WebElement> statuses = driver.findElementById("StatusCB").findElements(By.className("ListBoxItem")); // Statuses
    statuses.get(1).click();
}
```

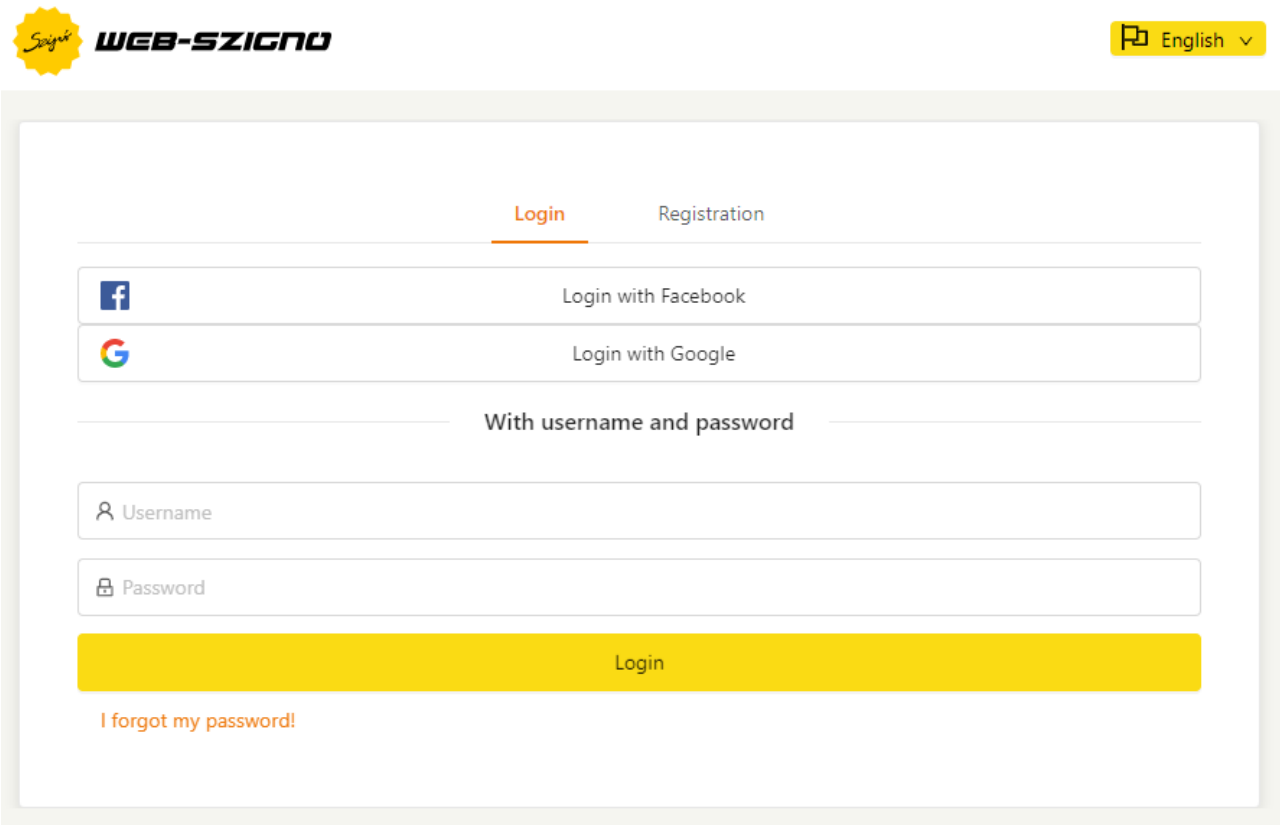
HW ToC

Appendix D: Example homework XXI. 2/1

Spring 2021/2022

Zsolt Zoltán Sajti: Websigno

- SUT (System Under Test): Websigno web application
  - Tested functions:
    - Login
    - File management
- Testing:
  - Model-based testing with GraphWalker
  - Glue code (adaptation code):
    - Using Selenium
  - Stress testing running multiple instances



↑ *UML sequence diagram of container creation*

API: Application Programing Interface
UML: Unified Modeling Language

HW ToC

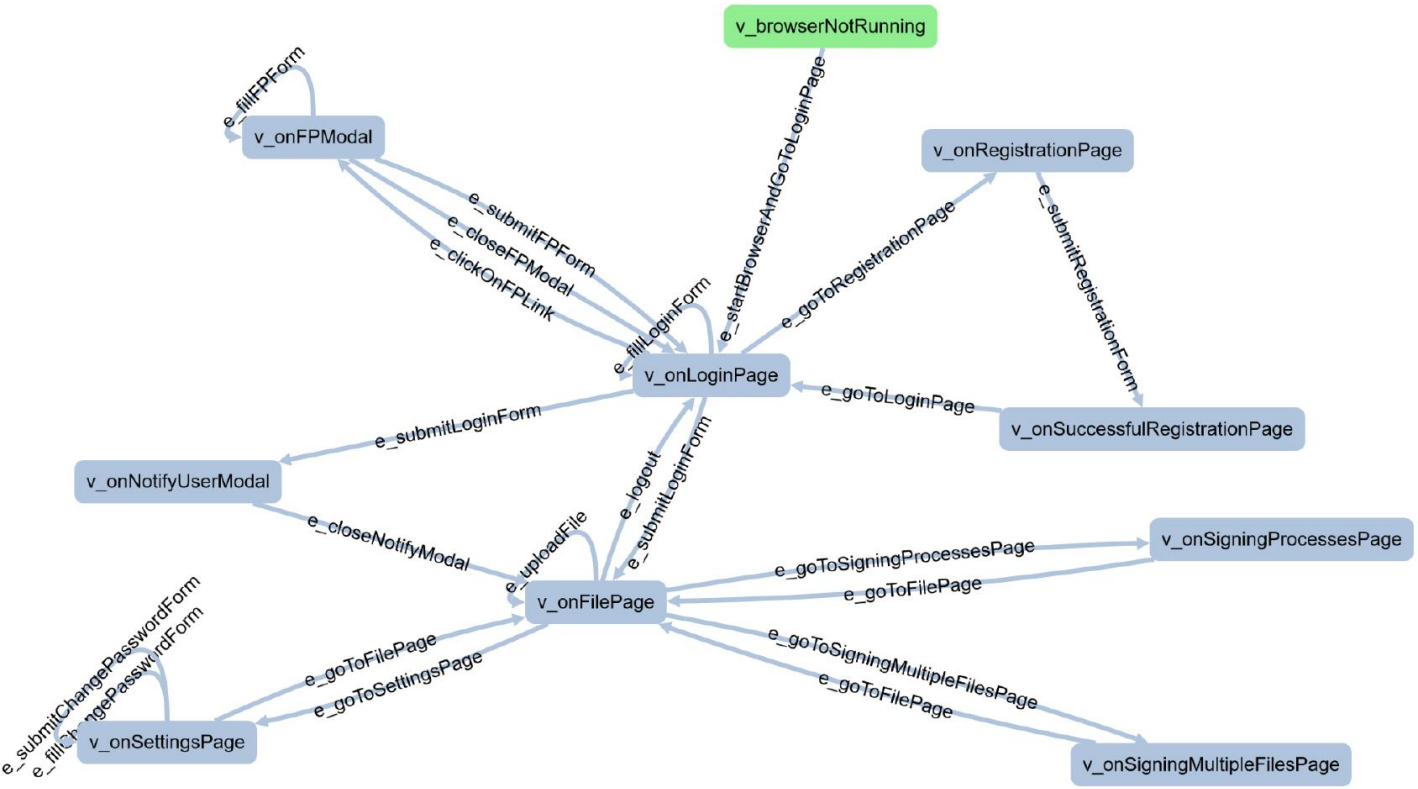
Appendix D: Example homework XXI. 2/2

Spring 2021/2022

Zsolt Zoltán Sajti: Websigno

Adaptation code example →

Model in GraphWalker:




```
package com.company.gs.mbt;
...
@GraphWalker
public class ComplexTest extends ExecutionContext implements Complex
  private WebDriver driver;
  ...
  @Override
  public void v_onLoginPage() {
    WebDriverUtil.waitForUrl(
      driver,
      GSProperties.get(Url.LOGIN_PAGE));
    assertEquals(
      GSProperties.get(Url.LOGIN_PAGE),
      driver.getCurrentUrl());
    assertTrue(
      WebDriverUtil.isAntTabActive(
        driver,
        GSProperties.get(XPath.LOGIN_TAB)));
  }
  ...
  @Override
  public void e_fillLoginForm() {
    WebDriverUtil.typeById(
      driver, GSProperties.get(Id.LOGIN_PAGE_EMAIL),
      GSProperties.get(Data.USER_EMAIL));
    WebDriverUtil.typeById(
      driver,
      GSProperties.get(Id.LOGIN_PAGE_PASSWORD),
      GSProperties.get(Data.USER_PASSWORD));
  }
  ...
```

HW ToC

Appendix D: Example homework XXII.

Spring 2021/2022

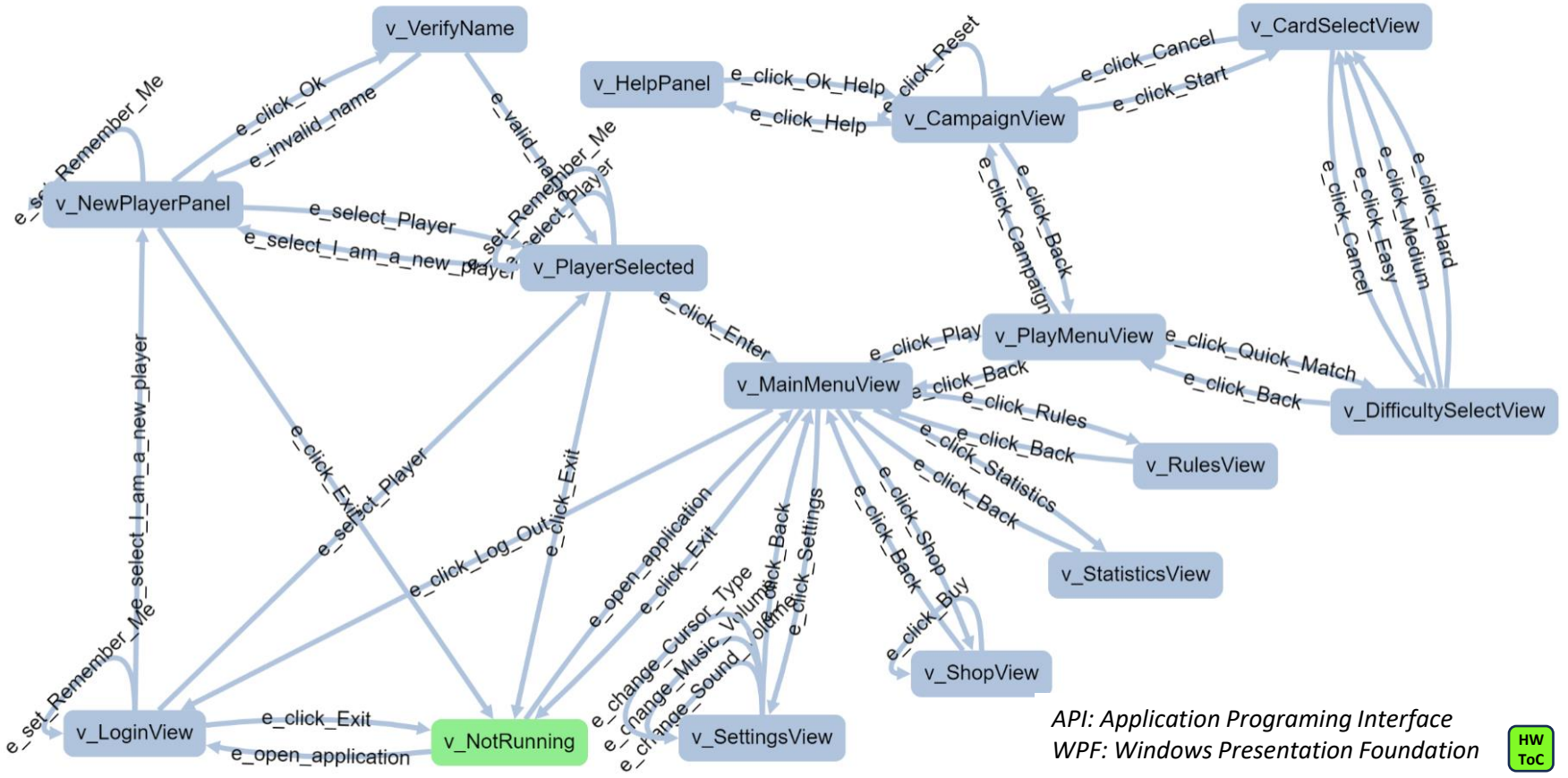
Ferenc András Somogyi:
Pazaak Card Game

Sample adaptation code →

```
233     @Override
        public void e_click_Medium() {
235         System.out.println("e_click_Medium");
236         if (driver.findElement(By.name("Medium")).isEnabled()) {
237             driver.findElement(By.name("Medium")).click();
238         }
239         else {
240             driver.findElement(By.name("Easy")).click();
241             System.out.println("NOT ENOUGH MONEY FOR MEDIUM LEVEL");
242         }
243         Wait();
244     }
```

- SUT (System Under Test):
  - Pazaak Card Game
    (WPF application)
- Testing:
  - Model-based testing with GraphWalker
  - Glue code (adaptation code):
    - Using Winium API calls, UISpy tool for element identifications

Model in GraphWalker →



API: Application Programing Interface
WPF: Windows Presentation Foundation

HW ToC

Appendix D: Example homework XXIII.

Spring 2021/2022

Barnabás Zahorán:
Neovim modes

- SUT (System Under Test): Neovim Vim-based text editor program
  - Testing the *Normal*, *Visual*, *Visual line*, *Visual block*, *Insert*, *Replace*, *Virtual Replace*, *Commandline*, *Ex* modes of Neovim

- Testing:
  - Model-based testing with Model ⟫ Test ⟫ Relax
  - Glue code (adaptation code):
    - Written in Python
    - Using Neovim's API for running RPCs through a TCP socket

M▶T▶R *Note that this example is included in the Model ⟫ Test ⟫ Relax repository under subfolder `sample_models/applications/Neovim`*

```
## Code snippet example to check if nvim starts in Normal mode, then by pressing i it goes to
Insert, then back to Normal with <Esc>:
>>> nvim.call('mode') == 'n'
>>> nvim.feedkeys('i')
>>> nvim.call('mode') == 'i'
>>> nvim.feedkeys(nvim.replace_termcodes("<Esc>"))
>>> nvim.call('mode') == 'n'
```

```
bash-5.1$ ./test_exec.py transitions.tsv testgen_test_suites/Neovim_modes_FSM-ATS_suite
Successfully loaded transition list from transitions.tsv
Successfully loaded test suite from testgen_test_suites/Neovim_modes_FSM-ATS_suite

================= Test suite info =================
File: testgen_test_suites/Neovim_modes_FSM-ATS_suite
Name: Neovim_modes_FSM
Id: 0
Method: ATS
Input sequence length: 652
====================================================
Running test sequence...

=================== Test results ==================
Total assertion count   652
Successful assertions   652
Failed assertion        0
Success rate            100.00 %
Execution time          10.2623 s
====================================================
```

```
start mode      i
key sequence    <C-c>
outcome mode    n
expected mode   n
result:         OK
------------------
start mode      n
key sequence    gQ
outcome mode    cv
expected mode   cv
result:         OK
------------------
start mode      cv
key sequence    vi<CR>
outcome mode    n
expected mode   n
result:         OK
------------------

=================== Test results ==================
Total assertion count   274
Successful assertions   274
Failed assertion        0
Success rate            100.00 %
Execution time          3.5577 s
====================================================
```

```
start mode      Rv
key sequence    <Ins>
outcome mode    i
expected mode   i
result:         OK
------------------
start mode      i
key sequence    <Ins>
outcome mode    Rv
expected mode   R
result:         NOT OK!
```

←↑ *Test execution with Model ⟫ Test ⟫ Relax using custom adaptation code*

API: Application Programing Interface
RPC: Remote Procedure Call
TCP: Transmission Control Protocol

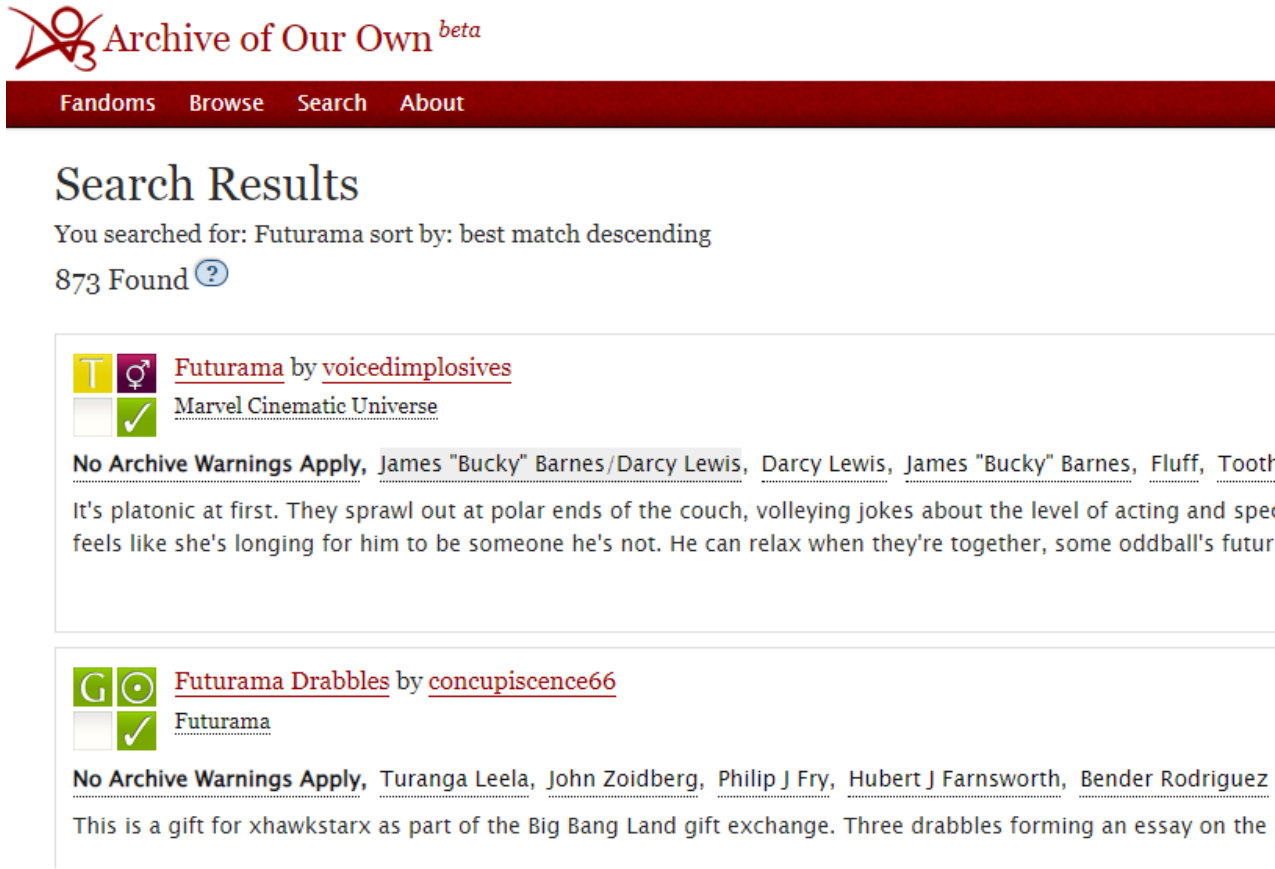
HW ToC

Appendix D: Example homework XXIV. 3/1

Fall 2022/2023

Laura Luca Nagy: Archive of our own

- SUT (System Under Test): Archive of Our Own website
  - An archive for transformative fan works (like fanfiction)
  - Tested functionalities:
    - Log in/Log out
    - Posting a new work
    - Searching works
    - Leave kudos and comment on the work page
- Testing:
  - Model-based testing with GraphWalker
    (test generation also done with Model 》 Test 》 Relax)
  - Glue code (adaptation code):
    - Written in C#
    - Using Selenide (framework powered by Selenium WebDriver) and JUnit



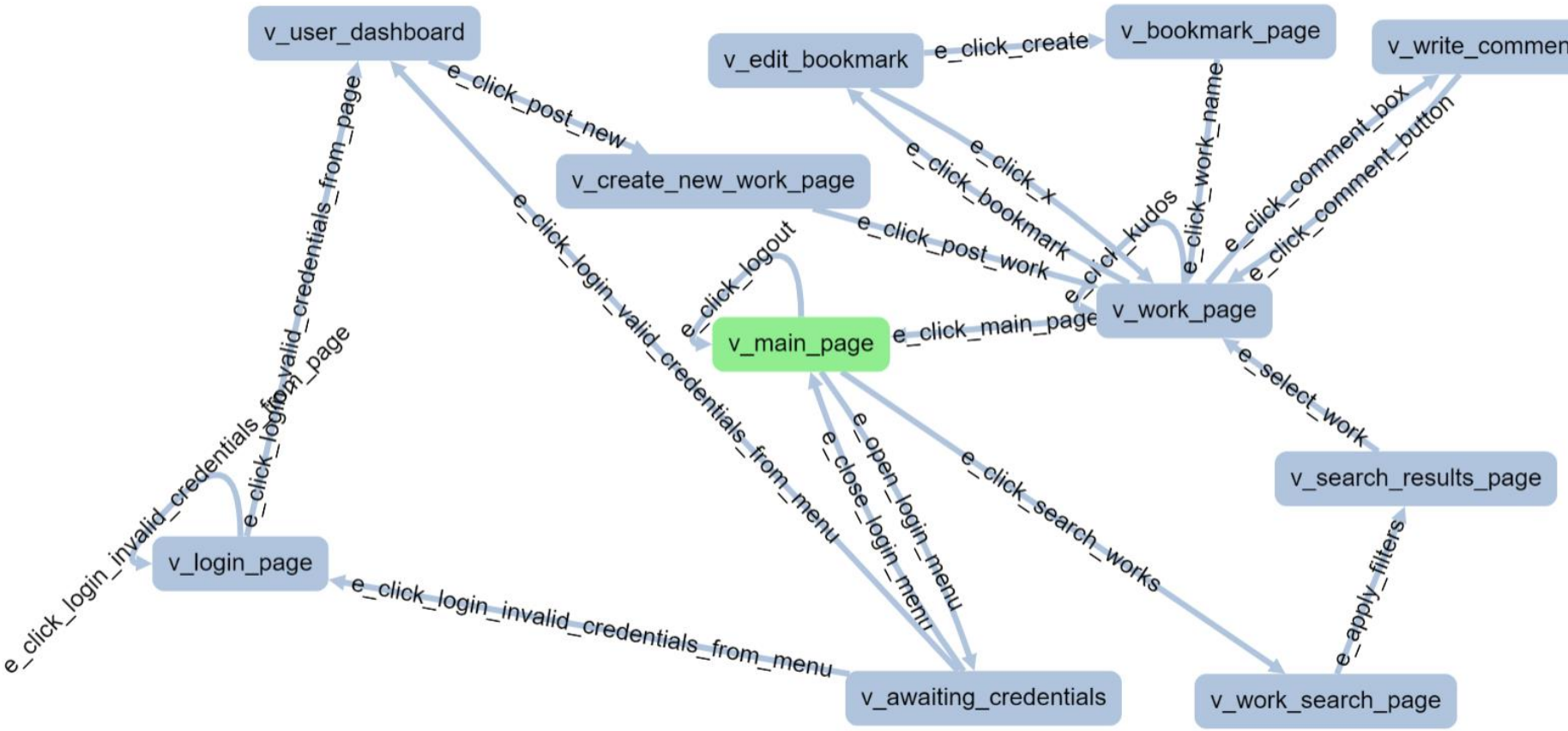
HW ToC

Appendix D: Example homework XXIV. 3/2

Fall 2022/2023

Laura Luca Nagy: Archive of our own

EFSM model in GraphWalker:



Test execution in GraphWalker:

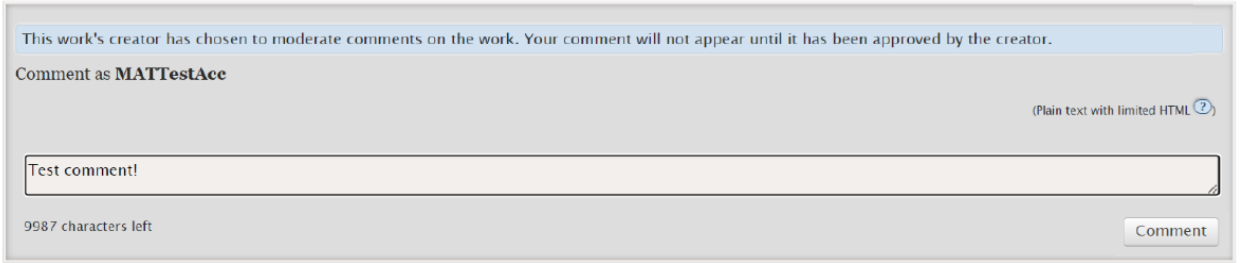
```
[INFO] ------------------------------------------------------------------------
[INFO]
[INFO] Result :
[INFO]
[INFO] {
  "totalFailedNumberOfModels": 0,
  "totalNotExecutedNumberOfModels": 0,
  "totalNumberOfUnvisitedVertices": 0,
  "verticesNotVisited": [],
  "totalNumberOfModels": 1,
  "totalCompletedNumberOfModels": 1,
  "totalNumberOfVisitedEdges": 20,
  "totalIncompleteNumberOfModels": 0,
  "edgesNotVisited": [],
  "vertexCoverage": 100,
  "totalNumberOfEdges": 20,
  "totalNumberOfVisitedVertices": 11,
  "edgeCoverage": 100,
  "totalNumberOfVertices": 11,
  "totalNumberOfUnvisitedEdges": 0
}
[INFO]
22:14:06.116 [AsyncHttpClient-1-1] DEBUG org.as
118 : 1
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:31 min
[INFO] Finished at: 2022-11-18T22:14:06+01:00
[INFO] ------------------------------------------------------------------------
```

HW ToC

Appendix D: Example homework XXIV. 3/3

Fall 2022/2023

Laura Luca Nagy: Archive of our own



← Comment functionality

```
@Override
 public void e_click_comment_box() {
   System.out.println("Traverses transition: e_click_comment_box");
   sleepOnRetryLater();
   $(By.name("comment[comment_content]")).scrollIntoView(false);
   $(By.name("comment[comment_content]")).click();
   sleepOnRetryLater();
   Assert.assertTrue(WebDriverRunner.getWebDriver().getCurrentUrl().matches("^https://archiveofourown.org/works/[0-9]*.*"));
 }

 @Override
 public void v_write_comment() {
   System.out.println("In state: v_write_comment");
   sleepOnRetryLater();
   $(By.name("comment[comment_content]")).setValue("Test comment!");
   Assert.assertTrue(WebDriverRunner.getWebDriver().getCurrentUrl().matches("^https://archiveofourown.org/works/[0-9]*.*"));
 }

 @Override
 public void e_click_comment_button() {
   System.out.println("Traverses transition: e_click_comment_button");
   sleepOnRetryLater();
   $(By.xpath("//input[@name='commit'][@value='Comment']")).scrollIntoView(false);
   $(By.xpath("//input[@name='commit'][@value='Comment']")).click();
   sleepOnRetryLater();
   $(By.xpath("//a[@href='/works/43059846']")).click();
   sleepOnRetryLater();
   Assert.assertTrue(WebDriverRunner.getWebDriver().getCurrentUrl().matches("^https://archiveofourown.org/works/[0-9]*.*"));
 }
```

← Sample adaptation code for check comment functionality

HW ToC

Appendix D: Example homework XXV. 2/1

Fall 2022/2023

Marcell Ferenc Hajdu: Telegram

- SUT (System Under Test): Telegram Client API
  - Safe chatting application
  - Tested functionalities:
    - Log in / log out
    - Create / delete chats
    - Send / delete / edit messages
    - Add / remove users, grant admin rights
- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in C#
    - Using Telegram API (TelegraApi + TDLib + WTelegramClient)

Source of the figure: https://screenlane.com/post/ios/telegram/

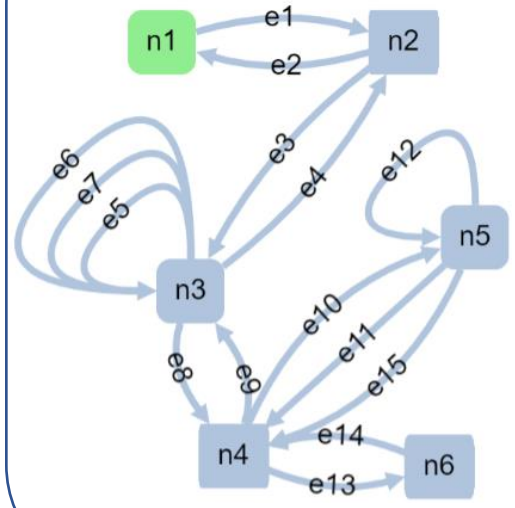
HW ToC

Appendix D: Example homework XXV. 2/2

Fall 2022/2023

Marcell Ferenc Hajdu: Telegram

EFSM model in GraphWalker:



n1: logged out
n2: logged in
n3: admin rights
n4: basic user
n5: messages system
n6: pinned messages system

e1: login
e2: logout
e3: create a chat, this adds 2 users' one as admin
e4: delete a chat
e5: promote a user to an admin user
e6: delete a user
e7: revoke a user's admin right
e8: add a new user
e9: delete a user(non admin)
e10: sending a new message
e11: deleting an existing message
e12: edit an existing message
e13: pin a message
e14: unpin a message
e15: receiving a message

Sample adaptation code for edit message functionality:

```
async Task EditMessageTest()
{
    var messageBase = await client.Messages_GetHistory(new InputPeerChat(chatId));
    int id = 0;

    foreach (var m in messageBase.Messages)
    {
        if (m is Message)
        {
            id = m.ID;
            break;
        }
    }


    await client.Messages_EditMessage(new InputPeerChat(chatId), id, false, "TestTest");

    messageBase = await client.Messages_GetHistory(new InputPeerChat(chatId));
    var message = messageBase.Messages.First(m => m.ID == id) as Message;

    CustomAssert.Assert( message is {message: "TestTest"}, nameof(EditMessageTest));
}
```

```
Loaded previous session
WTelegramClient 3.1.2 running under .NET 6.0.11
passed - LoginTest
passed - CreateChatTest
passed - DeleteUserTest
passed - AddUserTest
passed - DeleteChat
passed - CloseTest
passed - LoginTest
passed - CreateChatTest
passed - SendMessageTest
passed - EditMessageTest
passed - DeleteMessageTest
passed - LoginTest
passed - CreateChatTest
passed - SendMessageTest
passed - GetMessagesTest
passed - PinMessageTest
passed - RemovePinnedMessageTest
```

← ↓ *Test execution*

```
passed - LoginTest
passed - CreateChatTest
passed - PromoteToAdminTest
passed - RemoveAdminTest
passed - DeleteUserTest
Unhandled exception. TL.RpcException: 420 FLOOD_WAIT_X
   at WTelegram.Client.Invoke[T](IMethod`1 query)
   at Program.<>c__DisplayClass0_0.<<Main>$>g__AddUserTest|7>d.MoveNext() in I:\Prog\mat\extended\TelegramAPITest\TelegramAPITest\Program.cs:line 146
--- End of stack trace from previous location ---
   at Program.<Main>$(String[] args) in I:\Prog\mat\extended\TelegramAPITest\TelegramAPITest\Program.cs:line 83
   at Program.<Main>(String[] args)
```

HW ToC

Appendix D: Example homework XXVI. 2/1

Fall 2022/2023

Levente Jakab: Twitter API

- SUT (System Under Test): Twitter API v2 *(before Elon Musk…)*
  - Tested functionalities:
    - Create / delete / like / dislike / retweet / unretweet a tweet
    - Looking up a tweet or another user
    - Block / unblock / mute / unmute / follow / unfollow a user

- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax (MTR)
  - Glue code (adaptation code):
    - Written in Java:
      - Parses the test sequences generated by MTR
      - Maps the appropriate inputs to functions → request to the Twitter API
      - Check response
      - Print the result of the tests to the standard output

API: Application Programing Interface

HW ToC

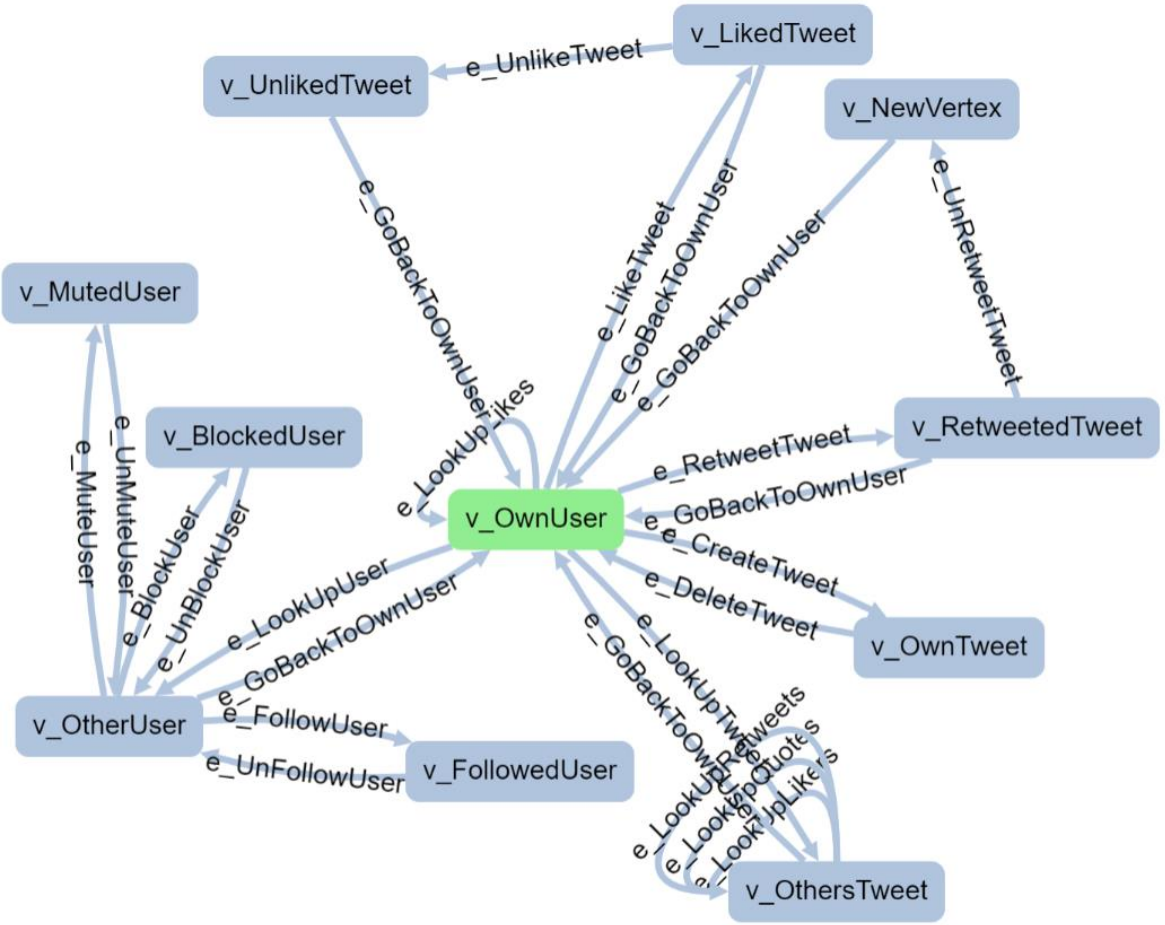
Appendix D: Example homework XXVI. 2/2

Fall 2022/2023

Levente Jakab: Twitter API

Adaptation code for the selected 3 MTR element →

FSM model in GraphWalker ↓



Input list from test sequence generated by MTR ↓

```
"input_list": [
    "like a Tweet",
    "unlike a Tweet",
    "go back to information about own user",
    "retweet a Tweet",
    "unretweet a Tweet",
    "go back to information about own user",
    "look up own liked Tweets",
    "look up information about a user",
    "block a user",
    "unblock a user",
    "mute a user",
    "unmute a user",
    "follow a user",
    "unfollow a user",
    "go back to information about own user",
    "look up Tweet information",
    "look up ReTweets of Tweet",
    "look up quote-Tweets of Tweet",
    "look up list of likers of a Tweet",
    "go back to information about own user",
    "retweet a Tweet",
    "go back to information about own user",
    "like a Tweet",
    "go back to information about own user",
    "create a new Tweet",
    "delete a Tweet"
],
```

```
unfollow a user
    Response: class UsersFollowingDeleteResponse {
    data: class ListFollowedResponseData {
        following: false
    }
    errors: null
}
go back to information about own user
    Response: class Get2UsersMeResponse {
    data: class User {
        createdAt: 2022-10-28T16:26:18Z
        description: null
        entities: null
        id: 1586031586306408450
        location: null
        name: DevJani
        pinnedTweetId: null
        profileImageUrl: null
        _protected: null
        publicMetrics: null
        url: null
        username: dev_nem_jani
        verified: false
        withheld: null
    }
    errors: null
    includes: null
}
look up Tweet information
    Response: class Get2TweetsIdResponse {
    data: class Tweet {
        attachments: null
        authorId: 2244994945
        contextAnnotations: null
        conversationId: null
        createdAt: 2022-10-24T19:00Z
        editControls: null
        editHistoryTweetIds: [1584620743018561537]
        entities: null
        geo: null
        id: 1584620743018561537
        inReplyToUserId: null
        lang: null
        nonPublicMetrics: null
        organicMetrics: null
        possiblySensitive: null
        promotedMetrics: null
        publicMetrics: null
        referencedTweets: null
        replySettings: null
        source: null
        text: if motivation != empty:
            keep_coding
        else:
            order_coffee
        withheld: null
    }
    errors: null
    includes: null
}
look up ReTweets of Tweet
    Response: class Get2TweetsIdRetweetedByResponse {
    data: [class User {
        createdAt: 2022-07-24T14:40:54Z
        description: null
        entities: null
```

HW ToC

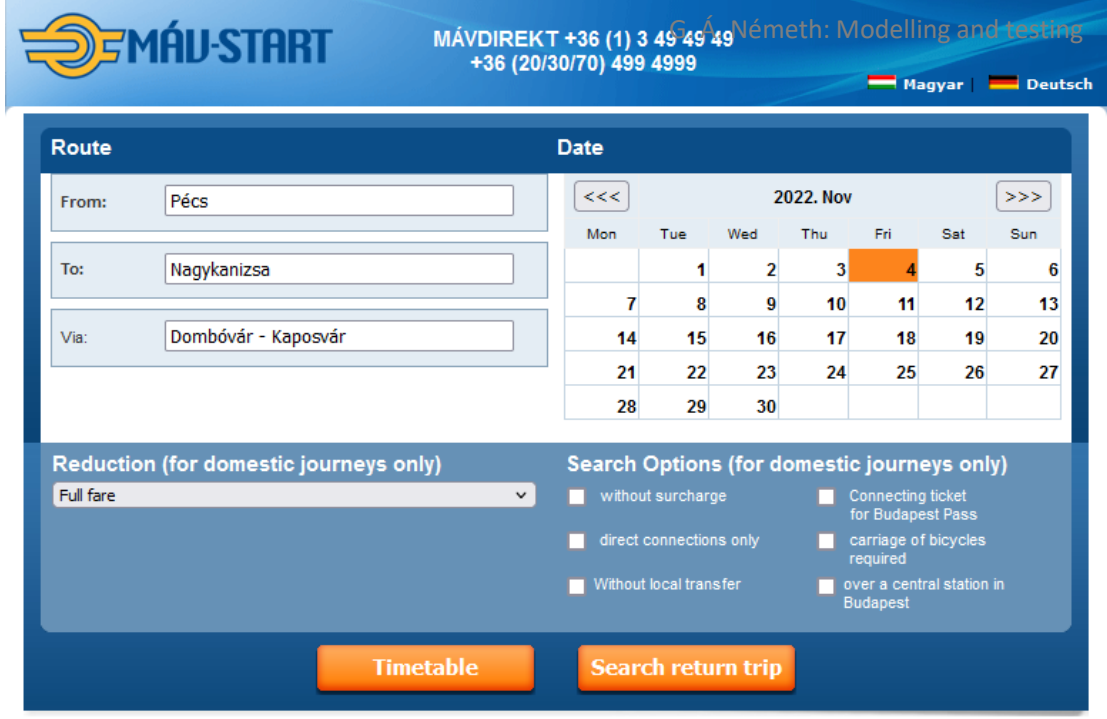
Appendix D: Example homework XXVII. 4/1

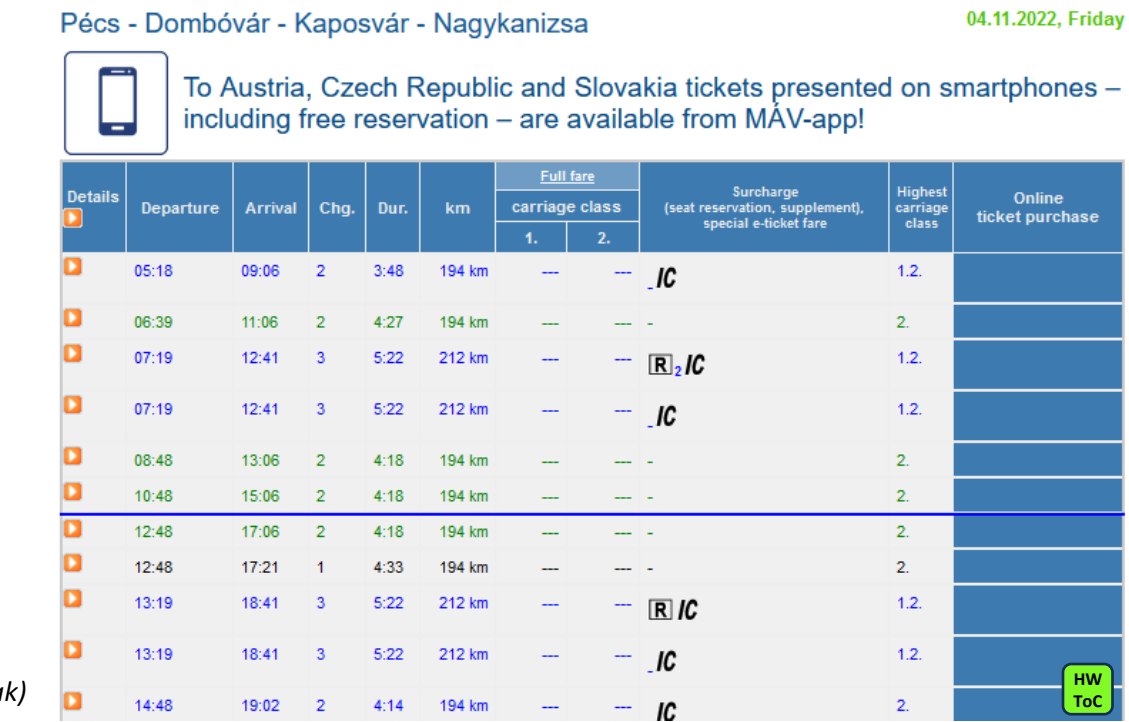
Fall 2022/2023

Bálint Rumpler: ELVIRA

- SUT (System Under Test): ELVIRA
  - Former timetable search and ticket purchasing website of MÁV
  - Tested functionalities:
    - Route planning:
      - Only departure and destination stations are added
      - Optional intermediate stations are added
      - Direct connections only
    - Timetable of stations and trains
    - Change language
- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax (MTR)
  - Glue code (adaptation code):
    - Written in Python, using Selenium for GUI testing
      - Parses the test sequences generated by MTR
      - Maps the appropriate inputs to functions
        → instruct Selenium what to do on the website
      - Check response
      - Print the result of the tests to the standard output

MÁV: Hungarian State Railways (Magyar Államvasutak)





HW ToC

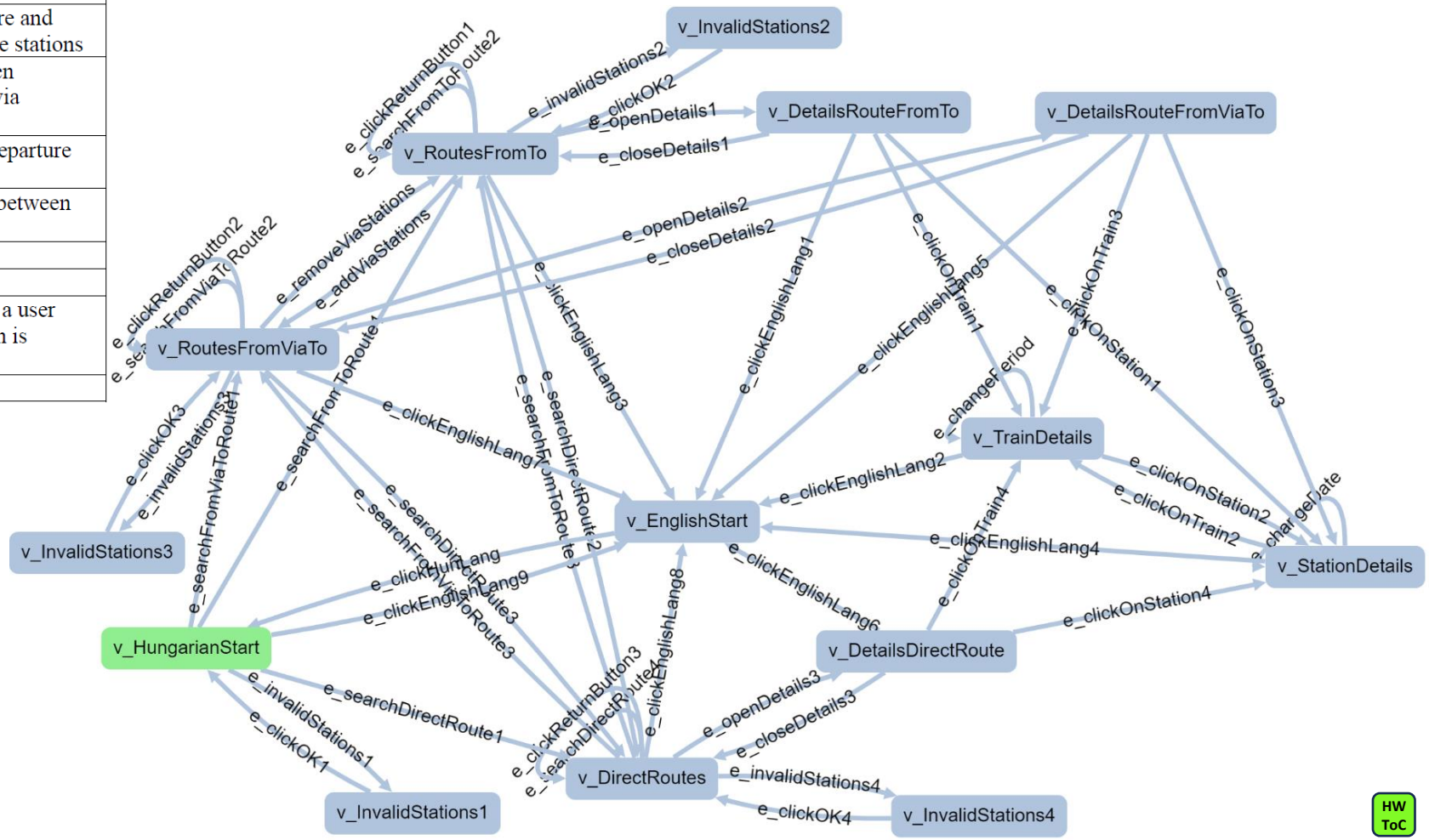
Appendix D: Example homework XXVII. 4/2

Fall 2022/2023

Bálint Rumpler: ELVIRA

Name of state (without _v prefix)	Description
HungarianStart (start state)	Home page of the website
RoutesFromTo	Page with routes between departure and destination station
DetailsRouteFromTo	Details of a searched route between departure and destination station
RoutesFromViaTo	Page with routes between departure and destination station via intermediate stations
DetailsRouteFromViaTo	Details of a searched route between departure and destination station via intermediate stations
DirectRoutes	Page with direct routes between departure and destination station
DetailsDirectRoutes	Details of a searched direct route between departure and destination station
TrainDetails	Timetable of a train
StationDetails	Timetable of a railway station
InvalidStations1, InvalidStations2, InvalidStations3, InvalidStations4	Error message that appears, when a user adds an invalid station, or a station is missing
EnglishStart	English version of the homepage

FSM model in GraphWalker:



HW ToC

Appendix D: Example homework XXVII. 4/3

Fall 2022/2023

Bálint Rumpler:
ELVIRA

Working of adaptation (glue) code:

1. Parses the test sequences generated by MTR

2. Each input in the FSM model corresponds to a function:

```
def fillDepartueViaDestinationAndDirectCheckboxOn():
    searchRoute("Pécs","Mohács","Villány",0,True)
    return "DirectRoutesUI"

def clickEnglishButton():
    changeLang()
    return "EnglishStartUI"

def clickHungarianButton():
    changeLang()
    return "HungarianStartUI"

def fillDepartueDestination():
    searchRoute("Pécs","Mohács","",0,False)
    return "RoutesFromToUI"

def clickOnDetailsButton():
    openCloseRouteDetails()
    return "RouteDetailsUI"
```

3. Each input function will call another function which will instruct Selenium to what to do on the website:

```
#testing search route feature
def searchRoute(departure, destination, via, direction, directOnly):
    #direction: 0 = departure-destination, 1 = destination-departure (return trip button)
    #directOnly: True = only direct routes, False = all routes

    #departure station
    departureSt = browser.find_element(By.ID, 'i')
    departureSt.click()
    departureSt.clear()
    departureSt.send_keys(departure)

    #destination station
    destinationSt = browser.find_element(By.ID, 'e')
    destinationSt.click()
    destinationSt.clear()
    destinationSt.send_keys(destination)

    #via (intermediate) station(s)
    viaSt = browser.find_element(By.ID, 'v')
    viaSt.click()
    viaSt.clear()
    viaSt.send_keys(via)

    #check direct routes only checkbox
    directCheckbox = browser.find_element(By.ID, 'sk')
    isCheckboxChecked = directCheckbox.is_selected()
    if (not directOnly and isCheckboxChecked) or (directOnly and not isCheckboxChecked):
        directCheckbox.click()

    if direction == 0:
        #click on timetable button
        browser.find_element(By.NAME, 'go').click()
    else :
        #click on return trip
        browser.find_element(By.ID, 'goback').click()
```

HW ToC

Appendix D: Example homework XXVII. 4/4

Fall 2022/2023

Bálint Rumpler: ELVIRA

4. The input function returns an output value. The runTest function will compare this output with the specification's output:

```python
def runTest(inputList, outputList):
    global browser
    browser = webdriver.Chrome()
    browser.get('https://elvira.mav-start.hu/elvira.dll/x/index?language=1')

    inputLength = len(inputList)
    success = 0
    fail = 0

    for index, input in enumerate(inputList):
        print(f'STEP {index+1}/{inputLength} | INPUT: {input}', end=" ")

        #All inputs in the FSM corresponds to a function (see below)
        inputFunction = globals()[input]
        output = inputFunction()
        time.sleep(1)

        print(f'| OUTPUT: {output} | ', end=" ")

        #Compare the output in specification FSM and output in implementation
        if output==outputList[index]:
            print('OK')
            success += 1
        else:
            print('FAIL')
            print(f"Test FAILED at STEP {index+1}/{inputLength} | INPUT: {input}")
            print(f"OUTPUT (SPECIFICATION): {outputList[index]} | OUTPUT (IMPLEMENTATION): {output}")
            fail += 1

    print("[INFO] Test ended")
    return success, fail
```

Test execution:

```
E:\EGYETEM\modelling and testing\hf\extended>python main.py testsuites/Elvira-Random_state-50_suite.json
TEST SUITE INFO
Model name:             Elvira
Input sequence length:  11
Method:                 Random

DevTools listening on ws://127.0.0.1:52339/devtools/browser/77fc80ca-3dff-4562-8f37-dcdbcda34205
STEP 1/11 | INPUT: clickTimetableWithInvalidStations | OUTPUT: invalidStationMessage |  OK
STEP 2/11 | INPUT: clickOkButton | OUTPUT: ClosedAlert |  OK
STEP 3/11 | INPUT: clickTimetableWithInvalidStations | OUTPUT: invalidStationMessage |  OK
STEP 4/11 | INPUT: clickOkButton | OUTPUT: ClosedAlert |  OK
STEP 5/11 | INPUT: fillDepartueDestination | OUTPUT: RoutesFromToUI |  OK
[10472:3336:1119/220935.688:ERROR:device_event_log_impl.cc(215)] [22:09:35.685] USB: usb_device_handle_win.cc:1048
0x1F)
[10472:3336:1119/220935.690:ERROR:device_event_log_impl.cc(215)] [22:09:35.686] USB: usb_device_handle_win.cc:1048
0x1F)
STEP 6/11 | INPUT: clickTimetableWithInvalidStations | OUTPUT: invalidStationMessage |  OK
STEP 7/11 | INPUT: clickOkButton | OUTPUT: ClosedAlert |  OK
STEP 8/11 | INPUT: fillDepartueDestination | OUTPUT: RoutesFromToUI |  OK
STEP 9/11 | INPUT: clickOnDetailsButton | OUTPUT: RouteDetailsUI |  OK
STEP 10/11 | INPUT: clickOnTrainNo | OUTPUT: TrainDetailsUI |  OK
STEP 11/11 | INPUT: clickOnStationName | OUTPUT: StationDetailsUI |  OK
[INFO] Test ended

TEST RESULTS
Total assertion count   11
Successful assertions   11
Failed assertion        0
Success rate            100.00 %
Time of execution:       16.723 sec
```

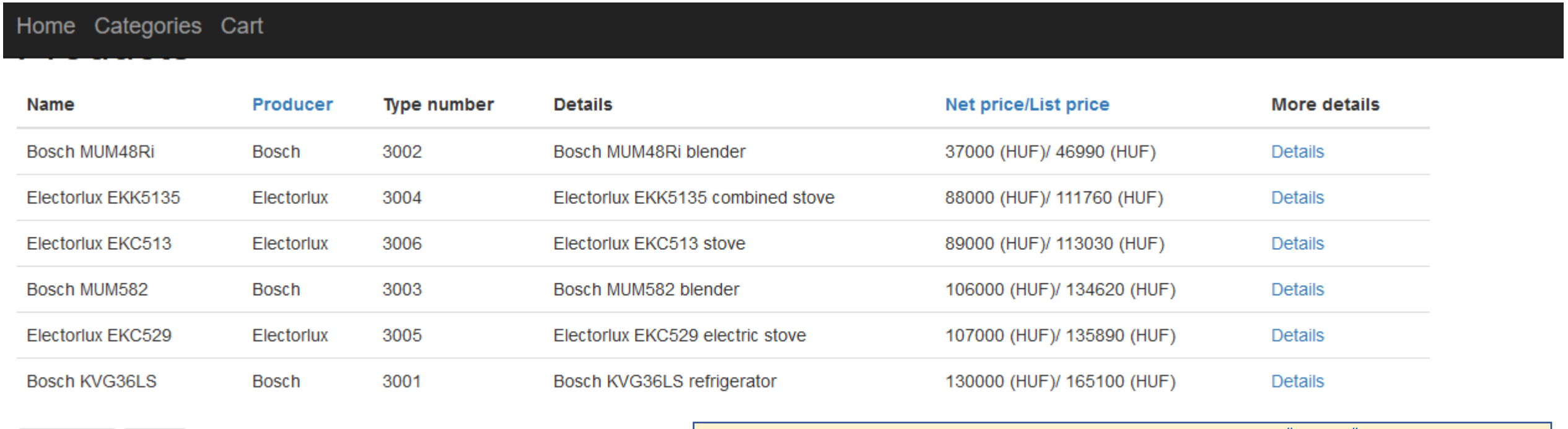
HW ToC

Appendix D: Example homework XXVIII. 4/1

Fall 2022/2023

Tomás Varga: ASP.NET Webshop

- SUT (System Under Test): ASP.NET Webshop
  - Tested functions:
    - Search products by categories
    - Add items to cart, remove items from
    - Confirm order form
- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in C#
    - Using Selenium with chromedriver



Note that this example is included in the Model 》 Test 》 Relax repository under subfolder `sample_models/applications/Webshop`

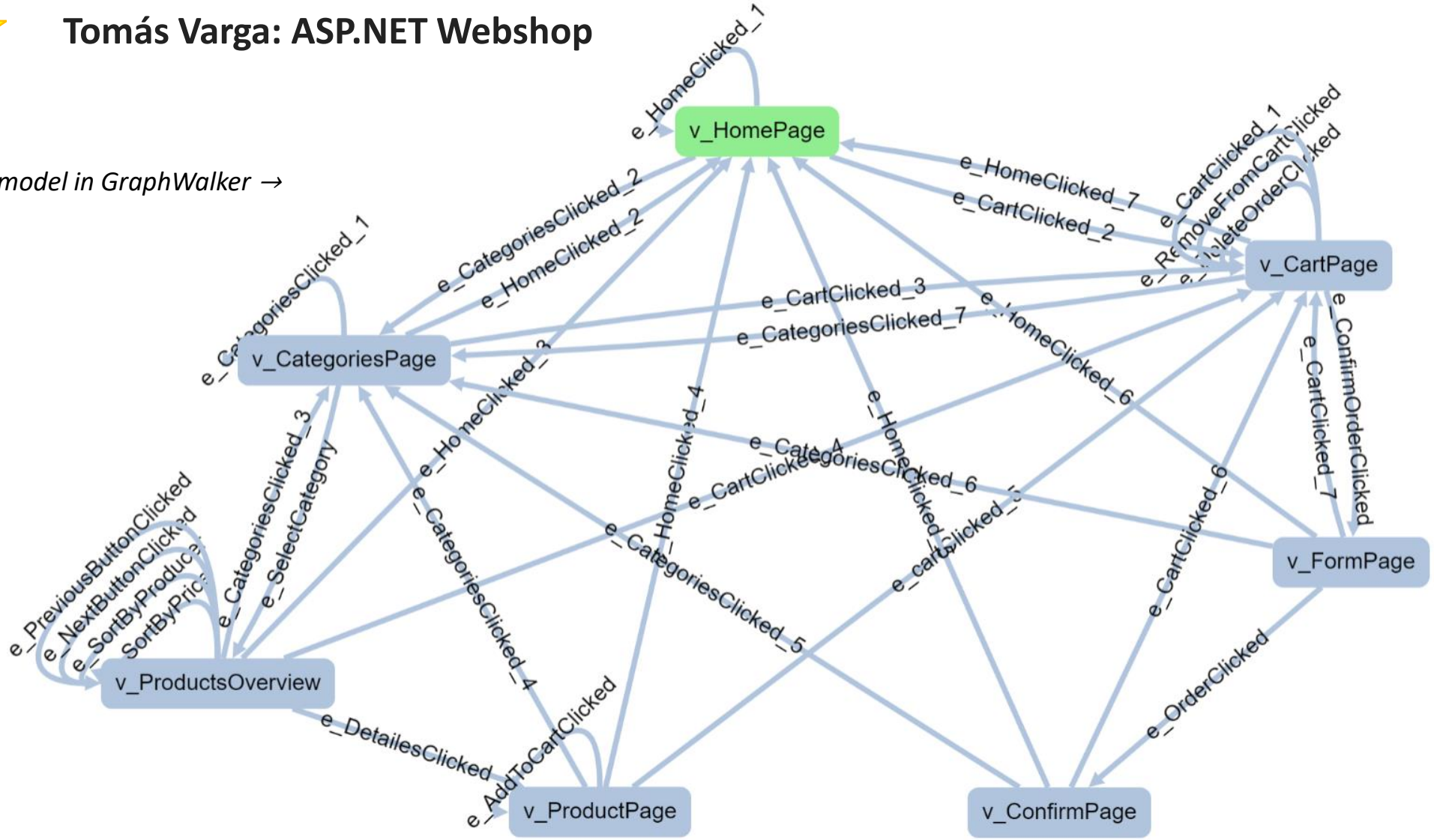
HW ToC

Appendix D: Example homework XXVIII. 4/2

Fall 2022/2023

Tomás Varga: ASP.NET Webshop

FSM model in GraphWalker →



M▸T▸R *Note that this example is included in the Model ⟫ Test ⟫ Relax repository under subfolder* `sample_models/applications/Webshop`

HW ToC

Appendix D: Example homework XXVIII. 4/3

Fall 2022/2023

Tomás Varga: ASP.NET Webshop

Adaptation code examples:

```
//simulate the addition of a product to the cart
        public static void AddToCart(ChromeDriver driver)
        {
            GoToCategoriesPage(driver);

            IWebElement selectCat = driver.FindElement(By.XPath(Dictionaries.xpaths["select a category"]));
            selectCat.Click();
            WebDriverWait waitCat = new WebDriverWait(driver, new TimeSpan(0, 0, 5));
            waitCat.Until(wt => wt.FindElement(By.XPath("/html/body/div/main/h1")));

            IWebElement pressDetails = driver.FindElement(By.XPath(Dictionaries.xpaths["press Details"]));
            pressDetails.Click();
            WebDriverWait waitProd = new WebDriverWait(driver, new TimeSpan(0, 0, 5));
            waitProd.Until(wt => wt.FindElement(By.XPath("/html/body/div/main/h1")));

            IWebElement addToCart = driver.FindElement(By.XPath(Dictionaries.xpaths["press Add to cart"]));
            addToCart.Click();
            WebDriverWait waitAdd = new WebDriverWait(driver, new TimeSpan(0, 0, 5));
            waitAdd.Until(wt => wt.FindElement(By.XPath("/html/body/div/main/h1")));
        }
…
//navigate to given URL
        public static void NavigateToURL(ChromeDriver driver, int i, string from, string input, string output)
        {
            if (input != "press Confirm" && from != "confirm page")
            {
                try
                {
                    driver.Navigate().GoToUrl(Dictionaries.urls[from]);
                }
                catch (Exception e)
                {
                    driver.Close();
                    throw new Exception("ERROR: Could not navigate to \"from\" URL!" +
                        "\n" + "test No.: " + i.ToString() + ", test from: " + from + ", test input: " + input + ", test output: " + output);
                }
            }
        }
```

Note that this example is included in the [Model ⟫ Test ⟫ Relax](#) repository under subfolder `sample_models/applications/Webshop`

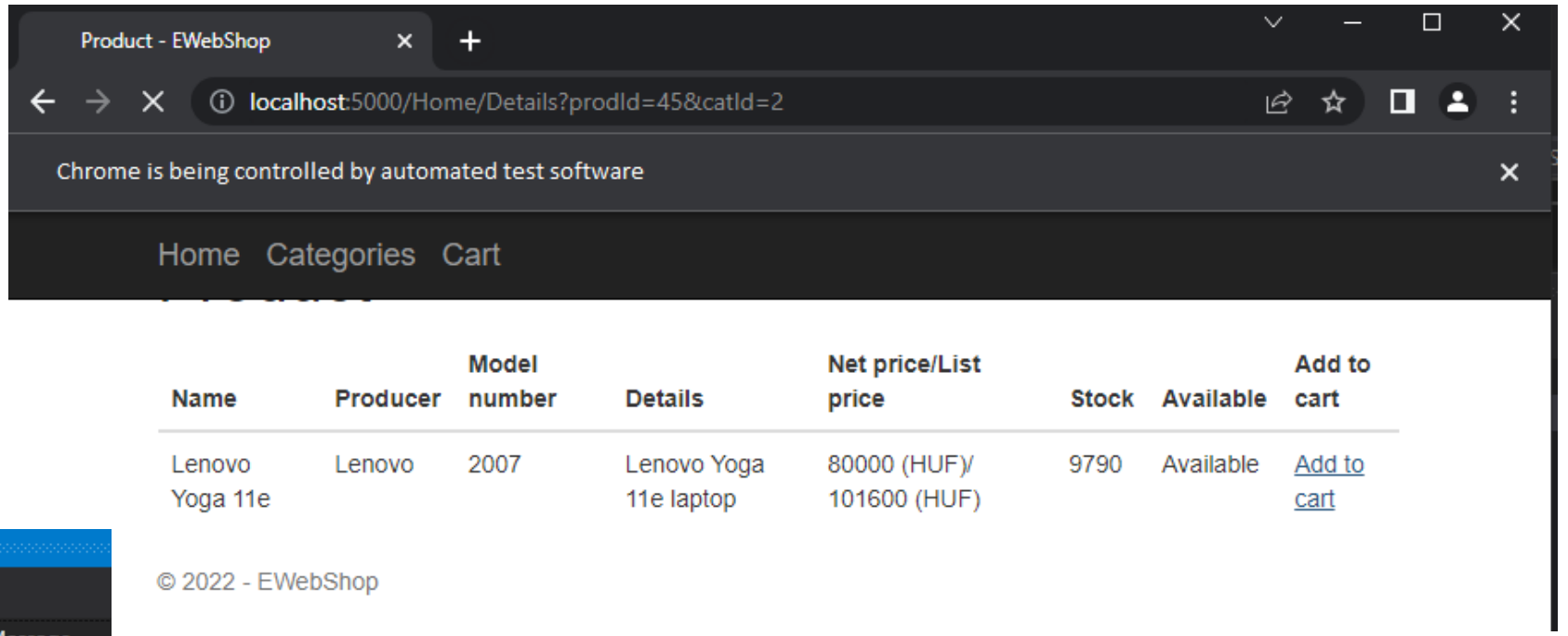
HW ToC

Appendix D: Example homework XXVIII. 4/4

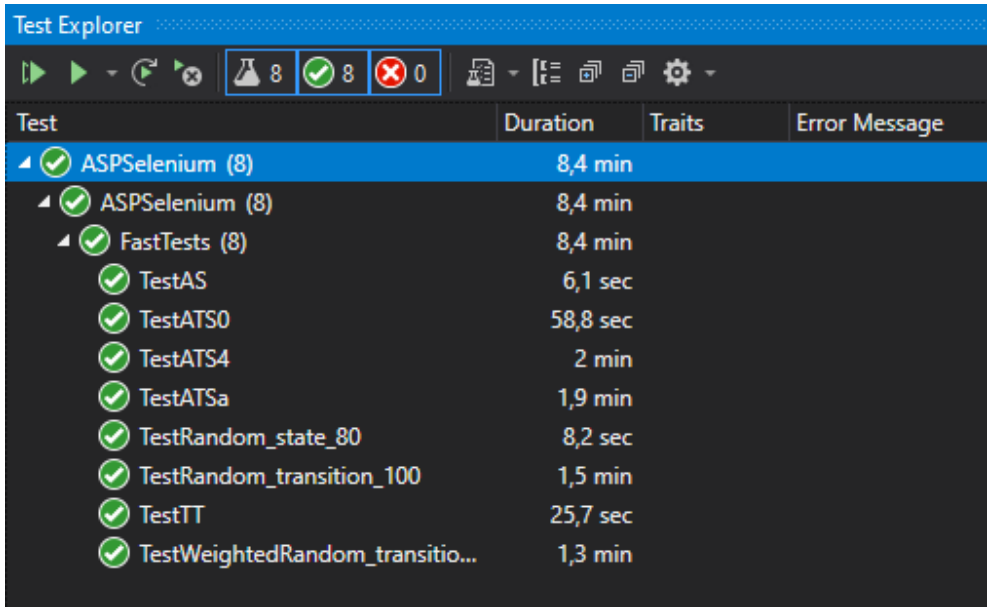
Fall 2022/2023

Tomás Varga: ASP.NET Webshop

Test execution with Selenium →



Test results:



Note that this example is included in the Model ⟫ Test ⟫ Relax repository under subfolder `sample_models/applications/Webshop`

HW ToC

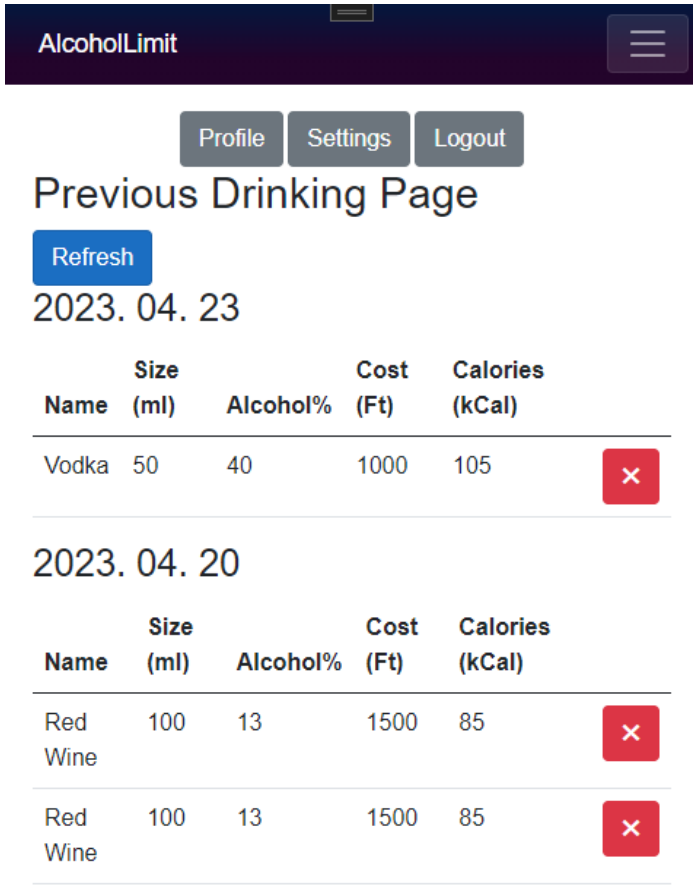
Appendix D: Example homework XXIX. 3/1

Spring 2022/2023

Ádám Bába: AlcoholLimit

- SUT (System Under Test): AlcoholLimit Android application
  - Tested functions:
    - Login
    - Start Drinking and Previous drinking pages
      - Record drink item, volume, alcohol% ...etc. properties and elapsed time
    - Set profile
      - Set threshold and notifications

- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
    - Using Appium (a tool built on Selenium) to automate GUI testing

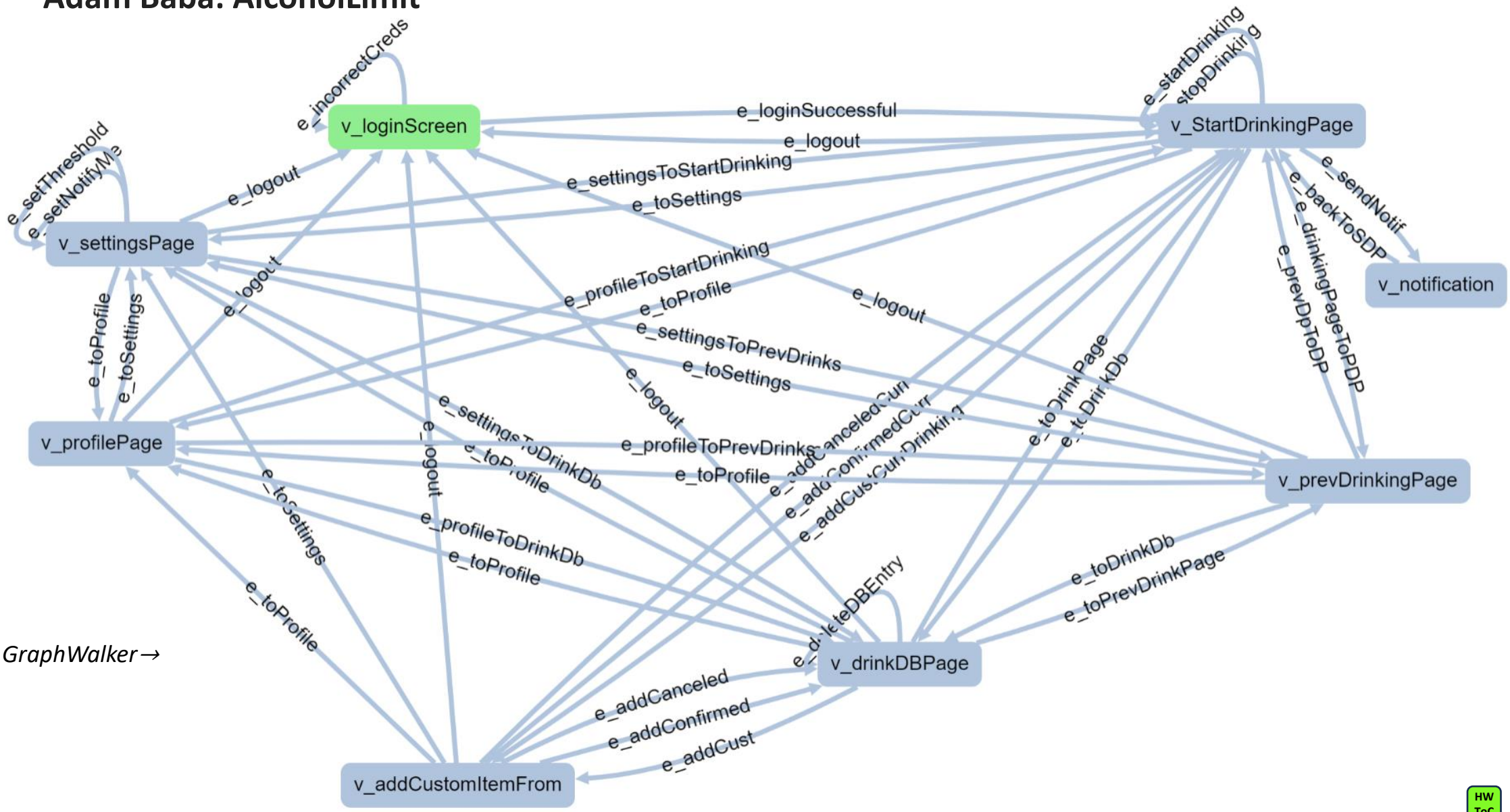


HW ToC

Appendix D: Example homework XXIX. 3/2

Spring 2022/2023

Ádám Bába: AlcoholLimit



EFSM model in GraphWalker→

HW ToC

Appendix D: Example homework XXIX. 3/3

Spring 2022/2023

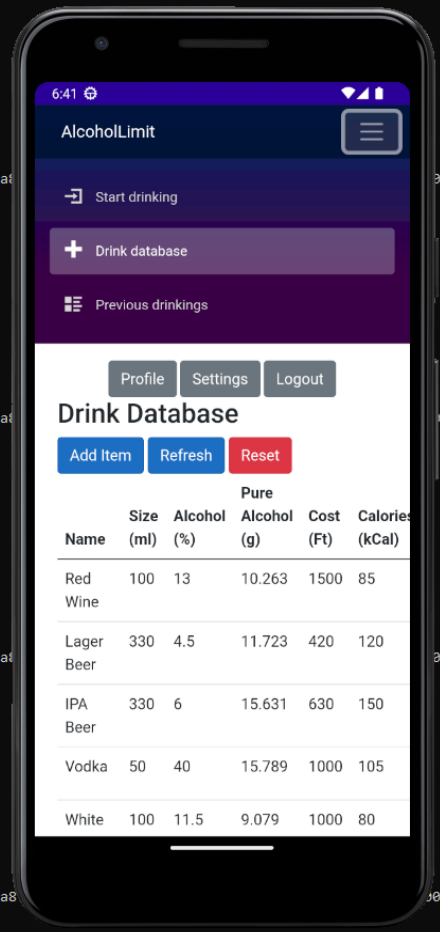
Ádám Bába: AlcoholLimit

Adaptation code examples:

```
 //Access an element by XPATH:
 def click_button(resource_id, driver):
        el = driver.find_elements(AppiumBy.XPATH, "//*[contains(@resource-id, '" + resource_id + "')]")
        print("Clicking button with resource-id: " + resource_id)
        el[0].click()
 //Access an element by coordinates:
 def tap_by_coords(x, y, driver):
        print("Tapping coordinates x: ", x, ", y: ", y)
        TouchAction(driver).tap(None, x, y, 1).perform()
...
 //Running the test itself:
class TestAppium(unittest.TestCase):
 def setUp(self) -> None:
   print('Start time: ', start)

   self.driver = webdriver.Remote(appium_server_url, capabilities)

   do_step('register', inputNavigationDict["register"], self.driver, False)
   self.driver.implicitly_wait(30)

 def tearDown(self) -> None:
   endTime = datetime.now()
   print('End time: ', endTime)

   duration = endTime - start
   print('Duration: ', duration.total_seconds())
   if self.driver:
     self.driver.quit()

 def test_run(self) -> None:
   i = 0
   for input in input_list:
     print('Current step number: ', i + 1,'\n')
     try:
       do_step(input, inputNavigationDict[input], self.driver, True)
     except StaleElementReferenceException as Exception:
       print('Retrying ' + input + ' after StaleElementException')
       do_step(input, inputNavigationDict[input], self.driver, True)

     try:
       check_result(output_list[i], outputAssertionDict[output_list[i]], self.driver, True)
     except StaleElementReferenceException as Exception:
       print('Retrying ' + input + ' after StaleElementException')
       check_result(output_list[i], outputAssertionDict[output_list[i]], self.driver, True)
     i += 1
```

Test execution:

```
Parancssor - python  test.py com.companyname.alcohollimit-Signed.apk "alcohol tracker-Random_transition-100_suite.json"
a8f33", element="00000000-0000-0026-0000-04870000000b")>]
Finished processing: start drinking page
---------------


Current step number:  24

Processing input: logout button clicked
Clicking button with resource-id: logout
Finished processing: logout button clicked
Expected output : login page
check_page el:  [<appium.webdriver.webelement.WebElement (session="b0075085-e85e-424e-8376-b710866a
a8f33", element="00000000-0000-0026-0000-04b40000000b")>]
Finished processing: login page
---------------


Current step number:  25

Processing input: login incorrectly
Filling field with resource-id: log-email with the following value: asd
Filling field with resource-id: log-pass with the following value: asd
Clicking button with resource-id: log-btn
Finished processing: login incorrectly
Expected output : login page
check_page el:  [<appium.webdriver.webelement.WebElement (session="b0075085-e85e-424e-8376-b710866a
a8f33", element="00000000-0000-0026-0000-04b40000000b")>]
Finished processing: login page
---------------


Current step number:  26

Processing input: login correctly
Filling field with resource-id: log-email with the following value: test@test.test
Filling field with resource-id: log-pass with the following value: test123
Clicking button with resource-id: log-btn
Finished processing: login correctly
Expected output : start drinking page
check_page el:  [<appium.webdriver.webelement.WebElement (session="b0075085-e85e-424e-8376-b710866a
a8f33", element="00000000-0000-0026-0000-04e50000000b")>]
Finished processing: start drinking page
---------------


Current step number:  27

Processing input: Drink database clicked
Clicking button with resource-id: navmenu_open
clicking:  drink-db
Tapping coordinates x:  262 , y:  489
Finished processing: Drink database clicked
Expected output : drink database
check_page el:  [<appium.webdriver.webelement.WebElement (session="b0075085-e85e-424e-8376-b710866a8
Finished processing: drink database
---------------


Current step number:  28

Processing input: start drinking page clicked
Clicking button with resource-id: navmenu_open
```



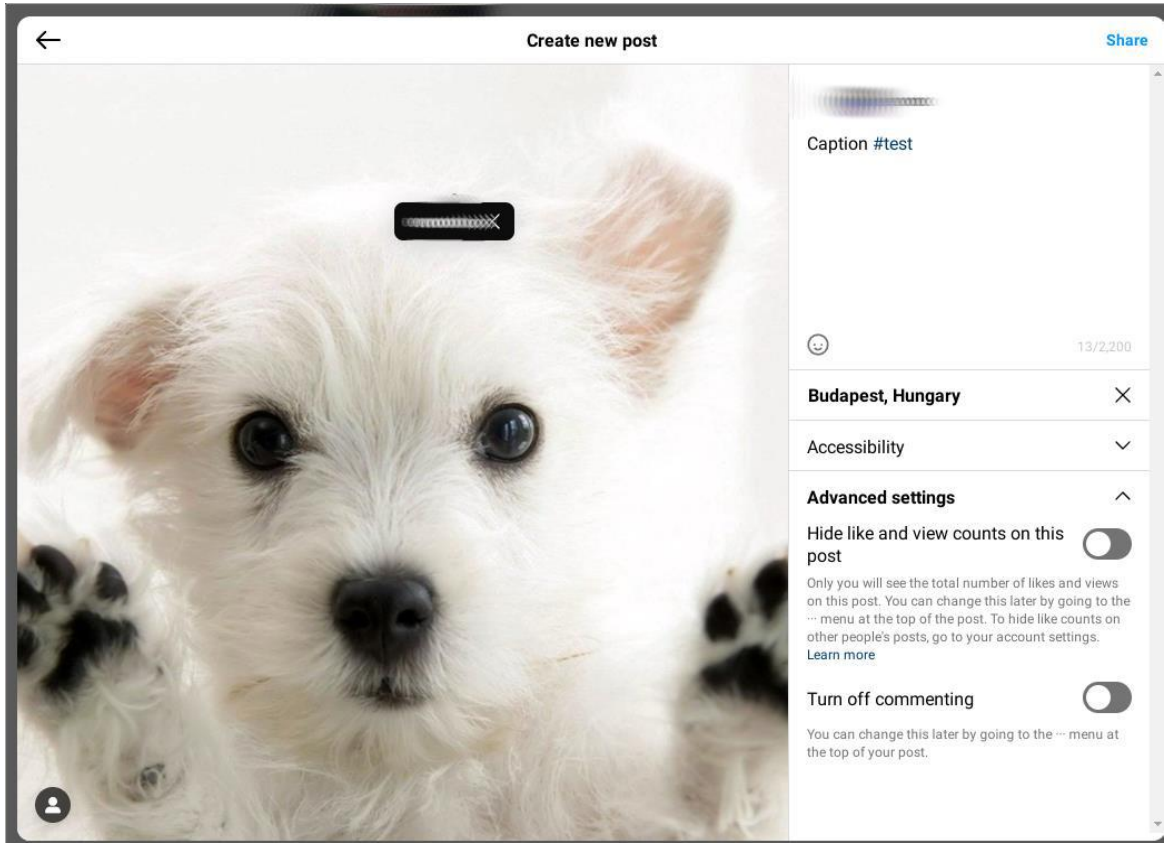
HW ToC

Appendix D: Example homework XXX. 5/1

Spring 2022/2023

Attila Barna: Instagram

- SUT (System Under Test): Instagram API
  - Mobile/web application that allows users to create and share photos and videos and interact with others
  - Tested functions:
    - Posting Photos and Videos
      - Add/remove posts
      - Upload media files
      - Add captions, hashtags, tag users and locations
      - Add filters and effects
      - Select post type (public/private…etc.), turn of comments
    - Direct Messaging
      - Send/receive messages/react to messages
      - Create groupchats
      - Add photos/videos
      - Blocks/unblocks users



- Testing:
  - Model has been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Test runner in Java (using GraphWalker CLI)
    - Using Selenium for GUI testing
    - OkHttp (Java library) for handle HTTP requests/responses
    - Test results are reported and recorded using JUnit

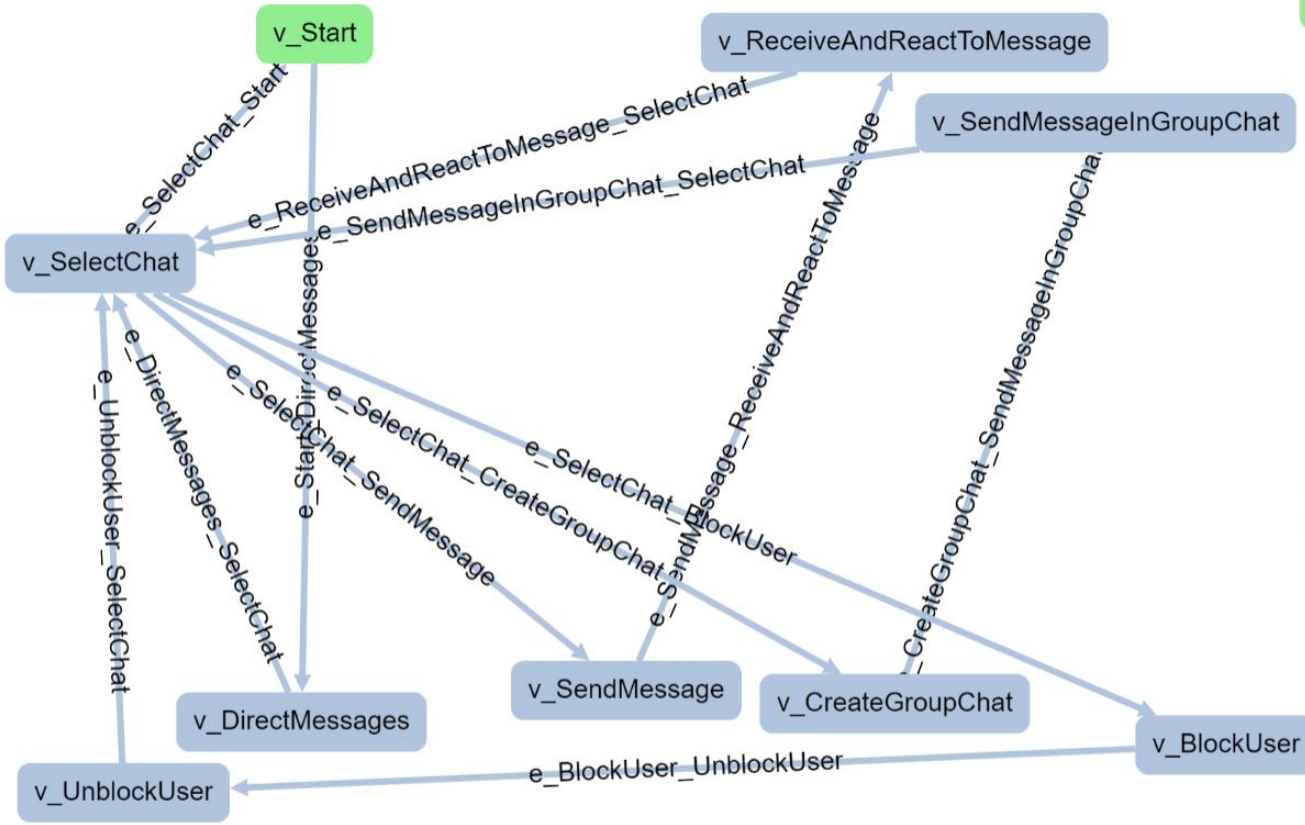
HW ToC

Appendix D: Example homework XXX. 5/2

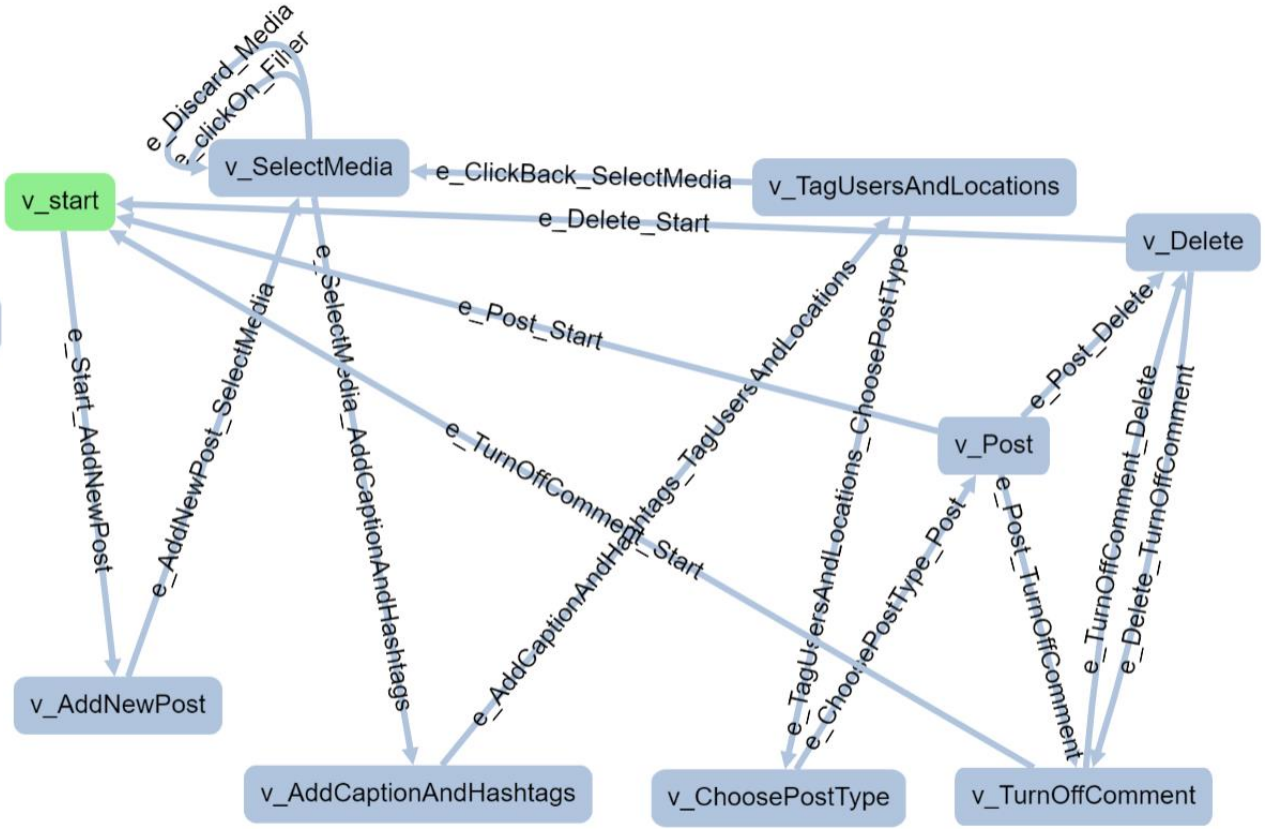
Spring 2022/2023

Attila Barna: Instagram

FSM model for direct messaging:



Model for posting photos and videos:



HW ToC

Appendix D: Example homework XXX. 5/3

Spring 2022/2023

Attila Barna: Instagram

Adaptation code:

- Standard Maven project
- The project includes the GraphWalker Java library for executing the imported test cases generated previously from the 2 models using Model 》 Test 》 Relax
- Selenium interacts with the web application (using ChromeDriver)
- OkHttp (Java library) handling HTTP requests and responses between the test runner and the application
- The dependencies are managed by Maven

Adaptation code examples using Selenium:

```
 /**
     * The sleep function pauses the execution of the thread for a specific duration of time
determined by the user.
     * @param time
     */
    protected void sleep(Long time) {
        try {
            TimeUnit.SECONDS.sleep(time);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    /**
     * The responsibility of the tryRemoveNotification function is to handle the closing of
Instagram's notification window, which may appear unexpectedly.
     */
    protected void tryRemoveNotification() {
        List<WebElement> elements = driver.findElements(By.xpath("//button[@class='_a9--
_a9_1']"));
        if (!elements.isEmpty()) {
            elements.forEach(WebElement::click);
            sleep(1L);
        }
    }

    /**
     * The function existsElement determines the availability of an element and does not generate
an exception.
     *
     * @param findBy
     * @return
     */
    protected Boolean existsElement(By findBy) {
        try {
            return !driver.findElements(findBy).isEmpty();
        } catch (Exception e) {
            return false;
        }
    }
```

```
    /**
     * The `findLast` function determines the last element in case of multiple identical elements.
     * @param findBy
     * @param fn
     * @return
     * @param <R>
     */
    protected <R> R findLast(By findBy, Function<WebElement, R> fn) {
        tryRemoveNotification();
        List<WebElement> elements = driver.findElements(findBy);
        if (!elements.isEmpty()) {
            return fn.apply(elements.get(elements.size() - 1));
        }
        throw new IllegalStateException("IllegalState at findLast.");
    }

    /**
     * The `tryOnElement` function performs an arbitrary operation on a given element,
     * if there are multiple elements, you can specify on which element to perform the operation.
     *
     * @param findBy
     * @param consumer
     * @param pos
     * @param sleep
     */
    protected void tryOnElement(By findBy, Consumer<WebElement> consumer, Integer pos, Long sleep)
{
        tryRemoveNotification();
        List<WebElement> elements = driver.findElements(findBy);
        if (!elements.isEmpty()) {
            consumer.accept(elements.get(pos));
            sleep(sleep);
        }
    }
```

HW ToC

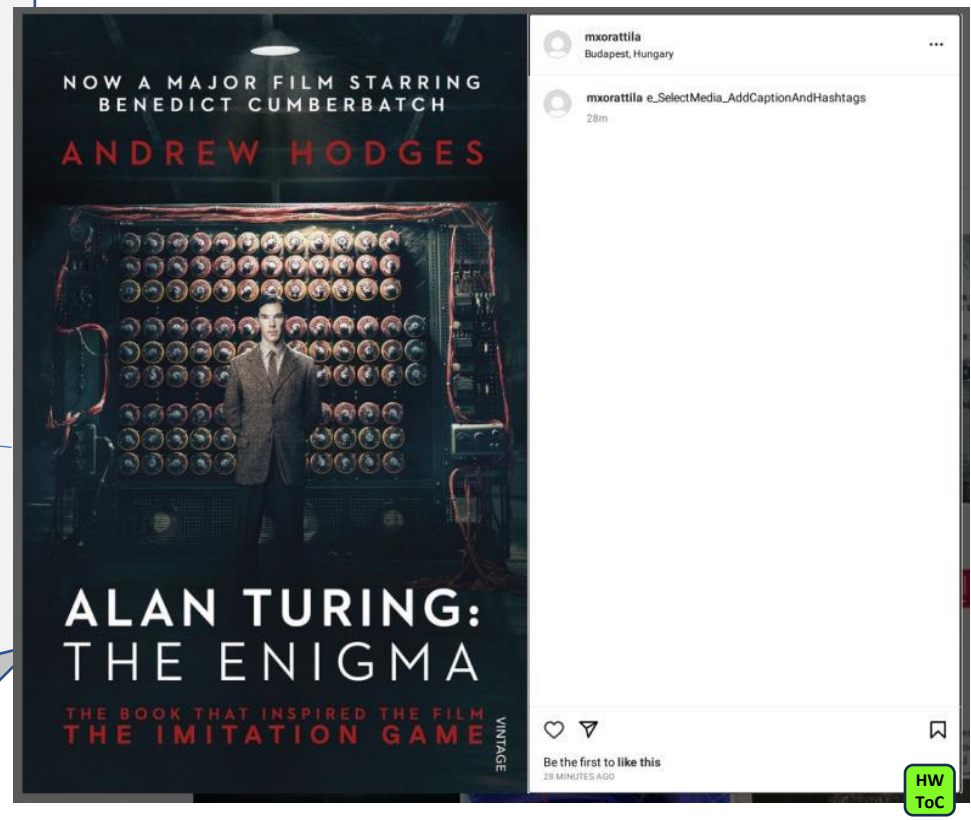
Appendix D: Example homework XXX. 5/4

Spring 2022/2023

Attila Barna: Instagram

Adaptation code examples using okHttp:

```
/**
* Creation of media content.
*/
@Override
public void e_ChoosePostType_Post() {
    try {
        this.uploadFile((bt, t) -> {
            bt.addFormDataPart("caption", "e_ChoosePostType_Post");
            bt.addFormDataPart("geotag_enabled","1");
            bt.addFormDataPart("location","{\"lat\":47.4719,\"lng\":19.0503,\"facebook_places_id\":\"106502519386806\"}");
            bt.addFormDataPart("disable_comments","1");
        });
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    sleep(2L);
}


/**
* `e_SelectMedia_AddCaptionAndHashtags` will upload media content with a tag and custom text.
*/
@Override
public void e_SelectMedia_AddCaptionAndHashtags() {
    try {
        this.uploadFile((bt, t) -> {
            bt.addFormDataPart("caption", "e_SelectMedia_AddCaptionAndHashtags");
            bt.addFormDataPart("geotag_enabled","1");
            bt.addFormDataPart("location","{\"lat\":47.4719,\"lng\":19.0503,\"facebook_places_id\":\"106502519386806\"}");
            bt.addFormDataPart("disable_comments","1");
        });
        sleep(2L);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```



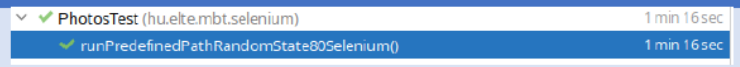
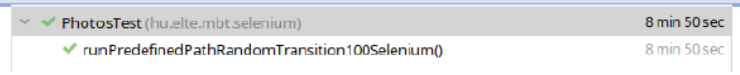
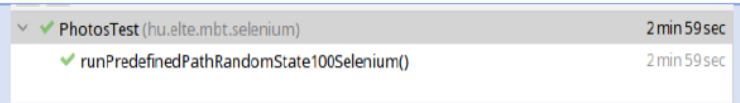
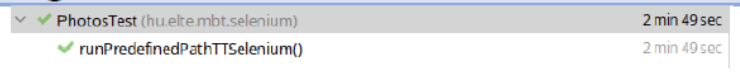
Appendix D: Example homework XXX. 5/5

Spring 2022/2023

Attila Barna: Instagram

The tests were run in the IntelliJ IDEA development environment.
The results are reported and recorded using JUnit.

Test results of GUI testing of posting photos and videos:

Method	Input sequence length	Result	Execution time
Random walk State 80%	11		1 min 16 sec
Random walk State 100%	112		8 min 50 sec
Random walk 100% transition	13		2 min 59 sec
All transition state	95		9 min 10 sec
All transition transition	561	It cannot be tested properly, Instagram disables the possibility of uploading images due to the high number of state transitions.	-
Transition Tour	31		2 min 49 sec
Harmonized state identifiers (Experimental)	63		4 min 48
H-Method	63		4 min 42 sec

HW ToC

Appendix D: Example homework XXXI. 3/1

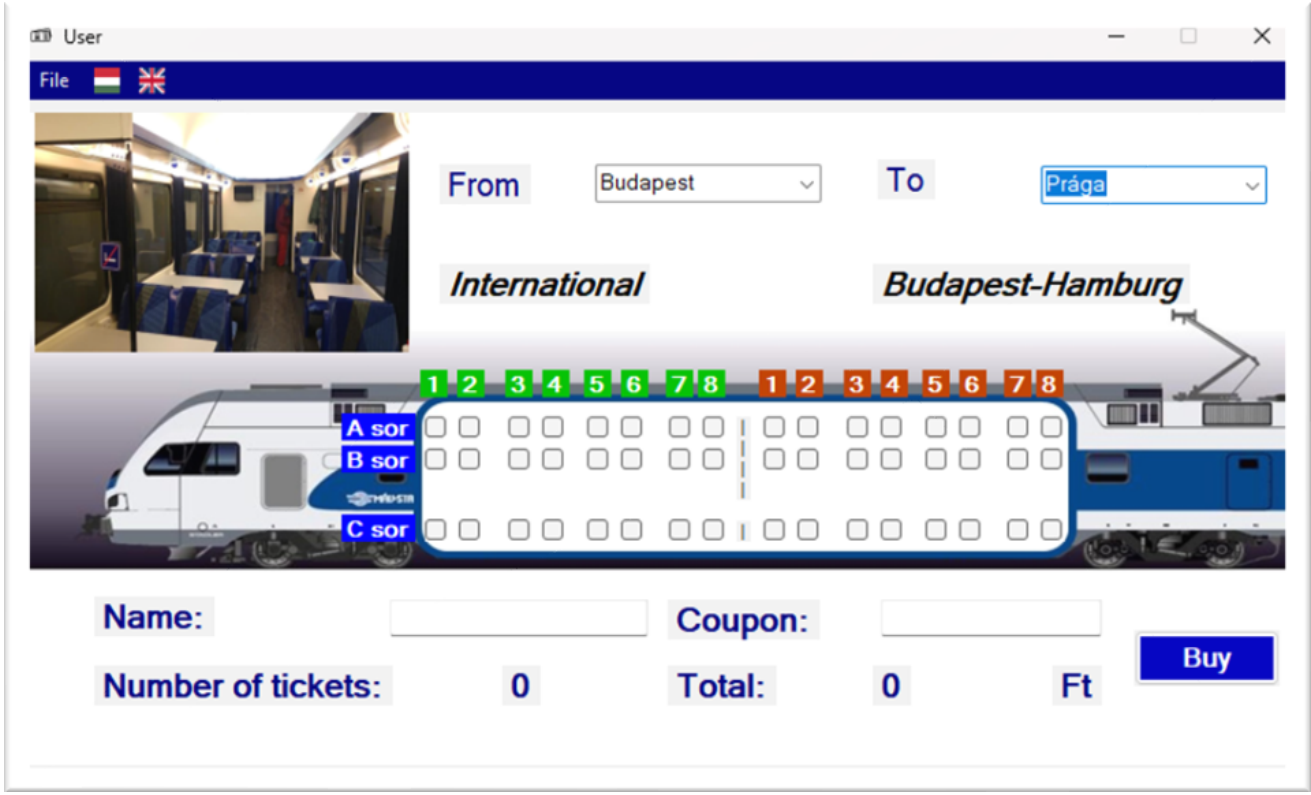
Spring 2022/2023

Barnabás Holicza:

Train ticket reservation application

- SUT (System Under Test): Train Ticket Reservation application
- Tested functions:
  - Logging (user or administrator)
  - User: Buy ticket in by selecting given seats in a GUI according to social distancing rules
  - Administrator: Display seating and statistics

- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in C#
    - Using Windows Application Driver (a service to support Selenium-like UI Test Automation on Windows Applications)



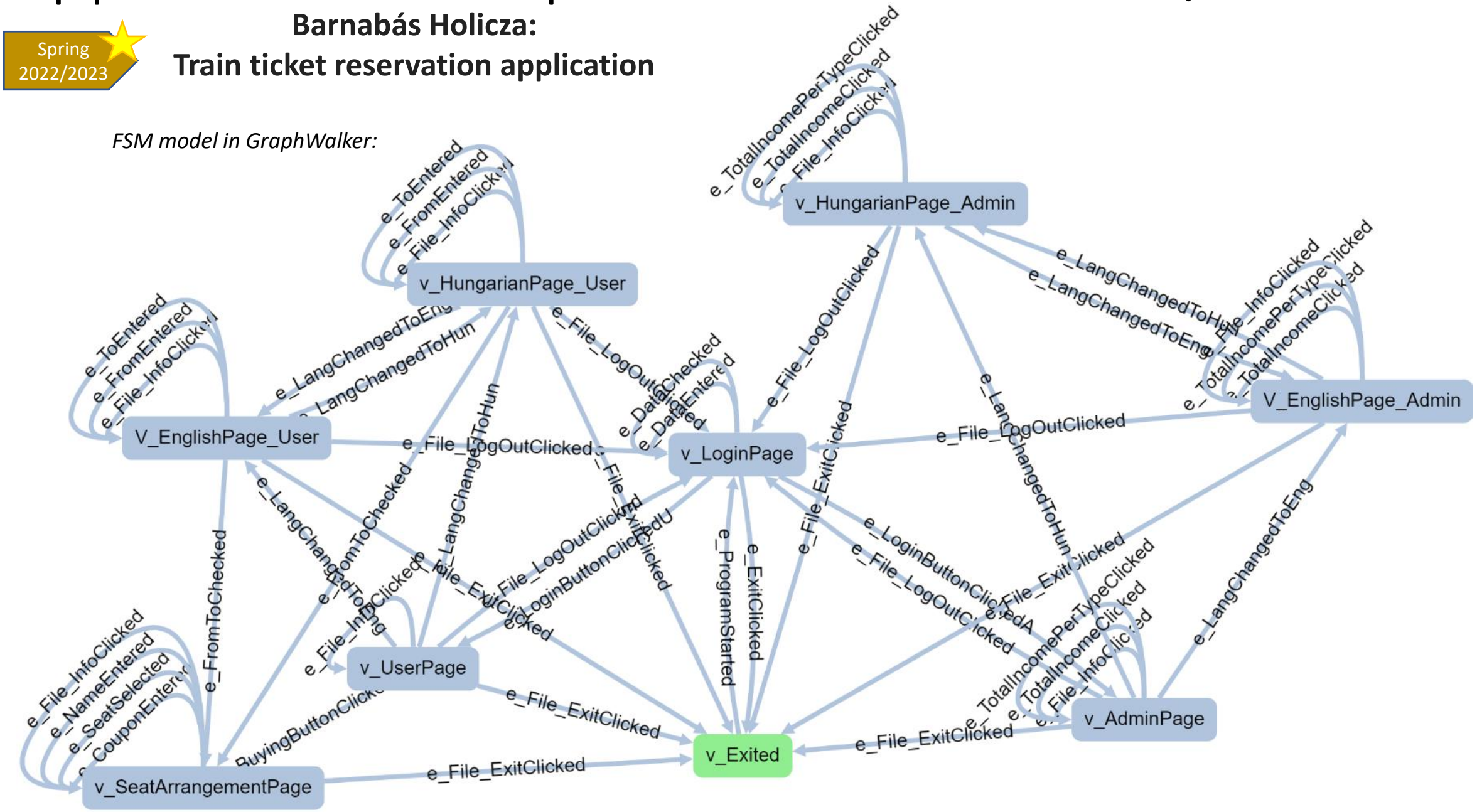
HW ToC

Appendix D: Example homework XXXI. 3/2

Spring 2022/2023

Barnabás Holicza:
Train ticket reservation application

FSM model in GraphWalker:



HW ToC

Appendix D: Example homework XXXI. 3/3

Spring 2022/2023

Barnabás Holicza:
Train ticket reservation application

Adaptation code output:

```
Enter ARIAN2_Jegyfoglalo.exe path:
C:\Users\HB-PC\Desktop\Modelling and testing\ticketme-main\ARIAN2_Jegyfoglalo\
ARIAN2_Jegyfoglalo\bin\Debug\ARIAN2_Jegyfoglalo.exe
Enter json file path:
.\results\TrainTicket-AS_suite.json
10
0 - started/login_page
1 - login_user/user_page
2 - change_lang_eu/english_user_page
3 - check_user/seat_page
4 - ticket_bought/user_page
5 - change_lang_hu/hungarian_user_page
6 - log_out/login_page
7 - login_admin/admin_page
8 - change_lang_ea/english_admin_page
9 - change_lang/hungarian_admin_page
10/10 test completed - took 15,5415855 s
```

The log generated by WinAppDriver shows the component that the program is currently interacting with:

```
=======================================
POST /session/8B99D00E-7FFA-4FAB-BB7A-3488CA36789A/element HTTP/1.1
Accept: application/json, image/png
Connection: Keep-Alive
Content-Length: 47
Content-Type: application/json;charset=utf-8
Host: 127.0.0.1:4723
User-Agent: selenium/3.141.0 (.net windows)
X-Idempotency-Key: 9c778b22-4b45-4b63-afc8-7b78d91d8cda

{"using":"accessibility id","value":"username"}
HTTP/1.1 200 OK
Content-Length: 96
Content-Type: application/json

{"sessionId":"8B99D00E-7FFA-4FAB-BB7A-3488CA36789A","status":0,"value":{"ELEMENT":"42.2296578"}}
```

Test statistics is printed after the execution of all tests:

```
10/10 test completed - took 15,5415855 s
```

Adaptation code example:

```
static void Main()
{
    Console.WriteLine("Enter ARIAN2_Jegyfoglalo.exe path: ");
    string path = Console.ReadLine();
    //C:\Users\HB-PC\Desktop\Modelling and testing\ticketme-main\ARIAN2_Jegyfoglalo\ARIAN2_Jegyfoglalo\bin\Debug\ARIAN2_Jegyfoglalo.exe
    //Program testing
    WindowsDriver<WindowsElement> train;
    AppiumOptions desiredCapabilities = new AppiumOptions();
    desiredCapabilities
        .AddAdditionalCapability("app", @""+path);
    train = new WindowsDriver<WindowsElement>(new Uri("http://127.0.0.1:4723/"), desiredCapabilities);
    if (train == null)
    {
        Console.WriteLine("App not started.");
        return;
    }
    //Reading JSON
    Console.WriteLine("Enter json file path: ");
    string path2 = Console.ReadLine();
    //.\results\TrainTicket-AS_suite.json
    string json = File.ReadAllText(@""+path2);
    JSON io = new JSON(json);
    List<string> edges = new List<string>();
    for (int i = 0; i < io.input_list.Length; i++)
    {
        edges.Add(io.input_list.GetValue(i) + "/" + io.output_list.GetValue(i));
    }
    counter = 0;
    List<string> noDupesEdges = edges.Distinct().ToList();

    var watch = System.Diagnostics.Stopwatch.StartNew();
    Console.WriteLine(edges.Count);
    for (int i = 0; i < edges.Count; i++)
    {
        System.Threading.Thread.Sleep(500);
        Console.Write(i+" - ");
        edgeFunctions(edges[i], train);
    }
    train.Quit();
    watch.Stop();

    var elapseds = watch.Elapsed.TotalSeconds;
    Console.WriteLine(counter+"/"+ edges.Count+" test completed - took "+ elapseds+" s");
}
```

HW ToC

Appendix D: Example homework XXXII. 4/1

Spring 2022/2023

Henrik Zoltán Varga: Dungeon crawler game

SUT (System Under Test): Dungeon crawler game (console application)

- Test possible interactions of the hero with its environment and determining if the game rule implementations are valid or not.

Game definitions & rules:

The game is played in a virtual dungeon, where the player must find a way to a diamond treasure, and then the way back to the exit. The below objects are present:

- **Player(h)**: The hero can be controlled by the player.
- **Wall(x)**: This object cannot be stepped on.
- **Exit(e)**: Reaching the exit results in ending the game either by the result of escaped or win depending on if the player acquired the treasure or not.
- **Diamond(d)**: The player can pick up the treasure by simply stepping on it.
- **Trap(t)**: If the player crosses a trap it transforms into a wall *(see the 2 figures on the right)*
- **Monster(m)**: Encountering a monster can result in 2 things. The player dies, if (s)he does not own a sword, or even if (s)he does, but have only 1 health point. Otherwise, the player defeats the monster but loses one health point. Monsters are static, not moving.
- **Sword(s)**: Picking up the sword can be done the same way as for a treasure, simply by stepping on it.
- **Potion(p):** If the hero has only 1 health points, reaching a potion increases it by 1.

Appendix D: Example homework XXXII. 4/2

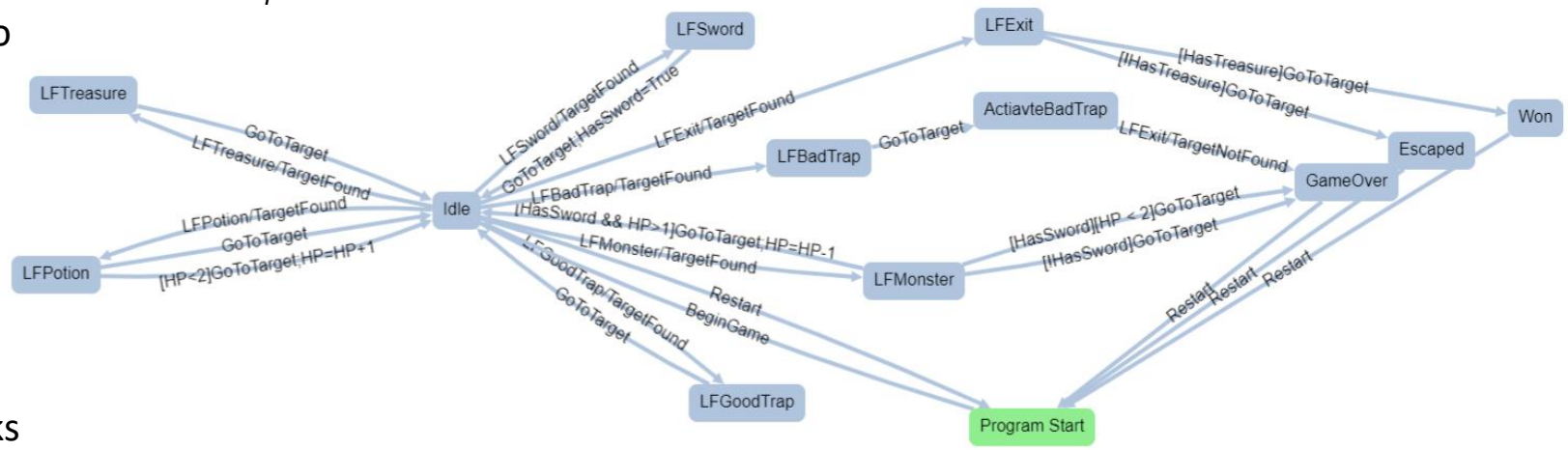
Spring 2022/2023

Henrik Zoltán Varga: Dungeon crawler game

Testing:

- Model has been created with GraphWalker Studio
- Model-based testing with Model 》 Test 》 Relax
- Glue code (adaptation code):
  - Written in C++, organized in 3 classes:
    - Logger: handles log file creation
    - Test Data: reading and processing test suite, determine if tests ends or not
    - Glue code: Execute a given action corresponding to given step and checks output
  - Additional helper python scripts:
    1. Model converter
    2. Parse test suite generated by Model 》 Test 》 Relax for C++ adaptation code)

EFSM model in GraphWalker:



Test generation with Model 》 Test 》 Relax:

```
F:\Uni\Msc\2ndSemester\ModelingAndTesting\MTR-2-0-69-R3BETA\MTR-2-0-69>mtr -o t -m random -f NewCLIGame_converted.json --graphviz --transition_list
[U-CTRL ] [info] Version: 2.0.69 R3: Kingfisher BETA, profile: DEFAULT, verbosity: 3, debug mode: off
[U-JP   ] [warning] No initialization actions defined in JSON model!
[U-LMI  ] [info] Model Name: CLIGame, Type: EFSM, Reliable reset: No
[U-CTRL ] [info] Created graphviz dot file of original model: ./graphviz-Random
[U-CTRL ] [info] Running Random test generation, coverage percent: 100.000000, coverage type: transition
[TG-RWe ] [info] Desired coverage reached: 100.0%,  Walk length: 1328
[TG-RWe ] [info] Finished computation at 2023-05-04 21:38:16.9445873
        elapsed time: (real time) 0.0014882 s
        elapsed time: (user time) 0.001000 s
[U-CTRL ] [info] Writing test suite

F:\Uni\Msc\2ndSemester\ModelingAndTesting\MTR-2-0-69-R3BETA\MTR-2-0-69>
```

HW ToC

Appendix D: Example homework XXXII. 4/3

Spring 2022/2023

Henrik Zoltán Varga: Dungeon crawler game

Adaptation code example:

```
 bool MBT_GlueCode::executeBadTrapAction(Input input, Output output, Game* game, MBT_TestData* data)
{
 Level* m_pLevel = game->getLevel();
 // We need all of them since we don't know prior which one is available
 std::vector<Object*> objectsOfType = m_pLevel->getObjectsOfType(TT_trap);
 if (objectsOfType.empty()) {
  data->getLatestTransition()->resultOutput = parseOutput(TargetNotFound);
  return false;
 }
 // Result list of 'nodes' on the way of the shortest path
 // Optional
 std::vector<Vector2D> objectsToIgnore = std::vector<Vector2D>();
 std::list<AStar::Node*> pathToBadTrap;
 AStar* aStar = new AStar(m_pLevel);
 int i = 0;
 bool foundBadTrap = false;
 while (!foundBadTrap && i < objectsOfType.size()) {
  pathToBadTrap = std::list<AStar::Node*>();

  std::vector<Object*> objectsOfTypeToIgnore = m_pLevel->getObjectsOfType(TT_monster);
  for (int i = 0; i < objectsOfTypeToIgnore.size(); i++) {
   objectsToIgnore.push_back(objectsOfTypeToIgnore[i]->getLocation());
  }

  while (!(aStar->solve(m_pLevel->getHero()->getLocation(), objectsOfType[i]->getLocation(), pathToBadTrap,
objectsToIgnore)) && i < objectsOfType.size())
  {
   // Returns true if found a solution. Here, we are not testing the AI, but the game functionality, so any
path will do.
   i += 1;
  }
  if (i == objectsOfType.size() || pathToBadTrap.empty()){
   data->getLatestTransition()->resultOutput = parseOutput(TargetNotFound);
   return false;
  }
  i += 1;

  std::list<AStar::Node*> pathToExit = std::list<AStar::Node*>();
  Vector2D badTrapLocation = pathToBadTrap.front()->location;
  m_pLevel->createObject<Wall>(badTrapLocation.x, badTrapLocation.y);
  Vector2D upperNeighbour = badTrapLocation;
  upperNeighbour.x = upperNeighbour.x - 1;
  Vector2D leftNeighbour = badTrapLocation;
  leftNeighbour.y = leftNeighbour.y - 1;
  Vector2D lowerNeighbour = badTrapLocation;
  lowerNeighbour.x = lowerNeighbour.x + 1;
  Vector2D rightNeighbour = badTrapLocation;
  rightNeighbour.y = rightNeighbour.y + 1;
  // Starting from a wall is a questionable thing.
  if (!(aStar->solve(upperNeighbour, m_pLevel->getObjectsOfType(TT_exit)[0]->getLocation(), pathToExit,
objectsToIgnore))) {
   foundBadTrap = true;
  }
  else {
   pathToExit = std::list<AStar::Node*>();
  }
  if (!foundBadTrap && !(aStar->solve(lowerNeighbour, m_pLevel->getObjectsOfType(TT_exit)[0]->getLocation(),
pathToExit, objectsToIgnore))) {
   foundBadTrap = true;
  }
  else {
   pathToExit = std::list<AStar::Node*>();
  }
  if (!foundBadTrap && !(aStar->solve(leftNeighbour, m_pLevel->getObjectsOfType(TT_exit)[0]->getLocation(),
pathToExit, objectsToIgnore))) {
   foundBadTrap = true;
  }
  else {
   pathToExit = std::list<AStar::Node*>();
  }
  if (!foundBadTrap && !(aStar->solve(rightNeighbour, m_pLevel->getObjectsOfType(TT_exit)[0]->getLocation(),
pathToExit, objectsToIgnore))) {
   foundBadTrap = true;
  }
  else {
   pathToExit = std::list<AStar::Node*>();
  }
  m_pLevel->createObject<Trap>(badTrapLocation.x, badTrapLocation.y);
 }
 if (!foundBadTrap) {
  data->getLatestTransition()->resultOutput = parseOutput(TargetNotFound);
  return false;
 }

 latestPathToObject.clear();
 latestPathToObject.swap(pathToBadTrap);
 data->getLatestTransition()->resultOutput = parseOutput(TargetFound);
 return true;
}
```

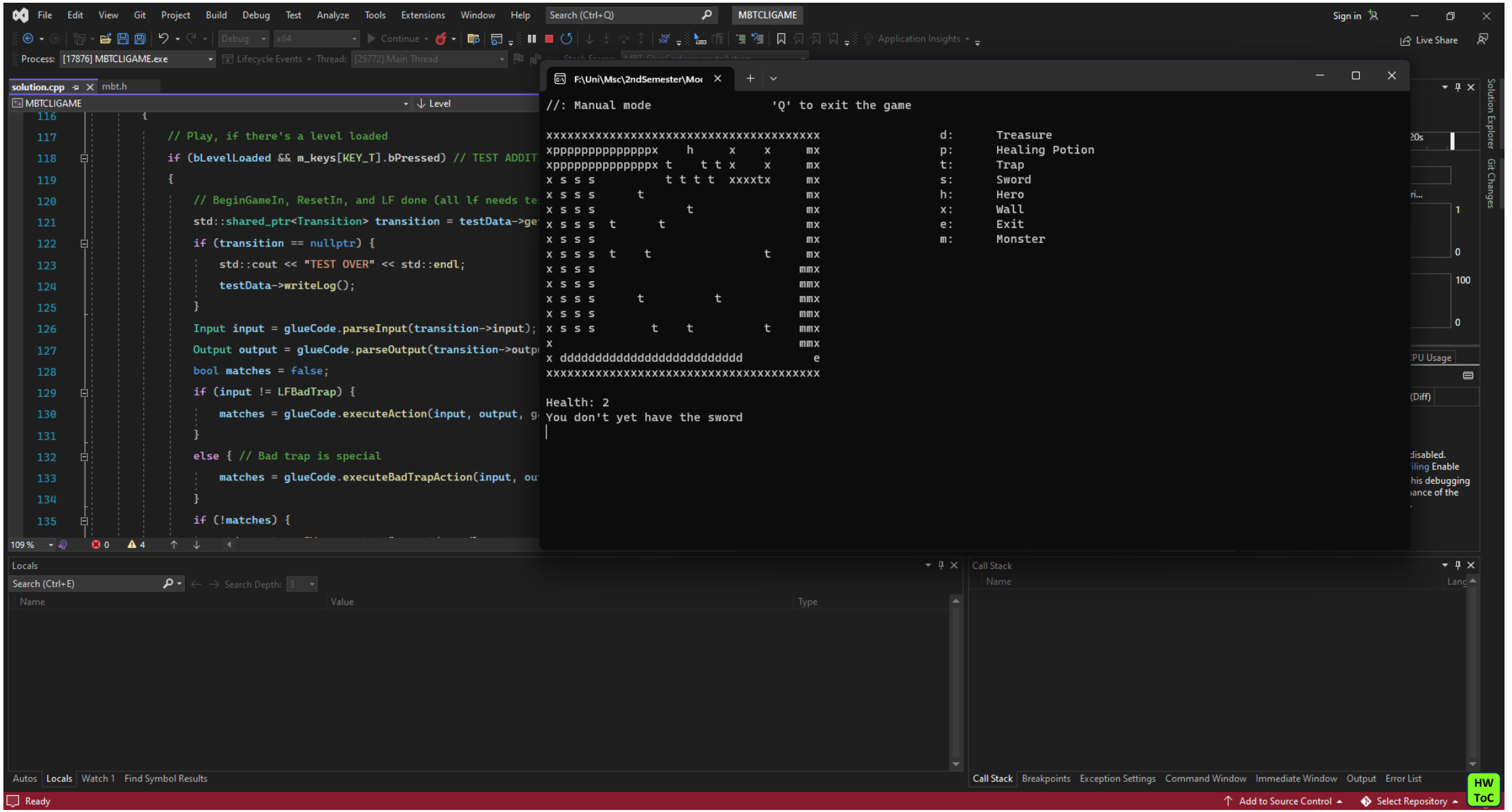
HW ToC

Appendix D: Example homework XXXII. 4/4

Spring 2022/2023

Henrik Zoltán Varga: Dungeon crawler game

Test execution →

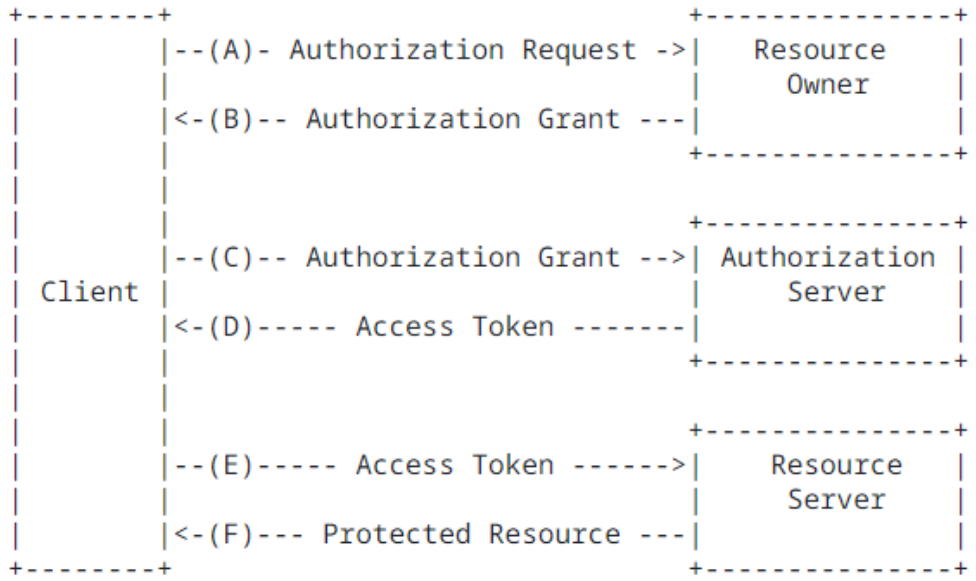


Appendix D: Example homework XXXIII. 3/1

Fall 2023/2024

Botond Dénes: OpenIddict Oauth2.0/OpenId Connect library

- SUT (System Under Test): OpenIddict Oauth2.0 / OpenId Connect library
  - Tested functionalities according to RFC 6749:
    - **Covered flows:**
      - Authorization Code Grant
      - Resource Owner Password Credentials Grant
      - Refresh Token Flow (description of related parameters)
    - **Alternate scenarios:**
      - When a refresh token is redeemed, a new one is issued, the old one is invalidated
    - **Error handling:**
      - The server must reject invalid authorization codes, access tokens and refresh tokens.
      - The server must reject expired authorization codes, access tokens and refresh tokens.
      - The server must reject already claimed authorization codes and refresh tokens.
      - The server must reject invalid password and usernames.



Abstract Protocol Flow of Oauth2.0
(Source of the figure)

- Testing:
  - EFSM Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in C#
    - Using API

M▶T▶R *Note that this example is included in the Model 》 Test 》 Relax repository under subfolder* `sample_models/applications/OpenIddict`

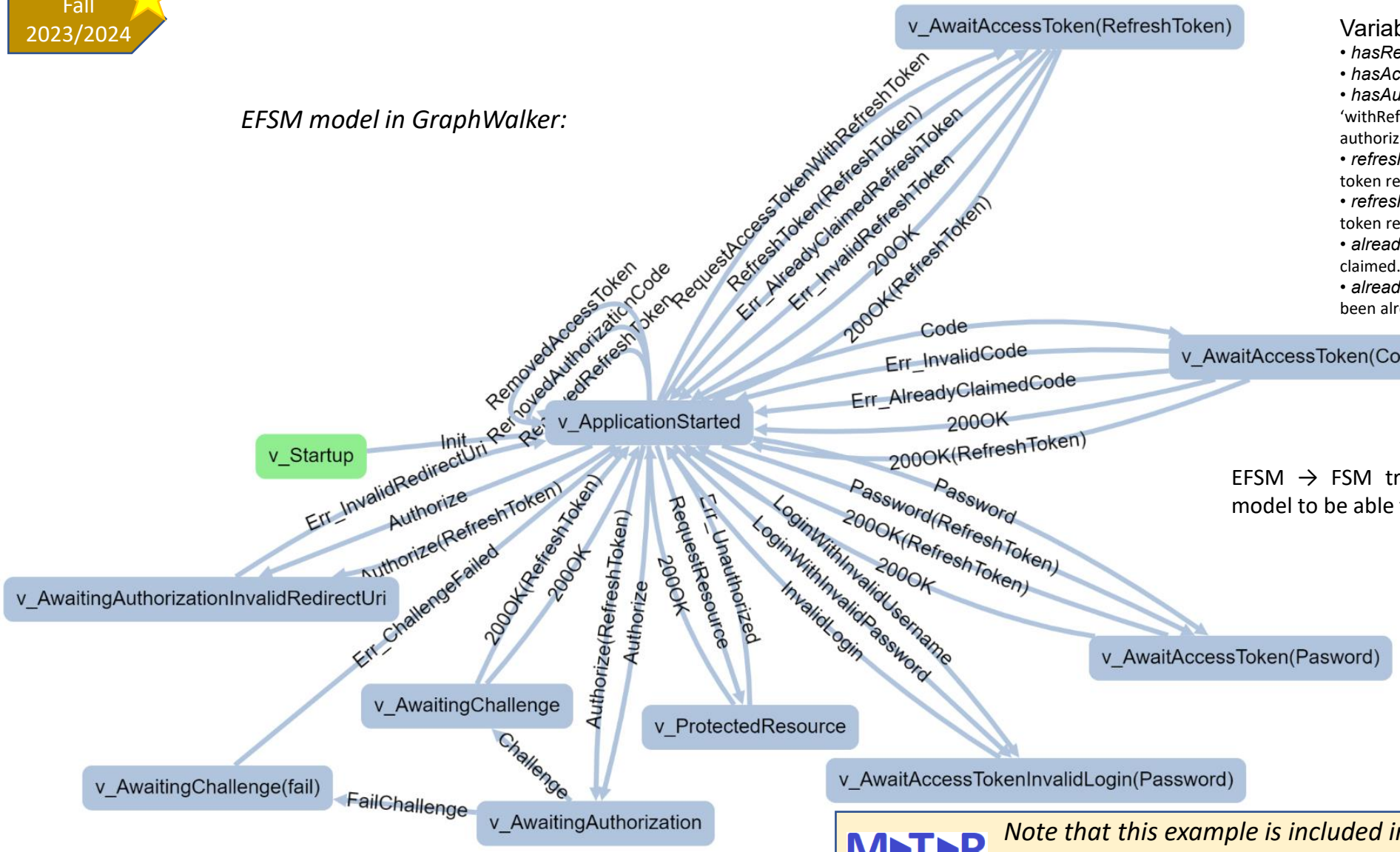
HW ToC

Appendix D: Example homework XXXIII. 3/2

Fall 2023/2024

Botond Dénes: OpenIddict Oauth2.0/OpenId Connect library

EFSM model in GraphWalker:



Variables:

- *hasRefreshToken*: represents if the user has a refresh token
- *hasAccessToken*: represents if the user has an access token
- *hasAuthorizationCode*: This is an enum with three values: 'no', 'withRefreshToken', 'withoutRefreshToken'. Represents if the user has an authorization code that will also request a refresh token or not.
- *refreshTokenRequest*: represents if the user has made a refresh token request in the password grant or the refresh token grant.
- *refreshTokenRequestCode*: represents if the user has made a refresh token request in the authorization code grant.
- *alreadyClaimedRefreshToken*: if the refresh token has been already claimed.
- *alreadyClaimedAuthorizationCode*: if the authorization code has been already claimed.

EFSM → FSM transformation has been applied on the EFSM model to be able to use FSM test generation algorithms

Note that this example is included in the [Model ⟫ Test ⟫ Relax] repository under subfolder `sample_models/applications/OpenIddict`

HW ToC

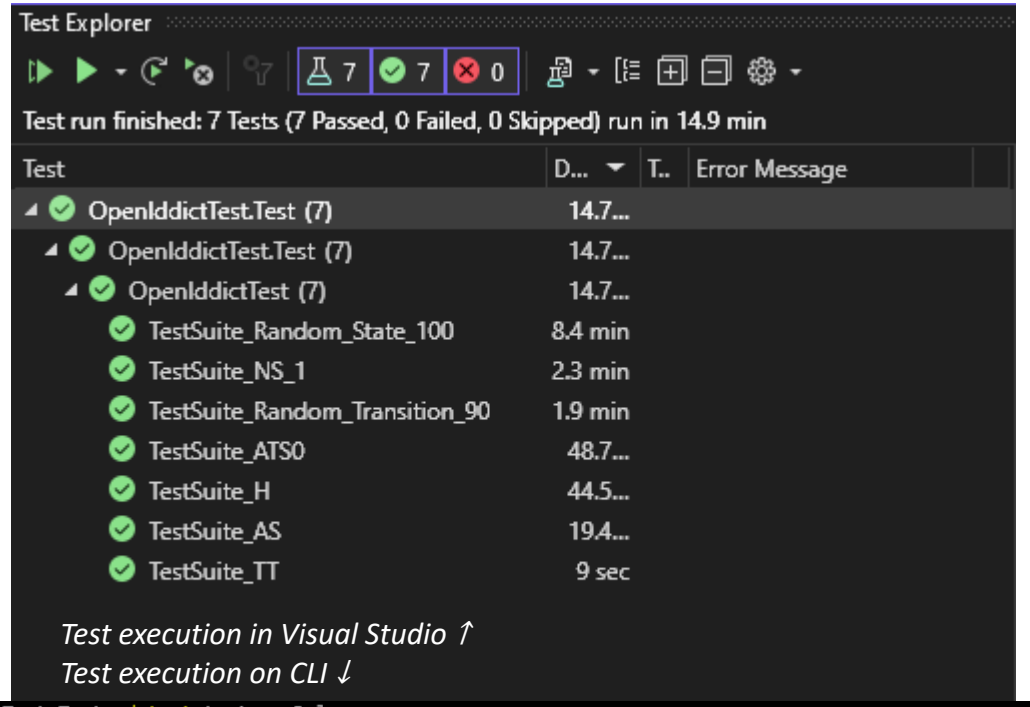
Appendix D: Example homework XXXIII. 3/3

Fall 2023/2024

Botond Dénes: OpenIddict Oauth2.0/OpenId Connect library

Adaptation code examples:

```
private async Task StartChallengeAsync(string username, string password)
{
        await CheckResponse(HttpStatusCode.Unauthorized);
        var jsonObject = JsonSerializer.Deserialize<Dictionary<string, string>>(await
        _response.Content.ReadAsStringAsync());
        Assert.IsNotNull(jsonObject);
        _challengeCode = jsonObject["challengeCode"];
        Assert.IsNotNull(_challengeCode);
        var pars = new Dictionary<string, string>()
        {
                { "response_type", "code" },
                { "client_id", "console" },
                { "client_secret", "388D45FA-B36B-4988-BA59-B187D329C207" },
                { "redirect_uri", "http://localhost:5215/redirect" },
                { "challenge_code", _challengeCode },
                { "username", username },
                { "password", password }
        };
        var content = new FormUrlEncodedContent(pars);
        _response = await _serverClient.PostAsync("http://localhost:5209/connect/authorize", content);
}

…
private async Task CompleteChallengeAsync()
{
        await CheckResponse(HttpStatusCode.Found);
        var location = _response.Headers.Location;
        Assert.IsNotNull(location);
        _authorizationCode = HttpUtility.ParseQueryString(location.Query).Get("code");
}

private async Task FailChallengeAsync()
{
        await CheckResponse(HttpStatusCode.Found);
        var location = _response.Headers.Location;
        Assert.IsNotNull(location);
        Assert.IsNotNull(HttpUtility.ParseQueryString(location.Query).Get("error"));
}
```



Test execution in Visual Studio ↑
Test execution on CLI ↓

```
PS E:\OpenIddictTest\OpenIddictTest.Test> dotnet test -c Release
  Determining projects to restore...
  All projects are up-to-date for restore.
  OpenIddictTest.Client -> E:\OpenIddictTest\OpenIddictTest.Client\bin\Release\net6.0\OpenIddictTest.Client.dll
  OpenIddictTest.Server -> E:\OpenIddictTest\OpenIddictTest.Server\bin\Release\net6.0\OpenIddictTest.Server.dll
  OpenIddictTest.Test -> E:\OpenIddictTest\OpenIddictTest.Test\bin\Release\net6.0\OpenIddictTest.Test.dll
Test run for E:\OpenIddictTest\OpenIddictTest.Test\bin\Release\net6.0\OpenIddictTest.Test.dll (.NETCoreApp,Version=v6.0)
Microsoft (R) Test Execution Command Line Tool Version 17.7.2 (x64)
Copyright (c) Microsoft Corporation.  All rights reserved.

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:     7, Skipped:     0, Total:     7, Duration: 15 m 28 s - OpenIddictTest.Test.dll (net6.0)
```

M▶T▶R *Note that this example is included in the [Model ⟫ Test ⟫ Relax] repository under subfolder* `sample_models/applications/OpenIddict`

HW ToC

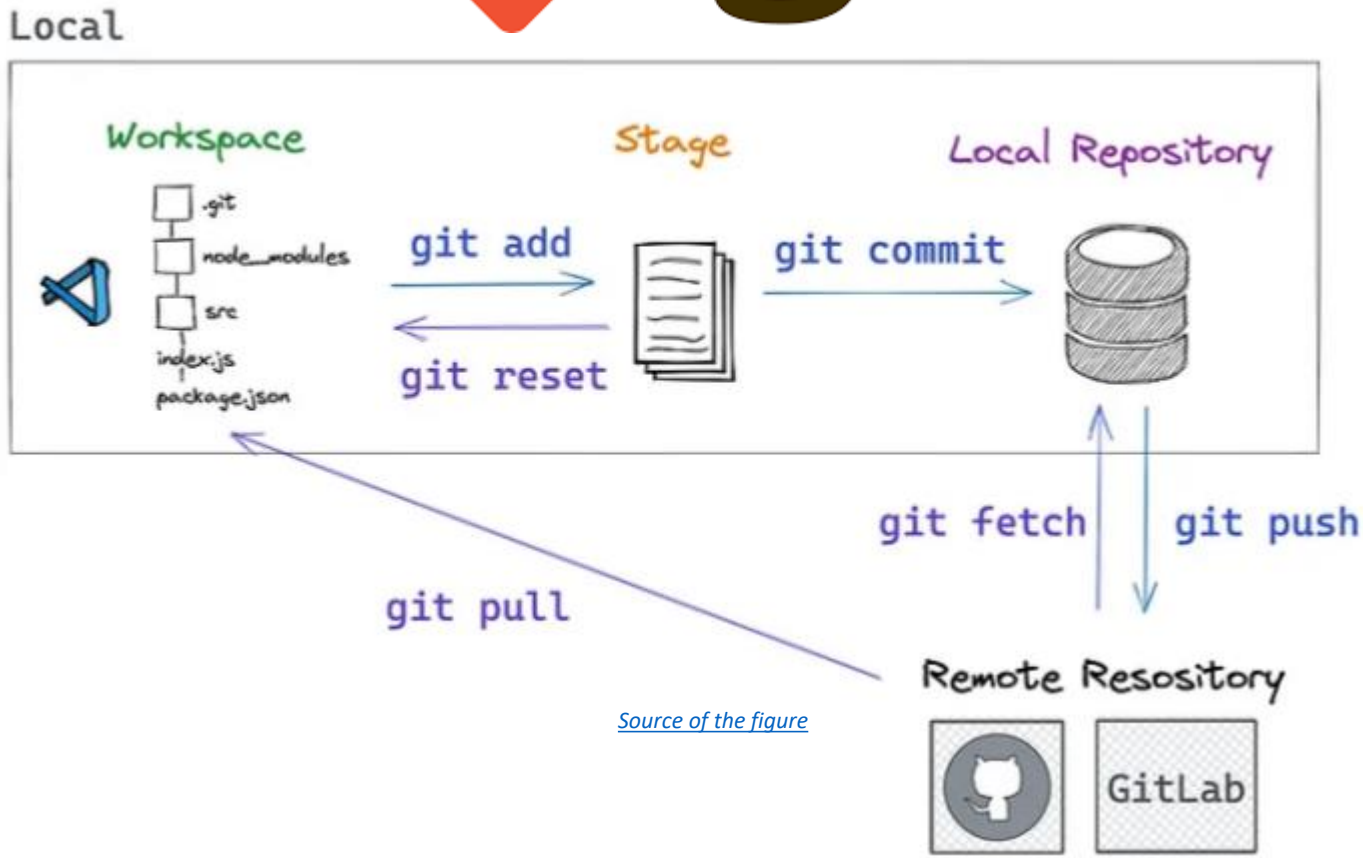
Appendix D: Example homework XXXIV. 4/1

Fall 2023/2024

Gábor Spaits: Git

- SUT (System Under Test): git version control system
  - Tested functionalities:
    - Changes and additions on git tree
  - Limitations:
    - One branch
    - Files are not tracked one by one

- Testing:
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
    - Using CLI
    - Full pipeline has been implemented



Source of the figure

Note that this example is included in the Model 》 Test 》 Relax repository under subfolder `sample_models/applications/gittree`

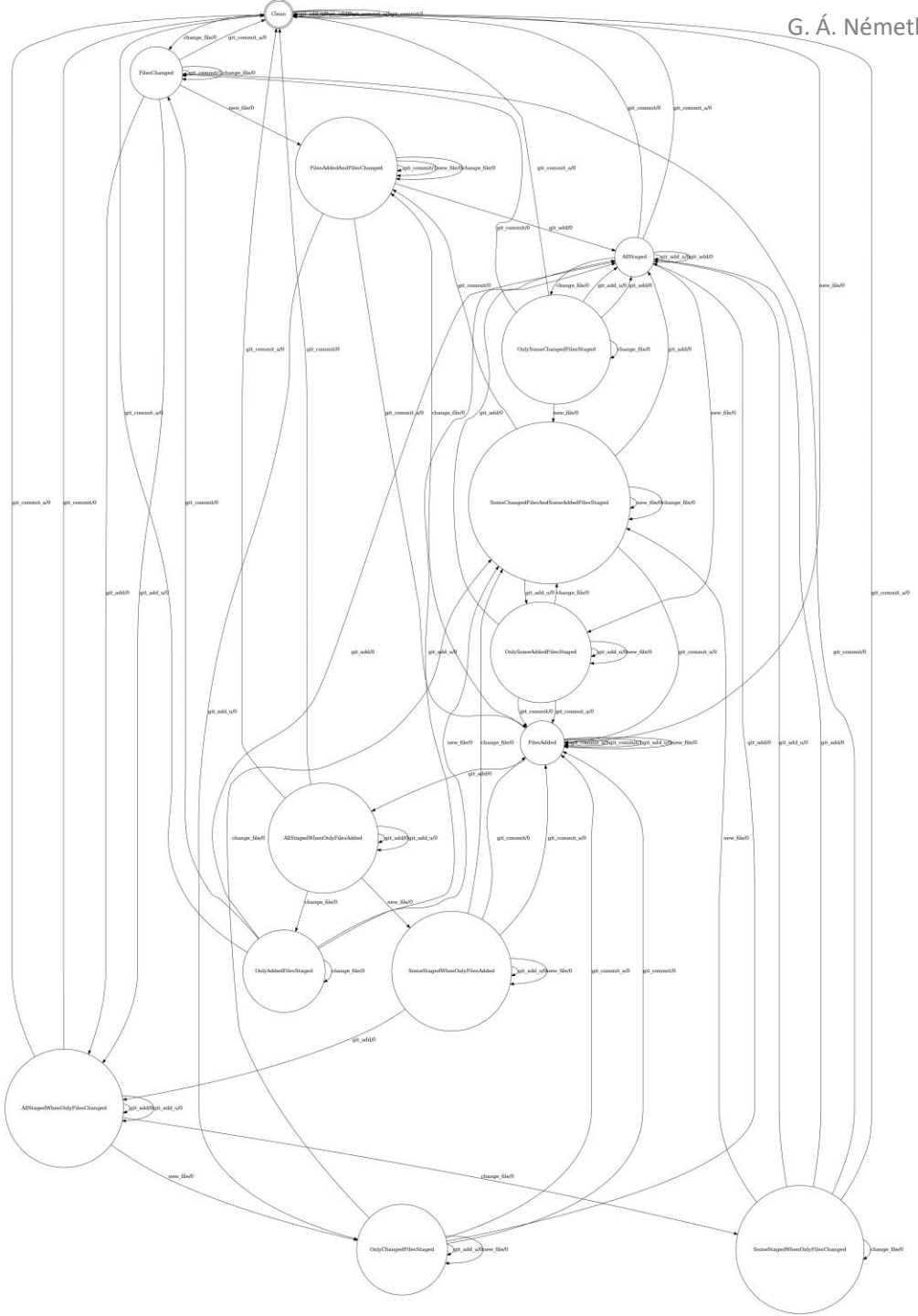
HW ToC

Appendix D:
Example homework XXXIV. 4/2

Fall 2023/2024

Gábor Spaits: Git

FSM model →



```
Kis Huba@DESKTOP-6FV1G44 MINGW64 ~/ModelTestRelax (313-weak-connectivity)
$ git pull
remote: Enumerating objects: 21, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (20/20), done.
remote: Total 21 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (21/21), 19.25 KiB | 65.00 KiB/s, done.
From gitlab.inf.elte.hu:nga/mbt-repository
   e28e30b9..5acd6b29  313-weak-connectivity -> origin/313-weak-connectivity
Updating e28e30b9..5acd6b29
Fast-forward
 docs/release_notes/release_notes.md                |  2 +-
 sample_models/test/FSM/WeaklyConnected.json        | 21 ++++++++++++----------
 .../common/src/algorithm_precondition_checks.cpp   |  2 +-
 src/algorithms/test_gen/src/TestGenRunner.cpp      |  2 +-
 src/test/TestGenerationTest.h                      |  2 +-
 5 files changed, 15 insertions(+), 14 deletions(-)

Kis Huba@DESKTOP-6FV1G44 MINGW64 ~/ModelTestRelax (313-weak-connectivity)
$
```

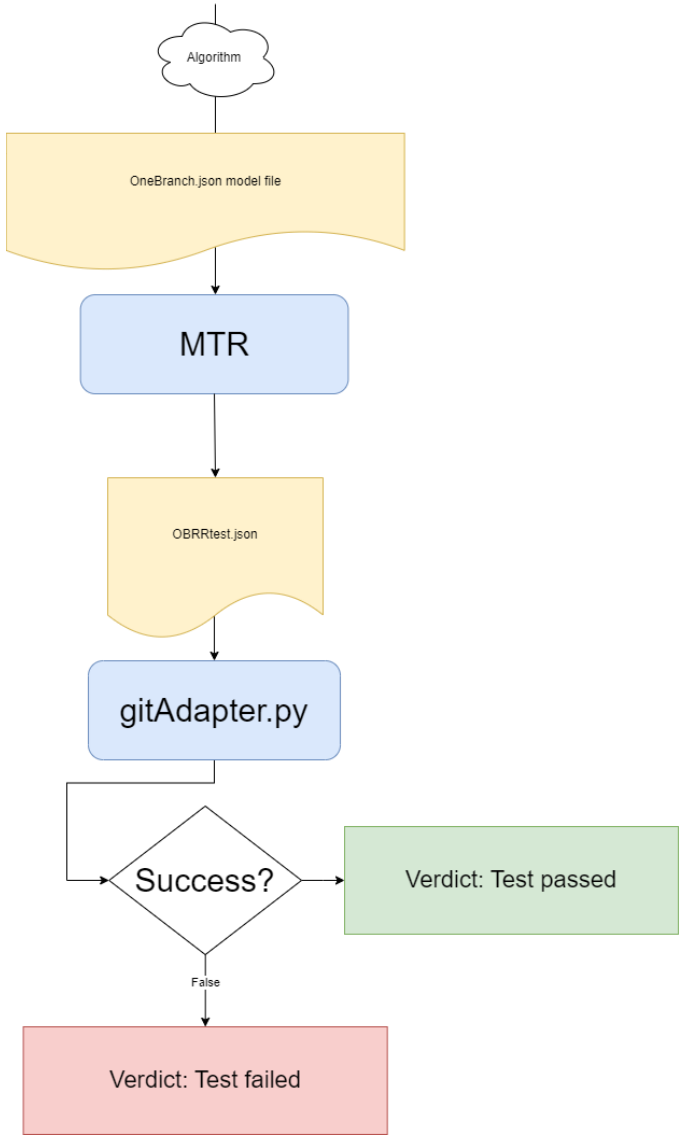
HW ToC

Appendix D: Example homework XXXIV. 4/3

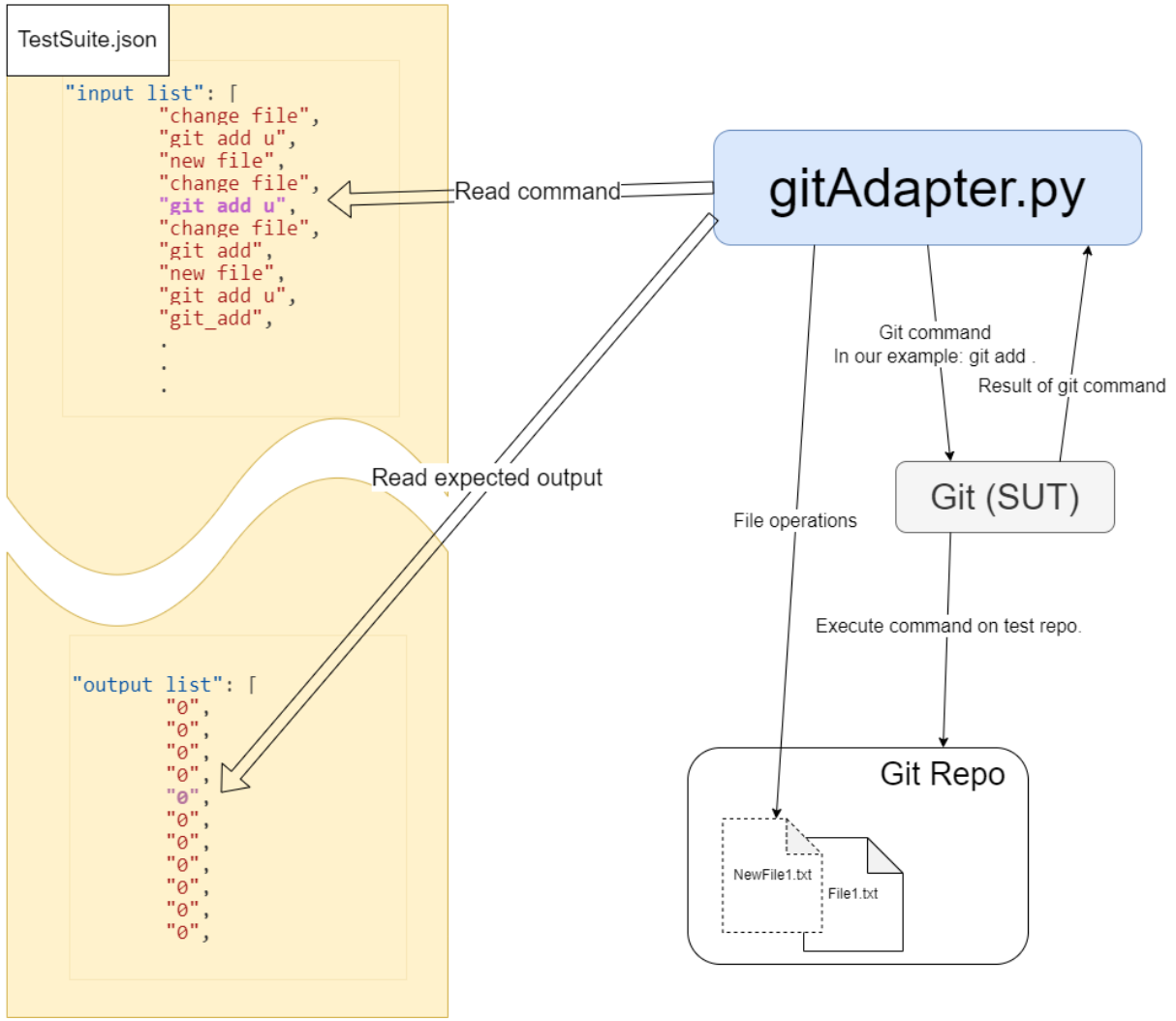
Fall 2023/2024

Gábor Spaits: Git

The final test pipeline:



Test execution:



Note that this example is included in the [Model ⟫ Test ⟫ Relax](#) repository under subfolder `sample_models/applications/gittree`

HW ToC

Appendix D: Example homework XXXIV. 4/4

Fall 2023/2024

Gábor Spaits: Git

Adaptation code examples:

The main loop doing the execution:

```
# Execute each operation in the test suite
for i in range(InputListLen):

    givenInput = InputList[i]
    expectedOutput = OutputList[i]

    # Since the reset is reliable we can skip the testing of it's
        correctness
    if givenInput != "<reset>":
        expectedOutput = int(OutputList[i][0])
    print("input:", givenInput, " output:", OutputList[i])

    # Execute the operation and check if it's output matches the
        expected
    match executeOneOperation(givenInput, expectedOutput):
        case (True, Txt):
            pass
        case (False, Reason):
            print("Error! input count:",i,"of",InputListLen,"
                message:", Reason)
            sys.exit(-1)
        case _:
            # If an operation is not implemented the test execution
                terminates
            print("Should Not be here")
            sys.exit(-1)
```

The code executed when `git add .` is the input:

```
def executeOneOperation(Operation, ReturnValue):
  match Operation:
          .
          .
          .
          case "git_add":
              return checkOutput(Operation, ReturnValue, git_add())
          .
          .
          .
          case _:
              print("OP not implemented:", Operation)
              return ""
```

```
def git_add():
  info = subprocess.run(["git", "add", "."], capture_output=True)
  return info.returncode
```

Note that this example is included in the [Model ⟫ Test ⟫ Relax] repository under subfolder `sample_models/applications/gittree`

HW ToC

Appendix D: Example homework XXXV.

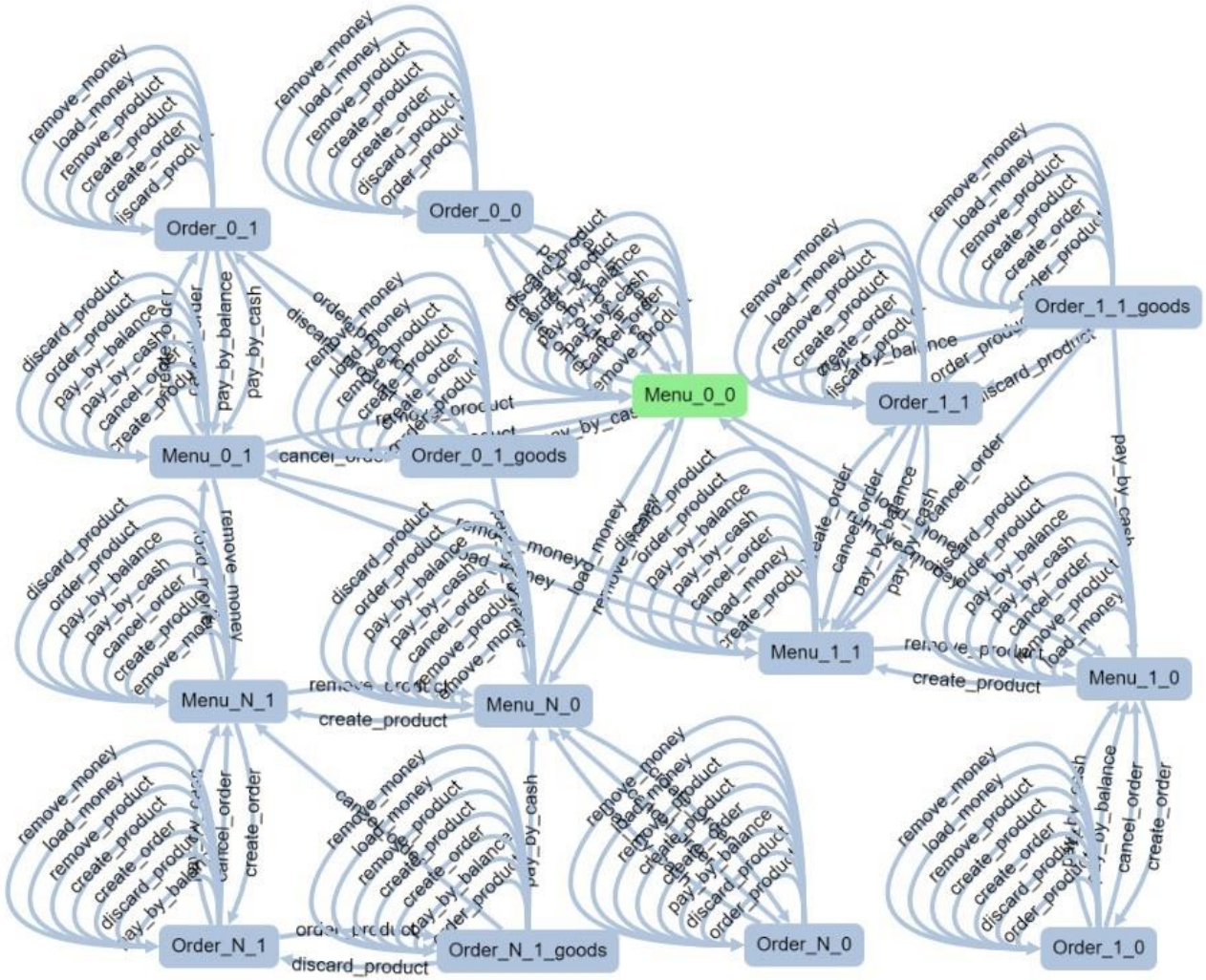
Fall 2023/2024

Gergő Szalay: Cookie Shop Backend

- SUT (System Under Test): Cookie Shop backend
  - Tested functionalities:
    - Managing the storage of the shop
    - Account management
    - Orders are made through the application
  - Limitations:
    - Only one customer
    - Only one product type
    - Limitations on the number of shopped products

- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax
    - Mutation testing has been also performed by insert faults to the…
      - (1) SUT,
      - (2) specification model and
      - (3) implementation
    - …to discover the fault coverage of different test generation algorithms
  - Glue code (adaptation code):
    - Written in Python
      - Load generated test suite, return value of the SUT is compared to the output expected from the generated test suite

EFSM model in GraphWalker:



HW ToC

Appendix D: Example homework XXXVI. 3/1

Fall 2023/2024

Bence Ladomérszky: MadTycoon

- SUT (System Under Test): MadTycoon tycoon simulator game
  - The player can design, organize and rule his own theme park
  - Tested functionalities:
    - Build roads, place a building or decoration
    - Change settings of a placed building
    - Loan money
    - See statistics

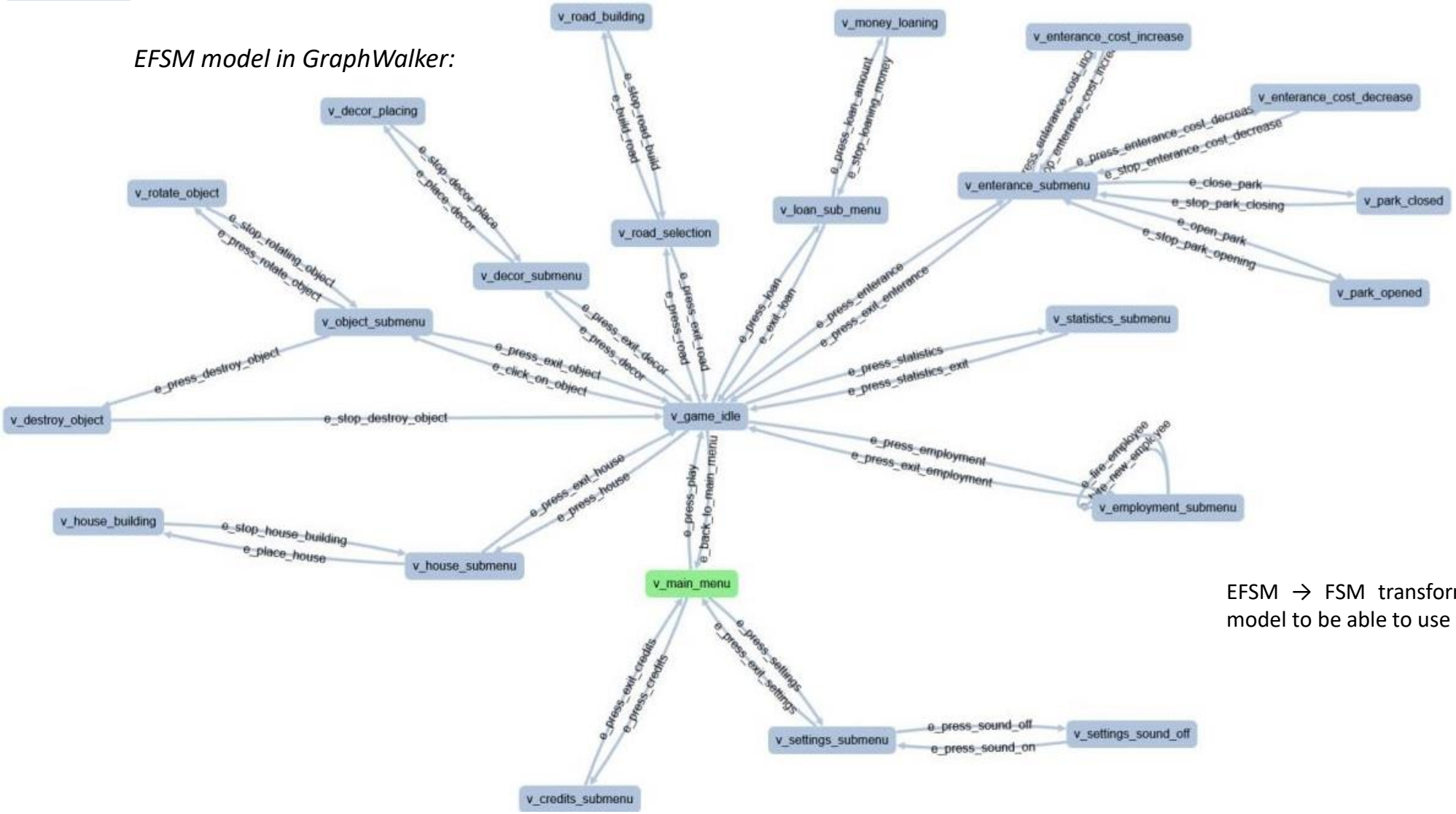
- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - GUI testing is written in Java (using intelIJ)

Appendix D: Example homework XXXVI. 3/2

Fall 2023/2024

Bence Ladomérszky: MadTycoon

EFSM model in GraphWalker:



EFSM → FSM transformation has been applied on the EFSM model to be able to use FSM test generation algorithms

HW ToC

Appendix D: Example homework XXXVI. 3/3

Fall 2023/2024

Bence Ladomérszky: MadTycoon

Sample adaptation code snippets ↓ →

Placing a house:

```
case "place_house":
    cur_menu.engine.hud.buildingMenu.doClick();
    cur_menu.engine.builder.setSelected(ScreenToWorld(new Point(houseOneX,houseOneY)));
    cur_menu.engine.builder.interact();
    err.add(checkOutput(output.getString(i),input.getString(i)));
    break;
```

```
case "house_placed":
    if (cur_menu.engine.builder.world.getGames().isEmpty()) {
        error++;
        error_info = new Errors(input, output, isPassed: false);
    }                   else{
        error_info = new Errors(input,output, isPassed: true);
    }
    break;
```

Error handling:

```
List<Errors> err = new ArrayList<>();
```

```
54 usages  new *
class Errors{
    2 usages
    public String input;
    2 usages
    public String output;
    2 usages
    public boolean isPassed;
    48 usages  new *
    public Errors(String input, String output, boolean isPassed){this.input = input; this.output = output; this.isPassed = isPassed;}
}
```

```
    1 usage  new *
    static void reportError() {
        try {
            FileWriter myWriter = new FileWriter( fileName: "result.txt");
            for (Errors err : errors) {
                myWriter.write( str: err.input + " - " + err.output + ((err.isPassed) ? " PASSED" : " FAILED") + "\n");
            }
            myWriter.close();
        } catch (IOException e) {
            System.out.println("An error occurred.");
        }
    }
}
```

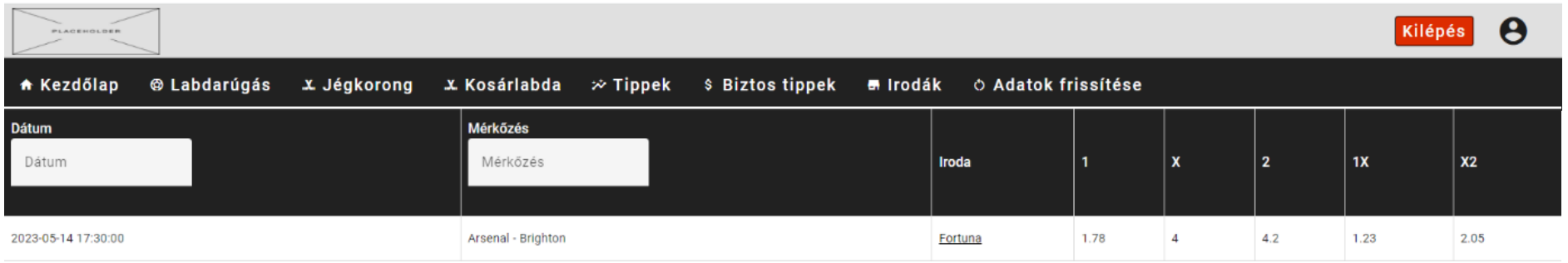
```
Test-sequence:  MadTycoon-transformed-example.json
press_settings - settings_open PASSED
press_play - game_started PASSED
press_house - house_menu_open PASSED
place_house - house_placed PASSED
stop_house_building - house_placed PASSED
press_exit_house - normal_hud PASSED
press_decor - decor_menu_open PASSED
press_decor - decor_menu_open PASSED
place_decor - decor_placed PASSED
stop_decor_place - decor_placed PASSED
press_exit_decor - normal_hud PASSED
press_house - house_menu_open PASSED
press_exit_house - normal_hud PASSED
press_road - road_placeable PASSED
build_road - road_placed PASSED
stop_road_build - road_placed PASSED
press_exit_road - road_exited PASSED
press_house - house_menu_open PASSED
press_exit_house - normal_hud PASSED
press_decor - decor_menu_open PASSED
press_decor - decor_menu_open PASSED
press_exit_decor - normal_hud PASSED
press_statistics - statistics_open PASSED
press_employment - employment_open PASSED
press_loan - loan_open PASSED
press_loan_amount - money_loaned PASSED
exit_loan - loan_closed PASSED
press_enterance - enterance_open PASSED
press_enterance_cost_increase - enterance_cost_increased PASSED
stop_enterance_cost_increase - enterance_cost_increased PASSED
press_enterance_cost_decrease - enterance_cost_decreased PASSED
stop_enterance_cost_decrease - enterance_cost_decreased PASSED
close_park - park_closed PASSED
open_park - park_opened PASSED
press_exit_enterance - enterance_open_close PASSED
press_credits - credits_open PASSED
Time taken: 2.396s
```

HW ToC

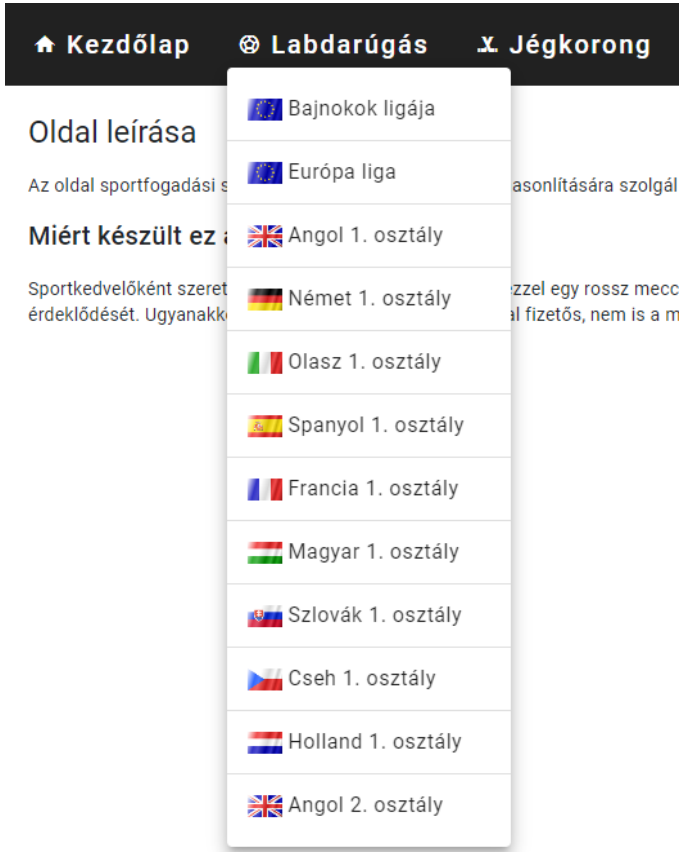
Appendix D: Example homework XXXVII. 3/1

Fall 2023/2024

Marián Vajda: Web application for betting odds



- SUT (System Under Test): Web application for betting odds
  - Tested functionalities:
    - Registration and login
    - Menu and navigation
    - Matches with the odds. Fields can be filtered, and the odds can be sorted.
    - Feedback form
    - User roles: admin/subscriber/non-subscriber/not logged
    - API requests



- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - GUI tests are written in Java, using Selenium
    - API testing with Postman

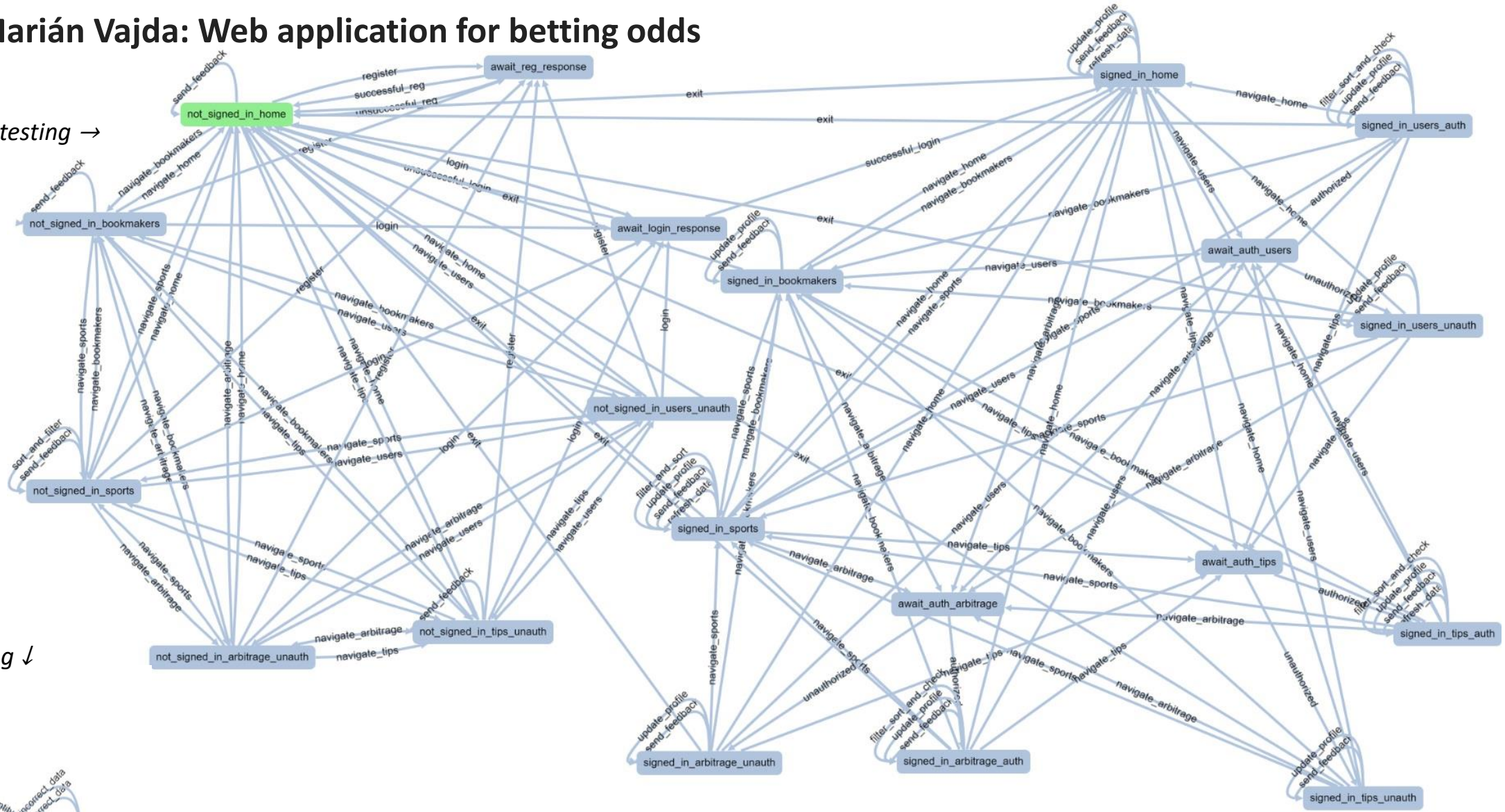
HW ToC

Appendix D: Example homework XXXVII. 3/2

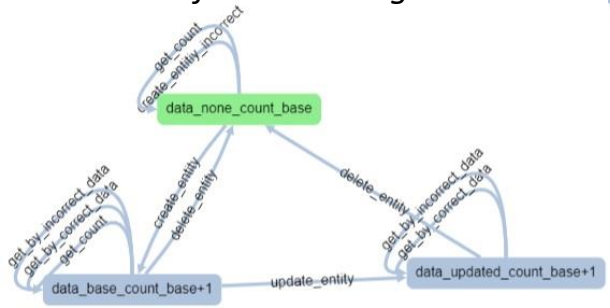
Fall 2023/2024

Marián Vajda: Web application for betting odds

FSM model for GUI testing →



FSM model for API testing ↓



HW ToC

Appendix D: Example homework XXXVII. 3/3

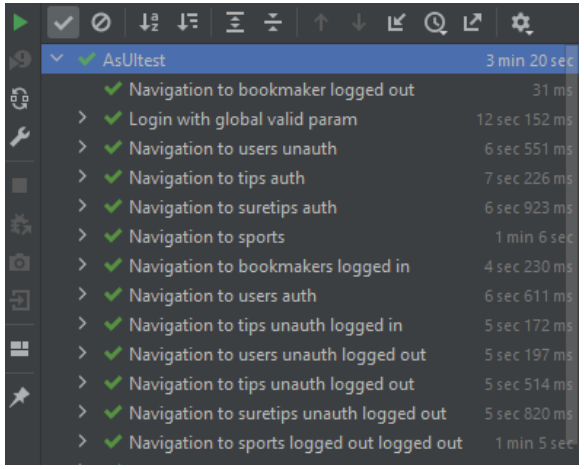
Fall 2023/2024

Marián Vajda: Web application for betting odds

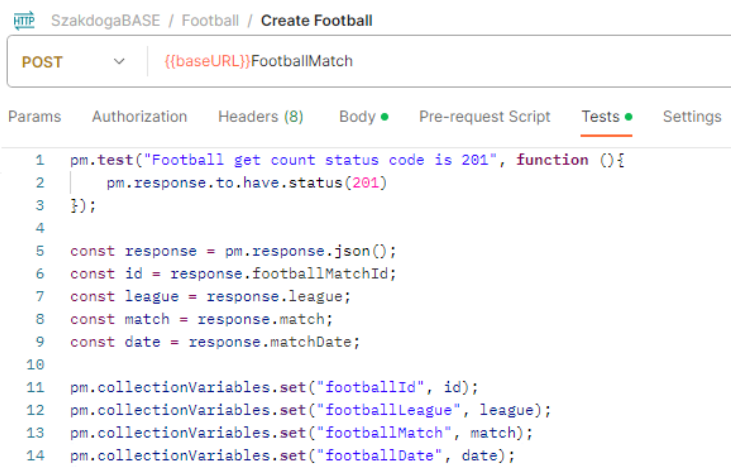
An adaptation code example for GUI testing (Open tips and sort it):

```
public class BaseEntityPageObject extends BasePageObject{
    public String entityUrl;
    /** Menu locators */
    protected By homeMenuElement = By.xpath("/html/body/app-root/body/app-menu/div/mat-toolbar/div[2]/a[1]");
    protected By footballMenuElement = By.xpath("/html/body/app-root/body/app-menu/div/mat-
toolbar/div[2]/button[1]");
    protected By icehockeyMenuElement = By.xpath("/html/body/app-root/body/app-menu/div/mat-
toolbar/div[2]/button[2]");
    protected By basketballMenuElement = By.xpath("/html/body/app-root/body/app-menu/div/mat-
toolbar/div[2]/button[3]");
    protected By tipsMenuElement = By.xpath("/html/body/app-root/body/app-menu/div/mat-toolbar/div[2]/a[2]");
    protected By suretipsMenuElement = By.xpath("/html/body/app-root/body/app-menu/div/mat-
toolbar/div[2]/a[3]");
    protected By bookmakersMenuElement = By.xpath("/html/body/app-root/body/app-menu/div/mat-
toolbar/div[2]/a[4]");
    protected By usersMenuElement = By.xpath("/html/body/app-root/body/app-menu/div/mat-toolbar/div[2]/a[5]");
    protected By refreshButton = By.xpath("/html/body/app-root/body/app-menu/div/mat-
toolbar/div[2]/button[4]");


    public BaseEntityPageObject(WebDriver driver, LogFactory log) {
        super(driver, log);
    }

    public void openMenuElement(By menu, By element){
        try{
            if (isOnPage(element)) {
                click(element);
                log.info("Element opened, was seeable by default");
            }
            else if (isOnPage(element) == false){
                click(menu);
                click(element);
                log.info("Element opened by dropdown, was not seeable by default");
            }
            log.info("open menu element: " + element);
        }
        catch (ElementNotInteractableException exp){
            log.interactionError(log,"open menu element", exp);
        }
    }

...

public class TipsPO extends BaseEntityPageObject {

    /** Main locator */
    public By errorMessageElement = By.xpath("/html/body/app-root/body/main/app-tips/body/h1");
    public By firstColumn = By.xpath("/html/body/app-root/body/main/app-tips/div/table/thead/tr/th[1]");
    protected By sortButtton4 = By.xpath("/html/body/app-root/body/main/app-
tips/div/table/thead/tr/th[4]/div/div[2]");
    public TipsPO(WebDriver driver, LogFactory log){
        super(driver, log);
        this.entityUrl = "tips";
    }

    public void openTipsByMenu(){
        click(tipsMenuElement);
    }

    public void sort(){
        click(sortButtton4);
    }

}
```

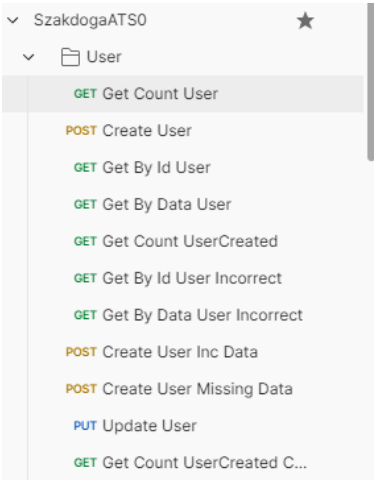
GUI testing with Selenium:



Tests of a Postman test request:



API testing



Summarize the results of a test execution (ATS API tests):

	executed	failed
iterations	1	0
requests	58	0
test-scripts	116	0
prerequest-scripts	68	0
assertions	77	0
total run duration: 5.9s		
total data received: 9.41kB (approx)		
average response time: 12ms [min: 4ms, max: 137ms, s.d.: 20ms]		

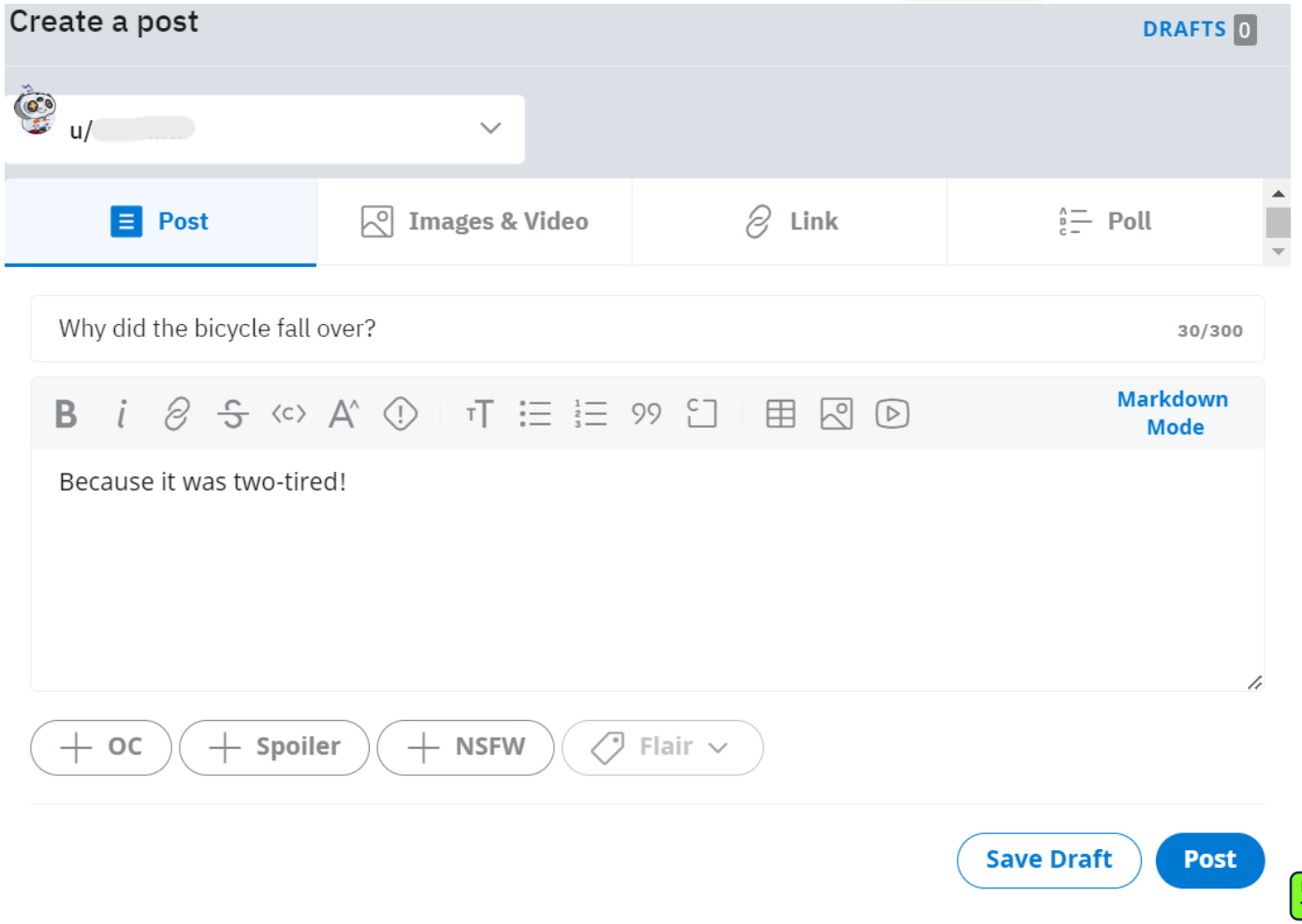
HW ToC

Appendix D: Example homework XXXVIII. 3/1

Fall 2023/2024

Anna Sike: Reddit

- SUT (System Under Test): Reddit social media platform
  - Tested post-related functionalities:
    - Voting
    - Save/unsave post
    - Hide/unhide post
    - Report post
    - Follow/unfollow post
    - Comment
    - Sorting
    - Create post
- Testing:
  - Model has been created with GraphWalker Studio
  - Model-based testing Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
    - Using Reddit API



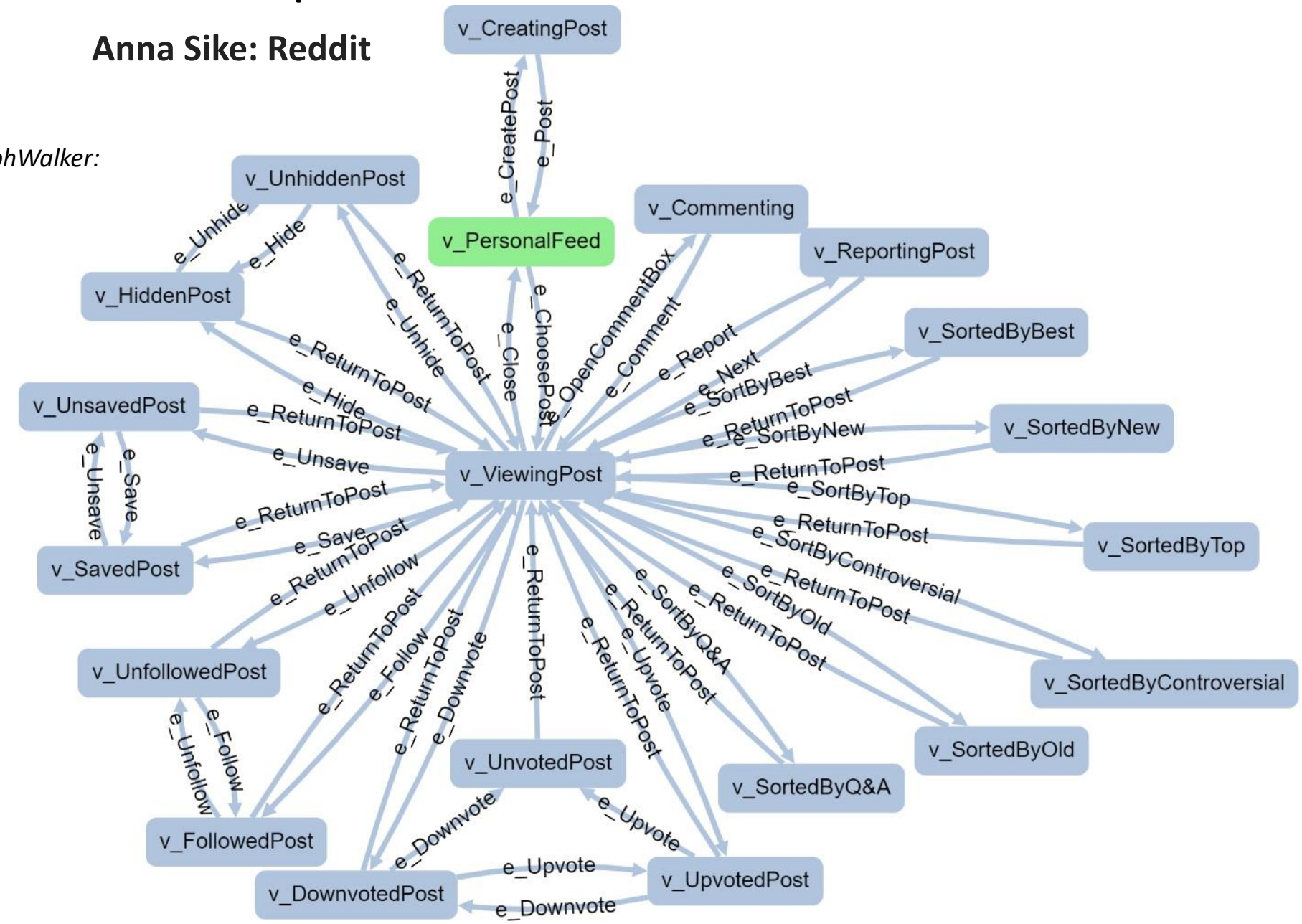
HW ToC

Appendix D: Example homework XXXVIII. 3/2

Fall 2023/2024

Anna Sike: Reddit

FSM model in GraphWalker:



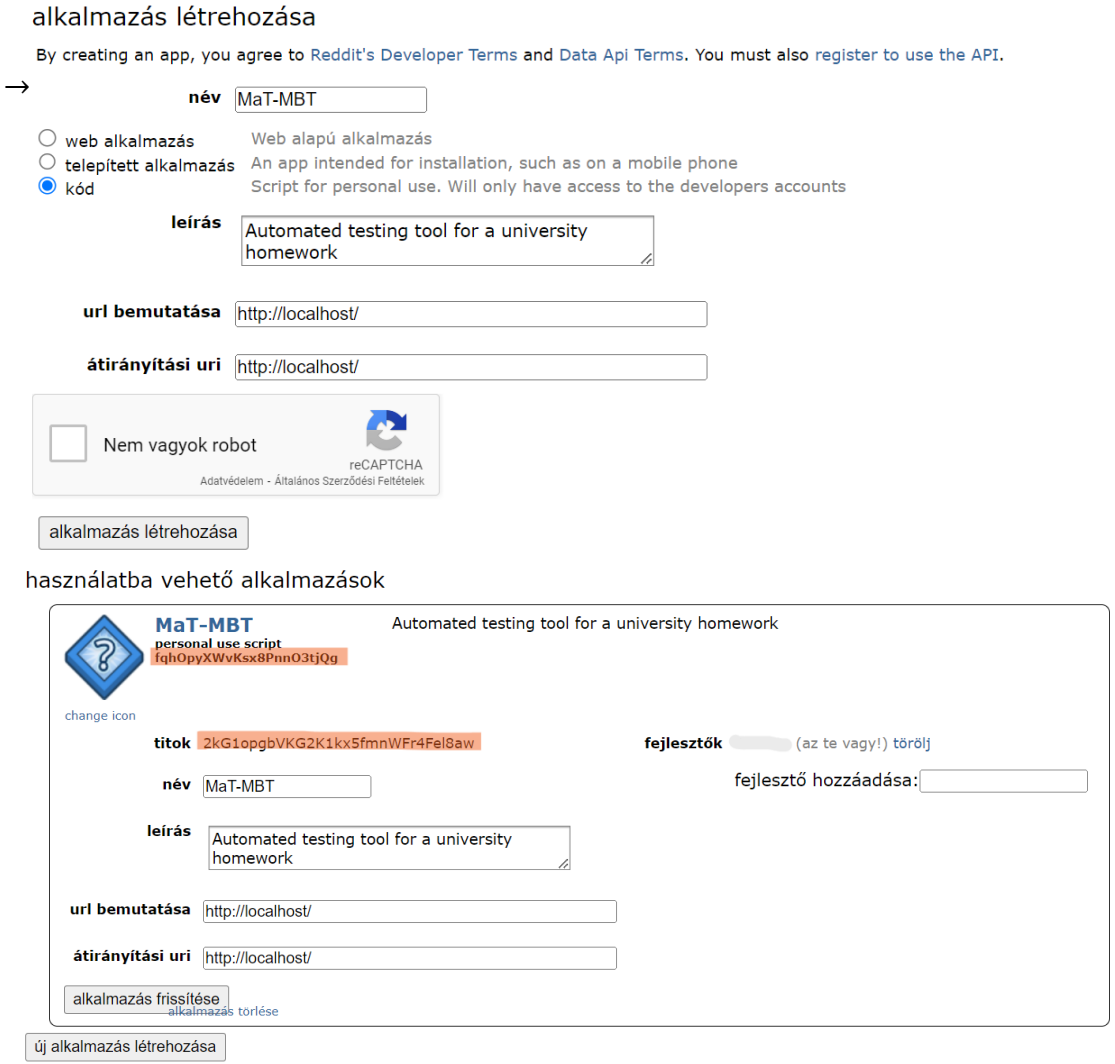
HW ToC

Appendix D: Example homework XXXVIII. 3/3

Fall 2023/2024

Anna Sike: Reddit

Creating Reddit app →



Adaptation code example for connection:

```python
from datetime import datetime
import json
import requests

# Reddit API credentials
CLIENT_ID = 'fqhOpyXWvKsx8PnnO3tjQg'
CLIENT_SECRET = '2kG1opgbVKG2K1kx5fmnWFr4Fel8aw'
REDDIT_USERNAME = '[redacted]'
REDDIT_PASSWORD = '[redacted]'
USER_AGENT = 'MaT-MBT/1.0 by [redacted]'

# Reddit API functions
BASE_URL = 'https://oauth.reddit.com'

# Store the access token globally for later use
ACCESS_TOKEN = None
```

Test results (ATS):

```
[INFO]       State: Viewing Post
[SUCCESS]    Step 'choose_post' passed.
[INFO]       Executing transition: e_OpenCommentBox for input 'open_comment_box'
[INFO]       Transition: Comment
[INFO]       State: Commenting
[SUCCESS]    Step 'open_comment_box' passed.
-----------------------------------------------
[INFO]       Finished the test suite
[INFO]       Test Verdict: PASS
[INFO]       Total Steps: 230
[INFO]       Failed Steps: 0
[INFO]       Cleaning up...
[SUCCESS]    Cleanup complete.
-----------------------------------------------
[INFO]       Test Run Time: 0:02:05.108707
```

HW ToC

Appendix D: Example homework XXXIX. 3/1

Fall 2023/2024

Barnabás Pőcze: SOCKS5 server

- SUT (System Under Test): SOCKS5 proxy server
  - Tested functionalities according to RFC1928:
    - Handshake
    - Authentication methods: No authentication (0x00), Username/password (0x02)
    - Commands: CONNECT, BIND, UDP_ASSOC
  - Limitations of the model:
    - Tests only a single client in isolation
    - Only two authentication methods are considered
    - BIND and UDP_ASSOC commands are only tested with wildcard IPv4/6 addresses
    - Checks a subset of potential messages
- Testing:
  - Model has been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
    - Using API

```
The SOCKS request is formed as follows:

     +----+-----+-------+------+----------+----------+
     |VER | CMD |  RSV  | ATYP | DST.ADDR | DST.PORT |
     +----+-----+-------+------+----------+----------+
     | 1  |  1  | X'00' |  1   | Variable |    2     |
     +----+-----+-------+------+----------+----------+

  Where:

       o  VER    protocol version: X'05'
       o  CMD
          o  CONNECT X'01'
          o  BIND X'02'
          o  UDP ASSOCIATE X'03'
       o  RSV    RESERVED
       o  ATYP   address type of following address
          o  IP V4 address: X'01'
          o  DOMAINNAME: X'03'
          o  IP V6 address: X'04'
       o  DST.ADDR       desired destination address
       o  DST.PORT desired destination port in network octet
          order
```

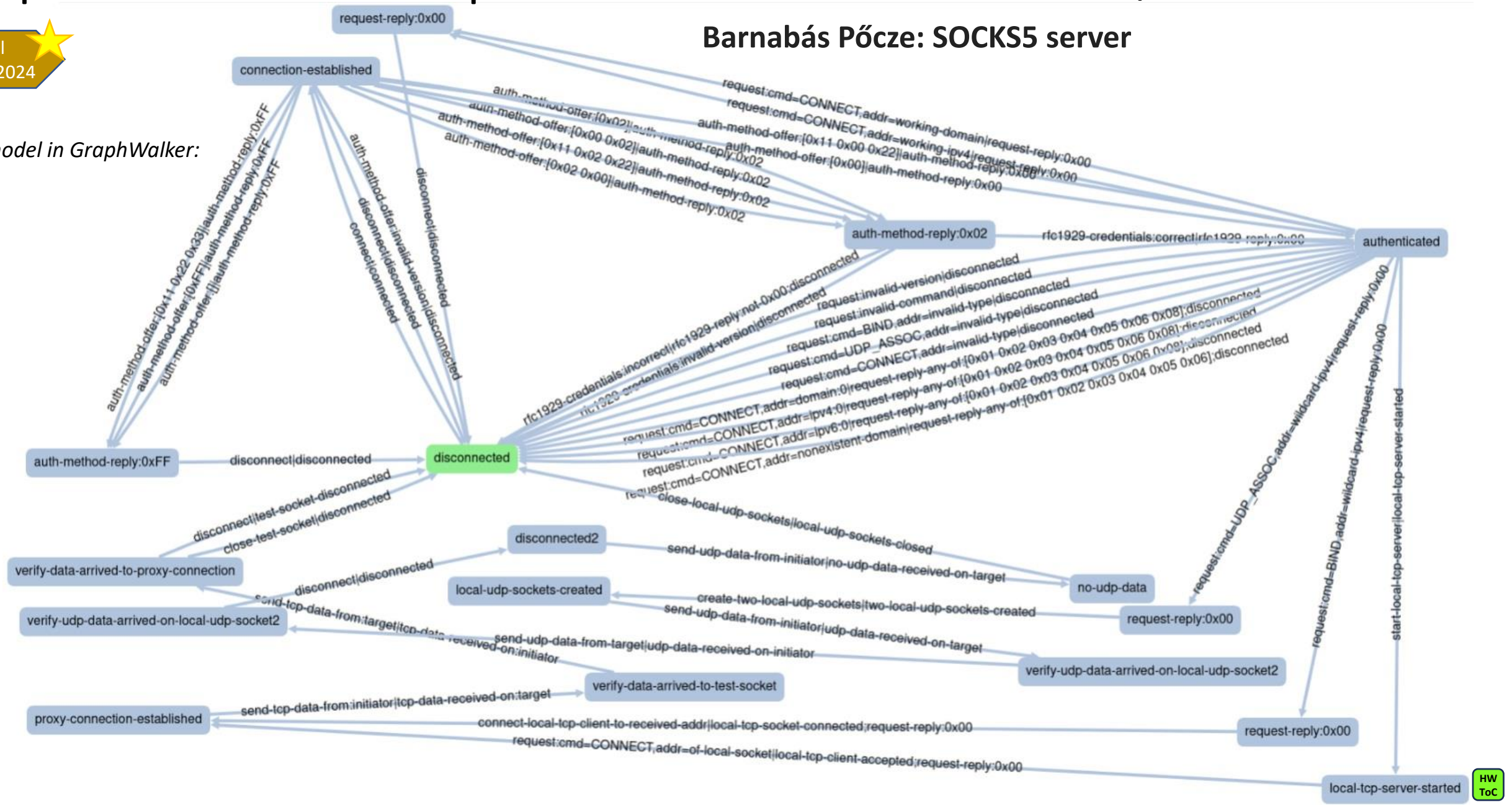
Format of the SOCKS request according to RFC 1928

HW ToC

Appendix D: Example homework XXXIX. 3/2

Fall 2023/2024

FSM model in GraphWalker:



Appendix D: Example homework XXXIX. 3/3

Fall 2023/2024

Barnabás Pőcze: SOCKS5 server

Adaptation code examples:

```
//The loop driving the testing process simply feeds all the inputs of the current step in order, then validates that all expected outputs are satisfied:
s: SocksTester = # ...
for (idx, step) in enumerate(steps):
 (inputs, outputs) = step
 for i in inputs:
  await s.feed(i)
 for o in outputs:
  await s.ensure(o)

//At the beginning of the connections, the client must send a list of the supported authentication methods:
case "rfc1929-credentials":
 match x.args:
  case "correct":
   data = SocksBuilder.rfc1929(1, "socks-test-user", "socks-test-password")
  case "incorrect":
   data = SocksBuilder.rfc1929(1, "wrong", "bad")
  case "invalid-version":
   data = SocksBuilder.rfc1929(99, "x", "x")
  case _:
   raise ValueError(f"invalid argument: {x.args}")

//For creating the byte strings, Python's struct module is used:
class SocksBuilder:
 # ...
 @staticmethod
 def rfc1929(version: int, username: str, password: str):
  return struct.pack("!BB", version, len(username)) + username.encode() \
   + struct.pack("!B", len(password)) + password.encode()

//Extracting information from the received messages:
port = struct.unpack("!H", await self.proxy_conn.read_exactly(2))[0]

//Send an actual SOCKS5 request:
case "request":
 match x.args:
  case "invalid-version":
   await self.send_request(99, 99, SocksCustomAddr(99, b""))
  case "invalid-command":
   await self.send_request(5, 99, SocksCustomAddr(99, b""))
  case {"cmd": str(command), "addr": str(addr)}:
   await self.send_request(5, parse_socks_cmd(command), parse_socks_addr(addr))
  case _:
   raise ValueError(f"invalid argument: {x.args}")

//Helper function responsible for returning a usable representation of the address from the parameter string:
def parse_socks_addr(s):
match s:
 case "invalid-type": return SocksCustomAddr(99, b"abcdef")
 case "domain:0": return SocksDomainAddr("example.com", 0)
 case "working-domain": return SocksDomainAddr("example.com", 80)
 case "nonexistent-domain": return SocksDomainAddr("does.not.exists.asd", 443)
 # ...

//Validate a reply to a request:
case "request-reply":
 reply = await asyncio.wait_for(self.read_reply(), TIMEOUT)
 assert reply.reply == x.args, \
  f"got reply code {reply.reply}, expected {x.args}"
case "request-reply-any-of":
 reply = await asyncio.wait_for(self.read_reply(), TIMEOUT)
 assert reply.reply in x.args, \
  f"got reply code {reply.reply}, expected any of {x.args}"

//An example for an address that is represented using Python dataclasses:
@dataclasses.dataclass
class SocksIPv4Addr:
 addr: ipaddress.IPv4Address
 port: int
# ...
SocksAddr = SocksIPv4Addr | SocksIPv6Addr | SocksDomainAddr | SocksCustomAddr
```

Test execution:

```
<< feed [12017/31926] Command(name='connect', args=None)
!! ensure [12017/31926] Command(name='connected', args=None)
<< feed [12018/31926] Command(name='auth-method-offer', args=[2])
<< send b'\x05\x01\x02'
!! ensure [12018/31926] Command(name='auth-method-reply', args=2)
>> receive b'\x05\x02'
<< feed [12019/31926] Command(name='rfc1929-credentials', args='correct')
<< send b'\x01\x0fsocks-test-user\x13socks-test-password'
!! ensure [12019/31926] Command(name='rfc1929-reply', args=0)
>> receive b'\x01\x00'
<< feed [12020/31926] Command(name='request', args={'cmd': 'CONNECT', 'addr': 'working-ipv4'})
<< send b'\x05\x01\x00\x01\x01\x01\x01\x01\x01\x005'
!! ensure [12020/31926] Command(name='request-reply', args=0)
>> receive b'\x05\x00\x00\x01'
>> receive b'\xc0\xa8\x1f\x81'
>> receive b'\x98\xa6'
<< feed [12021/31926] Command(name='disconnect', args=None)
!! ensure [12021/31926] Command(name='disconnected', args=None)
<< feed [12022/31926] Command(name='<reset>', args=None)
!! ensure [12022/31926] Command(name='', args=None)
```

Dante proxy server printing debug messages during test execution:

```
Nov 27 06:54:22 (1701068062.100646) sockd[1045]: debug: recv_clientrequest(): fd 8, client 127.0.0.1.40990, state->complete: 0, read so far: 3
Nov 27 06:54:22 (1701068062.100668) sockd[1045]: debug: socks_recvfrom(): fd 8, len 1, flags 0
Nov 27 06:54:22 (1701068062.100679) sockd[1045]: debug: socks_getfrombuffer(): fd = 8, get up to 1 decoded byte from read buffer which currently has 0 decoded, 0 e
ncoded.  Flags = 0
Nov 27 06:54:22 (1701068062.100690) sockd[1045]: debug: socks_freeinbuffer(): fd 8, which 0, free: 288
Nov 27 06:54:22 (1701068062.100701) sockd[1045]: debug: socks_freeinbuffer(): fd 8, which 0, free: 288
Nov 27 06:54:22 (1701068062.100724) sockd[1045]: debug: socks_recvfrom(): read 18/288 bytes from socket, fd 8, errno = 0 (no system error)
Nov 27 06:54:22 (1701068062.100735) sockd[1045]: debug: socks_freeinbuffer(): fd 8, which 0, free: 288
Nov 27 06:54:22 (1701068062.100746) sockd[1045]: debug: socks_addtobuffer(): fd = 8, add 17 unencoded bytes to read buffer which currently has 0 unencoded, 0 encod
ed.  Last bytes to add: 0x0, 0x50.  Data will be added after byte 0x0 which is at offset -1
Nov 27 06:54:22 (1701068062.100761) sockd[1045]: debug: socks_recvfrom(): fd 8, len 1, flags 0
Nov 27 06:54:22 (1701068062.100771) sockd[1045]: debug: socks_getfrombuffer(): fd = 8, get up to 1 decoded byte from read buffer which currently has 17 decoded, 0
encoded.  Flags = 0
Nov 27 06:54:22 (1701068062.100783) sockd[1045]: debug: socks_recvfrom(): read 1 byte from buf, 16 bytes left in buf
Nov 27 06:54:22 (1701068062.100792) sockd[1045]: debug: socks_recvfrom(): fd 8, len 1, flags 0
Nov 27 06:54:22 (1701068062.100803) sockd[1045]: debug: socks_getfrombuffer(): fd = 8, get up to 1 decoded byte from read buffer which currently has 16 decoded, 0
encoded.  Flags = 0
Nov 27 06:54:22 (1701068062.100813) sockd[1045]: debug: socks_recvfrom(): read 1 byte from buf, 15 bytes left in buf
Nov 27 06:54:22 (1701068062.100824) sockd[1045]: debug: socks_recvfrom(): fd 8, len 1, flags 0
Nov 27 06:54:22 (1701068062.100832) sockd[1045]: debug: socks_getfrombuffer(): fd = 8, get up to 1 decoded byte from read buffer which currently has 15 decoded, 0
encoded.  Flags = 0
```

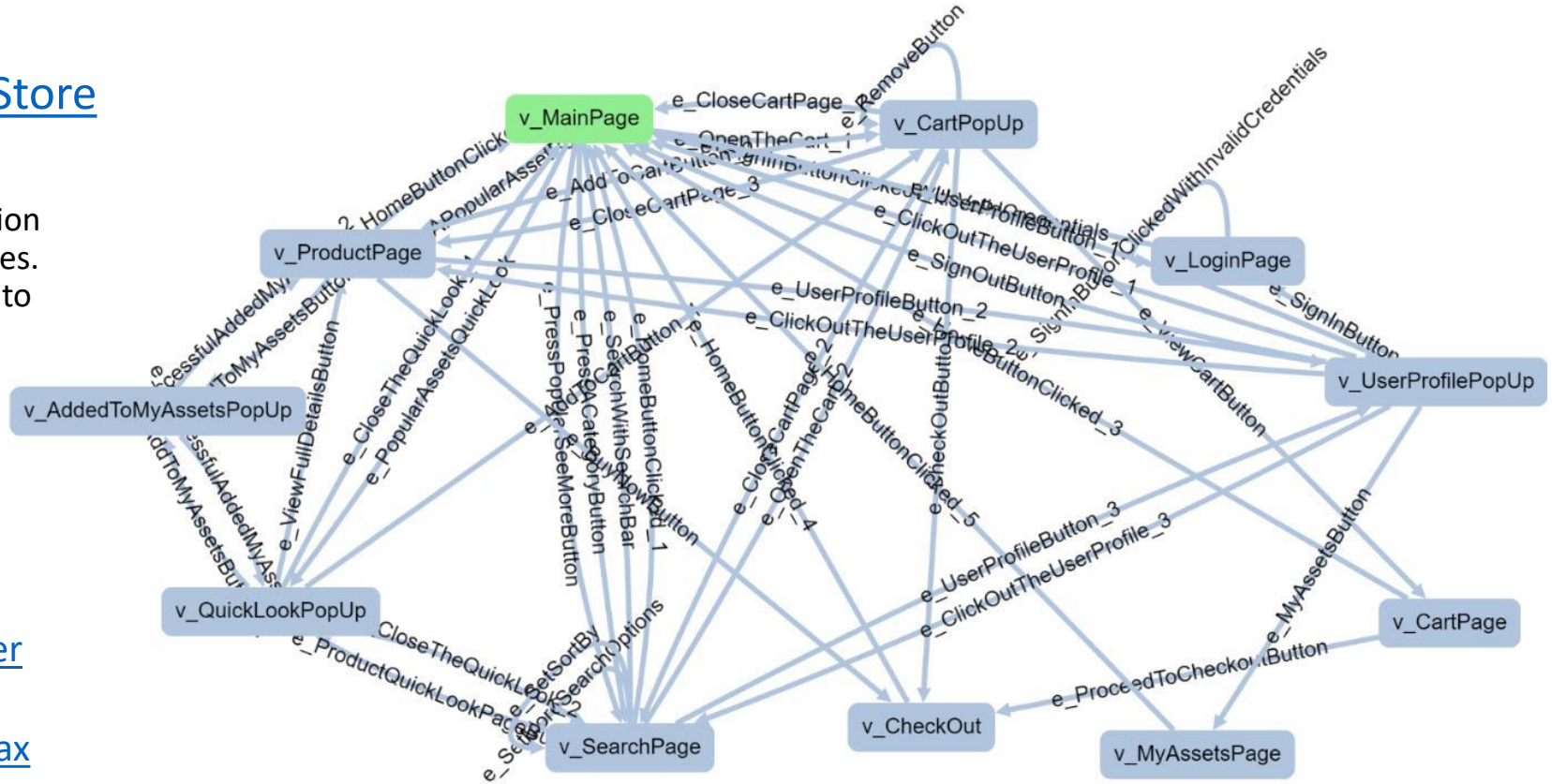
HW ToC

Appendix D: Example homework XXXX. 2/1

Spring 2023/2024

Máté Fekete: Unity Asset Store

- SUT (System Under Test): Unity Asset Store
  - Tested functionalities:
    - User Authentication: Login and logout
    - Asset Browsing: Search, filter, and navigation capabilities within different asset categories.
    - Asset Operations: Viewing details, adding to cart, and purchasing assets.
    - User Profile Management
  - Limitations of the model:
    - Excludes certain complex integrations and backend functionalities
- Testing:
  - Model has been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - GUI tests are written in TypeScript, using Selenium



EFSM model in GraphWalker

HW ToC

Appendix D: Example homework XXXX. 2/2

Spring 2023/2024

Máté Fekete: Unity Asset Store

Adaptation code examples:

```
public async findElementsCountByXpath(selector: string): Promise<number> {
        let locator: Locator = By.xpath(selector);
        try {
            return await this.driver.wait(until.elementsLocated(locator), waitTime / 2).then(
                elements => elements.length
            );
        }
        catch (error) {
            return 0;
        }
    }

    public async findFirstVisibleElement(xpath1: string, xpath2: string): Promise<WebElement | null> {
        const [element1, element2] = await Promise.all([
            this.findElementByXpath(xpath1, config.smallerWaitTimeInMilliseconds).catch(() => null),
            this.findElementByXpath(xpath2, config.smallerWaitTimeInMilliseconds).catch(() => null)
        ]);

        if (element1) {
            return element1;
        } else if (element2) {
            return element2;
        }
        throw null;
    }

    public async mouseMove(webElement: WebComponent) {
        let actions: Actions = this.driver.actions({ async: true });
        await actions.move({ origin: await webElement.getWebElement() }).perform();
    }

    public async clearCookies(): Promise<void> {
        await this.driver.manage().deleteAllCookies();
    }

    public async close(): Promise<void> {
        await this.driver.quit();
    }
```

Configuration file for setup of the Selenium testing environment for the Unity Asset Store:

```
{
    "testFileLocation": "./MaT_Unity_Asset_Store-Random-transition-50-test_suite.json",
    "headlessMode": false,
    "url": "https://assetstore.unity.com/",
    "userEmail": "barkitagno@gufum.com",
    "userPassword": "Alma1234",
    "delayBetweenTestCases": 150,
    "waitTimeInMilliseconds": 5000,
    "smallerWaitTimeInMilliseconds": 1000,
    "searchQuery": "characters"
}
```

Console Output of a text execution:

```
C:\Users\Máté\Downloads\Unity_Asset_Store_Selenium>ts-node testRunner.ts
--------------------------------------------------------------------------------
Test name: MaT_Unity_Asset_Store
Test method: Random
Test ID: e9fa51ce-fbff-4461-ac07-1c4f4c26403c
--------------------------------------------------------------------------------
Start testing...
[1/10] -> Input: click userPictureButton - Output: open userProfilePopUp -> Successful!
[2/10] -> Input: click signInButton - Output: open loginPage -> Successful!
[3/10] -> Input: add good userName and userPassword - Output: succesful login -> Successful!
[4/10] -> Input: click cartButton - Output: open cartPopUp -> Successful!
[5/10] -> Input: click outTheCartPopUp - Output: open mainPage -> Successful!
[6/10] -> Input: add searchText - Output: open searchPage -> Successful!
[7/10] -> Input: click userPictureButton - Output: open userProfilePopUp -> Successful!
[8/10] -> Input: click outTheUserProfile - Output: open searchPage -> Successful!
[9/10] -> Input: click userPictureButton - Output: open userProfilePopUp -> Successful!
[10/10] -> Input: click myAssetsButton - Output: open myAssetsPage -> Successful!
Testing completed...
--------------------------------------------------------------------------------
Testing has been completed. Here's the summary:
- Test running time: 34.121s
- Number of successful tests: 10 (100.00%)
- Number of failed tests: 0 (0.00%)
- Total tests run: 10
--------------------------------------------------------------------------------

C:\Users\Máté\Downloads\Unity_Asset_Store_Selenium>
```

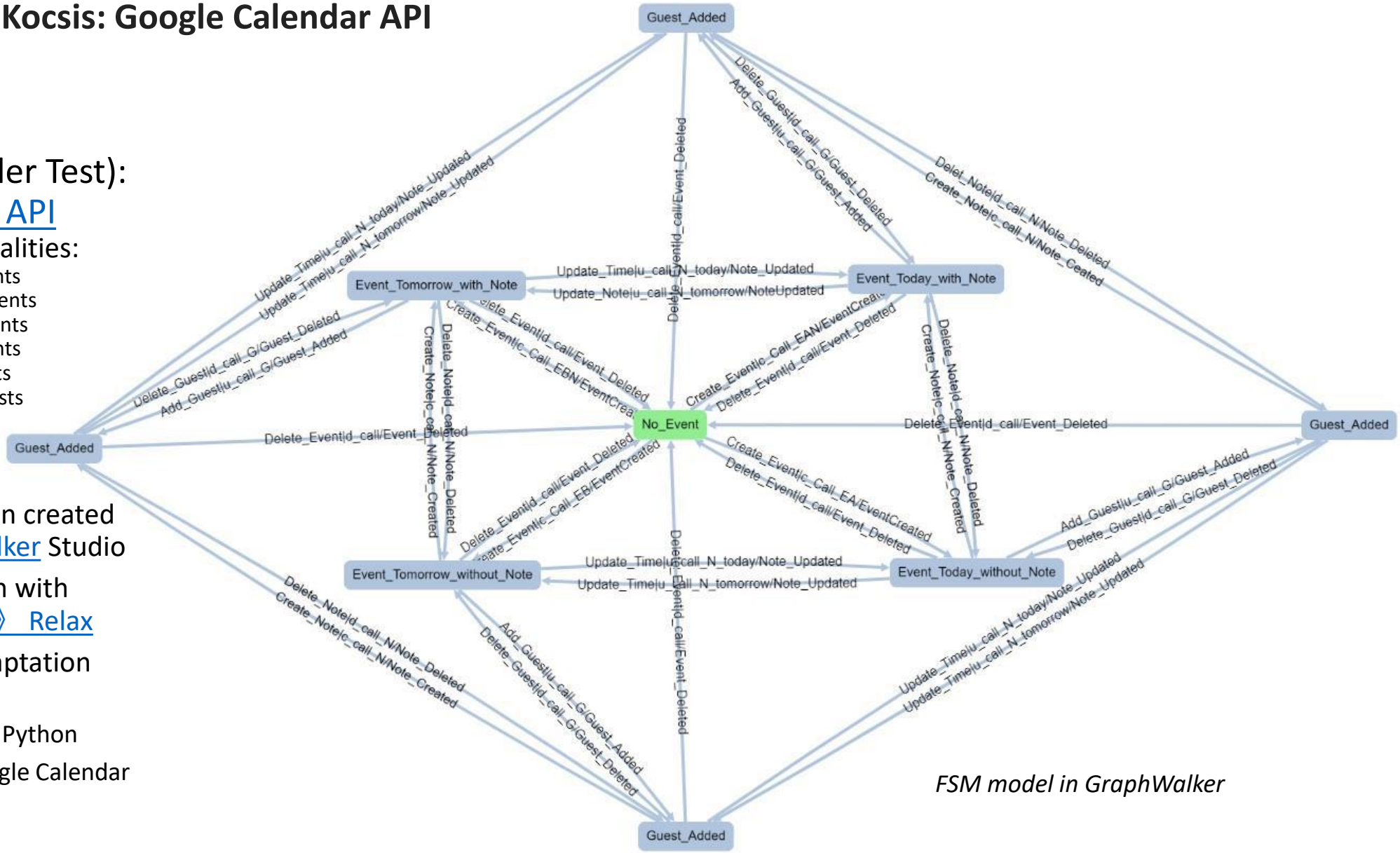
HW ToC

Appendix D: Example homework XXXXI.

Spring 2023/2024

Máté Kocsis: Google Calendar API

- SUT (System Under Test): Google Calendar API
  - Tested functionalities:
    - Creating Events
    - Retrieving Events
    - Updating Events
    - Deleting Events
    - Adding Guests
    - Deleting Guests
- Testing:
  - Model has been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
    - Using Google Calendar API



FSM model in GraphWalker

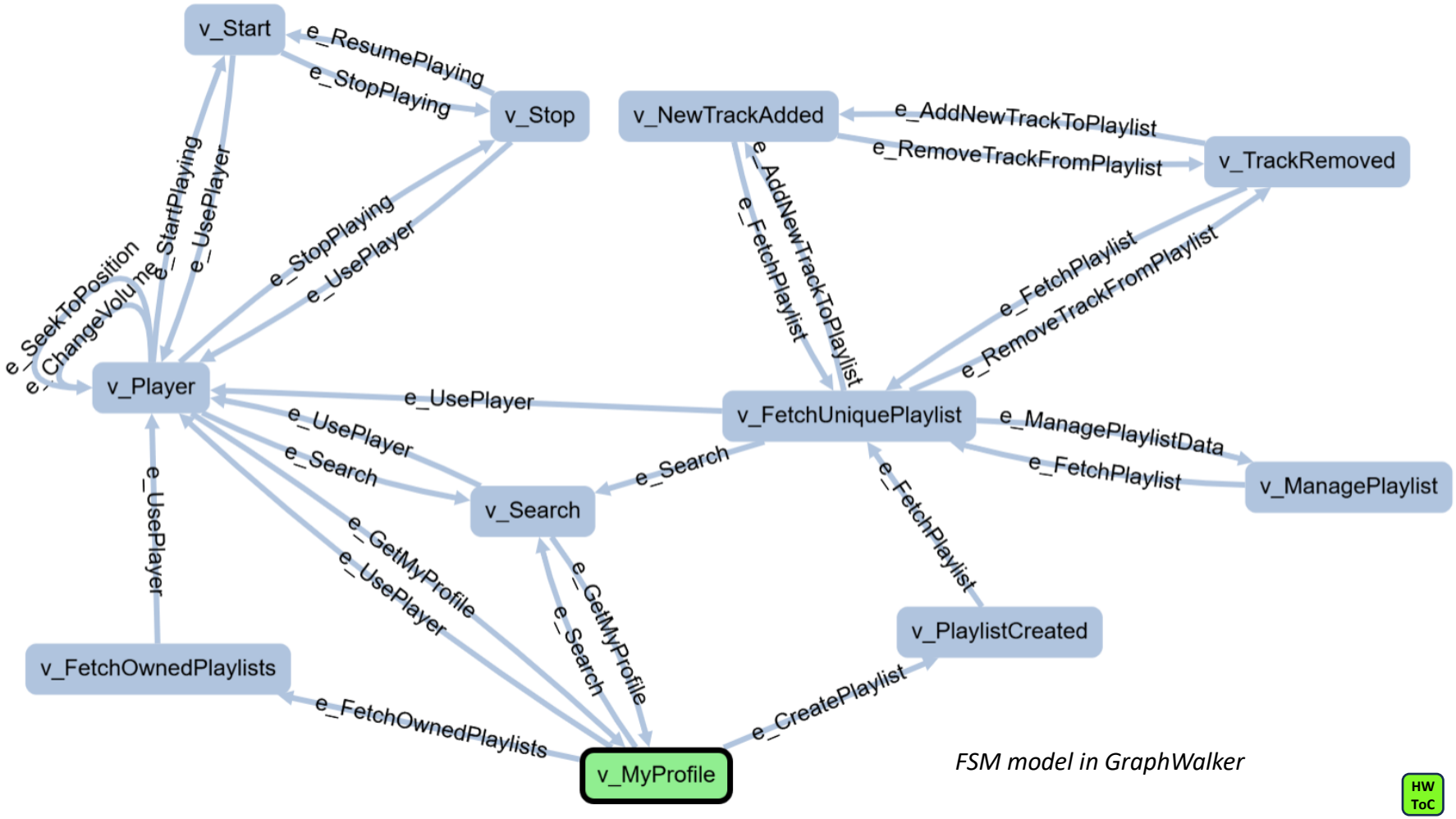
HW ToC

Appendix D: Example homework XXXXII. 3/1

Fall 2024/2025

Ábel Mandl: Spotify

- SUT (System Under Test): Spotify digital music service's Web API
  - Tested functionalities:
    - User Profile Retrieval
    - Playlist Management, including:
      - Fetching Playlists
      - Creating New Playlists
      - Modifying Playlist Details
      - Adding and Removing Tracks
      - Accessing Playlist Content
    - Track Search
    - Player Controls, such as:
      - Starting and Stopping Playback
      - Adjusting Volume
      - Seeking to Specific Positions
- Testing:
  - Model has been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Implemented in Python
    - Using Spotify Web API (Node.js)



FSM model in GraphWalker

HW ToC

Appendix D: Example homework XXXXII. 3/2

Fall 2024/2025

Ábel Mandl: Spotify

An adaptation code example:

```python
# Function to check the response body based on output keys
def check_response_body(response, output_key):
    try:
        response_body = response.json()

        if output_key == "CREATE_PLAYLIST_O":
            return "message" in response_body and "playlist" in response_body and response_body["message"] == "Playlist created"
        elif output_key == "FETCH_PLAYLIST_O":
            if "playlistId" in response_body and "tracks" in response_body:
                tracks = response_body["tracks"]
                # Check if tracks is a list and each track has the required fields
                return isinstance(tracks, list) and all(
                    "trackName" in track and "artists" in track and "album" in track and "durationMs" in track
                    for track in tracks
                )
            return False
        elif output_key == "REMOVE_TRACK_FROM_PLAYLIST_O":
            return "message" in response_body and response_body["message"] == "Tracks removed from playlist"
        elif output_key == "ADD_NEW_TRACK_TO_PLAYLIST_O":
            return "message" in response_body and response_body["message"] == "Tracks added to playlist"
        elif output_key == "USE_PLAYER_O":
            return "device" in response_body and isinstance(response_body["device"], dict)
        elif output_key == "GET_MY_PROFILE_O":
            return (
                "country" in response_body
                and "display_name" in response_body
                and "email" in response_body
                and "uri" in response_body
            )
        elif output_key == "FETCH_OWN_PLAYLISTS_O":
            return "message" in response_body and "playlists" in response_body and response_body["message"] == "Owned Playlists fetched successfully"
        elif output_key == "MANAGE_PLAYLIST_DATA_O":
            return "message" in response_body and response_body["message"] == "Playlist details updated successfully"
        elif output_key == "SEARCH_PERFORM_O":
            # You might want to add more specific checks here based on your API's response
            return "tracks" in response_body and "items" in response_body["tracks"]
        elif output_key == "SEEK_TO_POSITION_O":
            return "message" in response_body and response_body["message"] == "Seeked to position"
        elif output_key == "CHANGE_VOLUME_O":
            return "message" in response_body and response_body["message"] == "Volume changed"
        elif output_key == "START_PLAYING_O":
            return "message" in response_body and response_body["message"] == "Playback started or resumed"
        elif output_key == "STOP_PLAYING_O":
            return "message" in response_body and response_body["message"] == "Playback paused"
        else:
            return False  # Unknown output key
    except json.JSONDecodeError:
        print("Error: Invalid JSON response")
        return False
```

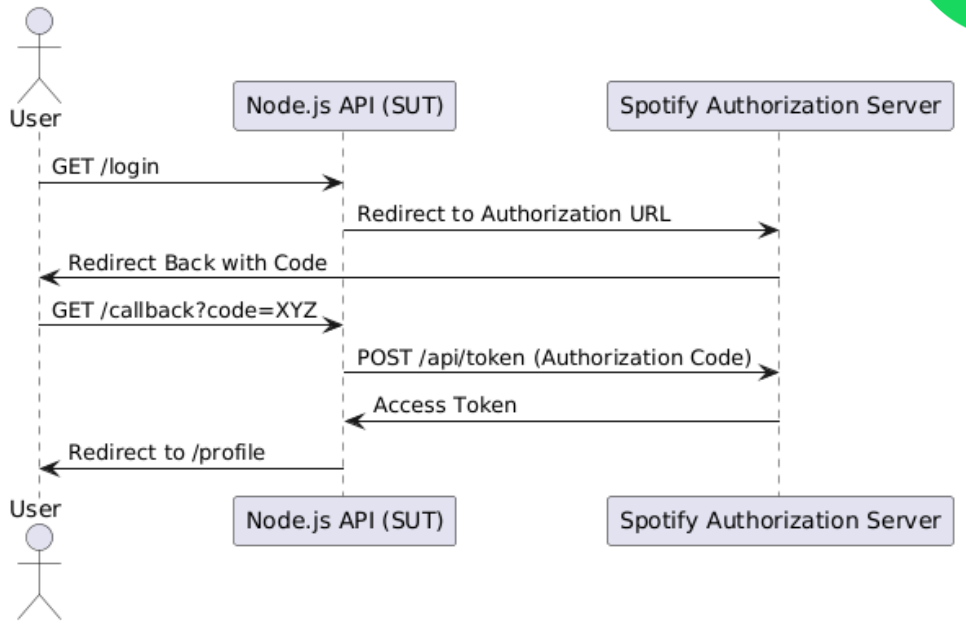
HW ToC

Appendix D: Example homework XXXXII. 3/3

Fall 2024/2025

Ábel Mandl: Spotify

Authentication and Authorization Flow:



Results of text execution:

Test Strategy	Total Tests	Successful tests	Failed tests	Test execution time (ms)
All-Transition-State	123	121	2	165651
1-Switch Coverage	194	189	5	257084
Random (100% transition cov.)	149	141	8	206322
Transition Tour	33	33	0	46380

Output of the script used for text execution:

```
✔ Test CREATE_PLAYLIST_I -> CREATE_PLAYLIST_O: was successful. (OK)
-> Captured sample_playlist_id: 4CxHOFyV8vNavUfp3qE4xA
✔ Test FETCH_PLAYLIST_I -> FETCH_PLAYLIST_O: was successful. (OK)
✔ Test REMOVE_TRACK_FROM_PLAYLIST_I -> REMOVE_TRACK_FROM_PLAYLIST_O: was successful. (OK)
✔ Test ADD_NEW_TRACK_TO_PLAYLIST_I -> ADD_NEW_TRACK_TO_PLAYLIST_O: was successful. (OK)
✔ Test REMOVE_TRACK_FROM_PLAYLIST_I -> REMOVE_TRACK_FROM_PLAYLIST_O: was successful. (OK)
✔ Test FETCH_PLAYLIST_I -> FETCH_PLAYLIST_O: was successful. (OK)
✔ Test ADD_NEW_TRACK_TO_PLAYLIST_I -> ADD_NEW_TRACK_TO_PLAYLIST_O: was successful. (OK)
✔ Test FETCH_PLAYLIST_I -> FETCH_PLAYLIST_O: was successful. (OK)
✔ Test USE_PLAYER_I -> USE_PLAYER_O: was successful. (OK)
✔ Test GET_MY_PROFILE_I -> GET_MY_PROFILE_O: was successful. (OK)
✔ Test FETCH_OWN_PLAYLISTS_I -> FETCH_OWN_PLAYLISTS_O: was successful. (OK)
✔ Test USE_PLAYER_I -> USE_PLAYER_O: was successful. (OK)
✔ Test GET_MY_PROFILE_I -> GET_MY_PROFILE_O: was successful. (OK)
✔ Test CREATE_PLAYLIST_I -> CREATE_PLAYLIST_O: was successful. (OK)
-> Captured sample_playlist_id: 3VaEXgUcUigxfIIPPs1qFb
✔ Test FETCH_PLAYLIST_I -> FETCH_PLAYLIST_O: was successful. (OK)
✔ Test MANAGE_PLAYLIST_DATA_I -> MANAGE_PLAYLIST_DATA_O: was successful. (OK)
✔ Test FETCH_PLAYLIST_I -> FETCH_PLAYLIST_O: was successful. (OK)
✔ Test SEARCH_PERFORM_I -> SEARCH_PERFORM_O: was successful. (OK)
-> Captured track URIs from search: ['spotify:track:6DCZcSspjsKoFjzjrWoCdn', 'spotify:trac
fy:track:2R0QwDcfbDfJkAzgk6eC6y', 'spotify:track:2wExHkTpLE370hvFrEdlnc']
✔ Test GET_MY_PROFILE_I -> GET_MY_PROFILE_O: was successful. (OK)
✔ Test USE_PLAYER_I -> USE_PLAYER_O: was successful. (OK)
✔ Test SEEK_TO_POSITION_I -> SEEK_TO_POSITION_O: was successful. (OK)
✔ Test CHANGE_VOLUME_I -> CHANGE_VOLUME_O: was successful. (OK)
✔ Test SEARCH_PERFORM_I -> SEARCH_PERFORM_O: was successful. (OK)
-> Captured track URIs from search: ['spotify:track:6DCZcSspjsKoFjzjrWoCdn', 'spotify:trac
fy:track:2R0QwDcfbDfJkAzgk6eC6y', 'spotify:track:2wExHkTpLE370hvFrEdlnc', 'spotify:track:3x
✔ Test USE_PLAYER_I -> USE_PLAYER_O: was successful. (OK)
✔ Test START_PLAYING_I -> START_PLAYING_O: was successful. (OK)
✔ Test STOP_PLAYING_I -> STOP_PLAYING_O: was successful. (OK)
✔ Test START_PLAYING_I -> START_PLAYING_O: was successful. (OK)
✔ Test USE_PLAYER_I -> USE_PLAYER_O: was successful. (OK)
✔ Test STOP_PLAYING_I -> STOP_PLAYING_O: was successful. (OK)
✔ Test USE_PLAYER_I -> USE_PLAYER_O: was successful. (OK)
✔ Test GET_MY_PROFILE_I -> GET_MY_PROFILE_O: was successful. (OK)
✔ Test SEARCH_PERFORM_I -> SEARCH_PERFORM_O: was successful. (OK)
-> Captured track URIs from search: ['spotify:track:6DCZcSspjsKoFjzjrWoCdn', 'spotify:trac
fy:track:2R0QwDcfbDfJkAzgk6eC6y', 'spotify:track:2wExHkTpLE370hvFrEdlnc', 'spotify:track:3x
track:3EhZlrrSF5MTwa68SNTwHh', 'spotify:track:2Ag1Lw6wZPTst7GtPyTcAP', 'spotify:track:6ZIcU
✔ Test GET_MY_PROFILE_I -> GET_MY_PROFILE_O: was successful. (OK)

Test Summary:
Total Tests: 33
Successful Tests: 33 ✔
Failed Tests: 0 ❌
Execution Time: 46244 ms
```

HW ToC

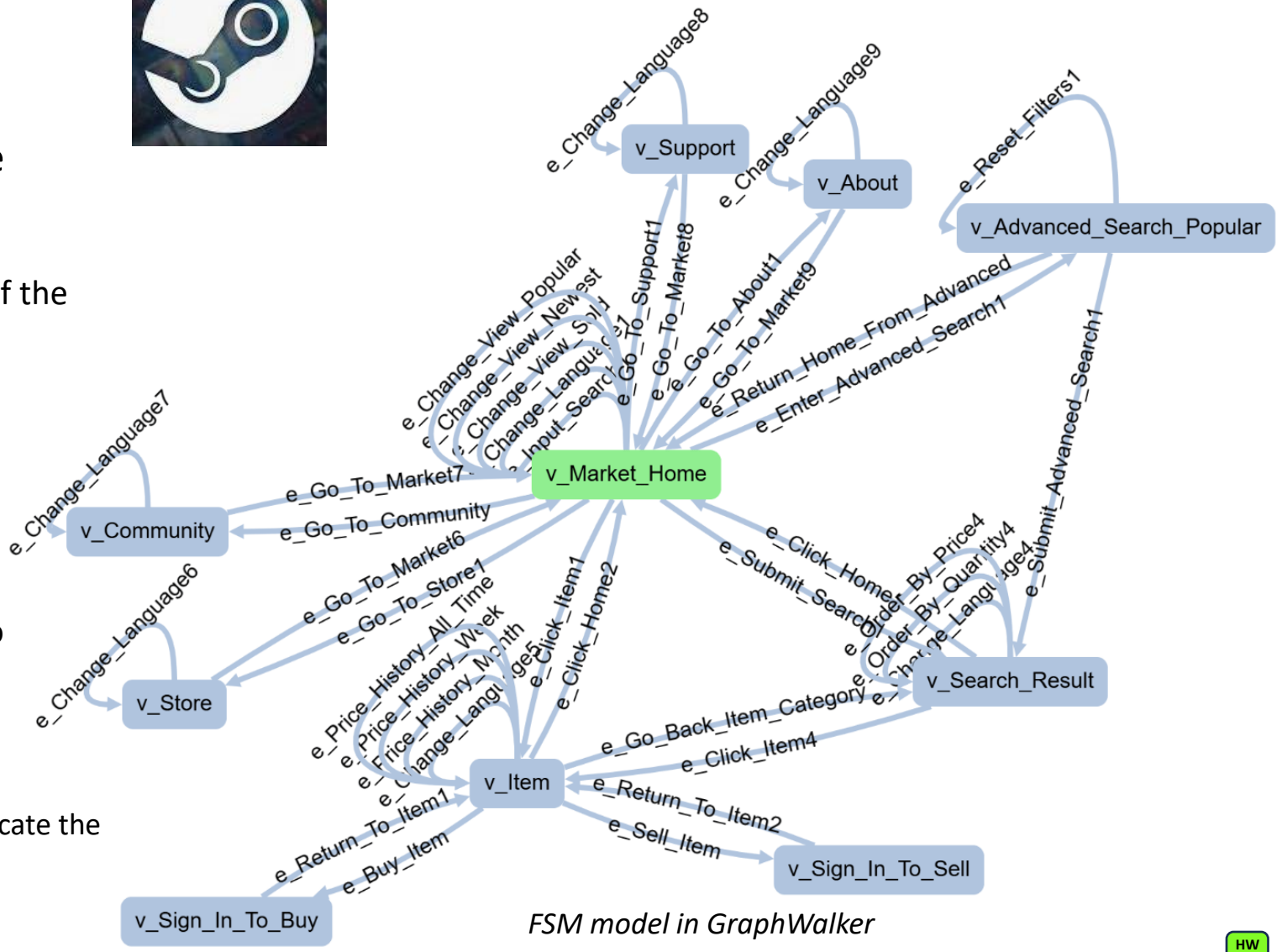
Appendix D: Example homework XXXXIII. 2/1

Fall 2024/2025

Tamás Márton: Steam Marketplace

- SUT (System Under Test): Steam online game distributing platform
  - Tested functionalities:
    - Reach and navigate through the homepage of the marketplace
    - Use the search bar
    - Use the advanced search
    - Check results
    - Navigate between the different pages

- Testing:
  - Model has been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
    - Using Selenium for GUI testing (XPath is used to locate the elements on the webpage)



FSM model in GraphWalker

HW ToC

Appendix D: Example homework XXXXIII. 2/2

Fall 2024/2025

Tamás Márton: Steam Marketplace

Adaptation code examples (definitions of few transitions):

```
def e_Return_To_Item2():
 global driver
 exit  =  WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.XPATH, '''//*[@id="modalContentDismiss"]''')))
 exit.click()
 time.sleep(1)
 login_popup_inv = WebDriverWait(driver, 20).until(EC.presence_of_element_located((By.XPATH, '//*[@id="modalBG"]')))
 if not login_popup_inv.is_displayed():
  return "returned_to_item"
 return "fail"

def e_Reset_Filters1():
 global driver
 correct=True
 reset  =  WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.XPATH, '''//*[@id="market_advancedsearch_dialog"]/div[2]/a''')))
 reset.click()

 filterlist = driver.find_element(By.XPATH, '''//*[@id="market_advancedsearch_filters"]''')
 filters = filterlist.find_elements(By.TAG_NAME, "input")

 for filter in filters:
  input_type = filter.get_attribute("type")
  if input_type == 'checkbox':
   if filter.is_selected():
    correct=False
 time.sleep(1)
 if correct:
  return "filters_reset"
 return "fail"

def e_Input_Advanced_Search1():
 global driver
 search_param = "Berlin"
 time.sleep(1)
 advanced_search_box = driver.find_element(By.ID, "advancedSearchBox")
 advanced_search_box.clear()
 advanced_search_box.send_keys(search_param)
 time.sleep(1)
 if advanced_search_box.get_attribute("value")==search_param:
  return "input_keys_advanced"
 return "fail"

def e_Submit_Advanced_Search1():
 global driver
 submit =  WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.XPATH,
'''//*[@id="market_advancedsearch_dialog"]/div[2]/div/span''')))
 submit.click()
 time.sleep(5)
 if 'https://steamcommunity.com/market/search?q=' in driver.current_url:
  return "loaded_results"
 return "fail"
```

The dictionary which is used to connect the inputs and outputs of tests to the adaptation code:

```
dict = {
"return_home_from_advanced":  {"function": e_Return_Home_From_Advanced, "expected_output":"loaded_home" },
"set_filters_advanced_search_p":  {"function": e_Set_Filters1, "expected_output":"filters_set" },
"click_home_from_item":  {"function": e_Click_Home, "expected_output":"loaded_home" },
"click_home_from_results":  {"function": e_Click_Home, "expected_output":"loaded_home" },
"change_language_from_item":  {"function": e_Change_Language, "expected_output":"language_changed" },
"click_root_item_category":  {"function": e_Go_Back_Item_Category, "expected_output":"loaded_item_root_category" },
…
}
```

Test summary:

Test suite	Execution time	Successful tests	All tests	Verdict
Random Walk (100%)	1 hour 29 minutes	179	179	SUCCESSFUL
All-State	8 minutes	16	16	SUCCESSFUL
Transition-Transition	20 minutes	42	42	SUCCESSFUL
All-State-Transition (depth=0)	1 hour 9 minutes	145	145	SUCCESSFUL
N-switch coverage (n=1)	3 hour 1 minute	377	377	SUCCESSFUL
All-Transition-Transition	3 hour 46 minutes	517	517	SUCCESSFUL
N-switch coverage (n=2)	Not finished	760	3111	NOT COMPLETE

HW ToC

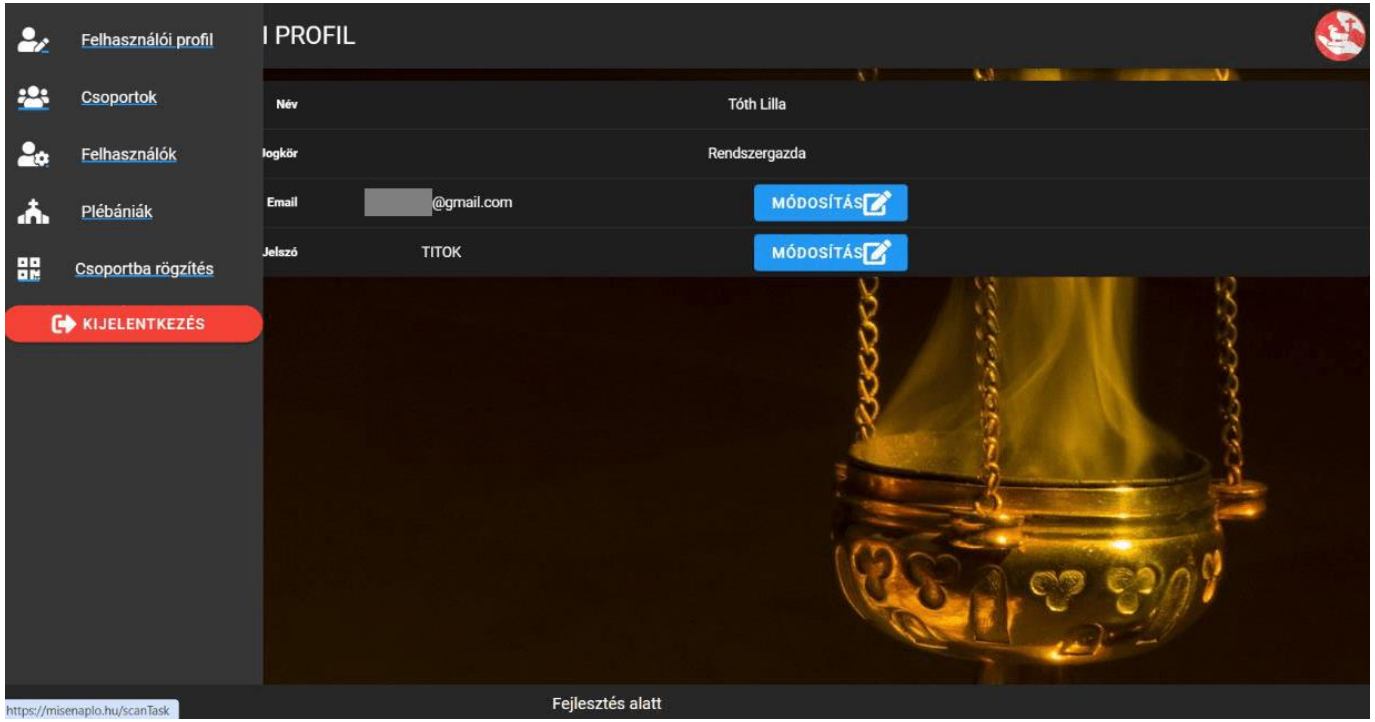
Appendix D: Example homework XXXXIV. 2/1

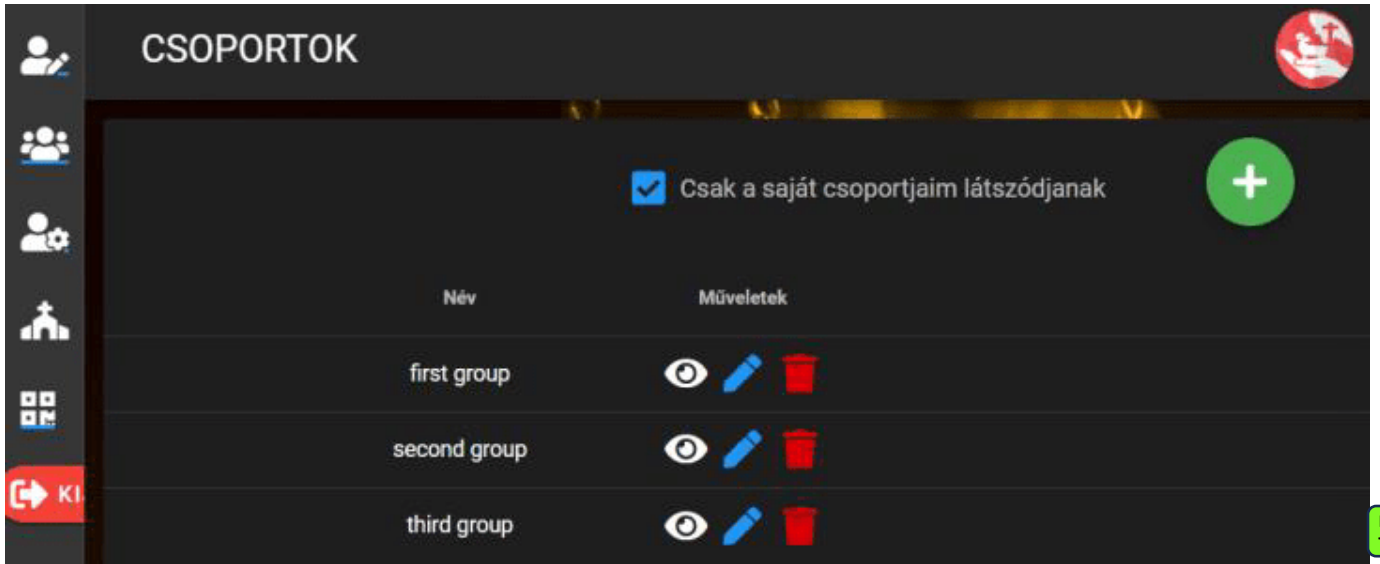
Fall 2024/2025

Lilla Tóth: MisEnapló

- SUT (System Under Test): MisEnapló web application
  - Tested functionalities:
    - Profile page:
      - Email address and password can be modified
    - Groups page:
      - Create, modify, and delete groups
    - Users page:
      - Add/remove users
    - Parishes page:
      - Add/remove parishes

- Testing:
  - Model has been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Javascript
    - Using Selenium with Chromedriver for GUI testing





HW ToC

Appendix D: Example homework XXXXIV. 2/2

Fall 2024/2025

Lilla Tóth: MisEnapló

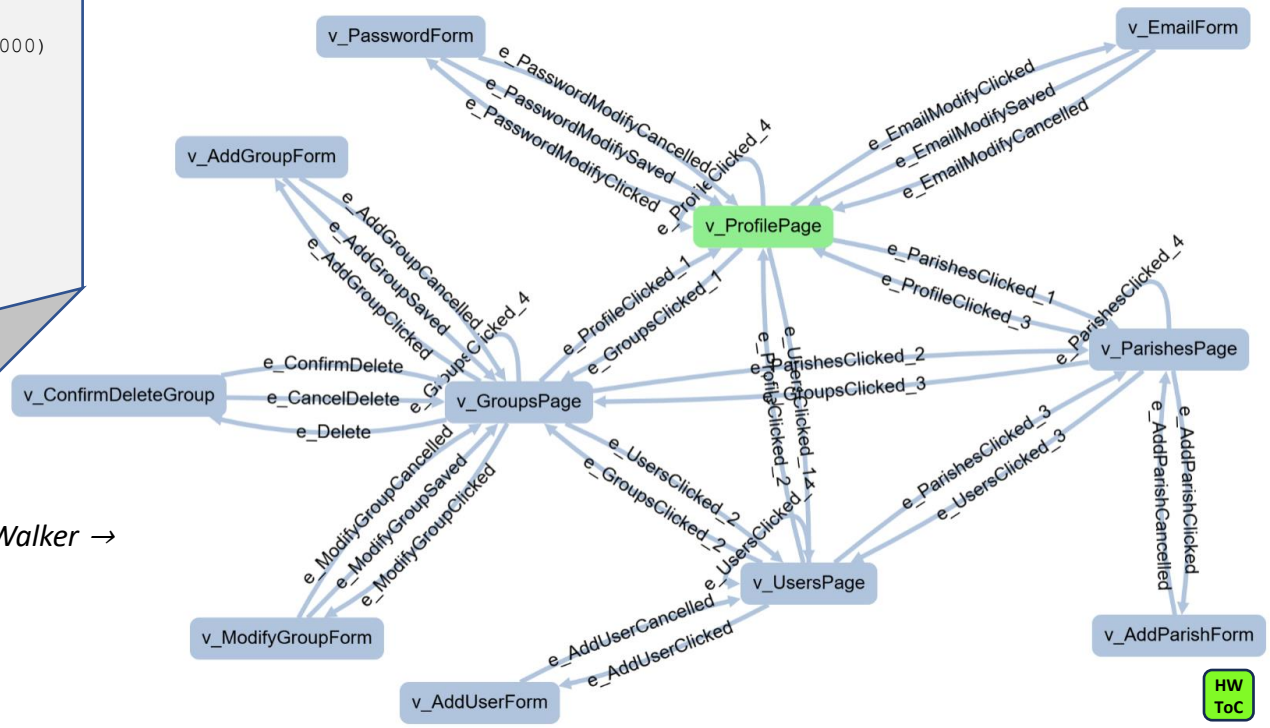
Adaptation code examples (definitions of few transitions):

```
async function savePassword() {
    const localDriver = await getDriver()
    await localDriver.findElement(By.xpath(extraHtmlMap["passwordFormNewPassword"])).clear()
    await localDriver.findElement(By.xpath(extraHtmlMap["passwordFormOldPassword"])).clear()
    await localDriver.findElement(By.xpath(extraHtmlMap["passwordFormNewPassword"])).sendKeys('123456')
    await localDriver.findElement(By.xpath(extraHtmlMap["passwordFormOldPassword"])).sendKeys('123456')

    await localDriver.wait(until.elementLocated(By.css(inputAndHtmlMap["press Save password"])), 5000)
    await localDriver.findElement(By.css(inputAndHtmlMap["press Save password"])).click()
}

async function saveNewGroup() {
    const localDriver = await getDriver()

    await localDriver.wait(until.elementLocated(By.xpath(extraHtmlMap["newGroupFormName"])), 5000)
    await localDriver.findElement(By.xpath(extraHtmlMap["newGroupFormName"])).sendKeys("0. class")

    await localDriver.wait(until.elementLocated(By.css(inputAndHtmlMap["press Save new group"])), 5000)
    await localDriver.findElement(By.css(inputAndHtmlMap["press Save new group"])).click()
}

async function clickCheckbox() {
    const localDriver = await getDriver()
    await localDriver.wait(until.elementLocated(By.css(extraHtmlMap["checkbox"])), 5000)
    const checkbox = await localDriver.findElement(By.css(extraHtmlMap["checkbox"]))
    const isClicked = await checkbox.isSelected()
    if(!isClicked) {
        await checkbox.click()
    }
    await localDriver.sleep(1000)
}
```

A test execution example:

```
D:\LILLA\Egyetem\MSc\1\Modelling_and_Testing\Adaptation_code\JavaScript>node index.js MisEnaplo-Random-state-50-test_suite.json

DevTools listening on ws://127.0.0.1:60425/devtools/browser/f680b9b0-e943-48e4-ac77-a506abdbaa6b
[INFO] DRIVER READY
[INFO] LOGGED IN
[INFO] 1. - PASS - INPUT: press Users item ||  OUTPUT: users page
[INFO] 2. - PASS - INPUT: press Add user ||  OUTPUT: user form
[INFO] 3. - PASS - INPUT: press Cancel user ||  OUTPUT: users page
[INFO] 4. - PASS - INPUT: press Groups item ||  OUTPUT: groups page
[INFO] 5. - PASS - INPUT: press Parishes item ||  OUTPUT: parishes page
[INFO] 6. - PASS - INPUT: press Parishes item ||  OUTPUT: parishes page
[INFO] 7. - PASS - INPUT: press Users item ||  OUTPUT: users page
[INFO] 8. - PASS - INPUT: press Parishes item ||  OUTPUT: parishes page
[INFO] 9. - PASS - INPUT: press Add parish ||  OUTPUT: parish form
[INFO] END OF TEST SEQUENCE
[INFO] DRIVER STOPPED

Navigation fail: 0 times
Timeout fail: 0 times
Success rate: 100%
Duration of execution: 0 min 4 sec
```

FSM model in GraphWalker →



HW ToC

Appendix D: Example homework XXXXV. 3/1

Fall 2024/2025

Márton Reményi: Wikipedia

- SUT (System Under Test): Wikipedia free encyclopedia
  - Tested functionalities:
    - User authentication
    - Watchlist editing
    - Searching articles
    - Viewing articles
    - Editing an article
    - Editing a talk page
    - Changing the appearance

- Testing:
  - Model has been created with GraphWalker Studio
  - EFSM → FSM conversation and test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
    - Using Selenium with Chromedriver for GUI testing

WIKIPEDIA The Free Encyclopedia

Search Wikipedia | Search

Green kingfisher

27 languages

Article | Talk | Read | Edit | View history | Tools

From Wikipedia, the free encyclopedia

Contents hide

(Top)
Taxonomy and systematics
Description
Distribution and habitat
Behavior
Feeding
Breeding
Status
References
External links

The **green kingfisher** (***Chloroceryle americana***) is a species of "water kingfisher" in the subfamily Cerylinae of the family Alcedinidae. It is found from southern Texas in the United States south through Central America, in every mainland South American country except Chile and Trinidad and Tobago.[2][3]

Taxonomy and systematics [edit]

The green kingfisher was formally described in 1788 by the German naturalist Johann Friedrich Gmelin in his revised and expanded edition of Carl Linnaeus's *Systema Naturae*. He placed it with the other kingfishers in the genus *Alcedo* and coined the binomial name *Alcedo americana*.[4] Gmelin based his description on "Le martin-pêcheur vert et blanc" from Cayenne that had been described and illustrated in 1780 by the French polymath Comte de Buffon and also the "white and green kingfisher" that had been described in 1782 by the English ornithologist John Latham.[5][6][7] The green kingfisher is now placed in the genus *Chloroceryle* that was introduced in 1848 by Johann Jakob Kaup.[8][9]

A molecular phylogenetic study published in 2006 found that the green kingfisher is a sister species to the larger green-and-rufous kingfisher (*C. inda*).[10]

Five subspecies are recognized:[9][11]

Image	Subspecies	Distribution
	C. a. hachisukai (Laubmann, 1941)	from extreme southern Arizona, southwestern New Mexico, and west central Texas in the US into northwestern Mexico to Nayarit
	C. a. septentrionalis (Sharpe, 1892)	from south central Texas and eastern Mexico south to northern Colombia and western Venezuela
	C. a. americana (Gmelin, JF, 1788)	South America east of the Andes from Venezuela east through the Guianas into northern and central Brazil and south to northeastern Bolivia; also Trinidad and Tobago

Green kingfisher

Male *C. a. americana* in the Pantanal, Brazil

Conservation status

Extinct | Threatened | Least Concern
EX EW CR EN VU NT LC

Least Concern (IUCN 3.1)[1]

Scientific classification

Domain:	Eukaryota
Kingdom:	Animalia
Phylum:	Chordata
Class:	Aves
Order:	Coraciiformes
Family:	Alcedinidae
Subfamily:	Cerylinae
Genus:	*Chloroceryle*
Species:	***C. americana***

Binomial name

Chloroceryle americana
(Gmelin, JF, 1788)

Subspecies

C. a. americana
C. a. mathewsii

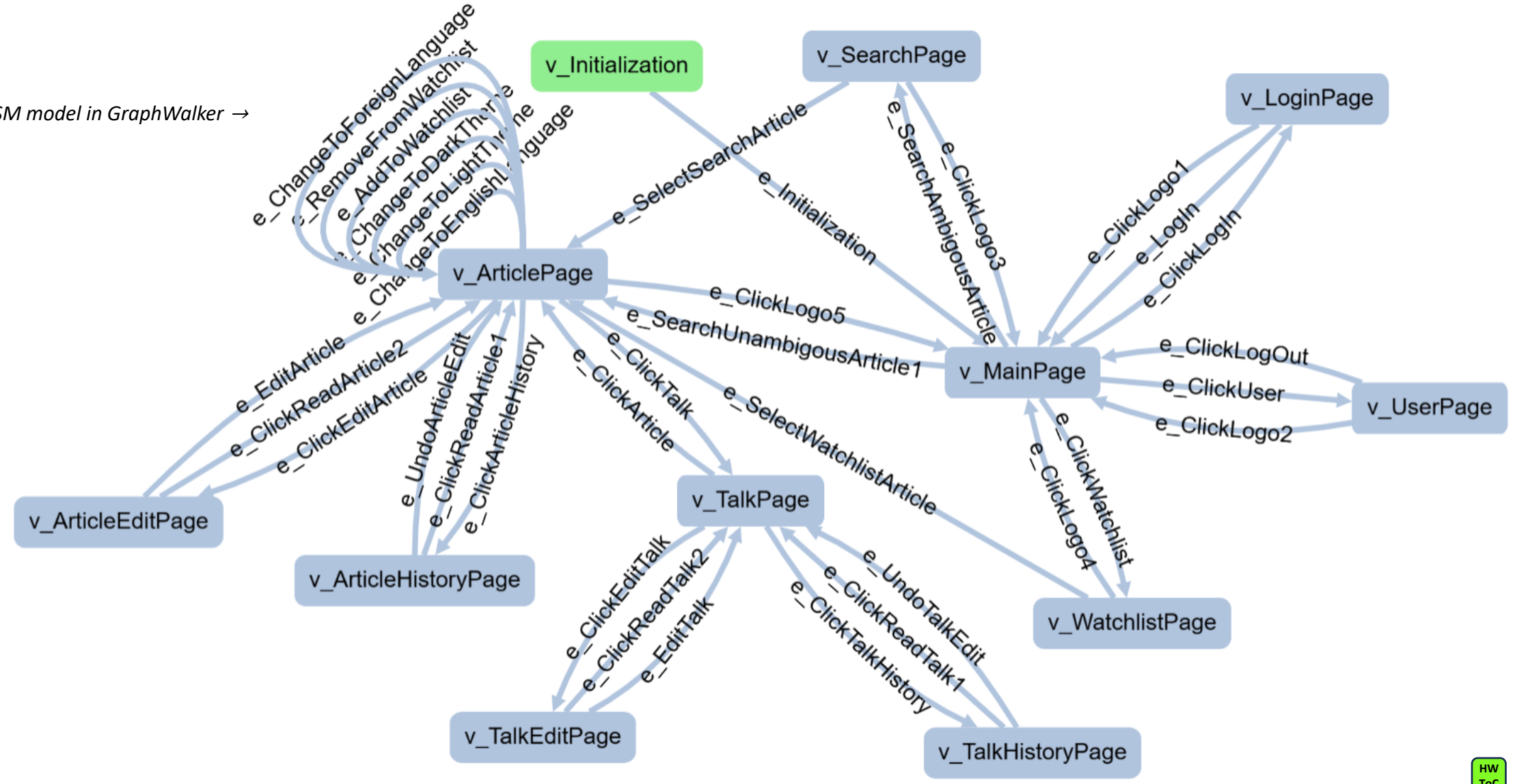
HW ToC

Appendix D: Example homework XXXXV. 3/2

Fall 2024/2025

Márton Reményi: Wikipedia

EFSM model in GraphWalker →



HW ToC

Appendix D: Example homework XXXXV. 3/3

Fall 2024/2025

Márton Reményi: Wikipedia

Adaptation code examples:

```
# Selenium input methods (adaptation of model transitions)
…
def click_talk():
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "#ca-talk"))).click()
    return "talk_page"

def click_read_talk():
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "#ca-view"))).click()
    return "talk_page"

def click_talk_history():
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "#ca-history"))).click()
    return "talk_history_page"

def click_edit_talk():
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "#ca-edit"))).click()
    return "talk_edit_page"

def edit_talk():
    last_URL = driver.current_url

    visit_sandbox_talk_edit()
    textbox =  wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "#wpTextbox1")))
    textbox.clear()
    textbox.send_keys("== New Discussion ==\nThis is a new discussion! :D")
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "#wpSaveWidget"))).click()

    driver.get(last_URL)
    return click_talk()

def undo_talk_edit():
    last_URL = driver.current_url

    visit_sandbox_talk_history()
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, ".mw-history-undo"))).click()
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "#wpSaveWidget"))).click()

    driver.get(last_URL)
    return click_talk()
…
# Mappings
input_map = {
    …
    "click_talk": click_talk,
    "click_read_talk": click_read_talk,
    "click_talk_history": click_talk_history,
    "click_edit_talk": click_edit_talk,
    "edit_talk": edit_talk,
    "undo_talk_edit": undo_talk_edit,
    "change_to_english_language": change_to_english_language,
    "change_to_foreign_language": change_to_foreign_language,
    "change_to_light_theme": change_to_light_theme,
    "change_to_dark_theme": change_to_dark_theme
}
…
#Iterating over the inputs
            bar = progressbar.ProgressBar(max_value=len(inputs))
            error_count = 0
            for i in range(len(inputs)):
                try:
                    if input_map[inputs[i]]() != outputs[i]:
                        print(f"\nError for input: {inputs[i]}")
                        error_count += 1
                except:
                    print(f"\nError for input: {inputs[i]}")
                    error_count += 1

                bar.update(i)
            bar.finish()
```

Test execution example:

```
DevTools listening on ws://127.0.0.1:51573/devtools/browser/fbb3dc75-5a9a-4ab8-a7ad-ce1d09dc0dea
100% (404 of 404) |########################################################################################################################################################################| Elapsed Time: 0:08:27 Time:  0:08:27

        > Success rate: 100.0%
        > Input count: 404
        > Elapsed time: 522.89s
```

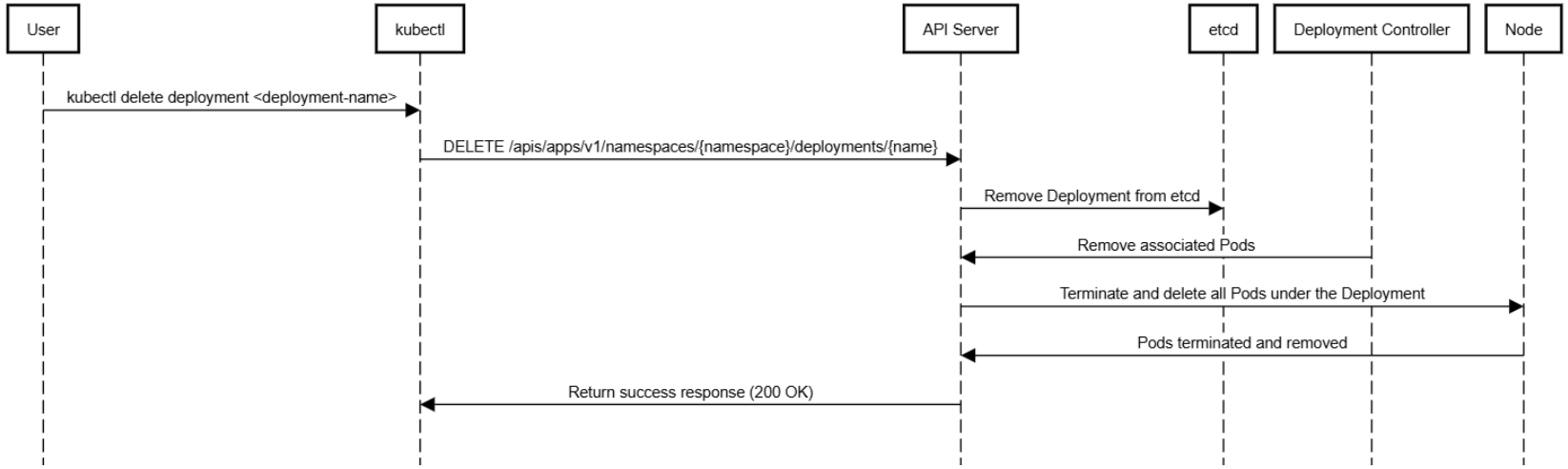
HW ToC

Appendix D: Example homework XXXXVI. 3/1

Fall 2024/2025

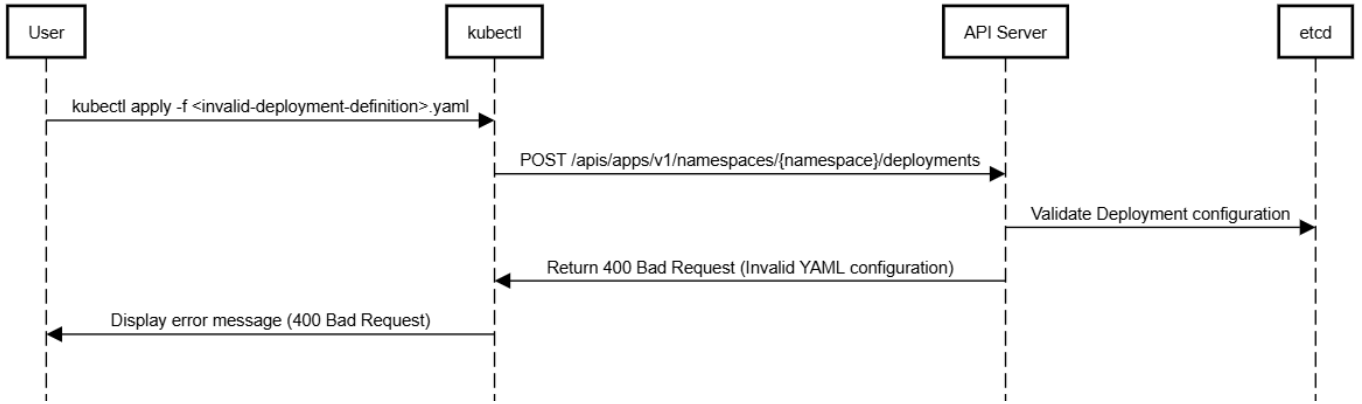
Anna Száraz: Kubernetes Pod Lifecycle Management

- SUT (System Under Test): Kubernetes open-source platform for automating the deployment, scaling, and management of containerized applications
  - Tested functionalities:
    - Create/Delete Pod, Final Deletion
    - Image Pull Success/Error
    - Containers Ready
    - Readiness Probe Success/Failure
    - Pod Success/Failure
    - Container Crash Loop/Crash Loop Timeout



Creating a Deployment

- Testing:
  - Model has been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Java
    - Interface with the Kubernetes cluster using *kubectl* commands



Error During Creating a Deployment

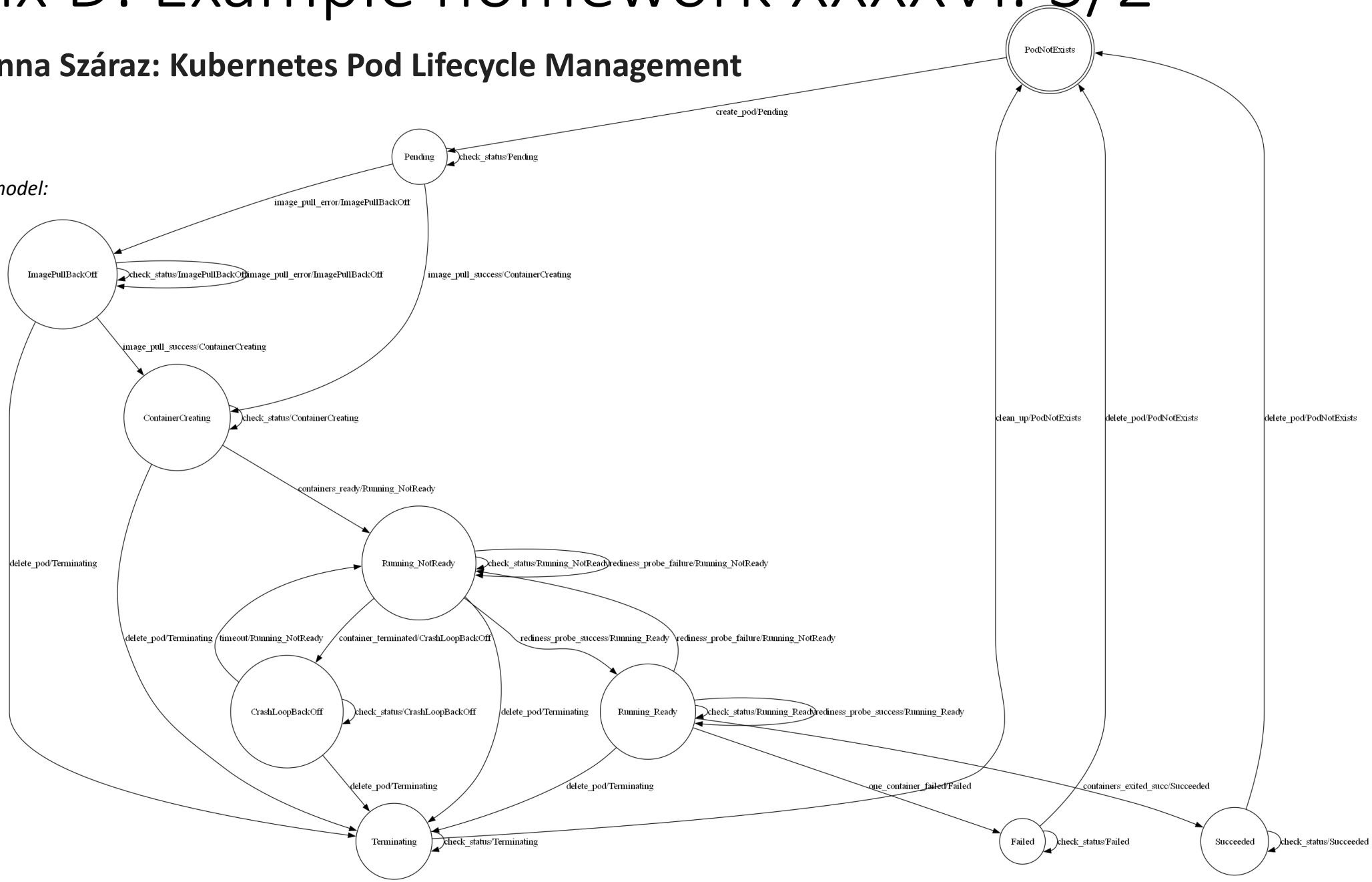
HW ToC

Appendix D: Example homework XXXXVI. 3/2

Fall 2024/2025

Anna Száraz: Kubernetes Pod Lifecycle Management

FSM model:



HW ToC

Appendix D: Example homework XXXXVI. 3/3

Fall 2024/2025

Anna Száraz: Kubernetes Pod Lifecycle Management

Adaptation code examples:

Parse JSON test suite and execute it:

```
    public void runTestSuite(TestSuite testSuite) throws FileNotFoundException, URISyntaxException, InterruptedException {
        long startTime = System.currentTimeMillis();
        boolean allTestPassed = true;
        boolean currentResult;

        KubectlUtils.deletePod("test-pod"); // fresh start

        List<String> inputs = testSuite.getInputList();
        List<String> expectedOutputs = testSuite.getOutputList();
        int testSuiteLength = inputs.size();

        // Iterate through all test cases
        for (int i = 0; i < testSuiteLength; i++) {
            String input = inputs.get(i);
            String expectedOutput = expectedOutputs.get(i);

            // Execute the corresponding InputExecutor for the given input
            if (inputFunctionMap.containsKey(input) && outputMap.containsKey(expectedOutput)) {
                System.out.println(STR."Running test for input: \{input}");
                System.out.println(STR."Expected output: \{expectedOutput}");
                currentResult = Utils.processInput(inputFunctionMap.get(input), outputMap.get(expectedOutput));
            } else {
                throw new IllegalArgumentException(STR."Unknown input: \{input}");
            }

            if (currentResult) {
                System.out.println("-------------------------------------------------");
                System.out.println(STR."Test (\{i+1}/\{testSuiteLength}) passed for input: \{input}");
                System.out.println("-------------------------------------------------");
            } else {
                System.out.println("-------------------------------------------------");
                System.out.println(STR."Test (\{i+1}/\{testSuiteLength}) failed for input: \{input}");
                System.out.println(STR."Expected: \{expectedOutput}");
                System.out.println("-------------------------------------------------");
            }

            allTestPassed = allTestPassed && currentResult;
        }

        long endTime = System.currentTimeMillis();
        System.out.println("\n");
        System.out.println(STR."Elapsed time: \{(endTime-startTime)/1000} s");
        if (allTestPassed) {
            System.out.println("All test passed!\n\n");
        } else {
            System.out.println("There were failures...\n\n");
        }
    }
}
```

createPod input Mapper:

```
package input_executors;

import output_mapper.OutputMapper;
import utils.KubectlUtils;

import java.io.FileNotFoundException;
import java.net.URISyntaxException;

public class PodCreateInputExecutor implements InputExecutor {

    @Override
    public boolean execute(OutputMapper expectedOutput) throws FileNotFoundException, URISyntaxException, InterruptedException {
        Thread.sleep(5000);
        System.out.println(KubectlUtils.applyPodDefinition("test-pod.yaml"));
        return expectedOutput.getState();
    }
}
```

Output Mapper for the Pending Output of the above transition:

```
package output_mapper;

import utils.KubectlUtils;

public class PendingOutputMapper implements OutputMapper {
    @Override
    public boolean getState() throws InterruptedException {
        return KubectlUtils.getPodStatus("test-pod", "Pending");
    }
}
```

Final steps of a test execution →

```
-------------------------------------------------
Running test for input: delete_pod
Expected output: Terminating
pod "test-pod" deleted
Actual status: Running
Pod state: test-pod   1/1   Terminating   0     8s
-------------------------------------------------
Test (5/6) passed for input: delete_pod
-------------------------------------------------
Running test for input: clean_up
Expected output: PodNotExists
Actual status: Error executing kubectl command.
Pod state: Error executing kubectl command.
-------------------------------------------------
Test (6/6) passed for input: clean_up
-------------------------------------------------


Elapsed time: 33 s
All test passed!
```

HW ToC

Appendix D: Example homework XXXXVII. 2/1

Fall 2024/2025

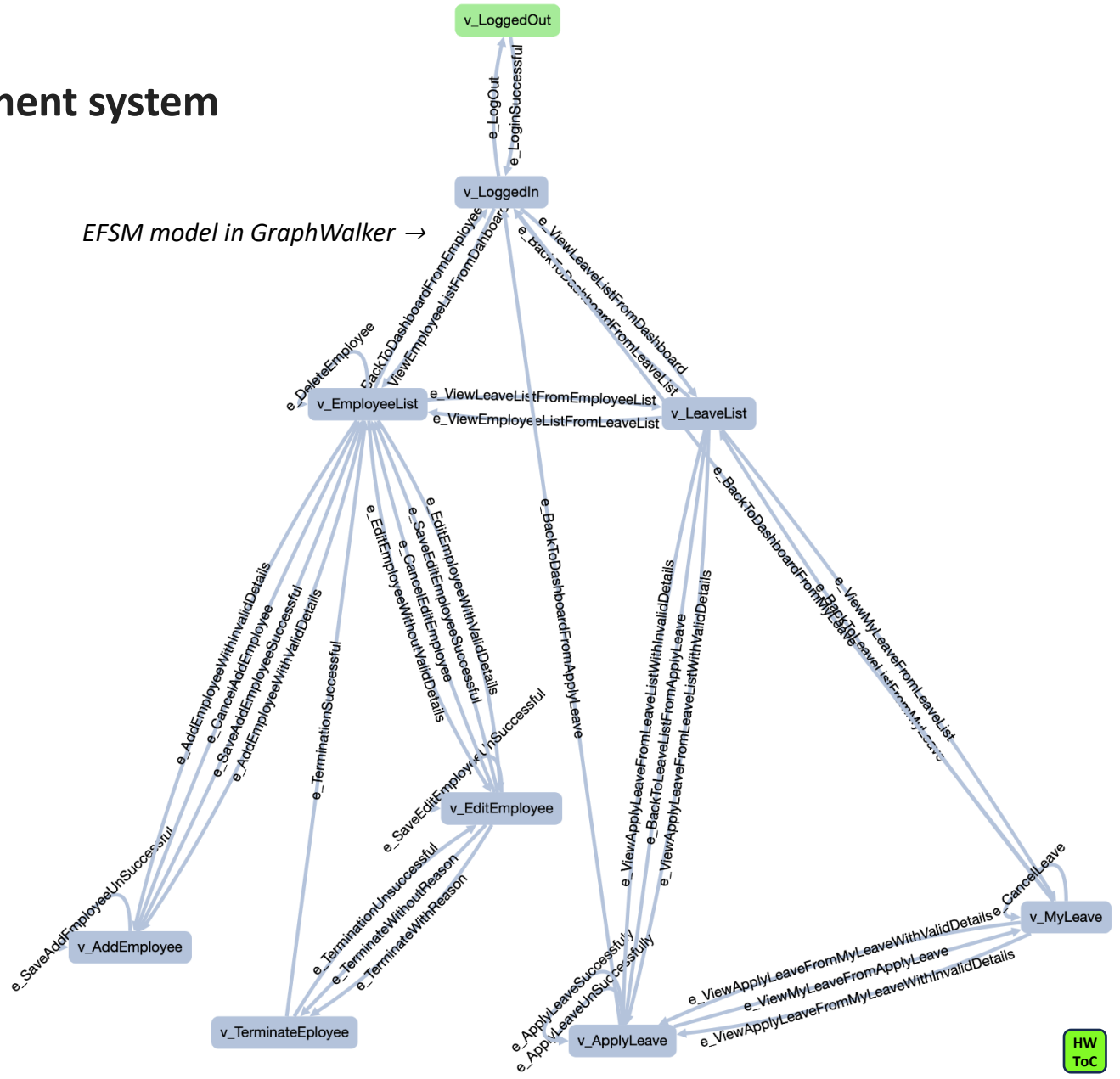
Miklós Mihály Vidovics:
OrangeHRM human resource management system

OrangeHRM
OPEN SOURCE HR MANAGEMENT

- SUT (System Under Test): OrangeHRM human resource management system
  - Tested functionalities:
    - Login and Logout
    - Display and edit employee data
    - Add and remove worker
    - Dismissing a worker
    - Apply and cancel leave request

- Testing:
  - Model has been created with GraphWalker Studio
  - EFSM → FSM conversation and test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
    - Using Selenium with Chromedriver for GUI testing

EFSM model in GraphWalker →



HW ToC

Appendix D: Example homework XXXXVII. 2/2

Fall 2024/2025

Miklós Mihály Vidovics: OrangeHRM human resource management system

Adaptation code examples (definitions of few transitions):

```python
def click_my_leave(self):
    try:
            el1 = self.wait_for_element_clickable(
                By.XPATH,
                "//span[contains(@class, 'oxd-main-menu-item--name') and text()='Leave']"
            )
            el1.click()
            self.wait_for_page_load()

            el2 = self.wait_for_element_clickable(
                By.XPATH,
                "//a[contains(@class, 'oxd-topbar-body-nav-tab-item') and text()='My Leave']"
            )
            el2.click()
            self.wait_for_page_load()

            self.wait_for_element(By.XPATH, "//h5[text()='My Leave List']")

    except Exception as e:
        self.logger.warning(f"{str(e)}")

def click_leave_list(self):
    try:
            el1 = self.wait_for_element_clickable(
                By.XPATH,
                "//span[contains(@class, 'oxd-main-menu-item--name') and text()='Leave']"
            )
            el1.click()
            self.wait_for_page_load()

            el2 = self.wait_for_element_clickable(
                By.XPATH,
                "//a[@class='oxd-topbar-body-nav-tab-item'][contains(text(), 'Leave List')]"
            )
            el2.click()
            self.wait_for_page_load()
            self.wait_for_element(By.XPATH, "//h5[text()='My Leave List']")

    except Exception as e:
        self.logger.warning(f"{str(e)}")
```

Test execution:

4-25-1 > MAT > Extended > src > log.txt

```
Test File: ../OrangeHRM-transformed-AS-test_suite.json
Duration: 27:39 (MM:SS)
Status: PASSED
--------------------------------------------------
Test File: ../OrangeHRM-transformed-ATS0-test_suite.json
Duration: 396:52 (MM:SS)
Status: PASSED
--------------------------------------------------
Test File: ../OrangeHRM-transformed-NS-1-test_suite.json
Duration: 360:43 (MM:SS)
Status: PASSED
--------------------------------------------------
Test File: ../OrangeHRM-transformed-Random-state-75-test_suite
Duration: 101:19 (MM:SS)
Status: PASSED
--------------------------------------------------
Test File: ../OrangeHRM-transformed-Random-transition-75-test_suite.json
Duration: 254:22 (MM:SS)
Status: PASSED
--------------------------------------------------
Test File: ../OrangeHRM-transformed-TT-test_suite.json
Duration: 71:17 (MM:SS)
Status: PASSED
--------------------------------------------------
```

HW ToC

Appendix D: Example homework XXXXVIII. 4/1

Fall 2024/2025

Norbert Ádám Mihalkó: Automatic transmission controller

- SUT (System Under Test): Simulator of an automatic transmission controller system in a car (Java Swing GUI)
  - Tested functionalities:
    - 4 states of automatic transmission: P (parking), R (reverse), N (neutral), D (drive)
    - 1st-6th gear position of D
    - Acceleration/deceleration between the -30…+150 km/h range in 5km/h steps

- Testing:
  - Model has been created with GraphWalker Studio
  - EFSM → FSM conversation and test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Java
    - GUI tests and model tests were both created
    - GUI testing with AssertJ Swing and custom functions

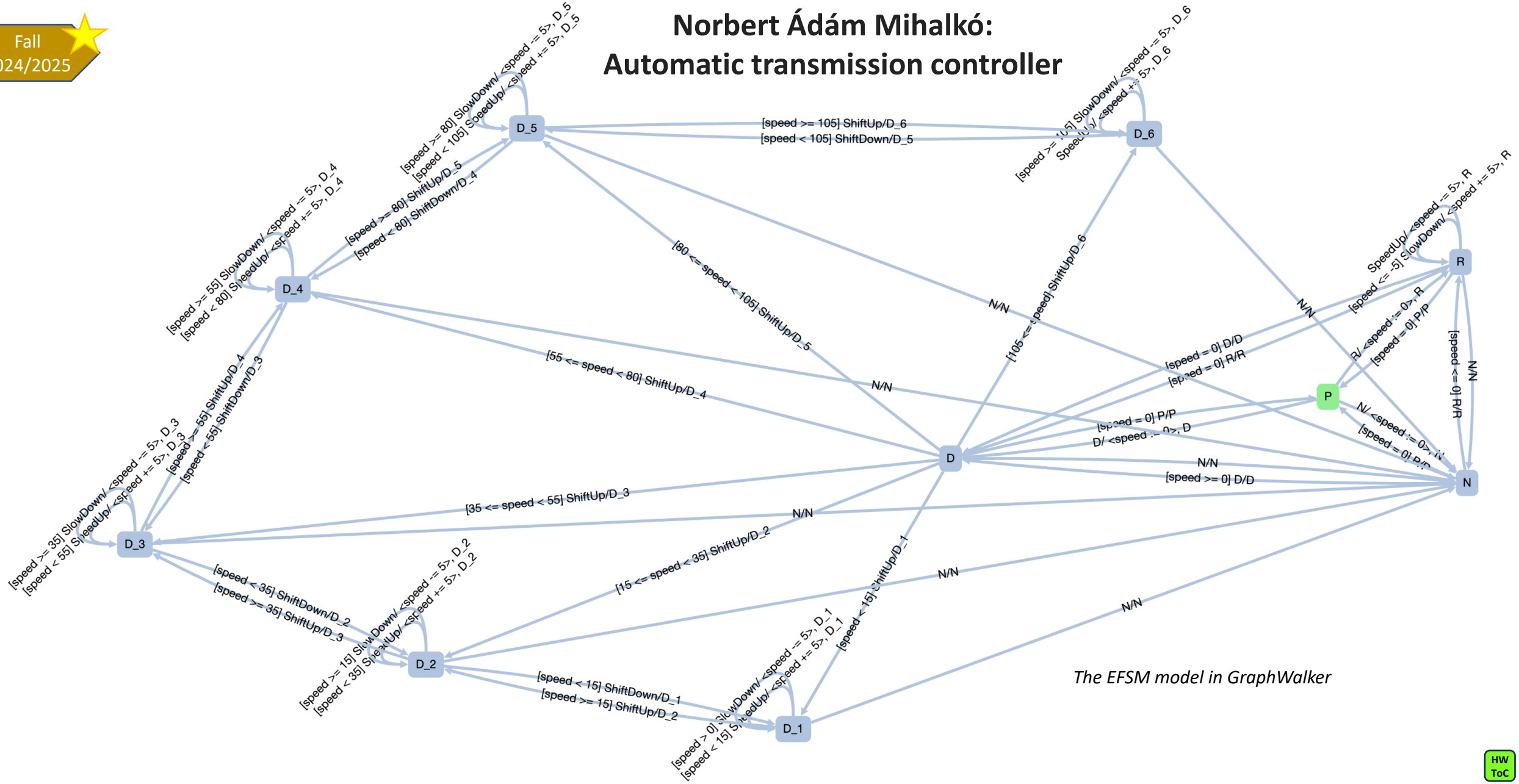
The GUI of automatic transmission system simulator

HW ToC

Appendix D: Example homework XXXXVIII. 4/2

Fall 2024/2025

Norbert Ádám Mihalkó:
Automatic transmission controller



The EFSM model in GraphWalker

HW ToC

Appendix D: Example homework XXXXVIII. 4/3

Fall 2024/2025

Norbert Ádám Mihalkó:
Automatic transmission controller

Adaptation code examples:

```
class AutomaticGuiTest {
    public static final String CSV_REPORT_PATH = "test_reports/AutomaticGuiTestsReport.csv";
    private static List<CsvRow> resultsRows;

    private FrameFixture window;

    private boolean runTestSuite(String testSuitePath) throws IOException {
        TestSuite testSuite = TestUtils.readJsonFileToTestSuite(testSuitePath);
        TestSuiteExecutor executor = new TestSuiteExecutor(window, testSuite.inputs(), testSuite.outputs());
        Long start = System.currentTimeMillis();
        Boolean isSuccess = executor.executeGui();
        Long finish = System.currentTimeMillis();
        Long timeElapsed = finish - start;
        Optional<Result> failedStep = executor.getResults().stream()
            .filter(result -> result.desiredOutput() != result.output())
            .findFirst();

        String failedAtIndex = failedStep.isEmpty() ? "-" : String.valueOf((failedStep.get().index() + 1));
        String failedAtInput = failedStep.isEmpty() ? "-" : failedStep.get().input().toString();
        String failedAtDesiredOutput = failedStep.isEmpty() ? "-" : failedStep.get().desiredOutput().toString();
        String failedAtOutput = failedStep.isEmpty() ? "-" : failedStep.get().output().toString();


        CsvRow resultRow = new CsvRow(isSuccess.toString(), testSuite.name(), testSuite.method(),
                String.valueOf(testSuite.inputs().length()), "Yes", timeElapsed.toString(), failedAtIndex,
                failedAtInput, failedAtDesiredOutput, failedAtOutput);
        resultsRows.add(resultRow);
        return isSuccess;
    }

    @BeforeAll
    public static void startUp() {
        resultsRows = new ArrayList<>();
    }

    @AfterEach
    public void finish() {
        window.cleanUp();
    }

    @BeforeEach
    public void setUp() throws InvocationTargetException, InterruptedException {
        FailOnThreadViolationRepaintManager.install();
        Gui gui = GuiActionRunner.execute(() -> new Gui());
        window = new FrameFixture(gui);
        window.show();
    }
    @AfterAll
    public static void makeCsvReport() throws IOException {
        TestUtils.makeCsvReport(resultsRows, CSV_REPORT_PATH);
    }

    @Test
    public void testAS() throws IOException {
        assertTrue(runTestSuite(TestUtils.AS_TEST_SUITE_PATH));
    }
```

```
public boolean execute(Transmission transmission) {
    IntStream.range(0, inputOutputPairs.size()).forEach(i -> {
        Integer index = i;
        State startingState = transmission.getCurrentState();

        Map.Entry<String, State> entry = inputOutputPairs.get(i);
        String inputStr = entry.getKey();
        Input input = Input.convertStringToInput(inputStr);
        State desiredOutput = entry.getValue();

        transmission.execute(inputStr);
        State output = transmission.getCurrentState();
        boolean isSucces = desiredOutput == output;
        results.add(new Result(index, startingState, input, desiredOutput, output, isSucces));
    });
    return results.stream()
            .map(result -> result.success())
            .reduce(true, (a, b) -> a && b);
}

public boolean executeGui() {
    if(window == null)
        throw new RuntimeException("Current executor is not useable for gui tests!");

    IntStream.range(0, inputOutputPairs.size()).forEach(i -> {
        Integer index = i;
        JPanelFixture controlPanel = window.panel("controlPanel");
        JLabelFixture stateLabel = controlPanel.label("currentGear");
        State startingState = State.convertStringToState(stateLabel.text());

        Map.Entry<String, State> entry = inputOutputPairs.get(i);
        String inputStr = entry.getKey();
        Input input = Input.convertStringToInput(inputStr);
        State desiredOutput = entry.getValue();

        JButtonFixture button = findButton(controlPanel, inputStr);
        clickOnButton(button.target());
        window.robot().waitForIdle();

        State output = State.convertStringToState(stateLabel.text());
        boolean isSucces = desiredOutput == output;
        results.add(new Result(index, startingState, input, desiredOutput, output, isSucces));
    });

    return results.stream()
            .map(result -> result.success())
            .reduce(true, (a, b) -> a && b);
}

private void clickOnButton(JButton button) {
    GuiActionRunner.execute(() -> {
        Arrays.stream(button.getActionListeners()).forEach(listener ->
            listener.actionPerformed(new ActionEvent(button, ActionEvent.ACTION_PERFORMED, ""))
        );
    });
    window.robot().waitForIdle();
}
```

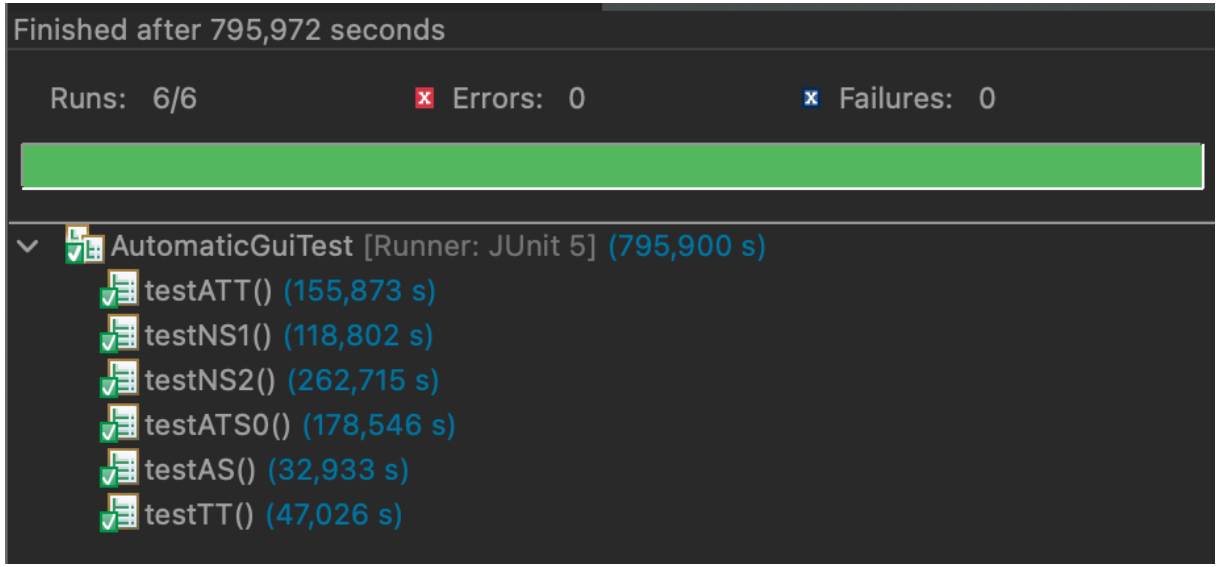
HW ToC

Appendix D: Example homework XXXXVIII. 4/4

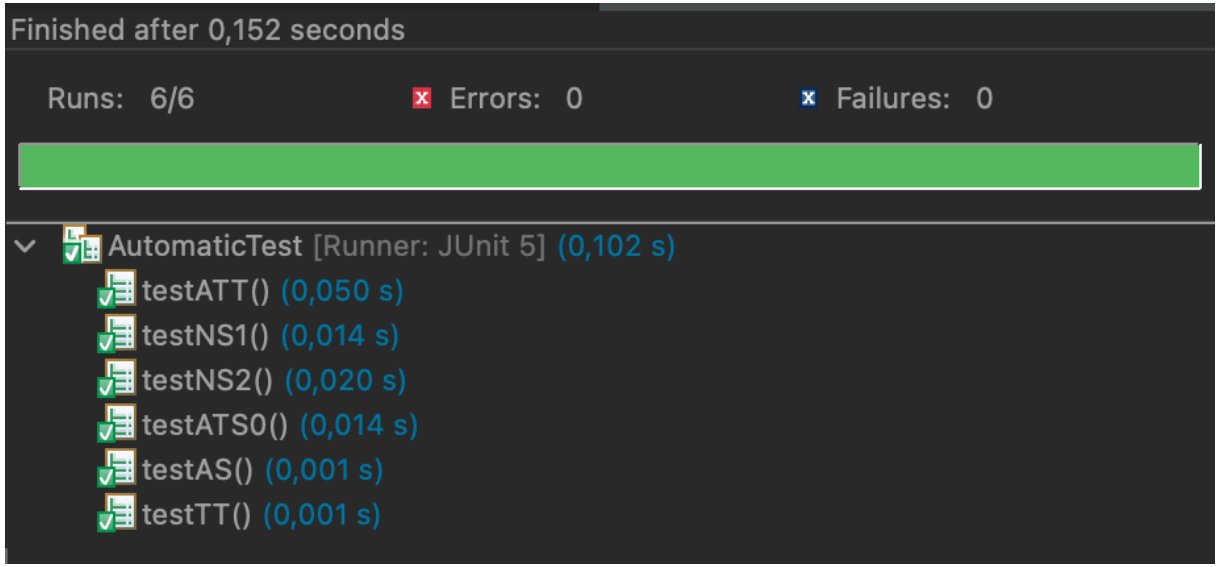
Fall 2024/2025

Norbert Ádám Mihalkó: Automatic transmission controller

Tests executed on GUI:



Tests executed on model only:



Test summary (ATS with 0 depth, N-SC with 1 and 2 switch coverage respectively):

Success	true											
Model name	Homework-transformed											
Test generation algorithm	ATT	NS	NS	ATS	AS	TT	ATT	NS	NS	ATS	AS	TT
Number of steps	1326	936	2179	1455	197	319	1326	936	2179	1455	197	319
Gui test	No						Yes					
Execution time [ms]	3	1	3	1	0	0	154145	108704	252522	168444	22845	36946
Failed first at step	-	-	-	-	-	-	-	-	-	-	-	-
Input	-	-	-	-	-	-	-	-	-	-	-	-
Desired output	-	-	-	-	-	-	-	-	-	-	-	-
Received output	-	-	-	-	-	-	-	-	-	-	-	-

HW ToC

Appendix D: Example homework XXXXIX. 3/1

Levente Székely: Ory Hydra Oauth2.0 and OpenID Connect server

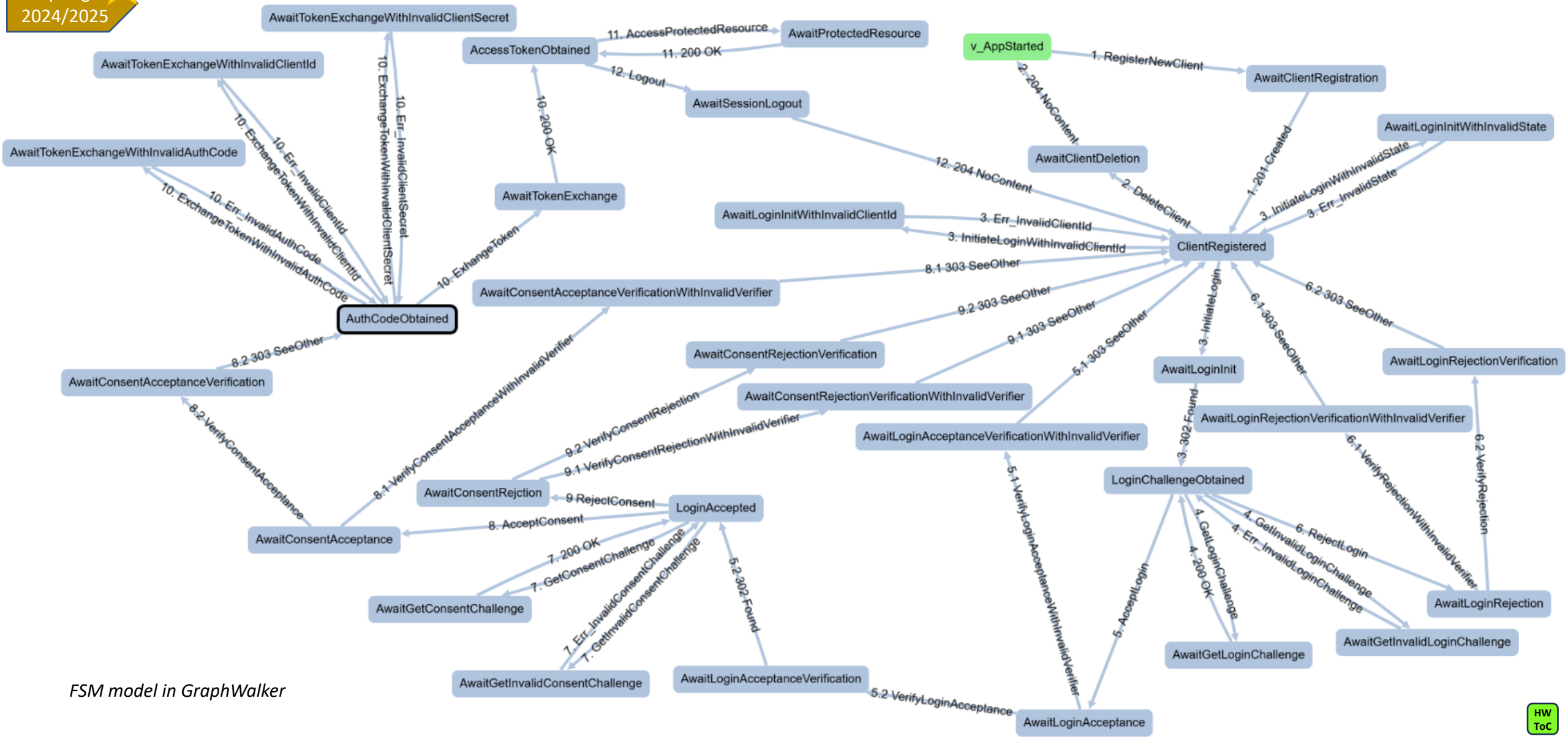
- SUT (System Under Test): Ory Hydra open source Oauth2.0 and OpenID Connect server
  - Tested functionalities:
    - Register a new client
    - Delete client registration
    - Login with authentication
      - Initiate the authorization code OAuth 2.0 flow and return a login challenge
      - Return an existing login challenge's metadata based on a valid login challenge ID
      - Accept/reject login requests for valid login challenge IDs
      - Return an existing consent challenge's metadata based on a valid consent challenge ID
      - Accept/reject a valid consent challenge
      - Exchange a valid authorization code for a valid access token
    - Access a protected resource using a valid access token
    - Suspend and active session
- Testing:
  - FSM model has been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in C#
    - Using Ory Hydra API

HW ToC

Appendix D: Example homework XXXXIX. 3/2

Spring 2024/2025

Levente Székely: Ory Hydra Oauth2.0 and OpenID Connect server



FSM model in GraphWalker

HW ToC

Appendix D: Example homework XXXXIX. 3/3

Spring 2024/2025

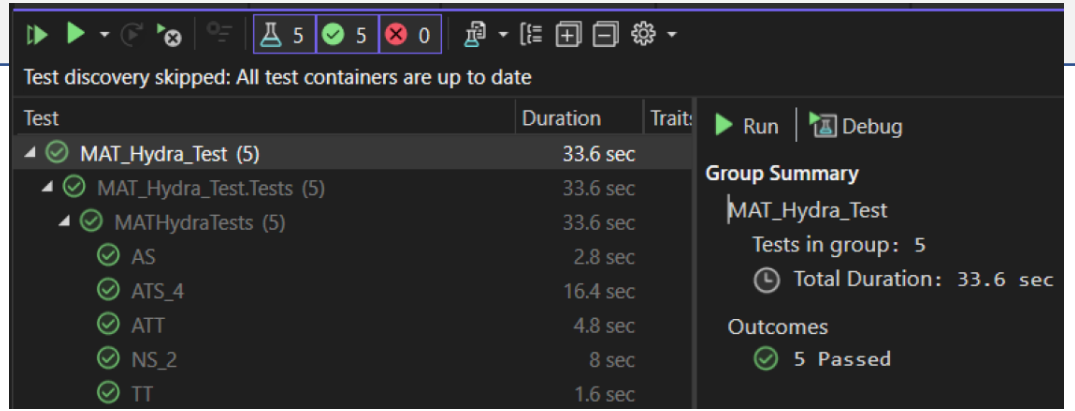
Levente Székely: Ory Hydra Oauth2.0 and OpenID Connect server

Adaptation code examples:

```
 private async Task AcceptLogin()
{
 var client = new HttpClient();
 var acceptanceMetadata = new ClientAcceptOAuth2LoginRequest(subject: "Test");
 var acceptanceStringContent = new  StringContent(JsonConvert.SerializeObject(acceptanceMetadata));
 _latestApiResponse = await
 client.PutAsync($"http://localhost:4445/admin/oauth2/auth/requests/login/accept?login_challenge={
_currentLoginChallenge}", acceptanceStringContent);
 Assert.Equal(HttpStatusCode.OK, _latestApiResponse!.StatusCode);
 var content = await _latestApiResponse.Content.ReadAsStringAsync();
 var response = JsonConvert.DeserializeObject<ClientOAuth2RedirectTo>(content);
 var parsedQuery = HttpUtility.ParseQueryString(response!.RedirectTo);
 Assert.Equal("login_verifier", parsedQuery.AllKeys[1]);
 _currentLoginVerifier = parsedQuery.Get(1)!;
}

private async Task StartLoginAcceptanceVerification()
{
 var handler = new HttpClientHandler { AllowAutoRedirect = false, CookieContainer = new() };
 var client = new HttpClient(handler);
 cookieCache.CachedCookies.TryGetValue(_currentState!, out var sessionCookies);
 sessionCookies!.TryGetValue(CookieType.Login, out var cookie);
 var url =
$"http://localhost:4444/oauth2/auth?client_id={_currentClientObject!.ClientId}&login_verifier={_cur
rentLoginVerifier!}&response_type=code&scope={HttpUtility.UrlEncode(_scopes)}&state={_currentState}
&redirect_uri={HttpUtility.UrlEncode("http://localhost:4444")}";
handler.CookieContainer.Add(new Uri(url), cookie!);
 _latestApiResponse = await client.GetAsync(url);
 Assert.Equal(HttpStatusCode.Found, _latestApiResponse.StatusCode);
}
```

```
private void AcceptLoginAcceptanceVerification()
{
 var headers = _latestApiResponse!.Headers.ToDictionary();
 Assert.True(headers.ContainsKey("Location"));
 Assert.Single(headers["Location"]);
 var parsedQuery = HttpUtility.ParseQueryString(headers["Location"].Single());
 Assert.Equal("consent_challenge", parsedQuery.AllKeys[0]!.Split("?").Last());
 Assert.True(headers.ContainsKey("Set-Cookie"));
 Assert.Equal(2, headers["Set-Cookie"].Count());
 var sessionCookie = headers["Set-Cookie"].First();
 cookieCache.CachedCookies.TryGetValue(_currentState!, out var innerDictionary);
 innerDictionary!.TryAdd(CookieType.Consent,
  new System.Net.Cookie(
   sessionCookie.Split("=").First(),
   sessionCookie.Split("=").ElementAt(1).Split(";").First()
   ));
 var consentCookie = headers["Set-Cookie"].Last();
 innerDictionary!.TryAdd(CookieType.Session,
  new System.Net.Cookie(
   consentCookie.Split("=").First(),
   consentCookie.Split("=").ElementAt(1).Split(";").First()
   ));
 _currentConsentChallenge = parsedQuery.Get(0);
}
```

Test execution →



HW ToC

Appendix D: Example homework L. 4/1

Zsolt Gyarmati: X Synchronization Extension Library

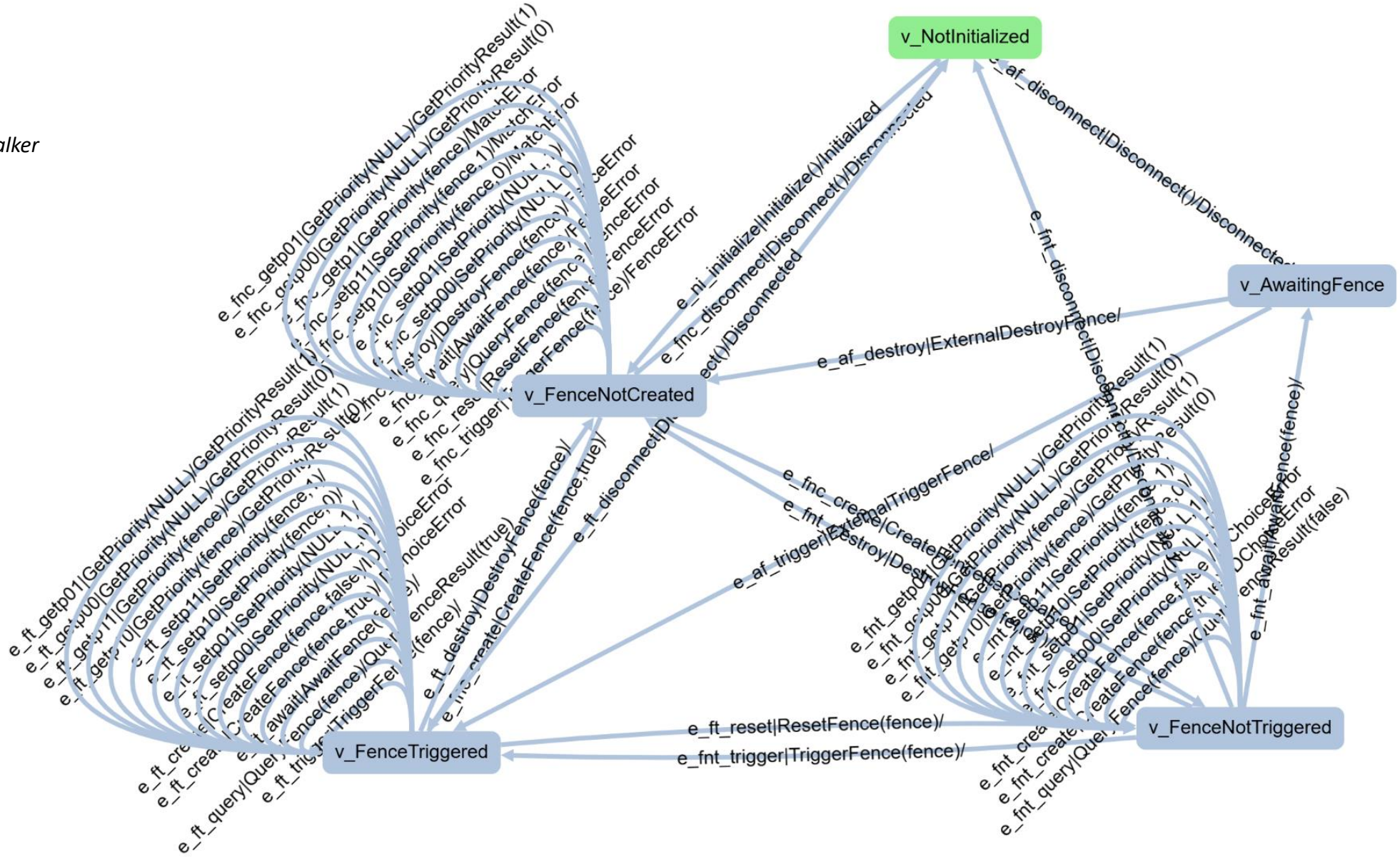
- SUT (System Under Test): X.Org Server, XSync Extension Library 3.1
  - Tested functionalities:
    - FENCE client-resource and its operations
    - GetPriority() and SetPriority() requests
      - The input symbols are X requests
      - The output symbols consist of Errors, regular request return values, and the empty symbol
- Testing:
  - EFSM model has been created with GraphWalker Studio
  - EFSM → FSM conversation and test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in C++17 using the low-level xcb X protocol library

Appendix D: Example homework L. 4/2

Spring 2024/2025

Zsolt Gyarmati: X Synchronization Extension Library

EFSM model in GraphWalker



HW ToC

Appendix D: Example homework L. 4/3

Spring 2024/2025

Zsolt Gyarmati: X Synchronization Extension Library

An example for adaptation code (input processing):

```
// ...
} else if (inputStr == "CreateFence(fence,false)") {
  if (cmain)
    x11output = cmain.checkRequest(xcb_sync_create_fence_checked(cmain, cmain.getRootWindow(), resFence, false));
} else if (inputStr == "CreateFence(fence,true)") {
  if (cmain)
    x11output = cmain.checkRequest(xcb_sync_create_fence_checked(cmain, cmain.getRootWindow(), resFence, true));
} else if (inputStr == "DestroyFence(fence)") {
  if (cmain)
    x11output = cmain.checkRequest(xcb_sync_destroy_fence_checked(cmain, resFence));
}
// ...
```

An example for test suite execution:

```
sh-5.2$ ./xsync_glue ./test_suites/Fence-efsm_trans-R-transition-100-test_suite.json
Info: opening test suite 'Fence-efsm_trans' (method: R)
[    1 /  1153,   OK]: [in: "Initialize()"] | [out: "Initialized"] | [expected: "Initialized"]
[    2 /  1153,   OK]: [in: "CreateFence(fence,false)"] | [out: ""] | [expected: ""]
...
[ 1152 /  1153,   OK]: [in: "AwaitFence(fence)"] | [out: ""] | [expected: ""]
[ 1153 /  1153,   OK]: [in: "ExternalTriggerFence"] | [out: ""] | [expected: ""]
Total test cases passed: 1018/1153 (88.291%)
Total test-suite execution took: 1.040968 secs
```

HW ToC

Appendix D: Example homework L. 4/4

Spring 2024/2025

Zsolt Gyarmati: X Synchronization Extension Library

Results of test execution:

Test suite	Execution time	Length	PASSED	FAILED	Pass rate
Random walk	1.040607 s	1153	1018	135	88.291%
Transition Tour	0.183568 s	114	108	6	94.737%
All-Transition-State	0.948166 s	530	506	24	95.472%
All-Transition-Transition	6.375579 s	10783	10771	12	99.889%
N-switch (N=1)	2.057138 s	1678	1575	103	93.862%

So why did all of the tests fail? Well, if we take a closer look at which test cases failed, we might have a clue:

```
...
[  599 /  1153,   OK]: [in: "SetPriority(NULL,1)"] | [out: ""] | [expected: ""]
[  600 /  1153, FAIL]: [in: "GetPriority(fence)"] | [out: "ValueError"] | [expected: "MatchError"]
[  601 /  1153, FAIL]: [in: "SetPriority(fence,1)"] | [out: "ValueError"] | [expected: "MatchError"]
[  602 /  1153,   OK]: [in: "QueryFence(fence)"] | [out: "FenceError"] | [expected: "FenceError"]
...
```

As it turns out, xorg-server deviates from the specification. The following is a direct quote from the official specification of the XSync protocol regarding both `SetPriority` and `GetPriority`:

> [...] A Match error is generated if client-resource is not None and does not name an existing resource in the server. [...]

In our case, the fence didn't exist yet, hence the `fence` id didn't name an *existing resource*. Apparently, instead of a `MatchError` the implementation instead returns a `ValueError` in this case.

HW ToC

Appendix D: Example homework LI. 3/1

Bence Vince Daka: Discord Web application

- SUT (System Under Test): Discord instant messaging and VoIP social platform
  - Tested functionalities:
    - User authentication (login/logout)
    - Navigation between Home, Server, Public Channel, Admin Channel, and DM views
    - Sending messages (public channels, direct messages)
    - Managing user status (Online, Do Not Disturb)
    - Role operations (create, assign, delete) and related permission logic
    - Channel management (public/admin creation and deletion)

- Testing:
  - EFSM model has been created with GraphWalker Studio
  - EFSM → FSM conversion and test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
      - using Discord API
      - Discord Web Application accessed through Selenium only used for a small illustrative scenario

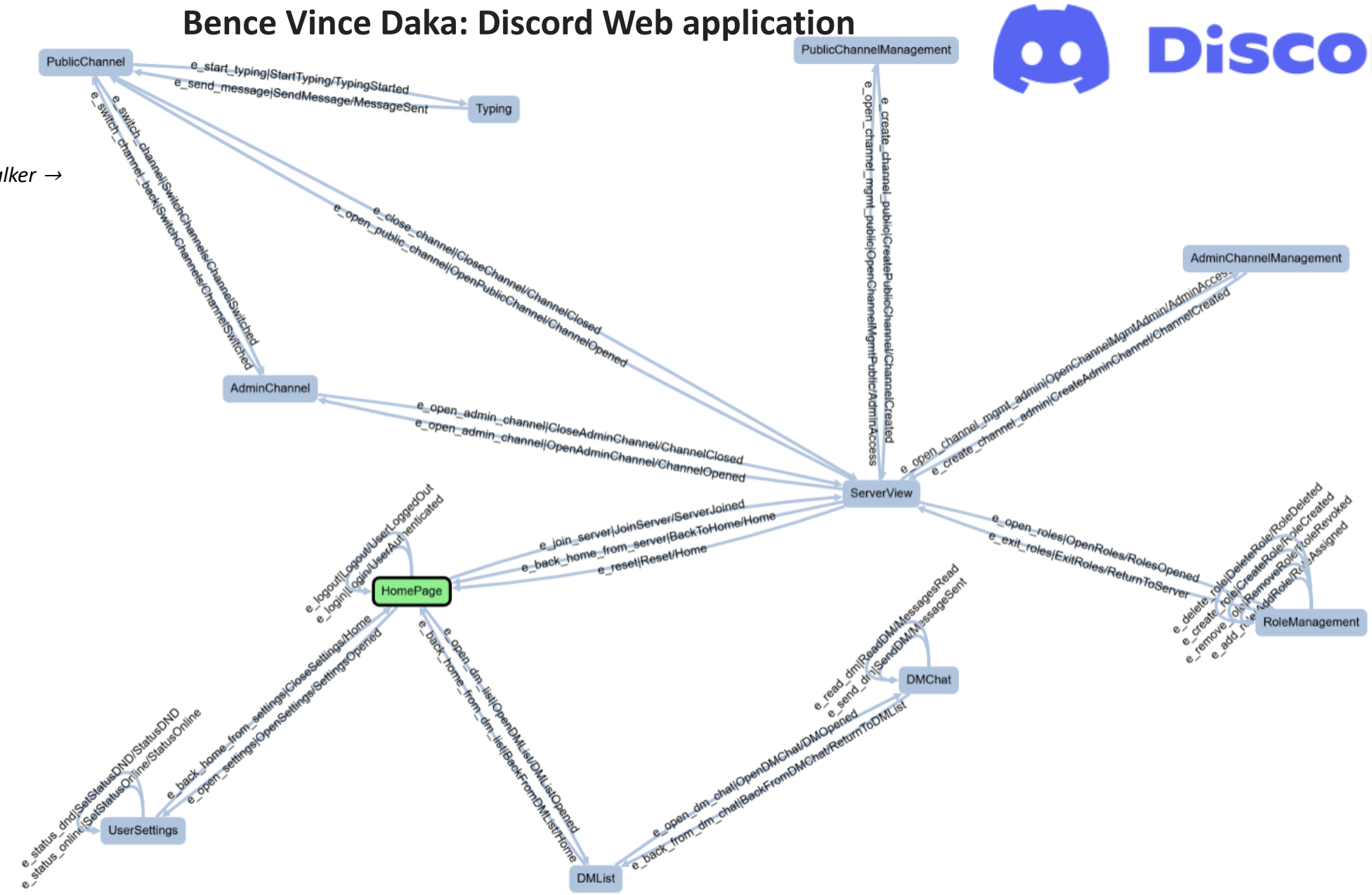
VoIP: Voice over IP

Appendix D: Example homework LI. 3/2

Fall 2025/2026

Bence Vince Daka: Discord Web application

EFSM model in GraphWalker →



HW ToC

Appendix D: Example homework LI. 3/3

Fall 2025/2026

Bence Vince Daka: Discord Web application

Adaptation code example for API testing:

```python
async def SetOnline(bot):
    state["status"] = "ONLINE"

    await guarded_call(
        bot.change_presence(status=discord.Status.online),
        action="change_presence(ONLINE)",
        category="presence",
    )

    logging.info("Status set to ONLINE.")
    return "StatusOnline"


async def SendDM(bot):
    guild = bot.get_guild(TEST_GUILD_ID)
    if guild is None:
        raise RuntimeError("Guild not found.")

    for member in guild.members[:3]:
        if runtime_state["simulation_mode"]:
            await smart_pause("send")
            logging.info(f"Simulated DM to {member.name}.")
            continue

        try:
            await guarded_call(
                member.send("Automated DM test message."),
                action=f"member.send({member.name})",
                category="send",
            )
            logging.info(f"DM sent to {member.name}")
        except Exception as e:
            logging.warning(f"Could not DM {member.name}: {e!r}")

    state["dm_unread_count"] = 1
    return "MessageSent"
```

Adaptation code example for GUI testing with Selenium:

```python
def Logout(current_state="PublicChannel"):
    """Logout transition with confirmation popup."""
    try:
        settings_btn = wait.until(EC.element_to_be_clickable(
            (By.XPATH, "//button[@aria-label='User Settings']")))
        settings_btn.click()
        pause(2)

        logout_btn = wait.until(EC.element_to_be_clickable((
            By.XPATH, "//div[@aria-label='Log Out' and @role='tab']"
        )))
        driver.execute_script("arguments[0].scrollIntoView(true);", logout_btn)
        pause(0.5)
        logout_btn.click()
        logging.info("Logout button clicked, waiting for confirmation...")

        confirm_btn = wait.until(EC.element_to_be_clickable((
            By.XPATH, "//button[contains(., 'Log Out') or contains(., 'log out')]"
        )))
        pause(0.5)
        confirm_btn.click()
        logging.info("Logged out successfully (confirmed popup).")

        wait.until(EC.presence_of_element_located((By.NAME, "email")))
        return "HomePage"

    except Exception as e:
        logging.error(f"Logout failed at state={current_state}: {e}")
        raise RuntimeError(f"Logout transition failed: {e}")

def OpenAdminChannel(current_state="ServerView"):
    """Open admin-channel."""
    try:
        channel = wait.until(EC.element_to_be_clickable(
            (By.XPATH, "//div[contains(text(), 'admin-channel')]")
        ))
        driver.execute_script("arguments[0].scrollIntoView(true);", channel)
        pause(0.5)
        channel.click()

        logging.info("Opened 'admin-channel'.")
        return "AdminChannel"
```

An example screenshot for API test execution:

```
20:41:14 [INFO] Exited role management.
20:41:14 [INFO] [OK] ExitRoles → ReturnToServer
20:41:14 [INFO] [9275/9277] Executing OpenPublicChannel
20:41:14 [INFO] Opened public channel view.
20:41:14 [INFO] [OK] OpenPublicChannel → ChannelOpened
20:41:14 [INFO] [9276/9277] Executing SwitchToAdmin
20:41:14 [INFO] Switched to admin channel (view).
20:41:14 [INFO] [OK] SwitchToAdmin → ChannelSwitched
20:41:14 [INFO] [9277/9277] Executing CloseAdminChannel
20:41:14 [INFO] Closed admin channel (view only).
20:41:14 [INFO] [OK] CloseAdminChannel → ChannelClosed
20:41:14 [INFO]
=== Test suite finished ===
20:41:14 [INFO]    Successful: 9277 / 9277
20:41:14 [INFO]    Errors: 0
20:41:14 [INFO]
```

Summary for API test execution results:

Test Suite	Success Rate	Input Count	Elapsed Time
Random Walk 50% (State)	99.8%	1926	379s
Random Walk 50% (Transition)	100%	3048	612s
Random Walk 100% (State)	99.9%	104,906	24,451s
Random Walk 100% (Transition)	99.9%	152,342	32,993s
All-State (AS)	100%	669	144s
Transition Tour (TT)	100%	1,360	261s
All-Transition-State (ATS)	100%	7,248	1,432s
n-Switch Coverage	100%	6,778	1,387s
HSI	100%	9,277	4,039s
H	100%	9,277	3,931s

HW ToC

Appendix D: Example homework LII. 3/1

Henriette Jurca: Jira

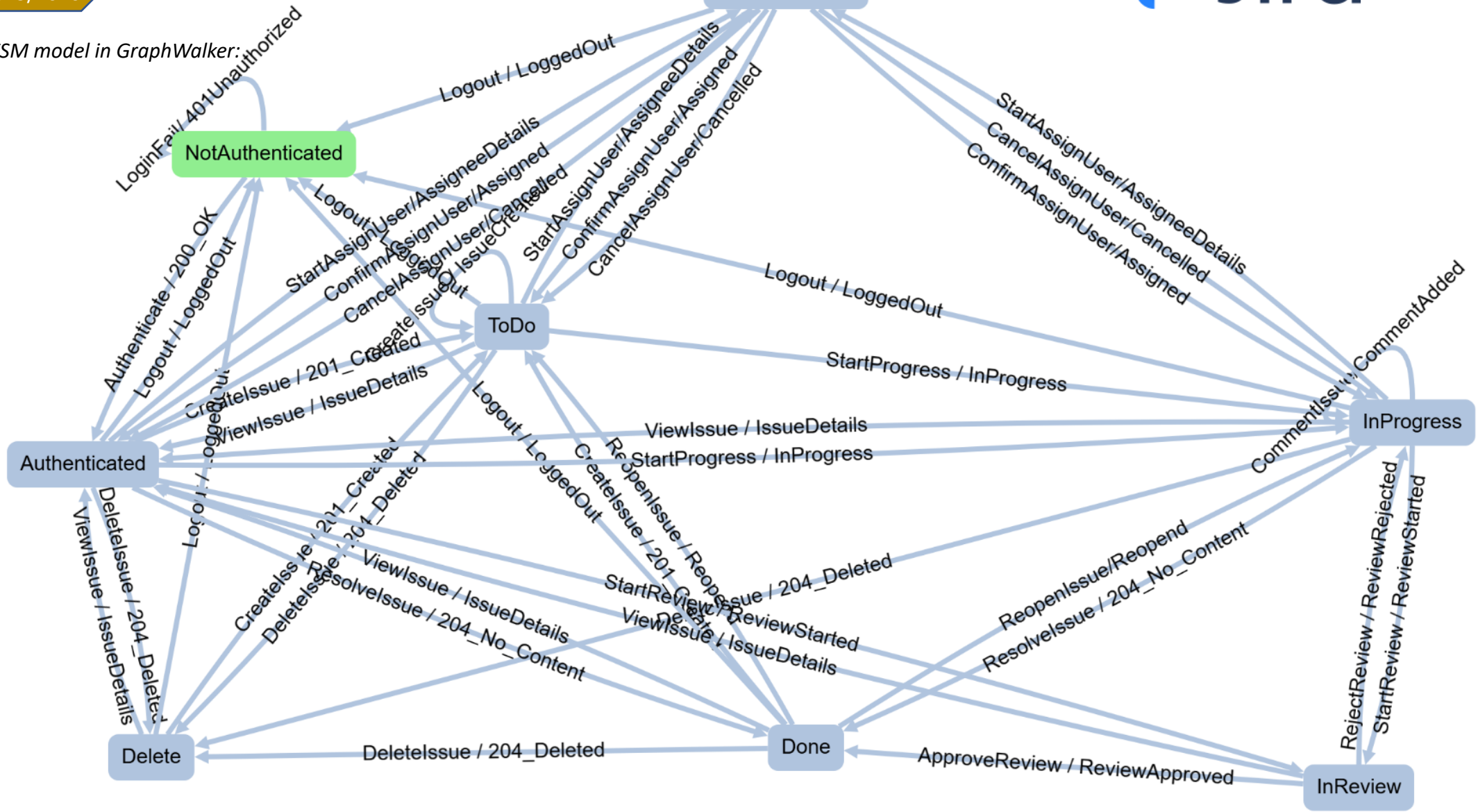
- SUT (System Under Test): Jira issue tracking system
  - Tested functionalities:
  - 1. Authentication and Session Handling:
    - Login success, Login failure, Logout
  - 2. Issue Lifecycle Management:
    - Create Issue, View Issue, Start Progress, Start Review, Approve Review, Reject Review, Resolve Issue, Reopen Issue, Delete Issue
  - 3. Issue Metadata Operations:
    - Assign User, Remove Assignee, Comment Issue, Get Assignee
  - 4. System State Awareness *(EFSM only)*:
    - Issue count tracking, Guard conditions, Automatic issue creation
  - 5. Negative and Edge-Case Handling
    - Unauthorized actions (401 or 403), Invalid transitions, Operations on deleted/non-existent issue, Simulated operations
- Testing:
  - EFSM model has been created with GraphWalker Studio
  - EFSM → FSM conversion and test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python using JIRA API

Appendix D: Example homework LII. 3/2

Fall 2025/2026

Henriette Jurca: Jira

EFSM model in GraphWalker:



HW ToC

Appendix D: Example homework LII. 3/3

Fall 2025/2026

Henriette Jurca: Jira

Adaptation code examples:

```python
import time
import requests

issues = []  # Nyilvántartás az issue-król

# SEGÉDFÜGGVÉNY
def print_step(name, status, details=""):
    print(f"{name:<25} -> {status} {details}")

def check_status(res, expected):
    if str(res.status_code) not in expected:
        raise Exception(f"Váratlan válasz: {res.status_code} != {expected}\n{res.text}")


# API FUNKCIÓK
def login_success(issue_id=None):
    if issue_id is None and len(issues) > 0:
        issue_id = issues[0]
    res = requests.get(f"{BASE_URL}/myself", auth=AUTH)
    check_status(res, "200")
    # print_step("Login success", res.status_code)
    return "login_success", issue_id

def login_fail(issue_id=None):
    try:
        res = requests.get(f"{BASE_URL}/myself", auth=("wrong_email", "wrong_token"))
        if res.status_code == 401:
            # print_step("login_fail", "401", "Simulated bad login")
            output = "401_Unauthorized"
        else:
            # print_step("login_fail", res.status_code, "Unexpected response")
            output = f"Unexpected_{res.status_code}"
    except Exception as e:
        # print_step("login_fail", "Error", str(e))
        output = "Error"
    return output, issue_id

def create_issue(issue_id=None):
    payload = {
        "fields": {
            "project": {"key": PROJECT_KEY},
            "summary": "Automated Test Issue",
            "issuetype": {"name": "Task"}
        }
    }
    res = requests.post(f"{BASE_URL}/issue", json=payload, auth=AUTH)
    check_status(res, "201")
    new_id = res.json()["id"]
    issues.append(new_id)
    # print_step("Create issue", res.status_code, f"ID={new_id}")
    return "201_Created", new_id
```

```python
def start_progress(issue_id):
    if issue_id is None and len(issues) > 0:
        issue_id = issues[0]
    if issue_id is None:
        # print_step("Start progress", "Failed", "issue is None")
        return "StartProgress_failed", issue_id
    transition_id = "11"
    res = requests.post(f"{BASE_URL}/issue/{issue_id}/transitions",
                        json={"transition": {"id": transition_id}}, auth=AUTH)
    check_status(res, "204")
    # print_step("Start progress", res.status_code)
    return "in_progress", issue_id

def resolve_issue(issue_id):
    if issue_id is None and len(issues) > 0:
        issue_id = issues[0]
    if issue_id is None:
        # print_step("Resolve issue", "Failed", "issue is None")
        return "ResolveIssue_failed", issue_id
    transition_id = "31"
    res = requests.post(f"{BASE_URL}/issue/{issue_id}/transitions",
                        json={"transition": {"id": transition_id}}, auth=AUTH)
    check_status(res, "204")
    # print_step("Resolve issue", res.status_code)
    return "204_No_Content", issue_id

def reopen_issue(issue_id):
    if issue_id is None and len(issues) > 0:
        issue_id = issues[0]
    if issue_id is None:
        # print_step("Reopen issue", "Failed", "issue is None")
        return "ReopenIssue_failed", issue_id
    res = requests.get(f"{BASE_URL}/issue/{issue_id}/transitions",
                       auth=AUTH, headers={"Accept": "application/json"})
    transitions = res.json().get("transitions", [])
    reopen_id = next((t["id"] for t in transitions if t["name"].lower() == "reopen"), None)
    if reopen_id:
        res2 = requests.post(f"{BASE_URL}/issue/{issue_id}/transitions",
                             json={"transition": {"id": reopen_id}}, auth=AUTH)
        if res2.status_code == 204:
            # print_step("Reopen issue", res2.status_code, f"Issue ID={issue_id} -> To Do")
            output = "200 Issue reopened successfully"
        else:
            # print_step("Reopen issue", "Error", f"Status code={res2.status_code}")
            output = "ReopenIssue_failed"
    else:
        # print_step("Reopen issue", "Simulated", f"Issue ID={issue_id}, Status=To Do")
        output = "200 Issue reopened successfully"
        # time.sleep(0.5)
    return output, issue_id
```

Summary for test execution results:

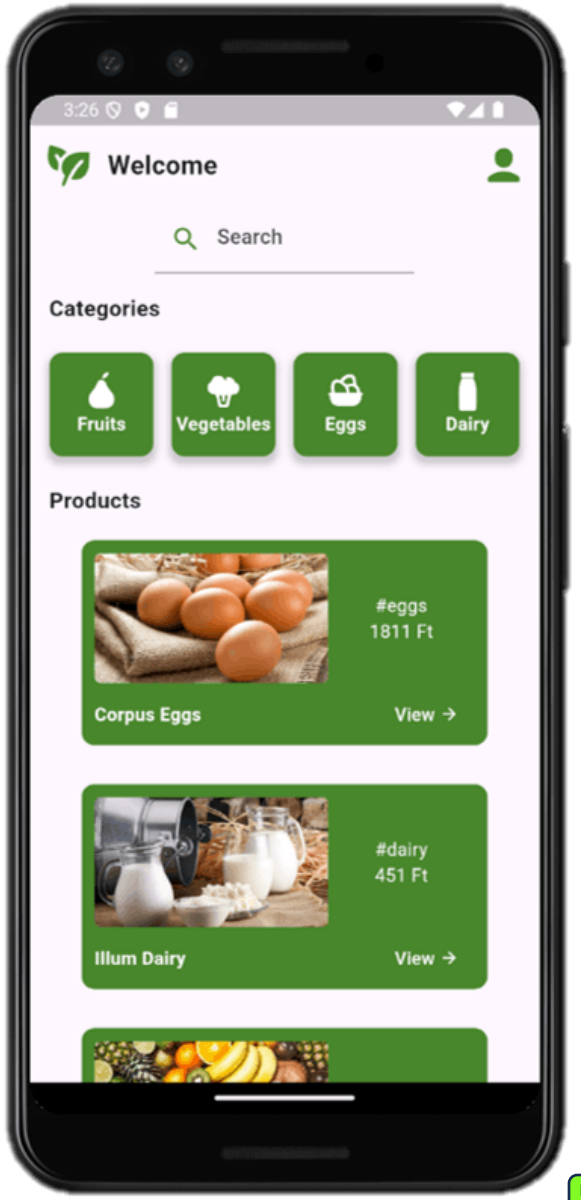
Test Suite File	Total Steps	Successful	Failed	Runtime (sec)
Jira-efsm_trans-AS-test_suite.json	36	36	0	25.943
Jira-efsm_trans-ATS0-test_suite.json	567	567	0	292.267
Jira-efsm_trans-DCP-test_suite.json	166	166	0	79.248
Jira-efsm_trans-TT-test_suite.json	166	166	0	91.726
Jira-efsm_trans-NS-1-test_suite.json	1134	1134	0	663.090

HW ToC

Appendix D: Example homework LIII. 4/1

Fall 2025/2026

Mónika Nick: Farmer2Customer market



- SUT (System Under Test): Farmer2Customer market Web application
  - Tested functionalities:
    - Registration, login
    - Consumers:
      - Browsing products
      - Creating and managing orders: place orders, and track their approval status
    - Producers:
      - Review and either approve or decline incoming product orders
    - Handling user role upgrades (from consumer to producer)
  - Limitations:
    - Although a producer could also act as a consumer, this scenario is not included
    - User upgrade requests are processed by an admin, who approves or declines them.
    - A consumer can place one order at a time
- Testing:
  - EFSM and FSM models have been created with GraphWalker Studio
  - Test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in Python
    - API testing

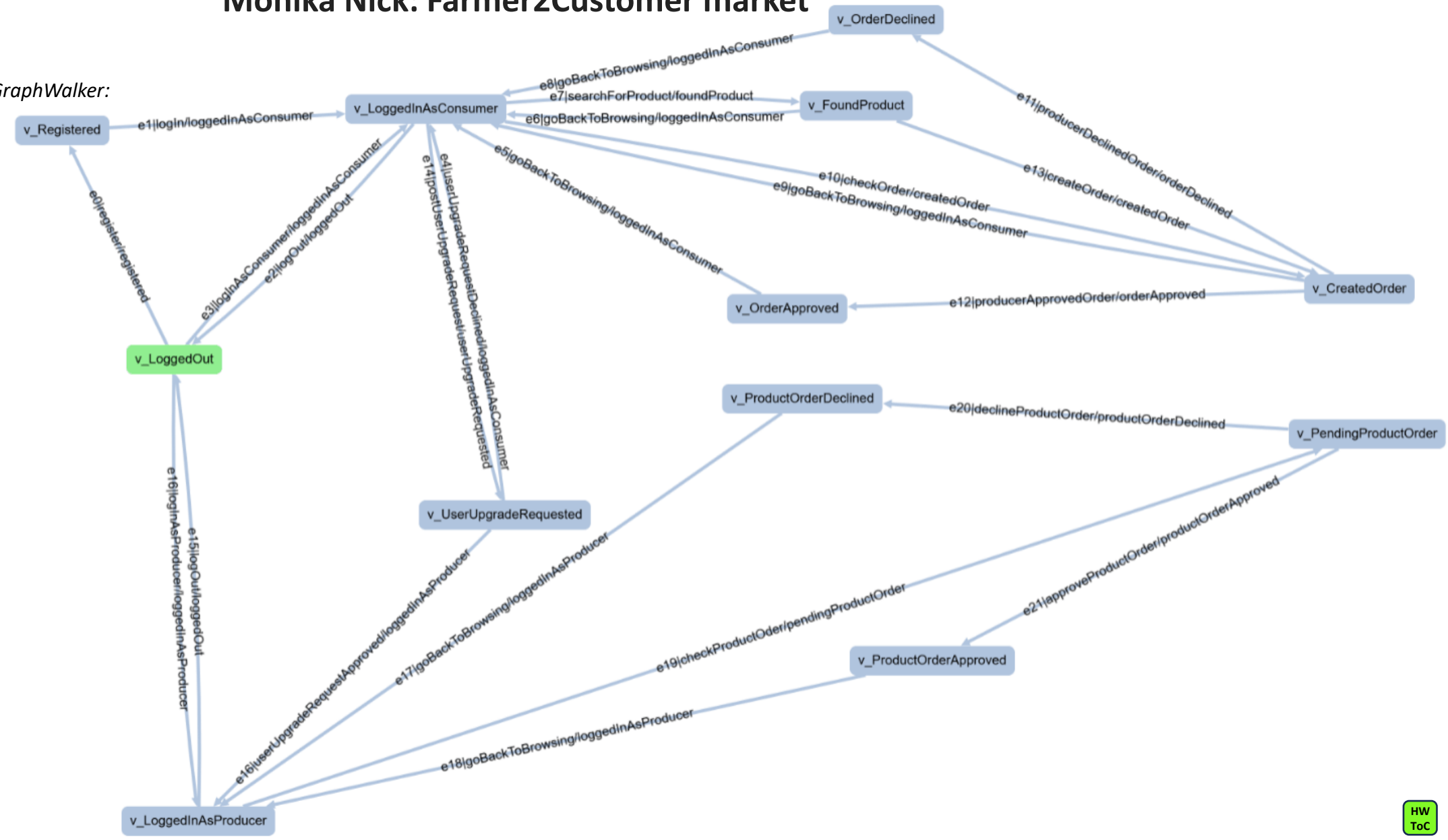
HW ToC

Appendix D: Example homework LIII. 4/2

Fall 2025/2026

Mónika Nick: Farmer2Customer market

EFSM model in GraphWalker:



HW ToC

Appendix D: Example homework LIII. 4/3

Fall 2025/2026

Mónika Nick: Farmer2Customer market

Adaptation code examples:

```
def foundProduct(self, response):
    if response is None:
        raise AssertionError("Response is None")

    if response.status_code != 200:
        raise AssertionError(f"Expected 200, got {response.status_code}")

    body = response.json()
    expected = self.client.DATA["getProduct"]["expectedResponse"]

    def check(expected, actual, path=""):
        for key, value in expected.items():
            if key not in actual:
                raise AssertionError(f"Missing '{path + key}' in response")
            if isinstance(value, dict):
                check(value, actual[key], path + key + ".")
            else:
                if actual[key] != value:
                    raise AssertionError(f"Incorrect value for '{path + key}': expected '{value}', got '{actual[key]}'")

    check(expected, body)

def userUpgradeRequested(self, response):
    if response is None:
        raise AssertionError("Response is None")

    body = response.json()
    cfg = self.client.DATA["userUpgradeRequest"]

    if "message" in body:
        expected_msg = cfg["message"]
        if body["message"] != expected_msg:
            raise AssertionError(f"Unexpected message: {body['message']}")
        return

    req = cfg["body"]

    if body.get("description") != req.get("description"):
        raise AssertionError("Description mismatch")

    if body.get("contact") != req.get("contact"):
        raise AssertionError("Contact mismatch")

    if "id" not in body:
        raise AssertionError("No 'id' in successful upgrade request response")
```

```
def createdOrder(self, response):
    if response is None:
        raise AssertionError("Response is None")

    if response.status_code not in (200, 201):
        raise AssertionError(f"Expected 200 or 201, got {response.status_code}")

    body = response.json()
    product_id = body.get("product", {}).get("id")
    order_id = body.get("id")
    cfg = self.client.DATA["createOrder"]

    expected_product_id = str(self.client.curProductId)
    expected_quantity = cfg["body"]["quantity"]

    if str(product_id) != expected_product_id and order_id != self.client.curOrderId:
        raise AssertionError(f"Not expected product or order Id: {product_id, order_id}")

    if body.get("quantity") != expected_quantity:
        raise AssertionError(f"Expected quantity {expected_quantity}, got {body.get('quantity')}")

    self.client.curOrderId = order_id

def pendingProductOrder(self, response):
    if response is None:
        raise AssertionError("Response is None")

    if response.status_code != 200:
        raise AssertionError(f"Expected 200, got {response.status_code}")

    body = response.json()
    order_id = body.get("id")

    if order_id != self.client.curOrderId:
        raise AssertionError(f"Expected order id {self.client.curOrderId}, got {order_id}")

def _orderStateAssert(self, response, expected_approval):
    if response is None:
        raise AssertionError("Response is None")

    if response.status_code not in (200, 201):
        raise AssertionError(f"Expected 200 or 201, got {response.status_code}")

    body = response.json()
    order_id = body.get("id")
    isOrderApproved = body.get("approved")

    if order_id != self.client.curOrderId:
        raise AssertionError(f"Not expected order Id: {order_id}")

    if isOrderApproved != expected_approval:
        raise AssertionError(f"Order not approved")
```

HW ToC

Appendix D: Example homework LIII. 4/4

Fall 2025/2026

Mónika Nick: Farmer2Customer market

An example execution of a test suite:

farmertoconsumer_model-efsm_trans-TT-test_suite.json | loader.p

Nick_Monika_bead > test_suites > farmertoconsumer_model-efsm_trans-TT-test_su

```
2       "test_suite": {
9           "input_list": [
46              "register",
47              "logIn",
48              "postUserUpgradeRequest",
49              "userUpgradeRequestApproved",
50              "logOut",
51              "logInAsConsumer",
52              "searchForProduct",
53              "goBackToBrowsing",
54              "logOut",
55              "logInAsProducer",
56              "checkProductOder",
57              "declineProductOrder",
58              "goBackToBrowsing",
59              "logOut",
60              "logInAsConsumer",
61              "postUserUpgradeRequest",
62              "userUpgradeRequestApproved",
63              "logOut",
64              "register",
65              "logIn",
66              "logOut",
67              "logInAsProducer",
68              "logOut",
69              "logInAsConsumer",
70              "postUserUpgradeRequest",
71              "userUpgradeRequestDeclined",
72              "searchForProduct",
73              "goBackToBrowsing",
74              "checkOrder",
75              "producerDeclinedOrder",
76              "goBackToBrowsing",
77              "postUserUpgradeRequest",
78              "userUpgradeRequestApproved",
79              "logOut",
80              "logInAsConsumer",
81              "searchForProduct",
82              "goBackToBrowsing",
83              "logOut",
84              "logInAsProducer",
85              "logOut"
```

TTtest.txt

testing > test_results > TTtest.txt

```
113 Running test 56 → input: logOut, expected output: loggedOut
114 ✓ OK
115 Running test 57 → input: logInAsProducer, expected output: loggedInAsProducer
116 ✓ OK
117 Running test 58 → input: logOut, expected output: loggedOut
118 ✓ OK
119 Running test 59 → input: logInAsConsumer, expected output: loggedInAsConsumer
120 ✓ OK
121 Running test 60 → input: postUserUpgradeRequest, expected output: userUpgradeRequested
122 ✓ OK
123 Running test 61 → input: userUpgradeRequestDeclined, expected output: loggedInAsConsumer
124 ✓ OK
125 Running test 62 → input: searchForProduct, expected output: foundProduct
126 ✓ OK
127 Running test 63 → input: goBackToBrowsing, expected output: loggedInAsConsumer
128 ✓ OK
129 Running test 64 → input: checkOrder, expected output: createdOrder
130 ✓ OK
131 Running test 65 → input: producerDeclinedOrder, expected output: orderDeclined
132 ✓ OK
133 Running test 66 → input: goBackToBrowsing, expected output: loggedInAsConsumer
134 ✓ OK
135 Running test 67 → input: postUserUpgradeRequest, expected output: userUpgradeRequested
136 ✓ OK
137 Running test 68 → input: userUpgradeRequestApproved, expected output: loggedInAsProducer
138 ✓ OK
139 Running test 69 → input: logOut, expected output: loggedOut
140 ✓ OK
141 Running test 70 → input: logInAsConsumer, expected output: loggedInAsConsumer
142 ✓ OK
143 Running test 71 → input: searchForProduct, expected output: foundProduct
144 ✓ OK
145 Running test 72 → input: goBackToBrowsing, expected output: loggedInAsConsumer
146 ✓ OK
147 Running test 73 → input: logOut, expected output: loggedOut
148 ✓ OK
149 Running test 74 → input: logInAsProducer, expected output: loggedInAsProducer
150 ✓ OK
151 Running test 75 → input: logOut, expected output: loggedOut
152 ✓ OK
153 ALL: 76, SUCCESS: 76, FAILED: 0
154 Execution time: 0:00:03.346014
```

farmertoconsumer_model-efsm_trans-TT-test_suite.json

Nick_Monika_bead > test_suites > farmertoconsumer_model

```
2       "test_suite": {
87          "output_list": [
124             "registered",
125             "loggedInAsConsumer",
126             "userUpgradeRequested",
127             "loggedInAsProducer",
128             "loggedOut",
129             "loggedInAsConsumer",
130             "foundProduct",
131             "loggedInAsConsumer",
132             "loggedOut",
133             "loggedInAsProducer",
134             "pendingProductOrder",
135             "productOrderDeclined",
136             "loggedInAsProducer",
137             "loggedOut",
138             "loggedInAsConsumer",
139             "userUpgradeRequested",
140             "loggedInAsProducer",
141             "loggedOut",
142             "registered",
143             "loggedInAsConsumer",
144             "loggedOut",
145             "loggedInAsProducer",
146             "loggedOut",
147             "loggedInAsConsumer",
148             "userUpgradeRequested",
149             "loggedInAsConsumer",
150             "foundProduct",
151             "loggedInAsConsumer",
152             "createdOrder",
153             "orderDeclined",
154             "loggedInAsConsumer",
155             "userUpgradeRequested",
156             "loggedInAsProducer",
157             "loggedOut",
158             "loggedInAsConsumer",
159             "foundProduct",
160             "loggedInAsConsumer",
161             "loggedOut",
162             "loggedInAsProducer",
163             "loggedOut"
```

HW ToC

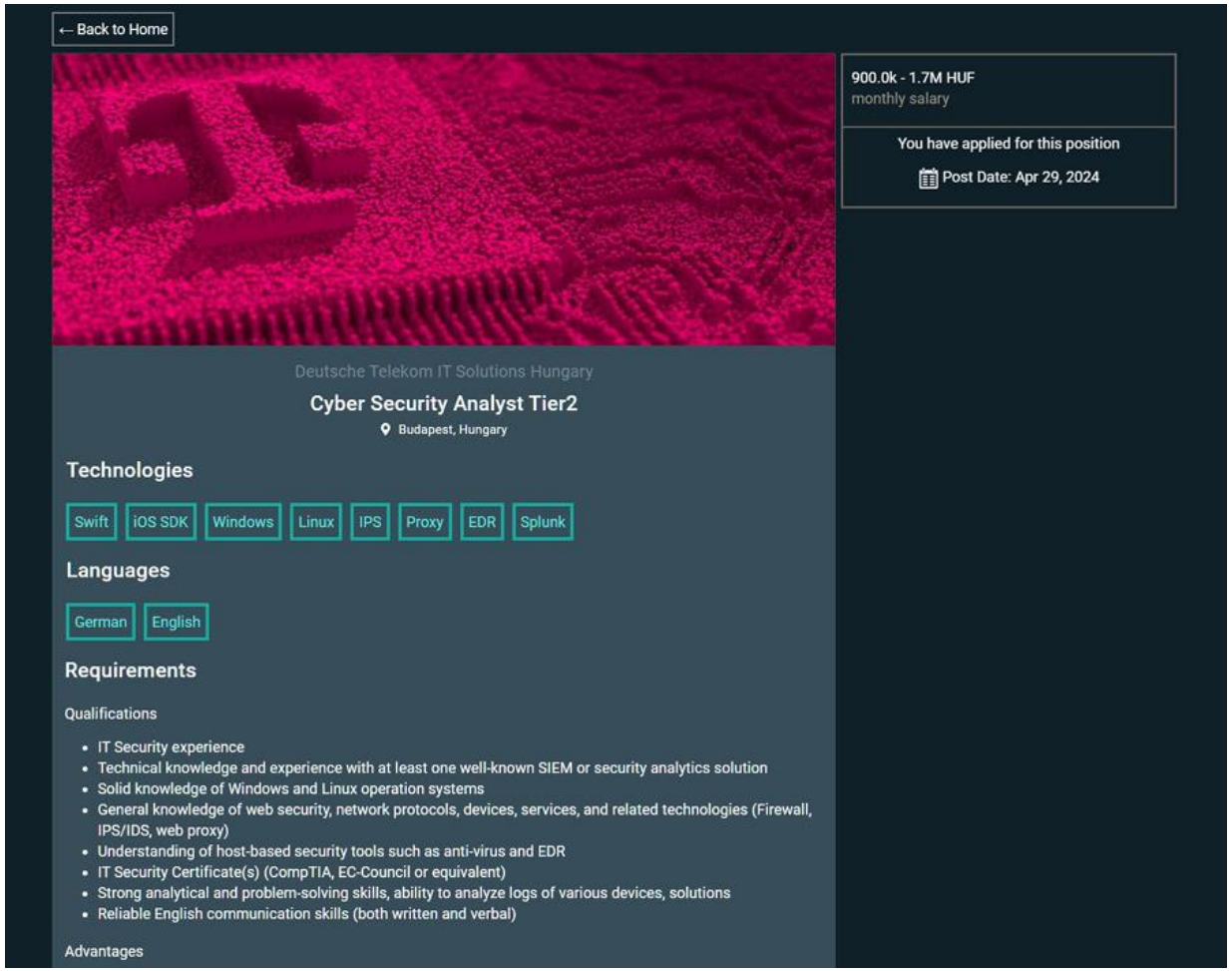
Appendix D: Example homework LIV. 3/1

Fall 2025/2026

Marcell Újvári: Job Hunting platform

- SUT (System Under Test): Job Hunting Web application
  - Tested functionalities:
    - User Authentication
    - User Registration
    - Job Browsing and Filtering
    - Job Application Flow

- Testing:
  - EFSM model has been created with GraphWalker Studio
  - EFSM → FSM conversion and test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in JavaScript
      - GUI testing with Playwright



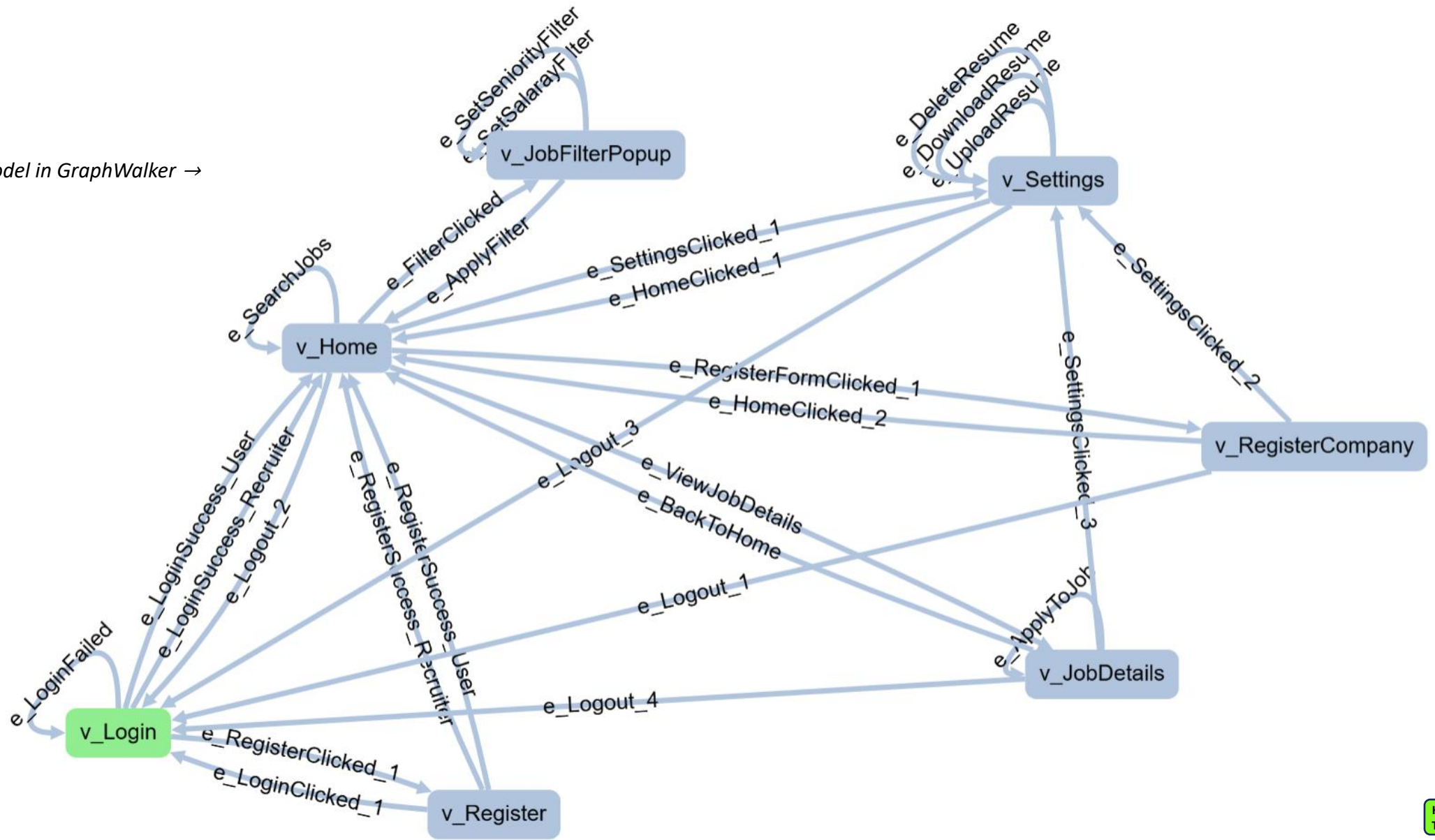
HW ToC

Appendix D: Example homework LIV. 3/2

Fall 2025/2026

Marcell Újvári: Job Hunting platform

EFSM model in GraphWalker →



HW ToC

Appendix D: Example homework LIV. 3/3

Fall 2025/2026

Marcell Újvári: Job Hunting platform

Adaptation code examples:

Test runner:

```
test.describe("Mode based test execution", () => {
  test.use({ storageState: { cookies: [], origins: [] } }); – 0ms (ran 6×)
  test.setTimeout(5 * 60 * 1000000);
  if (allTestPlanFiles.length === 0) {
    test("No test plans found", () => {
      console.log(`No .json files found in ${PLAN_DIRECTORY}.`);
      test.skip(true, "No test plans found in directory.");
    });
    return;
  }

  for (const planFilename of allTestPlanFiles) {
    test(`execute plan: ${planFilename}`, async ({ page }) => {

      const filePath = path.join(PLAN_DIRECTORY, planFilename);
      const testPlanRaw = JSON.parse(fs.readFileSync(filePath, "utf-8"));
      const { input_list, output_list } = testPlanRaw.test_suite;

      await page.goto("/"); – 499ms (ran 6×)

      for (let i = 0; i < input_list.length; i++) {
        const actionName = input_list[i];
        const expectedStateName = output_list[i];

        await test.step(`Step ${i + 1}: [${actionName}] -> Expect [${expectedStateName}]`, async () => {
          console.log("INPUT: " + actionName);
          await actionMap[actionName](page);
          console.log("OUTPUT: " + expectedStateName);
          await stateMap[expectedStateName](page);
        });
      }
    });
  }
});
```

Mapper:

```
export const actionMap: Record<string, (page: Page) => Promise<void>> = {
  login_user: async page => {
    await login_user(page);
  },
  login_recruiter: async page => {
    await login_recruiter(page);
  },
  login: async page => {
    await loginInvalid(page);
  },
  login_pressed: async page => {
    await loginInvalid(page);
  },
  logout_clicked: async page => {
    await logout(page);
  },
  register_pressed: async page => {
    await goToRegisterPage(page);
  },
  register_user: async page => {
    await registerUser(page);
  },
  register_recruiter: async page => {
    await registerRecruiter(page);
  },
  home_pressed: async page => {
    await navigateTo(page, "/home");
  },
  home_clicked: async page => {
    await navigateTo(page, "/home");
  },
```

Playwright Actions for GUI testing:

home.ts

e2e > tests > home.ts > filterSeniority

```
3   export const searchJobs = async (page: Page) => {
4     expect(page).toHaveURL("/home");
5
6     const searchBox = page.getByRole("textbox", {
7       name: "Enter your search term",
8     });
9
10    await searchBox.fill("Junior Frontend Developer");
11  };
12
13  export const openFilter = async (page: Page) => {
14    expect(page).toHaveURL("/home");
15
16    const filterButton = page.getByRole("button", { name: "Filters" });
17
18    await filterButton.click();
19  };
20
21  export const filterSeniority = async (page: Page) => {
22    expect(page).toHaveURL("/home");
23    const filterHeading = page.getByRole("heading", { name: "Filters" });
24    await expect(filterHeading).toBeVisible();
25
26    const seniorityPicker = page.getByRole("button", { name: "Seniority" });
27    const internCheckbox = page.locator('[id="0-input"]');
28
29    if (!(await internCheckbox.isVisible())) {
30      await seniorityPicker.click();
31    }
32
33    await internCheckbox.click();
34  };
```

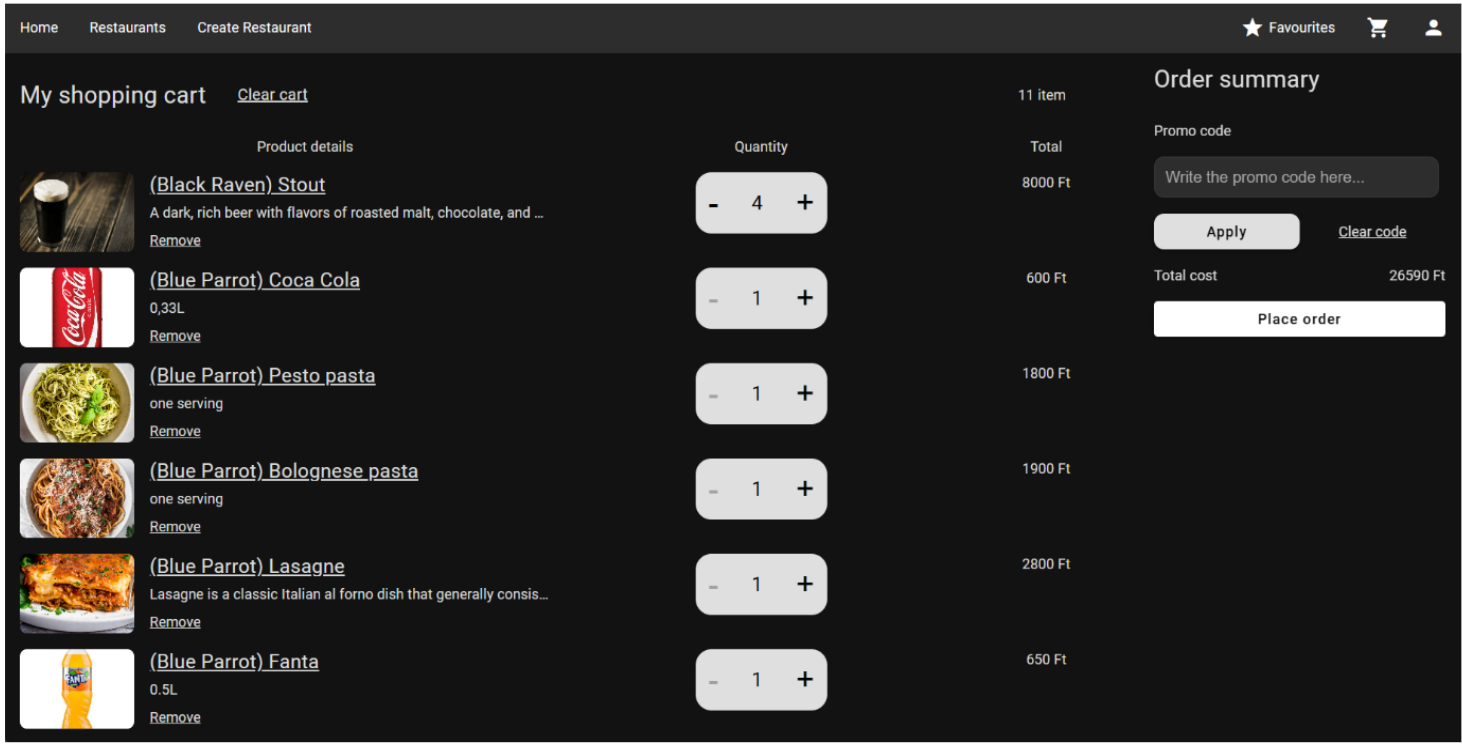
HW ToC

Appendix D: Example homework LV. 2/1

Fall 2025/2026

Levente Máté Smid: Food ordering

- SUT (System Under Test):
  Food ordering web application
  - Tested functionalities:
    - Login
    - Restaurant list page, filter
    - Restaurant detail page
    - Product detail
    - Add product to cart, place order
    - Edit profile page



- Testing:
  - EFSM model has been created with GraphWalker Studio
  - EFSM → FSM conversion and test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in C#
      - GUI testing with Playwright

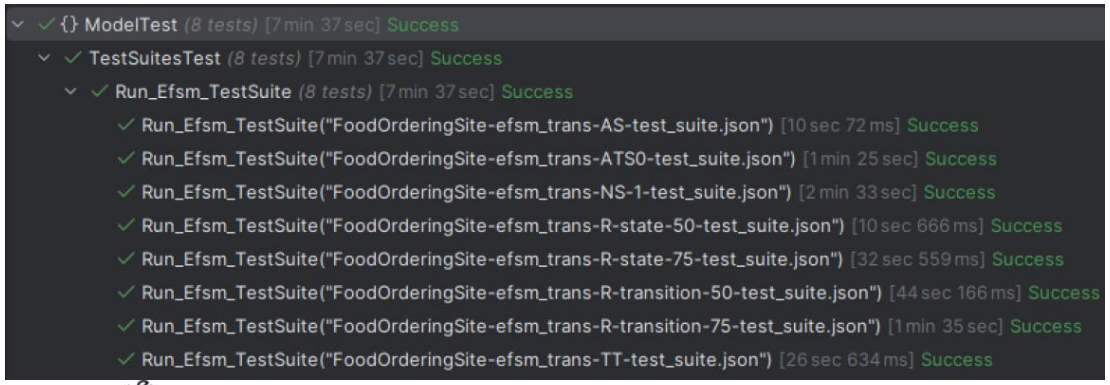
HW ToC

Appendix D: Example homework LV. 2/2

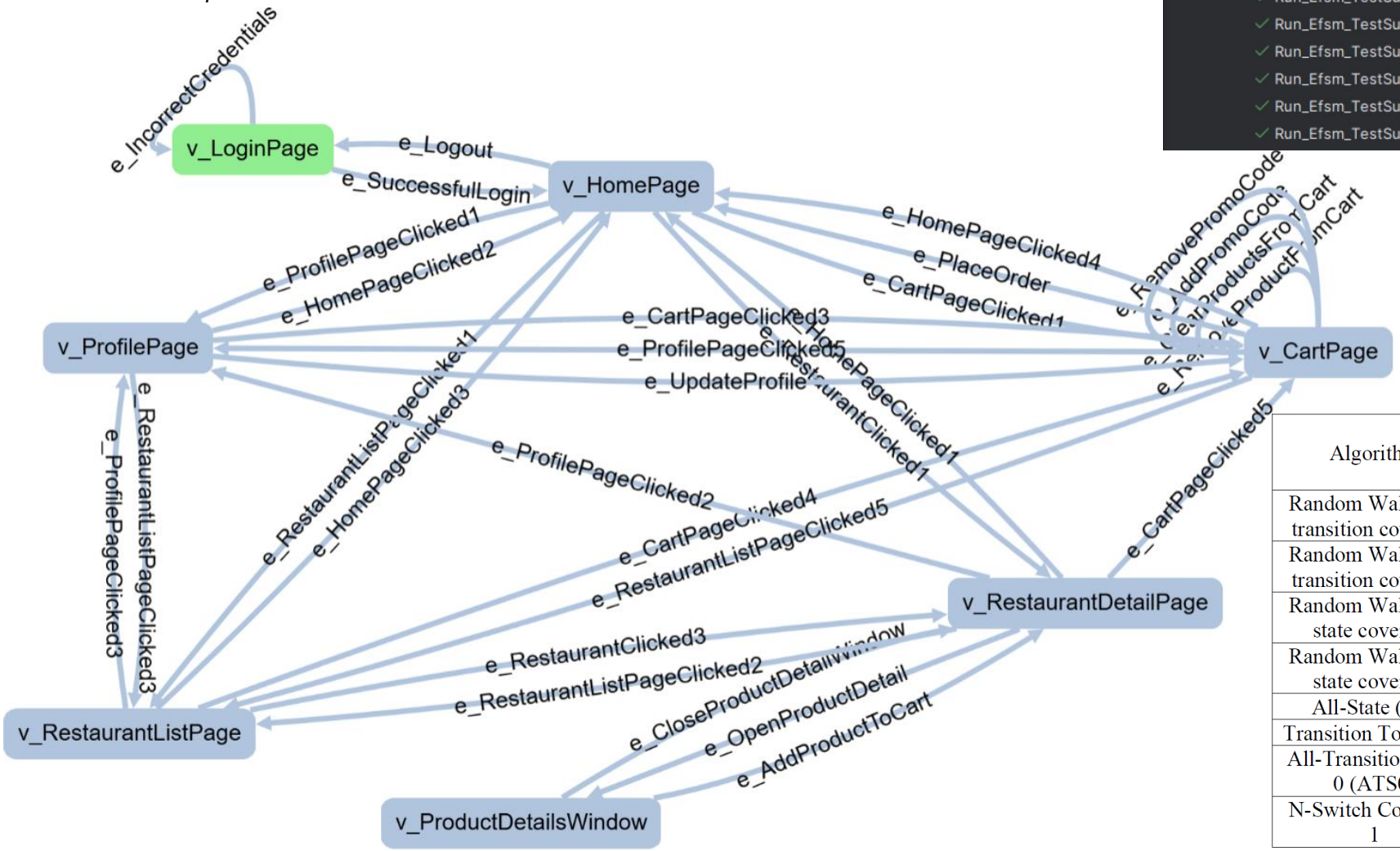
Fall 2025/2026

Levente Máté Smid: Food ordering

Test execution:



EFSM model in GraphWalker:



Summary of test executions:

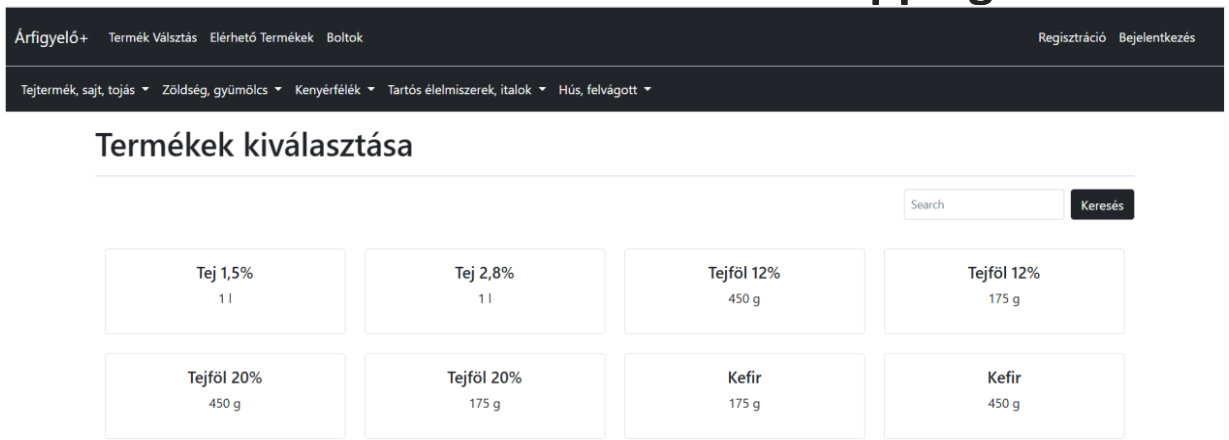
Algorithm	Test sequence length	MTR generation time	Test execution time	Result
Random Walk 75% transition coverage	447	0.0003953s	1m 35s	OK
Random Walk 50% transition coverage	231	0.0003032s	44s 166ms	OK
Random Walk 75% state coverage	216	0.0003158s	32s 559ms	OK
Random Walk 50% state coverage	34	0.0002832s	10s 666ms	OK
All-State (AS)	43	0.0013976s	10s 72ms	OK
Transition Tour (TT)	149	0.0070515s	26s 634ms	OK
All-Transition-State 0 (ATS0)	498	0.0509187s	1min 25s	OK
N-Switch Coverage 1	1010	0.0088584s	2min 33s	OK

HW ToC

Appendix D: Example homework LVI. 3/1

Fall 2025/2026

Balázs Szi-Benedek: PriceWatcher+ Shopping Assistant



- SUT (System Under Test): PriceWatcher+ Shopping Assistant ASP.NET web application
  - Tested functionalities:
    - Login
    - Filter and search product categories and subcategories
    - Select stores
    - Select general product categories (the application searches the cheapest brand)
    - Add products to

- Testing:
  - FSM model has been created with GraphWalker Studio
  - State minimization and test generation with Model 》 Test 》 Relax
  - Glue code (adaptation code):
    - Written in C#
      - GUI testing with Selenium

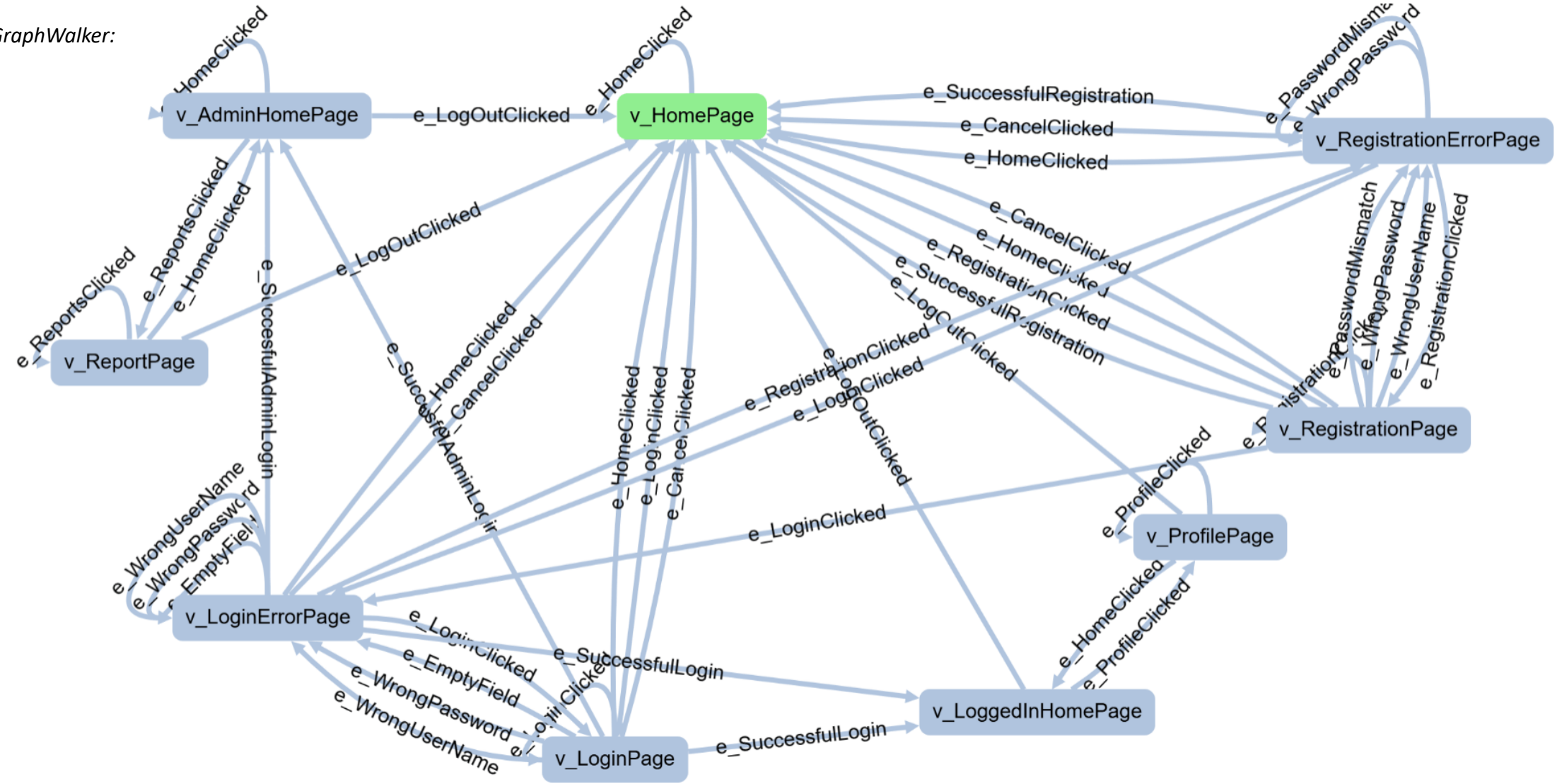
HW ToC

Appendix D: Example homework LVI. 3/2

Fall 2025/2026

Balázs Szi-Benedek: PriceWatcher+ Shopping Assistant

FSM model in GraphWalker:



HW ToC

Appendix D: Example homework LVI. 3/3

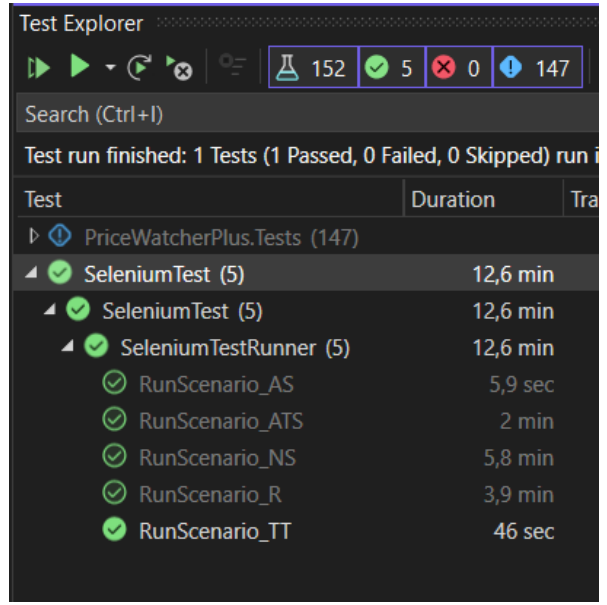
Fall 2025/2026

Balázs Szi-Benedek: PriceWatcher+ Shopping Assistant

Adaptation code example:

```
void RegistrationForm(string a)
{
    var u = _driver.FindElement(By.XPath(_xpaths["Registration username input"]));
    var mail = _driver.FindElement(By.XPath(_xpaths["Registration email input"]));
    var p1 = _driver.FindElement(By.XPath(_xpaths["Registration password input"]));
    var p2 = _driver.FindElement(By.XPath(_xpaths["Registration password again input"]));

    u.Clear(); mail.Clear(); p1.Clear(); p2.Clear();

    if (a.Contains("proper credential"))
    {
        u.SendKeys("newuser");
        mail.SendKeys("newuser@test.com");
        p1.SendKeys("User1234@");
        p2.SendKeys("User1234@");
    }
    else if (a.Contains("mismatched"))
    {
        u.SendKeys("newuser");
        mail.SendKeys("newuser@test.com");
        p1.SendKeys("User1234@");
        p2.SendKeys("User12345@");
    }
    else if (a.Contains("wrong password"))
    {
        u.SendKeys("newuser");
        mail.SendKeys("newuser@test.com");
        p1.SendKeys("wrongpass");
        p2.SendKeys("wrongpass");
    }
    else if (a.Contains("wrong username"))
    {
        u.SendKeys("user");
        mail.SendKeys("wrong@x.com");
        p1.SendKeys("User1234@");
        p2.SendKeys("User1234@");
    }

    Click("Registration submit button");
}
```

Test Execution:



Summary of test Execution:

Test Strategy	Parameters	Length of the test suite	Duration	Passed	Failed
Random	100% transition coverage	391	3.9 min	391	0
All-State	-	10	5.9 sec	10	0
Transition Tour	-	64	46 sec	64	0
All-Transition-State	ATS0	193	2 min	193	0
N-Switch Coverage	N=1	409	5.8 min	409	0

HW ToC

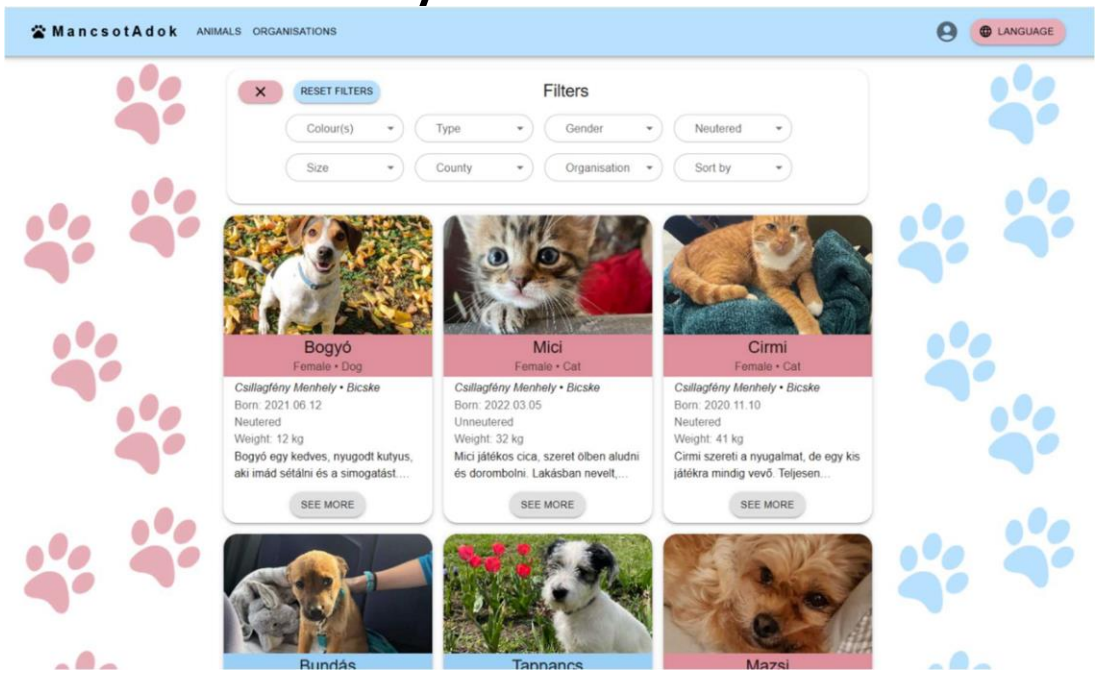
Appendix D: Example homework LVII. 3/1

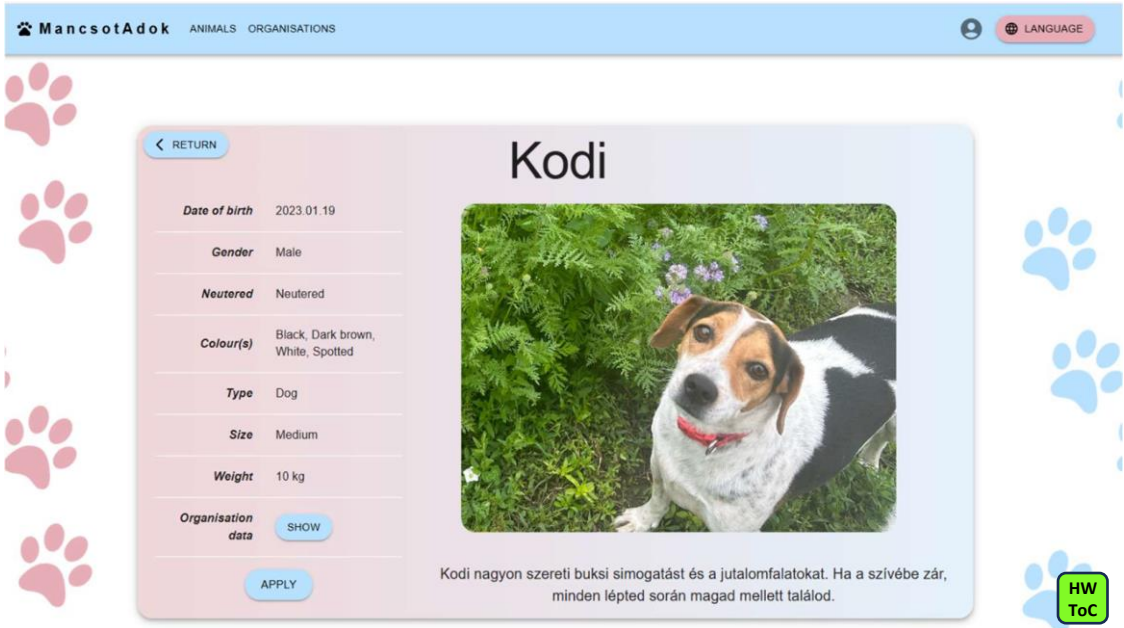
Fall 2025/2026

Sára Vadkerti: Online Animal Shelter

- SUT (System Under Test): MancsotAdok Online Animal Shelter
  - Tested functionalities:
    - Login
    - Language switch
    - Register page
    - Organizations page: displays a list of all registered rescue organizations
    - Animals page: displays a list of adoptable animals and includes a Filters panel
    - Animals details: presents all available information about a specific animal
  - Limitations:
    - Not tested:
      - Logged-in features:
        - applying for an animal, changing profile details,
        - filling out an application form, checking submitted applications
      - Organization features:
        - adding a new animal, managing existing animals,
        - checking applications or changing their status
      - Animals page features:
        - Number of animals for a given filter
      - Limited dataset of 10-20 animals

- Testing:
  - FSM model has been created with [GraphWalker](#) Studio
  - Test generation with [Model 》 Test 》 Relax](#)
  - Glue code (adaptation code):
    - Written in JavaScript
      - GUI testing with Playwright





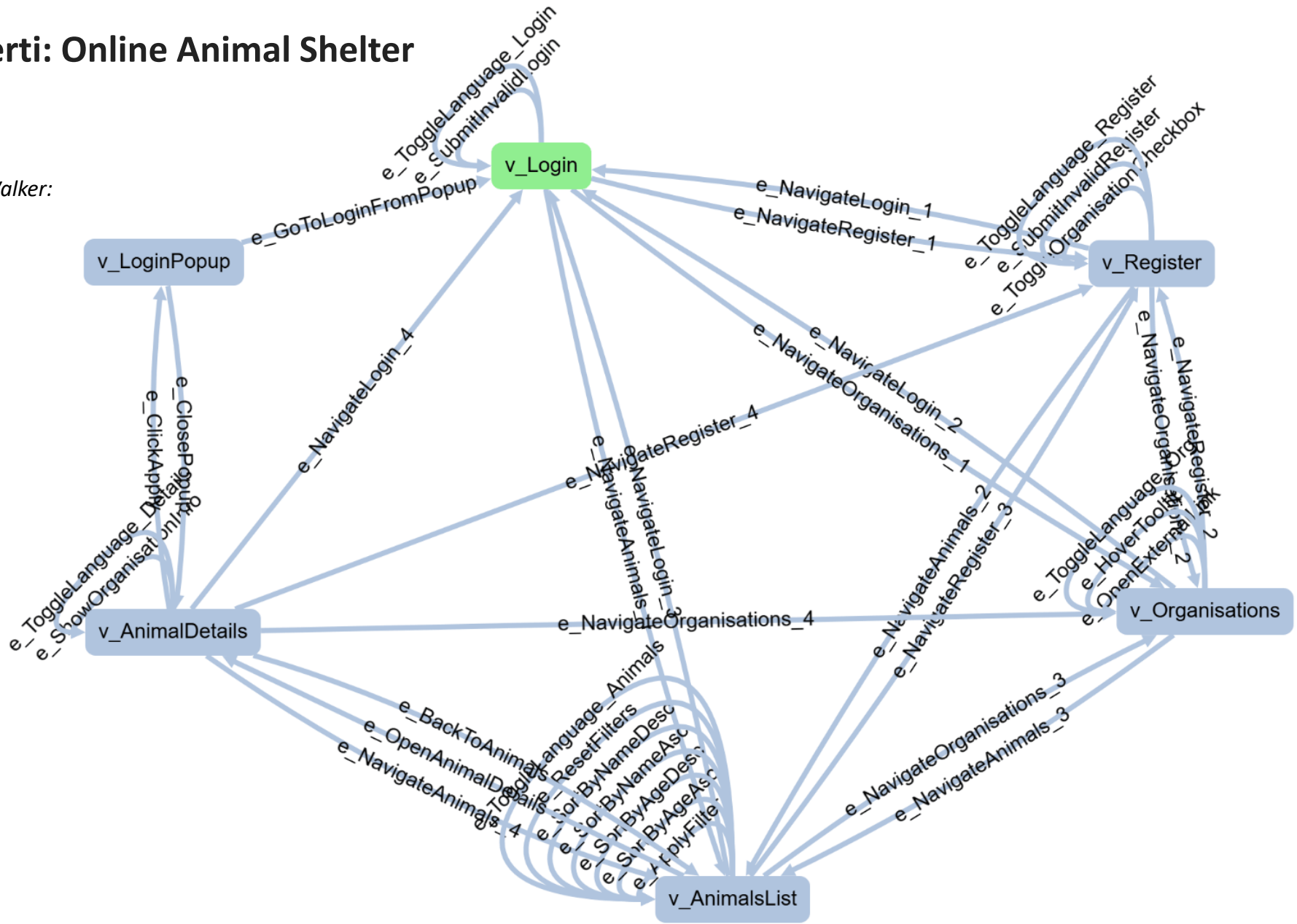
HW ToC

Appendix D: Example homework LVII. 3/2

Fall 2025/2026

Sára Vadkerti: Online Animal Shelter

FSM model in GraphWalker:



HW ToC

Appendix D: Example homework LVII. 3/3

Fall 2025/2026

Sára Vadkerti: Online Animal Shelter

Adaptation code examples (an implementation of few transitions):

```
async function performAction(page: Page, input: string) {
  switch (input) {
…

    case "e_NavigateAnimals":
      await page.locator('a[href="/animals"].MuiButton-
root').first().click();
      await expect(page).toHaveURL(/\/animals/);
      break;

…
    case "e_ApplyFilter":
      await applyFilters(page);
      await page.locator(".MuiButton-
containedSecondary").first().click();
      await page.locator('[data-testid="CloseIcon"]').click();

      break;

    case "e_SortByAgeAsc":
      await orderBy(page, 1, "age", "asc");
      break;
    case "e_SortByAgeDesc":
      await orderBy(page, 2, "age", "desc");
      break;
    case "e_SortByNameAsc":
      await orderBy(page, 3, "name", "asc");
      break;
    case "e_SortByNameDesc":
      await orderBy(page, 4, "name", "desc");
      break;

    case "e_ResetFilters":
      await applyFilters(page);
      await page.locator(".MuiButton-
containedSecondary").first().click();
      await expect(page).not.toHaveURL(/colour=/);
      await page.locator('[data-testid="CloseIcon"]').click();

      break;

    case "e_OpenAnimalDetails":
      await page.keyboard.press("Escape");
      await page.waitForTimeout(500);

      await page
        .locator('a[href^="/animal/"].MuiButton-contained')
        .first()
        .click({ force: true });
      await expect(page).toHaveURL(/\/animal\/\d+/);
      break;

    case "e_ShowOrganisationInfo":
      await page.locator(".MuiTableCell-root button").click();
      await expect(page.locator(".MuiDialog-
root")).toBeVisible();
      await page.keyboard.press("Escape");
      break;
…
    default:
      console.warn(`Unknown input: ${input}`);
      break;
  }
}
```

An example of test execution:

```
Running 1 test using 1 worker
[1/1] [chromium] › tests\animalshelter.glue.ts:249:9
[chromium] › tests\animalshelter.glue.ts:249:9 › Exec
alShelter_AS.json
Running suite: OnlineAnimalShelter_AS.json, steps: 6
1 - e_NavigateOrganisations
2 - e_NavigateAnimals
3 - e_OpenAnimalDetails
4 - e_ClickApply
5 - e_GoToLoginFromPopup
6 - e_NavigateRegister
  1 passed (6.0s)
```

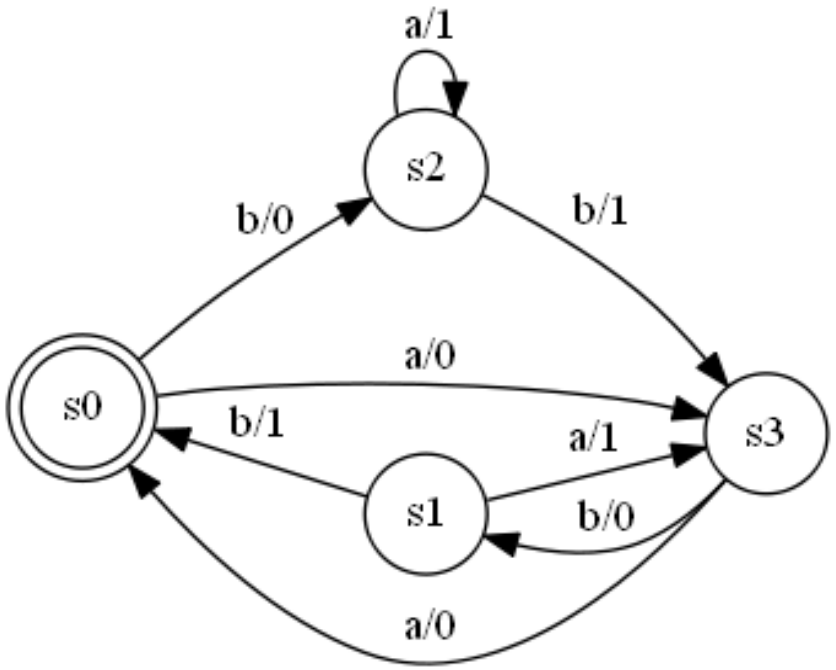
Summary of test execution:

Algorithm	Test length	Generation time (s)	Playwright execution
AS	6	0.0006467	6.0 seconds
AS_reset	6	0.0020959	6.6 seconds
TT	48	0.0125816	34.9 seconds
TT_reset	60	0.008	34.4 seconds
ATT	1226	0.0453509	7.6 minutes
ATT_reset	1675	0.00423356	11 minutes
ATS0	115	0.0198645	1.1 minutes
ATS0_reset	139	0.0224127	1.3 minutes
NS-1	432	0.0318049	4.7 minutes
NS-1_reset	616	0.0507831	6.6 minutes
H	108	0.0251617	1 minute
HSI	141	0.0098938	1.8 minutes

HW ToC

Appendix E: Example exams 2/1

1. Consider the following FSM (Finite State Machine), where double circle denotes the initial state, and each transition is labeled with *i/o*, where *i* and *o* denotes the input and output associated with the transition, respectively:



1.1 Pls. answer to the following questions and explain the reasons behind your answer:
- Is the given FSM deterministic?
- Is the given FSM completely specified?
- Is the given FSM reduced (or minimal)?

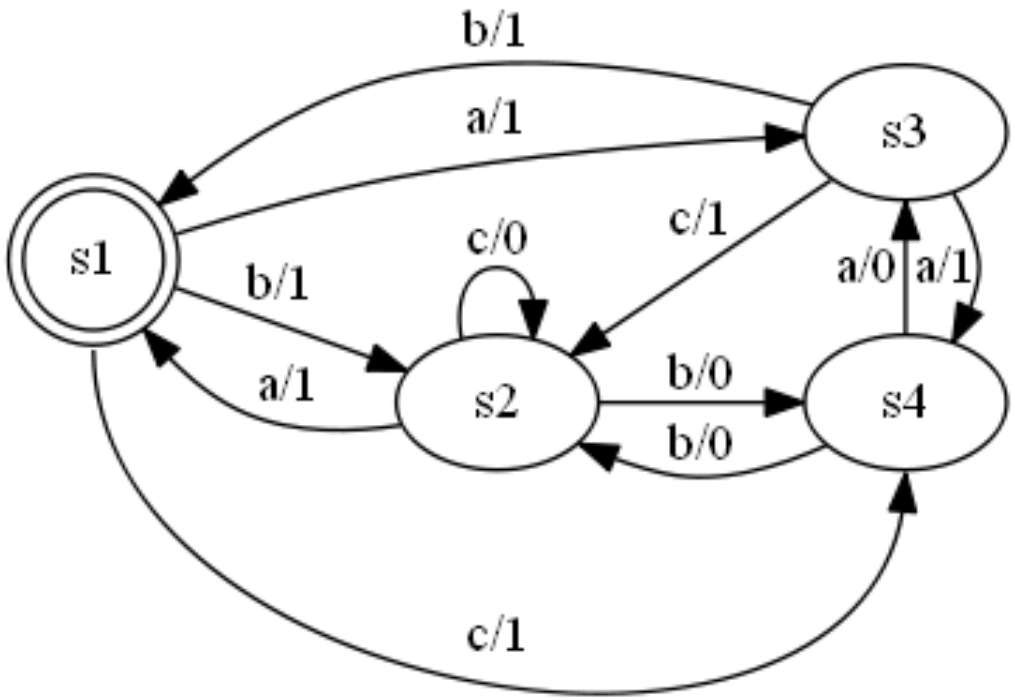
1.1 Explain the HIS/HSI (Harmonized State Identifiers) method:
- Which assumptions about FSMs are required in case of this method?
- Which types of faults are guaranteed to be found with this method?
- Explain the working of this method with the attached FSM! How state s2 can be verified?

2. In what way is the EFSM (Extended Finite State Machine) the extension of the "simple" FSM?

3. Which types of performance tests exist?

Appendix E: Example exams 2/2

1. Consider the following FSM (Finite State Machine), where double circle denotes the initial state, and each transition is labeled with *i/o*, where *i* and *o* denotes the input and output associated with the transition, respectively:



1.1 Is the given FSM completely specified? Why / why not?

1.1 Explain the TT (Transition Tour) method:
- Which assumptions about FSMs are required in case of this method?
- Which types of faults are guaranteed to be found with this method?
- Explain the working of this method with the attached FSM!

2. Models:

2.1 What are the Markov chains and how they are related to the FSMs?

2.2 What are Communicating Finite State Machines? In what way they are the extensions of “simple” FSMs, how they are work?

3. Which approaches are used to describe the behaviour of load generators in performance testing? What are the advantages and disadvantages of these approaches?

Model

Create a specification model.

Test

Generate test.

Relax

Watch the results!

https://modeltestrelax.org/

Appendix F: Model 》 Test 》 Relax MBT framework 31/1

- FSM model-based testing framework developed mainly for research and education purposes
- Free and open source – can be accessed in ELTE gitlab
- Has good documentation: user guide, developer guide, notes for contibutors, examples, sample test projects
- Developed in C++ using the graph algorithms and data structures of LEMON library
- Loose coupling with GraphWalker
  - Model import/export, test suite export to GraphWalker
- Different test generation methods:
  - Normal and Weighted Random Walks (with different stopping conditions)
  - AS (All-State)
  - TT (Transition Tour), Directed Chinese Postman
  - ATS (All-Transition-State)
  - ATT (All-Transition-Transition)
  - HSI-method
  - H-method
  - N-Switch Coverage
  - Reset Checking sequence
  - Model-based mutation testing with the above algorithms
- Also deals with EFSM and CEFSM models:
  - Variables, guard conditions and actions over variables, component machines and communications
  - EFSM → FSM conversion (with limitations on variable ranges and types), CEFSM → EFSM conversion

AS: All-State
ATS: All-Transition-State
ATT: All-Transition-Transition
CEFSM: Communicating Extended Finite State Machines
FSM: Finite State Machine
EFSM: Extended Finite State Machine
HSI: Harmonized State Identifiers
LEMON: Library for Efficient Modeling and Optimization in Networks
MBT: Model-Based Testing
TT: Transition Tour

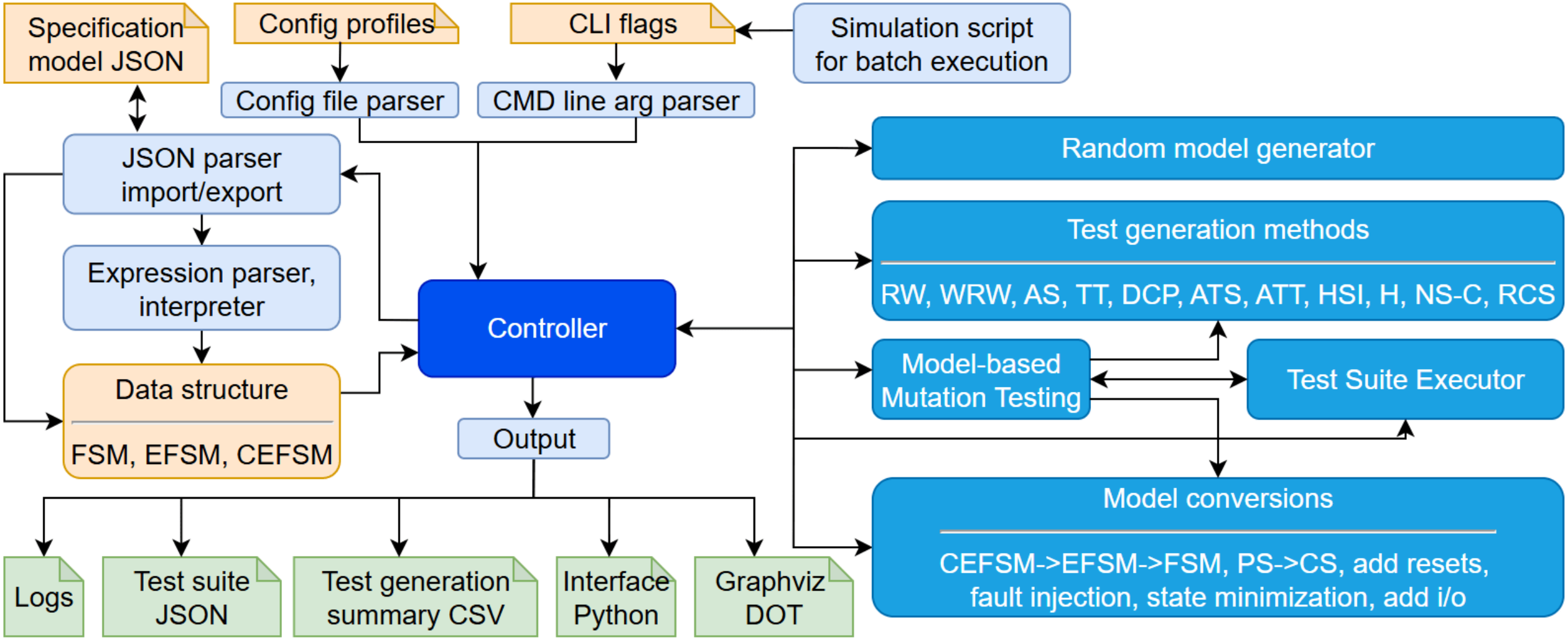
Appendix F: Model 》 Test 》 Relax MBT framework 31/2

Credits

Name	Role	Area	Timeframe
Gábor Árpád Németh	Product Owner / Supervisor	Concept, future directions, theory background, documentation	February 2020 –
Máté István Lugosi	Architect / Developer	• Core framework (import/export, data structures, logging) • Random graph generation, change injection V1 • Test generation algorithms: Random, AS, TT, DCP, ATS, ATT • Simulation script	February 2020 –
Ádám Bába	Developer	• Explicit reliable reset transitions implementation, Handle string, enum and struct variables, Handle ternary conditional expressions • CEFSM model handling, EFSM→FSM conversion	October 2022 -
Zsolt Csáky	Developer	• EFSM extension, EFSM→FSM conversion	February 2021 – May 2021
László Dervalics	Developer	• HSI test generation (legacy version)	February 2021 – May 2021
Tódor Dávid Nyeste	Developer	• H-method, HSI-method, ELTElive glue code (adaptation code) example	February 2022 – July 2022
Bálint Miksa Rumpler	Developer	• N-Switch Coverage test generation implementation • fault injection v2 • Model-based mutation testing	February 2024 – June 2025
Álmos Sőnfeld	Developer	• GraphWalker API	June 2022
Gábor Spaits	Developer	• Extend the inputs and outputs fields to an arbitrary user defined type	February 2024 – Sept 2024
Krisztián Tauber	Developer	• Create reports from test execution results, CsvManager	October 2023 -
Tomás Varga	Developer	• Weighted Random Walk test generation implementation, ASP.NET Webshop sample test project	October 2022 – June 2023
Levente Hunyadi	Technical advisor	-	Sept 2021 – Oct 2022

ATS: All-Transition-State
HSI: Harmonized State Identifiers
ATT: All-Transition-Transition
TT: Transition Tour
FSM: Finite State Machine
EFSM: Extended Finite State Machine

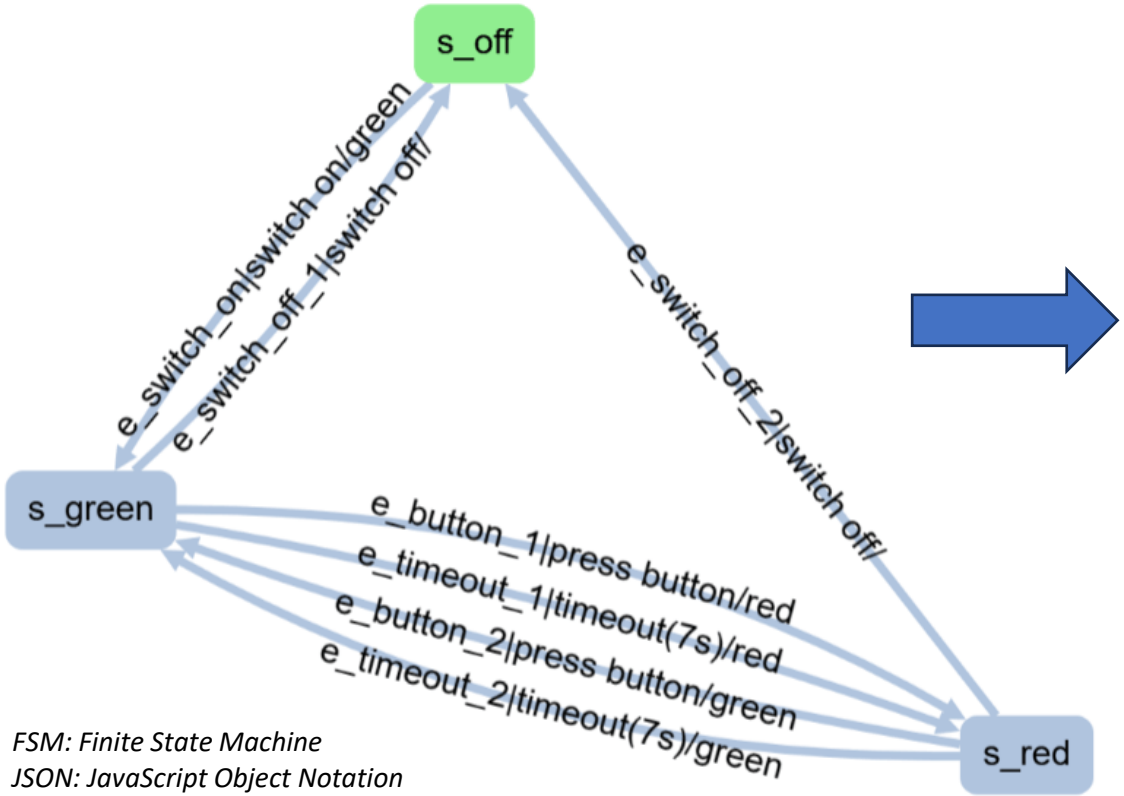
Appendix F: Model 》 Test 》 Relax MBT framework 31/3



The structure of the framework

Appendix F: Model 》 Test 》 Relax MBT framework 31/4

- **Interfaces 3/1:**
  - For FSM models (input/output) and for generated path (output):
    - Random graph generator
      - Parameters:
        - number of states, density, number of output symbols
        - completely/partially specified, have/do not have reset, reduced/not reduced
    - JSON parser (import/export models to/from GraphWalker)



FSM: Finite State Machine
JSON: JavaScript Object Notation


```
{
   "models":[
      {
         "name":"Toy_train_signal",
         "id":"MTR-toytrs-doi_10.36244/ICJ.2024.2.2",
         "hasReset":false,
         "hasReliableReset":false,
         "vertices":[
            {
               "id":"n0",
               "name":"s_green",
               "properties":{
                  "x":20,
                  "y":263.5
               }
            },
            {
               "id":"n1",
               "name":"s_red",
               "properties":{
                  "x":364,
                  "y":392
               }
            },
            {
               "id":"n2",
               "name":"s_off",
               "properties":{
                  "x":112,
                  "y":68
               }
            }
         ],
         "edges":[
            {
               "id":"e0",
               "name":"e_button_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n1",
               "input" : "press button",
               "output" : "red",
               "weight": "0.3",
               "cost" : "0.3"
            },
            {
               "id":"e1",
               "name":"e_switch_on",
               "sourceVertexId":"n2",
               "targetVertexId":"n0",
               "input" : "switch on",
               "output" : "green",
               "weight": "1",
               "cost" : "1"
            },
            …
         ],
         "startElementId":"n2"
      }
   ]
}
```

Appendix F: Model 》 Test 》 Relax MBT framework 31/5

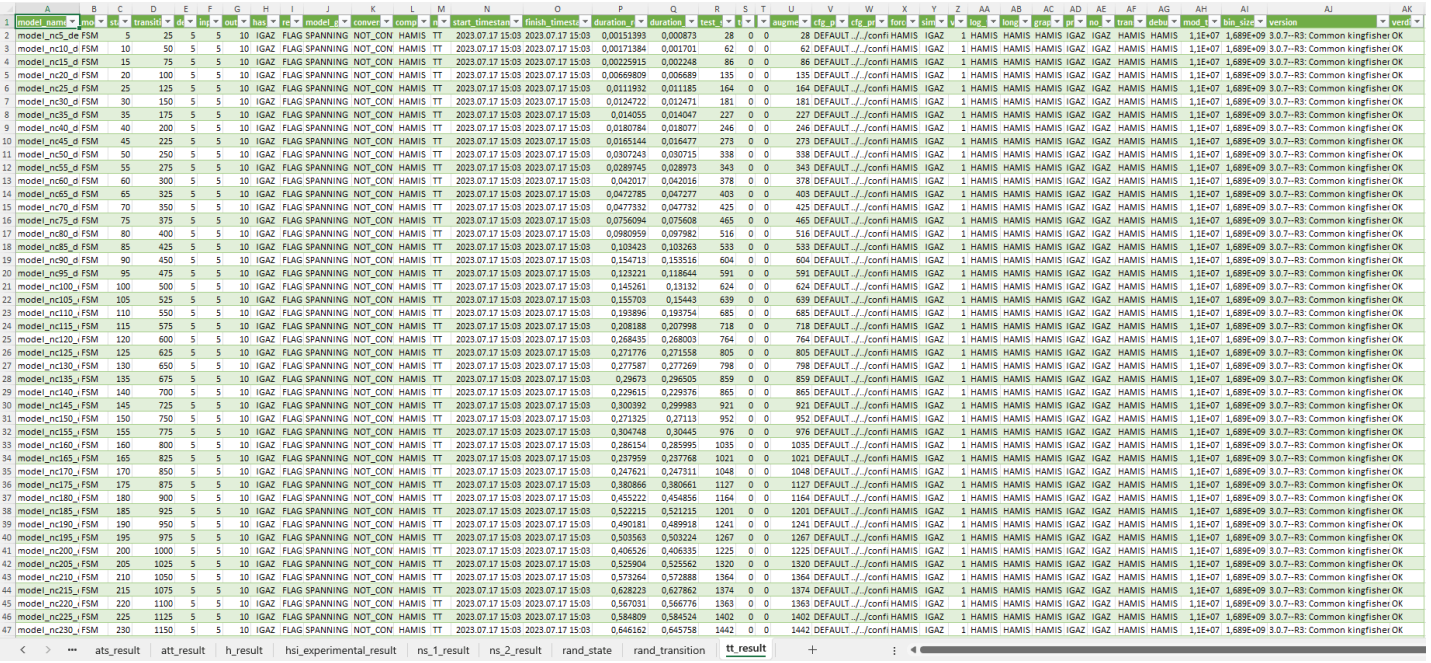
- **Interfaces 3/2:**
  - Program input settings:
    - Command line argument parser
    - Configuration profiles
  - Logger (output):
    - 6 levels: critical/error/warning/info/debug/trace
  - Model visualization with Graphviz dot file output
  - Export generated test cases into GraphWalker
  - Python interface file of the model

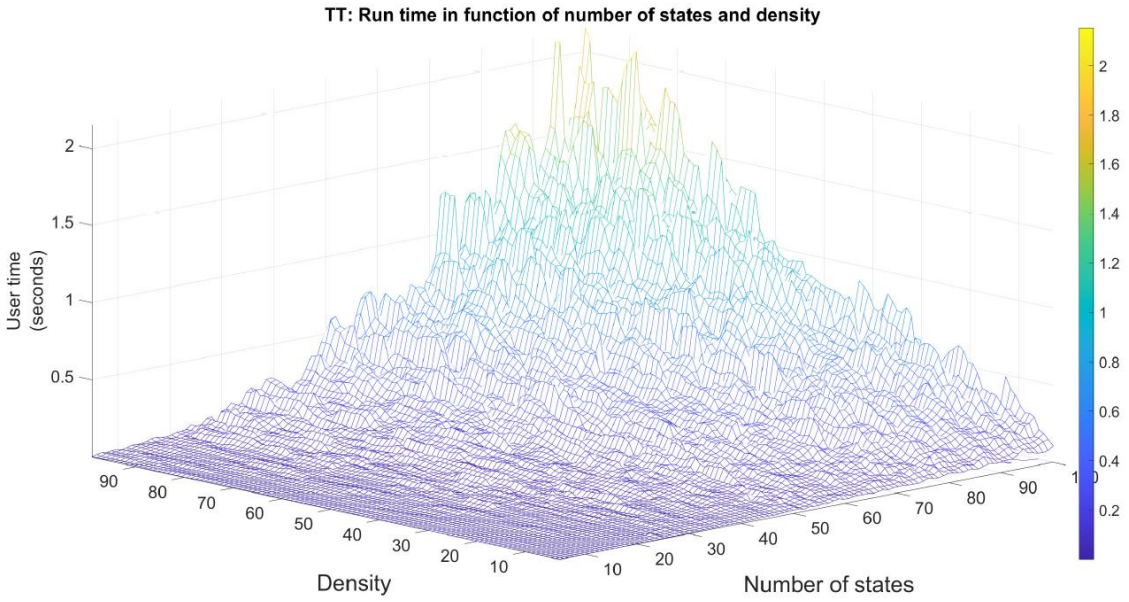
```
D:\MTR\MTR-1-0-6>MTR -m H -f sample_models/introduction/h_1.json --node_order=4 --extra_states=0
[U-CTRL ] [info] Version: 1.0.6-2-g5f0c9b1 R1: Bee hummingbird, debug mode: 0, verbosity: 3
[TGC-SCS] [info] Creating state cover set...
[TG-H   ] [info] Checking preconditions...
[TG-H   ] [info] The chosen node order is 4
[TG-H   ] [info] The chosen number of extra states is 0
[TG-H   ] [info] Collecting input and output symbols...
[TG-H   ] [info] Creating testing tree...
[TG-H   ] [info] Recreating the path to the node: n_0
[TG-H   ] [info] Finished the state cover for n_0, now inserting the traversal set...
[TG-H   ] [info] Recreating the path to the node: n_1
[TG-H   ] [info] Finished the state cover for n_1, now inserting the traversal set...
[TG-H   ] [info] Recreating the path to the node: n_2
[TG-H   ] [info] Finished the state cover for n_2, now inserting the traversal set...
[TG-H   ] [info] Recreating the path to the node: n_3
[TG-H   ] [info] Finished the state cover for n_3, now inserting the traversal set...
[TG-H   ] [info] Stage 2: appending testing tree with separating sequences of (SC-SC) pairs...
[TG-H   ] [info] Stage 3: appending testing tree with separating sequences of (SC-extra) pairs...
[TG-H   ] [info] Stage 4: appending testing tree with separating sequences of (extra-extra) pairs...
[TG-H   ] [info] Generating test suite...
[TG-H   ] [info] The length of the test suite is 40
[TG-H   ] [info] Test suite written: test_suites/h_1_H_suite
[TG-H   ] [info] finished computation at 2022-07-01 10:15:47.7669151, real time: 0.0253893 seconds
user time: 0.025000 s
memory usage: 0

D:\MTR\MTR-1-0-6>
```

GUI: Graphical User Interface

Appendix F: Model 》 Test 》 Relax MBT framework 31/6

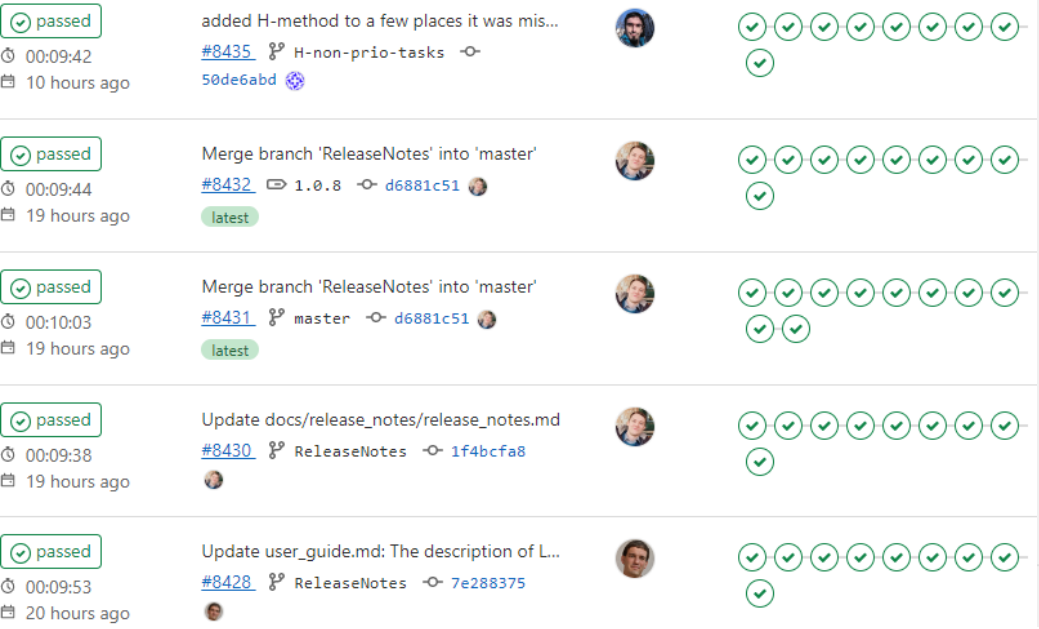
- **Interfaces 3/3:**
  - Simulation results of batch executions (output) – *CSV files and an xlsx summary file*:
    - Input model parameters (name, number of states, number of transitions, density, reset),
    - Used method (with specific input/output attributes),
    - Test generation time (user and real), used memory
    - Length of the generated test sequence (with specific sub arguments in case of TT, DCP, ATS, ATT, HIS, H and N-SC methods)
    - Error code and main params if a test generation fails on a model for some reasons
    - Feedback if a given injected fault has been found with the selected method

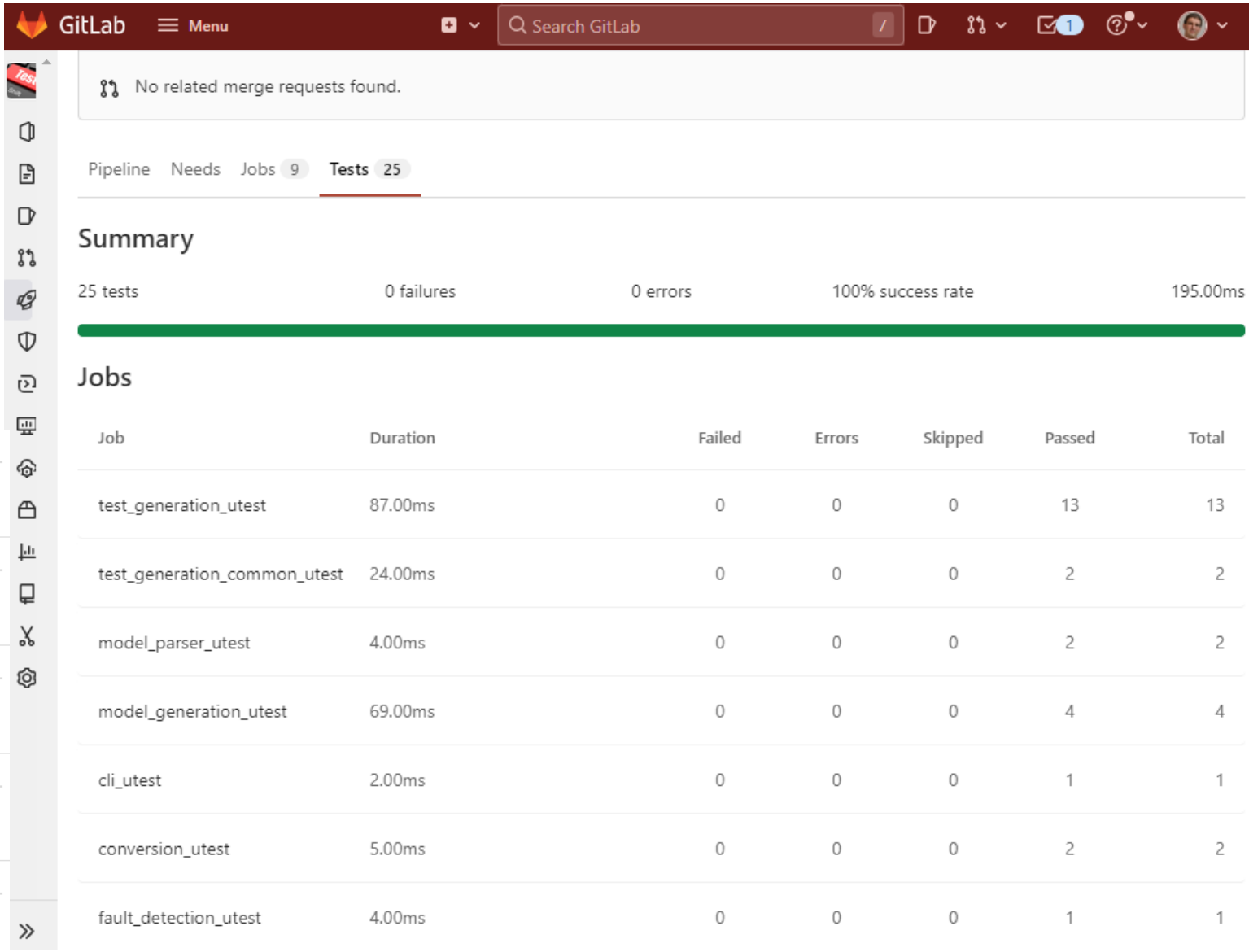




Appendix F: Model 》 Test 》 Relax MBT framework 31/7

- **Continuous integration:**
  - Automatic tests (that are created with Google tests) in ELTE gitlab
    - Build test
    - Tests for different modules (model parser, model conversion, test generation, CLI parser…etc.)
    - User Guide tests (automatic execution of sample commands)
    - …etc.





Appendix F: Model 》 Test 》 Relax MBT framework 31/8

- Different test generation algorithms:

n: number of states
p: number of input symbols
m: number of transitions

Test generation algorithm	Used model	Complexity of test generation	Complexity of test suite	Structure of test suite	Coverage	Notes
Random walk	FSM/EFSM	Not bounded	Not bounded	1 test sequence	Given percentage of state or transition coverage based on the preset stopping condition	Can be useful for exploratory testing, but impractical for testing a complex system due the possible length of the test sequence
Weighted Random Walk	FSM/EFSM	Not bounded	Not bounded	1 test sequence	Given percentage of state or transition coverage based on the preset stopping condition	Modified version of Random Walk, when the probabilities for each transition can be set
All-State	FSM	$O(n^2)$	$O(m)$	1 test sequence	100% state coverage	Heuristic algorithm
Transition Tour (TT)	FSM	$O(n^3+m)$	$O(m)$	1 test sequence	100% state- and transition coverage. It guarantees to discover all output faults, but does not guarantee to find transfer faults	The resulting test sequence is the shortest possible tour that provides 100% state- and transition coverage
Directed Chinese Postman (DCP)	FSM	$O(n^3+m)$	$O(m)$	1 test sequence	100% state- and transition coverage. It guarantees to discover all output faults, but does not guarantee to find transfer faults	Same as TT, but considers transition costs as well.
All-Transition-State (ATS)	FSM	$O(n^3+m)$ (ATS0 version), $O(\eta(n^3+m))$ (iterative version),	$O(m)$ (ATS0 version), $O(\eta \cdot m)$ (iterative version),	1 test sequence (with subparts)	It guarantees to discover all output faults and it finds virtually all transfer faults	Heuristic algorithm
All-Transition-Transition (ATT)	FSM	$O(m^2+n^3)$	$O(m^2)$	1 test sequence (with subparts)	It guarantees to discover all output faults and it finds virtually all transfer faults	Not efficient, naive implementation
Harmonized State Identifiers (HSI)	FSM	$O(p \cdot n^3)$	$O(p \cdot n^3)$	Structured test suite with multiple test sequences	It guarantees to discover output and transfer faults and given number of extra states in implementation	
Harmonized State Identifiers (HSI) **(legacy version)**	FSM	$O(p \cdot n^3)$	$O(p \cdot n^3)$	Structured test suite with multiple test sequences	It guarantees to discover output and transfer faults	This implementation version works only on completely specified FSMs and does not discover duplications in test seqs. More suitable for education purposes. Use HSI-method instead of this.
H-method	FSM	$O(p \cdot n^3)$	$O(p \cdot n^3)$	Structured test suite with multiple test sequences	It guarantees to discover output and transfer faults and given number of extra states in implementation	Improvement of the HSI-method
N-Switch coverage	FSM	$O((N+1) \cdot m^{(k+1) \cdot (N+1)})$, where $k=[0..N]$ iteration limit	$O((N+1) \cdot m^{N+1})$	1 test sequence	It covers all topologically possible, consecutive $N+1$ transitions	The test generation time and the size of the resulting test sequence dramatically increases if N or the size of the input model increases!
Test unreliable reset symbols	FSM	$O(n^2+n \cdot m)$ (`status` mode), $O(p \cdot n^2+n^3)$ (`sfs` mode)	$O(n^2)$ (`status` mode), $O(n^3)$ (`sfs` mode)	1 test sequence	It test reset symbols only	Can called implicitly by test generation algorithms that assume reliable reset symbols.

Appendix F: Model 》 Test 》 Relax MBT framework 31/9

- Different test generation algorithms:
  - **Random**
    - In each step a transition is chosen randomly and traversed until a given stop condition is fulfilled
      - Stop conditions: given percentage of state or transition coverage is reached
    - This approach can be useful for exploratory testing, but it is impractical for the functional testing of a large-scale software as the length of the test sequence can be much longer than the optimal solution.
  - **Weighted Random**
    - Modified version of Random Walk, when the probabilities for each transition can be set
      - Stop conditions: given percentage of state or transition coverage is reached
  - **All-State**
    - Produces a test sequence that visits every state of the FSM
    - It uses Nearest Neighbour heuristic which searches in each step for the closest unvisited state until such state exists.

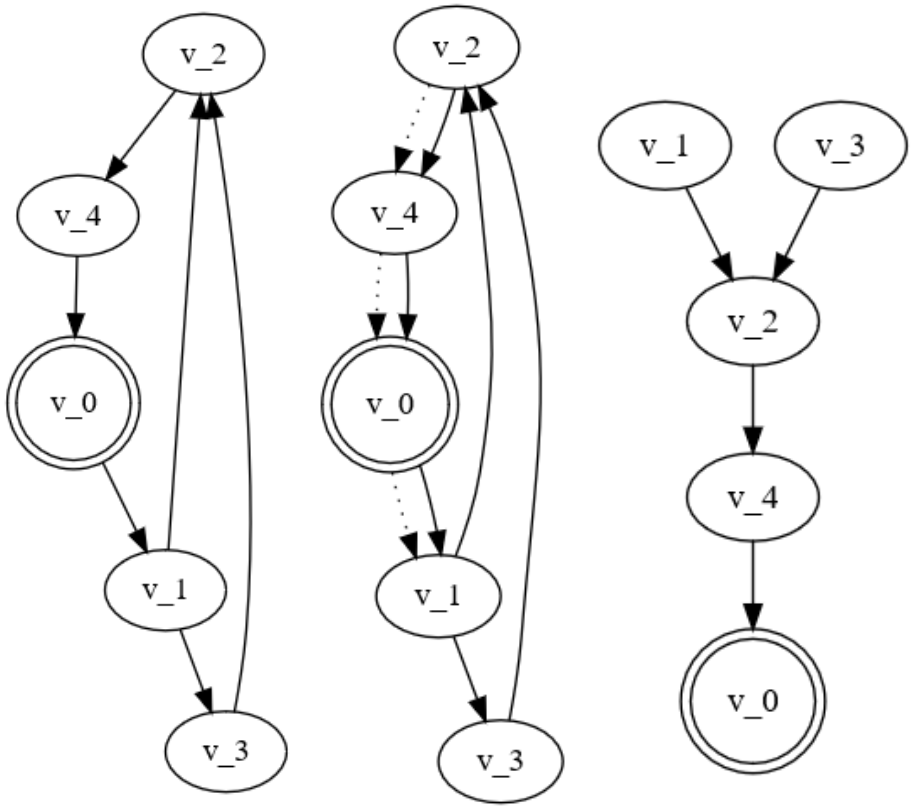
Appendix F: Model 》 Test 》 Relax MBT framework 31/10

- Different test generation algorithms:

  - **Transition Tour**
    - Shortest tour that provides 100 state and transition coverage
    - Based on the Directed Chinese Postman Problem

    - 2 main parts:
      1. Augmenting the original graph to make it Eulerian
         - The problem can be reduced to minimum weighted perfect matching on a bipartite graph
      2. Finding an Euler tour over an Eulerian graph
         - Using next-node representation → the problem can be reduced to a spanning tree, which edges are directed toward the root node

  - **Directed Chinese Postman**
    - Same as Transition Tour, but considers edge costs as well



↑ *input graph G* ↑ *Eulerian graph* G_E ↑ *spanning tree used in the Euler tour*

Appendix F: Model 》 Test 》 Relax MBT framework 31/11

- Different test generation algorithms:
  - **New algorithm for the ATS criteria of FSMs 4/1:**
    (i.e. only the first 2 conditions are considered)

    1) To satisfy condition 1:
       Create a *main sequence* using original graph *G*:
       1. **Preamble (AT):** Visit all transitions: Using the TT-method, except there is no need to return to the initial state at the end
       2. **Postamble (AS):** Visit all states: Using the Nearest Neighbour heuristic

    2) To satisfy condition 2:
       Create ***alternative sequences*** using filtered graphs of *G:*
       1. **Preamble (AT):** Create sequences that are as arc disjoint as possible
          - Using the TT-method on filtered graphs (without return to the initial state at the end)
       2. **Postamble (AS):** Visit all states: using the Nearest Neighbour heuristic

       Output of the algorithm:
       - Test suite: Concatenation of main sequence and alternative sequences

AT: All-Transition *ATS: All-Transition-State* *AS: All-State* *FSM: Finite State Machine* *TT: Transition Tour*

Appendix F: Model 》 Test 》 Relax MBT framework 31/12

- Different test generation algorithms:
  - **New algorithm for the ATS criteria of FSMs 4/2:**
    - The algorithm has 3 different versions for *alternative sequence* generation:

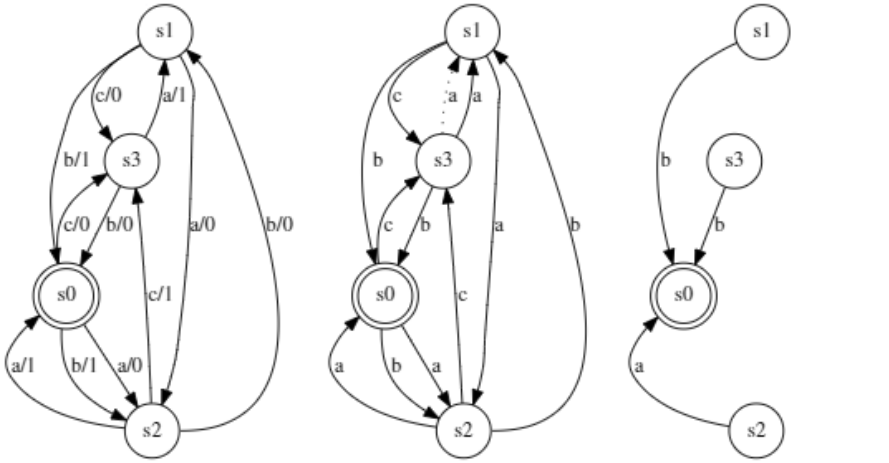
ID	notes	input	used graphs	output: test suite
ATS0	standard version	FSM M	original: G filtered: G^T, G^{ACT}	1 main sequence + 2 alternative seqs.
ATSa	version without iteration limit	FSM M	original: G filtered: G^T, G^{ACT}, G^{AC_k}	1 main sequence + max. $2n$ alternative seqs.
ATSx	version with iteration limit	FSM M, $depth$	original: G filtered: G^T, G^{ACT}, G^{AC_k}	1 main sequence + max. $depth + 2$ alternative seqs.

Appendix F: Model 》 Test 》 Relax MBT framework 31/13

- Different test generation algorithms:
  - **New algorithm for the ATS criteria of FSMs 4/3:**
    - Alternative sequence generation in standard version *(ATS0)*:
      - Using filtered subgraphs of G:
        - 1st alternative sequence: take the *TS* inverse spanning tree used for the Eulerian augmentation of *G* (G_E), and extend it with paths from the root node to each of the leaves → G^T graph → $G^T{}_E$ Eulerian graph
        - 2nd alternative sequence: filter out the arcs of G^T from *G*.
          - If the resulting graph won't be strongly connected, then re-enable some arcs of G_T → G^{ACT} graph → $G^{ACT}{}_E$ Eulerian graph
    - Optional, iterative extensions for alternative sequence generation *(ATSa, ATSx)*:
      - In the k^{th} iteration create filtered graphs $G^{ACk}{}_E$ that does not contain at least one arc that were re-enabled in all previous steps
        - *ATSa* terminates: (1) If for all arcs an arc disjoint sequence has been found or (2) if no more arc disjoint sequence is possible
        - *ATSx* terminates: (1) If iteration limit is reached or (2) if the previous condition of ATSa is met

Appendix F: Model 》 Test 》 Relax MBT framework 31/14

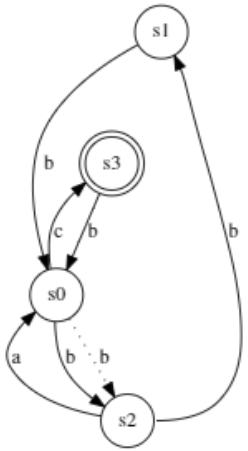
- Different test generation algorithms:
  - **New algorithm for the ATS criteria of FSMs 4/4:**



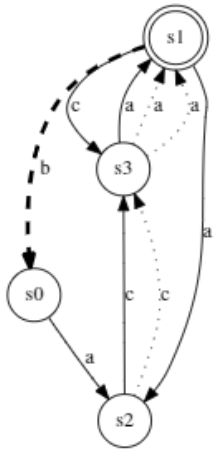
(a) FSM M

(b) Graph G_E

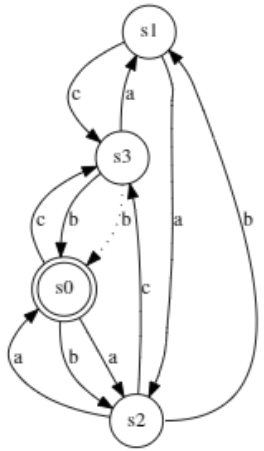
(c) Inverse spanning tree TS of graph G



(d) Graph G_E^T



(e) Graph G_E^{ACT}



(f) Graph $G_E^{AC_1}$

Legend

- *Double circle: Start state of a sequence*
- *Solid lines: original edges (i.e. transitions of FSM)*
- *Dotted lines: extra edges (that makes the graph balanced)*
- *Bold dashed lines: reenabled transitions that connect SCCs*

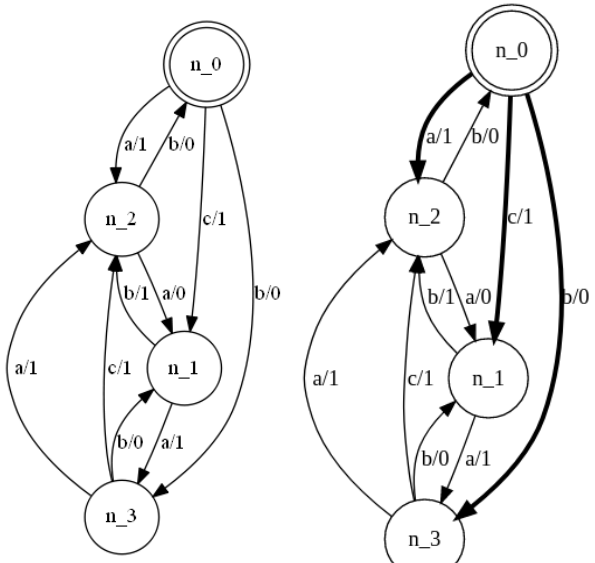
Resulting test suite	AT part (preamble)	AS part (postamble)
Main sequence (on G_E)	*bbacacabaacb*	*bbc*
1st alternative sequence (on G^T_E)	*bbbbbac*	*bbb*
2nd alternative sequence (on G^{ACT}_E)	*acabacac*	*aacab*
3rd alternative sequence (on G^{AC1}_E) *optional iterative step*	*bbacbaacac*	*aaa*

G. Á. Németh, M. Lugosi: Test generation algorithm for the All-Transition-State criteria of Finite State Machines. Infocommunications Journal. 13/3 (2021).
DOI: 10.36244/ICJ.2021.3.6

AT: All-Transition
AS: All-State
SCC: Strongly Connected Component
ATS: All-Transition-State
FSM: Finite State Machine

Appendix F: Model 》 Test 》 Relax MBT framework 31/15

- Different test generation algorithms:
  - **HSI-method**



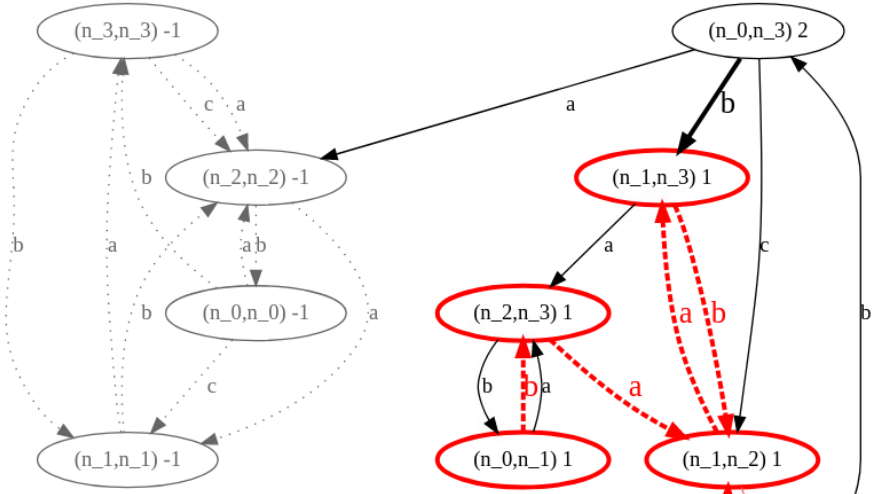
↑ *The input FSM*

↑ *FSM with the Q state cover set*

Legend

- *Double circle: start state*
- *Black bold edges: spanning tree edges (used in set Q)*

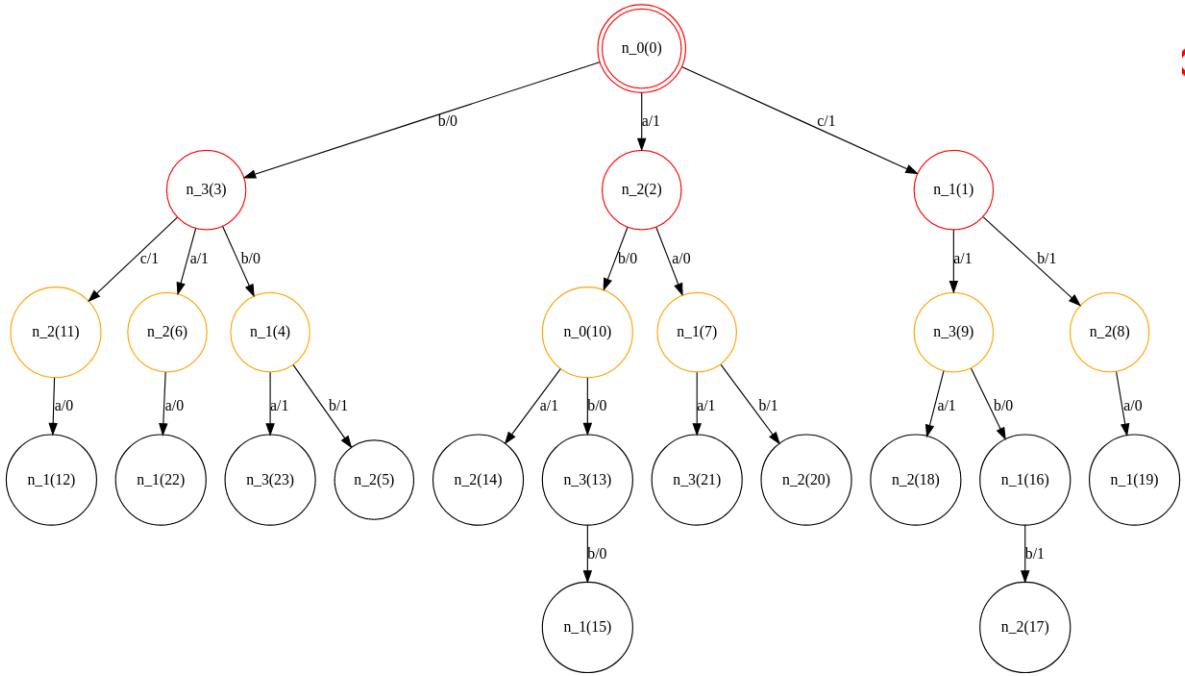
The auxiliary state pair graph with Z separating family of sequences→



Legend

- *Red ellipses: separating state pairs*
- *Red dashed edges: separating input*
- *Red bold dashed edges with bigger label: separating input selected by the algorithm*
- *Black bold edges: spanning forest edges (pointing to separating state pairs)*
- *Grey ellipses: identical state pairs*
- *Grey dotted edges: edges between identical state pairs*

Testing tree →

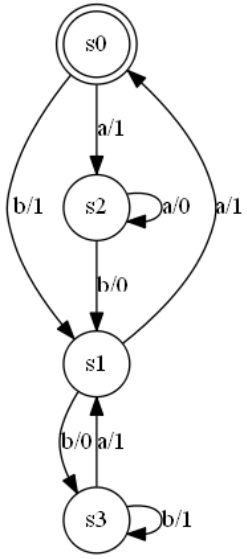


Legend

- Double red circle: start state
- Red circle: states reached by the state cover set *Q*'s sequences.
- Orange circle: states reached by the up to μ-n+1 long sequences mentioned in the 1. stage

Appendix F: Model 》 Test 》 Relax MBT framework 31/16

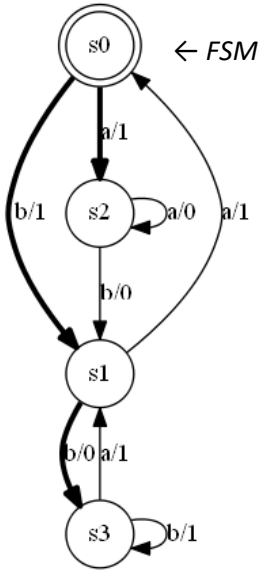
- Different test generation algorithms:
  - **HSI-method (legacy version)**



↑ *The input FSM*

```
[hsi] [info] Creating state cover set...
[hsi] [info] Q set: q(s0): [ ] q(s1): [ b ] q(s2): [ a ] q(s3): [ b b ]
[hsi] [info] Creating state pair graph...
[hsi] [info] Z set:
[hsi] [info] Z(s0): z(s0,s1): [ b ] z(s0,s2): [ b ] z(s0,s3): [ b b ]
[hsi] [info] Z(s1): z(s0,s1): [ b ] z(s1,s2): [ a ] z(s1,s3): [ b ]
[hsi] [info] Z(s2): z(s0,s2): [ b ] z(s1,s2): [ a ] z(s2,s3): [ a ]
[hsi] [info] Z(s3): z(s0,s3): [ b b ] z(s1,s3): [ b ] z(s2,s3): [ a ]
```

↑ *Generating Q state cover set and Z separating family of sequences*



← *FSM with the Q state cover set*

Legend

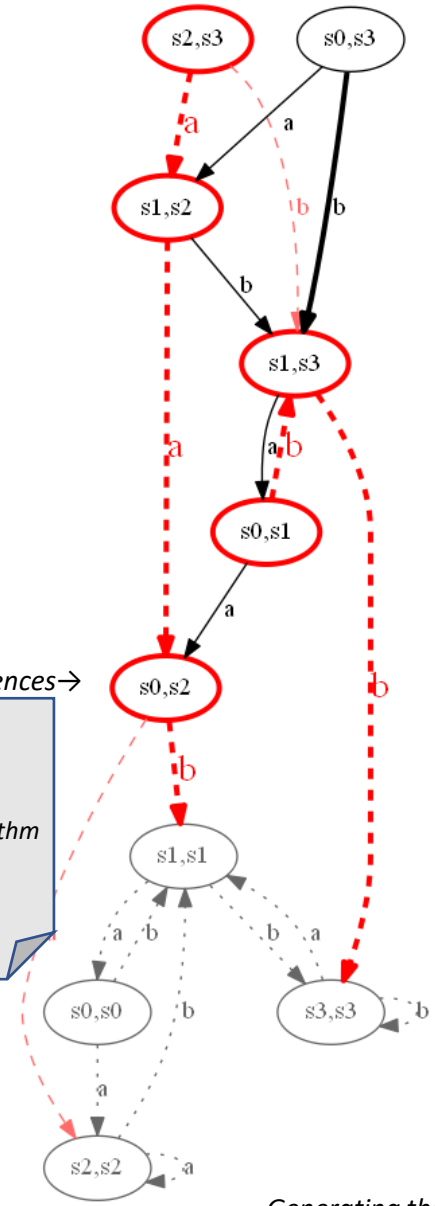
- *Double circle: start state*
- *Black bold edges: spanning tree edges (used in set Q)*

The auxiliary state pair graph with Z separating family of sequences →

Legend

- *Red ellipses: separating state pairs*
- *Red dashed edges: separating input*
- *Red bold dashed edges with bigger label: separating input selected by the algorithm*
- *Black bold edges: spanning forest edges (pointing to separating state pairs)*
- *Grey ellipses: identical state pairs*
- *Grey dotted edges: edges between identical state pairs*

This implementation version works only on completely specified FSMs and does not discover the duplications of test sequences.
More suitable for education purposes.
Usage of other HSI-method is recommended for actual testing.

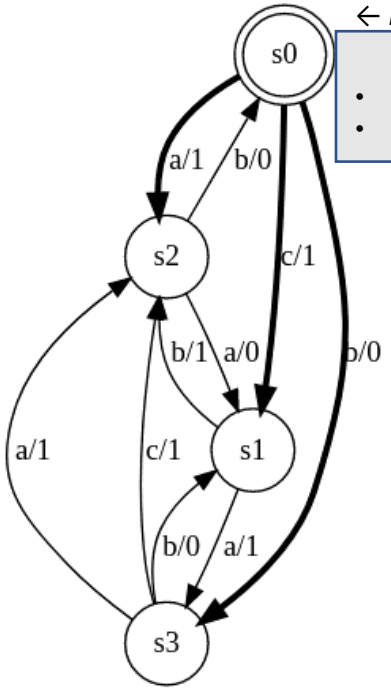



```
[hsi] [info] Generating test sequence...
[hsi] [info] Veryfiyng states...
[hsi] [info] Test state: s0
[hsi] [info] R ,  , b
[hsi] [info] R ,  , b
[hsi] [info] R ,  , b.b
[hsi] [info] Test state: s1
[hsi] [info] R , b. , b
[hsi] [info] R , b. , a
[hsi] [info] R , b. , b
[hsi] [info] Test state: s2
[hsi] [info] R , a. , b
[hsi] [info] R , a. , a
[hsi] [info] R , a. , a
[hsi] [info] Test state: s3
[hsi] [info] R , b.b. , b.b
[hsi] [info] R , b.b. , b
[hsi] [info] R , b.b. , a
[hsi] [info] Test transaction...
[hsi] [info] Test s0's [b] transition
[hsi] [info] already tested transaction
[hsi] [info] Test s0's [a] transition
[hsi] [info] already tested transaction
[hsi] [info] Test s1's [b] transition
[hsi] [info] already tested transaction
[hsi] [info] Test s1's [a] transition
[hsi] [info] R , b , a , b
[hsi] [info] R , b , a , b
[hsi] [info] R , b , a , b.b
[hsi] [info] Test s2's [b] transition
[hsi] [info] R , a , b , b
[hsi] [info] R , a , b , a
[hsi] [info] R , a , b , b
[hsi] [info] Test s2's [a] transition
[hsi] [info] R , a , a , b
[hsi] [info] R , a , a , a
[hsi] [info] R , a , a , a
[hsi] [info] Test s3's [b] transition
[hsi] [info] R , b.b , b , b.b
[hsi] [info] R , b.b , b , b
[hsi] [info] R , b.b , b , a
[hsi] [info] Test s3's [a] transition
[hsi] [info] R , b.b , a , b
[hsi] [info] R , b.b , a , a
[hsi] [info] R , b.b , a , b
```

Generating the entire test set (i.e. the checking sequence) ↑

Appendix F: Model 》 Test 》 Relax MBT framework 31/17

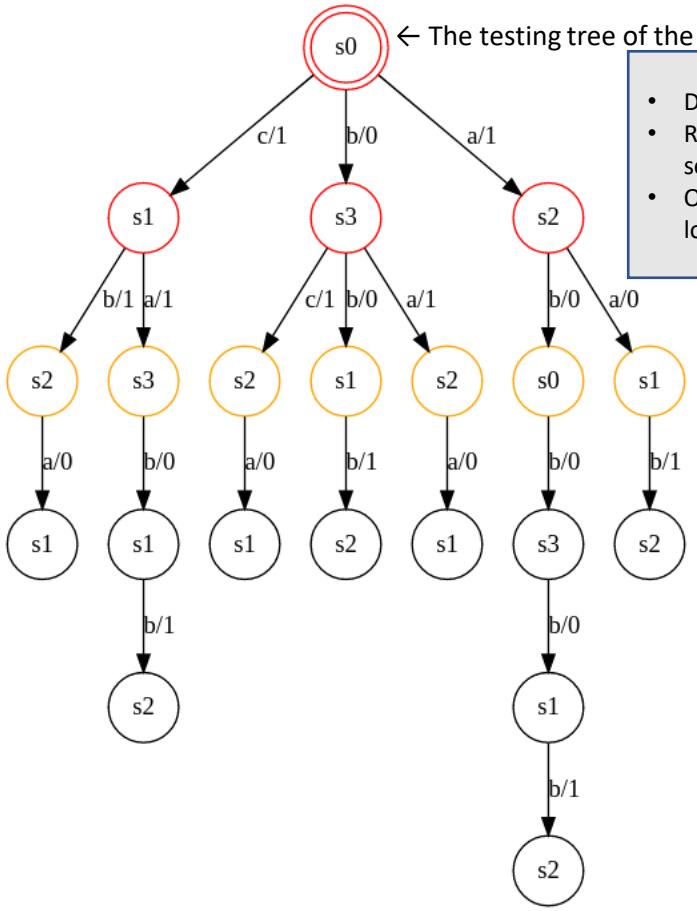
- Different test generation algorithms:
  - **H-method**



← *FSM with the Q state cover set*

Legend

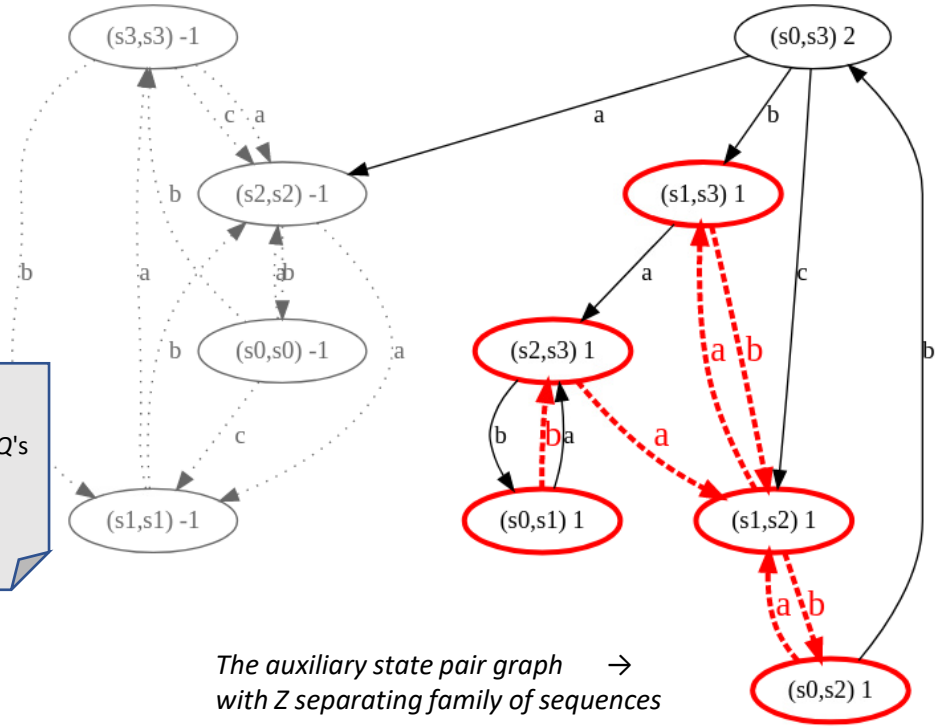
- *Double circle: start state*
- *Black bold edges: spanning tree edges (used in set Q)*



← The testing tree of the 4-complete test suite

Legend

- Double red circle: start state
- Red circle: states reached by the state cover set *Q*'s sequences.
- Orange circle: states reached by the up to μ-n+1 long sequences mentioned in the 1. stage



The auxiliary state pair graph →
with Z separating family of sequences

Legend

- Red ellipses: separating state pairs
- Red dashed edges: separating inputs
- Grey ellipses: identical state pairs
- Grey dotted edges: edges between identical state pairs
- Blue ellipses: non-minimal state pairs
- Blue dotted edges: edges of non-minimal state pairs
- Integer after the state pair: the length of the shortest possible separating sequence

Generating the entire test set (i.e. the checking sequence) ↑

Appendix F: Model 》 Test 》 Relax MBT framework 31/18

- Different test generation algorithms:
  - **N-Switch Coverage**
    - Covers all topologically possible, consecutive *N+1* transitions (including loops)
    - One can define the number of *N* with the `--ns_value` flag

    - The algorithm briefly works as follows:
      1. It creates an ordered list *L* for all topologically possible, consecutive *N+1* transitions.
      2. It always select an element of L that is not already covered and is the nearest form a given state.
      3. When a given element is covered, it also checks if other elements of list L is partially covered and augment the test sequence to cover it completely.
      4. The algorithm terminates if all elements of the list are covered.

Gábor Árpád Németh and Máté István Lugosi. MTR Model-Based Testing Framework. Infocommunications Journal, 16(2):11-18, June 2024. DOI: 10.36244/ICJ.2024.2.2

Appendix F: Model 》 Test 》 Relax MBT framework 31/19

- Different test generation algorithms:
  - **Test unreliable reset symbols**
    - Can be called directly or indirectly
      - Indirectly: If the user applies a test generation algorithms that needs reliable reset, but the model has unreliable resets → unreliable resets must be test first
    - For each state of the model, the following steps are applied:
      - Reach the given state with breath-first search.
      - Apply *reset* symbol.
      - Check if the end state of *reset* symbol is the initial state of the model.

Appendix F: Model 》 Test 》 Relax MBT framework 31/20

Model-based mutation testing:

1. Goes through the *Ω* list of mutation operators and generates all possible *M'* mutant models for the given ω_n operator
2. Generates *ts* test suite from *M* according to the *ALG* list of test gen algos and their params
3. Execute *ts* on each mutant model of *M'*
   - If the predefined *target_score* is reached, then at *ts* to the *TS* set of test suites
   - Otherwise continue 2.
4. Return *TS* set of test suites and the actually achieved *mutation_score*

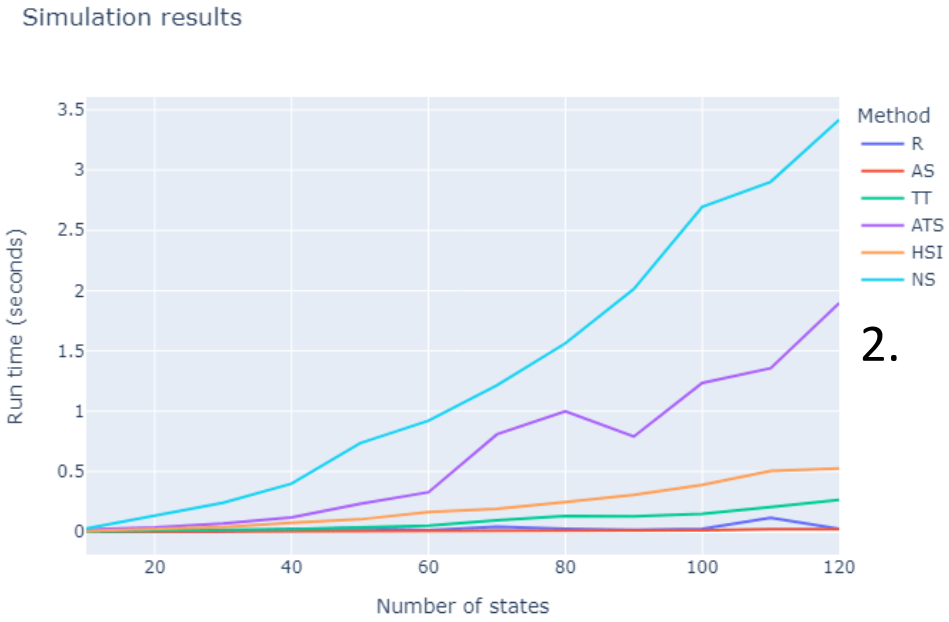
Algorithm 1: Model-based mutation testing (MBMT)

```
   input : M; Ω = {ω_1, .., ω_K}; target_score, ALG
   output: TS; mutation_score
1  T := {}, TS := {}
2  foreach ω_n ∈ Ω do
3      M' := generateMutants(M, ω_n)
4      foreach ts ∈ T do
5          execTestSuite(M', ts)
6          if reachedTarget(M',ts,target_score) and ts
            ∉ TS then
7              TS := TS ∪ {ts}
8      if ∄ ts ∈ T: reached target_score for ω_n then
9          while ¬ reachedTarget(M',ts,target_score)
            ∧ ∃ unused ∈ ALG do
10             alg := First unused element of ALG
11             ts := generateTestSuite(M, alg)
12             T := T ∪ {ts}
13             execTestSuite(M', ts)
14             if reachedTarget(M',ts,target_score)
                then
15                 TS := TS ∪ {ts}
16                 break // goto line 2
17     if ∄ ts ∈ T: reached target_score for ω_n then
18         cts := element of ALG with the highest score
19         TS := TS ∪ {cts}
20 mutation_score := calcMutScore(M, Ω, TS)
21 return TS, mutation_score
```

Gábor Árpád Németh. Model-based mutation testing for Finite State Machine specifications with MTR. Infocommunications Journal, 17(3): 84-91, September 2025. DOI: 10.36244/ICJ.2025.3.10

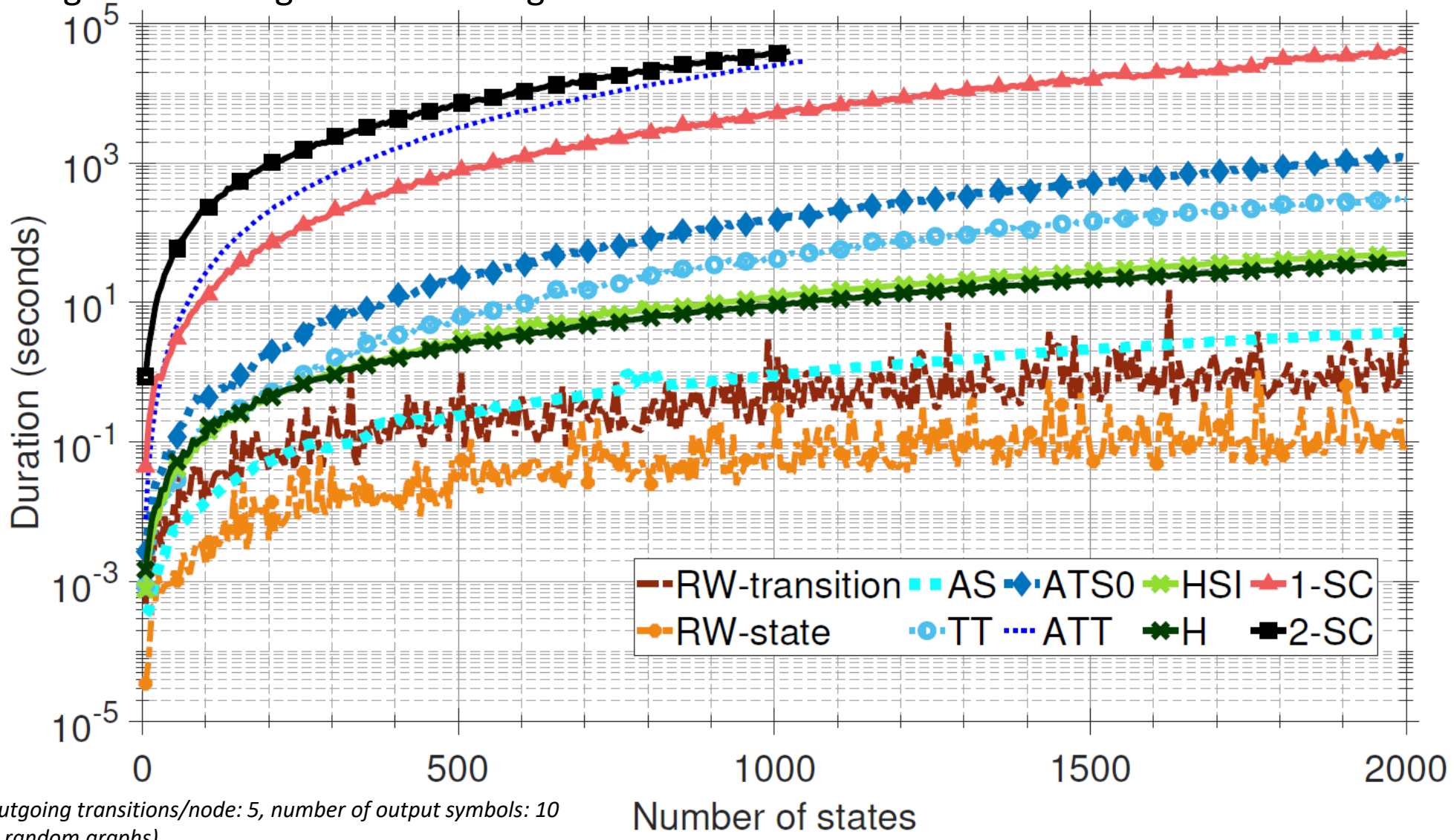
Appendix F: Model 》 Test 》 Relax MBT framework 31/21

- Simulation script:
  - Batch job can be executed:
    1. Model generation:
       - Run random model generation with any combination of these:
         - Number of states: fix / given range with given increments
         - Density: fix / given range with given increments
         - Number of output symbols: fix / given range with given increments
         - Completely specified / Partially specified
         - Have/do not have reliable reset
         - Reduced/non-reduced
    2. Test generation:
       - Apply the list of test generation algorithms with given parameters for a given list of models
       - The results will be collected in csv and excel files
       - In case of a server outage one can continue test generation
       - In case of test generation fails, the errors will be collected in a separate file
    3. Fault coverage investigations:
       - Apply given number and types of faults to the models and check if the tests created by given test generation algorithms can discover them
  - Draw diagram from the csv file created from batch test generation or fault coverage investigations



Appendix F: Model 》 Test 》 Relax MBT framework 31/22

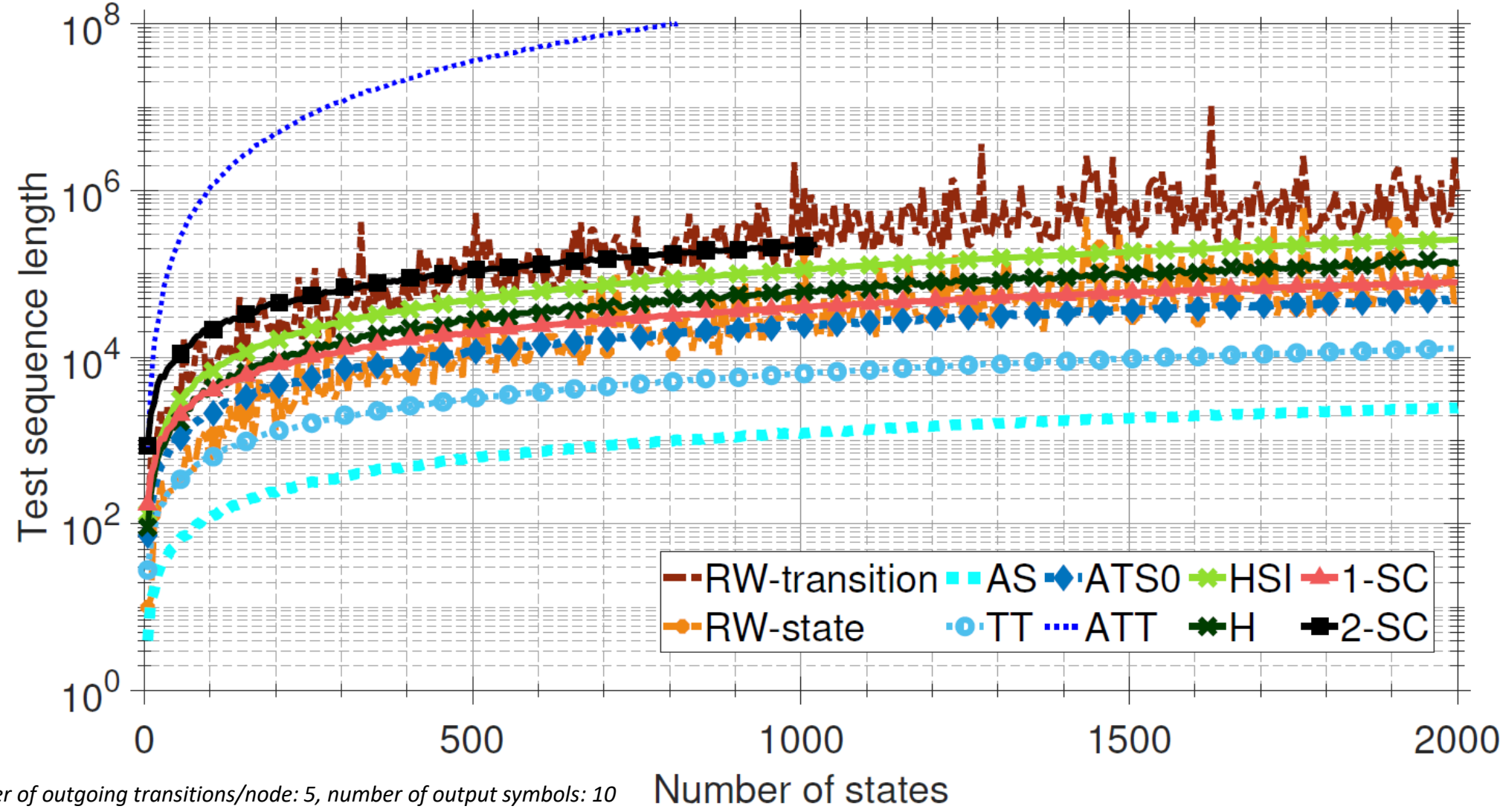
- Different test generation algorithms – test generation time:



Average number of outgoing transitions/node: 5, number of output symbols: 10 (Spanning tree based random graphs)

Appendix F: Model 》 Test 》 Relax MBT framework 31/23

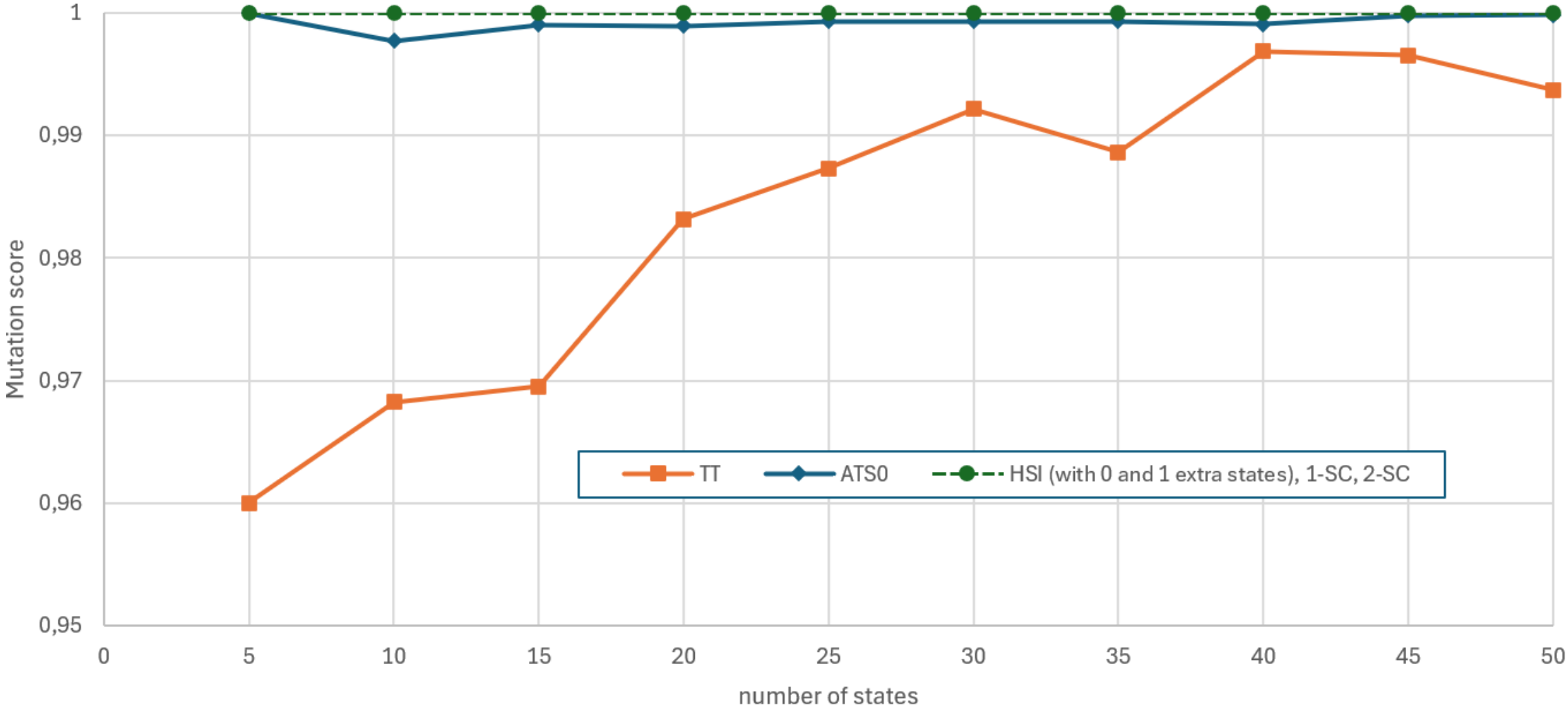
- Different test generation algorithms – length of the generated test sequences:



Average number of outgoing transitions/node: 5, number of output symbols: 10
(Spanning tree based random graphs)

Appendix F: Model 》 Test 》 Relax MBT framework 31/24

- Different test generation algorithms – fault coverage investigation for all possible transfer faults:



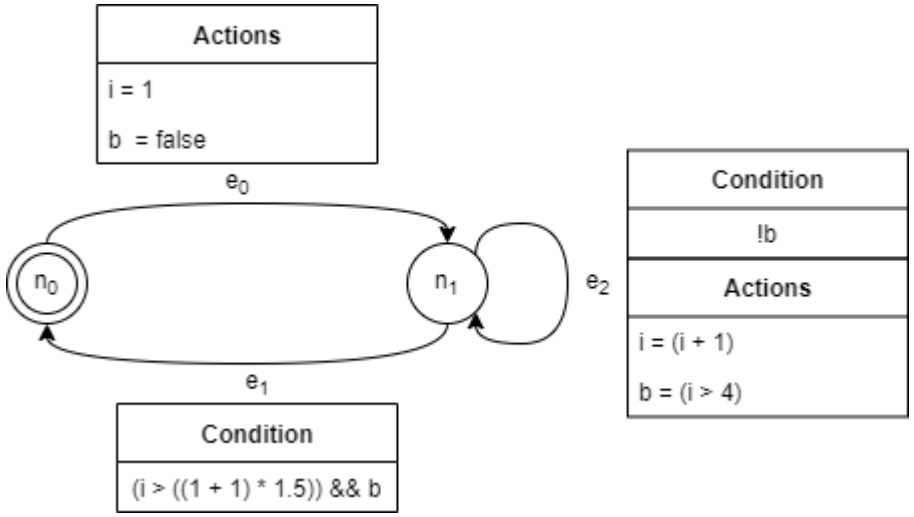
Average number of outgoing transitions/node: 5, number of output symbols: 10
(Spanning tree based random graphs)

Appendix F: Model 》 Test 》 Relax MBT framework 31/25

- **Handling of EFSM models:**

Handled variable types:
- Boolean
- Number (integer and float)
- String
- User-defined enum
- User defined struct

Ranges can be defined for all types of variables



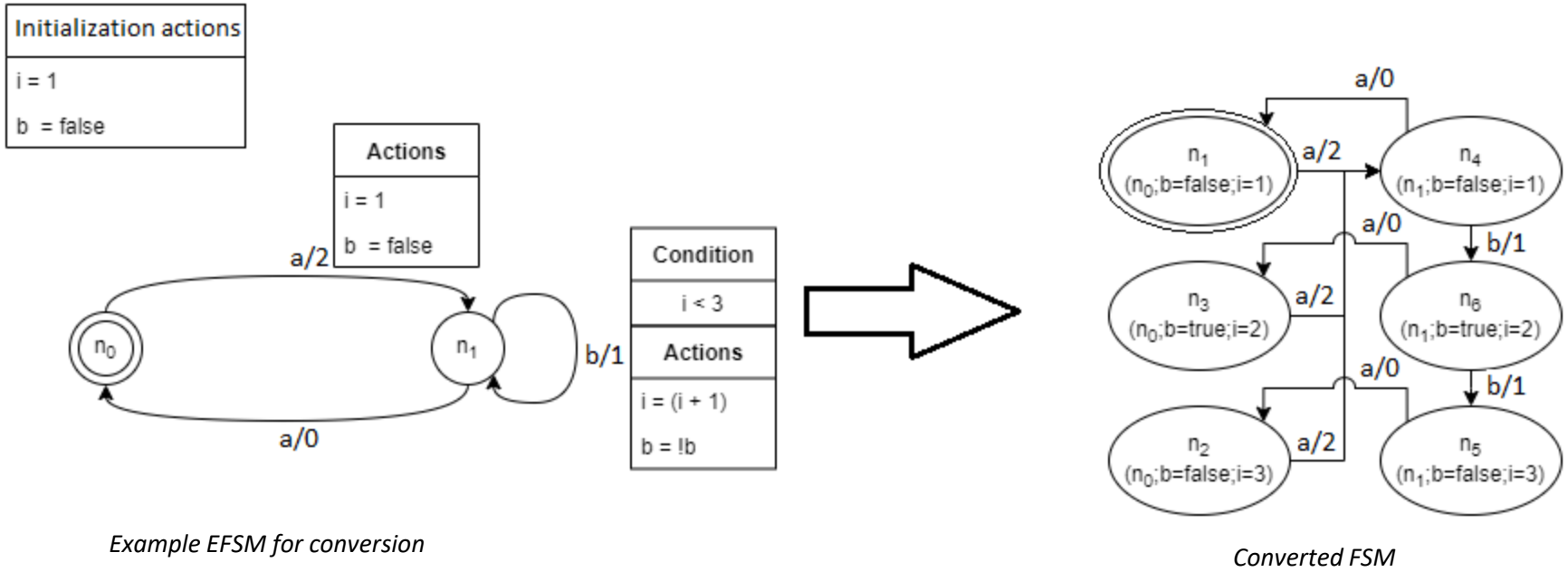
The following operators are handled:
- For all variable types:
  - direct assignment (=)
- For "boolean" variables - logical operators:
  - or (||), and (&&)
  - negation (!)
- For "number" variables:
  - addition (+), subtraction (-)
  - multiplication (*), devision (/)
  - increment by one (++), decrement by one (--)
  - addition assignment (+=)
  - subtraction assignment (-=)
- For "string" variables:
  - Concatenation ('a' . 'b')
  - Length (len('a'))
- For user-defined enum variables:
  - Addition assignment (+=)
  - Subtraction assignment (-=)
  - Increment (++), decrement (--)
- Ternary conditional expressions are also available.

EFSM: Extended Finite State Machine

Appendix F: Model 》 Test 》 Relax MBT framework 31/26

Conversions:

- **EFSM → FSM conversion**:



Example EFSM for conversion

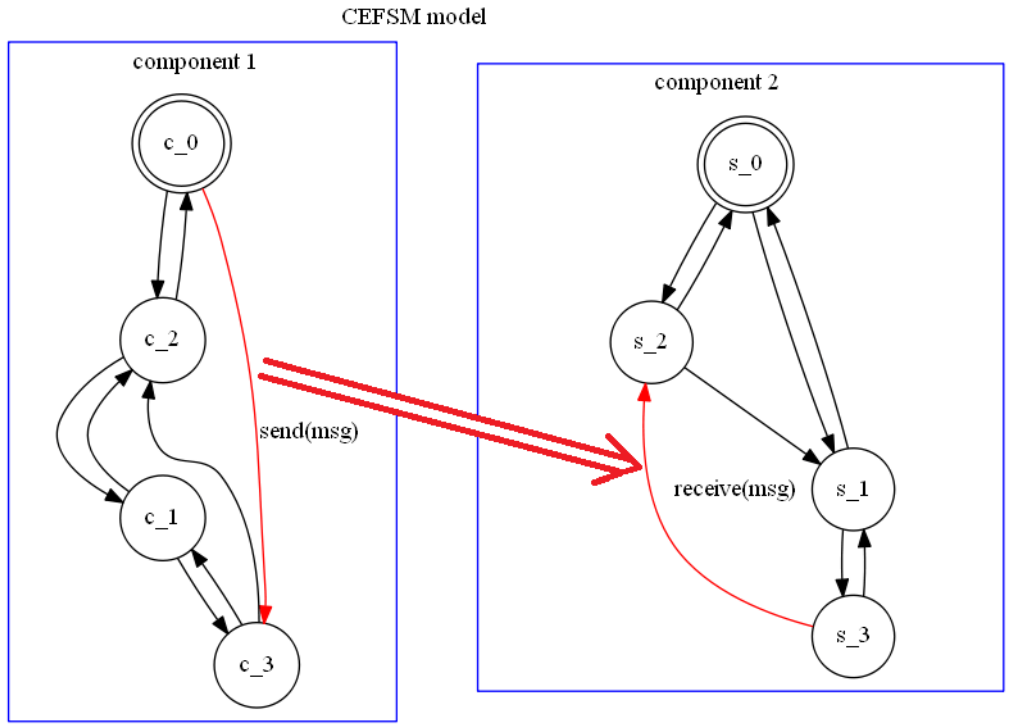
Converted FSM

- Parameter for conversion: possible value range for each variable
- Limitations: transformation works only for integer, Boolean, string and enum variables

FSM: Finite State Machine
EFSM: Extended Finite State Machine

Appendix F: Model 》 Test 》 Relax MBT framework 31/27

- **Handling of CEFSM models:**
  - List of participant models, called CEFSM Components
  - A transition communication can be either a **send** call or a **receive** call.
    - Elements of a send call:
      - `<comma_separated_list_of_targets>`: A list of models separated by commas. The targets of the message.
      - `!`: The "send" operator.
      - `<message>`: The message to be sent. Can be any kind of literal or variable that an EFSM model can handle.
      - `<block_clause>`: Specifies the block behaviour of the send.
      - `<payload_clause>`: Specifies the payload to be sent.
    - Elements of a receive call:
      - `untyped_struct_variable =`: An optional component of the call.
      - `<comma_separated_list_of_sources>`: A list of model ids separated by commas. The sources from which the communication waits messages.
      - `?`: The "receive" operator.
      - `<block_clause>`: Specifies the block behaviour of the send.
      - `<expecting_clause>`: Represents what kind of messages the current receive communication is waiting for.
      - `<comma_separated_list_of_expressions>`: Must be a comma separated list of viable expressions that the framework can handle.



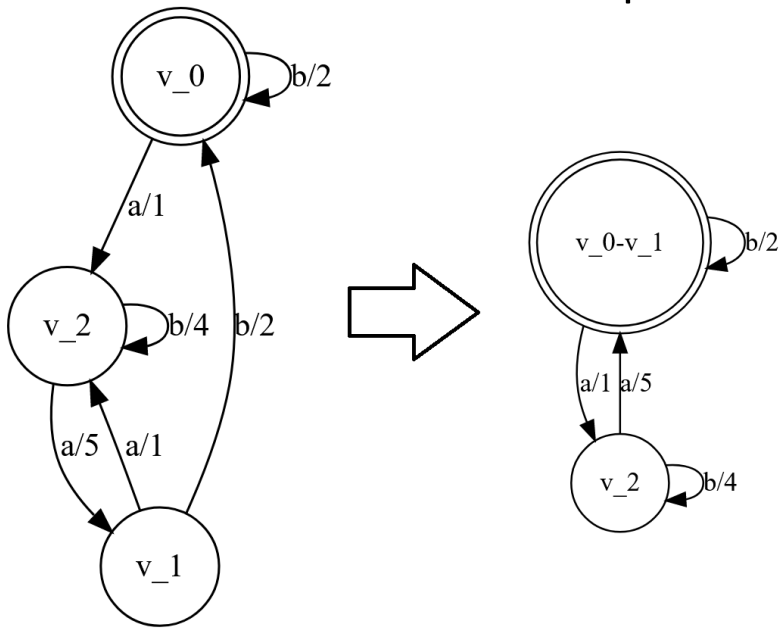
- **CEFSM → EFSM conversion**

CEFSM: Communicating Extended Finite State Machine
EFSM: Extended Finite State Machine

Appendix F: Model 》 Test 》 Relax MBT framework 31/28

Conversions:

- **State minimization:**
  - transform a not reduced FSM into an equivalent, reduced FSM by merging equivalent states



- **Partially specified FSM → Completely specified FSM conversion**:
  - For each undefined state-input pairs a loop transition is added without any output
  - A sink state can be also added for each each undefined state-input pairs without any output

FSM: Finite State Machine
EFSM: Extended Finite State Machine

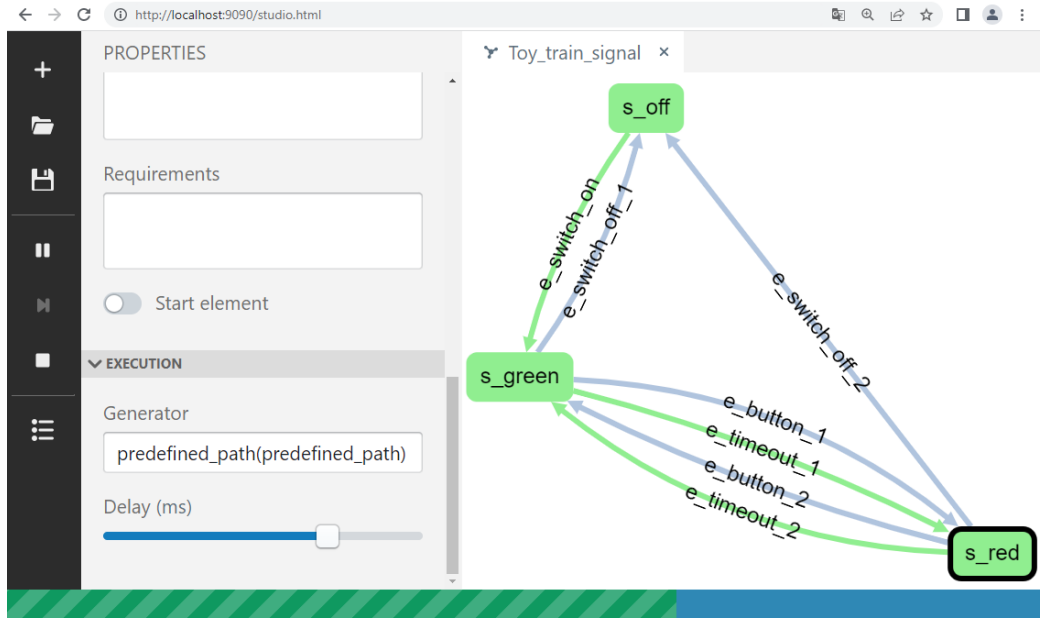
- **Fault injection**
  - Can be applied in Model-based mutation testing. To discover fault coverage of different test generations

Appendix F: Model 》 Test 》 Relax MBT framework 31/29

Test execution:

- Adaptation code skeleton generation from specification model

- Create a connection to an actual system:
  - Export test suite into GraphWalker
    - Generate test cases with MTR
    - Use the *predefined path* feature of GraphWalker to import generated testcases into GraphWalker



  - Write your own adaptation code – see delivered sample test projects

SUT: System Under Test

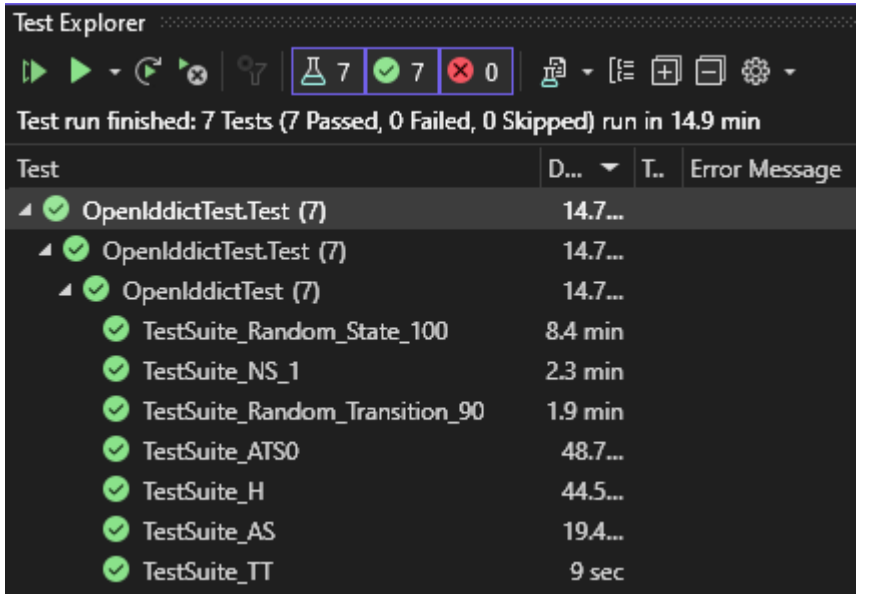
Appendix F: Model 》 Test 》 Relax MBT framework 31/30

Delivered sample test projects:

- The delivered Model 》 Test 》 Relax framework contains the following example testing projects with related documentation:
  - Git: Testing changes and additions on git version handling system
  - Ory Hydra: Testing the Ory Hydra OAuth2.0 and OpenID Connect server through an API
  - OpenIddict Oauth: Testing the Oauth2.0/OpenId Connect library through an API
  - *ASP.NET Webshop*: Testing the GUI of a simple ASP.NET Webshop application using Selenium
  - *ELTELive:* Functional testing of a streaming website through an API
  - *Neovim:* Testing the main modes of a linux text editor program through an API

- These example projects can be used as a starting point when one would like to create an own test project

- Each example project contains the following files:
  - An FSM or EFSM specification model
  - Adaptation code
  - SUT



SUT: System Under Test

Appendix F: Model 》 Test 》 Relax MBT framework 31/31

- Future directions:
  - Apply incremental test generation (TT, HSI, ATS) for changing specifications
  - Extend the list of test generation methods
  - Further improvements related to EFSM models (handling timers, test generation methods…etc.)
  - Handling CEFSM models, code→CEFSM transformation
  - Test case selection
  - Create adaptation code skeletons (GUI testing: Playwright, API testing)

- Long-term goal:
  - Create a logic for automatic test generation suggestion (based on the structure of the specification model, required coverage and available resources)

ATS: All-Transition-State
CEFSM: Communicating Extended Finite State Machines
EFSM: Extended Finite State Machine
HSI: Harmonized State Identifiers

Appendix G:
Traceability table for different Modelling and testing materials

Legend
Title of the topic with links / *Guest lecture* — slides in CM (Core Material, this file) / slides of guest lecturers
CM video in Hungarian / CM video in English / video of guest lecturers — `Auxiliary materials`

-	Introduction to testing, classification of testing CM/p9-47 MaT-HU-01-Bevezetes a tesztelesbe.mkv MaT-ENG-01-Introduction to testing.mkv
Requirement modelling & traceability CM/p48-81 Case study of changing reqs. CM/Appendix B/II MaT-HU-02-Kovetelmenyek modellezese es nyomonkovethetosege.mkv + MaT-HU-02SE-Valtozo kovetelmenyekrol esettanulmany-A Space Shuttle.mkv MaT-ENG-02-Requirement modelling and traceability.mkv + MaT-ENG-02SE-Case study of changing requirements-The Space Shuttle.mkv	Scenario-based and Keyword-driven testing CM/p82-97 Test automation CM/p98-123 MaT-HU-03-Scenario es kulcsszo alapu teszteles.mkv + MaT-HU-04-Teszt automatizacio.mkv MaT-ENG-03-Scenario-based testing and Keyword-driven testing.mkv + MaT-ENG-04-Test automation.mkv
MBT models, Petri nets, FSMs and their extensions / part 1 CM/p124-166 MaT-HU-06-MBT modellek-Petri halok Veges Automatak es kiterjeszteseik.mkv / 1. rész MaT-ENG-06-MBT models-Petri nets FSMs and extensions of FSMs.mkv / part 1	MBT models, Petri nets, FSMs and their extensions / part 2 CM/p124-166 MaT-HU-06-MBT modellek-Petri halok Veges Automatak es kiterjeszteseik.mkv / 2. rész MaT-ENG-06-MBT models-Petri nets FSMs and extensions of FSMs.mkv / part 2
MBT: coverage criteria, assumptions about FSMs, fault models CM/p167-186 MaT-HU-07-MBT-fedesi kriteriumok-elofeltevesek az FSMel kapcsolatban-hibamodellek.mkv MaT-ENG-07-MBT coverage criteria and assumptions about FSMs and fault models.mkv	MBT: test generation algorithms (classifications, TT, HSI, H) CM/p187-227 MaT-HU-08-MBT teszt generalo algoritmusok.mkv MaT-ENG-08-MBT test generation algorithms.mkv
MBT: Incremental test generation CM/p228-248 MaT-HU-09-MBT inkrementális teszt generalas.mkv MaT-ENG-09-MBT-incremental test generation.mkv	MBT: Model-based mutation testing CM/p249-265 No video!
MBT: Test case selection, conversion from code to EFSM, MBT tools, summary of MBT CM/p266-307, p353-371 MaT-HU-10-MBT-teszt eset szelektalas-kodrol EFSMre konverzio-eszkozok-osszefoglalo.mkv MaT-ENG-10-MBT-test case selection-code to EFSM conversion-MBT tools-MBT conclusion.mkv	*Máté I. Lugosi, G. Á. Németh: Model 》 Test 》 Relax MBT framework demo + information about the project*
GraphWalker MBT tool + further processing with Model 》 Test 》 Relax / part 1 CM/p308-352 MaT-HU-11-MBT-GraphWalker demo.mkv / 1. rész `Marklin signal project.zip`	GraphWalker MBT tool + further processing with Model 》 Test 》 Relax / part 2 CM/p308-352 MaT-HU-11-MBT-GraphWalker demo.mkv / 2. rész `Marklin signal project.zip`
Models in failure analysis: FMEA and FTA CM/p372-391 MaT-HU-05-FMEA es FTA.mkv MaT-ENG-05-FMEA and FTA.mkv `FMEA_sample_empty_table.xlsx`	All Fools' Day special edition CM/Appendix A MaT-HU-SE-Aprilis elsejei kulonkiadas.mkv `April1st_Feature_Specification.pdf`
Performance testing CM/p392-437, p480-481 MaT-HU-12-Teljesitmenyteszteles.mkv MaT-ENG-12-Performance testing.mkv	Wireshark protocol stack analyser & Apache Jmeter performance testing tool CM/p438-463 MaT-HU-13A-Wireshark protokol analizalo rovid demo.mkv + MaT-HU-13B-Apache JMeter teljesitmenyteszteles demo.mkv MaT-ENG-13-Apache JMeter demo.mkv `JmeterDemo.zip`
Guest lecture: G. Á. Németh: TitanSim: Perf. testing of telecomm. netws. with complex traffic mix CM/464-473 (only the main concept is shown here)	Model-driven performance testing methods for comm. Systems pCM/p474-479+ Future of testing CM/p482-492 MaT-HU-13C-Egy modell alapu megkozelites teljesitmenyteszteleshez.mkv + MaT-HU-14-A teszteles jovoje.mkv
Guest lecture: István Forgács: Harmony MBT tool Istvan Forgacs-4Test-Harmony.pdf Istvan Forgacs - ENG - 4Test Harmony.mp4	*Guest lecture: Antal Bali, István Móricz, Bálint Szél:Tricentis Tosca* Bali Antal-Tosca Core Concepts-v2.pdf + Istvan Moricz-Tosca demo.pdf Antal Bali and Istvan Moricz - ENG - Tricentis Tosca.mp4
Guest lecture: Péter Sótér: Continuous Integration and Continuous Delivery with Jenkins Peter Soter-CI_CD_Jenkins.pdf Peter Soter - HU - CI-CD with Jenkins - part1 - presentation.mkv Peter Soter - HU - CI-CD with Jenkins - part2 - Jenkins install and pipeline example.mkv	*Guest lecture: Kristóf Szabados: Code quality, code smells* Kristof Szabados-Code_smells.pdf Kristof Szabados - HU - Code quality code smells.mp4

Main ToC

Appendix H: Scheduling of topics during the semester Spring 2025/2026

Week	Tuesday 8.30-10.00	Thursday 12.00-14.00
1st (February 9-15)	Intro, discussions about homework and exam, Introduction to testing	Classification of testing, Requirements modelling, Modelling languages, Behavior trees, Traceability matrix
2nd (February 16-22)	Scenario-based testing, KDT, test automation, Test automatization , MBT: intro about models, petri nets	MBT: intro about models, petri nets, FSMs
3rd (February 23-March 1)	MBT: Extensions of FSMs: EFSMs, C(E)FSMs, PFSMs, Markov chains, test coverage criteria 2/1	
4th (March 2-8)		
5th (March 9-15)		*Homework task clarification deadline (till the end of the lecture)*
6th (March 16-22)		
7th (March 23-29)		
8th (March 30- April 5)	*Homework basic level deadline **Monday** 8.00 CET*	Holiday
9th (April 6-12)	Holiday	
10th (April 13-29)		
11th (April 20-26)	*Monday 8.00 CET: Homework extended level - early feedback deadline*	
12th (April 27-May 3)	*Monday 8.00 CET: Homework extended level – final deadline*	
13th (May 4-10)		
14th (May 11-17)	***Homework presentations***	***Pre-exam***

Appendix I: Syllabus

1. Introduction
   - Why is testing important? Scoring the risks, cost of defects
   - Classification of testing:
     - Participants: alpha / beta
     - Approach: black-box (specification-based) / white-box (structure-based) / grey-box
     - Testing in different phases of project life cycle: prototype, new functionality (green field investment), functionality of existing products (brown field investment)
     - Phase: smoke / sanity / regression
     - Levels of testing: unit, integration, system, acceptance
     - Types of testing based on role: functional, non-functional (performance, security, usability, etc.)
     - Scenario: positive / negative
     - Automatization:
       - From the view of test execution: manual / semi-automated / automated
       - From the view of test design: manually designed / (semi-)automatic generated
   - Bad examples form the industry that should be avoided
2. Requirement modelling and traceability
   - Feature specification document, use cases and scenarios
   - SDL (Specification and Description Language), MSC (Message Sequence s)
   - UML (Unified Modeling Language)
   - TDL (Test Description Language)
   - Behaviour trees
   - Traceability matrix
3. Test automation
   - Scenario-based testing, Gherkin language
   - KDT (Keyword-Driven Testing)
   - Waterfall, V-model, Agile, Test Driven Development (TDD), Continuous Integration (CI), Continuous Deployment (CD), DevOps methodologies, CI gating
   - Test pyramid
   - Scheduling of execution: commit / daily / weekly / before release tests
   - Live demo of a test automation server
4. Models in failure analysis
   - Bottom-up failure analysis with FMEA (Failure Mode and Effects Analysis)
   - Top-down, deductive failure analysis with FTA (Fault Tree Analyses)
5. MBT (Model-Based Testing)
   - Different models for behaviour description:
     - Petri nets
     - Behaviour trees
     - FSMs (Finite State Machines) and their extensions: STT (State Transition Systems), EFSMs (Extended FSMs), PFSMs (Probabilistic FSMs), C(E)FSMs (Communicating (E)FSMS)
     - Markov chains
   - 3 types of behaviour: specified, feasible (implemented), topologically possible
   - Black box testing, conformance testing
   - Test coverage criteria (structural and non-structural)
   - Fault models
   - Assumptions about FSMs
   - FSM-based test generation algorithms:
     - Classification: structure with 1 or 2 stages, adaptiveness, state identification approaches
     - Overview of existing methods
     - Description of the following methods:
       - Simplest solution with one stage
       - TT (Transition Tour method)
       - W / Wp / HSI (Harmonized State Identifiers) triple
       - H method
     - Incremental approach:
       - Goal, assumptions
       - 2 methods: incremental TT, incremental HSI
     - Model-based mutation testing
     - Test case selection:
       - Approximation methods, string edit distance metrics-based selection algorithm
   - Finding interoperability problems with code → EFSM transformation
   - Theory vs. practice of MBT
   - Comparison of different market leader tools
   - Live demo of a free and open-source MBT tool
   - Typical problems with MBT in practice / guidelines
   - Advantages / disadvantages of MBT
6. Performance testing
   - Types:
     - Load testing
     - Endurance (stability, soak) testing
     - Stress testing
     - Capacity (volume, flood) testing
     - Spike testing
     - Scalability testing
   - Scheduling of performance tests
   - Main steps of performance tests
   - Measured parameters
     - Hardware utilization
     - Characteristics of the tested system
   - Tools in performance testing, 3 main functionalities:
     - Load generation
       - Working principle
       - Approaches for behaviour description – trade-off between performance and complexity:
         - Hardwired (Packet generators)
         - Call-flow (Traffic playback tools)
         - Program code
         - Model-based descriptions (EFSMs…etc.)
     - Performance monitoring
       - Measured parameters
     - Log analyses
   - Comparison of industrial testing tools
   - Performance tests at different test levels
   - Connection with other testing attributes (functionality, security, usability…etc.)
   - Live demo of a packet analyzer tool
   - Live demo of a free and open-source performance testing tool
   - A model-driven performance testing method
7. The future of testing: AI (Artificial Intelligence):
   - working principle
   - possible applications in testing domain
   - free tools

Main ToC

Appendix J: List of Abbreviations

3GPP	3rd Generation Partnership Project
AI	Artificial Intelligence
API	Application Programing Interface
AS	All-State
AT	All-Transition
ATS	All-Transition-State
ATT	All-Transition-Transition
BA	Business Analyst
BDD	Behaviour Driven Development
BFS	Breadth-first search
BPML	Business Process Model and Notation
BVA	Boundary Value Analyses
CCS	State Cover Set
CEFSM	Communicating Extended Finite State Machine
CFSM	Communicating Finite State Machine
CD	Continuous Delivery
CI	Continuous Integration
CLI	Command Line Interface
CPP	Chinese Postman Problem
cps	Calls per Second
CPU	Central Processing Unit
CS	Characterizing Set
CR	Change Request
DCPP	Directed Chinese Postman Problem
DevOps	Software Development & IT Operations
DFS	Depth-first search
DLL	Dynamic Link Library
DS	Distinguishing Sequence
EFSM	Extended Finite State Machine
EP	Equivalence Partitioning
ETA	Extended Timed Automata
ETSI	European Telecomm. Standards Institute
FMEA	Failure Mode and Effects Analysis
FSM	Finite State Machine
FTA	Fault Tree Analyses
GoS	Grade of Service
GUI	Graphical User Interface
HSI	Harmonized State Identifiers
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
IP	Internet Protocol
IS	Identification Set
ISTQB	International Software Testing Qualifications Board
IT	Information Technology
IUT	Implementation Under Test
JSON	JavaScript Object Notation
KDT	Keyword-driven testing
LEMON	Library for Efficient Modeling and Optimization in Networks
LTS	Labeled Transition Systems
MBMT	Model-based Mutation Testing
MBT	Model-Based Testing
MT	Mutation testing
mps	Messages per Second
MÁV	Magyar Államvasutak → Hungarian State Railways
MSC	Message Sequence
OS	Operating System
PAND	Priority AND
PFSM	Probabilistic Finite State Machine
PHP	PHP: Hypertext Preprocessor
POM	Project Object Model
PTA	Probabilistic Timed Automata
RFC	Request For Comments
RPN	Risk Priority Number
RTM	Requirement Traceability Matrix
SCC	Strongly Connected Component
SCS	State Cover Set
SDL	Specification Description Language
SIP	Session Initiation Protocol
STT	State Transition Diagram
SUT	System/Software Under Test
SW	Software
TA	Test Automatization
TCFMM	Timed Communicating Finite Multistate Machine
TDD	Test Driven Development
TDL	Test Description Language
TDL-GR	TDL Graphical Representation
TDL-TO	TDL Test Objective
TDL-XF	TDL Exchange Format
TT	Transition Tour
TTCN-3	Testing and Test Control Notation version 3
TR	Trouble Report
UAC	User Agent Client
UAT	User Acceptance Testing
UFT	Unified Functional Testing
UI	User Interface
UIO seq.	Unique Input Output Sequence
UML	Unified Modeling Language
XOR	Exclusive OR

Main ToC

Acknowledgements

The project is supported by the Hungarian Government and co-financed by the European Social Fund (EFOP-3.6.3-VEKOP-16-2017-00001: Talent Management in Autonomous Vehicle Control Technologies).

HUNGARIAN GOVERNMENT

European Union
European Social Fund

INVESTING IN YOUR FUTURE